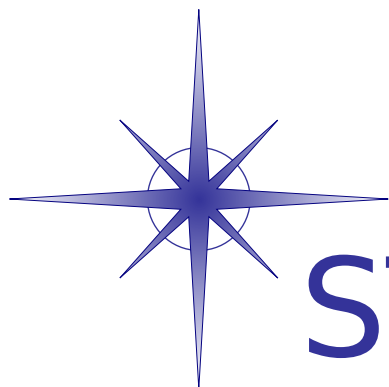




STAR-Dundee

STAR-System Documentation

Version 6.01

Generated by Doxygen 1.8.10

Thu Nov 13 2025 12:27:12

Contents

1	Introduction and Overview	1
1.1	Introduction	1
1.2	Supported Operating Systems	2
1.3	Installation Instructions	2
1.3.1	Installation Instructions for Windows	2
1.3.1.1	Windows XP USB Issue	2
1.3.1.2	Windows Vista USB Issue	3
1.3.1.3	Universal CRT	3
1.3.1.4	Windows 10 Kernel DMA Protection Issue	3
1.3.2	Installation Instructions for Linux	3
1.3.2.1	Command Line Installer	4
1.3.3	Installation Instructions for Linux with Secure Boot Enabled	5
1.3.4	Installation Instructions for QNX	6
1.3.5	Installation Instructions for VxWorks	6
1.3.6	Installation Instructions for RTEMS	6
1.3.7	Compatibility Issues with MXIe Systems	6
1.4	Getting Started	7
1.4.1	Getting Started on Windows	7
1.4.1.1	Getting Started on Cygwin	7
1.4.2	Getting Started on Linux	7
1.4.3	Getting Started on QNX	7
1.4.4	Getting Started on VxWorks	8
1.4.5	Getting Started on RTEMS	8
1.5	Migrating from Previous STAR-Dundee APIs	8
1.6	Supported Devices	9
1.6.1	Feature Comparison Table	10
1.7	Device User Manuals	11
1.8	Graphical Applications	11
1.9	CUBA Software	12
1.10	STAR-System APIs	12
1.10.1	STAR-API	12

1.10.2	Generic Configuration API	12
1.10.3	RMAP Packet Library	13
1.10.4	RMAP Target API	14
1.10.5	Generic Triggering API	14
1.10.6	SpaceFibre Configuration API	14
1.10.7	PCI Mk2 Packet Subsystem API	14
1.10.8	CCSDS/SPP/TF Packet Library API	15
1.11	Supported Languages	15
1.12	Example Programs	15
1.12.1	STAR-System Test	15
1.12.2	STAR Performance Tester	16
1.12.3	Device Configuration APIs Examples	16
1.12.4	RMAP Packet Library Example	17
1.12.5	CCSDS/SPP/TF Packet Library Example	17
1.12.6	Generic Triggering API Example	17
1.13	Known Issues	17
1.14	Change Log	18
1.14.1	Changes between STAR-System v6.00 and v6.01 - 13th November 2025	18
1.15	Support and Contact Information	19
2	STAR-System Applications	21
2.1	Graphical Applications	21
2.1.1	Device Configuration Application	21
2.1.2	Transmit Application	22
2.1.3	Receive Application	22
2.1.4	Source Application	22
2.1.5	Sink Application	23
2.1.6	Error Injection Application	23
2.1.7	Device Update Application	23
2.1.8	Time-code Application	23
2.1.9	RMAP Initiator Application	23
2.1.10	RMAP Target Application	24
2.1.11	Triggering Application	24
2.1.12	Ethernet Device Connector Application	24
2.1.13	Command-Line Arguments	24
2.1.14	Running the GUI Applications on QNX	25
2.2	CUBA Software	25
2.2.1	Data Format Conventions	26
2.2.2	SpaceWire Interactive Mode	26
2.2.3	RMAP Interactive Mode	27

2.2.4	Command File Directives	28
2.2.4.1	#INCLUDELIST Directive	28
2.2.4.2	#ALIAS Directive	28
2.2.5	SpaceWire Command Stream Mode	29
2.2.5.1	SpaceWire Send Command	29
2.2.5.2	SpaceWire Receive Command	29
2.2.5.3	Loop Command	30
2.2.5.4	Change Base Command	30
2.2.5.5	Example SpaceWire Command Stream Input File	30
2.2.5.6	SpaceWire Command Stream Output Files	31
2.2.5.7	Example SpaceWire Command Stream Output File	31
2.2.6	RMAP Command Stream Mode	31
2.2.6.1	RMAP Command Stream Input Files	31
2.2.6.2	Loop Command	32
2.2.6.3	Example RMAP Command Stream Input File	33
2.2.6.4	RMAP Command Stream Output Files	33
2.2.6.5	Example RMAP Command Stream Output File	34
2.2.7	CUBA Command Line Arguments	34
2.2.8	Backwards Compatibility Mode	35
3	STAR-System Devices	37
3.1	SpaceWire PCI Mk2 and cPCI Mk2	37
3.1.1	Overview	37
3.1.1.1	Summary	37
3.1.1.2	Hardware Description	37
3.1.2	Getting Started	39
3.1.3	Interfaces	39
3.1.3.1	SpaceWire Ports	39
3.1.3.2	Front Panel LEDs	39
3.1.4	Functions	40
3.1.4.1	SpaceWire Router	40
3.1.4.2	Interface Mode	40
3.1.4.3	Routing Mode	41
3.1.4.4	Link Operating Speed	42
3.1.4.5	Time-codes	43
3.1.5	Electrical Characteristics	43
3.1.6	Switching Characteristics	44
3.1.7	Grounding Information	45
3.2	SpaceWire PCIe	45
3.2.1	Overview	45

3.2.1.1	Summary	45
3.2.1.2	Hardware Description	45
3.2.2	Getting Started	46
3.2.3	Interfaces	47
3.2.3.1	SpaceWire Ports	47
3.2.3.2	Power LED	47
3.2.3.3	Front Panel LEDs	47
3.2.4	Functions	48
3.2.4.1	SpaceWire Router	48
3.2.4.2	Interface Mode	48
3.2.4.3	Routing Mode	49
3.2.4.4	Link Operating Speed	50
3.2.4.5	Time-codes	50
3.2.4.6	RMAP Target	51
3.2.5	Electrical Characteristics	52
3.2.6	Switching Characteristics	53
3.2.7	Grounding Information	54
3.3	SpaceWire PCIe Mk2	55
3.3.1	Overview	55
3.3.1.1	Hardware Description	55
3.3.2	Getting Started	56
3.3.3	Interfaces	57
3.3.3.1	SpaceWire Ports	57
3.3.3.2	Trigger Ports	58
3.3.3.3	Power LED	59
3.3.4	SpaceWire Router and SpaceWire Interfaces	59
3.3.4.1	SpaceWire Router	59
3.3.4.2	Interface Mode	59
3.3.4.3	Routing Mode	60
3.3.4.4	Link Operating Speed	61
3.3.4.5	Broadcast codes	61
3.3.4.6	Packet Timestamping	63
3.3.4.7	SpaceWire Interface Tristate Mode	65
3.3.5	RMAP Target	67
3.3.6	Absolute Maximum Ratings	68
3.3.7	Recommended Operating Conditions	69
3.3.8	Switching Characteristics	70
3.3.8.1	Encoder Switching Characteristics	70
3.3.8.2	Decoder Switching Characteristics	70
3.3.9	FMEA Compliant	71

3.3.10	SpaceWire Ground Connection	71
3.3.11	EMC Conformity	72
3.4	SpaceWire Brick Mk2	72
3.4.1	Overview	72
3.4.1.1	Summary	72
3.4.1.2	Hardware Description	72
3.4.2	Getting Started	73
3.4.3	Interfaces	73
3.4.3.1	SpaceWire Ports and LEDs	73
3.4.4	Functions	74
3.4.4.1	SpaceWire Router	74
3.4.4.2	Interface Mode	74
3.4.4.3	Routing Mode	76
3.4.4.4	Link Operating Speed	76
3.4.4.5	Time-codes	77
3.4.5	Electrical Characteristics	78
3.4.6	Switching Characteristics	78
3.4.7	Grounding Information	79
3.4.8	EMC Conformity	80
3.5	SpaceWire Router Mk2S	80
3.5.1	Overview	80
3.5.1.1	Summary	80
3.5.1.2	Hardware Description	81
3.5.2	Getting Started	82
3.5.3	Interfaces	82
3.5.3.1	SpaceWire Ports and LEDs	82
3.5.3.2	External FIFO Ports	83
3.5.4	Functions	88
3.5.4.1	SpaceWire Router	88
3.5.4.2	Interface Mode	88
3.5.4.3	Routing Mode	90
3.5.4.4	Link Operating Speed	91
3.5.4.5	Time-codes	92
3.5.5	Electrical Characteristics	92
3.5.6	Switching Characteristics	93
3.5.7	Grounding Information	94
3.5.8	EMC Conformity	95
3.6	SpaceWire Brick Mk3 and Brick Mk4	95
3.6.1	Overview	95
3.6.1.1	Summary	95

3.6.1.2	Hardware Description	96
3.6.2	Getting Started	96
3.6.3	Rear Panel Interfaces	97
3.6.3.1	USB Status LED	97
3.6.3.2	Micro-B USB Connector	99
3.6.4	Front Panel Interfaces	99
3.6.5	Functions	100
3.6.5.1	SpaceWire Router	100
3.6.5.2	Interface Mode	101
3.6.5.3	Routing Mode	102
3.6.5.4	Link Operating Speed	102
3.6.5.5	Time-codes	103
3.6.5.6	Packet Timestamping	103
3.6.6	SpaceWire Interface Tristate Mode	105
3.6.7	Electrical Characteristics	107
3.6.7.1	SpaceWire Connectors	107
3.6.7.2	External Trigger Connectors	108
3.6.7.3	FMECA Protection	110
3.6.8	Switching Characteristics	110
3.6.9	Grounding Information	111
3.6.10	EMC Conformity	111
3.7	SpaceWire PXI Interface and PXI Interface Mk2	112
3.7.1	Overview	112
3.7.2	Hardware Description	112
3.7.2.1	SpaceWire PXI Interface Card	113
3.7.2.2	SpaceWire PXI Interface with RMAP Target	114
3.7.3	Getting Started	115
3.7.3.1	Front Panel Interfaces	116
3.7.3.2	SpaceWire Interfaces	116
3.7.3.3	Trigger Interfaces	117
3.7.4	Supported Systems	118
3.7.4.1	cPCI Compatibility	119
3.7.4.2	PXI Compatibility	119
3.7.4.3	PXle Compatibility	120
3.7.5	Router and SpaceWire Interfaces	121
3.7.5.1	Interface Mode	121
3.7.5.2	Routing Mode	121
3.7.5.3	Link Operating Speed	122
3.7.5.4	Time-codes	122
3.7.5.5	Packet Timestamping	122

3.7.6	RMAP Target	124
3.7.6.1	Configuration and Access Modes	125
3.7.6.2	Memory Access	127
3.7.7	SpaceWire Interface Tristate Mode	127
3.7.8	Electrical Characteristics	131
3.7.8.1	Absolute Maximum Ratings	131
3.7.8.2	SpaceWire Functional Diagram	132
3.7.8.3	External Trigger Connectors	134
3.7.9	Switching Characteristics	135
3.8	SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2	136
3.8.1	Overview	136
3.8.2	Hardware Description	136
3.8.2.1	SpaceWire SpaceWire PXI 12 Port Router Card	137
3.8.3	Getting Started	138
3.8.3.1	Front Panel Interfaces	138
3.8.3.2	SpaceWire Interfaces	139
3.8.4	Supported Systems	139
3.8.4.1	cPCI Compatibility	140
3.8.4.2	PXI Compatibility	141
3.8.4.3	PXIe Compatibility	142
3.8.5	Router and SpaceWire Interfaces	143
3.8.5.1	Interface Mode	143
3.8.5.2	Routing Mode	144
3.8.5.3	Link Operating Speed	144
3.8.5.4	Time-codes	145
3.8.6	SpaceWire Interface Tristate Mode	145
3.8.7	Electrical Characteristics	149
3.8.7.1	Absolute Maximum Ratings	149
3.8.7.2	SpaceWire Functional Diagram	150
3.8.8	Switching Characteristics	152
3.9	SpaceWire GbE Brick Mk1	152
3.9.1	Overview	152
3.9.1.1	Summary	153
3.9.1.2	Hardware Description	153
3.9.2	Getting Started	153
3.9.3	Rear Panel Interfaces	155
3.9.3.1	Ethernet Status LED	156
3.9.4	Front Panel Interfaces	156
3.9.5	Functions	158
3.9.5.1	SpaceWire Router	158

3.9.5.2	Interface Mode	158
3.9.5.3	Routing Mode	159
3.9.5.4	Link Operating Speed	160
3.9.5.5	Time-codes	160
3.9.6	SpaceWire Interface Tristate Mode	161
3.9.7	Electrical Characteristics	163
3.9.7.1	SpaceWire Connectors	163
3.9.7.2	External Trigger Connectors	164
3.9.7.3	FMECA Protection	166
3.9.8	Switching Characteristics	166
3.9.9	Grounding Information	167
3.10	Other STAR-System Devices	168
3.10.1	Devices with Interface Capabilities	168
3.10.1.1	SpaceWire Physical Layer Tester	168
3.10.1.2	STAR Fire	168
3.10.1.3	STAR Fire Mk3	168
3.10.1.4	STAR-Ultra PCIe Interface	168
3.10.1.5	STAR-Ultra PCIe Single-Lane Router	168
3.10.2	Devices without Interface Capabilities	169
3.10.2.1	SpaceWire Conformance Tester Mk2	169
3.10.2.2	SpaceWire EGSE and SpaceWire EGSE Mk2	169
3.10.2.3	SpaceWire Link Analyser Mk3	169
3.10.2.4	SpaceWire Recorder and SpaceWire Recorder Mk2	169
3.11	Device Update Application	169
3.11.1	Using the Device Update Application	169
3.11.1.1	Before Running the Device Update Application	169
3.11.1.2	Performing a Device Update	170
3.11.1.3	Cancelling a Device Update	171
3.11.1.4	Errors During a Device Update	172
3.11.1.5	Other Messages and Notifications	172
4	Other Useful Information	175
4.1	VxWorks Installation and Getting Started Guide	175
4.1.1	Introduction	175
4.1.2	Installation Contents	175
4.1.3	VxWorks Image Include Components	176
4.1.4	Initialising The Driver	176
4.1.5	Quick Start Guide	177
4.1.5.1	Running The Pre-compiled Test Programs	177
4.1.5.2	Building the example programs	177

4.2	RTEMS Installation and Getting Started Guide	178
4.2.1	Introduction	178
4.2.2	Supported Targets	178
4.2.3	Installation	178
4.2.4	Driver Module	178
4.2.5	Device Configuration Service	178
4.2.6	RTEMS Workspace Configuration Options	178
4.3	Multi-Threading and Multi-Process Support	179
4.3.1	Multi-Threading Support	179
4.3.2	Multi-Process Support	179
4.4	Device Configuration Service	179
4.4.1	Channel 0	180
4.4.2	Windows Session 0	180
4.5	Error Logs	180
4.5.1	Log File Name and Location	181
4.5.2	Log File Format	181
4.6	Configuration API Functions Supported by Device Types	182
4.6.1	Generic Configuration API	182
4.6.2	SpaceWire cPCI Mk2 and SpaceWire PCI Mk2	195
4.6.3	SpaceWire PCIe and SpaceWire Physical Layer Tester	197
4.6.4	SpaceWire Brick Mk2	199
4.6.5	SpaceWire Router Mk2S	201
4.6.6	SpaceWire Brick Mk3 and Brick Mk4	203
4.6.7	SpaceWire PXI Devices	205
4.6.8	SpaceWire GbE Brick Mk1	207
4.6.9	SpaceWire PCIe Mk2	208
4.6.10	STAR-Ultra PCIe Interface and STAR-Ultra PCIe Single-Lane Router	211
4.7	Static Analysis	211
4.8	Open Source Components	211
4.8.1	Qt4 and Qt5 Libraries	211
4.8.2	Bip Buffer	230
4.9	Change Log	233
4.9.1	STAR-System v6.01 - 13th November 2025	233
4.9.2	STAR-System v6.00 - 7th October 2024	235
4.9.3	STAR-System v6.00 Beta 3 - 5th October 2023	236
4.9.4	STAR-System v6.00 Beta 2 - 29th August 2023	236
4.9.5	STAR-System v6.00 Beta 1 - 6th July 2023	237
4.9.6	STAR-System v5.04 - 20th February 2023	237
4.9.7	STAR-System v5.03 - 18th January 2023	237
4.9.8	STAR-System v5.02 - 25th November 2022	238

4.9.9	STAR-System v5.01 - 16th September 2022	238
4.9.10	STAR-System v5.00 - 17th February 2022	239
4.9.11	STAR-System v5.00 Beta 3 - 12th November 2021	240
4.9.12	STAR-System v5.00 Beta 2 - 30th September 2021	240
4.9.13	STAR-System v5.00 Beta 1 - 24th March 2021	241
4.9.14	STAR-System v4.05 - 29th January 2021	242
4.9.15	STAR-System v4.04 - 11th September 2020	242
4.9.16	STAR-System v4.03 - 28th July 2020	242
4.9.17	STAR-System v4.02 - 13th March 2020	243
4.9.18	STAR-System v4.01 - 29th November 2019	243
4.9.19	STAR-System v4.00 - 19th November 2019	243
4.9.20	STAR-System v4.00 Beta 4 - 16th October 2019	245
4.9.21	STAR-System v4.00 Beta 3 - 24th October 2018	245
4.9.22	STAR-System v4.00 Beta 2 - 16th October 2018	246
4.9.23	STAR-System v4.00 Beta 1 - 25th September 2018	246
4.9.24	STAR-System v3.10 - 23rd February 2018	247
4.9.25	STAR-System v3.9 - 6th December 2017	247
4.9.26	STAR-System v3.8 - 9th November 2017	247
4.9.27	STAR-System v3.7 - 13th October 2017	247
4.9.28	STAR-System v3.6 - 3rd April 2017	248
4.9.29	STAR-System v3.5 - 17th March 2017	248
4.9.30	STAR-System v3.4 - 9th March 2017	248
4.9.31	STAR-System v3.3 - 20th December 2016	249
4.9.32	STAR-System v3.2 - 24th November 2016	249
4.9.33	STAR-System v3.1 - 21st October 2016	249
4.9.34	STAR-System v3.0 - 26th August 2016	250
4.9.35	STAR-System v3.0 Beta 9 - 26th July 2016	251
4.9.36	STAR-System v3.0 Beta 8 - 6th May 2016	251
4.9.37	STAR-System v3.0 Beta 7 - 8th March 2016	251
4.9.38	STAR-System v3.0 Beta 6 - 21st January 2016	252
4.9.39	STAR-System v3.0 Beta 5 - 25th November 2015	252
4.9.40	STAR-System v3.0 Beta 4 - 13th November 2015	252
4.9.41	STAR-System v3.0 Beta 3 - 4th September 2015	252
4.9.42	STAR-System v3.0 Beta 2 - 24th July 2015	252
4.9.43	STAR-System v3.0 Beta 1 - 31st March 2015	253
4.9.44	STAR-System v2.4.1 - 4th August 2014	254
4.9.45	STAR-System v2.4 - 2nd May 2014	254
4.9.46	STAR-System v2.3 - 31st January 2014	254
4.9.47	STAR-System v2.2 - 5th November 2013	255
4.9.48	STAR-System v2.1 - 1st August 2013	255

4.9.49	STAR-System v2.0 - 3rd July 2013	256
4.9.50	STAR-System v2.0 Beta 4 - 21st May 2013	256
4.9.51	STAR-System v2.0 Beta 3 - 12th April 2013	257
4.9.52	STAR-System v2.0 Beta 2 - 19th March 2013	258
4.9.53	STAR-System v2.0 Beta 1 - 8th February 2013	258
4.9.54	STAR-System v1.12 - 14th November 2012	259
4.9.55	STAR-System v1.11 - 31st October 2012	259
4.9.56	STAR-System v1.10 - 27th September 2012	260
4.9.57	STAR-System v1.9 - 7th September 2012	260
4.9.58	STAR-System v1.8 - 24th August 2012	260
4.9.59	STAR-System v1.7 - 20th July 2012	260
4.9.60	STAR-System v1.6 - 29th June 2012	261
4.9.61	STAR-System v1.5 - 23rd April 2012	261
4.9.62	STAR-System v1.4 - 16th March 2012	262
4.9.63	STAR-System v1.3 - 14th February 2012	263
4.9.64	STAR-System v1.2 - 13th February 2012	263
4.9.65	STAR-System v1.1 - 20th December 2011	263
4.9.66	STAR-System v1.0 - 17th October 2011	265
4.9.67	STAR-System v0.8 (Beta 1) - 22nd August 2011	266
5	Deprecated List	267
6	Module Index	269
6.1	STAR-System C APIs	269
7	File Index	273
7.1	File List	273
8	Module Documentation	275
8.1	STAR-API	275
8.1.1	Detailed Description	276
8.1.2	Introduction	276
8.1.3	Structure	276
8.2	General API Functions	277
8.2.1	Detailed Description	277
8.2.2	Typedef Documentation	278
8.2.2.1	STAR_APP_ID	278
8.2.3	Function Documentation	278
8.2.3.1	STAR_destroyString(_Post_ptr_invalid_ In_z_ char *string)	278
8.2.3.2	STAR_destroyVersionInfo(_Post_ptr_invalid_ STAR_VERSION_INFO *p↔ VersionInfo)	278

8.2.3.3	STAR_destroyVersionInfoList(_Post_ptr_invalid_ STAR_VERSION_INFO *p↔ VersionInfoList)	278
8.2.3.4	STAR_getAllVersions(_Out_ U32 *count)	279
8.2.3.5	STAR_getApiVersion(void)	279
8.2.3.6	STAR_getApplicationAttachedToChannel(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	279
8.2.3.7	STAR_getApplicationID(void)	280
8.2.3.8	STAR_getApplicationName(void)	280
8.2.3.9	STAR_getApplicationNameForID(STAR_APP_ID appld)	280
8.2.3.10	STAR_setApplicationName(_In_z_ char *name)	281
8.3	Device and Driver Management	282
8.3.1	Detailed Description	286
8.3.2	Macro Definition Documentation	286
8.3.2.1	STAR_CHANNEL_MASK	286
8.3.2.2	STAR_DEVICE_ALL	287
8.3.2.3	STAR_DEVICE_BRICK_MK2	287
8.3.2.4	STAR_DEVICE_BRICK_MK3	287
8.3.2.5	STAR_DEVICE_BRICK_MK4	287
8.3.2.6	STAR_DEVICE_CONFIG_NOT_SUPPORTED	287
8.3.2.7	STAR_DEVICE_CONFIG_SUPPORTED	288
8.3.2.8	STAR_DEVICE_CONFORMANCE_TESTER	288
8.3.2.9	STAR_DEVICE_CONFORMANCE_TESTER_MK2	288
8.3.2.10	STAR_DEVICE_CPCI_MK2	288
8.3.2.11	STAR_DEVICE_EGSE	288
8.3.2.12	STAR_DEVICE_EGSE_MK2	288
8.3.2.13	STAR_DEVICE_IP_TUNNEL	289
8.3.2.14	STAR_DEVICE_LINK_ANALYSER	289
8.3.2.15	STAR_DEVICE_LINK_ANALYSER_MK2	289
8.3.2.16	STAR_DEVICE_LINK_ANALYSER_MK3	289
8.3.2.17	STAR_DEVICE_PCI	289
8.3.2.18	STAR_DEVICE_PCI_MK2	289
8.3.2.19	STAR_DEVICE_PCIE	290
8.3.2.20	STAR_DEVICE_PCIE_MK2	290
8.3.2.21	STAR_DEVICE_PXI_INTERFACE	290
8.3.2.22	STAR_DEVICE_PXI_INTERFACE_MK2	290
8.3.2.23	STAR_DEVICE_PXI_RMAP	290
8.3.2.24	STAR_DEVICE_PXI_RMAP_MK2	291
8.3.2.25	STAR_DEVICE_PXI_ROUTER_12	291
8.3.2.26	STAR_DEVICE_PXI_ROUTER_8	291
8.3.2.27	STAR_DEVICE_PXI_ROUTER_MK2	291

8.3.2.28	STAR_DEVICE_RECORDER	291
8.3.2.29	STAR_DEVICE_RECORDER_MK2	292
8.3.2.30	STAR_DEVICE_ROUTER_MK2S	292
8.3.2.31	STAR_DEVICE_ROUTER_USB	292
8.3.2.32	STAR_DEVICE_RTC	292
8.3.2.33	STAR_DEVICE_SERIAL_SIZE	292
8.3.2.34	STAR_DEVICE_SPLT	292
8.3.2.35	STAR_DEVICE_STAR_FIRE	293
8.3.2.36	STAR_DEVICE_STAR_FIRE_MK3	293
8.3.2.37	STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE	293
8.3.2.38	STAR_DEVICE_STAR_ULTRA_PCIE_SL_ROUTER	293
8.3.2.39	STAR_DEVICE_TRAFFIC_GENERATOR	293
8.3.2.40	STAR_DEVICE_TXRX_NOT_SUPPORTED	293
8.3.2.41	STAR_DEVICE_TXRX_SUPPORTED	294
8.3.2.42	STAR_DEVICE_TYPE	294
8.3.2.43	STAR_DEVICE_UNKNOWN	294
8.3.2.44	STAR_DEVICE_USB_BRICK	294
8.3.2.45	STAR_DEVICE_WBS_II	294
8.3.3	Typedef Documentation	295
8.3.3.1	STAR_DEVICE_ID	295
8.3.3.2	STAR_DEVICE_LISTENER_ID	295
8.3.3.3	STAR_DeviceEventFunc	295
8.3.3.4	STAR_DRIVER_ID	295
8.3.3.5	STAR_DRIVER_LISTENER_ID	295
8.3.3.6	STAR_DriverEventFunc	295
8.3.4	Enumeration Type Documentation	296
8.3.4.1	STAR_BUS_TYPE	296
8.3.4.2	STAR_DRIVER_TYPE	296
8.3.5	Function Documentation	297
8.3.5.1	STAR_destroyDeviceList(_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)	297
8.3.5.2	STAR_destroyDriverList(_Post_ptr_invalid_ STAR_DRIVER_ID *pDriverList)	298
8.3.5.3	STAR_getDeviceBusType(STAR_DEVICE_ID deviceId)	298
8.3.5.4	STAR_getDeviceBusTypeAsString(STAR_DEVICE_ID deviceId)	298
8.3.5.5	STAR_getDeviceChannels(STAR_DEVICE_ID deviceId)	299
8.3.5.6	STAR_getDeviceConfigCapabilities(STAR_DEVICE_ID deviceId)	299
8.3.5.7	STAR_getDeviceDriver(STAR_DEVICE_ID deviceId)	300
8.3.5.8	STAR_getDeviceFirmwareVersion(STAR_DEVICE_ID deviceId)	300
8.3.5.9	STAR_getDeviceIndex(STAR_DEVICE_ID deviceId)	300
8.3.5.10	STAR_getDeviceList(_Out_ U32 *count)	301
8.3.5.11	STAR_getDeviceListForDriver(STAR_DRIVER_ID driverId, _Out_ U32 *count)	302

8.3.5.12	STAR_getDeviceListForType(STAR_DEVICE_TYPE deviceType, _Out_ U32 *pCount)	302
8.3.5.13	STAR_getDeviceListForTypes(STAR_DEVICE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)	303
8.3.5.14	STAR_getDeviceName(STAR_DEVICE_ID deviceId)	304
8.3.5.15	STAR_getDeviceSerialNumber(STAR_DEVICE_ID deviceId)	304
8.3.5.16	STAR_getDeviceTxRxCapabilities(STAR_DEVICE_ID deviceId)	304
8.3.5.17	STAR_getDeviceType(STAR_DEVICE_ID deviceId)	306
8.3.5.18	STAR_getDeviceTypeAsString(STAR_DEVICE_ID deviceId)	306
8.3.5.19	STAR_getDriverList(int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 *count)	307
8.3.5.20	STAR_getDriverType(STAR_DRIVER_ID driverId)	308
8.3.5.21	STAR_getDriverVersion(STAR_DRIVER_ID driverId)	308
8.3.5.22	STAR_getLocalDeviceAttachedToChannel(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	309
8.3.5.23	STAR_isChannelOpen(STAR_DEVICE_ID deviceId, unsigned int channelNumber)	309
8.3.5.24	STAR_registerDeviceListener(_In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	309
8.3.5.25	STAR_registerDeviceListenerForDevice(STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)	310
8.3.5.26	STAR_registerDeviceListenerForDriver(STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	310
8.3.5.27	STAR_registerDriverListener(STAR_DriverEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	310
8.3.5.28	STAR_resetDevice(STAR_DEVICE_ID deviceId)	311
8.3.5.29	STAR_resetDeviceName(STAR_DEVICE_ID deviceId)	311
8.3.5.30	STAR_setDeviceName(STAR_DEVICE_ID deviceId, _In_z_ char *newName)	311
8.3.5.31	STAR_unregisterDeviceListener(STAR_DEVICE_LISTENER_ID listenerId)	312
8.3.5.32	STAR_unregisterDriverListener(STAR_DRIVER_LISTENER_ID listenerId)	312
8.4	Channel Management	313
8.4.1	Detailed Description	314
8.4.2	Typedef Documentation	314
8.4.2.1	STAR_CHANNEL_ID	314
8.4.2.2	STAR_CHANNEL_LISTENER_ID	314
8.4.2.3	STAR_ChannelEventFunc	314
8.4.3	Enumeration Type Documentation	315
8.4.3.1	STAR_CHANNEL_DIRECTION	315
8.4.3.2	STAR_CHANNEL_TYPE	315
8.4.4	Function Documentation	315
8.4.4.1	STAR_closeChannel(STAR_CHANNEL_ID channelId)	315
8.4.4.2	STAR_getChannelDirection(STAR_CHANNEL_ID channelId)	316

8.4.4.3	STAR_openChannelBetweenLocalDevices(STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB) . .	316
8.4.4.4	STAR_openChannelToLocalDevice(STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued) . . .	317
8.4.4.5	STAR_openChannelToLocalDeviceEx(_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)	318
8.4.4.6	STAR_registerChannelListener(_In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	318
8.4.4.7	STAR_registerChannelListenerForDevice(STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	319
8.4.4.8	STAR_registerChannelListenerForDriver(STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)	319
8.4.4.9	STAR_unregisterChannelListener(STAR_CHANNEL_LISTENER_ID listenerId) .	320
8.5	Stream Items	321
8.5.1	Detailed Description	324
8.5.2	Data Structure Documentation	324
8.5.2.1	struct STAR_IP_ADDRESS_TYPE	324
8.5.2.2	struct STAR_LINK_STATE_EVENT	325
8.5.2.3	struct STAR_SPACEWIRE_ADDRESS	326
8.5.2.4	struct STAR_DATA_CHUNK	327
8.5.2.5	struct STAR_SPACEWIRE_PACKET	328
8.5.2.6	struct STAR_TIMECODE	329
8.5.2.7	struct STAR_LINK_SPEED_EVENT	329
8.5.2.8	struct STAR_ERROR_IN_DATA_INJECT	330
8.5.2.9	struct STAR_TIMESTAMP_EVENT	331
8.5.2.10	struct STAR_BROADCAST_MESSAGE	332
8.5.2.11	struct STAR_STREAM_ITEM	333
8.5.3	Enumeration Type Documentation	334
8.5.3.1	STAR_BROADCAST_MESSAGE_STATUS_FLAG	334
8.5.3.2	STAR_EOP_TYPE	334
8.5.3.3	STAR_ERROR_IN_DATA_TYPE	335
8.5.3.4	STAR_IP_VERSION_TYPE	335
8.5.3.5	STAR_STREAM_ITEM_TYPE	335
8.5.3.6	STAR_TIMESTAMP_DIRECTION	336
8.5.3.7	STAR_TIMESTAMP_TYPE	336
8.5.4	Function Documentation	336
8.5.4.1	STAR_createAddress(_In_count_(pathLen) unsigned char *path, U16 pathLen) .	336
8.5.4.2	STAR_createBroadcastMessage(_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)	337

8.5.4.3	STAR_createDataChunk(_In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, int isStart, STAR_EOP_TYPE eopType)	337
8.5.4.4	STAR_createErrorInData(STAR_ERROR_IN_DATA_TYPE errorType)	338
8.5.4.5	STAR_createPacket(_In_opt_ STAR_SPACEWIRE_ADDRESS *address, _In_opt_count_(dataLen) unsigned char *data, unsigned int dataLen, STAR_EOP_TYPE eopType)	338
8.5.4.6	STAR_createTimeCode(U8 value)	339
8.5.4.7	STAR_destroyAddress(_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address)	339
8.5.4.8	STAR_destroyPacketData(_In_ _Post_ptr_invalid_ unsigned char *pData)	340
8.5.4.9	STAR_destroyStreamItem(_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item)	340
8.5.4.10	STAR_getBroadcastMessageChannel(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	341
8.5.4.11	STAR_getBroadcastMessageDataWord1(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	342
8.5.4.12	STAR_getBroadcastMessageDataWord2(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	342
8.5.4.13	STAR_getBroadcastMessageDelayedFlag(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	343
8.5.4.14	STAR_getBroadcastMessageLateFlag(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	344
8.5.4.15	STAR_getBroadcastMessageStatus(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	344
8.5.4.16	STAR_getBroadcastMessageType(_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	344
8.5.4.17	STAR_getChunkData(_In_ STAR_DATA_CHUNK *chunk, _Out_ unsigned int *len)	345
8.5.4.18	STAR_getChunkDataLength(_In_ STAR_DATA_CHUNK *chunk)	345
8.5.4.19	STAR_getChunkEop(_In_ STAR_DATA_CHUNK *chunk)	345
8.5.4.20	STAR_getChunkSop(_In_ STAR_DATA_CHUNK *chunk)	346
8.5.4.21	STAR_getLinkSpeedEventPort(_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)	346
8.5.4.22	STAR_getLinkSpeedEventSpeed(_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)	346
8.5.4.23	STAR_getLinkStateEventCount(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	347
8.5.4.24	STAR_getLinkStateEventPort(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	347
8.5.4.25	STAR_getPacketAddress(_In_ STAR_SPACEWIRE_PACKET *packet)	348
8.5.4.26	STAR_getPacketData(_In_ STAR_SPACEWIRE_PACKET *packet, _Out_ unsigned int *dataLength)	348
8.5.4.27	STAR_getPacketEOP(_In_ STAR_SPACEWIRE_PACKET *packet)	348
8.5.4.28	STAR_getPacketLength(_In_ STAR_SPACEWIRE_PACKET *packet)	349
8.5.4.29	STAR_getTimeCodeCount(_In_ STAR_TIMECODE *pTimeCode)	349
8.5.4.30	STAR_getTimeCodeValue(_In_ STAR_TIMECODE *pTimeCode)	349

8.5.4.31	STAR_getTimestampEventDirection(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)	350
8.5.4.32	STAR_getTimestampEventEndValue(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)	350
8.5.4.33	STAR_getTimestampEventRawCounters(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)	351
8.5.4.34	STAR_getTimestampEventStartValue(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)	352
8.5.4.35	STAR_getTimestampEventType(_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)	352
8.5.4.36	STAR_isLinkStateEventDisconnectError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	353
8.5.4.37	STAR_isLinkStateEventEscapeError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	353
8.5.4.38	STAR_isLinkStateEventLinkRunning(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	353
8.5.4.39	STAR_isLinkStateEventParityError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	354
8.5.4.40	STAR_isLinkStateEventReceiveCreditError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	354
8.5.4.41	STAR_isLinkStateEventTransmitCreditError(_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	354
8.5.4.42	STAR_setPacketAddress(_Inout_ STAR_SPACEWIRE_PACKET *packet, _In_ STAR_SPACEWIRE_ADDRESS *address)	355
8.5.4.43	STAR_setPacketEOP(_In_ STAR_SPACEWIRE_PACKET *packet, STAR_ENDPOINT_TYPE eopType)	356
8.5.4.44	STAR_setTimeCodeValue(_In_ STAR_TIMECODE *pTimeCode, U8 value)	356
8.6	Transfer Operations	357
8.6.1	Detailed Description	359
8.6.2	Macro Definition Documentation	359
8.6.2.1	STAR_RECEIVE_NULL	359
8.6.3	Typedef Documentation	360
8.6.3.1	STAR_TRANSFER_OPERATION	360
8.6.3.2	STAR_TransferOperationListenerFunc	360
8.6.4	Enumeration Type Documentation	360
8.6.4.1	STAR_RECEIVE_MASK	360
8.6.4.2	STAR_TRANSFER_STATUS	361
8.6.5	Function Documentation	361
8.6.5.1	STAR_cancelTransferOperation(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	361

8.6.5.2	STAR_cancelTransferOperationWaits(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	362
8.6.5.3	STAR_createRxOperation(int itemCount, STAR_RECEIVE_MASK mask)	362
8.6.5.4	STAR_createRxOperationEx(_In_count_(numBuffers) U8 **ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ STAR_RECEIVE_MASK mask)	363
8.6.5.5	STAR_createTxOperation(_In_count_(streamItemCount) STAR_STREAM_ITEM *ppItems, unsigned int streamItemCount)	364
8.6.5.6	STAR_destroyTransferItemList(_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **transferItemList, unsigned int transferItemListCount, int destroyItems)	364
8.6.5.7	STAR_disposeTransferOperation(_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION *pTransferOperation)	365
8.6.5.8	STAR_getNextTransferItem(STAR_STREAM_ITEM *pStreamItem)	365
8.6.5.9	STAR_getSpaceWireAddressPath(STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)	366
8.6.5.10	STAR_getSpaceWireAddressPathLength(STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)	366
8.6.5.11	STAR_getTransferItem(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, unsigned int index)	366
8.6.5.12	STAR_getTransferItemCount(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation)	367
8.6.5.13	STAR_getTransferItemList(_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)	367
8.6.5.14	STAR_getTransferOperationStatus(_In_ STAR_TRANSFER_OPERATION *pTransferOperation)	368
8.6.5.15	STAR_receivePacket(_In_ STAR_CHANNEL_ID channelId, _Out_bytecap_(*pPacketLength) void *pPacketData, _Inout_ unsigned int *pPacketLength, _Out_ STAR_EOP_TYPE *pEopType, _In_ int timeout)	368
8.6.5.16	STAR_registerTransferCompletionListener(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)	369
8.6.5.17	STAR_rxOperationExGetBroadcastMessage(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)	369
8.6.5.18	STAR_rxOperationExGetDataChunk(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)	370
8.6.5.19	STAR_rxOperationExGetItemType(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)	370
8.6.5.20	STAR_rxOperationExGetLinkSpeedEvent(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)	371
8.6.5.21	STAR_rxOperationExGetLinkStateEvent(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_STATE_EVENT *pLinkStateEvent)	372
8.6.5.22	STAR_rxOperationExGetNumItems(_In_ STAR_TRANSFER_OPERATION *pTransferOp)	372

8.6.5.23	STAR_rxOperationExGetStatus(_In_ STAR_TRANSFER_OPERATION *pTransferOp)	373
8.6.5.24	STAR_rxOperationExGetTimeCode(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMECODE *pTimeCode)	374
8.6.5.25	STAR_rxOperationExGetTimestampEvent(_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMESTAMP_EVENT *pTimestampEvent)	374
8.6.5.26	STAR_submitTransferOperation(_In_ STAR_CHANNEL_ID channelId, _In_ STAR_TRANSFER_OPERATION *transferOp)	375
8.6.5.27	STAR_submitTransferOperationList(STAR_CHANNEL_ID channelId, _In_ count(count) STAR_TRANSFER_OPERATION **transferOps, unsigned int count)	376
8.6.5.28	STAR_transmitPacket(_In_ STAR_CHANNEL_ID channelId, _In_opt_ bytecount(packetLength) void *pPacketData, _In_ unsigned int packetLength, _In_ STAR_EOP_TYPE eopType, _In_ int timeout)	377
8.6.5.29	STAR_unregisterTransferCompletionListener(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)	377
8.6.5.30	STAR_waitOnTransferOperationCompletion(_In_ STAR_TRANSFER_OPERATION *pTransferOperation, int timeout)	377
8.7	Virtual Devices and Drivers	379
8.7.1	Detailed Description	379
8.7.2	Function Documentation	379
8.7.2.1	STAR_isDeviceVirtual(STAR_DEVICE_ID deviceId)	379
8.7.2.2	STAR_isDriverVirtual(STAR_DRIVER_ID driverId)	379
8.8	Ethernet Devices and Drivers	381
8.8.1	Detailed Description	381
8.8.2	Function Documentation	381
8.8.2.1	STAR_closeEthernetDevice(STAR_DEVICE_ID deviceId)	381
8.8.2.2	STAR_closeEthernetDriver(STAR_DRIVER_ID driverId)	382
8.8.2.3	STAR_destroyIPAddresses(_Post_ptr_invalid_ STAR_IP_ADDRESS_TYPE **ppIPAdrs, _In_ U8 numDevices)	382
8.8.2.4	STAR_getDeviceIPAddress(STAR_DEVICE_ID deviceId, _Out_bytecap(IPAddressLength) U8 *pIPAddress, U8 IPAddressLength)	382
8.8.2.5	STAR_getIPAddresses(STAR_DRIVER_ID driverId, _Out_int *pNumDevices)	383
8.8.2.6	STAR_openEthernetDevice(unsigned char ipAddr[4])	383
8.8.2.7	STAR_openEthernetDriver(void)	383
8.9	Link Speed	384
8.9.1	Detailed Description	384
8.9.2	Enumeration Type Documentation	384
8.9.2.1	STAR_CFG_PCIMK2_LINK_FREQ	384
8.9.3	Function Documentation	385
8.9.3.1	CFG_PCIMK2_getLinkClockFrequency(STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ *pLinkFreq)	385

8.9.3.2	CFG_PCIMK2_setLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 link↔ Num, STAR_CFG_PCIMK2_LINK_FREQ linkFreq)	385
8.10	Link Speed	387
8.10.1	Detailed Description	387
8.10.2	Function Documentation	387
8.10.2.1	CFG_PCIE_MK2_getDecimalTxSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ double *pSignallingRate)	387
8.10.2.2	CFG_PCIE_MK2_getTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ U32 *pSignallingRate)	388
8.10.2.3	CFG_PCIE_MK2_setDecimalTxSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, double signallingRate)	388
8.10.2.4	CFG_PCIE_MK2_setTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate)	389
8.11	Broadcast Controller	390
8.11.1	Detailed Description	390
8.11.2	Function Documentation	390
8.11.2.1	CFG_PCIE_MK2_disableBroadcastControllerLegacyMode(STAR_DEVICE_ID deviceID)	390
8.11.2.2	CFG_PCIE_MK2_enableBroadcastControllerLegacyMode(STAR_DEVICE_ID deviceID)	391
8.11.2.3	CFG_PCIE_MK2_getBroadcastControllerLegacyModeEnabled(STAR_DEVICE↔ ID deviceID, _Out_ int *pEnabled)	391
8.11.2.4	CFG_PCIE_MK2_getInterruptCodeDistributionPorts(STAR_DEVICE_ID device↔ ID, _Out_ U32 *pPortMask)	392
8.11.2.5	CFG_PCIE_MK2_setInterruptCodeDistributionPorts(STAR_DEVICE_ID device↔ ID, U32 portMask)	393
8.12	Timestamping	394
8.12.1	Detailed Description	394
8.12.2	Function Documentation	394
8.12.2.1	CFG_PCIE_MK2_disableRxTimestampEventsOnPort(STAR_DEVICE_ID↔ D deviceID, U8 portNum)	394
8.12.2.2	CFG_PCIE_MK2_enableRxTimestampEventsOnPort(STAR_DEVICE_ID↔ D deviceID, U8 portNum)	395
8.12.2.3	CFG_PCIE_MK2_getPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, ↔ _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)	395
8.12.2.4	CFG_PCIE_MK2_getRxTimestampEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	396
8.12.2.5	CFG_PCIE_MK2_setPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)	396
8.12.2.6	CFG_PCIE_MK2_setTimestampMethod(STAR_DEVICE_ID deviceID, STAR↔ CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)	396
8.13	Error Injection	398
8.13.1	Detailed Description	398
8.13.2	Function Documentation	398

8.13.2.1	CFG_PCIE_MK2_injectErrors(STAR_DEVICE_ID deviceID, U8 portNum, _In_↔ STAR_CFG_BRICK_MK2_ERRORS *pErrors)	398
8.14	Link Speed	399
8.14.1	Detailed Description	399
8.14.2	Enumeration Type Documentation	399
8.14.2.1	STAR_CFG_BRICK_MK2_LINK_FREQ	399
8.14.3	Function Documentation	400
8.14.3.1	CFG_BRICK_MK2_getLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_BRICK_MK2_LINK_FREQ *pLinkFreq)	400
8.14.3.2	CFG_BRICK_MK2_setLinkClockFrequency(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_BRICK_MK2_LINK_FREQ linkFreq)	400
8.15	Interface Mode	402
8.15.1	Detailed Description	402
8.15.2	Function Documentation	402
8.15.2.1	CFG_BRICK_MK2_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	402
8.15.2.2	CFG_BRICK_MK2_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	403
8.15.2.3	CFG_BRICK_MK2_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	403
8.15.2.4	CFG_BRICK_MK2_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	404
8.15.2.5	CFG_BRICK_MK2_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	404
8.15.2.6	CFG_BRICK_MK2_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	405
8.15.2.7	CFG_BRICK_MK2_getPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)	405
8.15.2.8	CFG_BRICK_MK2_setPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, U8 address)	406
8.16	Error Injection	407
8.16.1	Detailed Description	407
8.16.2	Function Documentation	407
8.16.2.1	CFG_BRICK_MK2_injectErrors(STAR_DEVICE_ID deviceID, U8 portNum, _In_↔ _STAR_CFG_BRICK_MK2_ERRORS *pErrors)	407
8.17	Events	408
8.17.1	Detailed Description	408
8.17.2	Function Documentation	408
8.17.2.1	CFG_BRICK_MK2_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	408
8.17.2.2	CFG_BRICK_MK2_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	410
8.17.2.3	CFG_BRICK_MK2_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	410

8.17.2.4	CFG_BRICK_MK2_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	411
8.17.2.5	CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	411
8.17.2.6	CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	412
8.18	Link Speed	413
8.18.1	Detailed Description	413
8.18.2	Function Documentation	413
8.18.2.1	CFG_BRICK_MK3_getBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	413
8.18.2.2	CFG_BRICK_MK3_getTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	414
8.18.2.3	CFG_BRICK_MK3_setBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	414
8.18.2.4	CFG_BRICK_MK3_setTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	415
8.19	Timestamping	417
8.19.1	Detailed Description	418
8.19.2	Enumeration Type Documentation	418
8.19.2.1	STAR_CFG_BRICK_MK3_PULSE_FREQ	418
8.19.2.2	STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD	418
8.19.3	Function Documentation	418
8.19.3.1	CFG_BRICK_MK3_disableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	418
8.19.3.2	CFG_BRICK_MK3_enableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	419
8.19.3.3	CFG_BRICK_MK3_getPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)	419
8.19.3.4	CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	420
8.19.3.5	CFG_BRICK_MK3_getTimestampMethod(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pTimestampMethod)	420
8.19.3.6	CFG_BRICK_MK3_getTimestampValue(STAR_DEVICE_ID deviceID, _Out_ U32 *pValue)	421
8.19.3.7	CFG_BRICK_MK3_setPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)	421
8.19.3.8	CFG_BRICK_MK3_setTimestampMethod(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)	422
8.19.3.9	CFG_BRICK_MK3_setTimestampValue(STAR_DEVICE_ID deviceID, U32 value)	422
8.20	Link Speed	424
8.20.1	Detailed Description	424
8.20.2	Function Documentation	424

8.20.2.1	CFG_GBE_BRICK_MK1_getTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ U32 *pSignallingRate)	424
8.20.2.2	CFG_GBE_BRICK_MK1_setTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate)	425
8.21	Link Speed	426
8.21.1	Detailed Description	426
8.21.2	Function Documentation	426
8.21.2.1	CFG_PXI_getBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)	426
8.21.2.2	CFG_PXI_getLinkRateDivider(STAR_DEVICE_ID deviceID, _Out_ U8 *pDivider)	427
8.21.2.3	CFG_PXI_getTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)	427
8.21.2.4	CFG_PXI_setBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	428
8.21.2.5	CFG_PXI_setLinkRateDivider(STAR_DEVICE_ID deviceID, U8 divider)	429
8.21.2.6	CFG_PXI_setTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	429
8.22	Error Injection	431
8.22.1	Detailed Description	431
8.22.2	Function Documentation	431
8.22.2.1	CFG_PXI_injectError(STAR_DEVICE_ID deviceID, unsigned char port, SPW_ERROR error)	431
8.23	Interface Mode	432
8.23.1	Detailed Description	432
8.23.2	Function Documentation	432
8.23.2.1	CFG_PXI_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	432
8.23.2.2	CFG_PXI_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	433
8.23.2.3	CFG_PXI_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	433
8.23.2.4	CFG_PXI_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	434
8.23.2.5	CFG_PXI_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	434
8.23.2.6	CFG_PXI_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	434
8.23.2.7	CFG_PXI_getPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)	435
8.23.2.8	CFG_PXI_setPortRoutingAddress(STAR_DEVICE_ID deviceID, U8 portNum, U8 address)	435
8.24	Events	437
8.24.1	Detailed Description	437
8.24.2	Function Documentation	437
8.24.2.1	CFG_PXI_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	437
8.24.2.2	CFG_PXI_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	438

8.24.2.3	CFG_PXI_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	438
8.24.2.4	CFG_PXI_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	439
8.24.2.5	CFG_PXI_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	439
8.24.2.6	CFG_PXI_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	440
8.24.2.7	CFG_PXI_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	440
8.24.2.8	CFG_PXI_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	440
8.24.2.9	CFG_PXI_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	441
8.25	RMAP Target API	442
8.25.1	Detailed Description	442
8.26	Manual Authorisation	444
8.26.1	Detailed Description	444
8.26.2	Data Structure Documentation	444
8.26.2.1	struct RmapCommandParameters	444
8.26.3	Enumeration Type Documentation	445
8.26.3.1	REJECTION_REASON	445
8.26.4	Function Documentation	446
8.26.4.1	RMAP_TARGET_PXI_IF_authouriseWaitingCommand(STAR_DEVICE_ID deviceId, U32 target)	446
8.26.4.2	RMAP_TARGET_PXI_IF_getWaitingCommand(STAR_DEVICE_ID deviceId, U32 target, RmapCommandParameters *pCommand)	446
8.26.4.3	RMAP_TARGET_PXI_IF_isAuthRequired(STAR_DEVICE_ID deviceId, U32 target)	446
8.26.4.4	RMAP_TARGET_PXI_IF_rejectWaitingCommand(STAR_DEVICE_ID deviceId, U32 target, REJECTION_REASON reason)	447
8.27	Configuration	448
8.27.1	Detailed Description	450
8.27.2	Data Structure Documentation	450
8.27.2.1	struct TARGET_ENGINE	450
8.27.3	Typedef Documentation	451
8.27.3.1	TARGET_AUTH_CMD_MASK	451
8.27.4	Enumeration Type Documentation	451
8.27.4.1	IF_MODE	451
8.27.4.2	TARGET_AUTH_CMD	451
8.27.4.3	TARGET_AUTH_MODE	451
8.27.4.4	TARGET_STATUS	451
8.27.5	Function Documentation	452

8.27.5.1	RMAP_TARGET_PXI_IF_getAddressOffset(STAR_DEVICE_ID deviceId, U32 target, U32 *pOffset)	452
8.27.5.2	RMAP_TARGET_PXI_IF_getAuthCommands(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK *pCommands)	452
8.27.5.3	RMAP_TARGET_PXI_IF_getAuthControlMode(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE *pMode)	452
8.27.5.4	RMAP_TARGET_PXI_IF_getAuthKeyRange(STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)	453
8.27.5.5	RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange(STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)	453
8.27.5.6	RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange(STAR_DEVICE_ID deviceId, U32 target, U32 *pLowerBoundary, U32 *pUpperBoundary)	454
8.27.5.7	RMAP_TARGET_PXI_IF_getAuthProtocolId(STAR_DEVICE_ID deviceId, U32 target, U8 *pProtocolId)	454
8.27.5.8	RMAP_TARGET_PXI_IF_getInterfaceMode(STAR_DEVICE_ID deviceId, U32 port, IF_MODE *pMode)	455
8.27.5.9	RMAP_TARGET_PXI_IF_getStatus(STAR_DEVICE_ID deviceId, U32 target, TARGET_STATUS *pStatus)	456
8.27.5.10	RMAP_TARGET_PXI_IF_readMemory(STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, unsigned char *pBuffer)	456
8.27.5.11	RMAP_TARGET_PXI_IF_setAddressOffset(STAR_DEVICE_ID deviceId, U32 target, U32 offset)	457
8.27.5.12	RMAP_TARGET_PXI_IF_setAuthCommands(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK commands)	457
8.27.5.13	RMAP_TARGET_PXI_IF_setAuthControlMode(STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE mode)	458
8.27.5.14	RMAP_TARGET_PXI_IF_setAuthKeyRange(STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)	458
8.27.5.15	RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange(STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)	459
8.27.5.16	RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange(STAR_DEVICE_ID deviceId, U32 target, U32 lowerBoundary, U32 upperBoundary)	459
8.27.5.17	RMAP_TARGET_PXI_IF_setAuthProtocolId(STAR_DEVICE_ID deviceId, U32 target, U8 protocolId)	460
8.27.5.18	RMAP_TARGET_PXI_IF_setInterfaceMode(STAR_DEVICE_ID deviceId, U32 port, IF_MODE mode)	460
8.27.5.19	RMAP_TARGET_PXI_IF_writeMemory(STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, const unsigned char *pBuffer)	460
8.28	Notifications	462
8.28.1	Detailed Description	463
8.28.2	Typedef Documentation	463
8.28.2.1	NOTIF_TYPE_MASK	463
8.28.2.2	NotifListenerFunc	463
8.28.2.3	NotifListenerWithContextFunc	464
8.28.3	Enumeration Type Documentation	464
8.28.3.1	NOTIF_TYPE	464

8.28.4	Function Documentation	464
8.28.4.1	RMAP_TARGET_PXI_IF_getEnabledNotifications(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK *pNotifications)	465
8.28.4.2	RMAP_TARGET_PXI_IF_registerNotificationListener(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type, NotifListenerFunc listenerFunc)	466
8.28.4.3	RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type, NotifListenerWithContextFunc listenerFunc, void *pContext)	466
8.28.4.4	RMAP_TARGET_PXI_IF_setEnabledNotifications(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK notifications)	467
8.28.4.5	RMAP_TARGET_PXI_IF_startReceivingNotifications(STAR_DEVICE_ID deviceld)	467
8.28.4.6	RMAP_TARGET_PXI_IF_stopReceivingNotifications(STAR_DEVICE_ID deviceld)	468
8.28.4.7	RMAP_TARGET_PXI_IF_unregisterNotificationListener(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type)	468
8.28.4.8	RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext(STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE type)	468
8.29	Generic Triggering API	470
8.29.1	Detailed Description	471
8.29.2	Input Events	471
8.29.3	Output Actions	477
8.29.4	Internal Triggers	481
8.29.5	Triggering Resources	483
8.29.6	PCIe Differences	487
8.29.7	Examples	487
8.29.7.1	Transmitting Time-Codes Synchronised by an External Trigger	487
8.29.7.2	Packet Transmission Synchronised by a Counter	488
8.29.7.3	Multiple Packet Transmission Synchronised by Time-Codes	488
8.30	General	490
8.30.1	Detailed Description	490
8.30.2	Function Documentation	490
8.30.2.1	TRIGGER_CFG_getDeviceClockFreq(const STAR_DEVICE_ID deviceld, U32 *const pClockFreqHz)	490
8.30.2.2	TRIGGER_CFG_getDeviceClockRate(const STAR_DEVICE_ID deviceld)	491
8.30.2.3	TRIGGER_CFG_getTriggerMatrix(const STAR_DEVICE_ID deviceld)	491
8.30.2.4	TRIGGER_CFG_reset(const STAR_DEVICE_ID deviceld)	491
8.31	External Triggers	493
8.31.1	Detailed Description	494
8.31.2	Function Documentation	494
8.31.2.1	TRIGGER_CFG_disableExtTriggerEdgeDetectMode(const STAR_DEVICE_ID deviceld, const U32 extTrigger)	494
8.31.2.2	TRIGGER_CFG_disableExtTriggerInvert(const STAR_DEVICE_ID deviceld, const U32 extTrigger)	494

8.31.2.3	TRIGGER_CFG_disableExtTriggerOutput(const STAR_DEVICE_ID deviceId, const U32 extTrigger)	495
8.31.2.4	TRIGGER_CFG_enableExtTriggerEdgeDetectMode(const STAR_DEVICE_ID deviceId, const U32 extTrigger)	495
8.31.2.5	TRIGGER_CFG_enableExtTriggerInvert(const STAR_DEVICE_ID deviceId, const U32 extTrigger)	496
8.31.2.6	TRIGGER_CFG_enableExtTriggerOutput(const STAR_DEVICE_ID deviceId, const U32 extTrigger)	496
8.31.2.7	TRIGGER_CFG_getExtTriggerEdgeDetectModeEnabled(const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)	497
8.31.2.8	TRIGGER_CFG_getExtTriggerExtend(const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pExtend)	497
8.31.2.9	TRIGGER_CFG_getExtTriggerInputEvents(const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_EVENT_MASK *const pEvents)	498
8.31.2.10	TRIGGER_CFG_getExtTriggerInvertEnabled(const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)	498
8.31.2.11	TRIGGER_CFG_getExtTriggerOutputActions(const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_ACTION_MASK *const pActions)	499
8.31.2.12	TRIGGER_CFG_getExtTriggerOutputEnabled(const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)	499
8.31.2.13	TRIGGER_CFG_getNumberOfExtTriggers(const STAR_DEVICE_ID deviceId, U32 *const pNumExtTriggers)	499
8.31.2.14	TRIGGER_CFG_setExtTriggerExtend(const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 extend)	500
8.31.2.15	TRIGGER_CFG_setExtTriggerInputEvents(const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_EVENT_MASK events)	500
8.31.2.16	TRIGGER_CFG_setExtTriggerOutputActions(const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_ACTION_MASK actions)	501
8.32	Counters	502
8.32.1	Detailed Description	503
8.32.2	Function Documentation	503
8.32.2.1	TRIGGER_CFG_disableCounterAutoReload(const STAR_DEVICE_ID deviceId, const U32 counter)	503
8.32.2.2	TRIGGER_CFG_disableCounterLoadZero(const STAR_DEVICE_ID deviceId, const U32 counter)	504
8.32.2.3	TRIGGER_CFG_disableCounterStartMode(const STAR_DEVICE_ID deviceId, const U32 counter)	504
8.32.2.4	TRIGGER_CFG_disableCounterStartStopMode(const STAR_DEVICE_ID deviceId, const U32 counter)	505
8.32.2.5	TRIGGER_CFG_disableCounterTriggerCount(const STAR_DEVICE_ID deviceId, const U32 counter)	505
8.32.2.6	TRIGGER_CFG_enableCounterAutoReload(const STAR_DEVICE_ID deviceId, const U32 counter)	506

8.32.2.7	TRIGGER_CFG_enableCounterLoadZero(const STAR_DEVICE_ID deviceld, const U32 counter)	506
8.32.2.8	TRIGGER_CFG_enableCounterStartMode(const STAR_DEVICE_ID deviceld, const U32 counter)	507
8.32.2.9	TRIGGER_CFG_enableCounterStartStopMode(const STAR_DEVICE_ID deviceld, const U32 counter)	507
8.32.2.10	TRIGGER_CFG_enableCounterTriggerCount(const STAR_DEVICE_ID deviceld, const U32 counter)	508
8.32.2.11	TRIGGER_CFG_forceCounterReload(const STAR_DEVICE_ID deviceld, const U32 counter)	508
8.32.2.12	TRIGGER_CFG_getCounterAutoReloadEnabled(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled)	509
8.32.2.13	TRIGGER_CFG_getCounterInputEvents(const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, COUNTER_EVENT_MASK *const pEvents)	509
8.32.2.14	TRIGGER_CFG_getCounterLoadZeroEnabled(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled)	510
8.32.2.15	TRIGGER_CFG_getCounterOutputActions(const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, COUNTER_ACTION_MASK *const pActions)	510
8.32.2.16	TRIGGER_CFG_getCounterReloadValue(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pReloadValue)	511
8.32.2.17	TRIGGER_CFG_getCounterStartModeEnabled(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled)	511
8.32.2.18	TRIGGER_CFG_getCounterStartStopModeEnabled(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled)	512
8.32.2.19	TRIGGER_CFG_getCounterTriggerCountEnabled(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled)	512
8.32.2.20	TRIGGER_CFG_getCounterValue(const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pValue)	513
8.32.2.21	TRIGGER_CFG_getNumberOfCounters(const STAR_DEVICE_ID deviceld, U32 *const pNumCounters)	513
8.32.2.22	TRIGGER_CFG_setCounterInputEvents(const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, const COUNTER_EVENT_MASK events)	513
8.32.2.23	TRIGGER_CFG_setCounterOutputActions(const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, const COUNTER_ACTION_MASK actions)	514
8.32.2.24	TRIGGER_CFG_setCounterReloadValue(const STAR_DEVICE_ID deviceld, const U32 counter, const U32 reloadValue)	514
8.33	SpaceWire Ports	516
8.33.1	Detailed Description	517
8.33.2	Function Documentation	517
8.33.2.1	TRIGGER_CFG_disablePortSpill(const STAR_DEVICE_ID deviceld, const U32 port)	517
8.33.2.2	TRIGGER_CFG_disablePortTransmitPacketMode(const STAR_DEVICE_ID deviceld, const U32 port)	517
8.33.2.3	TRIGGER_CFG_disableSpWPortSpill(const STAR_DEVICE_ID deviceld, const U32 spwPort)	518
8.33.2.4	TRIGGER_CFG_disableSpWPortTransmitPacketMode(const STAR_DEVICE_ID deviceld, const U32 spwPort)	518

8.33.2.5	TRIGGER_CFG_enablePortSpill(const STAR_DEVICE_ID deviceId, const U32 port)	519
8.33.2.6	TRIGGER_CFG_enablePortTransmitPacketMode(const STAR_DEVICE_ID deviceId, const U32 port)	519
8.33.2.7	TRIGGER_CFG_enableSpWPortSpill(const STAR_DEVICE_ID deviceId, const U32 spwPort)	520
8.33.2.8	TRIGGER_CFG_enableSpWPortTransmitPacketMode(const STAR_DEVICE_ID deviceId, const U32 spwPort)	521
8.33.2.9	TRIGGER_CFG_getNumberOfSpWPorts(const STAR_DEVICE_ID deviceId, U32 *const pNumSpWPorts)	521
8.33.2.10	TRIGGER_CFG_getPortInputEvents(const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_EVENT_MASK *const pEvents)	522
8.33.2.11	TRIGGER_CFG_getPortOutputActions(const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_ACTION_MASK *const pActions)	522
8.33.2.12	TRIGGER_CFG_getPortSpillEnabled(const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)	523
8.33.2.13	TRIGGER_CFG_getPortTransmitPacketModeEnabled(const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)	523
8.33.2.14	TRIGGER_CFG_getSpWPortInputEvents(const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, SPW_PORT_EVENT_MASK *const pEvents)	524
8.33.2.15	TRIGGER_CFG_getSpWPortOutputActions(const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, SPW_PORT_ACTION_MASK *const pActions)	524
8.33.2.16	TRIGGER_CFG_getSpWPortSpillEnabled(const STAR_DEVICE_ID deviceId, const U32 spwPort, U32 *const pEnabled)	525
8.33.2.17	TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled(const STAR_DEVICE_ID deviceId, const U32 spwPort, U32 *const pEnabled)	525
8.33.2.18	TRIGGER_CFG_setPortInputEvents(const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_EVENT_MASK events)	526
8.33.2.19	TRIGGER_CFG_setPortOutputActions(const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_ACTION_MASK actions)	526
8.33.2.20	TRIGGER_CFG_setSpWPortInputEvents(const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, const SPW_PORT_EVENT_MASK events)	526
8.33.2.21	TRIGGER_CFG_setSpWPortOutputActions(const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, const SPW_PORT_ACTION_MASK actions)	527
8.34	Time-codes	528
8.34.1	Detailed Description	528
8.34.2	Function Documentation	528
8.34.2.1	TRIGGER_CFG_getNumberOfTimeCodeInterfaces(const STAR_DEVICE_ID deviceId, U32 *const pNumInterfaces)	528
8.34.2.2	TRIGGER_CFG_getTimeCodeInputEvents(const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_EVENT_MASK *const pEvents)	529
8.34.2.3	TRIGGER_CFG_getTimeCodeOutputActions(const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_ACTION_MASK *const pActions)	529

8.34.2.4	TRIGGER_CFG_setTimeCodeInputEvents(const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_EVENT_MASK events)	530
8.34.2.5	TRIGGER_CFG_setTimeCodeOutputActions(const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_ACTION_MASK actions)	530
8.35	Internal Triggers	531
8.35.1	Detailed Description	532
8.35.2	Function Documentation	532
8.35.2.1	TRIGGER_CFG_disableTrigger(const STAR_DEVICE_ID deviceId, const U32 trigger)	532
8.35.2.2	TRIGGER_CFG_enableTrigger(const STAR_DEVICE_ID deviceId, const U32 trigger)	532
8.35.2.3	TRIGGER_CFG_forceTrigger(const STAR_DEVICE_ID deviceId, const U32 trigger)	533
8.35.2.4	TRIGGER_CFG_getNumberOfTriggers(const STAR_DEVICE_ID deviceId, U32 *const pNumTriggers)	533
8.35.2.5	TRIGGER_CFG_getTriggerAndMask(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask)	534
8.35.2.6	TRIGGER_CFG_getTriggerEnabled(const STAR_DEVICE_ID deviceId, const U32 trigger, U32 *const pEnabled)	534
8.35.2.7	TRIGGER_CFG_getTriggerInputEvents(const STAR_DEVICE_ID deviceId, const U32 causeTrigger, const U32 trigger, TRIGGER_EVENT_MASK *const pEvents)	535
8.35.2.8	TRIGGER_CFG_getTriggerInputMode(const STAR_DEVICE_ID deviceId, const U32 trigger, TRIGGER_INPUT_MODE *const pMode)	535
8.35.2.9	TRIGGER_CFG_getTriggerInvertMask(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask)	536
8.35.2.10	TRIGGER_CFG_getTriggerSourceEventsAnded(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pAnded)	536
8.35.2.11	TRIGGER_CFG_getTriggerSourceEventsInverted(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pInverted)	537
8.35.2.12	TRIGGER_CFG_setTriggerAndMask(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 mask)	538
8.35.2.13	TRIGGER_CFG_setTriggerInputEvents(const STAR_DEVICE_ID deviceId, const U32 causeTrigger, const U32 trigger, const TRIGGER_EVENT_MASK events)	539
8.35.2.14	TRIGGER_CFG_setTriggerInputMode(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_INPUT_MODE mode)	539
8.35.2.15	TRIGGER_CFG_setTriggerInvertMask(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 mask)	540
8.35.2.16	TRIGGER_CFG_setTriggerSourceEventsAnded(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 anded)	540
8.35.2.17	TRIGGER_CFG_setTriggerSourceEventsInverted(const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 inverted)	541
8.36	SpaceFibre Ports	543

8.36.1	Detailed Description	543
8.36.2	Function Documentation	543
8.36.2.1	TRIGGER_CFG_getNumberOfSpFiPorts(const STAR_DEVICE_ID deviceld, U32 *const pNumSpFiPorts)	543
8.36.2.2	TRIGGER_CFG_getSpFiPortInputEvents(const STAR_DEVICE_ID device← Id, const U32 spfiPort, const U32 trigger, SPFI_PORT_EVENT_MASK *const pEvents)	544
8.36.2.3	TRIGGER_CFG_getSpFiPortOutputActions(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, SPFI_PORT_ACTION_MASK *const p← Actions)	544
8.36.2.4	TRIGGER_CFG_setSpFiPortInputEvents(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, const SPFI_PORT_EVENT_MASK events)	545
8.36.2.5	TRIGGER_CFG_setSpFiPortOutputActions(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, const SPFI_PORT_ACTION_MASK actions)	546
8.37	SpaceFibre Virtual Channels	547
8.37.1	Detailed Description	548
8.37.2	Function Documentation	548
8.37.2.1	TRIGGER_CFG_disableSpFiVCReceiveDataMode(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel)	548
8.37.2.2	TRIGGER_CFG_disableSpFiVCTransmitDataMode(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel)	548
8.37.2.3	TRIGGER_CFG_disableSpFiVCTransmitPacketMode(const STAR_DEVICE_ID ← D deviceld, const U32 spfiPort, const U32 virtualChannel)	549
8.37.2.4	TRIGGER_CFG_enableSpFiVCReceiveDataMode(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel)	549
8.37.2.5	TRIGGER_CFG_enableSpFiVCTransmitDataMode(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel)	549
8.37.2.6	TRIGGER_CFG_enableSpFiVCTransmitPacketMode(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel)	550
8.37.2.7	TRIGGER_CFG_getNumberOfSpFiVCs(const STAR_DEVICE_ID deviceld, U32 *const pNumSpFiVCs)	550
8.37.2.8	TRIGGER_CFG_getSpFiVCInputEvents(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC ← _EVENT_MASK *const pEvents)	551
8.37.2.9	TRIGGER_CFG_getSpFiVCOOutputActions(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_AC ← TION_MASK *const pActions)	551
8.37.2.10	TRIGGER_CFG_getSpFiVCReceiveDataModeEnabled(const STAR_DEVICE ← _ID deviceld, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)	552
8.37.2.11	TRIGGER_CFG_getSpFiVCTransmitDataModeEnabled(const STAR_DEVICE ← _ID deviceld, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)	552
8.37.2.12	TRIGGER_CFG_getSpFiVCTransmitPacketModeEnabled(const STAR_DEV ← ICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)	553
8.37.2.13	TRIGGER_CFG_setSpFiVCInputEvents(const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SP ← FI_VC_EVENT_MASK events)	553

8.37.2.14	TRIGGER_CFG_setSpFiVCOOutputActions(const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_V↔C_ACTION_MASK actions)	553
8.38	Defines	555
8.38.1	Detailed Description	555
8.39	Link Speed	556
8.39.1	Detailed Description	556
8.39.2	Function Documentation	556
8.39.2.1	CFG_ROUTER_MK2S_disablePrecisionTransmitRate(STAR_DEVICE_ID↔D deviceId)	556
8.39.2.2	CFG_ROUTER_MK2S_enablePrecisionTransmitRate(STAR_DEVICE_ID↔D deviceId)	557
8.39.2.3	CFG_ROUTER_MK2S_getPrecisionTransmitRate(STAR_DEVICE_ID deviceId, _Out_ double *pTransmitRate)	557
8.39.2.4	CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled(STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)	558
8.39.2.5	CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse(STAR_DEVICE_ID↔D deviceId, _Out_ int *pInUse)	558
8.39.2.6	CFG_ROUTER_MK2S_setPrecisionTransmitRate(STAR_DEVICE_ID deviceId, double transmitRate)	559
8.40	Events	560
8.40.1	Detailed Description	560
8.40.2	Function Documentation	561
8.40.2.1	CFG_MK2_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	561
8.40.2.2	CFG_MK2_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	561
8.40.2.3	CFG_MK2_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	562
8.40.2.4	CFG_MK2_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	562
8.40.2.5	CFG_MK2_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	563
8.40.2.6	CFG_MK2_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	563
8.40.2.7	CFG_MK2_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID↔deviceId, U8 portNum, _Out_ int *pEnabled)	564
8.40.2.8	CFG_MK2_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID↔D, U8 portNum, _Out_ int *pEnabled)	564
8.40.2.9	CFG_MK2_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)	565
8.41	Link Speed	566
8.41.1	Detailed Description	566
8.41.2	Data Structure Documentation	566
8.41.2.1	struct STAR_CFG_MK2_BASE_TRANSMIT_CLOCK	566

8.41.3	Function Documentation	567
8.41.3.1	CFG_MK2_getBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, ↔ _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	567
8.41.3.2	CFG_MK2_getLinkRateDivider(STAR_DEVICE_ID deviceID, U8 linkNum, ↔ Out_ U8 *pDivider)	568
8.41.3.3	CFG_MK2_getTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, ↔ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)	568
8.41.3.4	CFG_MK2_setBaseTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	569
8.41.3.5	CFG_MK2_setLinkRateDivider(STAR_DEVICE_ID deviceID, U8 linkNum, U8 di- vider)	570
8.41.3.6	CFG_MK2_setTransmitClock(STAR_DEVICE_ID deviceID, U8 linkNum, STAR↔ _CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)	571
8.42	Measured Link Speed	572
8.42.1	Detailed Description	572
8.42.2	Macro Definition Documentation	572
8.42.2.1	STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS	572
8.42.3	Function Documentation	572
8.42.3.1	CFG_MK2_getMeasuredLinkSpeed(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed)	572
8.43	Time-code Master	574
8.43.1	Detailed Description	574
8.43.2	Function Documentation	575
8.43.2.1	CFG_MK2_disableExternalTimeCodeSelection(STAR_DEVICE_ID deviceID)	575
8.43.2.2	CFG_MK2_disableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceID)	575
8.43.2.3	CFG_MK2_disableTimeCodeMaster(STAR_DEVICE_ID deviceID)	575
8.43.2.4	CFG_MK2_enableExternalTimeCodeSelection(STAR_DEVICE_ID deviceID)	576
8.43.2.5	CFG_MK2_enableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceID)	576
8.43.2.6	CFG_MK2_enableTimeCodeMaster(STAR_DEVICE_ID deviceID)	577
8.43.2.7	CFG_MK2_getExternalTimeCodeSelectionEnabled(STAR_DEVICE_ID device↔ ID, int *pEnabled)	577
8.43.2.8	CFG_MK2_getTimeCodeCounterBypassModeEnabled(STAR_DEVICE_ID ↔ D deviceID, int *pEnabled)	578
8.43.2.9	CFG_MK2_getTimeCodeMasterEnabled(STAR_DEVICE_ID deviceID, int *p↔ Enabled)	578
8.43.2.10	CFG_MK2_getTimeCodePeriod(STAR_DEVICE_ID deviceID, _Out_ U32 *p↔ Period)	579
8.43.2.11	CFG_MK2_setTimeCodePeriod(STAR_DEVICE_ID deviceID, U32 period)	579
8.44	Device Information	581
8.44.1	Detailed Description	581
8.44.2	Data Structure Documentation	581
8.44.2.1	struct STAR_CFG_MK2_HARDWARE_INFO	581
8.44.3	Function Documentation	582

8.44.3.1	CFG_MK2_getHardwareInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG↔ _MK2_HARDWARE_INFO *pInfo)	582
8.44.3.2	CFG_MK2_hardwareInfoToString(STAR_CFG_MK2_HARDWARE_INFO info, _Out_z_cap_c_(STAR_CFG_MK2_VERSION_STR_MAX_LEN) char *pVersion, _Out_z_cap_c_(STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN) char *p↔ BuildDate)	583
8.44.3.3	CFG_MK2_identify(STAR_DEVICE_ID deviceId)	583
8.45	Interface Mode	584
8.45.1	Detailed Description	585
8.45.2	Function Documentation	585
8.45.2.1	CFG_MK2_disableIdentifySource(STAR_DEVICE_ID deviceId)	585
8.45.2.2	CFG_MK2_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 port↔ Num)	585
8.45.2.3	CFG_MK2_disableInterfaceMode(STAR_DEVICE_ID deviceId)	585
8.45.2.4	CFG_MK2_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 port↔ Num)	586
8.45.2.5	CFG_MK2_enableIdentifySource(STAR_DEVICE_ID deviceId)	586
8.45.2.6	CFG_MK2_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 port↔ Num)	587
8.45.2.7	CFG_MK2_enableInterfaceMode(STAR_DEVICE_ID deviceId)	587
8.45.2.8	CFG_MK2_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 port↔ Num)	587
8.45.2.9	CFG_MK2_getIdentifySourceEnabled(STAR_DEVICE_ID deviceId, int *pEnabled)588	
8.45.2.10	CFG_MK2_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, int *pEnabled)	588
8.45.2.11	CFG_MK2_getInterfaceModeEnabled(STAR_DEVICE_ID deviceId, int *pEnabled)589	
8.45.2.12	CFG_MK2_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceId, U8 portNum, int *pEnabled)	589
8.45.2.13	CFG_MK2_getPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)	590
8.45.2.14	CFG_MK2_setPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, U8 address)	590
8.46	Error Injection	592
8.46.1	Detailed Description	592
8.46.2	Enumeration Type Documentation	592
8.46.2.1	SPW_ERROR	592
8.46.3	Function Documentation	593
8.46.3.1	CFG_MK2_injectError(STAR_DEVICE_ID deviceId, unsigned char port, SPW↔ _ERROR error)	593
8.47	Periodic Actions	594
8.47.1	Detailed Description	594
8.47.2	Enumeration Type Documentation	594
8.47.2.1	SPW_ACTION	594
8.47.3	Function Documentation	594

8.47.3.1	CFG_MK2_getDeviceClockRate(STAR_DEVICE_ID deviceId)	594
8.47.3.2	CFG_MK2_startPeriodicAction(STAR_DEVICE_ID deviceId, unsigned char port, U32 count, SPW_ACTION action)	595
8.47.3.3	CFG_MK2_stopPeriodicAction(STAR_DEVICE_ID deviceId, unsigned char port)	595
8.48	PCI Mk2 Packet Subsystem	597
8.48.1	Detailed Description	598
8.48.2	Data Structure Documentation	599
8.48.2.1	struct PKT_SUBSYS_STATISTICS	599
8.48.3	Typedef Documentation	600
8.48.3.1	PKT_SUBSYS_StatisticsFunc	600
8.48.3.2	STATISTICS_LOOP_ID	600
8.48.4	Enumeration Type Documentation	600
8.48.4.1	PKT_SUBSYS_PATTERN	600
8.48.5	Function Documentation	600
8.48.5.1	PKT_SUBSYS_cancelWaitsForPacketCheckingError(STAR_DEVICE_ID deviceId, unsigned int subsystem)	601
8.48.5.2	PKT_SUBSYS_fillBufferWithPattern(PKT_SUBSYS_PATTERN pattern, U16 bufferSize, _Out_bytecap_(bufferSize) void *pBuffer)	602
8.48.5.3	PKT_SUBSYS_getPacketCheckingMismatchCount(STAR_DEVICE_ID deviceId, unsigned int subsystem, U64 *errorCount)	602
8.48.5.4	PKT_SUBSYS_getSinkDataPointers(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 *startPointer, U16 *endPointer)	603
8.48.5.5	PKT_SUBSYS_getStatistics(STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_STATISTICS *pStatistics)	603
8.48.5.6	PKT_SUBSYS_readMemory(STAR_DEVICE_ID deviceId, U16 address, U16 readSize, _Inout_bytecap_(readSize) void *pBuffer)	604
8.48.5.7	PKT_SUBSYS_setGeneratorBlockingParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)	604
8.48.5.8	PKT_SUBSYS_setPacketCheckingFormat(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop)	605
8.48.5.9	PKT_SUBSYS_setPacketGeneratorFormat(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop, U16 gap)	606
8.48.5.10	PKT_SUBSYS_setPacketSinkParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)	607
8.48.5.11	PKT_SUBSYS_setSinkBlockingParameters(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)	608
8.48.5.12	PKT_SUBSYS_startGetStatisticsLoop(STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_StatisticsFunc statisticsHandler)	608
8.48.5.13	PKT_SUBSYS_startPacketChecking(STAR_DEVICE_ID deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)	609
8.48.5.14	PKT_SUBSYS_startPacketGeneration(STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 sequenceLength)	609
8.48.5.15	PKT_SUBSYS_startSink(STAR_DEVICE_ID deviceId, unsigned int subsystem)	610

8.48.5.16	PKT_SUBSYS_stopGetStatisticsLoop(STATISTICS_LOOP_ID loopId)	610
8.48.5.17	PKT_SUBSYS_stopPacketChecking(STAR_DEVICE_ID deviceId, unsigned int subsystem)	611
8.48.5.18	PKT_SUBSYS_stopPacketGeneration(STAR_DEVICE_ID deviceId, unsigned int subsystem)	611
8.48.5.19	PKT_SUBSYS_stopSink(STAR_DEVICE_ID deviceId, unsigned int subsystem)	611
8.48.5.20	PKT_SUBSYS_waitForPacketCheckingError(STAR_DEVICE_ID deviceId, unsigned int subsystem)	612
8.48.5.21	PKT_SUBSYS_waitForPacketGenerationSequenceComplete(STAR_DEVICE_ID deviceId, unsigned int subsystem, unsigned int timeout)	612
8.48.5.22	PKT_SUBSYS_writeMemory(STAR_DEVICE_ID deviceId, U16 address, U16 writeSize, _In_bytecount_(writeSize) void *pBuffer)	613
8.49	Generic Configuration API	614
8.49.1	Detailed Description	615
8.50	SpaceFibre Configuration API	616
8.50.1	Detailed Description	617
8.51	Other Device Configuration APIs	618
8.51.1	Detailed Description	619
8.51.2	Registers	620
8.52	Remote Devices	621
8.52.1	Detailed Description	621
8.52.2	Function Documentation	622
8.52.2.1	STAR_CFG_createRemoteDeviceIdentifier(STAR_DEVICE_ID deviceId, unsigned char localChannel, char *name, STAR_SPACEWIRE_ADDRESS *pathTo, STAR_SPACEWIRE_ADDRESS *returnPath)	622
8.52.2.2	STAR_CFG_destroyRemoteDeviceDescriptionList(_Post_ptr_invalid_ STAR_DEVICE_ID *pRemoteDeviceDescriptionList)	622
8.52.2.3	STAR_CFG_destroyRemoteDeviceIdentifier(STAR_DEVICE_ID remoteDeviceId)	622
8.52.2.4	STAR_CFG_getRemoteDeviceDescriptionList(_Out_ U32 *count)	623
8.52.2.5	STAR_CFG_getRemoteDeviceDescriptionNameString(STAR_DEVICE_ID deviceId)	623
8.52.2.6	STAR_CFG_getRemoteDeviceLocalChannel(STAR_DEVICE_ID remoteDeviceId, unsigned char *localChannel)	624
8.52.2.7	STAR_CFG_getRemoteDeviceLocalDevice(STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID *localDeviceId)	625
8.52.2.8	STAR_CFG_getRemoteDevicePath(STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pPath)	625
8.52.2.9	STAR_CFG_getRemoteDeviceRetPath(STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pRetPath)	625
8.53	Router Configuration API	627
8.53.1	Detailed Description	627
8.54	Mk2 Compatible Interface and Router Device Configuration API	628
8.54.1	Detailed Description	628
8.55	PCI Mk2 Configuration API	630

8.55.1 Detailed Description	630
8.56 Brick Mk2 Configuration API	631
8.56.1 Detailed Description	631
8.57 Brick Mk3 Configuration API	632
8.57.1 Detailed Description	632
8.58 GbE Brick Mk1 Configuration API	633
8.58.1 Detailed Description	633
8.59 Router Mk2S Configuration API	634
8.59.1 Detailed Description	634
8.60 PXI Configuration API	635
8.60.1 Detailed Description	635
8.61 PCIe Mk2 Configuration API	636
8.61.1 Detailed Description	636
8.62 Device Information	637
8.62.1 Detailed Description	638
8.62.2 Data Structure Documentation	638
8.62.2.1 struct STAR_CFG_DEVICE_IDENTIFIER_INFO	638
8.62.2.2 struct STAR_CFG_NETWORK_DISCOVERY_INFO	638
8.62.3 Macro Definition Documentation	639
8.62.3.1 STAR_CFG_DEVICE_STR_MAX_LEN	639
8.62.3.2 STAR_CFG_MANUFACTURER_STR_MAX_LEN	639
8.62.4 Enumeration Type Documentation	640
8.62.4.1 STAR_CFG_DEVICE_TYPE	640
8.62.5 Function Documentation	640
8.62.5.1 CFG_ROUTER_getDeviceIdentificationInfo(STAR_DEVICE_ID deviceID, _Out_↵ _STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)	640
8.62.5.2 CFG_ROUTER_getDeviceManufacturerAsString(STAR_CFG_DEVICE_IDEN↵ TIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_MANUFAC↵ TURER_STR_MAX_LEN) char *manufacturerStr)	640
8.62.5.3 CFG_ROUTER_getDeviceTypeAsString(STAR_CFG_DEVICE_IDENTIFIER_↵ INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_DEVICE_STR_MA↵ X_LEN) char *deviceTypeStr)	641
8.62.5.4 CFG_ROUTER_getNetworkDiscoveryInfo(STAR_DEVICE_ID deviceID, _Out_↵ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)	641
8.63 Port Status and Control	643
8.63.1 Detailed Description	644
8.63.2 Data Structure Documentation	644
8.63.2.1 struct STAR_CFG_CONFIG_PORT_ERRORS	644
8.63.2.2 struct STAR_CFG_SPW_LINK_ERRORS	646
8.63.2.3 struct STAR_CFG_SPW_LINK_STATUS	647
8.63.2.4 struct STAR_CFG_EXTERNAL_PORT_ERRORS	648
8.63.2.5 struct STAR_CFG_EXTERNAL_PORT_STATUS	648

8.63.3	Typedef Documentation	649
8.63.3.1	PORT_STATUS_CONTROL	649
8.63.4	Enumeration Type Documentation	649
8.63.4.1	STAR_CFG_PORT_TYPE	649
8.63.4.2	STAR_CFG_SPW_LINK_STATE	650
8.63.5	Function Documentation	650
8.63.5.1	CFG_ROUTER_clearPortErrors(STAR_DEVICE_ID deviceID, U8 portNum) . . .	650
8.63.5.2	CFG_ROUTER_getConfigPortErrors(PORT_STATUS_CONTROL portStatus↔ Control, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *errors)	651
8.63.5.3	CFG_ROUTER_getExternalPortErrors(PORT_STATUS_CONTROL port↔ StatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors) . . .	651
8.63.5.4	CFG_ROUTER_getExternalPortStatus(PORT_STATUS_CONTROL port↔ StatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)	652
8.63.5.5	CFG_ROUTER_getPortConnection(PORT_STATUS_CONTROL portStatus↔ Control)	652
8.63.5.6	CFG_ROUTER_getPortStatusControl(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL *portStatusControl)	652
8.63.5.7	CFG_ROUTER_getPortType(PORT_STATUS_CONTROL portStatusControl) . .	653
8.63.5.8	CFG_ROUTER_getSpaceWireLinkErrors(PORT_STATUS_CONTROL link↔ StatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)	653
8.63.5.9	CFG_ROUTER_getSpaceWireLinkStatus(PORT_STATUS_CONTROL link↔ StatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *status)	654
8.63.5.10	CFG_ROUTER_setSpaceWireLinkStatus(STAR_DEVICE_ID deviceID, U8 link↔ Num, STAR_CFG_SPW_LINK_STATUS linkStatus)	654
8.63.5.11	CFG_ROUTER_startLink(STAR_DEVICE_ID deviceID, U8 linkNum)	655
8.63.5.12	CFG_ROUTER_stopLink(STAR_DEVICE_ID deviceID, U8 linkNum)	655
8.64	Routing Table	657
8.64.1	Detailed Description	657
8.64.2	Data Structure Documentation	657
8.64.2.1	struct STAR_CFG_GAR_ENTRY	657
8.64.3	Function Documentation	658
8.64.3.1	CFG_ROUTER_getRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logical↔ Address, _Out_ STAR_CFG_GAR_ENTRY *entry)	658
8.64.3.2	CFG_ROUTER_setRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logical↔ Address, STAR_CFG_GAR_ENTRY entry)	659
8.65	User Configurable Registers	660
8.65.1	Detailed Description	660
8.65.2	Function Documentation	660
8.65.2.1	CFG_MK2_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, _Out_↔ U32 *pPortMask)	660
8.65.2.2	CFG_MK2_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, U32 portMask)	661
8.65.2.3	CFG_ROUTER_getGeneralPurpose(STAR_DEVICE_ID deviceID, _Out_ REG↔ ISTER *value)	661

8.65.2.4	CFG_ROUTER_getNetworkIdentity(STAR_DEVICE_ID deviceID, _Out_ REGISTER *value)	662
8.65.2.5	CFG_ROUTER_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, _Out_ U32 *portMask)	662
8.65.2.6	CFG_ROUTER_setGeneralPurpose(STAR_DEVICE_ID deviceID, REGISTER value)	663
8.65.2.7	CFG_ROUTER_setNetworkIdentity(STAR_DEVICE_ID deviceID, REGISTER value)	663
8.65.2.8	CFG_ROUTER_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, U32 portMask)	664
8.66	Router Control and Status Configuration	665
8.66.1	Detailed Description	666
8.66.2	Data Structure Documentation	666
8.66.2.1	struct STAR_CFG_ROUTER_GLOBAL_STATE	666
8.66.3	Enumeration Type Documentation	667
8.66.3.1	STAR_CFG_PORT_TIMEOUT	667
8.66.3.2	STAR_CFG_TIMEOUT_MODE	667
8.66.4	Function Documentation	668
8.66.4.1	CFG_MK2_getTimeCodeFlagMode(STAR_DEVICE_ID deviceID, _Out_ U8 *pMode)	668
8.66.4.2	CFG_MK2_setTimeCodeFlagMode(STAR_DEVICE_ID deviceID, U8 mode)	668
8.66.4.3	CFG_ROUTER_getRouterGlobalSettings(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *state)	669
8.66.4.4	CFG_ROUTER_getTimeCodeFlagMode(STAR_DEVICE_ID deviceID, _Out_ U8 *mode)	670
8.66.4.5	CFG_ROUTER_getTimeCodeValue(STAR_DEVICE_ID deviceID, _Out_ U8 *value)	670
8.66.4.6	CFG_ROUTER_setRouterGlobalSettings(STAR_DEVICE_ID deviceID, STAR_CFG_ROUTER_GLOBAL_STATE state)	671
8.66.4.7	CFG_ROUTER_setTimeCodeFlagMode(STAR_DEVICE_ID deviceID, U8 mode)	671
8.67	Triggering API	673
8.67.1	Detailed Description	673
8.67.2	Input Events	673
8.67.3	Output Actions	677
8.67.4	Internal Triggers	681
8.67.5	Triggering Resources	682
8.67.6	PCIe Differences	685
8.67.7	Examples	685
8.67.7.1	Transmitting Time-Codes Synchronised by an External Trigger	685
8.67.7.2	Packet Transmission Synchronised by a Counter	686
8.67.7.3	Multiple Packet Transmission Synchronised by Time-Codes	686
8.68	Events	688
8.68.1	Detailed Description	692

8.68.2	Typedef Documentation	692
8.68.2.1	COUNTER_ACTION_MASK	692
8.68.2.2	COUNTER_EVENT_MASK	692
8.68.2.3	EXT_TRIGGER_ACTION_MASK	692
8.68.2.4	EXT_TRIGGER_EVENT_MASK	692
8.68.2.5	SPFI_PORT_EVENT_MASK	692
8.68.2.6	SPFI_VC_EVENT_MASK	693
8.68.2.7	SPW_PORT_EVENT_MASK	693
8.68.2.8	TIME_CODE_ACTION_MASK	693
8.68.2.9	TIME_CODE_EVENT_MASK	693
8.68.2.10	TRIGGER_EVENT_MASK	693
8.68.3	Enumeration Type Documentation	693
8.68.3.1	COUNTER_EVENT	693
8.68.3.2	EXT_TRIGGER_EVENT	694
8.68.3.3	SPFI_PORT_EVENT	694
8.68.3.4	SPFI_VC_EVENT	694
8.68.3.5	SPW_PORT_EVENT	695
8.68.3.6	TIME_CODE_EVENT	695
8.68.3.7	TRIGGER_EVENT	695
8.68.4	Function Documentation	696
8.68.4.1	TRIGGER_BRICK_MK3_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	696
8.68.4.2	TRIGGER_BRICK_MK3_getExtTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)	697
8.68.4.3	TRIGGER_BRICK_MK3_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	697
8.68.4.4	TRIGGER_BRICK_MK3_getTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	698
8.68.4.5	TRIGGER_BRICK_MK3_getTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	698
8.68.4.6	TRIGGER_BRICK_MK3_setCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	699
8.68.4.7	TRIGGER_BRICK_MK3_setExtTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)	700
8.68.4.8	TRIGGER_BRICK_MK3_setPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events)	700
8.68.4.9	TRIGGER_BRICK_MK3_setTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	701
8.68.4.10	TRIGGER_BRICK_MK3_setTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	701
8.68.4.11	TRIGGER_PCIE_IF_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	702
8.68.4.12	TRIGGER_PCIE_IF_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	702

8.68.4.13	TRIGGER_PCIE_IF_getTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	703
8.68.4.14	TRIGGER_PCIE_IF_setCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	703
8.68.4.15	TRIGGER_PCIE_IF_setPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)	704
8.68.4.16	TRIGGER_PCIE_IF_setTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	704
8.68.4.17	TRIGGER_PCIE_MK2_getCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	704
8.68.4.18	TRIGGER_PCIE_MK2_getExtTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)	705
8.68.4.19	TRIGGER_PCIE_MK2_getPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	705
8.68.4.20	TRIGGER_PCIE_MK2_getTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	706
8.68.4.21	TRIGGER_PCIE_MK2_getTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	707
8.68.4.22	TRIGGER_PCIE_MK2_setCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	707
8.68.4.23	TRIGGER_PCIE_MK2_setExtTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)	708
8.68.4.24	TRIGGER_PCIE_MK2_setPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)	708
8.68.4.25	TRIGGER_PCIE_MK2_setTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	709
8.68.4.26	TRIGGER_PCIE_MK2_setTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	709
8.68.4.27	TRIGGER_PXI_IF_getCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	710
8.68.4.28	TRIGGER_PXI_IF_getExtTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)	710
8.68.4.29	TRIGGER_PXI_IF_getPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	711
8.68.4.30	TRIGGER_PXI_IF_getTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	712
8.68.4.31	TRIGGER_PXI_IF_getTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	712
8.68.4.32	TRIGGER_PXI_IF_setCounterInputEvents(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	713
8.68.4.33	TRIGGER_PXI_IF_setExtTriggerInputEvents(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)	713
8.68.4.34	TRIGGER_PXI_IF_setPortInputEvents(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)	714
8.68.4.35	TRIGGER_PXI_IF_setTimeCodeInputEvents(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	714

8.68.4.36	TRIGGER_PXI_IF_setTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	715
8.68.4.37	TRIGGER_PXI_ROUTER_getCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)	715
8.68.4.38	TRIGGER_PXI_ROUTER_getPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)	715
8.68.4.39	TRIGGER_PXI_ROUTER_getTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)	716
8.68.4.40	TRIGGER_PXI_ROUTER_getTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)	716
8.68.4.41	TRIGGER_PXI_ROUTER_setCounterInputEvents(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)	717
8.68.4.42	TRIGGER_PXI_ROUTER_setPortInputEvents(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events)	718
8.68.4.43	TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)	718
8.68.4.44	TRIGGER_PXI_ROUTER_setTriggerInputEvents(STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)	719
8.69	Actions	720
8.69.1	Detailed Description	723
8.69.2	Typedef Documentation	723
8.69.2.1	SPFI_PORT_ACTION_MASK	723
8.69.2.2	SPFI_VC_ACTION_MASK	723
8.69.2.3	SPW_PORT_ACTION_MASK	723
8.69.3	Enumeration Type Documentation	723
8.69.3.1	COUNTER_ACTION	723
8.69.3.2	EXT_TRIGGER_ACTION	724
8.69.3.3	SPFI_PORT_ACTION	724
8.69.3.4	SPFI_VC_ACTION	724
8.69.3.5	SPW_PORT_ACTION	724
8.69.3.6	TIME_CODE_ACTION	725
8.69.4	Function Documentation	725
8.69.4.1	TRIGGER_BRICK_MK3_getCounterOutputActions(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	725
8.69.4.2	TRIGGER_BRICK_MK3_getExtTriggerOutputActions(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)	726
8.69.4.3	TRIGGER_BRICK_MK3_getPortOutputActions(STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	726
8.69.4.4	TRIGGER_BRICK_MK3_getTimeCodeOutputActions(STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	726
8.69.4.5	TRIGGER_BRICK_MK3_setCounterOutputActions(STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	727
8.69.4.6	TRIGGER_BRICK_MK3_setExtTriggerOutputActions(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)	727

8.69.4.7	TRIGGER_BRICK_MK3_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	728
8.69.4.8	TRIGGER_BRICK_MK3_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	728
8.69.4.9	TRIGGER_PCIE_IF_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	729
8.69.4.10	TRIGGER_PCIE_IF_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	729
8.69.4.11	TRIGGER_PCIE_IF_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	730
8.69.4.12	TRIGGER_PCIE_IF_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	730
8.69.4.13	TRIGGER_PCIE_MK2_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	731
8.69.4.14	TRIGGER_PCIE_MK2_getExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)	731
8.69.4.15	TRIGGER_PCIE_MK2_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	731
8.69.4.16	TRIGGER_PCIE_MK2_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	732
8.69.4.17	TRIGGER_PCIE_MK2_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	732
8.69.4.18	TRIGGER_PCIE_MK2_setExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)	733
8.69.4.19	TRIGGER_PCIE_MK2_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	733
8.69.4.20	TRIGGER_PCIE_MK2_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	734
8.69.4.21	TRIGGER_PXI_IF_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	734
8.69.4.22	TRIGGER_PXI_IF_getExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)	735
8.69.4.23	TRIGGER_PXI_IF_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	735
8.69.4.24	TRIGGER_PXI_IF_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	736
8.69.4.25	TRIGGER_PXI_IF_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	736
8.69.4.26	TRIGGER_PXI_IF_setExtTriggerOutputActions(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)	736
8.69.4.27	TRIGGER_PXI_IF_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	737
8.69.4.28	TRIGGER_PXI_IF_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	737
8.69.4.29	TRIGGER_PXI_ROUTER_getCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)	738

8.69.4.30	TRIGGER_PXI_ROUTER_getPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)	738
8.69.4.31	TRIGGER_PXI_ROUTER_getTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)	739
8.69.4.32	TRIGGER_PXI_ROUTER_setCounterOutputActions(STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)	739
8.69.4.33	TRIGGER_PXI_ROUTER_setPortOutputActions(STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)	740
8.69.4.34	TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)	740
8.70	Configuration	741
8.70.1	Detailed Description	752
8.70.2	Enumeration Type Documentation	752
8.70.2.1	TRIGGER_INPUT_MODE	752
8.70.3	Function Documentation	753
8.70.3.1	TRIGGER_BRICK_MK3_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	753
8.70.3.2	TRIGGER_BRICK_MK3_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	753
8.70.3.3	TRIGGER_BRICK_MK3_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	753
8.70.3.4	TRIGGER_BRICK_MK3_disableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	754
8.70.3.5	TRIGGER_BRICK_MK3_disableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	754
8.70.3.6	TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	755
8.70.3.7	TRIGGER_BRICK_MK3_disableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	755
8.70.3.8	TRIGGER_BRICK_MK3_disableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	755
8.70.3.9	TRIGGER_BRICK_MK3_disablePortSpill(STAR_DEVICE_ID deviceId, U32 port)	756
8.70.3.10	TRIGGER_BRICK_MK3_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	756
8.70.3.11	TRIGGER_BRICK_MK3_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	757
8.70.3.12	TRIGGER_BRICK_MK3_enableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	757
8.70.3.13	TRIGGER_BRICK_MK3_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	758
8.70.3.14	TRIGGER_BRICK_MK3_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	758
8.70.3.15	TRIGGER_BRICK_MK3_enableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	759
8.70.3.16	TRIGGER_BRICK_MK3_enableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	759

8.70.3.17 TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceld, U32 extTrigger)	760
8.70.3.18 TRIGGER_BRICK_MK3_enableExtTriggerInvert(STAR_DEVICE_ID deviceld, U32 extTrigger)	760
8.70.3.19 TRIGGER_BRICK_MK3_enableExtTriggerOutput(STAR_DEVICE_ID deviceld, U32 extTrigger)	761
8.70.3.20 TRIGGER_BRICK_MK3_enablePortSpill(STAR_DEVICE_ID deviceld, U32 port)	761
8.70.3.21 TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceld, U32 port)	761
8.70.3.22 TRIGGER_BRICK_MK3_enableTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	762
8.70.3.23 TRIGGER_BRICK_MK3_forceCounterReload(STAR_DEVICE_ID deviceld, U32 counter)	762
8.70.3.24 TRIGGER_BRICK_MK3_forceTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	763
8.70.3.25 TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled(STAR_DEVICE_ID D deviceld, U32 counter, U32 *pEnabled)	763
8.70.3.26 TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled(STAR_DEVICE_ID D deviceld, U32 counter, U32 *pEnabled)	764
8.70.3.27 TRIGGER_BRICK_MK3_getCounterReloadValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pReloadValue)	765
8.70.3.28 TRIGGER_BRICK_MK3_getCounterStartModeEnabled(STAR_DEVICE_ID D deviceld, U32 counter, U32 *pEnabled)	765
8.70.3.29 TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled(STAR_DEVICE_ID ID deviceld, U32 counter, U32 *pEnabled)	766
8.70.3.30 TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	766
8.70.3.31 TRIGGER_BRICK_MK3_getCounterValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pValue)	766
8.70.3.32 TRIGGER_BRICK_MK3_getDeviceClockRate()	767
8.70.3.33 TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled(STAR_DEVICE_ID CE_ID deviceld, U32 extTrigger, U32 *pEnabled)	767
8.70.3.34 TRIGGER_BRICK_MK3_getExtTriggerExtend(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 *pExtend)	768
8.70.3.35 TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled(STAR_DEVICE_ID D deviceld, U32 extTrigger, U32 *pEnabled)	768
8.70.3.36 TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled(STAR_DEVICE_ID D deviceld, U32 extTrigger, U32 *pEnabled)	768
8.70.3.37 TRIGGER_BRICK_MK3_getPortSpillEnabled(STAR_DEVICE_ID deviceld, U32 port, U32 *pEnabled)	769
8.70.3.38 TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID E_ID deviceld, U32 port, U32 *pEnabled)	769
8.70.3.39 TRIGGER_BRICK_MK3_getTriggerAndMask(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	770
8.70.3.40 TRIGGER_BRICK_MK3_getTriggerEnabled(STAR_DEVICE_ID deviceld, U32 trigger, U32 *pEnabled)	771
8.70.3.41 TRIGGER_BRICK_MK3_getTriggerInputMode(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE *pMode)	771

8.70.3.42 TRIGGER_BRICK_MK3_getTriggerInvertMask(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	772
8.70.3.43 TRIGGER_BRICK_MK3_getTriggerMatrix()	772
8.70.3.44 TRIGGER_BRICK_MK3_reset(STAR_DEVICE_ID deviceld)	772
8.70.3.45 TRIGGER_BRICK_MK3_setCounterReloadValue(STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue)	773
8.70.3.46 TRIGGER_BRICK_MK3_setExtTriggerExtend(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 extend)	773
8.70.3.47 TRIGGER_BRICK_MK3_setTriggerAndMask(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask)	774
8.70.3.48 TRIGGER_BRICK_MK3_setTriggerInputMode(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode)	774
8.70.3.49 TRIGGER_BRICK_MK3_setTriggerInvertMask(STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask)	775
8.70.3.50 TRIGGER_PCIE_getTriggerMatrix()	775
8.70.3.51 TRIGGER_PCIE_IF_disableCounterAutoReload(STAR_DEVICE_ID deviceld, U32 counter)	775
8.70.3.52 TRIGGER_PCIE_IF_disableCounterLoadZero(STAR_DEVICE_ID deviceld, U32 counter)	776
8.70.3.53 TRIGGER_PCIE_IF_disableCounterStartMode(STAR_DEVICE_ID deviceld, U32 counter)	776
8.70.3.54 TRIGGER_PCIE_IF_disableCounterStartStopMode(STAR_DEVICE_ID device↔ Id, U32 counter)	777
8.70.3.55 TRIGGER_PCIE_IF_disableCounterTriggerCount(STAR_DEVICE_ID deviceld, U32 counter)	777
8.70.3.56 TRIGGER_PCIE_IF_disableTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	777
8.70.3.57 TRIGGER_PCIE_IF_enableCounterAutoReload(STAR_DEVICE_ID deviceld, U32 counter)	778
8.70.3.58 TRIGGER_PCIE_IF_enableCounterLoadZero(STAR_DEVICE_ID deviceld, U32 counter)	778
8.70.3.59 TRIGGER_PCIE_IF_enableCounterStartMode(STAR_DEVICE_ID deviceld, U32 counter)	779
8.70.3.60 TRIGGER_PCIE_IF_enableCounterStartStopMode(STAR_DEVICE_ID device↔ Id, U32 counter)	779
8.70.3.61 TRIGGER_PCIE_IF_enableCounterTriggerCount(STAR_DEVICE_ID deviceld, U32 counter)	779
8.70.3.62 TRIGGER_PCIE_IF_enableTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	780
8.70.3.63 TRIGGER_PCIE_IF_forceCounterReload(STAR_DEVICE_ID deviceld, U32 counter)	780
8.70.3.64 TRIGGER_PCIE_IF_forceTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	781
8.70.3.65 TRIGGER_PCIE_IF_getCounterAutoReloadEnabled(STAR_DEVICE_ID device↔ Id, U32 counter, U32 *pEnabled)	781
8.70.3.66 TRIGGER_PCIE_IF_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	781
8.70.3.67 TRIGGER_PCIE_IF_getCounterReloadValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pReloadValue)	782

8.70.3.68 TRIGGER_PCIE_IF_getCounterStartModeEnabled(STAR_DEVICE_ID device↔ Id, U32 counter, U32 *pEnabled)	782
8.70.3.69 TRIGGER_PCIE_IF_getCounterStartStopModeEnabled(STAR_DEVICE_ID↔ D deviceId, U32 counter, U32 *pEnabled)	783
8.70.3.70 TRIGGER_PCIE_IF_getCounterTriggerCountEnabled(STAR_DEVICE_ID↔ D deviceId, U32 counter, U32 *pEnabled)	784
8.70.3.71 TRIGGER_PCIE_IF_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	784
8.70.3.72 TRIGGER_PCIE_IF_getDeviceClockRate()	785
8.70.3.73 TRIGGER_PCIE_IF_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trig- ger, TRIGGER_TYPE type, U32 *pMask)	785
8.70.3.74 TRIGGER_PCIE_IF_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trig- ger, U32 *pEnabled)	785
8.70.3.75 TRIGGER_PCIE_IF_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trig- ger, TRIGGER_INPUT_MODE *pMode)	786
8.70.3.76 TRIGGER_PCIE_IF_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	786
8.70.3.77 TRIGGER_PCIE_IF_reset(STAR_DEVICE_ID deviceId)	787
8.70.3.78 TRIGGER_PCIE_IF_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	787
8.70.3.79 TRIGGER_PCIE_IF_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trig- ger, TRIGGER_TYPE type, U32 mask)	787
8.70.3.80 TRIGGER_PCIE_IF_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trig- ger, TRIGGER_INPUT_MODE mode)	788
8.70.3.81 TRIGGER_PCIE_IF_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	788
8.70.3.82 TRIGGER_PCIE_MK2_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	789
8.70.3.83 TRIGGER_PCIE_MK2_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	789
8.70.3.84 TRIGGER_PCIE_MK2_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	790
8.70.3.85 TRIGGER_PCIE_MK2_disableCounterStartStopMode(STAR_DEVICE_ID↔ D deviceId, U32 counter)	791
8.70.3.86 TRIGGER_PCIE_MK2_disableCounterTriggerCount(STAR_DEVICE_ID device↔ Id, U32 counter)	791
8.70.3.87 TRIGGER_PCIE_MK2_disableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	792
8.70.3.88 TRIGGER_PCIE_MK2_disableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	792
8.70.3.89 TRIGGER_PCIE_MK2_disableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	792
8.70.3.90 TRIGGER_PCIE_MK2_disablePortSpill(STAR_DEVICE_ID deviceId, U32 port) .	793
8.70.3.91 TRIGGER_PCIE_MK2_disablePortTransmitPacketMode(STAR_DEVICE_ID↔ D deviceId, U32 port)	793
8.70.3.92 TRIGGER_PCIE_MK2_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	794

8.70.3.93 TRIGGER_PCIE_MK2_enableCounterAutoReload(STAR_DEVICE_ID deviceld, U32 counter)	794
8.70.3.94 TRIGGER_PCIE_MK2_enableCounterLoadZero(STAR_DEVICE_ID deviceld, U32 counter)	794
8.70.3.95 TRIGGER_PCIE_MK2_enableCounterStartMode(STAR_DEVICE_ID deviceld, U32 counter)	795
8.70.3.96 TRIGGER_PCIE_MK2_enableCounterStartStopMode(STAR_DEVICE_ID deviceld, U32 counter)	795
8.70.3.97 TRIGGER_PCIE_MK2_enableCounterTriggerCount(STAR_DEVICE_ID deviceld, U32 counter)	796
8.70.3.98 TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceld, U32 extTrigger)	796
8.70.3.99 TRIGGER_PCIE_MK2_enableExtTriggerInvert(STAR_DEVICE_ID deviceld, U32 extTrigger)	797
8.70.3.100 TRIGGER_PCIE_MK2_enableExtTriggerOutput(STAR_DEVICE_ID deviceld, U32 extTrigger)	797
8.70.3.101 TRIGGER_PCIE_MK2_enablePortSpill(STAR_DEVICE_ID deviceld, U32 port)	798
8.70.3.102 TRIGGER_PCIE_MK2_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceld, U32 port)	798
8.70.3.103 TRIGGER_PCIE_MK2_enableTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	799
8.70.3.104 TRIGGER_PCIE_MK2_forceCounterReload(STAR_DEVICE_ID deviceld, U32 counter)	799
8.70.3.105 TRIGGER_PCIE_MK2_forceTrigger(STAR_DEVICE_ID deviceld, U32 trigger)	800
8.70.3.106 TRIGGER_PCIE_MK2_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	800
8.70.3.107 TRIGGER_PCIE_MK2_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	800
8.70.3.108 TRIGGER_PCIE_MK2_getCounterReloadValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pReloadValue)	801
8.70.3.109 TRIGGER_PCIE_MK2_getCounterStartModeEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	801
8.70.3.110 TRIGGER_PCIE_MK2_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	802
8.70.3.111 TRIGGER_PCIE_MK2_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceld, U32 counter, U32 *pEnabled)	802
8.70.3.112 TRIGGER_PCIE_MK2_getCounterValue(STAR_DEVICE_ID deviceld, U32 counter, U32 *pValue)	802
8.70.3.113 TRIGGER_PCIE_MK2_getDeviceClockRate()	803
8.70.3.114 TRIGGER_PCIE_MK2_getExtTriggerEdgeDetectModeEnabled(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 *pEnabled)	803
8.70.3.115 TRIGGER_PCIE_MK2_getExtTriggerExtend(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 *pExtend)	804
8.70.3.116 TRIGGER_PCIE_MK2_getExtTriggerInvertEnabled(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 *pEnabled)	804
8.70.3.117 TRIGGER_PCIE_MK2_getExtTriggerOutputEnabled(STAR_DEVICE_ID deviceld, U32 extTrigger, U32 *pEnabled)	804

8.70.3.118	TRIGGER_PCIE_MK2_getPortSpillEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	805
8.70.3.119	TRIGGER_PCIE_MK2_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	805
8.70.3.120	TRIGGER_PCIE_MK2_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	806
8.70.3.121	TRIGGER_PCIE_MK2_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	807
8.70.3.122	TRIGGER_PCIE_MK2_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	807
8.70.3.123	TRIGGER_PCIE_MK2_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	808
8.70.3.124	TRIGGER_PCIE_MK2_getTriggerMatrix()	808
8.70.3.125	TRIGGER_PCIE_MK2_reset(STAR_DEVICE_ID deviceId)	808
8.70.3.126	TRIGGER_PCIE_MK2_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	809
8.70.3.127	TRIGGER_PCIE_MK2_setExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)	809
8.70.3.128	TRIGGER_PCIE_MK2_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	810
8.70.3.129	TRIGGER_PCIE_MK2_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	810
8.70.3.130	TRIGGER_PCIE_MK2_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	811
8.70.3.131	TRIGGER_PXI_IF_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	811
8.70.3.132	TRIGGER_PXI_IF_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	812
8.70.3.133	TRIGGER_PXI_IF_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	812
8.70.3.134	TRIGGER_PXI_IF_disableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	812
8.70.3.135	TRIGGER_PXI_IF_disableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	813
8.70.3.136	TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	813
8.70.3.137	TRIGGER_PXI_IF_disableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	814
8.70.3.138	TRIGGER_PXI_IF_disableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	814
8.70.3.139	TRIGGER_PXI_IF_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	814
8.70.3.140	TRIGGER_PXI_IF_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	815
8.70.3.141	TRIGGER_PXI_IF_enableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	815
8.70.3.142	TRIGGER_PXI_IF_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	816

8.70.3.143	TRIGGER_PXI_IF_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	816
8.70.3.144	TRIGGER_PXI_IF_enableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	817
8.70.3.145	TRIGGER_PXI_IF_enableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	817
8.70.3.146	TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId, U32 extTrigger)	817
8.70.3.147	TRIGGER_PXI_IF_enableExtTriggerInvert(STAR_DEVICE_ID deviceId, U32 extTrigger)	818
8.70.3.148	TRIGGER_PXI_IF_enableExtTriggerOutput(STAR_DEVICE_ID deviceId, U32 extTrigger)	818
8.70.3.149	TRIGGER_PXI_IF_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	819
8.70.3.150	TRIGGER_PXI_IF_enableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	819
8.70.3.151	TRIGGER_PXI_IF_forceCounterReload(STAR_DEVICE_ID deviceId, U32 counter)	820
8.70.3.152	TRIGGER_PXI_IF_forceTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	820
8.70.3.153	TRIGGER_PXI_IF_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	821
8.70.3.154	TRIGGER_PXI_IF_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	821
8.70.3.155	TRIGGER_PXI_IF_getCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)	822
8.70.3.156	TRIGGER_PXI_IF_getCounterStartModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	823
8.70.3.157	TRIGGER_PXI_IF_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	823
8.70.3.158	TRIGGER_PXI_IF_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	824
8.70.3.159	TRIGGER_PXI_IF_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	824
8.70.3.160	TRIGGER_PXI_IF_getDeviceClockRate()	824
8.70.3.161	TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	825
8.70.3.162	TRIGGER_PXI_IF_getExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)	825
8.70.3.163	TRIGGER_PXI_IF_getExtTriggerInvertEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	825
8.70.3.164	TRIGGER_PXI_IF_getExtTriggerOutputEnabled(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)	826
8.70.3.165	TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	826
8.70.3.166	TRIGGER_PXI_IF_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	827
8.70.3.167	TRIGGER_PXI_IF_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	827

8.70.3.168	TRIGGER_PXI_IF_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	827
8.70.3.169	TRIGGER_PXI_IF_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	828
8.70.3.170	TRIGGER_PXI_IF_getTriggerMatrix()	828
8.70.3.171	TRIGGER_PXI_IF_reset(STAR_DEVICE_ID deviceId)	829
8.70.3.172	TRIGGER_PXI_IF_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	830
8.70.3.173	TRIGGER_PXI_IF_setExtTriggerExtend(STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)	830
8.70.3.174	TRIGGER_PXI_IF_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	831
8.70.3.175	TRIGGER_PXI_IF_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	831
8.70.3.176	TRIGGER_PXI_IF_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	832
8.70.3.177	TRIGGER_PXI_ROUTER_disableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	832
8.70.3.178	TRIGGER_PXI_ROUTER_disableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	833
8.70.3.179	TRIGGER_PXI_ROUTER_disableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	833
8.70.3.180	TRIGGER_PXI_ROUTER_disableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	834
8.70.3.181	TRIGGER_PXI_ROUTER_disableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	835
8.70.3.182	TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	835
8.70.3.183	TRIGGER_PXI_ROUTER_disableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	836
8.70.3.184	TRIGGER_PXI_ROUTER_enableCounterAutoReload(STAR_DEVICE_ID deviceId, U32 counter)	836
8.70.3.185	TRIGGER_PXI_ROUTER_enableCounterLoadZero(STAR_DEVICE_ID deviceId, U32 counter)	836
8.70.3.186	TRIGGER_PXI_ROUTER_enableCounterStartMode(STAR_DEVICE_ID deviceId, U32 counter)	837
8.70.3.187	TRIGGER_PXI_ROUTER_enableCounterStartStopMode(STAR_DEVICE_ID deviceId, U32 counter)	837
8.70.3.188	TRIGGER_PXI_ROUTER_enableCounterTriggerCount(STAR_DEVICE_ID deviceId, U32 counter)	838
8.70.3.189	TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode(STAR_DEVICE_ID deviceId, U32 port)	838
8.70.3.190	TRIGGER_PXI_ROUTER_enableTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	839
8.70.3.191	TRIGGER_PXI_ROUTER_forceCounterReload(STAR_DEVICE_ID deviceId, U32 counter)	839
8.70.3.192	TRIGGER_PXI_ROUTER_forceTrigger(STAR_DEVICE_ID deviceId, U32 trigger)	840

8.70.3.193	TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	840
8.70.3.194	TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	841
8.70.3.195	TRIGGER_PXI_ROUTER_getCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)	842
8.70.3.196	TRIGGER_PXI_ROUTER_getCounterStartModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	842
8.70.3.197	TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	843
8.70.3.198	TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled(STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)	843
8.70.3.199	TRIGGER_PXI_ROUTER_getCounterValue(STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)	843
8.70.3.200	TRIGGER_PXI_ROUTER_getDeviceClockRate()	844
8.70.3.201	TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled(STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)	844
8.70.3.202	TRIGGER_PXI_ROUTER_getTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	845
8.70.3.203	TRIGGER_PXI_ROUTER_getTriggerEnabled(STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)	846
8.70.3.204	TRIGGER_PXI_ROUTER_getTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)	846
8.70.3.205	TRIGGER_PXI_ROUTER_getTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)	847
8.70.3.206	TRIGGER_PXI_ROUTER_getTriggerMatrix()	847
8.70.3.207	TRIGGER_PXI_ROUTER_reset(STAR_DEVICE_ID deviceId)	847
8.70.3.208	TRIGGER_PXI_ROUTER_setCounterReloadValue(STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)	848
8.70.3.209	TRIGGER_PXI_ROUTER_setTriggerAndMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	848
8.70.3.210	TRIGGER_PXI_ROUTER_setTriggerInputMode(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)	849
8.70.3.211	TRIGGER_PXI_ROUTER_setTriggerInvertMask(STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)	849
8.71	General	851
8.71.1	Detailed Description	851
8.71.2	Function Documentation	851
8.71.2.1	CFG_getGeneralPurpose(STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)	851
8.71.2.2	CFG_getNetworkIdentity(STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)	852
8.71.2.3	CFG_getRouterGlobalSettings(STAR_DEVICE_ID deviceId, _Out_ STAR_CONFIG_ROUTER_GLOBAL_STATE *pState)	852
8.71.2.4	CFG_refreshDeviceInfo(STAR_DEVICE_ID deviceId)	853
8.71.2.5	CFG_setGeneralPurpose(STAR_DEVICE_ID deviceId, REGISTER value)	853

8.71.2.6	CFG_setNetworkIdentity(STAR_DEVICE_ID deviceId, REGISTER value)	854
8.71.2.7	CFG_setRouterGlobalSettings(STAR_DEVICE_ID deviceId, _In_ STAR_CFG↔ _ROUTER_GLOBAL_STATE *pState)	854
8.71.2.8	CFG_updateDeviceInfo(STAR_DEVICE_ID deviceId)	855
8.72	Hardware	856
8.72.1	Detailed Description	856
8.72.2	Data Structure Documentation	856
8.72.2.1	struct STAR_CFG_FPGA_INFO	856
8.72.3	Function Documentation	857
8.72.3.1	CFG_FPGAInfoToString(STAR_DEVICE_ID deviceId, _In_ STAR_CFG_FPG↔ A_INFO *pFPGAInfo, _Out_z_cap_c_ (STAR_CFG_MK2_VERSION_STR_M↔ AX_LEN) char *pVersion, _Out_z_cap_c_ (STAR_CFG_MK2_BUILD_DATE_↔ STR_MAX_LEN) char *pBuildDate)	857
8.72.3.2	CFG_getFPGAInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_FPGA_I↔ NFO *pFPGAInfo)	858
8.72.3.3	CFG_identify(STAR_DEVICE_ID deviceId)	858
8.73	Port Status and Control	860
8.73.1	Detailed Description	861
8.73.2	Function Documentation	861
8.73.2.1	CFG_clearPortErrors(STAR_DEVICE_ID deviceId, U8 portNum)	861
8.73.2.2	CFG_getConfigPortErrors(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)	861
8.73.2.3	CFG_getExternalPortErrors(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)	862
8.73.2.4	CFG_getExternalPortStatus(PORT_STATUS_CONTROL portStatusControl, _↔ Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)	862
8.73.2.5	CFG_getPortConnection(PORT_STATUS_CONTROL portStatusControl)	863
8.73.2.6	CFG_getPortStatusControl(STAR_DEVICE_ID deviceId, U8 portNum, _Out_↔ PORT_STATUS_CONTROL *pPortStatusControl)	863
8.73.2.7	CFG_getPortType(PORT_STATUS_CONTROL portStatusControl)	864
8.73.2.8	CFG_getSpaceWireLinkErrors(PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)	864
8.73.2.9	CFG_getSpaceWireLinkStatus(PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pStatus)	865
8.73.2.10	CFG_setSpaceWireLinkStatus(STAR_DEVICE_ID deviceId, U8 linkNum, _In_↔ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)	865
8.73.2.11	CFG_startLink(STAR_DEVICE_ID deviceId, U8 linkNum)	866
8.73.2.12	CFG_stopLink(STAR_DEVICE_ID deviceId, U8 linkNum)	867
8.74	Device Identifier	868
8.74.1	Detailed Description	868
8.74.2	Function Documentation	868
8.74.2.1	CFG_getDeviceIdentificationInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_C↔ FG_DEVICE_IDENTIFIER_INFO *pInfo)	868

8.74.2.2	CFG_getDeviceManufacturerAsString(_In_ STAR_CFG_DEVICE_IDENTIFIE← R_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER← ER_STR_MAX_LEN) char *pManufacturerStr)	869
8.74.2.3	CFG_getDeviceTypeAsString(_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)	869
8.74.2.4	CFG_getNetworkDiscoveryInfo(STAR_DEVICE_ID deviceId, _Out_ STAR_CF← G_NETWORK_DISCOVERY_INFO *pInfo)	870
8.75	Group Adaptive Routing	871
8.75.1	Detailed Description	871
8.75.2	Function Documentation	871
8.75.2.1	CFG_getRoutingTableEntry(STAR_DEVICE_ID deviceId, U8 logicalAddress, _← Out_ STAR_CFG_GAR_ENTRY *pEntry)	871
8.75.2.2	CFG_setRoutingTableEntry(STAR_DEVICE_ID deviceId, U8 logicalAddress, _← In_ STAR_CFG_GAR_ENTRY *pEntry)	872
8.76	Interface Mode	873
8.76.1	Detailed Description	874
8.76.2	Function Documentation	874
8.76.2.1	CFG_disableIdentifySource(STAR_DEVICE_ID deviceId)	874
8.76.2.2	CFG_disableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	874
8.76.2.3	CFG_disableInterfaceMode(STAR_DEVICE_ID deviceId)	875
8.76.2.4	CFG_disableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	875
8.76.2.5	CFG_enableIdentifySource(STAR_DEVICE_ID deviceId)	876
8.76.2.6	CFG_enableIdentifySourceOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	876
8.76.2.7	CFG_enableInterfaceMode(STAR_DEVICE_ID deviceId)	877
8.76.2.8	CFG_enableInterfaceModeOnPort(STAR_DEVICE_ID deviceId, U8 portNum)	878
8.76.2.9	CFG_getIdentifySourceEnabled(STAR_DEVICE_ID deviceId, _Out_ int *p← Enabled)	878
8.76.2.10	CFG_getIdentifySourceOnPortEnabled(STAR_DEVICE_ID deviceId, U8 port← Num, _Out_ int *pEnabled)	879
8.76.2.11	CFG_getInterfaceModeEnabled(STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)	879
8.76.2.12	CFG_getInterfaceModeOnPortEnabled(STAR_DEVICE_ID deviceId, U8 port← Num, _Out_ int *pEnabled)	880
8.76.2.13	CFG_getPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, _Out← _ U8 *pAddress)	881
8.76.2.14	CFG_setPortRoutingAddress(STAR_DEVICE_ID deviceId, U8 portNum, U8 ad← dress)	881
8.77	Time-codes	882
8.77.1	Detailed Description	883
8.77.2	Function Documentation	883
8.77.2.1	CFG_disableExternalTimeCodeSelection(STAR_DEVICE_ID deviceId)	883
8.77.2.2	CFG_disableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceId)	883
8.77.2.3	CFG_disableTimeCodeMaster(STAR_DEVICE_ID deviceId)	884
8.77.2.4	CFG_enableExternalTimeCodeSelection(STAR_DEVICE_ID deviceId)	884

8.77.2.5	CFG_enableTimeCodeCounterBypassMode(STAR_DEVICE_ID deviceID)	885
8.77.2.6	CFG_enableTimeCodeMaster(STAR_DEVICE_ID deviceID)	885
8.77.2.7	CFG_getExternalTimeCodeSelectionEnabled(STAR_DEVICE_ID deviceID, Out_int *pEnabled)	886
8.77.2.8	CFG_getTimeCodeCounterBypassModeEnabled(STAR_DEVICE_ID deviceID, Out_int *pEnabled)	886
8.77.2.9	CFG_getTimeCodeDistributionCapablePorts(STAR_DEVICE_ID deviceID, Out_U32 *pPortMask)	887
8.77.2.10	CFG_getTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, Out_U32 *pPortMask)	887
8.77.2.11	CFG_getTimeCodeFlagMode(STAR_DEVICE_ID deviceID, Out_U8 *pMode)	888
8.77.2.12	CFG_getTimeCodeMasterEnabled(STAR_DEVICE_ID deviceID, Out_int *pEnabled)	888
8.77.2.13	CFG_getTimeCodePeriod(STAR_DEVICE_ID deviceID, Out_U32 *pPeriod)	889
8.77.2.14	CFG_getTimeCodeValue(STAR_DEVICE_ID deviceID, Out_U8 *pValue)	889
8.77.2.15	CFG_setTimeCodeDistributionPorts(STAR_DEVICE_ID deviceID, U32 portMask)	890
8.77.2.16	CFG_setTimeCodeFlagMode(STAR_DEVICE_ID deviceID, U8 mode)	891
8.77.2.17	CFG_setTimeCodePeriod(STAR_DEVICE_ID deviceID, U32 period)	891
8.78	Events	892
8.78.1	Detailed Description	892
8.78.2	Function Documentation	892
8.78.2.1	CFG_disableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	892
8.78.2.2	CFG_disableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	893
8.78.2.3	CFG_disableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	894
8.78.2.4	CFG_enableSpeedChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	894
8.78.2.5	CFG_enableStateChangeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	895
8.78.2.6	CFG_enableTimeCodeEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	895
8.78.2.7	CFG_getSpeedChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, Out_int *pEnabled)	896
8.78.2.8	CFG_getStateChangeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, Out_int *pEnabled)	897
8.78.2.9	CFG_getTimeCodeEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, Out_int *pEnabled)	897
8.79	Transmit Link Speed	899
8.79.1	Detailed Description	899
8.79.2	Function Documentation	899
8.79.2.1	CFG_getTxSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, Out_double *pSignallingRate)	899
8.79.2.2	CFG_setTxSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, double signallingRate, Out_double *pSignallingRateSet)	900

8.80	Receive Link Speed	901
8.80.1	Detailed Description	901
8.80.2	Function Documentation	901
8.80.2.1	CFG_getRxSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ double *pSignallingRate)	901
8.81	Precision Links	902
8.81.1	Detailed Description	902
8.81.2	Function Documentation	902
8.81.2.1	CFG_disablePrecisionTransmitRate(STAR_DEVICE_ID deviceID)	902
8.81.2.2	CFG_enablePrecisionTransmitRate(STAR_DEVICE_ID deviceID)	903
8.81.2.3	CFG_getPrecisionTransmitRate(STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)	903
8.81.2.4	CFG_getPrecisionTransmitRateEnabled(STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)	904
8.81.2.5	CFG_getPrecisionTransmitRateInUse(STAR_DEVICE_ID deviceID, _Out_ int *pInUse)	905
8.81.2.6	CFG_setPrecisionTransmitRate(STAR_DEVICE_ID deviceID, double transmitRate)	905
8.82	Error Injection	906
8.82.1	Detailed Description	906
8.82.2	Function Documentation	906
8.82.2.1	CFG_injectError(STAR_DEVICE_ID deviceID, U8 portNum, SPW_ERROR error)	906
8.82.2.2	CFG_injectErrors(STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)	907
8.83	Timestamping	908
8.83.1	Detailed Description	908
8.83.2	Function Documentation	908
8.83.2.1	CFG_disableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	908
8.83.2.2	CFG_enableRxTimestampEventsOnPort(STAR_DEVICE_ID deviceID, U8 portNum)	909
8.83.2.3	CFG_getPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)	910
8.83.2.4	CFG_getRxTimestampEventsOnPortEnabled(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)	910
8.83.2.5	CFG_getTimestampMethod(STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pMethod)	911
8.83.2.6	CFG_getTimestampValue(STAR_DEVICE_ID deviceID, _Out_ U32 *pValue)	911
8.83.2.7	CFG_setPulseGeneratorFrequency(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)	912
8.83.2.8	CFG_setTimestampMethod(STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method)	912
8.83.2.9	CFG_setTimestampValue(STAR_DEVICE_ID deviceID, U32 value)	913
8.84	Periodic	914
8.84.1	Detailed Description	914

8.84.2	Function Documentation	914
8.84.2.1	CFG_getDeviceClockRate(STAR_DEVICE_ID deviceID)	914
8.84.2.2	CFG_startPeriodicAction(STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action)	915
8.84.2.3	CFG_stopPeriodicAction(STAR_DEVICE_ID deviceID, unsigned char port)	915
8.85	Broadcast Controller	916
8.85.1	Detailed Description	916
8.85.2	Function Documentation	916
8.85.2.1	CFG_disableBroadcastControllerLegacyMode(STAR_DEVICE_ID deviceID)	916
8.85.2.2	CFG_enableBroadcastControllerLegacyMode(STAR_DEVICE_ID deviceID)	917
8.85.2.3	CFG_getBroadcastControllerLegacyModeEnabled(STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)	917
8.85.2.4	CFG_getInterruptCodeDistributionPorts(STAR_DEVICE_ID deviceID, _Out_ U32 *pPortMask)	918
8.85.2.5	CFG_setInterruptCodeDistributionPorts(STAR_DEVICE_ID deviceID, U32 portMask)	918
8.86	Defines	920
8.86.1	Detailed Description	922
8.86.2	Macro Definition Documentation	923
8.86.2.1	CFG_STATUS_FAILURE	923
8.86.2.2	CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION	923
8.86.2.3	CFG_STATUS_SUCCESS	923
8.87	Device Information & Identification	924
8.87.1	Detailed Description	924
8.88	Device Information	925
8.88.1	Detailed Description	925
8.88.2	Function Documentation	925
8.88.2.1	CFG_SPFI_getDeviceInformation(STAR_DEVICE_ID deviceID, _Out_ SPFI_DEVICE_INFO *pDeviceInfo)	925
8.88.2.2	CFG_SPFI_getDeviceNodeType(SPFI_DEVICE_INFO deviceInfo)	926
8.88.2.3	CFG_SPFI_getDevicePortCount(SPFI_DEVICE_INFO deviceInfo)	926
8.88.2.4	CFG_SPFI_getDevicePortCountByType(STAR_DEVICE_ID deviceID, SPFI_PORT_TYPE portType, _Out_ U8 *pPortCount)	927
8.89	Device Identification	928
8.89.1	Detailed Description	928
8.89.2	Function Documentation	928
8.89.2.1	CFG_SPFI_getDeviceIdentificationInfo(STAR_DEVICE_ID deviceID, _Out_ SPFI_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo)	928
8.89.2.2	CFG_SPFI_getDeviceManufacturer(SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)	929

8.89.2.3	CFG_SPFI_getDeviceManufacturerAsString(SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_ z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)	929
8.89.2.4	CFG_SPFI_getDeviceType(SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)	930
8.89.2.5	CFG_SPFI_getDeviceTypeAsString(SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_ z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)	930
8.89.2.6	CFG_SPFI_getDeviceVersion(SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_ U8 *pMajorVersion, _Out_ U8 *pMinorVersion, _Out_ U8 *pPatchVersion)	931
8.90	Routing Table	932
8.90.1	Detailed Description	932
8.90.2	Function Documentation	933
8.90.2.1	CFG_SPFI_disableRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logicalAddress)	933
8.90.2.2	CFG_SPFI_disableRoutingTableEntryDeleteHeader(STAR_DEVICE_ID deviceID, U8 logicalAddress)	934
8.90.2.3	CFG_SPFI_enableRoutingTableEntry(STAR_DEVICE_ID deviceID, U8 logicalAddress)	934
8.90.2.4	CFG_SPFI_enableRoutingTableEntryDeleteHeader(STAR_DEVICE_ID deviceID, U8 logicalAddress)	935
8.90.2.5	CFG_SPFI_getRoutingTableEntryFlags(STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ SPFI_ROUTING_TABLE_ENTRY_FLAGS *pRoutingTableEntryFlags)	935
8.90.2.6	CFG_SPFI_getRoutingTableEntryPorts(STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ U32 *pRoutingTableEntryPorts)	936
8.90.2.7	CFG_SPFI_isRoutingTableEntryDeleteHeaderEnabled(SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)	936
8.90.2.8	CFG_SPFI_isRoutingTableEntryEnabled(SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)	937
8.90.2.9	CFG_SPFI_setRoutingTableEntryPorts(STAR_DEVICE_ID deviceID, U8 logicalAddress, U32 routingTableEntryPorts)	937
8.91	Network Management	939
8.91.1	Detailed Description	940
8.91.2	Function Documentation	940
8.91.2.1	CFG_SPFI_getBroadcastTimeout(STAR_DEVICE_ID deviceID, _Out_ double *pTimeout)	940
8.91.2.2	CFG_SPFI_getBroadcastTimeoutMaximum(STAR_DEVICE_ID deviceID, _Out_ double *pMaximumTimeout)	940
8.91.2.3	CFG_SPFI_getBroadcastTimeoutResolution(STAR_DEVICE_ID deviceID, _Out_ double *pResolution)	941
8.91.2.4	CFG_SPFI_getCurrentTime(STAR_DEVICE_ID deviceID, _Out_ U64 *pCurrentTime, _Out_ U8 *pSynchronised)	942
8.91.2.5	CFG_SPFI_getCurrentTimeSlot(STAR_DEVICE_ID deviceID, _Out_ U8 *pCurrentTimeSlot)	942

8.91.2.6	CFG_SPFI_getDeviceIdentifier(STAR_DEVICE_ID deviceId, _Out_ U32 *pDeviceIdentifier)	943
8.91.2.7	CFG_SPFI_getLinksWithErrors(STAR_DEVICE_ID deviceId, _Out_ U32 *pLinksWithErrors)	943
8.91.2.8	CFG_SPFI_getPortsReady(STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsReady)	944
8.91.2.9	CFG_SPFI_getPortsWithErrors(STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsWithErrors)	944
8.91.2.10	CFG_SPFI_getSpaceWireBC(STAR_DEVICE_ID deviceId, _Out_ U8 *pRouterBC)	945
8.91.2.11	CFG_SPFI_setBroadcastTimeout(STAR_DEVICE_ID deviceId, double broadcastTimeout)	945
8.91.2.12	CFG_SPFI_setDeviceIdentifier(STAR_DEVICE_ID deviceId, U32 deviceIdentifier)	946
8.91.2.13	CFG_SPFI_setSpaceWireBC(STAR_DEVICE_ID deviceId, U8 routerBC)	946
8.92	Vendor Specific	947
8.92.1	Detailed Description	948
8.92.2	Function Documentation	948
8.92.2.1	CFG_SPFI_disableRouterTimeout(STAR_DEVICE_ID deviceId)	948
8.92.2.2	CFG_SPFI_enableRouterTimeout(STAR_DEVICE_ID deviceId)	948
8.92.2.3	CFG_SPFI_getBroadcastTypesInformation(STAR_DEVICE_ID deviceId, _Out_ SPFI_BROADCAST_TYPES_INFO *pBroadcastTypesInfo)	949
8.92.2.4	CFG_SPFI_getRouterTimeout(STAR_DEVICE_ID deviceId, _Out_ double *pTimeout)	949
8.92.2.5	CFG_SPFI_getRouterTimeoutMaximum(STAR_DEVICE_ID deviceId, _Out_ double *pMaximumTimeout)	949
8.92.2.6	CFG_SPFI_getRouterTimeoutResolution(STAR_DEVICE_ID deviceId, _Out_ double *pResolution)	950
8.92.2.7	CFG_SPFI_getSpaceWireInterruptBroadcastType(SPFI_BROADCAST_TYPES_INFO *pBroadcastTypesInfo)	950
8.92.2.8	CFG_SPFI_getTimeCodeBroadcastType(SPFI_BROADCAST_TYPES_INFO *pBroadcastTypesInfo)	951
8.92.2.9	CFG_SPFI_getTimeSlotBroadcastType(SPFI_BROADCAST_TYPES_INFO *pBroadcastTypesInfo)	951
8.92.2.10	CFG_SPFI_isRouterTimeoutEnabled(STAR_DEVICE_ID deviceId, _Out_ U8 *pEnabled)	952
8.92.2.11	CFG_SPFI_setRouterTimeout(STAR_DEVICE_ID deviceId, double routerTimeout)	952
8.92.2.12	CFG_SPFI_setSpaceWireInterruptBroadcastType(STAR_DEVICE_ID deviceId, U8 interruptValue)	953
8.92.2.13	CFG_SPFI_setTimeCodeBroadcastType(STAR_DEVICE_ID deviceId, U8 timeCodeValue)	953
8.92.2.14	CFG_SPFI_setTimeSlotBroadcastType(STAR_DEVICE_ID deviceId, U8 timeSlotValue)	954
8.93	Virtual Channel Information	955
8.93.1	Detailed Description	955
8.94	Virtual Network Number	956

8.94.1	Detailed Description	956
8.94.2	Function Documentation	956
8.94.2.1	CFG_SPFI_getVCVN(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ U8 *pVirtualNetwork)	956
8.94.2.2	CFG_SPFI_setVCVN(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 virtualNetwork)	957
8.95	Status	958
8.95.1	Detailed Description	958
8.95.2	Function Documentation	958
8.95.2.1	CFG_SPFI_getVCStatus(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ SPFI_VC_STATUS *pStatus)	958
8.95.2.2	CFG_SPFI_isVCInputBufferFull(SPFI_VC_STATUS status)	959
8.95.2.3	CFG_SPFI_isVCInputBufferHalfFull(SPFI_VC_STATUS status)	959
8.95.2.4	CFG_SPFI_isVCInputBufferNotEmpty(SPFI_VC_STATUS status)	960
8.95.2.5	CFG_SPFI_isVCOutputBufferFull(SPFI_VC_STATUS status)	960
8.95.2.6	CFG_SPFI_isVCOutputBufferHalfFull(SPFI_VC_STATUS status)	961
8.95.2.7	CFG_SPFI_isVCOverused(SPFI_VC_STATUS status)	961
8.95.2.8	CFG_SPFI_isVCUnderused(SPFI_VC_STATUS status)	961
8.96	Errors	963
8.96.1	Detailed Description	963
8.96.2	Function Documentation	963
8.96.2.1	CFG_SPFI_clearVCErrors(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel)	963
8.96.2.2	CFG_SPFI_getVCErrors(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ SPFI_VC_ERRORS *pErrors)	964
8.96.2.3	CFG_SPFI_isVCAddressErrorDetected(SPFI_VC_ERRORS errors)	964
8.96.2.4	CFG_SPFI_isVCTimeoutDetected(SPFI_VC_ERRORS errors)	965
8.96.2.5	CFG_SPFI_isVCVNErrorsDetected(SPFI_VC_ERRORS errors)	965
8.97	Quality Of Service	966
8.97.1	Detailed Description	966
8.97.2	Function Documentation	966
8.97.2.1	CFG_SPFI_disableVCContinuousMode(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel)	966
8.97.2.2	CFG_SPFI_enableVCContinuousMode(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel)	967
8.97.2.3	CFG_SPFI_getVCBandwidthReservation(SPFI_QUALITY_OF_SERVICE qualityOfService)	968
8.97.2.4	CFG_SPFI_getVCPriority(SPFI_QUALITY_OF_SERVICE qualityOfService)	968
8.97.2.5	CFG_SPFI_getVCQualityOfService(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ SPFI_QUALITY_OF_SERVICE *pQualityOfService)	969
8.97.2.6	CFG_SPFI_isVCContinuousModeEnabled(SPFI_QUALITY_OF_SERVICE qualityOfService)	969

8.97.2.7	CFG_SPFI_setVCBandwidthReservation(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 bandwidth)	970
8.97.2.8	CFG_SPFI_setVCPriority(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 priority)	970
8.98	Time-slots	972
8.98.1	Detailed Description	972
8.98.2	Function Documentation	972
8.98.2.1	CFG_SPFI_getVCTimeSlots(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ U64 *pVCTimeSlots)	972
8.98.2.2	CFG_SPFI_setVCTimeSlots(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U64 timeSlots)	973
8.99	Data Rate	974
8.99.1	Detailed Description	974
8.99.2	Function Documentation	974
8.99.2.1	CFG_SPFI_getVCDataRate(STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ U64 *pTxDataRate, _Out_ U64 *pRxDataRate)	974
8.100	Port Information	975
8.100.1	Detailed Description	975
8.100.2	Function Documentation	975
8.100.2.1	CFG_SPFI_clearPortBroadcastError(STAR_DEVICE_ID deviceID, U8 port)	975
8.100.2.2	CFG_SPFI_getAXIBC(SPFI_PORT_INFO portInfo)	976
8.100.2.3	CFG_SPFI_getPortInformation(STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_PORT_INFO *pPortInfo)	976
8.100.2.4	CFG_SPFI_getPortType(SPFI_PORT_INFO portInfo)	977
8.100.2.5	CFG_SPFI_getPortVCCount(SPFI_PORT_INFO portInfo)	977
8.100.2.6	CFG_SPFI_getPortVCsWithErrors(STAR_DEVICE_ID deviceID, U8 port, _Out_ U32 *pVCsWithErrors)	978
8.100.2.7	CFG_SPFI_isPortBroadcastErrorDetected(SPFI_PORT_INFO portInfo)	978
8.100.2.8	CFG_SPFI_isPortReady(SPFI_PORT_INFO portInfo)	979
8.100.2.9	CFG_SPFI_setAXIBC(STAR_DEVICE_ID deviceID, U8 port, U8 axiBC)	979
8.101	Link Information	981
8.101.1	Detailed Description	981
8.102	Control	982
8.102.1	Detailed Description	983
8.102.2	Function Documentation	983
8.102.2.1	CFG_SPFI_disableLinkAutoStart(STAR_DEVICE_ID deviceID, U8 port)	983
8.102.2.2	CFG_SPFI_disableLinkInterfaceReset(STAR_DEVICE_ID deviceID, U8 port)	983
8.102.2.3	CFG_SPFI_disableLinkReset(STAR_DEVICE_ID deviceID, U8 port)	983
8.102.2.4	CFG_SPFI_disableLinkStart(STAR_DEVICE_ID deviceID, U8 port)	984
8.102.2.5	CFG_SPFI_enableLinkAutoStart(STAR_DEVICE_ID deviceID, U8 port)	984
8.102.2.6	CFG_SPFI_enableLinkInterfaceReset(STAR_DEVICE_ID deviceID, U8 port)	985
8.102.2.7	CFG_SPFI_enableLinkReset(STAR_DEVICE_ID deviceID, U8 port)	985

8.102.2.8	CFG_SPFI_enableLinkStart(STAR_DEVICE_ID deviceID, U8 port)	986
8.102.2.9	CFG_SPFI_getLinkControlSettings(STAR_DEVICE_ID deviceID, U8 port, Out_SPFI_LINK_CONTROL_SETTINGS *pLinkControlSettings)	986
8.102.2.10	CFG_SPFI_getLinkLanesRequestedCount(SPFI_LINK_CONTROL_SETTINGS linkControlSettings)	987
8.102.2.11	CFG_SPFI_isLinkAutoStartEnabled(SPFI_LINK_CONTROL_SETTINGS linkControlSettings)	988
8.102.2.12	CFG_SPFI_isLinkInterfaceResetEnabled(SPFI_LINK_CONTROL_SETTINGS linkControlSettings)	988
8.102.2.13	CFG_SPFI_isLinkResetEnabled(SPFI_LINK_CONTROL_SETTINGS linkControlSettings)	989
8.102.2.14	CFG_SPFI_isLinkStartEnabled(SPFI_LINK_CONTROL_SETTINGS linkControlSettings)	989
8.102.2.15	CFG_SPFI_setLinkLanesRequestedCount(STAR_DEVICE_ID deviceID, U8 port, U8 lanesRequestedCount)	990
8.103	Status	991
8.103.1	Detailed Description	991
8.103.2	Function Documentation	991
8.103.2.1	CFG_SPFI_clearLinkProtocolError(STAR_DEVICE_ID deviceID, U8 port)	991
8.103.2.2	CFG_SPFI_getLinkAlignmentState(SPFI_LINK_STATUS linkStatus)	992
8.103.2.3	CFG_SPFI_getLinkStatus(STAR_DEVICE_ID deviceID, U8 port, Out_SPFI_LINK_STATUS *pLinkStatus)	992
8.103.2.4	CFG_SPFI_isLinkErrorDetected(SPFI_LINK_STATUS linkStatus)	993
8.103.2.5	CFG_SPFI_isLinkEventDetected(SPFI_LINK_STATUS linkStatus)	993
8.103.2.6	CFG_SPFI_isLinkIdle(SPFI_LINK_STATUS linkStatus)	994
8.103.2.7	CFG_SPFI_isLinkLanesEventDetected(SPFI_LINK_STATUS linkStatus)	994
8.103.2.8	CFG_SPFI_isLinkProtocolErrorDetected(SPFI_LINK_STATUS linkStatus)	995
8.103.2.9	CFG_SPFI_isLinkReady(SPFI_LINK_STATUS linkStatus)	995
8.103.2.10	CFG_SPFI_isLinkReceiving(SPFI_LINK_STATUS linkStatus)	995
8.103.2.11	CFG_SPFI_isLinkTransmitting(SPFI_LINK_STATUS linkStatus)	996
8.104	Events	997
8.104.1	Detailed Description	997
8.104.2	Function Documentation	997
8.104.2.1	CFG_SPFI_clearLinkEvents(STAR_DEVICE_ID deviceID, U8 port)	997
8.104.2.2	CFG_SPFI_getLinkEvents(STAR_DEVICE_ID deviceID, U8 port, Out_SPFI_LINK_EVENTS *pLinkEvents)	998
8.104.2.3	CFG_SPFI_getLinkRetryCount(SPFI_LINK_EVENTS linkEvents)	998
8.104.2.4	CFG_SPFI_isLinkCRC16ErrorDetected(SPFI_LINK_EVENTS linkEvents)	999
8.104.2.5	CFG_SPFI_isLinkCRC8ErrorDetected(SPFI_LINK_EVENTS linkEvents)	999
8.104.2.6	CFG_SPFI_isLinkFarEndResetDetected(SPFI_LINK_EVENTS linkEvents)	1000
8.104.2.7	CFG_SPFI_isLinkFrameErrorDetected(SPFI_LINK_EVENTS linkEvents)	1000
8.105	Debug	1001
8.105.1	Detailed Description	1001

8.105.2 Function Documentation	1001
8.105.2.1 CFG_SPFI_disableLinkNearEndLoopback(STAR_DEVICE_ID deviceID, U8 port)	1001
8.105.2.2 CFG_SPFI_disableLinkRxScrambling(STAR_DEVICE_ID deviceID, U8 port)	1002
8.105.2.3 CFG_SPFI_disableLinkTxScrambling(STAR_DEVICE_ID deviceID, U8 port)	1002
8.105.2.4 CFG_SPFI_enableLinkNearEndLoopback(STAR_DEVICE_ID deviceID, U8 port)	1003
8.105.2.5 CFG_SPFI_enableLinkRxScrambling(STAR_DEVICE_ID deviceID, U8 port)	1003
8.105.2.6 CFG_SPFI_enableLinkTxScrambling(STAR_DEVICE_ID deviceID, U8 port)	1004
8.105.2.7 CFG_SPFI_getLinkDebugSettings(STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_DEBUG_SETTINGS *pLinkDebugSettings)	1004
8.105.2.8 CFG_SPFI_isLinkNearEndLoopbackEnabled(SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)	1005
8.105.2.9 CFG_SPFI_isLinkRxScramblingEnabled(SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)	1006
8.105.2.10 CFG_SPFI_isLinkTxScramblingEnabled(SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)	1006
8.106 Active Lanes	1008
8.106.1 Detailed Description	1008
8.106.2 Function Documentation	1008
8.106.2.1 CFG_SPFI_getLinkActiveLanes(STAR_DEVICE_ID deviceID, U8 port, _Out_ U32 *pActiveLanes)	1008
8.107 Lane Information	1009
8.107.1 Detailed Description	1009
8.108 Control	1010
8.108.1 Detailed Description	1011
8.108.2 Function Documentation	1011
8.108.2.1 CFG_SPFI_disableLaneErrorRecovery(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1011
8.108.2.2 CFG_SPFI_disableLaneReset(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1011
8.108.2.3 CFG_SPFI_disableLaneRx(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1012
8.108.2.4 CFG_SPFI_disableLaneTx(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1012
8.108.2.5 CFG_SPFI_enableLaneErrorRecovery(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1012
8.108.2.6 CFG_SPFI_enableLaneReset(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1013
8.108.2.7 CFG_SPFI_enableLaneRx(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1013
8.108.2.8 CFG_SPFI_enableLaneTx(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1014
8.108.2.9 CFG_SPFI_getLaneControlSettings(STAR_DEVICE_ID deviceID, U8 port, U8 lane, _Out_ SPFI_LANE_CONTROL_SETTINGS *pLaneControlSettings)	1014
8.108.2.10 CFG_SPFI_getLaneStandbyReason(SPFI_LANE_CONTROL_SETTINGS laneControlSettings)	1015
8.108.2.11 CFG_SPFI_isLaneErrorRecoveryEnabled(SPFI_LANE_CONTROL_SETTINGS laneControlSettings)	1015
8.108.2.12 CFG_SPFI_isLaneResetEnabled(SPFI_LANE_CONTROL_SETTINGS laneControlSettings)	1016

8.108.2.13	CFG_SPFI_isLaneRxEnabled(SPFI_LANE_CONTROL_SETTINGS lane↔ ControlSettings)	1016
8.108.2.14	CFG_SPFI_isLaneTxEnabled(SPFI_LANE_CONTROL_SETTINGS lane↔ ControlSettings)	1017
8.108.2.15	CFG_SPFI_setLaneStandbyReason(STAR_DEVICE_ID deviceID, U8 port, U8 lane, U8 standbyReason)	1017
8.109	Status	1019
8.109.1	Detailed Description	1019
8.109.2	Function Documentation	1020
8.109.2.1	CFG_SPFI_getLaneFarEndINIT3Flags(SPFI_LANE_STATUS laneStatus)	1020
8.109.2.2	CFG_SPFI_getLaneLOSReasonRx(SPFI_LANE_STATUS laneStatus)	1021
8.109.2.3	CFG_SPFI_getLaneStandbyReasonRx(SPFI_LANE_STATUS laneStatus)	1021
8.109.2.4	CFG_SPFI_getLaneState(SPFI_LANE_STATUS laneStatus)	1022
8.109.2.5	CFG_SPFI_getLaneStatus(STAR_DEVICE_ID deviceID, U8 port, U8 lane, _↔ Out_SPFI_LANE_STATUS *pLaneStatus)	1022
8.109.2.6	CFG_SPFI_isLaneActive(SPFI_LANE_STATUS laneStatus)	1023
8.109.2.7	CFG_SPFI_isLaneErrorDetected(SPFI_LANE_STATUS laneStatus)	1023
8.109.2.8	CFG_SPFI_isLaneEventDetected(SPFI_LANE_STATUS laneStatus)	1024
8.109.2.9	CFG_SPFI_isLaneNoSignalDetected(SPFI_LANE_STATUS laneStatus)	1024
8.109.2.10	CFG_SPFI_isLaneRxOnly(SPFI_LANE_STATUS laneStatus)	1025
8.109.2.11	CFG_SPFI_isLaneStarted(SPFI_LANE_STATUS laneStatus)	1026
8.109.2.12	CFG_SPFI_isLaneTxOnly(SPFI_LANE_STATUS laneStatus)	1026
8.110	Events	1028
8.110.1	Detailed Description	1028
8.110.2	Function Documentation	1028
8.110.2.1	CFG_SPFI_clearLaneEvents(STAR_DEVICE_ID deviceID, U8 port, U8 lane)	1028
8.110.2.2	CFG_SPFI_getLaneDisconnectCount(SPFI_LANE_EVENTS laneEvents)	1029
8.110.2.3	CFG_SPFI_getLaneEvents(STAR_DEVICE_ID deviceID, U8 port, U8 lane, _↔ Out_SPFI_LANE_EVENTS *pLaneEvents)	1029
8.110.2.4	CFG_SPFI_getLaneRxErrorCount(SPFI_LANE_EVENTS laneEvents)	1030
8.110.2.5	CFG_SPFI_isLaneFarEndErrorBurstDetected(SPFI_LANE_EVENTS laneEvents)	1030
8.110.2.6	CFG_SPFI_isLaneFarEndINIT1RxInActiveDetected(SPFI_LANE_EVENT↔ S laneEvents)	1031
8.110.2.7	CFG_SPFI_isLaneFarEndLOSDetected(SPFI_LANE_EVENTS laneEvents)	1031
8.110.2.8	CFG_SPFI_isLaneFarEndStandbyDetected(SPFI_LANE_EVENTS laneEvents)	1032
8.110.2.9	CFG_SPFI_isLaneINIT1RxDetected(SPFI_LANE_EVENTS laneEvents)	1032
8.110.2.10	CFG_SPFI_isLaneINIT2RxDetected(SPFI_LANE_EVENTS laneEvents)	1032
8.110.2.11	CFG_SPFI_isLaneInitTimeoutDetected(SPFI_LANE_EVENTS laneEvents)	1033
8.111	Data Signalling Rate	1034
8.111.1	Detailed Description	1034
8.111.2	Function Documentation	1034
8.111.2.1	CFG_SPFI_destroySpFiBitRates(U64 *pBitRates)	1034

8.111.2.2 CFG_SPFI_getSpFiBitRate(STAR_DEVICE_ID deviceID, U8 port, _Out_ U64 *pBitRate)	1034
8.111.2.3 CFG_SPFI_getSpFiBitRates(STAR_DEVICE_ID deviceID, _Out_ U32 *pCount)	1035
8.111.2.4 CFG_SPFI_setSpFiBitRate(STAR_DEVICE_ID deviceID, U8 port, U64 bitRate)	1035
8.112Types	1037
8.112.1 Detailed Description	1038
8.112.2 Typedef Documentation	1038
8.112.2.1 SPFI_BROADCAST_TYPES_INFO	1038
8.112.2.2 SPFI_DEVICE_IDENTIFIER_INFO	1038
8.112.2.3 SPFI_DEVICE_INFO	1038
8.112.2.4 SPFI_LANE_CONTROL_SETTINGS	1039
8.112.2.5 SPFI_LANE_EVENTS	1039
8.112.2.6 SPFI_LANE_STATUS	1039
8.112.2.7 SPFI_LINK_CONTROL_SETTINGS	1039
8.112.2.8 SPFI_LINK_DEBUG_SETTINGS	1039
8.112.2.9 SPFI_LINK_EVENTS	1039
8.112.2.10SPFI_LINK_STATUS	1039
8.112.2.11SPFI_PORT_INFO	1040
8.112.2.12SPFI_QUALITY_OF_SERVICE	1040
8.112.2.13SPFI_ROUTING_TABLE_ENTRY_FLAGS	1040
8.112.2.14SPFI_VC_ERRORS	1040
8.112.2.15SPFI_VC_STATUS	1040
8.112.2.16U64	1040
8.112.3 Enumeration Type Documentation	1041
8.112.3.1 SPFI_LANE_STATE	1041
8.112.3.2 SPFI_LINK_ALIGNMENT_STATE	1041
8.112.3.3 SPFI_NODE_TYPE	1041
8.112.3.4 SPFI_PORT_TYPE	1042
8.113Defines	1043
8.113.1 Detailed Description	1043
8.114RMAP Packet Library	1044
8.114.1 Detailed Description	1045
8.114.2 Functionality Provided	1046
8.114.3 Supported Platforms	1046
8.114.4 Using The Library	1046
8.114.5 Java Library	1046
8.114.6 Byte Alignment	1047
8.114.7 Contact Information	1047
8.114.8 Data Structure Documentation	1047
8.114.8.1 struct RMAP_PACKET	1047

8.114.9 Macro Definition Documentation	1049
8.114.9.1 RMAP_COMMAND_BIT	1049
8.114.9.2 RMAP_GET_VERSION_EDIT	1049
8.114.9.3 RMAP_GET_VERSION_MAJOR	1050
8.114.9.4 RMAP_GET_VERSION_MINOR	1050
8.114.9.5 RMAP_GET_VERSION_PATCH	1050
8.114.9.6 RMAP_INCREMENT_ADDRESS_BIT	1051
8.114.9.7 RMAP_PROTOCOL_IDENTIFIER	1051
8.114.9.8 RMAP_REPLY_ADDRESS_LENGTH_BITS	1051
8.114.9.9 RMAP_REPLY_BIT	1051
8.114.9.10 RMAP_RESERVED_BIT	1051
8.114.9.11 RMAP_VERIFY_BEFORE_WRITE_BIT	1052
8.114.9.12 RMAP_WRITE_OPERATION_BIT	1052
8.114.9.13 RMAPPACKETLIBRARY_CC	1052
8.114.10 Enumeration Type Documentation	1052
8.114.10.1 RMAP_PACKET_TYPE	1052
8.114.10.2 RMAP_STATUS	1052
8.114.11 Function Documentation	1053
8.114.11.1 RMAP_CalculateCRC(void *pBuffer, unsigned long len)	1053
8.114.11.2 RMAP_CalculateCRCWithSeed(void *pBuffer, unsigned long len, U8 crc)	1053
8.114.11.3 RMAP_FreeBuffer(void *pBuffer)	1054
8.114.11.4 RMAP_GetVersion(void)	1054
8.114.11.5 RMAP_IsCRCValid(void *pBuffer, unsigned long len, U8 crc)	1055
8.115 Functions for Building RMAP Packets	1057
8.115.1 Detailed Description	1059
8.115.2 Function Documentation	1059
8.115.2.1 RMAP_BuildReadCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1059
8.115.2.2 RMAP_BuildReadModifyWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 *pData, U8 *pMask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1060
8.115.2.3 RMAP_BuildReadModifyWriteRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1061

8.115.2.4	RMAP_BuildReadModifyWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 *pData, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1062
8.115.2.5	RMAP_BuildReadRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1063
8.115.2.6	RMAP_BuildReadReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1064
8.115.2.7	RMAP_BuildWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1065
8.115.2.8	RMAP_BuildWriteRegisterPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1066
8.115.2.9	RMAP_BuildWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment)	1067
8.115.2.10	RMAP_CalculateReadCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)	1068
8.115.2.11	RMAP_CalculateReadModifyWriteCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)	1068
8.115.2.12	RMAP_CalculateReadModifyWriteReplyPacketLength(unsigned long initiatorAddressLength, U32 dataLength, char alignment)	1069
8.115.2.13	RMAP_CalculateReadReplyPacketLength(unsigned long initiatorAddressLength, U32 dataLength, char alignment)	1069
8.115.2.14	RMAP_CalculateWriteCommandPacketLength(unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)	1070
8.115.2.15	RMAP_CalculateWriteReplyPacketLength(unsigned long initiatorAddressLength, char alignment)	1070
8.115.2.16	RMAP_FillReadCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1071

8.115.2.17	RMAP_FillReadModifyWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 *pData, U8 *pMask, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1072
8.115.2.18	RMAP_FillReadModifyWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 *pData, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1073
8.115.2.19	RMAP_FillReadReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1073
8.115.2.20	RMAP_FillWriteCommandPacket(U8 *pTargetAddress, unsigned long targetAddressLength, U8 *pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 *pData, U32 dataLength, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1074
8.115.2.21	RMAP_FillWriteReplyPacket(U8 *pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long *pRawPacketLength, RMAP_PACKET *pPacketStruct, char alignment, U8 *pRawPacket, unsigned long rawPacketLength)	1075
8.115.2.22	RMAP_SetProtocolIdentifier(RMAP_PACKET *pPacketStruct, U8 protocolIdentifier)	1076
8.116	Functions for Interpreting RMAP Packets	1078
8.116.1	Detailed Description	1079
8.116.2	Function Documentation	1079
8.116.2.1	RMAP_CheckPacketValid(void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)	1079
8.116.2.2	RMAP_CheckPacketValidIgnoreProtocol(void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)	1080
8.116.2.3	RMAP_GetAddress(RMAP_PACKET *pPacketStruct, U8 *pExtendedAddress)	1080
8.116.2.4	RMAP_GetData(RMAP_PACKET *pPacketStruct, U32 *pDataLength)	1081
8.116.2.5	RMAP_GetDataCRC(RMAP_PACKET *pPacketStruct)	1081
8.116.2.6	RMAP_GetHeaderCRC(RMAP_PACKET *pPacketStruct)	1082
8.116.2.7	RMAP_GetIncrementAddress(RMAP_PACKET *pPacketStruct)	1082
8.116.2.8	RMAP_GetKey(RMAP_PACKET *pPacketStruct)	1082
8.116.2.9	RMAP_GetMask(RMAP_PACKET *pPacketStruct, U8 *pMaskLength)	1083
8.116.2.10	RMAP_GetPacketType(RMAP_PACKET *pPacketStruct)	1083
8.116.2.11	RMAP_GetPerformAcknowledgement(RMAP_PACKET *pPacketStruct)	1084
8.116.2.12	RMAP_GetProtocolIdentifier(RMAP_PACKET *pPacketStruct)	1084
8.116.2.13	RMAP_GetReadLength(RMAP_PACKET *pPacketStruct)	1085

8.116.2.14	RMAP_GetReplyAddress(RMAP_PACKET *pPacketStruct, unsigned long *pReplyAddressLength)	1085
8.116.2.15	RMAP_GetStatus(RMAP_PACKET *pPacketStruct)	1086
8.116.2.16	RMAP_GetTargetAddress(RMAP_PACKET *pPacketStruct, unsigned long *pTargetAddressLength)	1087
8.116.2.17	RMAP_GetTransactionID(RMAP_PACKET *pPacketStruct)	1087
8.116.2.18	RMAP_GetVerifyBeforeWrite(RMAP_PACKET *pPacketStruct)	1088
8.117	CCSDS/SPP/TF Packet Library	1089
8.117.1	Detailed Description	1091
8.117.2	Functionality Provided	1091
8.117.3	Using The Library	1091
8.117.4	Contact Information	1091
8.117.5	Data Structure Documentation	1092
8.117.5.1	struct CCSDS_SPP_ENCAPS_HEADER	1092
8.117.5.2	struct M_PDU_PACKET	1092
8.117.5.3	struct CCSDS_TF_M_PDU_HEADER	1093
8.117.5.4	struct CCSDS_TF_PACKET_PRIMARY_HEADER	1094
8.117.5.5	struct CCSDS_TF_INSERT_ZONE	1095
8.117.5.6	struct CCSDS_TF_M_PDU_DATA_FIELD	1095
8.117.5.7	struct CCSDS_TF_M_PDU_PACKET	1096
8.117.5.8	struct CCSDS_SPP_PACKET_PRIMARY_HEADER	1097
8.117.5.9	struct CCSDS_SPP_PACKET_DATA_FIELD	1098
8.117.5.10	struct CCSDS_SPP_PACKET	1099
8.117.6	Macro Definition Documentation	1100
8.117.6.1	M_PDU_PACKET_MAX_SIZE	1100
8.117.7	Enumeration Type Documentation	1100
8.117.7.1	CCSDS_SPP_PACKET_TYPE	1100
8.117.7.2	CCSDS_SPP_STATUS	1100
8.118	Functions for building SPP Packets	1103
8.118.1	Detailed Description	1103
8.118.2	Function Documentation	1103
8.118.2.1	CCSDS_SPP_CreatePacket(_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength, _Out_ CCSDS_SPP_STATUS *const pStatus)	1104
8.118.2.2	CCSDS_SPP_CreatePTPHeader(_In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte, _Out_ CCSDS_SPP_STATUS *const pStatus)	1104

8.118.2.3	CCSDS_SPP_FillPacket(_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, ↵ _Out_ U8 *const pRawPacket, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET ↵ ET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, ↵ _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength)	1105
8.118.2.4	CCSDS_SPP_FillPTPHeader(_Out_ U8 *const pHeader, _In_ const U8 target↵ LogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte) ↵	1106
8.118.2.5	CCSDS_SPP_FreeBuffer(_Inout_ void *pBuffer)	1107
8.119	Functions for retrieving (reconstructing) and interpreting SPP Packets	1108
8.119.1	Detailed Description	1109
8.119.2	Function Documentation	1109
8.119.2.1	CCSDS_SPP_CheckAPID(_In_ const CCSDS_SPP_PACKET_TYPE type, _In↵ _ const U16 APID)	1109
8.119.2.2	CCSDS_SPP_GetAPID(_In_ const CCSDS_SPP_PACKET *const pPacket↵ Struct, _Out_ U16 *const pAPID)	1110
8.119.2.3	CCSDS_SPP_GetDataFieldLength(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pDataFieldLength)	1110
8.119.2.4	CCSDS_SPP_GetDataFieldPointer(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 **pDataField)	1110
8.119.2.5	CCSDS_SPP_GetEncapsulationHeaderLength(_In_ const CCSDS_SPP_PAC↵ KET *const pPacketStruct, _Out_ U16 *const pEncapsulationHeaderLength)	1111
8.119.2.6	CCSDS_SPP_GetEncapsulationHeaderPointer(_In_ const CCSDS_SPP_PAC↵ KET *const pPacketStruct, _Out_ U8 **pEncapsulationHeader)	1111
8.119.2.7	CCSDS_SPP_GetPacketType(_In_ const CCSDS_SPP_PACKET *const p↵ PacketStruct, _Out_ U8 *const pPacketType)	1111
8.119.2.8	CCSDS_SPP_GetPacketVersionNumber(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pVersionNumber)	1112
8.119.2.9	CCSDS_SPP_GetPrimaryHeaderLengthBytes(void)	1112
8.119.2.10	CCSDS_SPP_GetPTPHeaderLengthBytes(void)	1112
8.119.2.11	CCSDS_SPP_GetPTPProtocolIdentifier(_In_ const CCSDS_SPP_PACKE↵ T *const pPacketStruct, _Out_ U8 *const pProtocolIdentifier)	1113
8.119.2.12	CCSDS_SPP_GetPTPReservedByte(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pReservedByte)	1114
8.119.2.13	CCSDS_SPP_GetPTPTargetLogicalAddress(_In_ const CCSDS_SPP_PACKE↵ ET *const pPacketStruct, _Out_ U8 *const pTargetLogicalAddress)	1114
8.119.2.14	CCSDS_SPP_GetPTPUserApplicationByte(_In_ const CCSDS_SPP_PACKE↵ T *const pPacketStruct, _Out_ U8 *const pUserApplicationByte)	1114
8.119.2.15	CCSDS_SPP_GetSequenceCount(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pSequenceCount)	1115
8.119.2.16	CCSDS_SPP_GetSequenceFlags(_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pSequenceFlags)	1115
8.119.2.17	CCSDS_SPP_GetTelecommandSecondaryHeaderLengthBytes(void)	1115
8.119.2.18	CCSDS_SPP_IsSecondaryHeaderPresent(_In_ const CCSDS_SPP_PACKE↵ T *const pPacketStruct, _Out_ U8 *const pIsSecondaryHeaderPresent)	1116

8.119.2.19	CCSDS_SPP_RetrievePacket(_In_ const U8 *const pReceivedRawPacket, _↵ In_ const U8 retrievePTPHeader, _In_ const U16 encapsulationHeaderLength, _Out_ CCSDS_SPP_STATUS *const pStatus)	1116
8.120	Functions for building TF Packets	1117
8.120.1	Detailed Description	1117
8.120.2	Function Documentation	1117
8.120.2.1	CCSDS_TF_CreateIdlePacket(const U16 packetSize, U32 *const pRawPacket↵ Length, CCSDS_SPP_STATUS *const pStatus)	1117
8.120.2.2	CCSDS_TF_CreatePackets(_In_ U8 *const *const pCCSDS_SPP_Packets, ↵ _In_ U32 *const pCCSDS_SPP_PacketLengths, _In_ U16 const nrOfCCSD↵ S_SPP_Packets, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const U8 versionNumber, _In_ const U8 spaceCraftId, _In_ const U8 virtualChannelId, _Inout_ U32 *const pVirtual↵ ChannelFrameCount, _In_ const U8 replayFlag, _In_ const U8 virtualChannel↵ FrameCountUsageFlag, _Inout_ U8 const virtualChannelFrameCountCycle, ↵ _In_ const U8 *const pFrameHeaderErrorControl, _In_opt_ U8 *pInsertZoneData, _In_ U8 insertZoneDataLength, _Out_ U16 *pNumberOfTFPacketsCreated, ↵ _Out_ U16 *pTFPacketLengthBytes, _In_opt_ const U8 *const pOperational↵ ControlField, _In_opt_ const U8 *const pFrameErrorControlField, CCSDS_SP↵ P_STATUS *pStatus)	1118
8.121	Functions for retrieving (reconstructing) and interpreting TF Packets	1120
8.121.1	Detailed Description	1121
8.121.2	Function Documentation	1121
8.121.2.1	CCSDS_TF_Get_M_PDU_PacketMaxSize(void)	1121
8.121.2.2	CCSDS_TF_M_PDU_Get_M_PDU_DataField(_In_ const CCSDS_TF_M_PD↵ U_PACKET *const pPacketStruct, _Out_ const U8 *pM_PDU_DataField)	1121
8.121.2.3	CCSDS_TF_M_PDU_Get_M_PDU_FirstHeaderPointer(_In_ const CCSDS_T↵ F_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pM_PDU_First↵ HeaderPointer)	1122
8.121.2.4	CCSDS_TF_M_PDU_GetEncapsulationHeader(_In_ const CCSDS_TF_M_P↵ DU_PACKET *const pPacketStruct, _Out_ const U8 *pEncapsulationHeader, ↵ _Out_ U16 *const pEncapsulationHeaderLength)	1122
8.121.2.5	CCSDS_TF_M_PDU_GetFrameErrorControlField(_In_ const CCSDS_TF_M↵ PDU_PACKET *const pPacketStruct, _Out_ U16 *const pFrameErrorControlField) 123	
8.121.2.6	CCSDS_TF_M_PDU_GetFrameHeaderErrorControl(_In_ const CCSDS_TF↵ M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pFrameHeader↵ ErrorControl)	1124
8.121.2.7	CCSDS_TF_M_PDU_GetInsertZoneData(_In_ const CCSDS_TF_M_PDU_P↵ ACKET *const pPacketStruct, _Out_ U8 *pInsertZoneData, _Out_ U16 *const pInsertZoneDataLength)	1124
8.121.2.8	CCSDS_TF_M_PDU_GetOperationalControlField(_In_ const CCSDS_TF_M↵ PDU_PACKET *const pPacketStruct, _Out_ U32 *const pOperationalControlField) 125	
8.121.2.9	CCSDS_TF_M_PDU_GetReplayFlag(_In_ const CCSDS_TF_M_PDU_PACK↵ ET *const pPacketStruct, _Out_ U8 *const pReplayFlag)	1126
8.121.2.10	CCSDS_TF_M_PDU_GetSpacecraftID(_In_ const CCSDS_TF_M_PDU_PAC↵ KET *const pPacketStruct, _Out_ U8 *const pSpacecraftId)	1126
8.121.2.11	CCSDS_TF_M_PDU_GetVCFrameCountCycle(_In_ const CCSDS_TF_M_PD↵ U_PACKET *const pPacketStruct, _Out_ U8 *const pVCFrameCountCycle) . .	1126

8.121.2.12	CCSDS_TF_M_PDU_GetVCFrameCountUsageFlag(_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVCFrameCountUsageFlag)	1127
8.121.2.13	CCSDS_TF_M_PDU_GetVersionNumber(_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVersionNumber)	1127
8.121.2.14	CCSDS_TF_M_PDU_GetVirtualChannelFrameCount(_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U32 *const pVirtualChannelFrameCount)	1127
8.121.2.15	CCSDS_TF_M_PDU_GetVirtualChannelID(_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVirtualChannelId)	1128
8.121.2.16	CCSDS_TF_RetrievePacket(_In_ const U8 *const pRawPacket, _In_ const U16 encapsulationHeaderLength, _In_ const U8 frameHeaderErrorControlPresent, _In_ const U8 insertZoneDataLength, _In_ const U8 operationalControlFieldPresent, _In_ const U8 frameErrorControlFieldPresent, _Out_ CCSDS_SPP_STATUS *pStatus)	1128
8.122	STAR-Dundee Types	1130
8.122.1	Detailed Description	1130
9	File Documentation	1131
9.1	ccsds_spp_packet_library.h File Reference	1131
9.1.1	Detailed Description	1135
9.2	ccsds_spp_pus_services.h File Reference	1136
9.2.1	Detailed Description	1136
9.2.2	Function Documentation	1136
9.2.2.1	CCSDS_SPP_PUS_CreateST02DistributeOnOffDeviceCommand(_Out_opt CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ const U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const U16 numberOfDeviceAddresses, _In_ const U16 *const pDeviceAddresses, _Out_ CCSDS_SPP_STATUS *const pStatus, _Out_ U32 *const pRawPacketLength)	1136
9.2.2.2	CCSDS_SPP_PUS_CreateST02DistributeRegisterDumpCommand(_Out_opt CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ const U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const U16 numberOfRegisterAddresses, _In_ const U16 *const pRegisterAddresses, _Out_ CCSDS_SPP_STATUS *const pStatus, _Out_ U32 *const pRawPacketLength)	1137
9.3	cfg_api_brick_mk2.h File Reference	1138
9.3.1	Detailed Description	1139
9.4	cfg_api_brick_mk2_types.h File Reference	1139
9.4.1	Detailed Description	1140
9.4.2	Data Structure Documentation	1140
9.4.2.1	struct STAR_CFG_BRICK_MK2_ERRORS	1140
9.4.3	Macro Definition Documentation	1141
9.4.3.1	STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS	1141

9.5	cfg_api_brick_mk3.h File Reference	1142
9.5.1	Detailed Description	1143
9.6	cfg_api_brick_mk3_types.h File Reference	1143
9.6.1	Detailed Description	1143
9.7	cfg_api_gbe_brick_mk1.h File Reference	1143
9.7.1	Detailed Description	1144
9.8	cfg_api_generic.h File Reference	1144
9.8.1	Detailed Description	1152
9.8.2	Function Documentation	1152
9.8.2.1	CFG_getMeasuredLinkSpeed(STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U16 *pLinkSpeed)	1152
9.8.2.2	CFG_getTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ U32 *pSignallingRate)	1153
9.8.2.3	CFG_setTransmitSignallingRate(STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate)	1153
9.9	cfg_api_generic_common.h File Reference	1154
9.9.1	Detailed Description	1154
9.9.2	Data Structure Documentation	1154
9.9.2.1	struct STAR_DEVICE_INFO	1154
9.10	cfg_api_mk2.h File Reference	1155
9.10.1	Detailed Description	1158
9.10.2	Macro Definition Documentation	1158
9.10.2.1	CFG_API_MK2_MODULE_NAME	1158
9.11	cfg_api_mk2_types.h File Reference	1158
9.11.1	Detailed Description	1159
9.12	cfg_api_pci_mk2.h File Reference	1159
9.12.1	Detailed Description	1160
9.12.2	Macro Definition Documentation	1160
9.12.2.1	CFG_API_PCI_MK2_MODULE_NAME	1160
9.13	cfg_api_pci_mk2_types.h File Reference	1160
9.13.1	Detailed Description	1160
9.14	cfg_api_pcie_mk2.h File Reference	1161
9.14.1	Detailed Description	1162
9.15	cfg_api_pxi.h File Reference	1162
9.15.1	Detailed Description	1164
9.16	cfg_api_remote.h File Reference	1164
9.16.1	Detailed Description	1165
9.17	cfg_api_router.h File Reference	1165
9.17.1	Detailed Description	1167
9.18	cfg_api_router_mk2s.h File Reference	1167
9.18.1	Detailed Description	1167

9.19	cfg_api_router_types.h File Reference	1168
9.19.1	Detailed Description	1169
9.20	cfg_api_spfi.h File Reference	1169
9.20.1	Detailed Description	1178
9.21	cfg_api_spfi_types.h File Reference	1178
9.21.1	Detailed Description	1179
9.22	common.h File Reference	1180
9.22.1	Detailed Description	1180
9.22.2	Macro Definition Documentation	1180
9.22.2.1	STAR_API_CC	1180
9.22.2.2	STAR_API_MODULE_NAME	1181
9.23	general.h File Reference	1181
9.23.1	Detailed Description	1184
9.24	packet_subsystem.h File Reference	1184
9.24.1	Detailed Description	1186
9.24.2	Macro Definition Documentation	1187
9.24.2.1	PKT_SUBSYS_MODULE_NAME	1187
9.25	rmap_packet_library.h File Reference	1187
9.25.1	Detailed Description	1191
9.26	rmap_target_pxi_if.h File Reference	1192
9.26.1	Detailed Description	1194
9.27	rmap_target_types.h File Reference	1194
9.27.1	Detailed Description	1195
9.28	star-api.h File Reference	1195
9.28.1	Detailed Description	1196
9.29	star-dundee_annotations.h File Reference	1196
9.29.1	Detailed Description	1196
9.30	star-dundee_types.h File Reference	1196
9.30.1	Detailed Description	1197
9.31	stream_item.h File Reference	1197
9.31.1	Detailed Description	1200
9.32	stream_item_types.h File Reference	1200
9.32.1	Detailed Description	1201
9.33	transfer.h File Reference	1202
9.33.1	Detailed Description	1204
9.34	transfer_types.h File Reference	1204
9.34.1	Detailed Description	1205
9.35	triggering_brick_mk3.h File Reference	1205
9.35.1	Detailed Description	1209
9.36	triggering_generic.h File Reference	1209

9.36.1 Detailed Description	1215
9.37 triggering_pcie.h File Reference	1216
9.37.1 Detailed Description	1218
9.38 triggering_pcie_mk2.h File Reference	1218
9.38.1 Detailed Description	1222
9.39 triggering_pxi_if.h File Reference	1223
9.39.1 Detailed Description	1226
9.40 triggering_pxi_ro.h File Reference	1227
9.40.1 Detailed Description	1229
9.41 triggering_types.h File Reference	1230
9.41.1 Detailed Description	1232
9.41.2 Data Structure Documentation	1233
9.41.2.1 struct TRIGGER_MATRIX	1233
9.41.3 Typedef Documentation	1233
9.41.3.1 PORT_ACTION_MASK	1233
9.41.3.2 PORT_EVENT_MASK	1234
9.41.4 Enumeration Type Documentation	1234
9.41.4.1 PORT_ACTION	1234
9.41.4.2 PORT_EVENT	1235
9.41.4.3 TRIGGER_DEVICE	1235
9.41.4.4 TRIGGER_TYPE	1235
9.42 types.h File Reference	1236
9.42.1 Detailed Description	1239
9.42.2 Macro Definition Documentation	1239
9.42.2.1 CFG_INFO_ID_TYPE	1239
9.42.2.2 STAR_DEVICE_STAR_ULTRA_PCIE	1239
9.43 version.h File Reference	1239
9.43.1 Detailed Description	1240
9.43.2 Data Structure Documentation	1240
9.43.2.1 struct STAR_VERSION_INFO	1240
9.43.3 Macro Definition Documentation	1241
9.43.3.1 POPULATE_VERSION	1241
9.43.3.2 PRINT_FULL_VERSION_INFO	1241
9.43.3.3 PRINT_VERSION	1242
9.43.3.4 PRINT_VERSION_EDIT	1242
9.43.3.5 PRINT_VERSION_NUM	1242
9.43.3.6 PRINT_VERSION_PATCH	1242
9.43.3.7 VERSION_EDIT	1243
9.43.3.8 VERSION_MAJOR	1243
9.43.3.9 VERSION_MINOR	1243

9.43.3.10 VERSION_PATCH	1243
10 Example Documentation	1245
10.1 advanced_address_example.c	1245
10.2 advanced_continuous_receive_example.c	1247
10.3 advanced_multiple_packet_example.c	1248
10.4 advanced_receive_example.c	1250
10.5 advanced_send_and_receive_example.c	1251
10.6 advanced_send_example.c	1253
10.7 advanced_two_way_example.c	1254
10.8 all_version_example.c	1257
10.9 api_test_app.c	1257
10.10api_version_example.c	1258
10.11application_name_example.c	1258
10.12brickMk2_configuration_example.c	1259
10.13broadcast_message_example.c	1262
10.14ccsds_spp_tf_examples.c	1264
10.15channel_callback_example.c	1268
10.16check_packet_example.c	1269
10.17crc_example.c	1273
10.18data_signalling_rate_example.c	1273
10.19device_identifier_example.c	1276
10.20device_info_example.c	1277
10.21driver_list_example.c	1279
10.22ex_receive_example.c	1280
10.23firmware_version_example.c	1282
10.24lane_info_example.c	1283
10.25link_events.c	1289
10.26link_info_example.c	1295
10.27mk2_configuration_example.c	1304
10.28network_management_example.c	1306
10.29packet_subsystem_example.c	1310
10.30pciMk2_configuration_example.c	1315
10.31port_control_status_example.c	1316
10.32port_info_example.c	1318
10.33read_command_packet_example.c	1321
10.34read_reply_packet_example.c	1322
10.35rmap_target_example.c	1323
10.36rmw_command_packet_example.c	1328
10.37rmw_reply_packet_example.c	1329

10.38router_configuration_example.c	1330
10.39routerMk2S_configuration_example.c	1333
10.40routing_table_example.c	1334
10.41simple_receive_example.c	1336
10.42simple_send_and_receive_example.c	1337
10.43simple_send_example.c	1338
10.44simple_two_way_example.c	1339
10.45spfi_routing_table_example.c	1340
10.46star-test.c	1343
10.47star_performance_tester.c	1377
10.48time-code_example.c	1408
10.49time-code_test.c	1409
10.50timestamp_test.c	1416
10.51transfer_callback_example.c	1426
10.52transfer_operation_list_send_example.c	1427
10.53transfer_operation_list_send_receive_example.c	1429
10.54transmit_errors_example.c	1431
10.55triggering_example.c	1433
10.56triggering_generic_example.c	1457
10.57ui.c	1470
10.58user_registers_example.c	1473
10.59utilities.c	1474
10.60utility.c	1480
10.61vendor_specific_example.c	1483
10.62version_example.c	1486
10.63virtual_channel_info_example.c	1487
10.64write_command_packet_example.c	1500
10.65write_reply_packet_example.c	1501
Index	1503

Chapter 1

Introduction and Overview

Note that the HTML version of this documentation is available after installing STAR-System, and the latest version can be viewed online by registered users at <https://www.star-dundee.com/help/star-system/index.xhtml>. A PDF version of this documentation is also available for download from the STAR-Dundee website, again for registered users. See the [Support and Contact Information](#) section for more information on registering your product.

1.1 Introduction

STAR-System is the STAR-Dundee software suite provided with all new and future STAR-Dundee interface and router devices. It also provides the drivers used by other STAR-System devices. A full list of supported devices is provided in the [STAR-System Devices](#) section. If you are using a device which is not simply an interface or router, then it will include a separate user manual, focussing on its unique capabilities. Some of the features of STAR-System are still available for use with these devices and further information is included in the [Other STAR-System Devices](#) section.

The aims of STAR-System include:

- To provide high bandwidth and low latency packet transmission and reception.
- To provide additional test and debug capabilities than were available in previous STAR-Dundee software packages.
- To simplify migration between devices, operating systems and programming languages.

STAR-System was designed with the intention of not only allowing you to transmit and receive packets at high data rates, but also to provide you with the tools you're likely to need when testing or debugging a new device, for example. These include transmitting packets terminated with an EEP or with no end of packet marker, transmitting time-codes at a particular point in a packet, receiving packets and time-codes in the order in which they were carried across the SpaceWire link, etc.

STAR-System provides a consistent interface across multiple operating systems (see [Supported Operating Systems](#)). It provides a consistent interface for accessing numerous STAR-Dundee device types (see [Supported Devices](#)), with support for each new device added as that device is completed. It also supports multiple programming and scripting languages, with more to be added in the future, in a manner that is as consistent as is possible across these languages (see [Supported Languages](#)).

1.2 Supported Operating Systems

STAR-System is provided for Windows and Linux operating systems. The following versions of Windows are supported:

- Windows 10 (32- and 64-bit)
- Windows 11 (64-bit)

The Windows versions of STAR-System also support compiling and running applications under Cygwin.

Note that as Microsoft has dropped support for Windows XP and Windows Server 2003, STAR-System dropped support for Windows XP and Windows Server 2003 from version 4.02 onwards. Official support for all versions of Windows prior to Windows 10 was dropped in STAR-System v5.01 due to the end of Microsoft support for these platforms.

Please contact us if you require an older version of STAR-System for an older version of Windows.

2.6 - 6.17.7 Linux kernels for i386 and x86-64 processors are supported. An ARM release is also available for ARMv6, ARMv7 and ARMv8 targets including the Raspberry Pi and BeagleBone. For further information, or support for other ARM devices, please contact us.

QNX, VxWorks and RTEMS versions of STAR-System are available separately.

If you require support for other operating systems or other Linux kernels or architectures, please contact us using the [contact information](#) below.

1.3 Installation Instructions

1.3.1 Installation Instructions for Windows

To install STAR-System on Windows, double-click the `.msi` file for the Windows architecture you are installing on. For example, to install on a 64-bit version of Windows, double-click `star-system_win_x86-64_v6.01.msi`. To install on a 32-bit version of Windows, double-click `star-system_win_x86-32_v6.01.msi`.

After double-clicking the appropriate `.msi` file, the STAR-System installer should begin. Follow the instructions to install STAR-System, selecting the features that you wish to install.

If you encounter issues installing STAR-System on Windows, please make sure that you have the latest root certificates installed on your machine. These are available from Windows Update. It is recommended that you have the latest service pack and updates installed, particularly if using an older version of Windows (see below).

To remove Ethernet support, deselect the Ethernet driver from the "Drivers" category. This will invalidate the Ethernet Device Connector which can also be removed separately in the "GUI Applications" category.

1.3.1.1 Windows XP USB Issue

There is a bug in Windows XP which can cause PCs to bug check (blue screen) when transmitting and receiving over USB at the high rates possible using the STAR-System USB Driver. This will cause your PC to crash and reboot. A fix is available from Microsoft and it is recommended that this fix is installed (along with all other important updates) when running on Windows XP.

The fix can be obtained from: <http://support.microsoft.com/kb/969238>.

1.3.1.2 Windows Vista USB Issue

There is a bug in Windows Vista prior to Service Pack 2 which can cause USB traffic to be transferred out of sequence when transmitting and receiving over USB at the high rates possible using the STAR-System USB Driver. This can cause SpaceWire packets to be corrupted or received out of order. A fix for this issue was included in Service Pack 2 and it is highly recommended that the latest service pack is installed (along with all other important updates) when running on Windows Vista.

1.3.1.3 Universal CRT

The GUI applications require components from the Universal CRT which is built into Windows 10 onwards and older versions (Vista onwards) that have had updates applied. If you try to run the GUI applications on an older version of Windows that has not been updated then you may see an error stating that "api-ms-win-crt-runtime-l1-1-0.dll" is missing. To resolve this error you must either install the latest Windows updates or the [KB2999226](#) package.

1.3.1.4 Windows 10 Kernel DMA Protection Issue

In Windows 10 version 1803, there was a feature added called Kernel DMA Protection that is used to protect P↔Cs against DMA attacks using hot pluggable devices connected to PCIe interfaces. However, there is a known implementation issue that can cause kernel crashes or undefined behaviour.

This issue can affect the [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) if Kernel DMA Protection is enabled and they are connected via an external chassis such as a Thunderbolt chassis or a remote PXI/PXIe system.

For further information, and workarounds, see the following pages:

- [Kernel DMA Protection](#)
- [Stop code DRIVER_VERIFIER_DMA_VIOLATION when Kernel DMA Protection is enabled](#)

1.3.2 Installation Instructions for Linux

Prior to installing STAR-System on Linux, you should ensure that the gcc compiler, make and the kernel headers are installed on the target machine. These are required to build the drivers for your system.

On some systems, e.g. RedHat, it may also be necessary to install the "linux-kernel-devel" and/or "build-essential" packages in order to build the driver modules. On some versions of CentOS it may be necessary to install "kernel-devel-\$(uname -r)" and/or "kernel-headers-\$(uname -r)". On the Raspberry Pi this may be "raspberrypi-kernel-headers-\$(uname -r)".

To use the GUI applications you will also need to install Qt. Further information on installing the necessary Qt packages for Linux is available in the [Graphical Applications](#) section. The GUI applications are not currently available in the ARM release.

To install STAR-System on Linux, first extract the .tgz archive, then double-click the installation wizard for the Linux architecture you are installing on, or run the installation from the command line. For example, to install on a 64-bit x86 Linux kernel, run `star-system_linux_x86-64_v6.01`. To install on a 32-bit x86 kernel, run `star-system_linux_x86-32_v6.01`. To install STAR-System on an ARM version of Linux, first extract `star-system_linux_arm_v6.01.tgz` then refer to the [Command Line Installer](#) section.

After running the appropriate install script, the STAR-System installer GUI should begin. Follow the instructions to install STAR-System, selecting the features that you wish to install.

STAR-System requires root access during the installation process to install the drivers and associated files. Although on some Linux distributions the installer will ask for the appropriate password, note that on some distributions the installer will instead fail if the Linux distribution does not ask for the appropriate permissions. On these distributions you will need to execute the script as root. This can be done at the command line using `su` or `sudo`, depending on your distribution. For example, on Fedora:

```
$ su -  
$ ./star-system_linux_x86-64_v6.01
```

On Ubuntu, the following command can be used:

```
$ sudo ./star-system_linux_x86-64_v6.01
```

The installer builds drivers specifically for the current kernel. If you upgrade your kernel, or wish to use STAR-System on another kernel, please run the following build script to rebuild the drivers:

```
$ ./build_kernel_modules.sh
```

If the kernel is newer than officially supported, then the `--try-latest-kernel` argument can be used to attempt to build the kernel modules:

```
$ ./build_kernel_modules.sh --try-latest-kernel
```

Note that as with the installer you will need to be root or use `sudo` to run this script.

To remove Ethernet support, deselect the Ethernet driver from the "Drivers" category.

1.3.2.1 Command Line Installer

We also provide a command line installation script, `star-system_install_v6.01.sh` that includes a variety of options to allow feature selection. All features can be installed with the following command:

```
$ ./star-system_install_v6.01.sh --all
```

To attempt installation on a system with a newer kernel than officially supported you can run the installer with the `--try-latest-kernel` argument:

```
$ ./star-system_install_v6.01.sh --all --try-latest-kernel
```

The help argument can be used to display the available options:

```
$ ./star-system_install_v6.01.sh --help
```

Note again that as with the installer above, you will need to be root or use `sudo` to run this script.

On systems where PCI support is not available (e.g. some Raspberry Pi systems) or if you only wish to install the USB driver, for example, you may run the following command to install all features apart from the PCI drivers:

```
$ ./star-system_install_v6.01.sh --all --no-pci-driver --no-ultra-pcie-driver
```

Similarly, to remove Ethernet support, please run:

```
$ ./star-system_install_v6.01.sh --all --no-ethernet-driver
```

To install only the files required for development you can use the following command (this will install the header files and libraries but not the drivers or Device Configuration Service):

```
$ ./star-system_install_v6.01.sh --development
```


1.3.3 Installation Instructions for Linux with Secure Boot Enabled

Some Linux kernels require that kernel modules, such as the STAR-System drivers, be cryptographically signed by a trusted key in order to be loaded. This can often be the case for kernels running on UEFI systems with Secure Boot enabled.

If an error message such as:

```
Modprobe: ERROR: could not insert '<driver name>': Required key not available
```

is encountered during installation, this may indicate that Secure Boot is enabled and the STAR-System drivers should either be signed or Secure Boot be disabled.

The following steps describe the procedure required to sign and install the driver modules.

Firstly, create a directory for the signing keys:

```
$ mkdir ~/.ssl
```

```
$ cd ~/.ssl
```

Next, create the required keys:

```
$ openssl req -new -x509 -newkey rsa:2048 -keyout star.priv -outform DER  
-out star.der -nodes -days 36500 -subj "/CN=STAR-Dundee/"
```

NOTE : it is important to keep your private key secure, as it could be used to compromise any system which has your public key enrolled.

The next step is to sign the STAR-Dundee USB, PCI and Ultra PCIe drivers :

```
$ sudo /usr/src/kernels/$(uname -r)/scripts/sign-file sha256 star.priv  
star.der $(modinfo -n star_spw_usb)
```

```
$ sudo /usr/src/kernels/$(uname -r)/scripts/sign-file sha256 star.priv  
star.der $(modinfo -n star_spw_pci)
```

```
$ sudo /usr/src/kernels/$(uname -r)/scripts/sign-file sha256 star.priv  
star.der $(modinfo -n star_ultra_pcie)
```

Finally, the key must be "enrolled" to allow it to be recognised by the Secure Boot system:

```
$ sudo mokutil --import star.der
```

You will be asked to provide a password, which will be required after rebooting.

Reboot your PC, and select "Enroll MOK" from the displayed menu. The new key should be shown as a numeric entry, and the details can be checked by selecting the "View" option. Once the identity of the key has been confirmed, select "Continue" from the menu, then "Yes" when asked to confirm enrolling the key.

You will then be asked for the password you entered when enrolling the key, either as the full password, or specific characters from it.

The system will reboot again, and the signed drivers should now be available. There is no need to reinstall STAR-System or rebuild the drivers.

Note that key creation and enrolling only needs to be done once, and the same keys can be used for future updates or installations of STAR-System.

Alternatively, if it is possible to disable Secure Boot via the UEFI or BIOS settings, this should allow the drivers to be loaded without being signed.

If this is not possible, another method is to use the `mokutil` tool to disable Secure Boot. You may have to install this tool using your Linux distribution's package manager. To disable Secure Boot, use the command:

```
$ sudo mokutil --disable-validation
```

You will be asked to provide a password, which will be required after rebooting.

On rebooting a prompt will be displayed allowing Secure Boot to be disabled. You will either be asked for the password you entered earlier, or for specific characters from the password.

1.3.4 Installation Instructions for QNX

To install STAR-System on QNX, first extract the .tgz to the directory you wish to install the software in, then double-click the install script, or run the install script from the command line. For example, run:

```
$ ./install_star-system.sh
```

To remove Ethernet support, please run:

```
$ ./install_star-system.sh --no-ethernet-driver
```

Reboot the system and STAR-System should now be installed and ready to use.

1.3.5 Installation Instructions for VxWorks

Installation instructions for VxWorks are available in the [VxWorks Installation and Getting Started Guide](#).

1.3.6 Installation Instructions for RTEMS

Installation instructions for RTEMS are available in the [RTEMS Installation and Getting Started Guide](#).

1.3.7 Compatibility Issues with MXIe Systems

When using the [SpaceWire PXI Interface and PXI Interface Mk2](#) or [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) in a remote PXIe chassis using MXIe, there can occasionally be compatibility issues depending on the host PC.

If you experience issues when using a device remotely using MXIe, see the following page from NI for further information and troubleshooting steps to mitigate against MXIe / PC incompatibility:

[Mitigating MXI-Express PC Incompatibility](#)

1.4 Getting Started

There are a few things you should be aware of before you start using STAR-System and these are described below. After reading the section below for the operating system you're using, take a look at the [Graphical Applications](#) or some of the example programs such as [STAR-System Test](#) or [STAR Performance Tester](#) to start using the software, or read about the [STAR-System APIs](#) you can use to write your own applications.

1.4.1 Getting Started on Windows

After installation on Windows is complete, STAR-System should be ready to use. Note that the installer installs a process, `star-conf_service.exe` which is automatically executed when the installer completes, and will start when a user logs in to Windows. This process provides the Device Configuration Service for configuring devices, and should not consume any resources other than when STAR-System is in use. You will receive an error if you attempt to use some STAR-System features without this process running. See the [Device Configuration Service](#) section for further information.

To use the STAR-System C and C++ APIs on Windows, `.lib` and `.dll` files are included for building applications with a compiler such as Microsoft Visual C++ (MSVC). For 32-bit applications, the `.lib` files are installed in the `lib/x86-32` directory. For 64-bit applications, the `.lib` files are installed in the `lib/x86-64` directory. The `.dll` files are installed in the `bin` directory as well as the Windows system directories.

It is also possible to use the STAR-System C and C++ APIs on Windows using Cygwin, as described in the following section.

1.4.1.1 Getting Started on Cygwin

The example STAR-System applications provided as source code include Makefiles which allow these applications to be built under Cygwin. The header files and libraries also allow you to build your own STAR-System applications under Cygwin. For 32-bit applications you can use the normal STAR-System Windows `.lib` files. For 64-bit applications, some additional Cygwin-specific `.a` files are provided. Instructing your linker to look in the correct directory for the libraries (e.g. `lib/x86-64`) will result in the correct libraries being picked up. No further action is required. The example Makefiles demonstrate this.

1.4.2 Getting Started on Linux

After installation on Linux is complete, STAR-System should be ready to use. Note that the installer installs a process, `star_conf_service` which is automatically executed when the installer completes, and will run whenever your PC is started. This process provides the Device Configuration Service for configuring devices, and should not consume any resources other than when STAR-System is in use. You will receive an error if you attempt to use STAR-System without this process running. The start-up script which runs the Device Configuration Service is named `star-system.service` and is installed in the `/etc/systemd/system/` directory. On some Linux systems it may be named `star_device_config` and installed in the `/etc/rc.d/init.d/` or `/etc/init.d/` directory. See the [Device Configuration Service](#) section for further information.

1.4.3 Getting Started on QNX

After installation on QNX is complete, and the system rebooted, STAR-System should be ready to use.

On start-up, if a STAR-Dundee device is detected, the driver and the Device Configuration Service are started automatically. The driver and the Device Configuration Service are installed to `/sbin`.

For information, to start the driver manually, run `./star_pci`. To stop the driver, press CTRL + C or kill the process using `/bin/kill` (see the instructions for the Device Configuration Service below).

The Device Configuration Service must be running in order to use the device configuration functions. To start the service manually, run `./star_conf_service`. To stop the service send it the sigint signal:

```
/bin/kill -SIGINT <process_number>
```

To obtain the process number, execute `ps` and look for the `star_conf_service` process. See the [Device Configuration Service](#) section for further information.

1.4.4 Getting Started on VxWorks

Getting started instructions for VxWorks are available in the [VxWorks Installation and Getting Started Guide](#).

1.4.5 Getting Started on RTEMS

Getting started instructions for RTEMS are available in the [RTEMS Installation and Getting Started Guide](#).

1.5 Migrating from Previous STAR-Dundee APIs

STAR-Dundee's older drivers and APIs for our PCI and USB devices were developed and refined over a number of years. As a result, these systems are very reliable and provide very high performance.

However, these systems also have their limitations. The old PCI Driver in particular has been in existence for quite some time, and as a result there are issues on newer operating system releases. For example, it does not support 64-bit versions of Windows and Linux. A further limitation of these older systems is that a different API is used to access the PCI devices than is used to access the USB devices. As a result of this, it's quite awkward to write software which will work with both types of devices. Writing an application using STAR-System's APIs means that not only will it work with the currently supported STAR-Dundee devices, it will also work with future STAR-Dundee devices.

It is possible to migrate code from these older APIs to STAR-System and STAR-Dundee can provide support on this. Note also that the original SpaceWire Brick can be upgraded to the SpaceWire Brick Mk2, the SpaceWire PCI-2 can be upgraded to the SpaceWire PCI Mk2, the SpaceWire cPCI can be upgraded to the SpaceWire cPCI Mk2, and the SpaceWire Router Mk2 can be upgraded to the SpaceWire Router Mk2S, giving STAR-System support. Please contact us using the [contact information](#) below if you would like to upgrade a device.

Below is a list of some of the advantages of STAR-System over previous STAR-Dundee systems:

- A consistent API for new and future devices.
- An API which is consistent across all supported operating systems.
- Support for additional operating systems such as QNX and RTEMS (available separately).
- Support for 64-bit versions of Windows and Linux.
- Support for running 32-bit applications on 64-bit operating systems.
- Support for newer Linux kernels.
- Support for multiple applications accessing the same device at one time.
- APIs developed specifically for test and development, not just transmitting and receiving packets.
- Improved API documentation and examples.
- Improved performance with smaller packets.
- Direct support for multiple programming languages, including C, C++ and Python.
- Separately available LabVIEW wrapper.

If there are any additional features you would like added, please contact us using the [contact information](#) below.

1.6 Supported Devices

The SpaceWire PCI Mk2 and SpaceWire cPCI Mk2 were the first of the new STAR-Dundee devices to be supported by STAR-System. STAR-System was then updated to support the STAR-Dundee SpaceWire PCIe device. Version 2.0 of STAR-System added support for USB devices, specifically the STAR-Dundee SpaceWire Brick Mk2 and the SpaceWire Router Mk2S. Version 3.0 added support for the SpaceWire Brick Mk3 and SpaceWire PXI devices. Version 3.4 added support for the first SpaceFibre device, the STAR Fire, while version 3.5 added support for the SpaceWire PXI 12 Port Router device. Version 4.00 added support for the STAR Fire Mk3 and the SpaceWire PXI Mk2 devices, version 4.02 added support for the STAR-Ultra PCIe Interface and version 4.03 added support for the SpaceWire Brick Mk4. Most recently, version 5.00 added support for the first device with an Ethernet interface: the SpaceWire GbE Brick. Version 5.01 added support for the SpaceWire PCIe Mk2. Version 6.00 Beta 1 added support for the STAR-Ultra PCIe Single-Lane Router.

List of directly supported devices:

- SpaceWire Brick Mk2
- SpaceWire Brick Mk3
- SpaceWire Brick Mk4
- SpaceWire cPCI Mk2
- SpaceWire GbE Brick
- SpaceWire PCI Mk2
- SpaceWire PCIe
- SpaceWire PCIe Mk2
- SpaceWire Physical Layer Tester
- SpaceWire PXI Devices
- SpaceWire PXI Mk2 Devices
- SpaceWire Router Mk2S
- STAR Fire
- STAR Fire Mk3
- STAR-Ultra PCIe Interface
- STAR-Ultra PCIe Single-Lane Router

STAR-System also provides the drivers for other devices which do not offer standard transmit and receive functions. These include:

- SpaceWire Conformance Tester Mk2
- SpaceWire EGSE
- SpaceWire EGSE Mk2
- SpaceWire Link Analyser Mk3
- SpaceWire Recorder

- SpaceWire Recorder Mk2

Although STAR-System cannot currently be used with any other devices directly, it is still possible to configure other devices over SpaceWire using STAR-System's Device Configuration APIs and the Device Configuration application. The configuration that is possible includes getting and setting routing tables, starting and stopping links, and getting error information. In addition to the devices supported directly by STAR-System, the Device Configuration APIs and Device Configuration application currently support configuring the following devices over a SpaceWire network:

- SpaceWire-USB Brick
- SpaceWire Router-USB
- SpaceWire Router Mk2
- SpaceWire SpW-10X Router (AT7910E)

Other devices developed using STAR-Dundee's Router IP core (see <https://www.star-dundee.com/products/spacewire-ip-cores>) can also be configured over a SpaceWire network.

If you require support for another device, please contact us using the [contact information](#) below.

1.6.1 Feature Comparison Table

The table below summarises compatible features for STAR-System devices. Further details about individual generic configuration function compatibility is available in the [Configuration API Functions Supported by Device Types](#) section.

Product	Error Injection	Link Events	Time-code Events	Triggering	Periodic Actions	Packet Times-tamping	RMAP Target
SpaceWire Brick Mk2	Yes	Yes	No	No	No	No	No
SpaceWire Brick Mk3 and Brick Mk4	Yes	Yes	No	Yes	No	Yes	No
SpaceWire GbE Brick Mk1	Yes	Yes	No	Yes	No	No	No
SpaceWire Router Mk2S	Yes	Yes	No	No	No	No	No
SpaceWire PCI Mk2 and cPCI Mk2	Yes	No	No	No	Yes	No	No
SpaceWire PCIe	Yes	Yes	Yes	Yes	No	No	No
SpaceWire PCIe Mk2	Yes	Yes	Yes	Yes	No	Yes	No
SpaceWire Physical Layer Tester	Yes	Yes	No	No	No	No	No

SpaceWire PXI Interface and PXI Interface Mk2	Yes	Yes	Yes	Yes	No	Yes	Yes (RMAP Target card variant)
SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2	Yes	Yes	Yes	Yes	No	No	No
STAR Fire Mk3	Yes	Yes	No	No	No	No	No

1.7 Device User Manuals

The user manuals for the devices supported by STAR-System are available in the following sections:

- [SpaceWire PCI Mk2 and cPCI Mk2](#)
- [SpaceWire PCIe](#)
- [SpaceWire PCIe Mk2](#)
- [SpaceWire Brick Mk2](#)
- [SpaceWire Router Mk2S](#)
- [SpaceWire Brick Mk3 and Brick Mk4](#)
- [SpaceWire GbE Brick Mk1](#)
- [SpaceWire PXI Interface and PXI Interface Mk2](#)
- [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)

Information about other devices supported by STAR-System is available in the [Other STAR-System Devices](#) section.

Instructions to upgrade the firmware of a device are included in the [Device Update Application](#) section.

1.8 Graphical Applications

In addition to the command line example applications, a number of Graphical User Interface (GUI) applications are included, to provide simple, yet powerful, methods to interact with STAR-System and the supported devices. These applications are described in further detail in the [Graphical Applications](#) section.

1.9 CUBA Software

A further useful application provided with STAR-System is the CUBA software. This is an interactive tool designed to assist in the development and testing of SpaceWire systems.

The CUBA Software can be used to transmit and receive SpaceWire packets, either as raw packet data or using the RMAP protocol. Packet data can be entered at the command line, or read from files which allow simple scripting of the actions to be performed.

Received data can be displayed in the console or written to output files.

For more information on using the CUBA Software, refer to the [STAR-System CUBA Software](#) section.

1.10 STAR-System APIs

The STAR-System APIs described below are the C APIs which can be used to access supported STAR-System devices. The C++ API uses a similar structure.

1.10.1 STAR-API

Library:	star-api
Include:	star-api.h

The main Application Programming Interface (API) provided with STAR-System is STAR-API. This provides all the functions for transmitting and receiving traffic, getting the list of connected devices and drivers, registering for notification of events, etc.

All traffic transmitted and received using STAR-API is done so over channels to devices. A channel to a device must be opened before traffic can be transmitted or received. For most devices, when the device is in interface mode, a channel corresponds to a port on the device, e.g. all traffic transmitted over channel 1 will be sent over port 1 of the device. When a device is in router mode, a channel corresponds to an external port on the router. Channels can be opened to transmit traffic and/or to receive traffic. This allows one application to transmit traffic over a port, while another application receives from that port. Once an application has opened a channel to a device, no other application can open that same channel until it has been closed.

1.10.2 Generic Configuration API

Library:	star_conf_api_generic
Include:	cfg_api_generic.h , cfg_api_generic_common.h

The Generic Configuration API provides functions for configuring all STAR-System devices. These devices may be connected to the local PC or may be connected over a SpaceWire network. A number of functions common to all devices are provided.

For example, there are functions for getting and setting the routing tables of all routing devices and for starting and stopping each of the links on a device. Newer STAR-Dundee devices support a feature which causes the device to flash its LEDs when requested so the device can be visually identified. This can be useful when there are multiple devices in a PC, or multiple devices on a network. As this feature is not present in older devices, they will return an error code to indicate the function is not supported on the device.

1.10.3 RMAP Packet Library

Library:	rmap_packet_library
Include:	rmap_packet_library.h

The RMAP Packet Library was provided with STAR-Dundee's older USB and PCI APIs. The Remote Memory Access Protocol (RMAP) is a protocol designed specifically for reading and writing memory over SpaceWire. Its simplicity and flexibility means that it can also be used for many other purposes. For example, the Device Configuration API uses RMAP to read and write to registers of SpaceWire devices in order to configure them.

The RMAP Packet Library provides functions for constructing each type of RMAP packet, checking whether a packet is a valid RMAP packet, and accessing the fields in an RMAP packet. The STAR-System version of the library is named differently to previous versions to avoid conflicts with older versions, and to be consistent with the naming convention used throughout STAR-System.

1.10.4 RMAP Target API

Library:	star_rmap_target
Include:	rmap_target_pxi_if.h , rmap_target_types.h

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices. The RMAP targets can operate in automatic or manual authorisation mode. The API provides functions to set the automatic authorisation parameters, allowing ranges of valid values for the RMAP header parameters. When using the manual authorisation mode, the API provides functions to retrieve any pending commands and manually authorise or reject them.

In addition, there are functions to enable notifications when commands require authorisation or when commands are completed. Callback functions can be registered with the API and there are functions to start and stop receiving notifications.

1.10.5 Generic Triggering API

Library:	star_triggering_generic
Include:	triggering_generic.h

The Generic Triggering API provides functions for configuring and controlling the triggering capabilities of the SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2 and STAR-Ultra PCIe Interface devices. If a feature is not supported by a device, the API function calls will return a value to indicate this. Please see the relevant sections: [Input Events](#), [Output Actions](#) and [Internal Triggers](#) for more details about the supported triggering functionality for each STAR-Dundee device, individually. Please note that although the Triggering API functions can be used to configure triggering functionality on any STAR-Dundee device (that supports triggering functionality), the use of the Generic Triggering functions is from now on recommended.

1.10.6 SpaceFibre Configuration API

Library:	star_conf_api_spfi
Include:	cfg_api_spfi.h

The SpaceFibre Configuration API provides functions for configuring SpaceFibre devices. Please see [SpaceFibre Configuration API](#) for more details about the supported functions.

1.10.7 PCI Mk2 Packet Subsystem API

Library:	star_packet_subsystem
Include:	packet_subsystem.h

The PCI Mk2 device contains a single hardware packet generator and checker subsystem. This subsystem can be used to automatically generate and check SpaceWire packets at high speed without using the resources of the host PC.

1.10.8 CCSDS/SPP/TF Packet Library API

Library:	ccsds_spp_packet_library
Include:	ccsds_spp_packet_library.h

The STAR-Dundee CCSDS/SPP/TF Packet Library provides functions which can be called to create both SPP (Space Packet Protocol) and/or TF (Transfer Frame) packets, to retrieve the relevant values of all fields of a SPP and/or TF packet from an array of raw data and to check the validity of these fields.

1.11 Supported Languages

As well as the C API, C++ and Python versions of the APIs are also provided. Documentation for the C++ and Python APIs is available in separate documents, both of which provide an object-oriented version of the C API. The C API can also be used in C++ code, while both the C and C++ versions of the APIs can be imported into other programming languages such as Java and the .NET languages, and scripting languages such as Tcl.

A LabVIEW version of the APIs is available separately. Support for other languages is also planned.

If there is a particular language or application you would like us to support, please contact us using the [contact information](#) below.

1.12 Example Programs

STAR-System comes with a number of example programs with source code, demonstrating the features of each API. Some of these applications are described below.

The examples come with a Makefile, Visual Studio and Eclipse projects. The Eclipse projects can be accessed by pointing Eclipse to the following workspace locations when prompted:

- STAR-System/examples
- STAR-System/examples/config

You may need to run Visual Studio or Eclipse with admin or sudo rights to open the examples directly from the installation location.

1.12.1 STAR-System Test

STAR-System Test provides a number of common operations that you might want to try out when first using your device. These include loopback tests (transmitting packets out of one link and receiving them on another), double

loopback tests (transmitting and receiving on two links at the same time), transmitting packets to another destination, receiving packets from another source, and transmitting and receiving files.

These features have been provided with STAR-Dundee's test programs for previous PCI and USB systems, although STAR-System Test is more powerful than previous incarnations. It allows you to run multiple instances of the software and, for example, transmit from one software instance and receive in another. It also allows you to perform loopback tests between two devices. For example, if you have a SpaceWire PXI Interface device and a SpaceWire Brick Mk3 device, you can connect a SpaceWire cable between them and transmit packets from one to the other.

The source code for STAR-System Test can be used as a basis for your own code, with numerous examples of transmitting and receiving packets provided. The software also provides a Get Version option, so you can find exactly which versions of the various modules of STAR-System you have installed, and which devices are connected to your PC.

1.12.2 STAR Performance Tester

STAR-Dundee strives to get the best performance from our APIs, drivers and devices. For this reason, we use a number of tools to determine the throughput, latency and other performance figures we can achieve. STAR Performance Tester is one of those tools. It provides options to transmit and/or receive packets at the fastest possible rate achievable, using various packet sizes or random packet sizes, and to test the latency when transmitting and receiving. While the tests are being executed, the CPU usage is monitored and recorded, including the total CPU usage and the application's CPU usage. The rates, average packet transmission time (latency) and CPU usage achieved can be written to file, and the results files can then be imported in to Excel to graph data rates, packet rates, etc.

If you're interested to see the throughput that can be achieved using STAR-System with a particular packet size, for example, then STAR Performance Tester can give you that information. The source code provided can be very useful in your own code, as it is designed to achieve the best possible performance from the API and drivers.

Note that you can use the Performance Tester to transmit packets, receive packets, or to do both. If you set the Performance Tester to only transmit, you will need another application to receive those packets, or to configure hardware to drop the packets. It is a similar situation when receiving. You can run two instances of the Performance Tester, either on the same PC or on different PCs, with one instance transmitting and a second instance receiving, for example.

Also note that the latency times displayed are the average of the total time that it takes to transmit and receive a packet. This includes the time that the packet is on the SpaceWire link, so is larger than the actual latency introduced by the software. The data and packets rates shown are the rates at which packets are being transmitted on the link.

The Performance Tester will accept the test parameters as command line arguments. Just pass all the arguments that the application will ask for in order. For string arguments, you can include these in quotes if they contain spaces, or specify two quotes if an empty string is to be provided (e.g. an address path with no bytes).

A `-usechannel0` argument can also be passed to the Performance Tester, to enable the use of channel 0 to transmit and/or receive packets. This argument must be provided prior to any test parameters.

An application note is available from the STAR-Dundee website, describing the performance that can be achieved with a PCI Mk2 using the Performance Tester, and explaining in further detail the meaning of the figures. See <https://www.star-dundee.com/knowledge-base/star-system-performance>.

1.12.3 Device Configuration APIs Examples

The Device Configuration APIs Examples consist of a number of different small applications which demonstrate the use of functions in the Device Configuration APIs. These examples include getting and setting routing table entries, getting and setting the properties of ports, starting and stopping links, and enabling and disabling interface mode of the PCI Mk2.

These examples are used in the documentation for the APIs, and are useful starting points when writing code which uses the Device Configuration APIs.

1.12.4 RMAP Packet Library Example

The RMAP Packet Library Example software doesn't perform a particularly useful task, but its code provides detailed examples of how the RMAP Packet Library can be used to build various types of RMAP packets, check the validity of received RMAP packets, and access the fields of an RMAP packet.

Code from the RMAP Packet Library Example software is included in the documentation for the RMAP Packet Library, to demonstrate the use of the library's functions.

1.12.5 CCSDS/SPP/TF Packet Library Example

The CCSDS/SPP/TF Packet Library Example software doesn't perform a particularly useful task, but its code provides several basic examples of how to use the functions provided by the Packet Library. These functions can be used to:

1. Build SPP and/or TF packets
2. Retrieve complete SPP and/or TF packets from an array of raw packet bytes
3. Access the fields of SPP and/or TF packets

Code from the CCSDS/SPP/TF Library Example software is included in the documentation for the CCSDS/SPP/TF Packet Library, to demonstrate the use of the library's functions.

1.12.6 Generic Triggering API Example

The Generic Triggering Example software demonstrates the use of (some of) the functions defined in the Generic Triggering library. Among the examples are the following:

1. Generate pulse on external trigger when a valid time-code is received
2. Transmit packet when a pulse is received on the external trigger
3. Generate and transmit time-codes periodically

The Generic Triggering Example program also serves as a potential starting point for anyone wanting to make use of the Generic Triggering API functions in their own application.

1.13 Known Issues

The following list contains all known issues with this release of STAR-System:

- The `STAR_setPacketData()` function has been removed from this release of STAR-API as setting the data of a packet which had already been submitted to be transmitted over a channel would cause an error.
- As a result of the `STAR_setPacketData()` issue, STAR Performance Tester's random data test does not use a different length for each packet sent, but instead picks some random lengths which it uses repeatedly.
- Resetting the device, e.g. by calling `STAR_resetDevice()`, while a packet is being transmitted and/or received on the device, can cause the device to get in to a bad state.

If you encounter any problems not included in this list, please contact us using the [contact information](#) below.

1.14 Change Log

A full list of the changes introduced in each version is available in the [Change Log](#) section.

1.14.1 Changes between STAR-System v6.00 and v6.01 - 13th November 2025

- **New Features**

- Added a new and improved Triggering GUI application:
 - * Improved the visuals and usability throughout.
 - * Added the ability to save and load configurations to and from files.
 - * Added multiple example configurations for common use cases.
 - * Added a "Guided Tour" function to provide a tutorial for the application.
- Generic Triggering API:
 - * Added support for SpaceFibre ports and virtual channels.
 - * Added support for the [STAR-Ultra PCIe Interface](#) device.
 - * Added the `TRIGGER_CFG_getDeviceClockFreq()` function to retrieve the clock frequency used by the triggering system on the device.
 - * Added the following functions to retrieve the number of each resource:
 - `TRIGGER_CFG_getNumberOfExtTriggers()`
 - `TRIGGER_CFG_getNumberOfCounters()`
 - `TRIGGER_CFG_getNumberOfSpWPPorts()`
 - `TRIGGER_CFG_getNumberOfTimeCodeInterfaces()`
 - `TRIGGER_CFG_getNumberOfSpFiPorts()`
 - `TRIGGER_CFG_getNumberOfSpFIVCs()`
 - `TRIGGER_CFG_getNumberOfTriggers()`
 - * Added the following functions to make it easier to configure the AND/OR logic and invert state for events:
 - `TRIGGER_CFG_getTriggerSourceEventsAnded()`
 - `TRIGGER_CFG_setTriggerSourceEventsAnded()`
 - `TRIGGER_CFG_getTriggerSourceEventsInverted()`
 - `TRIGGER_CFG_setTriggerSourceEventsInverted()`
 - * Added the following functions for configuring SpaceFibre port events and actions:
 - `TRIGGER_CFG_getSpFiPortInputEvents()`
 - `TRIGGER_CFG_setSpFiPortInputEvents()`
 - `TRIGGER_CFG_getSpFiPortOutputActions()`
 - `TRIGGER_CFG_setSpFiPortOutputActions()`
 - * Added the following functions for configuring SpaceFibre virtual channel events and actions:
 - `TRIGGER_CFG_getSpFiVCInputEvents()`
 - `TRIGGER_CFG_setSpFiVCInputEvents()`
 - `TRIGGER_CFG_getSpFiVCOOutputActions()`
 - `TRIGGER_CFG_setSpFiVCOOutputActions()`
 - * Added the following functions for configuring SpaceFibre virtual channel resources:
 - `TRIGGER_CFG_enableSpFiVCReceiveDataMode()`
 - `TRIGGER_CFG_disableSpFiVCReceiveDataMode()`
 - `TRIGGER_CFG_enableSpFiVCTransmitDataMode()`
 - `TRIGGER_CFG_disableSpFiVCTransmitDataMode()`
 - `TRIGGER_CFG_enableSpFiVCTransmitPacketMode()`
 - `TRIGGER_CFG_disableSpFiVCTransmitPacketMode()`
 - * With the added support for SpaceFibre ports, the existing port functions e.g., `TRIGGER_CFG_setPortInputEvents()` have been deprecated in favour of protocol-specific functions e.g., `TRIGGER_CFG_setSpWPPortInputEvents()`.

- * Added improved documentation to the API's front page.
- Added option to the installers to disable installation of Ethernet support.
- Added installation of the End-User License Agreement as well as licenses used by external libraries.
- Added support for revision 2 of the SpaceWire Link Analyser Mk3, SpaceWire Conformance Tester Mk2, SpaceWire EGSE Mk2 and STAR Fire Mk3.
- Added the ability to save and load the state of the Device Configuration application to and from files.

- **Improvements**

- Fixed issue that could prevent temporary files used by STAR-System being created on Linux.
- Fixed issue that could result in shared memory files used by the STAR-System Device Configuration Service not being removed after uninstallation on Linux.
- Fixed issue that could result in drivers being partially installed on Linux when using the --development command-line installer option.
- Fixed issue that prevented the Linux command-line installer being usable outside of the current working directory.
- Fixed issue that could result in 32-bit and 64-bit libraries being installed incorrectly on recent multiarch Linux distributions.
- Fixed issue in the `CFG_setTxSignallingRate()` function that prevented it being used with the **SpaceWire PCIe Mk2**.
- Fixed issue that could result in the packet format dialogs being slow to load in the Source and Sink applications.
- Fixed issue with the image view's initial size in the Sink application.
- Fixed issue with the `CFG_getTimeCodeDistributionCapablePorts()` function returning an incorrect value.
- Updated the file format used when saving and loading the state of the graphical applications to use JSON.
- Various improvements and updates to the application notes.
- Various improvements and updates to the documentation.
- Linux kernel compatibility tested up to v6.17.7.

1.15 Support and Contact Information

For software updates and access to further technical information, please register your STAR-Dundee products at <http://www.star-dundee.com/product-registration> (or click the Product Registration link on our website: <http://www.star-dundee.com>). After registering your devices, you will be given an account with permissions to download updates for those devices. You can also register to receive e-mail notifications when new updates are available.

If you wish to contact STAR-Dundee with a support enquiry, please e-mail support@star-dundee.com. For general enquiries, please contact enquiries@star-dundee.com. Our full address is as follows:

STAR-Dundee Ltd.
STAR House
166 Nethergate
Dundee
DD1 4EE
Scotland, UK

Chapter 2

STAR-System Applications

STAR-System includes many applications which make it simple to perform common tasks.

These include graphical applications and command-line applications, some of which are provided as example code.

The [Graphical Applications](#) are described on a separate page.

The [CUBA Software](#), a command line application for performing and scripting transmit and receive operations or RMAP commands, is also documented on a separate page.

2.1 Graphical Applications

This section contains information on the Graphical User Interface (GUI) applications, provided with STAR-System.

Each GUI application contains context-sensitive help, explaining the purpose of each component of the application and how it can be used. The help can be accessed by pressing Shift + F1 when a component has focus, or by clicking the ? button in the application's title bar, then clicking on the component of interest. The help text will disappear when you click elsewhere in the application.

Note

These applications make use of Nokia's Qt libraries. The necessary libraries are installed by the STAR-System installer for Windows and QNX.

On Linux Qt may be installed by default, or will be available to download using your Linux distribution's package manager. STAR-System provides both Qt4 and Qt5 versions of the GUI applications. Qt5 brings the ability to transmit and receive webcam data to the Source and Sink applications and improves support for high DPI displays. The packages required for Qt4 are libqtcore4 and libqtgui4. Qt5 may require qt5-default or qt5-qtbase and libqt5multimedia5-plugins or qt5-qtmultimedia depending on the distribution. The STAR-System applications have been successfully tested on Qt versions 4.7, 4.8, and 5.5 and above.

Please see the [Qt4 and Qt5 Libraries](#) section for important information regarding the Qt Libraries.

Each of the GUI applications is described below. A description of the command-line arguments that can be passed to each of the applications is also provided in the [Command-Line Arguments](#) section.

2.1.1 Device Configuration Application

The Device Configuration application provides a window in which the properties of a device can be configured. From this application, links can be started or stopped, the device can be reset, the link speed can be changed, the routing table can be modified, etc.

The tabbed interface provides a tab for viewing the properties, and configuring those properties, for each of the ports on the device, including the configuration port and each of the external ports. Tabs are also included to view and configure the properties of the device which are local to the PC, such as the name used to refer to the device, and also the general device properties. The properties which are configurable can be changed by altering the value and pressing the corresponding Set button, which will only become enabled when the value has been changed.

Using the Device Configuration application, it is also possible to configure other supported devices on a SpaceWire network. To do this, check the Remote device check box, enter the path and return path to the device's configuration port (the path to the local device's configuration port are provided as an example), then click the Display Properties button. The properties of the remote device should be shown, although as this device is not local, the Local Properties tab will not be present. Note that to configure a remote device, the Device Configuration application needs to open a channel to transmit and receive the configuration packets. This means this channel cannot be used for data packets while the remote device's properties are being read or set.

Note that PCI/cPCI Mk2 and PCIe devices prior to version 1.07 have a bug which means that reading the value of the port routing registers always returns 0. This means that when viewing the properties of such devices, the port routing address will always be 0, and interface mode on port and identify source on port will always be disabled. The latest firmware version for these devices resolves this issue, and can be downloaded from the STAR-Dundee website. See the [Support and Contact Information](#) section for more information on registering your product.

2.1.2 Transmit Application

The Transmit application is a simple application for transmitting packets.

It allows the bytes of a packet to be entered in to a text box, then transmitted using the specified device and channel by pressing the Transmit Packet button. The base in which the packet is entered can be specified by changing the base in the drop down list. The end of packet marker that is used to terminate the packets transmitted can also be specified.

2.1.3 Receive Application

The Receive application is a simple application for receiving packets.

It allows packets to be received by pressing the Receive Packets button. This will cause all packets received on the specified device and channel to be displayed, along with their end of packet marker type. The base in which the packets are displayed can be altered by changing the base in the drop down list. Packets will continue to be received until the Stop Receiving button is pressed. The packets on display can be cleared by clicking the Clear Packets button.

2.1.4 Source Application

The Source application is a more powerful application for transmitting packets with complex formats at high speed.

It allows a packet format, and a schedule to which those packets should be transmitted, to be specified. Packets can be transmitted repeatedly or a specified number of times, and time delays can be included between packets. The packet format specified is saved for future use, and previously saved packet formats can be edited and reused. The same packet format can also be used in the [Sink Application](#) application to check the format of received packets.

Once the format and schedule have been defined, the packets can be transmitted. While the packets are being transmitted, the transmit statistics can be displayed. These statistics show the total number of characters transmit-

ted and the characters transmitted in the last second. Packet transmission can be stopped at any time by clicking the appropriate button.

2.1.5 Sink Application

The Sink application is a more powerful application for receiving packets at high speed, and checking the contents of packets for errors.

The Sink application allows packets to be received and the content of those packets to be checked for any deviation from the expected format. Received packets, or fields within those packets, can also be written to file. While the packets are being received, the receive statistics can be displayed. These statistics show the total number of characters received and the characters received in the last second. Any errors detected in the received packets will also be displayed in the statistics. Packet reception can be stopped at any time by clicking the appropriate button.

2.1.6 Error Injection Application

The Error Injection application allows errors to be injected onto a SpaceWire link. The types of error that may be injected are described in the documentation for [SPW_ERROR](#).

2.1.7 Device Update Application

The Device Update application allows in-the-field updating of the FPGAs in STAR-Dundee devices. When an update is required or recommended for a device, the appropriate update files will be made available for download from the STAR-Dundee website. See the [Support and Contact Information](#) section for more information on registering your product. These update files will be processed by the Device Update application, and applied to the target devices.

Details on the use of the Device Update Application can be found in the [Device Update Application](#) page.

2.1.8 Time-code Application

The Time-code application allows time-codes to be transmitted and received on STAR-Dundee devices. Other functions such as enabling a device as a time-code master and enabling time-code distribution ports are also provided.

2.1.9 RMAP Initiator Application

The RMAP Initiator application can be used to write to and read from memory in a target SpaceWire node using the remote memory access protocol (RMAP).

The RMAP Initiator transmits RMAP commands and receives any corresponding replies. RMAP write and read command properties are configurable. The data bytes to write to memory can be entered directly or defined in a file. The data bytes read from memory can be displayed on screen or written to file. The RMAP Initiator application compares the total number of bytes to write to or read from memory with the RMAP command data length specified, and transmits the appropriate number of commands required to achieve this. For greater control, the maximum number of posted commands associated with a transaction is configurable, as is the transaction time out. Status information indicates if any errors are detected, the number of command packets transmitted, the number of valid replies received, and the current transaction status.

2.1.10 RMAP Target Application

The RMAP Target application allows the RMAP targets within supported devices to be configured. This includes configuration of the authorisation parameters such as the valid memory range, valid key range, and valid command types.

Currently, the RMAP Target application is supported on the SpaceWire PXI Interface with RMAP Target only.

2.1.11 Triggering Application

The Triggering application allows simple editing and visualisation of triggering configurations on supported devices (see [Feature Comparison Table](#)).

It can be used to read and display a device's current configuration, write new configurations to the device, and provides example configurations for common use cases. Additionally, configurations can be exported for future use.

2.1.12 Ethernet Device Connector Application

The Ethernet Device Connector application allows connecting to the Ethernet devices, e.g. the GbE Brick Mk1.

The application lets you connect an Ethernet Device to STAR-System so that the device can be detected by other STAR-System applications, including the GUI applications, the command line applications and any other applications developed using the STAR-System APIs.

2.1.13 Command-Line Arguments

All STAR-System GUI Applications also support a number of command-line arguments to facilitate device set-up at program start. The following arguments are supported:

- **-0** or **--zero** Includes channel 0 as an option in the Device/Port Selector drop-down box options.
- **-s [serial #]** or **--serial [serial #]** Selects the device with serial number *serial #* as the default selected device from the Device Selector drop-down box.
*Note: If there is no device connected with serial number **serial #**, the argument is ignored.*

- `-c [ch #]` or `--channel [ch #]` Selects `ch #` as the default channel from the Device/Port Selector drop-down box.

Note: If the selected device does not have channel `ch #`, 1 (or 0) is chosen as the default.

Note: All the above arguments additionally support the Windows-based "/" delimiter for both long and short versions of the arguments.

2.1.14 Running the GUI Applications on QNX

The STAR-System GUI applications are implemented using Qt for QNX. This uses its own windowing system that is incompatible with QNX's Photon environment.

To use the GUI applications on QNX, QNX must be running in text mode and `io-display` must be running. To enter text mode from Photon, either `kill` the `Photon` process, or click Launch->Logout(End Photon session). In the login dialog, click Shutdown. The shutdown dialog appears. Enable Exit to text mode and click Ok.

Set up the following environment variables:

```
export QWS_KEYBOARD=qnx
export QWS_MOUSE_PROTO=qnx
export QWS_DISPLAY=qnx
```

To enable the mouse and keyboard under Qt, `devi-hid` must be run as follows: `/usr/photon/bin/devi-hid -Pr kbd mouse` This will prevent your current shell from accepting keyboard and mouse input, so be sure to either run this from a script that launches the GUI applications afterwards, or have a remote terminal available to launch applications.

When launching the GUI applications to run under QNX, the first application must be run with the option `-qws` in order to initialise the Qt windowing system and be designated as the GUI server. Only one Qt application should have this option specified.

2.2 CUBA Software

This section contains information on the STAR-System CUBA Software application.

The STAR-System CUBA Software is designed to support the development and testing of SpaceWire systems, allowing interactive or scripted sending and receiving of SpaceWire packets, including use of the RMAP protocol.

The STAR-System CUBA Software can be used to send and receive raw SpaceWire packets, either entered and displayed at the command line, or reading from command files and outputting received packets to an output file. It can also perform similar tasks using the SpaceWire Remote Memory Access Protocol (RMAP). RMAP commands to be performed may be entered at the command line, or supplied as a list of commands in a command file. The results of these operations can then be displayed on the command line or output to a file.

The CUBA Software can be run from the start menu, in which case it will start in its fully interactive mode. Alternatively it may be run from the command line with a number of optional arguments which can be used to modify its behaviour. These arguments are detailed in the [CUBA Command Line Arguments](#) section.

If there are no compatible SpaceWire devices connected when the software is run, it will display a message and exit. If there is only one device connected, this device will be used. If there is more than one device connected, and a valid device number has not been specified at the command line, the software will display a list of available devices and ask for one to be selected.

If the selected device has more than one channel available for use, and a valid channel number has not been specified at the command line, the software will ask for a channel number to be selected.

The command line options allow the application to be run in one of the command stream modes without requiring any input from the user. This can be useful for executing test scripts, for example, and the software can also be called from a batch file or shell script.

The following sections describe use of the CUBA software to send and receive raw SpaceWire packets in interactive and command stream modes, and similarly its use for RMAP packets in interactive and command stream modes.

2.2.1 Data Format Conventions

When entering data values for SpaceWire and RMAP packets, both in interactive mode and in command files, there are a number of conventions which may be used for numeric and other data.

The initial default base is hexadecimal, although this can be changed by commands in both interactive and command stream modes. Space-separated data values can be entered in the default base, and the base can be changed per data value if required by preceding it by a base change indicator. These are:

- 'b or 'B for binary - e.g. 'b10110100
- 'o or 'O for octal - e.g. 'O264
- 'd or 'D for decimal - e.g. 'd180
- 'x or 'X for hex - e.g. 'Xb4

ASCII character strings may be included in data fields by enclosing the required characters in double quotes - e.g. "Test string". The ASCII values of the characters will be used, and sent as individual bytes.

The contents of a data file may be used by entering the file name preceded by a hyphen - e.g. -testfile.dat. If the file name contains spaces it can be enclosed in double quotes - e.g. -"another file.txt". The contents of the file will be sent as raw data bytes.

All of the above data input modes can be combined as needed to define the data to be sent. Assuming the default base to be hex, the following is a valid data definition:

```
12 34 "characters" 'd234 'b10101100 -datafile.txt fe ff
```

Note that when the CUBA Software is operating in backwards compatibility mode, slightly different conventions apply. See the [Backwards Compatibility Mode](#) section for more details.

2.2.2 SpaceWire Interactive Mode

The SpaceWire Interactive Command/Reply Mode allows the bytes of a packet to be sent to be entered at the command line. After the bytes have been entered, pressing Enter will cause the bytes to be sent in a packet terminated by an EOP.

Once the packet has been sent, the software will wait for packets to be received. Each packet received will be shown at the command line, including the end of packet marker type. To stop the software waiting for packets, press any key. To receive packets without first sending one, just press Enter (without specifying any bytes to be sent) when the software asks for a packet to be sent.

The base used when entering packets to be sent and displaying any responses may be altered by entering base=<base>, where <base> is 2, 0b or i for binary; 8, 0o or o for octal; 10, 0d or n for decimal; or 16, 0x or h for hexadecimal. Note that when entering the base, any numbers entered are assumed to be decimal, regardless of the current base.

To exit the program, x should be entered.

2.2.3 RMAP Interactive Mode

The RMAP Interactive Command/Reply Mode allows RMAP commands to be specified at the command line. The software prompts for each of the fields in the command to be entered.

When this mode is entered a list of possible commands to be performed is provided:

- Read command
- Write command
- Read-modify-write command
- Change the default base
- Delete all fixed fields
- Exit

Selecting to perform a read, write or read-modify-write command will result in the software asking for values for each of the fields for those commands. If a value for a field has already been provided when performing a previous command, that value will be shown as the default value. Pressing Enter without entering a value will cause the software to use the default, otherwise a value can be entered. This will then become the new default.

If a value is entered followed by an x, then that value will become fixed, and all future commands will not ask for this field to be specified, the fixed value will be used instead. A default value can also be set to fixed by simply entering x. Fixed fields may be deleted by selecting the appropriate option from the main menu.

When a boolean value is expected, t, true, y or yes may be entered to indicate true, while f, false, n or no may be entered to indicate false. If a series of bytes is expected, each byte should be entered separated by a space, while strings can also be used, as described in the [Data Format Conventions](#) section.

Once the fields of a command have been specified, the command will be sent. If a response is expected by the command, the software will wait for the response. If a response is received it will be displayed. To stop waiting for a response, a key can be pressed. The main menu will be displayed once the command has completed or has been stopped.

The table below shows the valid fields for RMAP Interactive Command/Reply mode.

Fields for RMAP Interactive Command/Reply Mode

Field Name	Type	Command Write	Read	R-M-W
Target Address	Series of bytes	Yes	Yes	Yes
Initiator or Reply Address	Series of bytes	Yes	Yes	Yes
Verify Before Write	Boolean	Yes	—	—
Acknowledge	Boolean	Yes	—	—
Increment Address	Boolean	Yes	Yes	—
Key	1 byte number	Yes	Yes	Yes
Transaction Identifier	2 byte number	Yes	Yes	Yes

Read Write Address	4 byte number	Yes	Yes	Yes
Extended Read Write Address	1 byte number	Yes	Yes	Yes
Data Length	3 byte number	—	Yes	—
Header CRC	1 byte number	Yes	Yes	Yes
Data	Series of bytes	Yes	—	Yes
Mask	4 byte number	—	—	Yes
Data CRC	1 byte number	Yes	—	Yes

The base used when entering field values and displaying the field values in responses may be altered by selecting the appropriate menu option. The base should then be specified, with 2, 0b or i for binary; 8, 0o, or o for octal; 10, 0d or n for decimal; or 16, 0x or h for hexadecimal. Note that when entering the base, any numbers entered are assumed to be decimal, regardless of the current base. Once the base has been specified, the software will return to the menu.

To exit the program, x should be entered.

2.2.4 Command File Directives

Both the SpaceWire and RMAP command stream modes support common directives which are processed prior to processing and executing the commands in the input file provided. These are similar to pre-processor directives used in C and C++ programming, for example. All directives begin with a # character. The supported directives are described below.

2.2.4.1 #INCLUDELIST Directive

The #INCLUDELIST directive allows other command files to be included in a command file. This is similar to a #include in C and C++. The name of the file to be included should follow the #INCLUDELIST directive, and the command should be terminated with a semi-colon. The filename may contain spaces and the #INCLUDELIST may be specified in any mixture of upper and lowercase.

Where an #INCLUDELIST directive is encountered, the CUBA Software will replace the directive with the command file specified in the directive before processing and executing the commands. The included file could in turn include further files.

The example below includes the file `include file.cuba`. Note that quotes are not required around the filename, even though it contains spaces

```
#IncludeList include file.cuba;
```

2.2.4.2 #ALIAS Directive

The #ALIAS directive allows a string in the file and any included files to be replaced with a value specified in the #ALIAS directive. This is similar to a #define macro in C and C++. The #ALIAS directive should be followed by the alias string, an equals sign (=) and then the value to replace the alias string with. The directive should be terminated with a semi-colon. As with the #INCLUDELIST directive, the case of the #ALIAS text is not important. However the case of the alias string is important. Only text that is exactly the same as the string specified will be replaced.

Where an #ALIAS directive is encountered, the CUBA software will remove the alias directive and replace all occurrences of the alias string following the directive, including those included using the #INCLUDELIST directive, with the alias value after the equals sign. This value can include spaces and can be useful for specifying commonly used values, particularly those that might need to be changed in future.

For example, the path address to a device can be represented using an #ALIAS directive, as shown below:

```
#ALIAS ADDRESS=1 3;
```



```
S D:ADDRESS aa bb cc dd ee ff;
S D:ADDRESS 01 23 45 67 89;
```

2.2.5 SpaceWire Command Stream Mode

In the SpaceWire Command Stream Mode a file containing a list of SpaceWire commands to be performed must be specified. A file to output any packets received can also be specified, or left as blank if the received packets are of no interest.

A file to dump the pre-processed input file can also be specified. This will produce a file with all comments removed, and all directives (such as #ALIAS and #INCLUDELIST directives) processed. This can be useful to debug any problems with a command file.

There are four types of commands which can be included in a SpaceWire Command Stream: send packets, receive packets, loop commands or change the base in use. These are described below. Each command must be terminated with a semi-colon (;) and comments may be included by placing them between /* and */. The information in a comment is ignored.

2.2.5.1 SpaceWire Send Command

Send commands begin with S. An optional N: parameter can be used to indicate the number of packets to be sent. Each packet will be identical. The N: should be followed by the number of packets. If this parameter is not specified only one copy of the packet is sent.

By default, all packets are terminated with a normal end of packet marker. An optional E: parameter allows the end of packet marker type to be included at the end of the packet to be specified. The E: should be followed by O, N or F to indicate an EOP should be used (eOp, No or False), or E, Y or T to indicate an EEP should be used (eEp, Yes or True). The value provided is case-insensitive.

The data to be sent should be included following a D: parameter, each byte separated by a space. The following is an example send command (it is assumed the base is currently hexadecimal):

```
S D:11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
```

To send this packet ten times, terminated by an EEP, the following command could be used:

```
S N:a E:t D:11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
```

2.2.5.2 SpaceWire Receive Command

Receive commands begin with R. An optional N: parameter can be used to indicate the number of packets to be received. The N: should be followed by the number of packets. The following is an example receive command to receive a single packet:

```
R;
```

To receive 10 packets, the following command could be used:

```
R N:'d10;
```

Received packets can also be written to a file using the optional F: parameter. The file to write to should be included after the F: parameter, and should be placed in quotes if it contains spaces. The file specified will be appended to (not overwritten), so that the same file can be used for multiple commands. The data will be written to the file as raw bytes. The following example will write the received data to the file spw_output:

```
R F:spw_output;
```

Note that there are no markers inserted in the file to indicate the start or end of packets.

2.2.5.3 Loop Command

Loop commands begin with L and allow subsequent commands to be repeated. An N: parameter should be used to indicate the number of commands to be repeated. The N: should be followed by the number of commands to repeat. A T: parameter should be used to indicate the number of times to repeat the commands. The T: should be followed by the number of times to repeat the commands.

The following is an example loop command to send 5 packets and receive 5 packets:

```
L N:2 T:5;
  S D:00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
  R;
```

In the above example, the two commands following the loop command are repeated five times. Indentation has been used to highlight the commands being repeated.

Loop commands can be nested within one another. When this is done, the loop command and all the commands below it that are to be repeated are treated as one command. The following commands send 20 packets and receive twenty responses:

```
L N:2 T:5;
  S N:4 D:00 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;
  L N:2 T:2;
    R;
    R;
```

The first loop command will repeat the send command and the second loop command five times. The second loop command will repeat the two receive commands twice. As this loop command is being repeated 5 times, the two receive commands are each repeated a total of ten times.

2.2.5.4 Change Base Command

To change the base of values in the file, the B command can be used. Following the B and a space, the base to use should be entered, with 2, 0b or i indicating binary; 8, 0o or o indicating octal; 10, 0d or n indicating decimal; and 16, 0x or h indicating hexadecimal. Note that the values for the base are always entered in decimal. The following is an example command to change the base to decimal:

```
B 10;
```

The base used in output files can be changed in a similar way, using the O command. The following is an example command to change the output file's base to octal:

```
O 8;
```

Only the last command to change the output file's base will be used.

Note that the base commands are not included in loop commands, so cannot be repeated. The input base affects all text below the command, while the output base affects the entire output file.

2.2.5.5 Example SpaceWire Command Stream Input File

The following is an example command stream input file, containing comments explaining what the file does:

```
/* Send 10 packets */
S N:a D:1 3 11 22 33 44 55 66 77 88 99 aa bb cc dd ee ff;

/* Change the base to decimal */
B 10;

/* Receive 10 packets */
R N:10;
```

2.2.5.6 SpaceWire Command Stream Output Files

The output files produced when in SpaceWire Command Stream Mode contain the results of each receive operation. Each line in the output file corresponds to a receive operation in the command file (a receive operation may have multiple lines in the output file, e.g. if it is meant to receive multiple packets). The first character on the line indicates the success or failure of the operation, with the following values possible:

- F - Full packet received
- P - Partial packet received
- T - Timeout occurred (Partial packet might have been received)
- E - Error occurred

Following this character and a space is the contents of any packets read. The bytes of the packet will be shown in hexadecimal, unless an alternative base has been specified in the command file. If an end of packet marker has been received for a packet, this shall be included at the end of the line, with EOP indicating a normal end of packet marker and EEP indicating an error end of packet marker.

A comment line is included at the top of every output file, indicating the base in use.

The following is an example of a successfully read packet terminated with an EOP:

```
/* Base = Hexadecimal */
F 11 22 33 44 55 66 77 88 99 EOP
```

2.2.5.7 Example SpaceWire Command Stream Output File

The following is an example command stream output file, in which 8 packets have been successfully received terminated by an EOP, one has been received terminated by an EEP, and the final one timed out:

```
/* Base = Hexadecimal */
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF EOP
F 11 22 33 44 55 EEP
T
```

2.2.6 RMAP Command Stream Mode

In the RMAP Command Stream Mode a file containing a list of RMAP commands to be performed must be specified. A file to output any responses to these commands can also be specified, or left as blank if the responses are of no interest.

2.2.6.1 RMAP Command Stream Input Files

RMAP Command Stream input files take the form of a number of fields, each followed by a value. The end of a command is indicated by a blank line. The first command must have all fields required for that command specified,

other than the CRC field, which is automatically calculated. Subsequent commands can omit a field if it has been specified in a previous command.

The table below shows the fields which should be provided for each type of command. Note that R-M-W refers to the read-modify-write command. The first field specified should be the type of the command, which can be a write, read or read-modify-write. The field name does not need to be specified when specifying the command type. Subsequent fields can be in any order. To specify a field, the field's full name should be supplied, or 2 or more characters of its short name. This should be followed by a colon, then the value of the field, and then a semi-colon.

For the command type, w, r or m should be specified. If a series of bytes is expected, each byte should be a value between 0 and 0xFF, and should be separated from the next byte by a space. Additionally, strings and files can also be used. If a boolean value is expected, then either t, true, y or yes can be used to indicate true, while f, false, n or no can be used to indicate false.

If a 1 byte number is expected, a value between 0 and 0xFF should be entered.

If a 2 byte number is expected, a value between 0 and 0xFFFF should be entered.

If a 3 byte number is expected, a value between 0 and 0xFFFFFF should be entered.

If a 4 byte number is expected, a value between 0 and 0xFFFFFFFF should be entered.

The table below shows the valid fields for RMAP Command Stream mode.

Fields for RMAP Command Stream Input Files

Field Name	Short Name	Type	Command		
			Write	Read	R-M-W
CommandType	Command	W, R or M	W	R	M
TargetAddress	Target	Series of bytes	Yes	Yes	Yes
ReplyAddress	Reply	Series of bytes	Yes	Yes	Yes
VerifyBefore↔ Write	Verify	Boolean	Yes	—	—
Acknowledge	Acknowledge	Boolean	Yes	—	—
Increment↔ Address	Increment	Boolean	Yes	Yes	—
Key	Key	1 byte number	Yes	Yes	Yes
Transaction↔ Identifier	Transaction	2 byte number	Yes	Yes	Yes
ReadWrite↔ Address	Address	4 byte number	Yes	Yes	Yes
Extended↔ ReadWrite↔ Address	Extended	1 byte number	Yes	Yes	Yes
DataLength	Length	3 byte number	—	Yes	—
HeaderCRC	HeaderCRC	1 byte number	Yes	Yes	Yes
Data	Data	Series of bytes	Yes	—	Yes
Mask	Mask	4 byte number	—	—	Yes
DataCRC	CRC	1 byte number	Yes	—	Yes

Refer to the [Data Format Conventions](#) section for information on data formats for the contents of the fields

For command types that expect responses containing data (read and read-modify-write commands), an optional Filename field can be provided to specify a file to write the data to. The file to write to should be included after the Filename field, and should be placed in quotes if it contains spaces. The file specified will be appended to (not overwritten), so that the same file can be used for multiple commands. The data will be written to the file as raw bytes.

2.2.6.2 Loop Command

The RMAP Command Stream Mode includes a loop command similar to the one provided for the SpaceWire Command Stream Mode. Loop commands have a command type of l and allow subsequent commands to be repeated.

A `NumberOfRepeat` field should be used to indicate the number of commands to be repeated, while a `TimesToRepeat` parameter should be used to indicate the number of times to repeat the commands. Both fields accept numerical values, and only the first 2 characters of the field names are required.

Loop commands can be nested within one another. When this is done, the loop command and all the commands below it that are to be repeated are treated as one command.

Note that the base commands are not included in loop commands, so cannot be repeated.

2.2.6.3 Example RMAP Command Stream Input File

The following is an example RMAP command stream input file, containing comments explaining what the file does. Indentation has been used to highlight the commands repeated in loop commands.

```
/* Send a write command to address 0x106, using full field names */
CommandType: w;
TargetAddress: 1 0 FE;
VerifyBeforeWrite: T;
Acknowledge: T;
IncrementAddress: F;
Key: 20;
ReplyAddress: 4 FE;
TransactionIdentifier: 0;
ExtendedReadWriteAddress: 0;
ReadWriteAddress: 106;
Data: 01 23 45 67;

/* Change the output base to decimal */
B;
Output: 10;

/* Loop over the next 3 commands (which includes a loop) twice */
l;
TimesToRepeat: 2;
numbertorepeat: 3;

    /* Send a write command to address 0x101, using short field names and using defaults */
    w;
    Address: 101;
    Data: 89 ab cd ef;

    /* Send a write command to address 0x106, using decimal and octal values, */
    /* (very) short field names and using defaults */
    w;
    Ad: 106;
    Da: '021 34 '063 68;

    /* Loop over the next command twice */
    l;
    ti: 2;
    nu: 1;

    /* Read address 106, and write the data to rmap_out */
    r;
    file: rmap_out;
```

2.2.6.4 RMAP Command Stream Output Files

RMAP Command Stream output files contain the fields and their values of any received packets. These can be acknowledges to write commands, or responses to read and read-modify-write commands containing the data read.

If an expected response was not received, due to a timeout, then an error will be included in the file. This will show the type of the expected command and the expected transaction identifier.

The base in use to show the field values is shown in a comment at the top of the file.

2.2.6.5 Example RMAP Command Stream Output File

The following is an example command stream output file, in which the responses to two RMAP write commands have been successfully received, while a third has timed out:

```
/* Base = Hexadecimal */

Write Response
-----
Reply Address: FE
Data verification is enabled
Incrementing addresses is not enabled.
Status = success
Target Address: FE
TransactionIdentifier: 0
Header CRC: bd

Write Response
-----
Reply Address: FE
Data verification is enabled
Incrementing addresses is not enabled.
Status = success
Target Address: FE
TransactionIdentifier: 0
Header CRC: ba

Error receiving acknowledgement for command 3
```

2.2.7 CUBA Command Line Arguments

The arguments that can be passed to the CUBA Software are described below:

-help	Display information about the application and the arguments that can be passed to it, then exit.
-device=<device_num>	If more than one SpaceWire device is connected to the PC in use, the number of the device to be used by the software to send and receive packets can be specified by providing its number. The number of the first device connected is 0.
-channel=<channel_num>	If more than one SpaceWire channel is present on the selected device, the number of the channel to be used by the software to transmit and receive packets can be specified by providing its number.

<code>-all_devices=<enable></code>	By default the CUBA Software only accesses devices which can transmit and receive packets. Setting this parameter non-zero allows the device to be selected from all available SpaceWire devices								
<code>-mode=<mode></code>	Sets the mode the software operates in, where <code><mode></code> is one of the following values: <table> <tr> <td><code>interactive_↔ spacewire or iscr</code></td><td>Interactive SpaceWire Command/Reply Mode</td></tr> <tr> <td><code>interactive_↔ rmap or ircr</code></td><td>Interactive RMAP Command/Reply Mode</td></tr> <tr> <td><code>spacewire_↔ command or scs</code></td><td>SpaceWire Command Stream Mode</td></tr> <tr> <td><code>rmap_command or rcs</code></td><td>RMAP Command Stream Mode</td></tr> </table>	<code>interactive_↔ spacewire or iscr</code>	Interactive SpaceWire Command/Reply Mode	<code>interactive_↔ rmap or ircr</code>	Interactive RMAP Command/Reply Mode	<code>spacewire_↔ command or scs</code>	SpaceWire Command Stream Mode	<code>rmap_command or rcs</code>	RMAP Command Stream Mode
<code>interactive_↔ spacewire or iscr</code>	Interactive SpaceWire Command/Reply Mode								
<code>interactive_↔ rmap or ircr</code>	Interactive RMAP Command/Reply Mode								
<code>spacewire_↔ command or scs</code>	SpaceWire Command Stream Mode								
<code>rmap_command or rcs</code>	RMAP Command Stream Mode								
<code>-command_list=<file></code>	Specify the path to a file containing a command list. This argument is only valid in the command stream modes, and is ignored in the interactive modes.								
<code>-output=<file></code>	Specify the path to a file where any output will be recorded. This argument is only valid in the command stream modes, and is ignored in the interactive modes. The <code><file></code> may be left blank if no output file is required.								
<code>-timeout=<timeout></code>	Sets the timeout to be applied for any read operations, specified in milliseconds. This argument is only valid in the command stream modes, and is ignored in the interactive modes. If this option is not specified, the default value is 10 seconds.								
<code>-dump_cmdlist=<file></code>	Specify the path to a file which will be updated to contain the command file to be used after all pre-processing has been performed. This argument is only valid in the command stream modes, and is ignored in the interactive modes.								
<code>-backwards_compatibility=<1 or 0></code>	Enables or disables the backwards compatibility mode. If backwards compatibility is enabled, all scripts written for the original CUBA Software for the old SpaceWire USB API will work in the command stream modes. Note that there are some minor differences in data representation as noted below in the Backwards Compatibility Mode section.								

2.2.8 Backwards Compatibility Mode

Backwards Compatibility Mode is provided to allow the use of command stream files which were created for use with the previous version of the CUBA Software supplied with the Brick (Mk1), Router-USB and Router Mk2. There are a number of differences, particularly in the format of numeric data, which should be noted when using this mode.

To change the base used for a data item, the base indicator is preceded by a zero, i.e. '0b' for binary, '0o' for octal, '0d' for decimal and '0x' for hexadecimal (the base is not case-sensitive). This can lead to errors when using hexadecimal as the default base and the values 0b or 0d are used. These will be interpreted as base change requests with no following value, and will result in those values being omitted from the data.

In order to enter the values 0b and 0d they must either be input as simply 'b' and 'd', or explicitly in hex as '0x0b' and '0x0d'.

A further potential source of errors in command file data blocks is that data entered over multiple lines must have a trailing space at the end of each line. This separator must be present, otherwise the value at the end of the line will be concatenated with the first value on the next line. E.g. data entered as

```
Data: 12 34 ... .. 98 76 (no trailing space)
ab cd ... ..
```

would be interpreted as 12 34 98 76ab cd Since data is sent as bytes, the value '76ab' would be transmitted as only 'ab', omitting the '76' value.

Note that these are problems present in the original CUBA Software.

In backwards compatibility mode character strings may be entered using the single quote character. However, in the STAR-System version this character has been redefined to signal base change (see [Data Format Conventions](#)) and cannot be used for strings.

Command stream files which have been used successfully with the older version of the CUBA Software will continue to work correctly with the STAR-System CUBA Software when backwards compatibility is enabled. When writing new command files we recommend using the new conventions for formatting data.

Chapter 3

STAR-System Devices

The following sections describe the hardware devices supported by STAR-System, and how they can be updated:

- [SpaceWire PCI Mk2 and cPCI Mk2](#)
- [SpaceWire PCIe](#)
- [SpaceWire PCIe Mk2](#)
- [SpaceWire Brick Mk2](#)
- [SpaceWire Router Mk2S](#)
- [SpaceWire Brick Mk3 and Brick Mk4](#)
- [SpaceWire PXI Interface and PXI Interface Mk2](#)
- [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#)
- [SpaceWire GbE Brick Mk1](#)
- [Other STAR-System Devices](#)
- [Device Update Application](#)

3.1 SpaceWire PCI Mk2 and cPCI Mk2

Documentation for the STAR-Dundee SpaceWire PCI Mk2 and cPCI Mk2 devices is provided in this section.

3.1.1 Overview

3.1.1.1 Summary

This user manual provides an overview of the interfaces and features of the SpaceWire PCI Mk2 hardware.

For detailed information on using the SpaceWire PCI Mk2 card please consult the API reference documentation.

3.1.1.2 Hardware Description

The SpaceWire PCI Mk2 provides a PCI to SpaceWire interface device with three SpaceWire interfaces. Efficient host software support is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire PCI Mk2 board is shown below.

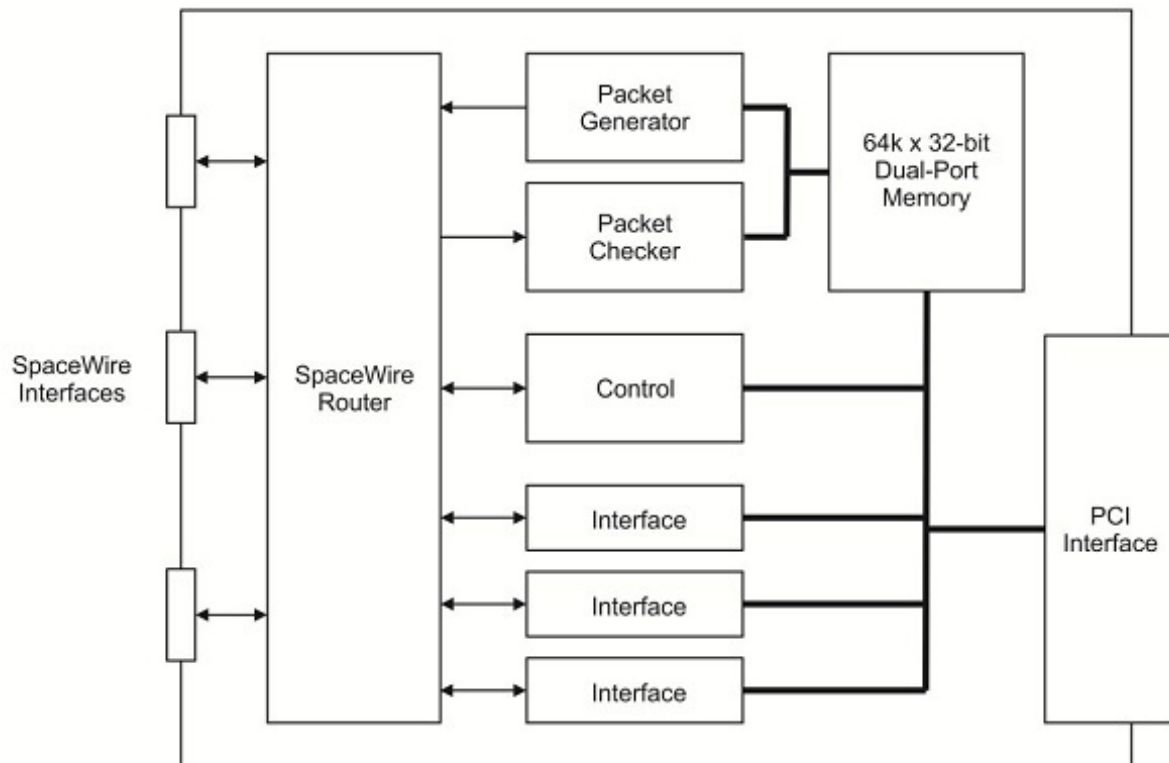


Figure 3.1: SpaceWire PCI Mk2 hardware overview

The three SpaceWire interfaces of the SpaceWire PCI Mk2 are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any another SpaceWire port, or the PCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the PCI channel for that link. There are three independent channels from the SpaceWire router to the PCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the SpaceWire PCI Mk2 regardless of the data flow.

The PCI interface is 32 bits wide and can operate at 33 or 66 MHz. It contains a DMA controller for rapid transfer of data to and from the SpaceWire PCI Mk2 board.

A hardware packet generator and checker are included on the SpaceWire PCI Mk2 to automatically generate and check SpaceWire packets at high speed without using host computer resources. The packets to be generated are stored in a dual-port memory on the PCI Mk2 board. Packets of any length can be generated with a separate header and cargo as required. When a packet is larger than the amount of RAM available on the on board dual port RAM the cargo part of the packet will be resent multiple times to satisfy the packet length required. The speed of packet data generation and the gap between packets can be controlled.

The packet checker receives an incoming packet and checks it against a template held in the dual-port memory, while the incoming packet can be stored in the dual-port memory. Any mismatches can be flagged to the host computer. The packet checker is very useful for the automatic testing of packets from high data-rate instruments. When interface mode is used the SpaceWire router can be configured to automatically route packets from any SpaceWire port to the packet checker. Alternatively in router mode the packets must have a header byte which routes packets to the packet checker address.

3.1.2 Getting Started

Note: Before inserting the SpaceWire PCI Mk2 card, the software and driver programs should first be installed on the host computer operating system. This ensures that the SpaceWire PCI Mk2 card is recognised by the host when it boots up.

The SpaceWire PCI Mk2 card fits into any standard PCI slot on a typical desktop PC. The card can be inserted into a standard 3.3 or 5 Volt PCI system.

3.1.3 Interfaces

3.1.3.1 SpaceWire Ports

The SpaceWire port at the bottom of the board (i.e. closest to the PCI bus, or motherboard), is SpaceWire port 3 and the furthest away is port 1. The port numbering is shown below.

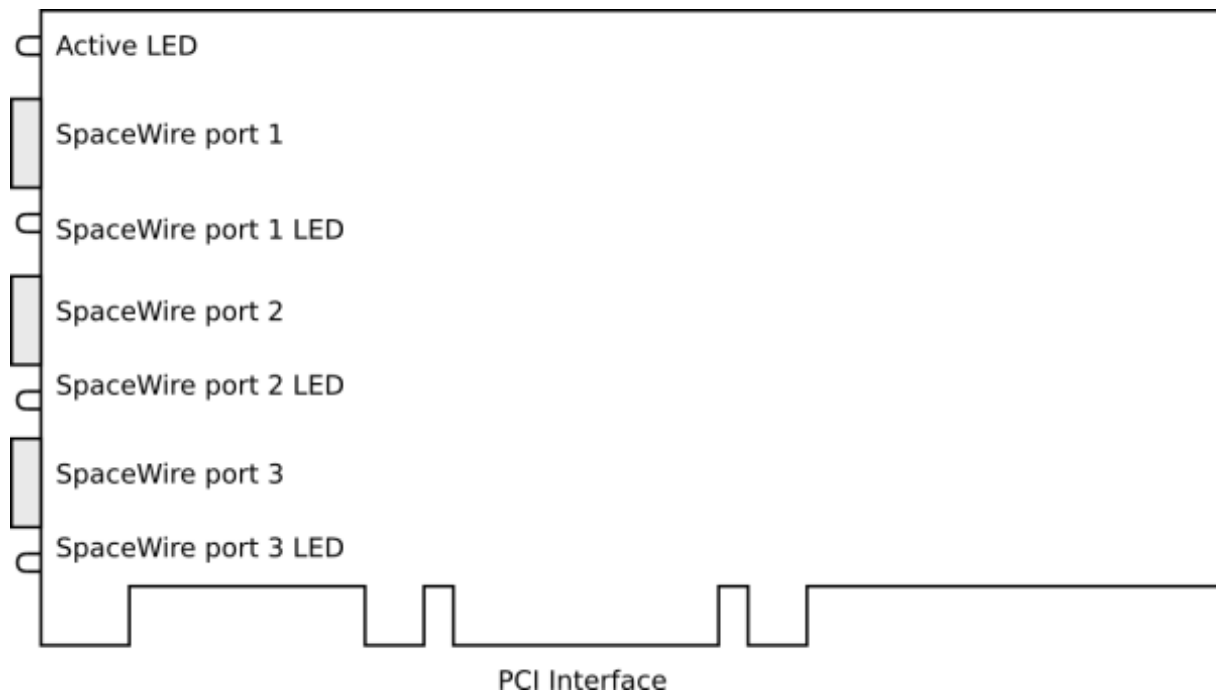


Figure 3.2: SpaceWire ports and LED positions

3.1.3.2 Front Panel LEDs

The PCI card has four front panel LEDs as shown in the image above. The LEDs are single colour red LEDs which are either on or off.

The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Mode	Description
----------	----------	-------------

No event	OFF	The LED is turned OFF when the link is not running and no errors are indicated.
Link running	ON	The LED is turned ON when the link is running. The link running LED is held for 200 ms after the link has stopped running and no error occurs.
Error	Fast flashing	The LED is flashed quickly at intervals of 100 ms.
Identify	Slow flashing	Identify mode is a convenience function which allows the user to visually determine which SpaceWire PCI board the software is talking to. The LED is flashed for a few seconds at intervals of 250 ms.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs flash red to indicate an error due to this noise. So, if you get a burst of noise on the input then you will see red flashes for around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.1.4 Functions

3.1.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire interfaces operate at a maximum rate of 200 Mbit/s.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 9 ports in total as listed below:

- 1 internal router configuration port
- 3 external SpaceWire ports
- 4 PCI interface external ports which are connected to the three SpaceWire ports and the configuration port
- 1 Packet generator/checker port

3.1.4.2 Interface Mode

In interface mode, a packet which is received on an external port, from a SpaceWire link or from the configuration port, will always be routed to the port specified by the Port Routing Register of the port. The SpaceWire PCI Mk2 card supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

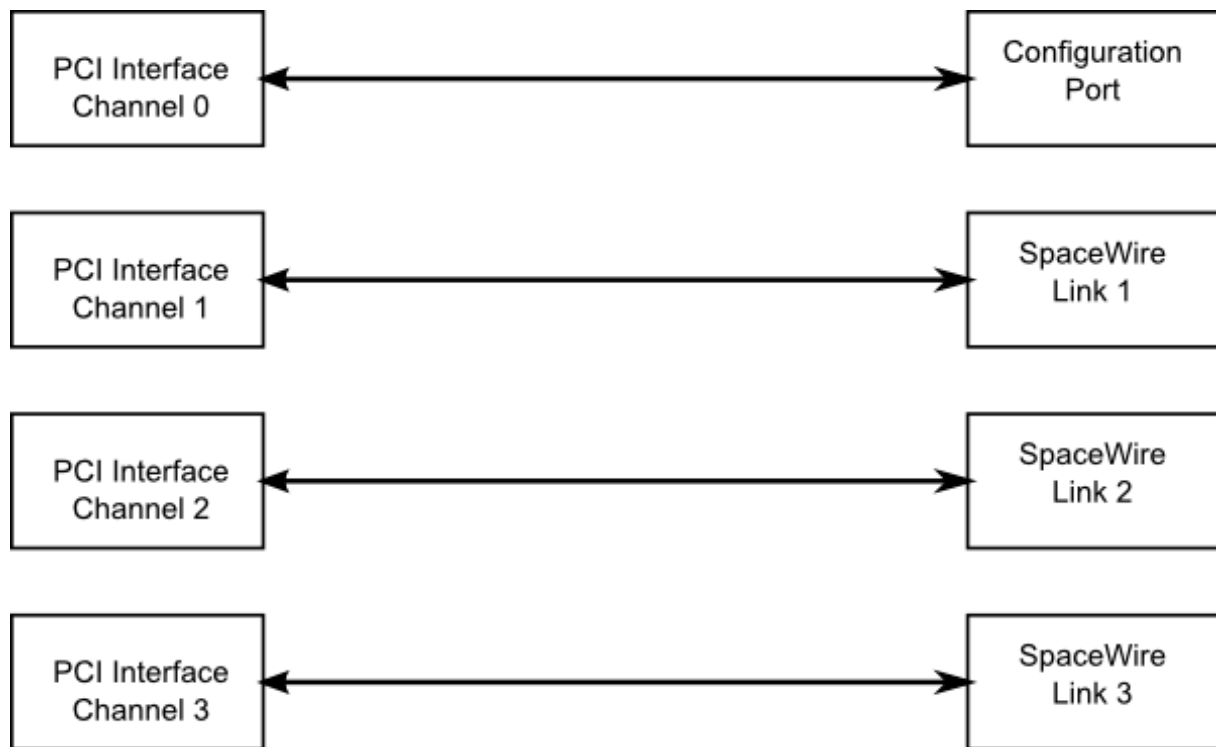


Figure 3.3: Interface mode routing connections

Source port	Destination port	Description
0	4	Configuration to external port 1 (host channel 0)
1	5	SpaceWire link 1 to external port 2 (host channel 1)
2	6	SpaceWire link 2 to external port 3 (host channel 2)
3	7	SpaceWire link 3 to external port 4 (host channel 3)
4	0	PCI interface channel 0 to configuration port
5	1	PCI interface channel 1 to SpaceWire link 1
6	2	PCI interface channel 2 to SpaceWire link 2
7	3	PCI interface channel 3 to SpaceWire link 3
8	1	Packet generator SpaceWire link 1

3.1.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source

port register.

- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.1.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

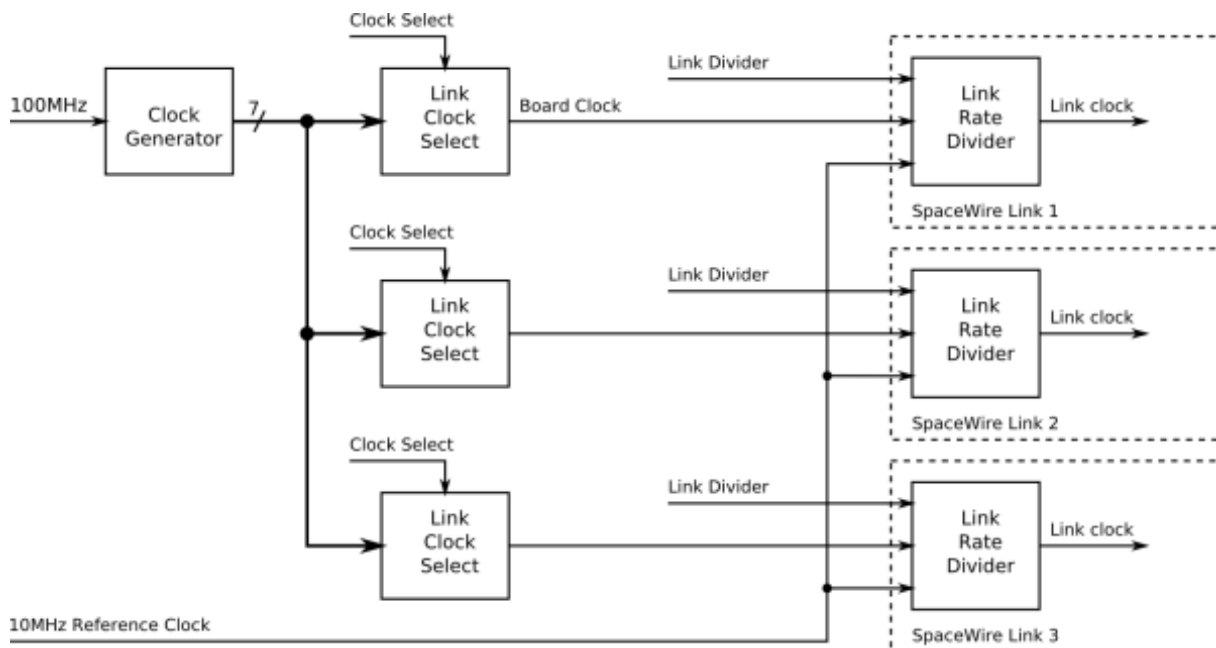


Figure 3.4: Link operating speed circuit diagram

The link operating speed is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link Clock Select (one for each SpaceWire link):
 - 120, 128, 140, 150, 160, 180, 200 MHz
- Link Rate Divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

The link rate can be programmed using the functions provided by the PCI Mk2 Configuration API and the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.1.4.5 Time-codes

A time-code can be received from the PCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external ports, only port 4, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

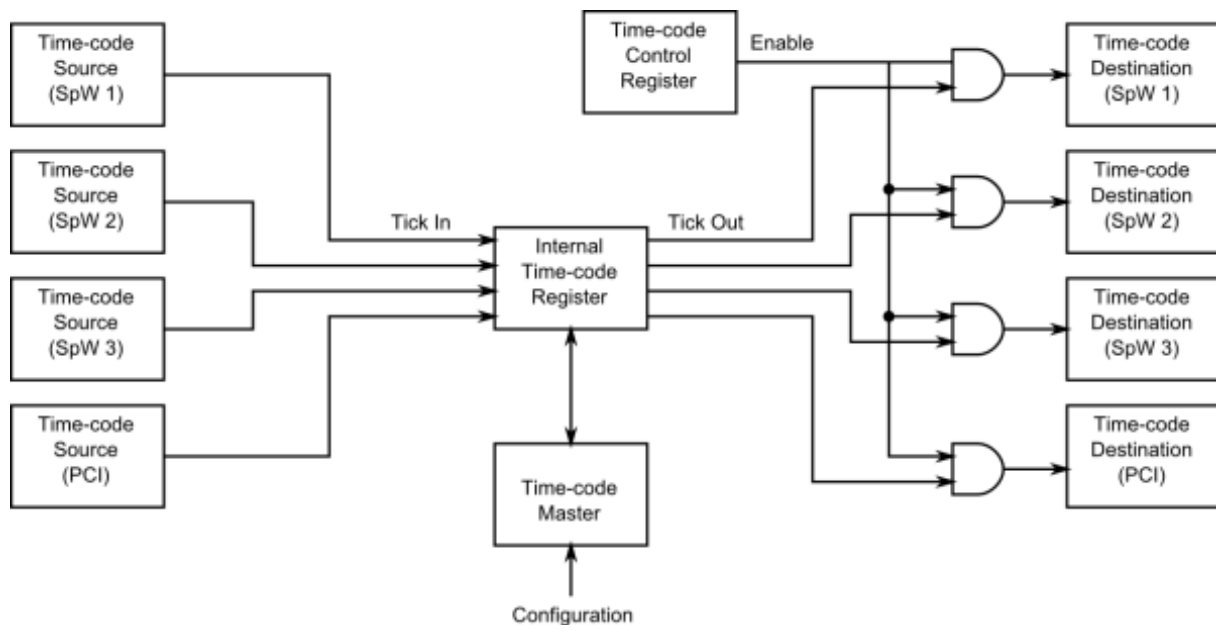


Figure 3.5: Time-code processing architecture

3.1.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3.8

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V _{ID} mV, Min	mV, Max	V _{ICM} V, Min	V, Max	V _{OD} mV, Min	mV, Max	V _{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2.8				

Dout, Sout					250	400	1.125	1.375
---------------	--	--	--	--	-----	-----	-------	-------

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

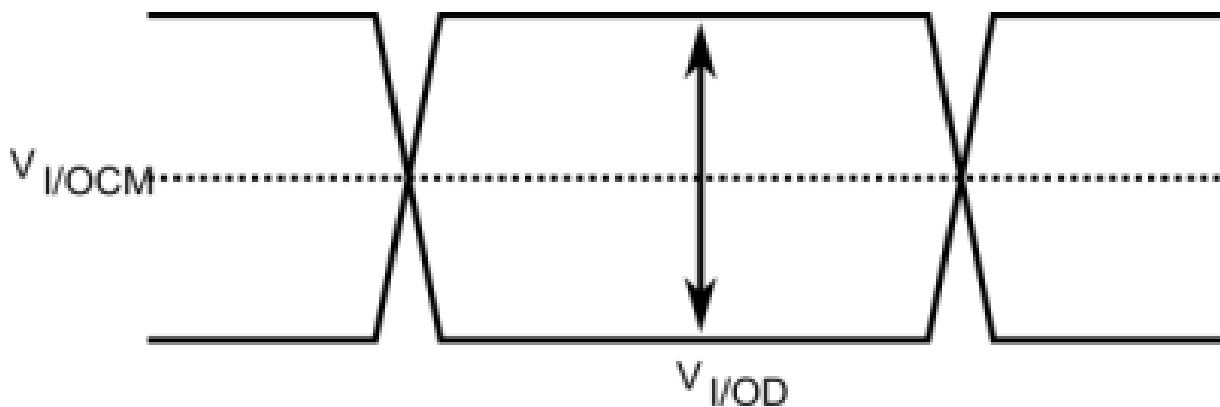


Figure 3.6: SpaceWire LVDS electrical specifications

3.1.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

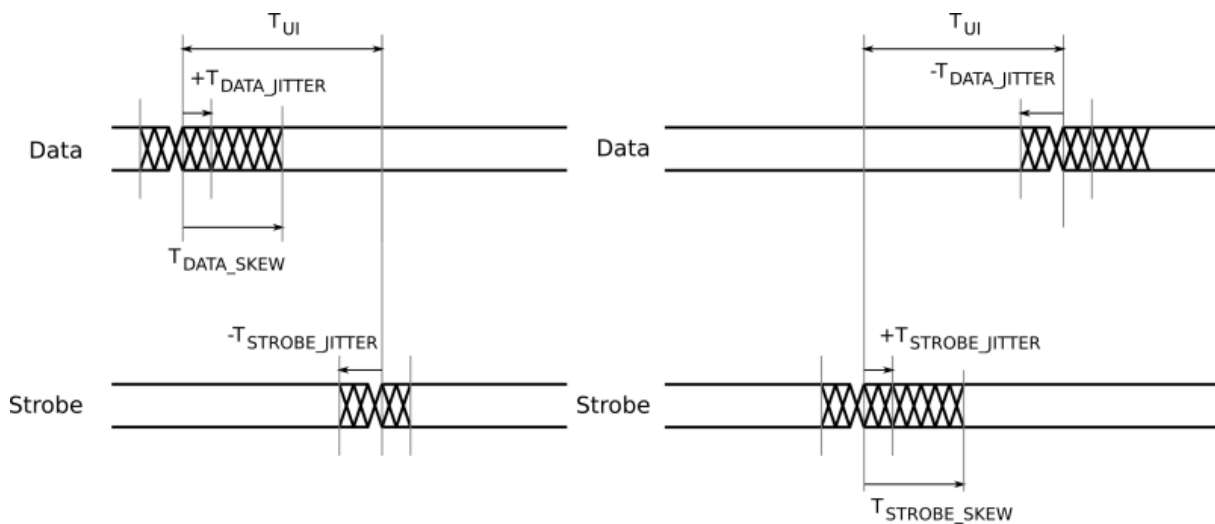


Figure 3.7: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.1.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.2 SpaceWire PCIe

Documentation for the STAR-Dundee SpaceWire PCIe device is provided in this section.

3.2.1 Overview

3.2.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire PCIe hardware.

For detailed information on using the SpaceWire PCIe card please consult the API reference documentation.

3.2.1.2 Hardware Description

The SpaceWire PCIe provides a PCIe to SpaceWire interface device with three SpaceWire interfaces. Efficient host software support is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire PCIe board is shown below.

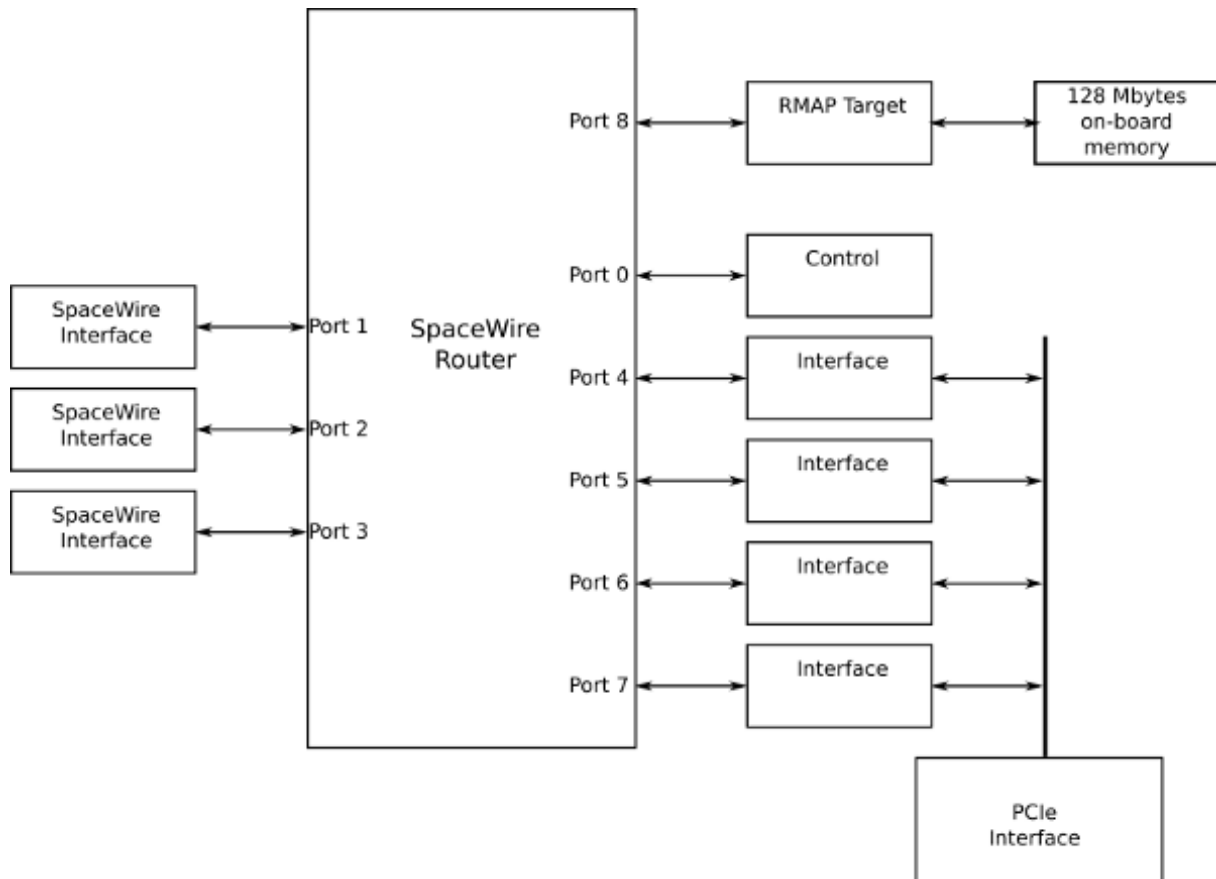


Figure 3.8: SpaceWire PCIe hardware overview

The three SpaceWire interfaces of the SpaceWire PCIe are fully compliant with the SpaceWire standard and operate at up to 300 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any another SpaceWire port, or the PCIe interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the PCIe channel for that link. There are three independent channels from the SpaceWire router to the PCIe interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the SpaceWire PCIe regardless of the data flow.

The single-lane PCIe interface is revision 1.1 compliant. It contains a DMA controller for rapid transfer of data to and from the SpaceWire PCIe board.

The SpaceWire PCIe includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.2.2 Getting Started

Note: Before inserting the SpaceWire PCIe card, the STAR-System software should first be installed on the host computer operating system. This ensures that the SpaceWire PCIe card is recognised by the host when it boots up.

The SpaceWire PCIe card fits into any standard PCIe slot on a typical desktop PC. It can fit into x1, x4 and x16 slots which are compliant to PCIe revision 1.1, 2 or 3. The card is powered entirely through the PCIe bus; no extra power supplies from the PC's ATX power supply are required.

3.2.3 Interfaces

3.2.3.1 SpaceWire Ports

The SpaceWire port at the bottom of the board (i.e. closest to the PCIe bus, or motherboard), is SpaceWire port 3 and the furthest away is port 1. The port numbering is shown below.

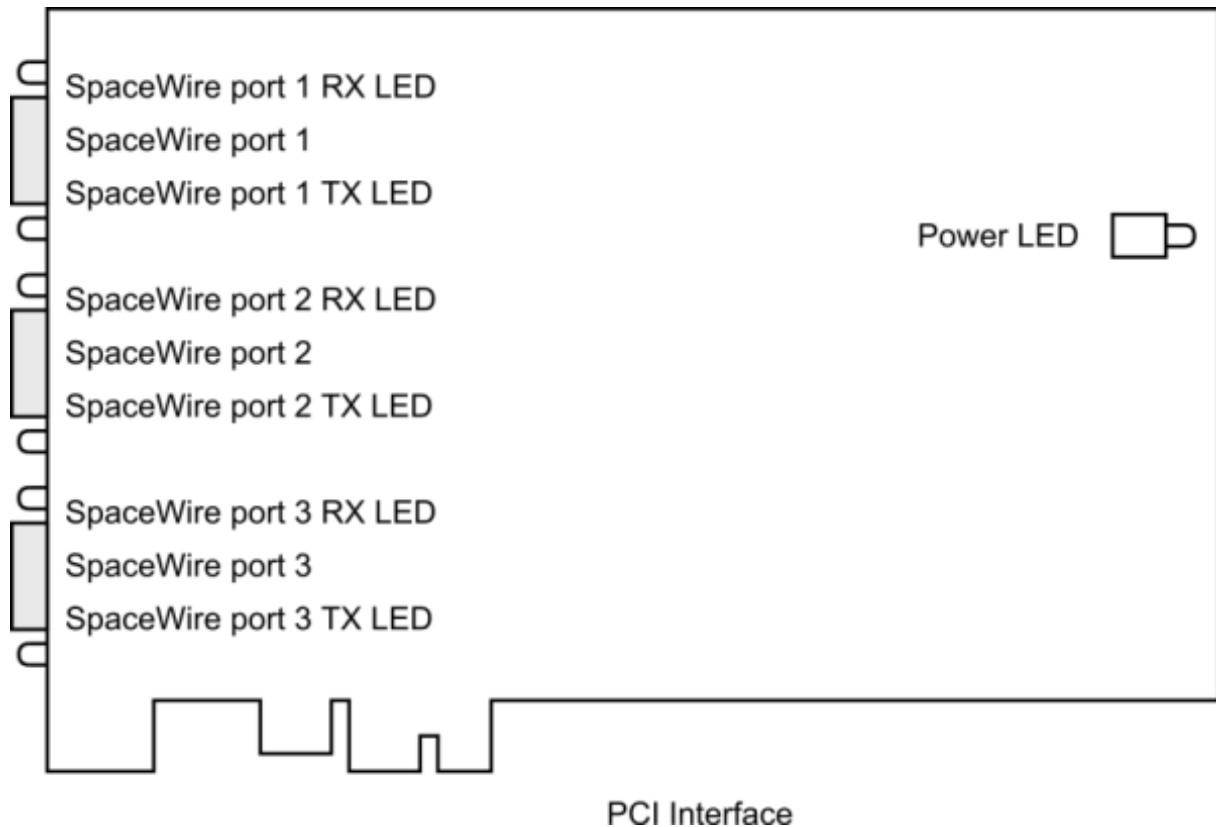


Figure 3.9: SpaceWire ports and LED positions

3.2.3.2 Power LED

When the PCIe card is powered up, the Power LED should glow a steady white. Should this LED glow, or flash any other colour, or fail to illuminate when power is applied to the board, then a hardware problem has occurred. Under these circumstances, the board must immediately be taken out of use and STAR-Dundee should be contacted for further instruction.

3.2.3.3 Front Panel LEDs

The PCIe card has six front panel tri-colour LEDs, two for each SpaceWire port. When viewed from the rear of the PC, the LED to the left of the link shows Received data, the LED to the right of the link shows Transmitted data.

The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	LED is steady amber.
Link running	Green	The LED is turned green when the link is running. The link running LED is held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The LED is turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire PCIe card can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.2.4 Functions

3.2.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire interfaces operate at a maximum rate of 300 Mbit/s.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 9 ports in total as listed below:

- 1 internal router configuration port
- 3 external SpaceWire ports
- 4 PCIe interface external ports which are connected to the three SpaceWire ports and the configuration port
- RMAP Target port

3.2.4.2 Interface Mode

In interface mode, a packet which is received on an external port, from a SpaceWire link or from the configuration port, will always be routed to the port specified by the Port Routing Register of the port. The SpaceWire PCIe card supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

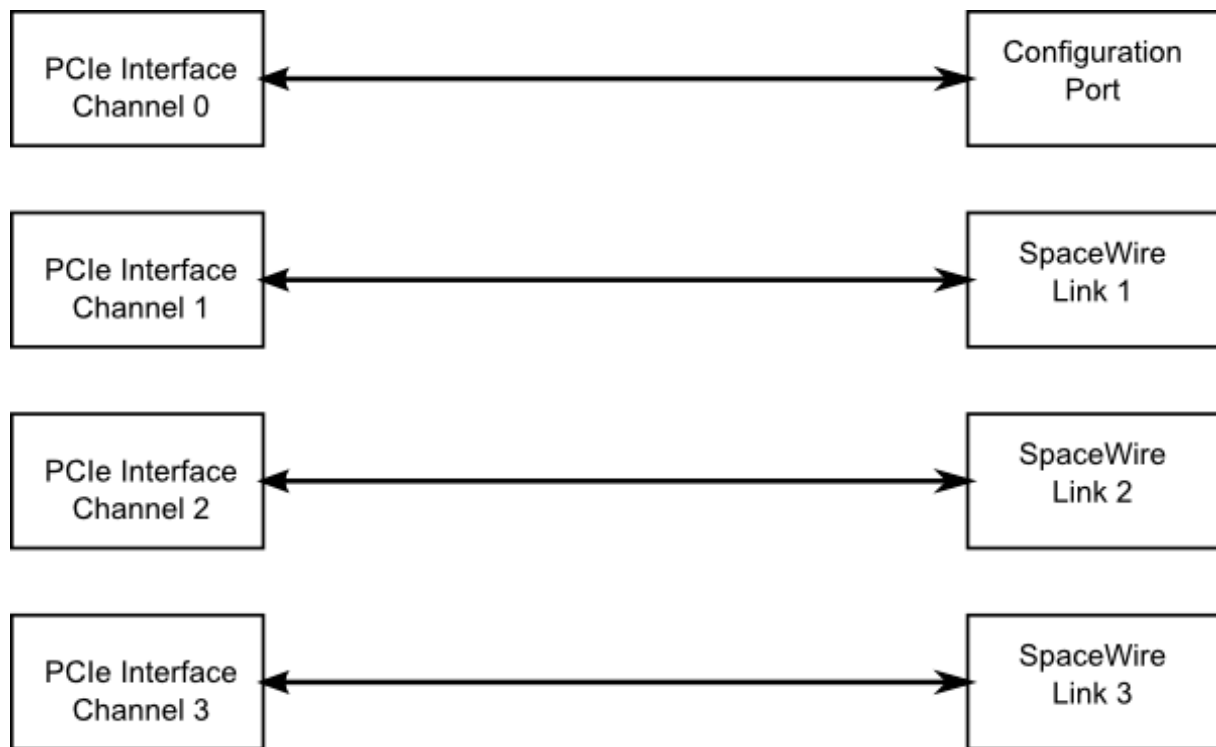


Figure 3.10: Interface mode routing connections

Source port	Destination port	Description
0	4	Configuration to external port 1 (host channel 0)
1	5	SpaceWire link 1 to external port 2 (host channel 1)
2	6	SpaceWire link 2 to external port 3 (host channel 2)
3	7	SpaceWire link 3 to external port 4 (host channel 3)
4	0	PCIe interface channel 0 to configuration port
5	1	PCIe interface channel 1 to SpaceWire link 1
6	2	PCIe interface channel 2 to SpaceWire link 2
7	3	PCIe interface channel 3 to SpaceWire link 3

3.2.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.

- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.2.4.4 Link Operating Speed

The link operating speed for each SpaceWire port is controlled by configuration parameters which can be set by the API or GUI application. The link clock rate selection logic for each port is shown in the figure below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

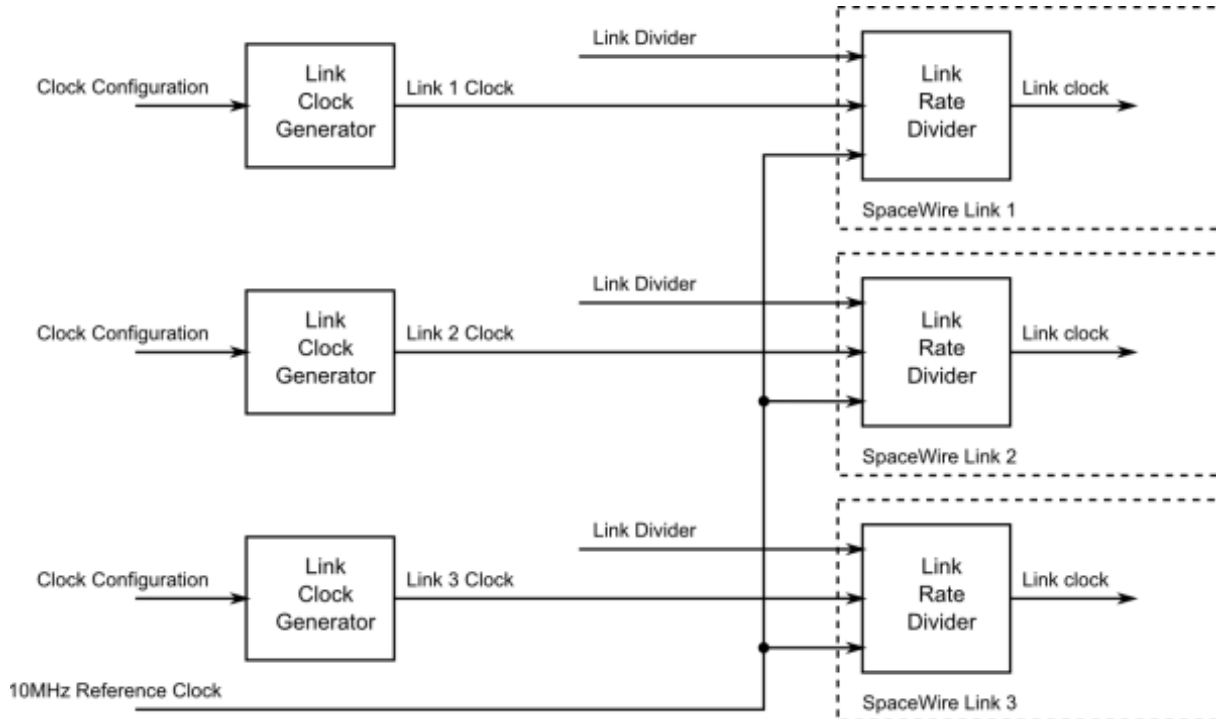


Figure 3.11: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using a programmable clock generator frequency. This is set by configuring a multiply and divide value to the input 100 MHz clock.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.2.4.5 Time-codes

A time-code can be received from the PCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external ports, only port 4, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

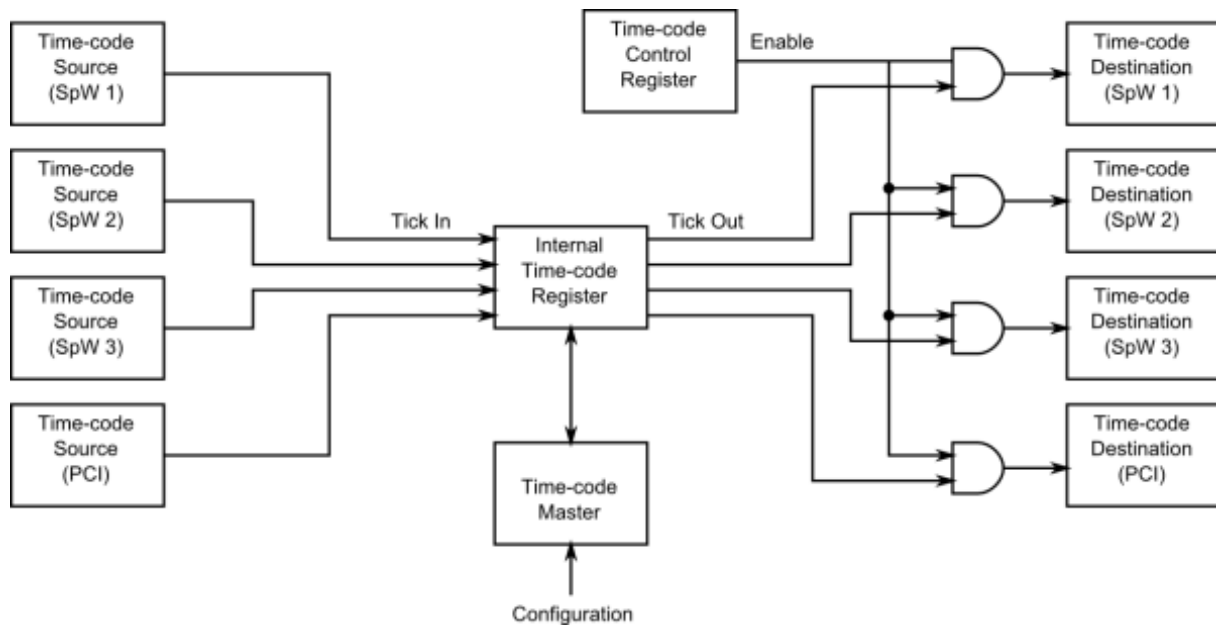


Figure 3.12: Time-code processing architecture

3.2.4.6 RMAP Target

The SpaceWire PCIe card includes an RMAP target port for rapid prototyping of a remote RMAP memory space. The port has access to 128 Mbytes of DDR3 memory. The port is accessible through the SpaceWire router allowing access from the PCIe interface or from the SpaceWire interfaces.

Supported authorisation parameters:

Target logical address: 0x00 - 0xFF	0xFE is the default target logical address
Protocol ID: 0x01	0x01 is the protocol ID for RMAP
Instruction:	The RMAP command type
Incrementing Read (command code 0b0011)	
Incrementing Write (command code 0b1001)	
Incrementing Write with reply (command code 0b1011)	
Read Modify Write (command code 0b0111)	
Key: 0x00 - 0xFF	The key allows the target application to filter commands
Initiator logical address: 0x00 - 0xFF	The logical address a reply should be sent to
Extended address: 0x02	To access the appropriate target address space
Data length: 0x0000_0000 - 0x00FF_FFFC	
Addressable range: 0x0000_0000 - 0xFFFF_FFFF	
Usable address range: 0x0000_0000 - 0x01FF_FFFF (addressed as 32 bit values)	

The bottom two bits of the data length field must be zero. Otherwise, the command will not be authorised.

All accesses are 32 bit aligned and the address in the RMAP command is interpreted as a word aligned address, i.e. address 0 and address 1 are separated by 4 bytes.

The external DDR3 memory range is 0x0000_0000 - 0x01FF_FFFF (128 Mbytes / 4)

The target does not support address range checking and will authorise access to any 32 bit address.

The extended address should be set to 0x02 for access to the 128 Mbyte external memory block.

3.2.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3.8

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V _{ID} mV, Min	mV, Max	V _{ICM} V, Min	V, Max	V _{OD} mV, Min	mV, Max	V _{OCM} V, Min	V, Max
Din, Sin	100	-	0.3	2.35				
Dout, Sout					247	454	1.125	1.375

V _{ID}	Input differential voltage
V _{ICM}	Input common mode voltage
V _{OD}	Output differential voltage
V _{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

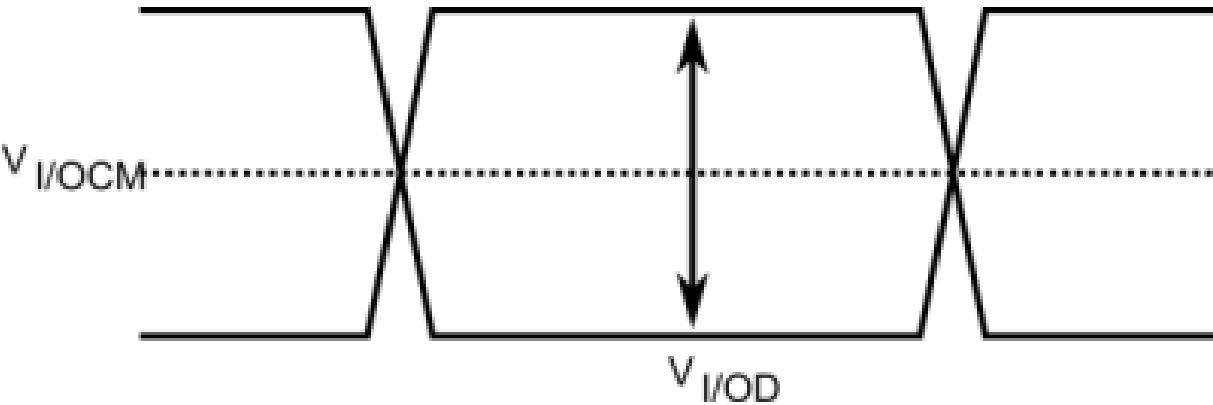


Figure 3.13: SpaceWire LVDS electrical specifications

3.2.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

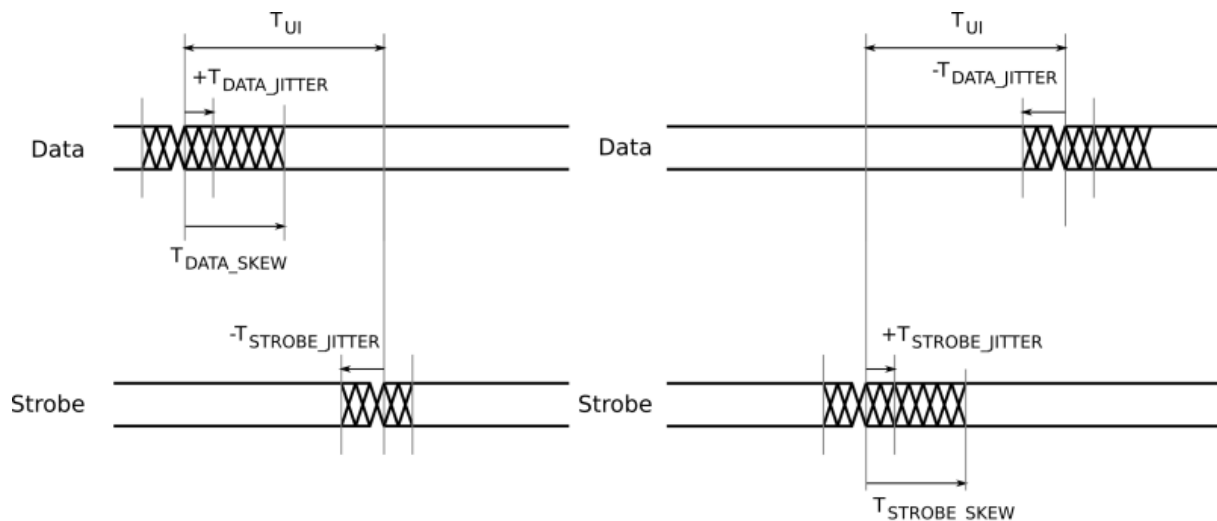


Figure 3.14: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$3 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.2.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.3 SpaceWire PCIe Mk2

Documentation for the STAR-Dundee SpaceWire PCIe Mk2 device is provided in this section.

3.3.1 Overview

This document provides an overview of the interfaces and features of the SpaceWire PCIe Mk2 card.

For detailed information on using the SpaceWire PCIe Mk2 card please consult the API reference documentation.

The PCIe Mk2 card is a versatile SpaceWire Interface, Router or simple RMAP Target capable endpoint. The card is designed to be an efficient host interface to a SpaceWire system or network using PCI Express.

It can take advantage of two bi-directional trigger connections on the card front panel to sequence events including the transmission of time-codes or data packets.

3.3.1.1 Hardware Description

The SpaceWire PCIe Mk2 provides a PCIe to SpaceWire interface device with three SpaceWire interfaces. Efficient host software is provided to support the rapid sending and receiving of SpaceWire packets into host PC memory.

The PCIe Mk2 card is shown in the figure below.

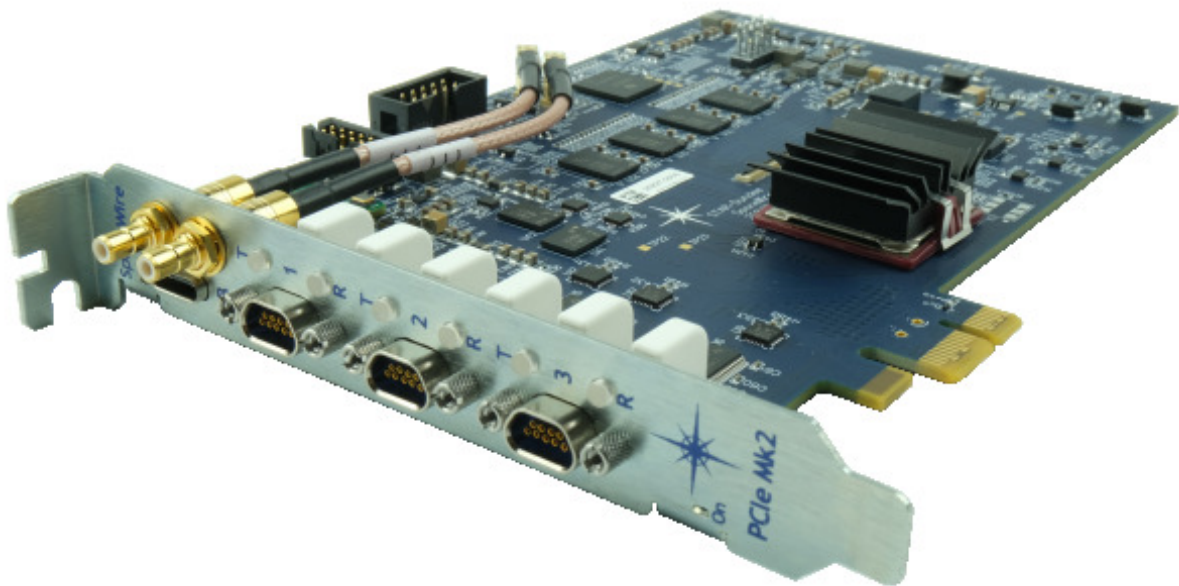


Figure 3.15: SpaceWire PCIe Mk2 card

The card is comprised of a front panel with three SpaceWire ports, two SMB trigger ports, and a set of LEDs used as status indicators. A x1 PCI Express connector is used to connect to a desktop PC with a compatible PCI Express slot.

A block diagram of the main functions provided by the card is shown below.

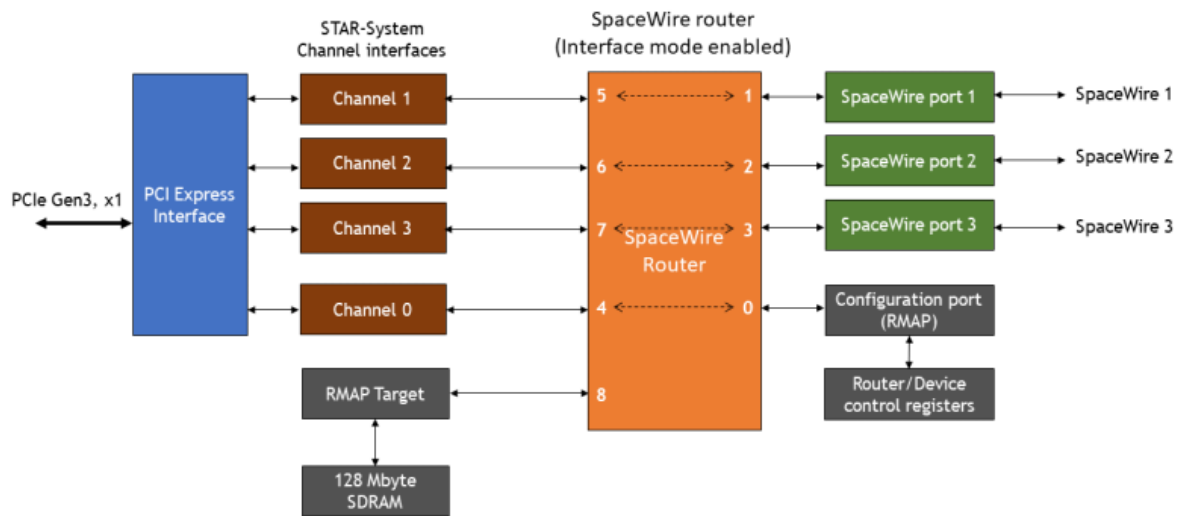


Figure 3.16: SpaceWire PCIe Mk2 hardware overview

The three SpaceWire interfaces of the SpaceWire PCIe Mk2 are fully compliant with the SpaceWire standard and operate at up to 400 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port or the PCIe interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the PCIe channel for that link. There are three independent channels from the SpaceWire router to the PCIe interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the SpaceWire PCIe Mk2 regardless of the data flow.

The single-lane PCIe interface is compliant with the PCI Express Base Specification revision 3.1. It contains a DMA controller for rapid transfer of data to and from the SpaceWire PCIe Mk2 board.

The SpaceWire PCIe Mk2 includes support for fault injection including support for parity, credit, and escape errors. Packet data corruption and EEP termination are also supported.

3.3.2 Getting Started

Note: Before inserting the SpaceWire PCIe Mk2 card, the STAR-System software should first be installed on the host computer operating system. This ensures that the host recognises the SpaceWire PCIe Mk2 card when it boots up.

The SpaceWire PCIe Mk2 card fits into any standard PCIe slot on a typical desktop PC. It is compatible with Gen-1, Gen-2, Gen-3 and Gen-4 PCIe slots of x1, x4, x8 and x16 widths. The card is powered entirely through the PCIe bus; no extra power supplies from the PC's ATX power supply are required.

3.3.3 Interfaces

The port numbering and LED position are shown in the figure below.

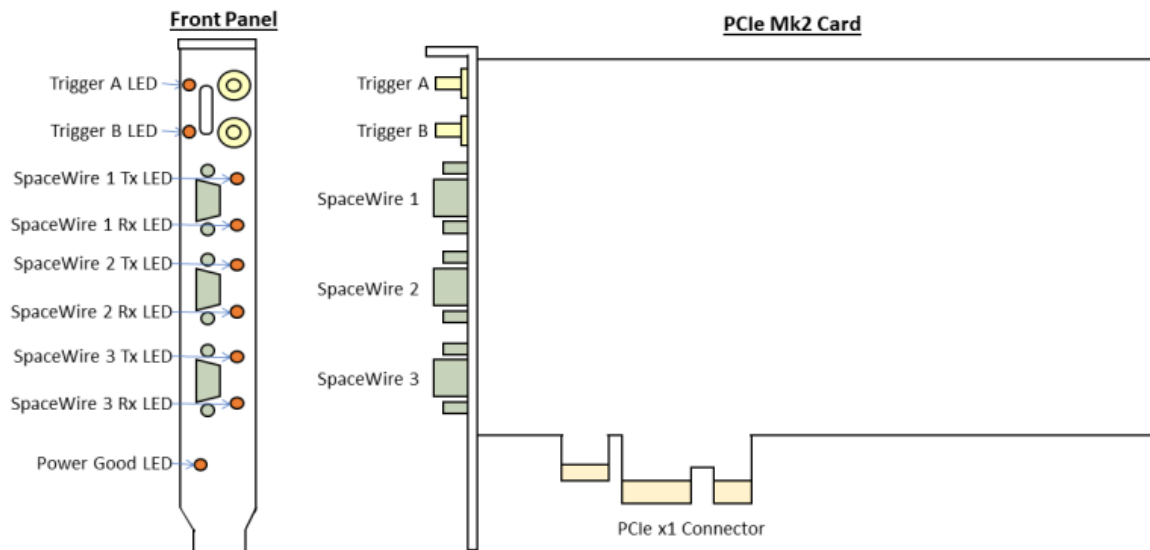


Figure 3.17: PCIe Mk2 card including SpaceWire port and LED positions

3.3.3.1 SpaceWire Ports

The PCIe Mk2 has three SpaceWire ports each operating at up to 400 Mbit/s. There are two independent, fully configurable, transmit clock generators; one for port 1 and the other for ports 2 and 3. Ports 2 and 3 therefore operate at the same transmit link speed. The receive speed automatically matches the speed of the incoming data. Each port is compliant with the ECSS SpaceWire standard. The connector used is the female 9-pin micro-D connector, as required by the SpaceWire standard.

The SpaceWire port at the bottom of the board (i.e. closest to the PCIe bus, or motherboard), is SpaceWire port 3 and the furthest away is port 1. There are two multicolour LEDs for each SpaceWire port, labelled T for Transmit and R for Receive. The R LED on each port shows received data for that port, and the T LED on each port shows transmitted data for that port. Together, the LEDs indicate the port status including link status and errors. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No Event	Amber	The T and R LEDs are steady amber.
Link Running	Green	The T and R LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.

Error	Red	The T and R LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the R LED turns blue. When data is transmitted on the link, the T LED turns blue.
No Credit	White	When no credit is available on the link for the PCIe Mk2 SpaceWire port to transmit, the T LED turns white. When the receive buffer at the far-end SpaceWire interface is blocked and it has not provided credit, the R LED turns white.
Identify	Flashing purple	The SpaceWire PCIe Mk2 card can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on unconnected inputs, or inputs which are connected but are not actively driven, can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid, it can cause the SpaceWire Interface to start up and then disconnect dependent on the control settings. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs.

The LVDS receivers on the card provide 25 mV of input voltage threshold hysteresis to avoid this happening all the time. This filters out most of the noise, but occasionally depending on the environment there may still be sufficient noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.3.3.2 Trigger Ports

The SpaceWire PCIe Mk2 card has two bi-directional SMB trigger interfaces which can be used to trigger actions such as sending a SpaceWire packet or indicate activity by providing a trigger output pulse.

An overview of the External Trigger circuit is shown in the figure below.

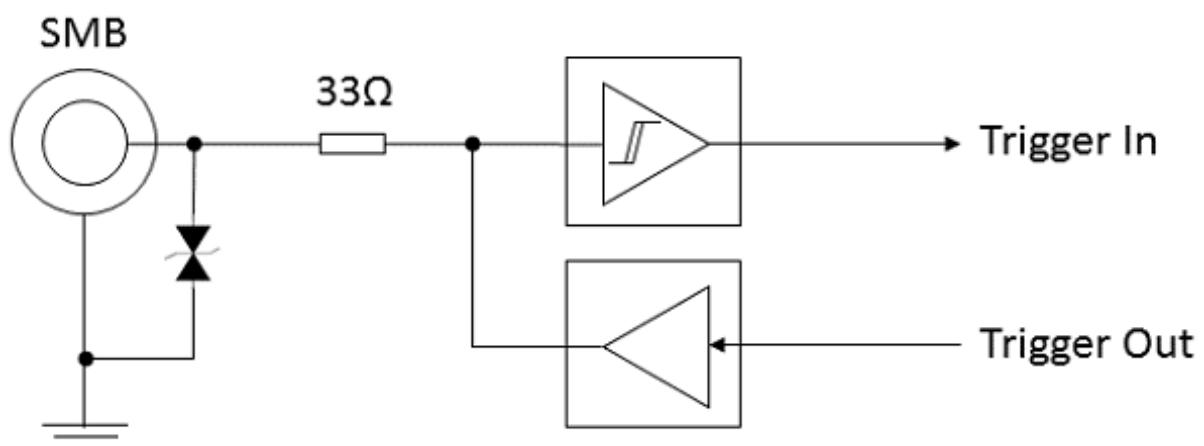


Figure 3.18: External trigger functional diagram

Two external SMB trigger interfaces are provided, which can be configured as input or output triggers. Each trigger is compatible with 3.3V logic levels including LVCMOS and LVTTTL. They are also 5V tolerant as noted in the absolute maximum and operating characteristics sections of this manual.

The triggers can be used for device synchronisation, triggering of SpaceWire packets and errors, or event signalling between cards or external equipment. The [Triggering Application](#) and [Triggering API](#) can be used to configure the triggers.

The trigger LEDs are set dependent on the trigger direction and the state of the trigger. When the trigger is an input the LED indicates if the trigger is enabled and when an external trigger occurs. When the trigger is an output the LED indicates if the trigger is armed and when the trigger is set. The LED colour map for the trigger LEDs is listed in the table below.

Function	LED Colour	Description
Not Active	Amber	Default state of the LED. The trigger is not enabled or armed.
Armed/Enabled	Green	Set to green when the trigger is an input and is enabled or when the trigger is an output and the trigger is armed.
Trigger	Blue	Set to blue when an input or output trigger occurs and the trigger is armed or enabled.

3.3.3.3 Power LED

When the PCIe Mk2 card is powered up, the Power LED should glow a steady green indicating power good. Should this LED glow or flash any other colour, or fail to illuminate when power is applied to the board, then a hardware problem has occurred. Under these circumstances, the board must immediately be taken out of use and STAR-↔ Dundee should be contacted for further instruction.

3.3.4 SpaceWire Router and SpaceWire Interfaces

3.3.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel/Microchip. The SpaceWire interfaces operate at a maximum rate of 400 Mbit/s.

Two modes of operation are supported by the Routing Switch, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 9 ports in total as listed below:

- 1 internal router configuration port
- 3 external SpaceWire ports
- 4 PCIe interface external ports which are connected to the three SpaceWire ports and the configuration port
- 1 RMAP Target port

3.3.4.2 Interface Mode

In interface mode, a packet which is received on an external port, from a SpaceWire link or the configuration port, will always be routed to the port specified by the Port Routing Register of the port. The SpaceWire PCIe Mk2 card supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

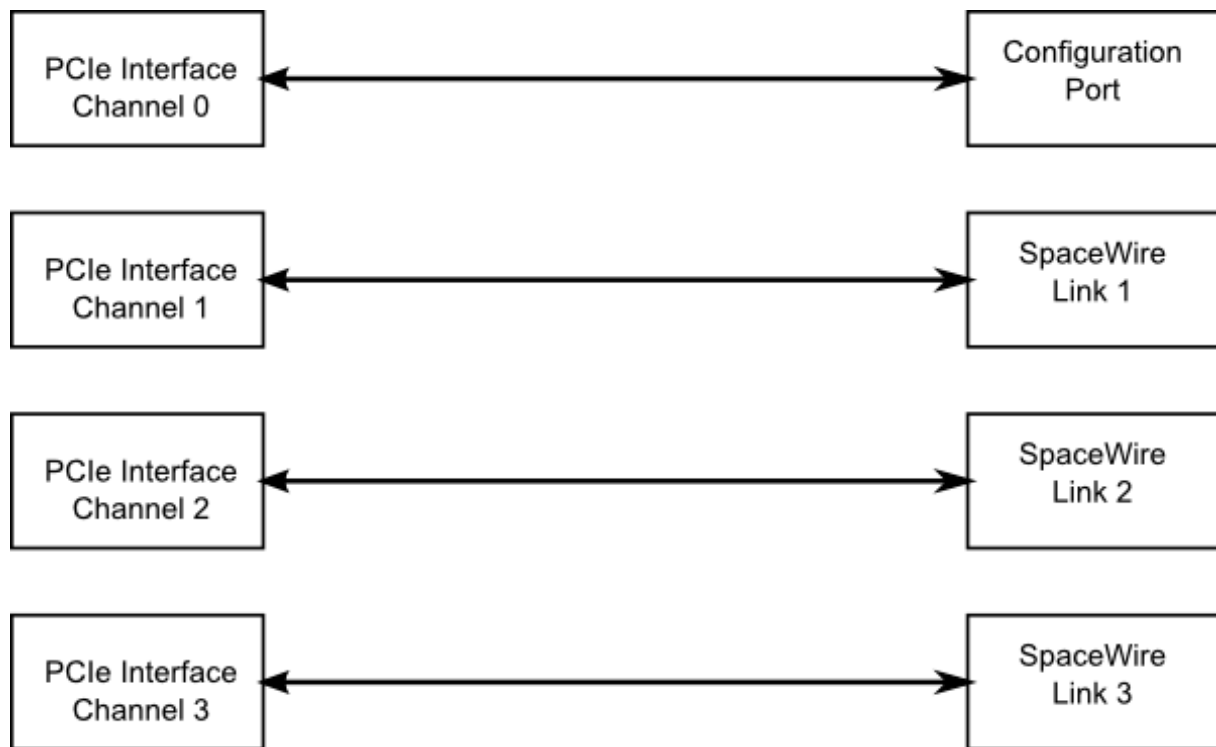


Figure 3.19: Interface mode routing connections

Source port	Destination port	Description
0	4	Configuration to external port 1 (host channel 0)
1	5	SpaceWire link 1 to external port 2 (host channel 1)
2	6	SpaceWire link 2 to external port 3 (host channel 2)
3	7	SpaceWire link 3 to external port 4 (host channel 3)
4	0	PCIe interface channel 0 to configuration port
5	1	PCIe interface channel 1 to SpaceWire link 1
6	2	PCIe interface channel 2 to SpaceWire link 2
7	3	PCIe interface channel 3 to SpaceWire link 3

3.3.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.

- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.3.4.4 Link Operating Speed

The SpaceWire port link operating speed is configurable between 2 and 400 Mbit/s. The speed can be set using the API or GUI application. An overview of the link clock generation logic is shown in the figure below. It consists of two reprogrammable clock generators which generate a frequency supporting 2-400 Mbit/s operation. A glitch-free multiplexer per-port ensures the link start-up frequency is always 10 Mbit/s.

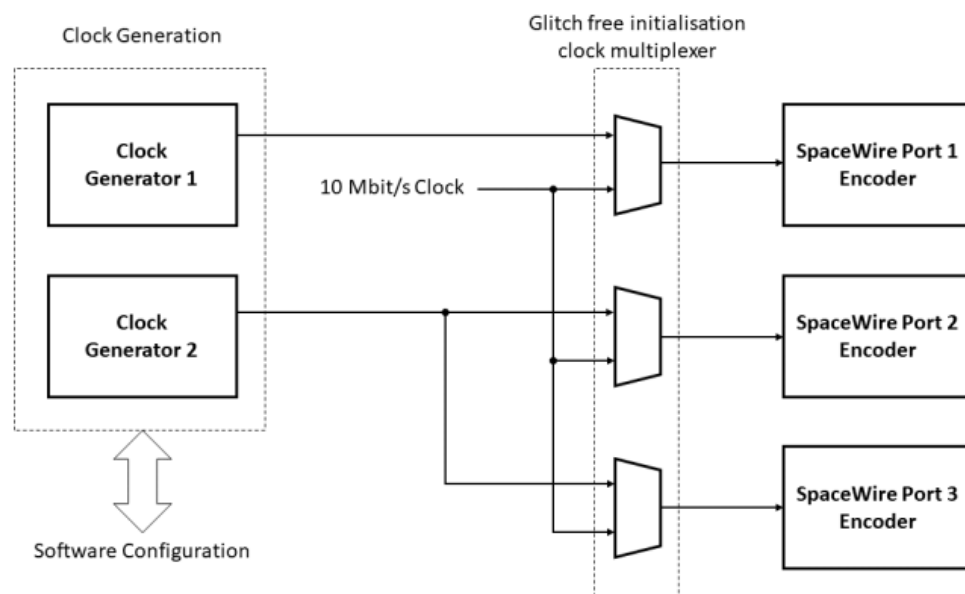


Figure 3.20: Link operating speed circuit diagram

After the PCIe Mk2 card is powered on or is reset, it defaults to a link operating speed of 200 Mbit/s after link start-up. During initialisation, the link speed is set to 10 Mbit/s.

SpaceWire port 1 uses a dedicated bit rate generator whereas SpaceWire ports 2 and 3 share a generator, meaning ports 2 and 3 operate at the same link speed except when one of the ports is initialising.

The link rate can be accessed and set using the [Device Configuration Application](#) or the functions provided by the [Generic Configuration API](#). Please refer to the API documentation for further details on how to set the clock speed.

3.3.4.5 Broadcast codes

The PCIe Mk2 supports reception and distribution of time-codes and distributed interrupt codes. Interrupt codes are supported and interrupt acknowledgement codes are discarded.

An overview of the Broadcast Controller is shown below. It is comprised of a set of enable registers to enable or disable reception or transmission per port, a time-code register with time-code master support, and an interrupt relay register.

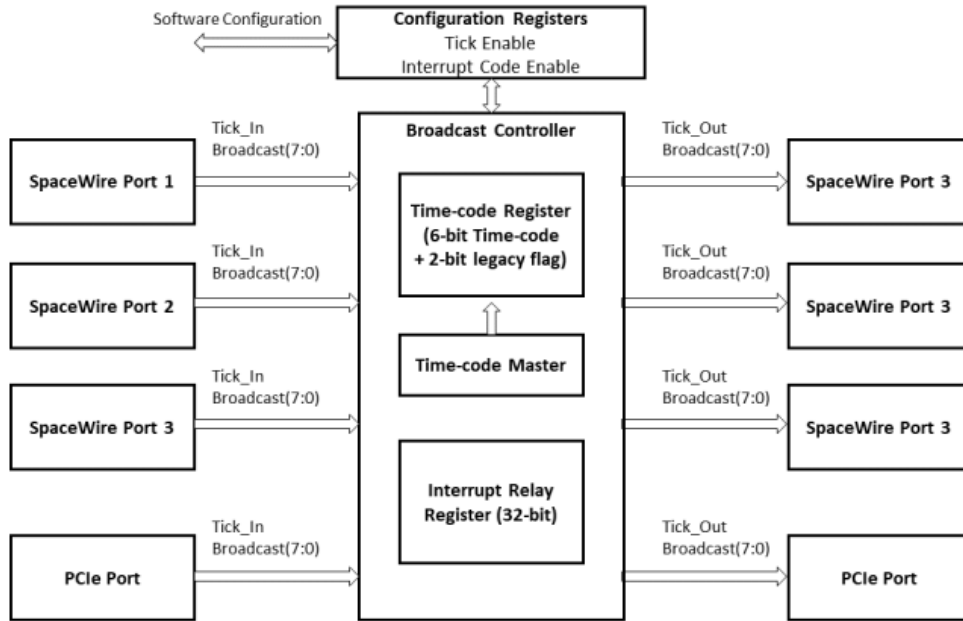


Figure 3.21: Broadcast code processing architecture

3.3.4.5.1 Broadcast code types

The Broadcast code type is identified using the top two bits of the broadcast code data field. The remaining 6-bits are interpreted dependent on the type field as listed below.

- A broadcast code with a type field 0b00 is interpreted as a time-code character.
- A distributed interrupt code is a broadcast code with a type field equal to 0b10.
- Types 0b01 and 0b11 are discarded except when [Broadcast Legacy Mode](#) is enabled.

3.3.4.5.2 Time-code Distribution

A time-code can be received from the PCI Express interface or a SpaceWire link. On reception of a time-code, either received by a SpaceWire interface or the PCIe interface time-code port, the time-code is passed to the time-code register. The time-code is acted upon dependent on the value of the tick enable register.

When [Broadcast Legacy Mode](#) is enabled, all broadcast code types are interpreted as time-codes and will be acted upon dependent on the time-code flags mode setting (see [Broadcast Controller](#) for the related API functions).

If the port is enabled to receive time-codes, the time-code register is updated and the time-code is forwarded if it is one more than the current value modulo 64. The value is forwarded to all ports except the port on which the time-code arrived. Ports which are not enabled to send or receive time-codes will not forward the time-code.

If [Time-code Register Bypass Mode](#) is enabled the value of the time-code register is always updated and the time-code will be forwarded to all ports except the port on which the time-code was received.

For time-code related API functions, including that used to access the time-code value register, see [Time-codes](#).

The router has a time-code master which can be configured to generate periodic time-code ticks. The generator is accurate to the 20 PPM onboard reference clock. An alternative method is available to generate time-code ticks with greater accuracy using the external trigger interface and the [Triggering API](#).

3.3.4.5.3 Interrupt Code Distribution

On reception of an interrupt code, received on a SpaceWire port or PCIe interface, the code is passed to the interrupt relay register. The interrupt is not acted on if the port is not enabled to receive distributed interrupt codes.

When the corresponding bit in the interrupt relay register is 0 the bit will be set to 1. A timeout timer is started to clear the interrupt after a timeout period and the interrupt is forwarded on all ports except the port on which the interrupt code arrived. Ports which are not enabled to send or receive interrupt codes will not forward the interrupt code.

A received interrupt code will be discarded if the interrupt bit is already set and the interrupt will not be forwarded. Receiving an interrupt code for a relay register bit which is already set indicates the period between interrupts should be increased.

The interrupt relay timeout is set to a fixed value of 500 microseconds.

3.3.4.5.4 Broadcast Legacy Mode

The PCIe Mk1 interpreted all broadcast code types as time-code as defined in the ECSS-E-ST-50-12C standard, whereas the PCIe Mk2 can support time-code and distributed interrupt code types as defined in the ECSS-E-ST-50-12C Rev.1 standard.

A legacy mode is available in the PCIe Mk2 which will enable the Mk1 behaviour to force all codes to be interpreted as time-codes.

If legacy mode is enabled, the broadcast code type is interpreted as a set of flags which are forwarded with the time-code. The Time-flags mode setting determines how the flags are interpreted, as listed below.

- When 0, the time-code flags are forwarded with the time-code value if it is one more (modulo 64) than the time-code register value.
- When 1, a time-code is discarded if its flags field is not equal to 0b00.

3.3.4.5.5 Time-code Register Bypass Mode

The time-code register can be programmed in bypass mode. On reception of a time-code in bypass mode, the following actions will occur.

- The time-code register is updated.
- The time-code is forwarded through all ports which are enabled to send time-codes except the port on which the time-code was received.

When enabled, a time-code which is received will be forwarded out of the other ports on the Router, without checking against the current value of the time-code register.

The Generic Configuration API [Time-codes](#) functions can be used to enable and disable time-code bypass mode.

3.3.4.6 Packet Timestamping

The SpaceWire PCIe Mk2 provides an optional timestamp feature for each received packet. A timestamp will be recorded when the first byte of a packet is received and when the end of packet marker is received. The timestamp is synchronised with an external pulse input on SMB trigger A.

Multiple SpaceWire PCIe Mk2 devices can be synchronised using the external pulse external time source to synchronise time across the devices.

3.3.4.6.1 Detailed Description

An overview of the timestamp functions provided by the SpaceWire PCIe Mk2 is shown in the figure below. Configuration settings are shown in *italics*.

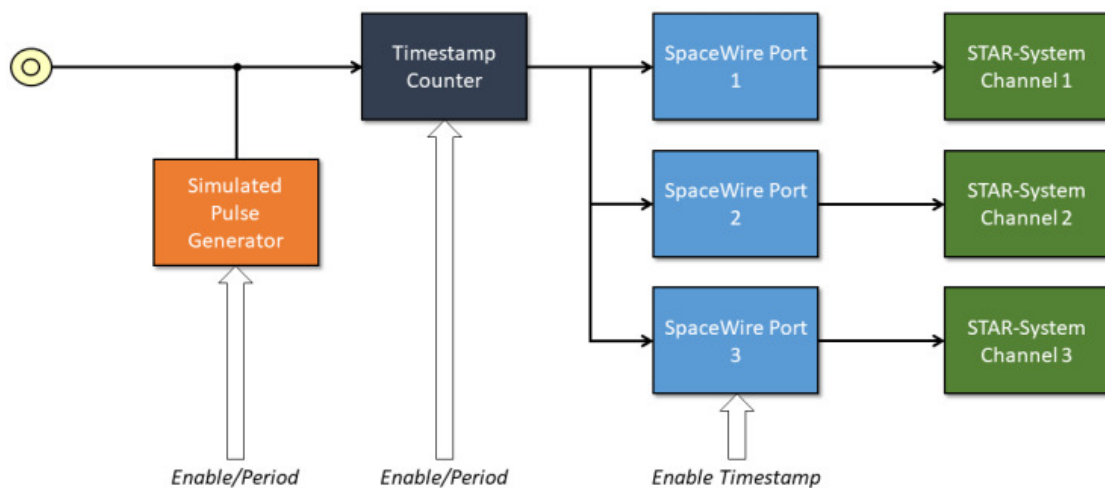


Figure 3.22: Timestamp system overview

Each SpaceWire port has a SpaceWire receiver which decodes serial data into link characters and data characters. Each data character can be an 8-bit packet data byte or an end of packet marker. The timestamping mechanism will record the timestamp for each start of packet byte and for each end of packet marker, effectively the start time and end time of each packet.

The timestamp counter is implemented as two counters, one which increments on the input SMB trigger or the pulse generator and one which increments on the system clock. The counters are identified as "SyncPulse" and "ClockCycles" respectively.

The packet start and end times are recorded using a combination of the counters described above, plus a capture register which captures the total number of clock cycles which elapsed in the previous period. An overview of the counter and register operation is listed below.

- The "SyncPulse" counter can be loaded by software to provide a starting value for the timestamp. The counter will increment on each trigger pulse.
- The "ClockCycles" counter is incremented on each system clock cycle and is reset on each pulse of the trigger. It provides a count of the system clock cycles which have elapsed since the last synchronisation pulse.
- The "TotalCycles" register is captured into a storage register on each synchronisation pulse to provide a mechanism to convert the "ClockCycles" elapsed time into fractions of the synchronisation period, i.e. $\frac{\text{ClockCycles}}{\text{TotalCycles}}$ is the fraction between 0 and 1 of the synchronisation period at which the event occurred.

The timestamp is synchronised using an external trigger pulse which supports frequencies equal to or greater than 1 Hz. The mechanism provides additional meta-data for each received SpaceWire packet which can be received by applications to compute the time at which the packet was decoded by the SpaceWire receiver.

A simulated pulse generator module can be used to simulate the external SMB trigger for testing and development purposes. The generator is programmed with the period between pulses in system clock cycles, and can be enabled or disabled.

3.3.4.6.2 Computing the "User" time from the recorded Timestamp

A method to convert the timestamp fields into "User" time is listed below. The raw timestamp is returned by the API therefore user applications can implement a different algorithm if required.

The synchronisation pulse frequency is set by the user application. In the equations below the frequency is referred to as F_{sync} .

The computed time is defined as x.y where x is the number of seconds and y indicates the fraction of seconds between 0 and 1.

The example steps to compute the "User" time are listed below.

1. The first step is to convert the hardware format into seconds and sub seconds as shown below.

$$\begin{aligned} Seconds &= SyncPulse / F_{sync} \\ SubSeconds &= ClockCycles / (TotalCycles * F_{sync}) \end{aligned}$$

2. The resultant values are then added together.

$$x.y = Seconds + SubSeconds$$

An example case is listed below.

1. The application setup is listed below:

- (a) F_{sync} is 10 Hz
- (b) "SyncPulse" counter is 1234
- (c) "ClockCycles" counter is 234567
- (d) "TotalCycles" counter is 301210

2. The resultant time in seconds and fractions of seconds is computed as:

$$\begin{aligned} Seconds &= 1234/10 \\ &= 123.4 \\ \\ SubSeconds &= \frac{234567}{301210 * F_{sync}} \\ &= 77.8749045516E - 3 \\ \\ x.y &= 123.477874905seconds \end{aligned}$$

3.3.4.7 SpaceWire Interface Tristate Mode

The SpaceWire ports on the PCIe Mk2 card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/↔ Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the Tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Interface Mode, Start, Disabled, or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.

If a link on the PCIe Mk2 card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

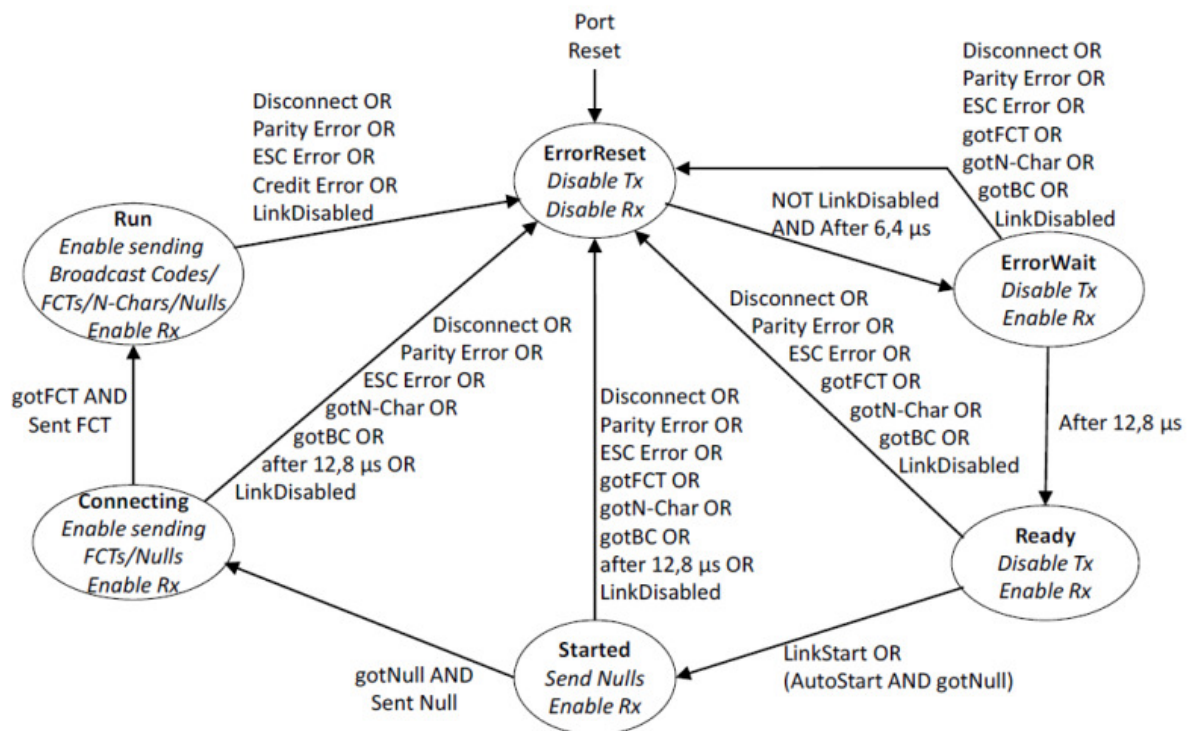


Figure 3.23: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled. The PCIe Mk2 will also disable the Tristate mode of a link when Interface Mode is enabled and software sends a packet to the port over the corresponding channel interface.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

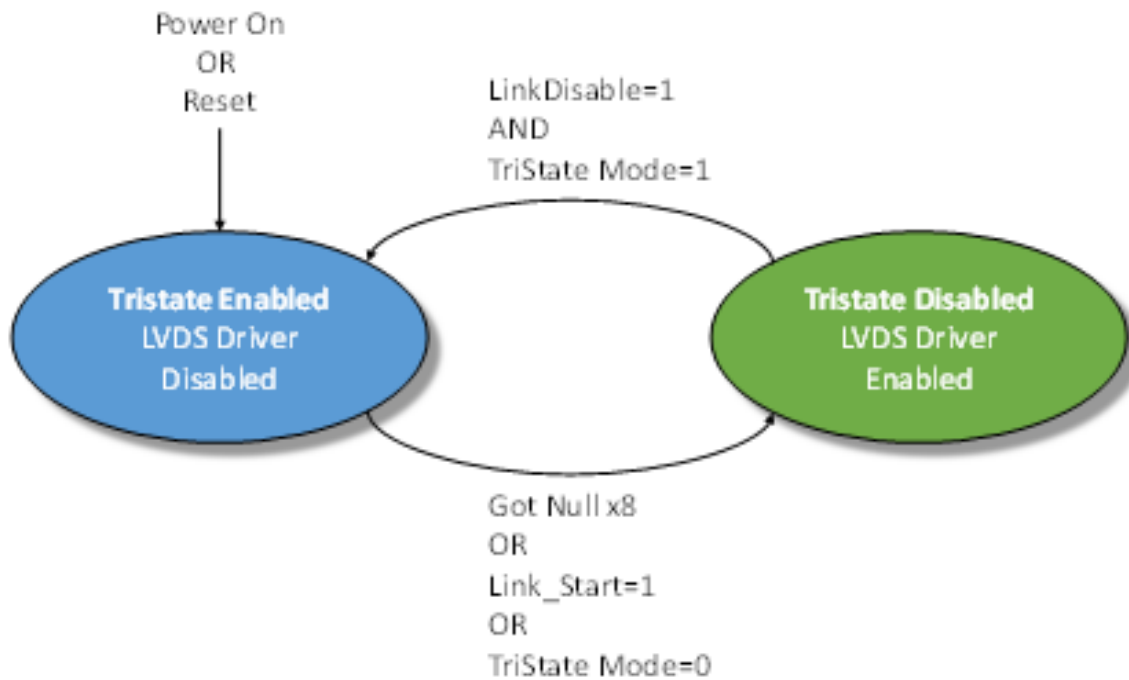


Figure 3.24: Tristate controller state machine

3.3.5 RMAP Target

The SpaceWire PCIe Mk2 card includes an RMAP target port for rapid prototyping of a remote RMAP memory space. The port has access to 128 Mbytes of DDR3 memory. The port is accessible through the SpaceWire router allowing access from the PCIe interface or from the SpaceWire interfaces.

The table below details the supported authorisation parameters.

Parameter	Value or range	Notes
Target logical address	0x00 - 0xFF	0xFE is the default target logical address.
Protocol ID	0x01	0x01 is the protocol ID for RMAP.
Instruction	Incrementing Read (command code 0b0011) Incrementing Write (command code 0b1001) Incrementing Write with reply (command code 0b1011) Read Modify Write (command code 0b0111)	Supported RMAP command types.

Key	0x00 - 0xFF	The key allows the target application to filter commands although is not checked.
Initiator logical address	0x00 - 0xFF	The logical address a reply should be sent to.
Extended address	0x02	To access the appropriate target address space.
Data length	0x0000_0000 - 0x00FF_FFFC	
Address range	0x0000_0000 - 0xFFFF_FFFF	
Usable address range	0x0000_0000 - 0x01FF_FFFF	Addressed as 32 bit values.

Notes:

- The bottom two bits of the data length field must be zero. Otherwise, the command will not be authorised.
- All accesses are 32 bit aligned and the address in the RMAP command is interpreted as a word aligned address, i.e. address 0 and address 1 are separated by 4 bytes.
- The external DDR3 memory range is 0x0000_0000 - 0x01FF_FFFF (128 Mbytes / 4).
- The target does not support address range checking and will authorise access to any 32 bit address.
- The extended address should be set to 0x02 for access to the 128 Mbyte external memory block.

3.3.6 Absolute Maximum Ratings

The SpaceWire port absolute maximum ratings are listed in the table below. Maximum voltages which are listed as To Be Announced (TBA) are being characterised and will be updated in a future STAR-System release. If you need further information, please contact us at support@star-dundee.com.

	Description	Min	Max	Units
V _{IN}	SpaceWire port Din+, Din-, Sin+, Sin- input voltage relative to GND	-0.5	TBA	V
	SpaceWire port Dout+, Dout-, Sout+, Sout- input voltage relative to GND	-0.5	TBA	V

The external trigger absolute maximum input ratings are defined in the table below. The device can withstand these maximum values but use at or near these values is not recommended.

Note that each trigger is only 5 Volt input tolerant when configured as an input (the default on power up), or when the device is powered down. Triggers configured as outputs are not 5 Volt tolerant, and may be damaged if an external voltage is applied to them. It is recommended to use 3.3 Volt logic when using the triggers.

Maximum voltages which are listed as To Be Announced (TBA) are being characterised and will be updated in a future STAR-System release. If you need further information, please contact us at support@star-dundee.com.

	Description	Condition	Min	Max	Units
V _{IN}	Voltage applied to trigger	Trigger configured as an input, or device powered down	-0.5	TBA	V

		Trigger configured as an output	-0.5	TBA	V
I _{OUT}	Continuous output current	Trigger configured as an output	-50	+50	mA

3.3.7 Recommended Operating Conditions

The SpaceWire port LVDS DC characteristics are listed in the table below. Maximum voltages which are listed as To Be Announced (TBA) are being characterised and will be updated in a future STAR-System release. If you need further information, please contact us at support@star-dundee.com.

Parameter	Min	Typical	Max	Units
Magnitude of differential input voltage (V _{id})	0.1		1	V
Input voltage range (any combination of common-mode or input signals)	0		TBA	V
SpaceWire input differential resistance	100			Ω
Output differential magnitude - 100 Ω load	247	350	454	mV
Output common mode range - 100 Ω load	1.125		1.375	V

The SMB Trigger DC characteristics are listed in the table below. Maximum voltages which are listed as To Be Announced (TBA) are being characterised and will be updated in a future STAR-System release. If you need further information, please contact us at support@star-dundee.com.

Parameter	Min	Max	Units
Trigger in. Low-level input voltage	0	0.8	V
Trigger in. High-level input voltage	2	TBA	V
Trigger out. Low-level output voltage (100 μA load)		0.1	V
Trigger out. High-level output voltage (100 μA load)	3.2		V
Trigger out. Low-level output voltage (24 mA load)		0.55	V
Trigger out. High-level output voltage (24 mA load)	2.3		V

3.3.8 Switching Characteristics

This section lists the encoder and decoder switching characteristics which are common for each SpaceWire Interface.

A SpaceWire serial interface is comprised of two signals in each direction, Data and Strobe. SpaceWire characters are encoded directly to the Data line including a parity and control bit per character. Strobe will toggle when Data does not change, providing a method to recover the serial clock using an XOR operation in the receiving device.

Differences in propagation delay between Data and Strobe, also known as the skew, will reduce the recovered clock pulse width and reduce margin when capturing data transitions. The ideal propagation delay is affected by the serial encoder, and by external factors including the LVDS drivers and the cable properties. The encoder and decoder are designed to provide a margin for skew and jitter as characterised in the sections below.

3.3.8.1 Encoder Switching Characteristics

The SpaceWire encoder switching characteristics are listed in the table below.

Parameter	Description	Min	Max	Units
$F_{ENC_BIT_RATE}$	Encoder bit rate range	2	400	Mbit/s
T_{ENC_SKEW}	Data or Strobe skew		350	ps
T_{ENC_JITTER}	Data or Strobe peak-to-peak jitter		110	ps

3.3.8.2 Decoder Switching Characteristics

The SpaceWire interface decoder characteristics are defined in the figure and table below.

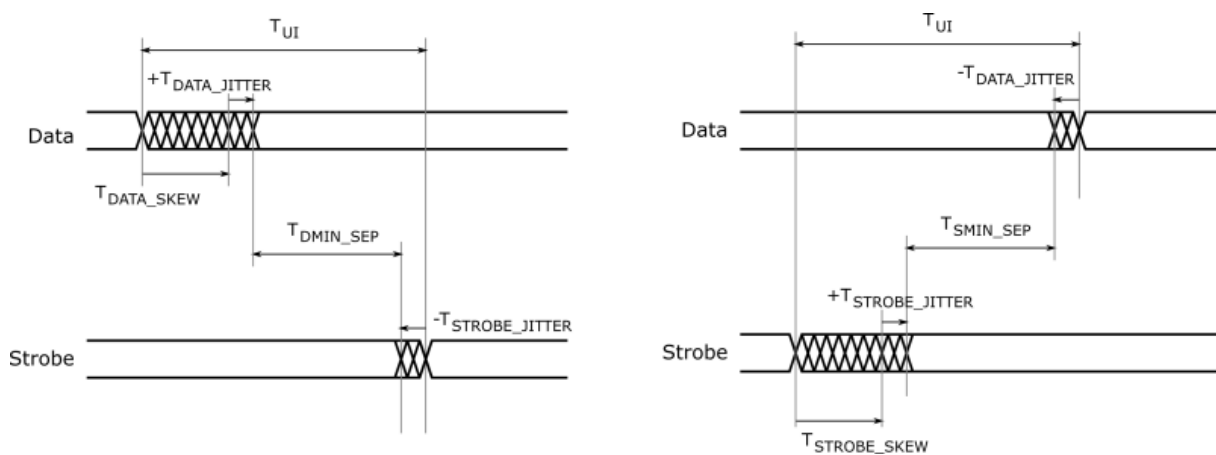


Figure 3.25: SpaceWire decoder switching characteristics

The minimum separation between data and strobe, characterised as T_{dmin_sep} and T_{smin_sep} , should be greater than zero when skew and jitter are taken into account.

Parameter	Description	Min	Max	Units
T _{UI}	Decoder bit period	2.5	500	ns
T _{DEC_DMIN_SEP}	Minimum separation between data and strobe when data is skewed relative to strobe	1.88		ns
T _{DEC_SMIN_SEP}	Minimum separation between data and strobe when strobe is skewed relative to data	1.88		ns
T _{DEC_MARGIN_400}	Skew and jitter margin at 400 Mbit/s (2.5 ns period)		0.62	ns
T _{DEC_MARGIN_200}	Skew and jitter margin at 200 Mbit/s (5 ns period)		3.12	ns

3.3.9 FMEA Compliant

The SpaceWire PCIe Mk2 incorporates extensive fault protection to meet most FMEA compliance requirements. Protection covers the input power voltages from the host PC, overvoltage of any of the point of load converters on the board, the output voltage on the SpaceWire ports, and the trigger output voltage. The SpaceWire ports are cold-sparing, so that the SpaceWire PCIe Mk2 board may be powered down, without adversely affecting any system it is connected to by a SpaceWire link. The PCIe Mk2 SpaceWire LVDS transmitters can be tri-stated.

A FMEA/FMECA guide will be available for the board to check the design against specific FMEA requirements.

3.3.10 SpaceWire Ground Connection

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and the ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.3.11 EMC Conformity

The SpaceWire PCIe Mk2 board is sold as a component for inclusion in a computer unit. EMC certification is the responsibility of the user.

3.4 SpaceWire Brick Mk2

Documentation for the STAR-Dundee SpaceWire Brick Mk2 device is provided in this section.

3.4.1 Overview

3.4.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire Brick Mk2 hardware.

For detailed information on using the SpaceWire Brick Mk2 device please consult the API reference documentation.

3.4.1.2 Hardware Description

The SpaceWire Brick Mk2 provides a USB to SpaceWire interface device with two SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire Brick Mk2 is shown below.

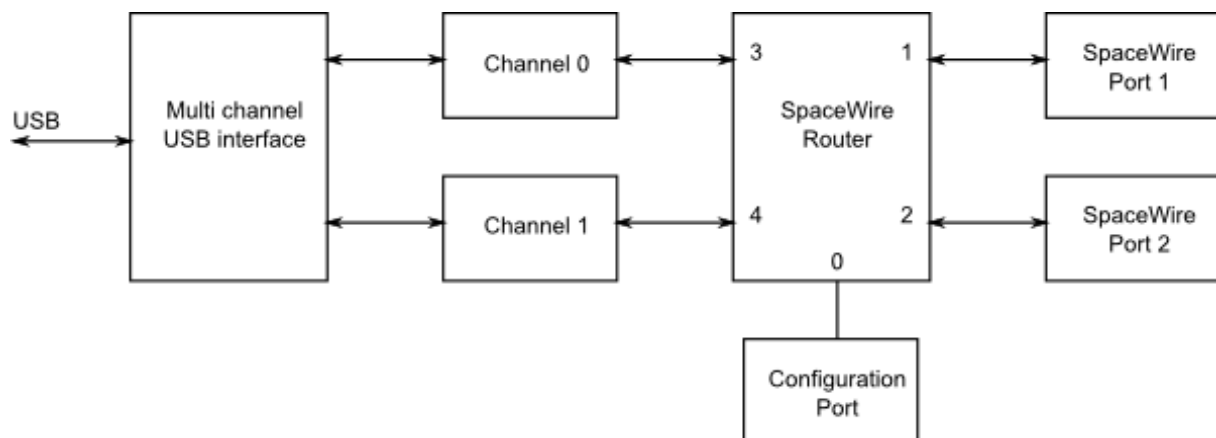


Figure 3.26: SpaceWire Brick Mk2 hardware overview

The two SpaceWire interfaces of the SpaceWire Brick Mk2 are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel. The behaviour when transmitting is described in the [Interface Mode](#) section. There is only one data channel which is shared between the two ports, so traffic flowing over one port may block traffic on the other port. There is also a separate control channel used to ensure that the host PC is always able to access the control, configuration and status space of the Brick Mk2 regardless of the data flow.

The USB interface is compliant to USB version 2.0, and can also be used in USB 1.1 and USB 3.0 ports.

The SpaceWire Brick Mk2 includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.4.2 Getting Started

The SpaceWire Brick Mk2 is powered from the USB cable; therefore no external power supply is required. The USB cable is inserted in the USB socket located at the rear of the Brick Mk2.

When plugged into a host PC or laptop the Brick Mk2 performs a power negotiation stage with the host PC before powering up the router. The LEDs at the front of the Brick Mk2 indicate whether the host has accepted the request from the SpaceWire Brick Mk2 to draw power from the USB bus. When first plugged in, the LEDs remain off until power has been granted to the device. Once the host has accepted the power request from the brick then the LEDs will turn to red for approximately 5 seconds while the brick disconnects and reconnects from the USB bus as a fully powered device.

After this phase the LEDs will default to their normal operating state e.g. with no cables connected all four LEDs are amber.

The SpaceWire Brick Mk2 software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Brick Mk2. This ensures that the Brick Mk2 is recognised by the host when it is connected.

3.4.3 Interfaces

3.4.3.1 SpaceWire Ports and LEDs

The SpaceWire port lowest down is SpaceWire router link 1 and the highest is router link 2. The USB port is router links 3 and 4. The port numbering is shown below.

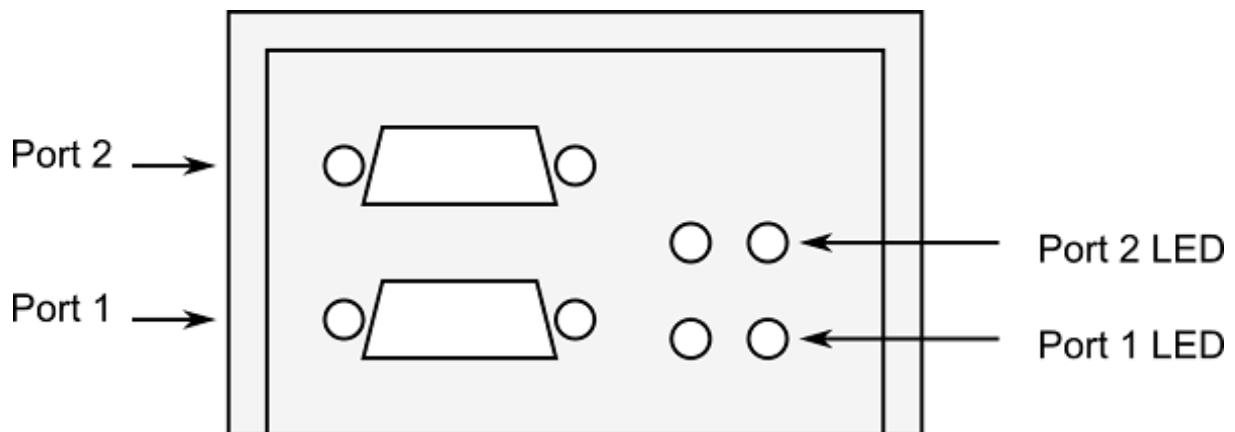


Figure 3.27: SpaceWire ports and LED positions

The Brick Mk2 has four tri-colour LEDs, two for each SpW port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire Brick Mk2 can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.4.4 Functions

3.4.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after a device reset is performed.

The SpaceWire router has 5 ports in total as listed below:

- 1 internal router configuration port
- 2 external SpaceWire ports
- 2 USB interface external ports, one of which may be used by the two SpaceWire ports, while the other may be used by the configuration port

3.4.4.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. Packets received from the USB external ports or the configuration port are routed

based on the first byte, as for routing mode. The SpaceWire Brick Mk2 supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

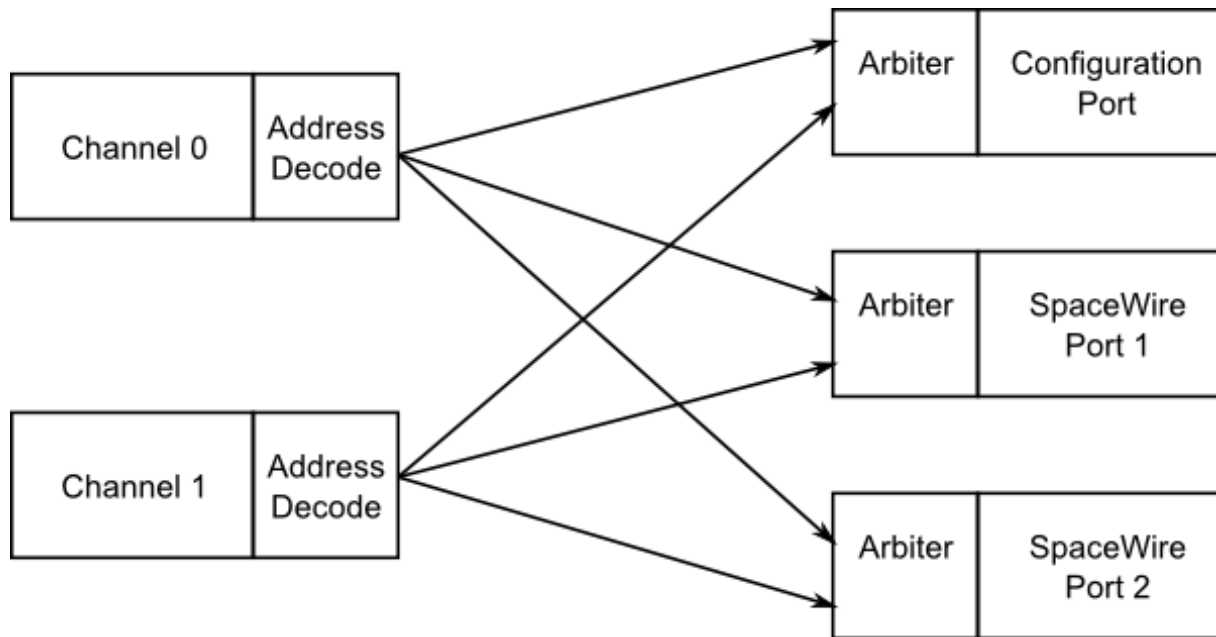


Figure 3.28: Interface mode routing connections, USB to SpaceWire

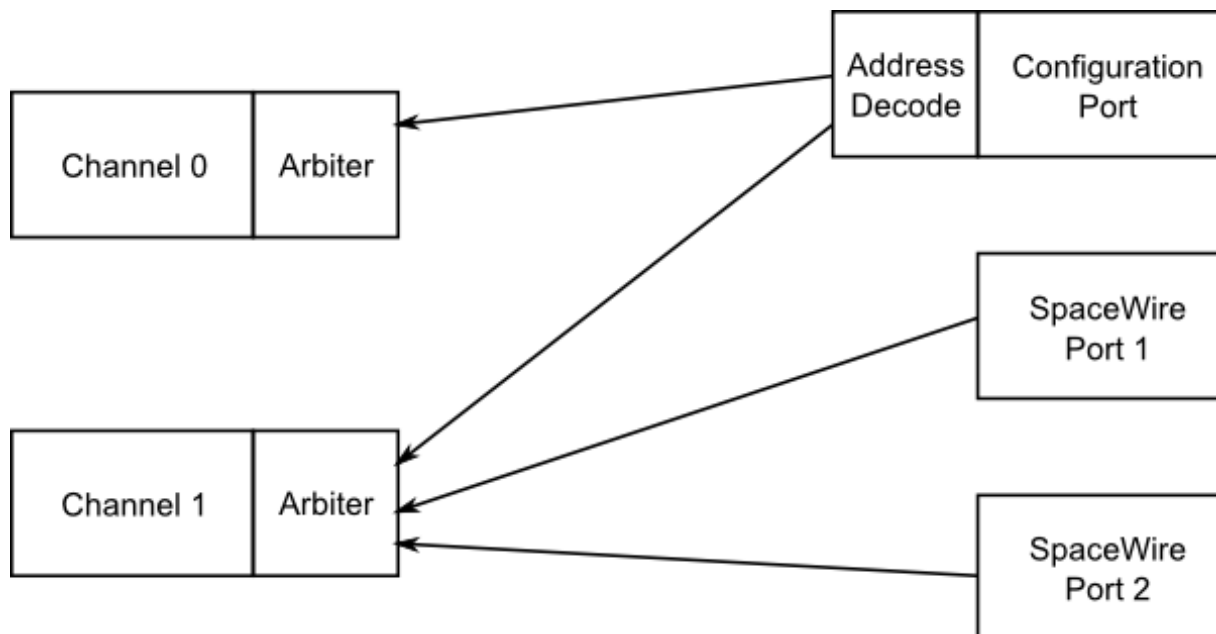


Figure 3.29: Interface mode routing connections, SpaceWire to USB

Source port	Destination port	Description
0	3 or 4	Configuration to USB interface channel (Reply packets routed based on command source)
1	4	SpaceWire link 1 to USB interface channel 1
2	4	SpaceWire link 2 to USB interface channel 1
3	0, 1 or 2	USB interface channel 0 to configuration port, SpaceWire links 1 and 2 (Packets routed based on first byte)
4	0, 1 or 2	USB interface channel 1 to configuration port, SpaceWire links 1 and 2 (Packets routed based on first byte)

3.4.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 9 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.4.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

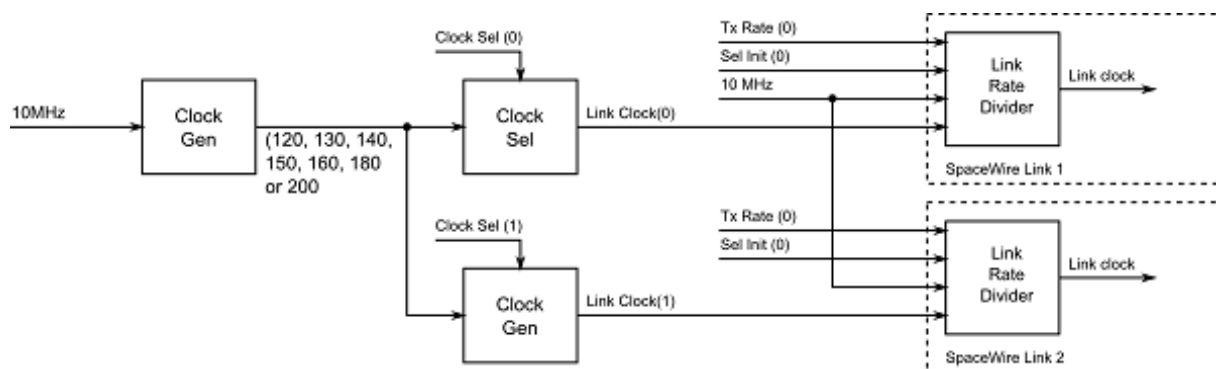


Figure 3.30: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link clock select (one for each SpaceWire link):
 - 120, 130, 140, 150, 160, 180, 200 MHz
- Link rate divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API and the Brick Mk2 Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.4.4.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the Time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 3, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

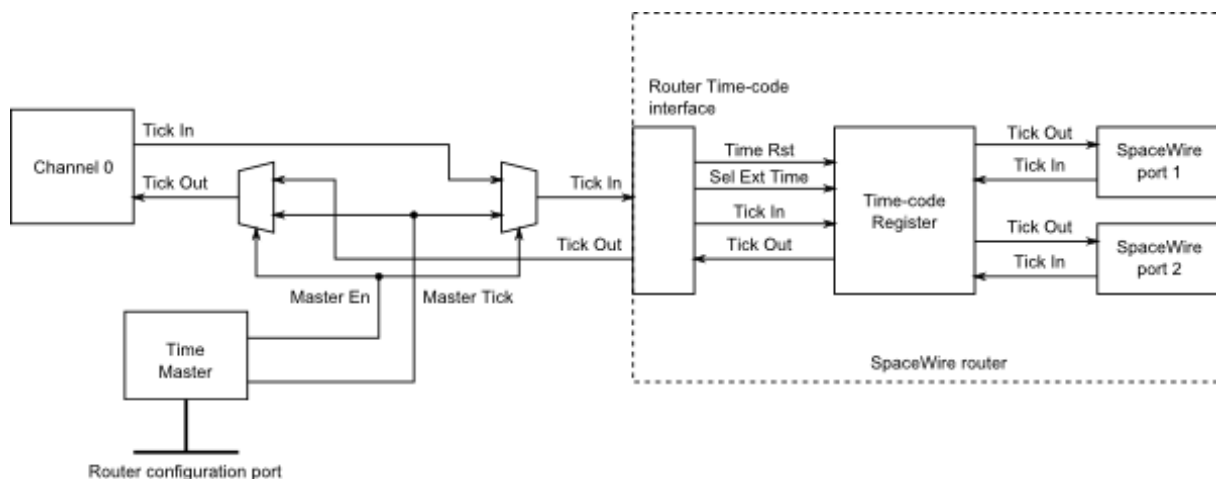


Figure 3.31: Time-code processing architecture

3.4.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V _{ID} mV, Min	mV, Max	V _{ICM} V, Min	V, Max	V _{OD} mV, Min	mV, Max	V _{OCM} V, Min	V, Max
Din, Sin	100	-	0.2	2				
Dout, Sout					250	400	1.125	1.375

V _{ID}	Input differential voltage
V _{ICM}	Input common mode voltage
V _{OD}	Output differential voltage
V _{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

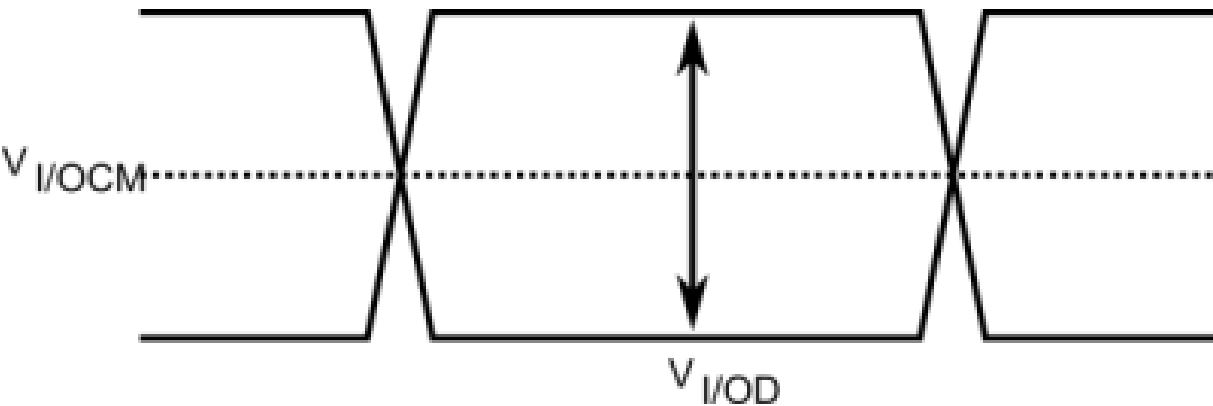


Figure 3.32: SpaceWire LVDS electrical specifications

3.4.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

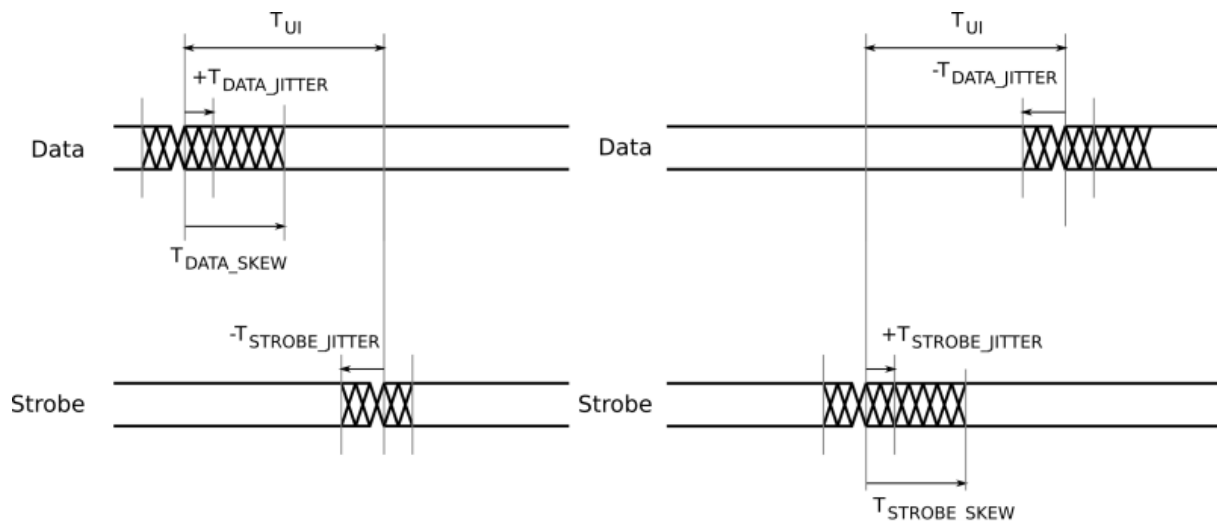


Figure 3.33: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.4.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.4.8 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Brick Mk2

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN 61326-1:1997 +A1: 1998 +A2:2001
47CFR:2002
ANSI C63-4:1992
CFR47 (2002) Part 15, Sub part B

following the provisions of the EMC Directive.

Dundee, 21st December 2012	
(place and date of issue)	(name and signature of authorised person)

Figure 3.34: S.M. Parkes

WARNING This is a class A product. In a domestic environment this product may cause radio interference in which case the user may be required to take adequate measures

3.5 SpaceWire Router Mk2S

Documentation for the STAR-Dundee SpaceWire Router Mk2S device is provided in this section.

3.5.1 Overview

3.5.1.1 Summary

This document provides an overview of the interfaces and features of the SpaceWire Router Mk2S hardware.

For detailed information on using the SpaceWire Router Mk2S device please consult the API reference documentation.

3.5.1.2 Hardware Description

The SpaceWire Router Mk2S provides a USB to SpaceWire interface device with eight SpaceWire interfaces. It also has two external FIFO ports and an external time-code port to permit interfacing to development boards and equipment. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

A block diagram of the SpaceWire Router Mk2S is shown below.

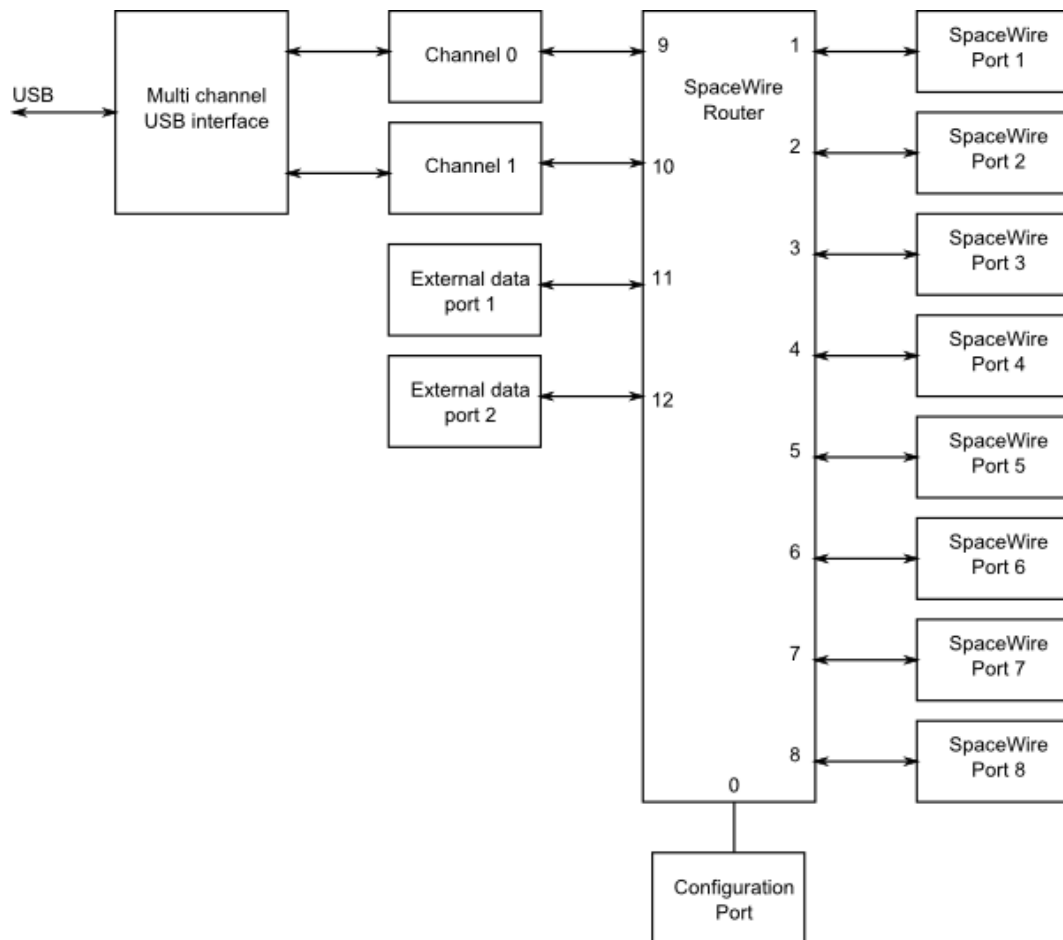


Figure 3.35: SpaceWire Router Mk2S hardware overview

The eight SpaceWire interfaces of the SpaceWire Router Mk2S are fully compliant with the SpaceWire standard and operate at up to 200 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel. There is only one data channel which is shared between the eight ports, so traffic flowing over one port may block traffic on another port. There is also a separate control channel used to ensure that the host PC is always able to access the control, configuration and status space of the Router Mk2S regardless of the data flow.

The USB interface is compliant to USB version 2.0, and can also be used in USB 1.1 and USB 3.0 ports.

The SpaceWire Router Mk2S includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.5.2 Getting Started

The SpaceWire Router Mk2S is supplied with a power supply and USB cable. The power supply jack should be plugged into the power socket at the rear of the SpaceWire Router Mk2S unit. When inserted, the SpaceWire transmit and receive LEDs should illuminate. The USB cable is inserted in the USB socket located at the rear of the Router Mk2S.

The SpaceWire Router Mk2S software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Router Mk2S. This ensures that the Router Mk2S is recognised by the host when it is connected.

Note: The USB cable need not be inserted if the Router Mk2S is part of a network and is only used to route data in the network. The USB cable is only needed if the Router Mk2S unit is to provide an interface from a SpaceWire network to a PC.

3.5.3 Interfaces

3.5.3.1 SpaceWire Ports and LEDs

The Router Mk2S has eight SpaceWire ports numbered 1 to 8. The USB port is router links 9 and 10. The Router Mk2S has two tri-colour LEDs for each SpaceWire port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
Identify	Flashing purple	The SpaceWire Router Mk2S can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.5.3.2 External FIFO Ports

The Router Mk2S has two external FIFO ports which may be used to interface development equipment. Each FIFO port is synchronous and runs at 30MHz. All signals are LVCMOS logic at 3.3V. The connector is a standard fine pitch SCSI-2 D-type, e.g. Molex 1587-6040. It is recommended that an insulation displacement (IDC) mating part is used. A typical mating part would be a Multicom 8E050P-32000AF.

The pinout for both external FIFO ports is as follows:

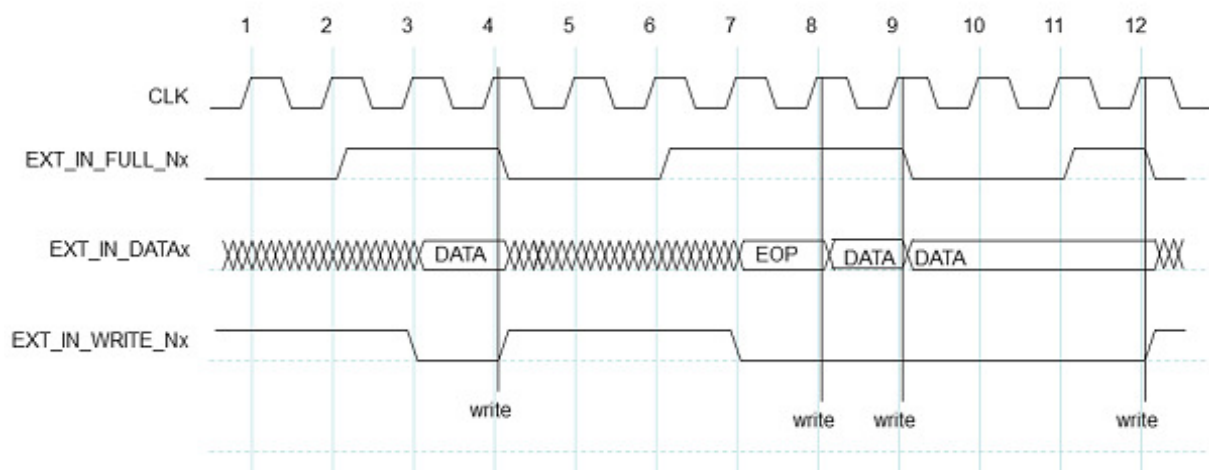
Pin	Signal	Symbol
1	Data Out (8)	EXT_OUT_DATA(0)
2	Data Out (7)	EXT_OUT_DATA(1)
3	Data Out (6)	EXT_OUT_DATA(2)
4	Data Out (5)	EXT_OUT_DATA(3)
5	Data Out (4)	EXT_OUT_DATA(4)
6	Data Out (3)	EXT_OUT_DATA(5)
7	Data Out (2)	EXT_OUT_DATA(6)
8	Data Out (1)	EXT_OUT_DATA(7)
9	Data Out (0)	EXT_OUT_DATA(8)
10	Output FIFO Empty (Active Low)	EXT_OUT_EMPTY_N
11	Input FIFO Full (Active Low)	EXT_IN_FULL_N
12	<i>Do Not Connect</i>	<i>NC</i>
13	System Clock (30MHz)	SYSCLK
14	Data In (8)	EXT_IN_DATA(0)
15	Data In (7)	EXT_IN_DATA(1)
16	Data In (6)	EXT_IN_DATA(2)
17	Data In (5)	EXT_IN_DATA(3)
18	Data In (4)	EXT_IN_DATA(4)
19	Data In (3)	EXT_IN_DATA(5)
20	Data In (2)	EXT_IN_DATA(6)
21	Data In (1)	EXT_IN_DATA(7)
22	Data In (0)	EXT_IN_DATA(8)
23	Input FIFO Write (Active Low)	EXT_IN_WRITE_N
24	Output FIFO Read (Active Low)	EXT_OUT_READ_N
25	<i>Do Not Connect</i>	<i>NC</i>
26	Ground	
27	Ground	
28	Ground	
29	Ground	
30	Ground	
31	Ground	
32	Ground	
33	Ground	
34	Ground	
35	Ground	
36	Ground	
37	Ground	

38	Ground	
39	Ground	
40	Ground	
41	Ground	
42	Ground	
43	Ground	
44	Ground	
45	Ground	
46	Ground	
47	Ground	
48	Ground	
49	Ground	
50	Ground	

Data into and out of the FIFO ports is 9 bits wide. The lowest 8 bits (0-7) are data, where bit 0 is the least significant; the highest bit (8) selects whether this word is data (set to zero) or an end of packet marker (set to one). If an end of packet marker is selected (bit 8 = '1') the data bits (0-7) are used to select an EOP (0x00) or an EEP (0x01). All other end of packet codes are reserved and should not be used.

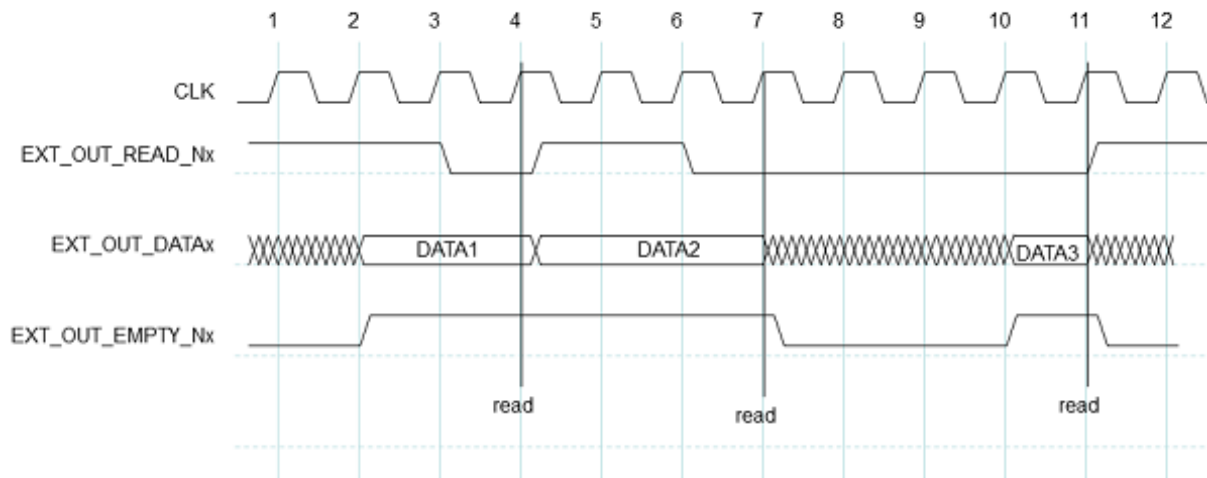
All data and control inputs (EXT_IN_DATA signals, EXT_IN_WRITE_N and EXT_OUT_READ_N) have internal pull-ups. Clock signals are source terminated for 50Ω.

3.5.3.2.1 External Port Write Cycle



The operation of the external port during write operations starts with the EXT_IN_FULL_N signal being de-asserted by the router (at clock cycle 2 above) to indicate to the external system that the router has room for more data and is ready to receive it through the external port. The external system then puts data onto the EXT_IN_DATA data lines and asserts EXT_IN_WRITE_N to transfer data into the external port on the next rising edge of SYSCLK. As long as there is room for new data (EXT_IN_FULL_N is inactive) the write access is performed as long as EXT_IN_WRITE_N is active. If no room is available the write access is ignored (cycles 10 and 11 in the diagram above) and will be performed when room has become available if EXT_IN_WRITE_N is still active. Therefore the data (EXT_IN_DATA) must be valid at that time.

3.5.3.2.2 External Port Read Cycle



Reading of the external port is illustrated in the diagram above. When data is available in the external port FIFO then it is placed on the EXT_OUT_DATA bus and the EXT_OUT_EMPTY_N signal is asserted to signal to the external system that data is available. This is done synchronously to the SYSCLK signal (e.g. clock cycle 2 in the diagram). When it is ready, the external system asserts the EXT_OUT_READ_N signal synchronously with the SYSCLK signal (e.g. clock cycle 3) and the data is then read out of the external port on the next rising edge of the SYSCLK (e.g. start of clock cycle 4). If there is no more data available in the FIFO then the EXT_OUT_EMPTY_N is de-asserted once the data has been read. If the FIFO contains more data to transfer then the EXT_OUT_EMPTY_N remains asserted, the new data is placed on the EXT_OUT_DATA bus and the external system can read it as soon as it is ready. The read access is ignored if there is no data available (EXT_OUT_EMPTY_N is active).

3.5.3.2.3 External Time-code Port

The Router Mk2S has a single time-code port which may be used to interface development equipment. All signals are LVCMOS logic at 3.3V. The connector is identical to that used for the external FIFO ports (SCSI-2 D-type, e.g. Molex 1587-6040). Again, it is recommended that an insulation displacement (IDC) mating part is used. A typical mating part would be a Multicomp 8E050P-32000AF.

The pinout for the external time-code port is as follows:

Pin	Signal	Symbol
1	Time Out (7)	EXT_TIME_OUT(7)
2	Time Out (6)	EXT_TIME_OUT(6)
3	Time Out (5)	EXT_TIME_OUT(5)
4	Time Out (4)	EXT_TIME_OUT(4)
5	Time Out (3)	EXT_TIME_OUT(3)
6	Time Out (2)	EXT_TIME_OUT(2)
7	Time Out (1)	EXT_TIME_OUT(1)
8	Time Out (0)	EXT_TIME_OUT(0)
9	Tick Out	EXT_TICK_OUT
10	<i>Do Not Connect</i>	<i>NC</i>
11	<i>Do Not Connect</i>	<i>NC</i>
12	<i>Do Not Connect</i>	<i>NC</i>
13	<i>Do Not Connect</i>	<i>NC</i>

14	Time In (7)	EXT_TIME_IN(7)
15	Time In (6)	EXT_TIME_IN(6)
16	Time In (5)	EXT_TIME_IN(5)
17	Time In (4)	EXT_TIME_IN(4)
18	Time In (3)	EXT_TIME_IN(3)
19	Time In (2)	EXT_TIME_IN(2)
20	Time In (1)	EXT_TIME_IN(1)
21	Time In (0)	EXT_TIME_IN(0)
22	Select External Time Data	SEL_EXT_TIME
23	Tick In	EXT_TICK_IN
24	Reset Internal Time Counter	TIME_CTR_RST
25	Router Reset (Active Low)	SYSRST_N
26	Ground	
27	Ground	
28	Ground	
29	Ground	
30	Ground	
31	Ground	
32	Ground	
33	Ground	
34	Ground	
35	Ground	
36	Ground	
37	Ground	
38	Ground	
39	Ground	
40	Ground	
41	Ground	
42	Ground	
43	Ground	
44	Ground	
45	Ground	
46	Ground	
47	Ground	
48	Ground	
49	Ground	
50	Ground	

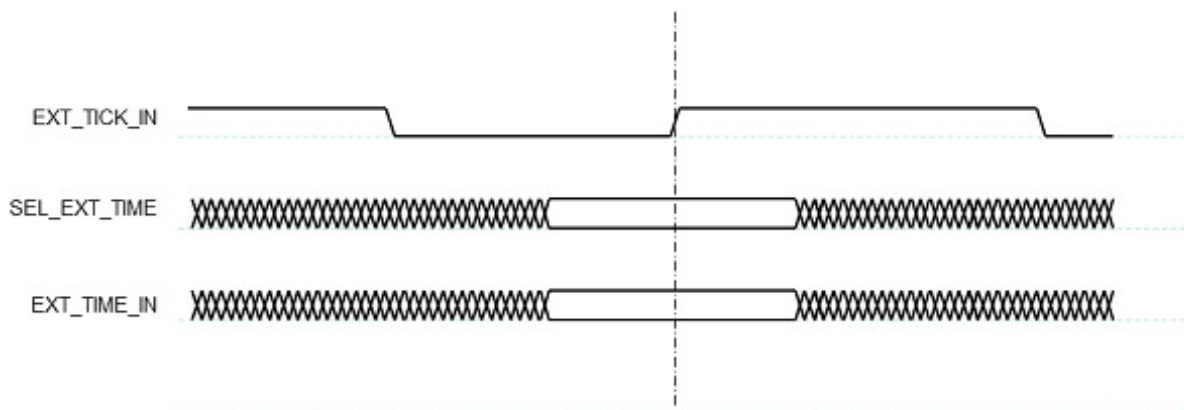
The time-code interface consists of a time-code source for generating time-codes to be transmitted over the SpaceWire link (the EXT_TIME_IN signals, SEL_EXT_TIME and EXT_TICK_IN) and a time-code sink for receiving time-codes from the SpaceWire links (the EXT_TIME_OUT signals and EXT_TICK_OUT). The tick signals indicate that a time-code should be transmitted (EXT_TICK_IN) or that one has been received (EXT_TICK_OUT) the time signals indicate the value of the time-code. Bit 0 is the least significant bit. The operation of the SEL_EXT_TIME and TIME_CTR_RST signals are described below.

All time inputs (EXT_TIME_IN signals, SEL_EXT_TIME, EXT_TICK_IN and TIME_CTR_RST) have internal pull-downs.

The SYSRST_N signal is bidirectional and can be used to reset the entire router. SYSRST_N is both logically and electrically equivalent to pressing the reset button. This signal is active low and has an internal pull-up. It must be driven by an open-drain driver. The router may also be reset via USB (software) and by the reset button. In both of these cases this signal will go low. If reset functionality is not required, it is safest to leave this signal unconnected.

3.5.3.2.4 Generating Time-codes

Time-codes can be generated by the router on request of the external system to which it is attached. A time-code is generated whenever the router detects a rising edge on the EXT_TICK_IN signal. The value of the time-code to be transmitted is either taken from the EXT_TIME_IN inputs or from the time-code counter inside the router. The time-code source used depends on the value of the SEL_EXT_TIME signal when EXT_TICK_IN signal has a rising edge. If SEL_EXT_TIME is 1 then the EXT_TIME_IN inputs are used to provide the contents of the time-code.

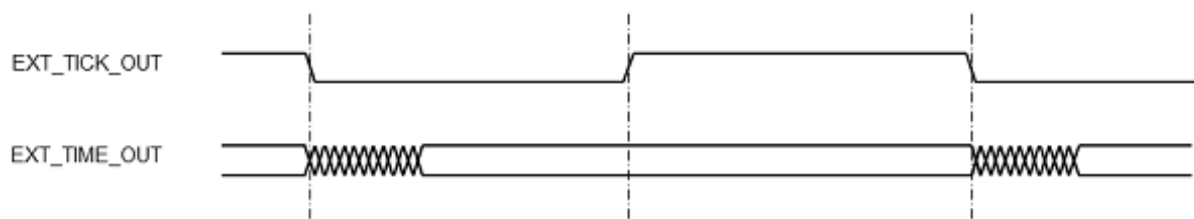


If SEL_EXT_TIME is 0 then the internal time-code counter provides the least-significant 6-bits of the time-code and the top two bits of EXT_TIME_IN (bits 7 and 6) provide the most-significant 2-bits. When using the EXT_TIME_IN inputs to provide the complete time-code, the time-code is only broadcast if it is a valid time-code i.e. is one more than the internal time register of the router (for more information refer to the SpaceWire standard).

Note: Only one router or node in a SpaceWire network should operate as a time master generating time-codes (again, see the SpaceWire standard for more information).

3.5.3.2.5 Receiving Time-codes

When a valid time-code is received by the router the value of this time-code (flags plus time value) will be placed on the EXT_TIME_OUT outputs and the EXT_TICK_OUT signal will be set to zero. The EXT_TICK_OUT signal is set to one a short time later, once the EXT_TIME_OUT outputs have stabilised, to indicate that these outputs are valid.



They then remain valid until the next time-code is received and the EXT_TICK_OUT signal will be set to zero.

3.5.3.2.6 Time Counter Reset

When a rising edge is detected on TIME_CTR_RST then the internal time-code register is reset to zero.



Figure 3.36: Time-code reset interface

3.5.4 Functions

3.5.4.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is router mode. The Router Mk2S will revert to router mode after a device reset is performed.

The SpaceWire router has 13 ports in total as listed below:

- 1 configuration port
- 8 SpaceWire ports
- 2 USB interface ports, one of which may be used by the SpaceWire ports, while the other may be used by the configuration port
- 2 parallel FIFO ports which are connected to the rear panel of the unit

3.5.4.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. Packets received from the USB external ports or the configuration port are routed based on the first byte, as for routing mode. The SpaceWire Router Mk2S supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

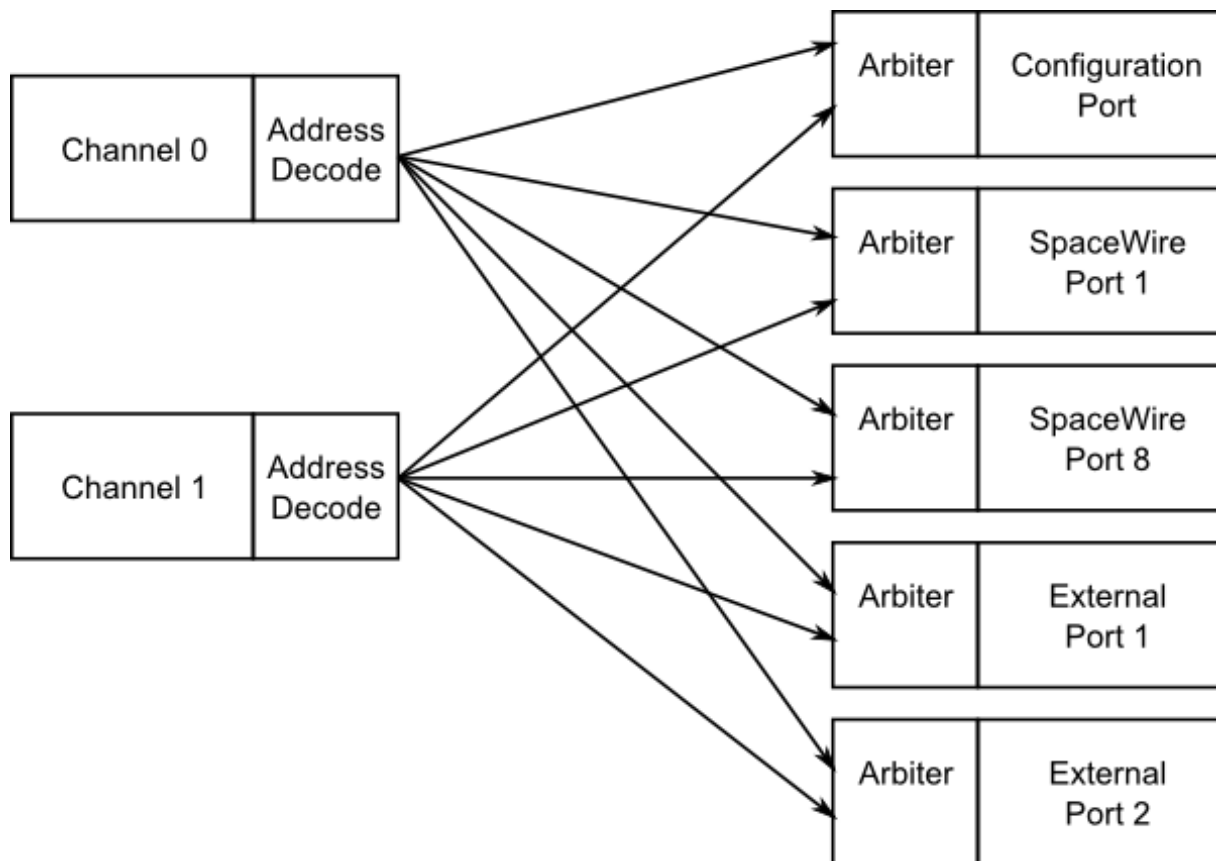


Figure 3.37: Interface mode routing connections, USB to SpaceWire

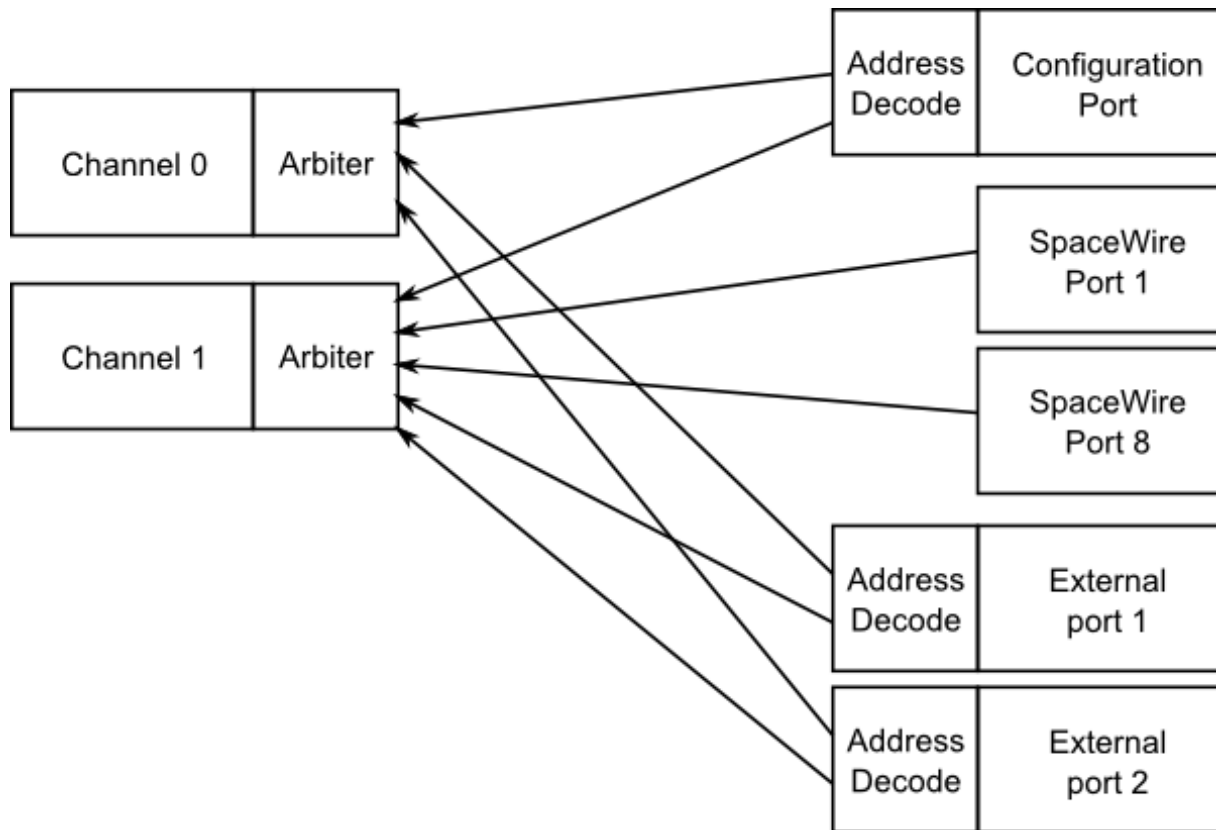


Figure 3.38: Interface mode routing connections, SpaceWire to USB

Source port	Destination port	Description
0	9 or 10	Configuration to USB interface channel (Reply packets routed based on command source)
1 to 8	10	SpaceWire links to USB interface channel 1
11, 12	10	External FIFO ports to USB interface channel 1
9	0 to 8, 11 or 12	USB interface channel 0 to configuration port, SpaceWire links, external FIFO ports (Packets routed based on first byte)
10	0 to 8, 11 or 12	USB interface channel 1 to configuration port, SpaceWire links, external FIFO ports (Packets routed based on first byte)

3.5.4.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 13 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.5.4.4 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire link is controlled by register settings. The clock selection logic for each link is described in the diagram below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

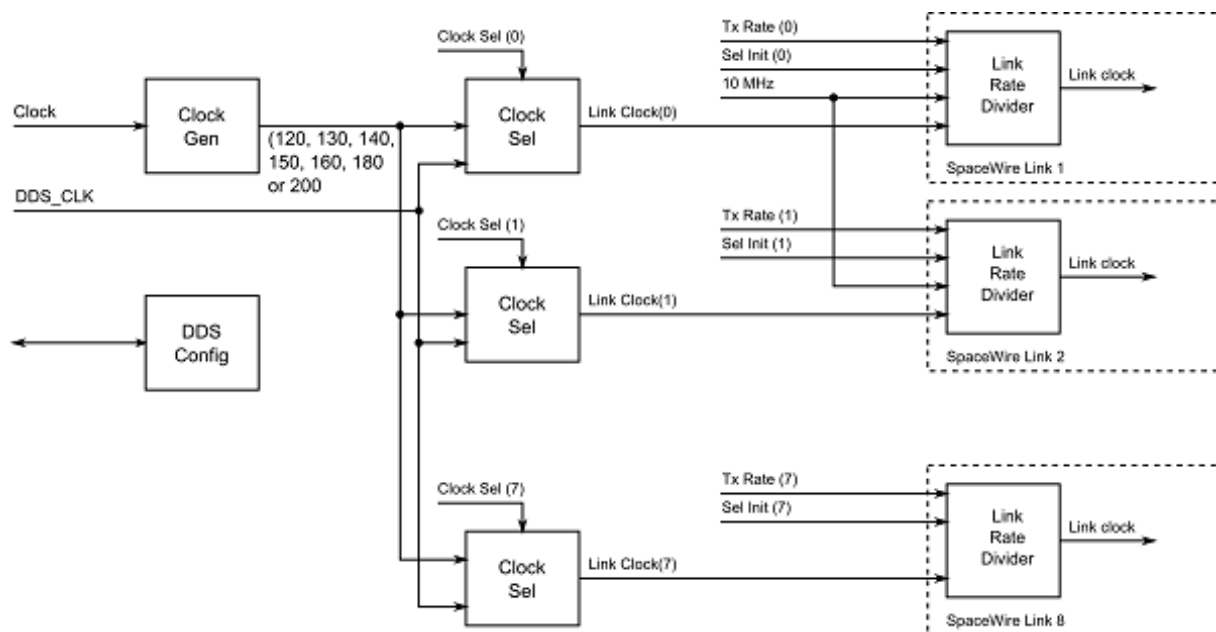


Figure 3.39: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using the following variables:

- Clock Generator:
 - Generates a set of clock frequencies for each link.
- Link clock select (one for each SpaceWire link):
 - 120, 130, 140, 150, 160, 180, 200 MHz
- Link rate divider (one for each SpaceWire link):
 - Divide by an even integer value.

The clock generator generates seven clock frequencies which can be individually selected by the link clock select modules.

Each link clock select module chooses between the set of clock generator frequencies using a link clock select register which is accessible through the SpaceWire router configuration port.

In addition, a precision transmit rate mode can be enabled, which generates any frequency the user requires between 2 and 200 Mbit/s and is accurate to more than 0.1 Mbit/s.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API and the Brick Mk2 Configuration API. The precision transmit rate can be configured using the Router Mk2S Configuration API. Please refer to the API documentation for further details on how to set these values.

3.5.4.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 9, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

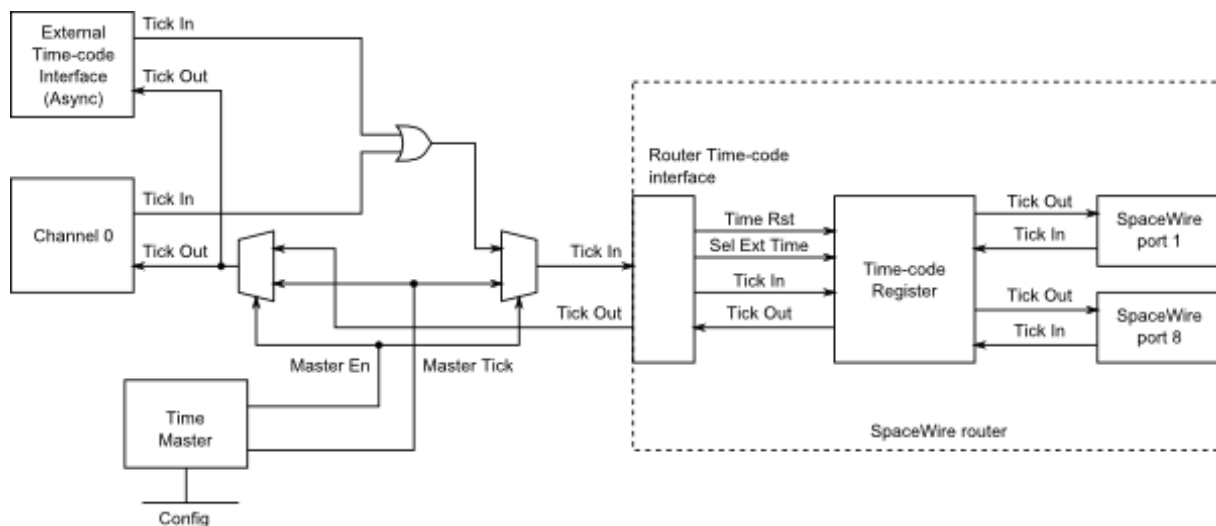


Figure 3.40: Time-code processing architecture

3.5.5 Electrical Characteristics

The SpaceWire interface absolute maximum input ratings are defined in the table below.

	Description	V _{MIN}	V _{MAX}
V _{IN}	Input voltage relative to GND	-0.5	3

The SpaceWire connector LVDS DC characteristics are listed in the table below.

	V_{ID}			V_{ICM}			V_{OD}			V_{OCM}		Unit
	Min	Typ	Max	Min	Typ	Max	Min	Typ	Max	Min	Max	
Din, Sin	100	350	600	300	1250	2200						mV
Dout, Sout							250	350	450	1125	1375	mV

V_{ID}	Input differential voltage
V_{ICM}	Input common mode voltage
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

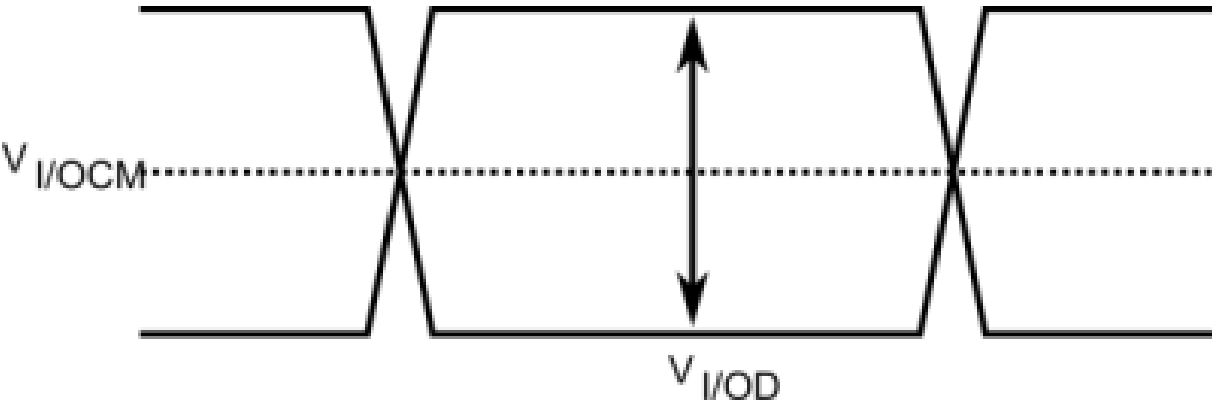


Figure 3.41: SpaceWire LVDS electrical specifications

3.5.6 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

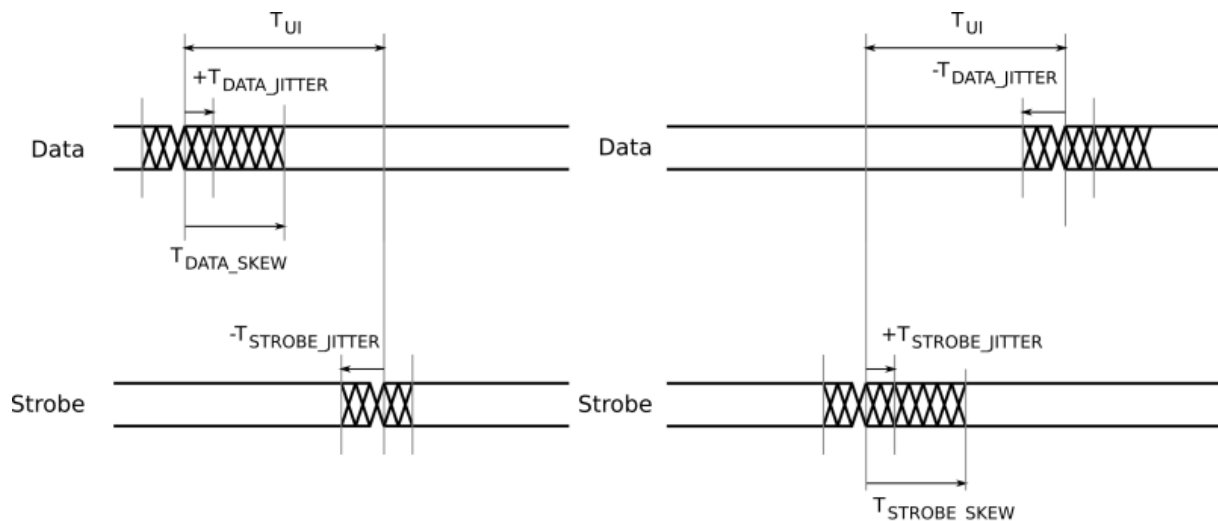


Figure 3.42: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew	0
T_{JITTER_OUT}	Maximum data/strobe jitter	0.7 (Peak to Peak)

3.5.7 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.5.8 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Router Mk2S

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN 61326-1:1997 +A1: 1998 +A2:2001
47CFR:2002
ANSI C63-4:1992
CFR47 (2002) Part 15, Sub part B

following the provisions of the EMC Directive.

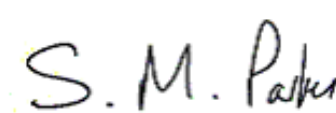
Dundee, 21st December 2012	
(place and date of issue)	(name and signature of authorised person)

Figure 3.43: S.M. Parkes

WARNING This is a class A product. In a domestic environment this product may cause radio interference in which case the user may be required to take adequate measures

3.6 SpaceWire Brick Mk3 and Brick Mk4

Documentation for the STAR-Dundee SpaceWire Brick Mk3 and Brick Mk4 devices is provided in this section.

3.6.1 Overview

3.6.1.1 Summary

This user manual provides an overview of the interfaces and features of the SpaceWire Brick Mk3 and Brick Mk4 hardware.

For detailed information on using the SpaceWire Brick Mk3 and Brick Mk4 devices please consult the STAR-System API reference documentation.

3.6.1.2 Hardware Description

The SpaceWire Brick Mk3/Mk4 provides a USB to SpaceWire interface device with two SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host P↔C memory.

A block diagram of the SpaceWire Brick Mk3/Mk4 is shown below.

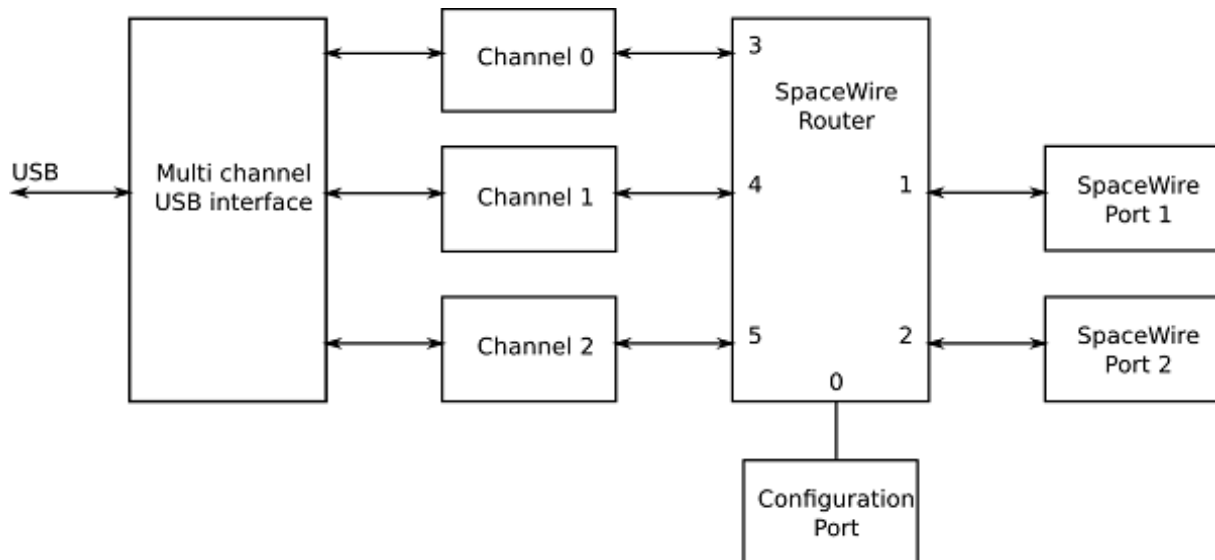


Figure 3.44: SpaceWire Brick Mk3/Mk4 hardware overview

The two SpaceWire interfaces of the SpaceWire Brick Mk3/Mk4 are fully compliant with the SpaceWire standard and operate at up to 300 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the USB interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the USB data channel for that link. There are two independent channels from the SpaceWire router to the USB interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the Brick Mk3/Mk4 regardless of the data flow.

The USB interface is compliant to USB version 3.0, as well as USB versions 1.1 and 2.0. This means it can be used in older PCs, albeit with lower throughput and higher latency achievable on the USB bus.

The SpaceWire Brick Mk3/Mk4 includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.6.2 Getting Started

The SpaceWire Brick Mk3/Mk4 is powered from the USB cable; therefore no external power supply is required. The USB cable is inserted in the USB socket located at the rear of the device.

When plugged into a host PC or laptop the Brick Mk3/Mk4 performs a power negotiation stage with the host PC before powering up the router. The LEDs at the front of the device indicate whether the host has accepted the request from the SpaceWire Brick Mk3/Mk4 to draw power from the USB bus. When first plugged in, the LEDs remain off until power has been granted to the device.

After this phase the LEDs will default to their normal operating state e.g. with no cables connected all four LEDs are amber.

The software and drivers, provided by STAR-System, should be installed on the host computer before connecting the Brick Mk3/Mk4. This ensures that the Brick Mk3/Mk4 is recognised by the host when it is connected.

3.6.3 Rear Panel Interfaces

The Rear panel features the Micro-B USB 3.0 connector with an LED to indicate USB activity and "power good". An image of this is shown below.



Figure 3.45: Micro USB 3.0 connector and LED

3.6.3.1 USB Status LED

The USB status LED provides three functions:

1. Low level power good indicator
2. Active USB interface version
3. USB data transaction in progress

3.6.3.1.1 Low level power good indicator

If the USB Status LED fails to light up (along with the LEDs on the front panel) after a few seconds when the Brick Mk3/Mk4 is connected to a USB port, then it is possible that the Host PC has not granted sufficient power on the USB bus to start up. In this event, disconnect the Brick Mk3/Mk4, and try plugging it into another USB port on the same computer. If it fails a second time, try using a USB hub with an external power supply.

If the USB Status LED fails to light up when the device is plugged into a few different computers, or if it turns off in the middle of a test, then this is an indication that one of the power supplies has failed. The Brick Mk3/Mk4 should be powered down and removed from the test system. Contact STAR-Dundee support at support@star-dundee.com for further advice in this situation.

3.6.3.1.2 Active USB interface version

The colour of the USB status LED is used to indicate the version of USB that the device is operating under. This is useful as the Brick Mk3/Mk4 is capable of operation under USB 1.1, 2.0 and 3.0. The colours are summarised in

the table below.

USB Status LED Colour	Active USB Interface Version
Blue	USB 3.0
Green	USB 2.0
Red	USB 1.1

Note that it is sometimes possible to plug the Brick Mk3/Mk4 into a USB 3.0 socket in such a way that it starts up in USB 2.0 mode. This is usually because the Type-A plug is inserted very slowly or, at a slight angle to the connector. If this happens, try unplugging and plugging the device back in.

3.6.3.1.3 USB data transaction in progress

The USB status LED can indicate when traffic is passing over the USB interface by blinking on and off. This behaviour is summarised in the table below.

USB Status LED	Status
Steady	No traffic is passing over the USB link
Blinking on and off rapidly	Traffic is passing over the USB link

Note that the LED will blink when data passes in either direction over any of the three USB endpoints.

3.6.3.2 Micro-B USB Connector

A suitable Type-A to Micro-B cable is supplied with the Brick Mk3/Mk4 along with a ferrite attached for EMC purposes. This cable can be plugged into any Type-A USB 1.1, 2.0 or 3.0 port on a host PC. The Brick will have higher throughput and lower latency on a USB 3.0 port compared to a USB 2.0 port, so it is recommended to use USB 3.0 when possible. Computers with a mixture of USB 2.0 and USB 3.0 ports usually colour the plastic portion of a USB 3.0 port blue to distinguish them. You can also look at the USB Status LED to see in which USB mode the Brick Mk3/Mk4 is operating.

Although it is possible to plug in other Type-A to Micro-B USB 2.0 or 3.0 cables, STAR-Dundee strongly recommends the use of the supplied cable as poor quality "off-the-shelf" cables can cause signal integrity problems in USB 3.0. EMC compliance may also be affected by using any other third party cables.

3.6.4 Front Panel Interfaces

The SpaceWire port on the left of the front panel is SpaceWire router link 1 and the port on the right is router link 2. The USB port is router links 3, 4 and 5. The front panel is shown below.

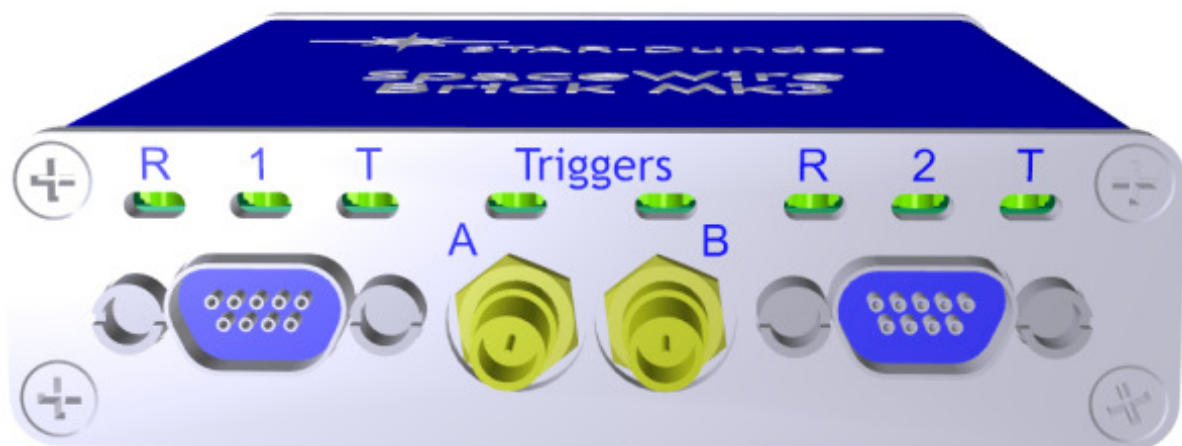


Figure 3.46: SpaceWire ports and LED positions

The Brick Mk3/Mk4 has six multicolour LEDs, three for each SpW port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The middle LED is currently unused. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the right hand LED turns white. When the Brick Mk3/Mk4 has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the left hand LED turns white.
Identify	Flashing purple	The Brick Mk3/Mk4 can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.6.5 Functions

3.6.5.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after power cycling or a device

reset is performed.

The SpaceWire router has 6 ports in total as listed below:

- 1 internal router configuration port
- 2 external SpaceWire ports
- 3 USB interface external ports. By default two of these are used by the two SpaceWire ports, while the other is used by the configuration port

3.6.5.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. The SpaceWire Brick Mk3/Mk4 supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

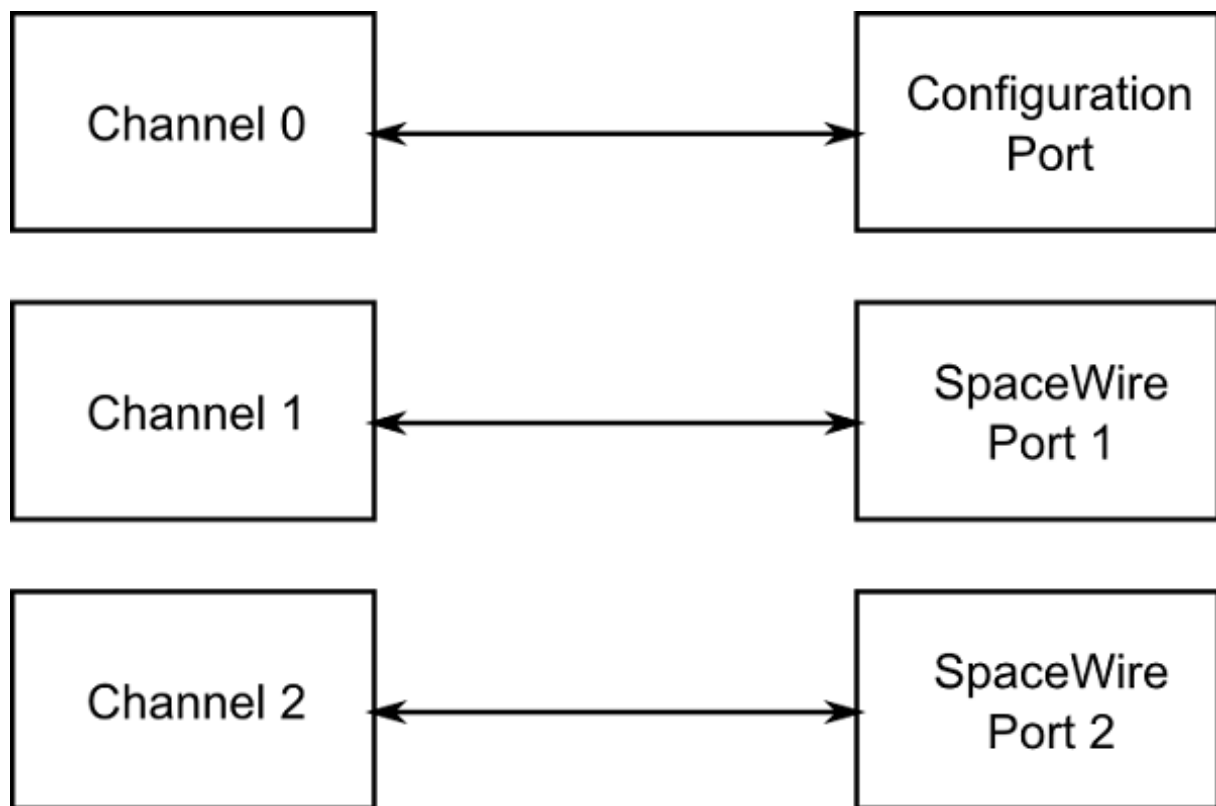


Figure 3.47: Interface mode routing connections

Source port	Destination port	Description
0	3	Configuration to external port 1 (host channel 0)
1	4	SpaceWire link 1 to external port 2 (host channel 1)

2	5	SpaceWire link 2 to external port 3 (host channel 2)
3	0	USB interface channel 0 to configuration port
4	1	USB interface channel 1 to SpaceWire link 1
5	2	USB interface channel 2 to SpaceWire link 2

3.6.5.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 6 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.6.5.4 Link Operating Speed

The link operating speed for each SpaceWire port is controlled by configuration parameters which can be set by the API or GUI application. The link clock rate selection logic for each port is shown in the figure below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

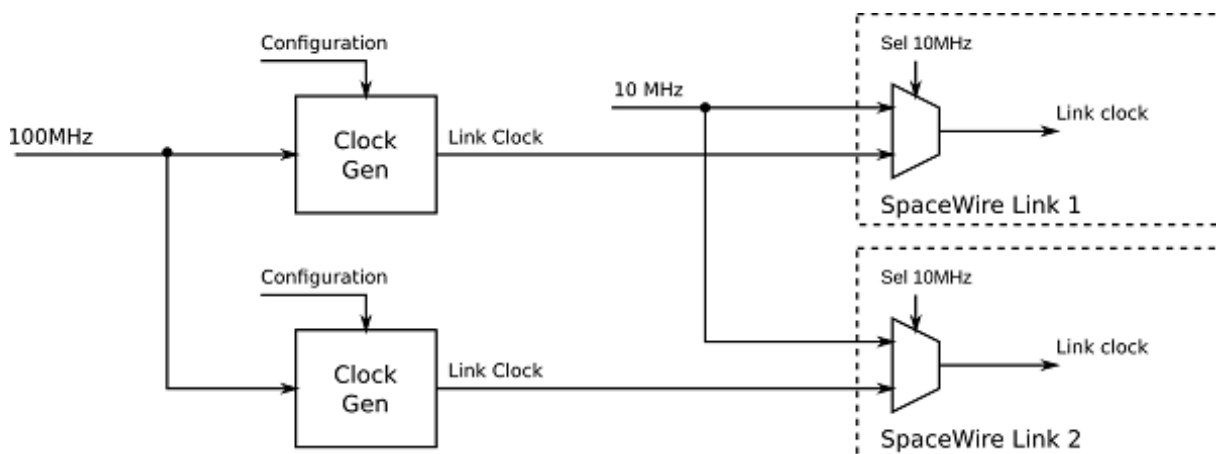


Figure 3.48: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using a programmable clock generator frequency. This is set by configuring a multiply and divide value to the input 100 MHz clock.

The link rate can be programmed using the functions provided by the Mk2 Device and Interface Configuration API. Please refer to the API documentation for further details on how to set this clock speed.

3.6.5.5 Time-codes

A time-code can be received from the USB interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external USB ports, only port 3, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

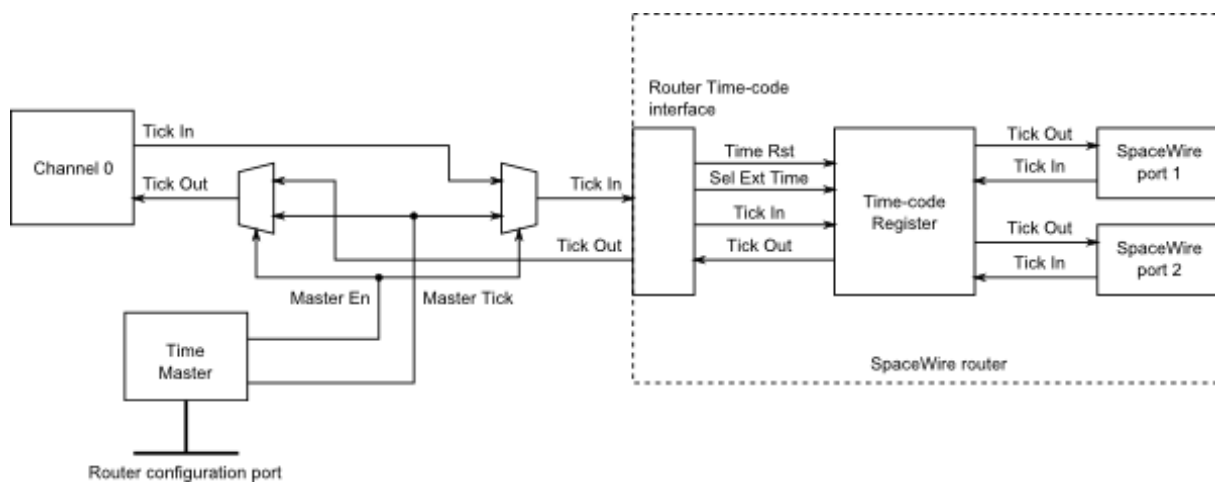


Figure 3.49: Time-code processing architecture

3.6.5.6 Packet Timestamping

The SpaceWire Brick Mk3/Mk4 provides an optional timestamp feature for each received packet. A timestamp will be recorded when the first byte of a packet is received and when the end of packet marker is received. The timestamp is synchronised with an external pulse input on SMB trigger A.

Multiple SpaceWire Brick Mk3/Mk4 devices can be synchronised using the external pulse external time source to synchronise time across the devices.

3.6.5.6.1 Detailed Description

An overview of the timestamp functions provided in by the SpaceWire Brick Mk3/Mk4 are shown in the figure below. Configuration settings are shown in *italics*.

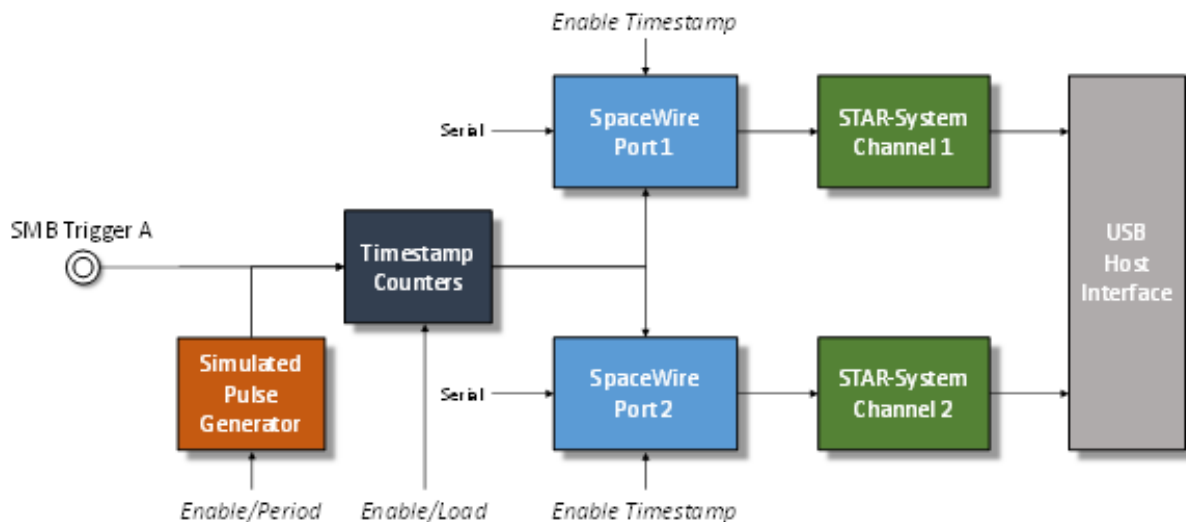


Figure 3.50: Timestamp system overview

Each SpaceWire port has a SpaceWire receiver which decodes serial data into link characters and data characters. Each data character can be an 8-bit packet data byte or an end of packet marker. The timestamping mechanism will record the timestamp for each start of packet byte and for each end of packet marker, effectively the start time and end time of each packet.

The timestamp counter is implemented as two counters, one which increments on the input SMB trigger or the pulse generator and one which increments on the system clock. The counters are identified as "SyncPulse" and "ClockCycles" respectively.

The packet start and end times are recorded using a combination of the counters described above, plus a capture register which captures the total number of clock cycles which elapsed in the previous period. An overview of the counter and register operation is listed below.

- The "SyncPulse" counter can be loaded by software to provide a starting value for the timestamp. The counter will increment on each trigger pulse.
- The "ClockCycles" counter is incremented on each system clock cycle and is reset on each pulse of the trigger. It provides a count of the system clock cycles which have elapsed since the last synchronisation pulse.
- The "TotalCycles" register is captured into a storage register on each synchronisation pulse to provide a mechanism to convert the "ClockCycles" elapsed time into fractions of the synchronisation period, i.e. $\frac{\text{ClockCycles}}{\text{TotalCycles}}$ is the fraction between 0 and 1 of the synchronisation period at which the event occurred.

The timestamp is synchronised using an external trigger pulse which supports frequencies equal to or greater than 1 Hz. The mechanism provides additional meta-data for each received SpaceWire packet which can be received by applications to compute the time at which the packet was decoded by the SpaceWire receiver.

A simulated pulse generator module can be used to simulate the external SMB trigger for testing and development purposes. The generator is programmed with the period between pulses in system clock cycles, and can be enabled or disabled.

3.6.5.6.2 Computing the "User" time from the recorded Timestamp

A method to convert the timestamp fields into "User" time is listed below. The raw timestamp is returned by the API therefore user applications can implement a different algorithm if required.

The synchronisation pulse frequency is set by the user application. In the equations below the frequency is referred to as F_{sync} .

The computed time is defined as x.y where x is the number of seconds and y indicates the fraction of seconds between 0 and 1.

The example steps to compute the "User" time are listed below.

1. The first step is to convert the hardware format into seconds and sub seconds as shown below.

$$\begin{aligned} \text{Seconds} &= \text{SyncPulse} / F_{\text{sync}} \\ \text{SubSeconds} &= \text{ClockCycles} / (\text{TotalCycles} * F_{\text{sync}}) \end{aligned}$$

2. The resultant values are then added together.

$$x.y = \text{Seconds} + \text{SubSeconds}$$

An example case is listed below.

1. The application setup is listed below:

- (a) F_{sync} is 10 Hz
- (b) "SyncPulse" counter is 1234
- (c) "ClockCycles" counter is 234567
- (d) "TotalCycles" counter is 301210

2. The resultant time in seconds and fractions of seconds is computed as:

$$\begin{aligned} \text{Seconds} &= 1234/10 \\ &= 123.4 \end{aligned}$$

$$\begin{aligned} \text{SubSeconds} &= \frac{234567}{301210 * F_{\text{sync}}} \\ &= 77.8749045516E - 3 \end{aligned}$$

$$x.y = 123.477874905 \text{seconds}$$

3.6.6 SpaceWire Interface Tristate Mode

The SpaceWire ports on the Brick Mk4 card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/↔ Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Interface Mode, Start, Disabled, or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.

If a link on the Brick Mk4 card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

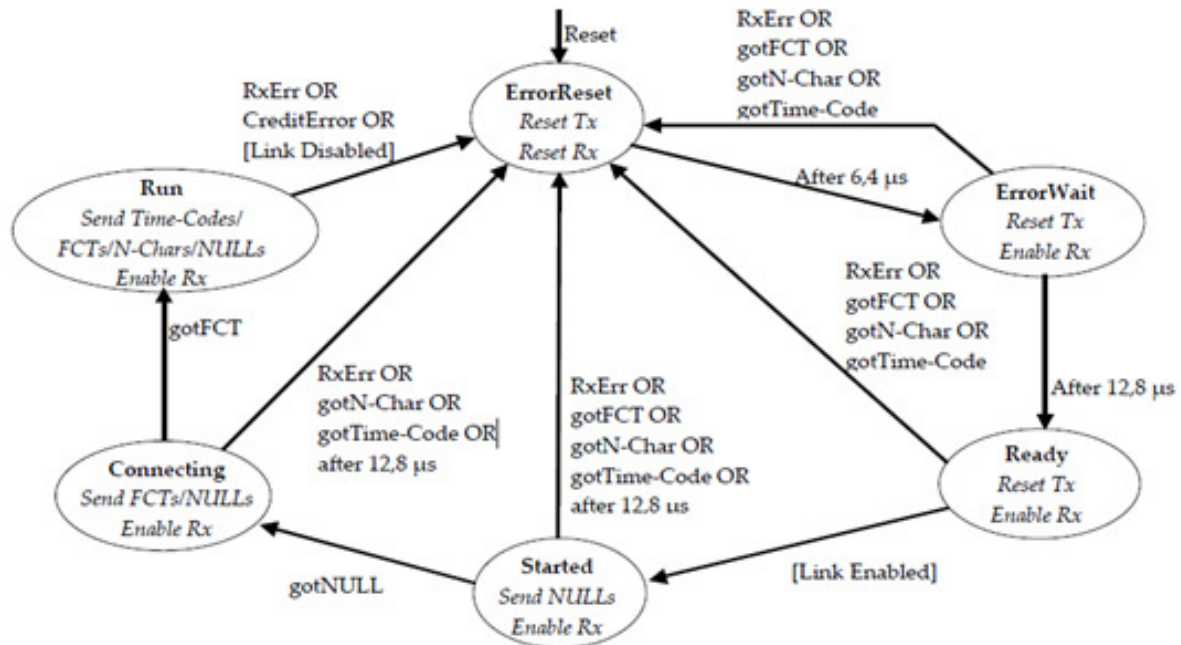


Figure 3.51: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled. Devices with FPGA version greater than or equal to version 1.00e08 will also disable the Tristate mode of a link when Interface Mode is enabled and software sends a packet to the port over the corresponding channel interface.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

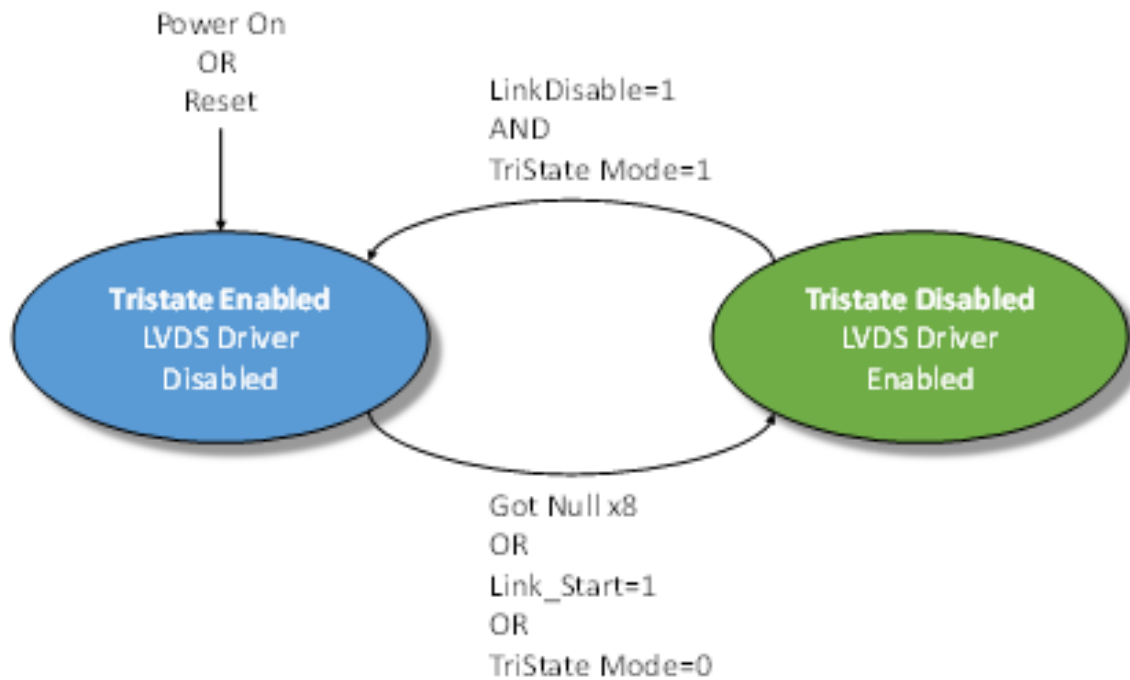


Figure 3.52: Tristate controller state machine

3.6.7 Electrical Characteristics

3.6.7.1 SpaceWire Connectors

3.6.7.1.1 Absolute Maximum Values

The SpaceWire interface **absolute maximum** input ratings are defined in the tables below. The device can withstand these maximum values but use at or near these values is not recommended.

3.6.7.1.1.1 Brick Mk3

		Description	V _{MIN}	V _{MAX}
V _{IN}	Data/Strobe Out	Input voltage relative to GND	-0.5	3.8
	Data/Strobe In	Input voltage relative to GND	-5	6

3.6.7.1.1.2 Brick Mk4

		Description	V _{MIN}	V _{MAX}
V _{IN}	Data/Strobe Out	Input voltage relative to GND	-0.3	3.6
	Data/Strobe In	Input voltage relative to GND	-5	4

Additional clamping diodes on all SpaceWire signals will engage for voltages below -0.3V and above 3V.

3.6.7.1.2 LVDS DC Characteristics

The SpaceWire connector LVDS DC characteristics are listed in the tables below.

3.6.7.1.2.1 Brick Mk3

	V_{ID}		V_{IN}		V_{OD}			V_{OCM}		
	mV, Min	mV, Max	V, Min	V, Max	mV, Min	mV, Typ	mV, Max	V, Min	V, Typ	V, Max
Din, Sin	100	3000	-4	5						
Dout, Sout					250	310	450	1.125	1.17	1.375

3.6.7.1.2.2 Brick Mk4

	V_{ID}		V_{IN}		V_{OD}			V_{OCM}		
	mV, Min	mV, Max	V, Min	V, Max	mV, Min	mV, Typ	mV, Max	V, Min	V, Typ	V, Max
Din, Sin	100	3000	-4	5						
Dout, Sout					247	340	454	1.125		1.375

V_{ID}	Input differential voltage
V_{IN}	Any combination of common mode or input signals
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

The common mode and differential identifiers are described in the diagram below.

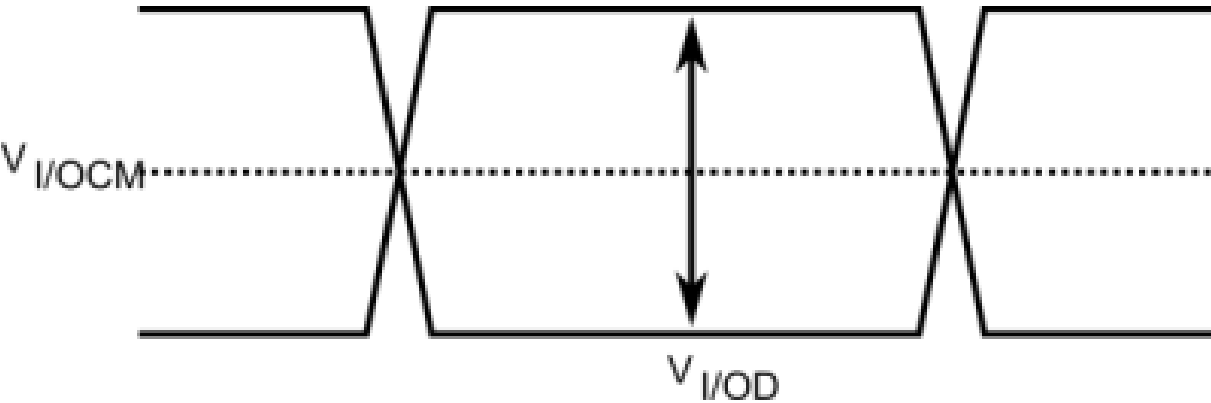


Figure 3.53: SpaceWire LVDS electrical specifications

3.6.7.2 External Trigger Connectors

3.6.7.2.1 External Trigger Functional Diagram

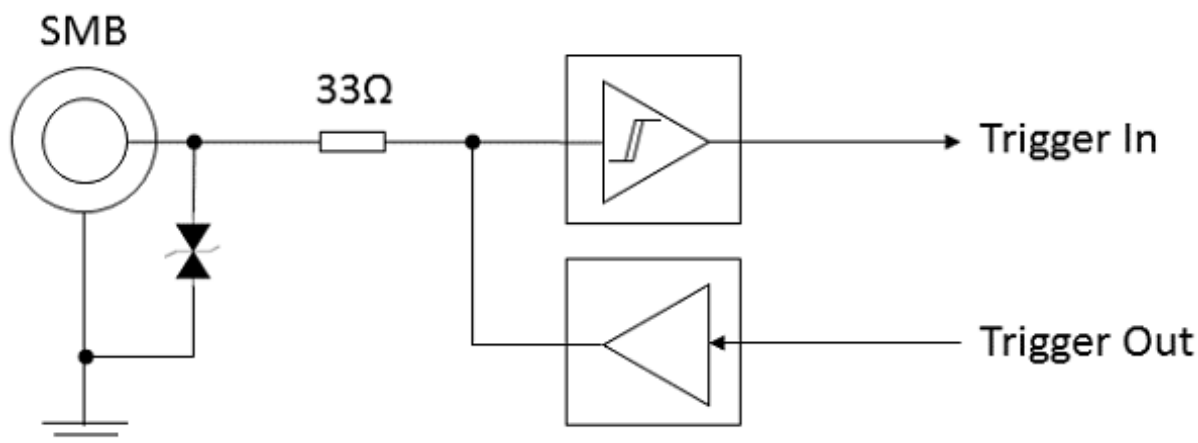


Figure 3.54: External trigger functional diagram

3.6.7.2.2 Absolute Maximum Values

The external trigger **absolute maximum** input ratings are defined in the table below. The device can withstand these maximum values but use at or near these values is not recommended. Note that each trigger is only 5 Volt input tolerant when configured as an input (the default on power up), or when the device is powered down. Triggers configured as outputs are not 5 Volt tolerant, and may be damaged if an external voltage is applied to them. It is recommended to use 3.3 Volt logic when using the triggers.

	Description	Condition	Min	Max	Unit
V_{IN}	Voltage applied to trigger	Trigger configured as input, or device powered down	-0.5	6.5	Volts
		Trigger configured as output	-0.5	3.8	Volts
I_{OUT}	Continuous output current	Trigger configured as output	-50	+50	mA

3.6.7.2.3 DC Characteristics

The DC characteristics of the external trigger is listed in the table below. The output is driven by 3.3 Volt LVCMOS logic.

	V_{IL} V, Min	V, Max	V_{IH} V, Min	V, Max	V_{OL} V, Max	V_{OH} V, Min
Trigger IN	0	0.8	2.0	5.5		
Trigger OUT (100 μ A load)					0.1	3.2
Trigger OUT (24 mA load)					0.55	2.3

V_{IL}	Input low voltage
V_{IH}	Input high voltage
V_{OL}	Output low voltage
V_{OH}	Output high voltage

3.6.7.3 FMECA Protection

A FMECA analysis report for the Brick Mk3/Mk4 is available on request.

3.6.8 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

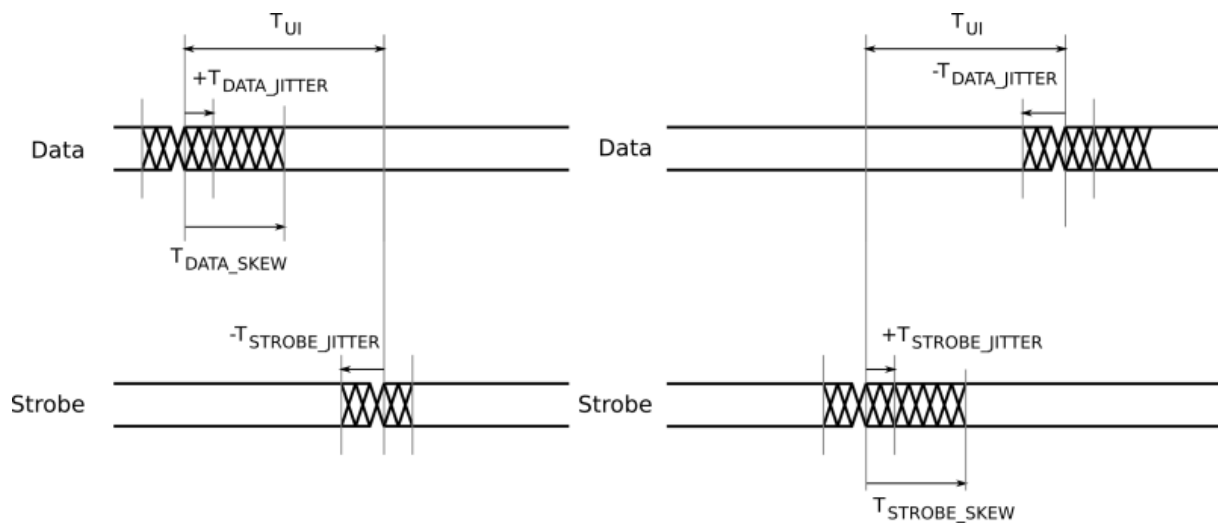


Figure 3.55: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate.	3.333
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 \cdot T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 \cdot T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.6.9 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.6.10 EMC Conformity

Declaration of Conformity

We

STAR-Dundee Ltd
STAR House, 166 Nethergate, Dundee, DD1 4EE, Scotland, UK

declare under our sole responsibility that the product

SpaceWire Brick Mk3

to which this declaration relates is in conformity with the following standard(s) or other normative documents

EN55022:2010
EN55024:2010

following the provisions of the EMC Directive.

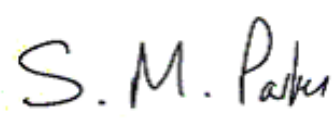
Dundee, 19th February 2015	
----------------------------	---

Figure 3.56: S.M. Parkes

(place and date of issue)	(name and signature of authorised person)
---------------------------	---

This is a class B product, and may therefore be used in a domestic environment.

3.7 SpaceWire PXI Interface and PXI Interface Mk2

Documentation for the STAR-Dundee SpaceWire PXI and PXI Mk2, Interface and Interface with RMAP Target devices is provided in this section.

3.7.1 Overview

This document provides an overview of the interfaces and features of the SpaceWire PXI and PXI Mk2, Interface card and Interface card with RMAP Target card.

The SpaceWire PXI card is a versatile SpaceWire Interface, Router or optional RMAP target device. The card is designed to be an efficient host interface to a SpaceWire system or network, by utilising a cPCI or PXI backplane.

The features of the SpaceWire PXI Interface card types are listed below.

- **SpaceWire PXI Interface:** 4 port SpaceWire Interface device with 4 data channels and 1 control channel for direct access to the SpaceWire ports through the PXI interface.
- **SpaceWire PXI Interface with RMAP Target:** 4 port SpaceWire Interface device with 4 embedded configurable Remote Memory Access Protocol (RMAP) targets and 1 GByte of DDR3 memory.

The Interface and RMAP Target cards can take advantage of the trigger connections on the PXI card front panel to sequence events in the card such as the transmission of time-codes or data packets. The interface card has four general purpose SMB trigger connectors which can be used to connect to the PXI backplane, sequence data transfers and events, and indicate events to an external device such as an oscilloscope.

The RMAP Target card has an option to directly connect up to four configurable RMAP targets to the SpaceWire interfaces to allow the card to act as a test card for remote memory transfers. Each RMAP target can be controlled and configured independently, and has access of up to 1 GByte of on board DDR3 memory.

Note that an upgrade is available for the Interface to include the RMAP Target functionality. Please contact STAR-Dundee for more information.

3.7.2 Hardware Description

The SpaceWire PXI card provides a cPCI or PXI system with four SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

The SpaceWire PXI card is shown in the figure below.

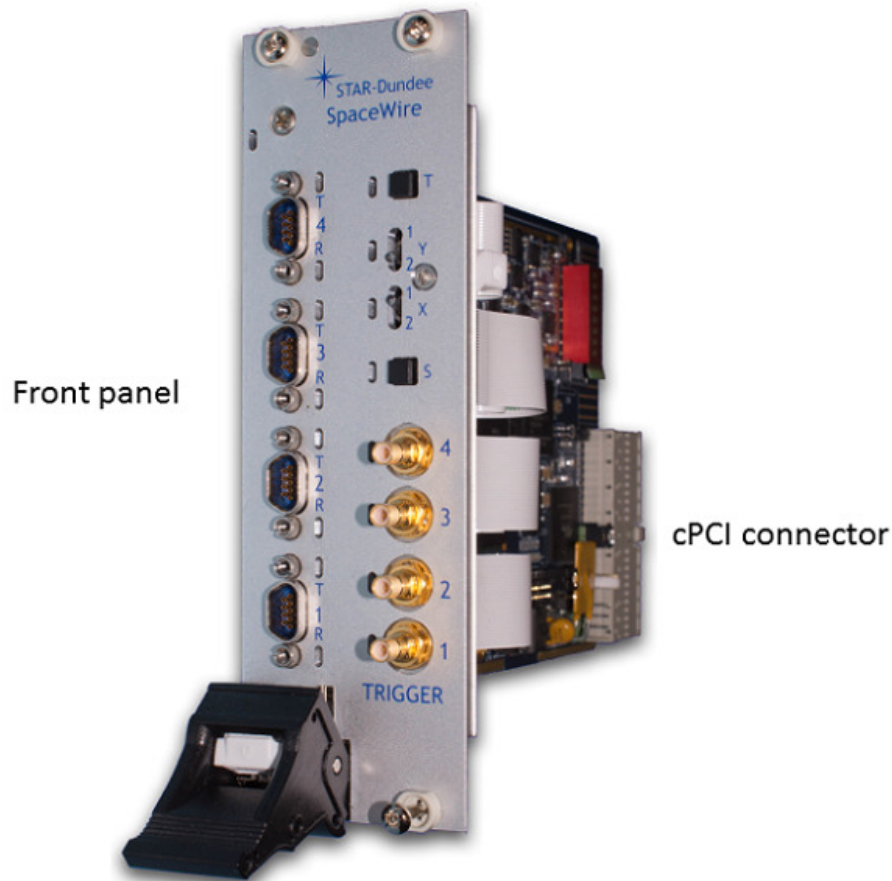


Figure 3.57: SpaceWire PXI Interface card

The card is comprised of a front panel where the SpaceWire and trigger interfaces are located, and a rear cPCI connector for insertion into a cPCI, PXI or PXIe rack - see the [Supported Systems](#) section.

The following sections describe the features of both cards.

3.7.2.1 SpaceWire PXI Interface Card

An overview of the SpaceWire PXI Interface card is shown in the figure below.

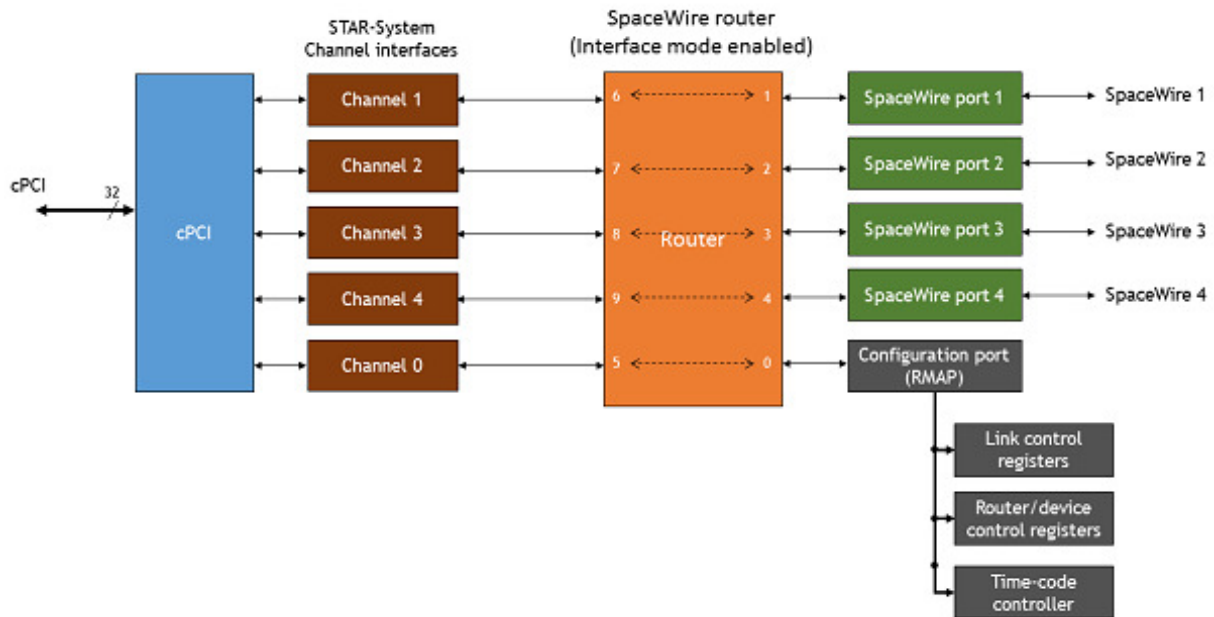


Figure 3.58: SpaceWire PXI Interface hardware overview

The four SpaceWire interfaces of the SpaceWire PXI Interface card are fully compliant with the SpaceWire standard and operate at up to 400 Mbit/s (FPGA versions prior to v1.03e05 supported a maximum link speed of 200 Mbit/s). They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode, where interface mode is the default mode. In Router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the cPCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire port are passed directly to the cPCI data channel for that port.

There are four independent channels from the SpaceWire router to the cPCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the PXI card regardless of the data flow.

The SpaceWire PXI card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.7.2.2 SpaceWire PXI Interface with RMAP Target

An overview of the functional blocks in the SpaceWire interface/RMAP card are shown in the figure below.

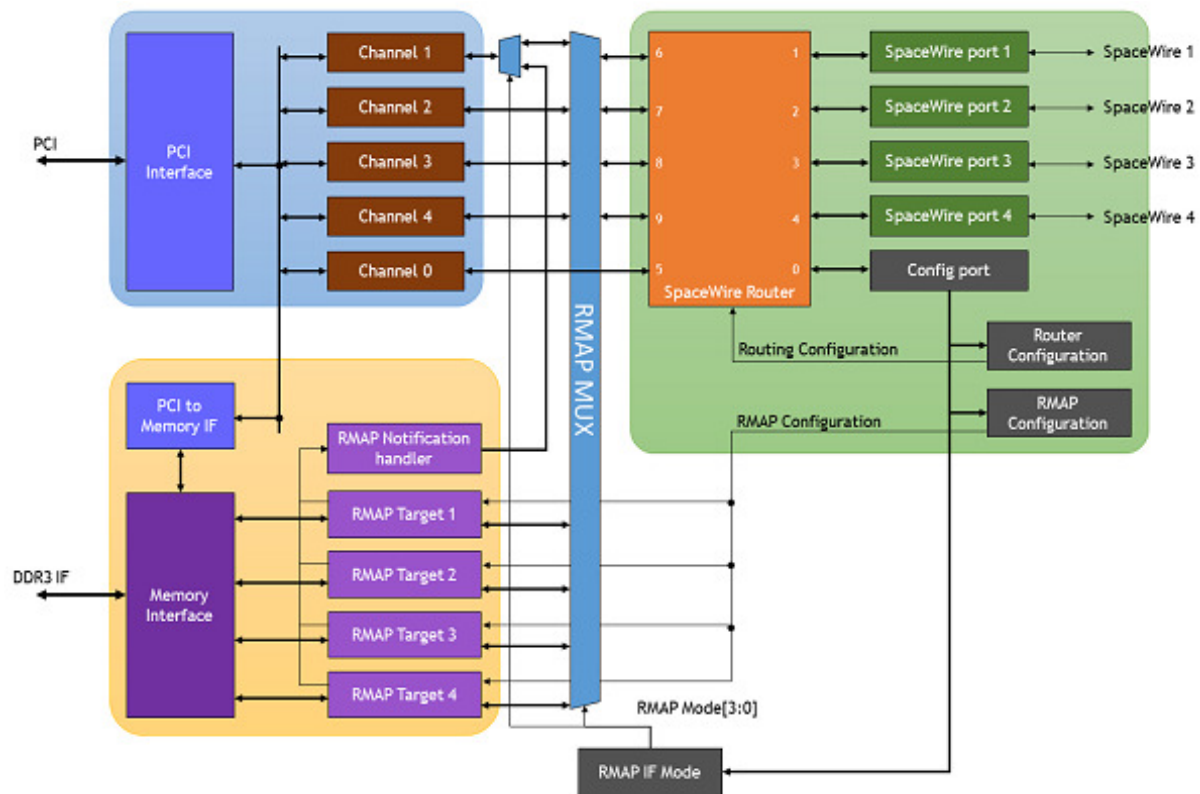


Figure 3.59: SpaceWire PXI Interface with RMAP Target hardware overview

The cPCI interface supports five channel interfaces, one of which is used by the STAR-System API for configuration purposes. The other channels are available for data transfer between the SpaceWire packets and application software dependent on the RMAP Mode setting. The SpaceWire ports and cPCI channel interfaces operate in a similar way to the PXI Interface card, listed in the section above.

The RMAP system comprises four RMAP targets, a notification handler and an access port to 1 GByte of on-board memory.

The SpaceWire interfaces and Router are configured using the STAR-System configuration API over the configuration port of the Router. The RMAP targets are also configured in the same way and are supported by the STAR-System **RMAP Target API**.

The RMAP IF Mode configuration bits control the RMAP MUX multiplexer and the RMAP Notification multiplexer. The configuration is identified as "RMAP Mode" in each of the following sections.

The SpaceWire PXI card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.7.3 Getting Started

The SpaceWire PXI card is powered over a cPCI or PXI backplane; therefore no external power supply is required. Correct anti-static discharge procedures should be observed when inserting the PXI card into the rack to ensure that no damage occurs to the components on the printed circuit board. Once the board is fully seated in the rack and the ejector handle is closed, it should be screwed in place using the four securing screws before use.

The SpaceWire PXI software and drivers, provided by STAR-System, should be installed on the host computer before connecting the SpaceWire PXI card. This ensures that the device is recognised by the host when it is

connected.

This document describes the host interface connections which are supported by the SpaceWire PXI card.

3.7.3.1 Front Panel Interfaces

The front panel interfaces of the SpaceWire PXI card are shown in the figure below.



Figure 3.60: SpaceWire PXI card front panel interfaces and LED positions

3.7.3.2 SpaceWire Interfaces

The SpaceWire PXI card has two multicolour LEDs for each SpW port. The lower LED on each port shows received data for that port, and the higher LED on each port shows transmitted data for that port. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The lower and higher LEDs are steady amber. Default state of the port, no activity and no NULLs transferred.

Link running	Green	The lower and higher LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The lower and higher LEDs are turned red if an error is detected when the port is in the Run state.
Data Transmitted/Received	Blue	When data is received on the link, the lower LED turns blue. When data is transmitted on the link, the higher LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the higher LED turns white. When the device has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the lower LED turns white.
Identify	Flashing purple	The SpaceWire PXI devices can be forced to flash their front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.7.3.3 Trigger Interfaces

The SpaceWire PXI card has four bi-directional trigger SMB trigger interfaces which can be used to trigger actions such as sending a SpaceWire packet or indicate activity by providing a trigger output pulse. The triggers are 5V tolerant and the output trigger pulse can be configured using the STAR-System [Triggering API](#).

The trigger LEDs are set dependent on the trigger direction and the state of the trigger. When the trigger is an input the LED indicates if the trigger is enabled and when an external trigger occurs. When the trigger is an output the LED indicates if the trigger is armed and when the trigger is set. The LED colour map for the trigger LEDs is listed in the table below.

Function	LED Colour	Description
Not active	Amber	Default state of the LED. The trigger is not enabled or armed.
Armed/Enabled	Green	Set to green when the trigger is an input and is enabled or when the trigger is an output and the trigger is armed.

Trigger	Blue	Set to blue when an input or output trigger occurs and the trigger is armed or enabled.
---------	------	---

3.7.4 Supported Systems

The cPCI and PXI specifications which are referenced in this section are listed in the table below.

Reference	Document	Description
[1]	PICMG 2.0 D3.0 CompactPCI Specification - 1999-10-01	cPCI Specification
[2]	PXI Hardware Specification - Revision 2.1 February 4, 2003	PXI Specification
[3]	PXI Express Hardware Specification - Revision 1.0 August 22, 2005	PXIe Specification

The SpaceWire PXI card is a standard 3U cPCI/PXI card which uses the 32-bit form factor defined in those standards. The card dimensions are listed below.

- Height: 100mm
- Length: 160 mm

An overview of the 3U 32-bit format PXI card is shown below and defined in Figure 2.1 of the PXI specification [2].

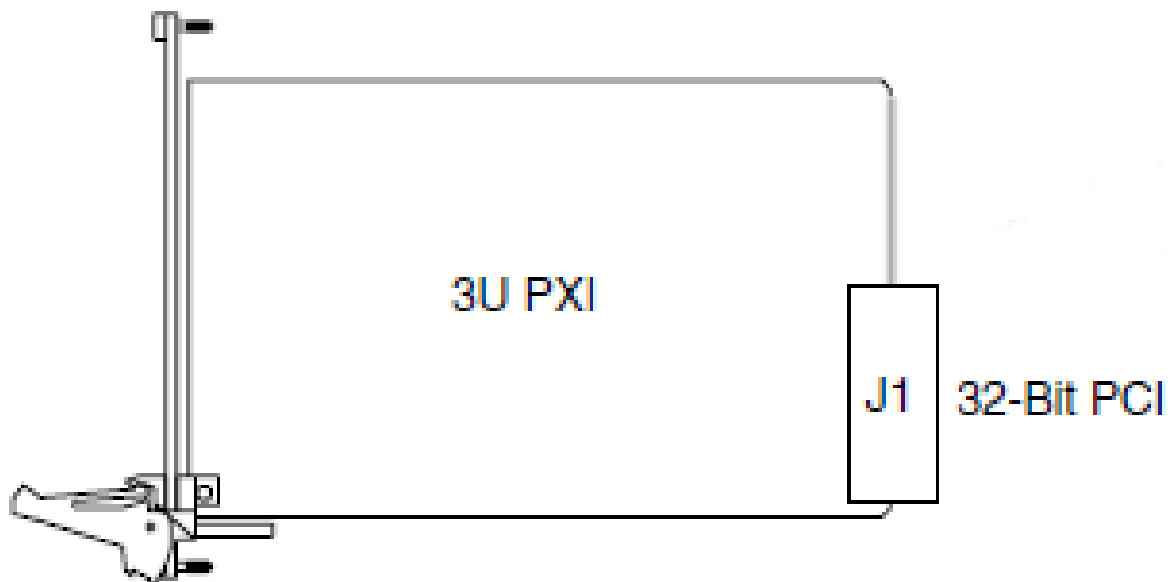


Figure 3.61: 3U PXI card form factor

3.7.4.1 cPCI Compatibility

The SpaceWire PXI card is compatible with all cPCI Peripheral slots defined in section 2.1 of the cPCI specification [1].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

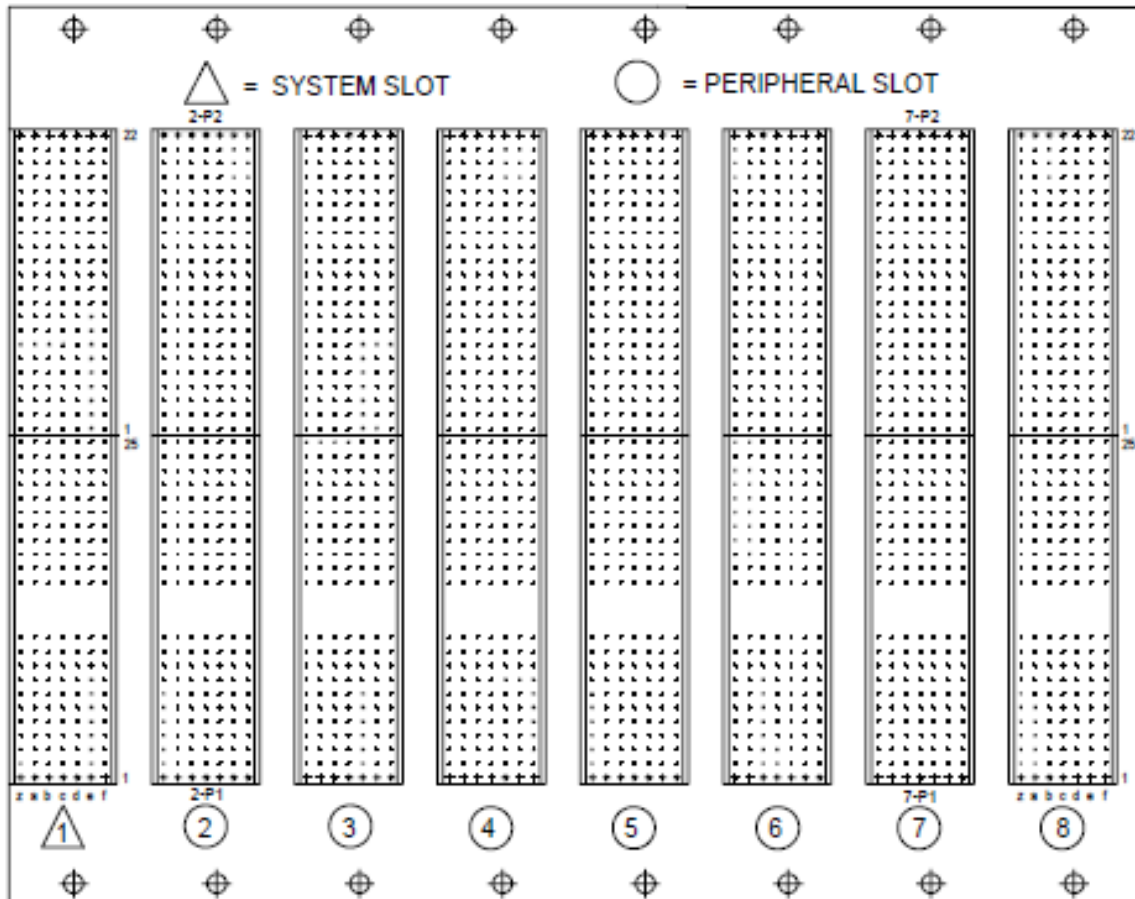


Figure 3.62: cPCI peripheral slots

3.7.4.2 PXI Compatibility

The SpaceWire PXI card is compatible with all PXI Peripheral slots defined in section 2.1 of the PXI specification [2].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

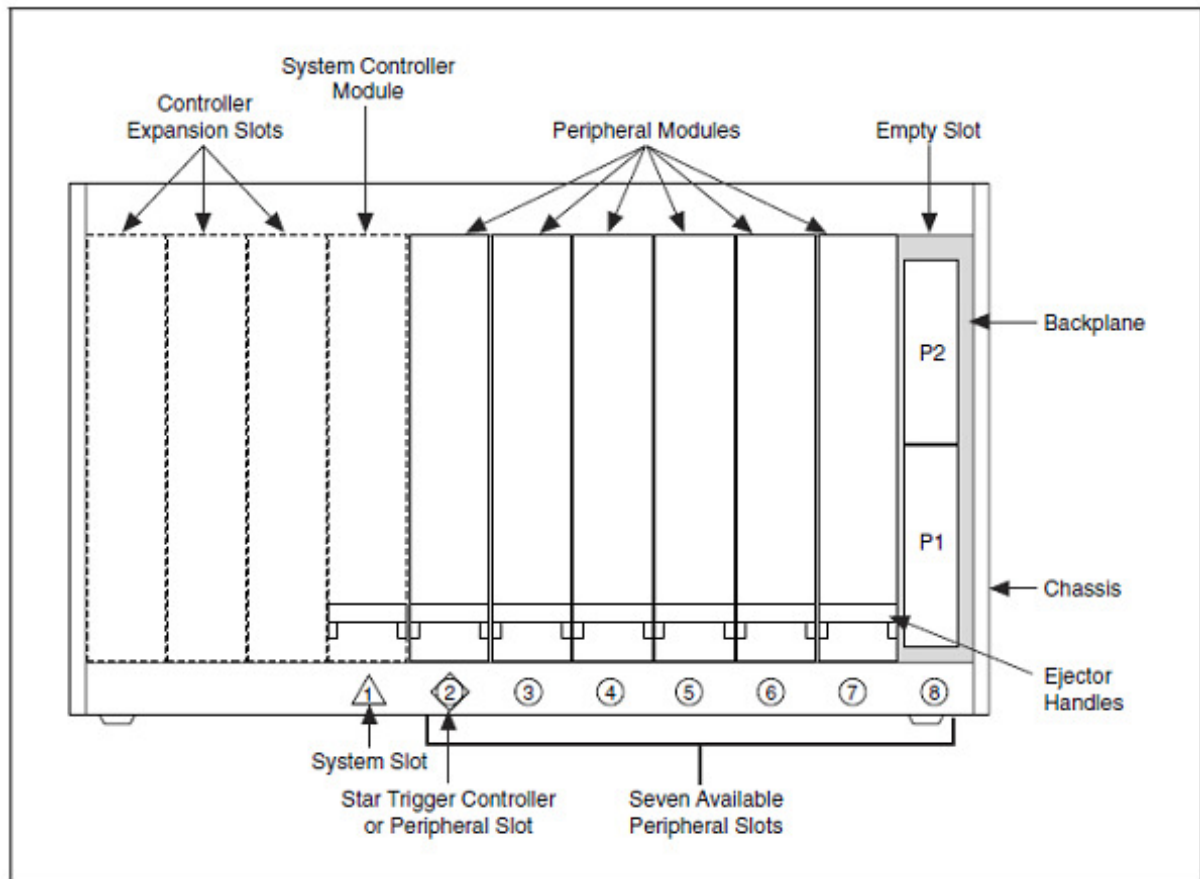


Figure 3.63: PXI peripheral slots

3.7.4.3 PXIe Compatibility

The SpaceWire PXI card is compatible with all PXIe PXI-1 and PXI Hybrid slots defined in section 2.1.1.5 and 2.1.1.6 of the PXIe specification [3].

PXI-1 slots are identified using a circle as shown in the reproduced figure below.

PXI Express Hybrid slots are identified using a filled circle with a capital H outside the circle, as shown in the reproduced figure below.

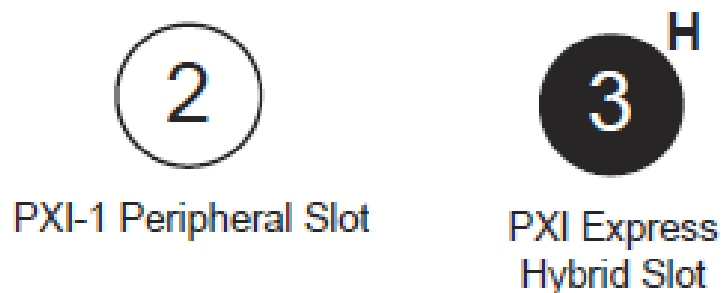


Figure 3.64: Compatible PXIe peripheral slots

3.7.5 Router and SpaceWire Interfaces

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire router can be configured using the STAR-System device configuration API functions.

The SpaceWire router has 10 ports in total as listed below:

- 1 internal router configuration port
- 4 external SpaceWire ports
- 5 cPCI channel interface external ports. By default four of these are used by the SpaceWire ports, while the other is used by the configuration port

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after power cycling or a device reset is performed.

3.7.5.1 Interface Mode

In interface mode, a packet which is received on a SpaceWire port will always be routed by the port routing configuration. The SpaceWire PXI card supports interface mode by utilising the port routing settings of the SpaceWire router. The default values are listed in the table below.

Source port	Destination port	Description
0	5	Configuration to external port 1 (host channel 0)
1	6	SpaceWire port 1 to external port 2 (host channel 1)
2	7	SpaceWire port 2 to external port 3 (host channel 2)
3	8	SpaceWire port 3 to external port 4 (host channel 3)
4	9	SpaceWire port 4 to external port 5 (host channel 4)
5	0	cPCI channel 0 to configuration port
6	1	cPCI channel 1 to SpaceWire port 1
7	2	cPCI channel 2 to SpaceWire port 2
8	3	cPCI channel 3 to SpaceWire port 3
9	4	cPCI channel 4 to SpaceWire port 4

3.7.5.2 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.

- Packets with unknown physical addresses, 10 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.7.5.3 Link Operating Speed

The SpaceWire link operating speed for each SpaceWire port is controlled by the STAR-System device configuration APIs. The link clock rate selection logic for each port is shown in the figure below.



Figure 3.65: SpaceWire link operating speed for each SpaceWire port

- Each SpaceWire link can be programmed to a link rate between 2 and 400 Mbit/s using the transmit clock configuration functions of the STAR-System device configuration APIs.
- The link operating speed after power on is 200 Mbit/s.
- Each SpaceWire link will operate at 10 Mbit/s at start-up.
- When the reconfigurable clock generator is reprogrammed the link rate will switch to 10 Mbit/s until the programming has completed, when the link is running.

3.7.5.4 Time-codes

A time-code can be received from the cPCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external cPCI ports, only port 5, corresponding to channel 0, can be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

3.7.5.5 Packet Timestamping

The SpaceWire PXI Interface provides an optional timestamp feature for each received packet. A timestamp will be recorded when the first byte of a packet is received and when the end of packet marker is received. The timestamp is synchronised with an external pulse input on SMB trigger A.

Multiple SpaceWire PXI Interface devices can be synchronised using the external pulse external time source to synchronise time across the devices.

3.7.5.5.1 Detailed Description

An overview of the timestamp functions provided by the SpaceWire PXI Interface are shown in the figure below. Configuration settings are shown in italics.

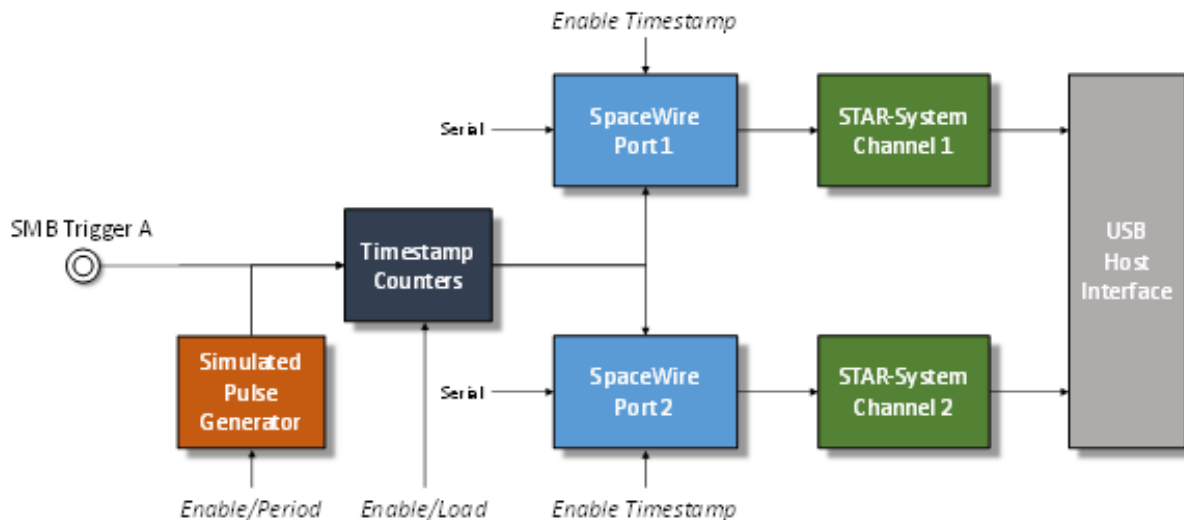


Figure 3.66: Timestamp system overview

Each SpaceWire port has a SpaceWire receiver which decodes serial data into link characters and data characters. Each data character can be an 8-bit packet data byte or an end of packet marker. The timestamping mechanism will record the timestamp for each start of packet byte and for each end of packet marker, effectively the start time and end time of each packet.

The timestamp counter is implemented as two counters, one which increments on the input SMB trigger or the pulse generator and one which increments on the system clock. The counters are identified as "SyncPulse" and "ClockCycles" respectively.

The packet start and end times are recorded using a combination of the counters described above, plus a capture register which captures the total number of clock cycles which elapsed in the previous period. An overview of the counter and register operation is listed below.

- The "SyncPulse" counter can be loaded by software to provide a starting value for the timestamp. The counter will increment on each trigger pulse.
- The "ClockCycles" counter is incremented on each system clock cycle and is reset on each pulse of the trigger. It provides a count of the system clock cycles which have elapsed since the last synchronisation pulse.
- The "TotalCycles" register is captured into a storage register on each synchronisation pulse to provide a mechanism to convert the "ClockCycles" elapsed time into fractions of the synchronisation period, i.e. $\frac{\text{ClockCycles}}{\text{TotalCycles}}$ is the fraction between 0 and 1 of the synchronisation period at which the event occurred.

The timestamp is synchronised using an external trigger pulse which supports frequencies equal to or greater than 1 Hz. The mechanism provides additional meta-data for each received SpaceWire packet which can be received by applications to compute the time at which the packet was decoded by the SpaceWire receiver.

A simulated pulse generator module can be used to simulate the external SMB trigger for testing and development purposes. The generator is programmed with the period between pulses in system clock cycles, and can be enabled

or disabled.

3.7.5.5.2 Computing the "User" time from the recorded Timestamp

A method to convert the timestamp fields into "User" time is listed below. The raw timestamp is returned by the API therefore user applications can implement a different algorithm if required.

The synchronisation pulse frequency is set by the user application. In the equations below the frequency is referred to as F_{sync} .

The computed time is defined as x.y where x is the number of seconds and y indicates the fraction of seconds between 0 and 1.

The example steps to compute the "User" time are listed below.

1. The first step is to convert the hardware format into seconds and sub seconds as shown below.

$$\begin{aligned} Seconds &= SyncPulse / F_{sync} \\ SubSeconds &= ClockCycles / (TotalCycles * F_{sync}) \end{aligned}$$

2. The resultant values are then added together.

$$x.y = Seconds + SubSeconds$$

An example case is listed below.

1. The application setup is listed below:

- (a) F_{sync} is 10 Hz
- (b) "SyncPulse" counter is 1234
- (c) "ClockCycles" counter is 234567
- (d) "TotalCycles" counter is 301210

2. The resultant time in seconds and fractions of seconds is computed as:

$$\begin{aligned} Seconds &= 1234 / 10 \\ &= 123.4 \\ SubSeconds &= \frac{234567}{301210 * F_{sync}} \\ &= 77.8749045516E - 3 \\ x.y &= 123.477874905seconds \end{aligned}$$

3.7.6 RMAP Target

The SpaceWire PXI Interface card with RMAP target has four embedded RMAP target ports which are compliant with the RMAP protocol standard document, ECSS-E-ST-50-52C.

Each RMAP target can be accessed through the SpaceWire ports or cPCI channel interfaces (with some restrictions) dependent on the configuration which is set through the configuration API. Each target can directly access up to 1 Gbyte of on-board DDR3 memory.

An overview of the RMAP functionality available for each of the RMAP targets is listed below and shown in the following figure.

- Decode RMAP command packets.
- Encode RMAP reply packets.
- Authorise commands using two modes of operation:
 - Notify the host when an RMAP command packet arrives. Wait for the host to authorise the command.
 - Authorise a command using a set of authorisation register which are configured by the host.
- Access memory directly or through a logical to physical address offset memory mapping.
 - Host notification can be enabled when the memory access has completed.

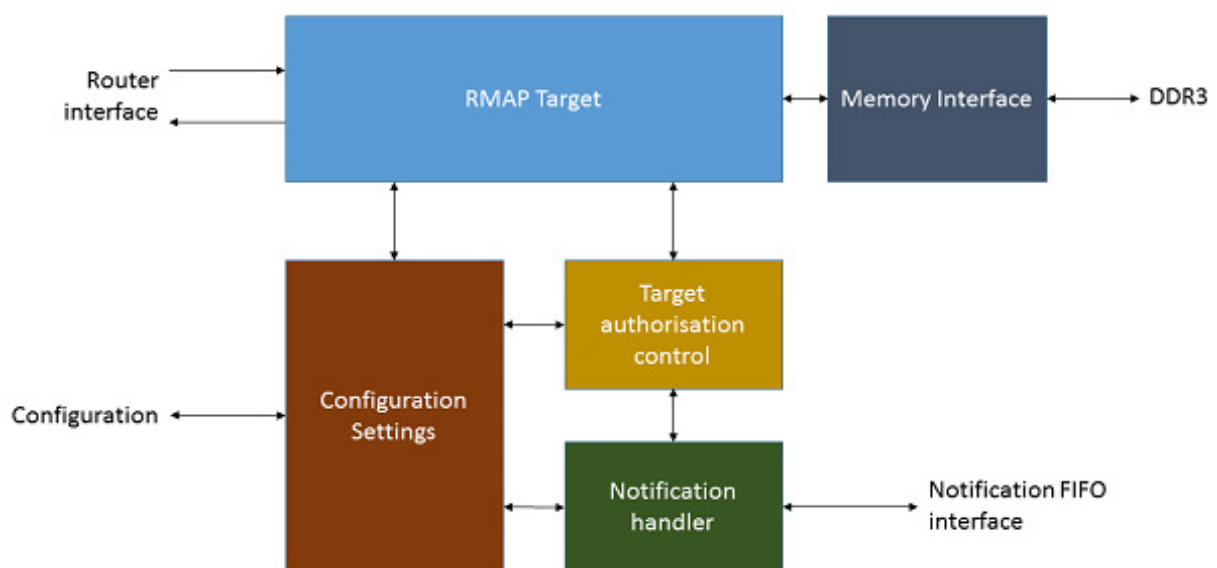


Figure 3.67: RMAP target architecture overview

3.7.6.1 Configuration and Access Modes

The configuration of the RMAP Target card is set using the STAR-System [RMAP Target API](#). The configuration items which determine the set-up of the RMAP card flow set-up is listed below.

Configuration	Default	Description
RMAP Mode[0]	Disabled	The RMAP mode flags change the settings of the RMAP MUX which determine if the RMAP target ports or the cPCI channel interfaces are connected to the SpaceWire Router ports. RMAP Mode[0] also controls the RMAP Notification multiplexer. When RMAP Mode[0] is enabled, any RMAP notifications will be sent to the STAR-System API using cPCI channel 1.
RMAP Mode[3:1]	Disabled	The RMAP mode flags change the settings of the RMAP MUX which determine if the RMAP target ports or the cPCI channel interfaces are connected to the SpaceWire Router ports.
Interface Mode	Enabled	The routing configuration of the SpaceWire router can be changed between Router Mode and Interface Mode. The default is Interface Mode. In Interface Mode, packets are routed dependent on a configurable port number and the leading address of the packet is not used. The default set-up connects SpaceWire port 1 to Router port 6, SpaceWire port 2 to Router port 7, and so on. The configuration port is connected to Router port 5. In Router Mode, packets are routed dependent on the leading packet address. Advanced routing and interface mode configuration set-ups are supported where some ports are configured in interface mode and have a fixed routing set-up, and some ports are configured to check the leading packet address to determine the packet destination.

RMAP Target Configuration		The RMAP Target configuration is set by the STAR-System RMAP Target API . Each target can be configured independently to access a different region of the 1 GByte memory space and respond to different logical addresses, commands, etc. The target configuration also sets the notification.
---------------------------	--	--

3.7.6.2 Memory Access

The on board memory interface is accessible from the RMAP targets and cPCI interface. The functionality of the memory interface is listed below:

- Four RMAP target interfaces for remote memory access and simulation.
- cPCI interface which is shared with RMAP target 1.
- Each RMAP target can be assigned a region of memory using a physical memory OFFSET and a lower and upper limit to memory access. Wrapping memory accesses are not supported.

The physical memory location which is accessed by an RMAP target is defined by the target OFFSET register and the address field in the RMAP packet.

The *RMAPOFFSET* address is defined in 1Mbyte regions as shown below.

$$PHYSICALMEMORYADDRESS = (2^{20} \times RMAPOFFSET) + RMAPADDRESS$$

The physical memory location which is accessed by an RMAP target is defined by the local bus memory OFFSET register (*LOCALOFFSET*) and the PLX local bus address pins.

The *LOCALOFFSET* address is defined in 16Mbyte chunks.

$$PHYSICALMEMORYADDRESS = (2^{24} \times LOCALOFFSET) + LA$$

3.7.7 SpaceWire Interface Tristate Mode

The SpaceWire ports on the PXI card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Start, Disabled or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.
If a link on the PXI card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains

LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

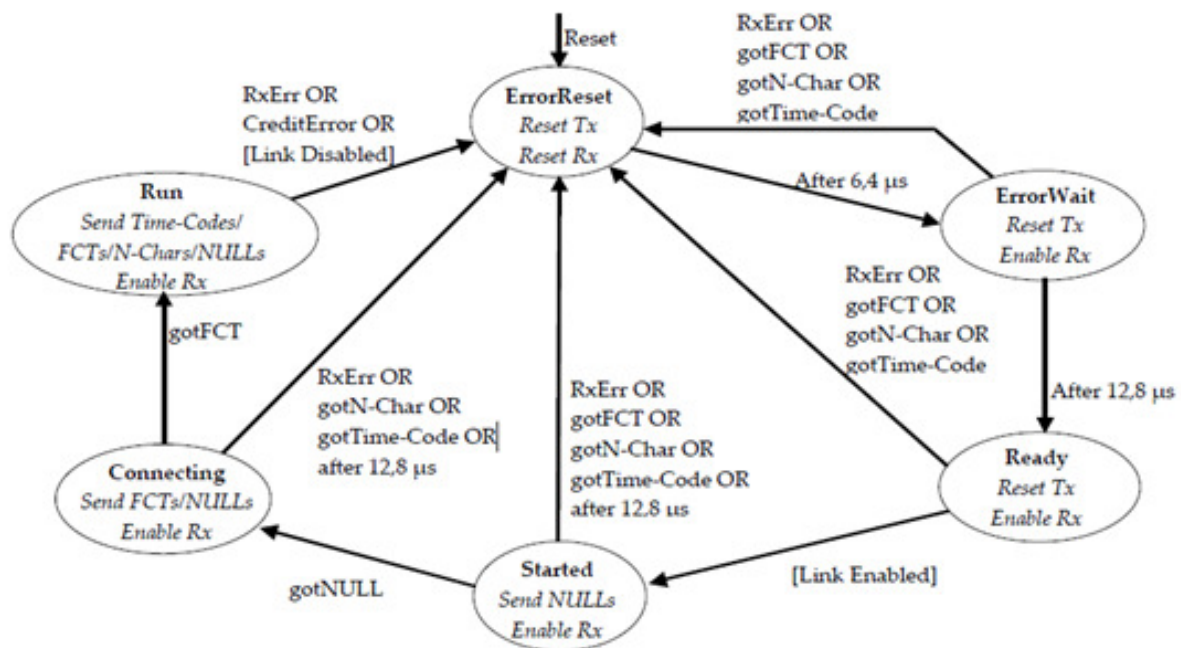


Figure 3.68: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

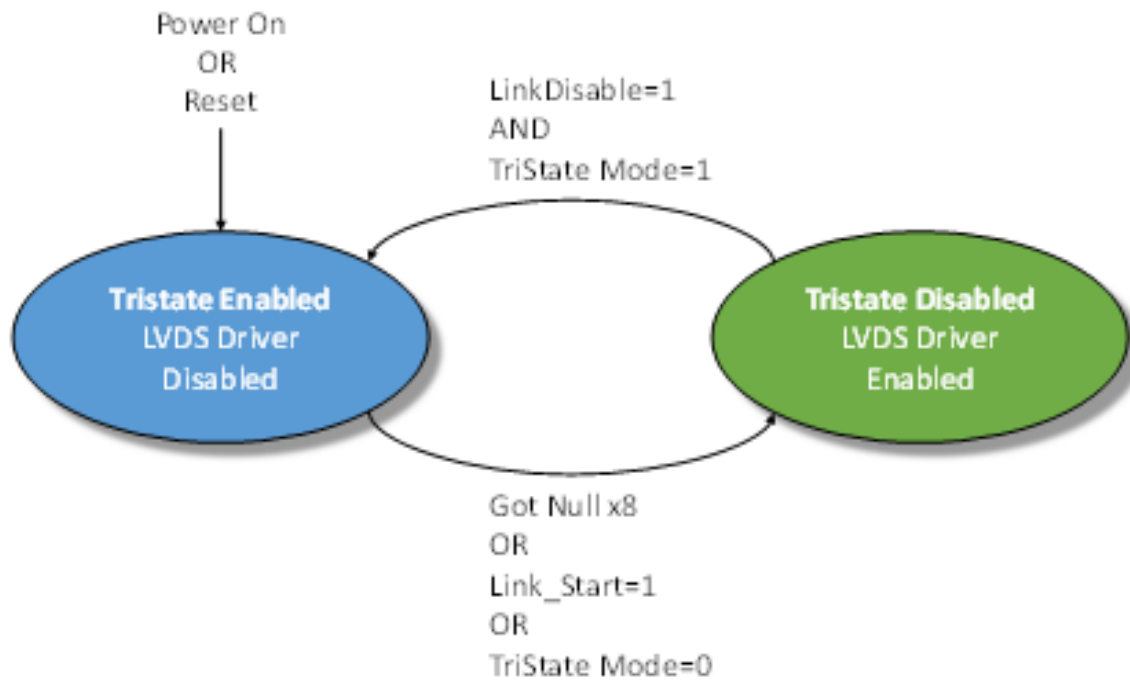


Figure 3.69: Tristate controller state machine

An approximate 60 microsecond delay occurs between the state machine moving from the Enabled to Disabled state, i.e. the LVDS drivers are not enabled until approximately 60 microseconds after the condition to enable the drivers has been detected.

A device which connects to the SpaceWire PXI card and sends NULLs can move through the SpaceWire link state machine a few times until the LVDS drivers are enabled. The exchange of silence period is 19.2 microseconds and a link which is enabled will send NULLs in the Started state for a period of 12.8 microseconds. The link will start normally after the 60 microsecond period – the user does not need to take any action to operate correctly with the PXI card in this mode.

An example start-up sequence when the PXI link is in AutoStart mode is shown in the figure below.

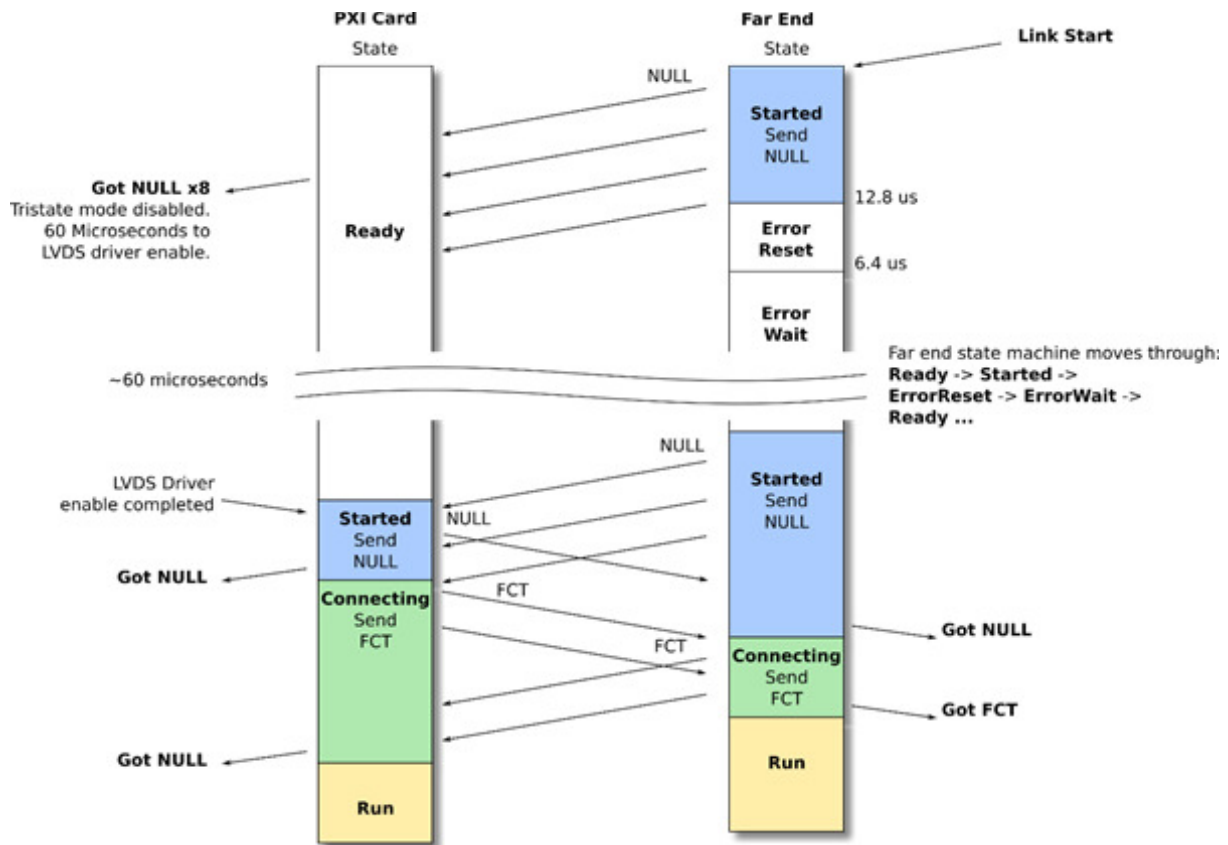


Figure 3.70: AutoStart link operation

In the figure above the Far End link state machine sends NULL characters after its Link Start control bit is set. The PXI card detects the NULL sequence and indicates to the Tristate controller state machine when 8 consecutive NULLs are received without error. The PXI card then enables its LVDS drivers after 60 microseconds and will remain in the Ready state until it detects a start-up NULL from the far end. Both ends then exchange NULL and FCT characters and start-up normally.

An example start-up sequence when the PXI link is set to start is shown below.

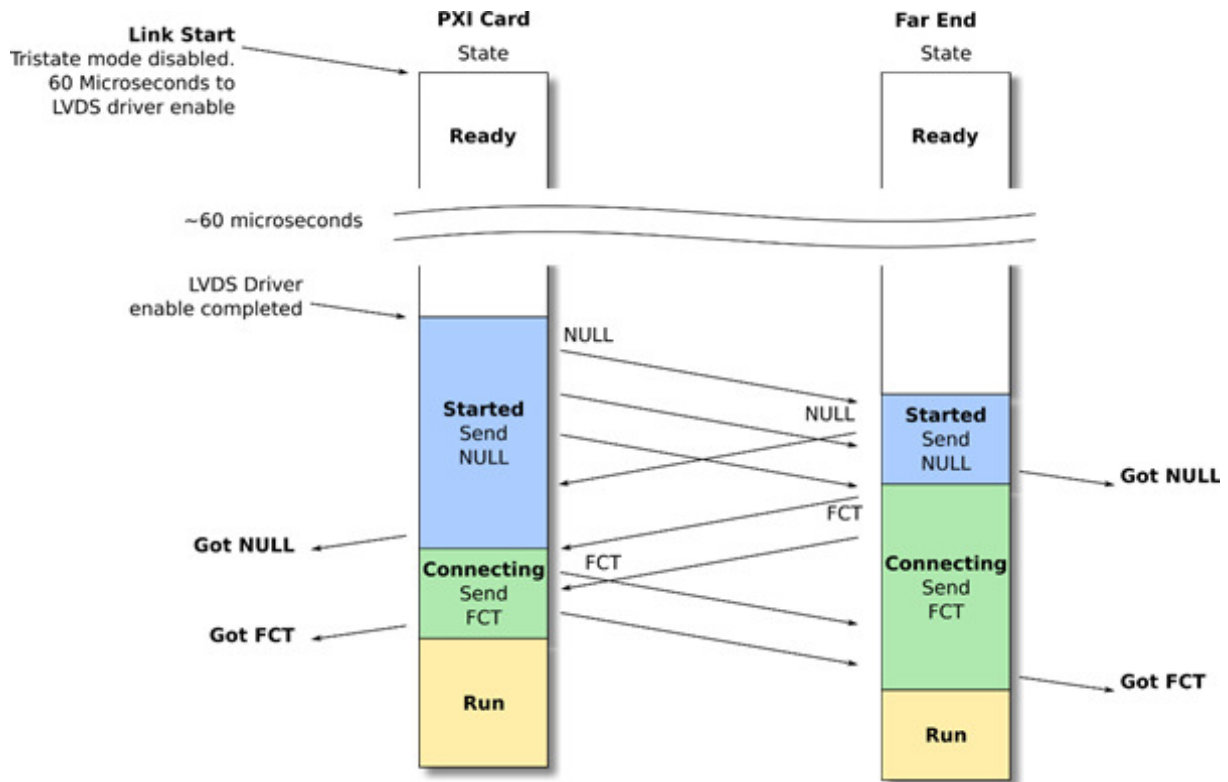


Figure 3.71: Link Start link operation

In the figure above the PXI Card Link Start bit is set and the LVDS driver enable sequence is started. After 60 microseconds the PXI card link will move to the Started state and will send NULL characters. The Far End will detect NULL characters, both ends will exchange NULLs and FCTs and will start-up normally.

Please note, if a SpaceWire Link should start-up on the first exchange of NULL characters then the Tristate setting for that link should be disabled.

3.7.8 Electrical Characteristics

3.7.8.1 Absolute Maximum Ratings

3.7.8.1.1 PXI Interface Mk1

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.7	4.3	V
SpaceWire input voltage on Vout+, Vout- (see the figure in the following section)	-0.5	4	V

External Trigger input voltage	-0.5	6	V
--------------------------------	------	---	---

3.7.8.1.2 PXI Interface Mk2

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.5	4.3	V
SpaceWire input voltage on Vout+, Vout- (see the figure in the following section)	-0.5	4.3	V
External Trigger input voltage	-0.5	6	V

3.7.8.2 SpaceWire Functional Diagram

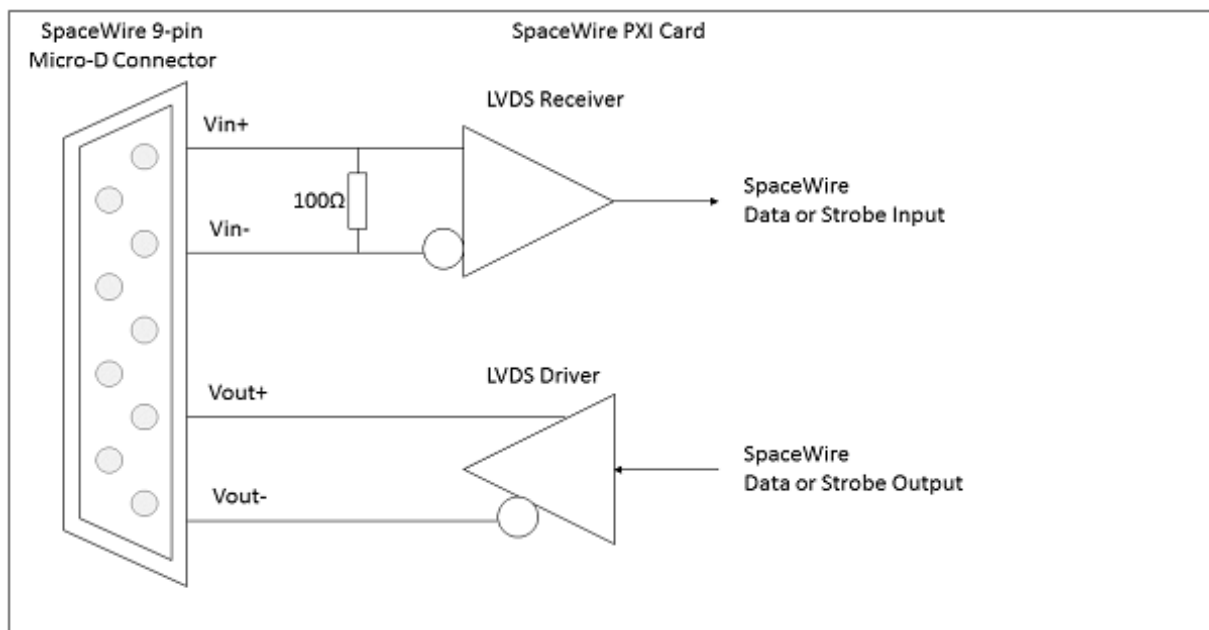


Figure 3.72: SpaceWire functional diagram (one LVDS driver/receiver shown)

Each SpaceWire port is comprised of a pair of LVDS drivers and LVDS receivers, one pair for data and strobe in, and one pair for data and strobe out.

3.7.8.2.1 SpaceWire Input Electrical Characteristics

The SpaceWire recommended input electrical characteristics are shown in the tables below.

3.7.8.2.1.1 PXI Interface Mk1

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-)	0.1	1	V

Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	4	V
SpaceWire input Differential resistance	98	102	Ω

3.7.8.2.1.2 PXI Interface Mk2

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-)	0.1	1	V
Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	3.3	V
SpaceWire input Differential resistance	98	102	Ω

3.7.8.2.2 SpaceWire Output Electrical Characteristics

The SpaceWire output electrical characteristics are shown in the tables below.

3.7.8.2.2.1 PXI Interface Mk1

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential mag- nitude - 100 Ω load	247	310	454	mV
SpaceWire output common mode voltage - 100 Ω load	1.125		1.375	V
Output voltage to an unpowered unit	Vin+/Vin-	1.12	1.16	V
	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	V
	Vout+/Vout- (Enabled - programmed by software)	0.898	1.602	V

3.7.8.2.2.2 PXI Interface Mk2

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential mag- nitude - 100 Ω load	247	330	454	mV
SpaceWire output common mode voltage - 100 Ω load	1.125		1.375	V
Output voltage to an unpowered unit	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	V

	Vout+ / Vout- (Enabled - programmed by software)	0.898		1.602	V
--	---	-------	--	-------	---

(1) Typical conditions at 25 °C

Additional information

The STAR-Dundee PXI card can programmatically tristate its LVDS drivers.

3.7.8.3 External Trigger Connectors

3.7.8.3.1 External Trigger Functional Diagram

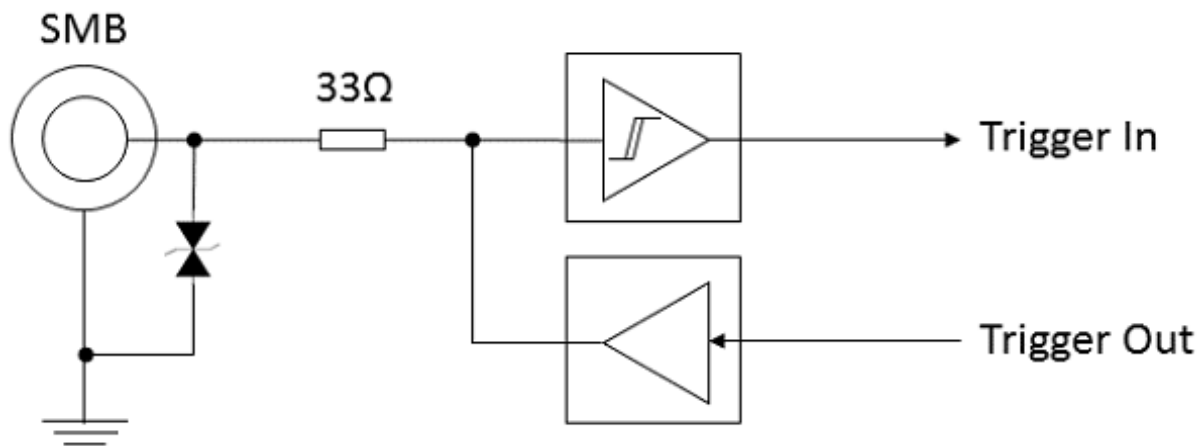


Figure 3.73: External trigger functional diagram

3.7.8.3.2 Absolute Maximum Values

The external trigger **absolute maximum** input ratings are defined in the table below. The device can withstand these maximum values but use at or near these values is not recommended. Note that each trigger is only 5 Volt input tolerant when configured as an input (the default on power up), or when the device is powered down. Triggers configured as outputs are not 5 Volt tolerant, and may be damaged if an external voltage is applied to them. It is recommended to use 3.3 Volt logic when using the triggers.

	Description	Condition	Min	Max	Unit
V _{IN}	Voltage applied to trigger	Trigger configured as input, or device powered down	-0.5	6.5	Volts
		Trigger configured as output	-0.5	3.8	Volts
I _{OUT}	Continuous output current	Trigger configured as output	-50	+50	mA

3.7.8.3.3 DC Characteristics

The DC characteristics of the external trigger is listed in the table below. The output is driven by 3.3 Volt LVCMOS logic.

	V_{IL} V, Min	V, Max	V_{IH} V, Min	V, Max	V_{OL} V, Max	V_{OH} V, Min
Trigger IN	0	0.8	2.0	5.5		
Trigger OUT (100 μ A load)					0.1	3.2
Trigger OUT (24 mA load)					0.55	2.3

V_{IL}	Input low voltage
V_{IH}	Input high voltage
V_{OL}	Output low voltage
V_{OH}	Output high voltage

These values apply to both the PXI Interface Mk1 and PXI Interface Mk2.

3.7.9 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

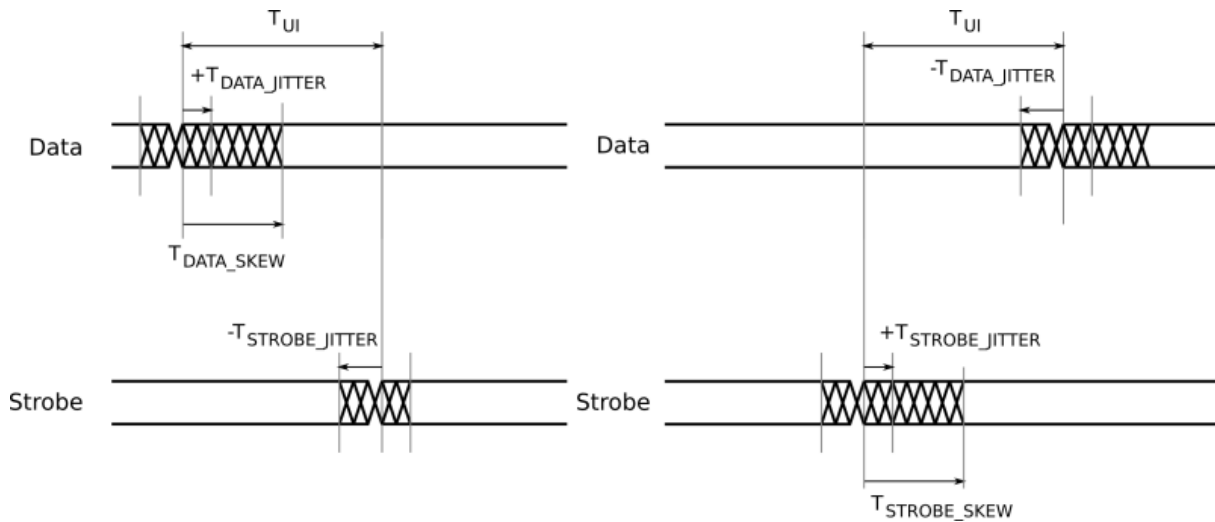


Figure 3.74: SpaceWire input switching characteristics

Parameter	Description	ns (Max)
T_{UI}	Transmitter bit rate.	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

Parameter	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0.1
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.8 SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

Documentation for the STAR-Dundee SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 card is provided in this section.

3.8.1 Overview

This document provides an overview of the interfaces and features of the SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 card.

The SpaceWire PXI 12 Port Router card is a versatile SpaceWire Interface or Router device. The card is designed to be an efficient host interface to a SpaceWire system or network, by utilising a cPCI or PXI backplane.

3.8.2 Hardware Description

The SpaceWire PXI 12 Port Router card provides a cPCI or PXI system with 12 SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host PC memory.

The SpaceWire PXI 12 Port Router card is shown in the figure below.

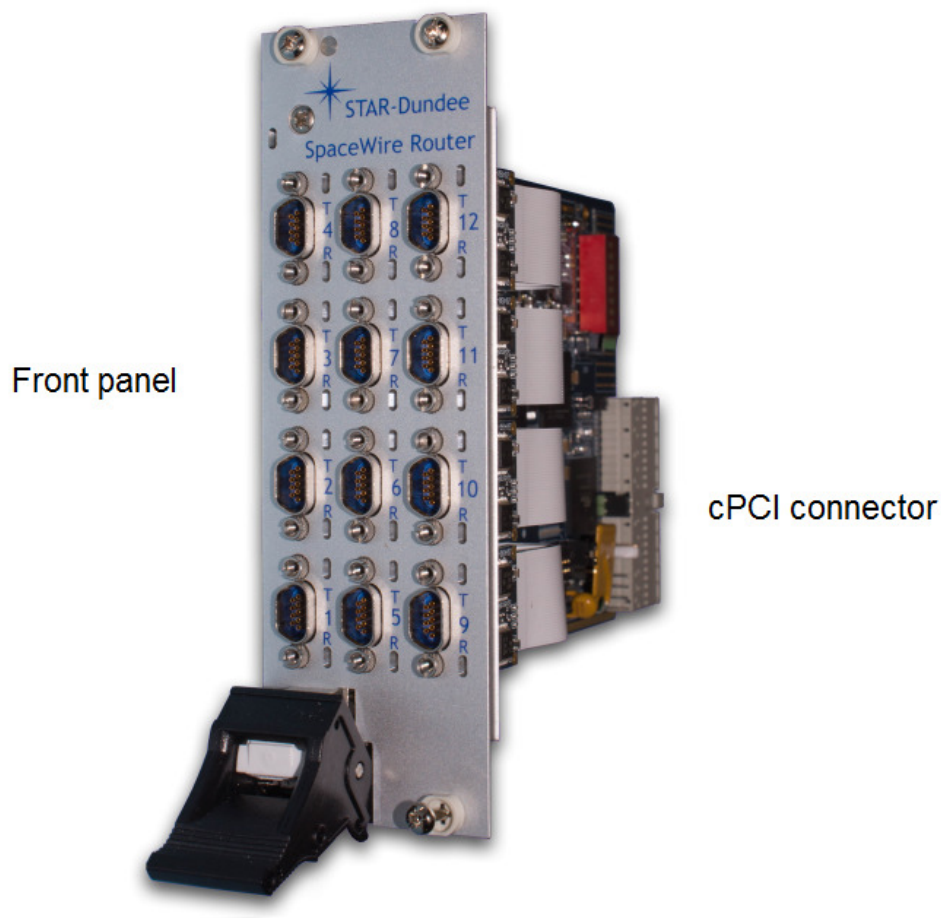


Figure 3.75: SpaceWire PXI 12 Port Router card

The card is comprised of a front panel where the SpaceWire interfaces are located, and a rear cPCI connector for insertion into a cPCI, PXI or PXIe rack - see the [Supported Systems](#) section.

The following section describes the features of the card.

3.8.2.1 SpaceWire SpaceWire PXI 12 Port Router Card

An overview of the SpaceWire PXI 12 Port Router card is shown in the figure below.

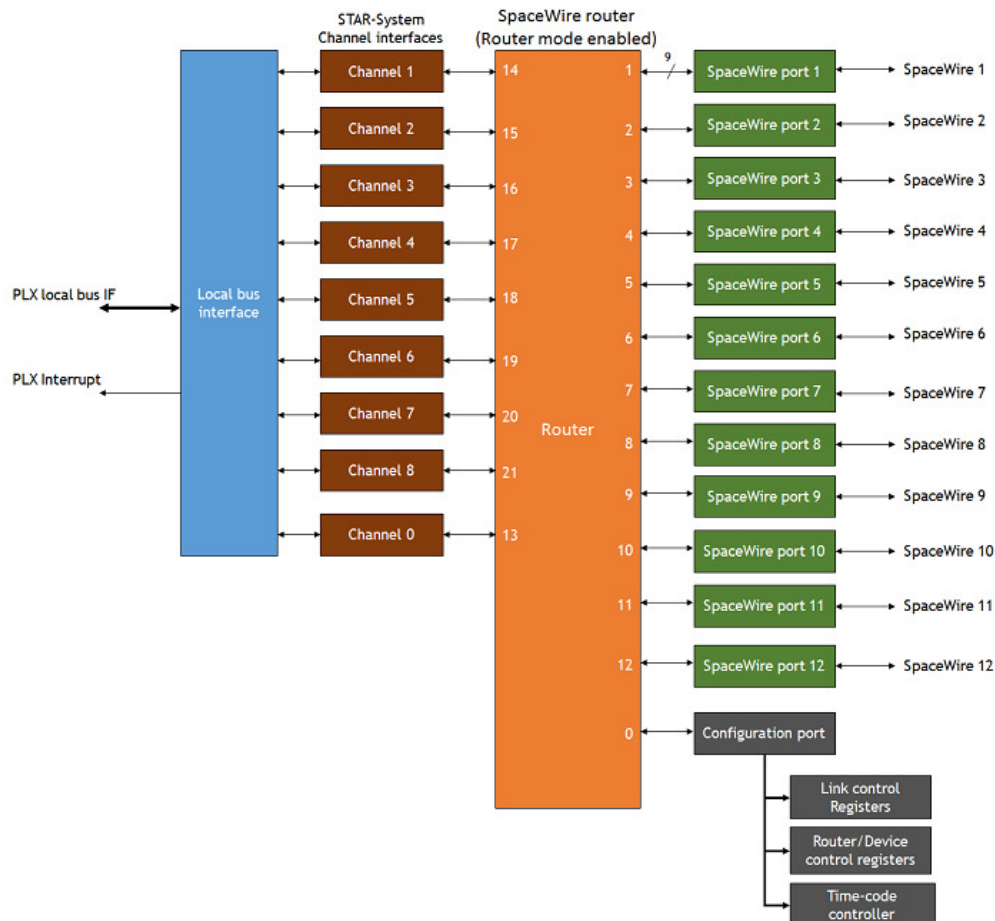


Figure 3.76: SpaceWire PXI 12 Port Router hardware overview

The 12 SpaceWire interfaces of the SpaceWire PXI Interface card are fully compliant with the SpaceWire standard and operate at up to 400 Mbit/s (FPGA versions prior to v1.03e01 supported a maximum link speed of 200 Mbit/s). They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode, where interface mode is the default mode. In Router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the cPCI interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire port are passed directly to the cPCI data channel for that port.

There are eight independent channels from the SpaceWire router to the cPCI interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the PXI card regardless of the data flow.

The SpaceWire PXI 12 Port Router card includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.8.3 Getting Started

The SpaceWire PXI 12 Port Router card is powered over a cPCI or PXI backplane; therefore no external power supply is required. Correct anti-static discharge procedures should be observed when inserting the PXI card into the rack to ensure that no damage occurs to the components on the printed circuit board. Once the board is fully seated in the rack and the ejector handle is closed, it should be screwed in place using the four securing screws before use.

The SpaceWire PXI software and drivers, provided by STAR-System, should be installed on the host computer before connecting the SpaceWire PXI 12 Port Router card. This ensures that the device is recognised by the host when it is connected.

This document describes the host interface connections which are supported by the SpaceWire PXI 12 Port Router card.

3.8.3.1 Front Panel Interfaces

The front panel interfaces of the SpaceWire PXI 12 Port Router card are shown in the figure below.

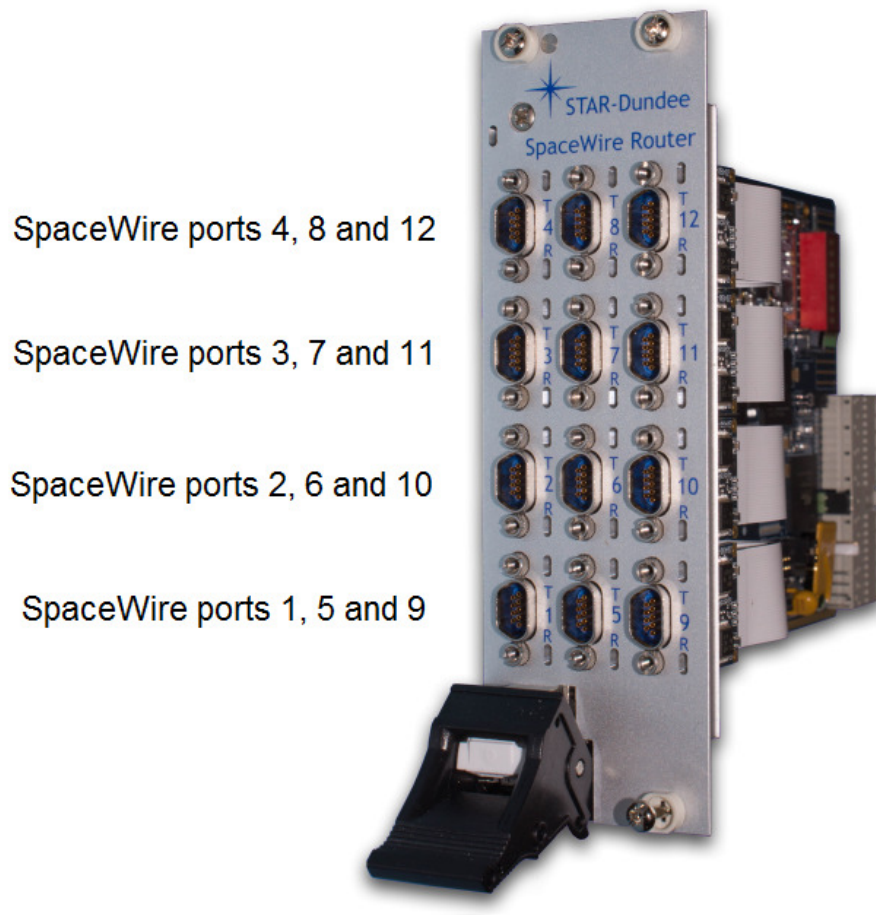


Figure 3.77: SpaceWire PXI 12 Port Router card front panel interfaces and LED positions

3.8.3.2 SpaceWire Interfaces

The SpaceWire PXI 12 Port Router card has two multicolour LEDs for each SpW port. The lower LED on each port shows received data for that port, and the higher LED on each port shows transmitted data for that port. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The lower and higher LEDs are steady amber. Default state of the port, no activity and no NULLs transferred.
Link running	Green	The lower and higher LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The lower and higher LEDs are turned red if an error is detected when the port is in the Run state.
Data Transmitted/Received	Blue	When data is received on the link, the lower LED turns blue. When data is transmitted on the link, the higher LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the higher LED turns white. When the device has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the lower LED turns white.
Identify	Flashing purple	The SpaceWire PXI devices can be forced to flash their front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.8.4 Supported Systems

The cPCI and PXI specifications which are referenced in this section are listed in the table below.

Reference	Document	Description
[1]	PICMG 2.0 D3.0 CompactPCI Specification - 1999-10-01	cPCI Specification

[2]	PXI Hardware Specification - Revision 2.1 February 4, 2003	PXI Specification
[3]	PXI Express Hardware Specification - Revision 1.0 August 22, 2005	PXIe Specification

The SpaceWire PXI 12 Port Router card is a standard 3U cPCI/PXI card which uses the 32-bit form factor defined in those standards. The card dimensions are listed below.

- Height: 100mm
- Length: 160 mm

An overview of the 3U 32-bit format PXI card is shown below and defined in Figure 2.1 of the PXI specification [2].

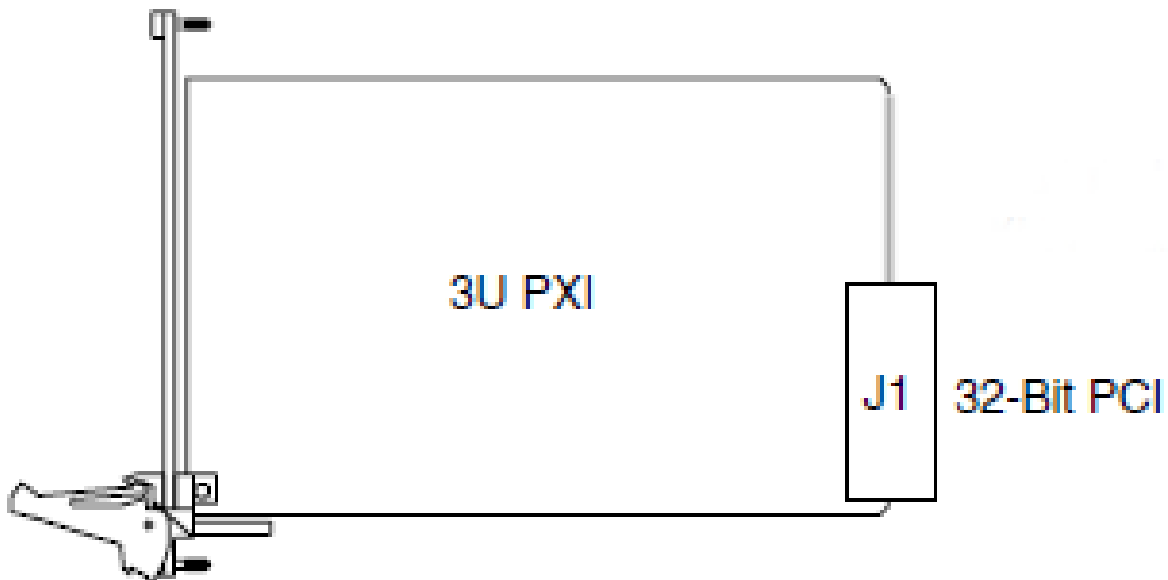


Figure 3.78: 3U PXI card form factor

3.8.4.1 cPCI Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all cPCI Peripheral slots defined in section 2.1 of the cPCI specification [1].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

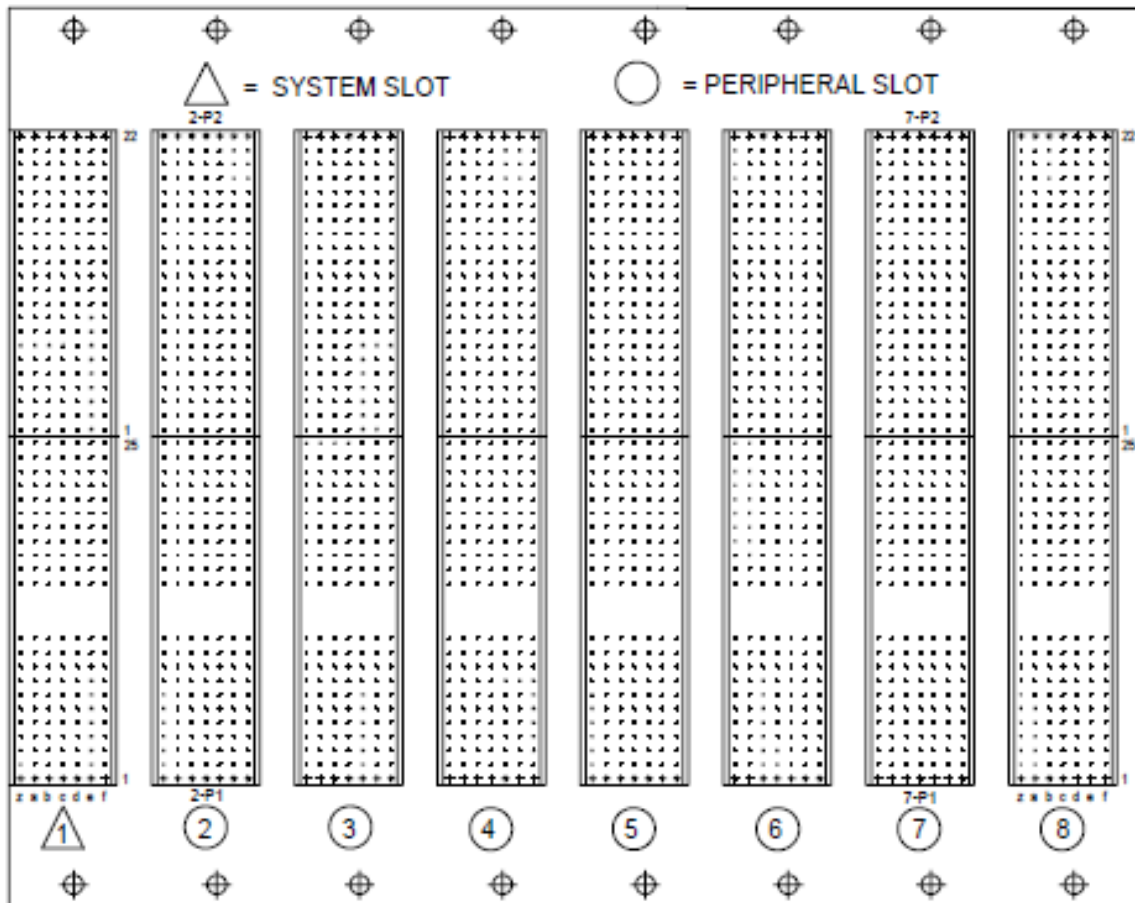


Figure 3.79: cPCI peripheral slots

3.8.4.2 PXI Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all PXI Peripheral slots defined in section 2.1 of the PXI specification [2].

Peripheral slots are identified using a circle as shown in the reproduced figure below.

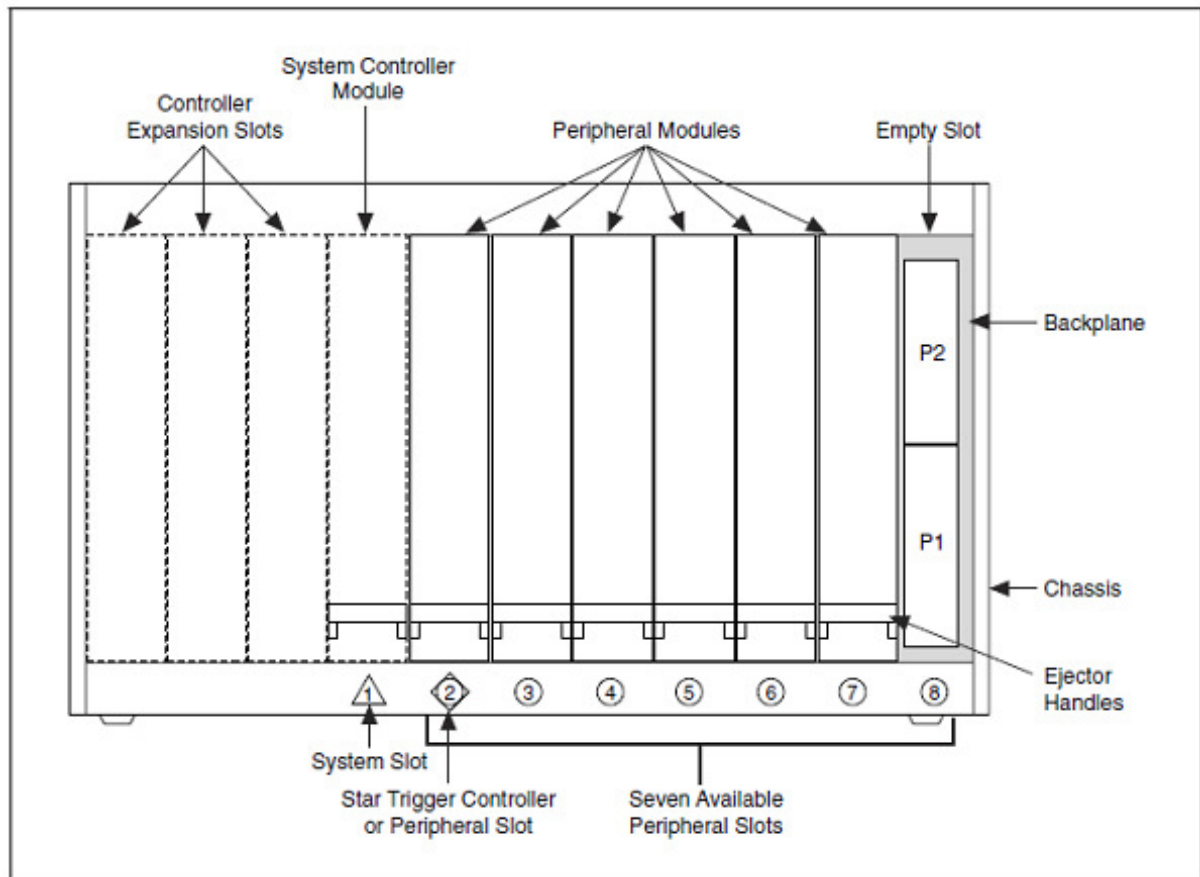


Figure 3.80: PXI peripheral slots

3.8.4.3 PXIe Compatibility

The SpaceWire PXI 12 Port Router card is compatible with all PXIe PXI-1 and PXI Hybrid slots defined in section 2.1.1.5 and 2.1.1.6 of the PXIe specification [3].

PXI-1 slots are identified using a circle as shown in the reproduced figure below.

PXI Express Hybrid slots are identified using a filled circle with a capital H outside the circle, as shown in the reproduced figure below.

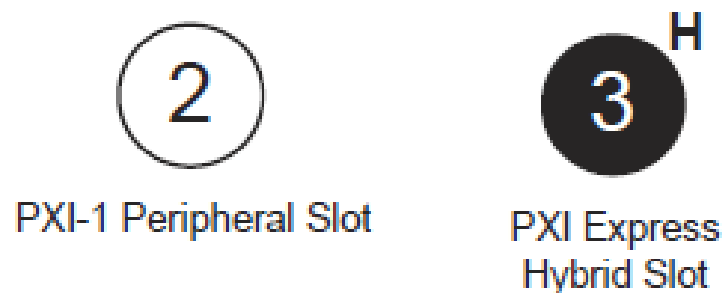


Figure 3.81: Compatible PXIe peripheral slots

3.8.5 Router and SpaceWire Interfaces

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel. The SpaceWire router can be configured using the STAR-System device configuration API functions.

The SpaceWire router has 22 ports in total as listed below:

- 1 internal router configuration port
- 12 external SpaceWire ports
- 9 cPCI channel interface external ports. By default eight of these are used by the SpaceWire ports, while the other is used by the configuration port

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is routing mode. The router will revert to routing mode after power cycling or a device reset is performed.

3.8.5.1 Interface Mode

In interface mode, a packet which is received on a SpaceWire port will always be routed by the port routing configuration. The SpaceWire PXI 12 Port Router card supports interface mode by utilising the port routing settings of the SpaceWire router. The default values are listed in the table below.

Source port	Destination port	Description
0	13	Configuration to external port 1 (host channel 0)
1	14	SpaceWire port 1 to external port 2 (host channel 1)
2	15	SpaceWire port 2 to external port 3 (host channel 2)
3	16	SpaceWire port 3 to external port 4 (host channel 3)
4	17	SpaceWire port 4 to external port 5 (host channel 4)
5	18	SpaceWire port 5 to external port 6 (host channel 5)
6	19	SpaceWire port 6 to external port 7 (host channel 6)
7	20	SpaceWire port 7 to external port 8 (host channel 7)
8	21	SpaceWire port 8 to external port 9 (host channel 8)
9	N/A	Not connected
10	N/A	Not connected
11	N/A	Not connected
12	N/A	Not connected
13	0	cPCI channel 0 to configuration port

14	1	cPCI channel 1 to SpaceWire port 1
15	2	cPCI channel 2 to SpaceWire port 2
16	3	cPCI channel 3 to SpaceWire port 3
17	4	cPCI channel 4 to SpaceWire port 4
18	5	cPCI channel 5 to SpaceWire port 5
19	6	cPCI channel 6 to SpaceWire port 6
20	7	cPCI channel 7 to SpaceWire port 7
21	8	cPCI channel 8 to SpaceWire port 8

3.8.5.2 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 22 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).
- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.8.5.3 Link Operating Speed

The SpaceWire link operating speed for each pair of SpaceWire ports is controlled by the STAR-System device configuration APIs. The link clock rate selection logic for each pair of ports is shown in the figure below.

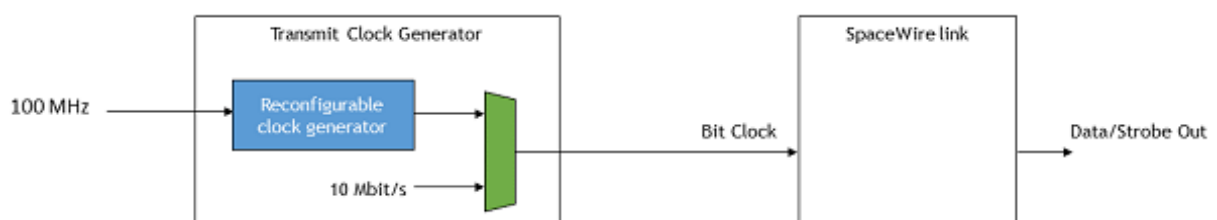


Figure 3.82: SpaceWire link operating speed for each pair of SpaceWire ports

- Each pair of SpaceWire links can be programmed to a link rate between 2 and 400 Mbit/s using the transmit clock configuration functions of the STAR-System device configuration APIs.
- The link operating speed after power on is 200 Mbit/s.

- Each SpaceWire link will operate at 10 Mbit/s at start-up.
- When the reconfigurable clock generator is reprogrammed the link rate will switch to 10 Mbit/s until the programming has completed, when the link is running.

3.8.5.4 Time-codes

A time-code can be received from the cPCI interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external cPCI ports, only port 5, corresponding to channel 0, can be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

3.8.6 SpaceWire Interface Tristate Mode

The SpaceWire ports on the PXI card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Start, Disabled or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.

If a link on the PXI card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

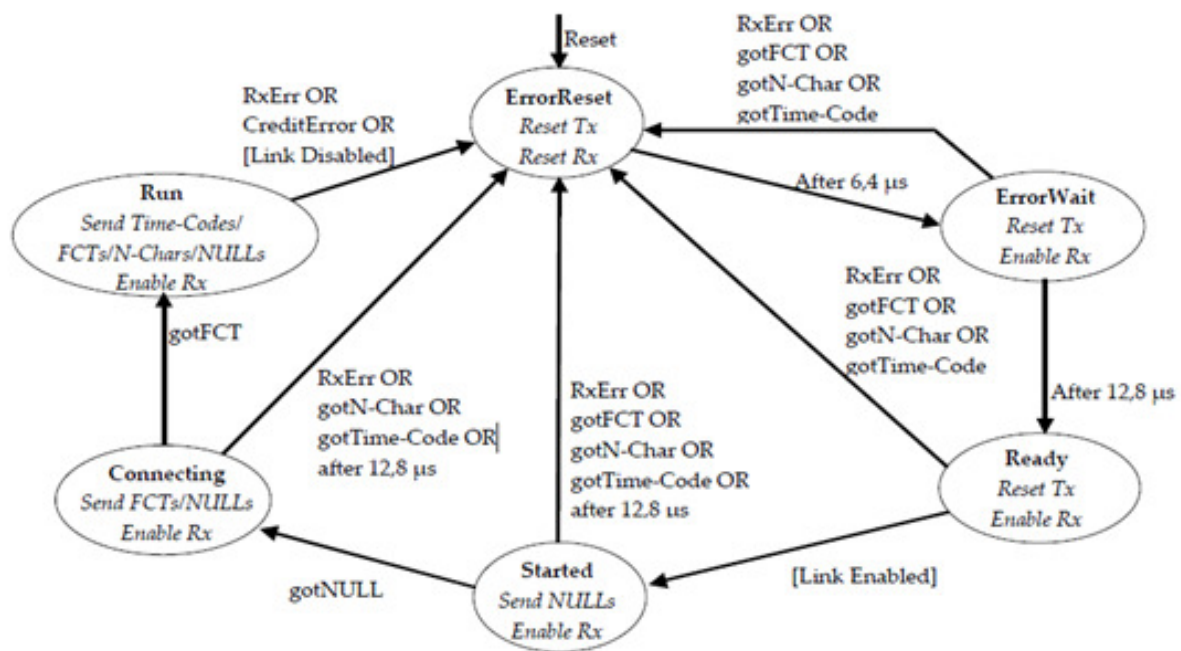


Figure 3.83: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

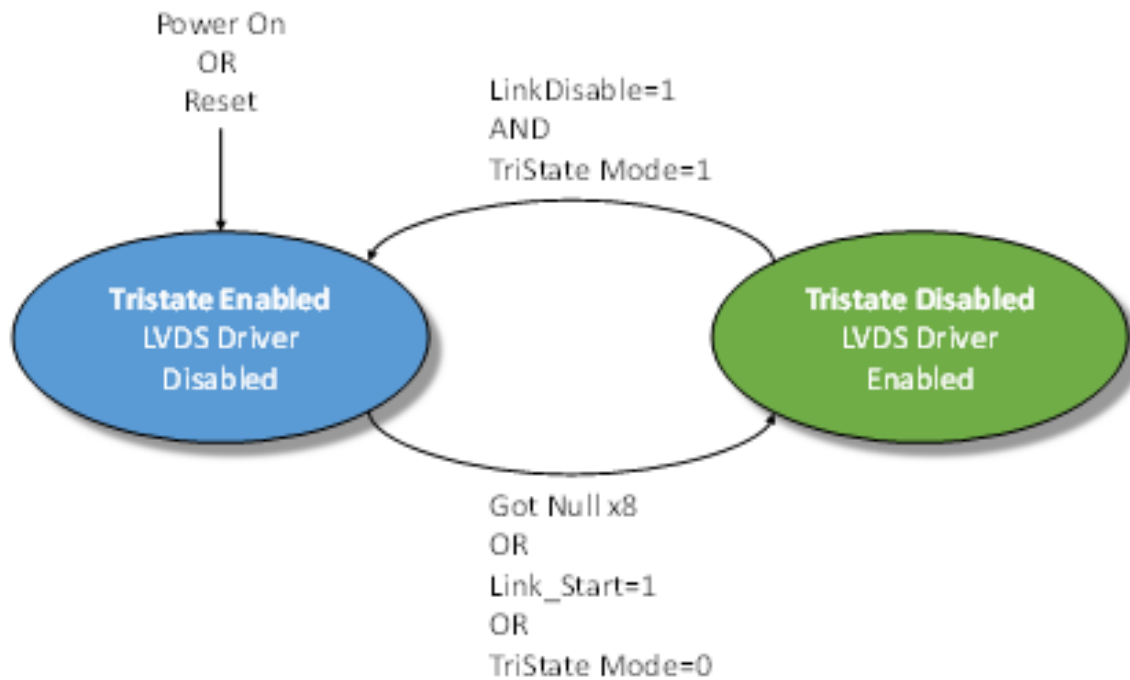


Figure 3.84: Tristate controller state machine

An approximate 60 microsecond delay occurs between the state machine moving from the Enabled to Disabled state, i.e. the LVDS drivers are not enabled until approximately 60 microseconds after the condition to enable the drivers has been detected.

A device which connects to the SpaceWire PXI card and sends NULLs can move through the SpaceWire link state machine a few times until the LVDS drivers are enabled. The exchange of silence period is 19.2 microseconds and a link which is enabled will send NULLs in the Started state for a period of 12.8 microseconds. The link will start normally after the 60 microsecond period – the user does not need to take any action to operate correctly with the PXI card in this mode.

An example start-up sequence when the PXI link is in AutoStart mode is shown in the figure below.

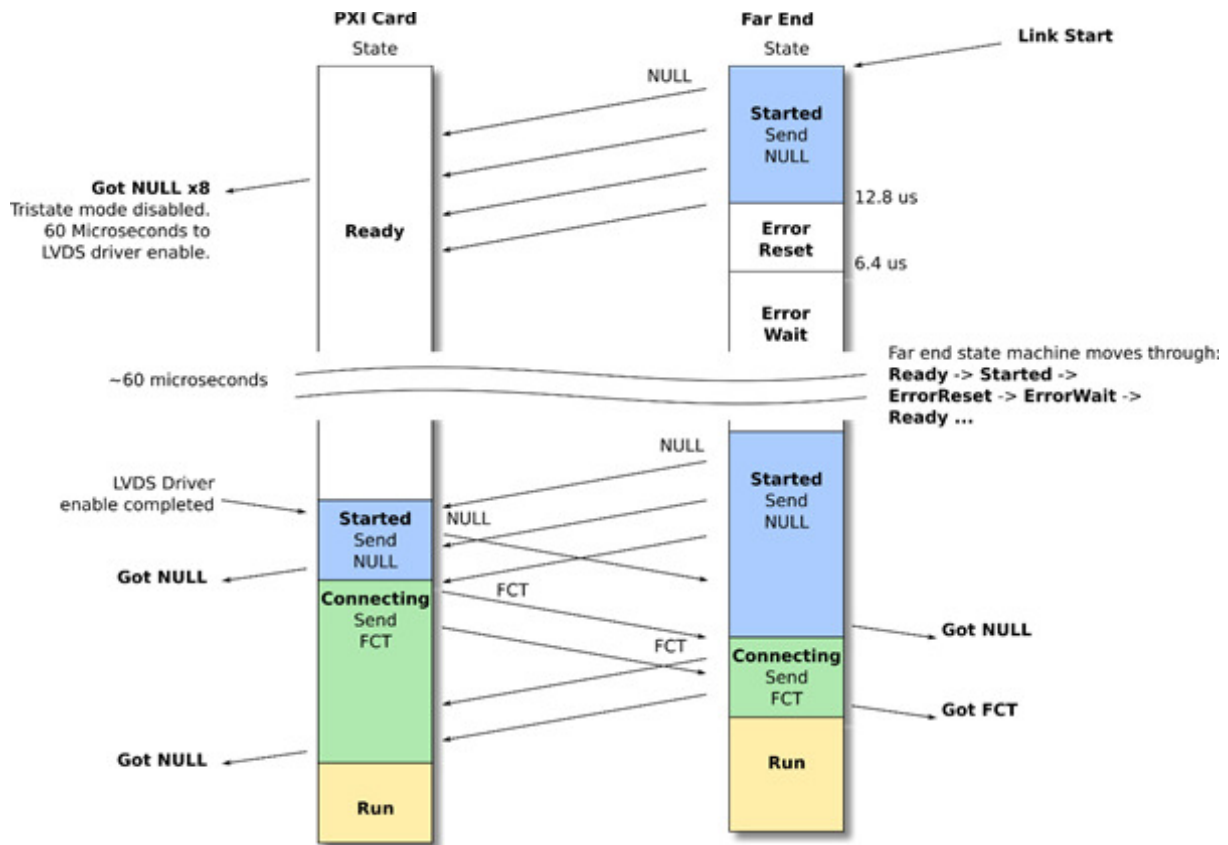


Figure 3.85: AutoStart link operation

In the figure above the Far End link state machine sends NULL characters after its Link Start control bit is set. The PXI card detects the NULL sequence and indicates to the Tristate controller state machine when 8 consecutive NULLs are received without error. The PXI card then enables its LVDS drivers after 60 microseconds and will remain in the Ready state until it detects a start-up NULL from the far end. Both ends then exchange NULL and FCT characters and start-up normally.

An example start-up sequence when the PXI link is set to start is shown below.

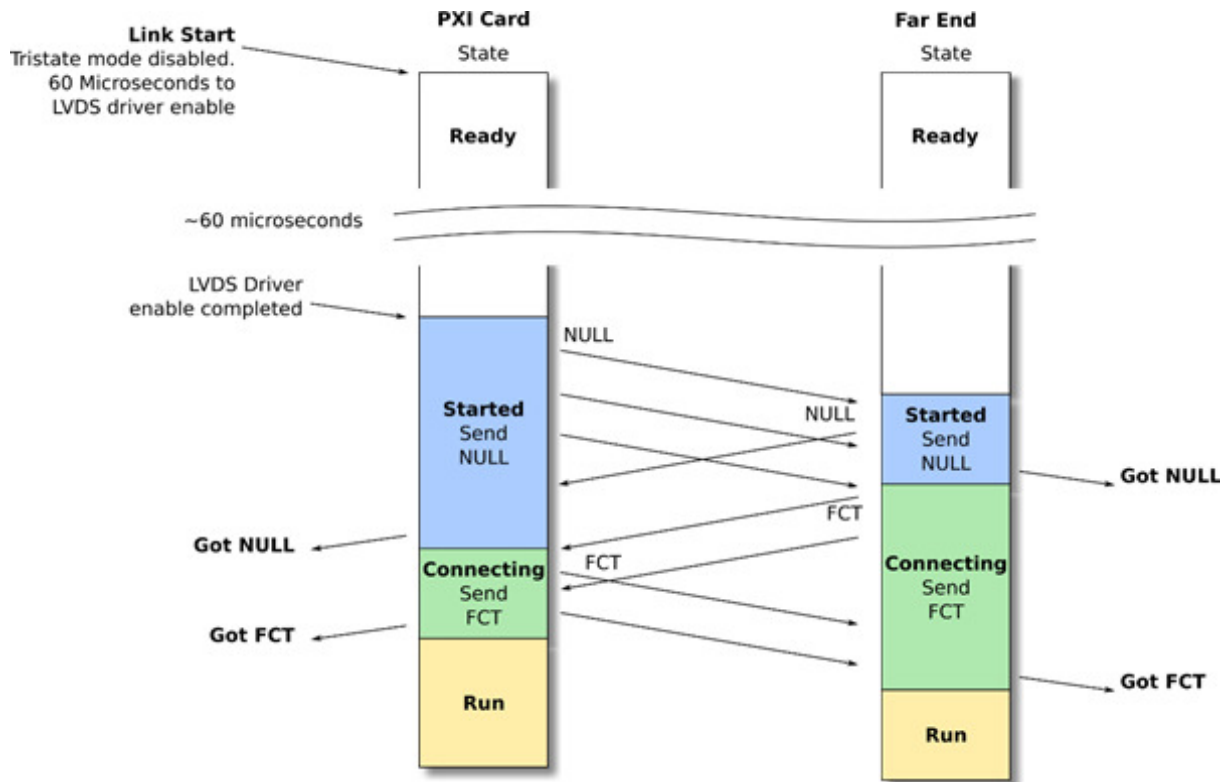


Figure 3.86: Link Start link operation

In the figure above the PXI Card Link Start bit is set and the LVDS driver enable sequence is started. After 60 microseconds the PXI card link will move to the Started state and will send NULL characters. The Far End will detect NULL characters, both ends will exchange NULLs and FCTs and will start-up normally.

Please note, if a SpaceWire Link should start-up on the first exchange of NULL characters then the Tristate setting for that link should be disabled.

3.8.7 Electrical Characteristics

3.8.7.1 Absolute Maximum Ratings

3.8.7.1.1 PXI 12 Port Router Mk1

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.7	4.3	V
SpaceWire input voltage on Vout+, Vout- (see the figure in the following section)	-0.5	4	V

3.8.7.1.2 PXI 12 Port Router Mk2

Parameter	Min	Max	Units
SpaceWire input voltage on Vin+, Vin- (see the figure in the following section)	-0.5	4.3	V
SpaceWire input voltage on Vout+, Vout- (see the figure in the following section)	-0.5	4.3	V

3.8.7.2 SpaceWire Functional Diagram

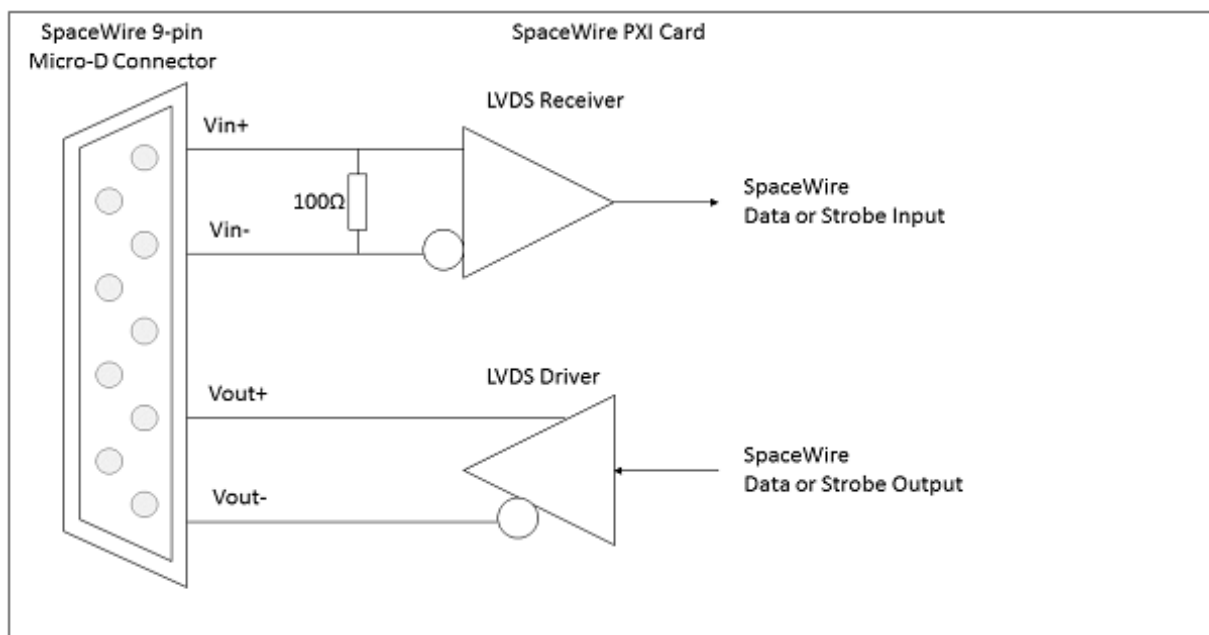


Figure 3.87: SpaceWire functional diagram (one LVDS driver/receiver shown)

Each SpaceWire port is comprised of a pair of LVDS drivers and LVDS receivers, one pair for data and strobe in, and one pair for data and strobe out.

3.8.7.2.1 SpaceWire Input Electrical Characteristics

The SpaceWire recommended input electrical characteristics are shown in the tables below.

3.8.7.2.1.1 PXI 12 Port Router Mk1

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-)	0.1	1	V
Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	4	V

SpaceWire input Differential resistance	98	102	Ω
--	----	-----	----------

3.8.7.2.1.2 PXI 12 Port Router Mk2

Parameter	Min	Max	Units
Differential Voltage range (Vin+ - Vin-)	0.1	1	V
Common Mode +/- Differential voltage range (any combination of common-mode or input signals)	0	3.3	V
SpaceWire input Differential resistance	98	102	Ω

3.8.7.2.2 SpaceWire Output Electrical Characteristics

The SpaceWire output electrical characteristics are shown in the tables below.

3.8.7.2.2.1 PXI 12 Port Router Mk1

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential mag- nitude - 100 Ω load	247	310	454	mV
SpaceWire output common mode voltage - 100 Ω load	1.125		1.375	V
Output voltage to an unpowered unit	Vin+/Vin-	1.12	1.16	V
	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	V
	Vout+/Vout- (Enabled - programmed by software)	0.898	1.602	V

3.8.7.2.2.2 PXI 12 Port Router Mk2

Parameter	Min	Typ (1)	Max	Units
SpaceWire output differential mag- nitude - 100 Ω load	247	330	454	mV
SpaceWire output common mode voltage - 100 Ω load	1.125		1.375	V
Output voltage to an unpowered unit	Vout+/Vout- (Tri-state - Default at power on)	High Z	High Z	V
	Vout+/Vout- (Enabled - programmed by software)	0.898	1.602	V

(1) Typical conditions at 25 °C

Additional information

The STAR-Dundee PXI card can programmatically tristate its LVDS drivers.

3.8.8 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

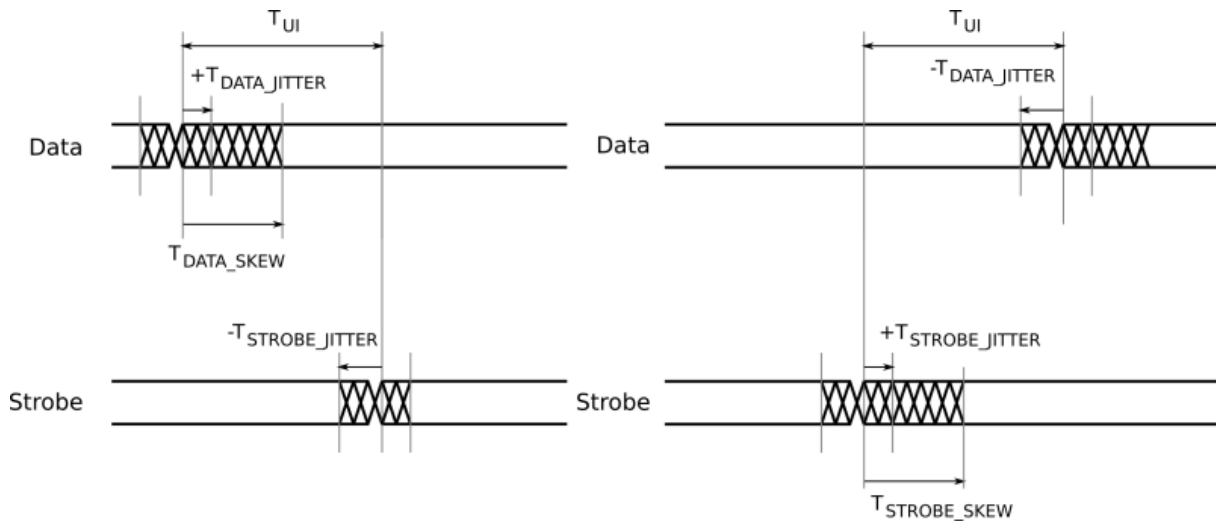


Figure 3.88: SpaceWire input switching characteristics

Parameter	Description	ns (Max)
T_{UI}	Transmitter bit rate.	5
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

Parameter	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0.1
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.9 SpaceWire GbE Brick Mk1

Documentation for the STAR-Dundee SpaceWire GbE Brick Mk1 device is provided in this section.

3.9.1 Overview

3.9.1.1 Summary

This user manual provides an overview of the interfaces and features of the SpaceWire GbE Brick Mk1 hardware.

For detailed information on using the SpaceWire GbE Brick Mk1 devices please consult the STAR-System API reference documentation.

3.9.1.2 Hardware Description

The SpaceWire GbE Brick Mk1 provides an Ethernet to SpaceWire interface device with two SpaceWire interfaces. Efficient host software is provided to support rapid sending and receiving of SpaceWire packets into host P↔C memory.

A block diagram of the SpaceWire GbE Mk1 is shown below.

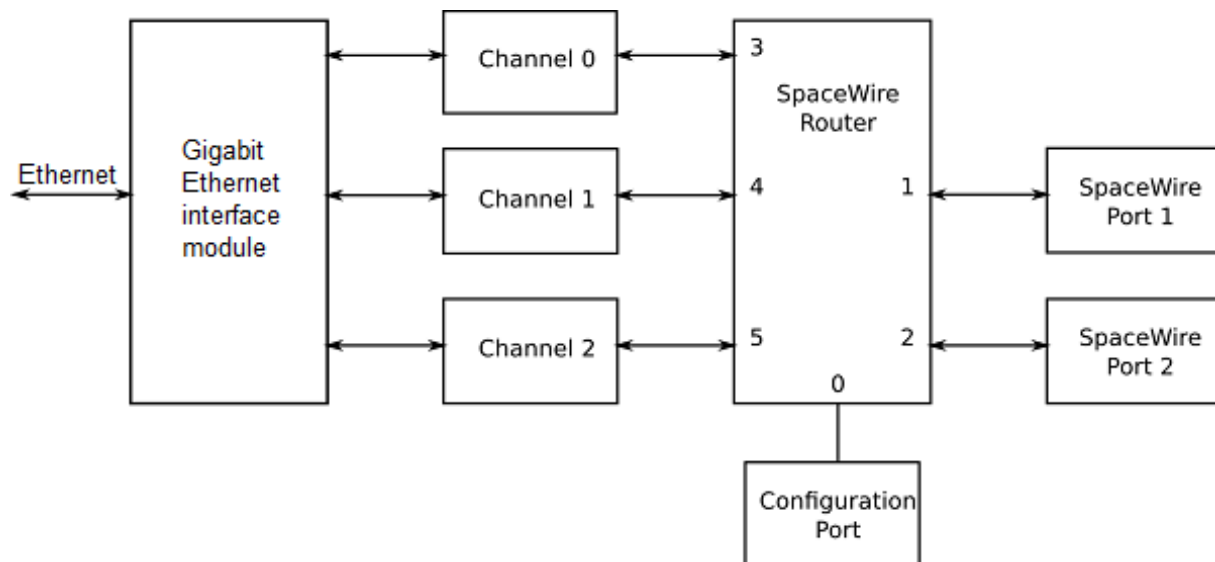


Figure 3.89: SpaceWire GbE Brick Mk1 hardware overview

The two SpaceWire interfaces of the SpaceWire GbE Brick Mk1 are fully compliant with the SpaceWire standard and operate at up to 400 Mbit/s. They are connected to a SpaceWire router which supports two modes of operation, interface mode and router mode. In router mode, packets from any SpaceWire port can be routed to any other SpaceWire port, or the Ethernet interface. In interface mode, the SpaceWire router is passive and packets which are received on a SpaceWire link are passed directly to the Ethernet data channel for that link. There are two independent channels from the SpaceWire router to the Ethernet interface, so traffic flowing over one port cannot block traffic for another port. In addition, there is a separate control channel, so that the host PC is always able to access the control, configuration and status space of the GbE Brick Mk1 regardless of the data flow.

The Ethernet interface is compliant to Gigabit Ethernet, variant 1000BASE-T, defined by the IEEE 802.3ab standard.

The SpaceWire GbE Brick Mk1 includes support for fault injection including support for parity errors, credit errors, escape errors, data corruption and EEP termination of packets.

3.9.2 Getting Started

The SpaceWire GbE Brick Mk1 is powered from the external power supply.

After powering the GbE Brick Mk1, the LEDs will default to their normal operating state e.g. with no cables connected all four LEDs are amber.

The software and drivers, provided by STAR-System, should be installed on the host computer before connecting the GbE Brick Mk1. This ensures that the GbE Brick Mk1 is recognised by the host when it is connected.

During initial configuration and for the best performance, the GbE Brick Mk1 should be connected directly to the PC over an Ethernet cable.

The default IP address of the GbE Brick Mk1 is 192.168.1.100. In order to communicate with the GbE Brick Mk1,

you must be on the same network as the GbE Brick Mk1. Therefore, change your Ethernet adapter settings as follows.

Windows

1. Type "View Network Connections" in the Windows Start menu and hit Enter
2. Double click on the Ethernet adapter the GbE Brick Mk1 is to be connected to
3. Click Properties
4. Select Internet Protocol Version 4 (TCP/IPv4) and click Properties
5. Click Use the following IP address and set the following values:
 - IP address: 192.168.1.199
 - Subnet mask: 255.255.255.0

Linux GNOME

1. Go to Settings -> Network -> Wired -> IPv4
2. Choose IPv4 Method "Manual"
3. Change the Address to "192.168.1.199", Netmask "255.255.255.0" and Gateway "192.168.1.1"

Linux CLI

Ubuntu 17.10 and later uses Netplan as the default network management tool.

1. Find the name given to your Ethernet interface with `ip link`
2. Edit `/etc/netplan/01-network-manager-all.yaml` by adding:

```
1 ethernets:
2   \<name-of-your-interface>:
3     dhcp4: no
4     addresses:
5       - 192.168.1.199/24
6     gateway4: 192.168.1.1
```

3. Once you have saved the file, execute `sudo netplan apply`

On earlier versions of Ubuntu:

1. Create or edit, if it already exists, the configuration file `/etc/network/interfaces`, as follows:

```
1 iface \<name-of-your-interface> inet static
2 address 192.168.1.199
3 netmask 255.255.255.0
4 gateway 192.168.1.1
```

2. For systemd hosts, execute `sudo systemctl restart networking.service`
For pre-systemd hosts, execute `sudo /etc/init.d/networking restart`

On CentOS and Red Hat Enterprise Linux (RHEL):

1. Edit the file `/etc/sysconfig/network-scripts/ifcfg-<name-of-your-interface>` to contain the following:

```
1 BOOTPROTO=static
2 IPADDR=192.168.1.199
3 NETMASK=255.255.255.0
4 GATEWAY=192.168.1.1
```

2. Execute `systemctl restart network.service` (CentOS 7 and RHEL 7) or
`nmcli networking off`
`nmcli networking on` (CentOS 8 and RHEL 8)

Note that the host IP address 192.168.1.199 is an example. Other addresses on the 192.168.1.0/24 subnet can be used.

If you wish to change the IP address of your GbE Brick Mk1, so that the device can be connected to your local network, simply access the GbE Brick Mk1 via any browser by navigating to the address <http://192.168.1.100>, then change the IP address, subnet and gateway, and hit Update.

Next, in order to be recognised by STAR-System, you need to connect to it. To do this:

- Run the [Ethernet Device Connector Application](#)
- Wait for your GbE Brick Mk1 to appear in the list of devices
- Select your GbE Brick Mk1
- Hit Connect

You are now able to use the GbE Brick Mk1 in the same way you would any other STAR-System device.

Note: Every time the Ethernet cable gets unplugged, you must re-connect using the [Ethernet Device Connector Application](#). The application can be closed once connected.

It is also possible to reset the device to its factory settings, including the original IP address of 192.168.1.100. If you wish to perform the factory reset, please contact STAR-Dundee (see [Support and Contact Information](#)) to obtain the correct version of FPGA and CPU files, and follow the steps below:

1. Locate the Reset hole at the [Rear Panel Interfaces](#)
2. Push and hold a paper clip into the hole for at least 10s until the LEDs go from purple to amber. The device has been factory reset.
3. Open the [Ethernet Device Connector Application](#) and connect the device
4. Once connected, open the [Device Update Application](#) and perform two updates:
 - FPGA with a .DAT file
 - CPU with a .CPU file

If you are using your Ethernet adapter to connect to the Internet, please remember to change the IP settings back to "Obtain an IP address automatically" or to the correct IP address for your network, once you have finished working with the GbE Brick Mk1.

3.9.3 Rear Panel Interfaces

The Rear panel features power adapter socket, heatsink and RJ45 connector (8P8C female jack) with LEDs to indicate Ethernet activity. An image of this is shown below.

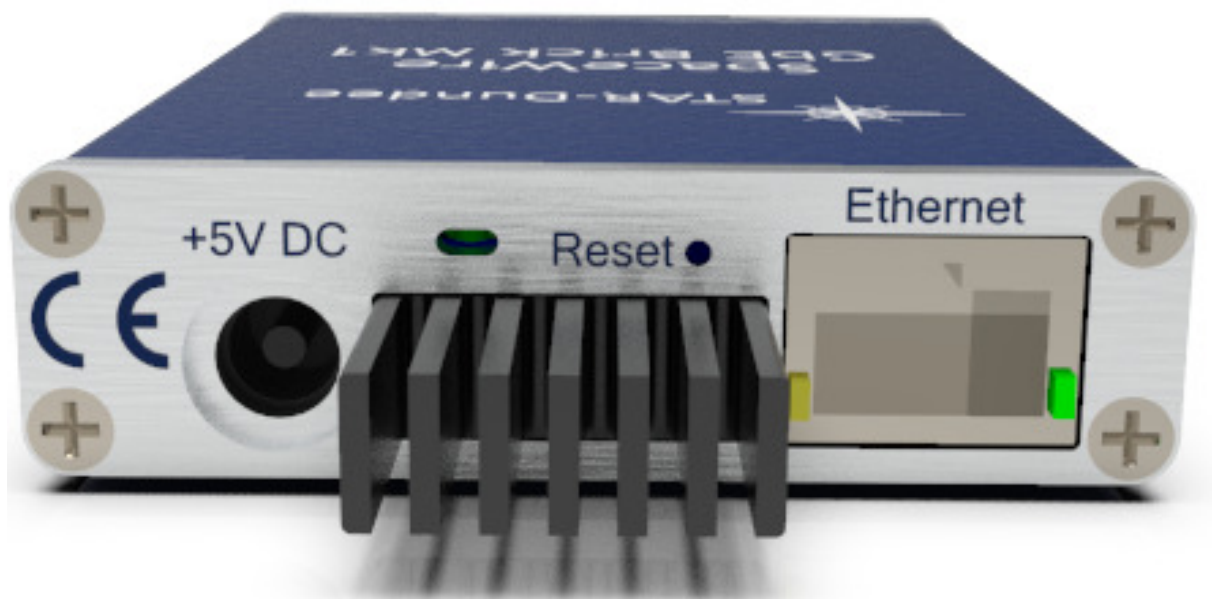


Figure 3.90: Power, heatsink and RJ45 jack with LEDs

3.9.3.1 Ethernet Status LED

The LEDs indicate the presence of an Ethernet link and also activity on the link. The mapping for amber and green LEDs is as shown in the table below.

State of the link	Amber	Green
1000Mbps, no activity	Off	On
1000Mbps, activity	Off	Blink
100Mbps, activity	Blink	Off
10Mbps, no activity	On	On
10Mbps, activity	Blink	Blink
No link	Off	Off

3.9.4 Front Panel Interfaces

The SpaceWire port on the left of the front panel is SpaceWire router link 1 and the port on the right is router link 2. The Ethernet port is router links 3, 4 and 5. The front panel is shown below.



Figure 3.91: SpaceWire ports and LED positions

The GbE Brick Mk1 has six multicolour LEDs, three for each SpW port. The LED on the left next to each link shows received data for that link, the LED on the right next to each link shows transmitted data for that link. The middle LED is currently unused. The functions of the SpaceWire port LEDs are defined in the table below.

Function	LED Colour	Description
No event	Amber	The left and right hand LEDs are steady amber.
Link running	Green	The left and right hand LEDs are turned green when the link is running. The link running LEDs are held for approximately half a second after the link has stopped running and no error occurs.
Error	Red	The left and right hand LEDs are turned red.
Data Transmitted/Received	Blue	When data is received on the link, the left hand LED turns blue. When data is transmitted on the link, the right hand LED turns blue.
No Credit	White	When no credit is available on the link for the device to transmit for a period of time, the right hand LED turns white. When the GbE Brick Mk1 has provided no credit on the link for the device at the other end of the link to transmit for a period of time, the left hand LED turns white.

Identify	Flashing purple	The GbE Brick Mk1 can be forced to flash its front panel LEDs to allow the device to be easily identified among a group of other devices. When this function is used the LEDs will flash purple for a short period of time.
----------	-----------------	---

Please note that there is an issue with all SpaceWire interfaces where a small amount of noise on the inputs can be translated by the LVDS receivers into a digital pattern and then interpreted by the receiver as a bit sequence. Since this bit sequence is invalid it causes the SpaceWire CODEC to start up and then disconnect. The SpaceWire interface LEDs go red on disconnect (ErrorReset state) and amber when ready (Ready state). So, if you get a burst of noise on the input then you will see a red flash of around half a second on the LEDs. To avoid this happening all the time we have a bias resistor network on the input of the LVDS receivers that provides a small bias current. This filters out most of the noise, but occasionally depending on the environment there may still be enough noise to trigger the reset to ready cycle and you see this as a red flash on the LEDs. This is the expected/normal operation. If you have any concerns, please contact support@star-dundee.com.

3.9.5 Functions

3.9.5.1 SpaceWire Router

The SpaceWire router inside this unit is compatible with the AT7910E radiation tolerant router (also known as the SpW-10X) manufactured by Atmel.

The SpaceWire router supports two modes of operation, interface mode and routing mode.

The default mode is interface mode. The router will revert to interface mode after power cycling or a device reset is performed.

The SpaceWire router has 6 ports in total as listed below:

- 1 internal router configuration port
- 2 external SpaceWire ports
- 3 Ethernet interface external ports. By default two of these are used by the two SpaceWire ports, while the other is used by the configuration port

3.9.5.2 Interface Mode

In interface mode, a packet which is received on a SpaceWire link will always be routed to the port specified by the port routing register of the port. The SpaceWire GbE Brick Mk1 supports interface mode by utilising the port routing registers of the SpaceWire router. The default values are listed in the image and table below.

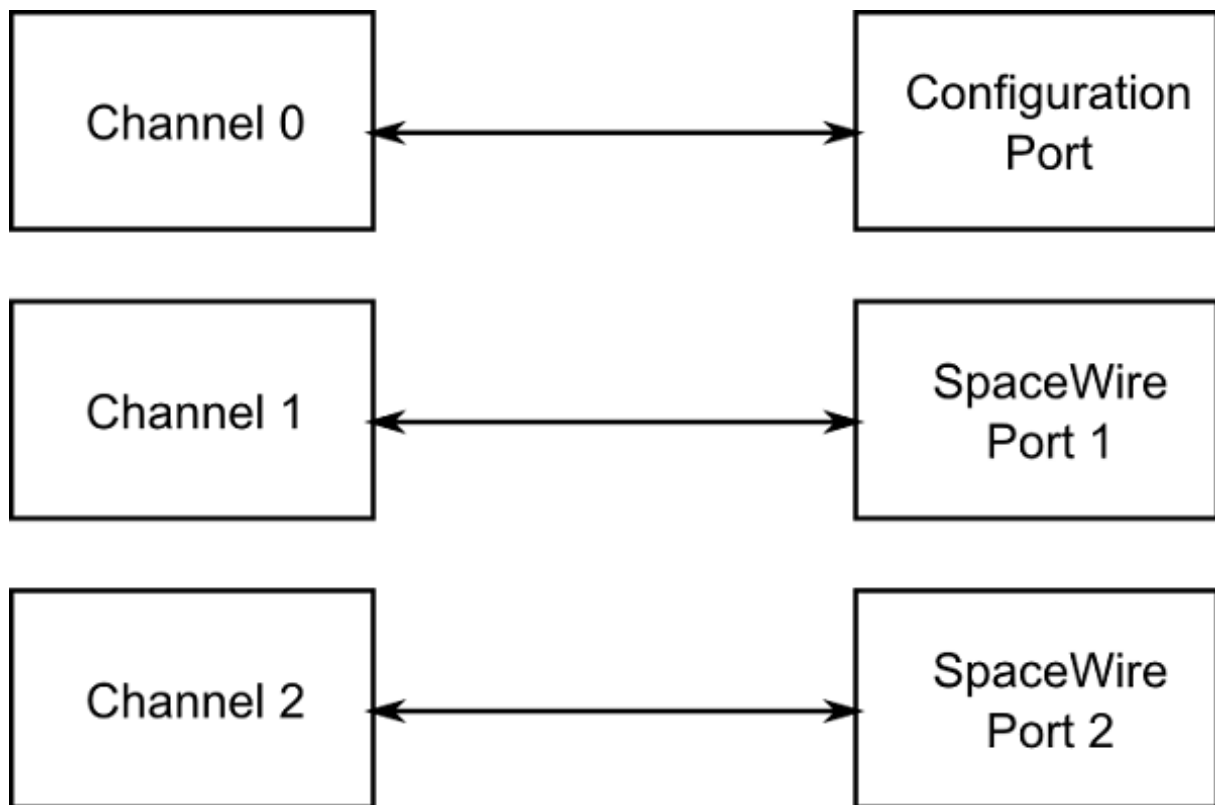


Figure 3.92: Interface mode routing connections

Source port	Destination port	Description
0	3	Configuration to external port 1 (host channel 0)
1	4	SpaceWire link 1 to external port 2 (host channel 1)
2	5	SpaceWire link 2 to external port 3 (host channel 2)
3	0	Ethernet interface channel 0 to configuration port
4	1	Ethernet interface channel 1 to SpaceWire link 1
5	2	Ethernet interface channel 2 to SpaceWire link 2

3.9.5.3 Routing Mode

In routing mode packets are routed dependent on the first byte of each SpaceWire packet. The routing algorithm is described below.

- Packets with known physical addresses are routed to the port indicated by the physical address at the head of the packet, i.e. a packet with address 0 is routed to the configuration port.
- Packets with unknown physical addresses, 6 to 31, are discarded and an address error is set in the source port register.
- Packets with logical address headers which are valid are routed to one of the set ports in the logical address lookup table (accessed via the configuration port).

- Packets with logical address headers which are marked as invalid are discarded and an address error is set in the source port register.

3.9.5.4 Link Operating Speed

The link operating speed for each SpaceWire port is controlled by configuration parameters which can be set by the API or GUI application. The link clock rate selection logic for each port is shown in the figure below.

The default link operating speed is 200 Mbit/s after link start-up and 10 Mbit/s during link initialisation.

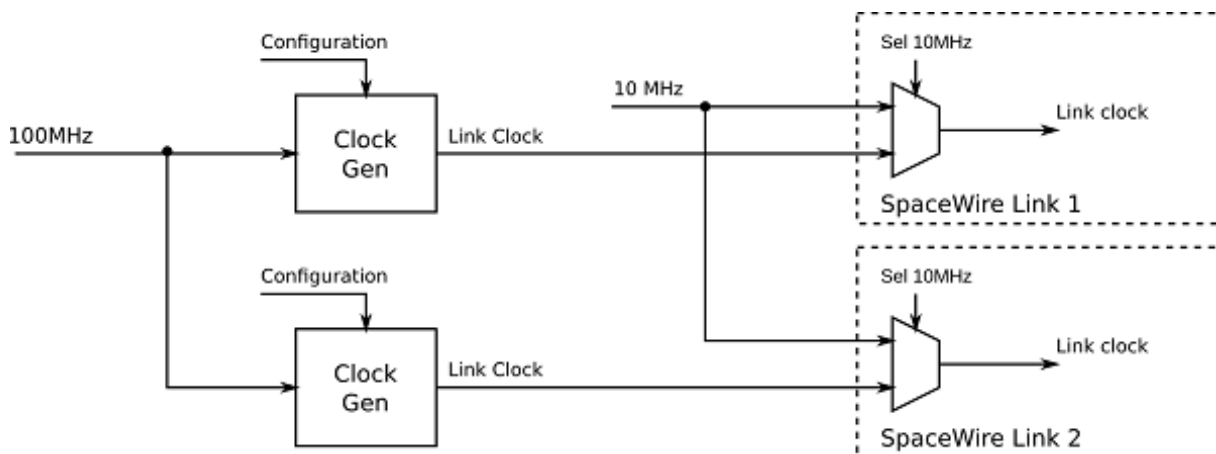


Figure 3.93: Link operating speed circuit diagram

The link operating speed for each SpaceWire link is set using a programmable clock generator frequency. This is set by configuring a multiply and divide value to the input 100 MHz clock.

The link rate can be programmed using the functions provided by the GbE Device ([CFG_GBE_BRICK_MK1↔_setTransmitSignallingRate\(\)](#)) and Generic Configuration API ([CFG_setTransmitClock\(\)](#)). Please refer to the API documentation for further details on how to set this clock speed.

3.9.5.5 Time-codes

A time-code can be received from the Ethernet interface or from a SpaceWire link. When a time-code is received, its value is checked against the value of the internal time-code register in the SpaceWire router. If the received time-code is one more than the value of the time-code register then the time-code will be written to the time-code register and propagated to other ports, except the port that was the source of the time-code.

Time-codes are only propagated on ports which are enabled to send time-codes as defined by the time-code control register. Note that of the external Ethernet ports, only port 3, corresponding to channel 0, can currently be used to send or receive time-codes.

The router has an internal time-code generation master, which can be controlled by the SpaceWire router configuration port to generate time-codes at a user defined rate.

An overview of the SpaceWire router time-code interface is shown below.

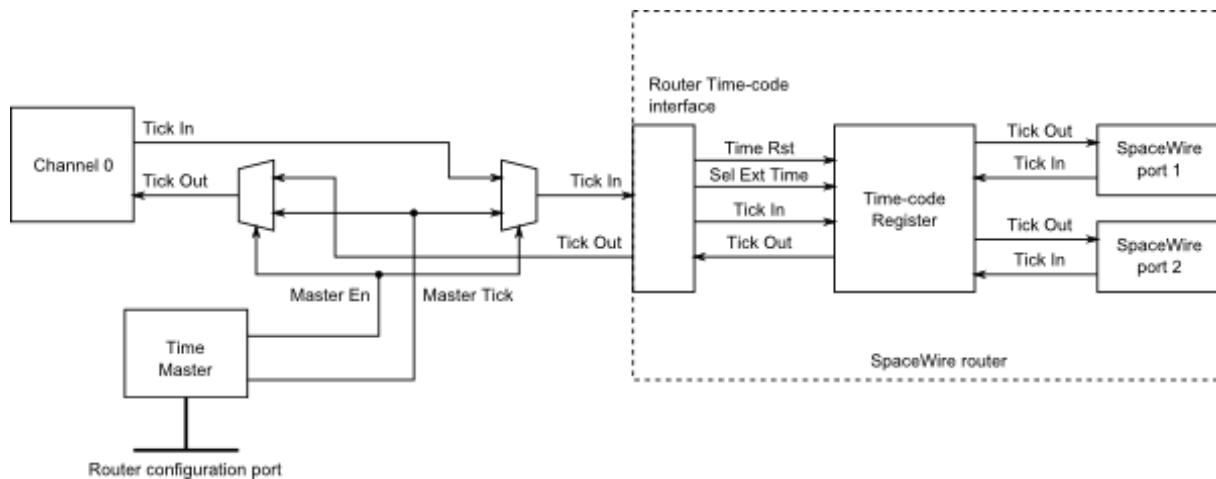


Figure 3.94: Time-code processing architecture

3.9.6 SpaceWire Interface Tristate Mode

The SpaceWire ports on the GbE Brick Mk1 card are comprised of four LVDS signal pairs, Data/Strobe in +/- and Data/Strobe out +/- . The Data/Strobe output pairs are connected to LVDS drivers which default to a high impedance Tristate mode after power on and after a device reset. The signal pairs are effectively disconnected when the tristate mode is enabled, providing protection for connected devices.

Each port includes a link monitoring component which can enable or disable the LVDS driver of a link dependent on a combination of the SpaceWire link settings, Start, Disabled or Tristate, and on the input bit pattern. The input bit pattern is monitored for a sequence of NULL characters without error to detect if a link should move out of Tristate mode and actively drive the Data/Strobe output signals.

The Tristate mode of each port is acted on by the following inputs.

- The default operating mode of all STAR-Dundee SpaceWire links is AutoStart. In the AutoStart mode the SpaceWire links remain in the Ready state and will wait for NULL characters before starting up.

If a link on the GbE Brick Mk1 card is in Tristate mode (LVDS drivers disabled) then the LinkEnabled check in the Ready state, **LinkEnabled=(Link Disabled=0 AND (LinkStart=1 OR (AutoStart=1 AND gotNULL=1))**, remains LOW until 8 valid NULL characters have been successfully received without error, i.e. the state machine cannot move to Started until the LinkDisabled condition in the function is cleared.

Further information on the Ready state transition is shown in the figure below.

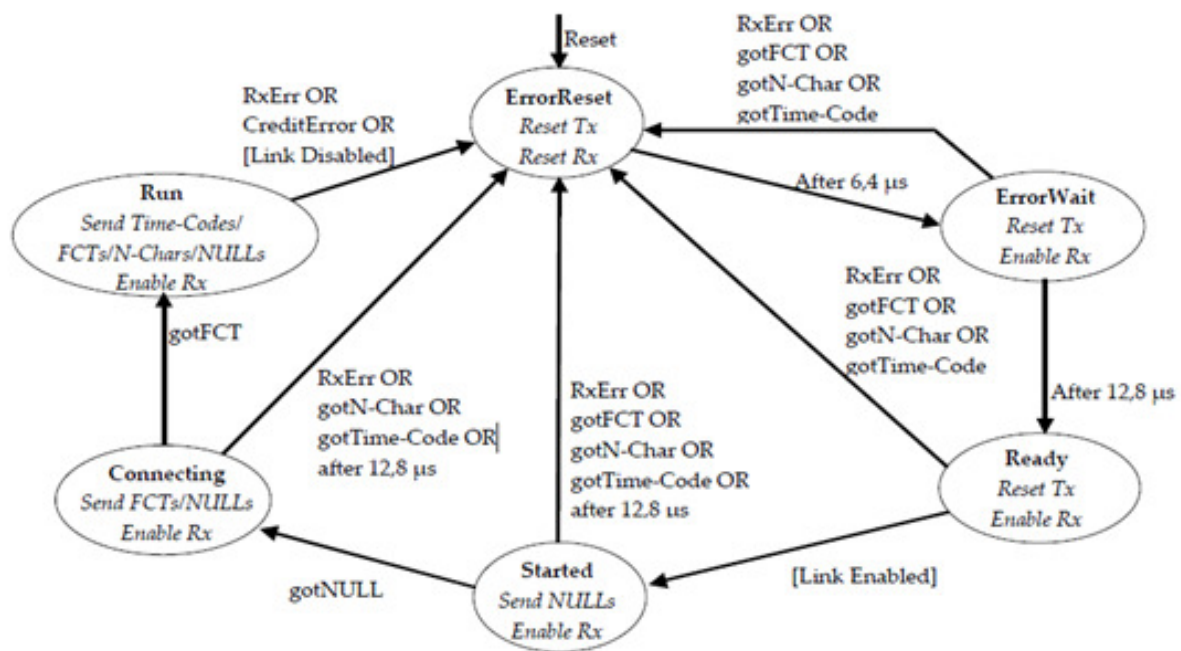


Figure 3.95: SpaceWire Interface state machine

- If the host software or network manager sets a link into LinkStart mode to force the link state machine to move from Ready to Started, or the Tristate mode enable bit is cleared, then the link will automatically move out of its Tristate mode and the SpaceWire port LVDS drivers will be enabled.
- If the host software or network manager sets the LinkDisabled mode to force the link into a disabled state, and the Tristate control bit is set, then the LVDS drivers of the corresponding SpaceWire port are disabled and are set to a high impedance state.

An overview of the Tristate mode settings and their effect on the link are shown in the state diagram below.

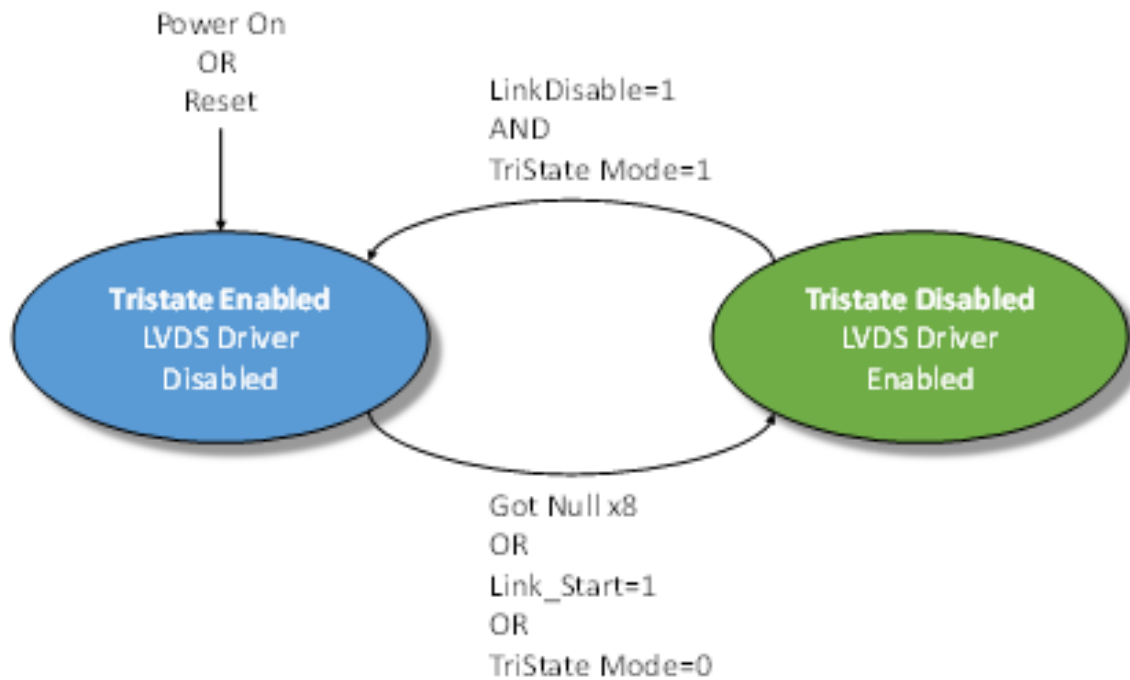


Figure 3.96: Tristate controller state machine

3.9.7 Electrical Characteristics

3.9.7.1 SpaceWire Connectors

3.9.7.1.1 Absolute Maximum Values

The SpaceWire interface **absolute maximum** input ratings are defined in the tables below. The device can withstand these maximum values but use at or near these values is not recommended.

		Description	V _{MIN}	V _{MAX}
V _{IN}	Data/Strobe Out	Input voltage relative to GND	-0.5	4
	Data/Strobe In	Input voltage relative to GND	-0.5	4

Additional clamping diodes on all SpaceWire signals will engage for voltages below -0.3V and above 3V.

3.9.7.1.2 LVDS DC Characteristics

The SpaceWire connector LVDS DC characteristics are listed in the tables below.

	V _{ID}		V _I		V _{OD}			V _{OCM}		
	mV, Min	mV, Max	V, Min	V, Max	mV, Min	mV, Typ	mV, Max	V, Min	V, Typ	V, Max
Din, Sin	100	1000	0	3.3						
Dout, Sout					247	350	454	1.125	N/A	1.375

V_{ID}	Input differential voltage
V_I	Input voltage ¹
V_{OD}	Output differential voltage
V_{OCM}	Output common mode voltage

¹Input voltage is a combination of common mode and differential input voltages

The common mode and differential identifiers are described in the diagram below.

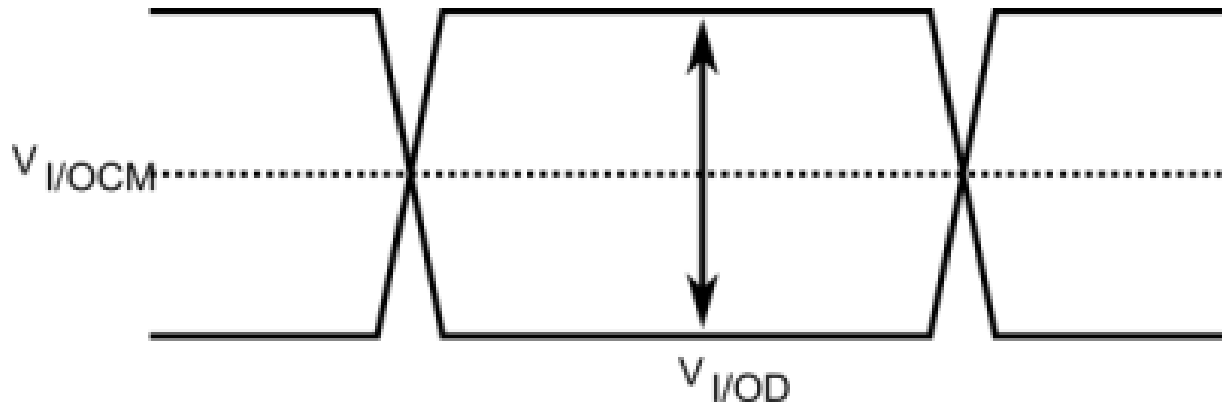


Figure 3.97: SpaceWire LVDS electrical specifications

3.9.7.2 External Trigger Connectors

3.9.7.2.1 External Trigger Functional Diagram

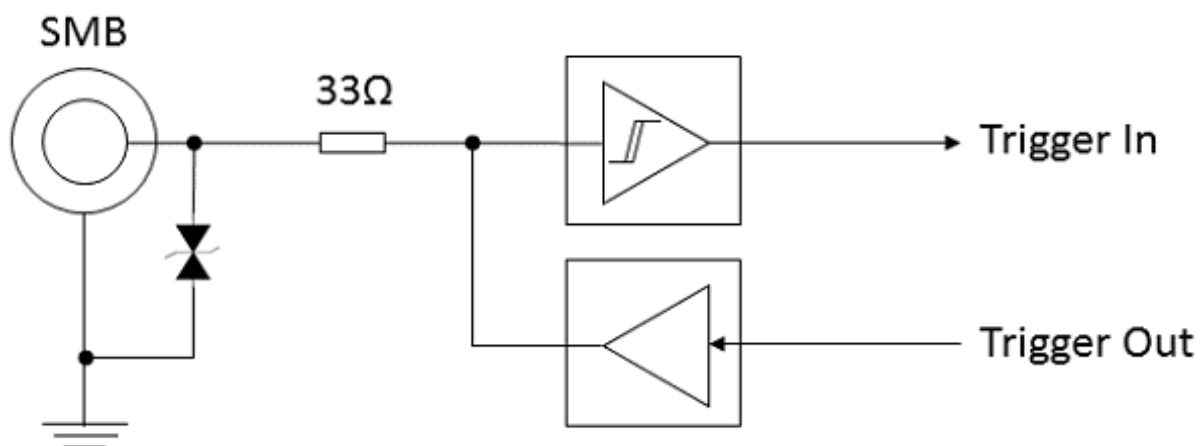


Figure 3.98: External trigger functional diagram

3.9.7.2.2 Absolute Maximum Values

The external trigger **absolute maximum** input ratings are defined in the table below. The device can withstand these maximum values but use at or near these values is not recommended. Note that each trigger is only 5 Volt input tolerant when configured as an input (the default on power up), or when the device is powered down. Triggers

configured as outputs are not 5 Volt tolerant, and may be damaged if an external voltage is applied to them. It is recommended to use 3.3 Volt logic when using the triggers.

	Description	Condition	Min	Max	Unit
V_{IN}	Voltage applied to trigger	Trigger configured as input, or device powered down	-0.5	6.5	Volts
		Trigger configured as output	-0.5	3.8	Volts
I_{OUT}	Continuous output current	Trigger configured as output	-50	+50	mA

3.9.7.2.3 DC Characteristics

The DC characteristics of the external trigger is listed in the table below. The output is driven by 3.3 Volt LVCMOS logic.

	V_{IL} V, Min	V, Max	V_{IH} V, Min	V, Max	V_{OL} V, Max	V_{OH} V, Min
Trigger IN	0	0.8	2.0	5.5		
Trigger OUT (100 μ A load)					0.1	3.2
Trigger OUT (24 mA load)					0.55	2.3

V_{IL}	Input low voltage
V_{IH}	Input high voltage
V_{OL}	Output low voltage
V_{OH}	Output high voltage

3.9.7.3 FMECA Protection

A FMECA analysis report for the GbE Brick Mk1 is available on request.

3.9.8 Switching Characteristics

The SpaceWire interface data/strobe input switching characteristics are defined in the figure and table below.

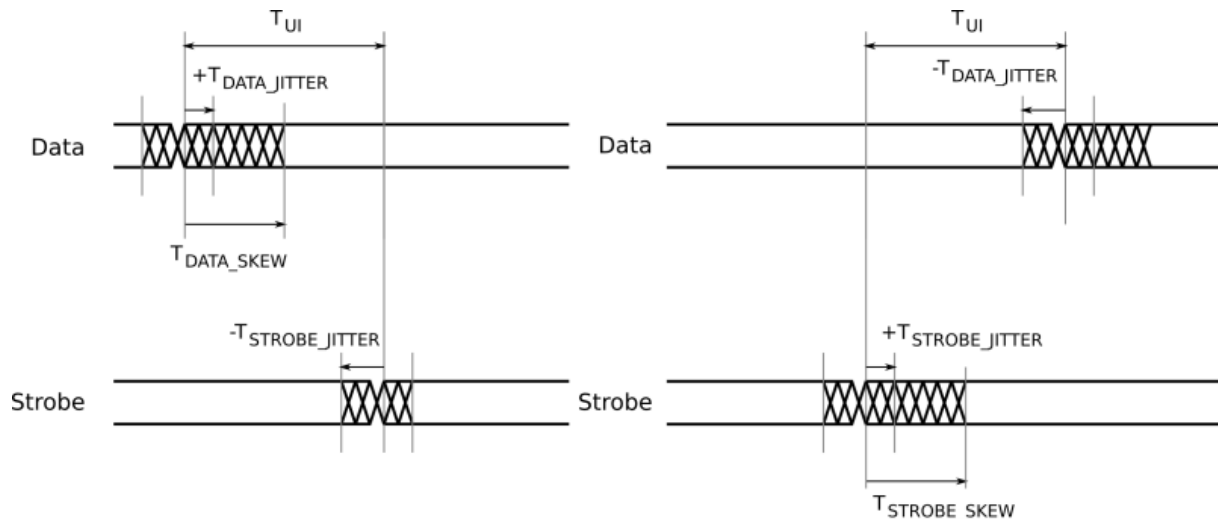


Figure 3.99: SpaceWire input switching characteristics

	Description	ns (Max)
T_{UI}	Transmitter bit rate.	3.333
$T_{DATA_SKEW_IN} (MAX)$	Maximum data skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$
$T_{STROBE_SKEW_IN} (MAX)$	Maximum strobe skew which can be tolerated by the receiver.	$2 - T_{DATA_JITTER_IN} - T_{STROBE_JITTER_IN}$

The data/strobe output switching characteristics are defined in the table below.

	Description	ns (Max)
T_{SKEW_OUT}	Maximum data or strobe skew.	0
T_{JITTER_OUT}	Maximum data/strobe jitter.	0.7 (Peak to Peak)

3.9.9 Grounding Information

When using LVDS it is very important that the grounds of the equipment at the two ends of each SpaceWire link are connected. Otherwise a significant common mode voltage can exist between the two units resulting in the maximum input voltage at one end or the other being exceeded. This could then result in damage to either unit.

The SpaceWire outer shield is connected to the backshell of the connectors at either end of a SpaceWire cable and when screwed into the mating SpaceWire connector on a unit ought to make a connection to the chassis and ground plane of that unit. Properly connecting a SpaceWire cable would then suffice to make the required ground connection between the two units being connected by SpaceWire.

STAR-Dundee equipment always has a low impedance connection between the backshell of the connector, the chassis and ground plane of the electronics in the unit. STAR-Dundee cables have the outer shield connected to the backshell of the connectors at either end of the cable.

When testing a new SpaceWire unit, it is recommended that you check that there is a low impedance connection between the backshell of the connector on your unit under test (UUT) and the chassis and ground plane in the UUT. If this is not low impedance then that problem should be resolved before connecting any SpaceWire equipment to the UUT. If there is a low impedance connection then please check the cable you are using and that there is a low impedance connection between the backshells of the connector at either end of the cable.

3.10 Other STAR-System Devices

In addition to the STAR-System devices with their own individual user manuals, there are several other STAR-↔Dundee devices which rely on STAR-System. These are described below.

3.10.1 Devices with Interface Capabilities

The following STAR-System devices offer similar functionality to the interface and router devices which have individual user manuals included in this documentation. They can therefore be used with the STAR-System applications and APIs. However, they also offer lots of other functionality so have separate user manuals which also describe their individual capabilities.

3.10.1.1 SpaceWire Physical Layer Tester

The **SpaceWire Physical Layer Tester**, or SPLT, provides the ability to manipulate SpaceWire signals to test a connected unit. While those signals are being manipulated, STAR-System can be used to transmit and receive packets on the SPLT's four ports. The SPLT's user manual explains how each of these ports can be accessed.

3.10.1.2 STAR Fire

The STAR Fire can act as a SpaceWire to SpaceFibre bridge, a SpaceFibre link analyser, or as an interface for transmitting and receiving on the SpaceWire and SpaceFibre ports. The initial version of the STAR Fire did not support STAR-System and relied on STAR-Dundee's older USB Driver. The device and software were soon updated, though, to support STAR-System for transmitting and receiving packets. Internally this updated version of the product is known as the STAR Fire Mk2. Please see the user manual for the STAR Fire to learn how packets can be routed to/from the SpaceWire and SpaceFibre ports.

3.10.1.3 STAR Fire Mk3

Like the original **STAR Fire**, the **STAR Fire Mk3** can act as a SpaceWire to SpaceFibre bridge, a SpaceFibre link analyser, or as an interface for transmitting and receiving on the SpaceWire and SpaceFibre links. The **STAR Fire Mk3** provides a higher speed USB 3.0 interface and much improved software. The STAR Fire Mk3 user manual provides information on routing packets to the SpaceWire and SpaceFibre ports, including using the USB 3.0 interface to transmit and receive packets at very high speed over the SpaceFibre ports.

3.10.1.4 STAR-Ultra PCIe Interface

The **STAR-Ultra PCIe Interface** is a SpaceFibre interface and SpaceFibre link analyser, capable of transmitting and receiving packets at very high speeds thanks to an eight-lane Gen 3 PCIe interface. The STAR-Ultra PCIe Interface user manual provides information on the mapping between STAR-System channels and the device's virtual channels.

3.10.1.5 STAR-Ultra PCIe Single-Lane Router

The **STAR-Ultra PCIe Single-Lane Router** is a SpaceFibre router, capable of routing, transmitting and receiving packets at very high speeds thanks to an eight-lane Gen 3 PCIe interface. The STAR-Ultra PCIe Single-Lane Router quick start guide provides information on the mapping between STAR-System channels and the device's virtual channels.

3.10.2 Devices without Interface Capabilities

The following STAR-System devices use the STAR-System drivers, but do not offer the ability to transmit and receive packets using the STAR-System APIs. Basic properties such as their name and serial number can be obtained, however, and device updates can be performed using the [Device Update Application](#), for example. These devices instead offer lots of other capabilities which are documented in their individual user manuals.

3.10.2.1 SpaceWire Conformance Tester Mk2

The [SpaceWire Conformance Tester Mk2](#) allows a device's conformance to the SpaceWire standard to be tested. It does this by transmitting specific patterns of bits on the link and checking the bits that are received. Due to this unique nature, it doesn't offer an interface for transmitting and receiving packets through STAR-System.

3.10.2.2 SpaceWire EGSE and SpaceWire EGSE Mk2

The SpaceWire EGSE and Device Simulator and its successor the [SpaceWire EGSE Mk2](#), can be used to perform real-time simulations of an instrument. This is achieved by writing scripts describing what the instrument should do, which are then downloaded to the device and executed on that device in real-time. Although the EGSE products do not offer standard interface capabilities, they do include an API which is built on top of STAR-System. Further information is available in the EGSE and EGSE Mk2 user manuals.

3.10.2.3 SpaceWire Link Analyser Mk3

The [SpaceWire Link Analyser Mk3](#) can be used to monitor and record the information crossing a Space↔Wire link. The recorded data can be viewed at various levels, from the individual bits through to a higher layer protocol level. Although the Link Analyser Mk3 doesn't include the ability to transmit and receive packets, it does include an API for configuring the capture of traffic, etc. This is built on top of STAR-System and further information is available in the Link Analyser Mk3's user manual.

3.10.2.4 SpaceWire Recorder and SpaceWire Recorder Mk2

The SpaceWire Recorder and its successor the [SpaceWire Recorder Mk2](#), can be used to monitor and record the traffic crossing up to four SpaceWire links. The Recorders include a large SSD for storing this data, meaning that the traffic can be recorded for much longer than is possible using the Link Analyser Mk3, although the Recorders are unable to record at the bit level.

3.11 Device Update Application

Documentation for the STAR-System Device Update Application is provided in this section.

3.11.1 Using the Device Update Application

3.11.1.1 Before Running the Device Update Application

Before running the Device Update Application, make sure that you have the correct firmware update file for your device.

Note also that updating devices can take a considerable length of time, sometimes several hours, and it is important that the computer used for updating, and the device being updated, are not powered down or disconnected during the update process. Doing so may leave your device in an inoperative state.

3.11.1.2 Performing a Device Update

The image below shows the Device Update Application in use.

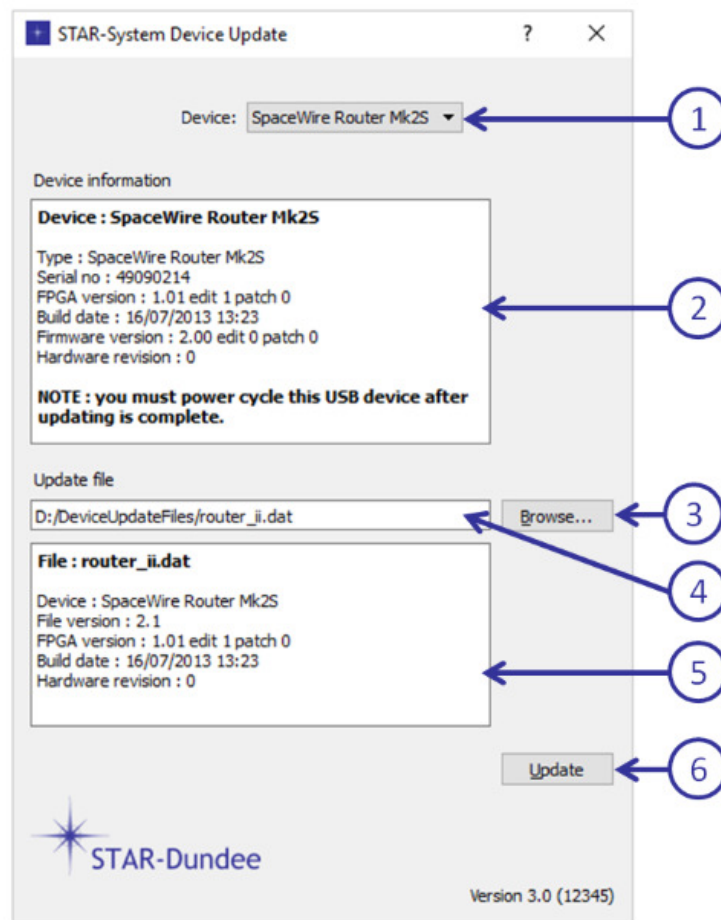


Figure 3.100: Device Update Application Main Window

The controls for the application are as follows:

1. Device selector dropdown - used to select the device to be updated
2. Device information panel - provides information about the selected device
3. Update file browse button - displays a file selection dialog for the update file
4. Update file path and name - displays the update file name, and can be used to enter the file name
5. Update file information - displays information about the update file
6. Update button - starts the update process

Note that the device selected in this example is a USB device, and the device information panel includes the information that it should be power cycled on completion of the update. For a PCI, PCIe, cPCI or PXI device, this will indicate that the computer should be power cycled to complete the update.

Once a device and an appropriate update file have been selected, the Update button (6) can be used to start the update process. A progress dialog will be displayed showing the status of the update:

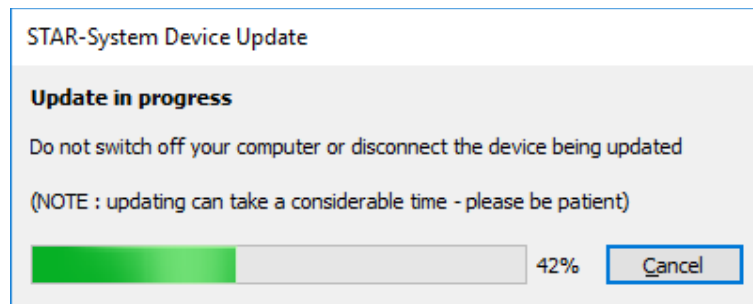


Figure 3.101: Device Update Progress Dialog

Again, please note that the update can take a considerable time.

Once the update is complete, a message box will be displayed:

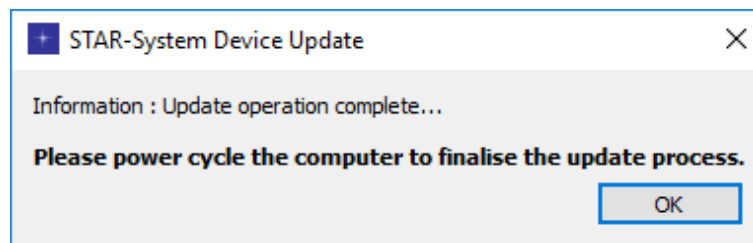


Figure 3.102: Device Update Complete

In this example the update was performed on a PCI, PCIe, cPCI or PXI device, and the computer must be power cycled to complete the update. For a USB device, the message will indicate that the device should be power cycled.

3.11.1.3 Cancelling a Device Update

The update can be cancelled at any time by pressing the Cancel button in the progress dialog, but please note that this may leave your device in an inoperative state. An additional dialog will be displayed to protect against accidental cancellation:

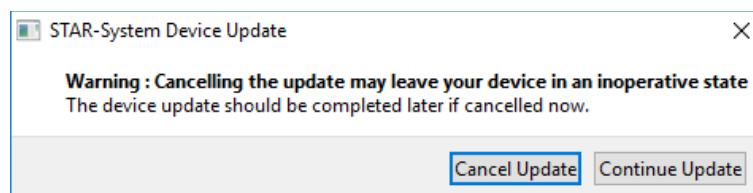


Figure 3.103: Device Update Cancel Dialog

If the update is cancelled, an additional notification will be displayed:

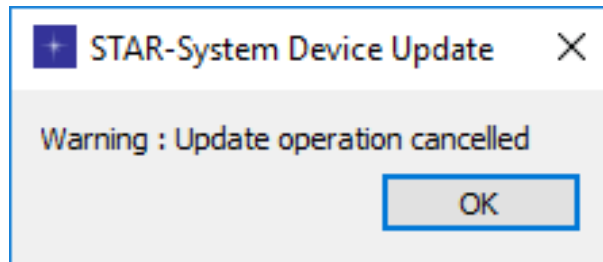


Figure 3.104: Device Update Cancelled Message

3.11.1.4 Errors During a Device Update

If an error occurs during the update process, a message box will be displayed giving details of the error, such as:

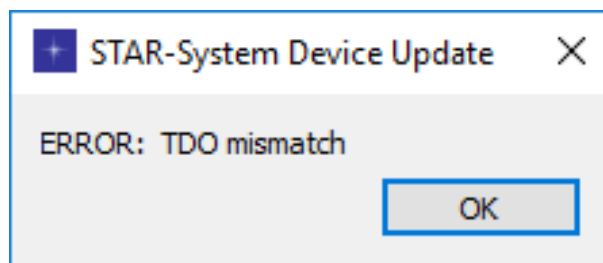


Figure 3.105: Device Update Error Message

If this occurs, please contact support@star-dundee.com with details of the error message, the device you were updating and the update file being used.

3.11.1.5 Other Messages and Notifications

There are a number of other messages and notifications which may appear when starting a device update operation, and each is described below.

If the update file is not for the type of device selected, an error message will be displayed. Either the correct device type or correct update file must be selected to allow the update to proceed.

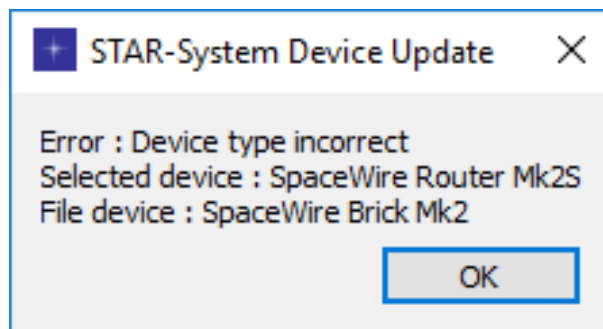


Figure 3.106: Device Type Incorrect Message

If the version of the update file is earlier than the version currently programmed into the device, a warning will be displayed. The update may be performed, but this should only be done if you are sure you wish to revert your device to an earlier version.

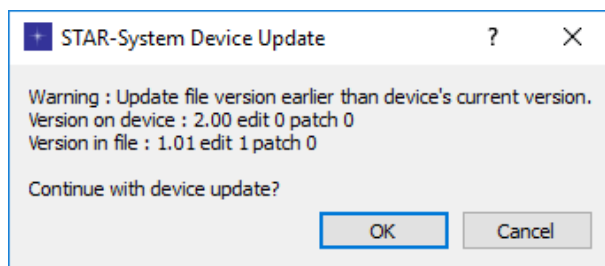


Figure 3.107: File Version Earlier Dialog

If the version of the update file is the same as the version currently programmed into the device, a warning will be displayed. The update may be performed, but this should only be done if you are sure this is what is required.

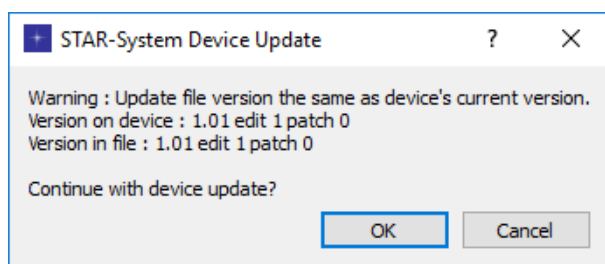


Figure 3.108: File Version The Same Dialog

Chapter 4

Other Useful Information

The following sections describe other information which may be useful when installing and using STAR-System:

- Installation instructions for specific operating systems:
 - [VxWorks Installation and Getting Started Guide](#)
 - [RTEMS Installation and Getting Started Guide](#)
- [Multi-Threading and Multi-Process Support](#)
- [Device Configuration Service](#)
- [Error Logs](#)
- Information on the C API:
 - [Configuration API Functions Supported by Device Types](#)
 - [Static Analysis](#)
- [Open Source Components](#)
- [Change Log](#)

4.1 VxWorks Installation and Getting Started Guide

4.1.1 Introduction

The VxWorks version of STAR-System provides a custom driver in the form of compiled libraries for the appropriate toolchain, low level device initialisation code provided as source, and example application code.

4.1.2 Installation Contents

This section contains a description of files provided.

`/bin`

- `device-configuration-service.out`: contains the `STAR_runConfigService()` function which runs the Device Configuration Service. This provides the facility to read or modify the properties of devices.
- `Star-SystemPerformanceTester.out`: Star-System Performance Tester functions can be run interactively from the `main` method or by providing command line arguments. Note that the file-handling functions have not been enabled for VxWorks.

- `Star-SystemTest.out`: A suite of general purpose STAR-System test functions. This can be run interactively by invoking `runInteractive()` or by providing command line arguments. Note that the file-handling functions have not been enabled for VxWorks.

/docs

- STAR-System documentation in HTML form.

/examples

- A suite of test/example code is provided including the source code to `Star-SystemPerformanceTester` and `Star-SystemTest.out`. Generic Makefiles are included but these may need to be adapted for use with your VxWorks setup.

/inc/star/

- Header files required to build STAR-System applications.

/lib

- STAR-System libraries (`.a`) compiled for specific VxWorks versions and toolchains.
- The following libraries should be included for all STAR-System application builds: `star-api.a`, `star-port.a`, `star-driver-interface.a`, and `star-VxWorks-driver.a`.
- The following libraries should also be included for all STAR-System applications using the configuration service: `star_conf_mssgr.a`, `star_conf_api_router.a`, `star_conf_api_mk2.a`, `star_conf_api_pci_mk2.a`, `star_rmap-initiator.a` and `rmap_packet_library.a`.

/sys

- This directory contains the driver board initialisation code provided as source; `sysSpaceWirePci.c` and `SpaceWirePci.h`. The function `sysSpaceWirePciInit()` must be called before running any STAR-System API calls.

4.1.3 VxWorks Image Include Components

In your VxWorks image build please make sure that the following components are included.

In operating system components (default), POSIX components: POSIX clocks (default), POSIX directory utilities (default), POSIX mman, POSIX process scheduling, POSIX semaphores (default), POSIX threads (default), POSIX timers (default) and sigevent notification library.

Depending on the BSP used, you may also need to include PCI and PCI autoconfiguration.

4.1.4 Initialising The Driver

Function `sysSpaceWirePciInit()` in file `sysSpaceWirePci.c`, adds an entry to the SpaceWire device resource table (globally defined) for each device found with the correct device and vendor ID. Function `sysSpaceWirePciInit()` must be called before running any STAR-System API calls and could be called from either `sysHwInit2()` or `userAppInit()` in the VxWorks image build. Before building, please define one of the targets at line 66 in `sysSpaceWirePci.c` or adapt the code for a new target. The default target is the Marvell CPC1750 PowerPC. Attempts to call `sysSpaceWirePciInit()` in `userAppInit()` may fail (depending on the BSP) if MMU memory mapping is required, in that instance please call it from either `sysHwInit()` or `sysHwInit2()`.

Function `sysSpaceWirePciInit()` may need to be modified for a specific target if there are any non-standard BSP issues. The following target specific implementations have been developed and tested by STAR-Dundee.

1. Marvell CPC1750 PowerPC, VxWorks 6.7
2. XCalibur 1002, VxWorks 6.7, 6.8

For any other target the following is suggested:

1. Create a new `#define` in `sysSpaceWirePci.c` for your new target.
2. Map BAR registers by reading the base address and storing in the resource table entry. If function `sysMmuMapAdd()` is required by your BSP then add an additional call `sysMmuMapAdd()` for BAR regions 0, 2 and 3.
3. Enable interrupts by obtaining the device IRQ and IRQ vector.
4. Specify whether the device is little or big endian.
5. Define any memory offset address for DMA.
6. Enable the PLX9056 command register.
7. Define read and write register functions.
8. Define interrupt connect and disconnect functions.
9. Define any other BSP cPCI specific tasks, e.g. the correct bus if there are multiple cPCI buses.

Please contact STAR-Dundee should any assistance be required for this procedure.

4.1.5 Quick Start Guide

4.1.5.1 Running The Pre-compiled Test Programs

Follow the instructions to initialise the driver in the above section. After booting VxWorks, download `Star-SystemTest.out` (for the correct toolchain). Connect links 1 and 3 with a SpaceWire cable then run test 2 to transmit and receive data by calling the `runInteractive()` function from a shell.

To test the router configuration service, download `device-configuration-service.out` and `Star-SystemTest.out`, call `STAR_runConfigServe()` which starts the configuration service, then call `runInteractive` and run the `IdentifyDevice` test which will flash the LEDs.

4.1.5.2 Building the example programs

The example programs are supplied with Makefiles which have not been configured for VxWorks. So to build the examples, either modify these Makefiles for your VxWorks set-up or use Workbench to create projects to build the test code. For example, to build STAR-System Test in Workbench please do the following:

1. Create a new downloadable kernel module project,
2. Add the STAR-System Test source code files to your project (or link to external content),
3. In the build properties, add the path to `/inc/star` to your include paths,
4. In the build properties include the 10 library files from the `/lib` directory, and
5. Build and download the test program.

4.2 RTEMS Installation and Getting Started Guide

4.2.1 Introduction

The RTEMS version of STAR-System includes all the APIs, examples and command-line test applications available on other STAR-System supported platforms. At the driver level, a PCI Driver is provided, supporting PCI Mk2, cPCI Mk2, PCIe and PXI devices.

4.2.2 Supported Targets

Currently only LEON targets are supported, with the code compiled for the LEON3. The RTEMS build of STAR-System makes use of the RTEMS Driver Manager, and so requires RTEMS version 4.10.

If you require support for another target, RTEMS version, or for USB devices, please contact STAR-Dundee using the information in the [Support and Contact Information](#) section.

4.2.3 Installation

The STAR-System RTEMS release is provided as a .tgz file which should be extracted to a location that can be accessed from your RTEMS build environment. The example and test applications can then be built using the following command in the directory of the program to be built:

```
make OPERATING_SYSTEM=RTEMS
```

If you wish to remove Ethernet support, please delete the TCP library libstar_tcp_driver.a.

4.2.4 Driver Module

The PCI Driver for STAR-System is provided as a library on RTEMS, named libstar_pci_driver.a. Any application which will make use of the STAR-System APIs to access a device must therefore link against this library, in addition to the other STAR-System libraries. The Makefiles for the STAR-System example applications demonstrate how to do this using `-lstar_pci_driver`.

4.2.5 Device Configuration Service

If you wish to read or modify the properties of a device by calling any of the Device Configuration API functions (those with names beginning `CFG_`) from your application, you must first call the `STAR_runConfigService()` function to start the Device Configuration Service. This function starts the Configuration Service, so should only be called once by an application. Further information on the functionality provided by the Device Configuration Service is available in the [Device Configuration Service](#) section.

4.2.6 RTEMS Workspace Configuration Options

There are no guarantees provided as to the resources used by the current version of STAR-System for RTEMS. The resources used by STAR-System include threads, mutexes, semaphores and condition variables. We therefore recommend setting these resources to be unlimited.

An example source file containing the definitions required to use the STAR-System PCI Driver is provided in the examples directory. This file, `star_rtems_init.c`, is used by all the example and test programs, and can also be used in your own applications. To start the Configuration Service, `STAR_CONFIG_API_REQUIRED` must be defined. This is performed in the example Makefiles by adding `-DSTAR_CONFIG_API_REQUIRED` during compilation.

4.3 Multi-Threading and Multi-Process Support

STAR-System and the various APIs provided, are designed to support multi-threaded and multi-process development.

4.3.1 Multi-Threading Support

All functions in the STAR-System APIs have been designed to be thread safe, and any function can be safely called in multiple threads at the same time.

Despite this thread-safety, there are some situations in which thread synchronisation may be required when calling STAR-System functions. If you are transmitting or receiving on a channel, in most situations you will only transmit from one thread at a time, or receive from one thread at a time. If, however, you wish to transmit from two different threads on the same channel, there is no guarantee of the order in which the packets will be sent, unless synchronisation is used between the threads. If you don't care about the order in which the packets are transmitted, one other thing to consider is that you should always transmit a full packet or make use of synchronisation between the threads. If you transmit the start of a packet, then transmit the end of the packet in a separate function call, the other thread may have transmitted a packet (or chunk of a packet) in the middle of this packet. Similar considerations should be made when receiving on one channel in multiple threads.

Note that the above doesn't apply to transmitting in one thread while receiving in another thread. These are completely independent operations, and it's unlikely synchronisation will be required, unless buffers or other resources are being shared between the two threads and may be modified or destroyed.

The documentation for individual functions highlight any potential issues when using these functions with multiple threads.

4.3.2 Multi-Process Support

STAR-System has also been designed to allow multiple applications to access a device at the same time. Any number of applications can call functions in the STAR-System APIs, and information such as device names can be set in one application and read in another.

Multiple applications can transmit and/or receive on a device at the same time. This is achieved through the use of channels, which must be opened by an application to a device before packets can be transmitted to that device or received from the device. Each application can open a different channel to a device, allowing one application to transmit to a device while another receives from the device, or for one application to transmit on link 1 while another transmits on link 2. See the [Channel Management](#) section of the STAR-API documentation for further information.

The Device Configuration APIs have been designed so that any number of applications (or threads) can query and/or configure a device simultaneously. The Device Configuration Service of STAR-System handles the synchronisation between each of the applications. See the [Device Configuration Service](#) for further information.

4.4 Device Configuration Service

The Device Configuration Service is a small application, named `star_conf_service.exe` on Windows and `star_conf_service` on other platforms, which runs in the background at all times.

The primary purpose of the Device Configuration Service is to allow multiple applications and/or threads to perform Device Configuration functions, either locally or over a SpaceWire network, all at the same time. It is responsible for ensuring each application/thread receives a response to the commands it initiated, and is effectively providing multiplexing/demultiplexing to allow multiple applications and threads to access a single resource. In addition, the Device Configuration Service also maintains state information, such as the channels which are open, the names assigned to devices, etc.

The intention of the Device Configuration Service is that it is always running. This reduces the time required for the first API function call to complete (as it doesn't need to wait on the process starting), and also allows the service to maintain state information after an application closes. By default, the Configuration Service will run at start-up on

Windows, Linux and QNX. When no STAR-System application is running, the process will not be doing anything, so it should be using very few resources. On VxWorks and RTEMS the service runs in a separate thread of the application, and must be started manually using a function call. See the [VxWorks Installation and Getting Started Guide](#) and [RTEMS Installation and Getting Started Guide](#) for further information.

4.4.1 Channel 0

Channel 0 is used by the [Device Configuration Service](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration function call. Therefore, if you are calling any configuration functions and immediately trying to open channel 0 (either using the APIs or supplied applications), it will fail as the configuration service still has the channel open and an appropriate delay may be required.

4.4.2 Windows Session 0

On Windows, the Device Configuration Service runs as a normal process so that it can communicate with user applications running as normal processes. If a user application is instead running as a Windows service, communication is not possible due to session isolation.

To support communication between the Device Configuration Service and a user application running in Session 0, there is an alternative Device Configuration Service provided that runs as a Windows service in Session 0.

The alternative Device Configuration Service is installed alongside the normal Device Configuration Service in the "bin" directory and is called "star_conf_service_session0.exe".

To create the service using the Windows Service Control Manager, the following command should be run in a command-prompt with administrator privileges using the full path to the executable e.g.:

```
sc create "STAR-System Device Configuration Service (Session 0)" bin←  
Path="c:\Program Files\STAR-Dundee\STAR-System\bin\star_conf_service_←  
session0.exe"
```

To start the service, open the Windows Services application, right-click the newly created service and select "Start".

To stop the service, open the Windows Task Manager and end the Device Configuration Service process.

To remove the service using the Windows Service Control Manager, the following command should be run in a command-prompt with administrator privileges using the name used to create the service e.g.:

```
sc delete "STAR-System Device Configuration Service (Session 0)"
```

Note that when using the Device Configuration Service in Session 0, the normal Device Configuration Service must not be running.

Note also that when running the Device Configuration Service in Session 0, it is not possible to use the Device Configuration GUI, as it runs as a normal process. Therefore, all device configuration must be performed using the API.

4.5 Error Logs

STAR-System uses an error log to indicate any serious errors encountered during its operation.

If you encounter any unexpected behaviour, or have trouble debugging an issue, check the error log to see if it can provide you with useful information.

4.5.1 Log File Name and Location

The error log will be located in your user directory, e.g. on Windows Vista onwards:

```
C:\Users\Stuart\
```

on Windows XP:

```
C:\Documents and Settings\Stuart\
```

on Linux and QNX:

```
/home/stuart/
```

or if you're logged in as root on Linux and QNX:

```
/root/
```

The log file is named `star-system.log`, and will never grow above a fixed size (which is currently 1 MByte). When the maximum size is reached, the file is renamed (to `star-system.0.log`) and a new log file is started. If log files have already been archived in this manner, they will be renamed to e.g. `star-system.1.log` before `star-system.0.log` is created. The maximum number of archived logs that will be maintained is currently four. The oldest archived log will be deleted when a fifth is created.

4.5.2 Log File Format

The log file format is quite simple:

```
<Date> <Time> <Message Type> [<Module Name>] <Message>
```

A typical error message looks as follows:

```
24-08-2011 15:39:53 ERROR [STAR-API] Could not communicate with Device Configuration Service
```

The `<Date>` and `<Time>` are the date and time provided by the PC's clock at which the issue occurred. The `<Message Type>` contains one of the following possible values `ERROR`, `WARNING`, `DEBUG` or `TRACE`, indicating the severity of the issue. Debug and trace messages should only be displayed if you have been provided with a special build of a module in order to debug a problem. The `<Module Name>` indicates the module which generated the log message. Possible values for the module name include:

- STAR-API
- Driver Interface
- STAR-Port
- STAR Device Configuration Messenger
- STAR Device Configuration Service
- STAR RMAP Initiator

The Device Configuration APIs may also output messages to the log file.

If you encounter problems with any of these modules, please contact STAR-Dundee's support team for further assistance using the e-mail address support@star-dundee.com.

4.6 Configuration API Functions Supported by Device Types

The following tables show the features provided by each device and the Configuration API functions which can be used to access them.

The Generic Configuration API provides functions for configuring all features on all devices. If a feature is not supported by a device, the API function calls will return a value to indicate this. Please refer to the [Generic Configuration API](#) documentation for further information and [cfg_api_generic.h](#) for the list of possible return values.

The other Configuration APIs were used to configure devices prior to the introduction of the Generic Configuration API in v4.00 of STAR-System. These APIs provide functions which are often limited to a range or single device. It is recommended that the Generic Configuration API is used for all new developments.

4.6.1 Generic Configuration API

Your device may require a firmware update to be compatible with some Generic Configuration API function calls. For any such calls, the required firmware version is documented below.

Feature	Device	Function
Get device identification information	Router Mk2S	Yes CFG_getDeviceIdentificationInfo()
	Brick Mk2	Yes CFG_getDeviceManufacturerAs ↔
	Brick Mk3	Yes String()
	Brick Mk4	Yes CFG_getDeviceTypeAsString()
	GbE Brick Mk1	Yes
	PCI Mk2 / cPCI Mk2	Yes
	PCle	Yes
	PCle Mk2	Yes
	Physical Layer Tester	Yes
	PXI Interface	Yes
	PXI Router	Yes
	PXI Interface Mk2	Yes
	PXI Router Mk2	Yes
	STAR Fire Mk3	Yes STAR-Ultra
	PCle Interface	Yes STAR-Ultra
	PCle Single-Lane Router	Yes
Network discovery information	Router Mk2S	Yes CFG_getNetworkDiscoveryInfo()
	Brick Mk2	Yes
	Brick Mk3	Yes
	Brick Mk4	Yes
	GbE Brick Mk1	Yes
	PCI Mk2 / cPCI Mk2	Yes
	PCle	Yes
	PCle Mk2	Yes
	Physical Layer Tester	Yes
	PXI Interface	Yes
	PXI Router	Yes
	PXI Interface Mk2	Yes
	PXI Router Mk2	Yes
	STAR Fire Mk3	Yes STAR-Ultra
	PCle Interface	No STAR-Ultra
	PCle Single-Lane Router	No

Port status and control	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_getPortStatusControl() CFG_getPortType() CFG_getPortConnection() CFG_getConfigPortErrors() CFG_getSpaceWireLinkErrors() CFG_getExternalPortErrors() CFG_getExternalPortStatus()
Clear port errors	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_clearPortErrors()
Get and set link status	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_getSpaceWireLinkStatus() CFG_setSpaceWireLinkStatus()

Start and stop links	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_startLink() CFG_stopLink()
Get and set routing table entries	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getRoutingTableEntry() CFG_setRoutingTableEntry()
Get and set network identity	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getNetworkIdentity() CFG_setNetworkIdentity()

Get and set general purpose register	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getGeneralPurpose() CFG_setGeneralPurpose()
Get and set time-code distribution ports	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getTimeCodeDistribution↔ Ports() CFG_setTimeCodeDistribution↔ Ports()
Get and set router global settings	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getRouterGlobalSettings() CFG_setRouterGlobalSettings()

Get current time-code value	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getTimeCodeValue()
Get and set time-code flag mode	Router Mk2S Yes Brick Mk2 No Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_getTimeCodeFlagMode() CFG_setTimeCodeFlagMode()
Enable and disable time-code master	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_enableTimeCodeMaster() CFG_disableTimeCodeMaster() CFG_getTimeCodeMaster↔ Enabled()

Get and set time-code period	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_getTimeCodePeriod() CFG_setTimeCodePeriod()
Enable and disable external time-code selection	Router Mk2S Yes Brick Mk2 No Brick Mk3 No Brick Mk4 No GbE Brick Mk1 No PCI Mk2 / cPCI Mk2 No PCIe No PCIe Mk2 No Physical Layer Tester No PXI Interface No PXI Router No PXI Interface Mk2 No PXI Router Mk2 No STAR Fire Mk3 No STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_enableExternalTimeCode↔ Selection() CFG_disableExternalTimeCode↔ Selection() CFG_getExternalTimeCode↔ SelectionEnabled()
Get FPGA information	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface Yes STAR-Ultra PCIe Single-Lane Router Yes	CFG_getFPGAInfo() CFG_FPGAInfoToString()

Identify device	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_identify()
Enable and disable interface mode	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_enableInterfaceMode() CFG_disableInterfaceMode() CFG_getInterfaceModeEnabled()
Enable and disable identify source	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_enableIdentifySource() CFG_disableIdentifySource() CFG_getIdentifySourceEnabled()

Enable and disable interface mode on port	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_enableInterfaceModeOn↔ Port() CFG_disableInterfaceModeOn↔ Port() CFG_getInterfaceModeOnPort↔ Enabled()
Enable and disable identify source on port	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_enableIdentifySourceOn↔ Port() CFG_disableIdentifySourceOn↔ Port() CFG_getIdentifySourceOnPort↔ Enabled()
Get and set port routing address	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCIe Yes PCIe Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCIe Interface No STAR-Ultra PCIe Single-Lane Router No	CFG_getPortRoutingAddress() CFG_setPortRoutingAddress()

Inject error	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 Yes PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_injectError()
Inject errors	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 No PCle No PCle Mk2 Yes Physical Layer Tester No PXI Interface No PXI Router No PXI Interface Mk2 No PXI Router Mk2 No STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_injectErrors()
Enable and disable link state change events	Router Mk2S Yes Brick Mk2 Yes Brick Mk3 Yes Brick Mk4 Yes GbE Brick Mk1 Yes PCI Mk2 / cPCI Mk2 No PCle Yes PCle Mk2 Yes Physical Layer Tester Yes PXI Interface Yes PXI Router Yes PXI Interface Mk2 Yes PXI Router Mk2 Yes STAR Fire Mk3 Yes STAR-Ultra PCle Interface No STAR-Ultra PCle Single-Lane Router No	CFG_enableStateChange↔ EventsOnPort() CFG_disableStateChange↔ EventsOnPort() CFG_getStateChangeEventsOn↔ PortEnabled()

Enable and disable link speed change events	Router Mk2S	Yes	CFG_enableSpeedChange↔ EventsOnPort() CFG_disableSpeedChange↔ EventsOnPort() CFG_getSpeedChangeEvents↔ OnPortEnabled()
	Brick Mk2	Yes	
	Brick Mk3	Yes	
	Brick Mk4	Yes	
	GbE Brick Mk1	Yes	
	PCI Mk2 / cPCI Mk2	No	
	PCIe	Yes	
	PCIe Mk2	Yes	
	Physical Layer Tester	Yes	
	PXI Interface	Yes	
	PXI Router	Yes	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	Yes	
	STAR Fire Mk3	Yes STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	
Transmit signalling rate	Router Mk2S	Yes	CFG_getTxSignallingRate() CFG_setTxSignallingRate()
	Brick Mk2	Yes	
	Brick Mk3 (from v1.01)	Yes	
	Brick Mk4	Yes	
	GbE Brick Mk1	Yes	
	PCI Mk2 / cPCI Mk2	Yes	
	PCIe (from v1.11)	Yes	
	PCIe Mk2	Yes	
	Physical Layer Tester		
	(from v2.00)	Yes	
	PXI Interface		
	(from v1.02 edit 3)	Yes	
	PXI Router		
	(from v1.02 edit 4)	Yes	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	Yes	
	STAR Fire Mk3	Yes STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	

Receive signalling rate	Router Mk2S	Yes	CFG_getRxSignallingRate()
	Brick Mk2	Yes	
	Brick Mk3	Yes	
	Brick Mk4	Yes	
	GbE Brick Mk1	Yes	
	PCI Mk2 / cPCI Mk2	No	
	PCIe	Yes	
	PCIe Mk2	Yes	
	Physical Layer Tester	Yes	
	PXI Interface	Yes	
	PXI Router	Yes	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	Yes	
	STAR Fire Mk3	Yes STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	
Precision transmit rate	Router Mk2S	Yes	CFG_enablePrecisionTransmit↔ Rate() CFG_disablePrecisionTransmit↔ Rate() CFG_getPrecisionTransmitRate↔ Enabled() CFG_getPrecisionTransmitRate↔ InUse() CFG_getPrecisionTransmitRate() CFG_setPrecisionTransmitRate()
	Brick Mk2	No	
	Brick Mk3	No	
	Brick Mk4	No	
	GbE Brick Mk1	No	
	PCI Mk2 / cPCI Mk2	No	
	PCIe	No	
	PCIe Mk2	No	
	Physical Layer Tester	No	
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	
Timestamping	Router Mk2S	No	CFG_getTimestampMethod() CFG_setTimestampMethod() CFG_enableRxTimestamp↔ EventsOnPort() CFG_disableRxTimestamp↔ EventsOnPort() CFG_getRxTimestampEvents↔ OnPortEnabled() CFG_getTimestampValue() CFG_setTimestampValue() CFG_getPulseGenerator↔ Frequency() CFG_setPulseGenerator↔ Frequency()
	Brick Mk2	No	
	Brick Mk3 (from v1.02)	Yes	
	Brick Mk4	Yes	
	GbE Brick Mk1	No	
	PCI Mk2 / cPCI Mk2	No	
	PCIe	No	
	PCIe Mk2	Yes	
	Physical Layer Tester	No	
	PXI Interface (from v1.04)	Yes	
	PXI Router	No	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	

Periodic actions	Router Mk2S	No	CFG_getDeviceClockRate()
	Brick Mk2	No	CFG_startPeriodicAction()
	Brick Mk3	No	CFG_stopPeriodicAction()
	Brick Mk4	No	
	GbE Brick Mk1	No	
	PCI Mk2 / cPCI Mk2	Yes	
	PCIe	No	
	PCIe Mk2	No	
	Physical Layer Tester	No	
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	
Enable and disable time-code counter bypass mode	Router Mk2S	No	CFG_enableTimeCodeCounter↔
	Brick Mk2	No	BypassMode()
	Brick Mk3	No	CFG_disableTimeCodeCounter↔
	Brick Mk4	No	BypassMode()
	GbE Brick Mk1	No	CFG_getTimeCodeCounter↔
	PCI Mk2 / cPCI Mk2	No	BypassModeEnabled()
	PCIe	Yes	
	PCIe Mk2	Yes	
	Physical Layer Tester	No	
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	
Enable and disable time-code events	Router Mk2S	No	CFG_enableTimeCodeEvents↔
	Brick Mk2	No	OnPort()
	Brick Mk3	No	CFG_disableTimeCodeEvents↔
	Brick Mk4	No	OnPort()
	GbE Brick Mk1	No	CFG_getTimeCodeEventsOn↔
	PCI Mk2 / cPCI Mk2	No	PortEnabled()
	PCIe	Yes	
	PCIe Mk2	Yes	
	Physical Layer Tester	No	
	PXI Interface		
	(from v1.02 edit 3)	Yes	
	PXI Router		
	(from v1.02 edit 4)	Yes	
	PXI Interface Mk2	Yes	
	PXI Router Mk2	Yes	
	STAR Fire Mk3	No STAR-Ultra	
	PCIe Interface	No STAR-Ultra	
	PCIe Single-Lane Router	No	

Enable and disable broadcast controller legacy mode	Router Mk2S	No	CFG_enableBroadcast↔ ControllerLegacyMode() CFG_disableBroadcast↔ ControllerLegacyMode() CFG_getBroadcastController↔ LegacyModeEnabled()
	Brick Mk2	No	
	Brick Mk3	No	
	Brick Mk4	No	
	GbE Brick Mk1	No	
	PCI Mk2 / cPCI Mk2	No	
	PCle	No	
	PCle Mk2	Yes	
	Physical Layer Tester	No	
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCle Interface	No STAR-Ultra	
	PCle Single-Lane Router	No	
Get and set interrupt code distribution ports	Router Mk2S	No	CFG_getInterruptCode↔ DistributionPorts() CFG_setInterruptCode↔ DistributionPorts()
	Brick Mk2	No	
	Brick Mk3	No	
	Brick Mk4	No	
	GbE Brick Mk1	No	
	PCI Mk2 / cPCI Mk2	No	
	PCle	No	
	PCle Mk2	Yes	
	Physical Layer Tester	No	
	PXI Interface	No	
	PXI Router	No	
	PXI Interface Mk2	No	
	PXI Router Mk2	No	
	STAR Fire Mk3	No STAR-Ultra	
	PCle Interface	No STAR-Ultra	
	PCle Single-Lane Router	No	

4.6.2 SpaceWire cPCI Mk2 and SpaceWire PCI Mk2

Further information on the features present in the SpaceWire cPCI Mk2 and PCI Mk2 can be found in the [SpaceWire PCI Mk2 and cPCI Mk2 user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_ROUTER_getTimeCodeDistributionPorts() CFG_ROUTER_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_ROUTER_getTimeCodeFlagMode() CFG_ROUTER_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_PCIMK2_getLinkClockFrequency() CFG_PCIMK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_MK2_enableInterfaceModeOnPort() CFG_MK2_disableInterfaceModeOnPort() CFG_MK2_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_MK2_enableIdentifySourceOnPort() CFG_MK2_disableIdentifySourceOnPort() CFG_MK2_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_MK2_getPortRoutingAddress() CFG_MK2_setPortRoutingAddress()
Inject errors	CFG_MK2_injectError()
Periodic actions	CFG_MK2_getDeviceClockRate() CFG_MK2_startPeriodicAction() CFG_MK2_stopPeriodicAction()
Enable and disable link state change events	<i>Not supported</i>
Enable and disable link speed change events	<i>Not supported</i>
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	<i>Not supported</i>
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.3 SpaceWire PCIe and SpaceWire Physical Layer Tester

Further information on the features present in the SpaceWire PCIe can be found in the [SpaceWire PCIe user manual](#). A brief overview of the [SpaceWire Physical Layer Tester](#) is provided in the [Other STAR-System Devices](#) section.

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_ROUTER_getTimeCodeDistributionPorts() CFG_ROUTER_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_ROUTER_getTimeCodeFlagMode() CFG_ROUTER_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_MK2_getBaseTransmitClock() CFG_MK2_setBaseTransmitClock() CFG_MK2_getTransmitClock() CFG_MK2_setTransmitClock()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode (PCIe only)	CFG_MK2_enableTimeCodeCounterBypassMode() CFG_MK2_disableTimeCodeCounterBypassMode() CFG_MK2_getTimeCodeCounterBypassModeEnabled()

Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_MK2_enableInterfaceModeOnPort() CFG_MK2_disableInterfaceModeOnPort() CFG_MK2_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_MK2_enableIdentifySourceOnPort() CFG_MK2_disableIdentifySourceOnPort() CFG_MK2_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_MK2_getPortRoutingAddress() CFG_MK2_setPortRoutingAddress()
Inject errors	CFG_MK2_injectError()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_MK2_enableStateChangeEventsOnPort() CFG_MK2_disableStateChangeEventsOnPort() CFG_MK2_getStateChangeEventsOnPortEnabled()
Enable and disable link speed change events	CFG_MK2_enableSpeedChangeEventsOnPort() CFG_MK2_disableSpeedChangeEventsOnPort() CFG_MK2_getSpeedChangeEventsOnPortEnabled()
Enable and disable time-code events (PCIe only)	CFG_MK2_enableTimeCodeEventsOnPort() CFG_MK2_disableTimeCodeEventsOnPort() CFG_MK2_getTimeCodeEventsOnPortEnabled()
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.4 SpaceWire Brick Mk2

Further information on the features present in the SpaceWire Brick Mk2 can be found in the [SpaceWire Brick Mk2 user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	<i>Not supported</i>
Get and set base transmit clock	CFG_BRICK_MK2_getLinkClockFrequency() CFG_BRICK_MK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	<i>Not supported</i>

4.6.5 SpaceWire Router Mk2S

Further information on the features present in the SpaceWire Router Mk2S can be found in the [SpaceWire Router Mk2S user manual](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	<i>Not supported</i>
Get and set base transmit clock	CFG_BRICK_MK2_getLinkClockFrequency() CFG_BRICK_MK2_setLinkClockFrequency()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	CFG_MK2_enableExternalTimeCodeSelection() CFG_MK2_disableExternalTimeCodeSelection() CFG_MK2_getExternalTimeCodeSelectionEnabled()
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()

Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	CFG_ROUTER_MK2S_enablePrecisionTransmit↔Rate() CFG_ROUTER_MK2S_disablePrecisionTransmit↔Rate() CFG_ROUTER_MK2S_getPrecisionTransmitRate↔Enabled() CFG_ROUTER_MK2S_getPrecisionTransmitRate↔InUse()
Get and set precision transmit rate	CFG_ROUTER_MK2S_getPrecisionTransmitRate() CFG_ROUTER_MK2S_setPrecisionTransmitRate()
Timestamping	<i>Not supported</i>

4.6.6 SpaceWire Brick Mk3 and Brick Mk4

Further information on the features present in the SpaceWire Brick Mk3 and Brick Mk4 can be found in the [SpaceWire Brick Mk3 and Brick Mk4](#) user manual.

Additionally, for Brick Mk4 devices, at power-up their SpaceWire ports default to Tri-State. This means the LVDS drivers are disabled and hence in a high impedance state.

A port's LVDS drivers can be enabled either by starting the link, by receiving data on the link, or by explicitly disabling tri-state with [CFG_setSpaceWireLinkStatus\(\)](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_BRICK_MK3_getBaseTransmitClock() (<i>Brick Mk3 only</i>) CFG_BRICK_MK3_setBaseTransmitClock() (<i>Brick Mk3 only</i>) CFG_BRICK_MK3_getTransmitClock() CFG_BRICK_MK3_setTransmitClock()
Get and set link rate divider	CFG_MK2_getLinkRateDivider() CFG_MK2_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()

Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode	<i>Not supported</i>
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_BRICK_MK2_enableInterfaceModeOnPort() CFG_BRICK_MK2_disableInterfaceModeOnPort() CFG_BRICK_MK2_getInterfaceModeOnPort↔Enabled()
Enable and disable identify source on port	CFG_BRICK_MK2_enableIdentifySourceOnPort() CFG_BRICK_MK2_disableIdentifySourceOnPort() CFG_BRICK_MK2_getIdentifySourceOnPort↔Enabled()
Get and set port routing address	CFG_BRICK_MK2_getPortRoutingAddress() CFG_BRICK_MK2_setPortRoutingAddress()
Inject errors	CFG_BRICK_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_BRICK_MK2_enableStateChangeEventsOn↔Port() CFG_BRICK_MK2_disableStateChangeEventsOn↔Port() CFG_BRICK_MK2_getStateChangeEventsOnPort↔Enabled()
Enable and disable link speed change events	CFG_BRICK_MK2_enableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_disableSpeedChangeEventsOn↔Port() CFG_BRICK_MK2_getSpeedChangeEventsOnPort↔Enabled()
Enable and disable time-code events	<i>Not supported</i>
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	CFG_BRICK_MK3_setTimestampMethod() CFG_BRICK_MK3_getTimestampMethod() CFG_BRICK_MK3_enableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_getRxTimestampEventsOnPort↔Enabled() CFG_BRICK_MK3_disableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_setTimestampValue() CFG_BRICK_MK3_getTimestampValue() CFG_BRICK_MK3_setPulseGeneratorFrequency() CFG_BRICK_MK3_getPulseGeneratorFrequency()

4.6.7 SpaceWire PXI Devices

Further information on the features present in the SpaceWire PXI devices can be found in the [SpaceWire PXI Interface and PXI Interface Mk2](#) or [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) user manuals.

Additionally, for PXI devices, at power-up their SpaceWire ports default to Tri-State. This means the LVDS drivers are disabled and hence in a high impedance state.

A port's LVDS drivers can be enabled either by starting the link, by receiving data on the link, or by explicitly disabling tri-state with [CFG_setSpaceWireLinkStatus\(\)](#).

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Get and set base transmit clock	CFG_PXI_getBaseTransmitClock() CFG_PXI_setBaseTransmitClock() CFG_PXI_getTransmitClock() CFG_PXI_setTransmitClock()
Get and set link rate divider	CFG_PXI_getLinkRateDivider() CFG_PXI_setLinkRateDivider()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode	<i>Not supported</i>

Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_PXI_enableInterfaceModeOnPort() CFG_PXI_disableInterfaceModeOnPort() CFG_PXI_getInterfaceModeOnPortEnabled()
Enable and disable identify source on port	CFG_PXI_enableIdentifySourceOnPort() CFG_PXI_disableIdentifySourceOnPort() CFG_PXI_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_PXI_getPortRoutingAddress() CFG_PXI_setPortRoutingAddress()
Inject errors	CFG_PXI_injectError()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_PXI_enableStateChangeEventsOnPort() CFG_PXI_disableStateChangeEventsOnPort() CFG_PXI_getStateChangeEventsOnPortEnabled()
Enable and disable link speed change events	CFG_PXI_enableSpeedChangeEventsOnPort() CFG_PXI_disableSpeedChangeEventsOnPort() CFG_PXI_getSpeedChangeEventsOnPortEnabled()
Enable and disable time-code events	CFG_PXI_enableTimeCodeEventsOnPort() CFG_PXI_disableTimeCodeEventsOnPort() CFG_PXI_getTimeCodeEventsOnPortEnabled()
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping (SpaceWire PXI Interface and PXI Interface Mk2 only)	CFG_BRICK_MK3_setTimestampMethod() CFG_BRICK_MK3_getTimestampMethod() CFG_BRICK_MK3_enableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_getRxTimestampEventsOnPort↔Enabled() CFG_BRICK_MK3_disableRxTimestampEventsOn↔Port() CFG_BRICK_MK3_setTimestampValue() CFG_BRICK_MK3_getTimestampValue() CFG_BRICK_MK3_setPulseGeneratorFrequency() CFG_BRICK_MK3_getPulseGeneratorFrequency()

4.6.8 SpaceWire GbE Brick Mk1

Further information on the features present in the SpaceWire GbE Brick Mk1 can be found in the [SpaceWire GbE Brick Mk1](#) user manual.

Additionally, just like for Brick Mk4 devices, at power-up their SpaceWire ports default to Tri-State. This means the LVDS drivers are disabled and hence in a high impedance state.

A port's LVDS drivers can be enabled either by starting the link, by receiving data on the link, or by explicitly disabling tri-state with [CFG_setSpaceWireLinkStatus\(\)](#).

Feature	Function
Get and set transmit signalling rate	CFG_GBE_BRICK_MK1_getTransmitSignallingRate() CFG_GBE_BRICK_MK1_setTransmitSignallingRate()

4.6.9 SpaceWire PCIe Mk2

Further information on the features present in the SpaceWire PCIe Mk2 can be found in the [SpaceWire PCIe Mk2 user manual](#).

It is recommended that the [Generic Configuration API](#) is used rather than the functions below. The Generic Configuration API provides functions for configuring all features on all devices, allowing software that supports multiple device types to be easily written.

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()
Network discovery information	CFG_ROUTER_getNetworkDiscoveryInfo()
Port status and control	CFG_ROUTER_getPortStatusControl() CFG_ROUTER_getPortType() CFG_ROUTER_getPortConnection() CFG_ROUTER_getConfigPortErrors() CFG_ROUTER_getSpaceWireLinkErrors() CFG_ROUTER_getExternalPortErrors() CFG_ROUTER_getExternalPortStatus()
Clear port errors	CFG_ROUTER_clearPortErrors()
Get and set link status	CFG_ROUTER_getSpaceWireLinkStatus() CFG_ROUTER_setSpaceWireLinkStatus()
Start and stop links	CFG_ROUTER_startLink() CFG_ROUTER_stopLink()
Get and set routing table entries	CFG_ROUTER_getRoutingTableEntry() CFG_ROUTER_setRoutingTableEntry()
Get and set network identity	CFG_ROUTER_getNetworkIdentity() CFG_ROUTER_setNetworkIdentity()
Get and set general purpose register	CFG_ROUTER_getGeneralPurpose() CFG_ROUTER_setGeneralPurpose()
Get and set time-code distribution ports	CFG_MK2_getTimeCodeDistributionPorts() CFG_MK2_setTimeCodeDistributionPorts()
Get and set router global settings	CFG_ROUTER_getRouterGlobalSettings() CFG_ROUTER_setRouterGlobalSettings()
Get current time-code value	CFG_ROUTER_getTimeCodeValue()
Get and set time-code flag mode	CFG_MK2_getTimeCodeFlagMode() CFG_MK2_setTimeCodeFlagMode()
Enable and disable time-code master	CFG_MK2_enableTimeCodeMaster() CFG_MK2_disableTimeCodeMaster() CFG_MK2_getTimeCodeMasterEnabled()
Get and set time-code period	CFG_MK2_getTimeCodePeriod() CFG_MK2_setTimeCodePeriod()
Enable and disable external time-code selection	<i>Not supported</i>
Enable and disable time-code counter bypass mode	CFG_MK2_enableTimeCodeCounterBypassMode() CFG_MK2_disableTimeCodeCounterBypassMode() CFG_MK2_getTimeCodeCounterBypassModeEnabled()
Get hardware information	CFG_MK2_getHardwareInfo() CFG_MK2_hardwareInfoToString()
Identify device	CFG_MK2_identify()
Enable and disable interface mode	CFG_MK2_enableInterfaceMode() CFG_MK2_disableInterfaceMode() CFG_MK2_getInterfaceModeEnabled()
Enable and disable identify source	CFG_MK2_enableIdentifySource() CFG_MK2_disableIdentifySource() CFG_MK2_getIdentifySourceEnabled()
Enable and disable interface mode on port	CFG_MK2_enableInterfaceModeOnPort() CFG_MK2_disableInterfaceModeOnPort() CFG_MK2_getInterfaceModeOnPortEnabled()

Enable and disable identify source on port	CFG_MK2_enableIdentifySourceOnPort() CFG_MK2_disableIdentifySourceOnPort() CFG_MK2_getIdentifySourceOnPortEnabled()
Get and set port routing address	CFG_MK2_getPortRoutingAddress() CFG_MK2_setPortRoutingAddress()
Inject errors	CFG_PCIE_MK2_injectErrors()
Periodic actions	<i>Not supported</i>
Enable and disable link state change events	CFG_MK2_enableStateChangeEventsOnPort() CFG_MK2_disableStateChangeEventsOnPort() CFG_MK2_getStateChangeEventsOnPortEnabled()
Enable and disable link speed change events	CFG_MK2_enableSpeedChangeEventsOnPort() CFG_MK2_disableSpeedChangeEventsOnPort() CFG_MK2_getSpeedChangeEventsOnPortEnabled()
Enable and disable time-code events	CFG_MK2_enableTimeCodeEventsOnPort() CFG_MK2_disableTimeCodeEventsOnPort() CFG_MK2_getTimeCodeEventsOnPortEnabled()
Get measured link speed	CFG_MK2_getMeasuredLinkSpeed()
Enable and disable precision transmit rate	<i>Not supported</i>
Get and set precision transmit rate	<i>Not supported</i>
Timestamping	CFG_PCIE_MK2_setTimestampMethod() CFG_BRICK_MK3_getTimestampMethod() CFG_PCIE_MK2_enableRxTimestampEventsOn↔ Port() CFG_PCIE_MK2_getRxTimestampEventsOnPort↔ Enabled() CFG_PCIE_MK2_disableRxTimestampEventsOn↔ Port() CFG_BRICK_MK3_setTimestampValue() CFG_BRICK_MK3_getTimestampValue() CFG_PCIE_MK2_setPulseGeneratorFrequency() CFG_PCIE_MK2_getPulseGeneratorFrequency()
Get and set transmit signalling rate	CFG_PCIE_MK2_setTransmitSignallingRate() CFG_PCIE_MK2_getTransmitSignallingRate() CFG_PCIE_MK2_setDecimalTxSignallingRate() CFG_PCIE_MK2_getDecimalTxSignallingRate()
Enable and disable broadcast controller legacy mode	CFG_PCIE_MK2_enableBroadcastController↔ LegacyMode() CFG_PCIE_MK2_getBroadcastControllerLegacy↔ ModeEnabled() CFG_PCIE_MK2_disableBroadcastController↔ LegacyMode()
Get and set interrupt code distribution ports	CFG_PCIE_MK2_setInterruptCodeDistributionPorts() CFG_PCIE_MK2_getInterruptCodeDistributionPorts()

4.6.10 STAR-Ultra PCIe Interface and STAR-Ultra PCIe Single-Lane Router

It is recommended that the [Generic Configuration API](#) is used rather than the functions below. The Generic Configuration API provides functions for configuring all features on all devices, allowing software that supports multiple device types to be easily written.

The [SpaceFibre Configuration API](#) is used to configure the SpaceFibre features of the STAR-Ultra PCIe Single-Lane Router.

Feature	Function
Get device identification information	CFG_ROUTER_getDeviceIdentificationInfo() CFG_ROUTER_getDeviceManufacturerAsString() CFG_ROUTER_getDeviceTypeAsString()

4.7 Static Analysis

A number of functions in the STAR-System APIs make use of SAL Annotations.

SAL is the Microsoft source-code annotation language (SAL), which provides a set of annotations to describe how a function uses its parameters, for example, the assumptions it makes about them and the guarantees it makes on finishing. The header file `<sal.h>` defines the annotations.

When using the Microsoft compiler with the `\analyze` flag, these annotations are used to provide information to the developer about possible defects, such as buffer overruns and NULL pointer dereferences. To disable the use of annotations on Windows, define the pre-processor variable:

```
NO_STAR_ANNOTATIONS
```

Note that these annotations are defined as nothing for non-Windows platforms.

For more information, please see the documentation available on the MSDN Library:

- [Code Analysis for C/C++ Overview](#)
- [SAL Annotations](#)

4.8 Open Source Components

The following sections list any open source components used within STAR-System.

4.8.1 Qt4 and Qt5 Libraries

The STAR-System GUI applications make use of the Qt4 and Qt5 libraries, as described in the [Graphical Applications](#) section.

The Qt5 libraries are distributed with STAR-System on Windows and the source code for these libraries may be downloaded from our website (<https://www.star-dundee.com/source/qt/qt-everywhere-src-5.12.12.zip>).

The Qt4 and Qt5 components are licensed under the terms of the GNU Lesser General Public License version 2.1 and version 3 respectively.

```
GNU LESSER GENERAL PUBLIC LICENSE
Version 2.1, February 1999
```

```
Copyright (C) 1991, 1999 Free Software Foundation, Inc.
51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.
```

[This is the first released version of the Lesser GPL. It also counts as the successor of the GNU Library Public License, version 2, hence the version number 2.1.]

Preamble

The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public Licenses are intended to guarantee your freedom to share and change free software--to make sure the software is free for all its users.

This license, the Lesser General Public License, applies to some specially designated software packages--typically libraries--of the Free Software Foundation and other authors who decide to use it. You can use it too, but we suggest you first think carefully about whether this license or the ordinary General Public License is the better strategy to use in any particular case, based on the explanations below.

When we speak of free software, we are referring to freedom of use, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish); that you receive source code or can get it if you want it; that you can change the software and use pieces of it in new free programs; and that you are informed that you can do these things.

To protect your rights, we need to make restrictions that forbid distributors to deny you these rights or to ask you to surrender these rights. These restrictions translate to certain responsibilities for you if you distribute copies of the library or if you modify it.

For example, if you distribute copies of the library, whether gratis or for a fee, you must give the recipients all the rights that we gave you. You must make sure that they, too, receive or can get the source code. If you link other code with the library, you must provide complete object files to the recipients, so that they can relink them with the library after making changes to the library and recompiling it. And you must show them these terms so they know their rights.

We protect your rights with a two-step method: (1) we copyright the library, and (2) we offer you this license, which gives you legal permission to copy, distribute and/or modify the library.

To protect each distributor, we want to make it very clear that there is no warranty for the free library. Also, if the library is modified by someone else and passed on, the recipients should know that what they have is not the original version, so that the original author's reputation will not be affected by problems that might be introduced by others.

Finally, software patents pose a constant threat to the existence of any free program. We wish to make sure that a company cannot effectively restrict the users of a free program by obtaining a restrictive license from a patent holder. Therefore, we insist that any patent license obtained for a version of the library must be consistent with the full freedom of use specified in this license.

Most GNU software, including some libraries, is covered by the ordinary GNU General Public License. This license, the GNU Lesser General Public License, applies to certain designated libraries, and is quite different from the ordinary General Public License. We use this license for certain libraries in order to permit linking those libraries into non-free programs.

When a program is linked with a library, whether statically or using a shared library, the combination of the two is legally speaking a combined work, a derivative of the original library. The ordinary General Public License therefore permits such linking only if the entire combination fits its criteria of freedom. The Lesser General Public License permits more lax criteria for linking other code with the library.

We call this license the "Lesser" General Public License because it does Less to protect the user's freedom than the ordinary General Public License. It also provides other free software developers Less of an advantage over competing non-free programs. These disadvantages are the reason we use the ordinary General Public License for many libraries. However, the Lesser license provides advantages in certain special circumstances.

For example, on rare occasions, there may be a special need to encourage the widest possible use of a certain library, so that it becomes a de-facto standard. To achieve this, non-free programs must be allowed to use the library. A more frequent case is that a free library does the same job as widely used non-free libraries. In this case, there is little to gain by limiting the free library to free software only, so we use the Lesser General Public License.

In other cases, permission to use a particular library in non-free programs enables a greater number of people to use a large body of free software. For example, permission to use the GNU C Library in non-free programs enables many more people to use the whole GNU operating system, as well as its variant, the GNU/Linux operating system.

Although the Lesser General Public License is Less protective of the users' freedom, it does ensure that the user of a program that is linked with the Library has the freedom and the wherewithal to run that program using a modified version of the Library.

The precise terms and conditions for copying, distribution and modification follow. Pay close attention to the difference between a "work based on the library" and a "work that uses the library". The former contains code derived from the library, whereas the latter must be combined with the library in order to run.

GNU LESSER GENERAL PUBLIC LICENSE
TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION

0. This License Agreement applies to any software library or other program which contains a notice placed by the copyright holder or other authorized party saying it may be distributed under the terms of this Lesser General Public License (also called "this License"). Each licensee is addressed as "you".

A "library" means a collection of software functions and/or data prepared so as to be conveniently linked with application programs (which use some of those functions and data) to form executables.

The "Library", below, refers to any such software library or work which has been distributed under these terms. A "work based on the Library" means either the Library or any derivative work under copyright law: that is to say, a work containing the Library or a portion of it, either verbatim or with modifications and/or translated straightforwardly into another language. (Hereinafter, translation is included without limitation in the term "modification".)

"Source code" for a work means the preferred form of the work for making modifications to it. For a library, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the library.

Activities other than copying, distribution and modification are not covered by this License; they are outside its scope. The act of running a program using the Library is not restricted, and output from such a program is covered only if its contents constitute a work based on the Library (independent of the use of the Library in a tool for writing it). Whether that is true depends on what the Library does and what the program that uses the Library does.

1. You may copy and distribute verbatim copies of the Library's complete source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact

all the notices that refer to this License and to the absence of any warranty; and distribute a copy of this License along with the Library.

You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee.

2. You may modify your copy or copies of the Library or any portion of it, thus forming a work based on the Library, and copy and distribute such modifications or work under the terms of Section 1 above, provided that you also meet all of these conditions:

- a) The modified work must itself be a software library.
- b) You must cause the files modified to carry prominent notices stating that you changed the files and the date of any change.
- c) You must cause the whole of the work to be licensed at no charge to all third parties under the terms of this License.
- d) If a facility in the modified Library refers to a function or a table of data to be supplied by an application program that uses the facility, other than as an argument passed when the facility is invoked, then you must make a good faith effort to ensure that, in the event an application does not supply such function or table, the facility still operates, and performs whatever part of its purpose remains meaningful.

(For example, a function in a library to compute square roots has a purpose that is entirely well-defined independent of the application. Therefore, Subsection 2d requires that any application-supplied function or table used by this function must be optional: if the application does not supply it, the square root function must still compute square roots.)

These requirements apply to the modified work as a whole. If identifiable sections of that work are not derived from the Library, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Library, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it.

Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Library.

In addition, mere aggregation of another work not based on the Library with the Library (or with a work based on the Library) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

3. You may opt to apply the terms of the ordinary GNU General Public License instead of this License to a given copy of the Library. To do this, you must alter all the notices that refer to this License, so that they refer to the ordinary GNU General Public License, version 2, instead of to this License. (If a newer version than version 2 of the ordinary GNU General Public License has appeared, then you can specify that version instead if you wish.) Do not make any other change in these notices.

Once this change is made in a given copy, it is irreversible for that copy, so the ordinary GNU General Public License applies to all subsequent copies and derivative works made from that copy.

This option is useful when you wish to copy part of the code of the Library into a program that is not a library.

4. You may copy and distribute the Library (or a portion or derivative of it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you accompany it with the complete corresponding machine-readable source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange.

If distribution of object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place satisfies the requirement to distribute the source code, even though third parties are not compelled to copy the source along with the object code.

5. A program that contains no derivative of any portion of the Library, but is designed to work with the Library by being compiled or linked with it, is called a "work that uses the Library". Such a work, in isolation, is not a derivative work of the Library, and therefore falls outside the scope of this License.

However, linking a "work that uses the Library" with the Library creates an executable that is a derivative of the Library (because it contains portions of the Library), rather than a "work that uses the library". The executable is therefore covered by this License. Section 6 states terms for distribution of such executables.

When a "work that uses the Library" uses material from a header file that is part of the Library, the object code for the work may be a derivative work of the Library even though the source code is not. Whether this is true is especially significant if the work can be linked without the Library, or if the work is itself a library. The threshold for this to be true is not precisely defined by law.

If such an object file uses only numerical parameters, data structure layouts and accessors, and small macros and small inline functions (ten lines or less in length), then the use of the object file is unrestricted, regardless of whether it is legally a derivative work. (Executables containing this object code plus portions of the Library will still fall under Section 6.)

Otherwise, if the work is a derivative of the Library, you may distribute the object code for the work under the terms of Section 6. Any executables containing that work also fall under Section 6, whether or not they are linked directly with the Library itself.

6. As an exception to the Sections above, you may also combine or link a "work that uses the Library" with the Library to produce a work containing portions of the Library, and distribute that work under terms of your choice, provided that the terms permit modification of the work for the customer's own use and reverse engineering for debugging such modifications.

You must give prominent notice with each copy of the work that the Library is used in it and that the Library and its use are covered by this License. You must supply a copy of this License. If the work during execution displays copyright notices, you must include the copyright notice for the Library among them, as well as a reference directing the user to the copy of this License. Also, you must do one of these things:

a) Accompany the work with the complete corresponding machine-readable source code for the Library including whatever changes were used in the work (which must be distributed under Sections 1 and 2 above); and, if the work is an executable linked with the Library, with the complete machine-readable "work that uses the Library", as object code and/or source code, so that the user can modify the Library and then relink to produce a modified executable containing the modified Library. (It is understood that the user who changes the contents of definitions files in the Library will not necessarily be able to recompile the application to use the modified definitions.)

b) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (1) uses at run time a

copy of the library already present on the user's computer system, rather than copying library functions into the executable, and (2) will operate properly with a modified version of the library, if the user installs one, as long as the modified version is interface-compatible with the version that the work was made with.

c) Accompany the work with a written offer, valid for at least three years, to give the same user the materials specified in Subsection 6a, above, for a charge no more than the cost of performing this distribution.

d) If distribution of the work is made by offering access to copy from a designated place, offer equivalent access to copy the above specified materials from the same place.

e) Verify that the user has already received a copy of these materials or that you have already sent this user a copy.

For an executable, the required form of the "work that uses the Library" must include any data and utility programs needed for reproducing the executable from it. However, as a special exception, the materials to be distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating system on which the executable runs, unless that component itself accompanies the executable.

It may happen that this requirement contradicts the license restrictions of other proprietary libraries that do not normally accompany the operating system. Such a contradiction means you cannot use both them and the Library together in an executable that you distribute.

7. You may place library facilities that are a work based on the Library side-by-side in a single library together with other library facilities not covered by this License, and distribute such a combined library, provided that the separate distribution of the work based on the Library and of the other library facilities is otherwise permitted, and provided that you do these two things:

a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities. This must be distributed under the terms of the Sections above.

b) Give prominent notice with the combined library of the fact that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

8. You may not copy, modify, sublicense, link with, or distribute the Library except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, link with, or distribute the Library is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance.

9. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Library or its derivative works. These actions are prohibited by law if you do not accept this License. Therefore, by modifying or distributing the Library (or any work based on the Library), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Library or works based on it.

10. Each time you redistribute the Library (or any work based on the Library), the recipient automatically receives a license from the original licensor to copy, distribute, link with or modify the Library subject to these terms and conditions. You may not impose any further restrictions on the recipients' exercise of the rights granted herein. You are not responsible for enforcing compliance by third parties with this License.

11. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Library at all. For example, if a patent license would not permit royalty-free redistribution of the Library by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Library.

If any portion of this section is held invalid or unenforceable under any particular circumstance, the balance of the section is intended to apply, and the section as a whole is intended to apply in other circumstances.

It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice.

This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License.

12. If the distribution and/or use of the Library is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Library under this License may add an explicit geographical distribution limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License.

13. The Free Software Foundation may publish revised and/or new versions of the Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Library does not specify a license version number, you may choose any version ever published by the Free Software Foundation.

14. If you wish to incorporate parts of the Library into other free programs whose distribution conditions are incompatible with these, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free status of all derivatives of our free software and of promoting the sharing and reuse of software generally.

NO WARRANTY

15. BECAUSE THE LIBRARY IS LICENSED FREE OF CHARGE, THERE IS NO WARRANTY FOR THE LIBRARY, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE LIBRARY "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE LIBRARY IS WITH YOU. SHOULD THE LIBRARY PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE LIBRARY AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE LIBRARY (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE LIBRARY TO OPERATE WITH ANY OTHER SOFTWARE), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Libraries

If you develop a new library, and you want it to be of the greatest possible use to the public, we recommend making it free software that everyone can redistribute and change. You can do so by permitting redistribution under these terms (or, alternatively, under the terms of the ordinary General Public License).

To apply these terms, attach the following notices to the library. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
\<one line to give the library's name and a brief idea of what it does.\>
Copyright (C) \<year\> \<name of author\>
```

```
This library is free software; you can redistribute it and/or
modify it under the terms of the GNU Lesser General Public
License as published by the Free Software Foundation; either
version 2.1 of the License, or (at your option) any later version.
```

```
This library is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU
Lesser General Public License for more details.
```

```
You should have received a copy of the GNU Lesser General Public
License along with this library; if not, write to the Free Software
Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA
```

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the library, if necessary. Here is a sample; alter the names:

```
Yoyodyne, Inc., hereby disclaims all copyright interest in the
library 'Frob' (a library for tweaking knobs) written by James Random Hacker.
```

```
\<signature of Ty Coon\>, 1 April 1990
Ty Coon, President of Vice
```

That's all there is to it!

GNU LESSER GENERAL PUBLIC LICENSE Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<http://fsf.org/>>
Everyone is permitted to copy and distribute verbatim copies
of this license document, but changing it is not allowed.

This version of the GNU Lesser General Public License incorporates the terms and conditions of version 3 of the GNU General Public License, supplemented by the additional permissions listed below.

0. Additional Definitions.

As used herein, "this License" refers to version 3 of the GNU Lesser General Public License, and the "GNU GPL" refers to version 3 of the GNU General Public License.

"The Library" refers to a covered work governed by this License, other than an Application or a Combined Work as defined below.

An "Application" is any work that makes use of an interface provided by the Library, but which is not otherwise based on the Library. Defining a subclass of a class defined by the Library is deemed a mode of using an interface provided by the Library.

A "Combined Work" is a work produced by combining or linking an Application with the Library. The particular version of the Library with which the Combined Work was made is also called the "Linked Version".

The "Minimal Corresponding Source" for a Combined Work means the Corresponding Source for the Combined Work, excluding any source code for portions of the Combined Work that, considered in isolation, are based on the Application, and not on the Linked Version.

The "Corresponding Application Code" for a Combined Work means the object code and/or source code for the Application, including any data and utility programs needed for reproducing the Combined Work from the Application, but excluding the System Libraries of the Combined Work.

1. Exception to Section 3 of the GNU GPL.

You may convey a covered work under sections 3 and 4 of this License without being bound by section 3 of the GNU GPL.

2. Conveying Modified Versions.

If you modify a copy of the Library, and, in your modifications, a facility refers to a function or data to be supplied by an Application that uses the facility (other than as an argument passed when the facility is invoked), then you may convey a copy of the modified version:

- a) under this License, provided that you make a good faith effort to ensure that, in the event an Application does not supply the function or data, the facility still operates, and performs whatever part of its purpose remains meaningful, or
- b) under the GNU GPL, with none of the additional permissions of this License applicable to that copy.

3. Object Code Incorporating Material from Library Header Files.

The object code form of an Application may incorporate material from a header file that is part of the Library. You may convey such object code under terms of your choice, provided that, if the incorporated material is not limited to numerical parameters, data structure layouts and accessors, or small macros, inline functions and templates (ten or fewer lines in length), you do both of the following:

- a) Give prominent notice with each copy of the object code that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the object code with a copy of the GNU GPL and this license document.

4. Combined Works.

You may convey a Combined Work under terms of your choice that, taken together, effectively do not restrict modification of the portions of the Library contained in the Combined Work and reverse engineering for debugging such modifications, if you also do each of the following:

- a) Give prominent notice with each copy of the Combined Work that the Library is used in it and that the Library and its use are covered by this License.
- b) Accompany the Combined Work with a copy of the GNU GPL and this license document.
- c) For a Combined Work that displays copyright notices during execution, include the copyright notice for the Library among these notices, as well as a reference directing the user to the copies of the GNU GPL and this license document.
- d) Do one of the following:
 - 0) Convey the Minimal Corresponding Source under the terms of this License, and the Corresponding Application Code in a form suitable for, and under terms that permit, the user to recombine or relink the Application with a modified version of the Linked Version to produce a modified Combined Work, in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.
 - 1) Use a suitable shared library mechanism for linking with the Library. A suitable mechanism is one that (a) uses at run time a copy of the Library already present on the user's computer system, and (b) will operate properly with a modified version of the Library that is interface-compatible with the Linked Version.
- e) Provide Installation Information, but only if you would otherwise be required to provide such information under section 6 of the GNU GPL, and only to the extent that such information is necessary to install and execute a modified version of the Combined Work produced by recombining or relinking the Application with a modified version of the Linked Version. (If you use option 4d0, the Installation Information must accompany the Minimal Corresponding Source and Corresponding Application Code. If you use option 4d1, you must provide the Installation Information in the manner specified by section 6 of the GNU GPL for conveying Corresponding Source.)

5. Combined Libraries.

You may place library facilities that are a work based on the Library side by side in a single library together with other library facilities that are not Applications and are not covered by this License, and convey such a combined library under terms of your choice, if you do both of the following:

- a) Accompany the combined library with a copy of the same work based on the Library, uncombined with any other library facilities, conveyed under the terms of this License.
- b) Give prominent notice with the combined library that part of it is a work based on the Library, and explaining where to find the accompanying uncombined form of the same work.

6. Revised Versions of the GNU Lesser General Public License.

The Free Software Foundation may publish revised and/or new versions of the GNU Lesser General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Library as you received it specifies that a certain numbered version of the GNU Lesser General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that published version or of any later version published by the Free Software Foundation. If the Library as you received it does not specify a version number of the GNU Lesser General Public License, you may choose any version of the GNU Lesser General Public License ever published by the Free Software Foundation.

If the Library as you received it specifies that a proxy can decide whether future versions of the GNU Lesser General Public License shall apply, that proxy's public statement of acceptance of any version is permanent authorization for you to choose that version for the Library.

GNU GENERAL PUBLIC LICENSE
Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <<https://fsf.org/>>
Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program--to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to

avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

TERMS AND CONDITIONS

0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to

control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.
- c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain

clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.

e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of

it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a

publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software

Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>
```

```
This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.
```

```
This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.
```

```
You should have received a copy of the GNU General Public License
```

along with this program. If not, see [<https://www.gnu.org/licenses/>](https://www.gnu.org/licenses/).

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author>
This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see [<https://www.gnu.org/licenses/>](https://www.gnu.org/licenses/).

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read [<https://www.gnu.org/licenses/why-not-lgpl.html>](https://www.gnu.org/licenses/why-not-lgpl.html).

4.8.2 Bip Buffer

The STAR-System Linux PCI driver (used by the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#)) makes use of the Bip Buffer described in the article "The Bip Buffer - The Circular Buffer with a Twist", authored by Simon Cooke and published on the CodeProject website.

This component is licensed under the terms of the Code Project Open License (CPOL) 1.02.

The Code Project Open License (CPOL) 1.02

Preamble

This License governs Your use of the Work. This License is intended to allow developers to use the Source Code and Executable Files provided as part of the Work in any application in any form.

The main points subject to the terms of the License are:

- Source Code and Executable Files can be used in commercial applications;
- Source Code and Executable Files can be redistributed; and
- Source Code can be modified to create derivative works.
- No claim of suitability, guarantee, or any warranty whatsoever is provided. The software is provided "as-is".
- The Article accompanying the Work may not be distributed or republished without the Author's consent

This License is entered between You, the individual or other entity reading or otherwise making use of the Work licensed pursuant to this License and the individual or other entity which offers the Work under the terms of this License ("Author").

License

THE WORK (AS DEFINED BELOW) IS PROVIDED UNDER THE TERMS OF THIS CODE PROJECT OPEN LICENSE ("LICENSE"). THE WORK IS PROTECTED BY COPYRIGHT AND/OR OTHER APPLICABLE LAW. ANY USE OF THE WORK OTHER THAN AS AUTHORIZED UNDER THIS LICENSE OR COPYRIGHT LAW IS PROHIBITED.

BY EXERCISING ANY RIGHTS TO THE WORK PROVIDED HEREIN, YOU ACCEPT AND AGREE TO BE BOUND BY THE TERMS OF THIS LICENSE. THE AUTHOR GRANTS YOU THE RIGHTS CONTAINED HEREIN IN CONSIDERATION OF YOUR ACCEPTANCE OF SUCH TERMS AND CONDITIONS. IF YOU DO NOT AGREE TO ACCEPT AND BE BOUND BY THE TERMS OF THIS LICENSE, YOU CANNOT MAKE ANY USE OF THE WORK.

1. Definitions.

- a. "Articles" means, collectively, all articles written by Author which describes how the Source Code and Executable Files for the Work may be used by a user.
- b. "Author" means the individual or entity that offers the Work under the terms of this License.
- c. "Derivative Work" means a work based upon the Work or upon the Work and other pre-existing works.
- d. "Executable Files" refer to the executables, binary files, configuration and any required data files included in the Work.
- e. "Publisher" means the provider of the website, magazine, CD-ROM, DVD or other medium from or by which the Work is obtained by You.
- f. "Source Code" refers to the collection of source code and configuration files used to create the Executable Files.
- g. "Standard Version" refers to such a Work if it has not been modified, or has been modified in accordance with the consent of the Author, such consent being in the full discretion of the Author.
- h. "Work" refers to the collection of files distributed by the Publisher, including the Source Code, Executable Files, binaries, data files, documentation, whitepapers and the Articles.
- i. "You" is you, an individual or entity wishing to use the Work and exercise your rights under this License.

2. Fair Use/Fair Use Rights. Nothing in this License is intended to reduce, limit, or restrict any rights arising from fair use, fair dealing, first sale or other limitations on the exclusive rights of the copyright owner under copyright law or other applicable laws.

3. License Grant. Subject to the terms and conditions of this License, the Author hereby grants You a worldwide, royalty-free, non-exclusive, perpetual (for the duration of the applicable copyright) license to exercise the rights in the Work as stated below:

- a. You may use the standard version of the Source Code or Executable Files in Your own applications.
- b. You may apply bug fixes, portability fixes and other modifications obtained from the Public Domain or from the Author. A Work modified in such a way shall still be considered the standard version and will be subject to this License.
- c. You may otherwise modify Your copy of this Work (excluding the Articles) in any way to create a Derivative Work, provided that You insert a prominent notice in each changed file stating how, when and where You changed that file.
- d. You may distribute the standard version of the Executable Files and Source Code or Derivative Work in aggregate with other (possibly commercial) programs as part of a larger (possibly commercial) software distribution.
- e. The Articles discussing the Work published in any form by the author may not be distributed or republished without the Author's consent. The author retains copyright to any such Articles. You may use the Executable Files and Source Code pursuant to this License but you may not repost or republish or otherwise distribute or make available the Articles, without the prior written consent of the Author.

Any subroutines or modules supplied by You and linked into the Source Code or Executable Files of this Work shall not be considered part of this Work and will not be subject to the terms of this License.

4. Patent License. Subject to the terms and conditions of this License, each Author hereby grants to You a perpetual, worldwide, non-exclusive, no-charge, royalty-free, irrevocable (except as stated in this section) patent license to make, have made, use, import, and otherwise transfer the Work.

5. Restrictions. The license granted in Section 3 above is expressly made subject to and limited by the following restrictions:

- a. You agree not to remove any of the original copyright, patent, trademark, and attribution notices and associated disclaimers that may appear in

- the Source Code or Executable Files.
- b. You agree not to advertise or in any way imply that this Work is a product of Your own.
 - c. The name of the Author may not be used to endorse or promote products derived from the Work without the prior written consent of the Author.
 - d. You agree not to sell, lease, or rent any part of the Work. This does not restrict you from including the Work or any part of the Work inside a larger software distribution that itself is being sold. The Work by itself, though, cannot be sold, leased or rented.
 - e. You may distribute the Executable Files and Source Code only under the terms of this License, and You must include a copy of, or the Uniform Resource Identifier for, this License with every copy of the Executable Files or Source Code You distribute and ensure that anyone receiving such Executable Files and Source Code agrees that the terms of this License apply to such Executable Files and/or Source Code. You may not offer or impose any terms on the Work that alter or restrict the terms of this License or the recipients' exercise of the rights granted hereunder. You may not sublicense the Work. You must keep intact all notices that refer to this License and to the disclaimer of warranties. You may not distribute the Executable Files or Source Code with any technological measures that control access or use of the Work in a manner inconsistent with the terms of this License.
 - f. You agree not to use the Work for illegal, immoral or improper purposes, or on pages containing illegal, immoral or improper material. The Work is subject to applicable export laws. You agree to comply with all such laws and regulations that may apply to the Work after Your receipt of the Work.
6. Representations, Warranties and Disclaimer. THIS WORK IS PROVIDED "AS IS", "WHERE IS" AND "AS AVAILABLE", WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES OR CONDITIONS OR GUARANTEES. YOU, THE USER, ASSUME ALL RISK IN ITS USE, INCLUDING COPYRIGHT INFRINGEMENT, PATENT INFRINGEMENT, SUITABILITY, ETC. AUTHOR EXPRESSLY DISCLAIMS ALL EXPRESS, IMPLIED OR STATUTORY WARRANTIES OR CONDITIONS, INCLUDING WITHOUT LIMITATION, WARRANTIES OR CONDITIONS OF MERCHANTABILITY, MERCHANTABLE QUALITY OR FITNESS FOR A PARTICULAR PURPOSE, OR ANY WARRANTY OF TITLE OR NON-INFRINGEMENT, OR THAT THE WORK (OR ANY PORTION THEREOF) IS CORRECT, USEFUL, BUG-FREE OR FREE OF VIRUSES. YOU MUST PASS THIS DISCLAIMER ON WHENEVER YOU DISTRIBUTE THE WORK OR DERIVATIVE WORKS.
7. Indemnity. You agree to defend, indemnify and hold harmless the Author and the Publisher from and against any claims, suits, losses, damages, liabilities, costs, and expenses (including reasonable legal or attorneys' fees) resulting from or relating to any use of the Work by You.
8. Limitation on Liability. EXCEPT TO THE EXTENT REQUIRED BY APPLICABLE LAW, IN NO EVENT WILL THE AUTHOR OR THE PUBLISHER BE LIABLE TO YOU ON ANY LEGAL THEORY FOR ANY SPECIAL, INCIDENTAL, CONSEQUENTIAL, PUNITIVE OR EXEMPLARY DAMAGES ARISING OUT OF THIS LICENSE OR THE USE OF THE WORK OR OTHERWISE, EVEN IF THE AUTHOR OR THE PUBLISHER HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.
9. Termination.
- a. This License and the rights granted hereunder will terminate automatically upon any breach by You of any term of this License. Individuals or entities who have received Derivative Works from You under this License, however, will not have their licenses terminated provided such individuals or entities remain in full compliance with those licenses. Sections 1, 2, 6, 7, 8, 9, 10 and 11 will survive any termination of this License.
 - b. If You bring a copyright, trademark, patent or any other infringement claim against any contributor over infringements You claim are made by the Work, your License from such contributor to the Work ends automatically.
 - c. Subject to the above terms and conditions, this License is perpetual (for the duration of the applicable copyright in the Work). Notwithstanding the above, the Author reserves the right to release the Work under different license terms or to stop distributing the Work at any time; provided, however that any such election will not serve to withdraw this License (or any other license that has been, or is required to be, granted under the terms of this License), and this License will continue in full force and effect unless terminated as stated above.

10. Publisher. The parties hereby confirm that the Publisher shall not, under any circumstances, be responsible for and shall not have any liability in respect of the subject matter of this License. The Publisher makes no warranty whatsoever in connection with the Work and shall not be liable to You or any party on any legal theory for any damages whatsoever, including without limitation any general, special, incidental or consequential damages arising in connection to this license. The Publisher reserves the right to cease making the Work available to You at any time without notice
11. Miscellaneous
 - a. This License shall be governed by the laws of the location of the head office of the Author or if the Author is an individual, the laws of location of the principal place of residence of the Author.
 - b. If any provision of this License is invalid or unenforceable under applicable law, it shall not affect the validity or enforceability of the remainder of the terms of this License, and without further action by the parties to this License, such provision shall be reformed to the minimum extent necessary to make such provision valid and enforceable.
 - c. No term or provision of this License shall be deemed waived and no breach consented to unless such waiver or consent shall be in writing and signed by the party to be charged with such waiver or consent.
 - d. This License constitutes the entire agreement between the parties with respect to the Work licensed herein. There are no understandings, agreements or representations with respect to the Work not specified herein. The Author shall not be bound by any additional provisions that may appear in any communication from You. This License may not be modified without the mutual written agreement of the Author and You.

4.9 Change Log

4.9.1 STAR-System v6.01 - 13th November 2025

• New Features

- Added a new and improved Triggering GUI application:
 - * Improved the visuals and usability throughout.
 - * Added the ability to save and load configurations to and from files.
 - * Added multiple example configurations for common use cases.
 - * Added a "Guided Tour" function to provide a tutorial for the application.
- Generic Triggering API:
 - * Added support for SpaceFibre ports and virtual channels.
 - * Added support for the [STAR-Ultra PCIe Interface](#) device.
 - * Added the `TRIGGER_CFG_getDeviceClockFreq()` function to retrieve the clock frequency used by the triggering system on the device.
 - * Added the following functions to retrieve the number of each resource:
 - `TRIGGER_CFG_getNumberOfExtTriggers()`
 - `TRIGGER_CFG_getNumberOfCounters()`
 - `TRIGGER_CFG_getNumberOfSpWPorts()`
 - `TRIGGER_CFG_getNumberOfTimeCodeInterfaces()`
 - `TRIGGER_CFG_getNumberOfSpFiPorts()`
 - `TRIGGER_CFG_getNumberOfSpFiVCs()`
 - `TRIGGER_CFG_getNumberOfTriggers()`
 - * Added the following functions to make it easier to configure the AND/OR logic and invert state for events:
 - `TRIGGER_CFG_getTriggerSourceEventsAnded()`
 - `TRIGGER_CFG_setTriggerSourceEventsAnded()`
 - `TRIGGER_CFG_getTriggerSourceEventsInverted()`
 - `TRIGGER_CFG_setTriggerSourceEventsInverted()`

- * Added the following functions for configuring SpaceFibre port events and actions:
 - [TRIGGER_CFG_getSpFiPortInputEvents\(\)](#)
 - [TRIGGER_CFG_setSpFiPortInputEvents\(\)](#)
 - [TRIGGER_CFG_getSpFiPortOutputActions\(\)](#)
 - [TRIGGER_CFG_setSpFiPortOutputActions\(\)](#)
- * Added the following functions for configuring SpaceFibre virtual channel events and actions:
 - [TRIGGER_CFG_getSpFiVCInputEvents\(\)](#)
 - [TRIGGER_CFG_setSpFiVCInputEvents\(\)](#)
 - [TRIGGER_CFG_getSpFiVCOOutputActions\(\)](#)
 - [TRIGGER_CFG_setSpFiVCOOutputActions\(\)](#)
- * Added the following functions for configuring SpaceFibre virtual channel resources:
 - [TRIGGER_CFG_enableSpFiVCReceiveDataMode\(\)](#)
 - [TRIGGER_CFG_disableSpFiVCReceiveDataMode\(\)](#)
 - [TRIGGER_CFG_enableSpFiVCTransmitDataMode\(\)](#)
 - [TRIGGER_CFG_disableSpFiVCTransmitDataMode\(\)](#)
 - [TRIGGER_CFG_enableSpFiVCTransmitPacketMode\(\)](#)
 - [TRIGGER_CFG_disableSpFiVCTransmitPacketMode\(\)](#)
- * With the added support for SpaceFibre ports, the existing port functions e.g., [TRIGGER_CFG_setPortInputEvents\(\)](#) have been deprecated in favour of protocol-specific functions e.g., [TRIGGER_CFG_setSpWPortInputEvents\(\)](#).
- * Added improved documentation to the API's front page.
- Added option to the installers to disable installation of Ethernet support.
- Added installation of the End-User License Agreement as well as licenses used by external libraries.
- Added support for revision 2 of the SpaceWire Link Analyser Mk3, SpaceWire Conformance Tester Mk2, SpaceWire EGSE Mk2 and STAR Fire Mk3.
- Added the ability to save and load the state of the Device Configuration application to and from files.

• Improvements

- Fixed issue that could prevent temporary files used by STAR-System being created on Linux.
- Fixed issue that could result in shared memory files used by the STAR-System Device Configuration Service not being removed after uninstallation on Linux.
- Fixed issue that could result in drivers being partially installed on Linux when using the `--development` command-line installer option.
- Fixed issue that prevented the Linux command-line installer being usable outside of the current working directory.
- Fixed issue that could result in 32-bit and 64-bit libraries being installed incorrectly on recent multiarch Linux distributions.
- Fixed issue in the [CFG_setTxSignallingRate\(\)](#) function that prevented it being used with the [SpaceWire PCIe Mk2](#).
- Fixed issue that could result in the packet format dialogs being slow to load in the Source and Sink applications.
- Fixed issue with the image view's initial size in the Sink application.
- Fixed issue with the [CFG_getTimeCodeDistributionCapablePorts\(\)](#) function returning an incorrect value.
- Updated the file format used when saving and loading the state of the graphical applications to use JSON.
- Various improvements and updates to the application notes.
- Various improvements and updates to the documentation.
- Linux kernel compatibility tested up to v6.17.7.

4.9.2 STAR-System v6.00 - 7th October 2024

• New Features

- SpaceFibre Configuration API
 - * Added the following functions:
 - [CFG_SPFI_getBroadcastTimeoutResolution\(\)](#)
 - [CFG_SPFI_getRouterTimeout\(\)](#)
 - [CFG_SPFI_setRouterTimeout\(\)](#)
 - [CFG_SPFI_getRouterTimeoutMaximum\(\)](#)
 - [CFG_SPFI_getRouterTimeoutResolution\(\)](#)
 - [CFG_SPFI_getVCDataRate\(\)](#)
 - [CFG_SPFI_getAXIBC\(\)](#)
 - [CFG_SPFI_setAXIBC\(\)](#)
 - * Added support for AXI ports to the following functions:
 - [CFG_SPFI_getVCVN\(\)](#)
 - [CFG_SPFI_setVCVN\(\)](#)
 - [CFG_SPFI_getVCStatus\(\)](#)
 - [CFG_SPFI_getVCErrors\(\)](#)
 - [CFG_SPFI_clearVCErrors\(\)](#)
 - [CFG_SPFI_getVCQualityOfService\(\)](#)
 - [CFG_SPFI_setVCBandwidthReservation\(\)](#)
 - [CFG_SPFI_setVCPriority\(\)](#)
 - [CFG_SPFI_enableVCContinuousMode\(\)](#)
 - [CFG_SPFI_disableVCContinuousMode\(\)](#)
 - [CFG_SPFI_getVCTimeSlots\(\)](#)
 - [CFG_SPFI_setVCTimeSlots\(\)](#)
 - * Renamed [CFG_SPFI_isLaneINIT1Detected\(\)](#) and [CFG_SPFI_isLaneINIT2Detected\(\)](#) to [CFG_SPFI_isLaneINIT1RxDetected\(\)](#) and [CFG_SPFI_isLaneINIT2RxDetected\(\)](#)
 - * Renamed [STAR_DEVICE_STAR_ULTRA_SL_ROUTER](#) constant to [STAR_DEVICE_STAR_ULTRA_PCIE_SL_ROUTER](#).
 - * Improved validation of [CFG_SPFI_setAXIBC\(\)](#) for STAR-Ultra PCIe Single-Lane Router devices, supporting AXI BC values from 0 to 63.
 - * Added example project.
- General
 - * Added the following functions:
 - [STAR_getTimeCodeCount\(\)](#)
 - [CFG_getTimeCodeDistributionCapablePorts\(\)](#)
 - * Deprecated [CFG_getMeasuredLinkSpeed\(\)](#), [CFG_getTransmitSignallingRate\(\)](#) and [CFG_setTransmitSignallingRate\(\)](#) and replaced with [CFG_getRxSignallingRate\(\)](#), [CFG_getTxSignallingRate\(\)](#) and [CFG_setTxSignallingRate\(\)](#), allowing more precision on supported devices.
 - * Following a kernel update, the drivers can now be rebuilt on Linux using the [build_kernel_modules.sh](#) script as described in [Installation Instructions for Linux](#).

• Improvements

- Fixed bug with error reporting in the [CUBA Software](#).
- Fixed bug in GUI applications that meant that [STAR-Ultra PCIe Interface](#) version information couldn't be displayed.
- Fixed bug in Time-code example which was setting invalid value for [CFG_setTimeCodeDistributionCapablePorts\(\)](#).
- Fixed bug in [CFG_SPFI_setLinkLanesRequestedCount\(\)](#) which did not support a value of 0 for link lanes requested.

- Fixed compilation issue in Broadcast Message example when using older compilers.
- Fixed [SpaceWire Router Mk2S](#) compatibility issue with [CFG_setTimeCodeDistributionPorts\(\)](#).
- Fixed [STAR Fire Mk2](#) compatibility issue with [CFG_setPortRoutingAddress\(\)](#).
- Fixed bug in the [Triggering API](#) and [Generic Triggering API](#) to prevent counter start and start/stop modes from being enabled at the same time.
- Fixed bug in the [Triggering API](#) and [Generic Triggering API](#) reset functions to correct the default counter reload value and external trigger edge detect mode.
- Fixed issue in the [CCSDS/SPP/TF Packet Library](#) to resolve linking issues on Windows and in C++ applications.
- Fixed bug in the [TRIGGER_CFG_getDeviceClockRate\(\)](#) function to correct the return value if an error occurs.
- Improved the Broadcast Message example to support type 0x20 and status 0x00.
- Deprecated [STAR_RECEIVE_NULL](#) constant and replaced it with [STAR_RECEIVE_NULLS](#).
- Deprecated [STAR_DEVICE_STAR_ULTRA_PCIE](#) and replaced it with [STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE](#).
- Visual Studio examples now use a consistent vcxproj file format.
- Corrected device compatibility for the following functions:
 - * [CFG_disableExternalTimeCodeSelection\(\)](#)
 - * [CFG_enableExternalTimeCodeSelection\(\)](#)
 - * [CFG_getExternalTimeCodeSelectionEnabled\(\)](#)
 - * [CFG_MK2_disableExternalTimeCodeSelection\(\)](#)
 - * [CFG_MK2_enableExternalTimeCodeSelection\(\)](#)
 - * [CFG_MK2_getExternalTimeCodeSelectionEnabled\(\)](#)
- Various improvements to the Linux installer.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v6.11.2.

4.9.3 STAR-System v6.00 Beta 3 - 5th October 2023

• New Features

- Added Broadcast Message example.
- Added support for configuring [SpaceWire PCIe Mk2](#) link speed in decimal to the [Device Configuration Application](#) and API functions:
 - * [CFG_PCIE_MK2_setDecimalTxSignallingRate\(\)](#)
 - * [CFG_PCIE_MK2_getDecimalTxSignallingRate\(\)](#)
- Various improvements to the [SpaceFibre Configuration API](#) and documentation.

• Improvements

- Fixed potential issue when hot-plugging STAR-Ultra devices connected using Thunderbolt.
- Fixed issue that could cause incorrect permissions to be set for device files on Linux during installation.

4.9.4 STAR-System v6.00 Beta 2 - 29th August 2023

• Improvements

- Fixed issue preventing USB firmware updates with revision 2 of the [SpaceWire Brick Mk4](#).
- Fixed issue causing a crash in the Python API when receiving packets with no data.
- Fixed issue in the Linux USB driver that could cause configuration functions to fail if called immediately after a device reset of [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices.
- Fixed missing STAR-Ultra driver type value in the [STAR_DRIVER_TYPE](#) enum.
- Various improvements to the [SpaceFibre Configuration API](#) and documentation.

4.9.5 STAR-System v6.00 Beta 1 - 6th July 2023

- **New Features**

- Added support for the [STAR-Ultra PCIe Single-Lane Router](#).
- Added a [SpaceFibre Configuration API](#).
- The state of many of the GUI applications can now be saved and loaded.

- **Improvements**

- Fixed issue where the port selector in the [Error Injection Application](#) was not populated correctly.
- Fixed issue where channel 0 could not be used for remote device configuration in the [Error Injection Application](#).
- Fixed issue in the [Triggering Application](#) which prevented fields being populated for the first device attached.
- Fixed issue in the [STAR-Ultra PCIe Interface](#), [STAR-Ultra PCIe Single-Lane Router](#), and [SpaceWire PCIe Mk2](#) Linux driver related to a missing kernel function in kernels prior to v3.12.27.
- Fixed issue in the [STAR-Ultra PCIe Interface](#), [STAR-Ultra PCIe Single-Lane Router](#), and [SpaceWire PCIe Mk2](#) drivers which caused version information to be returned as zero.
- Linux kernel compatibility tested up to v6.4.

4.9.6 STAR-System v5.04 - 20th February 2023

- **New Features**

- Added the [STAR_rxOperationExGetTimestampEvent\(\)](#) function to support timestamp events in ex receive operations.
- Added support for revision 2 of the SpaceWire Recorder Mk2.

- **Improvements**

- Added possibility to work with the contents of transfer operations in completion callbacks using the Python API.
- The [RMAP Initiator Application](#) now checks the EOP type in RMAP replies, and reports errors when EEPs are received.
- Fixed bug that caused some documentation to be missing in the [Graphical Applications](#) section.
- Fixed bug that caused the [Sink Application](#) to crash when using the field recording option.
- Linux kernel compatibility tested up to v6.1.12.

4.9.7 STAR-System v5.03 - 18th January 2023

- **New Features**

- In each application, the devices present in the device selector combo box are now being ordered and displayed in alphanumerical order.
- Using the Device Update application, one and only one update process can be run for a device at any given time.
- Added support for revision 2 of the SpaceWire Brick Mk4 and SpaceWire PXI Mk2 boards.

- **Improvements**

- Fixed bug with nested loops in CUBA software.
- Fixed bug when CRC used with webcam data in Source and Sink.
- Fixed bug in Time-codes application where flag state information was not reset when device was disconnected.

- Fixed bug in Source application that did not allow for the transmission of data at very high data rates.
- Fixed incompatibility between random number algorithm on Windows and Linux in Source and Sink.
- The `CFG_updateDeviceInfo()` configuration function is now deprecated, the newly added `CFG_refreshDeviceInfo()` should be used instead.
- Various improvements to the documentation and examples.
- Linux kernel compatibility tested up to v6.1.6.

4.9.8 STAR-System v5.02 - 25th November 2022

• New Features

- Added [Generic Triggering API](#).
- Added the following functions to get the clock rate used by the triggering system of each supported device type:
 - * `TRIGGER_BRICK_MK3_getDeviceClockRate()`
 - * `TRIGGER_PCIE_IF_getDeviceClockRate()`
 - * `TRIGGER_PCIE_MK2_getDeviceClockRate()`
 - * `TRIGGER_PXI_IF_getDeviceClockRate()`
 - * `TRIGGER_PXI_ROUTER_getDeviceClockRate()`
- Added C++ examples that demonstrate the use of the Generic Triggering API.
- Instead of the frequency, the period between consecutive time-codes (in microseconds) is now being specified in the Time-code GUI for the time-code master.
- Added support for the PXI with RMAP Target to the Python API.
- Added support for the PCIe MK2, PXI RMAP and PXI RMAP MK2 devices to the triggering functions in the Python API.
- It is now possible to pass in any kind of picklable Python object as the "context" argument (for the callback functions) for the following classes: `ChannelListener`, `DeviceListener`, `DriverListener` and `TransferCompletionListener` and retrieve them in the callback functions, when they execute.

• Improvements

- Fixed bug in the [STAR-Ultra PCIe Interface](#) and [SpaceWire PCIe Mk2](#) driver that caused errors in Linux debug kernels due to an invalid lock.
- Fixed bug in RMAP Target GUI application that caused incorrect values after saving and refreshing.
- Fixed bug that prevented the `CFG_getTimeCodeFlagMode()` and `CFG_setTimeCodeFlagMode()` functions being used with the [SpaceWire Router Mk2S](#).
- Fixed bug that prevented the test iteration being updated when running tests in the STAR-System Test application on Linux.
- Fixed bug that prevented the SpaceWire PXI Mk2 boards from working correctly with the [Generic Configuration API](#) on Linux.
- Fixed bug in the Python API that caused a mismatch in port numbers for the ports that have been specified in both the `setTimeCodeDistributionPorts` and `getTimeCodeDistributionPorts` functions.
- Fixed bug in the Python API that caused the `setPrecisionTransmitRate` function to fail.
- Linux kernel compatibility tested up to v6.0.9.

4.9.9 STAR-System v5.01 - 16th September 2022

• New Features

- Added support for [SpaceWire PCIe Mk2](#) devices.
- GUI applications now identify the device in the title bar.

- GUI applications now have an About menu dialog that displays information about the currently selected device as well as the current STAR-System version.
- GUI applications now offer the possibility (where applicable) to use software channel 0 for communicating with the selected device.
- Added Append option to the RMAP Initiator application, allowing output from sequential read transactions to be viewed simultaneously.
- Added timed test type to the Performance Tester.

- **Improvements**

- Fixed backward compatibility issue for earlier versions of the GbE Brick Mk1 configuration library on Linux.
- Fixed bug where Initial Value field in Data Pattern did not persist in the Source and Sink applications.
- Fixed bug in the Sink application that could cause memory to grow when checking packet format containing CRC type field(s).
- Fixed bug that could cause errors if configuration API functions were called immediately after a device reset for the Brick Mk3/Mk4.
- Fixed bug in [CCSDS/SPP/TF Packet Library](#) that prevented maximum possible value from being set for the sequence count.
- Fixed bug in [CCSDS/SPP/TF Packet Library](#) that prevented the possibility of the secondary header from not being present in case the CCSDS packet type was specified as telemetry/telecommand.
- Fixed issue with out of sequence data in Sink when only receiving one packet.
- Fixed bug in the Triggering GUI that caused the SpaceWire PXI Interface Mk2 with RMAP target to show up as not supported.
- Fixed bug in the Triggering GUI that caused some trigger output actions to be configurable for devices that don't support them.
- Fixed bug in Sink that caused extra file to be created when recording packets to individual files.
- Fixed bug that could cause uncommitted changes to a routing table to be inadvertently lost when saving.
- Fixed issue with CPU usage calculations in the Performance Tester.
- The PYTHONPATH environment variable is now automatically populated during installation.
- Added more precise timing between packets in the Source application.
- Added Seed option for random data patterns in Source and Sink.
- Various improvements to Linux installer.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v5.19.9.

4.9.10 STAR-System v5.00 - 17th February 2022

- Fixed bug in [CUBA Software](#) that prevented F command from working in SCS mode.
- Fixed bug in [CUBA Software](#) that could stop the comment end character '/' from being processed correctly.
- Fixed bug that could cause hang in STAR-System for LabVIEW when Initialise.vi was called repeatedly.
- Fixed error in RMAP_GET_VERSION* macros in [rmap_packet_library.h](#).
- Fixed linking issues when --as-needed GCC flag was used.
- Fixed issue preventing MSI-X interrupts being initialised in the [STAR-Ultra PCIe Interface](#) Linux driver on old kernels.
- Added --development flag to Linux installer that installs files required for development only.
- Added support for ARMv6, ARMv7 and ARMv8 Linux platforms.

- Added Python API.
- Added compile-time option to enable a low-latency IRQ handling mode in the PCI driver on Linux.
- Various improvements to the documentation and examples.
- Linux kernel compatibility tested up to v5.16.9.

4.9.11 STAR-System v5.00 Beta 3 - 12th November 2021

- Added text file RMAP reply data sink option to [RMAP Initiator Application](#).
- Fixed bug in Brick Mk2 configuration example.
- Fixed an issue where unnecessary dynamic memory allocation (for [CCSDS_SPP_PACKET](#) structures) would occur within the [CCSDS/SPP/TF Packet Library](#).
- Fixed a bug that prevented the [Generic Configuration API](#) to be usable on Linux OS.
- Linux kernel compatibility tested up to v5.15.1.
- Tested on Windows 11.

4.9.12 STAR-System v5.00 Beta 2 - 30th September 2021

- Added support for [SpaceWire GbE Brick Mk1](#) devices on Linux.
- Added Eclipse projects for examples.
- Added [RMAP Initiator Application](#).
- Added [Triggering Application](#).
- Added statistics graph to the [Source Application](#) and the [Sink Application](#).
- Added possibility to transmit/receive data present in multiple files in a folder using the [Source Application](#) and [Sink Application](#).
- Added the following functions:
 - [STAR_getDeviceIPAddress\(\)](#)
 - [STAR_getIPAddresses\(\)](#)
 - [STAR_destroyIPAddresses\(\)](#)
- Added Ethernet device example program.
- Added alternative version of the Device Configuration Service that runs in Windows Session 0.
- Fixed bug that prevented the [Sink Application](#) from saving files greater than 4 Gigabytes.
- Fixed file rotation issue when saving data from the [Sink Application](#).
- Fixed issue that prevented --serial argument from working in GUI applications.
- Fixed bug where TFEP packets were not being created correctly by the [CCSDS/SPP/TF Packet Library](#).
- Fixed build issue with time-code C example.
- Fixed bug that prevented device name from being set on Brick Mk4.
- Fixed bug that could allow the same channel to be opened more than once on Linux.
- Fixed bug when using ex receive operations that may result in an invalid number of items if the operation is cancelled while an item is being received.

- Fixed bug when using ex receive operations that may cause a crash if an operation is cancelled immediately after being submitted.
- Fixed bug in [Device Configuration Application](#) where routing table did not accept all hex characters.
- Fixed bug in [STAR_getDeviceConfigCapabilities\(\)](#) that returned the wrong status for [SpaceWire Recorder](#) and [SpaceWire Recorder Mk2](#) devices.
- Fixed bug in [Device Configuration Application](#) that caused error to be shown for [SpaceWire Recorder](#) and [SpaceWire Recorder Mk2](#) devices.
- Improvements to the GUI applications on high DPI screens.
- Various improvements to the Linux installer.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v5.14.8.

4.9.13 STAR-System v5.00 Beta 1 - 24th March 2021

- Added support for [SpaceWire GbE Brick Mk1](#) devices on Windows.
- Added [CCSDS/SPP/TF Packet Library](#).
- Added example showing use of alternative receive method using pre-allocated buffers.
- Added ability to copy serial number in [Device Configuration Application](#).
- Added the following functions to the [Triggering API](#) to reset the triggering resources and triggers:
 - [TRIGGER_BRICK_MK3_reset\(\)](#)
 - [TRIGGER_PCIE_IF_reset\(\)](#)
 - [TRIGGER_PXI_IF_reset\(\)](#)
 - [TRIGGER_PXI_ROUTER_reset\(\)](#)
- Added the following functions to support time-codes and link events in ex receive operations:
 - [STAR_rxOperationExGetTimeCode\(\)](#)
 - [STAR_rxOperationExGetLinkStateEvent\(\)](#)
 - [STAR_rxOperationExGetLinkSpeedEvent\(\)](#)
- Fixed bug that meant that some files were not copied to custom install locations on Linux.
- Fixed bug that prevented ex receive operations being used with SpaceWire PCIe and PCI devices.
- Improved the Makefiles of the examples to support compiling them on Linux without installing STAR-System.
- Various improvements to the Windows and Linux installers.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v5.11.8.

4.9.14 STAR-System v4.05 - 29th January 2021

- Fixed problem with Device Configuration Service failing to start on Manjaro Linux.
- Fixed potential memory leak in `STAR_getTransferItemList()`.
- Fixed bug that could prevent Sink from closing properly.
- Fixed issue in Device Configuration application that prevented Time-code Distribution ports from being set on PCIe, PCI Mk2 and SPLT devices.
- Corrected device compatibility check for `CFG_getTimeCodeFlagMode()` and `CFG_setTimeCodeFlagMode()`.
- Generic configuration API now checks device Tri-state compatibility.
- Added Brick Mk3/Mk4 spill option to Triggering API.
- Reduced delay when closing channels in the Linux PCI driver.
- Improved latency when receiving packets with the STAR-Ultra PCIe Interface.
- Various improvements to the Linux installer.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v5.10.11.

4.9.15 STAR-System v4.04 - 11th September 2020

- Fixed bug that prevented Error Injection application from working with the Brick Mk4.
- Fixed bug introduced in v4.02 that could cause memory usage to grow when using configuration operations.
- Fixed bug that could prevent Device Configuration Service from accessing devices.
- Added support for STAR-Ultra PCIe Interface devices on Linux.
- Added Brick Mk4 and PXI Mk2 support to Triggering API example.
- Various improvements to the Linux installer.
- Various improvements to the documentation.
- Linux kernel compatibility tested up to v5.8.8.

4.9.16 STAR-System v4.03 - 28th July 2020

- Added support for Brick Mk4 and Recorder Mk2 devices.
- Added support for STAR-Ultra PCIe Interface in Device Update application.
- Fixed bug that prevented newly connected devices from being detected by the GUI applications on some versions of Linux.
- Fixed bug that meant that devices were not detected if already connected prior to installation on some versions of Linux.
- Improvements to Performance Tester example code.
- Improvements to Linux installer error handling and kernel support.
- Linux kernel compatibility tested up to v5.7.10.
- Various improvements to the documentation and internals.

4.9.17 STAR-System v4.02 - 13th March 2020

- Fixed bug in STAR-System Test that caused incorrect link speed calculation for receive tests.
- Added support for STAR-Ultra PCIe Interface devices.
- Linux kernel compatibility tested up to v5.5.8.
- Various performance improvements.

4.9.18 STAR-System v4.01 - 29th November 2019

- Fixed bug in STAR-System Test and Time-code example programs due to incorrect error checking.
- Added Makefile and Visual Studio projects for device configuration examples.
- Linux kernel compatibility tested up to v5.4.
- Added port information to link state and speed change event structs, and also API functions [STAR_getLink↔StateEventPort\(\)](#) and [STAR_getLinkSpeedEventPort\(\)](#) to access it.

4.9.19 STAR-System v4.00 - 19th November 2019

- Added [Generic Configuration API](#).
- Added the following functions to support SpaceFibre broadcast message stream items:
 - [STAR_getBroadcastMessageChannel\(\)](#)
 - [STAR_getBroadcastMessageDataWord1\(\)](#)
 - [STAR_getBroadcastMessageDataWord2\(\)](#)
 - [STAR_getBroadcastMessageDelayedFlag\(\)](#)
 - [STAR_getBroadcastMessageLateFlag\(\)](#)
 - [STAR_getBroadcastMessageStatus\(\)](#)
 - [STAR_getBroadcastMessageType\(\)](#)
- Added time-code support for PXI cards.
- Added the following functions to support an alternative receive method using pre-allocated buffers:
 - [STAR_openChannelToLocalDeviceEx\(\)](#)
 - [STAR_createRxOperationEx\(\)](#)
 - [STAR_rxOperationExGetNumItems\(\)](#)
 - [STAR_rxOperationExGetItemType\(\)](#)
 - [STAR_rxOperationExGetDataChunk\(\)](#)
 - [STAR_rxOperationExGetBroadcastMessage\(\)](#)
 - [STAR_rxOperationExGetStatus\(\)](#)
- Improvements to Linux console installation script and GUI installer.
- Improved component scaling on high DPI and normal DPI displays.
- Fixed problem in [STAR_openChannelBetweenLocalDevices\(\)](#).
- Fixed bug where Sink application didn't always terminate when closed.
- Fixed bug that prevented Transmit application from transmitting over channel 0.
- Fixed issue that could cause increased CPU usage in the Sink application.

- Fixed bug that caused an error in the Performance Tester when log-like pattern was specified.
- Fixed bug in Time-codes GUI application where master frequency was not loaded correctly for the Brick Mk3.
- Documented installation steps for Linux installations with secure boot enabled.
- Documented PXI timestamping features.
- Linux kernel compatibility tested up to v5.3.9.
- Various performance and documentation improvements.
- Removed several functions from header files that were previously deprecated:
 - `CFG_BRICK_MK2_getMeasuredLinkSpeed()`
replaced by `CFG_getMeasuredLinkSpeed()`
 - `CFG_PCIMK2_getHardwareInfo()`
replaced by `CFG_getFPGAInfo()`
 - `CFG_PCIMK2_hardwareInfoToString()`
replaced by `CFG_FPGAInfoToString()`
 - `CFG_PCIMK2_enableTimeCodeMaster()`
replaced by `CFG_enableTimeCodeMaster()`
 - `CFG_PCIMK2_disableTimeCodeMaster()`
replaced by `CFG_disableTimeCodeMaster()`
 - `CFG_PCIMK2_setTimeCodePeriod()`
replaced by `CFG_setTimeCodePeriod()`
 - `CFG_PCIMK2_getTimeCodePeriod()`
replaced by `CFG_getTimeCodePeriod()`
 - `CFG_PCIMK2_identify()`
replaced by `CFG_identify()`
 - `CFG_PCIMK2_enableInterfaceMode()`
replaced by `CFG_enableInterfaceMode()`
 - `CFG_PCIMK2_enableInterfaceModeOnPort()`
replaced by `CFG_enableInterfaceModeOnPort()`
 - `CFG_PCIMK2_disableInterfaceMode()`
replaced by `CFG_disableInterfaceMode()`
 - `CFG_PCIMK2_disableInterfaceModeOnPort()`
replaced by `CFG_disableInterfaceModeOnPort()`
 - `CFG_PCIMK2_enableIdentifySource()`
replaced by `CFG_enableIdentifySource()`
 - `CFG_PCIMK2_disableIdentifySource()`
replaced by `CFG_disableIdentifySource()`
 - `CFG_PCIMK2_enableIdentifySourceOnPort()`
replaced by `CFG_enableIdentifySourceOnPort()`
 - `CFG_PCIMK2_disableIdentifySourceOnPort()`
replaced by `CFG_disableIdentifySourceOnPort()`
 - `CFG_PCIMK2_setPortRoutingAddress()`
replaced by `CFG_setPortRoutingAddress()`
 - `CFG_PCIMK2_getPortRoutingAddress()`
replaced by `CFG_getPortRoutingAddress()`
 - `CFG_PCIMK2_setLinkRateDivider()`
replaced by `CFG_setTransmitSignallingRate()`
 - `CFG_PCIMK2_getLinkRateDivider()`
replaced by `CFG_getTransmitSignallingRate()`
 - `STAR_CFG_getRemoteDeviceDescriptionName()`
replaced by `STAR_CFG_getRemoteDeviceDescriptionNameString()`

4.9.20 STAR-System v4.00 Beta 4 - 16th October 2019

- Brand new Linux installer allowing feature selection including the ability to install Qt5 versions of the GUI applications, required for new webcam features.
- Changes to Device Update application to improve usability.
- Added support for colour version of SpaceFibre Camera in Sink.
- Added port action to disable LVDS (`PORT_ACTION_DISABLE_LVDS`) to Triggering API.
- Added support for PXI Mk2 cards.
- Added timestamp support for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) cards.
- Fixed potential bug during initialisation of Link Analyser Mk3, STAR Fire Mk3, EGSE Mk2 and Conformance Tester Mk2.
- Fixed possible hang when transmitting high resolution webcam data in Source application.
- Fixed bug in "Select Packet Format" dialog in Sink application where leaving "Check the content of received packets for errors" unchecked would result in an error.
- Fixed bug in `CFG_BRICK_MK3_setTimestampValue()` where incorrect timestamp value was being set.
- Fixed bug in Time-codes GUI application where incorrect time-code period was being reported.
- Fixed bug where Sink application could crash when recording to file.
- Fixed bug in Device Configuration application where multiplier and divisor "Set" button was not being enabled in all circumstances.
- Fixed bug where Sink application would crash when selecting webcam field from "Frame Settings" dialog for second time.
- Fixed bug in Source application where webcam fields could not be deleted if the webcam is no longer attached.
- Fixed bug that could cause the Device Configuration application to get into a bad state when refreshing a remote device.
- Fixed bug in Source and Sink applications where it was possible to create a packet format with same name.
- Fixed bug in time-code handling for [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) cards.
- Various improvements to the documentation and internals.
- Linux kernel compatibility tested up to v5.0.0.

4.9.21 STAR-System v4.00 Beta 3 - 24th October 2018

- Added support for Conformance Tester Mk2 and EGSE Mk2 devices.
- Resolved issue where base selection dropdowns in the Device Configuration application would disappear on Linux.
- Devices are now listed under "STAR-Dundee Devices" in Device Manager on Windows.
- Linux kernel compatibility tested up to v4.19.

4.9.22 STAR-System v4.00 Beta 2 - 16th October 2018

- Fixed an issue in the Windows PCI driver that caused devices to get into a bad state after being reset.
- Fixed an issue in the Linux PCI and USB drivers that allowed invalid channels to be opened.
- Fixed an issue in the Source application that allowed an empty schedule to be transmitted.
- Added an icon to the Error Injection application.
- Various improvements to the documentation.

4.9.23 STAR-System v4.00 Beta 1 - 25th September 2018

- Added ability to transmit and receive webcam image data.
- Added device serial number to default device name so that it is easier to differentiate between multiple devices of the same type.
- Improved [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) device configuration example.
- Improvements to look and feel of GUI applications including maximise and minimise buttons.
- Custom device names are now stored in the device so that they persist when moved between computers.
- Added function [STAR_resetDeviceName\(\)](#) to allow a custom device name to be reset to a device's default name.
- Removed EEP statistics from the Source application as this only applies to the Sink.
- Sink application statistics now only show errors that are being checked.
- Added [STAR_getSpaceWireAddressPath\(\)](#) and [STAR_getSpaceWireAddressPathLength\(\)](#) functions.
- Fixed bug where Receive application could crash if an empty packet was received.
- Fixed bug that allowed time-code events to be enabled on incompatible versions of the [SpaceWire PCIe](#) card.
- Fixed bug where Sink showed packet too short errors when sequence error was encountered.
- Fixed possible crash when receiving link events on wrong channel.
- Fixed bug in Device Configuration Service that could result in the device not being recognised.
- Fixed bug that caused large packets to be split when received by the STAR Fire Mk3.
- Fixed bug that meant that time-codes could not be received in isolation on a channel.
- Fixed issue in `api_test_app` time-code example where the wrong channel was being used.
- Fixed bug in Device Configuration application where transmit frequency was not updated for [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#).
- Fixed bug where channel remained open if Linux application was terminated abruptly using Ctrl+C.
- Fixed issue with 32-bit/64-bit compatibility on recent Linux kernels.
- Various internal performance improvements.
- Various improvements to the documentation.
- Added functions [CFG_MK2_getTimeCodeDistributionPorts\(\)](#) and [CFG_MK2_setTimeCodeDistributionPorts\(\)](#). They are used to obtain and specify which output ports time-codes are forwarded on. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices.
- Added functions [CFG_MK2_getTimeCodeFlagMode\(\)](#) and [CFG_MK2_setTimeCodeFlagMode\(\)](#). They are used to get and set the time-code flag interpretation mode. They are used with Brick Mk2, Brick Mk3, Router Mk2S and PXI devices.

4.9.24 STAR-System v3.10 - 23rd February 2018

- Fixed memory leak in timestamp feature.
- Added batch file `star_performance_tester.bat` to call `star_performance_tester.exe` so test results can be viewed on screen.
- Added function `STAR_getChannelDirection()` to get the transfer direction in which a channel has been opened.
- Added function `STAR_getNextTransferItem()` to get the next item in a list of received stream items.

4.9.25 STAR-System v3.9 - 6th December 2017

- Linux kernel compatibility tested up to v4.14.3.
- Fixed issue that meant that Windows drivers were not recognised in a fresh install of Windows 7 or earlier.
- Fixed bug that prevented hexadecimal entry in some fields of the Device Configuration application.

4.9.26 STAR-System v3.8 - 9th November 2017

- Fixed issue that meant that Windows drivers were not recognised in some new installs of Windows 10.

4.9.27 STAR-System v3.7 - 13th October 2017

- Added ability to save and load routing table data in Device Configuration application.
- Added example programs for [Triggering API](#) and [RMAP Target API](#).
- Added `PORT_ACTION_STOP_RECEPTION` to [Triggering API](#) (supported by [SpaceWire Brick Mk3](#) and [Brick Mk4](#) hardware version 1.03) which causes a port to stop receiving whilst it is set.
- Linux kernel compatibility tested up to v4.13.4.
- Added new features to the C++ API including Triggering and RMAP Target support. See the [C++ API documentation](#) for more information.
- Fixed bug in Linux PCI driver where data was occasionally lost for packets less than 20 bytes.
- Fixed bug that could cause application to crash during infinite receive operations.
- Fixed hang in Performance Tester when very small packets were defined.
- Fixed crash in Device Configuration application when accessing pre-STAR-System devices as remote devices.
- Fixed bug where it was possible for time-code flags to be incorrectly set.
- Fixed bug in Windows PCI driver which occasionally caused duplicate data to be transmitted.
- Fixed bug that could allow the same channel to be opened more than once on Linux.
- Random data fields in packet formats are no longer checked by the Sink application.
- Added [Triggering API](#) documentation for [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#).
- Improved error handling for various API functions.
- Improvements to example project files and makefiles.
- Resolved various warnings in example code.

4.9.28 STAR-System v3.6 - 3rd April 2017

- Added support for timestamping received packets for the [SpaceWire Brick Mk3](#) and [Brick Mk4](#) device.
- Added support for injecting disconnect errors for the [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices.
- Added the [CFG_BRICK_MK3_setTimestampMethod\(\)](#), [CFG_BRICK_MK3_getTimestampMethod\(\)](#), [CFG_BRICK_MK3_enableRxTimestampEventsOnPort\(\)](#), [CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled\(\)](#), [CFG_BRICK_MK3_disableRxTimestampEventsOnPort\(\)](#), [CFG_BRICK_MK3_setTimestampValue\(\)](#), [CFG_BRICK_MK3_getTimestampValue\(\)](#), [CFG_BRICK_MK3_setPulseGeneratorFrequency\(\)](#) and [CFG_BRICK_MK3_getPulseGeneratorFrequency\(\)](#) functions to the [SpaceWire Brick Mk3](#) and [Brick Mk4](#) configuration API to support timestamping of received packets.
- Added the [STAR_getTimestampEventStartValue\(\)](#), [STAR_getTimestampEventEndValue\(\)](#), [STAR_getTimestampEventDirection\(\)](#), [STAR_getTimestampEventType\(\)](#) and [STAR_getTimestampEventRawCounters\(\)](#) functions to support timestamping of received packets.
- Added a description of the SpaceWire interface tristate behaviour for the ports on the [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices to their hardware manuals.
- Added example program to demonstrate the use of the timestamping feature for the [SpaceWire Brick Mk3](#) and [Brick Mk4](#).

4.9.29 STAR-System v3.5 - 17th March 2017

- Added support for the [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) device.
- Added support for the [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) device to the Triggering API.
- Fixed missing definitions in the Triggering API for Linux.
- Fixed bug that caused infinite receive operations to crash.
- Added additional documentation for the Triggering API.
- Added example program to demonstrate the use of the Triggering API.
- Added example program to demonstrate the use of the RMAP Target API.
- Fixed compiler warnings in the C++ API for old versions of GCC.
- Fixed compiler warnings in the C and C++ examples.
- Linux kernel compatibility tested up to v4.10.3.

4.9.30 STAR-System v3.4 - 9th March 2017

- Added millisecond resolution to timestamps in the STAR-System logs for Windows and Linux.
- Added support to the Device Configuration GUI application for the [STAR Fire Mk3](#) device.
- Added the [RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext\(\)](#) and [RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext\(\)](#) functions to the RMAP Target API to support user-supplied context variables when receiving RMAP target notification callbacks.
- Updated C++ API with new functionality. Information on the changes can be found in the change log section of the C++ API documentation.
- Fixed bug in the STAR-System Test application where the [STAR_INFINITE](#) value was incorrectly defined.

- Fixed bug in the Linux USB driver where if the size of a packet being transmitted was a multiple of the USB frame size, the transmission would hang.
- Fixed memory leak in the Link Events example program.
- Fixed memory leaks related to STAR-System callbacks.
- Improved resource clean-up during the shutdown procedure of STAR-System.
- Added external trigger voltage output information to the [SpaceWire Brick Mk3 and Brick Mk4](#) and [SpaceWire PXI Interface and PXI Interface Mk2](#) user manuals in the documentation.
- Various improvements to the documentation.

4.9.31 STAR-System v3.3 - 20th December 2016

- Added support for injecting disconnect errors when using [SpaceWire PXI Interface and PXI Interface Mk2](#) devices in the Error Injection GUI application.
- Added an API Test App example project which demonstrates how to implement many common applications of STAR-System.
- Added [SpaceWire PXI Interface and PXI Interface Mk2](#) configuration functions to replace the [SpaceWire Brick Mk2](#) functions that are currently used to control interface mode on port, identify source, port routing addresses, state events and speed events.
- Fixed issue in the drivers to support Linux kernel 4.8's hardened user copy function.
- Fixed HTML bug when viewing the [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) hardware manuals.
- Fixed bug in the GUI applications when distinguishing between different device hardware versions.
- Various improvements to the documentation.

4.9.32 STAR-System v3.2 - 24th November 2016

- Added an example application to demonstrate the use of the link events API.
- Added support for STAR Fire Mk3 devices.
- Added support for link events to the [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.
- Fixed bug in Sink GUI application which caused some recorded packet files to be lost.
- Fixed bug in Source and Sink GUI application where the statistics windows had incorrect font sizes on high DPI monitors.
- Fixed bug in the calculation of [SpaceWire PCI Mk2 and cPCI Mk2](#) register addresses which caused some functions to stop working.
- Improved readability of the device and file information fields in the Device Update GUI application.

4.9.33 STAR-System v3.1 - 21st October 2016

- Added new RMAP Target GUI to configure the RMAP targets within the [SpaceWire PXI Interface with RMAP Target](#) devices.
- Resizing the statistics views in the Source and Sink GUI applications now scales the font size accordingly.
- Added support in the [Triggering API](#) for controlling the triggering capabilities of [SpaceWire PCIe](#) devices. The [CFG_MK2_startPeriodicAction\(\)](#) and [CFG_MK2_stopPeriodicAction\(\)](#) functions have been deprecated for the [SpaceWire PCIe](#) devices because their functionality has been replaced by the [Triggering API](#).

- Added the `CFG_MK2_enableTimeCodeCounterBypassMode()`, `CFG_MK2_getTimeCodeCounterBypassModeEnabled()` and `CFG_MK2_disableTimeCodeCounterBypassMode()` functions to configure the time-code counter bypass mode within supported devices. When enabled, this mode allows the device to forward any received time-codes, including their flag values, without checking them for validity. When disabled, only valid time-codes are forwarded. A checkbox for enabling/disabling this feature will now appear in the Device Configuration GUI if supported by the device. Note that this feature is currently only supported by [SpaceWire PCIe](#) devices with hardware version 1.12 or above.
- Fixed bug in Source GUI application where the application would freeze on Linux after clicking the stop transmit button.
- Fixed bug when getting incorrect stream items after receiving multiple packets in a transfer operation.
- Linux kernel compatibility tested up to v4.8.2.
- Application notes are now installed in the `application_notes` directory.

4.9.34 STAR-System v3.0 - 26th August 2016

- A PDF version of this documentation is now available for download from the STAR-Dundee website in the support area for registered users, while the latest version of the HTML documentation is also available at <https://www.star-dundee.com/system/files/help/star-system/index.xhtml>.
- Added `CFG_MK2_enableTimeCodeEventsOnPort()`, `CFG_MK2_getTimeCodeEventsOnPortEnabled()` and `CFG_MK2_disableTimeCodeEventsOnPort()` to configure time-code events on individual ports of a [PCIe](#) device.
- Fixed bug in Sink application when attempting to record traffic to a directory which is not writeable.
- Added `CFG_MK2_enableStateChangeEventsOnPort()`, `CFG_MK2_getStateChangeEventsOnPortEnabled()` and `CFG_MK2_disableStateChangeEventsOnPort()` to configure state change events on individual ports of [PCIe](#) and [SPLT](#) devices.
- Added `CFG_MK2_enableSpeedChangeEventsOnPort()`, `CFG_MK2_getSpeedChangeEventsOnPortEnabled()` and `CFG_MK2_disableSpeedChangeEventsOnPort()` to configure speed change events on individual ports of [PCIe](#) and [SPLT](#) devices.
- Updated `CFG_MK2_getMeasuredLinkSpeed()` to work with all devices which support this feature, and removed `CFG_PXI_getMeasuredLinkSpeed()`.
- Added `STAR_destroyTransferItemList()` to destroy stream items returned by a call to `STAR_getTransferItemList()`. If this function is not called for the returned array of stream items, the memory will be leaked. This is a change of functionality from previous releases, and all code using `STAR_getTransferItemList()` must be updated. This change was necessary to deal with a bug in `STAR_getTransferItemList()` in previous releases which meant that the stream items pointed to by the returned array could disappear as more packets were received.
- Stream items created by receive operations can now be destroyed by calling `STAR_destroyStreamItem()` or `STAR_destroyTransferItemList()`. This change was necessary to ensure infinite (or very large) receive operations did not consume all the memory on the PC.
- Added the RMAP Target API for controlling the RMAP target in the SpaceWire PXI Interface with RMAP Target.
- Added the Triggering API for controlling the triggering capabilities of SpaceWire PXI Interface and Brick Mk3 devices.

4.9.35 STAR-System v3.0 Beta 9 - 26th July 2016

- Fixed bug in Device Configuration application which meant that when the Reset button was pressed, the window was not refreshed to show the new values. This bug was introduced in STAR-System v3.0 Beta 5.
- Added `STAR_getDeviceConfigCapabilities()` function and associated definitions of `STAR_DEVICE_CONFIG_SUPPORTED` and `STAR_DEVICE_CONFIG_NOT_SUPPORTED`. These values are used by `STAR_getDeviceConfigCapabilities()` to indicate if a device can be configured and can also be passed to `STAR_getDeviceListForType()` to get all devices which can or cannot be configured.
- Fixed bug in Device Configuration application which meant that the link speed divider and multiplier / divisor input boxes incorrectly allowed spaces.
- Fixed bug in Sink application where application could crash if starting recording to a file that had already reached the size limit. Now the file is correctly rotated.
- Fixed issue in Windows and Linux USB Driver that could cause a USB device to not transmit a packet until a subsequent packet was queued up to be transmitted.
- Added fix to potential permissions issue when running multiple applications under Linux with different user accounts.
- Improvements to the documentation.
- Linux kernel compatibility tested up to v4.7.

4.9.36 STAR-System v3.0 Beta 8 - 6th May 2016

- Fixed bug which caused `STAR_closeChannel()` to complete before the channel was closed. This could cause subsequent calls to open a channel to fail as the channel was still open.
- Fixed bug in Sink application where some of the statistics values were misaligned.
- Fixed bug in Windows USB Driver which could cause a blue screen if a channel was closed while a transmit or receive operation was in progress.
- Resolved code signing issue that could present SmartScreen warning during installation in newer versions of Windows.
- Improved documentation for Brick Mk3.
- Improved error reporting in Device Update Software.
- Added `CFG_MK2_getMeasuredLinkSpeed()` and `CFG_PXI_getMeasuredLinkSpeed()` functions to retrieve current link speed on a port.
- Linux kernel compatibility tested up to v4.5.4.

4.9.37 STAR-System v3.0 Beta 7 - 8th March 2016

- Added support for devices with improved SpaceWire clock duty cycle.
- Improvements to C++ API examples.
- Fixed bug to ensure 32- and 64-bit libraries are in the correct locations in Linux systems.
- Added composite multiplier and divider for clock frequency control in Device Configuration application.

4.9.38 STAR-System v3.0 Beta 6 - 21st January 2016

- Fixed an issue with IOCTL handling when running 32-bit apps on a 64-bit Linux kernel.
- Linux kernel compatibility tested up to v4.3.
- Added support for SpaceFibre Router PXI devices.

4.9.39 STAR-System v3.0 Beta 5 - 25th November 2015

- Fixed an issue with the Device Configuration API function `CFG_ROUTER_getConfigPortErrors()` where a late EEP was flagged when an early EEP was detected.
- Fixed an issue with the PCIe card on Linux.

4.9.40 STAR-System v3.0 Beta 4 - 13th November 2015

- Added support for SpaceWire PXI cards, with associated Configuration API.
- Tested on Windows 10, 32- and 64-bit.
- Tested on Linux 4.0.0 kernel, 32- and 64-bit.
- Reverted Glibc to version 1.12 for compatibility with older Linux systems.
- Reverted Qt to version 4.6.2 for compatibility with older Linux systems.

4.9.41 STAR-System v3.0 Beta 3 - 4th September 2015

- Fixed LabVIEW bug where an application can hang on restart when running in a loop.
- Added new Periodic Action functions to allow repetitive actions to be scheduled on the PCI-Mk2, cPCI-Mk2 and PCIe devices.
- Added function `STAR_getDeviceListForTypes()` to allow passing in an array of device types and to return an array of device IDs for all connected devices of the requested types.

4.9.42 STAR-System v3.0 Beta 2 - 24th July 2015

- Added new base box feature to GUI applications, which means the numerical base that values are entered in can be specified.
- Updated the Statistics dialog in the Sink application to have green error boxes when there are no errors, and to change red when there are errors. This makes errors much easier to spot.
- Completed support for SpaceWire Brick Mk3 devices in the Device Update software.
- Added new device type constant `STAR_DEVICE_ALL`, which can be passed to `STAR_getDeviceListForType()` to get all devices.
Note that as a result of this addition, the value of `STAR_DEVICE_TXRX_SUPPORTED` has changed from the value used in version 3.0 Beta 1.
- The CUBA Software, Source, Sink, Transmit, Receive, Error Injection and Time-codes applications have all been updated to only use devices which can transmit and receive packets.
- The Performance Tester and STAR-System Test have been updated to demonstrate the use of `STAR_getDeviceListForType()` and to improve the code.
- Renamed `STAR_canDeviceTransmitAndReceive()` function to `STAR_getDeviceTxRxCapabilities()`.

- Fixed bug in STAR-System core which could theoretically cause a problem during initialisation of the API.
- Completed CUBA Software documentation.
- Improved instructions on Linux installation and required modules.
- Fixed bug in STAR-System core which meant that a handle to a device was always kept open even after that device was removed. This could cause an issue on Linux when adding and removing devices.
- Fixed bug in STAR-System core which meant an application may hang when being closed on Linux if the Configuration Service was not running.

4.9.43 STAR-System v3.0 Beta 1 - 31st March 2015

- Added support for SpaceWire Brick Mk3 devices.
- Added support for STAR Fire devices with STAR-System support.
- Added support for Conformance Tester devices with STAR-System support.
- Added new Time-code GUI application to transmit and receive time-codes, and to configure the properties of devices related to time-codes.
- Added help to all GUI applications.
- Updated GUI applications to support command-line options:
 - `-0` to enable the use of channel 0 in the application.
 - `-s` to specify the serial number of the device to be used by the application.
 - `-c` to specify the channel on the device to be used by the application.

For more information, please see the [Graphical Applications](#) page.

- Added Error Injection GUI application to QNX installer.
- Minor improvements to the Device Configuration application when entering new values and when unable to read router properties.
- Added support for the HPPDSP, WBS and RTC devices to the Device Configuration GUI application.
- Updated Device Configuration GUI application to refresh all values on display following a reset.
- Added support for the STAR Fire, Link Analyser Mk2 and EGSE to the Device Update software.
- Updated Device Update application to support programming the EEPROM of Brick Mk3 devices, and a fast update mode for programming the FPGA of a Brick Mk3 (incomplete).
- Numerous minor improvements to the Source and Sink GUI applications when specifying the packet format.
- Replaced Copy and Paste buttons in the Source and Sink GUI application's Packet Format dialog with a Duplicate button.
- Fixed bug in Source and Sink GUI applications which meant a duplicate packet format was created when a packet format was renamed.
- Fixed bug in Source GUI application, which meant it would stop transmitting after it had transmitted 0xffffffff packets.
- Added data rate entry in the Statistics dialogs of the Source and Sink GUI applications.
- Added method to save packet formats to specific directories in Source and Sink GUI applications.
- Updated the Sink GUI application to allow unknown initial values for the sequence number field.
- Added the STAR-System version of the [CUBA Software](#).

- Improved Windows USB Driver, to provide additional buffer space and reduce the effect on the SpaceWire link of applications not receiving packets quickly enough. Unfortunately there is currently a bug in this code, see [Known Issues](#).
- Added `STAR_getDeviceListForType()` function.
- Added `STAR_canDeviceTransmitAndReceive()` function.
- Added definitions for all currently supported device types in `types.h`, along with definitions of `STAR_DEVICE_ICE_TXRX_SUPPORTED` and `STAR_DEVICE_TXRX_NOT_SUPPORTED`. These values are used by the above two functions.
- Added `STAR_createErrorInData()` function and added support for transmitting errors in sequence with data characters, when using devices which support this feature.
- Added support for Linux kernels up to v3.19.
- Fixed bug in Linux PCI Driver, which meant that some PCs were unable to map memory required to DMA to the device.
- Fixed bug when setting the name of a remote device using `STAR_CFG_createRemoteDeviceIdentifier()`.
- Fixed bug when receiving time-codes which meant non-zero flag values were incorrectly reported.
- Fixed bug in `CFG_BRICK_MK2_enableIdentifySourceOnPort()`, writing a value to an incorrect register.
- Fixed bug in reset feature of PCI drivers, which meant access to the device immediately after reset may put the device in a bad state.
- Fixed bug when resetting the API, which resulted in issues with some LabVIEW applications using the STAR-System LabVIEW Wrapper.
- Fixed bug in `CFG_MK2_setBaseTransmitClock()`, which could complete before the clock was successfully set, returning an error.
- Updated hardware manuals to address issues with channel numbering.

4.9.44 STAR-System v2.4.1 - 4th August 2014

- Added support for Linux kernel version 3.16.

4.9.45 STAR-System v2.4 - 2nd May 2014

- Release of STAR-System for RTEMS.

4.9.46 STAR-System v2.3 - 31st January 2014

- Fix to issue present in v2.2 release of Device Update software, which meant that device updates may fail with an error.
- Added support for 24-bit sequence numbers to the Source and Sink applications.
- Added support for SpaceWire Recorder device type.
- Support for CASTOR router configuration in Device Configuration application.
- Beta release of STAR-System for RTEMS.

4.9.47 STAR-System v2.2 - 5th November 2013

- Fixed problem with shortcuts to documentation on Linux. Wrong file extension was being used (.html rather than .xhtml).
- Fixed issues displaying some documentation files, including the hardware manuals and the GUI applications page, and resolved issues with tree view on left of documentation.
- Fixed memory leak in Device Configuration Service.
- Fixed bug in Device Configuration Service which could cause the service to get in to a bad state when a device was removed.
- Fixed bug in STAR-System core which could cause a crash if unexpected data was received from a device during device removal or addition.
- Fixed bug in STAR-System core which could cause a crash when submitting a receive operation which had previously been cancelled.
- Fix to compilation error in Linux USB Driver causing installation error when installed on kernels prior to 2.6.23.
- Fixed bug in STAR-System core which could cause a crash when creating a number of remote device identifiers.
- Fixed bug in Linux install script which may result in 64-bit libraries not being installed on 64-bit systems.
- Improvements to the Device Configuration application:
 - Errors on ports are now highlighted by changing the colour of the tab's text to red.
 - Clicking the Refresh button does not change the tab on display. Changing the device being displayed also does not change the tab.
- Improvements to the Performance Tester software:
 - Added support for `-usechannel0` argument, to allow channel 0 to be used to transmit and receive packets.
 - Fixed issue when performing rate and random tests, which could result in a receive test expecting to receive more packets than a transmit test would transmit.
- Fixed bug in Router Configuration Library: Invalid Destination Logical Address error was not being correctly reported.
- Added Cygwin support to Windows release.
- Fixed Time-flags mode was incorrectly reported for Brick Mk2, Router Mk2S and RTC devices.
- Fixed memory corruption leading to application crash when receiving link events.
- Added support for Linux kernels up to v3.11.
- Fix to issue in Windows PCI Driver, which could affect device update operations.

4.9.48 STAR-System v2.1 - 1st August 2013

- Fixed issues in Windows and Linux USB and PCI Drivers when hibernating a PC with a device connected which has a channel open.
- Fixed issues in documentation when using the document tree.
- Removed unnecessary dependency on libQtNetwork in GUI applications on Linux.
- On Windows, removed libraries for the current target (e.g. 32-bit libraries on 32-bit versions of Windows) from the lib install directory. Instead, only the libraries for each specific target are included in the appropriate sub-directory, e.g. lib/x86-32, lib/x86-64. Please use the libraries in these sub-directories when linking your code.

- Updated documentation for [STAR_TIMECODE](#) to indicate that time-codes can be transmitted on channel 0 only.
- Fixed bug in Linux USB Driver which could result in a kernel error when a USB device was removed on Linux.
- Removed ui.c and ui.h example files from API documentation, and moved them to their correct location in the examples.
- Added time-code test application to examples.
- Added FPGA version number information to STAR-System Test version information.
- Reduced size of documentation, and hence the installers. This was achieved by changing the images to be svg files. If you have problems viewing the images on your web browser, a version which uses gif files, and which should work on older browsers, is available for download from our website.

4.9.49 STAR-System v2.0 - 3rd July 2013

- Added C++ API.
- Fixed bug in Device Configuration application, which meant that invalid registers would be read for some devices, causing an error of 10 to be shown in the STAR-System log.
- Fixed bug in [STAR_CFG_destroyRemoteDeviceIdentifier\(\)](#) when passing in an invalid identifier, which would cause an exception.
- Added Error Injection GUI tool.
- Added function to support error injection for Mk2 compatible devices: [CFG_MK2_injectError\(\)](#).
- Fixed incorrect enum values for [STAR_CFG_PCIMK2_LINK_FREQ](#). Link Frequency values for 128MHz and 150MHz were swapped.
- Fixed issue with [CFG_ROUTER_stopLink\(\)](#), which did not reset the disable bit.
- Fixed bug in the Linux USB driver which caused a kernel panic if a device was unplugged while an operation was still in progress.
- Fixed bug that would cause [STAR_openChannelToLocalDevice\(\)](#) to fail.
- Fixed bug that would cause configuration operations to time out.
- Fixed bug that could crash the Device Configuration Service.
- Fixed memory leak in Device Configuration application, which could cause it to consume memory and eventually crash.

4.9.50 STAR-System v2.0 Beta 4 - 21st May 2013

- Fixed bug in STAR-API when calling [STAR_createRxOperation\(\)](#) with a stream item count of -1 to receive an infinite number of stream items. This call would always return an error.
- Reduced the time that the Device Configuration Service will keep a channel open following an operation to be a minimum of 1 second and a maximum of 2 seconds. In the previous release it was 10 seconds, which meant the channel could not be used to transmit or receive packets for quite some time.
- Fixed [STAR_CFG_getRemoteDeviceDescriptionNameString\(\)](#) not being exported correctly.
- Renamed GUI applications and their executable names as follows:
 - Simple Receive ([star_simple_rx](#)) => Receive ([star_receive](#))
 - Simple Transmit ([star_simple_tx](#)) => Transmit ([star_transmit](#))
 - Advanced Receive ([star_advanced_rx](#)) => Sink ([star_sink](#))

- Advanced Transmit (`star_advanced_tx`) => Source (`star_source`)

The Device Configuration application's filename has also been renamed from `star_config_gui` to `star_device_config`.

- Fixed bug in Sink application, when checking the packet format of a packet which contained a checksum/CRC and an address which was not present at the destination.
- Improved Source application to ensure random fields change between packets.
- Fixed bug in Linux PCI and USB drivers, which could cause a kernel oops when a device was first added (either by being plugged in, or when the driver was first loaded).
- Fixed bug in Linux USB driver, which meant a device reset would not work.
- Fixed bug in Windows USB Driver when transmitting, which could result in some packets not being transmitted.
- Fixed bug in `STAR_getApplicationID()` which meant it would fail if it was the first STAR-API function called in an application.
- Added noise warnings to Brick Mk2, PCIe and Router Mk2S hardware manuals.

4.9.51 STAR-System v2.0 Beta 3 - 12th April 2013

- Fixed bug in Device Configuration Service start-up script on Linux, which meant the service may not start on some Linux distributions. Also updated install and uninstall process to address related issues.
- Improved the end of line format used in header files and example code to use the operating system's native format.
- Addition of VxWorks documentation.
- Fixed synchronisation issue in STAR-System core, which could appear when registering a listener.
- Fixed bug in STAR-System core when receiving time-codes.
- Added the following functions to the Mk2 Configuration API to access external time-code selection settings:
 - `CFG_MK2_enableExternalTimeCodeSelection()`
 - `CFG_MK2_disableExternalTimeCodeSelection()`
 - `CFG_MK2_getExternalTimeCodeSelectionEnabled()`
- Added support for enabling and disabling external time-code selection to the Device Configuration GUI.
- Corrected type of parameter expected by `STAR_CFG_destroyRemoteDeviceDescriptionList()`.
- Updated the following listener functions to return an error if a device or driver is not specified:
 - `STAR_registerDeviceListenerForDriver()`
 - `STAR_registerDeviceListenerForDevice()`
 - `STAR_registerChannelListenerForDriver()`
 - `STAR_registerChannelListenerForDevice()`
- Fixed issues in Device Configuration Service, which could cause it to crash in some circumstances.
- Updated example Visual Studio project files to link to the appropriate libraries for 32-bit and 64-bit builds.
- Improved detection of addition and removal of devices.

4.9.52 STAR-System v2.0 Beta 2 - 19th March 2013

- Improved performance of Windows USB Driver.
- Improved memory usage of Windows USB Driver, to address occasional problem when adding and removing a USB device repeatedly.
- Fixed bug in Linux USB Driver which could potentially result in packets being corrupted or received out of order.
- Added option to configure remote devices in the Device Configuration GUI.
- Updated `STAR_CFG_createRemoteDeviceIdentifier()` to allow NULL addresses for the path and/or return path arguments.
- Improved `cfg_api_remote.h` to remove errors when including from C++ code.
- Fixed bug in Linux USB and PCI drivers when accessing invalid channel numbers.
- Disabled Set buttons in Device Configuration GUI until the corresponding value has been changed.
- Updated the Linux installer to include the option to perform a command-line install.

4.9.53 STAR-System v2.0 Beta 1 - 8th February 2013

- First VxWorks release.
- Added support for USB devices, including the Brick Mk2, the Router Mk2S and the SpaceWire Physical Layer Tester.
- Added support for new link state event (`STAR_LINK_STATE_EVENT`) and link speed event (`STAR_LINK↔_SPEED_EVENT`) stream types, which can be received from USB devices (but not transmitted).
- Added functions to access link state and link speed events:
 - `STAR_getLinkStateEventCount()`
 - `STAR_isLinkStateEventDisconnectError()`
 - `STAR_isLinkStateEventEscapeError()`
 - `STAR_isLinkStateEventLinkRunning()`
 - `STAR_isLinkStateEventParityError()`
 - `STAR_isLinkStateEventReceiveCreditError()`
 - `STAR_isLinkStateEventTransmitCreditError()`
 - `STAR_getLinkSpeedEventSpeed()`
- Fixed minor bugs in random and latency tests of STAR-System Performance Tester.
- Merged the STAR-Core and STAR-API modules in to a single module.
- Improved the documentation for remote devices.
- Fixed `STAR_CFG_getRemoteDeviceDescriptionList()` returning incorrect type.
- Added `STAR_CFG_destroyRemoteDeviceDescriptionList()` function.
- Fixed bug in Advanced Rx and Tx GUI applications, which resulted in packets containing incorrect bytes when specifying an address field containing more than one byte.
- Added Brick Mk2 and Router Mk2S Configuration APIs.
- Fixed bug in Windows Installer which meant Mk2 Configuration API was not included in the lib directory.
- Fixed access violation error when `STAR_registerTransferCompletionListener()` was called before any other STAR-API function.

- Fixed bug in [STAR_unregisterTransferCompletionListener\(\)](#) which unregistered all transfer completion listeners, instead of just the one specified.
- Fixed Windows installer not including file [cfg_api_mk2_types.h](#).
- Fixed bug in [CFG_ROUTER_getNetworkDiscoveryInfo\(\)](#) which resulted in the number of ports being returned in portCount being one less than the actual number of ports.
- Updated Simple Tx GUI application to allow the end of packet marker type of packets sent to be specified.
- Fixed bug in [CFG_ROUTER_clearPortErrors\(\)](#) which meant that errors were not being correctly cleared.
- Fixed bug in Device Configuration GUI which meant that the error status was not updated when the error was cleared.
- Added support for the notify for current option in STAR-API's register listener functions.
- Updated STAR-System Test to accept a path address when transmitting packets. This is required when using USB devices, or any devices in routing mode.
- Added [Configuration API Functions Supported by Device Types](#) page, listing the features and functions supported by each device type.
- Added Device Update GUI application.
- Fixed bug in Linux PCI Driver when receiving packets, which could result in a packet received by the device not being passed up to the API.

4.9.54 STAR-System v1.12 - 14th November 2012

- Fixed synchronisation bug in STAR-System initialisation, which occurred when the first function call to the API in an application, was made in multiple threads at the same time or in a very short space of time.
- Added [STAR_getPacketAddress\(\)](#) function.
- Fixed [STAR_getPacketData\(\)](#) reporting incorrect buffer length when address present.
- Fixed bug in Windows PCI Driver which could cause the PC to freeze when a device was reset. Also added note to [STAR_resetDevice\(\)](#) explaining the limitations of this function, and added a known issue for this.
- Updated documentation to describe the Device Configuration Service and the multi-threading and multi-processor support in STAR-System.
- Fixed bug which caused issues with functions which made use of the driver type, including [STAR_getDriverType\(\)](#) and [STAR_registerChannelListenerForDriver\(\)](#).
- Added [STAR_getDeviceDriver\(\)](#) function to get the driver of a device.
- Added [STAR_isDeviceVirtual\(\)](#) function to determine if a device is a virtual device.
- Added STAR-System Test and Performance Tester code to documentation as examples.
- Fixed bug in Windows PCI Driver when closing an application before closing channels which were open to transmit. Such channels would not be correctly cleaned up.

4.9.55 STAR-System v1.11 - 31st October 2012

- Update to Windows PCI driver to support firmware upgrades.

4.9.56 STAR-System v1.10 - 27th September 2012

- Fixed bug: `STAR_getDriverList()` now works correctly.
- Fixed bug: `CFG_MK2_setBaseTransmitClock()` now works correctly.
- Fixed bug: Callback functions registered with `STAR_registerTransferCompletionListener()` are now called correctly on completion of transmit operations.

4.9.57 STAR-System v1.9 - 7th September 2012

- Added: Advanced Rx GUI application can now record received packets and the fields of those packets to file.
- Fixed bug: Advanced Tx and Rx GUI applications did not check that a packet format had been selected when clicking OK in the Select Packet Format dialog.
- Fixed bug: STAR-System Test double loopback tests may encounter timeout errors when large packets or numbers of packets are used.
- Fixed bug: Advanced Tx application may transmit more packets than were requested when a number is specified.

4.9.58 STAR-System v1.8 - 24th August 2012

- Fixed bug: Linux PCI Driver may hang the system when transmitting and/or receiving on multiple links.
- Fixed bug: Attempts to create and transmit large (>65KB) packets would fail.
- Added: Support for multiple tests in the Performance Tester, so a double or triple loopback test can be performed.
- Added: Support for command line arguments in the Performance Tester.
- Change: The maximum permitted size for a single data chunk is now 0xffff. Attempts to create a data chunk larger than this using `STAR_createDataChunk()` will fail. `STAR_createPacket()` automatically breaks large input data buffers into data chunks of maximum size 0xffff.
- Fixed bug: Using Ctrl+C to quit an application on Windows or Linux which was transmitting and/or receiving could cause the application to hang.
- Improvement: Performance improvements to Windows PCI driver.
- Fixed bug: Copying a packet format in the Advanced Tx GUI application would result in an invalid packet format being created.
- Added: Checking of received packets added to the Advanced Rx GUI Application.
- Fixed bug: The Advanced Tx GUI application contained a bug building packets when a checksum covered a field prior to it in the packet, or covered a field which changed between packets, i.e. a sequence number.
- Fixed bug: The Advanced Tx GUI application contained a bug when adding a new field to a packet format with a checksum field selected. This could cause the checksum to cover the wrong fields.

4.9.59 STAR-System v1.7 - 20th July 2012

- Fixed bug: killing an application (e.g. using Ctrl+C) while it is transmitting or receiving on Windows could cause the application to get in to a bad state. Note that there is still another related bug related to this which can cause the application to hang.
- Fixed bug: restarting the Device Configuration Service after it had been stopped could fail. This can result in the Windows uninstall hanging if the Device Configuration Service had been stopped.

- Fixed bug: stopping the Device Configuration Service on Linux may not complete cleanly. This can result in the Linux install not starting the Device Configuration Service if an uninstall had previously been performed.
- Added: PCI Mk2 Packet Subsystem API.
- Change to End-User Licence Agreement to use new address for STAR-Dundee and to explicitly state the agreement covers "example software".
- Added byte ordering to sequence number and data pattern fields of Advanced Tx GUI.
- Added fixed value data pattern field type to Advanced Tx GUI.
- Added checksum/CRC field type to Advanced Tx GUI.
- Added length field type to Advanced Tx GUI.

4.9.60 STAR-System v1.6 - 29th June 2012

- Fixed bug: detaching from the STAR-System DLLs under Windows by calling FreeLibrary() resulted in dead-lock.
- Fixed bug: attempting to receive on an already open channel in the Simple Rx GUI application would result in the software getting in to a bad state.
- Added ability to specify schedule and format of packets to be transmitted in Advanced Tx GUI.
- Updated Device Configuration GUI to only show time-code distribution checkbox for supported ports.
- Fixed bugs in the following functions introduced in v1.5, which resulted in the wrong port being accessed (specified port - 1):
 - CFG_MK2_enableInterfaceModeOnPort()
 - CFG_MK2_getInterfaceModeOnPortEnabled()
 - CFG_MK2_disableInterfaceModeOnPort()
 - CFG_MK2_enableIdentifySourceOnPort()
 - CFG_MK2_getIdentifySourceOnPortEnabled()
 - CFG_MK2_disableIdentifySourceOnPort()
 - CFG_MK2_setPortRoutingAddress()
 - CFG_MK2_getPortRoutingAddress()
- Added support for cPCI Mk2 device type and cPCI bus type to STAR-API.
- Improvements to GUI applications to remove warnings when running from console.

4.9.61 STAR-System v1.5 - 23rd April 2012

- Improved performance of Device Configuration APIs.
- Fixed bug in STAR-System test when receiving to file under Windows which caused a spurious carriage return character to be added after writing a line feed character.
- Fixed synchronisation bug when using Device Configuration APIs which could cause operations to fail, and improved error reporting.
- Fixed bug in Router Configuration API when getting a routing table entry using `CFG_ROUTER_getRoutingTableEntry()`. The port mask was not being set correctly.
- Fixed bug in PCI Mk2 Configuration API when setting the link clock frequency using `CFG_PCIMK2_getLinkClockFrequency()`.

- Fixed bug in Linux PCI Driver, which resulted in the wrong device type being returned by calls to `STAR_getDeviceType()` on Linux.
- Fixed bug which could cause a shared memory area to become inaccessible when an application was closed. This could result in various issues, e.g. attempts to get the name of a device might fail.
- Fixed bug in Mk2 Configuration API when enabling time-code master using `CFG_MK2_enableTimeCodeMaster()`.
- Fixed bug in Mk2 Configuration API which resulted in the wrong port being accessed when getting and setting interface mode properties for a port using the following functions:
 - `CFG_MK2_enableInterfaceModeOnPort()`
 - `CFG_MK2_disableInterfaceModeOnPort()`
 - `CFG_MK2_enableIdentifySourceOnPort()`
 - `CFG_MK2_disableIdentifySourceOnPort()`
 - `CFG_MK2_setPortRoutingAddress()`
 - `CFG_MK2_getPortRoutingAddress()`
- Added new functions to Mk2 Configuration API:
 - `CFG_MK2_getTimeCodeMasterEnabled()`
 - `CFG_MK2_getInterfaceModeEnabled()`
 - `CFG_MK2_getIdentifySourceEnabled()`
 - `CFG_MK2_getInterfaceModeOnPortEnabled()`
 - `CFG_MK2_getIdentifySourceOnPortEnabled()`
- GUI Prototype improvements:
 - Device Configuration GUI:
 - * Added routing table tab to configure the routing table of a device.
 - * Added missing features to existing tabs.
 - Simple Tx and Rx GUIs:
 - * Added option to display/enter packets in ASCII.
 - Added Advanced Tx and Rx GUIs.

4.9.62 STAR-System v1.4 - 16th March 2012

- First full QNX release.
- Updated Router Configuration API to fix bug when getting error status using `CFG_ROUTER_getConfigPortErrors()`, `CFG_ROUTER_getSpaceWireLinkErrors()` and `CFG_ROUTER_getExternalPortErrors()`. The error structure provided was not being cleared.
- Fixed bug in Linux version of `STAR_resetDevice()`. The return value was incorrect. STAR-System Test now displays an error if the device cannot be reset.
- Updated Linux install and uninstall scripts to improve error checking and clean-up.
- Fixed issue when closing applications which have used a Device Configuration API. There was a bug when detaching from the Device Configuration Messenger.
- Fixed issue when running two applications at the same time, which could cause one of the applications not to terminate correctly. This was caused by a bug in the clean-up code.
- Improved output of STAR-System Test option to display device and version information.
- Added prototype GUI applications to installers.
- Added 32-bit DLLs to 64-bit Windows installer, to allow 32-bit applications to run on 64-bit Windows. Note that 32-bit Linux applications already run on 64-bit Linux.
- Fixed bug in RMAP Packet Library's `RMAP_CheckPacketValid()` function when checking a write command packet or read reply packet has a data length of 0.

4.9.63 STAR-System v1.3 - 14th February 2012

- Added Mk2 Interface and Router Device Configuration API files missing from v1.2 install.
- Added identify device option to STAR-System Test.
- Improvements to PCI Mk2 Device Configuration API example.
- Minor improvements to documentation.

4.9.64 STAR-System v1.2 - 13th February 2012

- First full Linux release.
- First release with PCIe support.
- Added Mk2 Interface and Router Device Configuration API.
- Deprecated functions in PCI Mk2 Device Configuration API which are present in Mk2 Interface and Router Device Configuration API.
- Fixed bug which could cause Device Configuration API set functions to timeout and return an error, despite the operation actually being performed successfully.
- Fixed bug in `STAR_closeChannel()` which could result in resources being accessed after they'd been freed, causing a segmentation fault on Linux.
- Fixed issues in Device Configuration Service, which could cause the service to exit unexpectedly, or to hang.
- Fixed occasional problems when starting a new application on Linux after previously starting and stopping multiple applications.
- Fixed memory leaks in system core.
- Updated API header files to address namespace issues.
- Fixed problem with Linux installer which could result in a logout at the end of installation.

4.9.65 STAR-System v1.1 - 20th December 2011

- First Linux (beta) release.
- First stable API release.
- Added remote device features to documentation and added further detail to documentation.
- Corrected issues with remote device function prototypes.
- `len` parameter of `STAR_getChunkData()` changed from `unsigned long *` to `unsigned int *`, to be consistent with other parameters and return types.
- Renamed `STAR_waitOnTransferCompletion()` to `STAR_waitOnTransferOperationCompletion()` to be consistent with other function names.
- Renamed `STAR_getTransferStatus()` to `STAR_getTransferOperationStatus()` to be consistent with other function names.
- Renamed `STAR_cancelTransferWaits()` to `STAR_cancelTransferOperationWaits()` to be consistent with other function names.
- Renamed `STAR_cancelReceiveOp()` to `STAR_cancelTransferOperation()` to be consistent with other function names.

- Renamed `STAR_disposeTransferOp()` to `STAR_disposeTransferOperation()` to be consistent with other function names.
- Renamed `STAR_CONNECTION_ID` to `STAR_CHANNEL_ID` to improve and simplify channel-related names and functions.
- Renamed `STAR_CONNECTION_LISTENER_ID` to `STAR_CHANNEL_LISTENER_ID` to improve and simplify channel-related names and functions.
- Removed `STAR_CONNECTION_ENDPOINT_ID` to improve and simplify channel-related names and functions.
- Renamed `STAR_ConnectionEventFunc` to `STAR_ChannelEventFunc` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_ENDPOINT_TYPE` to `STAR_CHANNEL_TYPE` and updated values to improve and simplify channel-related names and functions.
- Renamed `STAR_CONNECTION_DIRECTION` to `STAR_CHANNEL_DIRECTION` and updated values to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListener()` to `STAR_registerChannelListener()` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListenerForDriver()` to `STAR_registerChannelListenerForDriver()` and updated parameters to improve and simplify channel-related names and functions.
- Renamed `STAR_registerConnectionListenerForDevice()` to `STAR_registerChannelListenerForDevice()` and updated parameters (including adding `notifyForCurrent`) to improve and simplify channel-related names and functions.
- Renamed `STAR_unregisterConnectionListener()` to `STAR_unregisterChannelListener()` and updated parameter type to improve and simplify channel-related names and functions.
- Renamed `STAR_createConnectionBetween()` to `STAR_openChannelBetweenLocalDevices()` and updated return type to improve and simplify channel-related names and functions.
- Renamed `STAR_createConnectionTo()` to `STAR_openChannelToLocalDevice()` and updated parameters and return type to improve and simplify channel-related names and functions.
- Renamed `STAR_destroyConnection()` to `STAR_closeChannel()` and updated parameter to improve and simplify channel-related names and functions.
- Renamed `STAR_isChannelConnected()` to `STAR_isChannelOpen()` and updated return type to improve and simplify channel-related names and functions.
- Renamed `STAR_getDeviceConnectedToChannel()` to `STAR_getLocalDeviceAttachedToChannel()` to improve and simplify channel-related names and functions.
- Renamed `STAR_getApplicationConnectedToChannel()` to `STAR_getApplicationAttachedToChannel()` to improve and simplify channel-related names and functions.
- Updated `STAR_submitTransferOperation()` to use updated parameter type to improve and simplify channel-related names and functions.
- Updated `STAR_submitTransferOperationList()` to use updated parameter type to improve and simplify channel-related names and functions.
- Renamed `STAR_getPacketEop()` to `STAR_getPacketEOP()` to be consistent with other function names.
- Renamed `STAR_setPacketEop()` to `STAR_setPacketEOP()` to be consistent with other function names.
- Removed the `isUnlimited` parameter from `STAR_createRxOperation()`, with an item count of `-1` now resulting in an unlimited receive.
- Updated test programs and example code to use new function names, parameters and return types.

- Fixed issue with RMAP Packet Library file name on Windows.
- Fixed issue starting and stopping the Device Configuration Service on Windows.
- Added `STAR_transmitPacket()` and `STAR_receivePacket()` to provide simple methods for transmitting and receiving a single packet.
- Fixed bug when attempting to create a transfer operation prior to opening a channel.
- Removed deprecated functions, included prior to STAR-System, from the RMAP Packet Library.
- Replaced `STAR_freeBuffer()` with `STAR_destroyVersionInfo()`, `STAR_destroyVersionInfoList()`, `STAR_destroyDeviceList()`, `STAR_destroyDriverList()` and `STAR_destroyPacketData()`.
- Renamed the following header files so they are lower case and with words separated by underscores:
 - `streamItem.h` -> `stream_item.h`
 - `streamItemTypes.h` -> `stream_item_types.h`
 - `transferTypes.h` -> `transfer_types.h`
 - `cfg-api-remote.h` -> `cfg_api_remote.h`
 - `cfg-api-pci-mk2.h` -> `cfg_api_pci_mk2.h`
 - `cfg-api-pci-mk2_types.h` -> `cfg_api_pci_mk2_types.h`
 - `cfg-api-router.h` -> `cfg_api_router.h`
 - `cfg-api-router-types.h` -> `cfg_api_router_types.h`
- Renamed the following library files so they are lower case and with words separated by underscores:
 - `star-conf_pcimk2` -> `star_conf_api_pci_mk2`
 - `star-conf_router` -> `star_conf_api_router`
- Added `STAR_destroyString()` and updated the parameters and return types of the following functions to return a string:
 - `STAR_getApplicationName()`
 - `STAR_getApplicationNameForID()`
 - `STAR_getDeviceBusTypeAsString()`
 - `STAR_getDeviceTypeAsString()`
 - `STAR_getDeviceName()`
 - `STAR_getDeviceSerialNumber()`

4.9.66 STAR-System v1.0 - 17th October 2011

- First Windows release.
- Improved performance.
- Documentation improvements.
- Support for configuring remote devices using the Device Configuration API added.
- Issue when accessing the Device Configuration Service when it is not running addressed.
- `STAR_getApplicationConnectedToChannel()` function updated to return an application identifier.
- `STAR_getApplicationNameForID()` function added to get an application's name.
- `STAR_getApplicationID()` function added to get the current application's identifier.
- `STAR_getDeviceBusTypeAsString()` function added to get a string representation of a device's bus type.
- Added support for error logging to QNX.

- STAR-System Test Improvements:
 - Displays each device's firmware version.
 - Provides an option to reset a device.
 - Command line arguments accepted to display the properties of the modules and devices.
 - Added additional tests to transmit and receive files.
- Performance Tester updated to record the CPU usage during a test.

4.9.67 STAR-System v0.8 (Beta 1) - 22nd August 2011

First QNX (beta) release.

Chapter 5

Deprecated List

globalScope> Global CFG_getMeasuredLinkSpeed (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U16 *pLinkSpeed)

Function superseded by `CFG_getRxSignallingRate()`. Gets the current link speed measured on a specific port.

globalScope> Global CFG_getTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 *pSignallingRate)

Function superseded by `CFG_getTxSignallingRate()`. Gets the transmit bit rate for a given link.

globalScope> Global CFG_setTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, U32 signallingRate)

Function superseded by `CFG_setTxSignallingRate()`. Sets the transmit bit rate on a given link. The transmit bit rate of a link is given in Mbit/s. The bit rate will be between 2 Mbit/s and 400 Mbit/s inclusive, although 400 Mbit/s may not be achievable with every device type.

globalScope> Global CFG_updateDeviceInfo (STAR_DEVICE_ID deviceId)

Function superseded by `CFG_refreshDeviceInfo()`. This function is used to update a device's information held in the shared memory. This is only required for remote devices if they have been removed and replaced.

globalScope> Global PORT_ACTION

Replaced by `SPW_PORT_ACTION_MASK`.

globalScope> Global PORT_ACTION_MASK

Replaced by `SPW_PORT_ACTION_MASK`.

globalScope> Global PORT_EVENT

Replaced by `SPW_PORT_EVENT`.

globalScope> Global PORT_EVENT_MASK

Replaced by `SPW_PORT_EVENT_MASK`.

globalScope> Global STAR_DEVICE_STAR_ULTRA_PCIE

Replaced by `STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE`. The STAR-Dundee STAR-Ultra PCIe Interface device type.

globalScope> Global STAR_RECEIVE_NULL

Replaced by `STAR_RECEIVE_NULLS` for naming consistency. The `STAR_RECEIVE_NULL` value in the `STAR_RECEIVE_MASK` enum was replaced by the `STAR_RECEIVE_NULLS` value for naming consistency. This macro is included to maintain backwards compatibility.

globalScope> Global TRIGGER_CFG_disablePortSpill (const STAR_DEVICE_ID deviceId, const U32 port)

Function superseded by `TRIGGER_CFG_disableSpWPortSpill()`.

globalScope> Global TRIGGER_CFG_disablePortTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 port)

Function superseded by

`TRIGGER_CFG_disableSpWPortTransmitPacketMode()`.

globalScope> Global `TRIGGER_CFG_enablePortSpill` (const STAR_DEVICE_ID deviceId, const U32 port)

Function superseded by `TRIGGER_CFG_enableSpWPortSpill()`.

globalScope> Global `TRIGGER_CFG_enablePortTransmitPacketMode` (const STAR_DEVICE_ID deviceId, const U32 port)

Function superseded by

`TRIGGER_CFG_enableSpWPortTransmitPacketMode()`.

globalScope> Global `TRIGGER_CFG_getDeviceClockRate` (const STAR_DEVICE_ID deviceId)

Function superseded by `TRIGGER_CFG_getDeviceClockFreq()`. Gets the clock rate from the specified device that is being used as the clock rate in/with the triggering functions/functionality.

globalScope> Global `TRIGGER_CFG_getPortInputEvents` (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_EVENT_MASK *const pEvents)

Function superseded by `TRIGGER_CFG_getSpWPortInputEvents()`.

globalScope> Global `TRIGGER_CFG_getPortOutputActions` (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_ACTION_MASK *const pActions)

Function superseded by `TRIGGER_CFG_getSpWPortOutputActions()`.

globalScope> Global `TRIGGER_CFG_getPortSpillEnabled` (const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)

Function superseded by `TRIGGER_CFG_getSpWPortSpillEnabled()`.

globalScope> Global `TRIGGER_CFG_getPortTransmitPacketModeEnabled` (const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)

Function superseded by

`TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled()`.

globalScope> Global `TRIGGER_CFG_getTriggerMatrix` (const STAR_DEVICE_ID deviceId)

Function superseded by a number of new resource count retrieval functions e.g., `TRIGGER_CFG_getNumberOfExtTriggers`.

globalScope> Global `TRIGGER_CFG_setPortInputEvents` (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_EVENT_MASK events)

Function superseded by `TRIGGER_CFG_setSpWPortInputEvents()`. Sets the input events for a port which will cause an internal trigger to be set.

globalScope> Global `TRIGGER_CFG_setPortOutputActions` (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_ACTION_MASK actions)

Function superseded by `TRIGGER_CFG_setSpWPortOutputActions()`.

Class `TRIGGER_MATRIX`

Structure no longer needed due to new resource count retrieval

functions e.g., `TRIGGER_CFG_getNumberOfExtTriggers`. Hardware capabilities.

Chapter 6

Module Index

6.1 STAR-System C APIs

Here is a list of all modules:

STAR-API	275
General API Functions	277
Device and Driver Management	282
Virtual Devices and Drivers	379
Ethernet Devices and Drivers	381
Channel Management	313
Stream Items	321
Transfer Operations	357
RMAP Target API	442
Manual Authorisation	444
Configuration	448
Notifications	462
Generic Triggering API	470
General	490
External Triggers	493
Counters	502
SpaceWire Ports	516
Time-codes	528
Internal Triggers	531
SpaceFibre Ports	543
SpaceFibre Virtual Channels	547
Defines	555
PCI Mk2 Packet Subsystem	597
Generic Configuration API	614
General	851
Hardware	856
Port Status and Control	860
Device Identifier	868
Group Adaptive Routing	871
Interface Mode	873
Time-codes	882
Events	892
Transmit Link Speed	899
Receive Link Speed	901
Precision Links	902
Error Injection	906
Timestamping	908

Periodic	914
Broadcast Controller	916
Defines	920
SpaceFibre Configuration API	616
Device Information & Identification	924
Device Information	925
Device Identification	928
Routing Table	932
Network Management	939
Vendor Specific	947
Virtual Channel Information	955
Virtual Network Number	956
Status	958
Errors	963
Quality Of Service	966
Time-slots	972
Data Rate	974
Port Information	975
Link Information	981
Control	982
Status	991
Events	997
Debug	1001
Active Lanes	1008
Lane Information	1009
Control	1010
Status	1019
Events	1028
Data Signalling Rate	1034
Types	1037
Defines	1043
Other Device Configuration APIs	618
Remote Devices	621
Router Configuration API	627
Device Information	637
Port Status and Control	643
Routing Table	657
User Configurable Registers	660
Router Control and Status Configuration	665
Mk2 Compatible Interface and Router Device Configuration API	628
Events	560
Link Speed	566
Measured Link Speed	572
Time-code Master	574
Device Information	581
Interface Mode	584
Error Injection	592
Periodic Actions	594
PCI Mk2 Configuration API	630
Link Speed	384
Brick Mk2 Configuration API	631
Link Speed	399
Interface Mode	402
Error Injection	407
Events	408
Brick Mk3 Configuration API	632
Link Speed	413

Timestamping	417
GbE Brick Mk1 Configuration API	633
Link Speed	424
Router Mk2S Configuration API	634
Link Speed	556
PXI Configuration API	635
Link Speed	426
Error Injection	431
Interface Mode	432
Events	437
PCIe Mk2 Configuration API	636
Link Speed	387
Broadcast Controller	390
Timestamping	394
Error Injection	398
Triggering API	673
Events	688
Actions	720
Configuration	741
RMAP Packet Library	1044
Functions for Building RMAP Packets	1057
Functions for Interpreting RMAP Packets	1078
CCSDS/SPP/TF Packet Library	1089
Functions for building SPP Packets	1103
Functions for retrieving (reconstructing) and interpreting SPP Packets	1108
Functions for building TF Packets	1117
Functions for retrieving (reconstructing) and interpreting TF Packets	1120
STAR-Dundee Types	1130

Chapter 7

File Index

7.1 File List

Here is a list of all documented files with brief descriptions:

ccsds_spp_packet_library.h	
Declarations of enums, structures and functions provided by the CCSDS/SPP library . . .	1131
ccsds_spp_pus_services.h	
Declarations of functions provided by the PUS Services functionality in the CCSDS/SPP library	1136
cfg_api_brick_mk2.h	
Functions used to configure STAR-Dundee Brick Mk2 and compatible devices	1138
cfg_api_brick_mk2_types.h	
Types used with the STAR-Dundee Brick Mk2 Configuration API	1139
cfg_api_brick_mk3.h	
Functions used to configure STAR-Dundee Brick Mk3 and compatible devices	1142
cfg_api_brick_mk3_types.h	
Types used with the STAR-Dundee Brick Mk3 Configuration API	1143
cfg_api_gbe_brick_mk1.h	
Declaration of the functions provided by the STAR-Dundee GbE Brick Mk1 Configuration API	1143
cfg_api_generic.h	
Types used with the STAR-Dundee Generic Configuration API	1144
cfg_api_generic_common.h	
Additional types used with the STAR-Dundee Generic Configuration API	1154
cfg_api_mk2.h	
Functions used to configure STAR-Dundee Mk2 compatible devices	1155
cfg_api_mk2_types.h	
Types used with the STAR-Dundee Mk2 compatible device configuration API	1158
cfg_api_pci_mk2.h	
Functions used to configure STAR-Dundee PCI Mk2 devices	1159
cfg_api_pci_mk2_types.h	
Types used with the STAR-Dundee PCI Mk2 Configuration API	1160
cfg_api_pcie_mk2.h	
Functions used to configure STAR-Dundee PCIe Mk2 and compatible devices	1161
cfg_api_pxi.h	
Functions used to configure STAR-Dundee PXI devices	1162
cfg_api_remote.h	
Functions used to create, configure and destroy paths to remote devices	1164
cfg_api_router.h	
Functions used to configure STAR-Dundee routing devices	1165
cfg_api_router_mk2s.h	
Functions used to configure STAR-Dundee Router Mk2S devices	1167

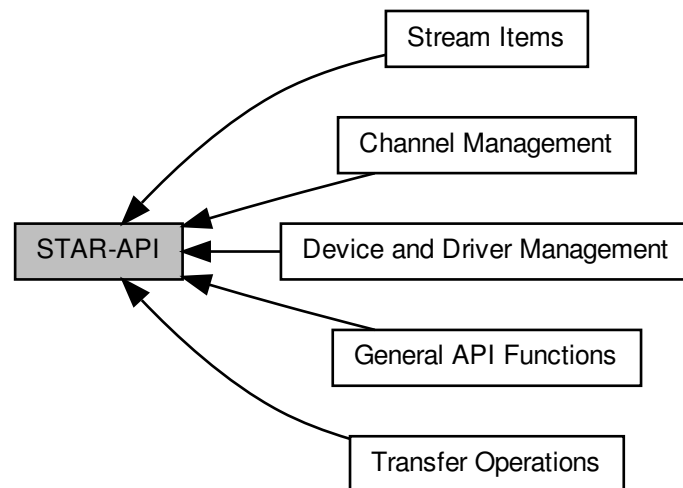
cfg_api_router_types.h	Types used with the STAR-Dundee Router API	1168
cfg_api_spfi.h	Functions used to configure SpaceFibre devices	1169
cfg_api_spfi_types.h	Types used with the STAR-Dundee SpaceFibre Configuration API	1178
common.h	STAR-Dundee API macros and types required by STAR API modules	1180
general.h	General functions provided by STAR-API	1181
packet_subsystem.h	Definitions of the functions provided by the PCI Mk2 packet subsystem API	1184
rmap_packet_library.h	Declarations of the functions provided by the STAR-Dundee RMAP Packet Library	1187
rmap_target_pxi_if.h	Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXi Interface devices	1192
rmap_target_types.h	Types used with the STAR-Dundee RMAP Target API	1194
star-api.h	Declarations of all functions provided by STAR-API	1195
star-dundee_annotations.h	Macros for previously undefined annotations macros	1196
star-dundee_types.h	Definitions of STAR-Dundee commonly used types	1196
stream_item.h	Functions create and modify items that can be sent over a SpaceWire network	1197
stream_item_types.h	Structures used to represent stream items such as packets, time-codes and link control to- kens	1200
transfer.h	Declarations of functions provided by STAR-API relating to transfer operations	1202
transfer_types.h	Types used to describe transmit and receive operations	1204
triggering_brick_mk3.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee SpaceWire Brick Mk3 and Brick Mk4 and SpaceWire GbE Brick Mk1 devices	1205
triggering_generic.h	The Generic Triggering API used to configure triggering functionality on all STAR-Dundee devices	1209
triggering_pcie.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices	1216
triggering_pcie_mk2.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe MK2 device	1218
triggering_pxi_if.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices	1223
triggering_pxi_ro.h	Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices	1227
triggering_types.h	Types used with the STAR-Dundee Triggering API	1230
types.h	Types, structures and function types used by STAR-API	1236
version.h	Version information for a module	1239

Chapter 8

Module Documentation

8.1 STAR-API

Collaboration diagram for STAR-API:



Modules

- [General API Functions](#)

These are functions that provide information about the API and the STAR-System stack itself.

- [Device and Driver Management](#)

Functions providing information on devices and drivers present.

- [Channel Management](#)

Used to open and close channels, between applications and devices.

- [Stream Items](#)

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

- [Transfer Operations](#)

Transfer operations are used to transmit and receive packets and time-codes.

8.1.1 Detailed Description

8.1.2 Introduction

This is the documentation for STAR-API, the C API of STAR-System, the STAR-Dundee software stack. This API is used to manage STAR-Dundee SpaceWire devices and the channels between them, and to transmit and receive data across those channels.

8.1.3 Structure

The API is broken down into a number of logically grouped sections. There are the General functions, which relate to the API itself, such as creating and disposing of devices and channels, and getting device and channel information.

8.2 General API Functions

These are functions that provide information about the API and the STAR-System stack itself.

Collaboration diagram for General API Functions:



Typedefs

- typedef CFG_INFO_ID_TYPE STAR_APP_ID
Unique value used to identify an application using STAR-System.

Functions

- STAR_VERSION_INFO *STAR_API_CC STAR_getApiVersion (void)
Gets the Version of STAR-API.
- void STAR_API_CC STAR_destroyVersionInfo (_Post_ptr_invalid_ STAR_VERSION_INFO *pVersionInfo)
Frees a version information structure previously created by a call to STAR_getApiVersion(), STAR_getDriverVersion() or STAR_getDeviceFirmwareVersion().
- _Ret_opt_cap_ count STAR_VERSION_INFO *STAR_API_CC STAR_getAllVersions (_Out_ U32 *count)
Gets an array of all the version info structures provided by all modules of STAR-System.
- void STAR_API_CC STAR_destroyVersionInfoList (_Post_ptr_invalid_ STAR_VERSION_INFO *pVersionInfoList)
Frees a version information list previously created by a call to STAR_getAllVersions().
- int STAR_API_CC STAR_setApplicationName (_In_z_ char *name)
Sets the name of the application using STAR-API.
- _Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getApplicationName (void)
Gets the name of the calling process (this process), set by STAR_setApplicationName().
- STAR_APP_ID STAR_API_CC STAR_getApplicationID (void)
Gets the application ID of the calling process.
- _Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getApplicationNameForID (STAR_APP_ID appld)
Gets the name of the application with the given ID.
- void STAR_API_CC STAR_destroyString (_Post_ptr_invalid_ _In_z_ char *string)
Destroy a string returned by STAR-API, such as from STAR_getApplicationName() or STAR_getDeviceTypeAsString().
- STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)
Returns the ID of an application attached to a given channel on a given device.

8.2.1 Detailed Description

These are functions that provide information about the API and the STAR-System stack itself.

8.2.2 Typedef Documentation

8.2.2.1 typedef CFG_INFO_ID_TYPE STAR_APP_ID

Unique value used to identify an application using STAR-System.

Added in version:

STAR-System v1.0 - 17th October 2011

8.2.3 Function Documentation

8.2.3.1 void STAR_API_CC STAR_destroyString (_Post_ptr_invalid_ _In_z_ char * *string*)

Destroy a string returned by STAR-API, such as from [STAR_getApplicationName\(\)](#) or [STAR_getDeviceTypeAsString\(\)](#).

Parameters

<i>in</i>	<i>string</i>	the string to be destroyed
-----------	---------------	----------------------------

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[application_name_example.c](#), [device_identifier_example.c](#), [link_events.c](#), [mk2_configuration_example.c](#), [port_control_status_example.c](#), [router_configuration_example.c](#), [routing_table_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_test.c](#), [triggering_example.c](#), [triggering_generic_example.c](#), [ui.c](#), [user_registers_example.c](#), and [utilities.c](#).

8.2.3.2 void STAR_API_CC STAR_destroyVersionInfo (_Post_ptr_invalid_ STAR_VERSION_INFO * *pVersionInfo*)

Frees a version information structure previously created by a call to [STAR_getApiVersion\(\)](#), [STAR_getDriverVersion\(\)](#) or [STAR_getDeviceFirmwareVersion\(\)](#).

Parameters

<i>pVersionInfo</i>	the version information structure to be freed
---------------------	---

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[api_version_example.c](#), and [star-test.c](#).

8.2.3.3 void STAR_API_CC STAR_destroyVersionInfoList (_Post_ptr_invalid_ STAR_VERSION_INFO * *pVersionInfoList*)

Frees a version information list previously created by a call to [STAR_getAllVersions\(\)](#).

Parameters

<i>pVersionInfoList</i>	the version information list to be freed
-------------------------	--

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`all_version_example.c`, and `star-test.c`.

8.2.3.4 `_Ret_opt_cap_count STAR_VERSION_INFO* STAR_API_CC STAR_getAllVersions ( _Out_ U32 * count )`

Gets an array of all the version info structures provided by all modules of STAR-System.

Parameters

<i>out</i>	<i>count</i>	Count of version information structures held in array.
------------	--------------	--

Returns

Array of version info structures containing STAR-System version info structures.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_destroyVersionInfoList()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`all_version_example.c`, and `star-test.c`.

8.2.3.5 `STAR_VERSION_INFO* STAR_API_CC STAR_getApiVersion ( void )`

Gets the Version of STAR-API.

Returns

Pointer to a version structure that will contain STAR-API's version info, or NULL if a memory allocation error occurred.

Note

This structure must be freed using `STAR_destroyVersionInfo()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`api_version_example.c`.

8.2.3.6 `STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel ( STAR_DEVICE_ID deviceId, unsigned int channelNumber )`

Returns the ID of an application attached to a given channel on a given device.

Parameters

<i>deviceId</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

Requested application ID, or 0 if no application is attached

Added in version:

STAR-System v1.1 - 20th December 2011

8.2.3.7 STAR_APP_ID STAR_API_CC STAR_getApplicationID (void)

Gets the application ID of the calling process.

Returns

The application ID for the calling process

Added in version:

STAR-System v1.0 - 17th October 2011

8.2.3.8 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getApplicationName (void)

Gets the name of the calling process (this process), set by [STAR_setApplicationName\(\)](#).

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Returns

the application name as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[application_name_example.c](#).

8.2.3.9 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getApplicationNameForID (STAR_APP_ID appld)

Gets the name of the application with the given ID.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>appld</i>	the application ID to get a corresponding name for
--------------	--

Returns

the application name as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

8.2.3.10 int STAR_API_CC STAR_setApplicationName (_In_z_ char * *name*)

Sets the name of the application using STAR-API.

This is the name provided to other processes calling `STAR_getApplicationNameForID()`.

Parameters

<i>in</i>	<i>name</i>	Null terminated string containing the application name
-----------	-------------	--

Returns

1 if app name was successfully set, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

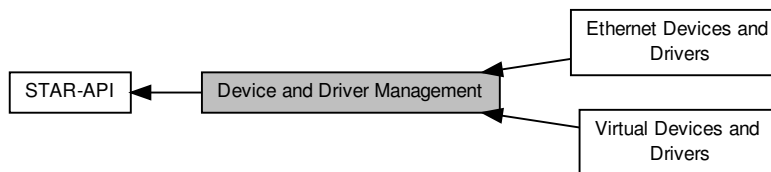
Examples:

`api_test_app.c`, `application_name_example.c`, `link_events.c`, `star-test.c`, `star_performance_tester.c`, `time-code_test.c`, and `timestamp_test.c`.

8.3 Device and Driver Management

Functions providing information on devices and drivers present.

Collaboration diagram for Device and Driver Management:



Modules

- **Virtual Devices and Drivers**

These are functions that are used to manage the properties of virtual drivers and devices.

- **Ethernet Devices and Drivers**

These are functions that are used to manage Ethernet drivers and devices.

Macros

- **#define STAR_DEVICE_TYPE U32**
Type of device that a device is.
- **#define STAR_DEVICE_SERIAL_SIZE 16**
Size in bytes (including null terminator) of a device serial number.
- **#define STAR_CHANNEL_MASK U32**
Type used to represent channels that exist on a device.
- **#define STAR_DEVICE_UNKNOWN 0U**
The "unknown" device type.
- **#define STAR_DEVICE_PCI 1U**
The STAR-Dundee SpaceWire PCI-2 and cPCI device type.
- **#define STAR_DEVICE_ROUTER_USB 2U**
The STAR-Dundee SpaceWire Router-USB device type.
- **#define STAR_DEVICE_USB_BRICK 3U**
The STAR-Dundee SpaceWire-USB Brick device type.
- **#define STAR_DEVICE_LINK_ANALYSER 4U**
The STAR-Dundee SpaceWire Link Analyser device type.
- **#define STAR_DEVICE_CONFORMANCE_TESTER 5U**
The STAR-Dundee SpaceWire Conformance Tester device type.
- **#define STAR_DEVICE_IP_TUNNEL 6U**
The STAR-Dundee SpaceWire IP-Tunnel device type.
- **#define STAR_DEVICE_ROUTER_MK2S 7U**
The STAR-Dundee SpaceWire Router Mk2S device type.
- **#define STAR_DEVICE_PCI_MK2 8U**
The STAR-Dundee SpaceWire PCI Mk2 device type.
- **#define STAR_DEVICE_BRICK_MK2 9U**

- The STAR-Dundee SpaceWire Brick Mk2 device type.*

 - #define `STAR_DEVICE_LINK_ANALYSER_MK2` 10U
- The STAR-Dundee SpaceWire Link Analyser Mk2 device type.*

 - #define `STAR_DEVICE_PCIE` 11U
- The STAR-Dundee SpaceWire PCI Express device type.*

 - #define `STAR_DEVICE_RTC` 12U
- The STAR-Dundee SpaceWire RTC device type.*

 - #define `STAR_DEVICE_EGSE` 13U
- The STAR-Dundee SpaceWire EGSE device type.*

 - #define `STAR_DEVICE_CPCI_MK2` 14U
- The STAR-Dundee SpaceWire cPCI Mk2 device type.*

 - #define `STAR_DEVICE_SPLT` 15U
- The STAR-Dundee SpaceWire Physical Layer Tester device type.*

 - #define `STAR_DEVICE_STAR_FIRE` 16U
- The STAR-Dundee STAR Fire device type.*

 - #define `STAR_DEVICE_WBS_II` 17U
- The STAR-Dundee Wide Band Spectrometer II device type.*

 - #define `STAR_DEVICE_RECORDER` 18U
- The STAR-Dundee SpaceWire Recorder device type.*

 - #define `STAR_DEVICE_BRICK_MK3` 19U
- The STAR-Dundee SpaceWire Brick Mk3 device type.*

 - #define `STAR_DEVICE_TRAFFIC_GENERATOR` 20U
- The Shimafuji Traffic Generator device type.*

 - #define `STAR_DEVICE_PXI_INTERFACE` 21U
- The STAR-Dundee SpaceWire PXI Interface device type.*

 - #define `STAR_DEVICE_PXI_RMAP` 22U
- The STAR-Dundee SpaceWire PXI RMAP device type.*

 - #define `STAR_DEVICE_PXI_ROUTER_8` 23U
- The STAR-Dundee SpaceWire PXI 8 port Router device type.*

 - #define `STAR_DEVICE_PXI_ROUTER_12` 24U
- The STAR-Dundee SpaceWire PXI 12 port Router device type.*

 - #define `STAR_DEVICE_SPFIPCI` 25U
- The STAR-Dundee SpaceFibre PCI Express device type.*

 - #define `STAR_DEVICE_STAR_FIRE_MK3` 26U
- The STAR-Dundee STAR Fire Mk3 device type.*

 - #define `STAR_DEVICE_PCI_MK3` 27U
- The STAR-Dundee SpaceWire PCI Mk3 device type.*

 - #define `STAR_DEVICE_LINK_ANALYSER_MK3` 28U
- The STAR-Dundee SpaceWire Link Analyser Mk3 device type.*

 - #define `STAR_DEVICE_CONFORMANCE_TESTER_MK2` 29U
- The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.*

 - #define `STAR_DEVICE_EGSE_MK2` 30U
- The STAR-Dundee SpaceWire EGSE Mk2 device type.*

 - #define `STAR_DEVICE_GBE_BRICK` 31U
- The STAR-Dundee SpaceWire GbE Brick device type.*

 - #define `STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE` 32U
- The STAR-Dundee STAR-Ultra PCIe Interface device type.*

 - #define `STAR_DEVICE_PXI_INTERFACE_MK2` 33U
- The STAR-Dundee SpaceWire PXI Interface Mk2 device type.*

 - #define `STAR_DEVICE_PXI_RMAP_MK2` 34U
- The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.*

- `#define STAR_DEVICE_PXI_ROUTER_MK2 35U`
The STAR-Dundee SpaceWire PXI Router Mk2 device type.
- `#define STAR_DEVICE_BRICK_MK4 36U`
The STAR-Dundee SpaceWire Brick Mk4 device type.
- `#define STAR_DEVICE_RECORDER_MK2 37U`
The STAR-Dundee SpaceWire Recorder Mk2 device type.
- `#define STAR_DEVICE_PCIE_MK2 38U`
The STAR-Dundee SpaceWire PCIe Mk2 device type.
- `#define STAR_DEVICE_STAR_ULTRA_PCIE_SL_ROUTER 39U`
The STAR-Dundee STAR-Ultra PCIe Single-Lane Router device type.
- `#define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU`
A device which is capable of being configured.
- `#define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU`
A device which is not capable of being configured.
- `#define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU`
A device which is capable of transmitting and receiving packets.
- `#define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00ffffeU`
A device which is not capable of transmitting and receiving packets.
- `#define STAR_DEVICE_ALL 0x00fffffU`
All devices.

Typedefs

- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID`
Unique value used to identify an individual driver.
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID`
Unique value used to identify an individual device.
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID`
Unique value used to identify an individual device listener.
- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID`
Unique value used to identify an individual driver listener.
- `typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)`
The function type for driver listener functions which are called when a new driver is added, or an existing one removed.
- `typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListener↵ Identifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, int deviceAdded, void *p↵ ContextInfo)`
The function type for device listener functions which are called when a device is added or removed.

Enumerations

- `enum STAR_BUS_TYPE {`
`STAR_BUS_UNKNOWN = 0, STAR_BUS_PCI = 1, STAR_BUS_USB = 2, STAR_BUS_PCIE = 3,`
`STAR_BUS_TCP = 4, STAR_BUS_CPCI = 5, STAR_BUS_VIRTUAL = 255 }`
The different types of bus that can be used to connect a SpaceWire device to a PC.
- `enum STAR_DRIVER_TYPE {`
`STAR_DRIVER_INVALID = 0, STAR_DRIVER_TYPE_USB = 1, STAR_DRIVER_TYPE_PCI = 2, STAR_↵`
`DRIVER_TYPE_TCPIP = 4,`
`STAR_DRIVER_TYPE_VIRTUAL = 5, STAR_DRIVER_TYPE_ULTRA_PCIE = 6, STAR_DRIVER_TYPE_↵`
`_PEP_PCIE = 7 }`
Available driver types.

Functions

- `_Ret_opt_cap_ count STAR_DRIVER_ID *STAR_API_CC STAR_getDriverList (int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 *count)`
Gets an array of driver identifiers for all drivers of the requested type(s).
- `void STAR_API_CC STAR_destroyDriverList (_Post_ptr_invalid_ STAR_DRIVER_ID *pDriverList)`
Frees a driver list previously created by a call to `STAR_getDriverList()`.
- `_Check_return_ STAR_DRIVER_LISTENER_ID STAR_API_CC STAR_registerDriverListener (STAR_↔ DriverEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a Driver is added or removed.
- `int STAR_API_CC STAR_unregisterDriverListener (STAR_DRIVER_LISTENER_ID listenerId)`
Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.
- `STAR_VERSION_INFO *STAR_API_CC STAR_getDriverVersion (STAR_DRIVER_ID driverId)`
Gets the Version of a given driver.
- `STAR_DRIVER_TYPE STAR_API_CC STAR_getDriverType (STAR_DRIVER_ID driverId)`
Gets the type (USB/PCI/specific virtual type identifier) of a given driver.
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceList (_Out_ U32 *count)`
Gets an array of identifiers for all devices present for all drivers.
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForType (STAR_DEVICE_↔ E_TYPE deviceType, _Out_ U32 *pCount)`
Gets an array of identifiers for all devices present for the given device type.
- `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForTypes (STAR_DEVI_↔ CE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)`
Gets an array of identifiers for all devices present for the given device types in an array.
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForDriver (STAR_DRIVE_↔ R_ID driverId, _Out_ U32 *count)`
Gets an array of device identifiers for all devices present for a specific driver.
- `void STAR_API_CC STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)`
Frees a device list previously created by a call to `STAR_getDeviceList()` or `STAR_getDeviceListForDriver()`.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener (_In_ STA_↔ R_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a device is added or removed.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.
- `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice (STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)`
Registers a Call-back function that will be called whenever a specific device is removed.
- `int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)`
Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.
- `int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID deviceId)`
Resets a device.
- `STAR_VERSION_INFO *STAR_API_CC STAR_getDeviceFirmwareVersion (STAR_DEVICE_ID deviceId)`
Gets the version of a device's firmware.
- `STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceId)`
Gets the bus type (PCI, USB, Virtual, etc) for the device.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceId)`
Gets a string representation of a device's bus type.
- `int STAR_API_CC STAR_getDeviceIndex (STAR_DEVICE_ID deviceId)`
Gets the device index by Driver.

- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID deviceId)`
Gets the device's type identifier.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID deviceId)`
Gets a string representation of a device's type.
- `STAR_DRIVER_ID STAR_API_CC STAR_getDeviceDriver (STAR_DEVICE_ID deviceId)`
Gets the identifier of the device's driver.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceName (STAR_DEVICE_ID deviceId)`
Gets the name of the device identified by a given identifier.
- `int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID deviceId, _In_z_ char *newName)`
Sets the name of the device (friendly name).
- `int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID deviceId)`
Resets a device's name to its default value.
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceSerialNumber (STAR_DEVICE_ID deviceId)`
Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities (STAR_DEVICE_ID deviceId)`
Determine whether the device is capable of transmitting and receiving packets on channels.
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID deviceId)`
Determine whether the device is capable of being configured.
- `STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID deviceId)`
Gets the channels on a device.
- `STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`
Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.
- `STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`
Returns the id of a device attached to a given channel on a given device.

8.3.1 Detailed Description

Functions providing information on devices and drivers present.

8.3.2 Macro Definition Documentation

8.3.2.1 `#define STAR_CHANNEL_MASK U32`

Type used to represent channels that exist on a device.

(bit 0 set = channel 0 exists, bit 2 set = channel 2 exists etc...).

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`link_events.c`, `star-test.c`, `star_performance_tester.c`, and `ui.c`.

8.3.2.2 `#define STAR_DEVICE_ALL 0x00ffffffU`

All devices.

Added in version:

STAR-System v3.0 Beta 2 - 24th July 2015

Examples:

`star-test.c`.

8.3.2.3 `#define STAR_DEVICE_BRICK_MK2 9U`

The STAR-Dundee SpaceWire Brick Mk2 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

`brickMk2_configuration_example.c`, and `link_events.c`.

8.3.2.4 `#define STAR_DEVICE_BRICK_MK3 19U`

The STAR-Dundee SpaceWire Brick Mk3 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

`link_events.c`, `timestamp_test.c`, `triggering_example.c`, and `triggering_generic_example.c`.

8.3.2.5 `#define STAR_DEVICE_BRICK_MK4 36U`

The STAR-Dundee SpaceWire Brick Mk4 device type.

Added in version:

STAR-System v4.03 - 28th July 2020

Examples:

`link_events.c`, `timestamp_test.c`, `triggering_example.c`, and `triggering_generic_example.c`.

8.3.2.6 `#define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU`

A device which is not capable of being configured.

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

8.3.2.7 `#define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU`

A device which is capable of being configured.

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

Examples:

`device_identifier_example.c`, `mk2_configuration_example.c`, `port_control_status_example.c`, `router_↵
configuration_example.c`, `routing_table_example.c`, `star-test.c`, `time-code_test.c`, and `user_registers_↵
example.c`.

8.3.2.8 `#define STAR_DEVICE_CONFORMANCE_TESTER 5U`

The STAR-Dundee SpaceWire Conformance Tester device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.9 `#define STAR_DEVICE_CONFORMANCE_TESTER_MK2 29U`

The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.

Added in version:

STAR-System v4.00 Beta 3 - 24th October 2018

8.3.2.10 `#define STAR_DEVICE_CPCI_MK2 14U`

The STAR-Dundee SpaceWire cPCI Mk2 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

`link_events.c`.

8.3.2.11 `#define STAR_DEVICE_EGSE 13U`

The STAR-Dundee SpaceWire EGSE device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.12 `#define STAR_DEVICE_EGSE_MK2 30U`

The STAR-Dundee SpaceWire EGSE Mk2 device type.

Added in version:

STAR-System v4.00 Beta 3 - 24th October 2018

8.3.2.13 #define STAR_DEVICE_IP_TUNNEL 6U

The STAR-Dundee SpaceWire IP-Tunnel device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.14 #define STAR_DEVICE_LINK_ANALYSER 4U

The STAR-Dundee SpaceWire Link Analyser device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.15 #define STAR_DEVICE_LINK_ANALYSER_MK2 10U

The STAR-Dundee SpaceWire Link Analyser Mk2 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.16 #define STAR_DEVICE_LINK_ANALYSER_MK3 28U

The STAR-Dundee SpaceWire Link Analyser Mk3 device type.

Added in version:

STAR-System v4.00 Beta 3 - 24th October 2018

8.3.2.17 #define STAR_DEVICE_PCI 1U

The STAR-Dundee SpaceWire PCI-2 and cPCI device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.18 #define STAR_DEVICE_PCI_MK2 8U

The STAR-Dundee SpaceWire PCI Mk2 device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

[link_events.c](#), [packet_subsystem_example.c](#), and [pciMk2_configuration_example.c](#).

8.3.2.19 `#define STAR_DEVICE_PCIE 11U`

The STAR-Dundee SpaceWire PCI Express device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

[link_events.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.20 `#define STAR_DEVICE_PCIE_MK2 38U`

The STAR-Dundee SpaceWire PCIe Mk2 device type.

Added in version:

STAR-System v5.01 - 16th September 2022

Examples:

[link_events.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.21 `#define STAR_DEVICE_PXI_INTERFACE 21U`

The STAR-Dundee SpaceWire PXI Interface device type.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Examples:

[link_events.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.22 `#define STAR_DEVICE_PXI_INTERFACE_MK2 33U`

The STAR-Dundee SpaceWire PXI Interface Mk2 device type.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Examples:

[link_events.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.23 `#define STAR_DEVICE_PXI_RMAP 22U`

The STAR-Dundee SpaceWire PXI RMAP device type.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Examples:

[link_events.c](#), [rmap_target_example.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.24 #define STAR_DEVICE_PXI_RMAP_MK2 34U

The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Examples:

[link_events.c](#), [rmap_target_example.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.25 #define STAR_DEVICE_PXI_ROUTER_12 24U

The STAR-Dundee SpaceWire PXI 12 port Router device type.

Added in version:

STAR-System v3.5 - 17th March 2017

Examples:

[link_events.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.26 #define STAR_DEVICE_PXI_ROUTER_8 23U

The STAR-Dundee SpaceWire PXI 8 port Router device type.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

8.3.2.27 #define STAR_DEVICE_PXI_ROUTER_MK2 35U

The STAR-Dundee SpaceWire PXI Router Mk2 device type.

Added in version:

STAR-System v4.00 Beta 4 - 16th October 2019

Examples:

[link_events.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.2.28 #define STAR_DEVICE_RECORDER 18U

The STAR-Dundee SpaceWire Recorder device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.29 #define STAR_DEVICE_RECORDER_MK2 37U

The STAR-Dundee SpaceWire Recorder Mk2 device type.

Added in version:

[STAR-System v4.03 - 28th July 2020](#)

8.3.2.30 #define STAR_DEVICE_ROUTER_MK2S 7U

The STAR-Dundee SpaceWire Router Mk2S device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[brickMk2_configuration_example.c](#), [link_events.c](#), and [routerMk2S_configuration_example.c](#).

8.3.2.31 #define STAR_DEVICE_ROUTER_USB 2U

The STAR-Dundee SpaceWire Router-USB device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

8.3.2.32 #define STAR_DEVICE_RTC 12U

The STAR-Dundee SpaceWire RTC device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

8.3.2.33 #define STAR_DEVICE_SERIAL_SIZE 16

Size in bytes (including null terminator) of a device serial number.

Added in version:

[STAR-System v1.0 - 17th October 2011](#)

8.3.2.34 #define STAR_DEVICE_SPLT 15U

The STAR-Dundee SpaceWire Physical Layer Tester device type.

Added in version:

[STAR-System v3.0 Beta 1 - 31st March 2015](#)

Examples:

[link_events.c](#).

8.3.2.35 #define STAR_DEVICE_STAR_FIRE 16U

The STAR-Dundee STAR Fire device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.36 #define STAR_DEVICE_STAR_FIRE_MK3 26U

The STAR-Dundee STAR Fire Mk3 device type.

Added in version:

STAR-System v3.2 - 24th November 2016

Examples:

link_events.c.

8.3.2.37 #define STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE 32U

The STAR-Dundee STAR-Ultra PCIe Interface device type.

Added in version:

STAR-System v6.00 - 7th October 2024

Examples:

broadcast_message_example.c, and triggering_generic_example.c.

8.3.2.38 #define STAR_DEVICE_STAR_ULTRA_PCIE_SL_ROUTER 39U

The STAR-Dundee STAR-Ultra PCIe Single-Lane Router device type.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

8.3.2.39 #define STAR_DEVICE_TRAFFIC_GENERATOR 20U

The Shimafuji Traffic Generator device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.40 #define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00ffffeU

A device which is not capable of transmitting and receiving packets.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.41 #define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU

A device which is capable of transmitting and receiving packets.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Declaration changed in version:

STAR-System v3.0 Beta 2 - 24th July 2015

Examples:

star-test.c, and star_performance_tester.c.

8.3.2.42 #define STAR_DEVICE_TYPE U32

Type of device that a device is.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

brickMk2_configuration_example.c, link_events.c, packet_subsystem_example.c, pciMk2_configuration_↔
example.c, routerMk2S_configuration_example.c, star-test.c, timestamp_test.c, ui.c, and utilities.c.

8.3.2.43 #define STAR_DEVICE_UNKNOWN 0U

The "unknown" device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

star-test.c, triggering_example.c, and triggering_generic_example.c.

8.3.2.44 #define STAR_DEVICE_USB_BRICK 3U

The STAR-Dundee SpaceWire-USB Brick device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.2.45 #define STAR_DEVICE_WBS_II 17U

The STAR-Dundee Wide Band Spectrometer II device type.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

8.3.3 Typedef Documentation

8.3.3.1 typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID

Unique value used to identify an individual device.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.3.2 typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID

Unique value used to identify an individual device listener.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.3.3 typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceId, int deviceAdded, void *pContextInfo)

The function type for device listener functions which are called when a device is added or removed.

Parameters

	<i>deviceListenerIdentifier</i>	The device listener that this event corresponds to.
	<i>driverIdentifier</i>	The driver on which the device was added or removed.
	<i>deviceId</i>	The device which was added or removed.
	<i>deviceAdded</i>	Whether the device has been added (1) or removed (0).
in	<i>pContextInfo</i>	A pointer to context information that can be passed to the function.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.3.4 typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID

Unique value used to identify an individual driver.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.3.5 typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID

Unique value used to identify an individual driver listener.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.3.6 typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListenerIdentifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)

The function type for driver listener functions which are called when a new driver is added, or an existing one removed.

Parameters

	<i>driverListenerIdentifier</i>	The driver listener that this event corresponds to.
	<i>driverIdentifier</i>	The driver which has been added or removed.
	<i>driverAdded</i>	Whether the driver was added (1) or removed (0).
in	<i>pContextInfo</i>	A pointer to optional context information that can be passed to the function.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.4 Enumeration Type Documentation**8.3.4.1 enum STAR_BUS_TYPE**

The different types of bus that can be used to connect a SpaceWire device to a PC.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_BUS_UNKNOWN The bus type of a device is unknown.

STAR_BUS_PCI The PCI bus type.

STAR_BUS_USB The USB bus type.

STAR_BUS_PCIE The PCI Express (PCIe) bus type.

Added in version:

STAR-System v1.2 - 13th February 2012

STAR_BUS_TCP The TCP/IP bus type, used by Ethernet devices.

STAR_BUS_CPCI The cPCI bus type.

Added in version:

STAR-System v1.6 - 29th June 2012

STAR_BUS_VIRTUAL The bus type for virtual devices.

8.3.4.2 enum STAR_DRIVER_TYPE

Available driver types.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_DRIVER_INVALID Invalid.

STAR_DRIVER_TYPE_USB USB Driver.

STAR_DRIVER_TYPE_PCI PCI Driver.

STAR_DRIVER_TYPE_TCPIP TCP/IP Driver.

STAR_DRIVER_TYPE_VIRTUAL Virtual Driver.

STAR_DRIVER_TYPE_ULTRA_PCIE Ultra PCIe Driver.

STAR_DRIVER_TYPE_PEP_PCIE Processor Endpoint PCIe Driver.

8.3.5 Function Documentation

8.3.5.1 void **STAR_API_CC** STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID * *pDeviceList*)

Frees a device list previously created by a call to [STAR_getDeviceList\(\)](#) or [STAR_getDeviceListForDriver\(\)](#).

Parameters

<i>pDeviceList</i>	the device list to be freed
--------------------	-----------------------------

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_send_example.c, device_identifier_example.c, driver_list_example.c, firmware_version_example.c, mk2_configuration_example.c, port_control_status_example.c, rmap_target_example.c, router_configuration_example.c, routing_table_example.c, simple_send_example.c, star-test.c, star_performance_tester.c, time-code_test.c, timestamp_test.c, transmit_errors_example.c, triggering_example.c, triggering_generic_example.c, ui.c, user_registers_example.c, and utilities.c.

8.3.5.2 void STAR_API_CC STAR_destroyDriverList (_Post_ptr_invalid_ STAR_DRIVER_ID * pDriverList)

Frees a driver list previously created by a call to STAR_getDriverList().

Parameters

<i>pDriverList</i>	the driver list to be freed
--------------------	-----------------------------

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

utilities.c.

8.3.5.3 STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceld)

Gets the bus type (PCI,USB, Virtual, etc) for the device.

Parameters

<i>deviceld</i>	the identifier of the device to get the bus type of
-----------------	---

Returns

1 if the driver is virtual, otherwise 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.4 _Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceld)

Gets a string representation of a device's bus type.

The string returned should be freed by calling STAR_destroyString().

Parameters

<i>deviceld</i>	the identifier of the device to get the bus type of
-----------------	---

Returns

the device's bus type as a null terminated string

Added in version:

STAR-System v1.0 - 17th October 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

star-test.c.

8.3.5.5 **STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID *deviceld*)**

Gets the channels on a device.

Parameters

<i>deviceld</i>	The device to check
-----------------	---------------------

Returns

A bit mask representing the channels on the device

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

link_events.c, star-test.c, star_performance_tester.c, ui.c, and utilities.c.

8.3.5.6 **STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID *deviceld*)**

Determine whether the device is capable of being configured.

This function can be used to determine whether a device can be configured using the Configuration APIs ([STAR_DEVICE_CONFIG_SUPPORTED](#)), or whether it is a special device such as a Link Analyser or Conformance Tester ([STAR_DEVICE_CONFIG_NOT_SUPPORTED](#)) which does not have a router with a configuration port.

Parameters

<i>deviceld</i>	the identifier of the device to get whether the device can be configured
-----------------	--

Returns

a value indicating whether the device is capable of being configured ([STAR_DEVICE_CONFIG_SUPPORTED](#)), not capable ([STAR_DEVICE_CONFIG_NOT_SUPPORTED](#)), or 0 if an error occurred

Added in version:

STAR-System v3.0 Beta 9 - 26th July 2016

Examples:

star-test.c.

8.3.5.7 **STAR_DRIVER_ID** **STAR_API_CC** **STAR_getDeviceDriver** (**STAR_DEVICE_ID** *deviceld*)

Gets the identifier of the device's driver.

Parameters

<i>deviceld</i>	the identifier of the device to get the driver of
-----------------	---

Returns

the driver for the device

Added in version:

STAR-System v1.12 - 14th November 2012

8.3.5.8 **STAR_VERSION_INFO*** **STAR_API_CC** **STAR_getDeviceFirmwareVersion** (**STAR_DEVICE_ID** *deviceld*)

Gets the version of a device's firmware.

Parameters

<i>deviceld</i>	Identifier for device to be checked
-----------------	-------------------------------------

Returns

Pointer to a version structure that will contain the device's version info, or NULL if a memory allocation error occurred.

Note

This structure must be freed using **STAR_destroyVersionInfo()** when no longer required.

Added in version:

STAR-System v1.0 - 17th October 2011

Examples:

`firmware_version_example.c`, and `star-test.c`.

8.3.5.9 **int** **STAR_API_CC** **STAR_getDeviceIndex** (**STAR_DEVICE_ID** *deviceld*)

Gets the device index by Driver.

Provided for backwards compatibility reasons, not for typical usage.

Parameters

<i>deviceld</i>	The device to check
-----------------	---------------------

Returns

The index number of the device, or -1 if the function call was unsuccessful

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`star-test.c`.

8.3.5.10 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceList ( _Out_ U32 * count )`

Gets an array of identifiers for all devices present for all drivers.

Parameters

<i>out</i>	<i>count</i>	Count of device identifiers held in the array.
------------	--------------	--

Returns

Array of identifiers for all devices present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[firmware_version_example.c](#), [star-test.c](#), and [utilities.c](#).

8.3.5.11 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForDriver ( STAR_DRIVER_ID driverId, _Out_ U32 * count )`

Gets an array of device identifiers for all devices present for a specific driver.

Parameters

	<i>driverId</i>	ID of driver to get devices for, obtained from a call to STAR_getDriverList()
<i>out</i>	<i>count</i>	Count of device identifiers held in the array.

Returns

Array of identifiers for devices for the given driver.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDeviceList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.12 `_Ret_opt_cap_pCount STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForType ( STAR_DEVICE_TYPE deviceType, _Out_ U32 * pCount )`

Gets an array of identifiers for all devices present for the given device type.

The device type may be the type of a specific device, e.g. [STAR_DEVICE_BRICK_MK3](#), or it may be one of the special device types, [STAR_DEVICE_TXRX_SUPPORTED](#), [STAR_DEVICE_TXRX_NOT_SUPPORTED](#), [STAR_DEVICE_CONFIG_SUPPORTED](#) or [STAR_DEVICE_CONFIG_NOT_SUPPORTED](#) to get the list of all devices which can or cannot transmit and receive packets or be configured, or [STAR_DEVICE_ALL](#) to get all devices.

Parameters

	<i>deviceType</i>	The type of device to get the list for
out	<i>pCount</i>	A pointer to a variable which will be updated to include the count of device identifiers held in the returned array.

Returns

Array of identifiers for all devices of the given type present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_destroyDeviceList()` when no longer required.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

`device_identifier_example.c`, `mk2_configuration_example.c`, `port_control_status_example.c`, `rmap_target_example.c`, `router_configuration_example.c`, `routing_table_example.c`, `star-test.c`, `star_performance_tester.c`, `time-code_test.c`, `user_registers_example.c`, and `utilities.c`.

8.3.5.13 `_Ret_opt_cap_ pCount STAR_DEVICE_ID* STAR_API_CC STAR_getDeviceListForTypes ( STAR_DEVICE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 * pCount )`

Gets an array of identifiers for all devices present for the given device types in an array.

Each device type may be the type of a specific device, e.g. `STAR_DEVICE_BRICK_MK3`, or it may be one of the special device types, `STAR_DEVICE_TXRX_SUPPORTED`, `STAR_DEVICE_TXRX_NOT_SUPPORTED`, `STAR_DEVICE_CONFIG_SUPPORTED` or `STAR_DEVICE_CONFIG_NOT_SUPPORTED`, to get the list of all devices which can or cannot transmit and receive packets or be configured, or `STAR_DEVICE_ALL` to get all devices.

Parameters

	<i>deviceTypes</i>	An array of device types to get the list for
	<i>numRequestedTypes</i>	The number of entries in the deviceTypes array
out	<i>pCount</i>	A pointer to a variable which will be updated to include the count of device identifiers held in the returned array.

Returns

Array of identifiers for all devices of the given type present.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_destroyDeviceList()` when no longer required.

Added in version:

STAR-System v3.0 Beta 3 - 4th September 2015

Examples:

`timestamp_test.c`, `triggering_example.c`, `triggering_generic_example.c`, and `ui.c`.

8.3.5.14 `_Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceName ( STAR_DEVICE_ID deviceld )`

Gets the name of the device identified by a given identifier.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the device name of
-----------------	--

Returns

the device's name as a null terminated string

Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

Declaration changed in version:

[STAR-System v1.1](#) - 20th December 2011

Examples:

[device_identifier_example.c](#), [mk2_configuration_example.c](#), [port_control_status_example.c](#), [router_configuration_example.c](#), [routing_table_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_test.c](#), [timestamp_test.c](#), [triggering_example.c](#), [triggering_generic_example.c](#), [ui.c](#), [user_registers_example.c](#), and [utilities.c](#).

8.3.5.15 `_Check_return_ _Ret_opt_z_ char* STAR_API_CC STAR_getDeviceSerialNumber ( STAR_DEVICE_ID deviceld )`

Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the serial number of
-----------------	--

Returns

the device's serial number as a null terminated string

Added in version:

[STAR-System v1.0](#) - 17th October 2011

Declaration changed in version:

[STAR-System v1.1](#) - 20th December 2011

Examples:

[star-test.c](#), [timestamp_test.c](#), [triggering_example.c](#), [triggering_generic_example.c](#), and [utilities.c](#).

8.3.5.16 `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities ( STAR_DEVICE_ID deviceld )`

Determine whether the device is capable of transmitting and receiving packets on channels.

This function can be used to determine whether a device can be used to route packets over channels and in and out of the device's SpaceWire links ([STAR_DEVICE_TXRX_SUPPORTED](#)), or whether it is a special device such as

a Link Analyser or Conformance Tester (`STAR_DEVICE_TXRX_NOT_SUPPORTED`) which cannot route packets. Note that some devices, such as the EGSE may be capable of transmitting and receiving packets, but a value of `STAR_DEVICE_TXRX_NOT_SUPPORTED` will be returned, as these devices do not simply route any packets transmitted or received on channels, and must be specially configured to transmit or receive packets.

Parameters

<i>deviceld</i>	the identifier of the device to get whether the device can transmit and receive
-----------------	---

Returns

a value indicating whether the device is capable of transmitting and receiving packets ([STAR_DEVICE_TX↔RX_SUPPORTED](#)), not capable ([STAR_DEVICE_TXRX_NOT_SUPPORTED](#)), or 0 if an error occurred

Added in version:

STAR-System v3.0 Beta 2 - 24th July 2015

8.3.5.17 STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID *deviceld*)

Gets the device's type identifier.

Parameters

<i>deviceld</i>	the identifier of the device to get the type of
-----------------	---

Returns

the type of the device

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[brickMk2_configuration_example.c](#), [link_events.c](#), [pciMk2_configuration_example.c](#), [routerMk2S_configuration↔_example.c](#), [timestamp_test.c](#), [triggering_example.c](#), and [triggering_generic_example.c](#).

8.3.5.18 _Check_return_ Ret_opt_z_char* STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID *deviceld*)

Gets a string representation of a device's type.

The string returned should be freed by calling [STAR_destroyString\(\)](#).

Parameters

<i>deviceld</i>	the identifier of the device to get the type of
-----------------	---

Returns

the device's type as a null terminated string

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[link_events.c](#), and [star-test.c](#).

8.3.5.19 `_Ret_opt_cap_count STAR_DRIVER_ID* STAR_API_CC STAR_getDriverList ( int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 * count )`

Gets an array of driver identifiers for all drivers of the requested type(s).

Parameters

<i>physicalDeviceDrivers</i>	Whether to include physical device drivers in the list
<i>virtualDeviceDrivers</i>	Whether to include virtual device drivers in the list
<i>out</i> <i>count</i>	Count of driver identifiers held in the array.

Returns

Array of identifiers for drivers.

Note

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using [STAR_destroyDriverList\(\)](#) when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[driver_list_example.c](#).

8.3.5.20 STAR_DRIVER_TYPE STAR_API_CC STAR_getDriverType (STAR_DRIVER_ID *driverId*)

Gets the type (USB/PCI/specific virtual type identifier) of a given driver.

Parameters

<i>driverId</i>	Identifier for driver to be checked
-----------------	-------------------------------------

Returns

Type of the driver

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[driver_list_example.c](#).

8.3.5.21 STAR_VERSION_INFO* STAR_API_CC STAR_getDriverVersion (STAR_DRIVER_ID *driverId*)

Gets the Version of a given driver.

Parameters

<i>driverId</i>	ID of driver to get version for, obtained from a call to STAR_getDriverList()
-----------------	---

Returns

Pointer to a version structure that will contain specific drivers version info, or NULL if there was an error.

Note

This structure must be freed using `STAR_destroyVersionInfo()` when no longer required.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.22 `STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel ( STAR_DEVICE_ID deviceld, unsigned int channelNumber )`

Returns the id of a device attached to a given channel on a given device.

Parameters

<i>deviceld</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

The attached device, or 0 if no device is attached

Added in version:

STAR-System v1.1 - 20th December 2011

8.3.5.23 `STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen ( STAR_DEVICE_ID deviceld, unsigned int channelNumber )`

Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.

Parameters

<i>deviceld</i>	The device to check
<i>channelNumber</i>	The channel number to check

Returns

Whether the channel is connected a device (`STAR_CHANNEL_TYPE_DEVICE`) or application (`STAR_CHANNEL_TYPE_APPLICATION`) if the channel is open, or not open (`STAR_CHANNEL_TYPE_NOT_OPEN`) if the channel is not open

Added in version:

STAR-System v1.1 - 20th December 2011

8.3.5.24 `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener ( _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a device is added or removed.

Parameters

<i>in</i>	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing devices to indicate that they've been added

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.25 `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice ( STAR_DEVICE_ID deviceld, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void * pContextInfo )`

Registers a Call-back function that will be called whenever a specific device is removed.

Parameters

	<i>deviceld</i>	Device to register device listener for
	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional A pointer to some context information that will be passed to the listener function when it is called

Returns

Non-Zero if call-back was registered successfully

8.3.5.26 `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver ( STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.

Parameters

	<i>driverId</i>	Driver to register device listener for
	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing devices to indicate that they've been added

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.27 `_Check_return_ STAR_DRIVER_LISTENER_ID STAR_API_CC STAR_registerDriverListener ( STAR_DriverEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a Driver is added or removed.

Parameters

	<i>listenerFunc</i>	Call-back Function
<i>in</i>	<i>pContextInfo</i>	Optional pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing drivers to indicate that they've been added

Returns

1 if call-back was registered successfully, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.28 int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID *deviceld*)

Resets a device.

This function will return an error if called with a Virtual Device's identifier.

Note

Calling this function while packets are being transmitted and/or received can cause the device to get in to a bad state.

Parameters

<i>deviceld</i>	Device to be reset
-----------------	--------------------

Returns

1 if the device was successfully reset, else 0

Added in version:

STAR-System v1.0 - 17th October 2011

Examples:

star-test.c.

8.3.5.29 int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID *deviceld*)

Resets a device's name to its default value.

Parameters

<i>deviceld</i>	Device to have it's name reset
-----------------	--------------------------------

Returns

1 if the device's name was successfully reset, else 0

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

8.3.5.30 int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID *deviceld*, _In_z_ char * *newName*)

Sets the name of the device (friendly name).

Parameters

	<i>deviceId</i>	The device to set a name for
<i>in</i>	<i>newName</i>	The new name for the device (newName must be null terminated and shorter than 256 characters). If newName is NULL then the device name is reset to the default.

Returns

1 if the name could be set, otherwise 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.31 int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)

Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.

Parameters

<i>listenerId</i>	Device Listener ID returned from a previous call to STAR_registerDeviceListener
-------------------	---

Returns

1 if call-back was unregistered successfully, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.3.5.32 int STAR_API_CC STAR_unregisterDriverListener (STAR_DRIVER_LISTENER_ID listenerId)

Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.

Parameters

<i>listenerId</i>	Driver Listener ID returned from a previous call to STAR_registerDriverListener
-------------------	---

Returns

1 if call-back was unregistered successfully

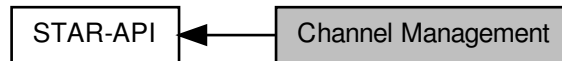
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.4 Channel Management

Used to open and close channels, between applications and devices.

Collaboration diagram for Channel Management:



Typedefs

- typedef `CFG_INFO_ID_TYPE STAR_CHANNEL_ID`
Unique value used to identify an open channel.
- typedef `CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`
Unique value used to identify an individual channel listener.
- typedef `void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channelIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdentifier, STAR_CHANNEL_ID channelIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`
The function type for channel listener functions which are called when a device channel is opened or closed.

Enumerations

- enum `STAR_CHANNEL_TYPE { STAR_CHANNEL_TYPE_NOT_OPEN, STAR_CHANNEL_TYPE_DEVICE, STAR_CHANNEL_TYPE_APPLICATION }`
Channel types.
- enum `STAR_CHANNEL_DIRECTION { STAR_CHANNEL_DIRECTION_IN = 1, STAR_CHANNEL_DIRECTION_OUT = 2, STAR_CHANNEL_DIRECTION_INOUT }`
Channel directions.

Functions

- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices (STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB)`
Open a channel between two devices on the specified channel numbers.
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice (STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued)`
Open a channel to a device on the specified channel number.
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx (_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)`
This function is an alternative to `STAR_openChannelToLocalDevice()` that uses an optimised receive strategy.
- `int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)`
Close a previously opened channel.
- `_Check_return_ STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection (STAR_CHANNEL_ID channelId)`
Get the direction information of an open channel.

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener (_In_↔ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever any channel is opened or closed.
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerFor↔ Device (STAR_DEVICE_ID deviceId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *p↔ ContextInfo, int notifyForCurrent)`
Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.
- `int STAR_API_CC STAR_unregisterChannelListener (STAR_CHANNEL_LISTENER_ID listenerId)`
Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.

8.4.1 Detailed Description

Used to open and close channels, between applications and devices.

8.4.2 Typedef Documentation

8.4.2.1 `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_ID`

Unique value used to identify an open channel.

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.2.2 `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`

Unique value used to identify an individual channel listener.

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.2.3 `typedef void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channelListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, STAR_CHANNEL_ID channelIdIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`

The function type for channel listener functions which are called when a device channel is opened or closed.

Parameters

<i>channel↔ ListenerIdentifier</i>	The channel listener that this event corresponds to.
<i>driverIdentifier</i>	The driver of the device on which the channel was opened or closed.
<i>deviceIdIdentifier</i>	The device on which the channel was opened or closed.

	<i>channelIdentifier</i>	The identifier of the channel which has been opened or closed.
	<i>channelOpened</i>	Whether the channel was opened (1) or closed (0).
	<i>channelNumber</i>	The channel number on the device of the channel which was opened or closed.
<i>in</i>	<i>pContextInfo</i>	A pointer to context information that can be passed to the function.

Note

The calling convention for this function type is `STAR_API_CC`

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.3 Enumeration Type Documentation**8.4.3.1 enum STAR_CHANNEL_DIRECTION**

Channel directions.

Added in version:

STAR-System v1.1 - 20th December 2011

Enumerator

STAR_CHANNEL_DIRECTION_IN Channel may be used to receive traffic.

STAR_CHANNEL_DIRECTION_OUT Channel may be used to transmit traffic.

STAR_CHANNEL_DIRECTION_INOUT Channel may be used to both receive and transmit traffic.

8.4.3.2 enum STAR_CHANNEL_TYPE

Channel types.

Added in version:

STAR-System v1.1 - 20th December 2011

Enumerator

STAR_CHANNEL_TYPE_NOT_OPEN Not open.

STAR_CHANNEL_TYPE_DEVICE Attached to a device.

STAR_CHANNEL_TYPE_APPLICATION Attached to an application.

8.4.4 Function Documentation**8.4.4.1 int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)**

Close a previously opened channel.

Parameters

<i>channelId</i>	Channel to close
------------------	------------------

Returns

1 if the channel was successfully closed, else 0. If a channel could not be closed, it was not open or has already been closed

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, broadcast_message_example.c, channel_callback_example.c, ex_receive_example.c, link_events.c, rmap_target_example.c, simple_receive_example.c, simple_send_and_receive_example.c, simple_send_example.c, simple_two_way_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

8.4.4.2 `_Check_return_STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection ( STAR_CHANNEL_ID channelID )`

Get the direction information of an open channel.

Parameters

<i>channelID</i>	Channel to get the direction information for
------------------	--

Returns

Direction information of the channel, or 0 if an error occurred

Added in version:

STAR-System v3.10 - 23rd February 2018

8.4.4.3 `_Check_return_STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices ( STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB )`

Open a channel between two devices on the specified channel numbers.

Parameters

<i>deviceA</i>	First device to attach to
<i>channelA</i>	Channel number on first device to attach to
<i>deviceB</i>	Second device to attach to
<i>channelB</i>	Channel number on second device to attach to

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.4.4 `_Check_return_STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice ( STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued )`

Open a channel to a device on the specified channel number.

When the channel is no longer required it should be closed by calling `STAR_closeChannel()`. Note that a channel can only be opened once in a particular direction, but it is possible, for example, to open a channel to receive, then open it again to transmit.

Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

The `isQueued` parameter determines whether received traffic is buffered when it is received. As well as buffering in the device, there is also buffering in the device driver, and if an application is open with a channel open to receive data, there is also buffering in STAR-API for this channel. As these buffers become full, the device will stop issuing credit, so the transmitting device can no longer transmit.

The `isQueued` parameter is a way to override this behaviour. When it is set to `FALSE`, packets will be dropped if there are no receive transfer operations waiting to receive them. If there are receive transfer operations waiting to receive them, packets will be received as normal.

The benefit of passing `FALSE` for the `isQueued` parameter is that the link shouldn't become blocked, as the device should always be issuing credit. The obvious disadvantage is that packets can be dropped, so this isn't a mode that is often used. In most circumstances it is recommended that `TRUE` is provided for this argument, although note that the argument will be ignored for transmit-only channels.

Parameters

<i>device</i>	the device to attach to
<i>direction</i>	whether this channel is for transmitting data from this application or receiving data into it, or both. Note that there is no performance penalty in opening a channel in both directions when only receiving, for example. It simply stops the channel from being opened for transmitting in another process or thread
<i>channelNumber</i>	the channel number on the device to attach to
<i>isQueued</i>	whether traffic received on this channel should be buffered if there is no receive op waiting to receive it

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

`advanced_address_example.c`, `advanced_continuous_receive_example.c`, `advanced_multiple_packet_example.c`, `advanced_send_and_receive_example.c`, `advanced_two_way_example.c`, `brickMk2_configuration_example.c`, `broadcast_message_example.c`, `channel_callback_example.c`, `link_events.c`, `rmap_target_example.c`, `simple_send_and_receive_example.c`, `simple_two_way_example.c`, `star-test.c`, `star_performance_tester.c`, `time-code_example.c`, `time-code_test.c`, `timestamp_test.c`, `transfer_callback_example.c`, `transfer_operation_list_send_example.c`, `transfer_operation_list_send_receive_example.c`, `ui.c`, and `utilities.c`.

8.4.4.5 `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx ( _In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued )`

This function is an alternative to `STAR_openChannelToLocalDevice()` that uses an optimised receive strategy.

Instead of creating dynamically allocated packets, data is copied directly into pre-allocated buffers.

To utilise the optimised receive strategy, the channel should be used with receive transfer operations created with the `STAR_createRxOperationEx()` function.

Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

When the channel is no longer required it should be closed by calling `STAR_closeChannel()`. Note that a channel can only be opened once in a particular direction, but it is possible, for example, to open a channel to receive, then open it again to transmit.

The `isQueued` parameter determines whether received traffic is buffered when it is received. As well as buffering in the device, there is also buffering in the device driver, and if an application is open with a channel open to receive data, there is also buffering in STAR-API for this channel. As these buffers become full, the device will stop issuing credit, so the transmitting device can no longer transmit.

The `isQueued` parameter is a way to override this behaviour. When it is set to `FALSE`, packets will be dropped if there are no receive transfer operations waiting to receive them. If there are receive transfer operations waiting to receive them, packets will be received as normal.

The benefit of passing `FALSE` for the `isQueued` parameter is that the link shouldn't become blocked, as the device should always be issuing credit. The obvious disadvantage is that data can be dropped, so this isn't a mode that is often used. In most circumstances it is recommended that `TRUE` is provided for this argument, although note that the argument will be ignored for transmit-only channels.

Parameters

<i>device</i>	the device to attach to
<i>direction</i>	whether this channel is for transmitting data from this application or receiving data into it, or both. Note that there is no performance penalty in opening a channel in both directions when only receiving, for example. It simply stops the channel from being opened for transmitting in another process or thread
<i>channelNumber</i>	the channel number on the device to attach to
<i>isQueued</i>	whether traffic received on this channel should be buffered if there is no receive op waiting to receive it

Returns

Channel identifier of new channel or 0 if an error occurred

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

Examples:

[ex_receive_example.c](#).

8.4.4.6 `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener ( _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever any channel is opened or closed.

Parameters

	<i>listenerFunc</i>	Call-back Function
in	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[channel_callback_example.c](#).

8.4.4.7 `_Check_return_STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDevice ( STAR_DEVICE_ID deviceld, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.

Parameters

	<i>deviceld</i>	Device to register channel listener for
	<i>listenerFunc</i>	Call-back Function
in	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.4.8 `_Check_return_STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver ( STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void * pContextInfo, int notifyForCurrent )`

Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.

Parameters

	<i>driverId</i>	Driver to register channel listener for
	<i>listenerFunc</i>	Call-back Function

<i>in</i>	<i>pContextInfo</i>	A pointer to some context information that will be passed to the listener function when it is called
	<i>notifyForCurrent</i>	Whether the function should be called for all existing channels to indicate that they've been opened

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v1.1 - 20th December 2011

8.4.4.9 int STAR_API_CC STAR_unregisterChannelListener (STAR_CHANNEL_LISTENER_ID *listenerId*)

Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.

Parameters

<i>listenerId</i>	Channel Listener ID returned from a previous call to a registerChannelListener function
-------------------	---

Returns

1 if call-back was unregistered successfully, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

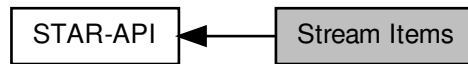
Examples:

channel_callback_example.c.

8.5 Stream Items

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

Collaboration diagram for Stream Items:



Data Structures

- struct [STAR_IP_ADDRESS_TYPE](#)
A structure consisting of: [More...](#)
- struct [STAR_LINK_STATE_EVENT](#)
Events that can occur which affect the state of the SpaceWire link. [More...](#)
- struct [STAR_SPACEWIRE_ADDRESS](#)
A structure used to describe a SpaceWire address. [More...](#)
- struct [STAR_DATA_CHUNK](#)
A structure used to describe a chunk of contiguous SpaceWire data, within a single packet. [More...](#)
- struct [STAR_SPACEWIRE_PACKET](#)
A structure used to describe a SpaceWire packet. [More...](#)
- struct [STAR_TIMECODE](#)
A structure used to describe a SpaceWire time-code. [More...](#)
- struct [STAR_LINK_SPEED_EVENT](#)
A structure used to describe a link speed change event. [More...](#)
- struct [STAR_ERROR_IN_DATA_INJECT](#)
A structure used to describe an error in data event. [More...](#)
- struct [STAR_TIMESTAMP_EVENT](#)
A structure used to describe a timestamp on receipt of data on the link. [More...](#)
- struct [STAR_BROADCAST_MESSAGE](#)
A structure used to describe a SpaceFibre broadcast message. [More...](#)
- struct [STAR_STREAM_ITEM](#)
A wrapper for Stream Item objects. [More...](#)

Enumerations

- enum [STAR_IP_VERSION_TYPE](#) { [STAR_IP_VERSION_NOT_DEFINED](#) = 0, [STAR_IPv4](#) = 4, [STAR_IPv6](#) = 16 }
The Internet Protocol version.
- enum [STAR_STREAM_ITEM_TYPE](#) {
[STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET](#), [STAR_STREAM_ITEM_TYPE_TIMECODE](#), [STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT](#), [STAR_STREAM_ITEM_TYPE_DATA_CHUNK](#),
[STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT](#), [STAR_STREAM_ITEM_TYPE_ERROR_INJECT](#),
[STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT](#), [STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE](#) }

The kinds of object that are represented as stream items.

- enum `STAR_EOP_TYPE` { `STAR_EOP_TYPE_INVALID`, `STAR_EOP_TYPE_EOP`, `STAR_EOP_TYPE_EEP`, `STAR_EOP_TYPE_NONE` }

The kinds of End of packet marker that may be present.

- enum `STAR_ERROR_IN_DATA_TYPE` { `STAR_ERROR_IN_DATA_PARITY`, `STAR_ERROR_IN_DATA_DISCONNECT` }

The types of error that may be injected in to data in a packet.

- enum `STAR_TIMESTAMP_TYPE` { `STAR_TIMESTAMP_TYPE_DATA`, `STAR_TIMESTAMP_TYPE_LINK_STATE`, `STAR_TIMESTAMP_TYPE_LINK_SPEED`, `STAR_TIMESTAMP_TYPE_TIMECODE`, `STAR_TIMESTAMP_TYPE_ERROR` }

An enum used to define different types of timestamps that can be received.

- enum `STAR_TIMESTAMP_DIRECTION` { `STAR_TIMESTAMP_DIRECTION_TRANSMIT`, `STAR_TIMESTAMP_DIRECTION_RECEIVE` }

An enum used to define the direction of a timestamp.

- enum `STAR_BROADCAST_MESSAGE_STATUS_FLAG` { `STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE` = 0x1, `STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED` = 0x2 }

An enum used to define the status flags of a broadcast message.

Functions

- void `STAR_API_CC STAR_destroyStreamItem` (`_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item`)
Disposes of a given stream item.
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createPacket` (`_In_opt_ STAR_SPACEWIRE_ADDRESS *address`, `_In_opt_count_ (dataLen) unsigned char *data`, `unsigned int dataLen`, `STAR_EOP_TYPE eopType`)
Creates a new SpaceWire packet stream item from user provided data.
- `_Check_return_ STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_createAddress` (`_In_count_ (pathLen) unsigned char *path`, `U16 pathLen`)
Creates a SpaceWire address.
- void `STAR_API_CC STAR_destroyAddress` (`_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address`)
Disposes of a SpaceWire address created with `STAR_createAddress()`.
- int `STAR_API_CC STAR_setPacketAddress` (`_Inout_ STAR_SPACEWIRE_PACKET *packet`, `_In_ STAR_SPACEWIRE_ADDRESS *address`)
Updates the address of a given packet.
- `_Ret_opt_bytecap_x_ dataLength unsigned char *STAR_API_CC STAR_getPacketData` (`_In_ STAR_SPACEWIRE_PACKET *packet`, `_Out_ unsigned int *dataLength`)
Gets the packet's data as an array of bytes.
- void `STAR_API_CC STAR_destroyPacketData` (`_In_ _Post_ptr_invalid_ unsigned char *pData`)
Frees packet data previously created by a call to `STAR_getPacketData()`.
- `STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_getPacketAddress` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets a copy of a packet's address structure.
- unsigned int `STAR_API_CC STAR_getPacketLength` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets the length of a given packet.
- `STAR_EOP_TYPE STAR_API_CC STAR_getPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`)
Gets the end of packet marker type of a packet.
- int `STAR_API_CC STAR_setPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`, `STAR_EOP_TYPE eopType`)
Sets the end of packet marker type of a packet.
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createDataChunk` (`_In_opt_count_ (dataLen) unsigned char *data`, `unsigned int dataLen`, `int isStart`, `STAR_EOP_TYPE eopType`)

Creates a data chunk stream item, used to refer to part of a packet.

- unsigned char *STAR_API_CC STAR_getChunkData (_In_ STAR_DATA_CHUNK *chunk, _Out_ unsigned int *len)

Gets a pointer to a data chunk stream item's data.

- unsigned int STAR_API_CC STAR_getChunkDataLength (_In_ STAR_DATA_CHUNK *chunk)

Gets the length of a data chunk stream item's data.

- STAR_EOP_TYPE STAR_API_CC STAR_getChunkEop (_In_ STAR_DATA_CHUNK *chunk)

Gets the End Of Packet marker type of a data chunk stream item.

- int STAR_API_CC STAR_getChunkSop (_In_ STAR_DATA_CHUNK *chunk)

Gets whether a data chunk stream item is at the start of a packet.

- STAR_STREAM_ITEM *STAR_API_CC STAR_createTimeCode (U8 value)

Creates a Time-code stream item.

- U8 STAR_API_CC STAR_getTimeCodeValue (_In_ STAR_TIMECODE *pTimeCode)

Gets the value of a given time-code stream item.

- void STAR_API_CC STAR_setTimeCodeValue (_In_ STAR_TIMECODE *pTimeCode, U8 value)

Sets the value of a given time-code stream item.

- U16 STAR_API_CC STAR_getTimeCodeCount (_In_ STAR_TIMECODE *pTimeCode)

Gets the count of a given time-code stream item.

- STAR_STREAM_ITEM *STAR_API_CC STAR_createErrorInData (STAR_ERROR_IN_DATA_TYPE errorType)

Creates an error in data stream item.

- U8 STAR_API_CC STAR_getLinkStateEventCount (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets the count of a given link state event stream item, indicating the number of events that have occurred.

- U8 STAR_API_CC STAR_getLinkStateEventPort (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets the port related to a given link state change event stream item.

- int STAR_API_CC STAR_isLinkStateEventDisconnectError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes a disconnect error.

- int STAR_API_CC STAR_isLinkStateEventEscapeError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes an escape error.

- int STAR_API_CC STAR_isLinkStateEventLinkRunning (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes a link running state change.

- int STAR_API_CC STAR_isLinkStateEventParityError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes a parity error.

- int STAR_API_CC STAR_isLinkStateEventReceiveCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes a receive credit error.

- int STAR_API_CC STAR_isLinkStateEventTransmitCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)

Gets whether a link state event stream item includes a transmit credit error.

- U32 STAR_API_CC STAR_getLinkSpeedEventSpeed (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)

Gets the link speed of a given link speed change event stream item.

- U8 STAR_API_CC STAR_getLinkSpeedEventPort (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)

Gets the port related to a given link speed change event stream item.

- void STAR_API_CC STAR_getTimestampEventStartValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)

Gets the start time value from a timestamp event stream item.

- `void STAR_API_CC STAR_getTimestampEventEndValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)`
Gets the end time value from a timestamp event stream item.
- `STAR_TIMESTAMP_TYPE STAR_API_CC STAR_getTimestampEventType (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`
Gets the timestamp type information from a timestamp event stream item.
- `STAR_TIMESTAMP_DIRECTION STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`
Gets the timestamp direction information from a timestamp event stream item.
- `void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)`
Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createBroadcastMessage (_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)`
Creates a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message channel from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageType (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message type from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageStatus (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message status from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageLateFlag (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message late flag from a broadcast message stream item.
- `U8 STAR_API_CC STAR_getBroadcastMessageDelayedFlag (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message delayed flag from a broadcast message stream item.
- `U32 STAR_API_CC STAR_getBroadcastMessageDataWord1 (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message first data word from a broadcast message stream item.
- `U32 STAR_API_CC STAR_getBroadcastMessageDataWord2 (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets the broadcast message second data word from a broadcast message stream item.

8.5.1 Detailed Description

Stream items are abstract representations of packets, time-codes and link control tokens that can be transmitted and received over a SpaceWire network.

The API has been designed such that users only interact with pointers to item types, so they need not have any knowledge of what is actually contained within the various item structure types. Therefore, any objects required must be created and destroyed using the API, and all uses of the object are through the provided item accessor and mutator functions. Data structure members may change in future versions of this API.

8.5.2 Data Structure Documentation

8.5.2.1 struct STAR_IP_ADDRESS_TYPE

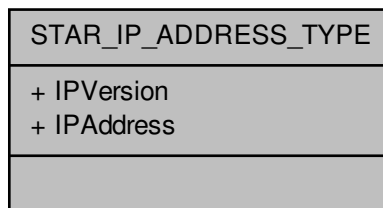
A structure consisting of:

- IP version
- IP address

Added in version:

STAR-System v5.00 Beta 2 - 30th September 2021

Collaboration diagram for STAR_IP_ADDRESS_TYPE:



Data Fields

U8 *	IPAddress	Pointer to the buffer containing the IP address. The length of the address is determined by the IPVersion: IPv4 = 4 bytes, IPv6 = 16 bytes.
STAR_IP_VERSION_TYPE	IPVersion	The Internet Protocol version.

8.5.2.2 struct STAR_LINK_STATE_EVENT

Events that can occur which affect the state of the SpaceWire link.

Note

Link state events are not supported by the PCI Mk2 card. The appropriate link state change event enable function must be called to allow these events to be generated.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

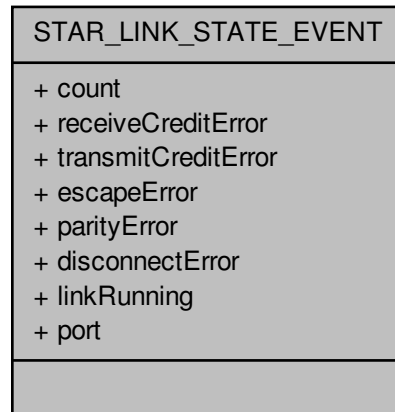
Declaration changed in version:

STAR-System v4.00 - 19th November 2019

Examples:

ex_receive_example.c, and link_events.c.

Collaboration diagram for STAR_LINK_STATE_EVENT:



Data Fields

U8	count	The number of times that the event(s) has occurred.
unsigned int	disconnect↔ Error: 1	Disconnect error.
unsigned int	escapeError: 1	Escape error.
unsigned int	linkRunning: 1	Link running.
unsigned int	parityError: 1	Parity error.
U8	port	The port on which the events occurred.
unsigned int	receiveCredit↔ Error: 1	Receive credit error.
unsigned int	transmitCredit↔ Error: 1	Transmit credit error.

8.5.2.3 struct STAR_SPACEWIRE_ADDRESS

A structure used to describe a SpaceWire address.

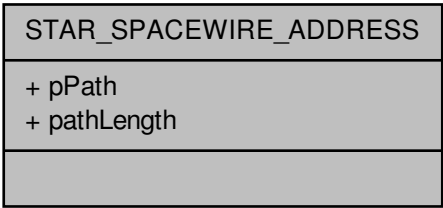
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, brickMk2_configuration_example.c, link_events.c, star-test.c, star_↔
performance_tester.c, transfer_operation_list_send_receive_example.c, and ui.c.

Collaboration diagram for STAR_SPACEWIRE_ADDRESS:



Data Fields

U16	pathLength	The length of the address.
U8 *	pPath	Array of SpaceWire path address elements.

8.5.2.4 struct STAR_DATA_CHUNK

A structure used to describe a chunk of contiguous SpaceWire data, within a single packet.

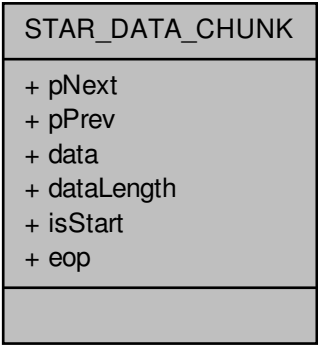
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`ex_receive_example.c`.

Collaboration diagram for STAR_DATA_CHUNK:



Data Fields

unsigned char *	data	Pointer to the data buffer itself.
U32	dataLength	Length of data in the buffer.
STAR_EOP_TYPE	eop	The type of end of packet marker this data chunk has, if any.
int	isStart	Whether the chunk of data represents the start of a new packet.
struct star_data_chunk *	pNext	
struct star_data_chunk *	pPrev	

8.5.2.5 struct STAR_SPACEWIRE_PACKET

A structure used to describe a SpaceWire packet.

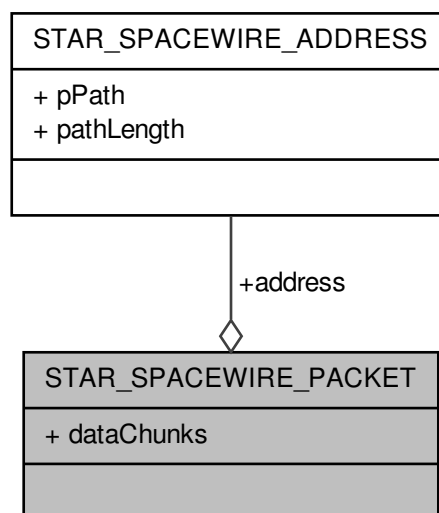
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_two_way_example.c, star-test.c, star_performance_tester.c, timestamp_test.c, transfer_callback_example.c, and transfer_operation_list_send_receive_example.c.

Collaboration diagram for STAR_SPACEWIRE_PACKET:



Data Fields

STAR_SPAC↔ EWIRE_ADD↔ RESS *	address	The packets address (optional) Note This member is provided for convenience only. There is no difference between specifying a packet with an address of [0xFE, 0xAB] and data of [0x01, 0x02, 0x03, 0x04], and specifying a packet with no explicit address, and data of [0xFE, 0xAB, 0x01, 0x02, 0x03, 0x04]
void *	dataChunks	The data that makes up the packet.

8.5.2.6 struct STAR_TIMECODE

A structure used to describe a SpaceWire time-code.

Note

Currently some STAR-System compatible devices only support transmitting time-codes using channel 0.

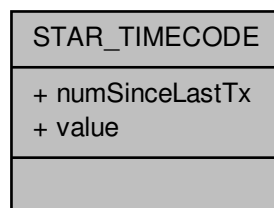
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[ex_receive_example.c](#), and [time-code_test.c](#).

Collaboration diagram for STAR_TIMECODE:



Data Fields

U16	numSinceLastTx	Count of the number of time-codes since the last time-code was transmitted.
U8	value	Value of the time-code.

8.5.2.7 struct STAR_LINK_SPEED_EVENT

A structure used to describe a link speed change event.

Note

Link speed events are not supported by the PCI Mk2 card. The appropriate link speed change event enable function must be called to allow these events to be generated.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

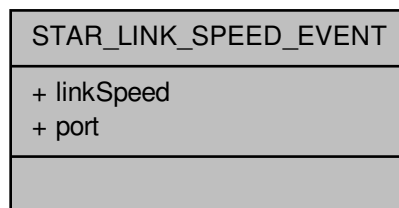
Declaration changed in version:

STAR-System v4.00 - 19th November 2019

Examples:

`brickMk2_configuration_example.c`, `ex_receive_example.c`, and `link_events.c`.

Collaboration diagram for STAR_LINK_SPEED_EVENT:

**Data Fields**

U32	linkSpeed	The new link speed which has been adopted, represented in bit/s. Note that this value is approximate.
U8	port	The port on which the event occurred.

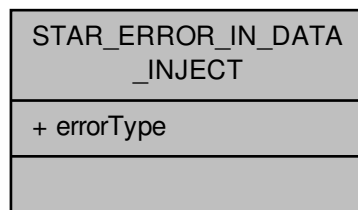
8.5.2.8 struct STAR_ERROR_IN_DATA_INJECT

A structure used to describe an error in data event.

Note

Not all devices support this item type, and that receiving these types is not possible.

Collaboration diagram for STAR_ERROR_IN_DATA_INJECT:

**Data Fields**

STAR_ERROR_IN_DATA_INJECT	errorType	Type of error to inject on a following data character.
---------------------------	-----------	--

8.5.2.9 struct STAR_TIMESTAMP_EVENT

A structure used to describe a timestamp on receipt of data on the link.

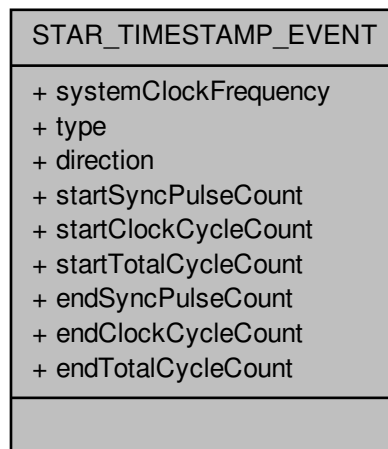
Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

Collaboration diagram for STAR_TIMESTAMP_EVENT:



Data Fields

STAR_TIMESTAMP_DIRECTION	direction	Whether timestamp relates to transmitted or received data.
U32	endClockCycleCount	The number of clock cycles counted when the end of the packet was transmitted or received.
U32	endSyncPulseCount	The number of synchronisation pulses when the end of the packet was transmitted or received.
U32	endTotalCycleCount	The number of clock cycles counted when the last synchronisation pulse before the end of the packet was received.
U32	startClockCycleCount	The number of clock cycles counted when the start of the packet was transmitted or received.
U32	startSyncPulseCount	The number of synchronisation pulses when the start of the packet was transmitted or received.
U32	startTotalCycleCount	The number of clock cycles counted when the last synchronisation pulse before the start of the packet was received.
U32	systemClockFrequency	The system clock frequency of the device.
STAR_TIMESTAMP_TYPE	type	The type of data that the timestamp represents.

8.5.2.10 struct STAR_BROADCAST_MESSAGE

A structure used to describe a SpaceFibre broadcast message.

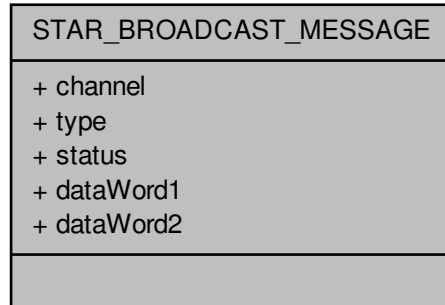
Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

broadcast_message_example.c.

Collaboration diagram for STAR_BROADCAST_MESSAGE:



Data Fields

U8	channel	The broadcast message's channel (BC field)
U32	dataWord1	The broadcast message's first data word.
U32	dataWord2	The broadcast message's second data word.
U8	status	The broadcast message's status (STATUS field).
U8	type	The broadcast message's type (B_TYPE field).

8.5.2.11 struct STAR_STREAM_ITEM

A wrapper for Stream Item objects.

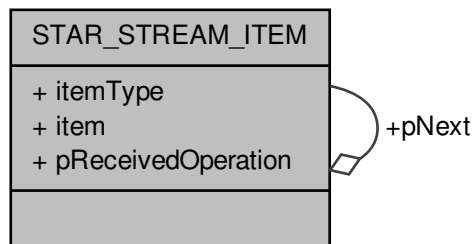
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet_example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send_example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, broadcast_message_example.c, link_events.c, rmap_target_example.c, star-test.c, star_performance_tester.c, time-code_example.c, time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c, transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

Collaboration diagram for STAR_STREAM_ITEM:



Data Fields

void *	item	A stream item object.
STAR_STREAM_ITEM_TYPE	itemType	The type of stream item pointed to by item.
struct STAR_STREAM_ITEM *	pNext	
void *	pReceivedOperation	

8.5.3 Enumeration Type Documentation

8.5.3.1 enum STAR_BROADCAST_MESSAGE_STATUS_FLAG

An enum used to define the status flags of a broadcast message.

Added in version:

STAR-System v4.00 - 19th November 2019

Enumerator

STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE The LATE flag of a broadcast message status field.

STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED The DELAYED flag of a broadcast message status field.

8.5.3.2 enum STAR_EOP_TYPE

The kinds of End of packet marker that may be present.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_EOP_TYPE_INVALID Error occurred determining EOP type.

STAR_EOP_TYPE_EOP End of Packet marker.

STAR_EOP_TYPE_EEP Error End of Packet marker.

STAR_EOP_TYPE_NONE No End of Packet marker present.

8.5.3.3 enum STAR_ERROR_IN_DATA_TYPE

The types of error that may be injected in to data in a packet.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Enumerator

STAR_ERROR_IN_DATA_PARITY Parity error.

STAR_ERROR_IN_DATA_DISCONNECT Disconnect error.

8.5.3.4 enum STAR_IP_VERSION_TYPE

The Internet Protocol version.

Added in version:

STAR-System v5.00 Beta 2 - 30th September 2021

Enumerator

STAR_IP_VERSION_NOT_DEFINED Initiator.

STAR_IPv4 Number of bytes to contain the IPv4 address.

STAR_IPv6 Number of bytes to contain the IPv6 address.

8.5.3.5 enum STAR_STREAM_ITEM_TYPE

The kinds of object that are represented as stream items.

Used by [STAR_STREAM_ITEM](#) to show what kind of item it is wrapping.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET Packet, see [STAR_SPACEWIRE_PACKET](#).

STAR_STREAM_ITEM_TYPE_TIMECODE Time-code, see: [STAR_TIMECODE](#).

STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT A change in the state of a link, see: [STAR_LINK_STATE_EVENT](#).

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

STAR_STREAM_ITEM_TYPE_DATA_CHUNK Contiguous chunk of data within a single packet, see: [STAR_DATA_CHUNK](#).

STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT A change in the speed of a link, see: [STAR_LINK_SPEED_EVENT](#).

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

STAR_STREAM_ITEM_TYPE_ERROR_INJECT An error injection control word, see: [STAR_ERROR_IN_DATA_INJECT](#).

STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT Timestamp of the last data received on the link, see: [STAR_TIMESTAMP_EVENT](#).

Added in version:

STAR-System v3.6 - 3rd April 2017

STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE SpaceFibre broadcast message, see: [STAR_BROADCAST_MESSAGE](#).

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[link_events.c](#).

8.5.3.6 enum STAR_TIMESTAMP_DIRECTION

An enum used to define the direction of a timestamp.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_TIMESTAMP_DIRECTION_TRANSMIT Timestamp is applied to transmitted data.

STAR_TIMESTAMP_DIRECTION_RECEIVE Timestamp is applied to received data.

8.5.3.7 enum STAR_TIMESTAMP_TYPE

An enum used to define different types of timestamps that can be received.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_TIMESTAMP_TYPE_DATA Timestamp for data.

STAR_TIMESTAMP_TYPE_LINK_STATE Timestamp for link state event.

STAR_TIMESTAMP_TYPE_LINK_SPEED Timestamp for link speed event.

STAR_TIMESTAMP_TYPE_TIMECODE Timestamp for time-code.

STAR_TIMESTAMP_TYPE_ERROR Timestamp for error in data.

8.5.4 Function Documentation

8.5.4.1 `_Check_return_ STAR_SPACEWIRE_ADDRESS* STAR_API_CC STAR_createAddress ( _In_count_(pathLen) unsigned char * path, U16 pathLen )`

Creates a SpaceWire address.

The address should be destroyed when it is no longer required using [STAR_destroyAddress\(\)](#).

Parameters

<i>in</i>	<i>path</i>	Address path
-----------	-------------	--------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>pathLen</i>	Length of the path buffer
----------------	---------------------------

Returns

Pointer to newly created address

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`advanced_address_example.c`, `brickMk2_configuration_example.c`, `link_events.c`, `star-test.c`, `star_↔`
`performance_tester.c`, `transfer_operation_list_send_receive_example.c`, and `ui.c`.

8.5.4.2 **STAR_STREAM_ITEM*** **STAR_API_CC** **STAR_createBroadcastMessage** (*_In_ U8 channel*, *_In_ U8 type*, *_In_ U8 status*, *_In_ U32 dataWord1*, *_In_ U32 dataWord2*)

Creates a broadcast message stream item.

This stream item should be destroyed when it is no longer required by calling `STAR_destroyStreamItem()`.

Parameters

<i>channel</i>	The broadcast message channel
<i>type</i>	The broadcast message type
<i>status</i>	The broadcast message status
<i>dataWord1</i>	The first data word of the broadcast message
<i>dataWord2</i>	The second data word of the broadcast message

Returns

Pointer to newly created broadcast message, or NULL if an error occurred.

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

`broadcast_message_example.c`.

8.5.4.3 **_Check_return_ STAR_STREAM_ITEM*** **STAR_API_CC** **STAR_createDataChunk** (*_In_opt_count_ (dataLen)* *unsigned char * data*, *unsigned int dataLen*, *int isStart*, **STAR_EOP_TYPE** *eopType*)

Creates a data chunk stream item, used to refer to part of a packet.

This stream item should be destroyed when it is no longer required by calling `STAR_destroyStreamItem()`.

Parameters

<i>in</i>	<i>data</i>	Pointer to start of chunk data
-----------	-------------	--------------------------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>dataLen</i>	The length of the chunk. The maximum size permitted for a data chunk is 0xffff. Attempts to create a chunk larger than this will fail.
<i>isStart</i>	Whether the chunk is the start of a packet (1) or not (0)
<i>eopType</i>	The end of packet marker type for the chunk (can be No EOP)

Returns

Pointer to newly created data chunk, or NULL if an error occurred.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`transmit_errors_example.c`.

8.5.4.4 **STAR_STREAM_ITEM* STAR_API_CC STAR_createErrorInData (STAR_ERROR_IN_DATA_TYPE *errorType*)**

Creates an error in data stream item.

This stream item can be transmitted to cause an error to occur on the next data character to be transmitted. The stream item should be destroyed when it is no longer required by calling `STAR_destroyStreamItem()`.

Parameters

<i>errorType</i>	the type of error to inject.
------------------	------------------------------

Returns

Error in data stream item.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Examples:

`transmit_errors_example.c`.

8.5.4.5 **_Check_return_ STAR_STREAM_ITEM* STAR_API_CC STAR_createPacket (_In_opt_ STAR_SPACEWIRE_ADDRESS * *address*, _In_opt_count_(dataLen) unsigned char * *data*, unsigned int *dataLen*, STAR_EOP_TYPE *eopType*)**

Creates a new SpaceWire packet stream item from user provided data.

This stream item should be destroyed when it is no longer required by calling `STAR_destroyStreamItem()`.

Parameters

<i>in</i>	<i>address</i>	Optional pointer to address created with STAR_createAddress()
-----------	----------------	---

Note

It is safe to dispose of this buffer after this function completes.

Parameters

<i>in</i>	<i>data</i>	Optional pointer to data buffer
-----------	-------------	---------------------------------

Note

The contents of this buffer are copied into data structures managed by the API. It is safe to dispose of this buffer after this function completes.

Parameters

<i>dataLen</i>	Length of the data buffer
<i>eopType</i>	End of packet marker type for the packet (may be none)

Returns

SpaceWire Packet item

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[advanced_address_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send_example.c](#), [advanced_two_way_example.c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [timestamp_test.c](#), [transfer_operation_list_send_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

8.5.4.6 STAR_STREAM_ITEM* STAR_API_CC STAR_createTimeCode (U8 value)

Creates a Time-code stream item.

This stream item should be destroyed when it is no longer required by calling [STAR_destroyStreamItem\(\)](#).

Parameters

<i>value</i>	Time-code value
--------------	-----------------

Returns

Time-code stream item

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Examples:

[time-code_example.c](#), and [time-code_test.c](#).

8.5.4.7 void STAR_API_CC STAR_destroyAddress (_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS * address)

Disposes of a SpaceWire address created with [STAR_createAddress\(\)](#).

Parameters

<i>in</i>	<i>address</i>	SpaceWire address to destroy
-----------	----------------	------------------------------

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [brickMk2_configuration_example.c](#), [link_events.c](#), [star-test.c](#), and [star_performance_tester.c](#).

8.5.4.8 void STAR_API_CC STAR_destroyPacketData (_In_ _Post_ptr_invalid_ unsigned char * *pData*)

Frees packet data previously created by a call to [STAR_getPacketData\(\)](#).

Parameters

<i>pData</i>	the data to be freed
--------------	----------------------

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_two_way_example.c](#), [star-test.c](#), [timestamp_test.c](#), [transfer_callback_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

8.5.4.9 void STAR_API_CC STAR_destroyStreamItem (_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM * *item*)

Disposes of a given stream item.

All stream items which have been explicitly created, e.g. by calls to [STAR_createDataChunk\(\)](#), [STAR_createPacket\(\)](#), etc. should be destroyed using this function.

Stream items returned by [STAR_getTransferItem\(\)](#) can optionally be disposed using this function. If these stream items are not disposed, they will continue to exist until the receive operation is disposed of by calling [STAR_disposeTransferOperation\(\)](#) or when the receive operation is resubmitted. For infinite receive operations (or receives for large numbers of packets) it is necessary to destroy the received stream items to ensure the PC's memory is not filled.

Parameters

<i>in</i>	<i>item</i>	Pointer to item to destroy
-----------	-------------	----------------------------

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send_example.c](#), [advanced_two_way_example.c](#), [broadcast_message_example.c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_example.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_operation_list_send_example.c](#), [transfer_operation_list_send_receive_example.c](#), and [transmit_errors_example.c](#).

8.5.4.10 U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message channel from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's channel

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[broadcast_message_example.c](#).

8.5.4.11 U32 STAR_API_CC STAR_getBroadcastMessageDataWord1 (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message first data word from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's first data word

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[broadcast_message_example.c](#).

8.5.4.12 U32 STAR_API_CC STAR_getBroadcastMessageDataWord2 (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message second data word from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's second data word

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[broadcast_message_example.c](#).

8.5.4.13 U8 STAR_API_CC STAR_getBroadcastMessageDelayedFlag (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message delayed flag from a broadcast message stream item.

Parameters

pBroadcastMessage ← the broadcast message

Returns

the broadcast message's delayed flag

Added in version:

STAR-System v4.00 - 19th November 2019

8.5.4.14 U8 STAR_API_CC STAR_getBroadcastMessageLateFlag (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message late flag from a broadcast message stream item.

Parameters

pBroadcastMessage ← the broadcast message

Returns

the broadcast message's late flag

Added in version:

STAR-System v4.00 - 19th November 2019

8.5.4.15 U8 STAR_API_CC STAR_getBroadcastMessageStatus (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message status from a broadcast message stream item.

Parameters

pBroadcastMessage ← the broadcast message

Returns

the broadcast message's status

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[broadcast_message_example.c](#).

8.5.4.16 U8 STAR_API_CC STAR_getBroadcastMessageType (_In_ STAR_BROADCAST_MESSAGE * *pBroadcastMessage*)

Gets the broadcast message type from a broadcast message stream item.

Parameters

<i>pBroadcastMessage</i>	the broadcast message
--------------------------	-----------------------

Returns

the broadcast message's type

Added in version:

STAR-System v4.00 - 19th November 2019

Examples:

[broadcast_message_example.c](#).

8.5.4.17 `unsigned char* STAR_API_CC STAR_getChunkData ( _In_ STAR_DATA_CHUNK * chunk, _Out_ unsigned int * len )`

Gets a pointer to a data chunk stream item's data.

Parameters

<i>chunk</i>	Data chunk
<i>len</i>	Pointer to user provided value that will be updated to the length of the returned data

Returns

Pointer to data, or NULL if no data is present. Note that this is a pointer to the chunk's data, not a copy of the data, so the data should not be freed, and will cease to exist if the data chunk is destroyed.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

8.5.4.18 `unsigned int STAR_API_CC STAR_getChunkDataLength ( _In_ STAR_DATA_CHUNK * chunk )`

Gets the length of a data chunk stream item's data.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

Length of data member

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.5.4.19 `STAR_EOP_TYPE STAR_API_CC STAR_getChunkEop ( _In_ STAR_DATA_CHUNK * chunk )`

Gets the End Of Packet marker type of a data chunk stream item.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

The EOP type of the chunk or STAR_EOP_TYPE_INVALID if an if an invalid chunk was passed in

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.5.4.20 int STAR_API_CC STAR_getChunkSop (_In_ STAR_DATA_CHUNK * *chunk*)

Gets whether a data chunk stream item is at the start of a packet.

Parameters

<i>chunk</i>	Data chunk
--------------	------------

Returns

Whether the chunk is at the start of a packet (1) or not (0), or -1 if an invalid chunk was passed in

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.5.4.21 U8 STAR_API_CC STAR_getLinkSpeedEventPort (_In_ STAR_LINK_SPEED_EVENT * *pLinkSpeedEvent*)

Gets the port related to a given link speed change event stream item.

Parameters

<i>pLinkSpeedEvent</i>	the link speed event to get the port for
------------------------	--

Returns

the port related to the link speed event

Added in version:

STAR-System v4.01 - 29th November 2019

Examples:

[link_events.c](#).

8.5.4.22 U32 STAR_API_CC STAR_getLinkSpeedEventSpeed (_In_ STAR_LINK_SPEED_EVENT * *pLinkSpeedEvent*)

Gets the link speed of a given link speed change event stream item.

Parameters

<i>pLinkSpeed</i>	the link speed event to get the link speed from
<i>Event</i>	

Returns

the link speed from the link speed event

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Examples:

brickMk2_configuration_example.c, and link_events.c.

8.5.4.23 U8 STAR_API_CC STAR_getLinkStateEventCount (_In_ STAR_LINK_STATE_EVENT * *pLinkStateEvent*)

Gets the count of a given link state event stream item, indicating the number of events that have occurred.

This count can be more than one if multiple link state events occurred in quick succession. If this is the case, only the most recent link state event type will be recorded.

Parameters

<i>pLinkStateEvent</i>	the link state event to get the count for
------------------------	---

Returns

the link state event's count

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.24 U8 STAR_API_CC STAR_getLinkStateEventPort (_In_ STAR_LINK_STATE_EVENT * *pLinkStateEvent*)

Gets the port related to a given link state change event stream item.

Parameters

<i>pLinkStateEvent</i>	the link state event to get the port for
------------------------	--

Returns

the port related to the link state event

Added in version:

STAR-System v4.01 - 29th November 2019

Examples:

link_events.c.

8.5.4.25 **STAR_SPACEWIRE_ADDRESS*** **STAR_API_CC** **STAR_getPacketAddress** (**_In_** **STAR_SPACEWIRE_PACKET *** *packet*)

Gets a copy of a packet's address structure.

Note that received packets will not have an address structure set. This function is only useful for accessing the address member of a packet already created by the user.

Note

As this function creates a copy of the address, this must be freed with a call to **STAR_destroyAddress()** when it is no longer in use.

Parameters

<i>in</i>	<i>packet</i>	Packet to get address member from.
-----------	---------------	------------------------------------

Returns

Pointer to copy of packet's address member.

Added in version:

STAR-System v1.12 - 14th November 2012

8.5.4.26 **_Ret_opt_bytecap_x_dataLength unsigned char*** **STAR_API_CC** **STAR_getPacketData** (**_In_** **STAR_SPACEWIRE_PACKET *** *packet*, **_Out_ unsigned int *** *dataLength*)

Gets the packet's data as an array of bytes.

This function copies the data from the various chunks that make up a packet into a new array.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
<i>out</i>	<i>dataLength</i>	Pointer to value to be updated to be size of array

Returns

Pointer to data, or NULL if no data is present

Note

As this function creates a new buffer to store the data, this buffer must be freed with a call to **STAR_destroyPacketData()** when it is no longer in use.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`advanced_address_example.c`, `advanced_continuous_receive_example.c`, `advanced_multiple_packet_example.c`, `advanced_receive_example.c`, `advanced_send_and_receive_example.c`, `advanced_two_way_example.c`, `star-test.c`, `timestamp_test.c`, `transfer_callback_example.c`, and `transfer_operation_list_send_receive_example.c`.

8.5.4.27 **STAR_EOP_TYPE** **STAR_API_CC** **STAR_getPacketEOP** (**_In_** **STAR_SPACEWIRE_PACKET *** *packet*)

Gets the end of packet marker type of a packet.

Parameters

<i>packet</i>	SpaceWire packet
---------------	------------------

Returns

The EOP type of the packet

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[star_performance_tester.c](#).

8.5.4.28 unsigned int STAR_API_CC STAR_getPacketLength (_In_ STAR_SPACEWIRE_PACKET * *packet*)

Gets the length of a given packet.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
-----------	---------------	------------------

Returns

The length in bytes of the packet (data + address)

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[star-test.c](#), and [star_performance_tester.c](#).

8.5.4.29 U16 STAR_API_CC STAR_getTimeCodeCount (_In_ STAR_TIMECODE * *pTimeCode*)

Gets the count of a given time-code stream item.

Where count is the number of time-codes since the last time-code was transmitted.

Parameters

<i>pTimeCode</i>	Time-code to get count from
------------------	-----------------------------

Returns

Time-code count

Added in version:

STAR-System v6.00 - 7th October 2024

8.5.4.30 U8 STAR_API_CC STAR_getTimeCodeValue (_In_ STAR_TIMECODE * *pTimeCode*)

Gets the value of a given time-code stream item.

Parameters

<i>pTimeCode</i>	Time-code to get value of
------------------	---------------------------

Returns

Time-code value

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

time-code_test.c.

8.5.4.31 **STAR_TIMESTAMP_DIRECTION** STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT * *pTimestampEvent*)

Gets the timestamp direction information from a timestamp event stream item.

Parameters

<i>pTimestampEvent</i>	the timestamp event to get the direction of
------------------------	---

Returns

the timestamp direction, or -1 if an invalid timestamp was passed in

Added in version:

STAR-System v3.6 - 3rd April 2017

8.5.4.32 **void** STAR_API_CC STAR_getTimestampEventEndValue (_In_ STAR_TIMESTAMP_EVENT * *pTimestampEvent*, U32 *syncPulseFrequency*, _Out_ U32 * *pSeconds*, _Out_ U32 * *pRemainder*)

Gets the end time value from a timestamp event stream item.

This value relates to the end of the data being received in whole seconds and the remainder in nanoseconds.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get timestamp from
	<i>syncPulseFrequency</i>	the frequency of synchronisation pulses in hertz
out	<i>pSeconds</i>	outputs the number of whole seconds of the timestamp
out	<i>pRemainder</i>	outputs the remainder in nanoseconds

Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

8.5.4.33 void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT *
pTimestampEvent, _Out_opt_ U32 * pStartSyncPulseCount, _Out_opt_ U32 * pStartClockCycleCount, _Out_opt_
U32 * pStartTotalCycleCount, _Out_opt_ U32 * pEndSyncPulseCount, _Out_opt_ U32 * pEndClockCycleCount,
_Out_opt_ U32 * pEndTotalCycleCount, _Out_opt_ U32 * pClockFrequency)

Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get timestamp from
out	<i>pStartSyncPulseCount</i>	outputs the start sync pulse counter
out	<i>pStartClockCycleCount</i>	outputs the start clock cycle counter
out	<i>pStartTotalCycleCount</i>	outputs the start clock cycle counter value recorded at the previous sync pulse
out	<i>pEndSyncPulseCount</i>	outputs the end sync pulse counter
out	<i>pEndClockCycleCount</i>	outputs the end sync pulse counter
out	<i>pEndTotalCycleCount</i>	outputs the end clock cycle counter value recorded at the previous sync pulse
out	<i>pClockFrequency</i>	outputs the clock frequency

Added in version:

STAR-System v3.6 - 3rd April 2017

8.5.4.34 void **STAR_API_CC** STAR_getTimestampEventStartValue (_In_ **STAR_TIMESTAMP_EVENT** * *pTimestampEvent*, **U32** *syncPulseFrequency*, _Out_ **U32** * *pSeconds*, _Out_ **U32** * *pRemainder*)

Gets the start time value from a timestamp event stream item.

This value relates to the start of the data being received in whole seconds and the remainder in nanoseconds.

Parameters

	<i>pTimestampEvent</i>	the timestamp event to get the timestamp from
	<i>syncPulseFrequency</i>	the frequency of synchronisation pulses in hertz
out	<i>pSeconds</i>	outputs the number of whole seconds of the timestamp
out	<i>pRemainder</i>	outputs the remainder in nanoseconds

Added in version:

STAR-System v3.6 - 3rd April 2017

Examples:

timestamp_test.c.

8.5.4.35 **STAR_TIMESTAMP_TYPE** **STAR_API_CC** STAR_getTimestampEventType (_In_ **STAR_TIMESTAMP_EVENT** * *pTimestampEvent*)

Gets the timestamp type information from a timestamp event stream item.

Parameters

<i>pTimestamp</i> ←→	the timestamp event to get the type of
<i>Event</i>	

Returns

the timestamp type, or -1 if an invalid timestamp was passed in

Added in version:

STAR-System v3.6 - 3rd April 2017

8.5.4.36 `int STAR_API_CC STAR_isLinkStateEventDisconnectError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a disconnect error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a disconnect error
------------------------	--

Returns

whether the link state event includes a disconnect error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.37 `int STAR_API_CC STAR_isLinkStateEventEscapeError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes an escape error.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for an escape error
------------------------	---

Returns

whether the link state event includes an escape error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.38 `int STAR_API_CC STAR_isLinkStateEventLinkRunning ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a link running state change.

Parameters

<i>pLinkStateEvent</i>	the link state event to check for a link running state change
------------------------	---

Returns

whether the link state event includes a link running state change

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Examples:

`link_events.c`.

8.5.4.39 `int STAR_API_CC STAR_isLinkStateEventParityError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a parity error.

Parameters

pLinkStateEvent the link state event to check for a parity error

Returns

whether the link state event includes a parity error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.40 `int STAR_API_CC STAR_isLinkStateEventReceiveCreditError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a receive credit error.

Parameters

pLinkStateEvent the link state event to check for a receive credit error

Returns

whether the link state event includes a receive credit error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.41 `int STAR_API_CC STAR_isLinkStateEventTransmitCreditError ( _In_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets whether a link state event stream item includes a transmit credit error.

Parameters

pLinkStateEvent the link state event to check for a transmit credit error

Returns

whether the link state event includes a transmit credit error

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

8.5.4.42 `int STAR_API_CC STAR_setPacketAddress ( _Inout_ STAR_SPACEWIRE_PACKET * packet, _In_ STAR_SPACEWIRE_ADDRESS * address )`

Updates the address of a given packet.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
<i>in</i>	<i>address</i>	SpaceWire packet

Returns

1 if the update was successful, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.5.4.43 `int STAR_API_CC STAR_setPacketEOP ( _In_ STAR_SPACEWIRE_PACKET * packet, STAR_EOP_TYPE eopType )`

Sets the end of packet marker type of a packet.

Parameters

<i>in</i>	<i>packet</i>	SpaceWire packet
	<i>eopType</i>	End of packet marker type

Returns

1 if EOP was successfully set, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

8.5.4.44 `void STAR_API_CC STAR_setTimeCodeValue ( _In_ STAR_TIMECODE * pTimeCode, U8 value )`

Sets the value of a given time-code stream item.

Parameters

<i>pTimeCode</i>	time-code to set value of
<i>value</i>	new value

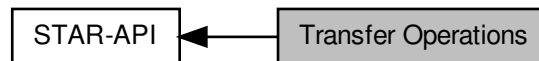
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.6 Transfer Operations

Transfer operations are used to transmit and receive packets and time-codes.

Collaboration diagram for Transfer Operations:



Macros

- `#define STAR_RECEIVE_NULL (STAR_RECEIVE_NULLS)`

Typedefs

- `typedef void STAR_TRANSFER_OPERATION`
A structure used to identify a transfer operation.
- `typedef void(STAR_API_CC * STAR_TransferOperationListenerFunc) (STAR_TRANSFER_OPERATION *pOperation, STAR_TRANSFER_STATUS status, void *pContextInfo)`
The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.

Enumerations

- `enum STAR_TRANSFER_STATUS {`
`STAR_TRANSFER_STATUS_NOT_STARTED, STAR_TRANSFER_STATUS_STARTED, STAR_TRANSFER_STATUS_COMPLETE, STAR_TRANSFER_STATUS_CANCELLED,`
`STAR_TRANSFER_STATUS_ERROR }`
Possible states a transfer operation can be in.
- `enum STAR_RECEIVE_MASK {`
`STAR_RECEIVE_PACKETS = 1U << 0, STAR_RECEIVE_CHUNKS = 1U << 1, STAR_RECEIVE_TIMECODES = 1U << 2, STAR_RECEIVE_FCTS = 1U << 3,`
`STAR_RECEIVE_NULLS = 1U << 4, STAR_RECEIVE_LINK_STATE_EVENTS = 1U << 5, STAR_RECEIVE_LINK_SPEED_EVENTS = 1U << 6, STAR_RECEIVE_TIMESTAMP_EVENTS = 1U << 7,`
`STAR_RECEIVE_BROADCAST_MESSAGES = 1U << 8 }`
Flags used to determine what kind of stream items to receive on a receive operation.

Functions

- `STAR_TRANSFER_STATUS STAR_API_CC STAR_receivePacket (_In_ STAR_CHANNEL_ID channelId, _Out_bytecap_(*pPacketLength) void *pPacketData, _Inout_ unsigned int *pPacketLength, _Out_ STAR_EOP_TYPE *pEopType, _In_ int timeout)`
Receive a single packet on a previously opened channel in to the buffer specified.
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_transmitPacket (_In_ STAR_CHANNEL_ID channelId, _In_opt_bytecount_(packetLength) void *pPacketData, _In_ unsigned int packetLength, _In_ STAR_EOP_TYPE eopType, _In_ int timeout)`

Transmit a single packet on a previously opened channel from the buffer specified.

- `_Check_return_ STAR_TRANSFER_OPERATION *STAR_API_CC STAR_createTxOperation (_In_count_ (streamItemCount) STAR_STREAM_ITEM **ppItems, unsigned int streamItemCount)`

Creates a transmit operation that can be submitted later.

- `_Check_return_ STAR_TRANSFER_OPERATION *STAR_API_CC STAR_createRxOperation (int itemCount, STAR_RECEIVE_MASK mask)`

Creates a receive operation that can then be submitted.

- `_Check_return_ STAR_TRANSFER_OPERATION *STAR_API_CC STAR_createRxOperationEx (_In_count_ (numBuffers) U8 **ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ STAR_RECEIVE_MASK mask)`

This function is an alternative to `STAR_createRxOperation()` that creates receive operations to be submitted to a channel opened with the `STAR_openChannelToLocalDeviceEx()` function.

- `int STAR_API_CC STAR_submitTransferOperation (_In_ STAR_CHANNEL_ID channelId, _In_ STAR_TRANSFER_OPERATION *transferOp)`

Submits a single transfer operation.

- `int STAR_API_CC STAR_submitTransferOperationList (STAR_CHANNEL_ID channelId, _In_count_ (count) STAR_TRANSFER_OPERATION **transferOps, unsigned int count)`

Submits a list of transfer operations.

- `STAR_TRANSFER_STATUS STAR_API_CC STAR_waitOnTransferOperationCompletion (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, int timeout)`

Blocks until a given transfer operation has completed, or the timeout period expires.

- `STAR_TRANSFER_STATUS STAR_API_CC STAR_getTransferOperationStatus (_In_ STAR_TRANSFER_OPERATION *pTransferOperation)`

Gets the status of a transfer operation.

- `void STAR_API_CC STAR_cancelTransferOperationWaits (_In_ STAR_TRANSFER_OPERATION *pTransferOperation)`

Cancels any waits in progress on a transfer operation.

- `unsigned int STAR_API_CC STAR_getTransferItemCount (_In_ STAR_TRANSFER_OPERATION *pReceiveOperation)`

Gets the current count of stream items received by the operation and available to be obtained using `STAR_getTransferItem()`.

- `STAR_STREAM_ITEM *STAR_API_CC STAR_getTransferItem (_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, unsigned int index)`

Gets the stream item at a given index of a receive operation.

- `STAR_STREAM_ITEM *STAR_API_CC STAR_getNextTransferItem (STAR_STREAM_ITEM *pStreamItem)`

Gets the next stream item in the list of received stream items.

- `_Ret_opt_cap_ count STAR_STREAM_ITEM **STAR_API_CC STAR_getTransferItemList (_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)`

Gets the entire list of transfer items associated with a receive.

- `void STAR_API_CC STAR_destroyTransferItemList (_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **transferItemList, unsigned int transferItemListCount, int destroyItems)`

Destroys a given stream item array, returned by a call to `STAR_getTransferItemList()`, optionally destroying each of the elements in the array.

- `int STAR_API_CC STAR_cancelTransferOperation (_In_ STAR_TRANSFER_OPERATION *pTransferOperation)`

Cancels a given transfer operation.

- `int STAR_API_CC STAR_disposeTransferOperation (_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION *pTransferOperation)`

Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation should be freed when the operation is no longer in use.

- `int STAR_API_CC STAR_registerTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)`

Registers a call-back function that will be called when an operation completes.

- `int STAR_API_CC STAR_unregisterTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)`
Unregisters a call-back function that will be called when an operation completes.
- `U8 *STAR_API_CC STAR_getSpaceWireAddressPath (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
Access function to get a SpaceWire address path.
- `U16 STAR_API_CC STAR_getSpaceWireAddressPathLength (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`
Access function to get a SpaceWire address path length.
- `int STAR_API_CC STAR_rxOperationExGetNumItems (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
Gets the number of items received by an ex receive operation.
- `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)`
Gets the stream item type for an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetDataChunk (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)`
Gets a data chunk from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`
Gets a broadcast message from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetTimestampEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`
Gets a timestamp event from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetLinkStateEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`
Gets a link state event from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetLinkSpeedEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`
Gets a link speed event from an item received by an ex receive operation.
- `int STAR_API_CC STAR_rxOperationExGetTimeCode (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMECODE *pTimeCode)`
Gets a time-code from an item received by an ex receive operation.
- `STAR_TRANSFER_STATUS STAR_API_CC STAR_rxOperationExGetStatus (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`
Gets the status of an ex receive operation.

8.6.1 Detailed Description

Transfer operations are used to transmit and receive packets and time-codes.

This group contains functions and data structures used to create, submit, and destroy these operations.

8.6.2 Macro Definition Documentation

8.6.2.1 #define STAR_RECEIVE_NULL (STAR_RECEIVE_NULLS)

Deprecated Replaced by `STAR_RECEIVE_NULLS` for naming consistency. The `STAR_RECEIVE_NULL` value in the `STAR_RECEIVE_MASK` enum was replaced by the `STAR_RECEIVE_NULLS` value for naming consistency. This macro is included to maintain backwards compatibility.

8.6.3 Typedef Documentation

8.6.3.1 typedef void STAR_TRANSFER_OPERATION

A structure used to identify a transfer operation.

Note

It is not intended for users of this library to access the members of this structure directly. Instead, use the provided accessor functions.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.6.3.2 typedef void(STAR_API_CC * STAR_TransferOperationListenerFunc) (STAR_TRANSFER_OPERATION *pOperation, STAR_TRANSFER_STATUS status, void *pContextInfo)

The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.

Parameters

<i>pOperation</i>	the operation which has completed
<i>status</i>	the status of the operation
<i>pContextInfo</i>	a pointer to user-specific data that is provided when the completion listener is registered

Note

The calling convention for this function type is [STAR_API_CC](#)

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.6.4 Enumeration Type Documentation

8.6.4.1 enum STAR_RECEIVE_MASK

Flags used to determine what kind of stream items to receive on a receive operation.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

[STAR_RECEIVE_PACKETS](#) [STAR_SPACEWIRE_PACKET](#)

[STAR_RECEIVE_CHUNKS](#) [STAR_DATA_CHUNK](#)

[STAR_RECEIVE_TIMECODES](#) [STAR_TIMECODE](#)

[STAR_RECEIVE_FCTS](#) Flow control characters. Note that transmitting and receiving FCTs is not currently supported.

[STAR_RECEIVE_NULLS](#) Null characters. Note that transmitting and receiving Nulls is not currently supported.

Added in version:

STAR-System v6.00 - 7th October 2024

STAR_RECEIVE_LINK_STATE_EVENTS STAR_LINK_STATE_EVENT

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

STAR_RECEIVE_LINK_SPEED_EVENTS STAR_LINK_SPEED_EVENT

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

STAR_RECEIVE_TIMESTAMP_EVENTS STAR_TIMESTAMP_EVENT

Added in version:

STAR-System v3.6 - 3rd April 2017

STAR_RECEIVE_BROADCAST_MESSAGES STAR_BROADCAST_MESSAGE

Added in version:

STAR-System v4.00 - 19th November 2019

8.6.4.2 enum STAR_TRANSFER_STATUS

Possible states a transfer operation can be in.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

STAR_TRANSFER_STATUS_NOT_STARTED Not yet started. When a transfer operation is created, this will be its status.

STAR_TRANSFER_STATUS_STARTED Transfer has begun. When a transfer operation is submitted, this will be its status, until it has completed.

STAR_TRANSFER_STATUS_COMPLETE Transfer has completed. Once a transmit operation has successfully transmitted all its traffic, or a receive operation has received all its requested traffic, this will be its status.

STAR_TRANSFER_STATUS_CANCELLED Transfer was cancelled. When a transfer is cancelled by calling `STAR_cancelTransferOperation()`, this will be its status.

STAR_TRANSFER_STATUS_ERROR An error occurred while processing the transfer. This will be the status of a transfer operation if there was an error creating it, submitting it, or there was an error while transmitting or receiving.

8.6.5 Function Documentation**8.6.5.1 int STAR_API_CC STAR_cancelTransferOperation (_In_ STAR_TRANSFER_OPERATION * pTransferOperation)**

Cancels a given transfer operation.

Note

It is not currently possible to cancel a transmit operation.

Parameters

<i>pTransfer↔ Operation</i>	Operation to cancel
---------------------------------	---------------------

Returns

1 if a receive was cancelled, else 0

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

brickMk2_configuration_example.c, link_events.c, and star_performance_tester.c.

8.6.5.2 void **STAR_API_CC** STAR_cancelTransferOperationWaits (_In_ STAR_TRANSFER_OPERATION *
pTransferOperation)

Cancels any waits in progress on a transfer operation.

Note that this does not cancel the transfer operation it simply cancels any calls to STAR_waitOnTransferOperation↔Completion().

Parameters

<i>pTransfer↔ Operation</i>	Operation to cancel waits on
---------------------------------	------------------------------

Added in version:

STAR-System v1.1 - 20th December 2011

8.6.5.3 _Check_return_ STAR_TRANSFER_OPERATION* STAR_API_CC STAR_createRxOperation (int *itemCount*,
STAR_RECEIVE_MASK *mask*)

Creates a receive operation that can then be submitted.

The receive operation returned should be disposed of by calling STAR_disposeTransferOperation() when it is no longer required.

Parameters

<i>itemCount</i>	The maximum number of stream items to receive (the size of an individual stream item is not limited). This can be -1 to receive an unlimited number of items
------------------	--

Note

When a receive operation receiving an unlimited (or very large) number of items, the received stream items must be destroyed to ensure the received stream items do not consume all the PC's memory.

Parameters

<i>mask</i>	Bitmask with flags set for the type of traffic one wishes to receive
-------------	--

Note

It is not possible to receive whole packets at the same time as link control tokens/data chunks

Returns

a pointer to a `STAR_TRANSFER_OPERATION` object

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

`advanced_address_example.c`, `advanced_continuous_receive_example.c`, `advanced_multiple_packet_example.c`, `advanced_receive_example.c`, `advanced_send_and_receive_example.c`, `advanced_two_way_example.c`, `brickMk2_configuration_example.c`, `broadcast_message_example.c`, `link_events.c`, `star-test.c`, `star_performance_tester.c`, `time-code_test.c`, `timestamp_test.c`, `transfer_callback_example.c`, and `transfer_operation_list_send_receive_example.c`.

8.6.5.4 `_Check_return_ STAR_TRANSFER_OPERATION* STAR_API_CC STAR_createRxOperationEx ( _In_count (numBuffers) U8 ** ppBuffers, _In_ unsigned int numBuffers, _In_ unsigned int bufferSize, _In_ STAR_RECEIVE_MASK mask )`

This function is an alternative to `STAR_createRxOperation()` that creates receive operations to be submitted to a channel opened with the `STAR_openChannelToLocalDeviceEx()` function.

Using the alternative receive operations, data is copied directly into pre-allocate buffers instead of dynamically allocated packets.

Note that receive operations created with `STAR_createRxOperation()` should not be submitted to a channel opened using `STAR_openChannelToLocalDeviceEx()`. Similarly, receive operations created with `STAR_createRxOperationEx()` should not be submitted to a channel opened using `STAR_openChannelToLocalDevice()`.

The receive operation returned should be disposed of by calling `STAR_disposeTransferOperation()` when it is no longer required.

Parameters

<i>ppBuffers</i>	An array of pointers to pre-allocated buffers that received stream items will be written to.
<i>numBuffers</i>	The number of buffers in the <i>ppBuffers</i> array.
<i>bufferSize</i>	The size of the buffers in the <i>ppBuffers</i> array.

Note

the size of the buffers should be large enough to receive the largest chunk generated by the hardware, which is generally 4096 or 8192 bytes.

Parameters

<i>mask</i>	Bitmask with flags set for the type of traffic one wishes to receive.
-------------	---

Note

Due to the use of fixed-size pre-allocated buffers, `STAR_RECEIVE_CHUNKS` should be used to receive data instead of `STAR_RECEIVE_PACKETS`.

Returns

a pointer to a [STAR_TRANSFER_OPERATION](#)

Added in version:

[STAR-System v4.00](#) - 19th November 2019

Examples:

[ex_receive_example.c](#).

8.6.5.5 [_Check_return_ STAR_TRANSFER_OPERATION*](#) [STAR_API_CC](#) [STAR_createTxOperation](#) ([_In_count_](#)(streamItemCount) [STAR_STREAM_ITEM](#) ** *ppltems*, unsigned int *streamItemCount*)

Creates a transmit operation that can be submitted later.

The transmit operation returned should be disposed of by calling [STAR_disposeTransferOperation\(\)](#) when it is no longer required.

Parameters

<i>ppltems</i>	Array of pointers to Stream Items to be sent
----------------	--

Note

It is safe to free this array after the function has completed.

Parameters

<i>streamItem</i> ↔ <i>Count</i>	Count of the items in the ppltems array
-------------------------------------	---

Returns

Pointer to a [STAR_TRANSFER_OPERATION](#)

Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_send_and_receive_](#)↔
[example.c](#), [advanced_send_example.c](#), [advanced_two_way_example.c](#), [broadcast_message_example.](#)↔
[c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_example.c](#), [time-code_test.c](#),
[timestamp_test.c](#), [transfer_operation_list_send_example.c](#), [transfer_operation_list_send_receive_example.c](#),
and [transmit_errors_example.c](#).

8.6.5.6 [void STAR_API_CC](#) [STAR_destroyTransferItemList](#) ([_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM](#) ** *transferItemList*, unsigned int *transferItemListCount*, int *destroyItems*)

Destroys a given stream item array, returned by a call to [STAR_getTransferItemList\(\)](#), optionally destroying each of the elements in the array.

If the stream items are not destroyed they will continue to exist until destroyed by calling [STAR_destroyStreamItem\(\)](#) or by disposing of the receive operation by calling [STAR_disposeTransferOperation\(\)](#). For infinite receive operations (or receives for large numbers of packets) it is necessary to destroy the received stream items to ensure the PC's memory is not filled.

Parameters

<i>in</i>	<i>transferItemList</i>	The array of stream items to destroy
	<i>transferItem↔ ListCount</i>	The number of elements in the array of stream items
	<i>destroyItems</i>	If non-zero, each stream item will also be destroyed

Added in version:

STAR-System v3.0 - 26th August 2016

Examples:

star_performance_tester.c.

8.6.5.7 `int STAR_API_CC STAR_disposeTransferOperation ( _In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION * pTransferOperation )`

Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation should be freed when the operation is no longer in use.

Note that this is a dispose function rather than a destroy function, as it only indicates to the API that the transfer operation should be destroyed when it is possible to do so. If the API is in the process of transmitting or receiving traffic on the operation it may not be possible to destroy the operation immediately.

Parameters

<i>pTransfer↔ Operation</i>	a pointer to the operation to be disposed of
---------------------------------	--

Returns

1 if operation was disposed of successfully, otherwise 0

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

advanced_address_example.c, advanced_continuous_receive_example.c, advanced_multiple_packet↔
example.c, advanced_receive_example.c, advanced_send_and_receive_example.c, advanced_send↔
example.c, advanced_two_way_example.c, brickMk2_configuration_example.c, broadcast_message↔
example.c, ex_receive_example.c, link_events.c, star-test.c, star_performance_tester.c, time-code_example.c,
time-code_test.c, timestamp_test.c, transfer_callback_example.c, transfer_operation_list_send_example.c,
transfer_operation_list_send_receive_example.c, and transmit_errors_example.c.

8.6.5.8 `STAR_STREAM_ITEM* STAR_API_CC STAR_getNextTransferItem ( STAR_STREAM_ITEM * pStreamItem )`

Gets the next stream item in the list of received stream items.

This is quicker than using `STAR_getTransferItem()`.

Parameters

<i>pStreamItem</i>	Current stream item
--------------------	---------------------

Returns

Next stream item

Added in version:

STAR-System v3.10 - 23rd February 2018

Examples:

star-test.c.

8.6.5.9 `U8* STAR_API_CC STAR_getSpaceWireAddressPath ( STAR_SPACEWIRE_ADDRESS * pSpaceWireAddress )`

Access function to get a SpaceWire address path.

Parameters

<i>pSpaceWire↔ Address</i>	A pointer to a <code>STAR_SPACEWIRE_ADDRESS</code> struct.
--------------------------------	--

Returns

A pointer to a SpaceWire address path.

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

8.6.5.10 `U16 STAR_API_CC STAR_getSpaceWireAddressPathLength ( STAR_SPACEWIRE_ADDRESS * pSpaceWireAddress )`

Access function to get a SpaceWire address path length.

Parameters

<i>pSpaceWire↔ Address</i>	A pointer to a <code>STAR_SPACEWIRE_ADDRESS</code> struct.
--------------------------------	--

Returns

A SpaceWire address path length.

Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

Examples:

star_performance_tester.c.

8.6.5.11 `STAR_STREAM_ITEM* STAR_API_CC STAR_getTransferItem ( _In_ STAR_TRANSFER_OPERATION * pReceiveOperation, unsigned int index )`

Gets the stream item at a given index of a receive operation.

Stream items returned can optionally be disposed by calling `STAR_destroyStreamItem()`. If these stream items are not disposed, they will continue to exist until the receive operation is disposed of by calling `STAR_disposeTransfer↔Operation()` or when the receive operation is resubmitted.

Parameters

<i>pReceive↔ Operation</i>	Operation to get item from
<i>index</i>	Item within operation to access

Returns

Item requested

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_two_way_example.c](#), [brickMk2_configuration_example.c](#), [broadcast_message_example.c](#), [link_events.c](#), [star-test.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_callback_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

8.6.5.12 `unsigned int STAR_API_CC STAR_getTransferItemCount ( _In_ STAR_TRANSFER_OPERATION * pReceiveOperation )`

Gets the current count of stream items received by the operation and available to be obtained using [STAR_getTransferItem\(\)](#).

Note that this may be less than the number of stream items received if some of those stream items have been destroyed by calling [STAR_destroyStreamItem\(\)](#) or [STAR_destroyTransferItemList\(\)](#) with the `destroyItems` parameter set to a non-zero value.

Parameters

<i>pReceive↔ Operation</i>	Operation to get received item count for
--------------------------------	--

Returns

Count of received stream items

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[advanced_multiple_packet_example.c](#), [brickMk2_configuration_example.c](#), [link_events.c](#), [star-test.c](#), and [timestamp_test.c](#).

8.6.5.13 `_Ret_opt_cap_count STAR_STREAM_ITEM** STAR_API_CC STAR_getTransferItemList ( _In_ STAR_TRANSFER_OPERATION * pReceiveOperation, _Out_ unsigned int * count )`

Gets the entire list of transfer items associated with a receive.

The array returned must be destroyed by calling [STAR_destroyTransferItemList\(\)](#).

Parameters

<i>in</i>	<i>pReceiveOperation</i>	Operation to get items from
<i>out</i>	<i>count</i>	Count of items in returned array

Returns

Array of pointers stream item associated with this receive

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

star_performance_tester.c.

8.6.5.14 **STAR_TRANSFER_STATUS STAR_API_CC STAR_getTransferOperationStatus (_In_ STAR_TRANSFER_OPERATION * *pTransferOperation*)**

Gets the status of a transfer operation.

Parameters

<i>pTransferOperation</i>	Operation to get status of
---------------------------	----------------------------

Returns

result status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

brickMk2_configuration_example.c, and star_performance_tester.c.

8.6.5.15 **STAR_TRANSFER_STATUS STAR_API_CC STAR_receivePacket (_In_ STAR_CHANNEL_ID *channelId*, _Out_bytecap_ **pPacketLength* void * *pPacketData*, _Inout_ unsigned int * *pPacketLength*, _Out_ STAR_EOP_TYPE * *pEopType*, _In_ int *timeout*)**

Receive a single packet on a previously opened channel in to the buffer specified.

This function is provided to simplify the process of receiving packets, but is not as efficient or as flexible as using [STAR_createRxOperation\(\)](#), [STAR_submitTransferOperation\(\)](#), etc. This function should not be used if high performance or low CPU utilisation is required. Note that if the received packet is longer than the buffer provided, the end of the packet will be dropped and the end of packet marker will indicate there was no end of packet marker. Note also that a limitation of this function is that the operation may timeout, but a packet is consumed and will not be received in a subsequent call to the function. To work around these issues use the alternative functions mentioned above.

Parameters

<i>channelId</i>	The identifier of a previously opened channel. The channel must have been opened as an in or in/out channel
<i>pPacketData</i>	Pointer to a previously allocated buffer which will be updated to contain the received packet
<i>pPacketLength</i>	Pointer to a variable which should contain the length of the buffer, and which will be updated to contain the length of the packet
<i>pEopType</i>	Pointer to a variable which will be updated to contain the end of packet marker type of the packet
<i>timeout</i>	The maximum time in milliseconds to wait for the packet to be received, or -1 to wait indefinitely

Returns

the status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[simple_receive_example.c](#), [simple_send_and_receive_example.c](#), and [simple_two_way_example.c](#).

8.6.5.16 `int STAR_API_CC STAR_registerTransferCompletionListener ( _In_ STAR_TRANSFER_OPERATION * pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void * pContextInfo )`

Registers a call-back function that will be called when an operation completes.

Note

Each callback is made using a separate thread, to ensure that STAR-System internal processing is not blocked by long-running callbacks. Due to operating system scheduling it is not possible to guarantee that callbacks will execute in the same order as they are triggered.

Parameters

<i>pTransfer↔ Operation</i>	Operation to add listener for
<i>listenerFunc</i>	Call-back function
<i>pContextInfo</i>	A pointer to memory which will be provided when the call-back function is called

Returns

Non-Zero if call-back was registered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[transfer_callback_example.c](#).

8.6.5.17 `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE * pBroadcastMessage )`

Gets a broadcast message from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the broadcast message from.
<i>pBroadcastMessage</i>	A pointer to a broadcast message that will be populated with data from the item.

Returns

1 if a broadcast message was successfully populated from the item, or -1 if an error occurred.

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

8.6.5.18 `int STAR_API_CC STAR_rxOperationExGetDataChunk ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK * pDataChunk )`

Gets a data chunk from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the data chunk from.
<i>pDataChunk</i>	A pointer to a data chunk that will be populated with data from the item.

Returns

1 if a data chunk was successfully populated from the item, or -1 if an error occurred.

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

Examples:

[ex_receive_example.c](#).

8.6.5.19 `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item )`

Gets the stream item type for an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the stream item type for.

Returns

The stream item type for an item received by an ex receive operation.

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

Examples:

[ex_receive_example.c](#).

8.6.5.20 `int STAR_API_CC STAR_rxOperationExGetLinkSpeedEvent (_In_ STAR_TRANSFER_OPERATION *
pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_SPEED_EVENT * pLinkSpeedEvent)`

Gets a link speed event from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the link speed event from.
<i>pLinkSpeed↔ Event</i>	A pointer to a link speed event that will be populated with data from the item.

Returns

1 if a link speed event was successfully populated from the item, or -1 if an error occurred.

Added in version:

[STAR-System v5.00 Beta 1 - 24th March 2021](#)

Examples:

[ex_receive_example.c](#).

8.6.5.21 `int STAR_API_CC STAR_rxOperationExGetLinkStateEvent ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_STATE_EVENT * pLinkStateEvent )`

Gets a link state event from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the link state event from.
<i>pLinkStateEvent</i>	A pointer to a link state event that will be populated with data from the item.

Returns

1 if a link state event was successfully populated from the item, or -1 if an error occurred.

Added in version:

[STAR-System v5.00 Beta 1 - 24th March 2021](#)

Examples:

[ex_receive_example.c](#).

8.6.5.22 `int STAR_API_CC STAR_rxOperationExGetNumItems ( _In_ STAR_TRANSFER_OPERATION * pTransferOp )`

Gets the number of items received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
--------------------	--

Returns

The number of items received by an ex receive operation, or -1 if an error occurred.

Added in version:

[STAR-System v4.00 - 19th November 2019](#)

Examples:

[ex_receive_example.c](#).

8.6.5.23 **STAR_TRANSFER_STATUS** **STAR_API_CC** **STAR_rxOperationExGetStatus** (**_In_**
STAR_TRANSFER_OPERATION * *pTransferOp*)

Gets the status of an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
--------------------	--

Returns

The status of the ex receive operation.

Added in version:

STAR-System v4.00 - 19th November 2019

8.6.5.24 `int STAR_API_CC STAR_rxOperationExGetTimeCode ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMECODE * pTimeCode )`

Gets a time-code from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the time-code from.
<i>pTimeCode</i>	A pointer to a time-code that will be populated with data from the item.

Returns

1 if a time-code was successfully populated from the item, or -1 if an error occurred.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Examples:

[ex_receive_example.c](#).

8.6.5.25 `int STAR_API_CC STAR_rxOperationExGetTimestampEvent ( _In_ STAR_TRANSFER_OPERATION * pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMESTAMP_EVENT * pTimestampEvent )`

Gets a timestamp event from an item received by an ex receive operation.

Parameters

<i>pTransferOp</i>	A pointer to a STAR_TRANSFER_OPERATION .
<i>item</i>	The index of the item to get the timestamp event from.
<i>pTimestampEvent</i>	A pointer to a timestamp event that will be populated with data from the item.

Returns

1 if a timestamp event was successfully populated from the item, or -1 if an error occurred.

Added in version:

STAR-System v5.04 - 20th February 2023

8.6.5.26 `int STAR_API_CC STAR_submitTransferOperation ( _In_ STAR_CHANNEL_ID channelId, _In_ STAR_TRANSFER_OPERATION * transferOp )`

Submits a single transfer operation.

Once a transfer operation has completed, it can be resubmitted. This can be useful for receiving a number of packets in a loop, or transmitting the same packet repeatedly. Note that if an operation is resubmitted before it has completed, the original operation will be cancelled and then the new one submitted.

Parameters

	<i>channelId</i>	Channel to submit operation on
<i>in</i>	<i>transferOp</i>	STAR_TRANSFER_OPERATION to submit

Returns

1 if operation was successfully submitted, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send_example.c](#), [advanced_two_way_example.c](#), [brickMk2_configuration_example.c](#), [broadcast_message_example.c](#), [ex_receive_example.c](#), [link_events.c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_tester.c](#), [time-code_example.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_callback_example.c](#), and [transmit_errors_example.c](#).

8.6.5.27 `int STAR_API_CC STAR_submitTransferOperationList ( STAR_CHANNEL_ID channelId, _In_count_(count) STAR_TRANSFER_OPERATION ** transferOps, unsigned int count )`

Submits a list of transfer operations.

Once a transfer operation has completed, it can be resubmitted. This can be useful for receiving a number of packets in a loop, or transmitting the same packet repeatedly. Note that if an operation is resubmitted before it has completed, the original operation will be cancelled and then the new one submitted. This means it is not useful to submit a list containing multiple instances of the same operation.

Parameters

	<i>channelId</i>	Channel to submit operations on
<i>in</i>	<i>transferOps</i>	Array of pointers to STAR_TRANSFER_OPERATION to be submitted
	<i>count</i>	Count of operations in array

Returns

1 if all operations were successfully submitted, else 0

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Declaration changed in version:

STAR-System v1.1 - 20th December 2011

Examples:

[transfer_operation_list_send_example.c](#), and [transfer_operation_list_send_receive_example.c](#).

8.6.5.28 STAR_TRANSFER_STATUS STAR_API_CC STAR_transmitPacket (_In_ STAR_CHANNEL_ID *channelId*, _In_opt_bytecount_(packetLength) void * *pPacketData*, _In_ unsigned int *packetLength*, _In_ STAR_EOP_TYPE *eopType*, _In_ int *timeout*)

Transmit a single packet on a previously opened channel from the buffer specified.

This function is provided to simplify the process of transmitting packets, but is not as efficient or as flexible as using [STAR_createTxOperation\(\)](#), [STAR_submitTransferOperation\(\)](#), etc. This function should not be used if high performance or low CPU utilisation is required.

Parameters

<i>channelId</i>	The identifier of a previously opened channel. The channel must have been opened as an out or in/out channel
<i>pPacketData</i>	A pointer to a previously allocated buffer which should contain the packet to be transmitted
<i>packetLength</i>	The length of the buffer, and therefore the length of the packet
<i>eopType</i>	The end of packet marker type to be added to the end of the packet
<i>timeout</i>	The maximum time in milliseconds to wait for the packet to be transmitted, or -1 to wait indefinitely

Returns

the status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

Examples:

[simple_send_and_receive_example.c](#), [simple_send_example.c](#), and [simple_two_way_example.c](#).

8.6.5.29 int STAR_API_CC STAR_unregisterTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION * *pTransferOperation*, STAR_TransferOperationListenerFunc *listenerFunc*)

Unregisters a call-back function that will be called when an operation completes.

Parameters

<i>pTransferOperation</i>	Operation to remove listener for
<i>listenerFunc</i>	Call-back function

Returns

Non-Zero if call-back was unregistered successfully

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[transfer_callback_example.c](#).

8.6.5.30 STAR_TRANSFER_STATUS STAR_API_CC STAR_waitOnTransferOperationCompletion (_In_ STAR_TRANSFER_OPERATION * *pTransferOperation*, int *timeout*)

Blocks until a given transfer operation has completed, or the timeout period expires.

Note that the operation will continue to transmit or receive once this function returns if it has not yet completed. The operation will only be stopped if it completes, there is an error, or it is cancelled by calling [STAR_cancelTransfer↵Operation\(\)](#). If the operation passed to this function has been submitted but has not yet completed when the function returns, the return value will be [STAR_TRANSFER_STATUS_STARTED](#) as the operation has started but not yet completed. This function can be called multiple times for the same operation to wait until that operation completes.

Parameters

<i>in</i>	<i>pTransfer↵Operation</i>	Operation to wait on
	<i>timeout</i>	Max time in milliseconds to wait, or -1 to wait indefinitely

Returns

result status of the operation

Added in version:

STAR-System v1.1 - 20th December 2011

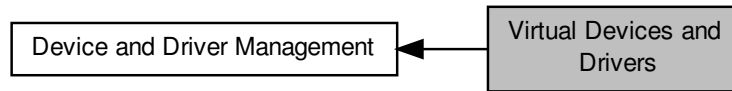
Examples:

[advanced_address_example.c](#), [advanced_continuous_receive_example.c](#), [advanced_multiple_packet_↵example.c](#), [advanced_receive_example.c](#), [advanced_send_and_receive_example.c](#), [advanced_send_↵example.c](#), [advanced_two_way_example.c](#), [brickMk2_configuration_example.c](#), [broadcast_message_↵example.c](#), [ex_receive_example.c](#), [link_events.c](#), [rmap_target_example.c](#), [star-test.c](#), [star_performance_↵tester.c](#), [time-code_example.c](#), [time-code_test.c](#), [timestamp_test.c](#), [transfer_operation_list_send_example.c](#), [transfer_operation_list_send_receive_example.c](#), and [transmit_errors_example.c](#).

8.7 Virtual Devices and Drivers

These are functions that are used to manage the properties of virtual drivers and devices.

Collaboration diagram for Virtual Devices and Drivers:



Functions

- `int STAR_API_CC STAR_isDriverVirtual (STAR_DRIVER_ID driverId)`
Gets whether or not a given driver is virtual.
- `int STAR_API_CC STAR_isDeviceVirtual (STAR_DEVICE_ID deviceId)`
Gets whether or not a given device is virtual.

8.7.1 Detailed Description

These are functions that are used to manage the properties of virtual drivers and devices.

8.7.2 Function Documentation

8.7.2.1 `int STAR_API_CC STAR_isDeviceVirtual ( STAR_DEVICE_ID deviceId )`

Gets whether or not a given device is virtual.

Parameters

<i>deviceId</i>	Identifier for device to be checked
-----------------	-------------------------------------

Returns

1 if the device is virtual, otherwise 0

Added in version:

STAR-System v1.12 - 14th November 2012

8.7.2.2 `int STAR_API_CC STAR_isDriverVirtual ( STAR_DRIVER_ID driverId )`

Gets whether or not a given driver is virtual.

Parameters

<i>driverId</i>	Identifier for driver to be checked
-----------------	-------------------------------------

Returns

1 if the driver is virtual, otherwise 0

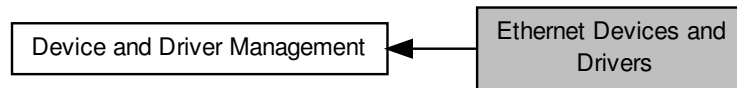
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

8.8 Ethernet Devices and Drivers

These are functions that are used to manage Ethernet drivers and devices.

Collaboration diagram for Ethernet Devices and Drivers:



Functions

- `STAR_DRIVER_ID STAR_API_CC STAR_openEthernetDriver (void)`
Opens the Ethernet driver.
- `int STAR_API_CC STAR_closeEthernetDriver (STAR_DRIVER_ID driverId)`
Closes the Ethernet driver.
- `STAR_DEVICE_ID STAR_API_CC STAR_openEthernetDevice (unsigned char ipAddr[4])`
Opens an Ethernet connected device.
- `int STAR_API_CC STAR_closeEthernetDevice (STAR_DEVICE_ID deviceId)`
Closes an open Ethernet connected device.
- `_Check_return_ int STAR_API_CC STAR_getDeviceIPAddress (STAR_DEVICE_ID deviceId, _Out_↔
bytecap_(IPAddressLength) U8 *pIPAddress, U8 IPAddressLength)`
Gets the IP address of a specific Ethernet device (i.e.
- `_Check_return_ STAR_IP_ADDRESS_TYPE **STAR_API_CC STAR_getIPAddresses (STAR_DRIVER↔
_ID driverId, _Out_ int *pNumDevices)`
*Scans for all attached Ethernet devices and stores their IP addresses in an array of IP address structures of STAR↔
_IP_ADDRESS_TYPE.*
- `int STAR_API_CC STAR_destroyIPAddresses (_Post_ptr_invalid_ STAR_IP_ADDRESS_TYPE **ppIP↔
Addrs, _In_ U8 numDevices)`
Frees the memory allocated by STAR_getIPAddresses().

8.8.1 Detailed Description

These are functions that are used to manage Ethernet drivers and devices.

8.8.2 Function Documentation

8.8.2.1 `int STAR_API_CC STAR_closeEthernetDevice ( STAR_DEVICE_ID deviceId )`

Closes an open Ethernet connected device.

Parameters

<i>deviceId</i>	The identifier of the device to close
-----------------	---------------------------------------

Returns

1 if the device was successfully closed, else 0

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

8.8.2.2 int STAR_API_CC STAR_closeEthernetDriver (STAR_DRIVER_ID *driverId*)

Closes the Ethernet driver.

Parameters

<i>driverId</i>	The identifier of the driver to close
-----------------	---------------------------------------

Returns

1 if the driver was successfully closed, else 0

Note

Closing the driver will automatically close any open Ethernet devices, and may result in data loss if transmits or receives are in progress.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

8.8.2.3 int STAR_API_CC STAR_destroyIPAddresses (_Post_ptr_invalid_ STAR_IP_ADDRESS_TYPE ** *ppIPAddr*, _In_ U8 *numDevices*)

Frees the memory allocated by STAR_getIPAddresses().

Parameters

out	<i>ppIPAddr</i>	<i>ppIPAddr</i> holds the address to the array of the IP Addresses returned by STAR_getIPAddresses()
in	<i>numDevices</i>	Number of Ethernet devices discovered by STAR_getIPAddresses()

Returns

1 if the memory was successfully freed, 0 otherwise

8.8.2.4 _Check_return_ int STAR_API_CC STAR_getDeviceIPAddress (STAR_DEVICE_ID *deviceId*, _Out_bytecap_(IPAddressLength) U8 * *pIPAddr*, U8 *IPAddressLength*)

Gets the IP address of a specific Ethernet device (i.e.

Device ID is known).

Parameters

<i>deviceId</i>	The identifier of the device
-----------------	------------------------------

<i>pIPAddress</i>	A pointer to the array the IP address shall be stored in
<i>IPAddressLength</i>	Size of the array in bytes (e.g. 4 bytes for IPv4)

Returns

1 if the IP address was successfully retrieved, else 0

Added in version:

STAR-System v5.00 Beta 2 - 30th September 2021

8.8.2.5 `_Check_return_ STAR_IP_ADDRESS_TYPE** STAR_API_CC STAR_getIPAddresses ( STAR_DRIVER_ID driverId, _Out_ int * pNumDevices )`

Scans for all attached Ethernet devices and stores their IP addresses in an array of IP address structures of `STAR_IP_ADDRESS_TYPE`.

The memory for the array is allocated dynamically. Must call `STAR_destroyIPAddresses()` when no longer using the array.

Parameters

	<i>driverId</i>	The identifier of the driver
out	<i>pNumDevices</i>	Number of discovered Ethernet devices. Negative value indicates failure in allocating the required memory.

Returns

An array of structs for each device or NULL if there was an error

8.8.2.6 `STAR_DEVICE_ID STAR_API_CC STAR_openEthernetDevice ( unsigned char ipAddr[4] )`

Opens an Ethernet connected device.

Parameters

	<i>ipAddr</i>	The IPv4 address of the device to open
--	---------------	--

Returns

The identifier of the opened device, or 0 if opening failed

Note

The Ethernet driver will automatically be opened, if it has not already been opened by calling `STAR_openEthernetDriver()`.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

8.8.2.7 `STAR_DRIVER_ID STAR_API_CC STAR_openEthernetDriver ( void )`

Opens the Ethernet driver.

Returns

The identifier of the opened driver, or 0 if opening failed

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

8.9 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



Enumerations

- enum `STAR_CFG_PCIMK2_LINK_FREQ` {
`STAR_CFG_PCIMK2_LINK_FREQ_120` = 0, `STAR_CFG_PCIMK2_LINK_FREQ_128` = 5, `STAR_CFG_PCIMK2_LINK_FREQ_140` = 1, `STAR_CFG_PCIMK2_LINK_FREQ_150` = 6,
`STAR_CFG_PCIMK2_LINK_FREQ_160` = 2, `STAR_CFG_PCIMK2_LINK_FREQ_180` = 3, `STAR_CFG_PCIMK2_LINK_FREQ_200` = 4 }

Frequencies a link may run at.

Functions

- int `STAR_API_CC_CFG_PCIMK2_setLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, `STAR_CFG_PCIMK2_LINK_FREQ` linkFreq)
Sets the Link Clock Frequency on a given link.
- int `STAR_API_CC_CFG_PCIMK2_getLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, `STAR_CFG_PCIMK2_LINK_FREQ` *pLinkFreq)
Gets the Link Clock Frequency for a given link.

8.9.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

8.9.2 Enumeration Type Documentation

8.9.2.1 enum `STAR_CFG_PCIMK2_LINK_FREQ`

Frequencies a link may run at.

These are divided by a divider set by `CFG_PCIMK2_setLinkRateDivider()` to generate the desired link speed.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Enumerator

`STAR_CFG_PCIMK2_LINK_FREQ_120` 120MHz

`STAR_CFG_PCIMK2_LINK_FREQ_128` 128MHz

Declaration changed in version:

STAR-System v2.0 - 3rd July 2013

STAR_CFG_PCIMK2_LINK_FREQ_140 140MHz

STAR_CFG_PCIMK2_LINK_FREQ_150 150MHz

Declaration changed in version:

STAR-System v2.0 - 3rd July 2013

STAR_CFG_PCIMK2_LINK_FREQ_160 160MHz

STAR_CFG_PCIMK2_LINK_FREQ_180 180MHz

STAR_CFG_PCIMK2_LINK_FREQ_200 200MHz

8.9.3 Function Documentation

8.9.3.1 `int STAR_API_CC_CFG_PCIMK2_getLinkClockFrequency ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ * pLinkFreq )`

Gets the Link Clock Frequency for a given link.

Note

This function is for use with SpaceWire PCI Mk2 and cPCI Mk2 devices only. For SpaceWire PCIe and SPLT devices please use `CFG_MK2_getBaseTransmitClock()` instead. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices please use `CFG_BRICK_MK2_getLinkClockFrequency()`. For SpaceWire Brick Mk3 and Brick Mk4 devices, please use `CFG_BRICK_MK3_getBaseTransmitClock()`.

Parameters

	<i>deviceID</i>	Device to get the links frequency from.
	<i>linkNum</i>	Link to get frequency for.
out	<i>pLinkFreq</i>	Pointer to value that will be updated with the given links frequency.

Returns

1 if source frequency was successfully obtained, else 0.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

8.9.3.2 `int STAR_API_CC_CFG_PCIMK2_setLinkClockFrequency ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_PCIMK2_LINK_FREQ linkFreq )`

Sets the Link Clock Frequency on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{LinkClockRate}{LinkClockRateDivider}$$

Where *LinkClockRate* is linkFreq and *LinkClockRateDivider* is set by `CFG_MK2_setLinkRateDivider()`.

Note

This function is for use with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices only. For [SpaceWire PCIe](#) and [SPLT](#) devices please use [CFG_MK2_setBaseTransmitClock\(\)](#) instead. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices please use [CFG_BRICK_MK2_setLinkClockFrequency\(\)](#). For [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices, please use [CFG_BRICK_MK3_setBaseTransmitClock\(\)](#).

Parameters

<i>deviceID</i>	Device to modify a links frequency on.
<i>linkNum</i>	Link to have its frequency modified.
<i>linkFreq</i>	Link clock frequency to be set.

Returns

1 if source frequency was successfully set, else 0.

Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

8.10 Link Speed

Functions in this group deal with setting the link speeds of PCIe Mk2 devices.

Collaboration diagram for Link Speed:



Functions

- int `STAR_API_CC_CFG_PCIE_MK2_setTransmitSignallingRate` (`STAR_DEVICE_ID` deviceID, U8 linkNum, U32 signallingRate)
Sets the transmit bit rate on a given link.
- int `STAR_API_CC_CFG_PCIE_MK2_getTransmitSignallingRate` (`STAR_DEVICE_ID` deviceID, U8 linkNum, _Out_ U32 *pSignallingRate)
Gets the transmit bit rate for a given link.
- int `STAR_API_CC_CFG_PCIE_MK2_setDecimalTxSignallingRate` (`STAR_DEVICE_ID` deviceID, U8 link↔Num, double signallingRate)
Sets the transmit bit rate on a given link.
- int `STAR_API_CC_CFG_PCIE_MK2_getDecimalTxSignallingRate` (`STAR_DEVICE_ID` deviceID, U8 link↔Num, _Out_ double *pSignallingRate)
Gets the transmit bit rate for a given link.

8.10.1 Detailed Description

Functions in this group deal with setting the link speeds of PCIe Mk2 devices.

8.10.2 Function Documentation

8.10.2.1 int `STAR_API_CC_CFG_PCIE_MK2_getDecimalTxSignallingRate` (`STAR_DEVICE_ID` deviceID, U8 linkNum, _Out_ double * pSignallingRate)

Gets the transmit bit rate for a given link.

Parameters

	<i>deviceID</i>	Device to get the transmit bit rate from.
	<i>linkNum</i>	Link to get the transmit bit rate for.
out	<i>pSignallingRate</i>	Pointer to decimal value that will be updated with the given link's transmit bit rate in Mbit/s.

Returns

1 if transmit bit rate was successfully obtained, else 0.

Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

Supported devices:

SpaceWire PCIe Mk2

8.10.2.2 `int STAR_API_CC_CFG_PCIE_MK2_getTransmitSignallingRate ( STAR_DEVICE_ID deviceID, U8 linkNum, Out_ U32 * pSignallingRate )`

Gets the transmit bit rate for a given link.

The transmit bit rate is rounded to the nearest integer. Use `CFG_PCIE_MK2_getDecimalTxSignallingRate()` to access the decimal transmit bit rate.

Parameters

	<i>deviceID</i>	Device to get the transmit bit rate from.
	<i>linkNum</i>	Link to get the transmit bit rate for.
out	<i>pSignallingRate</i>	Pointer to integer value that will be updated with the given link's transmit bit rate in Mbit/s.

Returns

1 if transmit bit rate was successfully obtained, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.10.2.3 `int STAR_API_CC_CFG_PCIE_MK2_setDecimalTxSignallingRate ( STAR_DEVICE_ID deviceID, U8 linkNum, double signallingRate )`

Sets the transmit bit rate on a given link.

The transmit bit rate of a link is given in Mbit/s. Decimal bit rates between 2 Mbit/s and 400 Mbit/s inclusive are supported.

An exact rate may not be achievable and may be rounded up or down slightly. Use `CFG_PCIE_MK2_getDecimalTxSignallingRate()` to access the rate set.

Parameters

	<i>deviceID</i>	Device to modify a link's transmit bit rate on.
	<i>linkNum</i>	Link to have its transmit bit rate modified.
	<i>signallingRate</i>	Decimal transmit bit rate in Mbit/s.

Returns

1 if link's transmit bit rate was successfully set, else 0.

Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

Supported devices:

SpaceWire PCIe Mk2

8.10.2.4 `int STAR_API_CC_CFG_PCIE_MK2_setTransmitSignallingRate ( STAR_DEVICE_ID deviceID, U8 linkNum, U32 signallingRate )`

Sets the transmit bit rate on a given link.

The transmit bit rate of a link is given in Mbit/s. Integer bit rates between 2 Mbit/s and 400 Mbit/s inclusive are supported.

An exact rate may not be achievable and may be rounded up or down slightly. Use [CFG_PCIE_MK2_getDecimalTxSignallingRate\(\)](#) to access the rate set.

Parameters

<i>deviceID</i>	Device to modify a link's transmit bit rate on.
<i>linkNum</i>	Link to have its transmit bit rate modified.
<i>signallingRate</i>	Integer transmit bit rate in Mbit/s.

Returns

1 if link's transmit bit rate was successfully set, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.11 Broadcast Controller

Functions in this group deal with PCIe Mk2 broadcasts.

Collaboration diagram for Broadcast Controller:



Functions

- `int STAR_API_CC_CFG_PCIE_MK2_enableBroadcastControllerLegacyMode (STAR_DEVICE_ID deviceID)`
Enables broadcast controller legacy mode.
- `int STAR_API_CC_CFG_PCIE_MK2_getBroadcastControllerLegacyModeEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`
Gets whether broadcast legacy mode has been enabled for the device.
- `int STAR_API_CC_CFG_PCIE_MK2_disableBroadcastControllerLegacyMode (STAR_DEVICE_ID deviceID)`
Disables broadcast controller legacy mode.
- `int STAR_API_CC_CFG_PCIE_MK2_setInterruptCodeDistributionPorts (STAR_DEVICE_ID deviceID, U32 portMask)`
This function is used to specify which output ports interrupt codes are forwarded on.
- `int STAR_API_CC_CFG_PCIE_MK2_getInterruptCodeDistributionPorts (STAR_DEVICE_ID deviceID, _Out_ U32 *pPortMask)`
This function is used to obtain which output ports interrupt codes are forwarded on.

8.11.1 Detailed Description

Functions in this group deal with PCIe Mk2 broadcasts.

These can be used to support distributed interrupt codes.

8.11.2 Function Documentation

8.11.2.1 `int STAR_API_CC_CFG_PCIE_MK2_disableBroadcastControllerLegacyMode ( STAR_DEVICE_ID deviceID )`

Disables broadcast controller legacy mode.

When disabled, interrupt codes are supported. A broadcast code with type field 0b00 is interpreted as a time-code. A broadcast code with type field 0b10 is interpreted as a distributed interrupt. Broadcasts with type fields equal to 0b01 or 0b11 are discarded.

Parameters

<i>deviceID</i>	Device to disable broadcast controller legacy mode for.
-----------------	---

Returns

1 if broadcast controller legacy mode was successfully disabled for the device, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.11.2.2 `int STAR_API_CC_CFG_PCIE_MK2_enableBroadcastControllerLegacyMode ( STAR_DEVICE_ID deviceId )`

Enables broadcast controller legacy mode.

When enabled, all broadcast codes are interpreted as time-codes. Codes which do not have flags field set to 0b00 (time-code) will be forwarded according to the time-code flag mode.

Parameters

<i>deviceId</i>	Device to enable broadcast controller legacy mode for.
-----------------	--

Returns

1 if broadcast controller legacy mode was successfully enabled for the device, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.11.2.3 `int STAR_API_CC_CFG_PCIE_MK2_getBroadcastControllerLegacyModeEnabled ( STAR_DEVICE_ID deviceId, _Out_ int * pEnabled )`

Gets whether broadcast legacy mode has been enabled for the device.

When broadcast legacy mode is disabled, interrupt codes are supported. A broadcast code with type field 0b00 is interpreted as a time-code. A broadcast code with type field 0b10 is interpreted as a distributed interrupt. Broadcasts with type fields equal to 0b01 or 0b11 are discarded. When broadcast legacy mode is enabled, all broadcast codes are interpreted as time-codes. Codes which do not have flags field set to 0b00 (time-code) will be forwarded according to the time-code flag mode.

Parameters

	<i>deviceId</i>	Device to check.
<i>out</i>	<i>pEnabled</i>	User supplied value that will be updated to 1 if broadcast controller legacy mode is enabled for the device, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.11.2.4 `int STAR_API_CC_CFG_PCIE_MK2_getInterruptCodeDistributionPorts ( STAR_DEVICE_ID deviceID, _Out_ U32 * pPortMask )`

This function is used to obtain which output ports interrupt codes are forwarded on.

Parameters

	<i>deviceId</i>	Device to get the interrupt code distribution ports for.
out	<i>pPortMask</i>	Bit mask of ports that interrupt code distribution is enabled on.

Note

Bit 1 corresponds to port 1.

Returns

1 if the interrupt code distribution ports could be obtained, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.11.2.5 `int STAR_API_CC_CFG_PCIE_MK2_setInterruptCodeDistributionPorts ( STAR_DEVICE_ID deviceId, U32 portMask )`

This function is used to specify which output ports interrupt codes are forwarded on.

Parameters

	<i>deviceId</i>	Device to set the interrupt code distribution ports for.
	<i>portMask</i>	Bit mask of ports that interrupt code distribution should be enabled on.

Note

Bit 1 corresponds to port 1.

Returns

1 if the interrupt code distribution ports could be set, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

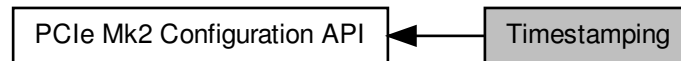
Supported devices:

SpaceWire PCIe Mk2

8.12 Timestamping

Functions in this group deal with PCIe Mk2 packet timestamping.

Collaboration diagram for Timestamping:



Functions

- `int STAR_API_CC_CFG_PCIE_MK2_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)`
Sets the timestamping method to use for the given device.
- `int STAR_API_CC_CFG_PCIE_MK2_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively enables receive timestamp events on a given port.
- `int STAR_API_CC_CFG_PCIE_MK2_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`
Gets whether receive timestamp events are enabled on a specific port.
- `int STAR_API_CC_CFG_PCIE_MK2_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`
Selectively disables receive timestamp events on a given port.
- `int STAR_API_CC_CFG_PCIE_MK2_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)`
Sets the frequency of each generated pulse.
- `int STAR_API_CC_CFG_PCIE_MK2_getPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)`
Gets the frequency of each generated pulse.

8.12.1 Detailed Description

Functions in this group deal with PCIe Mk2 packet timestamping.

These can be used to receive start of packet and end of packet timestamps for data that is received on the SpaceWire links.

8.12.2 Function Documentation

8.12.2.1 `int STAR_API_CC_CFG_PCIE_MK2_disableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables receive timestamp events on a given port.

Disabling receive timestamp events will result in no timestamp information being provided after the packet.

Parameters

<i>deviceID</i>	Device to disable receive timestamp events for a port on.
<i>portNum</i>	Port to disable receive timestamp events on.

Returns

1 if receive timestamp events were successfully disabled, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.12.2.2 `int STAR_API_CC_CFG_PCIE_MK2_enableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables receive timestamp events on a given port.

Enabling receive timestamp events will result in the timestamp that the last packet was received at to be provided as a STAR_STREAM_ITEM_TYPE_RX_TIMESTAMP.

Parameters

<i>deviceID</i>	Device to enable receive timestamp events for a port on.
<i>portNum</i>	Port to enable receive timestamp events on.

Returns

1 if receive timestamp packets were successfully enabled, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.12.2.3 `int STAR_API_CC_CFG_PCIE_MK2_getPulseGeneratorFrequency ( STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ * pFrequency )`

Gets the frequency of each generated pulse.

Parameters

<i>deviceID</i>	Device to get the pulse generator frequency value for.
<i>out</i> <i>pFrequency</i>	Pointer to value that will be updated with the frequency between each pulse cycle.

Returns

1 if frequency value was successfully obtained, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.12.2.4 `int STAR_API_CC_CFG_PCIE_MK2_getRxTimestampEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether receive timestamp events are enabled on a specific port.

Parameters

	<i>deviceId</i>	Device to get whether receive timestamp events are enabled for a port.
	<i>portNum</i>	Port to get receive timestamp events for.
out	<i>pEnabled</i>	User supplied value that will be updated to 1 if receive timestamp events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.12.2.5 `int STAR_API_CC_CFG_PCIE_MK2_setPulseGeneratorFrequency ( STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency )`

Sets the frequency of each generated pulse.

Parameters

	<i>deviceId</i>	Device to set the pulse generator frequency for.
	<i>frequency</i>	The frequency value to set.

Returns

1 if frequency was successfully set, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.12.2.6 `int STAR_API_CC_CFG_PCIE_MK2_setTimestampMethod ( STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod )`

Sets the timestamping method to use for the given device.

Note

Timestamp method can be accessed using `CFG_BRICK_MK3_getTimestampMethod`.

The device will trigger on the rising edge of the external pulse but the Triggering API can be used to change it to trigger on the falling edge using e.g. `TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode()` and `TRIGGER_BRICK_MK3_enableExtTriggerInvert()`.

Parameters

<i>deviceID</i>	Device to set the timestamping method for.
<i>timestamp</i> ↔ <i>Method</i>	Timestamp method to enable for device.

Returns

1 if timestamp method was successfully set, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

8.13 Error Injection

Functions in this group deal with error injection.

Collaboration diagram for Error Injection:



Functions

- `int STAR_API_CC_CFG_PCIE_MK2_injectErrors (STAR_DEVICE_ID deviceId, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`
Inject the specified errors on a given port.

8.13.1 Detailed Description

Functions in this group deal with error injection.

8.13.2 Function Documentation

8.13.2.1 `int STAR_API_CC_CFG_PCIE_MK2_injectErrors ( STAR_DEVICE_ID deviceId, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS * pErrors )`

Inject the specified errors on a given port.

Parameters

<i>deviceId</i>	Device to inject the errors on.
<i>portNum</i>	Port to inject the errors on.
<i>pErrors</i>	Pointer to structure specifying which errors are to be injected.

Returns

1 if the errors were successfully injected, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

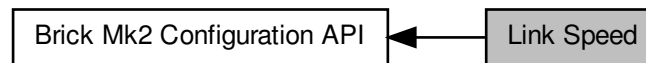
Supported devices:

SpaceWire PCIe Mk2

8.14 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



Enumerations

- enum `STAR_CFG_BRICK_MK2_LINK_FREQ` {
`STAR_CFG_BRICK_MK2_LINK_FREQ_120 = 0`, `STAR_CFG_BRICK_MK2_LINK_FREQ_130 = 1`, `STAR_CFG_BRICK_MK2_LINK_FREQ_140 = 2`, `STAR_CFG_BRICK_MK2_LINK_FREQ_150 = 3`,
`STAR_CFG_BRICK_MK2_LINK_FREQ_160 = 4`, `STAR_CFG_BRICK_MK2_LINK_FREQ_180 = 5`, `STAR_CFG_BRICK_MK2_LINK_FREQ_200 = 6` }

Frequencies a link on a Brick Mk2 compatible device may run at.

Functions

- int `STAR_API_CC_CFG_BRICK_MK2_setLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, `STAR_CFG_BRICK_MK2_LINK_FREQ` linkFreq)
Sets the Link Clock Frequency on a given link.
- int `STAR_API_CC_CFG_BRICK_MK2_getLinkClockFrequency` (`STAR_DEVICE_ID` deviceId, U8 linkNum, _Out_ `STAR_CFG_BRICK_MK2_LINK_FREQ` *pLinkFreq)
Gets the Link Clock Frequency for a given link.

8.14.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

8.14.2 Enumeration Type Documentation

8.14.2.1 enum `STAR_CFG_BRICK_MK2_LINK_FREQ`

Frequencies a link on a Brick Mk2 compatible device may run at.

These are divided by a divider set by `CFG_MK2_setLinkRateDivider()` to generate the desired link speed.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Enumerator

`STAR_CFG_BRICK_MK2_LINK_FREQ_120` 120MHz

`STAR_CFG_BRICK_MK2_LINK_FREQ_130` 130MHz

STAR_CFG_BRICK_MK2_LINK_FREQ_140 140MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_150 150MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_160 160MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_180 180MHz
STAR_CFG_BRICK_MK2_LINK_FREQ_200 200MHz

8.14.3 Function Documentation

8.14.3.1 **int STAR_API_CC_CFG_BRICK_MK2_getLinkClockFrequency (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_BRICK_MK2_LINK_FREQ * pLinkFreq)**

Gets the Link Clock Frequency for a given link.

Note

This function is for use with SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices only. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices please use [CFG_MK2_getBaseTransmitClock\(\)](#). For SpaceWire PCI Mk2 and cPCI Mk2 devices please use [CFG_PCIMK2_setLinkClockFrequency\(\)](#). For SpaceWire Brick Mk3 and Brick Mk4 devices, please use [CFG_BRICK_MK3_getBaseTransmitClock\(\)](#).

Parameters

	<i>deviceId</i>	Device to get the link's frequency from.
	<i>linkNum</i>	Link to get frequency for.
out	<i>pLinkFreq</i>	Pointer to value that will be updated with the given links frequency.

Returns

1 if source frequency was successfully obtained, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S

8.14.3.2 **int STAR_API_CC_CFG_BRICK_MK2_setLinkClockFrequency (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_BRICK_MK2_LINK_FREQ linkFreq)**

Sets the Link Clock Frequency on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{LinkClockRate}{LinkClockRateDivider}$$

Where *LinkClockRate* is linkFreq and *LinkClockRateDivider* is set by [CFG_MK2_setLinkRateDivider\(\)](#).

Note

This function is for use with SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices only. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices please use [CFG_MK2_setBaseTransmitClock\(\)](#). For SpaceWire PCI Mk2 and cPCI Mk2 devices please use [CFG_PCIMK2_setLinkClockFrequency\(\)](#). For SpaceWire Brick Mk3 and Brick Mk4 devices, please use [CFG_BRICK_MK3_setBaseTransmitClock\(\)](#).

Parameters

<i>deviceID</i>	Device to modify a link's frequency on.
<i>linkNum</i>	Link to have its frequency modified.
<i>linkFreq</i>	Link clock frequency to be set.

Returns

1 if source frequency was successfully set, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Router Mk2S

8.15 Interface Mode

Functions in this group deal with managing the device in Interface mode.

Collaboration diagram for Interface Mode:



Functions

- `int STAR_API_CC CFG_BRICK_MK2_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables interface mode on a given port of a supported USB device.
- `int STAR_API_CC CFG_BRICK_MK2_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether interface mode is enabled on a specific port of a supported USB device.
- `int STAR_API_CC CFG_BRICK_MK2_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables interface mode on a given port of a supported USB device.
- `int STAR_API_CC CFG_BRICK_MK2_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Enables identification of the source port of a received packet on a given port, when in interface mode.
- `int STAR_API_CC CFG_BRICK_MK2_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether identify source is enabled for a specific port.
- `int STAR_API_CC CFG_BRICK_MK2_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Disables source port identification for a given port.
- `int STAR_API_CC CFG_BRICK_MK2_setPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, U8 address)`
Sets the address a packet received on a given port should be routed to when interface mode is enabled.
- `int STAR_API_CC CFG_BRICK_MK2_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`
Gets the address a packet received on a given port should be routed to when interface mode is enabled.

8.15.1 Detailed Description

Functions in this group deal with managing the device in Interface mode.

8.15.2 Function Documentation

8.15.2.1 `int STAR_API_CC CFG_BRICK_MK2_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable source port identification for a given port on.
<i>portNum</i>	Port to disable source identification on.

Returns

1 if source identification was successfully disabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.2 `int STAR_API_CC_CFG_BRICK_MK2_disableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables interface mode on a given port of a supported USB device.

When interface mode is disabled, the device operates in routing mode.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable interface mode for a port on.
<i>portNum</i>	Port to disable interface mode on.

Returns

1 if interface mode was successfully disabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.3 `int STAR_API_CC_CFG_BRICK_MK2_enableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Enables identification of the source port of a received packet on a given port, when in interface mode.

Interface mode ([CFG_MK2_enableInterfaceMode\(\)](#)) and Identify Source ([CFG_MK2_enableIdentifySource\(\)](#)) must be enabled globally for this to have an effect.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable source port identification for a given port on.
<i>portNum</i>	Port to enable source identification on.

Returns

1 if source identification was successfully enabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1](#) - 8th February 2013

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.4 `int STAR_API_CC_CFG_BRICK_MK2_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables interface mode on a given port of a supported USB device.

Interface mode must be enabled globally ([CFG_MK2_enableInterfaceMode\(\)](#)) for interface mode on a port to be enabled.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable interface mode for a port on.
<i>portNum</i>	Port to enable interface mode on.

Returns

1 if interface mode was successfully enabled, else 0.

Added in version:

[STAR-System v2.0 Beta 1](#) - 8th February 2013

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.5 `int STAR_API_CC_CFG_BRICK_MK2_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether identify source is enabled for a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to check.
<i>portNum</i>	Port to check.
<i>pEnabled</i>	User supplied value that will be updated to 1 if identify source is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.6 `int STAR_API_CC_CFG_BRICK_MK2_getInterfaceModeOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether interface mode is enabled on a specific port of a supported USB device.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to get interface mode enabled for a port on.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if interface mode is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.7 `int STAR_API_CC_CFG_BRICK_MK2_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

	<i>deviceID</i>	Device to get port routing address from.
	<i>portNum</i>	Port to get port routing address from.
<i>out</i>	<i>pAddress</i>	Pointer to value that will be updated to contain the address.

Returns

1 if the address could be obtained, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.15.2.8 `int STAR_API_CC_CFG_BRICK_MK2_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

	<i>deviceID</i>	Device to have one of its port's port routing address changed.
	<i>portNum</i>	Port to have its port routing address modified.
	<i>address</i>	New port routing address.

Returns

1 if the address could be set, else 0.

Added in version:

[STAR-System v2.0 Beta 1 - 8th February 2013](#)

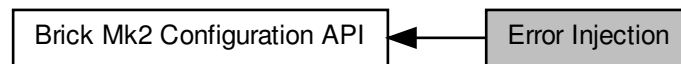
Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.16 Error Injection

Functions in this group deal with error injection.

Collaboration diagram for Error Injection:



Functions

- `int STAR_API_CC_CFG_BRICK_MK2_injectErrors (STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`
Inject the specified errors on a given port.

8.16.1 Detailed Description

Functions in this group deal with error injection.

8.16.2 Function Documentation

8.16.2.1 `int STAR_API_CC_CFG_BRICK_MK2_injectErrors ( STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS * pErrors )`

Inject the specified errors on a given port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to inject the errors on.
<i>portNum</i>	Port to inject the errors on.
<i>pErrors</i>	Pointer to structure specifying which errors are to be injected.

Returns

1 if the errors were successfully injected, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

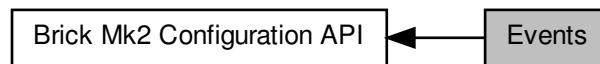
Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#)

8.17 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



Functions

- `int STAR_API_CC_CFG_BRICK_MK2_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables state change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether state change events are enabled on a specific port.
- `int STAR_API_CC_CFG_BRICK_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables state change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables speed change events on a given port.
- `int STAR_API_CC_CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether speed change events are enabled on a specific port.
- `int STAR_API_CC_CFG_BRICK_MK2_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables speed change events on a given port.

8.17.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

8.17.2 Function Documentation

8.17.2.1 `int STAR_API_CC_CFG_BRICK_MK2_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable speed change events for a port on.
<i>portNum</i>	Port to disable speed change events on.

Returns

1 if speed change events were successfully disabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.17.2.2 `int STAR_API_CC_CFG_BRICK_MK2_disableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3 and Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to disable state change events for a port on.
<i>portNum</i>	Port to disable state change events on.

Returns

1 if state change events were successfully disabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.17.2.3 `int STAR_API_CC_CFG_BRICK_MK2_enableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3 and Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable speed change events for a port on.
<i>portNum</i>	Port to enable speed change events on.

Returns

1 if speed change events were successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.17.2.4 `int STAR_API_CC_CFG_BRICK_MK2_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceID</i>	Device to enable state change events for a port on.
<i>portNum</i>	Port to enable state change events on.

Returns

1 if state change events were successfully enabled, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.17.2.5 `int STAR_API_CC_CFG_BRICK_MK2_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceId</i>	Device to get whether speed change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if speed change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.17.2.6 `int STAR_API_CC_CFG_BRICK_MK2_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

Note

This function is used by [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices. Please refer to the [Configuration API Functions Supported by Device Types](#) page for details of which functions are supported by each device.

Parameters

<i>deviceId</i>	Device to get whether state change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if state change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

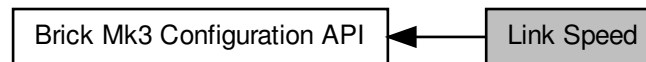
Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3

8.18 Link Speed

Functions in this group deal with setting the link speeds of Brick Mk3 devices.

Collaboration diagram for Link Speed:



Functions

- `int STAR_API_CC_CFG_BRICK_MK3_setBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the base transmit clock rate on a given link of a Brick Mk3 device:
- `int STAR_API_CC_CFG_BRICK_MK3_getBaseTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, ↔_Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`
Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.
- `int STAR_API_CC_CFG_BRICK_MK3_setTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the transmit clock rate on a given link.
- `int STAR_API_CC_CFG_BRICK_MK3_getTransmitClock (STAR_DEVICE_ID deviceID, U8 linkNum, ↔_Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *pClockRateParams)`
Gets the transmit clock rate parameters for a given link.

8.18.1 Detailed Description

Functions in this group deal with setting the link speeds of Brick Mk3 devices.

8.18.2 Function Documentation

8.18.2.1 `int STAR_API_CC_CFG_BRICK_MK3_getBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, ↔_Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.

Note

This function is currently for use with Brick Mk3 devices before hardware version v1.01.
 For later versions use `CFG_BRICK_MK3_getTransmitClock()`.

The SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester, SpaceWire Brick Mk2 and SpaceWire Router Mk2S do not support this function. For SpaceWire PCI Mk2 and cPCI Mk2 devices, please use `CFG_PCIMK2_getLinkClockFrequency()` instead. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices, please use either `CFG_MK2_getBaseTransmitClock()` or `CFG_MK2_get↔TransmitClock()` depending on hardware version. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices, please use `CFG_BRICK_MK2_getLinkClockFrequency()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the base transmit clock rate from.
	<i>linkNum</i>	Link to get base transmit clock rate for.
out	<i>pClockRateParams</i>	A pointer to an existing structure that will be updated to contain the given link's base transmit clock rate parameters.

Returns

1 if the base transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Supported devices:

Brick Mk3 (prior to version 1.01)

8.18.2.2 `int STAR_API_CC_CFG_BRICK_MK3_getTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the transmit clock rate parameters for a given link.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.01 and [Brick Mk4](#) devices. For earlier versions use [CFG_BRICK_MK3_getBaseTransmitClock\(\)](#).

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the transmit clock rate from.
	<i>linkNum</i>	Link to get the transmit clock rate for.
out	<i>pClockRateParams</i>	Pointer to value that will be updated with the given link's transmit clock rate parameters.

Returns

1 if transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

Supported devices:

Brick Mk3 (version 1.01 or greater), [Brick Mk4](#)

8.18.2.3 `int STAR_API_CC_CFG_BRICK_MK3_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link of a Brick Mk3 device:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_MK2_setLinkRateDivider()` function.

Note

This function is currently for use with Brick Mk3 devices before hardware version v1.01.
For later versions use `CFG_BRICK_MK3_setTransmitClock()`.

The SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester, SpaceWire Brick Mk2 and SpaceWire Router Mk2S do not support this function. For SpaceWire PCI Mk2 and cPCI Mk2 devices, please use `CFG_PCIMK2_setLinkClockFrequency()` instead. For SpaceWire PCIe and SpaceWire Physical Layer Tester devices, please use either `CFG_MK2_setBaseTransmitClock()` or `CFG_MK2_setTransmitClock()` depending on hardware version. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices, please use `CFG_BRICK_MK2_setLinkClockFrequency()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a link's base transmit clock rate on.
<i>linkNum</i>	Link to have its base transmit clock rate modified.
<i>clockRateParams</i>	Structure containing the base transmit clock rate parameters to be set.

Returns

1 if link's Base transmit clock frequency was successfully set, else 0.

Added in version:

STAR-System v3.0 Beta 1 - 31st March 2015

Supported devices:

Brick Mk3 (prior to version 1.01)

8.18.2.4 `int STAR_API_CC CFG_BRICK_MK3_setTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s.

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.01 and [Brick Mk4](#) devices. For earlier versions use `CFG_BRICK_MK3_setBaseTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a link's transmit clock rate on.
<i>linkNum</i>	Link to have its transmit clock rate modified.
<i>clockRate</i> ↔ <i>Params</i>	Transmit clock rate parameters to be set.

Returns

1 if link's transmit clock frequency was successfully set, else 0.

Added in version:

[STAR-System v3.0 Beta 7 - 8th March 2016](#)

Supported devices:

[Brick Mk3](#) (version 1.01 or greater), [Brick Mk4](#)

8.19 Timestamping

Functions in this group deal with timestamping related functions for the Brick Mk3.

Collaboration diagram for Timestamping:



Enumerations

- enum STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD { STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE = 0, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER = 1, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR = 2 }
Timestamps can be generated by either an external trigger signal or by an internal pulse generator.
- enum STAR_CFG_BRICK_MK3_PULSE_FREQ { STAR_CFG_BRICK_MK3_PULSE_FREQ_1 = 0, STAR_CFG_BRICK_MK3_PULSE_FREQ_10 = 1, STAR_CFG_BRICK_MK3_PULSE_FREQ_100 = 2, STAR_CFG_BRICK_MK3_PULSE_FREQ_1000 = 3 }
Defines frequencies that can be used by the global pulse generator.

Functions

- int STAR_API_CC_CFG_BRICK_MK3_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod)
Sets the timestamping method to use for the given device.
- int STAR_API_CC_CFG_BRICK_MK3_getTimestampMethod (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pTimestampMethod)
Gets the timestamping method that is currently in use for the given device.
- int STAR_API_CC_CFG_BRICK_MK3_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)
Selectively enables receive timestamp events on a given port.
- int STAR_API_CC_CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)
Gets whether receive timestamp events are enabled on a specific port.
- int STAR_API_CC_CFG_BRICK_MK3_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)
Selectively disables receive timestamp events on a given port.
- int STAR_API_CC_CFG_BRICK_MK3_setTimestampValue (STAR_DEVICE_ID deviceId, U32 value)
Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.
- int STAR_API_CC_CFG_BRICK_MK3_getTimestampValue (STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)
Gets the current timestamp value.
- int STAR_API_CC_CFG_BRICK_MK3_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)
Sets the frequency of each generated pulse.
- int STAR_API_CC_CFG_BRICK_MK3_getPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)
Gets the frequency of each generated pulse.

8.19.1 Detailed Description

Functions in this group deal with timestamping related functions for the Brick Mk3.

The functions in this group can be used to receive start of packet and end of packet timestamps for data that is received on either SpaceWire link.

8.19.2 Enumeration Type Documentation

8.19.2.1 enum STAR_CFG_BRICK_MK3_PULSE_FREQ

Defines frequencies that can be used by the global pulse generator.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_CFG_BRICK_MK3_PULSE_FREQ_1 1Hz
STAR_CFG_BRICK_MK3_PULSE_FREQ_10 10Hz
STAR_CFG_BRICK_MK3_PULSE_FREQ_100 100Hz
STAR_CFG_BRICK_MK3_PULSE_FREQ_1000 1KHz

8.19.2.2 enum STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD

Timestamps can be generated by either an external trigger signal or by an internal pulse generator.

This enum defines different methods of timestamping that are available.

Added in version:

STAR-System v3.6 - 3rd April 2017

Enumerator

STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE No timestamp method.
STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER External trigger used for timestamp sync.
STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR Internal pulse generator used for timestamp sync.

8.19.3 Function Documentation

8.19.3.1 int STAR_API_CC_CFG_BRICK_MK3_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)

Selectively disables receive timestamp events on a given port.

Disabling receive timestamp events will result in no timestamp information being provided after the packet.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.02, [Brick Mk4](#) devices, [PXI Interface](#) devices from hardware version v1.04 and [PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to disable receive timestamp events for a port on.
<i>portNum</i>	Port to disable receive timestamp events on.

Returns

1 if receive timestamp events were successfully disabled, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.2 `int STAR_API_CC_CFG_BRICK_MK3_enableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables receive timestamp events on a given port.

Enabling receive timestamp events will result in the timestamp that the last packet was received at to be provided as a STAR_STREAM_ITEM_TYPE_RX_TIMESTAMP.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.02, [Brick Mk4](#) devices, [PXI Interface](#) devices from hardware version v1.04 and [PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to enable receive timestamp events for a port on.
<i>portNum</i>	Port to enable receive timestamp events on.

Returns

1 if receive timestamp packets were successfully enabled, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.3 `int STAR_API_CC_CFG_BRICK_MK3_getPulseGeneratorFrequency ( STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ * pFrequency )`

Gets the frequency of each generated pulse.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.02, [Brick Mk4](#) devices, [PXI Interface](#) devices from hardware version v1.04 and [PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the pulse generator frequency value for.
out	<i>pFrequency</i>	Pointer to value that will be updated with the frequency between each pulse cycle.

Returns

1 if frequency value was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.4 `int STAR_API_CC_CFG_BRICK_MK3_getRxTimestampEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether receive timestamp events are enabled on a specific port.

Note

This function is currently for use with Brick Mk3 devices from hardware version v1.02, Brick Mk4 devices, PXI Interface devices from hardware version v1.04 and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get whether receive timestamp events are enabled for a port.
	<i>portNum</i>	Port to get receive timestamp events for.
out	<i>pEnabled</i>	User supplied value that will be updated to 1 if receive timestamp events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.5 `int STAR_API_CC_CFG_BRICK_MK3_getTimestampMethod ( STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD * pTimestampMethod )`

Gets the timestamping method that is currently in use for the given device.

Note

This function is currently for use with Brick Mk3 devices from hardware version v1.02, Brick Mk4 devices, PXI Interface devices from hardware version v1.04 and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the timestamping method for.
out	<i>pTimestampMethod</i>	Pointer to value that will be updated with the given link's timestamp method.

Returns

1 if timestamp method was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2, SpaceWire PCIe Mk2

8.19.3.6 int STAR_API_CC_CFG_BRICK_MK3_getTimestampValue (STAR_DEVICE_ID deviceID, _Out_ U32 * pValue)

Gets the current timestamp value.

Note

This function is currently for use with Brick Mk3 devices from hardware version v1.02, Brick Mk4 devices, PXI Interface devices from hardware version v1.04 and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the timestamp value for.
out	<i>pValue</i>	Pointer to value that will be updated with the given link's timestamp value.

Returns

1 if timestamp value was successfully obtained, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2, SpaceWire PCIe Mk2

8.19.3.7 int STAR_API_CC_CFG_BRICK_MK3_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceID, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)

Sets the frequency of each generated pulse.

Note

This function is currently for use with Brick Mk3 devices from hardware version v1.02, Brick Mk4 devices, PXI Interface devices from hardware version v1.04 and PXI Interface Mk2 devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Device to set the pulse generator frequency for.
<i>frequency</i>	The frequency value to set.

Returns

1 if frequency was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.8 `int STAR_API_CC_CFG_BRICK_MK3_setTimestampMethod ( STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD timestampMethod )`

Sets the timestamping method to use for the given device.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.02, [Brick Mk4](#) devices, [PXI Interface](#) devices from hardware version v1.04 and [PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

The device will trigger on the rising edge of the external pulse but the Triggering API can be used to change it to trigger on the falling edge using e.g. `TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode()` and `TRIGGER_BRICK_MK3_enableExtTriggerInvert()`.

Parameters

<i>deviceId</i>	Device to set the timestamping method for.
<i>timestamp↔ Method</i>	Timestamp method to enable for device.

Returns

1 if timestamp method was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2

8.19.3.9 `int STAR_API_CC_CFG_BRICK_MK3_setTimestampValue ( STAR_DEVICE_ID deviceId, U32 value )`

Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.

Note

This function is currently for use with [Brick Mk3](#) devices from hardware version v1.02, [Brick Mk4](#) devices, [PXI Interface](#) devices from hardware version v1.04 and [PXI Interface Mk2](#) devices.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Device to set the timestamp value for.
<i>value</i>	Timestamp value to set for device.

Returns

1 if timestamp value was successfully set, else 0.

Added in version:

STAR-System v3.6 - 3rd April 2017

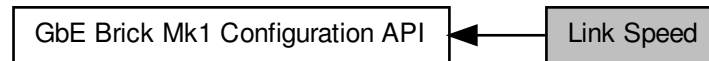
Supported devices:

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2, SpaceWire PCIe Mk2

8.20 Link Speed

Functions in this group deal with setting the link speeds of GbE Brick Mk1 devices.

Collaboration diagram for Link Speed:



Functions

- `int STAR_API_CC_CFG_GBE_BRICK_MK1_setTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, U32 signallingRate)`
Sets the transmit bit rate on a given link.
- `int STAR_API_CC_CFG_GBE_BRICK_MK1_getTransmitSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 *pSignallingRate)`
Gets the transmit bit rate for a given link.

8.20.1 Detailed Description

Functions in this group deal with setting the link speeds of GbE Brick Mk1 devices.

8.20.2 Function Documentation

8.20.2.1 `int STAR_API_CC_CFG_GBE_BRICK_MK1_getTransmitSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 * pSignallingRate )`

Gets the transmit bit rate for a given link.

Note

This function is currently for use with [SpaceWire GbE Brick Mk1](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceId</i>	Device to get the transmit bit rate from.
	<i>linkNum</i>	Link to get the transmit bit rate for.
out	<i>pSignallingRate</i>	Pointer to value that will be updated with the given link's transmit bit rate in Mbit/s.

Returns

1 if transmit bit rate was successfully obtained, else 0.

Added in version:

STAR-System v4.04 - 11th September 2020

Supported devices:

SpaceWire GbE Brick Mk1

8.20.2.2 int STAR_API_CC_CFG_GBE_BRICK_MK1_setTransmitSignallingRate (STAR_DEVICE_ID *deviceId*, U8 *linkNum*, U32 *signallingRate*)

Sets the transmit bit rate on a given link.

The transmit bit rate of a link is given in Mbit/s. The bit rate will be between 2 Mbit/s and 400 Mbit/s inclusive.

An exact rate may not be achievable and may be rounded up or down slightly.

Note

This function is currently for use with [SpaceWire GbE Brick Mk1](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Device to modify a link's transmit bit rate on.
<i>linkNum</i>	Link to have its transmit bit rate modified.
<i>signallingRate</i>	Transmit bit rate in Mbit/s.

Returns

1 if link's transmit bit rate was successfully set, else 0.

Added in version:

STAR-System v4.04 - 11th September 2020

Supported devices:

SpaceWire GbE Brick Mk1

8.21 Link Speed

Functions in this group deal with setting the link speeds of PXI devices devices.

Collaboration diagram for Link Speed:



Functions

- `int STAR_API_CC_CFG_PXI_setLinkRateDivider (STAR_DEVICE_ID deviceId, U8 divider)`
Sets the Link rate divider for a PXI device.
- `int STAR_API_CC_CFG_PXI_getLinkRateDivider (STAR_DEVICE_ID deviceId, _Out_ U8 *pDivider)`
Gets the Link rate divider for a PXI device.
- `int STAR_API_CC_CFG_PXI_setBaseTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the base transmit clock rate on a given link:
- `int STAR_API_CC_CFG_PXI_getBaseTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)`
Gets the base transmit clock rate parameters for a given link.
- `int STAR_API_CC_CFG_PXI_setTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams)`
Sets the transmit clock rate on a given link.
- `int STAR_API_CC_CFG_PXI_getTransmitClock (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK *clockRateParams)`
Gets the transmit clock rate parameters for a given link.

8.21.1 Detailed Description

Functions in this group deal with setting the link speeds of PXI devices devices.

8.21.2 Function Documentation

8.21.2.1 `int STAR_API_CC_CFG_PXI_getBaseTransmitClock ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * clockRateParams )`

Gets the base transmit clock rate parameters for a given link.

Note

This function is currently for use with PXI devices before hardware version v1.01. For later versions use `CFG_PXI_getTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get the base transmit clock rate from.
	<i>linkNum</i>	Link to get base transmit clock rate for.
out	<i>clockRateParams</i>	Pointer to value that will be updated with the given link's Base transmit clock rate parameters.

Returns

1 if Base transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (prior to version 1.01), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (prior to version 1.01)

8.21.2.2 int STAR_API_CC_CFG_PXI_getLinkRateDivider (STAR_DEVICE_ID *deviceID*, _Out_ U8 * *pDivider*)

Gets the Link rate divider for a PXI device.

Note

This function is currently for use with PXI devices before hardware version V1.1. No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to get a links divider from.
out	<i>pDivider</i>	Value of the divider.

Returns

1 if was successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.21.2.3 int STAR_API_CC_CFG_PXI_getTransmitClock (STAR_DEVICE_ID *deviceID*, U8 *linkNum*, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * *clockRateParams*)

Gets the transmit clock rate parameters for a given link.

Note

This function is currently for use with PXI devices from hardware version v1.01. For earlier versions use `CFG_PXI_getBaseTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, getting the transmit clock frequency for link 1 will return the transmit clock frequency also shared by link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to get the transmit clock frequency for each pair of links.

Parameters

	<i>deviceID</i>	Device to get the transmit clock rate from.
	<i>linkNum</i>	Link to get the transmit clock rate for.
out	<i>clockRate↔ Params</i>	Pointer to value that will be updated with the given link's transmit clock rate parameters.

Returns

1 if transmit clock frequency parameters were successfully obtained, else 0.

Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (version 1.01 or greater), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (version 1.01 or greater)

8.21.2.4 `int STAR_API_CC_CFG_PXI_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_PXI_setLinkRateDivider()` function.

Note

This function is currently for use with PXI devices before hardware version v1.01. For later versions use `CFG_PXI_setTransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

Parameters

	<i>deviceID</i>	Device to modify a link's base transmit clock rate on.
	<i>linkNum</i>	Link to have its base transmit clock rate modified.
	<i>clockRate↔ Params</i>	Base transmit clock rate parameters to be set.

Returns

1 if link's Base transmit clock frequency was successfully set, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (prior to version 1.01), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (prior to version 1.01)

8.21.2.5 `int STAR_API_CC_CFG_PXI_setLinkRateDivider ( STAR_DEVICE_ID deviceID, U8 divider )`

Sets the Link rate divider for a PXI device.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

Where *BaseClockRate* is set by the appropriate function (i.e. `CFG_PXI_setBaseTransmitClock()`) and *LinkClockRateDivider* = *divider*.

Note

It is never necessary to set a link rate divider higher than 100, as the minimum transmit rate permitted by SpaceWire is 2 Mbit/s (200 Mbit/s Max / 100).

If an odd number (other than 1) is specified as the divider parameter, the previous even integer will be used instead. For example: if a divider value of 3 is specified, the actual divider set shall be 2.

This function is currently for use with PXI devices before hardware version V1.1. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to modify a links divider on.
<i>divider</i>	Value of the new divider. Valid input: 1, Even numbers 2 through 126.

Returns

1 if divider was successfully set, else 0.

Added in version:

STAR-System v3.0 Beta 4 - 13th November 2015

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.21.2.6 `int STAR_API_CC_CFG_PXI_setTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

Note

This function is currently for use with PXI devices from hardware version v1.01. For earlier versions use [CFG_PXI_setBaseTransmitClock\(\)](#).

No check is made by this function to ensure it is being used with a compatible device.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, setting the transmit clock frequency for link 1 will also affect link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to set the transmit clock frequency for each pair of links.

Parameters

<i>deviceID</i>	Device to modify a link's transmit clock rate on.
<i>linkNum</i>	Link to have its transmit clock rate modified.
<i>clockRate</i> ↔ <i>Params</i>	Transmit clock rate parameters to be set.

Returns

1 if link's transmit clock frequency was successfully set, else 0.

Added in version:

[STAR-System v3.0 Beta 7 - 8th March 2016](#)

Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2](#) (version 1.01 or greater), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) (version 1.01 or greater)

8.22 Error Injection

Functions in this group deal with injecting errors onto SpaceWire links.

Collaboration diagram for Error Injection:



Functions

- `int STAR_API_CC CFG_PXI_injectError (STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error)`

Immediately injects the specified error on a given device's port.

8.22.1 Detailed Description

Functions in this group deal with injecting errors onto SpaceWire links.

8.22.2 Function Documentation

- 8.22.2.1 `int STAR_API_CC CFG_PXI_injectError ( STAR_DEVICE_ID deviceId, unsigned char port, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

Note

This function is currently only compatible with [SpaceWire PXI Interface and PXI Interface Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceId</i>	Identifier of the device to have the error injected on.
<i>port</i>	Port the error is to be injected on.
<i>error</i>	SpaceWire error to be injected.

Returns

1 if the error information was successfully sent to the device, else 0.

Added in version:

[STAR-System v3.0 Beta 4 - 13th November 2015](#)

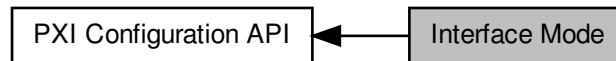
Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#)

8.23 Interface Mode

Functions in this group deal with managing the device in Interface mode.

Collaboration diagram for Interface Mode:



Functions

- `int STAR_API_CC_CFG_PXI_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables interface mode on a given port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether interface mode is enabled on a specific port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables interface mode on a given port of a supported PXI device.
- `int STAR_API_CC_CFG_PXI_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Enables identification of the source port of a received packet on a given port, when in interface mode.
- `int STAR_API_CC_CFG_PXI_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether identify source is enabled for a specific port.
- `int STAR_API_CC_CFG_PXI_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Disables source port identification for a given port.
- `int STAR_API_CC_CFG_PXI_setPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, U8 address)`
Sets the address a packet received on a given port should be routed to when interface mode is enabled.
- `int STAR_API_CC_CFG_PXI_getPortRoutingAddress (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 *pAddress)`
Gets the address a packet received on a given port should be routed to when interface mode is enabled.

8.23.1 Detailed Description

Functions in this group deal with managing the device in Interface mode.

8.23.2 Function Documentation

8.23.2.1 `int STAR_API_CC_CFG_PXI_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

Parameters

<i>deviceID</i>	Device to disable source port identification for a given port on.
<i>portNum</i>	Port to disable source identification on.

Returns

1 if source identification was successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.2 int **STAR_API_CC_CFG_PXI_disableInterfaceModeOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Selectively disables interface mode on a given port of a supported PXI device.

When interface mode is disabled, the device operates in routing mode.

Parameters

<i>deviceID</i>	Device to disable interface mode for a port on.
<i>portNum</i>	Port to disable interface mode on.

Returns

1 if interface mode was successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.3 int **STAR_API_CC_CFG_PXI_enableIdentifySourceOnPort** (**STAR_DEVICE_ID** *deviceID*, **U8** *portNum*)

Enables identification of the source port of a received packet on a given port, when in interface mode.

Interface mode (**CFG_MK2_enableInterfaceMode()**) and Identify Source (**CFG_MK2_enableIdentifySource()**) must be enabled globally for this to have an effect.

Parameters

<i>deviceID</i>	Device to enable source port identification for a given port on.
<i>portNum</i>	Port to enable source identification on.

Returns

1 if source identification was successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.4 int STAR_API_CC_CFG_PXI_enableInterfaceModeOnPort (STAR_DEVICE_ID *deviceId*, U8 *portNum*)

Selectively enables interface mode on a given port of a supported PXI device.

Interface mode must be enabled globally (CFG_MK2_enableInterfaceMode()) for interface mode on a port to be enabled.

Parameters

<i>deviceId</i>	Device to enable interface mode for a port on.
<i>portNum</i>	Port to enable interface mode on.

Returns

1 if interface mode was successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.5 int STAR_API_CC_CFG_PXI_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID *deviceId*, U8 *portNum*, _Out_ int * *pEnabled*)

Gets whether identify source is enabled for a specific port.

Parameters

<i>deviceId</i>	Device to check.
<i>portNum</i>	Port to check.
<i>pEnabled</i>	User supplied value that will be updated to 1 if identify source is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.6 int STAR_API_CC_CFG_PXI_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID *deviceId*, U8 *portNum*, _Out_ int * *pEnabled*)

Gets whether interface mode is enabled on a specific port of a supported PXI device.

Parameters

<i>deviceId</i>	Device to get interface mode enabled for a port on.
-----------------	---

<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if interface mode is enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.7 `int STAR_API_CC_CFG_PXI_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

Parameters

	<i>deviceID</i>	Device to get port routing address from.
	<i>portNum</i>	Port to get port routing address from.
<i>out</i>	<i>pAddress</i>	Pointer to value that will be updated to contain the address.

Returns

1 if the address could be obtained, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.23.2.8 `int STAR_API_CC_CFG_PXI_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

Note

This is an advanced feature and not necessary for normal usage.

Parameters

<i>deviceID</i>	Device to have one of its port's port routing address changed.
-----------------	--

<i>portNum</i>	Port to have its port routing address modified.
<i>address</i>	New port routing address.

Returns

1 if the address could be set, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

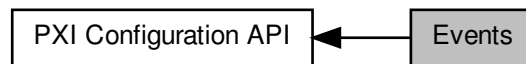
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



Functions

- `int STAR_API_CC CFG_PXI_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables state change events on a given port.
- `int STAR_API_CC CFG_PXI_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether state change events are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables state change events on a given port.
- `int STAR_API_CC CFG_PXI_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables speed change events on a given port.
- `int STAR_API_CC CFG_PXI_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether speed change events are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables speed change events on a given port.
- `int STAR_API_CC CFG_PXI_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively enables time-code notifications on a channel attached to a given port.
- `int STAR_API_CC CFG_PXI_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`
Gets whether time-code notifications are enabled on a specific port.
- `int STAR_API_CC CFG_PXI_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`
Selectively disables time-code notifications on a channel attached to a given port.

8.24.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

8.24.2 Function Documentation

8.24.2.1 `int STAR_API_CC CFG_PXI_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

Parameters

<i>deviceID</i>	Device to disable speed change events for a port on.
<i>portNum</i>	Port to disable speed change events on.

Returns

1 if speed change events were successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24.2.2 int STAR_API_CC_CFG_PXI_disableStateChangeEventsOnPort (STAR_DEVICE_ID *deviceID*, U8 *portNum*)

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

Parameters

<i>deviceID</i>	Device to disable state change events for a port on.
<i>portNum</i>	Port to disable state change events on.

Returns

1 if state change events were successfully disabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24.2.3 int STAR_API_CC_CFG_PXI_disableTimeCodeEventsOnPort (STAR_DEVICE_ID *deviceID*, U8 *portNum*)

Selectively disables time-code notifications on a channel attached to a given port.

Note

This function is currently only compatible with [PXI Interface](#) devices from hardware version v1.02 edit 3, [PXI Interface Mk2](#) devices [PXI Router](#) devices from hardware version v1.02 edit 4 and with [PXI Router Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to disable time-code notifications for a port on.
-----------------	--

<i>portNum</i>	Port to disable time-code notifications on.
----------------	---

Returns

1 if time-code notifications were successfully disabled, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

Supported devices:

PXI Interface (version 1.02 edit 3 or greater), PXI Interface Mk2, PXI Router (version 1.02 edit 4 or greater), PXI Router Mk2

8.24.2.4 `int STAR_API_CC_CFG_PXI_enableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

Parameters

<i>deviceID</i>	Device to enable speed change events for a port on.
<i>portNum</i>	Port to enable speed change events on.

Returns

1 if speed change events were successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24.2.5 `int STAR_API_CC_CFG_PXI_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

Parameters

<i>deviceID</i>	Device to enable state change events for a port on.
<i>portNum</i>	Port to enable state change events on.

Returns

1 if state change events were successfully enabled, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24.2.6 `int STAR_API_CC_CFG_PXI_enableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables time-code notifications on a channel attached to a given port.

Note

This function is currently only compatible with [PXI Interface](#) devices from hardware version v1.02 edit 3, [PXI Interface Mk2](#) devices [PXI Router](#) devices from hardware version v1.02 edit 4 and with [PXI Router Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to enable time-code notifications for a port on.
<i>portNum</i>	Port to enable time-code notifications on.

Returns

1 if time-codes were successfully enabled, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

Supported devices:

[PXI Interface](#) (version 1.02 edit 3 or greater), [PXI Interface Mk2](#), [PXI Router](#) (version 1.02 edit 4 or greater), [PXI Router Mk2](#)

8.24.2.7 `int STAR_API_CC_CFG_PXI_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

Parameters

<i>deviceID</i>	Device to get whether speed change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if speed change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

[SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#)

8.24.2.8 `int STAR_API_CC_CFG_PXI_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

Parameters

<i>deviceID</i>	Device to get whether state change events are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if state change events are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v3.3 - 20th December 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.24.2.9 `int STAR_API_CC_CFG_PXI_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

Note

This function is currently only compatible with [PXI Interface](#) devices from hardware version v1.02 edit 3, [PXI Interface Mk2](#) devices [PXI Router](#) devices from hardware version v1.02 edit 4 and with [PXI Router Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Parameters

<i>deviceID</i>	Device to get whether time-code notifications are enabled for a port.
<i>portNum</i>	Port to get information for.
<i>pEnabled</i>	User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0.

Returns

1 if the information could be obtained from the device, else 0.

Added in version:

STAR-System v4.00 - 19th November 2019

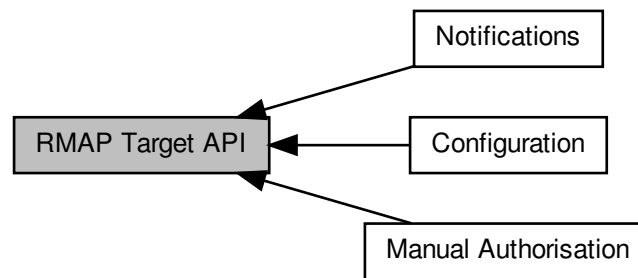
Supported devices:

[PXI Interface](#) (version 1.02 edit 3 or greater), [PXI Interface Mk2](#), [PXI Router](#) (version 1.02 edit 4 or greater), [PXI Router Mk2](#)

8.25 RMAP Target API

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices.

Collaboration diagram for RMAP Target API:



Modules

- **Manual Authorisation**

Functions in this group deal with the manual RMAP target authorisation.

- **Configuration**

Functions in this group deal with setting and getting RMAP target configuration parameters.

- **Notifications**

Functions in this group deal with the RMAP target notification handling.

8.25.1 Detailed Description

The RMAP Target API provides functions for configuring and controlling the RMAP targets within PXI Interface with RMAP Target devices.

The RMAP targets can operate in automatic or manual authorisation mode. The API provides functions to set the automatic authorisation parameters, allowing ranges of valid values for the RMAP header parameters. When using the manual authorisation mode, the API provides functions to retrieve any pending commands and manually authorise or reject them.

In addition, there are functions to enable notifications when commands require authorisation or when commands are completed. Callback functions can be registered with the API and there are functions to start and stop receiving notifications.

More information about the RMAP target hardware can be found in the [SpaceWire PXI Interface and PXI Interface Mk2 Device User Manual](#).

Several examples of how to use the RMAP Target API are included in an application note entitled "Using the STAR-System RMAP Target API for the PXI Interface Board". The application note can be found in the following directory on Windows, assuming the default installation directory:

`C:\Program Files\STAR-Dundee\STAR-System\application_notes`

Or the following directory on Linux:

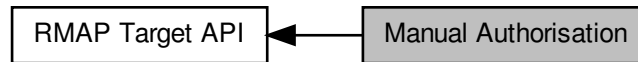
`/usr/local/STAR-Dundee/STAR-System/application_notes`

Additionally, there is a full code example include in the list of examples (see `rmap_target_example.c`).

8.26 Manual Authorisation

Functions in this group deal with the manual RMAP target authorisation.

Collaboration diagram for Manual Authorisation:



Data Structures

- struct [RmapCommandParameters](#)
Parameters of an RMAP command. [More...](#)

Enumerations

- enum [REJECTION_REASON](#) { [REJECTION_REASON_LOGICAL_ADDRESS](#), [REJECTION_REASON_KEY](#), [REJECTION_REASON_PROTOCOL_ID](#), [REJECTION_REASON_OTHER](#) }
Reason for rejecting a waiting RMAP command.

Functions

- int [STAR_API_CC RMAP_TARGET_PXI_IF_isAuthRequired](#) (STAR_DEVICE_ID deviceId, U32 target)
Gets whether or not there is an RMAP command waiting to be authorised.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_getWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target, [RmapCommandParameters](#) *pCommand)
Gets the parameters of the RMAP command waiting to be authorised.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_authoriseWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target)
Authorises a waiting RMAP command.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_rejectWaitingCommand](#) (STAR_DEVICE_ID deviceId, U32 target, [REJECTION_REASON](#) reason)
Rejects a waiting RMAP command.

8.26.1 Detailed Description

Functions in this group deal with the manual RMAP target authorisation.

8.26.2 Data Structure Documentation

8.26.2.1 struct [RmapCommandParameters](#)

Parameters of an RMAP command.

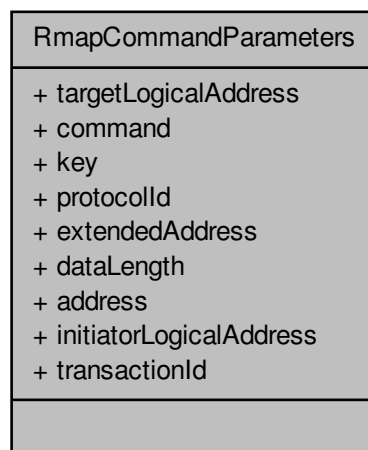
Added in version:

STAR-System v3.0 - 26th August 2016

Examples:

`rmap_target_example.c`.

Collaboration diagram for RmapCommandParameters:



Data Fields

U32	address	
U8	command	
U32	dataLength	
U8	extended↔ Address	
U8	initiatorLogical↔ Address	
U8	key	
U8	protocolId	
U8	targetLogical↔ Address	
U16	transactionId	

8.26.3 Enumeration Type Documentation

8.26.3.1 enum REJECTION_REASON

Reason for rejecting a waiting RMAP command.

Added in version:

STAR-System v3.0 - 26th August 2016

8.26.4 Function Documentation

8.26.4.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_authoriseWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target )`

Authorises a waiting RMAP command.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to authorise a waiting RMAP command for.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

8.26.4.2 `int STAR_API_CC RMAP_TARGET_PXI_IF_getWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target, RmapCommandParameters * pCommand )`

Gets the parameters of the RMAP command waiting to be authorised.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get the waiting RMAP command parameters from.
<i>pCommand</i>	Pointer to a value to be updated with the RMAP command parameters.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

8.26.4.3 `int STAR_API_CC RMAP_TARGET_PXI_IF_isAuthRequired ( STAR_DEVICE_ID deviceld, U32 target )`

Gets whether or not there is an RMAP command waiting to be authorised.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to check whether authorisation is required.

Returns

1 if the target has a command waiting to be authorised, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.26.4.4 `int STAR_API_CC RMAP_TARGET_PXI_IF_rejectWaitingCommand ( STAR_DEVICE_ID deviceld, U32 target, REJECTION_REASON reason )`

Rejects a waiting RMAP command.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to reject a waiting RMAP command for.
<i>reason</i>	Reason for rejecting the waiting RMAP command.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

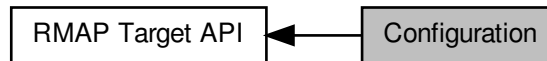
Examples:

`rmap_target_example.c`.

8.27 Configuration

Functions in this group deal with setting and getting RMAP target configuration parameters.

Collaboration diagram for Configuration:



Data Structures

- struct `TARGET_ENGINE`
Hardware capabilities. [More...](#)

Typedefs

- typedef U32 `TARGET_AUTH_CMD_MASK`
Mask describing which commands are authorised.

Enumerations

- enum `TARGET_AUTH_MODE` { `TARGET_AUTH_MODE_MANUAL` = 0, `TARGET_AUTH_MODE_AUTO←`
`MATIC` = 1 }
Method of authorising incoming RMAP commands.
- enum `TARGET_STATUS` {
`TARGET_STATUS_SUCCESS` = 0, `TARGET_STATUS_GENERAL_ERROR` = 1, `TARGET_STATUS_H←`
`EADER_EOP_ERROR` = 2, `TARGET_STATUS_HEADER_EEP_ERROR` = 3,
`TARGET_STATUS_PID_ERROR` = 4, `TARGET_STATUS_REPLY_ERROR` = 5, `TARGET_STATUS_HE←`
`ADER_CRC_ERROR` = 6, `TARGET_STATUS_HEADER_EEP_AFTER_CRC` = 7,
`TARGET_STATUS_PACKET_TYPE_ERROR` = 8, `TARGET_STATUS_COMMAND_TYPE_ERROR` = 9,
`TARGET_STATUS_RMW_DATALEN_ERROR` = 10, `TARGET_STATUS_CARGO_TOO_LARGE` = 11,
`TARGET_STATUS_KEY_ERROR` = 12, `TARGET_STATUS_LOGICAL_ADDR_ERROR` = 13, `TARGET←`
`_STATUS_AUTHORISED_ERROR` = 14, `TARGET_STATUS_VERIFY_BUFFER_OVERRUN` = 15,
`TARGET_STATUS_DATA_CRC_ERROR` = 16, `TARGET_STATUS_BUS_ERROR` = 17, `TARGET_STA←`
`TUS_DATA_EOP_ERROR` = 18, `TARGET_STATUS_DATA_EEP_ERROR` = 19,
`TARGET_STATUS_DATA_EEP_AFTER_CRC` = 20 }
Status of the RMAP target.
- enum `TARGET_AUTH_CMD` {
`TARGET_AUTH_CMD_READ` = (1 << 0), `TARGET_AUTH_CMD_READ_INCR` = (1 << 1), `TARGET←`
`_AUTH_CMD_READ_MODIFY_WRITE` = (1 << 2), `TARGET_AUTH_CMD_WRITE` = (1 << 3),
`TARGET_AUTH_CMD_WRITE_INCR` = (1 << 4), `TARGET_AUTH_CMD_WRITE_REPLY` = (1 << 5),
`TARGET_AUTH_CMD_WRITE_INCR_REPLY` = (1 << 6), `TARGET_AUTH_CMD_WRITE_VERIFY` = (1
<< 7),
`TARGET_AUTH_CMD_WRITE_VERIFY_INCR` = (1 << 8), `TARGET_AUTH_CMD_WRITE_VERIFY_R←`
`EPPLY` = (1 << 9), `TARGET_AUTH_CMD_WRITE_VERIFY_INCR_REPLY` = (1 << 10), `TARGET_AUT←`
`H_CMD_ALL` = (0x7FF),
`TARGET_AUTH_CMD_NONE` = (0) }

Command types which are authorised by the target.

- enum IF_MODE { IF_MODE_RMAP_DISABLED = 0, IF_MODE_RMAP_ENABLED = 1 }

Control whether or not the RMAP target is enabled for a port.

Functions

- int STAR_API_CC RMAP_TARGET_PXI_IF_getStatus (STAR_DEVICE_ID deviceId, U32 target, TARGET_STATUS *pStatus)
Gets the RMAP target status for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 offset)
Sets the address offset for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 *pOffset)
Gets the address offset from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE mode)
Sets the authorisation control mode for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE *pMode)
Gets the authorisation control mode from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)
Sets the authorised target logical address range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)
Gets the authorised target logical address range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 protocolId)
Sets the authorised protocol ID for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 *pProtocolId)
Gets the authorised protocol ID from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK commands)
Sets the authorised RMAP commands for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK *pCommands)
Gets the authorised RMAP commands from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthKeyRange (STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)
Sets the authorised key range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthKeyRange (STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)
Gets the authorised key range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange (STAR_DEVICE_ID deviceId, U32 target, U32 lowerBoundary, U32 upperBoundary)
Sets the authorised memory address range for a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange (STAR_DEVICE_ID deviceId, U32 target, U32 *pLowerBoundary, U32 *pUpperBoundary)
Gets the authorised memory address range from a specific target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_setInterfaceMode (STAR_DEVICE_ID deviceId, U32 port, IF_MODE mode)

Sets the RMAP interface mode for a specific port.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_getInterfaceMode` (STAR_DEVICE_ID deviceId, U32 port, IF↔_MODE *pMode)

Gets the RMAP interface mode for a specific port.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_readMemory` (STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, unsigned char *pBuffer)

Reads an area of RMAP target memory into a user-supplied buffer.

- int `STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory` (STAR_DEVICE_ID deviceId, U32 target, U32 address, U32 length, const unsigned char *pBuffer)

Writes an area of RMAP target memory from a user-supplied buffer.

8.27.1 Detailed Description

Functions in this group deal with setting and getting RMAP target configuration parameters.

8.27.2 Data Structure Documentation

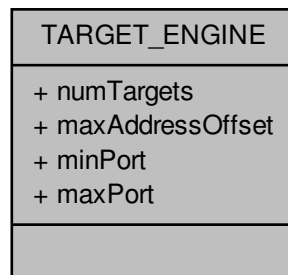
8.27.2.1 struct TARGET_ENGINE

Hardware capabilities.

Added in version:

STAR-System v3.0 - 26th August 2016

Collaboration diagram for TARGET_ENGINE:



Data Fields

U32	maxAddress↔Offset	
U32	maxPort	
U32	minPort	

U32	numTargets	
-----	------------	--

8.27.3 Typedef Documentation

8.27.3.1 typedef U32 TARGET_AUTH_CMD_MASK

Mask describing which commands are authorised.

Added in version:

STAR-System v3.0 - 26th August 2016

8.27.4 Enumeration Type Documentation

8.27.4.1 enum IF_MODE

Control whether or not the RMAP target is enabled for a port.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

IF_MODE_RMAP_DISABLED When RMAP is disabled, the port acts as a normal SpaceWire interface.

IF_MODE_RMAP_ENABLED When RMAP is enabled, the port acts as an RMAP target.

8.27.4.2 enum TARGET_AUTH_CMD

Command types which are authorised by the target.

Added in version:

STAR-System v3.0 - 26th August 2016

8.27.4.3 enum TARGET_AUTH_MODE

Method of authorising incoming RMAP commands.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

TARGET_AUTH_MODE_MANUAL When set to manual, each incoming command must be authorised or rejected by software.

TARGET_AUTH_MODE_AUTOMATIC When set to automatic, each incoming command is automatically authorised or rejected based on the authorised parameter values.

8.27.4.4 enum TARGET_STATUS

Status of the RMAP target.

Added in version:

STAR-System v3.0 - 26th August 2016

8.27.5 Function Documentation

8.27.5.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset ( STAR_DEVICE_ID deviceld, U32 target, U32 * pOffset )`

Gets the address offset from a specific target.

The address offset determines where the target's memory region begins.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get address offset from.
<i>pOffset</i>	Pointer to a value to be updated with the target's address offset.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.2 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthCommands ( STAR_DEVICE_ID deviceld, U32 target, TARGET_AUTH_CMD_MASK * pCommands )`

Gets the authorised RMAP commands from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised RMAP commands from.
<i>pCommands</i>	Pointer to a value to be updated with the target's authorised commands.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.3 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthControlMode ( STAR_DEVICE_ID deviceld, U32 target, TARGET_AUTH_MODE * pMode )`

Gets the authorisation control mode from a specific target.

Authorisation can be set to either TARGET_AUTH_MODE_MANUAL or TARGET_AUTH_MODE_AUTOMATIC. When set to TARGET_AUTH_MODE_MANUAL, the authorisation of RMAP commands is done manually by the user application. When set to TARGET_AUTH_MODE_AUTOMATIC, the authorisation of RMAP commands is done automatically using the expected parameter fields.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorisation mode from.
<i>pMode</i>	Pointer to a value to be updated with the target's authorisation mode.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.4 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthKeyRange ( STAR_DEVICE_ID deviceld, U32 target, U8 * pLowest, U8 * pHighest )`

Gets the authorised key range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised key range from.
<i>pLowest</i>	Pointer to a value to be updated with the lowest key authorised.
<i>pHighest</i>	Pointer to a value to be updated with the highest key authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.5 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U8 * pLowest, U8 * pHighest )`

Gets the authorised target logical address range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised logical address range from.
<i>pLowest</i>	Pointer to a value to be updated with the lowest target logical address authorised.
<i>pHighest</i>	Pointer to a value to be updated with the highest target logical address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.6 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U32 * pLowerBoundary, U32 * pUpperBoundary )`

Gets the authorised memory address range from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised memory address range from.
<i>pLowerBoundary</i>	Pointer to a value to be updated with the lowest memory address authorised.
<i>pUpperBoundary</i>	Pointer to a value to be updated with the highest memory address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.7 `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthProtocolId ( STAR_DEVICE_ID deviceld, U32 target, U8 * pProtocolId )`

Gets the authorised protocol ID from a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get authorised protocol ID from.
<i>pProtocolId</i>	Pointer to a value to be updated with the target's authorised protocol ID.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.8 `int STAR_API_CC RMAP_TARGET_PXI_IF_getInterfaceMode ( STAR_DEVICE_ID deviceld, U32 port, IF_MODE * pMode )`

Gets the RMAP interface mode for a specific port.

Parameters

<i>deviceld</i>	Device containing the port.
<i>port</i>	Number of the port (6 - 9).
<i>pMode</i>	Pointer to a value to be updated with the port's interface mode.

Returns

1 if the port's RMAP interface mode was successfully modified, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.9 `int STAR_API_CC RMAP_TARGET_PXI_IF_getStatus ( STAR_DEVICE_ID deviceld, U32 target, TARGET_STATUS * pStatus )`

Gets the RMAP target status for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to get status from.
<i>pStatus</i>	Pointer to a value to be updated with the target's status.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.10 `int STAR_API_CC RMAP_TARGET_PXI_IF_readMemory ( STAR_DEVICE_ID deviceld, U32 target, U32 address, U32 length, unsigned char * pBuffer )`

Reads an area of RMAP target memory into a user-supplied buffer.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to read memory from.
<i>address</i>	The address in the target to read from.
<i>length</i>	The length to read.
<i>pBuffer</i>	Pointer to a buffer to read memory into.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.27.5.11 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset ( STAR_DEVICE_ID deviceld, U32 target, U32 offset )`

Sets the address offset for a specific target.

The address offset determines where the target's memory region begins.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set address offset for.
<i>offset</i>	Address offset in MBytes.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

8.27.5.12 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthCommands ( STAR_DEVICE_ID deviceld, U32 target, TARGET_AUTH_CMD_MASK commands )`

Sets the authorised RMAP commands for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised RMAP commands for.
<i>commands</i>	Authorised commands mask.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

8.27.5.13 int **STAR_API_CC** RMAP_TARGET_PXI_IF_setAuthControlMode (STAR_DEVICE_ID *deviceld*, U32 *target*, TARGET_AUTH_MODE *mode*)

Sets the authorisation control mode for a specific target.

Authorisation can be set to either TARGET_AUTH_MODE_MANUAL or TARGET_AUTH_MODE_AUTOMATIC. When set to TARGET_AUTH_MODE_MANUAL, the authorisation of RMAP commands is done manually by the user application. When set to TARGET_AUTH_MODE_AUTOMATIC, the authorisation of RMAP commands is done automatically using the expected parameter fields.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorisation mode for.
<i>mode</i>	Authorisation mode.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

8.27.5.14 int **STAR_API_CC** RMAP_TARGET_PXI_IF_setAuthKeyRange (STAR_DEVICE_ID *deviceld*, U32 *target*, U8 *lowest*, U8 *highest*)

Sets the authorised key range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised key range for.
<i>lowest</i>	Lowest key authorised.
<i>highest</i>	Highest key authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

8.27.5.15 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U8 lowest, U8 highest )`

Sets the authorised target logical address range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised logical address range for.
<i>lowest</i>	Lowest target logical address authorised.
<i>highest</i>	Highest target logical address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

8.27.5.16 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange ( STAR_DEVICE_ID deviceld, U32 target, U32 lowerBoundary, U32 upperBoundary )`

Sets the authorised memory address range for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised memory address range for.
<i>lowerBoundary</i>	Lowest memory address authorised.
<i>upperBoundary</i>	Highest memory address authorised.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

8.27.5.17 `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthProtocolId ( STAR_DEVICE_ID deviceld, U32 target, U8 protocolId )`

Sets the authorised protocol ID for a specific target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set authorised protocol ID for.
<i>protocolId</i>	Authorised protocol ID.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

8.27.5.18 `int STAR_API_CC RMAP_TARGET_PXI_IF_setInterfaceMode ( STAR_DEVICE_ID deviceld, U32 port, IF_MODE mode )`

Sets the RMAP interface mode for a specific port.

Parameters

<i>deviceld</i>	Device containing the port.
<i>port</i>	Number of the port (6 - 9).
<i>mode</i>	Interface mode of the port.

Returns

1 if the port's RMAP interface mode was successfully modified, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

`rmap_target_example.c`.

8.27.5.19 `int STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory ( STAR_DEVICE_ID deviceld, U32 target, U32 address, U32 length, const unsigned char * pBuffer )`

Writes an area of RMAP target memory from a user-supplied buffer.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to write memory to.
<i>address</i>	The address in the target to write to.
<i>length</i>	The length to write.
<i>pBuffer</i>	Pointer to a buffer to write memory from.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

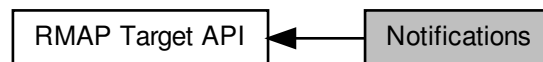
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.28 Notifications

Functions in this group deal with the RMAP target notification handling.

Collaboration diagram for Notifications:



Typedefs

- typedef U32 NOTIF_TYPE_MASK
Mask describing which notifications are enabled.
- typedef void(STAR_API_CC * NotifListenerFunc) (U32 target, void *pNotif)
Notification listener function pointer: target is the index of the target which generated the notification.
- typedef void(STAR_API_CC * NotifListenerWithContextFunc) (STAR_DEVICE_ID deviceId, U32 target, void *pNotif, void *pContext)
Notification listener with context function pointer: deviceId is the ID of the device that issued the notification target is the index of the target which generated the notification.

Enumerations

- enum NOTIF_TYPE { NOTIF_TYPE_AUTH_REQUEST = (1 << 0), NOTIF_TYPE_CMD_COMPLETE = (1 << 2), NOTIF_TYPE_ALL = (0x5), NOTIF_TYPE_NONE = (0) }
Types of notifications.

Functions

- int STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE_MASK notifications)
Set the notifications that are enabled for a target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE_MASK *pNotifications)
Get the notifications that are enabled for a target.
- int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type, NotifListenerFunc listenerFunc)
Register a notification listener function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type, NotifListenerWithContextFunc listenerFunc, void *pContext)
Register a notification listener with context function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type)
Unregister a notification listener function.
- int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext (STAR_DEVICE_ID deviceId, U32 target, NOTIF_TYPE type)
Unregister a notification listener with context function.

- int [STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications \(STAR_DEVICE_ID deviceId\)](#)
Start receiving notifications for a device.
- int [STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications \(STAR_DEVICE_ID deviceId\)](#)
Stop receiving notifications for a device.

8.28.1 Detailed Description

Functions in this group deal with the RMAP target notification handling.

8.28.2 Typedef Documentation

8.28.2.1 typedef U32 NOTIF_TYPE_MASK

Mask describing which notifications are enabled.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

8.28.2.2 typedef void(STAR_API_CC * NotifListenerFunc) (U32 target, void *pNotif)

Notification listener function pointer: target is the index of the target which generated the notification.

pNotif is a pointer to the notification packet which has one of two formats depending on the notification type.

Authorisation request notification:

Byte [0] : 0xFE

Byte [1] : 0xF0

Byte [2] : 0x10 (Notification type)

Byte [3] : Notification packet cargo length

Byte [4] : Target index

Byte [5] : Current time-code at time of event

Command complete notification:

Byte [0] : 0xFE

Byte [1] : 0xF0

Byte [2] : 0x12 (Notification type)

Byte [3] : Notification packet cargo length

Byte [4] : Target index

Byte [5] : Current time-code at time of event

Byte [6] : Target logical address

Byte [7] : Protocol ID

Byte [8] : Command

Byte [9] : Key

Byte [10] : Initiator logical address

Byte [11 - 12] : Transaction ID (MSB first)

Byte [13] : Extended address

Byte [14 - 17] : Address (MSB first)

Byte [18 - 20] : Data length (MSB first)

Byte [21] : Status

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

8.28.2.3 `typedef void(STAR_API_CC * NotifListenerWithContextFunc) (STAR_DEVICE_ID deviceld, U32 target, void *pNotif, void *pContext)`

Notification listener with context function pointer: deviceld is the ID of the device that issued the notification target is the index of the target which generated the notification.

pNotif is a pointer to the notification packet which has one of two formats depending on the notification type. pContext is a pointer to a user supplied context variable

Authorisation request notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x10 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event

Command complete notification:

Byte [0] : 0xFE
 Byte [1] : 0xF0
 Byte [2] : 0x12 (Notification type)
 Byte [3] : Notification packet cargo length
 Byte [4] : Target index
 Byte [5] : Current time-code at time of event
 Byte [6] : Target logical address
 Byte [7] : Protocol ID
 Byte [8] : Command
 Byte [9] : Key
 Byte [10] : Initiator logical address
 Byte [11 - 12] : Transaction ID (MSB first)
 Byte [13] : Extended address
 Byte [14 - 17] : Address (MSB first)
 Byte [18 - 20] : Data length (MSB first)
 Byte [21] : Status

Added in version:

STAR-System v3.0 - 26th August 2016

8.28.3 Enumeration Type Documentation

8.28.3.1 enum NOTIF_TYPE

Types of notifications.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

NOTIF_TYPE_AUTH_REQUEST Authorisation request notification.

NOTIF_TYPE_CMD_COMPLETE Command complete notification.

NOTIF_TYPE_ALL All notifications.

NOTIF_TYPE_NONE No notifications.

8.28.4 Function Documentation

8.28.4.1 `int STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications ( STAR_DEVICE_ID deviceld, U32 target, NOTIF_TYPE_MASK * pNotifications )`

Get the notifications that are enabled for a target.

Parameters

<i>deviceId</i>	Device containing the target.
<i>target</i>	Target to get enabled notifications from.
<i>pNotifications</i>	Pointer to a value to be updated with the enabled notifications.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.28.4.2 **int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener (STAR_DEVICE_ID *deviceId*, U32 *target*, NOTIF_TYPE *type*, NotifListenerFunc *listenerFunc*)**

Register a notification listener function.

Parameters

<i>deviceId</i>	Device containing the target.
<i>target</i>	Target to register a notification listener function for.
<i>type</i>	Type of notification to register the listener function for.
<i>listenerFunc</i>	Pointer to a notification listener function.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.28.4.3 **int STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext (STAR_DEVICE_ID *deviceId*, U32 *target*, NOTIF_TYPE *type*, NotifListenerWithContextFunc *listenerFunc*, void * *pContext*)**

Register a notification listener with context function.

Parameters

<i>deviceId</i>	Device containing the target.
<i>target</i>	Target to register a notification listener function for.
<i>type</i>	Type of notification to register the listener function for.
<i>listenerFunc</i>	Pointer to a notification listener with context function.
<i>pContext</i>	Pointer to a user-supplied context.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.4 - 9th March 2017

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

8.28.4.4 **int STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications (STAR_DEVICE_ID *deviceld*, U32 *target*, NOTIF_TYPE_MASK *notifications*)**

Set the notifications that are enabled for a target.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to set enabled notifications for.
<i>notifications</i>	Enabled notifications mask

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

rmap_target_example.c.

8.28.4.5 **int STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications (STAR_DEVICE_ID *deviceld*)**

Start receiving notifications for a device.

Parameters

<i>deviceld</i>	Device to start receiving notifications for.
-----------------	--

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

Examples:

[rmap_target_example.c](#).

8.28.4.6 int STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications (STAR_DEVICE_ID *deviceld*)

Stop receiving notifications for a device.

Parameters

<i>deviceld</i>	Device to stop receiving notifications for.
-----------------	---

Returns

1 if the operation was successful, else 0.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2 \(with RMAP Target\)](#)

Examples:

[rmap_target_example.c](#).

8.28.4.7 int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener (STAR_DEVICE_ID *deviceld*, U32 *target*, NOTIF_TYPE *type*)

Unregister a notification listener function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to unregister a notification listener with context function for.
<i>type</i>	Type of notification to unregister the listener with context function for.

Returns

1 if the operation was successful, else 0.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

Supported devices:

[SpaceWire PXI Interface and PXI Interface Mk2 \(with RMAP Target\)](#)

Examples:

[rmap_target_example.c](#).

8.28.4.8 int STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext (STAR_DEVICE_ID *deviceld*, U32 *target*, NOTIF_TYPE *type*)

Unregister a notification listener with context function.

Parameters

<i>deviceld</i>	Device containing the target.
<i>target</i>	Target to unregister a notification listener with context function for.
<i>type</i>	Type of notification to unregister the listener with context function for.

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.4 - 9th March 2017

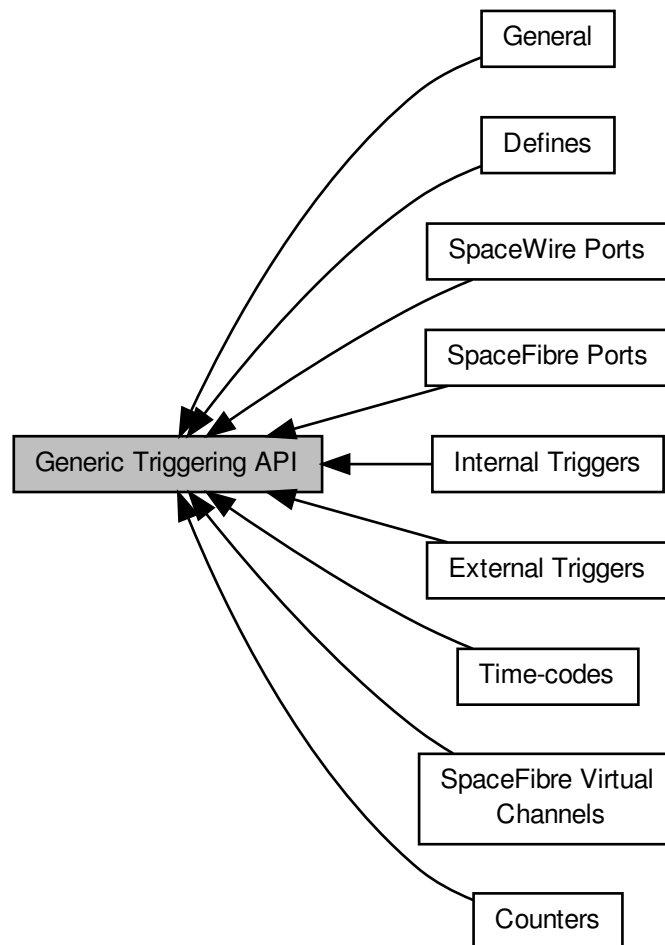
Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2 (with RMAP Target)

8.29 Generic Triggering API

The Generic Triggering API provides functions for configuring and controlling the triggering capabilities of the SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2 and STAR-Ultra PCIe Interface devices.

Collaboration diagram for Generic Triggering API:



Modules

- **General**

Functions in this group are used to configure general triggering functionality.

- **External Triggers**

Functions in this group are used to configure triggering functionality for the external triggers.

- **Counters**

Functions in this group are used to configure triggering functionality for the counters.

- **SpaceWire Ports**

Functions in this group are used to configure triggering functionality for the SpW ports.

- [Time-codes](#)

Functions in this group are used to configure triggering functionality for the time-code engine(s).

- [Internal Triggers](#)

Functions in this group are used to configure triggering functionality for the internal triggers.

- [SpaceFibre Ports](#)

Functions in this group are used to configure triggering functionality for SpaceFibre Ports.

- [SpaceFibre Virtual Channels](#)

Functions in this group are used to configure triggering functionality for SpaceFibre Virtual Channels.

- [Defines](#)

Function return error values are defined here.

8.29.1 Detailed Description

The Generic Triggering API provides functions for configuring and controlling the triggering capabilities of the [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#), [SpaceWire PCIe](#), [SpaceWire PCIe Mk2](#) and [STAR-Ultra PCIe Interface](#) devices.

If a feature is not supported by a device, the API function calls will return a value to indicate this. Please see the relevant sections: [Input Events](#), [Output Actions](#) and [Internal Triggers](#) for more details about the supported triggering functionality for each STAR-Dundee device individually. Alternatively, the supported devices are listed for each Generic Triggering configuration function. You can view the Generic Triggering functions here: [General](#), [External Triggers](#), [Counters](#), [SpaceWire Ports](#), [Time-codes](#), [Internal Triggers](#), [SpaceFibre Ports](#) and [SpaceFibre Virtual Channels](#).

The Generic Triggering API allows events that occur on triggering resources such as external triggers, counters, time-code interfaces, SpaceWire ports, SpaceFibre ports or SpaceFibre virtual channels to cause actions. For example, the triggers could be configured to transmit a packet on a SpaceWire port whenever an external trigger detects the rising edge of an input signal.

The three main concepts of the Generic Triggering API are:

- **Input Events:** these are events that occur on triggering resources.
- **Output Actions:** these are actions that are performed by triggering resources.
- **Internal Triggers:** these are used to control the input events that cause output actions.

These concepts are described in the following sections.

8.29.2 Input Events

An input event is an event that occurs on one of the triggering resources. These events may be configured to set one or more internal triggers. Each triggering resource has one or more events that can occur. For example, SpaceWire ports have events for when they detect certain characters, errors or receive/transmit time-codes.

The following tables provide a description of the input events for each triggering resource.

External Trigger Input Events

Input Event	Description	Supported Devices						
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STARUltra PCIe Interface	
EXT_TRIGGER_EVENT_IN	This event occurs when the external trigger receives a signal.	Yes	Yes	No	No	Yes	Yes	

Counter Input Events

Input Event	Description	Supported Devices						
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STARUltra PCIe Interface	
COUNTER_EVENT_COUNT	This event occurs when the counter decrements.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_EVENT_ZERO_SIGNAL	This event occurs when the counter reaches zero.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_EVENT_ZERO	This event is active while the counter is equal to zero.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_EVENT_RELOAD	This event occurs when the counter reloads.	Yes	Yes	Yes	Yes	Yes	Yes	

SpaceWire Port Input Events

Input Event	Description	Supported Devices						
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STARUltra PCIe Interface	

SPW_P↔ ORT_E↔ VENT_↔ RX_SOP	This event occurs when the port receives a Start of Packet (SOP).	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_↔ RX_EOP	This event occurs when the port receives an End of Packet (EOP).	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_↔ RX_EEP	This event occurs when the port receives an Error End of Packet (EEP).	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_T↔ X_SOP	This event occurs when the port transmits an SOP.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_T↔ X_EOP	This event occurs when the port transmits an EOP.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_T↔ X_PKT_↔ PENDING	This event is active when a packet queued for transmission.	Yes	Yes	Yes	Yes	Yes	No

SPW_P↔ ORT_E↔ VENT_↔ RUNNING	This event is active when the link attached to a port is running.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_P↔ ARITY_↔ ERROR	This event occurs when a port detects a parity error.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_E↔ SCAPE↔ _ERROR	This event occurs when a port detects an escape error.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_↔ CREDIT↔ _ERROR	This event occurs when a port detects a credit error.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_↔ DISCON↔ NECT	This event occurs when the link attached to a port disconnects.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_↔ RX_TIM↔ ECODE	This event occurs when a port receives a time-code.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_E↔ VENT_T↔ X_TIME↔ CODE	This event occurs when a port transmits a time-code.	Yes	Yes	Yes	No	Yes	No

Time-Code Interface Input Events

Input Event	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCle	Space↔ Wire PCle Mk2	STAR↔ Ultra PCle Interface

TIME_C↔ ODE_E↔ VENT_T↔ ICK	This event occurs when a time-code interface receives its next valid time-code.	Yes	Yes	Yes	No	Yes	No
-------------------------------------	---	-----	-----	-----	----	-----	----

SpaceFibre Port Input Events

Input Event	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCle	Space↔ Wire PCle Mk2	STAR↔ Ultra PCle Interface
SPFI_P↔ ORT_E↔ VENT_↔ CONNE↔ CTED	This event occurs when the link moves from not connected to connected.	No	No	No	No	No	Yes
SPFI_P↔ ORT_E↔ VENT_↔ DISCON↔ NECTED	This event occurs when the link moves from connected to not connected.	No	No	No	No	No	Yes
SPFI_P↔ ORT_E↔ VENT_T↔ X_BM	This event occurs when the port transmits a broadcast message.	No	No	No	No	No	Yes
SPFI_P↔ ORT_E↔ VENT_↔ RX_BM	This event occurs when the port receives a broadcast message.	No	No	No	No	No	Yes

SpaceFibre Virtual Channel Input Events

Input Event	Description	Supported Devices					
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STARUltra PCIe Interface
SPFI_V↔ C_EVE↔ NT_RX_↔ SOP	This event occurs when the virtual channel receives an SOP.	No	No	No	No	No	Yes
SPFI_V↔ C_EVE↔ NT_RX_↔ EOP	This event occurs when the virtual channel receives an EOP.	No	No	No	No	No	Yes
SPFI_V↔ C_EVE↔ NT_RX_↔ EEP	This event occurs when the virtual channel receives an EEP.	No	No	No	No	No	Yes
SPFI_V↔ C_EVE↔ NT_TX_↔ SOP	This event occurs when the virtual channel transmits an SOP.	No	No	No	No	No	Yes
SPFI_V↔ C_EVE↔ NT_TX_↔ EOP	This event occurs when the virtual channel transmits an EOP.	No	No	No	No	No	Yes

SPFI_V↔ C_EVE↔ NT_TX_↔ PKT_PE↔ NDING	This event is active when the virtual channel has a packet queued for transmission.	No	No	No	No	No	Yes
--	---	----	----	----	----	----	-----

8.29.3 Output Actions

An output action is an action that is performed by a triggering resource. The actions are caused by the setting of an internal trigger, if configured to do so. Each triggering resource has one or more actions that can be performed. For example, a SpaceWire port can inject errors or transmit packets, if configured in packet transmit mode.

The following tables provide a description of the output actions for each triggering resource.

External Trigger Output Actions

Output Action	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCIe	Space↔ Wire PCIe Mk2	STAR↔ Ultra PCIe Interface
EXT_TR↔ IGGER_↔ ACTION↔ _OUT	This action causes an external trigger to output a signal.	Yes	Yes	No	No	Yes	No

Counter Output Actions

Output Action	Description	Supported Devices						
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STAR-Ultra PCIe Interface	
COUNTER_ACTION_COUNT	This action causes a counter to decrement by one.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_ACTION_RELOAD	This action causes a counter to be set to its configured reload value.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_ACTION_START	This action causes a counter to be start, if start mode is enabled.	Yes	Yes	Yes	Yes	Yes	Yes	
COUNTER_ACTION_STOP	This action causes a counter to be stopped, if start-stop mode is enabled.	Yes	Yes	Yes	Yes	Yes	Yes	

SpaceWire Port Output Actions

Output Action	Description	Supported Devices						
		SpaceWire Brick Mk3/Mk4	SpaceWire PXI Mk1/Mk2 Interface	SpaceWire PXI Mk1/Mk2 Router	SpaceWire PCIe	SpaceWire PCIe Mk2	STAR-Ultra PCIe Interface	
SPW_PORT_ACTION_TRANSMIT_PKT	This action causes a port to transmit a queued packet, if transmit packet mode is enabled for the port.	Yes	Yes	Yes	Yes	Yes	No	

SPW_P↔ ORT_A↔ CTION_↔ DISCON↔ NECT	This action causes the link attached to a port to disconnect.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ PARITY↔ _ERROR	This action causes a port to inject a parity error.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ ESCAP↔ E_ERR↔ OR	This action causes a port to inject an escape error.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ INSERT↔ _FCT	This action causes a port to insert a Flow Control Token (FCT).	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ SUPPR↔ ESS_FCT	This action causes a port to suppress the next FCT to be transmitted.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ INCR_C↔ REDIT	This action causes a port to increment its credit.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ DECR_↔ CREDIT	This action causes a port to decrement its credit.	Yes	Yes	Yes	Yes	Yes	No
SPW_P↔ ORT_A↔ CTION_↔ STOP_↔ RECEP↔ TION	This action causes a port to stop receiving whilst it is set.	Yes	No	No	No	Yes	No

SPW_P↔ ORT_A↔ CTION_↔ DISABL↔ E_LVDS	This action causes a port to disable its LVDS drivers whilst it is set.	Yes	No	No	No	Yes	No
--	---	-----	----	----	----	-----	----

Time-Code Interface Output Actions

Output Action	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCle	Space↔ Wire PCle Mk2	STAR-↔ Ultra PCle Interface
TIME_C↔ ODE_A↔ CTION_↔ TX	This action causes a time-code interface to transmit its next valid time-code.	Yes	Yes	Yes	No	Yes	No

SpaceFibre Port Output Actions

Output Action	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCle	Space↔ Wire PCle Mk2	STAR-↔ Ultra PCle Interface
SPFI_P↔ ORT_A↔ CTION_↔ DISCON↔ NECT	This action causes a Spacefibre port to disconnect the link.	No	No	No	No	No	Yes

SpaceFibre Virtual Channel Output Actions

Output Action	Description	Supported Devices					
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCle	Space↔ Wire PCle Mk2	STAR↔ Ultra PCle Interface
SPFI_V↔ C_ACTI↔ ON_TR↔ ANSMIT↔ _PKT	This action causes a VC to transmit a queued packet, if transmit packet mode is enabled for the VC.	No	No	No	No	No	Yes
SPFI_V↔ C_ACTI↔ ON_TR↔ ANSMIT↔ _DATA	This action causes a VC to transmit data while set, if transmit data mode is enabled for the VC.	No	No	No	No	No	Yes
SPFI_V↔ C_ACTI↔ ON_RE↔ CEIVE_↔ DATA	This action causes a VC to receive data while set, if receive data mode is enabled for the VC.	No	No	No	No	No	Yes

8.29.4 Internal Triggers

Internal triggers are used to control which input events cause output actions. An internal trigger can receive one or more input events which cause it to be set. Once set, the internal trigger causes one or more output actions.

An internal trigger is configured by taking a mask, for each triggering resource, that describes which events will cause it to be set. The internal trigger is then configured to operate in AND or OR mode. If operating in AND mode, the internal trigger will be set when all of its input events occur. If operating in OR mode, the internal trigger will be set when any of its input events occur. To control which output actions that the internal trigger will cause, it takes a mask, for each triggering resource, that describes which output actions will be performed when it is set.

The following diagram illustrates the input events, internal triggers, and output actions architecture.

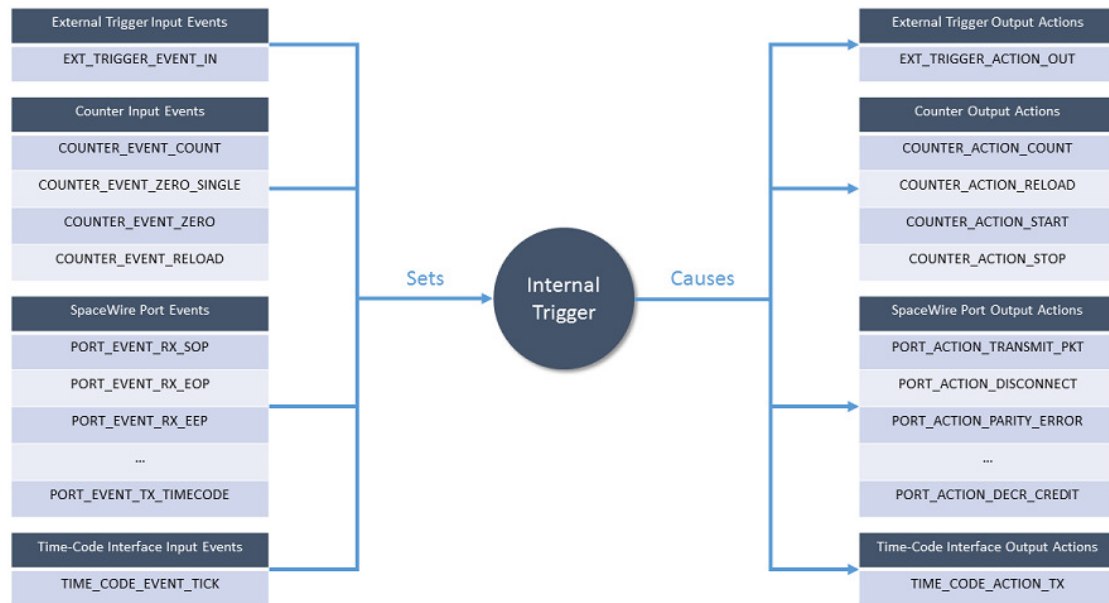


Figure 8.1: Generic Triggering API Architecture

The diagram above shows that an internal trigger is set by one or more input events. Once set, the internal trigger causes one or more output actions.

The following diagram shows an example of a triggering configuration.

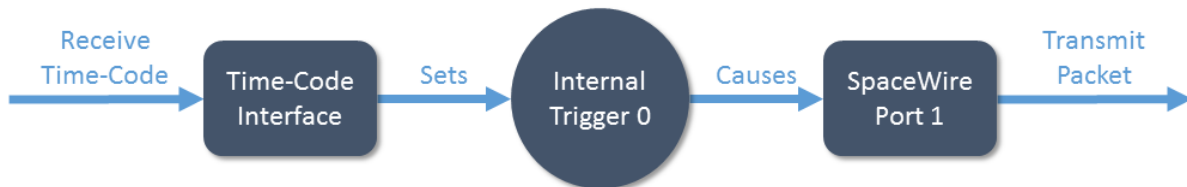


Figure 8.2: Generic Triggering API Example

In this example, a time-code received event on the time-code interface is configured to set internal trigger 0. Once set, internal trigger 0 causes a packet to be transmitted on SpaceWire port 1. The code for this example is shown below.

- `~~~~{.c} /* Enable port transmit packet mode on SpaceWire port 1 */ TRIGGER_CFG_enableSpWPort(`
`TransmitPacketMode(deviceId, 1);`
`/* TIME_CODE_EVENT_TICK event on the time-code interface causes internal trigger 0 to be set */ TRIG`
`GER_CFG_setTimeCodeInputEvents(deviceId, 0, 0, TIME_CODE_EVENT_TICK);`
`/* Internal trigger 0 causes SPW_PORT_ACTION_TRANSMIT_PKT action on SpaceWire port 1 */ TRIG`
`GER_CFG_setPortOutputActions(deviceId, 1, 0, SPW_PORT_ACTION_TRANSMIT_PKT);`
`/* Enable internal trigger 0 */ TRIGGER_CFG_enableTrigger(deviceId, 0);`
- `~~~~`

In the code listed above, SpaceWire port 1 is first configured to have "Port Transmit Mode" enabled. This means that any packets submitted for transmission on the port by STAR-System will be held in a buffer until explicitly instructed to transmit by an action. In the second line, internal trigger 0 is configured to be set by

the TIME_CODE_EVENT_TICK event from the time-code interface. Next, internal trigger 0 is configured to cause the SPW_PORT_ACTION_TRANSMIT_PKT action on SpaceWire port 1 when set. Finally, internal trigger 0 is enabled to allow it to receive events and cause actions.

8.29.5 Triggering Resources

There are seven types of triggering resource: external triggers, counters, SpaceWire ports, time-code interfaces, SpaceFibre ports, SpaceFibre virtual channels, and internal triggers. Each device that is supported by the Generic Triggering API has zero or more of each of these resources. The following table lists the number of triggering resources in each supported device.

Triggering Resources

Type	Description	Number of Resources						
		Space↔ Wire Brick Mk3/Mk4	Space↔ Wire PXI Mk1/Mk2 Interface	Space↔ Wire PXI Mk1/Mk2 Router	Space↔ Wire PCIe (see note below)	Space↔ Wire PCIe Mk2	STAR↔ Ultra PCIe Interface	
External Trigger	An external trigger can receive signals causing events and transmit signals as actions.	2	4	0	0	2	1	
Counter	A counter can operate in free-running mode as a timer or in triggered counting mode as a counter.	4	4	4	6	4	4	
SpaceWire Port	A SpaceWire port can cause events when it receives characters or errors and perform actions such as transmitting packets or injecting errors.	2	4	12	3	3	0	
Time↔ Code Interface	A time-code interface can cause events by receiving valid time-codes and transmit valid time-codes as actions.	1	1	1	0	1	0	

Space↔ Fibre Port	A Space↔ Fibre port can cause events when it changes state or trans- mits/receives broadcast messages and perform actions such as discon- necting the link.	0	0	0	0	0	2
Space↔ Fibre Virtual Channel	A Space↔ Fibre VC can cause events when it trans- mits/receives packets and perform actions such as transmit- ting packets.	0	0	0	0	0	8 per port
Internal Trigger	An internal trigger is set by receiving one or more input events. When set, it causes actions to be performed on other triggering resources.	8	8	4	6	8	8

Note that the number of resources supported by SpaceWire PCIe devices was increased in version 1.11e06. Previous versions support only 3 counters and internal triggers.

The external trigger, counter, SpaceWire port, SpaceFibre VC, and internal trigger triggering resources have various configuration settings. The following tables describe the configuration options for each of these triggering resources.

External Trigger Configuration

Setting	Description
Edge Detect Mode	When enabled, this setting causes the input event to occur on the rising edge of the input signal.
Invert	When enabled, this setting inverts the output signal and inverts the input signal before it reaches the detection mechanism.
Output	When enabled, the output action causes the external trigger to output a signal for the duration specified by the Extend value.
Extend Value	The extend value is the duration, in clock cycles, of the external trigger's output signal caused by an output action.

Counter Configuration

Setting	Description
Auto Reload	When enabled, this setting causes the counter to be automatically reloaded with its configured reload value when the counter reaches zero.
Trigger Count	When enabled, the counter must explicitly receive a count action before it is decremented. This allows the counter to operate as a counter rather than a free-running timer.
Start Mode	When enabled, the counter will wait for a start action before any counting or reloading can take place. When the counter reaches zero, it may be reloaded but no further counting or reloading will take place until the next start action occurs.
Start Stop Mode	When enabled, the counter will wait for a start action before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.
Load Zero	When enabled, reload triggered events will only occur when the counter value is zero.
Reload Value	The reload value is the value that the counter will be set to when it is reloaded.
Force Reload	The force reload setting causes the counter to be reloaded.

SpaceWire Port Configuration

Setting	Description
Transmit Packet Mode	When enabled, packets are queued for transmission on the SpaceWire port. If a packet transmit action occurs, a single packet will be transmitted from the queue.

SpaceFibre Virtual Channel Configuration

Setting	Description
Transmit Packet Mode	When enabled, packets are queued for transmission on the SpaceFibre VC. If a packet transmit action occurs, a single packet will be transmitted from the queue.

Transmit Data Mode	When enabled, the SpaceFibre VC will transmit data while the SPFI_VC_ACTION_TRANSMIT_DATA action is set.
Receive Data Mode	When enabled, the SpaceFibre VC will receive data while the SPFI_VC_ACTION_RECEIVE_DATA action is set.

Internal Trigger Configuration

Setting	Description
Input Mode	An internal trigger can be configured to operate in AND or OR mode. When operating in AND mode, the internal trigger is set when all of its input events occur. When operating in OR mode, the internal trigger is set when any of its input events occur.
Enabled	When enabled, the internal trigger can be set by its input events and cause its output actions to occur.
Force Trigger	The force trigger setting causes the internal trigger to be set.
Invert Mask	The invert mask determines, for each triggering resource, if the input events should be inverted before reaching the internal trigger.
AND Mask	The AND Mask determines, for each triggering resource, if the input events should be checked when the internal trigger is operating in AND mode.

8.29.6 PCIe Differences

The SpaceWire PCIe device's triggering support has several differences compared to the other devices. These differences are listed below.

- There is no time-code interface so the functions to trigger on valid time-code events and perform time-code transmit actions are not supported. However, the SpaceWire port time-code receive event is still supported so it can be used to trigger on time-codes.
- The SpaceWire port time-code transmit event is not supported by the SpaceWire PCIe device.
- There are no explicit packet transmit mode enable/disable functions. Instead, if any packet transmit action is enabled on a port, packet transmit mode is enabled implicitly.

8.29.7 Examples

The following sections describe several examples showing how to use the Generic Triggering API.

8.29.7.1 Transmitting Time-Codes Synchronised by an External Trigger

In this example, the triggers of a device will be configured so that time-codes are transmitted by the device every time the rising edge of an external trigger input signal is detected. The code for this example is listed below.

- ```
~~~~{.c} /* Enable external trigger 0 edge detect mode */ TRIGGER_CFG_enableExtTriggerEdgeDetect↵
Mode(deviceId, 0);

/* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal trigger 0 to be set */ TRIGGER↵
_CFG_setExtTriggerInputEvents(deviceId, 0, 0, EXT_TRIGGER_EVENT_IN);
```

```
/* Internal trigger 0 causes TIME_CODE_ACTION_TX action on the time-code interface */ TRIGGER_CFG↵
G_setTimeCodeOutputActions(deviceId, 0, 0, TIME_CODE_ACTION_TX);

/* Enable internal trigger 0 */ TRIGGER_CFG_enableTrigger(deviceId, 0);
```

- ~~~

In the code listed above, external trigger 0 is first configured to enable edge detect mode. This means that the EXT\_TRIGGER\_EVENT\_IN event will occur once, on the rising edge of the input signal. Otherwise, the event would be active for as long as the input signal is high (unless the invert setting is enabled, in which case the event would be active for as long as the input signal is low).

Next, internal trigger 0 is configured so that the EXT\_TRIGGER\_EVENT\_IN event from external trigger 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the TIME\_CODE\_ACTI↵  
ON\_TX action on the time-code interface. Finally, internal trigger 0 is enabled to allow the event to cause the action.

### 8.29.7.2 Packet Transmission Synchronised by a Counter

In this example, the triggers of a device will be configured so that packets submitted for transmission by STAR-↵  
System to SpaceWire port 1 will be queued and transmitted every time a free-running counter reloads. The code for this example is listed below.

- ~~~{.c} /\* Set counter 0 reload value to 60 million clock cycles \*/ TRIGGER\_CFG\_setCounterReload↵  
Value(deviceId, 0, 60000000);

/\* Enable counter auto reload \*/ TRIGGER\_CFG\_enableCounterAutoReload(deviceId, 0);

/\* Enable port transmit packet mode on SpaceWire port 1 \*/ TRIGGER\_CFG\_enableSpWPortTransmit↵  
PacketMode(deviceId, 1);

/\* COUNTER\_EVENT\_RELOAD event on counter 0 causes internal trigger 0 to be set \*/ TRIGGER\_CFG↵  
\_setCounterInputEvents(deviceId, 0, 0, COUNTER\_EVENT\_RELOAD);

/\* Internal trigger 0 causes SPW\_PORT\_ACTION\_TRANSMIT\_PKT action on SpaceWire port 1 \*/ TRIG↵  
GER\_CFG\_setSpWPortOutputActions(deviceId, 1, 0, SPW\_PORT\_ACTION\_TRANSMIT\_PKT);

/\* Enable internal trigger 0 \*/ TRIGGER\_CFG\_enableTrigger(deviceId, 0);

- ~~~

In the code listed above, counter 0 is first configured to set its reload value to 60 million clock cycles and enable its auto-reload setting. SpaceWire port 1 is then configured to enable its transmit packet mode.

Next, internal trigger 0 is configured so that the COUNTER\_EVENT\_RELOAD event from counter 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the SPW\_PORT\_ACTION\_T↵  
RANSMIT\_PKT action on SpaceWire port 1. Finally, internal trigger 0 is enabled to allow the event to cause the action.

### 8.29.7.3 Multiple Packet Transmission Synchronised by Time-Codes

In this example, the triggers of a device will be configured so that packets submitted for transmission by STAR-↵  
System to SpaceWire port 1 will be queued and transmitted in groups of 4 every time the device receives a valid time-code. The code for this example is listed below.

- ~~~{.c} /\* Enable counter trigger count on counter 0 so the counter only counts on triggers \*/ TRIGGER↵  
\_CFG\_enableCounterTriggerCount(deviceId, 0);

/\* Set counter 0 reload value to 3 \*/ TRIGGER\_CFG\_setCounterReloadValue(deviceId, 0, 3);

/\* Enable port transmit packet mode on SpaceWire port 1 \*/ TRIGGER\_CFG\_enableSpWPortTransmit↵  
PacketMode(deviceId, 1);

/\* TIME\_CODE\_EVENT\_TICK event on the time-code interface causes internal trigger 0 to be set \*/ TRIG↵  
GER\_CFG\_setTimeCodeInputEvents(deviceId, 0, 0, TIME\_CODE\_EVENT\_TICK);

```

/* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on counter 0 */ TRIGGER_CFG_set↵
CounterOutputActions(deviceId, 0, 0, COUNTER_ACTION_RELOAD);

/* Internal trigger 0 causes SPW_PORT_ACTION_TRANSMIT_PKT action on SpaceWire port 1 */ TRIG↵
GER_CFG_setSpWPortOutputActions(deviceId, 1, 0, SPW_PORT_ACTION_TRANSMIT_PKT);

/* SPW_PORT_EVENT_TX_EOP event on port 1 causes internal trigger 1 to be set */ TRIGGER_CFG_↵
setSpWPortInputEvents(deviceId, 1, 1, SPW_PORT_EVENT_TX_EOP);

/* Internal trigger 1 causes COUNTER_ACTION_COUNT action on counter 0 */ TRIGGER_CFG_set↵
CounterOutputActions(deviceId, 0, 1, COUNTER_ACTION_COUNT);

/* COUNTER_EVENT_COUNT event on counter 0 causes internal trigger 2 to be set */ TRIGGER_CFG_↵
setCounterInputEvents(deviceId, 0, 2, COUNTER_EVENT_COUNT);

/* Internal trigger 2 causes SPW_PORT_ACTION_TRANSMIT_PKT action on SpaceWire port 1 */ TRIG↵
GER_CFG_setPortOutputActions(deviceId, 1, 2, SPW_PORT_ACTION_TRANSMIT_PKT);

/* Enable internal triggers 0, 1 and 2 */ TRIGGER_CFG_enableTrigger(deviceId, 0); TRIGGER_CFG_↵
enableTrigger(deviceId, 1); TRIGGER_CFG_enableTrigger(deviceId, 2);

```

- ~~~

In the code listed above, counter 0 is configured to enable its trigger count mode. This means that the counter will only decrement when it receives a COUNTER\_ACTION\_COUNT action. Additionally, counter 0 has its reload value set to 3. Auto-reload is not enabled in this case because reloading will be controlled by the triggers. SpaceWire Port 1 is then configured to have its transmit packet mode enabled as described in the previous example.

Next, internal trigger 0 is configured so that the TIME\_CODE\_EVENT\_TICK event from the time-code interface will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes two actions to be performed: firstly, counter 0 is reloaded using the COUNTER\_ACTION\_RELOAD action, and secondly, the first packet in the group is transmitted on port 1 using the SPW\_PORT\_ACTION\_TRANSMIT\_PKT action. Internal trigger 1 is configured so that the SPW\_PORT\_EVENT\_TX\_EOP event on port 1 will cause it to be set. Internal trigger 1 is also configured so that, when set, it causes the COUNTER\_ACTION\_COUNT action on counter 0. Internal trigger 2 is configured so that the COUNTER\_EVENT\_COUNT event from counter 0 causes it to be set. Internal trigger 2 is also configured so that, when set, it causes the SPW\_PORT\_ACTION\_TRANSMIT\_PKT action on port 1. Finally, internal triggers 0, 1 and 2 are enabled to allow the events to cause the actions.

To summarise this example, when a time-code is received by the device it reloads the packet counter and transmits the first packet in the group of 4. When the EOP from the packet is transmitted, it decrements the packet counter. When the packet counter is decremented it transmits another packet. The transmitting EOPs continue causing the packet counter to decrement and more packets to be transmitted until the counter reaches 0, at which point the transmission stops. When the next time-code is received, the packet counter is reloaded and the process begins again, transmitting the next group of 4 packets.

## 8.30 General

Functions in this group are used to configure general triggering functionality.

Collaboration diagram for General:



### Functions

- `int STAR_API_CC TRIGGER_CFG_reset (const STAR_DEVICE_ID deviceId)`  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_CFG_getTriggerMatrix (const STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC TRIGGER_CFG_getDeviceClockRate (const STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC TRIGGER_CFG_getDeviceClockFreq (const STAR_DEVICE_ID deviceId, U32 *const pClockFreqHz)`  
*Gets the clock frequency (in hertz) from the specified device.*

### 8.30.1 Detailed Description

Functions in this group are used to configure general triggering functionality.

### 8.30.2 Function Documentation

**8.30.2.1** `int STAR_API_CC TRIGGER_CFG_getDeviceClockFreq ( const STAR_DEVICE_ID deviceId, U32 *const pClockFreqHz )`

Gets the clock frequency (in hertz) from the specified device.

**Parameters**

|                     |                                                                 |
|---------------------|-----------------------------------------------------------------|
| <i>deviceId</i>     | Identifier of the device to query.                              |
| <i>pClockFreqHz</i> | Pointer to value to be updated with the device clock frequency. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c.`

### 8.30.2.2 `int STAR_API_CC TRIGGER_CFG_getDeviceClockRate ( const STAR_DEVICE_ID deviceId )`

**Deprecated** Function superseded by `TRIGGER_CFG_getDeviceClockFreq()`. Gets the clock rate from the specified device that is being used as the clock rate in/with the triggering functions/functionality.

**Parameters**


---

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceId</i> | Identifier of the device to query. |
|-----------------|------------------------------------|

---

**Returns**

The device's clock rate that is being used by the triggering functionality in Hz.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

### 8.30.2.3 `TRIGGER_MATRIX STAR_API_CC TRIGGER_CFG_getTriggerMatrix ( const STAR_DEVICE_ID deviceId )`

**Deprecated** Function superseded by a number of new resource count retrieval functions e.g., `TRIGGER_CFG_getNumberOfExtTriggers`.

Retrieves the trigger matrix of selected device

**Parameters**


---

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceId</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

---

**Returns**

The trigger matrix of the selected device.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

### 8.30.2.4 `int STAR_API_CC TRIGGER_CFG_reset ( const STAR_DEVICE_ID deviceId )`

Resets the triggering configuration to its default state.



---

**Parameters**

---

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

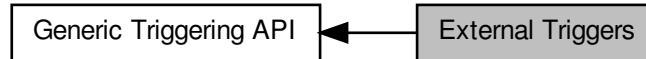
**Examples:**

`triggering_generic_example.c`.

## 8.31 External Triggers

Functions in this group are used to configure triggering functionality for the external triggers.

Collaboration diagram for External Triggers:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setExtTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_EVENT_MASK events)`  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_EVENT_MASK *const pEvents)`  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setExtTriggerOutputActions (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerOutputActions (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_ACTION_MASK *const pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerEdgeDetectMode (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerEdgeDetectMode (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerEdgeDetectModeEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerInvert (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerInvert (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerInvertEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether invert is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerOutput (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerOutput (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables output for a specific external trigger.*

- `int STAR_API_CC TRIGGER_CFG_getExtTriggerOutputEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether output is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_setExtTriggerExtend (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 extend)`  
*Sets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerExtend (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfExtTriggers (const STAR_DEVICE_ID deviceId, U32 *const pNumExtTriggers)`  
*Gets the number of external trigger resources available on the specified device.*

### 8.31.1 Detailed Description

Functions in this group are used to configure triggering functionality for the external triggers.

### 8.31.2 Function Documentation

#### 8.31.2.1 `int STAR_API_CC TRIGGER_CFG_disableExtTriggerEdgeDetectMode ( const STAR_DEVICE_ID deviceId, const U32 extTrigger )`

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

##### Parameters

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

##### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

##### Added in version:

STAR-System v5.02 - 25th November 2022

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

#### 8.31.2.2 `int STAR_API_CC TRIGGER_CFG_disableExtTriggerInvert ( const STAR_DEVICE_ID deviceId, const U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the output trigger signal is not inverted, and the input trigger signal is not inverted before the detection mechanism.

### Parameters

---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

---

### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

### Added in version:

STAR-System v5.02 - 25th November 2022

### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.31.2.3** `int STAR_API_CC TRIGGER_CFG_disableExtTriggerOutput ( const STAR_DEVICE_ID deviceld, const U32 extTrigger )`

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.

### Parameters

---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

---

### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

### Added in version:

STAR-System v5.02 - 25th November 2022

### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

### Examples:

`triggering_generic_example.c`.

**8.31.2.4** `int STAR_API_CC TRIGGER_CFG_enableExtTriggerEdgeDetectMode ( const STAR_DEVICE_ID deviceld, const U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

### Parameters

---

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c`.

**8.31.2.5** `int STAR_API_CC TRIGGER_CFG_enableExtTriggerInvert ( const STAR_DEVICE_ID deviceld, const U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the output trigger signal is inverted, and the input trigger signal is inverted before the detection mechanism.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.31.2.6** `int STAR_API_CC TRIGGER_CFG_enableExtTriggerOutput ( const STAR_DEVICE_ID deviceld, const U32 extTrigger )`

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.31.2.7** `int STAR_API_CC TRIGGER_CFG_getExtTriggerEdgeDetectModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, U32 *const pEnabled )`

Gets whether edge detect mode is enabled for a specific external trigger.

**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.31.2.8** `int STAR_API_CC TRIGGER_CFG_getExtTriggerExtend ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, U32 *const pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the external trigger. |
|-----------------|-----------------------------------------|

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2

**8.31.2.9** `int STAR_API_CC TRIGGER_CFG_getExtTriggerInputEvents ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_EVENT_MASK *const pEvents )`

Gets the input events from an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.31.2.10** `int STAR_API_CC TRIGGER_CFG_getExtTriggerInvertEnabled ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, U32 *const pEnabled )`

Gets whether invert is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

8.31.2.11 `int STAR_API_CC_TRIGGER_CFG_getExtTriggerOutputActions ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_ACTION_MASK *const pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

8.31.2.12 `int STAR_API_CC_TRIGGER_CFG_getExtTriggerOutputEnabled ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, U32 *const pEnabled )`

Gets whether output is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

8.31.2.13 `int STAR_API_CC_TRIGGER_CFG_getNumberOfExtTriggers ( const STAR_DEVICE_ID deviceld, U32 *const pNumExtTriggers )`

Gets the number of external trigger resources available on the specified device.



**Parameters**

|                        |                                                                                |
|------------------------|--------------------------------------------------------------------------------|
| <i>deviceId</i>        | Identifier of the device.                                                      |
| <i>pNumExtTriggers</i> | Pointer to value to be updated with the number of available external triggers. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c`.

**8.31.2.14** `int STAR_API_CC TRIGGER_CFG_setExtTriggerExtend ( const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceId</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |
| <i>extend</i>     | Extend duration in cycles.                  |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2

**Examples:**

`triggering_generic_example.c`.

**8.31.2.15** `int STAR_API_CC TRIGGER_CFG_setExtTriggerInputEvents ( const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                 |
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.31.2.16** `int STAR_API_CC TRIGGER_CFG_setExtTriggerOutputActions ( const STAR_DEVICE_ID deviceld, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.           |
| <i>extTrigger</i> | External trigger to set output actions for.       |
| <i>trigger</i>    | Internal trigger which causes the output actions. |
| <i>actions</i>    | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

## 8.32 Counters

Functions in this group are used to configure triggering functionality for the counters.

Collaboration diagram for Counters:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setCounterInputEvents (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, const COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getCounterInputEvents (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, COUNTER_EVENT_MASK *const pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setCounterOutputActions (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, const COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getCounterOutputActions (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, COUNTER_ACTION_MASK *const pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableCounterAutoReload (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_disableCounterAutoReload (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterAutoReloadEnabled (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_enableCounterTriggerCount (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_disableCounterTriggerCount (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Disables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterTriggerCountEnabled (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pEnabled)`  
*Gets whether trigger count is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_enableCounterStartMode (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Enables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_disableCounterStartMode (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Disables start mode for a specific counter.*

- `int STAR_API_CC TRIGGER_CFG_getCounterStartModeEnabled (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pEnabled)`  
*Gets whether start mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_enableCounterStartStopMode (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Enables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_disableCounterStartStopMode (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Disables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterStartStopModeEnabled (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pEnabled)`  
*Gets whether start stop mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_enableCounterLoadZero (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Enables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_disableCounterLoadZero (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Disables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterLoadZeroEnabled (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pEnabled)`  
*Gets whether load zero is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_setCounterReloadValue (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 reloadValue)`  
*Sets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterReloadValue (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pReloadValue)`  
*Gets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_forceCounterReload (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Force the reload of a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterValue (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pValue)`  
*Gets the counter value for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfCounters (const STAR_DEVICE_ID deviceId, U32 *const pNumCounters)`  
*Gets the number of counter resources available on the specified device.*

### 8.32.1 Detailed Description

Functions in this group are used to configure triggering functionality for the counters.

A counter can operate in free-running mode as a timer or in triggered counting mode as a counter.

### 8.32.2 Function Documentation

#### 8.32.2.1 `int STAR_API_CC TRIGGER_CFG_disableCounterAutoReload ( const STAR_DEVICE_ID deviceId, const U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

### 8.32.2.2 `int STAR_API_CC TRIGGER_CFG_disableCounterLoadZero ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

### 8.32.2.3 `int STAR_API_CC TRIGGER_CFG_disableCounterStartMode ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

#### 8.32.2.4 `int STAR_API_CC TRIGGER_CFG_disableCounterStartStopMode ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

#### 8.32.2.5 `int STAR_API_CC TRIGGER_CFG_disableCounterTriggerCount ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

|                |                                      |
|----------------|--------------------------------------|
| <i>counter</i> | Counter to disable trigger count on. |
|----------------|--------------------------------------|

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.6** `int STAR_API_CC TRIGGER_CFG_enableCounterAutoReload ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**


---

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.7** `int STAR_API_CC TRIGGER_CFG_enableCounterLoadZero ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

---

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.8** `int STAR_API_CC_TRIGGER_CFG_enableCounterStartMode ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start/stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**


---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.9** `int STAR_API_CC_TRIGGER_CFG_enableCounterStartStopMode ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start/stop mode are mutually exclusive and enabling one will disable the other.



**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.10** `int STAR_API_CC TRIGGER_CFG_enableCounterTriggerCount ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.11** `int STAR_API_CC TRIGGER_CFG_forceCounterReload ( const STAR_DEVICE_ID deviceld, const U32 counter )`

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.12** `int STAR_API_CC TRIGGER_CFG_getCounterAutoReloadEnabled ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.13** `int STAR_API_CC TRIGGER_CFG_getCounterInputEvents ( const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, COUNTER_EVENT_MASK *const pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>trigger</i>  | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |
| <i>pEvents</i>  | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.14** `int STAR_API_CC_TRIGGER_CFG_getCounterLoadZeroEnabled ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.15** `int STAR_API_CC_TRIGGER_CFG_getCounterOutputActions ( const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, COUNTER_ACTION_MASK *const pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to get output actions from. |

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.16** `int STAR_API_CC_TRIGGER_CFG_getCounterReloadValue ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

#### Parameters

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.17** `int STAR_API_CC_TRIGGER_CFG_getCounterStartModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled )`

Gets whether start mode is enabled for a specific counter.

#### Parameters

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.18** `int STAR_API_CC_TRIGGER_CFG_getCounterStartStopModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.19** `int STAR_API_CC_TRIGGER_CFG_getCounterTriggerCountEnabled ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.20** `int STAR_API_CC TRIGGER_CFG_getCounterValue ( const STAR_DEVICE_ID deviceld, const U32 counter, U32 *const pValue )`

Gets the counter value for a specific counter.

#### Parameters

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.32.2.21** `int STAR_API_CC TRIGGER_CFG_getNumberOfCounters ( const STAR_DEVICE_ID deviceld, U32 *const pNumCounters )`

Gets the number of counter resources available on the specified device.

#### Parameters

|                     |                                                                       |
|---------------------|-----------------------------------------------------------------------|
| <i>deviceld</i>     | Identifier of the device.                                             |
| <i>pNumCounters</i> | Pointer to value to be updated with the number of available counters. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

#### Examples:

`triggering_generic_example.c`.

**8.32.2.22** `int STAR_API_CC TRIGGER_CFG_setCounterInputEvents ( const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, const COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.23** `int STAR_API_CC TRIGGER_CFG_setCounterOutputActions ( const STAR_DEVICE_ID deviceld, const U32 counter, const U32 trigger, const COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.32.2.24** `int STAR_API_CC TRIGGER_CFG_setCounterReloadValue ( const STAR_DEVICE_ID deviceld, const U32 counter, const U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

---

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

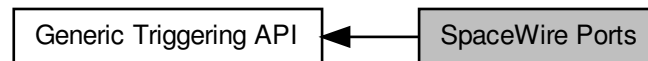
[triggering\\_generic\\_example.c](#).



## 8.33 SpaceWire Ports

Functions in this group are used to configure triggering functionality for the SpW ports.

Collaboration diagram for SpaceWire Ports:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setPortInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` port, const `U32` trigger, const `PORT_EVENT_MASK` events)
- `int STAR_API_CC TRIGGER_CFG_getPortInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` port, const `U32` trigger, `PORT_EVENT_MASK` \*const pEvents)
- `int STAR_API_CC TRIGGER_CFG_setPortOutputActions` (const `STAR_DEVICE_ID` deviceId, const `U32` port, const `U32` trigger, const `PORT_ACTION_MASK` actions)
- `int STAR_API_CC TRIGGER_CFG_getPortOutputActions` (const `STAR_DEVICE_ID` deviceId, const `U32` port, const `U32` trigger, `PORT_ACTION_MASK` \*const pActions)
- `int STAR_API_CC TRIGGER_CFG_enablePortTransmitPacketMode` (const `STAR_DEVICE_ID` deviceId, const `U32` port)
- `int STAR_API_CC TRIGGER_CFG_disablePortTransmitPacketMode` (const `STAR_DEVICE_ID` deviceId, const `U32` port)
- `int STAR_API_CC TRIGGER_CFG_getPortTransmitPacketModeEnabled` (const `STAR_DEVICE_ID` deviceId, const `U32` port, `U32` \*const pEnabled)
- `int STAR_API_CC TRIGGER_CFG_enablePortSpill` (const `STAR_DEVICE_ID` deviceId, const `U32` port)
- `int STAR_API_CC TRIGGER_CFG_disablePortSpill` (const `STAR_DEVICE_ID` deviceId, const `U32` port)
- `int STAR_API_CC TRIGGER_CFG_getPortSpillEnabled` (const `STAR_DEVICE_ID` deviceId, const `U32` port, `U32` \*const pEnabled)
- `int STAR_API_CC TRIGGER_CFG_setSpWPortInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort, const `U32` trigger, const `SPW_PORT_EVENT_MASK` events)  
*Sets the input events for a SpaceWire port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort, const `U32` trigger, `SPW_PORT_EVENT_MASK` \*const pEvents)  
*Gets the input events from a SpaceWire port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpWPortOutputActions` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort, const `U32` trigger, const `SPW_PORT_ACTION_MASK` actions)  
*Sets the output actions for a SpaceWire port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortOutputActions` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort, const `U32` trigger, `SPW_PORT_ACTION_MASK` \*const pActions)  
*Gets the output actions from a SpaceWire port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableSpWPortTransmitPacketMode` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort)  
*Enables packet transmit mode for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpWPortTransmitPacketMode` (const `STAR_DEVICE_ID` deviceId, const `U32` spwPort)  
*Disables packet transmit mode for a specific SpaceWire port.*

- `int STAR_API_CC TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled (const STAR_DEVICE_ID deviceld, const U32 spwPort, U32 *const pEnabled)`  
*Gets whether packet transmit mode is enabled for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_enableSpWPortSpill (const STAR_DEVICE_ID deviceld, const U32 spwPort)`  
*Enables spilling for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpWPortSpill (const STAR_DEVICE_ID deviceld, const U32 spwPort)`  
*Disables spilling for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortSpillEnabled (const STAR_DEVICE_ID deviceld, const U32 spwPort, U32 *const pEnabled)`  
*Gets whether spilling is enabled for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfSpWPorts (const STAR_DEVICE_ID deviceld, U32 *const pNumSpWPorts)`  
*Gets the number of SpaceWire port resources available on the specified device.*

### 8.33.1 Detailed Description

Functions in this group are used to configure triggering functionality for the SpW ports.

### 8.33.2 Function Documentation

#### 8.33.2.1 `int STAR_API_CC TRIGGER_CFG_disablePortSpill ( const STAR_DEVICE_ID deviceld, const U32 port )`

**Deprecated** Function superseded by `TRIGGER_CFG_disableSpWPortSpill()`.

Disables spilling for a specific port.

#### Parameters

|                 |                              |
|-----------------|------------------------------|
| <i>deviceld</i> | Device containing the port.  |
| <i>port</i>     | Port to disable spilling on. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

#### 8.33.2.2 `int STAR_API_CC TRIGGER_CFG_disablePortTransmitPacketMode ( const STAR_DEVICE_ID deviceld, const U32 port )`

**Deprecated** Function superseded by `TRIGGER_CFG_disableSpWPortTransmitPacketMode()`.

Disables packet transmit mode for a specific port. When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

8.33.2.3 `int STAR_API_CC TRIGGER_CFG_disableSpWPortSpill ( const STAR_DEVICE_ID deviceld, const U32 spwPort )`

Disables spilling for a specific SpaceWire port.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.  |
| <i>spwPort</i>  | SpaceWire port to disable spilling on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

8.33.2.4 `int STAR_API_CC TRIGGER_CFG_disableSpWPortTransmitPacketMode ( const STAR_DEVICE_ID deviceld, const U32 spwPort )`

Disables packet transmit mode for a specific SpaceWire port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                                    |
|-----------------|----------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.              |
| <i>spwPort</i>  | SpaceWire port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

8.33.2.5 `int STAR_API_CC TRIGGER_CFG_enablePortSpill ( const STAR_DEVICE_ID deviceld, const U32 port )`

**Deprecated** Function superseded by `TRIGGER_CFG_enableSpWPortSpill()`.

Enables spilling for a specific port. While enabled AND the port has transmit packet mode enabled, queued packets are spilled.

**Parameters**

|                 |                             |
|-----------------|-----------------------------|
| <i>deviceld</i> | Device containing the port. |
| <i>port</i>     | Port to enable spilling on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

8.33.2.6 `int STAR_API_CC TRIGGER_CFG_enablePortTransmitPacketMode ( const STAR_DEVICE_ID deviceld, const U32 port )`

**Deprecated** Function superseded by `TRIGGER_CFG_enableSpWPortTransmitPacketMode()`.

Enables packet transmit mode for a specific port. When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

8.33.2.7 `int STAR_API_CC TRIGGER_CFG_enableSpWPortSpill ( const STAR_DEVICE_ID deviceId, const U32 spwPort )`

Enables spilling for a specific SpaceWire port.

While enabled AND the SpaceWire port has transmit packet mode enabled, queued packets are spilled.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the port.           |
| <i>spwPort</i>  | SpaceWire port to enable spilling on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

**8.33.2.8** `int STAR_API_CC TRIGGER_CFG_enableSpWPortTransmitPacketMode ( const STAR_DEVICE_ID deviceld, const U32 spwPort )`

Enables packet transmit mode for a specific SpaceWire port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.             |
| <i>spwPort</i>  | SpaceWire port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

**Examples:**

`triggering_generic_example.c`.

**8.33.2.9** `int STAR_API_CC TRIGGER_CFG_getNumberOfSpWPorts ( const STAR_DEVICE_ID deviceld, U32 *const pNumSpWPorts )`

Gets the number of SpaceWire port resources available on the specified device.

**Parameters**

|                 |                           |
|-----------------|---------------------------|
| <i>deviceld</i> | Identifier of the device. |
|-----------------|---------------------------|

---

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| <i>pNumSpWPorts</i> | Pointer to value to be updated with the number of available SpaceWire ports. |
|---------------------|------------------------------------------------------------------------------|

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

8.33.2.10 `int STAR_API_CC TRIGGER_CFG_getPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 port, const U32 trigger, PORT_EVENT_MASK *const pEvents )`

**Deprecated** Function superseded by `TRIGGER_CFG_getSpWPortInputEvents()`.

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

8.33.2.11 `int STAR_API_CC TRIGGER_CFG_getPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 port, const U32 trigger, PORT_ACTION_MASK *const pActions )`

**Deprecated** Function superseded by `TRIGGER_CFG_getSpWPortOutputActions()`.

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

8.33.2.12 `int STAR_API_CC TRIGGER_CFG_getPortSpillEnabled ( const STAR_DEVICE_ID deviceld, const U32 port, U32 *const pEnabled )`

**Deprecated** Function superseded by `TRIGGER_CFG_getSpWPortSpillEnabled()`.

Gets whether spilling is enabled for a specific port.

**Parameters**

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                   |
| <i>port</i>     | Port to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if spilling is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

8.33.2.13 `int STAR_API_CC TRIGGER_CFG_getPortTransmitPacketModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 port, U32 *const pEnabled )`

**Deprecated** Function superseded by

`TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled()`.

Gets whether packet transmit mode is enabled for a specific port.



**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

**8.33.2.14** `int STAR_API_CC TRIGGER_CFG_getSpWPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spwPort, const U32 trigger, SPW_PORT_EVENT_MASK *const pEvents )`

Gets the input events from a SpaceWire port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.                          |
| <i>spwPort</i>  | SpaceWire port to get input events from.                       |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

**8.33.2.15** `int STAR_API_CC TRIGGER_CFG_getSpWPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spwPort, const U32 trigger, SPW_PORT_ACTION_MASK *const pActions )`

Gets the output actions from a SpaceWire port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.             |
| <i>spwPort</i>  | SpaceWire port to get output actions from.        |
| <i>trigger</i>  | Internal trigger which causes the output actions. |

---

*pActions* Pointer to a value which will be updated with the actions mask.

---

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

**8.33.2.16** `int STAR_API_CC_TRIGGER_CFG_getSpWPortSpillEnabled ( const STAR_DEVICE_ID deviceld, const U32 spwPort, U32 *const pEnabled )`

Gets whether spilling is enabled for a specific SpaceWire port.

#### Parameters

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.                                         |
| <i>spwPort</i>  | SpaceWire port to check.                                                      |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if spilling is enabled, else 0. |

---

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4 (versions 1.05e15 and 1.00e09, respectively, and greater), SpaceWire GbE Brick Mk1, SpaceWire PCIe Mk2

**8.33.2.17** `int STAR_API_CC_TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 spwPort, U32 *const pEnabled )`

Gets whether packet transmit mode is enabled for a specific SpaceWire port.

#### Parameters

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.                                                     |
| <i>spwPort</i>  | SpaceWire port to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

---

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

8.33.2.18 `int STAR_API_CC TRIGGER_CFG_setPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 port, const U32 trigger, const PORT_EVENT_MASK events )`

**Deprecated** Function superseded by `TRIGGER_CFG_setSpWPortInputEvents()`. Sets the input events for a port which will cause an internal trigger to be set.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

8.33.2.19 `int STAR_API_CC TRIGGER_CFG_setPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 port, const U32 trigger, const PORT_ACTION_MASK actions )`

**Deprecated** Function superseded by `TRIGGER_CFG_setSpWPortOutputActions()`.

Sets the output actions for a port that are caused by an internal trigger being set.

#### Parameters

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

8.33.2.20 `int STAR_API_CC TRIGGER_CFG_setSpWPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spwPort, const U32 trigger, const SPW_PORT_EVENT_MASK events )`

Sets the input events for a SpaceWire port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.                   |
| <i>spwPort</i>  | SpaceWire port to set input events for.                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.33.2.21** `int STAR_API_CC TRIGGER_CFG_setSpWPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spwPort, const U32 trigger, const SPW_PORT_ACTION_MASK actions )`

Sets the output actions for a SpaceWire port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceWire port.             |
| <i>spwPort</i>  | SpaceWire port to set output actions for.         |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

## 8.34 Time-codes

Functions in this group are used to configure triggering functionality for the time-code engine(s).

Collaboration diagram for Time-codes:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setTimeCodeInputEvents (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getTimeCodeInputEvents (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_EVENT_MASK *const pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setTimeCodeOutputActions (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getTimeCodeOutputActions (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_ACTION_MASK *const pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfTimeCodeInterfaces (const STAR_DEVICE_ID deviceId, U32 *const pNumInterfaces)`  
*Gets the number of time code resources available on the specified device.*

### 8.34.1 Detailed Description

Functions in this group are used to configure triggering functionality for the time-code engine(s).

### 8.34.2 Function Documentation

**8.34.2.1** `int STAR_API_CC TRIGGER_CFG_getNumberOfTimeCodeInterfaces ( const STAR_DEVICE_ID deviceId, U32 *const pNumInterfaces )`

Gets the number of time code resources available on the specified device.

#### Parameters

|                       |                                                                                  |
|-----------------------|----------------------------------------------------------------------------------|
| <i>deviceId</i>       | Identifier of the device.                                                        |
| <i>pNumInterfaces</i> | Pointer to value to be updated with the number of available time code resources. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.34.2.2** `int STAR_API_CC TRIGGER_CFG_getTimeCodeInputEvents ( const STAR_DEVICE_ID deviceld, const U32 timeCode, const U32 trigger, TIME_CODE_EVENT_MASK *const pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

**8.34.2.3** `int STAR_API_CC TRIGGER_CFG_getTimeCodeOutputActions ( const STAR_DEVICE_ID deviceld, const U32 timeCode, const U32 trigger, TIME_CODE_ACTION_MASK *const pActions )`

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

**8.34.2.4** `int STAR_API_CC TRIGGER_CFG_setTimeCodeInputEvents ( const STAR_DEVICE_ID deviceld, const U32 timeCode, const U32 trigger, const TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.34.2.5** `int STAR_API_CC TRIGGER_CFG_setTimeCodeOutputActions ( const STAR_DEVICE_ID deviceld, const U32 timeCode, const U32 trigger, const TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

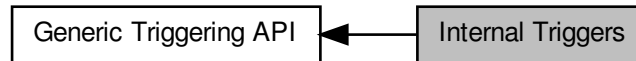
**Examples:**

[triggering\\_generic\\_example.c](#).

## 8.35 Internal Triggers

Functions in this group are used to configure triggering functionality for the internal triggers.

Collaboration diagram for Internal Triggers:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setTriggerInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` causeTrigger, const `U32` trigger, const `TRIGGER_EVENT_MASK` events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerInputEvents` (const `STAR_DEVICE_ID` deviceId, const `U32` causeTrigger, const `U32` trigger, `TRIGGER_EVENT_MASK` \*const pEvents)  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerInputMode` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_INPUT_MODE` mode)  
*Sets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerInputMode` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, `TRIGGER_INPUT_MODE` \*const pMode)  
*Gets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableTrigger` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger)  
*Enables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableTrigger` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger)  
*Disables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerEnabled` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, `U32` \*const pEnabled)  
*Gets whether a trigger is enabled.*
- `int STAR_API_CC TRIGGER_CFG_forceTrigger` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger)  
*Force the triggering of a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerInvertMask` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_TYPE` type, const `U32` mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerInvertMask` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_TYPE` type, `U32` \*const pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerAndMask` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_TYPE` type, const `U32` mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerAndMask` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_TYPE` type, `U32` \*const pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerSourceEventsInverted` (const `STAR_DEVICE_ID` deviceId, const `U32` trigger, const `TRIGGER_TYPE` type, const `U32` source, const `U32` inverted)



*Set the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.*

- int [STAR\\_API\\_CC TRIGGER\\_CFG\\_getTriggerSourceEventsInverted](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) trigger, const [TRIGGER\\_TYPE](#) type, const [U32](#) source, [U32](#) \*const pInverted)

*Get the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.*

- int [STAR\\_API\\_CC TRIGGER\\_CFG\\_setTriggerSourceEventsAnded](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) trigger, const [TRIGGER\\_TYPE](#) type, const [U32](#) source, const [U32](#) anded)

*Set whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.*

- int [STAR\\_API\\_CC TRIGGER\\_CFG\\_getTriggerSourceEventsAnded](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) trigger, const [TRIGGER\\_TYPE](#) type, const [U32](#) source, [U32](#) \*const pAnded)

*Get whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.*

- int [STAR\\_API\\_CC TRIGGER\\_CFG\\_getNumberOfTriggers](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, [U32](#) \*const pNumTriggers)

*Gets the number of internal trigger resources available on the specified device.*

### 8.35.1 Detailed Description

Functions in this group are used to configure triggering functionality for the internal triggers.

### 8.35.2 Function Documentation

#### 8.35.2.1 int [STAR\\_API\\_CC TRIGGER\\_CFG\\_disableTrigger](#) ( const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) trigger )

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

#### Parameters

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

#### 8.35.2.2 int [STAR\\_API\\_CC TRIGGER\\_CFG\\_enableTrigger](#) ( const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) trigger )

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

triggering\_generic\_example.c.

### 8.35.2.3 `int STAR_API_CC TRIGGER_CFG_forceTrigger ( const STAR_DEVICE_ID deviceld, const U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

### 8.35.2.4 `int STAR_API_CC TRIGGER_CFG_getNumberOfTriggers ( const STAR_DEVICE_ID deviceld, U32 *const pNumTriggers )`

Gets the number of internal trigger resources available on the specified device.

**Parameters**

|                     |                                                                                |
|---------------------|--------------------------------------------------------------------------------|
| <i>deviceld</i>     | Identifier of the device.                                                      |
| <i>pNumTriggers</i> | Pointer to value to be updated with the number of available internal triggers. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c`.

**8.35.2.5** `int STAR_API_CC TRIGGER_CFG_getTriggerAndMask ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.6** `int STAR_API_CC TRIGGER_CFG_getTriggerEnabled ( const STAR_DEVICE_ID deviceld, const U32 trigger, U32 *const pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.7** `int STAR_API_CC TRIGGER_CFG_getTriggerInputEvents ( const STAR_DEVICE_ID deviceld, const U32 causeTrigger, const U32 trigger, TRIGGER_EVENT_MASK *const pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

#### Parameters

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.8** `int STAR_API_CC TRIGGER_CFG_getTriggerInputMode ( const STAR_DEVICE_ID deviceld, const U32 trigger, TRIGGER_INPUT_MODE *const pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

#### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.9** `int STAR_API_CC TRIGGER_CFG_getTriggerInvertMask ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.10** `int STAR_API_CC TRIGGER_CFG_getTriggerSourceEventsAnded ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pAnded )`

Get whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.

By default, OR logic is applied.

**Parameters**

|                 |                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                                                         |
| <i>trigger</i>  | The internal trigger number.                                                                                    |
| <i>type</i>     | The source type.                                                                                                |
| <i>source</i>   | The source number.                                                                                              |
| <i>pAnded</i>   | Pointer to the value to be updated with the AND state. A value of zero indicates OR logic, otherwise AND logic. |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.11** `int STAR_API_CC TRIGGER_CFG_getTriggerSourceEventsInverted ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pInverted )`

Get the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.

By default, the signal will not be inverted.

**Parameters**

|                  |                                                                                                                                           |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i>  | Device containing the internal trigger.                                                                                                   |
| <i>trigger</i>   | The internal trigger number.                                                                                                              |
| <i>type</i>      | The source type.                                                                                                                          |
| <i>source</i>    | The source number.                                                                                                                        |
| <i>pInverted</i> | Pointer to the value to be updated with the invert state. A value of zero indicates that the signal is not inverted, else it is inverted. |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

8.35.2.12 `int STAR_API_CC TRIGGER_CFG_setTriggerAndMask ( const STAR_DEVICE_ID deviceld, const U32 trigger,  
const TRIGGER_TYPE type, const U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.13** `int STAR_API_CC TRIGGER_CFG_setTriggerInputEvents ( const STAR_DEVICE_ID deviceld, const U32 causeTrigger, const U32 trigger, const TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.14** `int STAR_API_CC TRIGGER_CFG_setTriggerInputMode ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.



**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.15** `int STAR_API_CC TRIGGER_CFG_setTriggerInvertMask ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, const U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.16** `int STAR_API_CC TRIGGER_CFG_setTriggerSourceEventsAnded ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 anded )`

Set whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.

By default, OR logic is applied.

**Parameters**

|                 |                                                                                                     |
|-----------------|-----------------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                                             |
| <i>trigger</i>  | The internal trigger number.                                                                        |
| <i>type</i>     | The source type.                                                                                    |
| <i>source</i>   | The source number.                                                                                  |
| <i>anded</i>    | The required AND state. A value of zero requests that OR logic should be used, otherwise AND logic. |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

**8.35.2.17** `int STAR_API_CC TRIGGER_CFG_setTriggerSourceEventsInverted ( const STAR_DEVICE_ID deviceld, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 inverted )`

Set the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.

By default, the signal is not inverted.

**Parameters**

|                 |                                                                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                                                                                   |
| <i>trigger</i>  | The internal trigger number.                                                                                                              |
| <i>type</i>     | The source type.                                                                                                                          |
| <i>source</i>   | The source number.                                                                                                                        |
| <i>inverted</i> | The required invert state. A value of zero requests that the signal should not be inverted, otherwise that the signal should be inverted. |

**Note**

TRIGGER\_TYPE\_SPFI\_VC is currently not supported for the type parameter

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, STAR-Ultra PCIe Interface

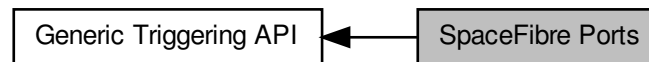
Examples:

[triggering\\_generic\\_example.c](#).

## 8.36 SpaceFibre Ports

Functions in this group are used to configure triggering functionality for SpaceFibre Ports.

Collaboration diagram for SpaceFibre Ports:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setSpFiPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, const SPFI_PORT_EVENT_MASK events)`  
*Sets the input events for a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, SPFI_PORT_EVENT_MASK *const pEvents)`  
*Gets the input events from a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, const SPFI_PORT_ACTION_MASK actions)`  
*Sets the output actions for a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, SPFI_PORT_ACTION_MASK *const pActions)`  
*Gets the output actions from a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfSpFiPorts (const STAR_DEVICE_ID deviceId, U32 *const pNumSpFiPorts)`  
*Gets the number of SpaceFibre ports on the specified device.*

### 8.36.1 Detailed Description

Functions in this group are used to configure triggering functionality for SpaceFibre Ports.

### 8.36.2 Function Documentation

**8.36.2.1** `int STAR_API_CC TRIGGER_CFG_getNumberOfSpFiPorts ( const STAR_DEVICE_ID deviceId, U32 *const pNumSpFiPorts )`

Gets the number of SpaceFibre ports on the specified device.

#### Parameters

|                      |                                                                     |
|----------------------|---------------------------------------------------------------------|
| <i>deviceId</i>      | Identifier of the device.                                           |
| <i>pNumSpFiPorts</i> | Pointer to value to be updated with the number of SpaceFibre ports. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.36.2.2** `int STAR_API_CC TRIGGER_CFG_getSpFiPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, SPFI_PORT_EVENT_MASK *const pEvents )`

Gets the input events from a SpaceFibre port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceFibre port.                         |
| <i>spfiPort</i> | SpaceFibre port to get input events from.                      |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.36.2.3** `int STAR_API_CC TRIGGER_CFG_getSpFiPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, SPFI_PORT_ACTION_MASK *const pActions )`

Gets the output actions from a SpaceFibre port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceFibre port.                          |
| <i>spfiPort</i> | SpaceFibre port to get output actions from.                     |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface (version 1.07 edit 5 and greater)

8.36.2.4 `int STAR_API_CC TRIGGER_CFG_setSpFiPortInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, const SPFI_PORT_EVENT_MASK events )`

Sets the input events for a SpaceFibre port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceFibre port.                  |
| <i>spfiPort</i> | SpaceFibre port to set input events for.                |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.36.2.5** `int STAR_API_CC TRIGGER_CFG_setSpFiPortOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 trigger, const SPFI_PORT_ACTION_MASK actions )`

Sets the output actions for a SpaceFibre port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the SpaceFibre port.            |
| <i>spfiPort</i> | SpaceFibre port to set output actions for.        |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface (version 1.07 edit 5 and greater)

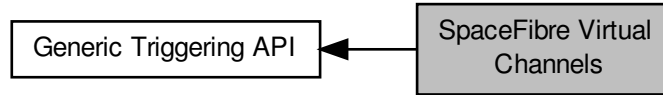
**Examples:**

`triggering_generic_example.c`.

## 8.37 SpaceFibre Virtual Channels

Functions in this group are used to configure triggering functionality for SpaceFibre Virtual Channels.

Collaboration diagram for SpaceFibre Virtual Channels:



### Functions

- `int STAR_API_CC TRIGGER_CFG_setSpFiVCInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_EVENT_MASK events)`  
*Sets the input events for a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiVCInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_EVENT_MASK *const pEvents)`  
*Gets the input events from a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiVCOOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_ACTION_MASK actions)`  
*Sets the output actions for a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiVCOOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_ACTION_MASK *const pActions)`  
*Gets the output actions from a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableSpFiVCTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`  
*Enable transmit packet mode on a virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpFiVCTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`  
*Disable transmit packet mode on a virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiVCTransmitPacketModeEnabled (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)`  
*Gets whether packet transmit mode is enabled for a specific virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_enableSpFiVCTransmitDataMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`  
*Enable transmit data mode on a virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpFiVCTransmitDataMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`  
*Disable transmit data mode on a virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiVCTransmitDataModeEnabled (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)`  
*Gets whether transmit data mode is enabled for a specific virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_enableSpFiVCReceiveDataMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`  
*Enable receive data mode on a virtual channel of a SpaceFibre port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpFiVCReceiveDataMode (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel)`



*Disable receive data mode on a virtual channel of a SpaceFibre port.*

- `int STAR_API_CC TRIGGER_CFG_getSpFIVCReceiveDataModeEnabled (const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled)`

*Gets whether receive data mode is enabled for a specific virtual channel of a SpaceFibre port.*

- `int STAR_API_CC TRIGGER_CFG_getNumberOfSpFIVCs (const STAR_DEVICE_ID deviceld, U32 *const pNumSpFIVCs)`

*Gets the number of virtual channels on each SpaceFibre port on the specified device.*

### 8.37.1 Detailed Description

Functions in this group are used to configure triggering functionality for SpaceFibre Virtual Channels.

### 8.37.2 Function Documentation

#### 8.37.2.1 `int STAR_API_CC TRIGGER_CFG_disableSpFIVCReceiveDataMode ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel )`

Disable receive data mode on a virtual channel of a SpaceFibre port.

When disabled, the virtual channel does not wait for the RECEIVE\_DATA action to be set before receiving data.

##### Parameters

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

##### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

##### Added in version:

STAR-System v6.01 - 13th November 2025

##### Supported devices:

STAR-Ultra PCIe Interface

#### 8.37.2.2 `int STAR_API_CC TRIGGER_CFG_disableSpFIVCTransmitDataMode ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel )`

Disable transmit data mode on a virtual channel of a SpaceFibre port.

When disabled, the virtual channel does not wait for the TRANSMIT\_DATA action to be set before data transmission.

##### Parameters

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

##### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

##### Added in version:

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

8.37.2.3 `int STAR_API_CC_TRIGGER_CFG_disableSpFIVCTransmitPacketMode ( const STAR_DEVICE_ID deviceld,  
const U32 spfiPort, const U32 virtualChannel )`

Disable transmit packet mode on a virtual channel of a SpaceFibre port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

8.37.2.4 `int STAR_API_CC_TRIGGER_CFG_enableSpFIVCReceiveDataMode ( const STAR_DEVICE_ID deviceld, const  
U32 spfiPort, const U32 virtualChannel )`

Enable receive data mode on a virtual channel of a SpaceFibre port.

When enabled, the virtual channel will receive data while the RECEIVE\_DATA action is set.

**Parameters**

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

8.37.2.5 `int STAR_API_CC_TRIGGER_CFG_enableSpFIVCTransmitDataMode ( const STAR_DEVICE_ID deviceld, const  
U32 spfiPort, const U32 virtualChannel )`

Enable transmit data mode on a virtual channel of a SpaceFibre port.

When enabled, the virtual channel will transmit data while the TRANSMIT\_DATA action is set.

**Parameters**

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.37.2.6** `int STAR_API_CC TRIGGER_CFG_enableSpFiVCTransmitPacketMode ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel )`

Enable transmit packet mode on a virtual channel of a SpaceFibre port.

When enabled, the virtual channel will not transmit a packet until it receives a transmit packet action.

**Parameters**

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c`.

**8.37.2.7** `int STAR_API_CC TRIGGER_CFG_getNumberOfSpFiVCs ( const STAR_DEVICE_ID deviceld, U32 *const pNumSpFiVCs )`

Gets the number of virtual channels on each SpaceFibre port on the specified device.

**Parameters**

|                 |                           |
|-----------------|---------------------------|
| <i>deviceld</i> | Identifier of the device. |
|-----------------|---------------------------|

---

|                    |                                                                     |
|--------------------|---------------------------------------------------------------------|
| <i>pNumSpFiVCs</i> | Pointer to value to be updated with the number of virtual channels. |
|--------------------|---------------------------------------------------------------------|

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**Examples:**

[triggering\\_generic\\_example.c](#).

**8.37.2.8** `int STAR_API_CC TRIGGER_CFG_getSpFiVCInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_EVENT_MASK *const pEvents )`

Gets the input events from a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.

**Parameters**

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port.                         |
| <i>spfiPort</i>       | The SpaceFibre port number.                                    |
| <i>virtualChannel</i> | The virtual channel number.                                    |
| <i>trigger</i>        | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>        | Pointer to a value which will be updated with the events mask. |

---

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.37.2.9** `int STAR_API_CC TRIGGER_CFG_getSpFiVCOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_ACTION_MASK *const pActions )`

Gets the output actions from a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.

**Parameters**

|                       |                                        |
|-----------------------|----------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port. |
| <i>spfiPort</i>       | The SpaceFibre port number.            |
| <i>virtualChannel</i> | The virtual channel number.            |

---

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.37.2.10** `int STAR_API_CC_TRIGGER_CFG_getSpFiVCReceiveDataModeEnabled ( const STAR_DEVICE_ID deviceld,  
const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled )`

Gets whether receive data mode is enabled for a specific virtual channel of a SpaceFibre port.

**Parameters**

|                       |                                                                                        |
|-----------------------|----------------------------------------------------------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port.                                                 |
| <i>spfiPort</i>       | The SpaceFibre port number.                                                            |
| <i>virtualChannel</i> | The virtual channel number.                                                            |
| <i>pEnabled</i>       | User supplied value that will be updated to 1 if receive data mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.37.2.11** `int STAR_API_CC_TRIGGER_CFG_getSpFiVCTransmitDataModeEnabled ( const STAR_DEVICE_ID deviceld,  
const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled )`

Gets whether transmit data mode is enabled for a specific virtual channel of a SpaceFibre port.

**Parameters**

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port.                                                  |
| <i>spfiPort</i>       | The SpaceFibre port number.                                                             |
| <i>virtualChannel</i> | The virtual channel number.                                                             |
| <i>pEnabled</i>       | User supplied value that will be updated to 1 if transmit data mode is enabled, else 0. |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**8.37.2.12** `int STAR_API_CC TRIGGER_CFG_getSpFiVCTransmitPacketModeEnabled ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, U32 *const pEnabled )`

Gets whether packet transmit mode is enabled for a specific virtual channel of a SpaceFibre port.

#### Parameters

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port.                                                    |
| <i>spfiPort</i>       | The SpaceFibre port number.                                                               |
| <i>virtualChannel</i> | The virtual channel number.                                                               |
| <i>pEnabled</i>       | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

STAR-Ultra PCIe Interface

**8.37.2.13** `int STAR_API_CC TRIGGER_CFG_setSpFiVCInputEvents ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_EVENT_MASK events )`

Sets the input events for a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.

#### Parameters

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| <i>deviceld</i>       | Device containing the SpaceFibre port.                  |
| <i>spfiPort</i>       | The SpaceFibre port number.                             |
| <i>virtualChannel</i> | The virtual channel number.                             |
| <i>trigger</i>        | Internal trigger which is affected by the input events. |
| <i>events</i>         | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

#### Added in version:

STAR-System v6.01 - 13th November 2025

#### Supported devices:

STAR-Ultra PCIe Interface

**8.37.2.14** `int STAR_API_CC TRIGGER_CFG_setSpFiVCOOutputActions ( const STAR_DEVICE_ID deviceld, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_ACTION_MASK actions )`

Sets the output actions for a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.

**Parameters**

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| <i>deviceId</i>       | Device containing the SpaceFibre port.            |
| <i>spfiPort</i>       | The SpaceFibre port number.                       |
| <i>virtualChannel</i> | The virtual channel number.                       |
| <i>trigger</i>        | Internal trigger which causes the output actions. |
| <i>actions</i>        | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, 0 if the operation failed, or a negative error code as defined in [Defines](#).

**Added in version:**

STAR-System v6.01 - 13th November 2025

**Supported devices:**

STAR-Ultra PCIe Interface

**Examples:**

`triggering_generic_example.c`.

## 8.38 Defines

Function return error values are defined here.

Collaboration diagram for Defines:



### Macros

- `#define TRIGGER_CFG_SUCCESS (1)`  
*Operation succeeded.*
- `#define TRIGGER_CFG_FAILURE (0)`  
*Operation failed.*
- `#define TRIGGER_CFG_STATUS_GET_DEVICE_INFO_FAILURE (-1)`  
*Failed to get "Device Information" for this device.*
- `#define TRIGGER_CFG_STATUS_NOT_SUPPORTED_BY_DEVICE (-2)`  
*Functionality not supported by this device.*

### 8.38.1 Detailed Description

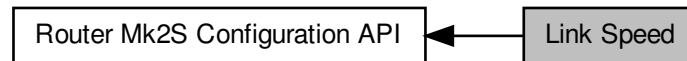
Function return error values are defined here.



## 8.39 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



### Functions

- `int STAR_API_CC_CFG_ROUTER_MK2S_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Enables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled on a Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceID)`  
*Disables precision transmit rate on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceID, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use on a SpaceWire Router Mk2S device.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_getPrecisionTransmitRate (STAR_DEVICE_ID deviceID, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*
- `int STAR_API_CC_CFG_ROUTER_MK2S_setPrecisionTransmitRate (STAR_DEVICE_ID deviceID, double transmitRate)`  
*Set the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*

### 8.39.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

### 8.39.2 Function Documentation

#### 8.39.2.1 `int STAR_API_CC_CFG_ROUTER_MK2S_disablePrecisionTransmitRate ( STAR_DEVICE_ID deviceID )`

Disables precision transmit rate on a SpaceWire Router Mk2S device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_ROUTER_MK2S_setPrecisionTransmitRate()`. Note that after being disabled there may be a short delay (less than 250 microseconds) before precision transmit rate is actually disabled. Call `CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse` to determine if precision transmit rate is no longer in use.

**Parameters**


---

|                 |                                               |
|-----------------|-----------------------------------------------|
| <i>deviceID</i> | Device to disable precision transmit rate on. |
|-----------------|-----------------------------------------------|

---

**Returns**

1 if precision transmit rate was successfully disabled, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

### 8.39.2.2 int STAR\_API\_CC\_CFG\_ROUTER\_MK2S\_enablePrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID* )

Enables precision transmit rate on a [SpaceWire Router Mk2S](#) device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_ROUTER\\_MK2S\\_setPrecisionTransmitRate\(\)](#). Note that after being enabled it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_ROUTER\\_MK2S\\_getPrecisionTransmitRateInUse](#) to determine if precision transmit rate is actually in use.

**Parameters**


---

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | Device to enable precision transmit rate on. |
|-----------------|----------------------------------------------|

---

**Returns**

1 if precision transmit rate was successfully enabled, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

### 8.39.2.3 int STAR\_API\_CC\_CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ double \* *pTransmitRate* )

Get the current precision transmit rate on a [SpaceWire Router Mk2S](#) device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_ROUTER\\_MK2S\\_setPrecisionTransmitRate\(\)](#).

**Parameters**


---

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <i>deviceID</i>      | Device to get the precision transmit rate.                                                 |
| <i>pTransmitRate</i> | User supplied value that will be updated to contain the precision transmit rate in Mbit/s. |

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

8.39.2.4 `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateEnabled ( STAR_DEVICE_ID deviceId,  
_Out_ int * pEnabled )`

Determine if precision transmit rate is enabled on a Router Mk2S device.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_ROUTER_MK2S_setPrecisionTransmitRate()`. Note that although precision transmit rate may be enabled, it may not yet be in use. It can take up to 250 microseconds before precision transmit rate is actually used. Call `CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse` to determine if precision transmit rate is actually in use.

**Parameters**

|                 |                                                                                              |
|-----------------|----------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether precision transmit rate is enabled.                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if precision transmit rate is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

8.39.2.5 `int STAR_API_CC CFG_ROUTER_MK2S_getPrecisionTransmitRateInUse ( STAR_DEVICE_ID deviceId, _Out_  
int * pInUse )`

Determine whether precision transmit rate is in use on a SpaceWire Router Mk2S device.

After being enabled or disabled, it can take up to 250 microseconds before precision transmit is actually used or not used. This function determines whether precision transmit rate is currently in use and can be used to determine if an enable or disable operation has completed.

**Parameters**

|                 |                                                                                             |
|-----------------|---------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether precision transmit rate is in use.                                    |
| <i>pInUse</i>   | User supplied value that will be updated to 1 if precision transmit rate is in use, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

**Supported devices:**

SpaceWire Router Mk2S

8.39.2.6 `int STAR_API_CC_CFG_ROUTER_MK2S_setPrecisionTransmitRate ( STAR_DEVICE_ID deviceId, double transmitRate )`

Set the current precision transmit rate on a [SpaceWire Router Mk2S](#) device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number.

#### Parameters

|                     |                                                |
|---------------------|------------------------------------------------|
| <i>deviceId</i>     | Device to set the precision transmit rate for. |
| <i>transmitRate</i> | the precision transmit rate in Mbit/s to set.  |

#### Returns

1 if the rate for the device was successfully set, else 0.

#### Added in version:

[STAR-System v2.0 Beta 1](#) - 8th February 2013

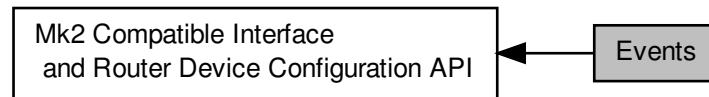
#### Supported devices:

[SpaceWire Router Mk2S](#)

## 8.40 Events

Functions in this group deal with enabling and disabling link events.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC CFG_MK2_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC CFG_MK2_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables state change events on a given port.*
- `int STAR_API_CC CFG_MK2_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables speed change events on a given port.*
- `int STAR_API_CC CFG_MK2_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether speed change events are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables speed change events on a given port.*
- `int STAR_API_CC CFG_MK2_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC CFG_MK2_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether time-code notifications are enabled on a specific port.*
- `int STAR_API_CC CFG_MK2_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceID, U8 portNum)`  
*Selectively disables time-code notifications on a the channel attached to a given port.*

#### 8.40.1 Detailed Description

Functions in this group deal with enabling and disabling link events.

## 8.40.2 Function Documentation

### 8.40.2.1 `int STAR_API_CC_CFG_MK2_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) link speed events are received on the channel associated with the port rather than channel 0.

#### Parameters

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable speed change events for a port on. |
| <i>portNum</i>  | Port to disable speed change events on.              |

#### Returns

1 if speed change events were successfully disabled, else 0.

#### Added in version:

[STAR-System v3.0 - 26th August 2016](#)

#### Supported devices:

[SpaceWire PCIe](#) (version 1.10 edit 4 and greater), [SpaceWire PCIe Mk2](#), [SpaceWire Physical Layer Tester](#) (version 2.00 and greater)

### 8.40.2.2 `int STAR_API_CC_CFG_MK2_disableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

#### Note

This function is currently only compatible with [SpaceWire PCIe](#) devices from hardware version v1.10 edit 4, and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

On the [SpaceWire PCIe](#) state events are received on the channel associated with the port rather than channel 0.

#### Parameters

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable state change events for a port on. |
| <i>portNum</i>  | Port to disable state change events on.              |

**Returns**

1 if state change events were successfully disabled, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester (version 2.00 and greater)

**8.40.2.3 int STAR\_API\_CC\_CFG\_MK2\_disableTimeCodeEventsOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )**

Selectively disables time-code notifications on a the channel attached to a given port.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4, with No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Device to disable time-code notifications for a port on. |
| <i>portNum</i>  | Port to disable time-code notifications on.              |

**Returns**

1 if time-code notifications were successfully disabled, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2

**8.40.2.4 int STAR\_API\_CC\_CFG\_MK2\_enableSpeedChangeEventsOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )**

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4, and SpaceWire Physical Layer Tester devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

On the SpaceWire PCIe link speed events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable speed change events for a port on. |
| <i>portNum</i>  | Port to enable speed change events on.              |

**Returns**

1 if speed change events were successfully enabled, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester (version 2.00 and greater)

**8.40.2.5 int STAR\_API\_CC\_CFG\_MK2\_enableStateChangeEventsOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )**

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4, and SpaceWire Physical Layer Tester devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

On the SpaceWire PCIe state events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable state change events for a port on. |
| <i>portNum</i>  | Port to enable state change events on.              |

**Returns**

1 if state change events were successfully enabled, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester (version 2.00 and greater)

**8.40.2.6 int STAR\_API\_CC\_CFG\_MK2\_enableTimeCodeEventsOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )**

Selectively enables time-code notifications on a the channel attached to a given port.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4. No check is made by this function to ensure it is being used with a compatible device.



**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Device to enable time-code notifications for a port on. |
| <i>portNum</i>  | Port to enable time-code notifications on.              |

**Returns**

1 if time-codes were successfully enabled, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2

**8.40.2.7** `int STAR_API_CC_CFG_MK2_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4, and SpaceWire Physical Layer Tester devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to get whether speed change events are enabled for a port.                         |
| <i>portNum</i>  | Port to get information for.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if speed change events are enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester (version 2.00 and greater)

**8.40.2.8** `int STAR_API_CC_CFG_MK2_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4, and SpaceWire Physical Layer Tester devices from hardware version v2.00. No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether state change events are enabled for a port.                         |
| <i>portNum</i>  | Port to get information for.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if state change events are enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester (version 2.00 and greater)

8.40.2.9 `int STAR_API_CC_CFG_MK2_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

**Note**

This function is currently only compatible with SpaceWire PCIe devices from hardware version v1.10 edit 4. No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | Device to get whether time-code notifications are enabled for a port.                                 |
| <i>portNum</i>  | Port to get information for.                                                                          |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

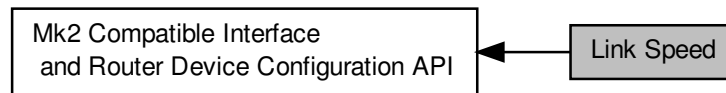
**Supported devices:**

SpaceWire PCIe (version 1.10 edit 4 and greater), SpaceWire PCIe Mk2

## 8.41 Link Speed

Functions in this group deal with setting the link speeds on the device.

Collaboration diagram for Link Speed:



### Data Structures

- struct [STAR\\_CFG\\_MK2\\_BASE\\_TRANSMIT\\_CLOCK](#)

Base transmit clock rate control properties. [More...](#)

### Functions

- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, U8 divider)  
Sets the Link rate divider for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getLinkRateDivider](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U8 \*pDivider)  
Gets the Link rate divider for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
Sets the base transmit clock rate on a given link:
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getBaseTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
Gets the base transmit clock rate parameters for a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_setTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
Sets the transmit clock rate on a given link.
- int [STAR\\_API\\_CC\\_CFG\\_MK2\\_getTransmitClock](#) (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
Gets the transmit clock rate parameters for a given link.

#### 8.41.1 Detailed Description

Functions in this group deal with setting the link speeds on the device.

#### 8.41.2 Data Structure Documentation

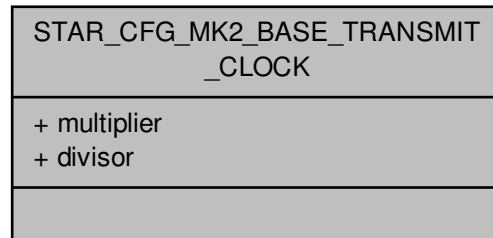
##### 8.41.2.1 struct STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK

Base transmit clock rate control properties.

Added in version:

STAR-System v1.2 - 13th February 2012

Collaboration diagram for STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK:



#### Data Fields

|     |            |                                                |
|-----|------------|------------------------------------------------|
| U16 | divisor    | Divisor value. Valid range 1:256 inclusive.    |
| U16 | multiplier | Multiplier value. Valid range 2:256 inclusive. |

### 8.41.3 Function Documentation

8.41.3.1 `int STAR_API_CC CFG_MK2_getBaseTransmitClock ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ STAR_CFG_MK2_BASE_TRANSMIT_CLOCK * pClockRateParams )`

Gets the base transmit clock rate parameters for a given link.

#### Note

This function is currently for use with [SpaceWire PCIe](#) devices before hardwareversion v1.11 and [SpaceWire Physical Layer Tester](#) devices before hardware version v2.00.  
For later versions use [CFG\\_MK2\\_getTransmitClock\(\)](#).

The [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) do not support this function. For [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices, please use [CFG\\_PCIMK2\\_getLinkClockFrequency\(\)](#) instead. For [SpaceWire Brick Mk2](#) and [SpaceWire Router Mk2S](#) devices, please use [CFG\\_BRICK\\_MK2\\_getLinkClockFrequency\(\)](#). For [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices, please use [CFG\\_BRICK\\_MK3\\_getBaseTransmitClock\(\)](#) or [CFG\\_BRICK\\_MK3\\_getTransmitClock\(\)](#) depending on hardware version.

No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceId</i> | Device to get the base transmit clock rate from. |
| <i>linkNum</i>  | Link to get base transmit clock rate for.        |

|     |                               |                                                                                                  |
|-----|-------------------------------|--------------------------------------------------------------------------------------------------|
| out | <i>pClockRate↔<br/>Params</i> | Pointer to value that will be updated with the given link's Base transmit clock rate parameters. |
|-----|-------------------------------|--------------------------------------------------------------------------------------------------|

**Returns**

1 if Base transmit clock frequency parameters were successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCIe (prior to version 1.11), SpaceWire Physical Layer Tester (prior to version 2.00)

**8.41.3.2** int **STAR\_API\_CC\_CFG\_MK2\_getLinkRateDivider** ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *linkNum*, **\_Out\_ U8 \*** *pDivider* )

Gets the Link rate divider for a given link.

**Parameters**

|     |                 |                                      |
|-----|-----------------|--------------------------------------|
|     | <i>deviceId</i> | Device to get a links divider from.. |
|     | <i>linkNum</i>  | Link to get divider for.             |
| out | <i>pDivider</i> | Value of the divider.                |

**Returns**

1 if was successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe (prior to version 1.11), SpaceWire Router Mk2S, SpaceWire Physical Layer Tester (prior to version 2.00)

**8.41.3.3** int **STAR\_API\_CC\_CFG\_MK2\_getTransmitClock** ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *linkNum*, **\_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*** *pClockRateParams* )

Gets the transmit clock rate parameters for a given link.

**Note**

This function is currently for use with SpaceWire PCIe devices from hardware version v1.11 and SpaceWire Physical Layer Tester devices from hardware version v2.00. For earlier versions use **CFG\_MK2\_getBase↔TransmitClock()**.

No check is made by this function to ensure it is being used with a compatible device.

**Parameters**

|     |                               |                                                                                             |
|-----|-------------------------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>               | Device to get the transmit clock rate from.                                                 |
|     | <i>linkNum</i>                | Link to get the transmit clock rate for.                                                    |
| out | <i>pClockRate↔<br/>Params</i> | Pointer to value that will be updated with the given link's transmit clock rate parameters. |

#### Returns

1 if transmit clock frequency parameters were successfully obtained, else 0.

#### Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

#### Supported devices:

SpaceWire PCIe (version 1.11 and greater), SpaceWire Physical Layer Tester (version 2.00 and greater)

**8.41.3.4** `int STAR_API_CC_CFG_MK2_setBaseTransmitClock ( STAR_DEVICE_ID deviceID, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the base transmit clock rate on a given link:

$$BaseClockRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR}$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate must be between 5 MHz and 200 MHz.

The transmit rate of the link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

The *LinkClockRateDivider* can be set using the `CFG_MK2_setLinkRateDivider()` function.

#### Note

This function is currently for use with SpaceWire PCIe devices before hardware version v1.11 and SpaceWire Physical Layer Tester devices before hardware version v2.0.  
For later versions use `CFG_MK2_setTransmitClock()`.

The SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire Brick Mk2, SpaceWire Router Mk2S and Space↔Wire Brick Mk3 and Brick Mk4 do not support this function. For SpaceWire PCI Mk2 and cPCI Mk2 devices, please use `CFG_PCIMK2_setLinkClockFrequency()` instead. For SpaceWire Brick Mk2 and SpaceWire Router Mk2S devices, please use `CFG_BRICK_MK2_setLinkClockFrequency()`. For SpaceWire Brick Mk3 and Brick Mk4 devices, please use `CFG_BRICK_MK3_setBaseTransmitClock()` or `CFG_BRICK_MK3_set↔TransmitClock()` depending on hardware version.

No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|  |                 |                                                        |
|--|-----------------|--------------------------------------------------------|
|  | <i>deviceID</i> | Device to modify a link's base transmit clock rate on. |
|--|-----------------|--------------------------------------------------------|

|                   |                                                     |
|-------------------|-----------------------------------------------------|
| <i>linkNum</i>    | Link to have its base transmit clock rate modified. |
| <i>clockRate↔</i> | Base transmit clock rate parameters to be set.      |
| <i>Params</i>     |                                                     |

**Returns**

1 if link's Base transmit clock frequency was successfully set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCIe (prior to version 1.11), SpaceWire Physical Layer Tester (prior to version 2.00)

### 8.41.3.5 int STAR\_API\_CC\_CFG\_MK2\_setLinkRateDivider ( STAR\_DEVICE\_ID *deviceId*, U8 *linkNum*, U8 *divider* )

Sets the Link rate divider for a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{BaseClockRate}{LinkClockRateDivider} \times 2$$

Where *BaseClockRate* is set by the appropriate function (e.g. `CFG_MK2_setBaseTransmitClock()` for a Space↔Wire PCIe device) and *LinkClockRateDivider* = divider.

**Note**

It is never necessary to set a link rate divider higher than 100, as the minimum transmit rate permitted by SpaceWire is 2 Mbit/s (i.e. 200 Mbit/s / 100).

If an odd number (other than 1) is specified as the divider parameter, the previous even integer will be used instead. For example: if a divider value of 3 is specified, the actual divider set shall be 2.

Link rate divider is limited to 64 on PCIe hardware versions prior to v1.11. We recommend upgrading to the latest firmware. If using the latest firmware then it is necessary to use `CFG_MK2_setTransmitClock()` rather than `CFG_MK2_setBaseTransmitClock()`. The `CFG_MK2_setLinkRateDivider()` function does not apply to PCIe hardware v1.11 or later and SPLT hardware v2.00 or later.

**Parameters**

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>deviceId</i> | Device to modify a links divider on.                                  |
| <i>linkNum</i>  | Link to have its divider modified.                                    |
| <i>divider</i>  | Value of the new divider. Valid input: 1, Even numbers 2 through 126. |

**Returns**

1 if divider was successfully set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe (prior to version 1.11), SpaceWire Router Mk2S, SpaceWire Physical Layer Tester (prior to version 2.00)

8.41.3.6 `int STAR_API_CC_CFG_MK2_setTransmitClock ( STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_MK2_BASE_TRANSMIT_CLOCK clockRateParams )`

Sets the transmit clock rate on a given link.

The transmit rate of a link is given by:

$$LinkTransmitRate = \frac{100\text{MHz} \times MULTIPLIER}{DIVISOR} \times 2$$

The *MULTIPLIER* and *DIVISOR* are properties of the `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK` structure, passed as a parameter. The output clock rate will be between 1 MHz and 200 MHz giving data rates between 2 Mbit/s and 400 Mbit/s.

Note that for data rates below 10 Mbit/s, an exact rate may not be achievable and it may be rounded up or down slightly.

#### Note

This function is currently for use with [SpaceWire PCIe](#) devices from hardware version v1.11 and [SpaceWire Physical Layer Tester](#) devices from hardware version v2.00. For earlier versions use `CFG_MK2_setBase↔TransmitClock()`.

No check is made by this function to ensure it is being used with a compatible device.

#### Parameters

|                         |                                                   |
|-------------------------|---------------------------------------------------|
| <i>deviceId</i>         | Device to modify a link's transmit clock rate on. |
| <i>linkNum</i>          | Link to have its transmit clock rate modified.    |
| <i>clockRate↔Params</i> | Transmit clock rate parameters to be set.         |

#### Returns

1 if link's transmit clock frequency was successfully set, else 0.

#### Added in version:

STAR-System v3.0 Beta 7 - 8th March 2016

#### Supported devices:

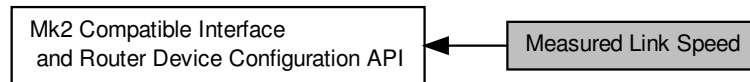
[SpaceWire PCIe](#) (version 1.11 and greater), [SpaceWire Physical Layer Tester](#) (version 2.00 and greater)



## 8.42 Measured Link Speed

Functions in this group deal with reading the measured link speed.

Collaboration diagram for Measured Link Speed:



### Macros

- `#define STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS 100`  
*The units in Kbit/s used to represent the link speed returned when calling `CFG_MK2_getMeasuredLinkSpeed()`.*

### Functions

- `int STAR_API_CC CFG_MK2_getMeasuredLinkSpeed (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U16 *pLinkSpeed)`  
*Gets the current link speed measured on a specific port.*

#### 8.42.1 Detailed Description

Functions in this group deal with reading the measured link speed.

#### 8.42.2 Macro Definition Documentation

##### 8.42.2.1 `#define STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS 100`

The units in Kbit/s used to represent the link speed returned when calling `CFG_MK2_getMeasuredLinkSpeed()`.

Added in version:

STAR-System v3.0 Beta 8 - 6th May 2016

#### 8.42.3 Function Documentation

##### 8.42.3.1 `int STAR_API_CC CFG_MK2_getMeasuredLinkSpeed ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U16 *pLinkSpeed )`

Gets the current link speed measured on a specific port.

##### Parameters

---

|                   |                                                                                                                                                           |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i>   | Device to get the measured link speed for a port.                                                                                                         |
| <i>portNum</i>    | Port to get information for.                                                                                                                              |
| <i>pLinkSpeed</i> | User supplied value that will be updated to contain the measured link speed in 100 Kbit/s units, see <a href="#">STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS</a> . |

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.0 Beta 8 - 6th May 2016

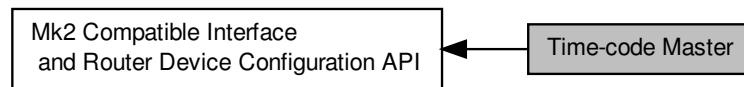
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

## 8.43 Time-code Master

Functions in this group deal with the management of the device as a time-code master.

Collaboration diagram for Time-code Master:



### Functions

- `int STAR_API_CC_CFG_MK2_enableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Enables the device as a time-code master.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeMasterEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether the device has been enabled as a time-code master.*
- `int STAR_API_CC_CFG_MK2_disableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Disables the device as a time-code master.*
- `int STAR_API_CC_CFG_MK2_enableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Enables external time-code selection for the device.*
- `int STAR_API_CC_CFG_MK2_getExternalTimeCodeSelectionEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether external time-code selection has been enabled for the device.*
- `int STAR_API_CC_CFG_MK2_disableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Disables external time-code selection for the device.*
- `int STAR_API_CC_CFG_MK2_setTimeCodePeriod (STAR_DEVICE_ID deviceId, U32 period)`  
*Sets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_MK2_getTimeCodePeriod (STAR_DEVICE_ID deviceId, _Out_ U32 *pPeriod)`  
*Gets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_MK2_enableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Enables time-code counter bypass mode for the device.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeCounterBypassModeEnabled (STAR_DEVICE_ID deviceId, int *pEnabled)`  
*Gets whether time-code counter bypass mode has been enabled for the device.*
- `int STAR_API_CC_CFG_MK2_disableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Disables time-code counter bypass mode for the device.*

### 8.43.1 Detailed Description

Functions in this group deal with the management of the device as a time-code master.

## 8.43.2 Function Documentation

### 8.43.2.1 `int STAR_API_CC_CFG_MK2_disableExternalTimeCodeSelection ( STAR_DEVICE_ID deviceID )`

Disables external time-code selection for the device.

When external time-code selection is disabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device.

#### Note

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

#### Parameters

---

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable external time-code selection for. |
|-----------------|------------------------------------------------------------------------|

---

#### Returns

1 if external time-code selection was successfully disabled for the device, else 0.

#### Added in version:

[STAR-System v2.0 Beta 3 - 12th April 2013](#)

#### Supported devices:

[SpaceWire Router Mk2S](#)

### 8.43.2.2 `int STAR_API_CC_CFG_MK2_disableTimeCodeCounterBypassMode ( STAR_DEVICE_ID deviceID )`

Disables time-code counter bypass mode for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded.

#### Note

This function is currently only supported by [SpaceWire PCIe](#) devices with at least hardware version v1.12.

#### Parameters

---

|                 |                                                                         |
|-----------------|-------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable time-code counter bypass mode for. |
|-----------------|-------------------------------------------------------------------------|

---

#### Returns

1 if time-code counter bypass mode was successfully disabled for the device, else 0.

#### Added in version:

[STAR-System v3.1 - 21st October 2016](#)

#### Supported devices:

[SpaceWire PCIe](#) (version 1.12 and greater), [SpaceWire PCIe Mk2](#)

### 8.43.2.3 `int STAR_API_CC_CFG_MK2_disableTimeCodeMaster ( STAR_DEVICE_ID deviceID )`

Disables the device as a time-code master.

**Parameters**

---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to disable as a time-code master |
|-----------------|------------------------------------------------------------|

---

**Returns**

1 if the device was successfully unset as a time code master, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.43.2.4 int STAR\_API\_CC\_CFG\_MK2\_enableExternalTimeCodeSelection ( STAR\_DEVICE\_ID deviceID )**

Enables external time-code selection for the device.

When external time-code selection is enabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Note**

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

**Parameters**

---

|                 |                                                                       |
|-----------------|-----------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to enable external time-code selection for. |
|-----------------|-----------------------------------------------------------------------|

---

**Returns**

1 if external time-code selection was successfully enabled for the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 3 - 12th April 2013

**Supported devices:**

[SpaceWire Router Mk2S](#)

**8.43.2.5 int STAR\_API\_CC\_CFG\_MK2\_enableTimeCodeCounterBypassMode ( STAR\_DEVICE\_ID deviceID )**

Enables time-code counter bypass mode for the device.

When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices with at least hardware version v1.12.

**Parameters**


---

|                 |                                                                        |
|-----------------|------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to enable time-code counter bypass mode for. |
|-----------------|------------------------------------------------------------------------|

---

**Returns**

1 if time-code counter bypass mode was successfully enabled for the device, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe (version 1.12 and greater), SpaceWire PCIe Mk2

#### 8.43.2.6 int STAR\_API\_CC\_CFG\_MK2\_enableTimeCodeMaster ( STAR\_DEVICE\_ID *deviceID* )

Enables the device as a time-code master.

**Parameters**


---

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to enable as a time-code master. |
|-----------------|------------------------------------------------------------|

---

**Returns**

1 if the device was successfully set as a time code master, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.43.2.7 int STAR\_API\_CC\_CFG\_MK2\_getExternalTimeCodeSelectionEnabled ( STAR\_DEVICE\_ID *deviceID*, int \* *pEnabled* )

Gets whether external time-code selection has been enabled for the device.

When external time-code selection is disabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device. When external time-code selection is enabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Note**

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

**Parameters**

|                 |                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to check.                                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if external time-code selection is enabled for the device, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v2.0 Beta 3 - 12th April 2013

**Supported devices:**

SpaceWire Router Mk2S

**8.43.2.8** `int STAR_API_CC_CFG_MK2_getTimeCodeCounterBypassModeEnabled ( STAR_DEVICE_ID deviceId, int * pEnabled )`

Gets whether time-code counter bypass mode has been enabled for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded. When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices with at least hardware version v1.12.

**Parameters**

|                 |                                                                                                                   |
|-----------------|-------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i> | The Mk2 compatible device to check.                                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code counter bypass mode is enabled for the device, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

[SpaceWire PCIe](#) (version 1.12 and greater), [SpaceWire PCIe Mk2](#)

**8.43.2.9** `int STAR_API_CC_CFG_MK2_getTimeCodeMasterEnabled ( STAR_DEVICE_ID deviceId, int * pEnabled )`

Gets whether the device has been enabled as a time-code master.

**Parameters**

|                 |                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to check.                                                                   |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the device is enabled as a time-code master, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.43.2.10 int STAR\_API\_CC\_CFG\_MK2\_getTimeCodePeriod ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U32 \* *pPeriod* )

Gets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|     |                 |                                                                                    |
|-----|-----------------|------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get time-code period from                                                |
| out | <i>pPeriod</i>  | A pointer to a variable that will be updated to contain the period in microseconds |

**Returns**

1 if the time-code period was successfully obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.43.2.11 int STAR\_API\_CC\_CFG\_MK2\_setTimeCodePeriod ( STAR\_DEVICE\_ID *deviceID*, U32 *period* )

Sets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**



---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to set time-code period for   |
| <i>period</i>   | The period to be set in microseconds |

---

**Returns**

1 if the time-code period was successfully set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

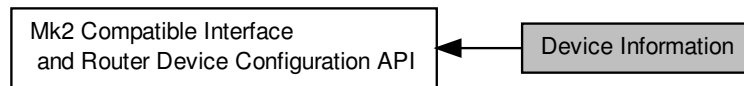
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

## 8.44 Device Information

Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.

Collaboration diagram for Device Information:



### Data Structures

- struct `STAR_CFG_MK2_HARDWARE_INFO`

Hardware version, and synthesis date of the FPGA code. [More...](#)

### Functions

- int `STAR_API_CC_CFG_MK2_getHardwareInfo` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_MK2\_HARDWARE\_INFO \*pInfo)

Reads information about the hardware version of a device and updates the `STAR_CFG_MK2_HARDWARE_INFO` structure pointed to by the passed in pointer to contain the received version information.

- void `STAR_API_CC_CFG_MK2_hardwareInfoToString` (STAR\_CFG\_MK2\_HARDWARE\_INFO info, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)

Creates a string representation of a `STAR_CFG_MK2_HARDWARE_INFO` structure.

- int `STAR_API_CC_CFG_MK2_identify` (STAR\_DEVICE\_ID deviceId)

Flashes the front panel LEDs of a device.

#### 8.44.1 Detailed Description

Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.

#### 8.44.2 Data Structure Documentation

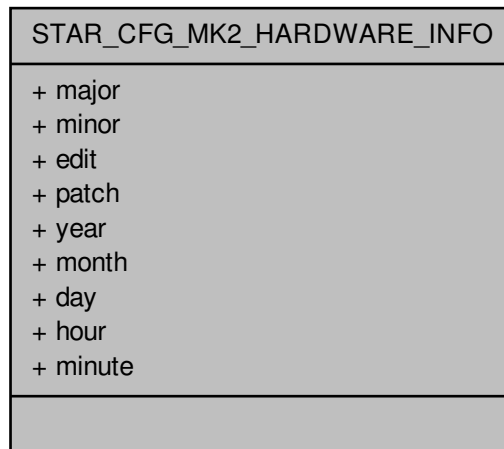
##### 8.44.2.1 struct STAR\_CFG\_MK2\_HARDWARE\_INFO

Hardware version, and synthesis date of the FPGA code.

Added in version:

STAR-System v1.2 - 13th February 2012

Collaboration diagram for STAR\_CFG\_MK2\_HARDWARE\_INFO:



#### Data Fields

|     |        |                                |
|-----|--------|--------------------------------|
| U8  | day    | Day value of time and date.    |
| U8  | edit   | Version number edit.           |
| U8  | hour   | Hour value of time and date.   |
| U8  | major  | Major version number.          |
| U8  | minor  | Minor version number.          |
| U8  | minute | Minute value of time and date. |
| U8  | month  | Month value of time and date.  |
| U8  | patch  | Version number patch.          |
| U16 | year   | Year value of time and date.   |

### 8.44.3 Function Documentation

8.44.3.1 `int STAR_API_CC CFG_MK2_getHardwareInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_MK2_HARDWARE_INFO * pInfo )`

Reads information about the hardware version of a device and updates the `STAR_CFG_MK2_HARDWARE_INFO` structure pointed to by the passed in pointer to contain the received version information.

#### Parameters

|     |                 |                                                                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Identifier for the Mk2 compatible device to get hardware info from.                                                                 |
| out | <i>pInfo</i>    | Pointer to a <code>STAR_CFG_MK2_HARDWARE_INFO</code> structure that will be updated to contain hardware information for the device. |

#### Returns

1 if the hardware version was successfully read.

Added in version:

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.44.3.2** void **STAR\_API\_CC** CFG\_MK2\_hardwareInfoToString ( **STAR\_CFG\_MK2\_HARDWARE\_INFO** info, \_Out\_z\_cap\_c\_(**STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN**) char \* *pVersion*, \_Out\_z\_cap\_c\_(**STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN**) char \* *pBuildDate* )

Creates a string representation of a **STAR\_CFG\_MK2\_HARDWARE\_INFO** structure.

**Parameters**

|     |                   |                                                                                                                                                                       |
|-----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>info</i>       | Hardware information obtained from a call to <a href="#">CFG_MK2_getHardwareInfo()</a> .                                                                              |
| out | <i>pVersion</i>   | String of length <a href="#">STAR_CFG_MK2_VERSION_STR_MAX_LEN</a> that will be updated to contain a null-terminated string representation of the hardware version.    |
| out | <i>pBuildDate</i> | String of length <a href="#">STAR_CFG_MK2_VERSION_STR_MAX_LEN</a> that will be updated to contain a null-terminated string representation of the hardware build date. |

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.44.3.3** int **STAR\_API\_CC** CFG\_MK2\_identify ( **STAR\_DEVICE\_ID** *deviceID* )

Flashes the front panel LEDs of a device.

This can be used to identify the physical device to which a device identifier refers to.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its LEDs flashed. |
|-----------------|----------------------------------|

**Returns**

1 if the device successfully identified itself, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

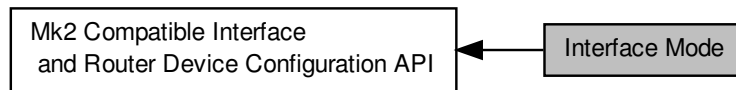
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

## 8.45 Interface Mode

Functions in this group deal managing the device in Interface mode.

Collaboration diagram for Interface Mode:



### Functions

- int `STAR_API_CC_CFG_MK2_enableInterfaceMode` (STAR\_DEVICE\_ID deviceId)
 

*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_MK2_setPortRoutingAddress()`).*
- int `STAR_API_CC_CFG_MK2_getInterfaceModeEnabled` (STAR\_DEVICE\_ID deviceId, int \*pEnabled)
 

*Gets whether interface mode has been enabled on a device.*
- int `STAR_API_CC_CFG_MK2_enableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)
 

*Selectively enables interface mode on a given port.*
- int `STAR_API_CC_CFG_MK2_getInterfaceModeOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, int \*pEnabled)
 

*Gets whether interface mode is enabled on a specific port.*
- int `STAR_API_CC_CFG_MK2_disableInterfaceMode` (STAR\_DEVICE\_ID deviceId)
 

*Disables interface mode on a device.*
- int `STAR_API_CC_CFG_MK2_disableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)
 

*Selectively disables interface mode on a given port.*
- int `STAR_API_CC_CFG_MK2_enableIdentifySource` (STAR\_DEVICE\_ID deviceId)
 

*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- int `STAR_API_CC_CFG_MK2_getIdentifySourceEnabled` (STAR\_DEVICE\_ID deviceId, int \*pEnabled)
 

*Gets whether identify source is enabled.*
- int `STAR_API_CC_CFG_MK2_disableIdentifySource` (STAR\_DEVICE\_ID deviceId)
 

*Disables identification of source ports for interface mode globally.*
- int `STAR_API_CC_CFG_MK2_enableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)
 

*Enables identification of source port during interface mode for a given port.*
- int `STAR_API_CC_CFG_MK2_getIdentifySourceOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, int \*pEnabled)
 

*Gets whether identify source is enabled for a specific port.*
- int `STAR_API_CC_CFG_MK2_disableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)
 

*Disables source port identification for a given port.*
- int `STAR_API_CC_CFG_MK2_setPortRoutingAddress` (STAR\_DEVICE\_ID deviceId, U8 portNum, U8 address)
 

*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int `STAR_API_CC_CFG_MK2_getPortRoutingAddress` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U8 \*pAddress)
 

*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*

### 8.45.1 Detailed Description

Functions in this group deal managing the device in Interface mode.

### 8.45.2 Function Documentation

#### 8.45.2.1 `int STAR_API_CC_CFG_MK2_disableIdentifySource ( STAR_DEVICE_ID deviceID )`

Disables identification of source ports for interface mode globally.

##### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification on. |
|-----------------|--------------------------------------------------|

##### Returns

1 if source identification was successfully disabled globally, else 0.

##### Added in version:

STAR-System v1.2 - 13th February 2012

##### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.2 `int STAR_API_CC_CFG_MK2_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

##### Parameters

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification for a given port on. |
| <i>portNum</i>  | Port to disable source identification on.                         |

##### Returns

1 if source identification was successfully disabled, else 0.

##### Added in version:

STAR-System v1.2 - 13th February 2012

##### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.3 `int STAR_API_CC_CFG_MK2_disableInterfaceMode ( STAR_DEVICE_ID deviceID )`

Disables interface mode on a device.

When interface mode is disabled, the device operates in routing mode.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to disable interface mode on. |
|-----------------|--------------------------------------|

**Returns**

1 if interface mode was successfully disabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.4 int STAR\_API\_CC\_CFG\_MK2\_disableInterfaceModeOnPort ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum* )

Selectively disables interface mode on a given port.

When interface mode is disabled, the device operates in routing mode.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to disable interface mode for a port on. |
| <i>portNum</i>  | Port to disable interface mode on.              |

**Returns**

1 if interface mode was successfully disabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.5 int STAR\_API\_CC\_CFG\_MK2\_enableIdentifySource ( STAR\_DEVICE\_ID *deviceID* )

When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.

For each source port on which this behaviour is desired, a call to [CFG\\_MK2\\_enableIdentifySourceOnPort\(\)](#) must be made.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification on. |
|-----------------|-------------------------------------------------|

**Returns**

1 if source identification was successfully enabled globally, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.6 `int STAR_API_CC_CFG_MK2_enableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Enables identification of source port during interface mode for a given port.

Interface mode (`CFG_MK2_enableInterfaceMode()`) and Identify Source (`CFG_MK2_enableIdentifySource()`) must be enabled globally for this to have an effect.

**Parameters**

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification for a given port on. |
| <i>portNum</i>  | Port to enable source identification on.                         |

**Returns**

1 if source identification was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.7 `int STAR_API_CC_CFG_MK2_enableInterfaceMode ( STAR_DEVICE_ID deviceID )`

In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_MK2_setPortRoutingAddress()`).

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceID</i> | Device to enable interface mode on. |
|-----------------|-------------------------------------|

**Returns**

1 if interface mode was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.8 `int STAR_API_CC_CFG_MK2_enableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables interface mode on a given port.

Interface mode must be enabled globally (`CFG_MK2_enableInterfaceMode()`) for interface mode on a port to be enabled.



**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceID</i> | Device to enable interface mode for a port on. |
| <i>portNum</i>  | Port to enable interface mode on.              |

**Returns**

1 if interface mode was successfully enabled, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.9 `int STAR_API_CC_CFG_MK2_getIdentifySourceEnabled ( STAR_DEVICE_ID deviceID, int * pEnabled )`

Gets whether identify source is enabled.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to check.                                                                     |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.10 `int STAR_API_CC_CFG_MK2_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, int * pEnabled )`

Gets whether identify source is enabled for a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pEnabled is always 0.

**Parameters**

|                 |                  |
|-----------------|------------------|
| <i>deviceID</i> | Device to check. |
|-----------------|------------------|

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>portNum</i>  | Port to check.                                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.11 int STAR\_API\_CC\_CFG\_MK2\_getInterfaceModeEnabled ( STAR\_DEVICE\_ID *deviceID*, int \* *pEnabled* )

Gets whether interface mode has been enabled on a device.

**Parameters**

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <i>deviceID</i> | The Mk2 compatible device to check.                                                 |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

**Returns**

1 if the information could be obtained from the device, else 0

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.45.2.12 int STAR\_API\_CC\_CFG\_MK2\_getInterfaceModeOnPortEnabled ( STAR\_DEVICE\_ID *deviceID*, U8 *portNum*, int \* *pEnabled* )

Gets whether interface mode is enabled on a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in *pEnabled* is always 0.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to get interface mode enabled for a port on. |
| <i>portNum</i>  | Port to get information for.                        |

---

|                 |                                                                                     |
|-----------------|-------------------------------------------------------------------------------------|
| <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |
|-----------------|-------------------------------------------------------------------------------------|

---

**Returns**

1 if the information could be obtained from the device, else 0.

**Added in version:**

STAR-System v1.5 - 23rd April 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

**8.45.2.13** `int STAR_API_CC_CFG_MK2_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pAddress is always 0.

**Parameters**

|            |                 |                                                               |
|------------|-----------------|---------------------------------------------------------------|
|            | <i>deviceID</i> | Device to get port routing address from.                      |
|            | <i>portNum</i>  | Port to get port routing address from.                        |
| <i>out</i> | <i>pAddress</i> | Pointer to value that will be updated to contain the address. |

**Returns**

1 if the address could be obtained, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

**8.45.2.14** `int STAR_API_CC_CFG_MK2_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

**Note**

This is an advanced feature and not necessary for normal usage.

**Parameters**

---

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| <i>deviceID</i> | Device to have one of its ports port routing address changed. |
| <i>portNum</i>  | Port to have its port routing address modified.               |
| <i>address</i>  | New port routing address.                                     |

---

**Returns**

1 if the address could be set, else 0.

**Added in version:**

STAR-System v1.2 - 13th February 2012

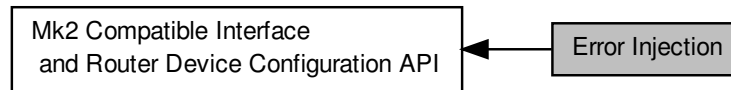
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

## 8.46 Error Injection

Functions in this group deal with injecting errors onto SpaceWire links.

Collaboration diagram for Error Injection:



### Enumerations

- enum `SPW_ERROR` {  
`SPW_ERROR_DISCONNECT` = 1, `SPW_ERROR_PARITY` = 2, `SPW_ERROR_ESCAPE` = 3, `SPW_ERROR_INSERT_FCT` = 4,  
`SPW_ERROR_SUPPRESS_FCT` = 5, `SPW_ERROR_INCREMENT_CREDIT` = 6, `SPW_ERROR_DECREMENT_CREDIT` = 7 }

*Errors that can be injected onto the SpaceWire link.*

### Functions

- int `STAR_API_CC_CFG_MK2_injectError` (`STAR_DEVICE_ID` deviceId, unsigned char port, `SPW_ERROR` error)

*Immediately injects the specified error on a given device's port.*

#### 8.46.1 Detailed Description

Functions in this group deal with injecting errors onto SpaceWire links.

#### 8.46.2 Enumeration Type Documentation

##### 8.46.2.1 enum `SPW_ERROR`

Errors that can be injected onto the SpaceWire link.

Added in version:

STAR-System v2.0 - 3rd July 2013

#### Enumerator

**`SPW_ERROR_DISCONNECT`** Causes the SpaceWire link to disconnect. The link enters ErrorReset and the Data/Strobe outputs transition LOW.

**`SPW_ERROR_PARITY`** Inserts a parity error on the next SpaceWire character to be transmitted by flipping the parity bit.

**`SPW_ERROR_ESCAPE`** Transmits two escape characters in sequence.

**`SPW_ERROR_INSERT_FCT`** Sends an FCT character. This can be used to force FCT errors on the SpaceWire link.

***SPW\_ERROR\_SUPPRESS\_FCT*** Discards the next FCT character to be transmitted on the link. This can be used to simulate the loss of an FCT character.

***SPW\_ERROR\_INCREMENT\_CREDIT*** Increments the transmit credit available to send data by 8. This can be used to force a data overflow in the opposite end of the link, as too much data will be transmitted.

***SPW\_ERROR\_DECREMENT\_CREDIT*** Decrements the transmit credit count by 1. This can be used to reduce the rate at which data can be transmitted.

### 8.46.3 Function Documentation

8.46.3.1 `int STAR_API_CC_CFG_MK2_injectError ( STAR_DEVICE_ID deviceID, unsigned char port, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire Physical Layer Tester](#) and [SpaceWire PCIe](#) devices. No check is made by this function to ensure it is being used with a compatible device. For [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices, please use [CFG\\_BRICK\\_MK2\\_injectErrors\(\)](#).

Error injection makes use of the same on-board registers as periodic actions on devices which support it, therefore error injection cannot be used on those devices when periodic actions are in progress.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have the error injected on. |
| <i>port</i>     | Port the error is to be injected on.                    |
| <i>error</i>    | SpaceWire error to be injected.                         |

#### Returns

1 if the error information was successfully sent to the device, else 0.

#### Added in version:

STAR-System v2.0 - 3rd July 2013

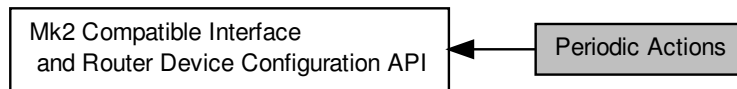
#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire Physical Layer Tester](#)

## 8.47 Periodic Actions

Functions in this group deal with controlling periodic actions on SpaceWire links.

Collaboration diagram for Periodic Actions:



### Enumerations

- enum `SPW_ACTION` { `SPW_TRANSMIT_PACKET` = 0 }  
*Actions which can be preformed at specified periodic intervals.*

### Functions

- int `STAR_API_CC_CFG_MK2_startPeriodicAction` (`STAR_DEVICE_ID` deviceId, unsigned char port, U32 count, `SPW_ACTION` action)  
*Starts a periodic action on a given device's port.*
- int `STAR_API_CC_CFG_MK2_stopPeriodicAction` (`STAR_DEVICE_ID` deviceId, unsigned char port)  
*Stops a periodic action on a given device's port.*
- U32 `STAR_API_CC_CFG_MK2_getDeviceClockRate` (`STAR_DEVICE_ID` deviceId)  
*Gets a device's internal clock rate.*

#### 8.47.1 Detailed Description

Functions in this group deal with controlling periodic actions on SpaceWire links.

#### 8.47.2 Enumeration Type Documentation

##### 8.47.2.1 enum `SPW_ACTION`

Actions which can be preformed at specified periodic intervals.

Added in version:

STAR-System v3.0 Beta 3 - 4th September 2015

Enumerator

**`SPW_TRANSMIT_PACKET`** Transmits a SpaceWire packet.

#### 8.47.3 Function Documentation

##### 8.47.3.1 U32 `STAR_API_CC_CFG_MK2_getDeviceClockRate` ( `STAR_DEVICE_ID` deviceId )

Gets a device's internal clock rate.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceID</i> | Identifier of the device to query. |
|-----------------|------------------------------------|

**Returns**

The device's clock rate in Hz, or zero if the device does not support periodic actions.

**Added in version:**

STAR-System v3.0 Beta 3 - 4th September 2015

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**8.47.3.2** `int STAR_API_CC_CFG_MK2_startPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action )`

Starts a periodic action on a given device's port.

To stop a periodic action use `CFG_MK2_stopPeriodicAction()`.

**Note**

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.

Periodic actions make use of the same on-board registers as error injection, therefore error injection cannot be used when periodic actions are in progress.

**Parameters**

|                 |                                                                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have periodic action created for.                                                                                     |
| <i>port</i>     | Port the periodic action is to be performed on.                                                                                                   |
| <i>count</i>    | Number of device clock cycles between action repetitions. See <a href="#">CFG_MK2_getDeviceClockRate()</a> for obtaining the device's clock rate. |
| <i>action</i>   | Action to be performed. This can be <code>SPW_TRANSMIT_PACKET</code> or any of the <code>SPW_ERROR</code> values.                                 |

**Returns**

1 if the periodic action was successfully started, else 0.

**Added in version:**

STAR-System v3.0 Beta 3 - 4th September 2015

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**8.47.3.3** `int STAR_API_CC_CFG_MK2_stopPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port )`

Stops a periodic action on a given device's port.

Periodic actions are started by calling `CFG_MK2_startPeriodicAction()`.

**Note**

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices. No check is made by this function to ensure it is being used with a compatible device.



**Parameters**

---

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device performing the periodic action. |
| <i>port</i>     | Port the periodic action is being performed on.          |

---

**Returns**

1 if the periodic action was successfully stopped, else 0.

**Added in version:**

STAR-System v3.0 Beta 3 - 4th September 2015

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

## 8.48 PCI Mk2 Packet Subsystem

Functions in this group deal with setting the packet subsystem parameters for PCI Mk2 Devices.

### Data Structures

- struct `PKT_SUBSYS_STATISTICS`

*Statistics that can be obtained from the PCI Mk2 device packet sink subsystem. [More...](#)*

### Typedefs

- typedef `void(STAR_API_CC * PKT_SUBSYS_StatisticsFunc) (STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_STATISTICS statistics)`

*The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.*

- typedef `U32 STATISTICS_LOOP_ID`

*Type of identifier used to identify a statistics polling thread.*

### Enumerations

- enum `PKT_SUBSYS_PATTERN` {  
`PKT_SUBSYS_PATTERN_ALL_ZEROES, PKT_SUBSYS_PATTERN_ALL_ONES, PKT_SUBSYS_PATTERN_ALTERNATING_BITS, PKT_SUBSYS_PATTERN_INCREMENTING_BYTES, PKT_SUBSYS_PATTERN_DECREMENTING_BYTES, PKT_SUBSYS_PATTERN_WALKING_ONES, PKT_SUBSYS_PATTERN_WALKING_ZEROES` }

*Available patterns to use with `PKT_SUBSYS_fillBufferWithPattern()`*

### Functions

- void `STAR_API_CC PKT_SUBSYS_fillBufferWithPattern (PKT_SUBSYS_PATTERN pattern, U16 bufferSize, U16 *pBuffer)`

*Convenience function that fills a user provided buffer with a known pattern.*

- int `STAR_API_CC PKT_SUBSYS_writeMemory (STAR_DEVICE_ID deviceId, U16 address, U16 writeSize, U16 *pBuffer)`

*Writes a buffer to the PCI Mk2's dual port memory.*

- int `STAR_API_CC PKT_SUBSYS_readMemory (STAR_DEVICE_ID deviceId, U16 address, U16 readSize, U16 *pBuffer)`

*Reads from the PCI Mk2's dual port memory to a user allocated buffer.*

- int `STAR_API_CC PKT_SUBSYS_setPacketGeneratorFormat (STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop, U16 gap)`

*Sets up the format of the packet to be inserted by the packet generator into the router.*

- int `STAR_API_CC PKT_SUBSYS_setGeneratorBlockingParameters (STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)`

*Sets up the packet generation subsystem blocking parameters.*

- int `STAR_API_CC PKT_SUBSYS_startPacketGeneration (STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 sequenceLength)`

*Starts the packet generator inserting packets into the router.*

- int `STAR_API_CC PKT_SUBSYS_waitForPacketGenerationSequenceComplete (STAR_DEVICE_ID deviceId, unsigned int subsystem, unsigned int timeout)`

*Blocks the current thread until packet generation has stopped.*

- int `STAR_API_CC_PKT_SUBSYS_stopPacketGeneration` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet generator inserting packets into the router.*
- int `STAR_API_CC_PKT_SUBSYS_setPacketSinkParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)  
*Sets up the circular buffer for the packet sink subsystem.*
- int `STAR_API_CC_PKT_SUBSYS_setSinkBlockingParameters` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)  
*Sets up the packet sink subsystem blocking parameters.*
- int `STAR_API_CC_PKT_SUBSYS_startSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Starts the packet sink reading packets into memory.*
- int `STAR_API_CC_PKT_SUBSYS_stopSink` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet sink reading packets into memory.*
- int `STAR_API_CC_PKT_SUBSYS_getSinkDataPointers` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 \*startPointer, U16 \*endPointer)  
*Gets the address for the start and end of valid data in the circular buffer in device memory.*
- int `STAR_API_CC_PKT_SUBSYS_startPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)  
*Starts a packet checker running on a packet sink.*
- int `STAR_API_CC_PKT_SUBSYS_stopPacketChecking` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Stops the packet checker.*
- int `STAR_API_CC_PKT_SUBSYS_waitForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.*
- int `STAR_API_CC_PKT_SUBSYS_cancelWaitsForPacketCheckingError` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem)  
*Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.*
- int `STAR_API_CC_PKT_SUBSYS_setPacketCheckingFormat` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, `STAR_EOP_TYPE` eop)  
*Sets up the packet checker to check packets match the format of packets provided by user entered parameters.*
- int `STAR_API_CC_PKT_SUBSYS_getPacketCheckingMismatchCount` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, U64 \*errorCount)  
*Gets the number of mismatched data characters observed on the SpaceWire interface.*
- int `STAR_API_CC_PKT_SUBSYS_getStatistics` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_STATISTICS` \*pStatistics)  
*Gets statistics for a given subsystem.*
- `STATISTICS_LOOP_ID` `STAR_API_CC_PKT_SUBSYS_startGetStatisticsLoop` (`STAR_DEVICE_ID` deviceId, unsigned int subsystem, `PKT_SUBSYS_StatisticsFunc` statisticsHandler)  
*Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.*
- void `STAR_API_CC_PKT_SUBSYS_stopGetStatisticsLoop` (`STATISTICS_LOOP_ID` loopId)  
*Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.*

### 8.48.1 Detailed Description

Functions in this group deal with setting the packet subsystem parameters for PCI Mk2 Devices.

The PCI Mk2 device contains a single hardware packet generator and checker subsystem. This subsystem can be used to automatically generate and check SpaceWire packets at high speed without using the resources of the host PC.

On the device there is a dual-port memory that is used to store the format for packets to be generated and checked, and also used for the circular buffer used by the packet sink. Packets of any length can be generated with a separate header and cargo, which is specified by pointers to blocks of this shared memory. Additionally, it is possible to control the rate at which packets are sent. The user can set the number of clock cycles when no data is sent, (blocking) and also the number of clock cycles between packets (gap).

The packet sink can be used to read incoming packets directly to a user defined circular buffer in the dual-port memory. A packet checker can be run on each packets sink. The packet sink must be running on a subsystem in order for statistics to be gathered or packet checking to occur. Statistics are updated once per second.

The packet checker receives incoming packets and checks them against a template held in the dual-port memory. If a mismatched character is detected, then the packet checker stops the packet sink it is running on after a specified number of data characters are received. This allows the user to set up a circular buffer (packet sink) that can be read back to show what data preceded and followed the mismatched character, aiding debugging.

#### Attention

Currently the PCI Mk2 hardware supports only a single packet subsystem. Attempts to use subsystems 2 or 3 will result functions returning error. Note that subsystem 1 is connected to port 8 of the PCI Mk2 device's internal router.

## 8.48.2 Data Structure Documentation

### 8.48.2.1 struct PKT\_SUBSYS\_STATISTICS

Statistics that can be obtained from the PCI Mk2 device packet sink subsystem.

Statistics are updated once per second.

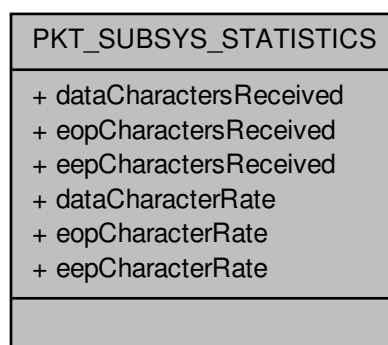
#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Examples:

`packet_subsystem_example.c`.

Collaboration diagram for PKT\_SUBSYS\_STATISTICS:



## Data Fields

|     |                                  |                                                                                                                |
|-----|----------------------------------|----------------------------------------------------------------------------------------------------------------|
| U32 | dataCharacter↔<br>Rate           | Number of data characters received in the last sampling period. The sampling period is one second in duration. |
| U64 | data↔<br>Characters↔<br>Received | Number of data characters received since the PCI Mk2 was last reset.                                           |
| U32 | eepCharacter↔<br>Rate            | Number of EEP characters received in the last sampling period. The sampling period is one second in duration.  |
| U64 | eepCharacters↔<br>Received       | Number of EEP characters received since the PCI Mk2 was last reset.                                            |
| U32 | eopCharacter↔<br>Rate            | Number of EOP characters received in the last sampling period. The sampling period is one second in duration.  |
| U64 | eopCharacters↔<br>Received       | Number of EOP characters received since the PCI Mk2 was last reset.                                            |

## 8.48.3 Typedef Documentation

8.48.3.1 `typedef void(STAR_API_CC * PKT_SUBSYS_StatisticsFunc) (STAR_DEVICE_ID deviceId, unsigned int subsystem, PKT_SUBSYS_STATISTICS statistics)`

The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.

## Parameters

|                   |                                                                       |
|-------------------|-----------------------------------------------------------------------|
| <i>deviceId</i>   | Identifier for the device statistics were obtained from.              |
| <i>subsystem</i>  | Subsystem number (indexed from 1) that statistics were obtained from. |
| <i>statistics</i> | Newly gathered statistics.                                            |

Added in version:

STAR-System v1.7 - 20th July 2012

8.48.3.2 `typedef U32 STATISTICS_LOOP_ID`

Type of identifier used to identify a statistics polling thread.

Used when manually stopping a statistics polling thread.

Added in version:

STAR-System v1.7 - 20th July 2012

## 8.48.4 Enumeration Type Documentation

8.48.4.1 `enum PKT_SUBSYS_PATTERN`

Available patterns to use with `PKT_SUBSYS_fillBufferWithPattern()`

Added in version:

STAR-System v1.7 - 20th July 2012

## 8.48.5 Function Documentation

8.48.5.1 `int STAR_API_CC PKT_SUBSYS_cancelWaitsForPacketCheckingError ( STAR_DEVICE_ID deviceld, unsigned int subsystem )`

Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet sink was stopped, or 0 if an error occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**8.48.5.2** void **STAR\_API\_CC** PKT\_SUBSYS\_fillBufferWithPattern ( PKT\_SUBSYS\_PATTERN *pattern*, U16 *bufferSize*, \_Out\_bytecap\_(bufferSize) void \* *pBuffer* )

Convenience function that fills a user provided buffer with a known pattern.

**Parameters**

|                   |                                        |
|-------------------|----------------------------------------|
| <i>pattern</i>    | Pattern to generate.                   |
| <i>bufferSize</i> | Size in bytes of user provided buffer. |
| <i>pBuffer</i>    | Pointer to user allocated buffer.      |

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.3** int **STAR\_API\_CC** PKT\_SUBSYS\_getPacketCheckingMismatchCount ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, U64 \* *errorCount* )

Gets the number of mismatched data characters observed on the SpaceWire interface.

**Parameters**

|            |                   |                                                            |
|------------|-------------------|------------------------------------------------------------|
|            | <i>deviceld</i>   | Identifier for the device to start the packet checking on. |
|            | <i>subsystem</i>  | Subsystem number to configure (indexed from 1)             |
| <i>out</i> | <i>errorCount</i> | Count of mismatched characters.                            |

**Returns**

1 if packet sink was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.4** int **STAR\_API\_CC** PKT\_SUBSYS\_getSinkDataPointers ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, U16 \* *startPointer*, U16 \* *endPointer* )

Gets the address for the start and end of valid data in the circular buffer in device memory.

Note that if the endPointer is less than the startPointer then the end pointer has wrapped round since the sink was last started.

**Parameters**

|     |                     |                                                                       |
|-----|---------------------|-----------------------------------------------------------------------|
|     | <i>deviceld</i>     | Identifier for the device to get the memory location for.             |
|     | <i>subsystem</i>    | Subsystem number to get information from (indexed from 1).            |
| out | <i>startPointer</i> | 32-bit word address of start of acquired data in the circular buffer. |
| out | <i>endPointer</i>   | 32-bit word address of end of acquired data in the circular buffer.   |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.5** int **STAR\_API\_CC** PKT\_SUBSYS\_getStatistics ( STAR\_DEVICE\_ID *deviceld*, unsigned int *subsystem*, PKT\_SUBSYS\_STATISTICS \* *pStatistics* )

Gets statistics for a given subsystem.

Note that a sink must be started for statistics to be updated.

**Parameters**

|     |                    |                                                                                                                                        |
|-----|--------------------|----------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceld</i>    | Identifier for the device to stop the packet sink on.                                                                                  |
|     | <i>subsystem</i>   | Subsystem number to configure (indexed from 1).                                                                                        |
| out | <i>pStatistics</i> | Pointer to a user allocated PKT_SUBSYS_STATISTICS structure. that will be updated with the current values of the statistics registers. |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2



**8.48.5.6** `int STAR_API_CC PKT_SUBSYS_readMemory ( STAR_DEVICE_ID deviceld, U16 address, U16 readSize,  
_Inout_bytecap_(readSize) void * pBuffer )`

Reads from the PCI Mk2's dual port memory to a user allocated buffer.

#### Note

The dual port memory is made up of 64K 32 bit words. Therefore the valid address range is 0 through 65535. Attempts to read from addresses beyond this range will fail.

#### Parameters

|                |                 |                                                                                                                                      |
|----------------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------|
|                | <i>deviceld</i> | Identifier for the device to read from.                                                                                              |
|                | <i>address</i>  | Word address to read from in the device's dual port memory. Address values 0 through 65536 are valid.                                |
|                | <i>readSize</i> | Size in bytes to read from the device memory into user provided buffer. This must be a multiple of 4 (only entire words may be read) |
| <i>in, out</i> | <i>pBuffer</i>  | Pointer to user allocated buffer.                                                                                                    |

#### Returns

0 if the memory was successfully read from to, else 1.

#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

#### Examples:

`packet_subsystem_example.c`.

**8.48.5.7** `int STAR_API_CC PKT_SUBSYS_setGeneratorBlockingParameters ( STAR_DEVICE_ID deviceld, unsigned int  
subsystem, U16 blockFrequency, U16 blockDuration )`

Sets up the packet generation subsystem blocking parameters.

When blocked, no packet data will be inserted into the router by the packet generator.

#### Parameters

|  |                       |                                                                                           |
|--|-----------------------|-------------------------------------------------------------------------------------------|
|  | <i>deviceld</i>       | Identifier for the device to set up the packet generator subsystem for.                   |
|  | <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                            |
|  | <i>blockFrequency</i> | Number of clock cycles between the end of a blocking operation and the start of the next. |
|  | <i>blockDuration</i>  | Number of clock cycles which blocking will occur over.                                    |

#### Returns

1 if packet generation was stopped.

#### Added in version:

STAR-System v1.7 - 20th July 2012

#### Supported devices:

SpaceWire PCI Mk2 and cPCI Mk2

8.48.5.8 `int STAR_API_CC PKT_SUBSYS_setPacketCheckingFormat ( STAR_DEVICE_ID deviceld, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop )`

Sets up the packet checker to check packets match the format of packets provided by user entered parameters.

**Parameters**

|                      |                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------|
| <i>deviceId</i>      | Identifier for the device to set up the packet checking subsystem for.                        |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                                |
| <i>headerPointer</i> | Address of the expected packet header in the device's memory.                                 |
| <i>headerLength</i>  | Length of the expected packet header in bytes.                                                |
| <i>bodyPointer</i>   | Starting address of the area in device memory to be used for the expected packet body values. |
| <i>bodyLength</i>    | Expected Length of the packet body in bytes.                                                  |
| <i>packetLength</i>  | Expected length of entire packet.                                                             |
| <i>eop</i>           | Type of end of packet identifier to be used. Valid values are EOP or EEP.                     |

**Returns**

1 if the packet format was successfully set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

**8.48.5.9** `int STAR_API_CC PKT_SUBSYS_setPacketGeneratorFormat ( STAR_DEVICE_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR_EOP_TYPE eop, U16 gap )`

Sets up the format of the packet to be inserted by the packet generator into the router.

**Parameters**

|                      |                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------|
| <i>deviceId</i>      | Identifier for the device to set up the packet generator subsystem for.                   |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                            |
| <i>headerPointer</i> | Starting word address of the packet header in the device's memory.                        |
| <i>headerLength</i>  | Length of the packet header in bytes.                                                     |
| <i>bodyPointer</i>   | Starting word address of the area in device memory to be used for the packet body values. |
| <i>bodyLength</i>    | Number of consecutive words in memory the packet generator will use to build packets.     |
| <i>packetLength</i>  | Desired length of entire packet.                                                          |
| <i>eop</i>           | Type of end of packet identifier to be used. Valid values are EOP or EEP.                 |
| <i>gap</i>           | Number of clock cycles between the end of one packet and the start of the next.           |

**Returns**

1 if the packet format was successfully set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

8.48.5.10 int **STAR\_API\_CC\_PKT\_SUBSYS\_setPacketSinkParameters** ( **STAR\_DEVICE\_ID** *deviceId*, unsigned int *subsystem*, **U16** *memoryPointer*, **U16** *memoryLength* )

Sets up the circular buffer for the packet sink subsystem.

**Parameters**

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <i>deviceld</i>      | Identifier for the device to set up the packet sink subsystem for.                         |
| <i>subsystem</i>     | Subsystem number to configure (indexed from 1)                                             |
| <i>memoryPointer</i> | Starting word address for the circular buffer used to record packets received by the sink. |
| <i>memoryLength</i>  | Length of the circular buffer in words.                                                    |

**Returns**

1 if the packet sink parameters were set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

#### 8.48.5.11 int **STAR\_API\_CC\_PKT\_SUBSYS\_setSinkBlockingParameters** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, U16 *blockFrequency*, U16 *blockDuration* )

Sets up the packet sink subsystem blocking parameters.

When blocked, no packet data will be read from router into packet sink memory.

**Parameters**

|                       |                                                                                           |
|-----------------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i>       | Identifier for the device to set up the packet generator subsystem for.                   |
| <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                            |
| <i>blockFrequency</i> | Number of clock cycles between the end of a blocking operation and the start of the next. |
| <i>blockDuration</i>  | Number of clock cycles which blocking will occur over.                                    |

**Returns**

1 if packet sink blocking parameters were set.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

#### 8.48.5.12 **STATISTICS\_LOOP\_ID** **STAR\_API\_CC\_PKT\_SUBSYS\_startGetStatisticsLoop** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, **PKT\_SUBSYS\_StatisticsFunc** *statisticsHandler* )

Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.

If an error occurs obtaining the statistics, the thread terminates. The thread will keep running until stopped with **PKT\_SUBSYS\_stopGetStatisticsLoop()** even if the packet sink is disabled (though no statistics are updated when the packet sink is disabled).

**Note**

If packet checking is enabled and a character mismatch occurs, the packet sink is disabled. This causes statistics to stop being gathered until the sink is restarted.

**Parameters**

|                          |                                                                               |
|--------------------------|-------------------------------------------------------------------------------|
| <i>deviceld</i>          | Identifier for the device to stop the packet sink on.                         |
| <i>subsystem</i>         | Subsystem number to configure (indexed from 1).                               |
| <i>statisticsHandler</i> | Pointer to function that will be called when updated statistics are obtained. |

**Returns**

Identifier for the loop that can be used to stop it.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.13** int **STAR\_API\_CC\_PKT\_SUBSYS\_startPacketChecking** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, int *disableSinkOnError*, **U16** *disableSinkdelay* )

Starts a packet checker running on a packet sink.

If a character mismatch is detected the packet sink is disabled after a specified number of data characters are received. Note that a packet sink must be running to enable checking of incoming packets.

**Parameters**

|                           |                                                                                                                   |
|---------------------------|-------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i>           | Identifier for the device to start the packet checking on.                                                        |
| <i>subsystem</i>          | Subsystem number to configure (indexed from 1).                                                                   |
| <i>disableSinkOnError</i> | Whether to disable the packet sink if a mismatched character is received (1) or not (0).                          |
| <i>disableSinkdelay</i>   | The number of bytes to be received after a character mismatch is detected and before the packet sink is disabled. |

**Returns**

1 if packet checking was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.14** int **STAR\_API\_CC\_PKT\_SUBSYS\_startPacketGeneration** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, **U16** *sequenceLength* )

Starts the packet generator inserting packets into the router.

**Parameters**

|                       |                                                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceId</i>       | Identifier for the device to set up the packet generator subsystem for.                                                               |
| <i>subsystem</i>      | Subsystem number to configure (indexed from 1)                                                                                        |
| <i>sequenceLength</i> | Number of packets that will be inserted before generation stops. Specify a sequence length of zero to enable continuous transmission. |

**Returns**

1 if packet generation was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

#### 8.48.5.15 `int STAR_API_CC PKT_SUBSYS_startSink ( STAR_DEVICE_ID deviceId, unsigned int subsystem )`

Starts the packet sink reading packets into memory.

**Parameters**

|                  |                                                        |
|------------------|--------------------------------------------------------|
| <i>deviceId</i>  | Identifier for the device to start the packet sink on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)         |

**Returns**

1 if packet sink was started.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

`packet_subsystem_example.c`.

#### 8.48.5.16 `void STAR_API_CC PKT_SUBSYS_stopGetStatisticsLoop ( STATISTICS_LOOP_ID loopId )`

Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.

**Parameters**

|               |                                  |
|---------------|----------------------------------|
| <i>loopId</i> | Identifier for the loop to stop. |
|---------------|----------------------------------|

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.17** int **STAR\_API\_CC\_PKT\_SUBSYS\_stopPacketChecking** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem* )

Stops the packet checker.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet checking was stopped

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.18** int **STAR\_API\_CC\_PKT\_SUBSYS\_stopPacketGeneration** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem* )

Stops the packet generator inserting packets into the router.

**Parameters**

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device to set up the packet generator subsystem for. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)                          |

**Returns**

1 if packet generation was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**8.48.5.19** int **STAR\_API\_CC\_PKT\_SUBSYS\_stopSink** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem* )

Stops the packet sink reading packets into memory.



**Parameters**

|                  |                                                       |
|------------------|-------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device to stop the packet sink on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1)        |

**Returns**

1 if packet sink was stopped.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

**8.48.5.20** int **STAR\_API\_CC\_PKT\_SUBSYS\_waitForPacketCheckingError** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem* )

Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.

**Parameters**

|                  |                                                          |
|------------------|----------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device packet checking is started on. |
| <i>subsystem</i> | Subsystem number to configure (indexed from 1).          |

**Returns**

1 if packet sink was stopped, or 0 if an error occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**8.48.5.21** int **STAR\_API\_CC\_PKT\_SUBSYS\_waitForPacketGenerationSequenceComplete** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned int *subsystem*, unsigned int *timeout* )

Blocks the current thread until packet generation has stopped.

**Parameters**

|                  |                                                                            |
|------------------|----------------------------------------------------------------------------|
| <i>deviceld</i>  | Identifier for the device with the packet generator subsystem to wait for. |
| <i>subsystem</i> | Subsystem number to wait on (indexed from 1)                               |
| <i>timeout</i>   | Timeout period in milliseconds. Provide a value of 0 to wait indefinitely. |

**Returns**

0 if error occurred communicating with device, 1 if packet generation completed or -1 if timeout occurred.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

**Examples:**

packet\_subsystem\_example.c.

8.48.5.22 `int STAR_API_CC_PKT_SUBSYS_writeMemory ( STAR_DEVICE_ID deviceId, U16 address, U16 writeSize,  
_In_bytecount_(writeSize) void * pBuffer )`

Writes a buffer to the PCI Mk2's dual port memory.

**Note**

The dual port memory is made up of 64K 32 bit words. Therefore the valid address range is 0 through 65535. Attempts to write to addresses beyond this range will fail.

**Parameters**

|    |                  |                                                                                                                                    |
|----|------------------|------------------------------------------------------------------------------------------------------------------------------------|
|    | <i>deviceId</i>  | Identifier for the device to write to.                                                                                             |
|    | <i>address</i>   | Word address to write to in the device's dual port memory. Address values 0 through 65536 are valid.                               |
|    | <i>writeSize</i> | Length in bytes of user provided buffer to write to device memory. This must be a multiple of 4 (only entire words may be written) |
| in | <i>pBuffer</i>   | Pointer to user allocated buffer.                                                                                                  |

**Returns**

0 if the memory was successfully written to, else 1.

**Added in version:**

STAR-System v1.7 - 20th July 2012

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2

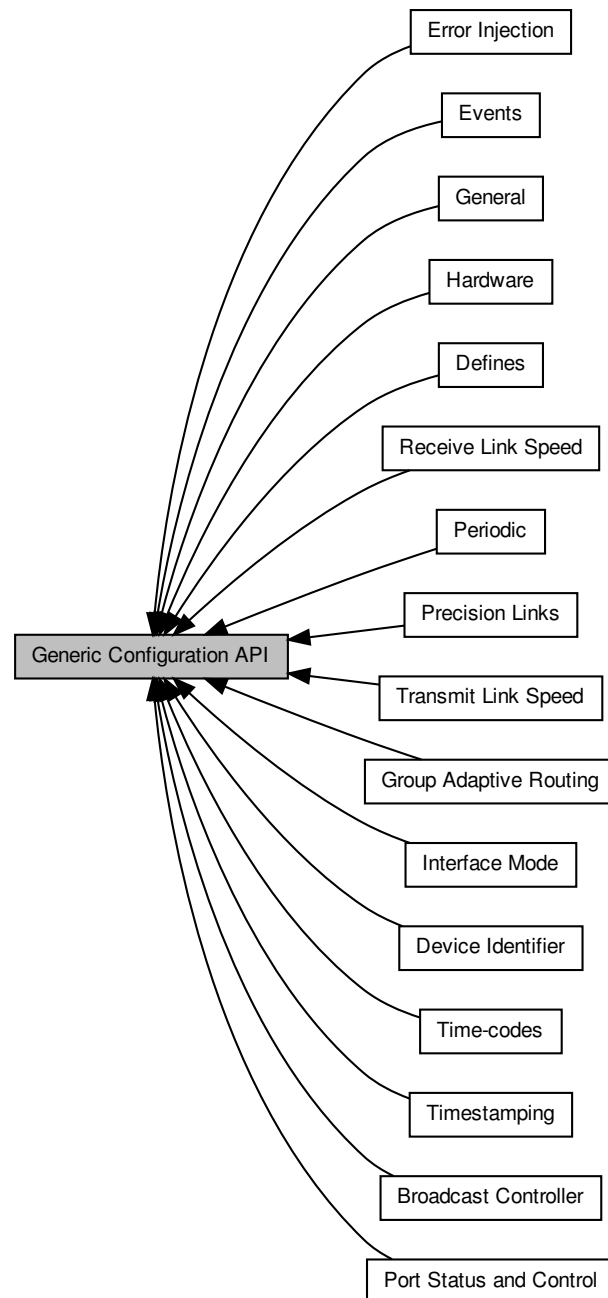
**Examples:**

packet\_subsystem\_example.c.

## 8.49 Generic Configuration API

The Generic Configuration API provides functions for configuring all features on all devices.

Collaboration diagram for Generic Configuration API:



### Modules

- General

*Functions in this group are general functions.*

- **Hardware**

*Functions in this group are hardware related functions.*

- **Port Status and Control**

*Functions in this group deal with port status and control.*

- **Device Identifier**

*Functions in this group deal with device identification.*

- **Group Adaptive Routing**

*Functions in this group deal with group adaptive routing.*

- **Interface Mode**

*Functions in this group deal with interface mode.*

- **Time-codes**

*Functions in this group deal with time-codes.*

- **Events**

*Functions in this group deal with events.*

- **Transmit Link Speed**

*Functions in this group deal with controlling transmit link speed.*

- **Receive Link Speed**

*Functions in this group deal with measuring receive link speed.*

- **Precision Links**

*Functions in this group deal with controlling link speed with greater precision.*

- **Error Injection**

*Functions in this group deal with error injection.*

- **Timestamping**

*Functions in this group deal with timestampings.*

- **Periodic**

*Functions in this group deal with periodic functions.*

- **Broadcast Controller**

*Functions in this group deal with broadcasts.*

- **Defines**

*Function return error values are defined here.*

### 8.49.1 Detailed Description

The Generic Configuration API provides functions for configuring all features on all devices.

If a feature is not supported by a device, the API function calls will return a value to indicate this. Please refer to file [cfg\\_api\\_generic.h](#) for the possible return values.

As well as configuring devices connected directly to the PC, the Generic Device Configuration API can also be used to configure devices over a SpaceWire network. This is achieved by creating a remote device identifier (see [Remote Devices](#)) and passing this remote device identifier as the identifier of the device to be configured.

The [Configuration API Functions Supported by Device Types](#) lists the functions supported by each device.

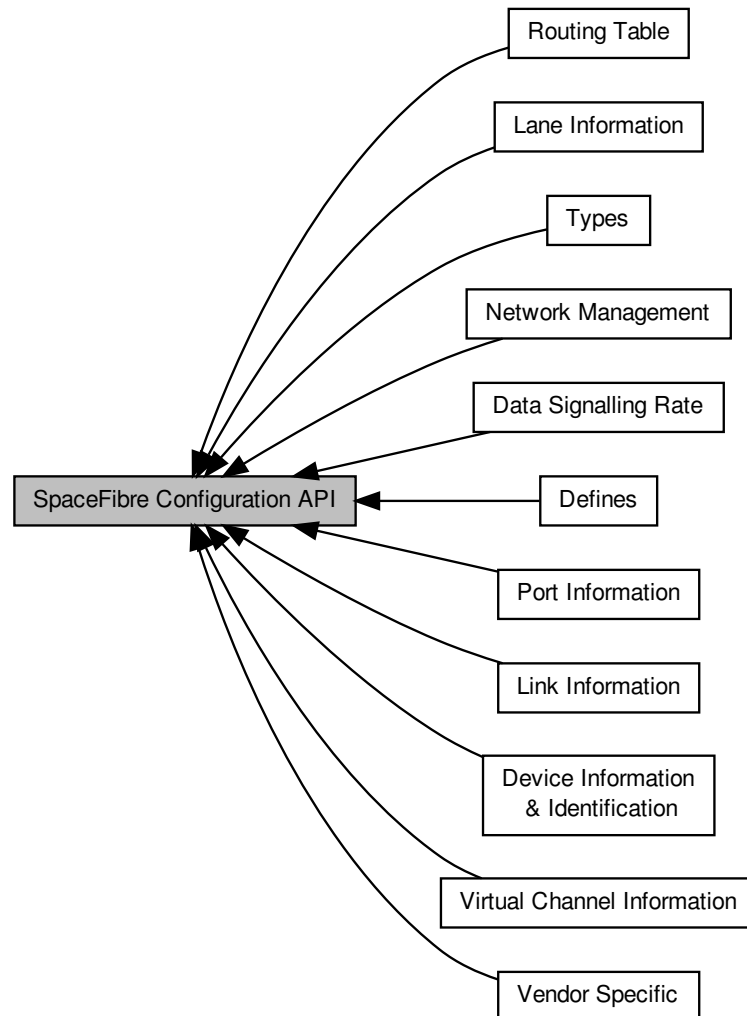
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.50 SpaceFibre Configuration API

The SpaceFibre Configuration API provides functions for configuring SpaceFibre devices.

Collaboration diagram for SpaceFibre Configuration API:



### Modules

- **Device Information & Identification**

*Functions in this group are used to access device information such as the type of node, number of ports, manufacturer identifier and device identifier.*

- **Routing Table**

*Functions in this group are used to access and configure routing table entries.*

- **Network Management**

*Functions in this group deal are used to access and configure network management fields such as the device identifier, broadcast timeout, port ready flags, port errors, links errors, current timeslot, time and router broadcast channel.*

- **Vendor Specific**

*Functions in this group are used to access and configure vendor specific functionality including router timeout and broadcast types.*

- **Virtual Channel Information**

*Functions in this group are used to access and configure virtual channel information such as the virtual network number, status fields, errors and quality of service parameters.*

- **Port Information**

*Functions in this group are used to access and configure port information such as the port type, number of virtual channels, ready status, broadcast error and virtual channel errors.*

- **Link Information**

*Functions in this group are used to access and configure link information such as the control fields, status and errors.*

- **Lane Information**

*Functions in this group are used to access and configure lane information such as the status and errors.*

- **Data Signalling Rate**

*Functions in this group are used to access and configure SpaceFibre lane data signalling rate.*

- **Types**

*Types used with the STAR-Dundee SpaceFibre Configuration API.*

- **Defines**

*Function return error values specific to the SpaceFibre Configuration API are defined here in addition to the [general error codes](#) that may also be returned.*

### 8.50.1 Detailed Description

The SpaceFibre Configuration API provides functions for configuring SpaceFibre devices.

Functions are divided into three categories: accessor functions read one or more values from the device, sub-accessor functions read a field within one of the values retrieved with an accessor function, and mutator functions write (or read-modify-write) one or more values to the device.

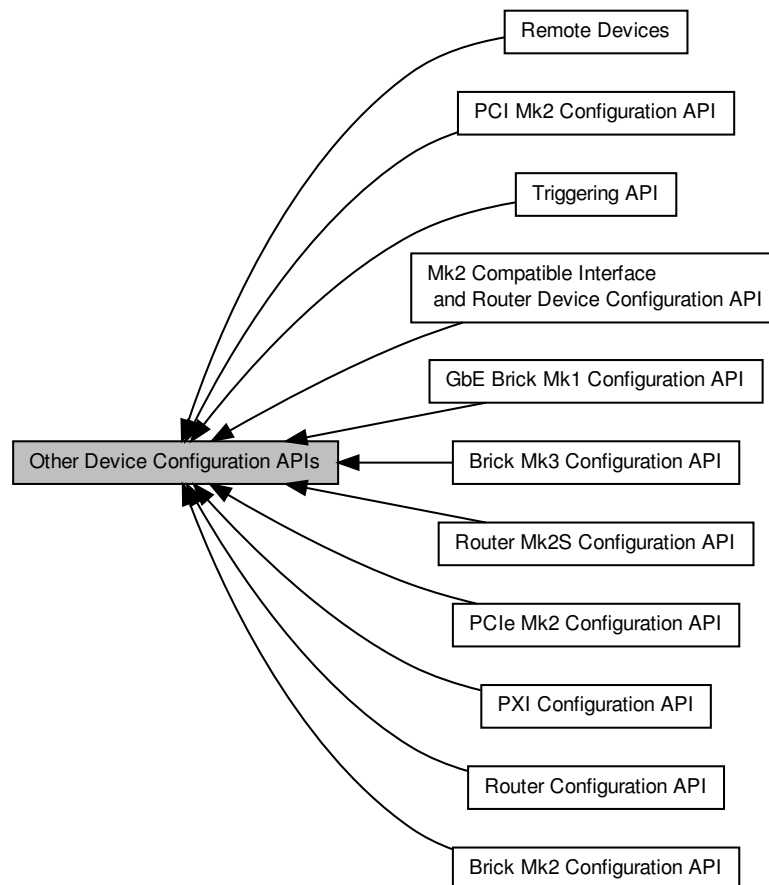
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.51 Other Device Configuration APIs

The Device Configuration APIs provide functions for configuring devices.

Collaboration diagram for Other Device Configuration APIs:



### Modules

- [Remote Devices](#)

*Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.*

- [Router Configuration API](#)

*The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.*

- [Mk2 Compatible Interface and Router Device Configuration API](#)

*The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#) and [Brick Mk4](#).*

- [PCI Mk2 Configuration API](#)

*The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).*

- [Brick Mk2 Configuration API](#)

The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3 and Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

- [Brick Mk3 Configuration API](#)

The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

- [GbE Brick Mk1 Configuration API](#)

The GbE Brick Mk1 Configuration API provides functions for configuring features of [SpaceWire GbE Brick Mk1](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#), [Brick Mk2 Configuration API](#) or [Brick Mk3 Configuration API](#).

- [Router Mk2S Configuration API](#)

The Router Mk2S Configuration API provides functions for configuring features specific to the [SpaceWire Router Mk2S](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

- [PXI Configuration API](#)

The PXI Configuration API provides functions for configuring features of [SpaceWire PXI Interface and PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

- [PCIe Mk2 Configuration API](#)

The PCIe Mk2 Configuration API provides functions for configuring features specific to the [SpaceWire PCIe Mk2](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk3 Configuration API](#).

- [Triggering API](#)

The Triggering API provides functions for configuring and controlling the triggering capabilities of the [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#), [SpaceWire PCIe](#) and [SpaceWire PCIe Mk2](#) devices.

### 8.51.1 Detailed Description

The Device Configuration APIs provide functions for configuring devices.

As well as configuring devices connected directly to the PC, the Device Configuration API can also be used to configure devices over a SpaceWire network. This is achieved by creating a remote device identifier (see [Remote Devices](#)) and passing this remote device identifier as the identifier of the device to be configured.

Note that the [Generic Configuration API](#) should be used for all new developments rather than the original Device Configuration APIs. The Generic Configuration API works with all device types, so different functions do not need called based on the device type, as can be the case with the other Configuration APIs.

The Device Configuration APIs consist of a number of sub-APIs which provide functions for configuring different devices. The [Router Configuration API](#) provides general functions for configuring router devices. Supported devices include the STAR-Dundee [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire PCIe Mk2](#), [SpaceWire Physical Layer Tester](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), the [SpaceWire Router-USB](#), the [SpaceWire Router Mk2](#), the [SpaceWire-USB Brick](#) and the [ESA SpW-10X](#). Additional APIs then provide functions specific to individual devices. Included is the [Mk2 Compatible Interface and Router Device Configuration API](#) that supports all Mk2 compatible interface and router devices. This includes the [SpaceWire PCIe](#), [SpaceWire PCIe Mk2](#), [SpaceWire PCI Mk2 and cPCI Mk2](#), [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) and [SpaceWire GbE Brick Mk1](#) devices. Also, the [PCI Mk2 Configuration API](#) is included, which provides additional functions specifically for configuring the link speed of the [SpaceWire PCI Mk2 and cPCI Mk2](#). The [Brick Mk2 Configuration API](#) includes functions for configuring features specific to the [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#), [SpaceWire Brick Mk3 and Brick Mk4](#) and [SpaceWire GbE Brick Mk1](#), although note that the [SpaceWire Brick Mk3 and Brick Mk4](#) uses separate base transmit clock functions provided by the [Brick Mk3 Configuration API](#), while the [SpaceWire GbE Brick Mk1](#) uses the [GbE Brick Mk1 Configuration API](#). The [Router Mk2S Configuration API](#) includes functions for configuring the precision transmit rate features of the [SpaceWire Router Mk2S](#). The [PXI Configuration API](#) provides separate



functions for configuring base transmit clock and link rate divider for the [SpaceWire PXI Interface and PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#) devices. The [PCIe Mk2 Configuration API](#) provides separate functions for configuring link speed, interrupt code functionality and timestamping for the [SpaceWire PCIe Mk2](#).

The [Configuration API Functions Supported by Device Types](#) lists the functions supported by each device.

### 8.51.2 Registers

A number of the configuration functions in this API perform an operation on single registers. It is more efficient to read the value of this register once, and then keep this value in memory to get the required information/set required parameters, than to read the register for every operation. Functions that operate on a single register are grouped together in this API.

## 8.52 Remote Devices

Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.

Collaboration diagram for Remote Devices:



### Functions

- `STAR_DEVICE_ID STAR_API_CC STAR_CFG_createRemoteDeviceIdentifier (STAR_DEVICE_ID deviceId, unsigned char localChannel, char *name, STAR_SPACEWIRE_ADDRESS *pathTo, STAR_SPACEWIRE_ADDRESS *returnPath)`  
*Creates a Remote Device Identifier.*
- `int STAR_API_CC STAR_CFG_destroyRemoteDeviceIdentifier (STAR_DEVICE_ID remoteDeviceId)`  
*Destroys a Remote Device Identifier.*
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList (_Out_ U32 *count)`  
*Gets an array of all remote devices that have been created by all processes.*
- `void STAR_API_CC STAR_CFG_destroyRemoteDeviceDescriptionList (_Post_ptr_invalid_ STAR_DEVICE_ID *pRemoteDeviceDescriptionList)`  
*Frees a remote device description list previously created by a call to `STAR_CFG_getRemoteDeviceDescriptionList()`.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel (STAR_DEVICE_ID remoteDeviceId, unsigned char *localChannel)`  
*Gets the number of the local device channel used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice (STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID *localDeviceId)`  
*Gets the local device used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDevicePath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pPath)`  
*Gets the path to the specified remote device.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath (STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS **pRetPath)`  
*Gets the return path from to the specified remote device.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString (STAR_DEVICE_ID deviceId)`  
*Gets the name of the remote device description identified by a given identifier.*

### 8.52.1 Detailed Description

Functions in this group deal with creating and destroying remote device identifiers, and obtaining the properties associated with a remote device identifier.

A remote device identifier can only be used to configure a device using the Configuration APIs.

## 8.52.2 Function Documentation

### 8.52.2.1 **STAR\_DEVICE\_ID** **STAR\_API\_CC** **STAR\_CFG\_createRemoteDeviceIdentifier** ( **STAR\_DEVICE\_ID** *deviceld*, unsigned char *localChannel*, char \* *name*, **STAR\_SPACEWIRE\_ADDRESS** \* *pathTo*, **STAR\_SPACEWIRE\_ADDRESS** \* *returnPath* )

Creates a Remote Device Identifier.

Remote device Identifiers are used to describe to the Configuration API how to reach a remote device.

#### Parameters

|                     |                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceld</i>     | Local Device communication with the remote device is initiated on.                                                                                                                                                                                                                                                                                                                   |
| <i>localChannel</i> | The number of the channel on the local device on which communication with the remote device is made. This channel cannot be used for any other purpose when a Device Configuration operation is being performed.                                                                                                                                                                     |
| <i>name</i>         | The name to be used to identify the remote device                                                                                                                                                                                                                                                                                                                                    |
| <i>pathTo</i>       | Path to the remote device. This should be the path to the remote device's configuration port (port 0), including a default logical address. This means that the path will normally end with "0 254".                                                                                                                                                                                 |
| <i>returnPath</i>   | Return path from the remote device to reach the localChannel. Configuration response packets are automatically sent out of the port on which the command is received, so this port number is not required. The path should also be terminated with a default logical address. This means that <i>returnPath</i> is normally one byte shorter than <i>pathTo</i> and ends with "254". |

#### Returns

Device ID for the remote device.

#### Added in version:

STAR-System v1.0 - 17th October 2011

#### Examples:

brickMk2\_configuration\_example.c, and link\_events.c.

### 8.52.2.2 **void** **STAR\_API\_CC** **STAR\_CFG\_destroyRemoteDeviceDescriptionList** ( **\_Post\_ptr\_invalid\_** **STAR\_DEVICE\_ID** \* *pRemoteDeviceDescriptionList* )

Frees a remote device description list previously created by a call to [STAR\\_CFG\\_getRemoteDeviceDescriptionList\(\)](#).

#### Parameters

|                                                 |                       |
|-------------------------------------------------|-----------------------|
| <i>pRemote↔<br/>Device↔<br/>DescriptionList</i> | The list to be freed. |
|-------------------------------------------------|-----------------------|

#### Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

#### Declaration changed in version:

STAR-System v2.0 Beta 3 - 12th April 2013

### 8.52.2.3 **int** **STAR\_API\_CC** **STAR\_CFG\_destroyRemoteDeviceIdentifier** ( **STAR\_DEVICE\_ID** *remoteDeviceld* )

Destroys a Remote Device Identifier.

**Parameters**


---

|                       |                                                             |
|-----------------------|-------------------------------------------------------------|
| <i>remoteDeviceId</i> | The device identifier of the remote device to be destroyed. |
|-----------------------|-------------------------------------------------------------|

---

**Returns**

Whether the remote device identifier was successfully destroyed.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**Examples:**

brickMk2\_configuration\_example.c, and link\_events.c.

#### 8.52.2.4 `_Ret_opt_cap_count STAR_DEVICE_ID* STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList ( _Out_ U32 * count )`

Gets an array of all remote devices that have been created by all processes.

**Parameters**


---

|            |              |                                                                                                              |
|------------|--------------|--------------------------------------------------------------------------------------------------------------|
| <i>out</i> | <i>count</i> | Pointer to user allocated variable that will be updated to contain the count of items in the returned array. |
|------------|--------------|--------------------------------------------------------------------------------------------------------------|

---

**Returns**

Array of remote device description identifiers.

**Note**

This function returns a snapshot of the current state of the system and is not automatically updated. This array must be freed using `STAR_CFG_destroyRemoteDeviceDescriptionList()` when no longer required.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**Declaration changed in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

#### 8.52.2.5 `_Check_return_ _Ret_opt_z_char* STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString ( STAR_DEVICE_ID deviceId )`

Gets the name of the remote device description identified by a given identifier.

The string returned should be freed by calling `STAR_destroyString()`.

**Parameters**


---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | The remote device description to check. |
|-----------------|-----------------------------------------|

---

**Returns**

Device name as a null terminated string.

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

8.52.2.6 `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel ( STAR_DEVICE_ID remoteDeviceId, unsigned char * localChannel )`

Gets the number of the local device channel used to communicate with the remote device on.

**Parameters**

|     |                       |                                                    |
|-----|-----------------------|----------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.              |
| out | <i>localChannel</i>   | Value that will be updated with the local channel. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**8.52.2.7** `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice ( STAR_DEVICE_ID remoteDeviceId, STAR_DEVICE_ID * localDeviceID )`

Gets the local device used to communicate with the remote device on.

**Parameters**

|     |                       |                                                      |
|-----|-----------------------|------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                |
| out | <i>localDeviceID</i>  | Value that will be updated with the local device ID. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**8.52.2.8** `int STAR_API_CC STAR_CFG_getRemoteDevicePath ( STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS ** pPath )`

Gets the path to the specified remote device.

The address should be destroyed when it is no longer required using `STAR_destroyAddress()`.

**Parameters**

|     |                       |                                                         |
|-----|-----------------------|---------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                   |
| out | <i>pPath</i>          | Value that will be updated with pointer to the address. |

**Returns**

1 on success, else 0.

**Added in version:**

STAR-System v1.0 - 17th October 2011

**8.52.2.9** `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath ( STAR_DEVICE_ID remoteDeviceId, _Out_opt_ STAR_SPACEWIRE_ADDRESS ** pRetPath )`

Gets the return path from to the specified remote device.

The address should be destroyed when it is no longer required using `STAR_destroyAddress()`.

**Parameters**

---

|     |                       |                                                         |
|-----|-----------------------|---------------------------------------------------------|
|     | <i>remoteDeviceId</i> | Remote device to get information for.                   |
| out | <i>pRetPath</i>       | Value that will be updated with pointer to the address. |

---

**Returns**

1 on success, else 0

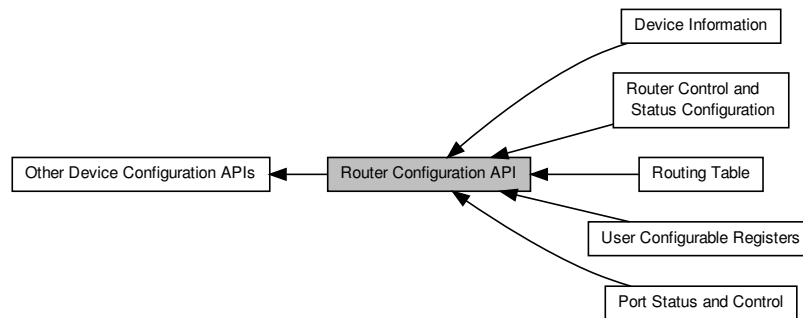
**Added in version:**

STAR-System v1.0 - 17th October 2011

## 8.53 Router Configuration API

The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.

Collaboration diagram for Router Configuration API:



### Modules

- **Device Information**

*Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.*

- **Port Status and Control**

*Functions in this group deal with the management of the configuration port, link ports, and external ports.*

- **Routing Table**

*Functions in this group deal with the management of a device's routing table.*

- **User Configurable Registers**

*Functions in this group deal with reading and writing to the user configurable registers.*

- **Router Control and Status Configuration**

*Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.*

### 8.53.1 Detailed Description

The Router Configuration API provides functions for configuring features common to a number of SpaceWire routing devices.

The devices directly supported by the Router Configuration API include the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), the [SpaceWire PCIe](#), the [SpaceWire PCIe Mk2](#), the [SpaceWire Brick Mk2](#), the [SpaceWire Router Mk2S](#), the [SpaceWire Brick Mk3](#) and [Brick Mk4](#), the [SpaceWire Router-USB](#), the [SpaceWire Router Mk2](#), the [SpaceWire-USB Brick](#) and the [ESA SpW-10X](#).

#### Note

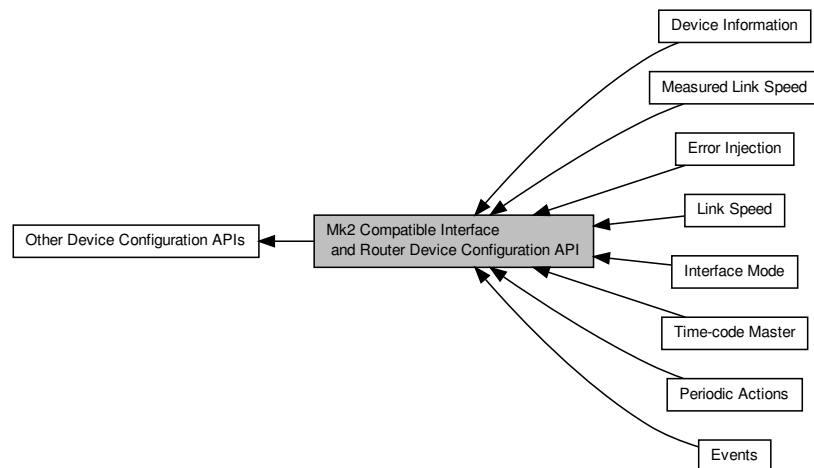
If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.



## 8.54 Mk2 Compatible Interface and Router Device Configuration API

The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#) and [Brick Mk4](#).

Collaboration diagram for Mk2 Compatible Interface and Router Device Configuration API:



### Modules

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

- [Measured Link Speed](#)

*Functions in this group deal with reading the measured link speed.*

- [Time-code Master](#)

*Functions in this group deal with the management of the device as a time-code master.*

- [Device Information](#)

*Functions in this group deal with hardware information, such as obtaining version information, or flashing the LEDs.*

- [Interface Mode](#)

*Functions in this group deal managing the device in Interface mode.*

- [Error Injection](#)

*Functions in this group deal with injecting errors onto SpaceWire links.*

- [Periodic Actions](#)

*Functions in this group deal with controlling periodic actions on SpaceWire links.*

### 8.54.1 Detailed Description

The Mk2 compatible device API provides functions for configuring features common across the Mk2 range of devices, including the [SpaceWire Brick Mk3](#) and [Brick Mk4](#).

**Note**

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.55 PCI Mk2 Configuration API

The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Collaboration diagram for PCI Mk2 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

#### 8.55.1 Detailed Description

The PCI Mk2 Configuration API provides functions for configuring features of the [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

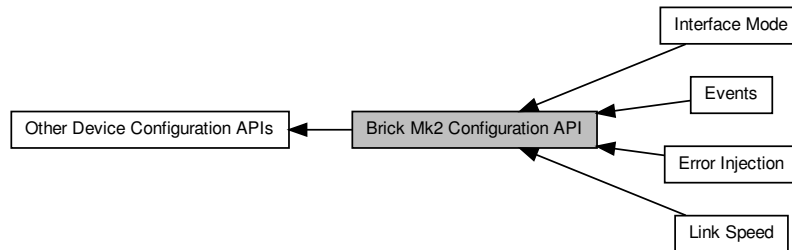
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.56 Brick Mk2 Configuration API

The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Collaboration diagram for Brick Mk2 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

- [Interface Mode](#)

*Functions in this group deal with managing the device in Interface mode.*

- [Error Injection](#)

*Functions in this group deal with error injection.*

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

### 8.56.1 Detailed Description

The Brick Mk2 Configuration API provides functions for configuring features of [SpaceWire Brick Mk2](#), [SpaceWire Router Mk2S](#) and [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#) or [Mk2 Compatible Interface and Router Device Configuration API](#).

Note that the [SpaceWire Brick Mk3](#) and [Brick Mk4](#) uses the base transmit clock functions provided by the [Mk2 Compatible Interface and Router Device Configuration API](#).

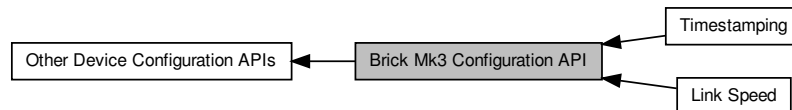
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.57 Brick Mk3 Configuration API

The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for Brick Mk3 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of Brick Mk3 devices.*

- [Timestamping](#)

*Functions in this group deal with timestamping related functions for the Brick Mk3.*

### 8.57.1 Detailed Description

The Brick Mk3 Configuration API provides functions for configuring features of [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

These functions are for configuring the base transmit clock of Brick Mk3 devices.

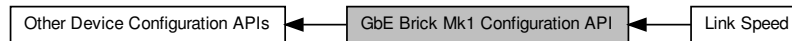
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.58 GbE Brick Mk1 Configuration API

The GbE Brick Mk1 Configuration API provides functions for configuring features of [SpaceWire GbE Brick Mk1](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#), [Brick Mk2 Configuration API](#) or [Brick Mk3 Configuration API](#).

Collaboration diagram for GbE Brick Mk1 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of GbE Brick Mk1 devices.*

### 8.58.1 Detailed Description

The GbE Brick Mk1 Configuration API provides functions for configuring features of [SpaceWire GbE Brick Mk1](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#), [Brick Mk2 Configuration API](#) or [Brick Mk3 Configuration API](#).

These functions are for configuring the transmit bit rate on a given link of GbE Brick Mk1 devices.

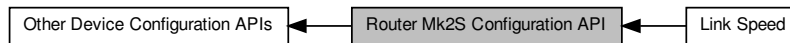
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.59 Router Mk2S Configuration API

The Router Mk2S Configuration API provides functions for configuring features specific to the [SpaceWire Router Mk2S](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for Router Mk2S Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds on the device.*

### 8.59.1 Detailed Description

The Router Mk2S Configuration API provides functions for configuring features specific to the [SpaceWire Router Mk2S](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface and Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

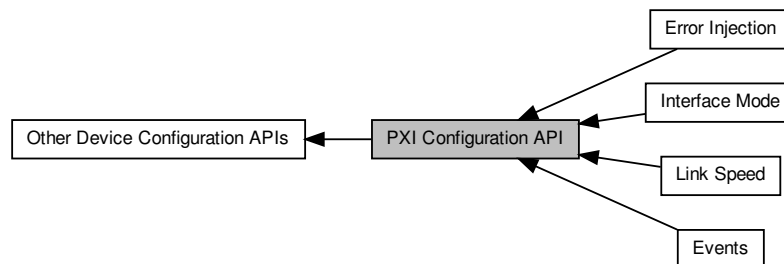
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.60 PXI Configuration API

The PXI Configuration API provides functions for configuring features of [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

Collaboration diagram for PXI Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of PXI devices devices.*

- [Error Injection](#)

*Functions in this group deal with injecting errors onto SpaceWire links.*

- [Interface Mode](#)

*Functions in this group deal with managing the device in Interface mode.*

- [Events](#)

*Functions in this group deal with enabling and disabling link events.*

### 8.60.1 Detailed Description

The PXI Configuration API provides functions for configuring features of [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) and [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#) devices specific to those devices, and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk2 Configuration API](#).

These functions are for configuring the base transmit clock and link rate dividers of PXI devices.

#### Note

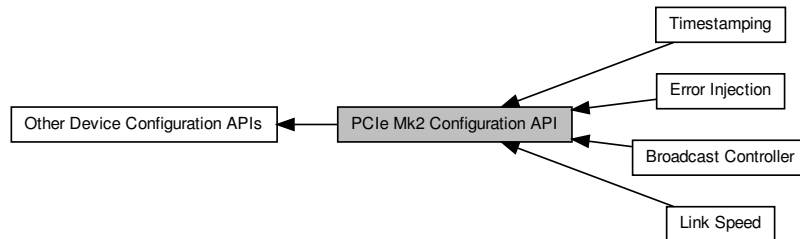
If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.



## 8.61 PCIe Mk2 Configuration API

The PCIe Mk2 Configuration API provides functions for configuring features specific to the [SpaceWire PCIe Mk2](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk3 Configuration API](#).

Collaboration diagram for PCIe Mk2 Configuration API:



### Modules

- [Link Speed](#)

*Functions in this group deal with setting the link speeds of PCIe Mk2 devices.*

- [Broadcast Controller](#)

*Functions in this group deal with PCIe Mk2 broadcasts.*

- [Timestamping](#)

*Functions in this group deal with PCIe Mk2 packet timestamping.*

- [Error Injection](#)

*Functions in this group deal with error injection.*

### 8.61.1 Detailed Description

The PCIe Mk2 Configuration API provides functions for configuring features specific to the [SpaceWire PCIe Mk2](#), and not covered in the [Router Configuration API](#), [Mk2 Compatible Interface](#) and [Router Device Configuration API](#) or [Brick Mk3 Configuration API](#).

These functions are for configuring transmit link speed, interrupt code functionality, timestamping and error injection.

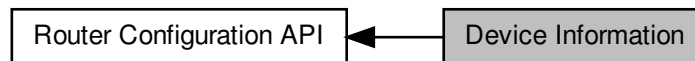
#### Note

If opening channel 0 in your own application, please be aware that the [Device Configuration Service](#) uses [Channel 0](#) to communicate with the device whenever a configuration function is called. The service keeps the channel open for around 200 ms after the last configuration operation is called so an appropriate delay may be required.

## 8.62 Device Information

Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.

Collaboration diagram for Device Information:



### Data Structures

- struct `STAR_CFG_DEVICE_IDENTIFIER_INFO`  
Information obtained from a Device's Identifier Register, identifying the router's version and type. [More...](#)
- struct `STAR_CFG_NETWORK_DISCOVERY_INFO`  
Information obtained from a device's network discovery register. [More...](#)

### Macros

- `#define STAR_CFG_MANUFACTURER_STR_MAX_LEN 256`  
Maximum length for the string representation of a manufacturer name.
- `#define STAR_CFG_DEVICE_STR_MAX_LEN 256`  
Maximum length for the string representation of a device type.

### Enumerations

- enum `STAR_CFG_DEVICE_TYPE` { `STAR_CFG_DEVICE_TYPE_ROUTER`, `STAR_CFG_DEVICE_TYPE_UNKNOWN`, `STAR_CFG_DEVICE_TYPE_INVALID` }  
Device types permitted within the network discovery register.

### Functions

- `int STAR_API_CC CFG_ROUTER_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
Reads the device identifier information of a device, i.e.
- `void STAR_API_CC CFG_ROUTER_getDeviceManufacturerAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *manufacturerStr)`  
If the manufacturer is known, this function obtains a string representation of the manufacturers name.
- `void STAR_API_CC CFG_ROUTER_getDeviceTypeAsString (STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *deviceTypeStr)`  
If the manufacturer and device type is known, this function obtains a string representation of the device's type.
- `int STAR_API_CC CFG_ROUTER_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
Reads the network discovery information of a device.

### 8.62.1 Detailed Description

Functions in this group deal with discovery of device information, such as device type, number of ports/links, and manufacturer information.

### 8.62.2 Data Structure Documentation

#### 8.62.2.1 struct STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO

Information obtained from a Device's Identifier Register, identifying the router's version and type.

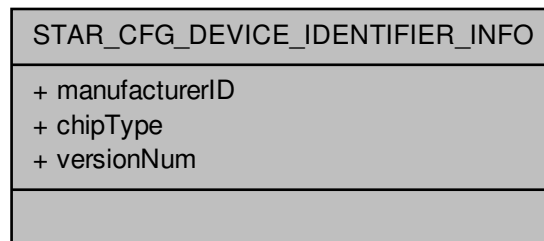
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`device_identifier_example.c`, and `star-test.c`.

Collaboration diagram for STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO:



#### Data Fields

|     |                |                                                                        |
|-----|----------------|------------------------------------------------------------------------|
| U8  | chipType       | Identity code for the SpaceWire chip from the particular manufacturer. |
| U16 | manufacturerID | Manufacturer ID number (STAR-Dundee: 1)                                |
| U8  | versionNum     | Device version number.                                                 |

#### 8.62.2.2 struct STAR\_CFG\_NETWORK\_DISCOVERY\_INFO

Information obtained from a device's network discovery register.

Information that can be used to determine the network layout.

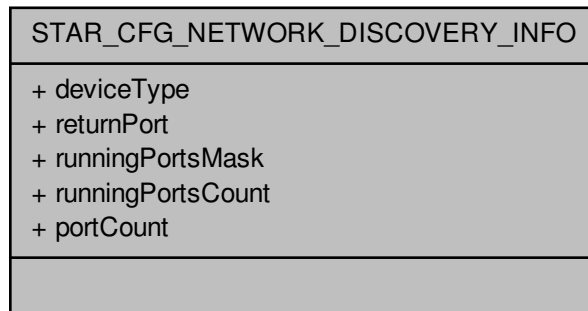
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`device_identifier_example.c`.

Collaboration diagram for STAR\_CFG\_NETWORK\_DISCOVERY\_INFO:



#### Data Fields

|                       |                        |                                                                                                 |
|-----------------------|------------------------|-------------------------------------------------------------------------------------------------|
| STAR_CFG_DEVICE_TYPE↔ | deviceType             | Type of device.                                                                                 |
| U8                    | portCount              | Count of the number of ports the device has.                                                    |
| U8                    | returnPort             | Indicates the input port number which was used to access the network discovery register.        |
| U8                    | runningPorts↔<br>Count | Count of running ports.                                                                         |
| U32                   | runningPorts↔<br>Mask  | Bitmask of ports which are in the run state.<br><br><b>Note</b><br>Bit 1 corresponds to port 1. |

### 8.62.3 Macro Definition Documentation

#### 8.62.3.1 #define STAR\_CFG\_DEVICE\_STR\_MAX\_LEN 256

Maximum length for the string representation of a device type.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

device\_identifier\_example.c, device\_info\_example.c, and star-test.c.

#### 8.62.3.2 #define STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN 256

Maximum length for the string representation of a manufacturer name.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`device_identifier_example.c`, `device_info_example.c`, and `star-test.c`.

**8.62.4 Enumeration Type Documentation****8.62.4.1 enum STAR\_CFG\_DEVICE\_TYPE**

Device types permitted within the network discovery register.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Enumerator**

**STAR\_CFG\_DEVICE\_TYPE\_ROUTER** Router device.

**STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN** Unknown device type.

**STAR\_CFG\_DEVICE\_TYPE\_INVALID** Invalid device type.

**8.62.5 Function Documentation****8.62.5.1 int STAR\_API\_CC\_CFG\_ROUTER\_getDeviceIdentificationInfo ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pInfo* )**

Reads the device identifier information of a device, i.e.  
the Manufacturer ID, Chip type and version.

**Parameters**

|     | <i>deviceId</i> | Device to obtain info from.                                                              |
|-----|-----------------|------------------------------------------------------------------------------------------|
| out | <i>pInfo</i>    | Structure that will be updated to contain the Identification information for the device. |

**Returns**

1 if device identifier info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.62.5.2 void STAR\_API\_CC\_CFG\_ROUTER\_getDeviceManufacturerAsString ( STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO \* *pDeviceIdentifierInfo*, \_Out\_z\_cap\_c\_ (STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN) char \* *manufacturerStr* )**

If the manufacturer is known, this function obtains a string representation of the manufacturers name.

**Parameters**

|     |                              |                                                                                                                                                                |
|-----|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <code>CFG_ROUTER_getDeviceIdentificationInfo()</code> .                                   |
| out | <i>manufacturerStr</i>       | User allocated zero terminated string of max length <code>STAR_CFG_MANUFACTURER_STR_MAX_LEN</code> that will be updated with the manufacturers name, if known. |

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.62.5.3** `void STAR_API_CC CFG_ROUTER_getDeviceTypeAsString ( STAR_CFG_DEVICE_IDENTIFIER_INFO * pDeviceIdentifierInfo, _Out_z_cap_c(STAR_CFG_DEVICE_STR_MAX_LEN) char * deviceTypeStr )`

If the manufacturer and device type is known, this function obtains a string representation of the device's type.

**Parameters**

|     |                              |                                                                                                                                                   |
|-----|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <code>CFG_ROUTER_getDeviceIdentificationInfo()</code> .                      |
| out | <i>deviceTypeStr</i>         | User allocated zero terminated string of max length <code>STAR_CFG_DEVICE_STR_MAX_LEN</code> that will be updated with the device name, if known. |

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.62.5.4** `int STAR_API_CC CFG_ROUTER_getNetworkDiscoveryInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO * pInfo )`

Reads the network discovery information of a device.

This information can be used by a network manager to determine the layout of the network.

**Parameters**

|     |                 |                                                                                             |
|-----|-----------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to obtain info from.                                                                 |
| out | <i>pInfo</i>    | Structure that will be updated to contain the network discovery information for the device. |

**Returns**

1 if device identifier info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

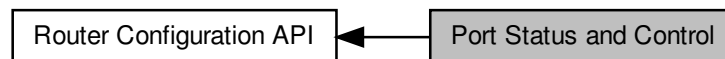
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

## 8.63 Port Status and Control

Functions in this group deal with the management of the configuration port, link ports, and external ports.

Collaboration diagram for Port Status and Control:



### Data Structures

- struct [STAR\\_CFG\\_CONFIG\\_PORT\\_ERRORS](#)  
*Errors that may be present on a device's configuration port. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_ERRORS](#)  
*Errors that may be present on a SpaceWire Link. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_STATUS](#)  
*Status of a SpaceWire link. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_ERRORS](#)  
*Errors that may be present on an external port. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_STATUS](#)  
*Status of an external port. [More...](#)*

### Typedefs

- typedef [REGISTER\\_PORT\\_STATUS\\_CONTROL](#)  
*The type used to represent a port status and control register value.*

### Enumerations

- enum [STAR\\_CFG\\_PORT\\_TYPE](#) { [STAR\\_CFG\\_PORT\\_TYPE\\_CONFIGURATION](#), [STAR\\_CFG\\_PORT\\_TYPE\\_LINK](#), [STAR\\_CFG\\_PORT\\_TYPE\\_EXTERNAL](#), [STAR\\_CFG\\_PORT\\_TYPE\\_INVALID](#) }  
*Port types for Routing devices.*
- enum [STAR\\_CFG\\_SPW\\_LINK\\_STATE](#) { [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_ERROR\\_RESET](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_ERROR\\_WAIT](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_READY](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_STARTED](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_CONNECTING](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_RUN](#), [STAR\\_CFG\\_SPW\\_LINK\\_STATE\\_INVALID](#) }  
*The state of the interface state machine in the SpaceWire link.*

### Functions

- int [STAR\\_API\\_CFG\\_ROUTER\\_getPortStatusControl](#) ([STAR\\_DEVICE\\_ID](#) deviceID, U8 portNum, [\\_OUT\\_PORT\\_STATUS\\_CONTROL](#) \*portStatusControl)  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*



- `STAR_CFG_PORT_TYPE STAR_API_CC CFG_ROUTER_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `U8 STAR_API_CC CFG_ROUTER_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC CFG_ROUTER_clearPortErrors (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC CFG_ROUTER_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *errors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkErrors (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkStatus (PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *status)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_SPW_LINK_STATUS linkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC CFG_ROUTER_startLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC CFG_ROUTER_stopLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC CFG_ROUTER_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)`  
*Gets the status of an external port.*

### 8.63.1 Detailed Description

Functions in this group deal with the management of the configuration port, link ports, and external ports.

### 8.63.2 Data Structure Documentation

#### 8.63.2.1 struct STAR\_CFG\_CONFIG\_PORT\_ERRORS

Errors that may be present on a device's configuration port.

If a member of this structure is set, then the error it represents is present.

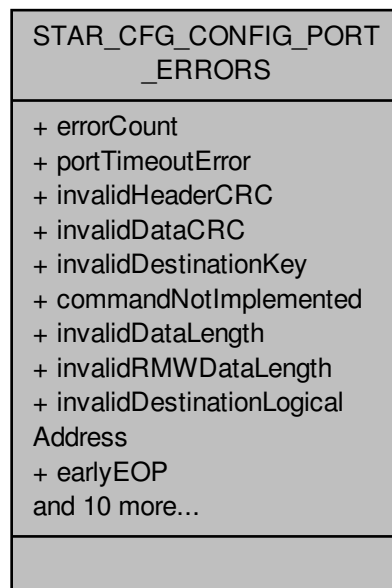
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`port_control_status_example.c`.

Collaboration diagram for STAR\_CFG\_CONFIG\_PORT\_ERRORS:



#### Data Fields

|      |                                  |                                                                                                                                                          |
|------|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | cargoTooLarge                    | The RMAP command packet is too large.                                                                                                                    |
| char | commandNotImplemented            | The command not implemented bit is set when the command code is a valid RMAP code but the command is not supported by the SpaceWire Router.              |
| char | earlyEEP                         | The early EEP bit is set when the command packet is terminated before the end of packet with an EEP.                                                     |
| char | earlyEOP                         | The early EOP bit is set when the command packet is terminated before the end of packet with an EOP.                                                     |
| char | errorCount                       | Count of errors present on the configuration port.                                                                                                       |
| char | invalidDataCRC                   | The invalid data CRC is set when the data field of the packet is corrupted and the CRC does not match the internally generated CRC.                      |
| char | invalidDataLength                | The invalid data length bit is set when a data length error is detected.                                                                                 |
| char | invalidDestinationKey            | The invalid destination key bit is set when the destination key in the command packet is invalid.                                                        |
| char | invalidDestinationLogicalAddress | The invalid destination logical address bit is set when the destination logical address in the command packet is not the default value of 254.           |
| char | invalidHeaderCRC                 | The Invalid header CRC bit is set when the header CRC is invalid.                                                                                        |
| char | invalidRegisterAddress           | The invalid register address bit is set when an unknown register address is given in the command packet or a write is attempted to a read only register. |

|      |                                         |                                                                                                                                                          |
|------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | invalidRMW↔<br>DataLength               | The read modify write command data length is invalid. The expected length is 8.                                                                          |
| char | lateEEP                                 | The late EEP bit is set when the command packet is not terminated correctly and trailing bytes are detected before the end of packet.                    |
| char | lateEOP                                 | The late EOP bit is set when the command packet is not terminated correctly and trailing bytes are detected before the end of packet.                    |
| char | portTimeout↔<br>Error                   | The port timeout error bit is set when a timeout event is detected by the configuration port routing logic.                                              |
| char | sourceLogical↔<br>AddressError          | The source logical address error bit is set when an invalid source logical address is received.                                                          |
| char | sourcePath↔<br>AddressError             | This bit is set when an invalid source address path is received.<br><br><b>Note</b><br><br>This error is not used for the 10X.                           |
| char | unsupported↔<br>Protocol                | The unsupported protocol error bit is set when a command packet is received with a protocol identifier which is not the RMAP protocol identifier (0x01). |
| char | unusedRMAP↔<br>CommandOr↔<br>PacketType | The command code is an unused command code or the packet type is invalid.                                                                                |
| char | verifyBuffer↔<br>Overrun                | The verify buffer overrun error bit is set when a verified write command is performed and the data length is not 4.                                      |

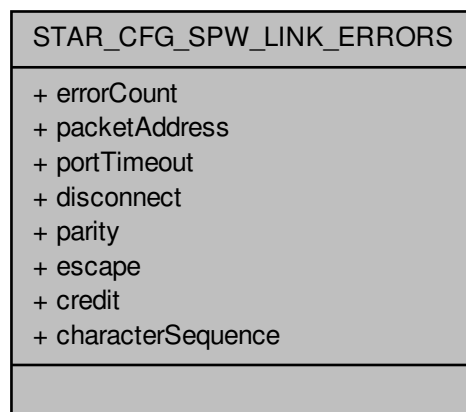
#### 8.63.2.2 struct STAR\_CFG\_SPW\_LINK\_ERRORS

Errors that may be present on a SpaceWire Link.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_SPW\_LINK\_ERRORS:



**Data Fields**

|      |                        |                                                                                                                                        |
|------|------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| char | character↔<br>Sequence | A character sequence error occurred on the link. A time-code or data character was received before the first FCT.                      |
| char | credit                 | A credit error occurred on the link.                                                                                                   |
| char | disconnect             | A disconnect error occurred on the link. No activity occurred on the link for the disconnect timeout period.                           |
| char | errorCount             | Count of errors present on the link port.                                                                                              |
| char | escape                 | An invalid escape code was received on the link. (ESC-ESC, ESC-EOP, or ESC-EEP).                                                       |
| char | packetAddress          | Packet received with an invalid address, either due to an unknown port, or an invalid logical address.                                 |
| char | parity                 | A parity was detected on a character parity bit.                                                                                       |
| char | portTimeout            | The port has become blocked for a period of time. A packet could not be routed to a destination port before the port timeout occurred. |

**8.63.2.3 struct STAR\_CFG\_SPW\_LINK\_STATUS**

Status of a SpaceWire link.

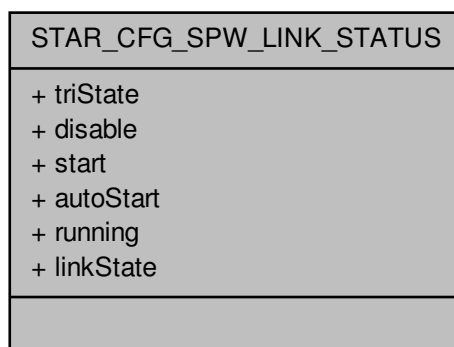
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[brickMk2\\_configuration\\_example.c](#), [link\\_events.c](#), [port\\_control\\_status\\_example.c](#), and [timestamp\\_test.c](#).

Collaboration diagram for STAR\_CFG\_SPW\_LINK\_STATUS:

**Data Fields**

|      |           |                                                                                                                                                                                                                              |
|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | autoStart | When set the SpaceWire link will auto-start as defined in the SpaceWire standard. The SpaceWire port will wait until the other end of the link tries to make a connection (sending NULLs) and will then automatically start. |
|------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|                                   |           |                                                                                                                                                                                                                  |
|-----------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char                              | disable   | When set then the SpaceWire link will be disabled as defined in the SpaceWire standard. The SpaceWire port will not start and will not respond to any attempt to make a connection by the other end of the link. |
| STAR_CFG_S↔<br>PW_LINK_ST↔<br>ATE | linkState | State of the interface state machine.                                                                                                                                                                            |
| char                              | running   | Set when the SpaceWire interface state machine is in the Run state.                                                                                                                                              |
| char                              | start     | When set then the SpaceWire link will initiate start-up as defined in the SpaceWire standard. The SpaceWire port will try to make a connection with the other end of the link.                                   |
| char                              | triState  | When set, the SpaceWire link LVDS drivers are in tri-state mode.                                                                                                                                                 |

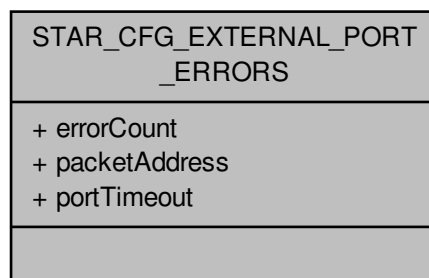
#### 8.63.2.4 struct STAR\_CFG\_EXTERNAL\_PORT\_ERRORS

Errors that may be present on an external port.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_EXTERNAL\_PORT\_ERRORS:



#### Data Fields

|      |               |                                                                                                                                                                                    |
|------|---------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | errorCount    | Count of errors present on the external port.                                                                                                                                      |
| char | packetAddress | Packet received with an invalid address, either due to an unknown port, or an invalid logical address. This is also generated when an empty packet is passed to the external port. |
| char | portTimeout   | The port has become blocked for a period of time. A packet could not be routed to a destination port before the port timeout occurred.                                             |

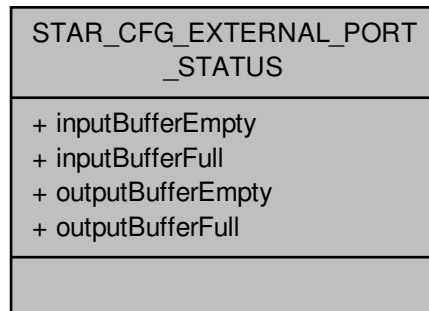
#### 8.63.2.5 struct STAR\_CFG\_EXTERNAL\_PORT\_STATUS

Status of an external port.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Collaboration diagram for STAR\_CFG\_EXTERNAL\_PORT\_STATUS:



#### Data Fields

|      |                        |                                                                                                                                                      |
|------|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | inputBuffer↔<br>Empty  | The external port input buffer is empty.<br><br><b>Note</b><br>The input buffer writes data to the SpaceWire router.                                 |
| char | inputBufferFull        | The external port input buffer is full.                                                                                                              |
| char | outputBuffer↔<br>Empty | The external output port buffer is empty.<br><br><b>Note</b><br>The output buffer writes data to the external device connected to the external port. |
| char | outputBufferFull       | The external output port buffer is full.                                                                                                             |

### 8.63.3 Typedef Documentation

#### 8.63.3.1 typedef REGISTER\_PORT\_STATUS\_CONTROL

The type used to represent a port status and control register value.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### 8.63.4 Enumeration Type Documentation

#### 8.63.4.1 enum STAR\_CFG\_PORT\_TYPE

Port types for Routing devices.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Enumerator**

**STAR\_CFG\_PORT\_TYPE\_CONFIGURATION** Configuration port.  
**STAR\_CFG\_PORT\_TYPE\_LINK** SpaceWire Link port.  
**STAR\_CFG\_PORT\_TYPE\_EXTERNAL** External port.  
**STAR\_CFG\_PORT\_TYPE\_INVALID** Invalid value. This should never occur

**8.63.4.2 enum STAR\_CFG\_SPW\_LINK\_STATE**

The state of the interface state machine in the SpaceWire link.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Enumerator**

**STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RESET** Error Reset.  
**STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT** Error Wait.  
**STAR\_CFG\_SPW\_LINK\_STATE\_READY** Ready.  
**STAR\_CFG\_SPW\_LINK\_STATE\_STARTED** Started.  
**STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING** Connecting.  
**STAR\_CFG\_SPW\_LINK\_STATE\_RUN** Run.  
**STAR\_CFG\_SPW\_LINK\_STATE\_INVALID** Invalid value.

**8.63.5 Function Documentation****8.63.5.1 int STAR\_API\_CC CFG\_ROUTER\_clearPortErrors ( STAR\_DEVICE\_ID deviceId, U8 portNum )**

Clears all errors on a given port.

**Note**

Errors on a port are latched until cleared with this function.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Device to clear port errors on.      |
| <i>portNum</i>  | The port to have its errors cleared. |

**Returns**

1 if the port port errors were successfully cleared, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.2** `int STAR_API_CC CFG_ROUTER_getConfigPortErrors ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS * errors )`

Gets any errors present on the config port.

These are errors that arise when malformed or invalid configuration commands are sent to the configuration port.

#### Note

Errors on the configuration port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

#### Parameters

|     |                          |                                                                                                          |
|-----|--------------------------|----------------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> .       |
| out | <i>errors</i>            | User provided error structure that will be updated to show any errors present on the configuration port. |

#### Returns

0 if the portStatusControl value passed in is not for a configuration port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.3** `int STAR_API_CC CFG_ROUTER_getExternalPortErrors ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS * errors )`

Gets any errors present on an external port.

#### Note

Errors on a port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

#### Parameters

|     |                          |                                                                                                     |
|-----|--------------------------|-----------------------------------------------------------------------------------------------------|
|     | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> .  |
| out | <i>errors</i>            | User provided error structure that will be updated to show any errors present on the external port. |

#### Returns

0 if the portStatusControl value passed in is not for an external port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester



**8.63.5.4** `int STAR_API_CC CFG_ROUTER_getExternalPortStatus ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS * status )`

Gets the status of an external port.

#### Parameters

|            |                          |                                                                                                    |
|------------|--------------------------|----------------------------------------------------------------------------------------------------|
|            | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> . |
| <i>out</i> | <i>status</i>            | User provided error structure that will be updated to show the status of the external port.        |

#### Returns

0 if the portStatusControl value passed in is not for an external port, otherwise 1.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.5** `U8 STAR_API_CC CFG_ROUTER_getPortConnection ( PORT_STATUS_CONTROL portStatusControl )`

Identifies the output port to which the source port is currently connected whilst routing is in operation.

#### Parameters

|  |                          |                                                                                                    |
|--|--------------------------|----------------------------------------------------------------------------------------------------|
|  | <i>portStatusControl</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> . |
|--|--------------------------|----------------------------------------------------------------------------------------------------|

#### Returns

Output port connected. This will have a value of 31 if there is no current port connected.

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.6** `int STAR_API_CC CFG_ROUTER_getPortStatusControl ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ PORT_STATUS_CONTROL * portStatusControl )`

Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.

**Parameters**

|     |                                |                                                                |
|-----|--------------------------------|----------------------------------------------------------------|
|     | <i>deviceID</i>                | Device to obtain info from.                                    |
|     | <i>portNum</i>                 | The port number to determine the type of.                      |
| out | <i>portStatus↔<br/>Control</i> | Value that will be updated with the ports status/control value |

**Returns**

1 if device port type info was successfully read, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.63.5.7 **STAR\_CFG\_PORT\_TYPE STAR\_API\_CC CFG\_ROUTER\_getPortType ( PORT\_STATUS\_CONTROL *portStatusControl* )**

Identifies the type of port attributed to a given port number.

**Parameters**

|  |                                |                                                                                                    |
|--|--------------------------------|----------------------------------------------------------------------------------------------------|
|  | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_ROUTER_getPortStatusControl()</code> . |
|--|--------------------------------|----------------------------------------------------------------------------------------------------|

**Returns**

Type of the port

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.63.5.8 **int STAR\_API\_CC CFG\_ROUTER\_getSpaceWireLinkErrors ( PORT\_STATUS\_CONTROL *linkStatusControl*, \_Out\_ STAR\_CFG\_SPW\_LINK\_ERRORS \* *errors* )**

Gets any errors present on a SpaceWire link.

**Note**

Errors on a port are latched until cleared with `CFG_ROUTER_clearPortErrors()`.

**Parameters**

|     |                          |                                                                                                        |
|-----|--------------------------|--------------------------------------------------------------------------------------------------------|
|     | <i>linkStatusControl</i> | Port Status/Control value obtained from a call to <a href="#">CFG_ROUTER_getPort↔StatusControl()</a> . |
| out | <i>errors</i>            | User provided error structure that will be updated to show any errors present on the SpaceWire link.   |

**Returns**

0 if the portStatusControl value passed in is not for a SpaceWire link, otherwise 1.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.9** `int STAR_API_CC CFG_ROUTER_getSpaceWireLinkStatus ( PORT_STATUS_CONTROL linkStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS * status )`

Gets the status of a SpaceWire Link.

**Parameters**

|     |                          |                                                                                                        |
|-----|--------------------------|--------------------------------------------------------------------------------------------------------|
|     | <i>linkStatusControl</i> | Port Status/Control value obtained from a call to <a href="#">CFG_ROUTER_getPort↔StatusControl()</a> . |
| out | <i>status</i>            | User provided error structure that will be updated to show the status of the SpaceWire link.           |

**Returns**

0 if the portStatusControl value passed in is not for a SpaceWire link port, otherwise 1.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.63.5.10** `int STAR_API_CC CFG_ROUTER_setSpaceWireLinkStatus ( STAR_DEVICE_ID deviceId, U8 linkNum, STAR_CFG_SPW_LINK_STATUS linkStatus )`

Sets the state of a SpaceWire Link.

**Parameters**

|                   |                                                                                   |
|-------------------|-----------------------------------------------------------------------------------|
| <i>deviceID</i>   | Device to have its link state set.                                                |
| <i>linkNum</i>    | The link to set state for.                                                        |
| <i>linkStatus</i> | The values within this structure are used to set the state of the SpaceWire Link. |

**Note**

The linkState member of this structure is ignored by this function.

**Returns**

1 if the link state was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.63.5.11 int STAR\_API\_CC\_CFG\_ROUTER\_startLink ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum* )

Starts a SpaceWire link by setting the start bit, and clearing the disable bit.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link started. |
| <i>linkNum</i>  | The link to start.               |

**Returns**

1 if the link was successfully started, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.63.5.12 int STAR\_API\_CC\_CFG\_ROUTER\_stopLink ( STAR\_DEVICE\_ID *deviceID*, U8 *linkNum* )

Stops a SpaceWire link by clearing the start bit, and setting the disable bit.

**Note**

If auto-start is enabled the link will start again as soon as it receives NULLs.

**Parameters**

---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its link stopped. |
| <i>linkNum</i>  | The link to stop.                |

---

**Returns**

1 if the link was successfully stopped, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

## 8.64 Routing Table

Functions in this group deal with the management of a device's routing table.

Collaboration diagram for Routing Table:



### Data Structures

- struct `STAR_CFG_GAR_ENTRY`

*A Routing table entry. [More...](#)*

### Functions

- int `STAR_API_CC_CFG_ROUTER_getRoutingTableEntry` (`STAR_DEVICE_ID` deviceID, U8 logicalAddress, `_Out_ STAR_CFG_GAR_ENTRY *`entry)
- Gets a routing table entry for a given logical address.*
- int `STAR_API_CC_CFG_ROUTER_setRoutingTableEntry` (`STAR_DEVICE_ID` deviceID, U8 logicalAddress, `STAR_CFG_GAR_ENTRY` entry)

*Sets a routing table entry for a given logical address.*

#### 8.64.1 Detailed Description

Functions in this group deal with the management of a device's routing table.

This is the table that maps Logical Addresses to one or more physical ports. There is one entry in the routing table for each possible Logical Address.

#### Warning

Care must be taken to avoid infinite loops when setting up routing tables. If, for example, there is a SpaceWire link made between two ports of the router, and a logical address routes a packet out of one of these ports, it will arrive back at the other, and then be routed out again continually, forever. If the packet is large enough it may block instead.

#### 8.64.2 Data Structure Documentation

##### 8.64.2.1 struct `STAR_CFG_GAR_ENTRY`

A Routing table entry.

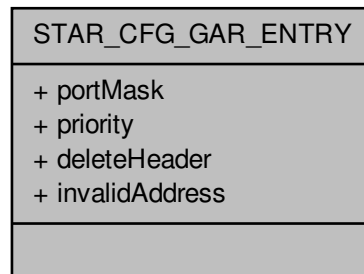
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`routing_table_example.c.`

Collaboration diagram for STAR\_CFG\_GAR\_ENTRY:

**Data Fields**

|      |                |                                                                                                                                                                                                                                          |
|------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | deleteHeader   | When set the leading header byte of the input packet will be removed before it is transferred to the output port.                                                                                                                        |
| char | invalidAddress | When set this indicates that the corresponding logical address is invalid. In this case, any packets arriving at the router with an invalid address are spilt and an address error is reported in the port status register.              |
| U32  | portMask       | Bitmask of output ports the logical address will arbitrate for. Valid bits set are 1 through 28.<br><br><b>Note</b><br>Bit 1 corresponds to port 1<br>It is not possible to access the configuration port (0) through logical addresses. |
| char | priority       | Packets with the logical address for this entry with the priority bit set will be granted access to a particular output port in preference to packets whose logical addresses in the routing table have their priority bit set to zero.  |

**8.64.3 Function Documentation**

**8.64.3.1** `int STAR_API_CC CFG_ROUTER_getRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY * entry )`

Gets a routing table entry for a given logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Device to get GAR table entry for.          |
| <i>logicalAddress</i> | Logical address to get GAR table entry for. |

|            |              |                                                    |
|------------|--------------|----------------------------------------------------|
| <i>out</i> | <i>entry</i> | The GAR table entry for the given logical address. |
|------------|--------------|----------------------------------------------------|

**Returns**

1 if the entry was successfully obtained from the device, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.64.3.2 int STAR\_API\_CC\_CFG\_ROUTER\_setRoutingTableEntry ( STAR\_DEVICE\_ID *deviceId*, U8 *logicalAddress*, STAR\_CFG\_GAR\_ENTRY *entry* )

Sets a routing table entry for a given logical address.

**Parameters**

|                       |                                                                              |
|-----------------------|------------------------------------------------------------------------------|
| <i>deviceId</i>       | Device to set GAR table entry on.                                            |
| <i>logicalAddress</i> | Logical address to set GAR table entry for. Valid values are 32 through 255. |

**Note**

Logical address 255 is reserved for future use and should not be used. See section 10.3.3n of the SpaceWire standard for more information.

**Parameters**

|              |                                                    |
|--------------|----------------------------------------------------|
| <i>entry</i> | The GAR table entry for the given logical address. |
|--------------|----------------------------------------------------|

**Returns**

1 if the entry was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

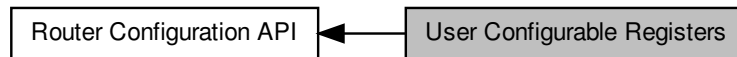
SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester



## 8.65 User Configurable Registers

Functions in this group deal with reading and writing to the user configurable registers.

Collaboration diagram for User Configurable Registers:



### Functions

- `int STAR_API_CC_CFG_MK2_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_MK2_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_setNetworkIdentity (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_getNetworkIdentity (STAR_DEVICE_ID deviceId, _Out_ REGISTER *value)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_setGeneralPurpose (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_getGeneralPurpose (STAR_DEVICE_ID deviceId, _Out_ REGISTER *value)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *portMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*

### 8.65.1 Detailed Description

Functions in this group deal with reading and writing to the user configurable registers.

These registers have no effect upon the operation of the router.

### 8.65.2 Function Documentation

**8.65.2.1** `int STAR_API_CC_CFG_MK2_getTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask )`

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|     |                  |                                                                     |
|-----|------------------|---------------------------------------------------------------------|
|     | <i>deviceID</i>  | Device to get the time-code distribution ports for.                 |
| out | <i>pPortMask</i> | Bit mask of ports that time code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v4.00 Beta 1 - 25th September 2018

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

#### 8.65.2.2 int STAR\_API\_CC\_CFG\_MK2\_setTimeCodeDistributionPorts ( STAR\_DEVICE\_ID *deviceID*, U32 *portMask* )

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

|  |                 |                                                                                                                                                       |
|--|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code distribution ports for.                                                                                                   |
|  | <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 13 depending on the device. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v4.00 Beta 1 - 25th September 2018

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCIe Mk2

#### 8.65.2.3 int STAR\_API\_CC\_CFG\_ROUTER\_getGeneralPurpose ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ REGISTER \* *value* )

Gets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|            |                 |                                          |
|------------|-----------------|------------------------------------------|
|            | <i>deviceId</i> | Device to get the network identity from. |
| <i>out</i> | <i>value</i>    | Value of the register                    |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.65.2.4** `int STAR_API_CC_CFG_ROUTER_getNetworkIdentity ( STAR_DEVICE_ID deviceId, _Out_ REGISTER * value )`

Gets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|            |                 |                                          |
|------------|-----------------|------------------------------------------|
|            | <i>deviceId</i> | Device to get the network identity from. |
| <i>out</i> | <i>value</i>    | Value of the register.                   |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.65.2.5** `int STAR_API_CC_CFG_ROUTER_getTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceId, _Out_ U32 * portMask )`

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|     |                 |                                                                     |
|-----|-----------------|---------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get the time-code distribution ports for.                 |
| out | <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 8.65.2.6 int STAR\_API\_CC\_CFG\_ROUTER\_setGeneralPurpose ( STAR\_DEVICE\_ID *deviceID*, REGISTER *value* )

Sets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|  |                 |                                                |
|--|-----------------|------------------------------------------------|
|  | <i>deviceID</i> | Device to set the general purpose register on. |
|  | <i>value</i>    | Value to set the general purpose register to.  |

**Returns**

1 if the register was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.65.2.7 int STAR\_API\_CC\_CFG\_ROUTER\_setNetworkIdentity ( STAR\_DEVICE\_ID *deviceID*, REGISTER *value* )

Sets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceID</i> | Device to set the network identity for. |
| <i>value</i>    | Value to set the network identity to.   |

---

**Returns**

1 if the register was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.65.2.8 int STAR\_API\_CC\_CFG\_ROUTER\_setTimeCodeDistributionPorts ( STAR\_DEVICE\_ID deviceID, U32 portMask )**

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

---

|                 |                                                                                                                                                       |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Device to set the time-code distribution ports for.                                                                                                   |
| <i>portMask</i> | Bit mask of ports that time code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 11 depending on the device. |

---

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

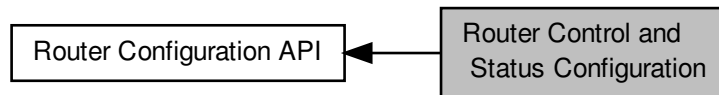
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

## 8.66 Router Control and Status Configuration

Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.

Collaboration diagram for Router Control and Status Configuration:



### Data Structures

- struct `STAR_CFG_ROUTER_GLOBAL_STATE`

Global router settings. [More...](#)

### Enumerations

- enum `STAR_CFG_PORT_TIMEOUT` {  
`STAR_CFG_PORT_TIMEOUT_100US = 0x0`, `STAR_CFG_PORT_TIMEOUT_1MS = 0x1`, `STAR_CFG_PORT_TIMEOUT_10MS = 0x2`, `STAR_CFG_PORT_TIMEOUT_100MS = 0x3`,  
`STAR_CFG_PORT_TIMEOUT_1S = 0x4` }

Length of port timeout.

- enum `STAR_CFG_TIMEOUT_MODE` { `STAR_CFG_TIMEOUT_MODE_BLOCKING`, `STAR_CFG_TIMEOUT_MODE_WATCHDOG` }

Timeout mode used by the router.

### Functions

- int `STAR_API_CC_CFG_MK2_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_MK2_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)`  
*Obtains the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *mode)`  
*Obtains the time-code flag interpretation mode:*
- int `STAR_API_CC_CFG_ROUTER_getTimeCodeValue (STAR_DEVICE_ID deviceId, _Out_ U8 *value)`  
*Obtains the current value of the router's internal time-code counter.*
- int `STAR_API_CC_CFG_ROUTER_setRouterGlobalSettings (STAR_DEVICE_ID deviceId, STAR_CFG_ROUTER_GLOBAL_STATE state)`  
*Sets the router's global settings.*
- int `STAR_API_CC_CFG_ROUTER_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *state)`  
*Gets the router's global settings.*

### 8.66.1 Detailed Description

Functions in this group deal with configuring the router's internal state, such as timeout periods self addressing, time code forwarding etc.

### 8.66.2 Data Structure Documentation

#### 8.66.2.1 struct STAR\_CFG\_ROUTER\_GLOBAL\_STATE

Global router settings.

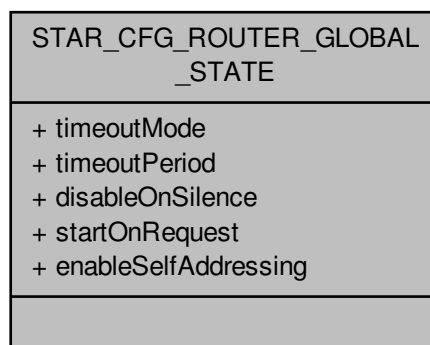
Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

rmap\_target\_example.c, router\_configuration\_example.c, and timestamp\_test.c.

Collaboration diagram for STAR\_CFG\_ROUTER\_GLOBAL\_STATE:



## Data Fields

|                       |      |                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                       | char | disableOn↔<br>Silence     | If true, links will be disabled after the timeout period when data transfer completes. The link will only be disabled if the link was started automatically.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|                       | char | enableSelf↔<br>Addressing | If true, a packet can be routed out of the port on which it arrived on. This is for debugging purposes. If this is false and a packet is to be routed through the same port an address error is reported and the packet is discarded. If false, and a group adaptive routing packet is received (a packet which can be routed through two or more ports, dependent on the group adaptive routing table contents) which can be routed through the port it arrived on then the packet is routed through one of the other ports and not the port on which the packet arrived on. An address error is not reported. |
|                       | char | startOnRequest            | If true, links will be automatically started when they have data to transfer. If the link cannot be started packets are discarded after the timeout period.                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| STAR_CFG_TIMEOUT_MODE |      | timeoutMode               | Timeout mode.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| STAR_CFG_PORT_TIMEOUT |      | timeoutPeriod             | Timeout period.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

## 8.66.3 Enumeration Type Documentation

## 8.66.3.1 enum STAR\_CFG\_PORT\_TIMEOUT

Length of port timeout.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_PORT\_TIMEOUT\_100US** 60-80us

**STAR\_CFG\_PORT\_TIMEOUT\_1MS** ~1.3ms

**STAR\_CFG\_PORT\_TIMEOUT\_10MS** ~10ms

**STAR\_CFG\_PORT\_TIMEOUT\_100MS** ~82ms

**STAR\_CFG\_PORT\_TIMEOUT\_1S** ~1.3s

## 8.66.3.2 enum STAR\_CFG\_TIMEOUT\_MODE

Timeout mode used by the router.

This affects how blocked packets will be handled.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

## Enumerator

**STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING** Blocking allowed. When blocking mode is enabled packets will wait forever to be routed unless the packet is routed to a port that is not started. In this case the packet will be discarded.



**STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG** Watchdog timer mode. When watchdog mode is enabled packets which are waiting to be routed at source ports will be discarded after the timeout period. Packet tails will also be discarded if the packet becomes blocked for the timeout period. In this case the packet will be ended with an EEP.

## 8.66.4 Function Documentation

### 8.66.4.1 `int STAR_API_CC_CFG_MK2_getTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, _Out_ U8 * pMode )`

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

#### Parameters

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceID</i> | Device to get the time-code flag mode from. |
| out | <i>pMode</i>    | Time code flag mode,                        |

#### Returns

1 if the register was successfully obtained, else 0.

#### Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire PCIe Mk2

### 8.66.4.2 `int STAR_API_CC_CFG_MK2_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, U8 mode )`

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

#### Parameters

|  |                 |                                             |
|--|-----------------|---------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code flag modes for. |
|  | <i>mode</i>     | Mode to set the time code flag mode to,     |

#### Returns

1 if the register was successfully obtained, else 0.

#### Added in version:

STAR-System v4.00 Beta 1 - 25th September 2018

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Router Mk2S, SpaceWire PCIe Mk2

8.66.4.3 `int STAR_API_CC CFG_ROUTER_getRouterGlobalSettings ( STAR_DEVICE_ID deviceId, _Out_  
STAR_CFG_ROUTER_GLOBAL_STATE * state )`

Gets the router's global settings.

**Parameters**

|            |                 |                                     |
|------------|-----------------|-------------------------------------|
|            | <i>deviceID</i> | Device to get global settings from. |
| <i>out</i> | <i>state</i>    | Global settings for the router.     |

**Returns**

1 if the entry was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.66.4.4 int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeFlagMode ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U8 \* *mode* )

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Note**

This function is currently only supported by SpaceWire PCIe and SpaceWire PCI Mk2 and cPCI Mk2 devices with at least hardware version v1.14 edit 4 and SpaceWire Physical Layer Tester devices with at least hardware version v2.03.

**Parameters**

|            |                 |                                             |
|------------|-----------------|---------------------------------------------|
|            | <i>deviceID</i> | Device to get the time-code flag mode from. |
| <i>out</i> | <i>mode</i>     | Time code flag mode,                        |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

#### 8.66.4.5 int STAR\_API\_CC\_CFG\_ROUTER\_getTimeCodeValue ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U8 \* *value* )

Obtains the current value of the router's internal time-code counter.

•

**Parameters**

|            |                 |                                         |
|------------|-----------------|-----------------------------------------|
|            | <i>deviceID</i> | Device to get the time-code value from. |
| <i>out</i> | <i>value</i>    | Time-code value,                        |

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.66.4.6 `int STAR_API_CC_CFG_ROUTER_setRouterGlobalSettings ( STAR_DEVICE_ID deviceID, STAR_CFG_ROUTER_GLOBAL_STATE state )`

Sets the router's global settings.

**Parameters**

|  |                 |                                    |
|--|-----------------|------------------------------------|
|  | <i>deviceID</i> | Device to set global settings for. |
|  | <i>state</i>    | Global settings for the router.    |

**Returns**

1 if the entry was successfully set, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

#### 8.66.4.7 `int STAR_API_CC_CFG_ROUTER_setTimeCodeFlagMode ( STAR_DEVICE_ID deviceID, U8 mode )`

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) and [SpaceWire PCI Mk2 and cPCI Mk2](#) devices with at least hardware version v1.14 edit 4 and [SpaceWire Physical Layer Tester](#) devices with at least hardware version v2.03.

**Parameters**

---

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceID</i> | Device to set the time-code flag modes for. |
| <i>mode</i>     | Mode to set the time code flag mode to,     |

---

**Returns**

1 if the register was successfully obtained, else 0.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

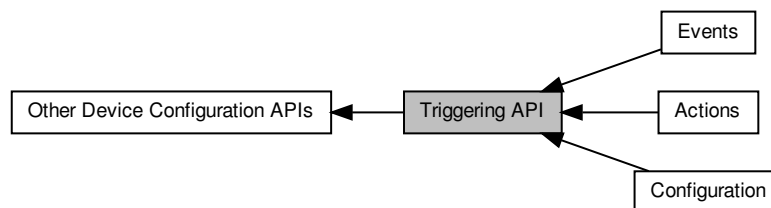
**Supported devices:**

SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire Physical Layer Tester

## 8.67 Triggering API

The Triggering API provides functions for configuring and controlling the triggering capabilities of the [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#), [SpaceWire PCIe](#) and [SpaceWire PCIe Mk2](#) devices.

Collaboration diagram for Triggering API:



### Modules

- [Events](#)

*Functions in this group deal with setting and getting trigger input events.*

- [Actions](#)

*Functions in this group deal with setting and getting trigger output actions.*

- [Configuration](#)

*Functions in this group deal with configuring the different triggering subsystems.*

### 8.67.1 Detailed Description

The Triggering API provides functions for configuring and controlling the triggering capabilities of the [SpaceWire Brick Mk3 and Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#), [SpaceWire PCIe](#) and [SpaceWire PCIe Mk2](#) devices.

The Triggering API allows events that occur on triggering resources such as external trigger ports, counters, time-code interfaces or SpaceWire ports to cause actions. For example, the triggers could be configured to transmit a packet on a port whenever an external trigger detects the rising edge of an input signal.

The three main concepts of the Triggering API are:

- **Input Events:** these are events that occur on triggering resources.
- **Output Actions:** these are actions that are performed by triggering resources.
- **Internal Triggers:** these are used to control the input events that cause output actions.

These concepts are described in the following sections.

### 8.67.2 Input Events

An input event is an event that occurs on one of the triggering resources. These events may be configured to set one or more internal triggers. Each triggering resource has one or more events that can occur. For example, SpaceWire ports have events for when they detect certain characters, errors or receive/transmit time-codes.

The following tables provide a description of the input events for each triggering resource.

**External Trigger Input Events**

| Input Event                    | Description                                                    | Supported Devices            |               |            |      |          |
|--------------------------------|----------------------------------------------------------------|------------------------------|---------------|------------|------|----------|
|                                |                                                                | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |
| EXT_TRI↔<br>GGER_EV↔<br>ENT_IN | This event occurs when the external trigger receives a signal. | Yes                          | Yes           | No         | No   | Yes      |

### Counter Input Events

| Input Event                              | Description                                              | Supported Devices            |               |            |      |          |
|------------------------------------------|----------------------------------------------------------|------------------------------|---------------|------------|------|----------|
|                                          |                                                          | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |
| COUNTE↔<br>R_EVENT↔<br>_COUNT            | This event occurs when the counter decrements.           | Yes                          | Yes           | Yes        | Yes  | Yes      |
| COUNTE↔<br>R_EVENT↔<br>_ZERO_SI↔<br>NGLE | This event occurs when the counter reaches zero.         | Yes                          | Yes           | Yes        | Yes  | Yes      |
| COUNTE↔<br>R_EVENT↔<br>_ZERO             | This event is active while the counter is equal to zero. | Yes                          | Yes           | Yes        | Yes  | Yes      |
| COUNTE↔<br>R_EVENT↔<br>_RELOAD           | This event occurs when the counter reloads.              | Yes                          | Yes           | Yes        | Yes  | Yes      |

### SpaceWire Port Input Events

| Input Event                 | Description                                                       | Supported Devices            |               |            |      |          |
|-----------------------------|-------------------------------------------------------------------|------------------------------|---------------|------------|------|----------|
|                             |                                                                   | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |
| PORT_EV↔<br>ENT_RX_↔<br>SOP | This event occurs when the port receives a Start of Packet (SOP). | Yes                          | Yes           | Yes        | Yes  | Yes      |



|                                         |                                                                        |     |     |     |     |     |
|-----------------------------------------|------------------------------------------------------------------------|-----|-----|-----|-----|-----|
| PORT_EV↔<br>ENT_RX↔<br>EOP              | This event occurs when the port receives an End of Packet (EOP).       | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_RX↔<br>EEP              | This event occurs when the port receives an Error End of Packet (EEP). | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_TX↔<br>SOP              | This event occurs when the port transmits an SOP.                      | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_TX↔<br>EOP              | This event occurs when the port transmits an EOP.                      | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_TX↔<br>PKT_PEN↔<br>DING | This event is active when a port has a packet queued for transmission. | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_RUN↔<br>NING            | This event is active when the link attached to a port is running.      | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_PAR↔<br>ITY_ERR↔<br>OR  | This event occurs when a port detects a parity error.                  | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_ESC↔<br>APE_ERR↔<br>OR  | This event occurs when a port detects an escape error.                 | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_CRE↔<br>DIT_ERR↔<br>OR  | This event occurs when a port detects a credit error.                  | Yes | Yes | Yes | Yes | Yes |

|                                 |                                                                 |     |     |     |     |     |
|---------------------------------|-----------------------------------------------------------------|-----|-----|-----|-----|-----|
| PORT_EV↔<br>ENT_DIS↔<br>CONNECT | This event occurs when the link attached to a port disconnects. | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_RX↔<br>TIMECODE | This event occurs when a port receives a time-code.             | Yes | Yes | Yes | Yes | Yes |
| PORT_EV↔<br>ENT_TX↔<br>TIMECODE | This event occurs when a port transmits a time-code.            | Yes | Yes | Yes | No  | Yes |

### Time-Code Interface Input Events

| Input Event                    | Description                                                                     | Supported Devices            |               |            |      |          |
|--------------------------------|---------------------------------------------------------------------------------|------------------------------|---------------|------------|------|----------|
|                                |                                                                                 | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |
| TIME_CO↔<br>DE_EVEN↔<br>T_TICK | This event occurs when a time-code interface receives its next valid time-code. | Yes                          | Yes           | Yes        | No   | Yes      |

### 8.67.3 Output Actions

An output action is an action that is performed by a triggering resource. The actions are caused by the setting of an internal trigger, if configured to do so. Each triggering resource has one or more actions that can be performed. For example, a SpaceWire port can inject errors or transmit packets, if configured in packet transmit mode.

The following tables provide a description of the output actions for each triggering resource.

#### External Trigger Output Actions

| Output Action                         | Description                                                | Supported Devices            |               |            |      |          |
|---------------------------------------|------------------------------------------------------------|------------------------------|---------------|------------|------|----------|
|                                       |                                                            | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |
| EXT_TRI↔<br>GGER_A↔<br>CTION_O↔<br>UT | This action causes an external trigger to output a signal. | Yes                          | Yes           | No         | No   | Yes      |

## Counter Output Actions

| Output Action                   | Description                                                                | Supported Devices            |               |            |      |          |  |
|---------------------------------|----------------------------------------------------------------------------|------------------------------|---------------|------------|------|----------|--|
|                                 |                                                                            | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |  |
| COUNT↔<br>R_ACTION↔<br>N_COUNT  | This action causes a counter to decrement by one.                          | Yes                          | Yes           | Yes        | Yes  | Yes      |  |
| COUNT↔<br>R_ACTION↔<br>N_RELOAD | This action causes a counter to be set to its configured reload value.     | Yes                          | Yes           | Yes        | Yes  | Yes      |  |
| COUNT↔<br>R_ACTION↔<br>N_START  | This action causes a counter to be start, if start mode is enabled.        | Yes                          | Yes           | Yes        | Yes  | Yes      |  |
| COUNT↔<br>R_ACTION↔<br>N_STOP   | This action causes a counter to be stopped, if start-stop mode is enabled. | Yes                          | Yes           | Yes        | Yes  | Yes      |  |

### SpaceWire Port Output Actions

| Output Action                           | Description                                                                                             | Supported Devices            |               |            |      |          |  |
|-----------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------|---------------|------------|------|----------|--|
|                                         |                                                                                                         | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCle | PCle Mk2 |  |
| PORT_AC↔<br>TION_TR↔<br>ANSMIT_↔<br>PKT | This action causes a port to transmit a queued packet, if transmit packet mode is enabled for the port. | Yes                          | Yes           | Yes        | Yes  | Yes      |  |

|                              |                                                                         |     |     |     |     |     |
|------------------------------|-------------------------------------------------------------------------|-----|-----|-----|-----|-----|
| PORT_ACTION_DISCONNECT       | This action causes the link attached to a port to disconnect.           | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_PARITY_ERROR     | This action causes a port to inject a parity error.                     | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_ESCAPE_ERROR     | This action causes a port to inject an escape error.                    | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_INSERT_FCT       | This action causes a port to insert a Flow Control Token (FCT).         | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_SUPPRESS_FCT     | This action causes a port to suppress the next FCT to be transmitted.   | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_INCREMENT_CREDIT | This action causes a port to increment its credit.                      | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_DECREMENT_CREDIT | This action causes a port to decrement its credit.                      | Yes | Yes | Yes | Yes | Yes |
| PORT_ACTION_STOP_RECEPTION   | This action causes a port to stop receiving whilst it is set.           | Yes | No  | No  | No  | Yes |
| PORT_ACTION_DISABLE_LVDS     | This action causes a port to disable its LVDS drivers whilst it is set. | Yes | No  | No  | No  | Yes |

#### Time-Code Interface Output Actions

| Output Action | Description | Supported Devices |  |
|---------------|-------------|-------------------|--|
|---------------|-------------|-------------------|--|

|                     |                                                                                | Brick Mk3/Mk4, GbE Brick Mk1 | PXI Interface | PXI Router | PCIe | PCIe Mk2 |
|---------------------|--------------------------------------------------------------------------------|------------------------------|---------------|------------|------|----------|
| TIME_CODE_ACTION_TX | This action causes a time-code interface to transmit its next valid time-code. | Yes                          | Yes           | Yes        | No   | Yes      |

### 8.67.4 Internal Triggers

Internal triggers are used to control which input events cause output actions. An internal trigger can receive one or more input events which cause it to be set. Once set, the internal trigger causes one or more output actions.

An internal trigger is configured by taking a mask, for each triggering resource, that describes which events will cause it to be set. The internal trigger is then configured to operate in AND or OR mode. If operating in AND mode, the internal trigger will be set when all of its input events occur. If operating in OR mode, the internal trigger will be set when any of its input events occur. To control which output actions that the internal trigger will cause, it takes a mask, for each triggering resource, that describes which output actions will be performed when it is set.

The following diagram illustrates the input events, internal triggers, and output actions architecture.

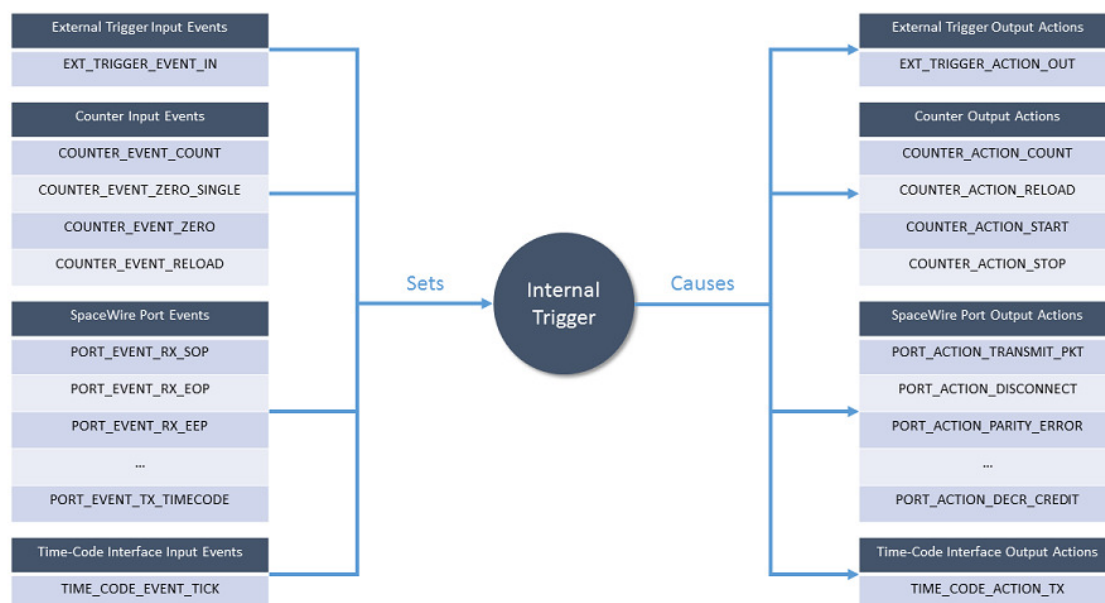


Figure 8.3: Triggering API Architecture

The diagram above shows that an internal trigger is set by one or more input events. Once set, the internal trigger causes one or more output actions.

The following diagram shows an example of a triggering configuration.

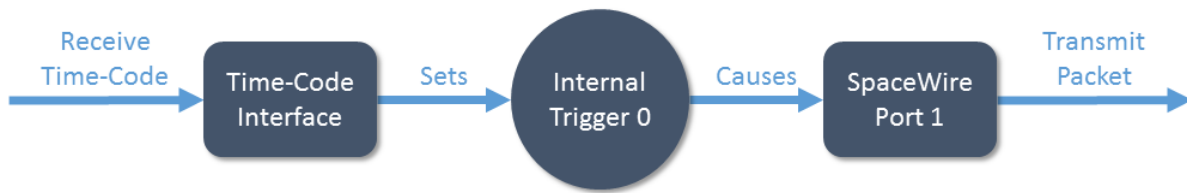


Figure 8.4: Triggering API Example

In this example, a time-code received event on the time-code interface is configured to set internal trigger 0. Once set, internal trigger 0 causes a packet to be transmitted on SpaceWire port 1. The code for this example, using a Brick Mk3, is shown below.

- ~~~{.c} /\* Enable port transmit packet mode on port 1 \*/ TRIGGER\_BRICK\_MK3\_enablePortTransmit↵  
PacketMode(deviceId, 1);  
  
/\* TIME\_CODE\_EVENT\_TICK event on the time-code interface causes internal trigger 0 to be set \*/ TRIG↵  
GER\_BRICK\_MK3\_setTimeCodeInputEvents(deviceId, 0, 0, TIME\_CODE\_EVENT\_TICK);  
  
/\* Internal trigger 0 causes PORT\_ACTION\_TRANSMIT\_PKT action on port 1 \*/ TRIGGER\_BRICK\_MK3↵  
\_setPortOutputActions(deviceId, 1, 0, PORT\_ACTION\_TRANSMIT\_PKT);  
  
/\* Enable internal trigger 0 \*/ TRIGGER\_BRICK\_MK3\_enableTrigger(deviceId, 0);
- ~~~

In the code listed above, SpaceWire port 1 is first configured to have "Port Transmit Mode" enabled. This means that any packets submitted for transmission on the port by STAR-System will be held in a buffer until explicitly instructed to transmit by an action. In the second line, internal trigger 0 is configured to be set by the TIME\_CODE\_EVENT\_TICK event from the time-code interface. Next, internal trigger 0 is configured to cause the PORT\_ACTION\_TRANSMIT\_PKT action on SpaceWire port 1 when set. Finally, internal trigger 0 is enabled to allow it to receive events and cause actions.

### 8.67.5 Triggering Resources

There are five types of triggering resource: external triggers, counters, SpaceWire ports, time-code interfaces and internal triggers. Each device that is supported by the Triggering API has zero or more of each of these resources. The following table lists the number of triggering resources in each supported device.

#### Triggering Resources

| Type             | Description                                                                             | Number of Resources |               |            |                       |          |
|------------------|-----------------------------------------------------------------------------------------|---------------------|---------------|------------|-----------------------|----------|
|                  |                                                                                         | Brick Mk3           | PXI Interface | PXI Router | PCIe (see note below) | PCIe Mk2 |
| External Trigger | An external trigger can receive signals causing events and transmit signals as actions. | 2                   | 4             | 0          | 0                     | 2        |

|                     |                                                                                                                                               |   |   |    |   |   |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|---|---|----|---|---|
| Counter             | A counter can operate in free-running mode as a timer or in triggered counting mode as a counter.                                             | 4 | 4 | 4  | 6 | 4 |
| SpaceWire Port      | A SpaceWire port can cause events when it receives characters or errors and perform actions such as transmitting packets or injecting errors. | 2 | 4 | 12 | 3 | 3 |
| Time-Code Interface | A time-code interface can cause events by receiving valid time-codes and transmit valid time-codes as actions.                                | 1 | 1 | 1  | 0 | 1 |
| Internal Trigger    | An internal trigger is set by receiving one or more input events. When set, it causes actions to be performed on other triggering resources.  | 8 | 8 | 4  | 6 | 8 |

Note that the number of resources supported by PCIe devices was increased in version 1.11e06. Previous versions support only 3 counters and internal triggers.

The external trigger, counter, SpaceWire port and internal trigger triggering resources have various configuration settings. The following tables describe the configuration options for each of these triggering resources.

#### External Trigger Configuration

| Setting | Description |
|---------|-------------|
|---------|-------------|



|                  |                                                                                                                                |
|------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Edge Detect Mode | When enabled, this setting causes the input event to occur on the rising edge of the input signal.                             |
| Invert           | When enabled, this setting inverts the output signal and inverts the input signal before it reaches the detection mechanism.   |
| Output           | When enabled, the output action causes the external trigger to output a signal for the duration specified by the Extend value. |
| Extend Value     | The extend value is the duration, in clock cycles, of the external trigger's output signal caused by an output action.         |

### Counter Configuration

| Setting         | Description                                                                                                                                                                                                                                        |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Auto Reload     | When enabled, this setting causes the counter to be automatically reloaded with its configured reload value when the counter reaches zero.                                                                                                         |
| Trigger Count   | When enabled, the counter must explicitly receive a count action before it is decremented. This allows the counter to operate as a counter rather than a free-running timer.                                                                       |
| Start Mode      | When enabled, the counter will wait for a start action before any counting or reloading can take place. When the counter reaches zero, it may be reloaded but no further counting or reloading will take place until the next start action occurs. |
| Start Stop Mode | When enabled, the counter will wait for a start action before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.                                                                  |
| Load Zero       | When enabled, reload triggered events will only occur when the counter value is zero.                                                                                                                                                              |
| Reload Value    | The reload value is the value that the counter will be set to when it is reloaded.                                                                                                                                                                 |
| Force Reload    | The force reload setting causes the counter to be reloaded.                                                                                                                                                                                        |

### SpaceWire Port Configuration

| Setting              | Description                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Transmit Packet Mode | When enabled, packets are queued for transmission on the SpaceWire port. If a packet transmit action occurs, a single packet will be transmitted from the queue. |

### Internal Trigger Configuration

| Setting    | Description                                                                                                                                                                                                                                                |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input Mode | An internal trigger can be configured to operate in AND or OR mode. When operating in AND mode, the internal trigger is set when all of its input events occur. When operating in OR mode, the internal trigger is set when any of its input events occur. |

|               |                                                                                                                                                  |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| Enabled       | When enabled, the internal trigger can be set by its input events and cause its output actions to occur.                                         |
| Force Trigger | The force trigger setting causes the internal trigger to be set.                                                                                 |
| Invert Mask   | The invert mask determines, for each triggering resource, if the input events should be inverted before reaching the internal trigger.           |
| AND Mask      | The AND Mask determines, for each triggering resource, if the input events should be checked when the internal trigger is operating in AND mode. |

## 8.67.6 PCIe Differences

The PCIe device's triggering support has several differences compared to the Brick Mk3/Mk4, GbE Brick Mk1, PXI devices and the PCIe Mk2 device. These differences are listed below.

- There is no time-code interface so the functions to trigger on valid time-code events and perform time-code transmit actions are not supported. However, the SpaceWire port time-code receive event is still supported so it can be used to trigger on time-codes.
- The SpaceWire port time-code transmit event is not supported by the PCIe device.
- There are no explicit packet transmit mode enable/disable functions. Rather, if any packet transmit action is enabled on a port, packet transmit mode is enabled implicitly.

## 8.67.7 Examples

The following sections describe several examples showing how to use the Triggering API.

### 8.67.7.1 Transmitting Time-Codes Synchronised by an External Trigger

In this example, the triggers of a Brick Mk3 will be configured so that time-codes are transmitted by the Brick Mk3 every time the rising edge of an external trigger input signal is detected. The code for this example is listed below.

- ```
~~~~{.c} /* Enable external trigger 0 edge detect mode */ TRIGGER_BRICK_MK3_enableExtTriggerEdge↵
DetectMode(deviceId, 0);

/* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal trigger 0 to be set */ TRIGGER↵
_BRICK_MK3_setExtTriggerInputEvents(deviceId, 0, 0, EXT_TRIGGER_EVENT_IN);

/* Internal trigger 0 causes TIME_CODE_ACTION_TX action on the time-code interface */ TRIGGER_BR↵
ICK_MK3_setTimeCodeOutputActions(deviceId, 0, 0, TIME_CODE_ACTION_TX);

/* Enable internal trigger 0 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
```

- ~~~~

In the code listed above, external trigger 0 is first configured to enable edge detect mode. This means that the EXT\_TRIGGER\_EVENT\_IN event will occur once, on the rising edge of the input signal. Otherwise, the event would be active for as long as the input signal is high (unless the invert setting is enabled, in which case the event would be active for as long as the input signal is low).

Next, internal trigger 0 is configured so that the EXT\_TRIGGER\_EVENT\_IN event from external trigger 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the TIME\_CODE\_ACTI↵
ON\_TX action on the time-code interface. Finally, internal trigger 0 is enabled to allow the event to cause the action.

### 8.67.7.2 Packet Transmission Synchronised by a Counter

In this example, the triggers of a Brick Mk3 will be configured so that packets submitted for transmission by STAR-System to port 1 will be queued and transmitted every time a free-running counter reloads. The code for this example is listed below.

- `~~~~{.c} /* Set counter 0 reload value to 60 million clock cycles */ TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 60000000);`  
`/* Enable counter auto reload */ TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId, 0);`  
`/* Enable port transmit packet mode on port 1 */ TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(deviceId, 1);`  
`/* COUNTER_EVENT_RELOAD event on counter 0 causes internal trigger 0 to be set */ TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 0, COUNTER_EVENT_RELOAD);`  
`/* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on port 1 */ TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);`  
`/* Enable internal trigger 0 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);`
- `~~~~`

In the code listed above, counter 0 is first configured to set its reload value to 60 million clock cycles and enable its auto-reload setting. SpaceWire port 1 is then configured to enable its transmit packet mode.

Next, internal trigger 0 is configured so that the COUNTER\_EVENT\_RELOAD event from counter 0 will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes the PORT\_ACTION\_TRANSMIT\_PKT action on SpaceWire port 1. Finally, internal trigger 0 is enabled to allow the event to cause the action.

### 8.67.7.3 Multiple Packet Transmission Synchronised by Time-Codes

In this example, the triggers of a Brick Mk3 will be configured so that packets submitted for transmission by STAR-System to port 1 will be queued and transmitted in groups of 4 every time the Brick Mk3 receives a valid time-code. The code for this example is listed below.

- `~~~~{.c} /* Enable counter trigger count on counter 0 so the counter only counts on triggers */ TRIGGER_BRICK_MK3_enableCounterTriggerCount(deviceId, 0);`  
`/* Set counter 0 reload value to 3 */ TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId, 0, 3);`  
`/* Enable port transmit packet mode on port 1 */ TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(deviceId, 1);`  
`/* TIME_CODE_EVENT_TICK event on the time-code interface causes internal trigger 0 to be set */ TRIGGER_BRICK_MK3_setTimeCodeInputEvents(deviceId, 0, 0, TIME_CODE_EVENT_TICK);`  
`/* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on counter 0 */ TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 0, COUNTER_ACTION_RELOAD);`  
`/* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on port 1 */ TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 0, PORT_ACTION_TRANSMIT_PKT);`  
`/* PORT_EVENT_TX_EOP event on port 1 causes internal trigger 1 to be set */ TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1, 1, PORT_EVENT_TX_EOP);`  
`/* Internal trigger 1 causes COUNTER_ACTION_COUNT action on counter 0 */ TRIGGER_BRICK_MK3_setCounterOutputActions(deviceId, 0, 1, COUNTER_ACTION_COUNT);`  
`/* COUNTER_EVENT_COUNT event on counter 0 causes internal trigger 2 to be set */ TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId, 0, 2, COUNTER_EVENT_COUNT);`  
`/* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on port 1 */ TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 1, 2, PORT_ACTION_TRANSMIT_PKT);`  
`/* Enable internal triggers 0, 1 and 2 */ TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0); TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1); TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);`

- ~~~

In the code listed above, counter 0 is configured to enable its trigger count mode. This means that the counter will only decrement when it receives a `COUNTER_ACTION_COUNT` action. Additionally, counter 0 has its reload value set to 3. Auto-reload is not enabled in this case because reloading will be controlled by the triggers. SpaceWire Port 1 is then configured to have its transmit packet mode enabled as described in the previous example.

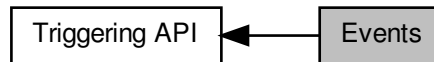
Next, internal trigger 0 is configured so that the `TIME_CODE_EVENT_TICK` event from the time-code interface will cause it to be set. Internal trigger 0 is also configured so that, when set, it causes two actions to be performed: firstly, counter 0 is reloaded using the `COUNTER_ACTION_RELOAD` action, and secondly, the first packet in the group is transmitted on port 1 using the `PORT_ACTION_TRANSMIT_PKT` action. Internal trigger 1 is configured so that the `PORT_EVENT_TX_EOP` event on port 1 will cause it to be set. Internal trigger 1 is also configured so that, when set, it causes the `COUNTER_ACTION_COUNT` action on counter 0. Internal trigger 2 is configured so that the `COUNTER_EVENT_COUNT` event from counter 0 causes it to be set. Internal trigger 2 is also configured so that, when set, it causes the `PORT_ACTION_TRANSMIT_PKT` action on port 1. Finally, internal triggers 0, 1 and 2 are enabled to allow the events to cause the actions.

To summarise this example, when a time-code is received by the Brick Mk3, it reloads the packet counter and transmits the first packet in the group of 4. When the EOP from the packet is transmitted, it decrements the packet counter. When the packet counter is decremented it transmits another packet. The transmitting EOPs continue causing the packet counter to decrement and more packets to be transmitted until the counter reaches 0, at which point the transmission stops. When the next time-code is received, the packet counter is reloaded and the process begins again, transmitting the next group of 4 packets.

## 8.68 Events

Functions in this group deal with setting and getting trigger input events.

Collaboration diagram for Events:



### Typedefs

- typedef U32 EXT\_TRIGGER\_EVENT\_MASK  
*External trigger event mask.*
- typedef U32 EXT\_TRIGGER\_ACTION\_MASK  
*External trigger action mask.*
- typedef U32 COUNTER\_EVENT\_MASK  
*Counter event mask.*
- typedef U32 COUNTER\_ACTION\_MASK  
*Counter action mask.*
- typedef U32 SPW\_PORT\_EVENT\_MASK  
*SpaceWire port event mask.*
- typedef U32 SPFI\_PORT\_EVENT\_MASK  
*SpaceFibre port event mask.*
- typedef U32 SPFI\_VC\_EVENT\_MASK  
*SpaceFibre virtual channel event mask.*
- typedef U32 TIME\_CODE\_EVENT\_MASK  
*Time-code event mask.*
- typedef U32 TIME\_CODE\_ACTION\_MASK  
*Time-code action mask.*
- typedef U32 TRIGGER\_EVENT\_MASK  
*Trigger event mask.*

### Enumerations

- enum EXT\_TRIGGER\_EVENT { EXT\_TRIGGER\_EVENT\_IN = 1 << 0, EXT\_TRIGGER\_EVENT\_NONE = 0, EXT\_TRIGGER\_EVENT\_ALL = 0xf }
- External trigger events.*
- enum COUNTER\_EVENT { COUNTER\_EVENT\_COUNT = 1 << 0, COUNTER\_EVENT\_ZERO\_SINGLE = 1 << 1, COUNTER\_EVENT\_ZERO = 1 << 2, COUNTER\_EVENT\_RELOAD = 1 << 3, COUNTER\_EVENT\_NONE = 0, COUNTER\_EVENT\_ALL = 0xf }
- Counter events.*

- enum SPW\_PORT\_EVENT {  
 SPW\_PORT\_EVENT\_RX\_SOP = PORT\_EVENT\_RX\_SOP, SPW\_PORT\_EVENT\_RX\_EOP = PORT\_EVENT\_RX\_EOP, SPW\_PORT\_EVENT\_RX\_EEP = PORT\_EVENT\_RX\_EEP, SPW\_PORT\_EVENT\_TX\_SOP = PORT\_EVENT\_TX\_SOP,  
 SPW\_PORT\_EVENT\_TX\_EOP = PORT\_EVENT\_TX\_EOP, SPW\_PORT\_EVENT\_TX\_PKT\_PENDING = PORT\_EVENT\_TX\_PKT\_PENDING, SPW\_PORT\_EVENT\_RUNNING = PORT\_EVENT\_RUNNING, SPW\_PORT\_EVENT\_PARITY\_ERROR = PORT\_EVENT\_PARITY\_ERROR,  
 SPW\_PORT\_EVENT\_ESCAPE\_ERROR = PORT\_EVENT\_ESCAPE\_ERROR, SPW\_PORT\_EVENT\_CREDIT\_ERROR = PORT\_EVENT\_CREDIT\_ERROR, SPW\_PORT\_EVENT\_DISCONNECT = PORT\_EVENT\_DISCONNECT, SPW\_PORT\_EVENT\_RX\_TIME\_CODE = PORT\_EVENT\_RX\_TIME\_CODE,  
 SPW\_PORT\_EVENT\_TX\_TIME\_CODE = PORT\_EVENT\_TX\_TIME\_CODE, SPW\_PORT\_EVENT\_NONE = PORT\_EVENT\_NONE, SPW\_PORT\_EVENT\_ALL = PORT\_EVENT\_ALL, SPW\_PORT\_EVENT\_ALL\_PCIE = PORT\_EVENT\_ALL\_PCIE }

*SpaceWire port events.*

- enum SPFI\_PORT\_EVENT {  
 SPFI\_PORT\_EVENT\_CONNECTED = 1 << 0, SPFI\_PORT\_EVENT\_DISCONNECTED = 1 << 1, SPFI\_PORT\_EVENT\_TX\_BM = 1 << 2, SPFI\_PORT\_EVENT\_RX\_BM = 1 << 3,  
 SPFI\_PORT\_EVENT\_NONE = 0, SPFI\_PORT\_EVENT\_ALL = 0xf }

*SpaceFibre port events.*

- enum SPFI\_VC\_EVENT {  
 SPFI\_VC\_EVENT\_RX\_SOP = 1 << 0, SPFI\_VC\_EVENT\_RX\_EOP = 1 << 1, SPFI\_VC\_EVENT\_RX\_EEP = 1 << 2, SPFI\_VC\_EVENT\_TX\_SOP = 1 << 3,  
 SPFI\_VC\_EVENT\_TX\_EOP = 1 << 4, SPFI\_VC\_EVENT\_TX\_PKT\_PENDING = 1 << 5, SPFI\_VC\_EVENT\_NONE = 0, SPFI\_VC\_EVENT\_ALL = 0x3f }

*SpaceFibre virtual channel events.*

- enum TIME\_CODE\_EVENT { TIME\_CODE\_EVENT\_TICK = 1 << 0, TIME\_CODE\_EVENT\_NONE = 0, TIME\_CODE\_EVENT\_ALL = 0x1 }

*Time-code events.*

- enum TRIGGER\_EVENT { TRIGGER\_EVENT\_IN = 1 << 0, TRIGGER\_EVENT\_NONE = 0, TRIGGER\_EVENT\_ALL = 0x1 }

*Internal trigger events.*

## Functions

- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setExtTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setCounterInputEvents (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setPortInputEvents (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setTimeCodeInputEvents (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getExtTriggerInputEvents (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getCounterInputEvents (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_BRICK_MK3_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_setTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)

*Sets the input events for a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PCIE_MK2_getTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*



- `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)`  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)`  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)`  
*Sets the input events for a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`  
*Gets the input events from a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)`  
*Sets the input events for a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`



*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

### 8.68.1 Detailed Description

Functions in this group deal with setting and getting trigger input events.

### 8.68.2 Typedef Documentation

#### 8.68.2.1 typedef U32 COUNTER\_ACTION\_MASK

Counter action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.2 typedef U32 COUNTER\_EVENT\_MASK

Counter event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.3 typedef U32 EXT\_TRIGGER\_ACTION\_MASK

External trigger action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.4 typedef U32 EXT\_TRIGGER\_EVENT\_MASK

External trigger event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.5 typedef U32 SPFI\_PORT\_EVENT\_MASK

SpaceFibre port event mask.

Added in version:

STAR-System v6.01 - 13th November 2025

#### 8.68.2.6 `typedef U32 SPFI_VC_EVENT_MASK`

SpaceFibre virtual channel event mask.

Added in version:

STAR-System v6.01 - 13th November 2025

#### 8.68.2.7 `typedef U32 SPW_PORT_EVENT_MASK`

SpaceWire port event mask.

Added in version:

STAR-System v6.01 - 13th November 2025

#### 8.68.2.8 `typedef U32 TIME_CODE_ACTION_MASK`

Time-code action mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.9 `typedef U32 TIME_CODE_EVENT_MASK`

Time-code event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

#### 8.68.2.10 `typedef U32 TRIGGER_EVENT_MASK`

Trigger event mask.

Added in version:

STAR-System v3.0 - 26th August 2016

### 8.68.3 Enumeration Type Documentation

#### 8.68.3.1 `enum COUNTER_EVENT`

Counter events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**`COUNTER_EVENT_COUNT`** Event occurs when the counter decrements.

**`COUNTER_EVENT_ZERO_SINGLE`** Event occurs when the counter reaches zero (once)

**COUNTER\_EVENT\_ZERO** Event is active while the counter is equal to zero.

**COUNTER\_EVENT\_RELOAD** Event occurs when the counter reloads.

**COUNTER\_EVENT\_NONE** No events.

**COUNTER\_EVENT\_ALL** All events.

### 8.68.3.2 enum EXT\_TRIGGER\_EVENT

External trigger events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**EXT\_TRIGGER\_EVENT\_IN** Event occurs when the external trigger has received input.

**EXT\_TRIGGER\_EVENT\_NONE** No events.

**EXT\_TRIGGER\_EVENT\_ALL** All events.

### 8.68.3.3 enum SPFI\_PORT\_EVENT

SpaceFibre port events.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPFI\_PORT\_EVENT\_CONNECTED** Event occurs when the link moves from not connected to connected.

**SPFI\_PORT\_EVENT\_DISCONNECTED** Event occurs when the link moves from connected to not connected.

**SPFI\_PORT\_EVENT\_TX\_BM** Event occurs when the port transmits a broadcast message.

**SPFI\_PORT\_EVENT\_RX\_BM** Event occurs when the port receives a broadcast message.

**SPFI\_PORT\_EVENT\_NONE** No events.

**SPFI\_PORT\_EVENT\_ALL** All events.

### 8.68.3.4 enum SPFI\_VC\_EVENT

SpaceFibre virtual channel events.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPFI\_VC\_EVENT\_RX\_SOP** Event occurs when the virtual channel receives an SOP.

**SPFI\_VC\_EVENT\_RX\_EOP** Event occurs when the virtual channel receives an EOP.

**SPFI\_VC\_EVENT\_RX\_EEP** Event occurs when the virtual channel receives an EEP.

**SPFI\_VC\_EVENT\_TX\_SOP** Event occurs when the virtual channel transmits an SOP.

**SPFI\_VC\_EVENT\_TX\_EOP** Event occurs when the virtual channel transmits an EOP.

**SPFI\_VC\_EVENT\_TX\_PKT\_PENDING** Event is active when the virtual channel has a packet queued for transmission.

**SPFI\_VC\_EVENT\_NONE** No events.

**SPFI\_VC\_EVENT\_ALL** All events.

### 8.68.3.5 enum SPW\_PORT\_EVENT

SpaceWire port events.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPW\_PORT\_EVENT\_RX\_SOP** Event occurs when the port receives an SOP.  
**SPW\_PORT\_EVENT\_RX\_EOP** Event occurs when the port receives an EOP.  
**SPW\_PORT\_EVENT\_RX\_EEP** Event occurs when the port receives an EEP.  
**SPW\_PORT\_EVENT\_TX\_SOP** Event occurs when the port transmits an SOP.  
**SPW\_PORT\_EVENT\_TX\_EOP** Event occurs when the port transmits an EOP.  
**SPW\_PORT\_EVENT\_TX\_PKT\_PENDING** Event is active when the port has a packet queued for transmission.  
**SPW\_PORT\_EVENT\_RUNNING** Event is active when the link attached to the port is running.  
**SPW\_PORT\_EVENT\_PARITY\_ERROR** Event occurs when the port detects a parity error.  
**SPW\_PORT\_EVENT\_ESCAPE\_ERROR** Event occurs when the port detects an escape error.  
**SPW\_PORT\_EVENT\_CREDIT\_ERROR** Event occurs when the port detects a credit error.  
**SPW\_PORT\_EVENT\_DISCONNECT** Event occurs when the port disconnects.  
**SPW\_PORT\_EVENT\_RX\_TIME\_CODE** Event occurs when the port receives a time-code.  
**SPW\_PORT\_EVENT\_TX\_TIME\_CODE** Event occurs when the port transmits a time-code.  
**SPW\_PORT\_EVENT\_NONE** No events.  
**SPW\_PORT\_EVENT\_ALL** All events.

### 8.68.3.6 enum TIME\_CODE\_EVENT

Time-code events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TIME\_CODE\_EVENT\_TICK** Event occurs when the next valid time-code is received.  
**TIME\_CODE\_EVENT\_NONE** No events.  
**TIME\_CODE\_EVENT\_ALL** All events.

### 8.68.3.7 enum TRIGGER\_EVENT

Internal trigger events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TRIGGER\_EVENT\_IN** Event occurs when the internal trigger has received input.  
**TRIGGER\_EVENT\_NONE** No events.  
**TRIGGER\_EVENT\_ALL** All events.

#### 8.68.4 Function Documentation

8.68.4.1 `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK * pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

8.68.4.2 `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK * pEvents )`

Gets the input events from an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

8.68.4.3 `int STAR_API_CC TRIGGER_BRICK_MK3_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.68.4.4** `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.68.4.5** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

8.68.4.6 `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.



**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**8.68.4.7** `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                 |
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[triggering\\_example.c](#).

**8.68.4.8** `int STAR_API_CC TRIGGER_BRICK_MK3_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[triggering\\_example.c](#).

**8.68.4.9** `int STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[triggering\\_example.c](#).

**8.68.4.10** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

8.68.4.11 **int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_EVENT\_MASK \* *pEvents* )**

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

8.68.4.12 **int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getPortInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *port*, U32 *trigger*, PORT\_EVENT\_MASK \* *pEvents* )**

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.68.4.13** `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.68.4.14** `int STAR_API_CC TRIGGER_PCIE_IF_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

**8.68.4.15** `int STAR_API_CC TRIGGER_PCIE_IF_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.1 - 21st October 2016

#### Supported devices:

SpaceWire PCIe

**8.68.4.16** `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

#### Parameters

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.1 - 21st October 2016

#### Supported devices:

SpaceWire PCIe

**8.68.4.17** `int STAR_API_CC TRIGGER_PCIE_MK2_getCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK * pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

#### Parameters

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

**8.68.4.18** `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK * pEvents )`

Gets the input events from an external trigger which will cause an internal trigger to be set.

#### Parameters

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

**8.68.4.19** `int STAR_API_CC TRIGGER_PCIE_MK2_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

#### Parameters

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.68.4.20** `int STAR_API_CC TRIGGER_PCIE_MK2_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.68.4.21** `int STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.68.4.22 `int STAR_API_CC TRIGGER_PCIE_MK2_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.



**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**8.68.4.23** `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                 |
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[triggering\\_example.c](#).

**8.68.4.24** `int STAR_API_CC TRIGGER_PCIE_MK2_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

8.68.4.25 `int STAR_API_CC TRIGGER_PCIE_MK2_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

8.68.4.26 `int STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.68.4.27 **int STAR\_API\_CC\_TRIGGER\_PXI\_IF\_getCounterInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_EVENT\_MASK \* *pEvents* )**

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

8.68.4.28 **int STAR\_API\_CC\_TRIGGER\_PXI\_IF\_getExtTriggerInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *extTrigger*, U32 *trigger*, EXT\_TRIGGER\_EVENT\_MASK \* *pEvents* )**

Gets the input events from an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                                |
|-------------------|----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                        |
| <i>extTrigger</i> | External trigger to get input events from.                     |
| <i>trigger</i>    | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>    | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.68.4.29** `int STAR_API_CC TRIGGER_PXI_IF_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.68.4.30** `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

8.68.4.31 `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.68.4.32** `int STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events )`

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.68.4.33** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events )`

Sets the input events for an external trigger which will cause an internal trigger to be set.

**Parameters**

|                   |                                                         |
|-------------------|---------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                 |
| <i>extTrigger</i> | External trigger to set input events for.               |
| <i>trigger</i>    | Internal trigger which is affected by the input events. |
| <i>events</i>     | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.68.4.34** `int STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events )`

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.68.4.35** `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events )`

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.68.4.36** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events )`

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.68.4.37** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterInputEvents ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK * pEvents )`

Gets the input events from a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                 |
| <i>counter</i>  | Counter to get input events from.                              |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2



**8.68.4.38** `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortInputEvents ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK * pEvents )`

Gets the input events from a port which will cause an internal trigger to be set.

#### Parameters

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                    |
| <i>port</i>     | Port to get input events from.                                 |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.68.4.39** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeInputEvents ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK * pEvents )`

Gets the input events from a time-code engine which will cause an internal trigger to be set.

#### Parameters

|                 |                                                                |
|-----------------|----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                        |
| <i>timeCode</i> | Time-code engine to get input events from.                     |
| <i>trigger</i>  | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>  | Pointer to a value which will be updated with the events mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.68.4.40** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputEvents ( STAR_DEVICE_ID deviceld, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK * pEvents )`

Gets the input events for an internal trigger which will cause another internal trigger to be set.

#### Parameters

|                     |                                                                |
|---------------------|----------------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                        |
| <i>causeTrigger</i> | Internal trigger to get input events from.                     |
| <i>trigger</i>      | Internal trigger which is affected by the input events.        |
| <i>pEvents</i>      | Pointer to a value which will be updated with the events mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.68.4.41 **int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setCounterInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_EVENT\_MASK *events* )**

Sets the input events for a counter which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                          |
| <i>counter</i>  | Counter to set input events for.                        |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

8.68.4.42 **int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setPortInputEvents ( STAR\_DEVICE\_ID *deviceld*, U32 *port*, U32 *trigger*, PORT\_EVENT\_MASK *events* )**

Sets the input events for a port which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                             |
| <i>port</i>     | Port to set input events for.                           |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.68.4.43** int **STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setTimeCodeInputEvents** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *timeCode*, **U32** *trigger*, **TIME\_CODE\_EVENT\_MASK** *events* )

Sets the input events for a time-code engine which will cause an internal trigger to be set.

**Parameters**

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                 |
| <i>timeCode</i> | Time-code engine to set input events for.               |
| <i>trigger</i>  | Internal trigger which is affected by the input events. |
| <i>events</i>   | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.68.4.44** int **STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setTriggerInputEvents** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *causeTrigger*, **U32** *trigger*, **TRIGGER\_EVENT\_MASK** *events* )

Sets the input events for an internal trigger which will cause another internal trigger to be set.

**Parameters**

|                     |                                                         |
|---------------------|---------------------------------------------------------|
| <i>deviceld</i>     | Device containing the internal trigger.                 |
| <i>causeTrigger</i> | Internal trigger to set input events for.               |
| <i>trigger</i>      | Internal trigger which is affected by the input events. |
| <i>events</i>       | Mask describing the input events.                       |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

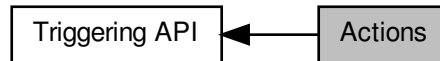
**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

## 8.69 Actions

Functions in this group deal with setting and getting trigger output actions.

Collaboration diagram for Actions:



### Typedefs

- typedef `U32 SPW_PORT_ACTION_MASK`  
*SpaceWire port action mask.*
- typedef `U32 SPFI_PORT_ACTION_MASK`  
*SpaceFibre port action mask.*
- typedef `U32 SPFI_VC_ACTION_MASK`  
*SpaceFibre virtual channel action mask.*

### Enumerations

- enum `EXT_TRIGGER_ACTION` { `EXT_TRIGGER_ACTION_OUT` = 1 << 0, `EXT_TRIGGER_ACTION_NONE` = 0, `EXT_TRIGGER_ACTION_ALL` = 0x1 }  
*External trigger actions.*
- enum `COUNTER_ACTION` { `COUNTER_ACTION_COUNT` = 1 << 0, `COUNTER_ACTION_RELOAD` = 1 << 1, `COUNTER_ACTION_START` = 1 << 2, `COUNTER_ACTION_STOP` = 1 << 3, `COUNTER_ACTION_NONE` = 0, `COUNTER_ACTION_ALL` = 0xf }
- enum `SPW_PORT_ACTION` { `SPW_PORT_ACTION_TRANSMIT_PKT` = `PORT_ACTION_TRANSMIT_PKT`, `SPW_PORT_ACTION_DISCONNECT` = `PORT_ACTION_DISCONNECT`, `SPW_PORT_ACTION_PARITY_ERROR` = `PORT_ACTION_ON_PARITY_ERROR`, `SPW_PORT_ACTION_ESCAPE_ERROR` = `PORT_ACTION_ESCAPE_ERROR`, `SPW_PORT_ACTION_INSERT_FCT` = `PORT_ACTION_INSERT_FCT`, `SPW_PORT_ACTION_SUPPRESS_FCT` = `PORT_ACTION_SUPPRESS_FCT`, `SPW_PORT_ACTION_INCR_CREDIT` = `PORT_ACTION_INCR_CREDIT`, `SPW_PORT_ACTION_DECR_CREDIT` = `PORT_ACTION_DECR_CREDIT`, `SPW_PORT_ACTION_STOP_RECEPTION` = `PORT_ACTION_STOP_RECEPTION`, `SPW_PORT_ACTION_DISABLE_LVDS` = `PORT_ACTION_DISABLE_LVDS`, `SPW_PORT_ACTION_NONE` = `PORT_ACTION_NONE`, `SPW_PORT_ACTION_ALL` = `PORT_ACTION_ALL`, `SPW_PORT_ACTION_ALL_PCIE` = `PORT_ACTION_ALL_PCIE` }  
*SpaceWire port actions.*
- enum `SPFI_PORT_ACTION` { `SPFI_PORT_ACTION_DISCONNECT` = 1 << 0, `SPFI_PORT_ACTION_NONE` = 0, `SPFI_PORT_ACTION_ALL` = 0x1 }  
*SpaceFibre port actions.*
- enum `SPFI_VC_ACTION` { `SPFI_VC_ACTION_TRANSMIT_PKT` = 1 << 0, `SPFI_VC_ACTION_TRANSMIT_DATA` = 1 << 1, `SPFI_VC_ACTION_RECEIVE_DATA` = 1 << 2, `SPFI_VC_ACTION_NONE` = 0, `SPFI_VC_ACTION_ALL` = 0x7 }

*SpaceFibre virtual channel actions.*

- enum `TIME_CODE_ACTION` { `TIME_CODE_ACTION_TX` = 1 << 0, `TIME_CODE_ACTION_NONE` = 0, `TIME_CODE_ACTION_ALL` = 0x1 }

*Time-code actions.*

## Functions

- int `STAR_API_CC_TRIGGER_BRICK_MK3_setExtTriggerOutputActions` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 trigger, `EXT_TRIGGER_ACTION_MASK` actions)

*Sets the output actions for an external trigger that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_setCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_setPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTimeCodeOutputActions` (`STAR_DEVICE_ID` deviceId, U32 timeCode, U32 trigger, `TIME_CODE_ACTION_MASK` actions)

*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_getExtTriggerOutputActions` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 trigger, `EXT_TRIGGER_ACTION_MASK` \*pActions)

*Gets the output actions from an external trigger that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_getCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_getPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTimeCodeOutputActions` (`STAR_DEVICE_ID` deviceId, U32 timeCode, U32 trigger, `TIME_CODE_ACTION_MASK` \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_IF_setCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_IF_setPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_IF_getPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setPortOutputActions` (`STAR_DEVICE_ID` deviceId, U32 port, U32 trigger, `PORT_ACTION_MASK` actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterOutputActions` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 trigger, `COUNTER_ACTION_MASK` \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- `int STAR_API_CC TRIGGER_PCIE_MK2_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

### 8.69.1 Detailed Description

Functions in this group deal with setting and getting trigger output actions.

### 8.69.2 Typedef Documentation

#### 8.69.2.1 typedef U32 SPFI\_PORT\_ACTION\_MASK

SpaceFibre port action mask.

Added in version:

STAR-System v6.01 - 13th November 2025

#### 8.69.2.2 typedef U32 SPFI\_VC\_ACTION\_MASK

SpaceFibre virtual channel action mask.

Added in version:

STAR-System v6.01 - 13th November 2025

#### 8.69.2.3 typedef U32 SPW\_PORT\_ACTION\_MASK

SpaceWire port action mask.

Added in version:

STAR-System v6.01 - 13th November 2025

### 8.69.3 Enumeration Type Documentation

#### 8.69.3.1 enum COUNTER\_ACTION

Counter actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**COUNTER\_ACTION\_COUNT** Decrement the counter.

**COUNTER\_ACTION\_RELOAD** Reload the counter.

**COUNTER\_ACTION\_START** Start the counter.

**COUNTER\_ACTION\_STOP** Stop the counter.

**COUNTER\_ACTION\_NONE** No actions.

**COUNTER\_ACTION\_ALL** All actions.

### 8.69.3.2 enum EXT\_TRIGGER\_ACTION

External trigger actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**EXT\_TRIGGER\_ACTION\_OUT** Output the external trigger.

**EXT\_TRIGGER\_ACTION\_NONE** No actions.

**EXT\_TRIGGER\_ACTION\_ALL** All actions.

### 8.69.3.3 enum SPFI\_PORT\_ACTION

SpaceFibre port actions.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPFI\_PORT\_ACTION\_DISCONNECT** Disconnect the link.

**SPFI\_PORT\_ACTION\_NONE** No actions.

**SPFI\_PORT\_ACTION\_ALL** All actions.

### 8.69.3.4 enum SPFI\_VC\_ACTION

SpaceFibre virtual channel actions.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPFI\_VC\_ACTION\_TRANSMIT\_PKT** Transmit a pending packet.

**SPFI\_VC\_ACTION\_TRANSMIT\_DATA** Transmit pending data.

**SPFI\_VC\_ACTION\_RECEIVE\_DATA** Receive pending data.

**SPFI\_VC\_ACTION\_NONE** No actions.

**SPFI\_VC\_ACTION\_ALL** All actions.

### 8.69.3.5 enum SPW\_PORT\_ACTION

SpaceWire port actions.

Added in version:

STAR-System v6.01 - 13th November 2025

Enumerator

**SPW\_PORT\_ACTION\_TRANSMIT\_PKT** Transmit a pending packet.



**SPW\_PORT\_ACTION\_DISCONNECT** Disconnect the SpaceWire port.

**SPW\_PORT\_ACTION\_PARITY\_ERROR** Inject a parity error.

**SPW\_PORT\_ACTION\_ESCAPE\_ERROR** Inject an escape error.

**SPW\_PORT\_ACTION\_INSERT\_FCT** Insert an FCT.

**SPW\_PORT\_ACTION\_SUPPRESS\_FCT** Suppress the next FCT to be transmitted.

**SPW\_PORT\_ACTION\_INCR\_CREDIT** Increment credit.

**SPW\_PORT\_ACTION\_DECR\_CREDIT** Decrement credit.

**SPW\_PORT\_ACTION\_STOP\_RECEPTION** Stop reception.

Supported devices:

SpaceWire Brick Mk3 and Brick Mk4 (version 1.03 or greater).

**SPW\_PORT\_ACTION\_DISABLE\_LVDS** Disable LVDS drivers.

Supported devices:

SpaceWire Brick Mk3 and Brick Mk4 (version 1.05 or greater), SpaceWire PXI Interface and PXI Interface Mk2 (v1.05 or later), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 (v1.05 or later).

**SPW\_PORT\_ACTION\_NONE** No actions.

**SPW\_PORT\_ACTION\_ALL** All actions.

#### 8.69.3.6 enum TIME\_CODE\_ACTION

Time-code actions.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TIME\_CODE\_ACTION\_TX** Transmit a time-code.

**TIME\_CODE\_ACTION\_NONE** No actions.

**TIME\_CODE\_ACTION\_ALL** All actions.

### 8.69.4 Function Documentation

**8.69.4.1** int **STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getCounterOutputActions** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** *trigger*, **COUNTER\_ACTION\_MASK** \* *pActions* )

Gets the output actions from a counter that are caused by an internal trigger being set.

Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.0 - 26th August 2016

Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.69.4.2** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK * pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

#### Parameters

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.69.4.3** `int STAR_API_CC TRIGGER_BRICK_MK3_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

#### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.69.4.4** `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK * pActions )`

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

#### Parameters

---

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.69.4.5** `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

**8.69.4.6** `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set output actions for. |

---

|                |                                                   |
|----------------|---------------------------------------------------|
| <i>trigger</i> | Internal trigger which causes the output actions. |
| <i>actions</i> | Mask describing the output actions                |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.69.4.7** `int STAR_API_CC TRIGGER_BRICK_MK3_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**


---

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

**8.69.4.8** `int STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**


---

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

**8.69.4.9** `int STAR_API_CC TRIGGER_PCIE_IF_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.69.4.10** `int STAR_API_CC TRIGGER_PCIE_IF_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the port.      |
| <i>port</i>     | Port to get output actions from. |

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.69.4.11** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_setCounterOutputActions** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** *trigger*, **COUNTER\_ACTION\_MASK** *actions* )

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.69.4.12** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_setPortOutputActions** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *port*, **U32** *trigger*, **PORT\_ACTION\_MASK** *actions* )

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

Examples:

triggering\_example.c.

**8.69.4.13** `int STAR_API_CC TRIGGER_PCIE_MK2_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

#### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

**8.69.4.14** `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK * pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

#### Parameters

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

**8.69.4.15** `int STAR_API_CC TRIGGER_PCIE_MK2_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.69.4.16 **int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_getTimeCodeOutputActions ( STAR\_DEVICE\_ID *deviceld*, U32 *timeCode*, U32 *trigger*, TIME\_CODE\_ACTION\_MASK \* *pActions* )**

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.69.4.17 **int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_setCounterOutputActions ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *trigger*, COUNTER\_ACTION\_MASK *actions* )**

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |



**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[triggering\\_example.c](#).

**8.69.4.18** `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.           |
| <i>extTrigger</i> | External trigger to set output actions for.       |
| <i>trigger</i>    | Internal trigger which causes the output actions. |
| <i>actions</i>    | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**8.69.4.19** `int STAR_API_CC TRIGGER_PCIE_MK2_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

8.69.4.20 `int STAR_API_CC TRIGGER_PCIE_MK2_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceId</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

8.69.4.21 `int STAR_API_CC TRIGGER_PXI_IF_getCounterOutputActions ( STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceId</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.69.4.22** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK * pActions )`

Gets the output actions from an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                                 |
|-------------------|-----------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                         |
| <i>extTrigger</i> | External trigger to get output actions from.                    |
| <i>trigger</i>    | Internal trigger which causes the output actions.               |
| <i>pActions</i>   | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.69.4.23** `int STAR_API_CC TRIGGER_PXI_IF_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

8.69.4.24 `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK * pActions )`

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.69.4.25** `int STAR_API_CC TRIGGER_PXI_IF_setCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.69.4.26** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerOutputActions ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions )`

Sets the output actions for an external trigger that are caused by an internal trigger being set.

**Parameters**

|                   |                                                   |
|-------------------|---------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.           |
| <i>extTrigger</i> | External trigger to set output actions for.       |
| <i>trigger</i>    | Internal trigger which causes the output actions. |
| <i>actions</i>    | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.69.4.27** `int STAR_API_CC TRIGGER_PXI_IF_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.69.4.28** `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions.               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.69.4.29** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK * pActions )`

Gets the output actions from a counter that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                  |
| <i>counter</i>  | Counter to get output actions from.                             |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.69.4.30** `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK * pActions )`

Gets the output actions from a port that are caused by an internal trigger being set.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                     |
| <i>port</i>     | Port to get output actions from.                                |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.69.4.31 `int STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK * pActions )`

Gets the output actions from a time-code engine that are caused by an internal trigger being set.

#### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.                         |
| <i>timeCode</i> | Time-code engine to get output actions from.                    |
| <i>trigger</i>  | Internal trigger which causes the output actions.               |
| <i>pActions</i> | Pointer to a value which will be updated with the actions mask. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.69.4.32 `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterOutputActions ( STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions )`

Sets the output actions for a counter that are caused by an internal trigger being set.

#### Parameters

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                    |
| <i>counter</i>  | Counter to set output actions for.                |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.5 - 17th March 2017

#### Supported devices:

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

#### Examples:

`triggering_example.c`.

8.69.4.33 `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions ( STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions )`

Sets the output actions for a port that are caused by an internal trigger being set.



**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                       |
| <i>port</i>     | Port to set output actions for.                   |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**8.69.4.34** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeOutputActions ( STAR_DEVICE_ID deviceld, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions )`

Sets the output actions for a time-code engine that are caused by an internal trigger being set.

**Parameters**

|                 |                                                   |
|-----------------|---------------------------------------------------|
| <i>deviceld</i> | Device containing the time-code engine.           |
| <i>timeCode</i> | Time-code engine to set output actions for.       |
| <i>trigger</i>  | Internal trigger which causes the output actions. |
| <i>actions</i>  | Mask describing the output actions                |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

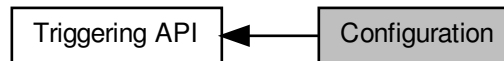
**Examples:**

[triggering\\_example.c](#).

## 8.70 Configuration

Functions in this group deal with configuring the different triggering subsystems.

Collaboration diagram for Configuration:



### Enumerations

- enum `TRIGGER_INPUT_MODE` { `TRIGGER_INPUT_MODE_OR` = 0, `TRIGGER_INPUT_MODE_AND` = 1 }  
*Internal trigger input modes.*

### Functions

- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether invert is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Enables output for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput` (`STAR_DEVICE_ID` deviceId, U32 extTrigger)  
*Disables output for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether output is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerExtend` (`STAR_DEVICE_ID` deviceId, U32 extTrigger, U32 extend)  
*Sets the extend duration for a specific external trigger.*

- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether trigger count is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start stop mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether load zero is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)`  
*Sets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)`

- Gets the reload value for a specific counter.*

  - int `STAR_API_CC TRIGGER_BRICK_MK3_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables spilling for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables spilling for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getPortSpillEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether spilling is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)

*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)

*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_BRICK_MK3_reset` (STAR\_DEVICE\_ID deviceId)

*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_BRICK_MK3_getTriggerMatrix` ()

- Retrieves the trigger matrix of the Brick Mk3 device.*

  - int `STAR_API_CC TRIGGER_BRICK_MK3_getDeviceClockRate ()`

*Returns the clock rate used by the triggering system of the Brick Mk3 device.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables auto reload for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables auto reload for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

*Gets whether auto reload is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterTriggerCountEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

*Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartStopModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterLoadZeroEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`

*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)`

*Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)`

*Gets the reload value for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TR↔IGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TR↔IGGER\_TYPE type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_reset` (STAR\_DEVICE\_ID deviceId)  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC_TRIGGER_PCIE_getTriggerMatrix` ()  
*Retrieves the trigger matrix of the PCIe device.*
- int `STAR_API_CC_TRIGGER_PCIE_IF_getDeviceClockRate` ()  
*Returns the clock rate used by the triggering system of the PCIe device.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_reset` (STAR\_DEVICE\_ID deviceId)  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC_TRIGGER_PCIE_MK2_getTriggerMatrix` ()  
*Retrieves the trigger matrix of the PCIe MK2 device.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getDeviceClockRate` ()  
*Returns the clock rate used by the triggering system of the PCIe MK2 device.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables trigger count for a specific counter.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

- Gets whether trigger count is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables load zero for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables load zero for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether load zero is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)

*Sets the reload value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)

*Gets the reload value for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)

*Sets the input mode for a specific internal trigger.*

  - int `STAR_API_CC_TRIGGER_PCIE_MK2_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)



- Gets the input mode for a specific internal trigger.*

  - int `STAR_API_CC TRIGGER_PCIE_MK2_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerEdgeDetectModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether edge detect mode is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerInvertEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether invert is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables output for a specific external trigger.*



- `int STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 ext↔Trigger)`  
*Disables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerOutputEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether output is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 ext↔Trigger, U32 extend)`  
*Sets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 ext↔Trigger, U32 *pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_enablePortSpill (STAR_DEVICE_ID deviceId, U32 port)`  
*Enables spilling for a specific port.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_disablePortSpill (STAR_DEVICE_ID deviceId, U32 port)`  
*Disables spilling for a specific port.*
- `int STAR_API_CC TRIGGER_PCIE_MK2_getPortSpillEnabled (STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)`  
*Gets whether spilling is enabled for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInvertEnabled (STAR_DEVICE_ID deviceId, U32 ext↔Trigger, U32 *pEnabled)`  
*Gets whether invert is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 ext↔Trigger)`  
*Disables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputEnabled (STAR_DEVICE_ID deviceId, U32 ext↔Trigger, U32 *pEnabled)`  
*Gets whether output is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 extend)`  
*Sets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerExtend (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*

- `int STAR_API_CC TRIGGER_PXI_IF_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterTriggerCountEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether trigger count is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableCounterStartMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start mode for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterStartModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableCounterStartStopMode (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables start stop mode for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterStartStopModeEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether start stop mode is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableCounterLoadZero (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables load zero for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterLoadZeroEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether load zero is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 reloadValue)`  
*Sets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterReloadValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pReloadValue)`  
*Gets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_forceCounterReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Force the reload of a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterValue (STAR_DEVICE_ID deviceId, U32 counter, U32 *pValue)`  
*Gets the counter value for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_IF_enablePortTransmitPacketMode (STAR_DEVICE_ID deviceId, U32 port)`  
*Enables packet transmit mode for a specific port.*

- `int STAR_API_CC TRIGGER_PXI_IF_disablePortTransmitPacketMode (STAR_DEVICE_ID deviceId, U32 port)`  
*Disables packet transmit mode for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled (STAR_DEVICE_ID deviceId, U32 port, U32 *pEnabled)`  
*Gets whether packet transmit mode is enabled for a specific port.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)`  
*Sets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)`  
*Gets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_enableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Enables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_disableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Disables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerEnabled (STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)`  
*Gets whether a trigger is enabled.*
- `int STAR_API_CC TRIGGER_PXI_IF_forceTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Force the triggering of a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_IF_reset (STAR_DEVICE_ID deviceId)`  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_PXI_IF_getTriggerMatrix ()`  
*Retrieves the trigger matrix of the PXI Interface device.*
- `int STAR_API_CC TRIGGER_PXI_IF_getDeviceClockRate ()`  
*Returns the clock rate used by the triggering system of the PXI Interface device.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`

*Disables trigger count for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterTriggerCountEnabled (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterStartMode (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disableCounterStartMode (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterStartModeEnabled (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterStartStopMode (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disableCounterStartStopMode (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterStartStopModeEnabled (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterLoadZero (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables load zero for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disableCounterLoadZero (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables load zero for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterLoadZeroEnabled (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether load zero is enabled for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_setCounterReloadValue (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)

*Sets the reload value for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterReloadValue (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)

*Gets the reload value for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_forceCounterReload (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getCounterValue (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enablePortTransmitPacketMode (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disablePortTransmitPacketMode (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables packet transmit mode for a specific port.*

- int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getPortTransmitPacketModeEnabled (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether packet transmit mode is enabled for a specific port.*

- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE mode)`  
*Sets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputMode (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_INPUT_MODE *pMode)`  
*Gets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_enableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Enables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_disableTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Disables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled (STAR_DEVICE_ID deviceId, U32 trigger, U32 *pEnabled)`  
*Gets whether a trigger is enabled.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger (STAR_DEVICE_ID deviceId, U32 trigger)`  
*Force the triggering of a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 mask)`  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerAndMask (STAR_DEVICE_ID deviceId, U32 trigger, TRIGGER_TYPE type, U32 *pMask)`  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_reset (STAR_DEVICE_ID deviceId)`  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerMatrix ()`  
*Retrieves the trigger matrix of the PXI Router device.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getDeviceClockRate ()`  
*Returns the clock rate used by the triggering system of the PXI Router device.*

### 8.70.1 Detailed Description

Functions in this group deal with configuring the different triggering subsystems.

### 8.70.2 Enumeration Type Documentation

#### 8.70.2.1 enum TRIGGER\_INPUT\_MODE

Internal trigger input modes.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**TRIGGER\_INPUT\_MODE\_OR** Trigger is a sum of its inputs.

**TRIGGER\_INPUT\_MODE\_AND** Trigger is a product of its inputs.

### 8.70.3 Function Documentation

8.70.3.1 `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

#### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### Examples:

`triggering_example.c`.

8.70.3.2 `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

#### Parameters

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

8.70.3.3 `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.4 `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.5 `int STAR_API_CC TRIGGER_BRICK_MK3_disableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.6 `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

##### Parameters

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.7 `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the output trigger signal is not inverted, and the input trigger signal is not inverted before the detection mechanism.

##### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

##### Examples:

`timestamp_test.c`.

#### 8.70.3.8 `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.



**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.9 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disablePortSpill ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Disables spilling for a specific port.

**Note**

This function is currently only supported by SpaceWire Brick Mk3 and Brick Mk4 devices with FPGA versions v1.05e15 and v1.00e09, respectively.

**Parameters**

|                 |                              |
|-----------------|------------------------------|
| <i>deviceld</i> | Device containing the port.  |
| <i>port</i>     | Port to disable spilling on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v4.05 - 29th January 2021

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4

**8.70.3.10 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disablePortTransmitPacketMode ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.11 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_disableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c.

**8.70.3.12 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.13** `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.14** `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.15 `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

#### 8.70.3.16 `int STAR_API_CC TRIGGER_BRICK_MK3_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

**8.70.3.17** `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

**Parameters**

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.18** `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the output trigger signal is inverted, and the input trigger signal is inverted before the detection mechanism.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c.

#### 8.70.3.19 `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.

##### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

##### Examples:

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

#### 8.70.3.20 `int STAR_API_CC TRIGGER_BRICK_MK3_enablePortSpill ( STAR_DEVICE_ID deviceld, U32 port )`

Enables spilling for a specific port.

While enabled AND the port has transmit packet mode enabled, queued packets are spilled.

##### Note

This function is currently only supported by [SpaceWire Brick Mk3 and Brick Mk4](#) devices with FPGA versions v1.05e15 and v1.00e09, respectively.

##### Parameters

|                 |                             |
|-----------------|-----------------------------|
| <i>deviceld</i> | Device containing the port. |
| <i>port</i>     | Port to enable spilling on. |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v4.05 - 29th January 2021

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4

#### 8.70.3.21 `int STAR_API_CC TRIGGER_BRICK_MK3_enablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[triggering\\_example.c](#).

**8.70.3.22 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_enableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**8.70.3.23 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_forceCounterReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
|-----------------|--------------------------------|

---

|                |                    |
|----------------|--------------------|
| <i>counter</i> | Counter to reload. |
|----------------|--------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

triggering\_example.c.

**8.70.3.24** `int STAR_API_CC TRIGGER_BRICK_MK3_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.25** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

---

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

---



8.70.3.26 `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceId, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.27** `int STAR_API_CC_TRIGGER_BRICK_MK3_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.28** `int STAR_API_CC_TRIGGER_BRICK_MK3_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.29** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.30** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.31** `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.32 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getDeviceClockRate ( )**

Returns the clock rate used by the triggering system of the Brick Mk3 device.

**Returns**

The clock rate used by the triggering system of the Brick Mk3 device.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.33 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getExtTriggerEdgeDetectModeEnabled ( STAR\_DEVICE\_ID deviceld, U32 extTrigger, U32 \* pEnabled )**

Gets whether edge detect mode is enabled for a specific external trigger.

**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.34** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

#### Parameters

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                          |
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.35** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether invert is enabled for a specific external trigger.

#### Parameters

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.36** `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether output is enabled for a specific external trigger.

#### Parameters

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.37** int **STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getPortSpillEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *port*, **U32** \* *pEnabled* )

Gets whether spilling is enabled for a specific port.

**Note**

This function is currently only supported by [SpaceWire Brick Mk3 and Brick Mk4](#) devices with FPGA versions v1.05e15 and v1.00e09, respectively.

**Parameters**

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                   |
| <i>port</i>     | Port to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if spilling is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v4.05 - 29th January 2021

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4

**8.70.3.38** int **STAR\_API\_CC TRIGGER\_BRICK\_MK3\_getPortTransmitPacketModeEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *port*, **U32** \* *pEnabled* )

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.39** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.40** `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.41 `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

##### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.42 `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

##### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

##### Returns

1 if the operation was successful, else 0.

##### Added in version:

STAR-System v3.0 - 26th August 2016

##### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### 8.70.3.43 `TRIGGER_MATRIX STAR_API_CC TRIGGER_BRICK_MK3_getTriggerMatrix ( )`

Retrieves the trigger matrix of the Brick Mk3 device.



**Returns**

The trigger matrix of the Brick Mk3 device.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4

**8.70.3.44 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_reset ( STAR\_DEVICE\_ID *deviceld* )**

Resets the triggering configuration to its default state.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.45 int STAR\_API\_CC TRIGGER\_BRICK\_MK3\_setCounterReloadValue ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *reloadValue* )**

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.46** `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

#### Parameters

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |
| <i>extend</i>     | Extend duration in cycles.                  |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

#### Examples:

`timestamp_test.c`, and `triggering_example.c`.

**8.70.3.47** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

#### Parameters

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.48** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**8.70.3.49** `int STAR_API_CC TRIGGER_BRICK_MK3_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1

**Examples:**

`triggering_example.c`.

**8.70.3.50** `TRIGGER_MATRIX STAR_API_CC TRIGGER_PCIE_getTriggerMatrix ( )`

Retrieves the trigger matrix of the PCIe device.

**Returns**

The trigger matrix of the PCIe device.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PCIe

**8.70.3.51 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.52 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.53 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_disableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.54** int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_disableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.55** int STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_disableCounterTriggerCount ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

8.70.3.56 `int STAR_API_CC TRIGGER_PCIE_IF_disableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.57** `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

**8.70.3.58** `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.59 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.60 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_enableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe



**8.70.3.61**    `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.62**    `int STAR_API_CC TRIGGER_PCIE_IF_enableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

**8.70.3.63**    `int STAR_API_CC TRIGGER_PCIE_IF_forceCounterReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Force the reload of a specific counter.

**Parameters**

---

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.64 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_forceTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Force the triggering of a specific internal trigger.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.65 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_getCounterAutoReloadEnabled ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 \* *pEnabled* )**

Gets whether auto reload is enabled for a specific counter.

**Parameters**

---

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

8.70.3.66 `int STAR_API_CC_TRIGGER_PCIE_IF_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceId, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.67** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterReloadValue** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pReloadValue* )

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.68** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterStartModeEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**8.70.3.69** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterStartStopModeEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**8.70.3.70** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterTriggerCountEnabled** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pEnabled* )

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

Added in version:

STAR-System v3.1 - 21st October 2016

Supported devices:

SpaceWire PCIe

**8.70.3.71** int **STAR\_API\_CC\_TRIGGER\_PCIE\_IF\_getCounterValue** ( **STAR\_DEVICE\_ID** *deviceld*, **U32** *counter*, **U32** \* *pValue* )

Gets the counter value for a specific counter.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.72 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_getDeviceClockRate ( )**

Returns the clock rate used by the triggering system of the PCIe device.

**Returns**

The clock rate used by the triggering system of the PCIe device.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

**8.70.3.73 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_getTriggerAndMask ( STAR\_DEVICE\_ID deviceld, U32 trigger, TRIGGER\_TYPE type, U32 \* pMask )**

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

8.70.3.74 `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerEnabled ( STAR_DEVICE_ID deviceId, U32 trigger, U32 *  
pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.75** `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.76** `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |



**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.77 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_reset ( STAR\_DEVICE\_ID *deviceld* )**

Resets the triggering configuration to its default state.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PCIe

**8.70.3.78 int STAR\_API\_CC TRIGGER\_PCIE\_IF\_setCounterReloadValue ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 *reloadValue* )**

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**Examples:**

triggering\_example.c.

8.70.3.79 `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger,  
TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.80** `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPU↵T\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

**8.70.3.81** `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.1 - 21st October 2016

**Supported devices:**

SpaceWire PCIe

### 8.70.3.82 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_disableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

### 8.70.3.83 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_disableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.84** `int STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.85** `int STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.86** `int STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.87** `int STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

**Parameters**

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.88** `int STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the output trigger signal is not inverted, and the input trigger signal is not inverted before the detection mechanism.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c.

**8.70.3.89** `int STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

#### Examples:

timestamp\_test.c, and triggering\_example.c.

**8.70.3.90** `int STAR_API_CC TRIGGER_PCIE_MK2_disablePortSpill ( STAR_DEVICE_ID deviceld, U32 port )`

Disables spilling for a specific port.

#### Parameters

|                 |                              |
|-----------------|------------------------------|
| <i>deviceld</i> | Device containing the port.  |
| <i>port</i>     | Port to disable spilling on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

**8.70.3.91** `int STAR_API_CC TRIGGER_PCIE_MK2_disablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.70.3.92 `int STAR_API_CC TRIGGER_PCIE_MK2_disableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

`timestamp_test.c`.

8.70.3.93 `int STAR_API_CC TRIGGER_PCIE_MK2_enableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |



**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.94** `int STAR_API_CC TRIGGER_PCIE_MK2_enableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.95** `int STAR_API_CC TRIGGER_PCIE_MK2_enableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.96** `int STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

**8.70.3.97** `int STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

**8.70.3.98** `int STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

#### Parameters

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

#### Examples:

timestamp\_test.c, and triggering\_example.c.

**8.70.3.99** `int STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the output trigger signal is inverted, and the input trigger signal is inverted before the detection mechanism.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

#### Examples:

timestamp\_test.c.

**8.70.3.100** `int STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.

**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.101 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_enablePortSpill ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Enables spilling for a specific port.

While enabled AND the port has transmit packet mode enabled, queued packets are spilled.

**Parameters**

|                 |                             |
|-----------------|-----------------------------|
| <i>deviceld</i> | Device containing the port. |
| <i>port</i>     | Port to enable spilling on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.102 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_enablePortTransmitPacketMode ( STAR\_DEVICE\_ID *deviceld*, U32 *port* )**

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

**8.70.3.103** `int STAR_API_CC TRIGGER_PCIE_MK2_enableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.104** `int STAR_API_CC TRIGGER_PCIE_MK2_forceCounterReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

triggering\_example.c.

### 8.70.3.105 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_forceTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

### 8.70.3.106 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_getCounterAutoReloadEnabled ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 \* *pEnabled* )

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

### 8.70.3.107 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_getCounterLoadZeroEnabled ( STAR\_DEVICE\_ID *deviceld*, U32 *counter*, U32 \* *pEnabled* )

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.108** `int STAR_API_CC_TRIGGER_PCIE_MK2_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.109** `int STAR_API_CC_TRIGGER_PCIE_MK2_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

**8.70.3.110** `int STAR_API_CC_TRIGGER_PCIE_MK2_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

**8.70.3.111** `int STAR_API_CC_TRIGGER_PCIE_MK2_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

Added in version:

STAR-System v5.01 - 16th September 2022

Supported devices:

SpaceWire PCIe Mk2

**8.70.3.112** `int STAR_API_CC_TRIGGER_PCIE_MK2_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.



**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.113 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_getDeviceClockRate ( )**

Returns the clock rate used by the triggering system of the PCIe MK2 device.

**Returns**

The clock rate used by the triggering system of the PCIe MK2 device.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.114 int STAR\_API\_CC TRIGGER\_PCIE\_MK2\_getExtTriggerEdgeDetectModeEnabled ( STAR\_DEVICE\_ID deviceld, U32 extTrigger, U32 \* pEnabled )**

Gets whether edge detect mode is enabled for a specific external trigger.

**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.115** `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

#### Parameters

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                          |
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

**8.70.3.116** `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerInvertEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether invert is enabled for a specific external trigger.

#### Parameters

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v5.01 - 16th September 2022

#### Supported devices:

SpaceWire PCIe Mk2

**8.70.3.117** `int STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerOutputEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether output is enabled for a specific external trigger.

#### Parameters

---

---

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.118** `int STAR_API_CC TRIGGER_PCIE_MK2_getPortSpillEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether spilling is enabled for a specific port.

**Parameters**

---

|                 |                                                                               |
|-----------------|-------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                   |
| <i>port</i>     | Port to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if spilling is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.119** `int STAR_API_CC TRIGGER_PCIE_MK2_getPortTransmitPacketModeEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

---

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

8.70.3.120 `int STAR_API_CC TRIGGER_PCIE_MK2_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger,  
TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.121** `int STAR_API_CC TRIGGER_PCIE_MK2_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.122** `int STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.         |
| <i>trigger</i>  | Internal trigger to get trigger input mode for. |

---

|              |                                                     |
|--------------|-----------------------------------------------------|
| <i>pMode</i> | Pointer to value to be updated with the input mode. |
|--------------|-----------------------------------------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.123** `int STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.124** `TRIGGER_MATRIX STAR_API_CC TRIGGER_PCIE_MK2_getTriggerMatrix ( )`

Retrieves the trigger matrix of the PCIe MK2 device.

**Returns**

The trigger matrix of the PCIe MK2 device.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.125** `int STAR_API_CC TRIGGER_PCIE_MK2_reset ( STAR_DEVICE_ID deviceld )`

Resets the triggering configuration to its default state.

**Parameters**

---

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.126** `int STAR_API_CC TRIGGER_PCIE_MK2_setCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

---

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

`timestamp_test.c`, and `triggering_example.c`.

**8.70.3.127** `int STAR_API_CC TRIGGER_PCIE_MK2_setExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

---

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |

---

---

|               |                            |
|---------------|----------------------------|
| <i>extend</i> | Extend duration in cycles. |
|---------------|----------------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.128** `int STAR_API_CC TRIGGER_PCIE_MK2_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.129** `int STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
|-----------------|-----------------------------------------|

---



|                |                                             |
|----------------|---------------------------------------------|
| <i>trigger</i> | Internal trigger to set the input mode for. |
| <i>mode</i>    | The input mode.                             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.130** `int STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**8.70.3.131** `int STAR_API_CC TRIGGER_PXI_IF_disableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.132 int STAR\_API\_CC\_TRIGGER\_PXI\_IF\_disableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.133 int STAR\_API\_CC\_TRIGGER\_PXI\_IF\_disableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.134 int STAR\_API\_CC\_TRIGGER\_PXI\_IF\_disableCounterStartStopMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

---

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.135** `int STAR_API_CC TRIGGER_PXI_IF_disableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.136** `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables edge detect mode for a specific external trigger.

When disabled, the trigger will always be set when the input trigger is high.

**Parameters**

---

|                   |                                                  |
|-------------------|--------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.          |
| <i>extTrigger</i> | External trigger to disable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

### 8.70.3.137 `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables invert for a specific external trigger.

When disabled, the output trigger signal is not inverted, and the input trigger signal is not inverted before the detection mechanism.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable invert on.  |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### Examples:

timestamp\_test.c.

### 8.70.3.138 `int STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Disables output for a specific external trigger.

When disabled, the external trigger's output action does not cause the trigger output signal to be pulsed.

#### Parameters

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to disable output on.  |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### Examples:

timestamp\_test.c, and triggering\_example.c.

### 8.70.3.139 `int STAR_API_CC TRIGGER_PXI_IF_disablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.140 int STAR\_API\_CC TRIGGER\_PXI\_IF\_disableTrigger ( STAR\_DEVICE\_ID deviceld, U32 trigger )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c.

**8.70.3.141 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterAutoReload ( STAR\_DEVICE\_ID deviceld, U32 counter )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.142 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.143 int STAR\_API\_CC TRIGGER\_PXI\_IF\_enableCounterStartMode ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.144** `int STAR_API_CC TRIGGER_PXI_IF_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

#### Parameters

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### Examples:

`triggering_example.c`.

**8.70.3.145** `int STAR_API_CC TRIGGER_PXI_IF_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

#### Parameters

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

#### Examples:

`triggering_example.c`.

**8.70.3.146** `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables edge detect mode for a specific external trigger.

When enabled, the trigger will be set on the rising edge of the input trigger signal.

**Parameters**

---

|                   |                                                 |
|-------------------|-------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.         |
| <i>extTrigger</i> | External trigger to enable edge detect mode on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.147** `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerInvert ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables invert for a specific external trigger.

When enabled, the output trigger signal is inverted, and the input trigger signal is inverted before the detection mechanism.

**Parameters**

---

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable invert on.   |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c.

**8.70.3.148** `int STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerOutput ( STAR_DEVICE_ID deviceld, U32 extTrigger )`

Enables output for a specific external trigger.

When enabled, the external trigger's output action causes the trigger output signal to be pulsed for a duration specified by the extend duration value.



**Parameters**

|                   |                                         |
|-------------------|-----------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger. |
| <i>extTrigger</i> | External trigger to enable output on.   |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.149** `int STAR_API_CC TRIGGER_PXI_IF_enablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

**8.70.3.150** `int STAR_API_CC TRIGGER_PXI_IF_enableTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

8.70.3.151 `int STAR_API_CC TRIGGER_PXI_IF_forceCounterReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Force the reload of a specific counter.

**Parameters**

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

triggering\_example.c.

8.70.3.152 `int STAR_API_CC TRIGGER_PXI_IF_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.153** `int STAR_API_CC_TRIGGER_PXI_IF_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.154** `int STAR_API_CC_TRIGGER_PXI_IF_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

8.70.3.155 `int STAR_API_CC TRIGGER_PXI_IF_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter,  
U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.156** `int STAR_API_CC_TRIGGER_PXI_IF_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.157** `int STAR_API_CC_TRIGGER_PXI_IF_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.158** `int STAR_API_CC TRIGGER_PXI_IF_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

#### Parameters

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.159** `int STAR_API_CC TRIGGER_PXI_IF_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.

#### Parameters

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.160** `int STAR_API_CC TRIGGER_PXI_IF_getDeviceClockRate ( )`

Returns the clock rate used by the triggering system of the PXI Interface device.

#### Returns

The clock rate used by the triggering system of the PXI Interface device.

#### Added in version:

STAR-System v5.02 - 25th November 2022

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

[timestamp\\_test.c](#), and [triggering\\_example.c](#).

**8.70.3.161** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether edge detect mode is enabled for a specific external trigger.

**Parameters**

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                               |
| <i>extTrigger</i> | External trigger to check.                                                            |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if edge detect mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

[STAR-System v3.0 - 26th August 2016](#)

**Supported devices:**

[SpaceWire PXI Interface](#) and [PXI Interface Mk2](#)

**8.70.3.162** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pExtend )`

Gets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                                                  |
|-------------------|------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                          |
| <i>extTrigger</i> | External trigger to get the extend duration for.                 |
| <i>pExtend</i>    | Point to value to be updated with the extend duration in cycles. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

[STAR-System v3.0 - 26th August 2016](#)

**Supported devices:**

[SpaceWire PXI Interface](#) and [PXI Interface Mk2](#)

**8.70.3.163** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInvertEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether invert is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if invert is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.164** `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputEnabled ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 * pEnabled )`

Gets whether output is enabled for a specific external trigger.

**Parameters**

|                   |                                                                             |
|-------------------|-----------------------------------------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.                                     |
| <i>extTrigger</i> | External trigger to check.                                                  |
| <i>pEnabled</i>   | User supplied value that will be updated to 1 if output is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.165** `int STAR_API_CC TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled ( STAR_DEVICE_ID deviceld, U32 port, U32 * pEnabled )`

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2



**8.70.3.166** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the AND/OR mask for a trigger type for a specific internal trigger.

#### Parameters

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.167** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

#### Parameters

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.168** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.             |
| <i>trigger</i>  | Internal trigger to get trigger input mode for.     |
| <i>pMode</i>    | Pointer to value to be updated with the input mode. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.169** `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.170** `TRIGGER_MATRIX STAR_API_CC TRIGGER_PXI_IF_getTriggerMatrix ( )`

Retrieves the trigger matrix of the PXI Interface device.

**Returns**

The trigger matrix of the PXI Interface device.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

8.70.3.171 `int STAR_API_CC TRIGGER_PXI_IF_reset ( STAR_DEVICE_ID deviceld )`

Resets the triggering configuration to its default state.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.172** `int STAR_API_CC TRIGGER_PXI_IF_setCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.173** `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerExtend ( STAR_DEVICE_ID deviceld, U32 extTrigger, U32 extend )`

Sets the extend duration for a specific external trigger.

The extend duration is the duration, in cycles, of the trigger output signal pulse when a trigger output action occurs.

**Parameters**

|                   |                                             |
|-------------------|---------------------------------------------|
| <i>deviceld</i>   | Device containing the external trigger.     |
| <i>extTrigger</i> | External trigger to set extend duration on. |

---

|               |                            |
|---------------|----------------------------|
| <i>extend</i> | Extend duration in cycles. |
|---------------|----------------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**Examples:**

timestamp\_test.c, and triggering\_example.c.

**8.70.3.174** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | AND/OR mask.                                               |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.0 - 26th August 2016

**Supported devices:**

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.175** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
|-----------------|-----------------------------------------|

---

|                |                                             |
|----------------|---------------------------------------------|
| <i>trigger</i> | Internal trigger to set the input mode for. |
| <i>mode</i>    | The input mode.                             |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.176** `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

#### Parameters

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |
| <i>mask</i>     | Invert mask.                                               |

#### Returns

1 if the operation was successful, else 0.

#### Added in version:

STAR-System v3.0 - 26th August 2016

#### Supported devices:

SpaceWire PXI Interface and PXI Interface Mk2

**8.70.3.177** `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterAutoReload ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables auto reload for a specific counter.

When disabled, the counter will not automatically reload with its reload value when the counter reaches zero.

#### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceld</i> | Device containing the counter.     |
| <i>counter</i>  | Counter to disable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.178** `int STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterLoadZero ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables load zero for a specific counter.

When disabled, reload triggered events occur even when the counter is not zero.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to disable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.179** `int STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to disable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.70.3.180 `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables start stop mode for a specific counter.

When disabled, the counter will not wait for a start action to occur before any counting or reloading can take place.



**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceld</i> | Device containing the counter.         |
| <i>counter</i>  | Counter to disable start stop mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.181** `int STAR_API_CC TRIGGER_PXI_ROUTER_disableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Disables trigger count for a specific counter.

When disabled, the counter will be decremented every clock cycle.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceld</i> | Device containing the counter.       |
| <i>counter</i>  | Counter to disable trigger count on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.182** `int STAR_API_CC TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Disables packet transmit mode for a specific port.

When disabled, packets do not wait for a transmit packet action to occur before transmission.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the port.              |
| <i>port</i>     | Port to disable packet transmit mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.183 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_disableTrigger ( STAR\_DEVICE\_ID *deviceld*, U32 *trigger* )**

Disables a specific internal trigger.

When disabled, the trigger is not set by its input events and does not cause output actions to occur.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to disable.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.184 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterAutoReload ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables auto reload for a specific counter.

When enabled, the counter will automatically reload with its reload value when the counter reaches zero.

**Parameters**

|                 |                                   |
|-----------------|-----------------------------------|
| <i>deviceld</i> | Device containing the counter.    |
| <i>counter</i>  | Counter to enable auto reload on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.185 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableCounterLoadZero ( STAR\_DEVICE\_ID *deviceld*, U32 *counter* )**

Enables load zero for a specific counter.

When enabled, reload triggered events only occur when the counter is zero.

**Parameters**

|                 |                                 |
|-----------------|---------------------------------|
| <i>deviceld</i> | Device containing the counter.  |
| <i>counter</i>  | Counter to enable load zero on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.186** `int STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start mode for a specific counter.

When enabled, the counter will wait for a counter start action to occur before any counting or reloading can take place. When the counter reaches zero, the counter may be reloaded but no counting or reloading can take place until the next start action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceld</i> | Device containing the counter.   |
| <i>counter</i>  | Counter to enable start mode on. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.187** `int STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartStopMode ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables start stop mode for a specific counter.

When enabled, the counter will wait for a start action to occur before any counting or reloading can take place. The counter will continue to count and reload until a stop action occurs.

Note that start mode and start stop mode are mutually exclusive and enabling one will disable the other.

**Parameters**

---

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceld</i> | Device containing the counter.        |
| <i>counter</i>  | Counter to enable start stop mode on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**8.70.3.188** `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterTriggerCount ( STAR_DEVICE_ID deviceld, U32 counter )`

Enables trigger count for a specific counter.

When enabled, the counter will only be decremented when a count action occurs.

**Parameters**

---

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceld</i> | Device containing the counter.      |
| <i>counter</i>  | Counter to enable trigger count on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

[triggering\\_example.c](#).

**8.70.3.189** `int STAR_API_CC TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode ( STAR_DEVICE_ID deviceld, U32 port )`

Enables packet transmit mode for a specific port.

When enabled, packets are only transmitted when a transmit packet action occurs.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the port.             |
| <i>port</i>     | Port to enable packet transmit mode on. |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.190 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_enableTrigger ( STAR\_DEVICE\_ID deviceld, U32 trigger )**

Enables a specific internal trigger.

When enabled, the trigger can be set by its input events and cause output actions to occur.

**Parameters**

---

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to enable.             |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.191 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_forceCounterReload ( STAR\_DEVICE\_ID deviceld, U32 counter )**

Force the reload of a specific counter.

**Parameters**

---

|                 |                                |
|-----------------|--------------------------------|
| <i>deviceld</i> | Device containing the counter. |
| <i>counter</i>  | Counter to reload.             |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.192** `int STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger ( STAR_DEVICE_ID deviceld, U32 trigger )`

Force the triggering of a specific internal trigger.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger. |
| <i>trigger</i>  | Internal trigger to trigger.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.193** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether auto reload is enabled for a specific counter.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                   |
| <i>counter</i>  | Counter to check.                                                                |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if auto reload is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.70.3.194 `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled ( STAR_DEVICE_ID deviceId, U32 counter, U32 * pEnabled )`

Gets whether load zero is enabled for a specific counter.

**Parameters**

|                 |                                                                                |
|-----------------|--------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                 |
| <i>counter</i>  | Counter to check.                                                              |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if load zero is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.195** `int STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pReloadValue )`

Gets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                     |                                                       |
|---------------------|-------------------------------------------------------|
| <i>deviceld</i>     | Device containing the counter.                        |
| <i>counter</i>      | Counter to get the reload value for.                  |
| <i>pReloadValue</i> | Pointer to value to be updated with the reload value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.196** `int STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterStartModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                 |
|-----------------|---------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                  |
| <i>counter</i>  | Counter to check.                                                               |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017



**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.197** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether start stop mode is enabled for a specific counter.

**Parameters**

|                 |                                                                                      |
|-----------------|--------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                       |
| <i>counter</i>  | Counter to check.                                                                    |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if start stop mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.198** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pEnabled )`

Gets whether trigger count is enabled for a specific counter.

**Parameters**

|                 |                                                                                    |
|-----------------|------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                                                     |
| <i>counter</i>  | Counter to check.                                                                  |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if trigger count is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.199** `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 * pValue )`

Gets the counter value for a specific counter.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceld</i> | Device containing the counter.                         |
| <i>counter</i>  | Counter to get the value for.                          |
| <i>pValue</i>   | Pointer to value to be updated with the counter value. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.200 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getDeviceClockRate ( )**

Returns the clock rate used by the triggering system of the PXI Router device.

**Returns**

The clock rate used by the triggering system of the PXI Router device.

**Added in version:**

STAR-System v5.02 - 25th November 2022

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.201 int STAR\_API\_CC TRIGGER\_PXI\_ROUTER\_getPortTransmitPacketModeEnabled ( STAR\_DEVICE\_ID deviceld, U32 port, U32 \* pEnabled )**

Gets whether packet transmit mode is enabled for a specific port.

**Parameters**

|                 |                                                                                           |
|-----------------|-------------------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the port.                                                               |
| <i>port</i>     | Port to check.                                                                            |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if packet transmit mode is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

```
8.70.3.202 int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerAndMask (STAR_DEVICE_ID deviceld, U32 trigger,
 TRIGGER_TYPE type, U32 * pMask)
```

Gets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the AND/OR mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the AND/OR mask.            |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.203** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled ( STAR_DEVICE_ID deviceld, U32 trigger, U32 * pEnabled )`

Gets whether a trigger is enabled.

**Parameters**

|                 |                                                                                  |
|-----------------|----------------------------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                                          |
| <i>trigger</i>  | Internal trigger to check.                                                       |
| <i>pEnabled</i> | User supplied value that will be updated to 1 if the trigger is enabled, else 0. |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.204** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE * pMode )`

Gets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all of its input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any of its input events occur.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.         |
| <i>trigger</i>  | Internal trigger to get trigger input mode for. |

---

|              |                                                     |
|--------------|-----------------------------------------------------|
| <i>pMode</i> | Pointer to value to be updated with the input mode. |
|--------------|-----------------------------------------------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.205** `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 * pMask )`

Gets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                         |
| <i>trigger</i>  | Internal trigger to get the invert mask for a trigger type for. |
| <i>type</i>     | Trigger type.                                                   |
| <i>pMask</i>    | Pointer to value to be updated with the invert mask.            |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.206** `TRIGGER_MATRIX STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerMatrix ( )`

Retrieves the trigger matrix of the PXI Router device.

**Returns**

The trigger matrix of the PXI Router device.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.207** `int STAR_API_CC TRIGGER_PXI_ROUTER_reset ( STAR_DEVICE_ID deviceld )`

Resets the triggering configuration to its default state.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceld</i> | Device containing the triggering system. |
|-----------------|------------------------------------------|

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.208** `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterReloadValue ( STAR_DEVICE_ID deviceld, U32 counter, U32 reloadValue )`

Sets the reload value for a specific counter.

The reload value is the value that will be loaded into the counter when a reload occurs.

**Parameters**

|                    |                                      |
|--------------------|--------------------------------------|
| <i>deviceld</i>    | Device containing the counter.       |
| <i>counter</i>     | Counter to set the reload value for. |
| <i>reloadValue</i> | Reload value.                        |

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**Examples:**

triggering\_example.c.

**8.70.3.209** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the AND/OR mask for a trigger type for a specific internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set AND/OR mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |

---

|             |              |
|-------------|--------------|
| <i>mask</i> | AND/OR mask. |
|-------------|--------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.210** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputMode ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_INPUT_MODE mode )`

Sets the input mode for a specific internal trigger.

The trigger input mode can be either TRIGGER\_INPUT\_MODE\_AND or TRIGGER\_INPUT\_MODE\_OR. If set to TRIGGER\_INPUT\_MODE\_AND, the trigger occurs when all source input events occur. If set to TRIGGER\_INPUT\_MODE\_OR, the trigger occurs when any source input events occur.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.     |
| <i>trigger</i>  | Internal trigger to set the input mode for. |
| <i>mode</i>     | The input mode.                             |

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

**8.70.3.211** `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask ( STAR_DEVICE_ID deviceld, U32 trigger, TRIGGER_TYPE type, U32 mask )`

Sets the invert mask for a trigger type for a specific internal trigger.

The invert mask for a trigger type determines if a source of input events should have its signal inverted before reaching the internal trigger.

**Parameters**

|                 |                                                            |
|-----------------|------------------------------------------------------------|
| <i>deviceld</i> | Device containing the internal trigger.                    |
| <i>trigger</i>  | Internal trigger to set invert mask for a trigger type on. |
| <i>type</i>     | Trigger type.                                              |

---

---

|             |              |
|-------------|--------------|
| <i>mask</i> | Invert mask. |
|-------------|--------------|

---

**Returns**

1 if the operation was successful, else 0.

**Added in version:**

STAR-System v3.5 - 17th March 2017

**Supported devices:**

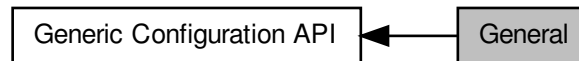
SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2



## 8.71 General

Functions in this group are general functions.

Collaboration diagram for General:



### Functions

- `int STAR_API_CC_CFG_updateDeviceInfo (STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC_CFG_refreshDeviceInfo (STAR_DEVICE_ID deviceId)`  
*This function is used to update a device's information held in the shared memory.*
- `int STAR_API_CC_CFG_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`  
*Gets the router's global settings.*
- `int STAR_API_CC_CFG_setRouterGlobalSettings (STAR_DEVICE_ID deviceId, _In_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`  
*Sets the router's global settings.*
- `int STAR_API_CC_CFG_getNetworkIdentity (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_setNetworkIdentity (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the network identity register.*
- `int STAR_API_CC_CFG_getGeneralPurpose (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_setGeneralPurpose (STAR_DEVICE_ID deviceId, REGISTER value)`  
*Sets the value of the general purpose register.*

#### 8.71.1 Detailed Description

Functions in this group are general functions.

#### 8.71.2 Function Documentation

##### 8.71.2.1 `int STAR_API_CC_CFG_getGeneralPurpose ( STAR_DEVICE_ID deviceId, _Out_ REGISTER * pValue )`

Gets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|     |                 |                                                  |
|-----|-----------------|--------------------------------------------------|
|     | <i>deviceId</i> | Device to get the general purpose register from. |
| out | <i>pValue</i>   | Value of the register.                           |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[user\\_registers\\_example.c](#).

#### 8.71.2.2 int STAR\_API\_CC\_CFG\_getNetworkIdentity ( STAR\_DEVICE\_ID deviceId, \_Out\_ REGISTER \* pValue )

Gets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|     |                 |                                          |
|-----|-----------------|------------------------------------------|
|     | <i>deviceId</i> | Device to get the network identity from. |
| out | <i>pValue</i>   | Value of the register.                   |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[user\\_registers\\_example.c](#).

#### 8.71.2.3 int STAR\_API\_CC\_CFG\_getRouterGlobalSettings ( STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_ROUTER\_GLOBAL\_STATE \* pState )

Gets the router's global settings.

**Parameters**

|     |                 |                                     |
|-----|-----------------|-------------------------------------|
|     | <i>deviceId</i> | Device to get global settings from. |
| out | <i>pState</i>   | Global settings for the router.     |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[rmap\\_target\\_example.c](#), [router\\_configuration\\_example.c](#), and [timestamp\\_test.c](#).

**8.71.2.4 int STAR\_API\_CC\_CFG\_refreshDeviceInfo ( STAR\_DEVICE\_ID deviceId )**

This function is used to update a device's information held in the shared memory.

This is only required for remote devices if they have been removed and replaced.

**Parameters**

|  |                 |                                       |
|--|-----------------|---------------------------------------|
|  | <i>deviceId</i> | Device to update the information for. |
|--|-----------------|---------------------------------------|

**Returns**

0 if the device's information was successfully refreshed, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.03 - 18th January 2023

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.71.2.5 int STAR\_API\_CC\_CFG\_setGeneralPurpose ( STAR\_DEVICE\_ID deviceId, REGISTER value )**

Sets the value of the general purpose register.

This is a user defined 32 bit value that can be set by the user as required for their purposes. This value has no effect on the operation of the router.

**Parameters**

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceId</i> | Device to set the general purpose register on. |
| <i>value</i>    | Value to set the general purpose register to.  |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [router↔Mk2S\\_configuration\\_example.c](#), and [user\\_registers\\_example.c](#).

#### 8.71.2.6 `int STAR_API_CC_CFG_setNetworkIdentity ( STAR_DEVICE_ID deviceId, REGISTER value )`

Sets the value of the network identity register.

This is a 32 bit value that can be used to identify the device, typically set by a network manager. It may also be used for any other purpose. This value has no effect on the operation of the router.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Device to set the network identity for. |
| <i>value</i>    | Value to set the network identity to.   |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[user\\_registers\\_example.c](#).

#### 8.71.2.7 `int STAR_API_CC_CFG_setRouterGlobalSettings ( STAR_DEVICE_ID deviceId, _In_ STAR_CFG_ROUTER_GLOBAL_STATE * pState )`

Sets the router's global settings.

**Parameters**

|           |                 |                                    |
|-----------|-----------------|------------------------------------|
|           | <i>deviceID</i> | Device to set global settings for. |
| <i>in</i> | <i>pState</i>   | Global settings for the router.    |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[rmap\\_target\\_example.c](#), [router\\_configuration\\_example.c](#), and [timestamp\\_test.c](#).

#### 8.71.2.8 `int STAR_API_CC CFG_updateDeviceInfo ( STAR_DEVICE_ID deviceID )`

**Deprecated** Function superseded by [CFG\\_refreshDeviceInfo\(\)](#). This function is used to update a device's information held in the shared memory. This is only required for remote devices if they have been removed and replaced.

**Parameters**

|  |                 |                                       |
|--|-----------------|---------------------------------------|
|  | <i>deviceID</i> | Device to update the information for. |
|--|-----------------|---------------------------------------|

**Returns**

1 if the device's information was successfully updated, else 0.

**Added in version:**

STAR-System v4.00 - 19th November 2019

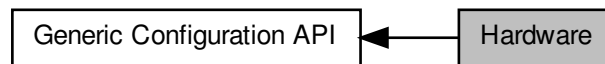
**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

## 8.72 Hardware

Functions in this group are hardware related functions.

Collaboration diagram for Hardware:



### Data Structures

- struct [STAR\\_CFG\\_FPGA\\_INFO](#)

*Defines a structure used to store a device's FPGA version info. [More...](#)*

### Functions

- int [STAR\\_API\\_CC\\_CFG\\_getFPGAInfo](#) (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo)

*Reads information about the hardware version of a device and updates the [STAR\\_CFG\\_FPGA\\_INFO](#) structure pointed to by the passed in pointer to contain the received version information.*

- void [STAR\\_API\\_CC\\_CFG\\_FPGAInfoToString](#) (STAR\_DEVICE\_ID deviceId, \_In\_ STAR\_CFG\_FPGA\_INFO \*pFPGAInfo, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \*pVersion, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \*pBuildDate)

*Creates a string representation of a [STAR\\_CFG\\_FPGA\\_INFO](#) structure.*

- int [STAR\\_API\\_CC\\_CFG\\_identify](#) (STAR\_DEVICE\_ID deviceId)

*Flashes the front panel LEDs of a device.*

### 8.72.1 Detailed Description

Functions in this group are hardware related functions.

### 8.72.2 Data Structure Documentation

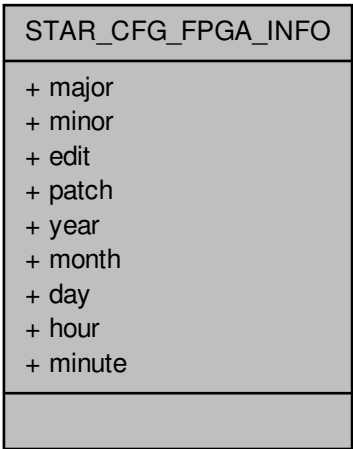
#### 8.72.2.1 struct STAR\_CFG\_FPGA\_INFO

Defines a structure used to store a device's FPGA version info.

#### Examples:

[brickMk2\\_configuration\\_example.c](#), [link\\_events.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [routerMk2S\\_configuration\\_example.c](#), [star-test.c](#), and [timestamp\\_test.c](#).

Collaboration diagram for STAR\_CFG\_FPGA\_INFO:



Data Fields

|     |        |                            |
|-----|--------|----------------------------|
| U8  | day    | FPGA build day.            |
| U16 | edit   | FPGA edit number.          |
| U8  | hour   | FPGA build hour.           |
| U8  | major  | FPGA major version number. |
| U8  | minor  | FPGA minor version number. |
| U8  | minute | FPGA build minute.         |
| U8  | month  | FPGA build month.          |
| U8  | patch  | FPGA patch number.         |
| U16 | year   | FPGA build year.           |

8.72.3 Function Documentation

8.72.3.1 void STAR\_API\_CC CFG\_FPGAInfoToString ( STAR\_DEVICE\_ID *deviceId*, \_In\_ STAR\_CFG\_FPGA\_INFO \* *pFPGAInfo*, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_VERSION\_STR\_MAX\_LEN) char \* *pVersion*, \_Out\_z\_cap\_c\_(STAR\_CFG\_MK2\_BUILD\_DATE\_STR\_MAX\_LEN) char \* *pBuildDate* )

Creates a string representation of a STAR\_CFG\_FPGA\_INFO structure.

Parameters

|     |                  |                                                                                                                                                                    |
|-----|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>  | Identifier for the device to get the info string from.                                                                                                             |
| in  | <i>pFPGAInfo</i> | FPGA information obtained from a call to <a href="#">CFG_getFPGAInfo()</a> .                                                                                       |
| out | <i>pVersion</i>  | String of length <a href="#">STAR_CFG_MK2_VERSION_STR_MAX_LEN</a> that will be updated to contain a null-terminated string representation of the hardware version. |

|     |                   |                                                                                                                                                                    |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| out | <i>pBuildDate</i> | String of length <code>STAR_CFG_MK2_VERSION_STR_MAX_LEN</code> that will be updated to contain a null-terminated string representation of the hardware build date. |
|-----|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`brickMk2_configuration_example.c`, `link_events.c`, `mk2_configuration_example.c`, `pciMk2_configuration_example.c`, and `routerMk2S_configuration_example.c`.

**8.72.3.2** `int STAR_API_CC CFG_getFPGAInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_FPGA_INFO * pFPGAInfo )`

Reads information about the hardware version of a device and updates the `STAR_CFG_FPGA_INFO` structure pointed to by the passed in pointer to contain the received version information.

**Parameters**

|     |                  |                                                                                                                             |
|-----|------------------|-----------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>  | Identifier for the device to get hardware info from.                                                                        |
| out | <i>pFPGAInfo</i> | Pointer to a <code>STAR_CFG_FPGA_INFO</code> structure that will be updated to contain hardware information for the device. |

**Returns**

0 for success, or a negative error code upon failure as defined in `Defines`.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`brickMk2_configuration_example.c`, `link_events.c`, `mk2_configuration_example.c`, `pciMk2_configuration_example.c`, `routerMk2S_configuration_example.c`, `star-test.c`, and `timestamp_test.c`.

**8.72.3.3** `int STAR_API_CC CFG_identify ( STAR_DEVICE_ID deviceId )`

Flashes the front panel LEDs of a device.

This can be used to identify the physical device to which a device identifier refers to.



---

**Parameters**

---

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceID</i> | Device to have its LEDs flashed. |
|-----------------|----------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

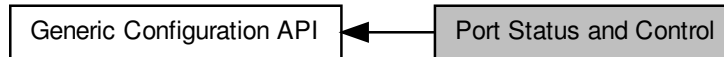
**Examples:**

[brickMk2\\_configuration\\_example.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [router↔Mk2S\\_configuration\\_example.c](#), and [star-test.c](#).

## 8.73 Port Status and Control

Functions in this group deal with port status and control.

Collaboration diagram for Port Status and Control:



### Functions

- `int STAR_API_CC_CFG_clearPortErrors (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Clears all errors on a given port.*
- `int STAR_API_CC_CFG_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC_CFG_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceId, U8 linkNum, _In_ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC_CFG_startLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC_CFG_stopLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC_CFG_getSpaceWireLinkErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC_CFG_getSpaceWireLinkStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pStatus)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC_CFG_getPortStatusControl (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ PORT_STATUS_CONTROL *pPortStatusControl)`  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE STAR_API_CC_CFG_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `int STAR_API_CC_CFG_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)`  
*Gets the status of an external port.*
- `U8 STAR_API_CC_CFG_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*

8.73.1 Detailed Description

Functions in this group deal with port status and control.

8.73.2 Function Documentation

8.73.2.1 int STAR\_API\_CC CFG\_clearPortErrors ( STAR\_DEVICE\_ID deviceID, U8 portNum )

Clears all errors on a given port.

**Note**  
Errors on a port are latched until cleared with this function.

|            |                                      |
|------------|--------------------------------------|
| Parameters |                                      |
| deviceID   | Device to clear port errors on.      |
| portNum    | The port to have its errors cleared. |

**Returns**  
0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**  
STAR-System v4.00 - 19th November 2019

**Supported devices:**  
All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**  
[brickMk2\\_configuration\\_example.c](#), and [port\\_control\\_status\\_example.c](#).

8.73.2.2 int STAR\_API\_CC CFG\_getConfigPortErrors ( PORT\_STATUS\_CONTROL portStatusControl, \_Out\_ STAR\_CFG\_CONFIG\_PORT\_ERRORS \* pErrors )

Gets any errors present on the config port.  
These are errors that arise when malformed or invalid configuration commands are sent to the configuration port.

**Note**  
Errors on the configuration port are latched until cleared with [CFG\\_clearPortErrors\(\)](#).

|                        |                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------|
| Parameters             |                                                                                                |
| portStatus↔<br>Control | Port Status/Control value obtained from a call to <a href="#">CFG_getPortStatusControl()</a> . |

|                  |                      |                                                                                                          |
|------------------|----------------------|----------------------------------------------------------------------------------------------------------|
| <code>out</code> | <code>pErrors</code> | User provided error structure that will be updated to show any errors present on the configuration port. |
|------------------|----------------------|----------------------------------------------------------------------------------------------------------|

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

#### Examples:

`port_control_status_example.c`.

**8.73.2.3** `int STAR_API_CC CFG_getExternalPortErrors ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS * pErrors )`

Gets any errors present on an external port.

#### Note

Errors on a port are latched until cleared with `CFG_clearPortErrors()`.

#### Parameters

|                  |                                |                                                                                                     |
|------------------|--------------------------------|-----------------------------------------------------------------------------------------------------|
|                  | <code>portStatusControl</code> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .         |
| <code>out</code> | <code>pErrors</code>           | User provided error structure that will be updated to show any errors present on the external port. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.73.2.4** `int STAR_API_CC CFG_getExternalPortStatus ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS * pStatus )`

Gets the status of an external port.

**Parameters**

|     |                                |                                                                                             |
|-----|--------------------------------|---------------------------------------------------------------------------------------------|
|     | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
| out | <i>pStatus</i>                 | User provided error structure that will be updated to show the status of the external port. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.73.2.5 U8 STAR\_API\_CC CFG\_getPortConnection ( PORT\_STATUS\_CONTROL portStatusControl )**

Identifies the output port to which the source port is currently connected whilst routing is in operation.

**Parameters**

|  |                                |                                                                                             |
|--|--------------------------------|---------------------------------------------------------------------------------------------|
|  | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> . |
|--|--------------------------------|---------------------------------------------------------------------------------------------|

**Returns**

Output port connected. This will have a value of 31 if there is no current port connected.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`port_control_status_example.c`.

**8.73.2.6 int STAR\_API\_CC CFG\_getPortStatusControl ( STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ PORT\_STATUS\_CONTROL \* pPortStatusControl )**

Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.

**Parameters**

|     |                                 |                                                                 |
|-----|---------------------------------|-----------------------------------------------------------------|
|     | <i>deviceID</i>                 | Device to obtain info from.                                     |
|     | <i>portNum</i>                  | The port number to determine the type of.                       |
| out | <i>pPortStatus↔<br/>Control</i> | Value that will be updated with the ports status/control value. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), [link\\_events.c](#), [port\\_control\\_status\\_example.c](#), and [timestamp\\_test.c](#).

### 8.73.2.7 **STAR\_CFG\_PORT\_TYPE** **STAR\_API\_CC** **CFG\_getPortType ( PORT\_STATUS\_CONTROL *portStatusControl* )**

Identifies the type of port attributed to a given port number.

**Parameters**

|  |                                |                                                                                                |
|--|--------------------------------|------------------------------------------------------------------------------------------------|
|  | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <a href="#">CFG_getPortStatusControl()</a> . |
|--|--------------------------------|------------------------------------------------------------------------------------------------|

**Returns**

Type of the port.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[port\\_control\\_status\\_example.c](#).

### 8.73.2.8 **int** **STAR\_API\_CC** **CFG\_getSpaceWireLinkErrors ( PORT\_STATUS\_CONTROL *portStatusControl*, \_Out\_ STAR\_CFG\_SPW\_LINK\_ERRORS \* *pErrors* )**

Gets any errors present on a SpaceWire link.

**Note**

Errors on a port are latched until cleared with `CFG_clearPortErrors()`.

**Parameters**

|     |                                |                                                                                                      |
|-----|--------------------------------|------------------------------------------------------------------------------------------------------|
|     | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .          |
| out | <i>pErrors</i>                 | User provided error structure that will be updated to show any errors present on the SpaceWire link. |

**Returns**

0 for success, or a negative error code upon failure as defined in `Defines`.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.73.2.9** `int STAR_API_CC CFG_getSpaceWireLinkStatus ( PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS * pStatus )`

Gets the status of a SpaceWire Link.

**Parameters**

|     |                                |                                                                                              |
|-----|--------------------------------|----------------------------------------------------------------------------------------------|
|     | <i>portStatus↔<br/>Control</i> | Port Status/Control value obtained from a call to <code>CFG_getPortStatusControl()</code> .  |
| out | <i>pStatus</i>                 | User provided error structure that will be updated to show the status of the SpaceWire link. |

**Returns**

0 for success, or a negative error code upon failure as defined in `Defines`.

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`brickMk2_configuration_example.c`, `link_events.c`, `port_control_status_example.c`, and `timestamp_test.c`.

**8.73.2.10** `int STAR_API_CC CFG_setSpaceWireLinkStatus ( STAR_DEVICE_ID deviceID, U8 linkNum, _In_ STAR_CFG_SPW_LINK_STATUS * pLinkStatus )`

Sets the state of a SpaceWire Link.

**Parameters**

|           |                    |                                                                                   |
|-----------|--------------------|-----------------------------------------------------------------------------------|
|           | <i>deviceID</i>    | Device to have its link state set.                                                |
|           | <i>linkNum</i>     | The link to set state for.                                                        |
| <i>in</i> | <i>pLinkStatus</i> | The values within this structure are used to set the state of the SpaceWire Link. |

**Note**

The linkState member of this structure is ignored by this function.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

brickMk2\_configuration\_example.c, link\_events.c, port\_control\_status\_example.c, and timestamp\_test.c.

### 8.73.2.11 int STAR\_API\_CC\_CFG\_startLink ( STAR\_DEVICE\_ID deviceID, U8 linkNum )

Starts a SpaceWire link by setting the start bit, and clearing the disable bit.

**Parameters**

|  |                 |                                  |
|--|-----------------|----------------------------------|
|  | <i>deviceID</i> | Device to have its link started. |
|  | <i>linkNum</i>  | The link to start.               |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

link\_events.c, and port\_control\_status\_example.c.



#### 8.73.2.12 int STAR\_API\_CC\_CFG\_stopLink ( STAR\_DEVICE\_ID *deviceId*, U8 *linkNum* )

Stops a SpaceWire link by clearing the start bit, and setting the disable bit.

##### Note

If auto-start is enabled the link will start again as soon as it receives NULLs.

##### Parameters

|                 |                                  |
|-----------------|----------------------------------|
| <i>deviceId</i> | Device to have its link stopped. |
| <i>linkNum</i>  | The link to stop.                |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

##### Examples:

[link\\_events.c](#), and [port\\_control\\_status\\_example.c](#).

## 8.74 Device Identifier

Functions in this group deal with device identification.

Collaboration diagram for Device Identifier:



### Functions

- `int STAR_API_CC CFG_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC CFG_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*

### 8.74.1 Detailed Description

Functions in this group deal with device identification.

### 8.74.2 Function Documentation

#### 8.74.2.1 `int STAR_API_CC CFG_getDeviceIdentificationInfo ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO * pInfo )`

Reads the device identifier information of a device, i.e.

the Manufacturer ID, Chip type and version.

#### Parameters

|     | <i>deviceId</i> | Device to obtain info from.                                                              |
|-----|-----------------|------------------------------------------------------------------------------------------|
| out | <i>pInfo</i>    | Structure that will be updated to contain the Identification information for the device. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[device\\_identifier\\_example.c](#), and [star-test.c](#).

```
8.74.2.2 void STAR_API_CC CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO
* pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *
pManufacturerStr)
```

If the manufacturer is known, this function obtains a string representation of the manufacturers name.

**Parameters**

|     |                              |                                                                                                                                                                   |
|-----|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <a href="#">CFG_getDeviceIdentificationInfo()</a> .                                          |
| out | <i>pManufacturerStr</i>      | User allocated zero terminated string of max length <a href="#">STAR_CFG_MANUFACTURER_STR_MAX_LEN</a> that will be updated with the manufacturers name, if known. |

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[device\\_identifier\\_example.c](#), and [star-test.c](#).

```
8.74.2.3 void STAR_API_CC CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *
pDeviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char * pDeviceTypeStr)
```

If the manufacturer and device type is known, this function obtains a string representation of the device's type.

**Parameters**

|     |                              |                                                                                                                                                      |
|-----|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| in  | <i>pDeviceIdentifierInfo</i> | Pointer to device identification information obtained from a call to <a href="#">CFG_getDeviceIdentificationInfo()</a> .                             |
| out | <i>pDeviceTypeStr</i>        | User allocated zero terminated string of max length <a href="#">STAR_CFG_DEVICE_STR_MAX_LEN</a> that will be updated with the device name, if known. |

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[device\\_identifier\\_example.c](#), and [star-test.c](#).

**8.74.2.4** `int STAR_API_CC CFG_getNetworkDiscoveryInfo ( STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO * pInfo )`

Reads the network discovery information of a device.

This information can be used by a network manager to determine the layout of the network.

**Parameters**

|     |                 |                                                                                             |
|-----|-----------------|---------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to obtain info from.                                                                 |
| out | <i>pInfo</i>    | Structure that will be updated to contain the network discovery information for the device. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

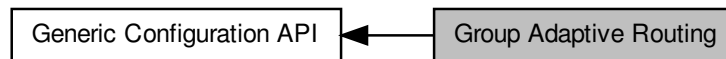
**Examples:**

[device\\_identifier\\_example.c](#).

## 8.75 Group Adaptive Routing

Functions in this group deal with group adaptive routing.

Collaboration diagram for Group Adaptive Routing:



### Functions

- `int STAR_API_CC_CFG_getRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY *pEntry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_setRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress, _In_ STAR_CFG_GAR_ENTRY *pEntry)`  
*Sets a routing table entry for a given logical address.*

#### 8.75.1 Detailed Description

Functions in this group deal with group adaptive routing.

#### 8.75.2 Function Documentation

**8.75.2.1** `int STAR_API_CC_CFG_getRoutingTableEntry ( STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY * pEntry )`

Gets a routing table entry for a given logical address.

##### Parameters

|            |                       |                                                    |
|------------|-----------------------|----------------------------------------------------|
|            | <i>deviceId</i>       | Device to get GAR table entry for.                 |
|            | <i>logicalAddress</i> | Logical address to get GAR table entry for.        |
| <i>out</i> | <i>pEntry</i>         | The GAR table entry for the given logical address. |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`routing_table_example.c.`

**8.75.2.2** `int STAR_API_CC CFG_setRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _In_ STAR_CFG_GAR_ENTRY * pEntry )`

Sets a routing table entry for a given logical address.

**Parameters**

|                       |                                                                              |
|-----------------------|------------------------------------------------------------------------------|
| <i>deviceID</i>       | Device to set GAR table entry on.                                            |
| <i>logicalAddress</i> | Logical address to set GAR table entry for. Valid values are 32 through 255. |

**Note**

Logical address 255 is reserved for future use and should not be used. See section 10.3.3n of the SpaceWire standard for more information.

**Parameters**

|           |               |                                                    |
|-----------|---------------|----------------------------------------------------|
| <i>in</i> | <i>pEntry</i> | The GAR table entry for the given logical address. |
|-----------|---------------|----------------------------------------------------|

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`routing_table_example.c.`

## 8.76 Interface Mode

Functions in this group deal with interface mode.

Collaboration diagram for Interface Mode:



### Functions

- `int STAR_API_CC CFG_getPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ U8 *pAddress)`  
*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC CFG_setPortRoutingAddress (STAR_DEVICE_ID deviceId, U8 portNum, U8 address)`  
*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- `int STAR_API_CC CFG_disableIdentifySource (STAR_DEVICE_ID deviceId)`  
*Disables identification of source ports for interface mode globally.*
- `int STAR_API_CC CFG_enableIdentifySource (STAR_DEVICE_ID deviceId)`  
*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- `int STAR_API_CC CFG_getIdentifySourceEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether identify source is enabled.*
- `int STAR_API_CC CFG_disableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Disables source port identification for a given port.*
- `int STAR_API_CC CFG_enableIdentifySourceOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Enables identification of source port during interface mode for a given port.*
- `int STAR_API_CC CFG_getIdentifySourceOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether identify source is enabled for a specific port.*
- `int STAR_API_CC CFG_disableInterfaceMode (STAR_DEVICE_ID deviceId)`  
*Disables interface mode on a device.*
- `int STAR_API_CC CFG_enableInterfaceMode (STAR_DEVICE_ID deviceId)`  
*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_setPortRoutingAddress()`).*
- `int STAR_API_CC CFG_getInterfaceModeEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether interface mode has been enabled on a device.*
- `int STAR_API_CC CFG_disableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables interface mode on a given port.*
- `int STAR_API_CC CFG_enableInterfaceModeOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables interface mode on a given port.*
- `int STAR_API_CC CFG_getInterfaceModeOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether interface mode is enabled on a specific port.*

### 8.76.1 Detailed Description

Functions in this group deal with interface mode.

### 8.76.2 Function Documentation

#### 8.76.2.1 `int STAR_API_CC CFG_disableIdentifySource ( STAR_DEVICE_ID deviceID )`

Disables identification of source ports for interface mode globally.

##### Parameters

|                 |                                                  |
|-----------------|--------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification on. |
|-----------------|--------------------------------------------------|

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

##### Examples:

[brickMk2\\_configuration\\_example.c](#).

#### 8.76.2.2 `int STAR_API_CC CFG_disableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Disables source port identification for a given port.

##### Parameters

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| <i>deviceID</i> | Device to disable source port identification for a given port on. |
| <i>portNum</i>  | Port to disable source identification on.                         |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

##### Examples:

[brickMk2\\_configuration\\_example.c](#), and [routerMk2S\\_configuration\\_example.c](#).



**8.76.2.3** `int STAR_API_CC CFG_disableInterfaceMode ( STAR_DEVICE_ID deviceID )`

Disables interface mode on a device.

When interface mode is disabled, the device operates in routing mode.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Device to disable interface mode on. |
|-----------------|--------------------------------------|

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[link\\_events.c](#), [packet\\_subsystem\\_example.c](#), and [rmap\\_target\\_example.c](#).

**8.76.2.4** `int STAR_API_CC CFG_disableInterfaceModeOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables interface mode on a given port.

When interface mode is disabled, the device operates in routing mode.

**Parameters**

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to disable interface mode for a port on. |
| <i>portNum</i>  | Port to disable interface mode on.              |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), and [routerMk2S\\_configuration\\_example.c](#).

#### 8.76.2.5 `int STAR_API_CC CFG_enableIdentifySource ( STAR_DEVICE_ID deviceID )`

When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.

For each source port on which this behaviour is desired, a call to `CFG_enableIdentifySourceOnPort()` must be made.

##### Parameters

|                 |                                                 |
|-----------------|-------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification on. |
|-----------------|-------------------------------------------------|

##### Returns

0 for success, or a negative error code upon failure as defined in `Defines`.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

##### Examples:

`brickMk2_configuration_example.c`, `mk2_configuration_example.c`, `pciMk2_configuration_example.c`, and `routerMk2S_configuration_example.c`.

#### 8.76.2.6 `int STAR_API_CC CFG_enableIdentifySourceOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Enables identification of source port during interface mode for a given port.

Interface mode (`CFG_enableInterfaceMode()`) and Identify Source (`CFG_enableIdentifySource()`) must be enabled globally for this to have an effect.

##### Parameters

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <i>deviceID</i> | Device to enable source port identification for a given port on. |
| <i>portNum</i>  | Port to enable source identification on.                         |

##### Returns

0 for success, or a negative error code upon failure as defined in `Defines`.

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

##### Examples:

`brickMk2_configuration_example.c`, `mk2_configuration_example.c`, `pciMk2_configuration_example.c`, and `routerMk2S_configuration_example.c`.

#### 8.76.2.7 int STAR\_API\_CC\_CFG\_enableInterfaceMode ( STAR\_DEVICE\_ID *deviceId* )

In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by [CFG\\_setPortRoutingAddress\(\)](#)).

**Parameters**


---

|                 |                                     |
|-----------------|-------------------------------------|
| <i>deviceID</i> | Device to enable interface mode on. |
|-----------------|-------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), [link\\_events.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [routerMk2S\\_configuration\\_example.c](#), and [timestamp\\_test.c](#).

#### 8.76.2.8 int STAR\_API\_CC CFG\_enableInterfaceModeOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )

Selectively enables interface mode on a given port.

Interface mode must be enabled globally ([CFG\\_enableInterfaceMode\(\)](#)) for interface mode on a port to be enabled.

**Parameters**


---

|                 |                                                |
|-----------------|------------------------------------------------|
| <i>deviceID</i> | Device to enable interface mode for a port on. |
| <i>portNum</i>  | Port to enable interface mode on.              |

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), and [routerMk2S\\_configuration\\_example.c](#).

#### 8.76.2.9 int STAR\_API\_CC CFG\_getIdentifySourceEnabled ( STAR\_DEVICE\_ID deviceID, \_Out\_ int \* pEnabled )

Gets whether identify source is enabled.

**Parameters**

|     |                 |                                                                                      |
|-----|-----------------|--------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to check.                                                                     |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.76.2.10** `int STAR_API_CC_CFG_getIdentifySourceOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether identify source is enabled for a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pEnabled is always 0.

**Parameters**

|     |                 |                                                                                      |
|-----|-----------------|--------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to check.                                                                     |
|     | <i>portNum</i>  | Port to check.                                                                       |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if identify source is enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

`brickMk2_configuration_example.c`, and `routerMk2S_configuration_example.c`.

**8.76.2.11** `int STAR_API_CC_CFG_getInterfaceModeEnabled ( STAR_DEVICE_ID deviceID, _Out_ int * pEnabled )`

Gets whether interface mode has been enabled on a device.

**Parameters**

|     |                 |                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | The device to check.                                                                |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.76.2.12** `int STAR_API_CC_CFG_getInterfaceModeOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether interface mode is enabled on a specific port.

**Note**

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pEnabled is always 0.

**Parameters**

|     |                 |                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get interface mode enabled for a port on.                                 |
|     | <i>portNum</i>  | Port to get information for.                                                        |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if interface mode is enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), and [routerMk2S\\_configuration\\_example.c](#).

**8.76.2.13** `int STAR_API_CC_CFG_getPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ U8 * pAddress )`

Gets the address a packet received on a given port should be routed to when interface mode is enabled.

#### Note

This is an advanced feature and not necessary for normal usage.

SpaceWire PCI Mk2 and cPCI Mk2 and SpaceWire PCIe devices prior to version 1.07 have a bug which means that the value returned in pAddress is always 0.

#### Parameters

|     |                 |                                                               |
|-----|-----------------|---------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get port routing address from.                      |
|     | <i>portNum</i>  | Port to get port routing address from.                        |
| out | <i>pAddress</i> | Pointer to value that will be updated to contain the address. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**8.76.2.14** `int STAR_API_CC_CFG_setPortRoutingAddress ( STAR_DEVICE_ID deviceID, U8 portNum, U8 address )`

Sets the address a packet received on a given port should be routed to when interface mode is enabled.

#### Note

This is an advanced feature and not necessary for normal usage.

#### Parameters

|  |                 |                                                               |
|--|-----------------|---------------------------------------------------------------|
|  | <i>deviceID</i> | Device to have one of its ports port routing address changed. |
|  | <i>portNum</i>  | Port to have its port routing address modified.               |
|  | <i>address</i>  | New port routing address.                                     |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

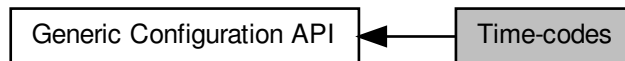
#### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

## 8.77 Time-codes

Functions in this group deal with time-codes.

Collaboration diagram for Time-codes:



### Functions

- `int STAR_API_CC_CFG_getTimeCodeDistributionCapablePorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports are capable of forwarding time-codes on.*
- `int STAR_API_CC_CFG_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)`  
*Obtains the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_setTimeCodeFlagMode (STAR_DEVICE_ID deviceId, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_getTimeCodeValue (STAR_DEVICE_ID deviceId, _Out_ U8 *pValue)`  
*Obtains the current value of the router's internal time-code counter.*
- `int STAR_API_CC_CFG_disableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Disables the device as a time-code master.*
- `int STAR_API_CC_CFG_enableTimeCodeMaster (STAR_DEVICE_ID deviceId)`  
*Enables the device as a time-code master.*
- `int STAR_API_CC_CFG_getTimeCodeMasterEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether the device has been enabled as a time-code master.*
- `int STAR_API_CC_CFG_disableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Disables external time-code selection for the device.*
- `int STAR_API_CC_CFG_enableExternalTimeCodeSelection (STAR_DEVICE_ID deviceId)`  
*Enables external time-code selection for the device.*
- `int STAR_API_CC_CFG_getExternalTimeCodeSelectionEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Gets whether external time-code selection has been enabled for the device.*
- `int STAR_API_CC_CFG_setTimeCodePeriod (STAR_DEVICE_ID deviceId, U32 period)`  
*Sets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_getTimeCodePeriod (STAR_DEVICE_ID deviceId, _Out_ U32 *pPeriod)`  
*Gets the period between time-code master ticks.*
- `int STAR_API_CC_CFG_disableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`  
*Disables time-code counter bypass mode for the device.*
- `int STAR_API_CC_CFG_enableTimeCodeCounterBypassMode (STAR_DEVICE_ID deviceId)`



*Enables time-code counter bypass mode for the device.*

- `int STAR_API_CC_CFG_getTimeCodeCounterBypassModeEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`

*Gets whether time-code counter bypass mode has been enabled for the device.*

### 8.77.1 Detailed Description

Functions in this group deal with time-codes.

### 8.77.2 Function Documentation

#### 8.77.2.1 `int STAR_API_CC_CFG_disableExternalTimeCodeSelection ( STAR_DEVICE_ID deviceId )`

Disables external time-code selection for the device.

When external time-code selection is disabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device.

#### Note

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

#### Parameters

---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceId</i> | The device to disable external time-code selection for. |
|-----------------|---------------------------------------------------------|

---

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire Router Mk2S](#)

#### Examples:

[time-code\\_test.c](#).

#### 8.77.2.2 `int STAR_API_CC_CFG_disableTimeCodeCounterBypassMode ( STAR_DEVICE_ID deviceId )`

Disables time-code counter bypass mode for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded.

#### Note

This function is currently only supported by [SpaceWire PCIe](#) devices.

**Parameters**


---

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | The device to disable time-code counter bypass mode for. |
|-----------------|----------------------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe (version 1.12 and greater), SpaceWire PCIe Mk2

**8.77.2.3 int STAR\_API\_CC\_CFG\_disableTimeCodeMaster ( STAR\_DEVICE\_ID *deviceID* )**

Disables the device as a time-code master.

**Parameters**


---

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | The device to disable as a time-code master. |
|-----------------|----------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

brickMk2\_configuration\_example.c, and time-code\_test.c.

**8.77.2.4 int STAR\_API\_CC\_CFG\_enableExternalTimeCodeSelection ( STAR\_DEVICE\_ID *deviceID* )**

Enables external time-code selection for the device.

When external time-code selection is enabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Note**

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

**Parameters**

---

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceID</i> | The device to enable external time-code selection for. |
|-----------------|--------------------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**Examples:**

time-code\_test.c.

**8.77.2.5 int STAR\_API\_CC\_CFG\_enableTimeCodeCounterBypassMode ( STAR\_DEVICE\_ID deviceID )**

Enables time-code counter bypass mode for the device.

When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices.

**Parameters**

---

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | The device to enable time-code counter bypass mode for. |
|-----------------|---------------------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire PCIe (version 1.12 and greater), SpaceWire PCIe Mk2

**8.77.2.6 int STAR\_API\_CC\_CFG\_enableTimeCodeMaster ( STAR\_DEVICE\_ID deviceID )**

Enables the device as a time-code master.

**Parameters**

---

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceID</i> | The device to enable as a time-code master. |
|-----------------|---------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), [mk2\\_configuration\\_example.c](#), [pciMk2\\_configuration\\_example.c](#), [routerMk2S\\_configuration\\_example.c](#), and [time-code\\_test.c](#).

**8.77.2.7** `int STAR_API_CC_CFG_getExternalTimeCodeSelectionEnabled ( STAR_DEVICE_ID deviceId, _Out_ int * pEnabled )`

Gets whether external time-code selection has been enabled for the device.

When external time-code selection is disabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is ignored, and the next valid time-code is transmitted by the device. When external time-code selection is enabled and a time-code is transmitted from an external source such as a time-code port, the value specified for the time-code is used. Note that the time-code will only be transmitted by the device if it is the next valid time-code.

**Note**

This setting only applies to the [External Time-code Port](#) on the rear panel of the [SpaceWire Router Mk2S](#) when using the SEL\_EXT\_TIME pin which can automatically increment on EXT\_TICK\_IN.

**Parameters**

|     |                 |                                                                                                                  |
|-----|-----------------|------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | The device to check.                                                                                             |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if external time-code selection is enabled for the device, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

[SpaceWire Router Mk2S](#)

**8.77.2.8** `int STAR_API_CC_CFG_getTimeCodeCounterBypassModeEnabled ( STAR_DEVICE_ID deviceId, _Out_ int * pEnabled )`

Gets whether time-code counter bypass mode has been enabled for the device.

When time-code counter bypass mode is disabled, any time-code which is received will be checked against the current value of the counter for validity before being forwarded. When time-code counter bypass mode is enabled, any time-code which is received will be forwarded out of the other ports on the router, without checking against the current value of the counter. The time-code register will be updated on each received time-code.

**Note**

This function is currently only supported by [SpaceWire PCIe](#) devices.

**Parameters**

|     |                 |                                                                                                                   |
|-----|-----------------|-------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | The device to check.                                                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code counter bypass mode is enabled for the device, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

[STAR-System v4.00 - 19th November 2019](#)

**Supported devices:**

[SpaceWire PCIe](#) (version 1.12 and greater), [SpaceWire PCIe Mk2](#)

**8.77.2.9** `int STAR_API_CC_CFG_getTimeCodeDistributionCapablePorts ( STAR_DEVICE_ID deviceID, _Out_ U32 * pPortMask )`

This function is used to obtain which output ports are capable of forwarding time-codes on.

**Parameters**

|     |                  |                                                                      |
|-----|------------------|----------------------------------------------------------------------|
|     | <i>deviceID</i>  | Device to get the time-code distribution ports for.                  |
| out | <i>pPortMask</i> | Bit mask of all ports that time-code distribution can be enabled on. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

[STAR-System v6.00 - 7th October 2024](#)

**Supported devices:**

All STAR-System Devices ([SpaceWire Router Mk2S](#), [SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [STAR Fire Mk3](#), [SpaceWire Router Mk2S](#), [SpaceWire PXI Interface and PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2](#), [SpaceWire PCI Mk2](#) and [cPCI Mk2](#), [SpaceWire PCIe](#), [SpaceWire PCIe Mk2](#), [SpaceWire Physical Layer Tester](#))

**Examples:**

`time-code_test.c`.

**8.77.2.10** `int STAR_API_CC_CFG_getTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceID, _Out_ U32 * pPortMask )`

This function is used to obtain which output ports time-codes are forwarded on.

**Parameters**

|     |                  |                                                                     |
|-----|------------------|---------------------------------------------------------------------|
|     | <i>deviceID</i>  | Device to get the time-code distribution ports for.                 |
| out | <i>pPortMask</i> | Bit mask of ports that time-code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[router\\_configuration\\_example.c](#).

#### 8.77.2.11 int STAR\_API\_CC\_CFG\_getTimeCodeFlagMode ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U8 \* *pMode* )

Obtains the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceID</i> | Device to get the time-code flag mode from. |
| out | <i>pMode</i>    | Time-code flag mode.                        |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

**Examples:**

[router\\_configuration\\_example.c](#).

#### 8.77.2.12 int STAR\_API\_CC\_CFG\_getTimeCodeMasterEnabled ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ int \* *pEnabled* )

Gets whether the device has been enabled as a time-code master.

**Parameters**

|     |                 |                                                                                                       |
|-----|-----------------|-------------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | The device to check.                                                                                  |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if the device is enabled as a time-code master, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

#### 8.77.2.13 int STAR\_API\_CC\_CFG\_getTimeCodePeriod ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U32 \* *pPeriod* )

Gets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|     |                 |                                                                                     |
|-----|-----------------|-------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get time-code period from.                                                |
| out | <i>pPeriod</i>  | A pointer to a variable that will be updated to contain the period in microseconds. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[brickMk2\\_configuration\\_example.c](#), and [time-code\\_test.c](#).

#### 8.77.2.14 int STAR\_API\_CC\_CFG\_getTimeCodeValue ( STAR\_DEVICE\_ID *deviceId*, \_Out\_ U8 \* *pValue* )

Obtains the current value of the router's internal time-code counter.

•

**Parameters**

|     |                 |                                             |
|-----|-----------------|---------------------------------------------|
|     | <i>deviceID</i> | Device to get the time-code flag mode from. |
| out | <i>pValue</i>   | Time-code value,                            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[router\\_configuration\\_example.c](#).

#### 8.77.2.15 `int STAR_API_CC_CFG_setTimeCodeDistributionPorts ( STAR_DEVICE_ID deviceID, U32 portMask )`

This function is used to specify which output ports time-codes are forwarded on.

**Parameters**

|  |                 |                                                                                                                                                       |
|--|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>deviceID</i> | Device to set the time-code distribution ports for.                                                                                                   |
|  | <i>portMask</i> | Bit mask of ports that time-code distribution should be enabled on. Valid ports to forward time-codes on are 1 through to 13 depending on the device. |

**Note**

Bit 1 corresponds to port 1 and bit 0 is unused.  
Not all external ports allow time-code forwarding. Check your device's user manual for more details.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

[router\\_configuration\\_example.c](#), and [time-code\\_test.c](#).



**8.77.2.16** int **STAR\_API\_CC\_CFG\_setTimeCodeFlagMode** ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *mode* )

Sets the time-code flag interpretation mode:

0 - Time-code control bit flags are distributed with valid time-code values regardless of the value of the time-code control flags

1 - When the time-code control bit flags are "00" then valid time-codes are distributed. When the time-code control flags are not "00" then the time-code is discarded and the internal time-code register is not updated.

**Parameters**

|                 |                                             |
|-----------------|---------------------------------------------|
| <i>deviceId</i> | Device to set the time-code flag modes for. |
| <i>mode</i>     | Mode to set the time-code flag mode to.     |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester

**8.77.2.17** int **STAR\_API\_CC\_CFG\_setTimeCodePeriod** ( **STAR\_DEVICE\_ID** *deviceId*, **U32** *period* )

Sets the period between time-code master ticks.

This is the number of microseconds between time-codes.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceId</i> | Device to set time-code period for.   |
| <i>period</i>   | The period to be set in microseconds. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

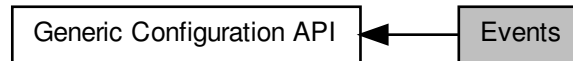
**Examples:**

brickMk2\_configuration\_example.c, mk2\_configuration\_example.c, pciMk2\_configuration\_example.c, router↔Mk2S\_configuration\_example.c, and time-code\_test.c.

## 8.78 Events

Functions in this group deal with events.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC_CFG_disableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables speed change events on a given port.*
- `int STAR_API_CC_CFG_enableSpeedChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables speed change events on a given port.*
- `int STAR_API_CC_CFG_getSpeedChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether speed change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_disableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables state change events on a given port.*
- `int STAR_API_CC_CFG_enableStateChangeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables state change events on a given port.*
- `int STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether state change events are enabled on a specific port.*
- `int STAR_API_CC_CFG_disableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC_CFG_enableTimeCodeEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- `int STAR_API_CC_CFG_getTimeCodeEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether time-code notifications are enabled on a specific port.*

### 8.78.1 Detailed Description

Functions in this group deal with events.

### 8.78.2 Function Documentation

#### 8.78.2.1 `int STAR_API_CC_CFG_disableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables speed change events on a given port.

Disabling speed change events will result in speed change event traffic no longer being received on channel 0 when the link speed changes.

**Note**

On the [SpaceWire PCIe](#), link speed events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable speed change events for a port on. |
| <i>portNum</i>  | Port to disable speed change events on.              |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**Examples:**

[brickMk2\\_configuration\\_example.c](#), and [link\\_events.c](#).

### 8.78.2.2 `int STAR_API_CC CFG_disableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively disables state change events on a given port.

Disabling state change events will result in state change event traffic no longer being received on channel 0 when the link changes to running or disconnected.

**Note**

On the [SpaceWire PCIe](#), state events are received on the channel associated with the port rather than channel 0.

**Parameters**

|                 |                                                      |
|-----------------|------------------------------------------------------|
| <i>deviceID</i> | Device to disable state change events for a port on. |
| <i>portNum</i>  | Port to disable state change events on.              |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**Examples:**

[link\\_events.c](#).

### 8.78.2.3 `int STAR_API_CC_CFG_disableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables time-code notifications on a the channel attached to a given port.

#### Note

This function is currently compatible with [SpaceWire PCIe](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) devices.

#### Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceId</i> | Device to disable time-code notifications for a port on. |
| <i>portNum</i>  | Port to disable time-code notifications on.              |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire PCIe Mk2](#), [SpaceWire PCIe](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#)

### 8.78.2.4 `int STAR_API_CC_CFG_enableSpeedChangeEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively enables speed change events on a given port.

Enabling speed change events will result in speed change event traffic being received on channel 0 when the link speed changes.

#### Note

On the [SpaceWire PCIe](#), link speed events are received on the channel associated with the port rather than channel 0.

#### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceId</i> | Device to enable speed change events for a port on. |
| <i>portNum</i>  | Port to enable speed change events on.              |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire Brick Mk2](#), [SpaceWire Brick Mk3](#) and [Brick Mk4](#), [SpaceWire GbE Brick Mk1](#), [SpaceWire Router Mk2S](#), [STAR Fire Mk3](#), [SpaceWire PCIe Mk2](#), [SpaceWire PCIe](#), [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#), [SpaceWire PXI 12 Port Router](#) and [PXI 12 Port Router Mk2](#), [SpaceWire Physical Layer Tester](#)

#### Examples:

[brickMk2\\_configuration\\_example.c](#), and [link\\_events.c](#).

#### 8.78.2.5 `int STAR_API_CC_CFG_enableStateChangeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables state change events on a given port.

Enabling state change events will result in state change event traffic being received on channel 0 when the link changes to running or disconnected.

##### Note

On the [SpaceWire PCIe](#), state events are received on the channel associated with the port rather than channel 0.

##### Parameters

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <i>deviceID</i> | Device to enable state change events for a port on. |
| <i>portNum</i>  | Port to enable state change events on.              |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

##### Examples:

[link\\_events.c](#).

#### 8.78.2.6 `int STAR_API_CC_CFG_enableTimeCodeEventsOnPort ( STAR_DEVICE_ID deviceID, U8 portNum )`

Selectively enables time-code notifications on a the channel attached to a given port.

##### Note

This function is currently compatible with [SpaceWire PCIe](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.

##### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Device to enable time-code notifications for a port on. |
| <i>portNum</i>  | Port to enable time-code notifications on.              |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v4.00 - 19th November 2019

##### Supported devices:

SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2

8.78.2.7 `int STAR_API_CC CFG_getSpeedChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceID, U8 portNum, _Out_ int * pEnabled )`

Gets whether speed change events are enabled on a specific port.

**Parameters**

|     |                 |                                                                                           |
|-----|-----------------|-------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get whether speed change events are enabled for a port.                         |
|     | <i>portNum</i>  | Port to get information for.                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if speed change events are enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.78.2.8** `int STAR_API_CC_CFG_getStateChangeEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether state change events are enabled on a specific port.

**Parameters**

|     |                 |                                                                                           |
|-----|-----------------|-------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get whether state change events are enabled for a port.                         |
|     | <i>portNum</i>  | Port to get information for.                                                              |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if state change events are enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**8.78.2.9** `int STAR_API_CC_CFG_getTimeCodeEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether time-code notifications are enabled on a specific port.

**Note**

This function is currently compatible with [SpaceWire PCIe](#) devices and [SpaceWire PXI Interface and PXI Interface Mk2](#) devices.

**Parameters**

|     |                 |                                                                                                       |
|-----|-----------------|-------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get whether time-code notifications are enabled for a port.                                 |
|     | <i>portNum</i>  | Port to get information for.                                                                          |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if time-code notifications on port are enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

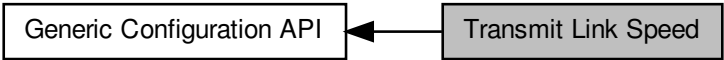
SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2



## 8.79 Transmit Link Speed

Functions in this group deal with controlling transmit link speed.

Collaboration diagram for Transmit Link Speed:



### Functions

- `int STAR_API_CC_CFG_getTxSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ double *pSignallingRate)`  
*Gets the transmit bit rate for a given link in Mbit/s.*
- `int STAR_API_CC_CFG_setTxSignallingRate (STAR_DEVICE_ID deviceId, U8 linkNum, double signallingRate, _Out_ double *pSignallingRateSet)`  
*Sets the transmit bit rate on a given link.*

### 8.79.1 Detailed Description

Functions in this group deal with controlling transmit link speed.

### 8.79.2 Function Documentation

**8.79.2.1** `int STAR_API_CC_CFG_getTxSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ double *pSignallingRate )`

Gets the transmit bit rate for a given link in Mbit/s.

#### Note

This function is compatible with all devices.

#### Parameters

|     |                        |                                                                                          |
|-----|------------------------|------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>        | Device to get the transmit bit rate from.                                                |
|     | <i>linkNum</i>         | Link to get the transmit bit rate for.                                                   |
| out | <i>pSignallingRate</i> | Pointer to value that will be updated with the given link's transmit bit rate in Mbit/s. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 - 7th October 2024

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

brickMk2\_configuration\_example.c, and pciMk2\_configuration\_example.c.

**8.79.2.2** `int STAR_API_CC_CFG_setTxSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, double signallingRate, _Out_ double * pSignallingRateSet )`

Sets the transmit bit rate on a given link.

The transmit bit rate of a link is given in Mbit/s. The bit rate will be between 2 Mbit/s and 400 Mbit/s inclusive, although 400 Mbit/s may not be achievable with every device type.

**Note**

This function is compatible with all devices. However, depending upon the capabilities of the device, an exact bit rate may not be achievable.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, setting the transmit bit rate for link 1 will also affect link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to set the transmit clock frequency for each pair of links.

**Parameters**

|                           |                                                 |
|---------------------------|-------------------------------------------------|
| <i>deviceId</i>           | Device to modify a link's transmit bit rate on. |
| <i>linkNum</i>            | Link to have its transmit bit rate modified.    |
| <i>signallingRate</i>     | Transmit bit rate in Mbit/s.                    |
| <i>pSignallingRateSet</i> | The actual bit rate that was set in Mbit/s.     |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 - 7th October 2024

**Supported devices:**

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

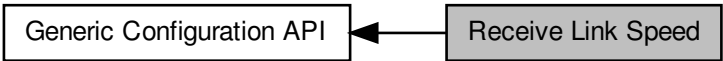
**Examples:**

brickMk2\_configuration\_example.c, link\_events.c, mk2\_configuration\_example.c, and pciMk2\_configuration\_example.c.

## 8.80 Receive Link Speed

Functions in this group deal with measuring receive link speed.

Collaboration diagram for Receive Link Speed:



### Functions

- `int STAR_API_CC_CFG_getRxSignallingRate (STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ double *pSignallingRate)`  
*Gets the receive bit rate for a given link in Mbit/s.*

#### 8.80.1 Detailed Description

Functions in this group deal with measuring receive link speed.

#### 8.80.2 Function Documentation

**8.80.2.1** `int STAR_API_CC_CFG_getRxSignallingRate ( STAR_DEVICE_ID deviceID, U8 linkNum, _Out_ double *pSignallingRate )`

Gets the receive bit rate for a given link in Mbit/s.

**Parameters**

|            |                        |                                                                                         |
|------------|------------------------|-----------------------------------------------------------------------------------------|
|            | <i>deviceID</i>        | Device to get the receive bit rate for.                                                 |
|            | <i>linkNum</i>         | Link to get the receive bit rate for.                                                   |
| <i>out</i> | <i>pSignallingRate</i> | Pointer to value that will be updated with the given link's receive bit rate in Mbit/s. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 - 7th October 2024

**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester

**Examples:**

[brickMk2\\_configuration\\_example.c](#), and [link\\_events.c](#).

## 8.81 Precision Links

Functions in this group deal with controlling link speed with greater precision.

Collaboration diagram for Precision Links:



### Functions

- `int STAR_API_CC_CFG_disablePrecisionTransmitRate (STAR_DEVICE_ID deviceId)`  
*Disables precision transmit rate.*
- `int STAR_API_CC_CFG_enablePrecisionTransmitRate (STAR_DEVICE_ID deviceId)`  
*Enables precision transmit rate.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateEnabled (STAR_DEVICE_ID deviceId, _Out_ int *pEnabled)`  
*Determine if precision transmit rate is enabled.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRateInUse (STAR_DEVICE_ID deviceId, _Out_ int *pInUse)`  
*Determine whether precision transmit rate is in use.*
- `int STAR_API_CC_CFG_getPrecisionTransmitRate (STAR_DEVICE_ID deviceId, _Out_ double *pTransmitRate)`  
*Get the current precision transmit rate on a device in Mbit/s.*
- `int STAR_API_CC_CFG_setPrecisionTransmitRate (STAR_DEVICE_ID deviceId, double transmitRate)`  
*Set the current precision transmit rate on a device in Mbit/s.*

### 8.81.1 Detailed Description

Functions in this group deal with controlling link speed with greater precision.

### 8.81.2 Function Documentation

#### 8.81.2.1 `int STAR_API_CC_CFG_disablePrecisionTransmitRate ( STAR_DEVICE_ID deviceId )`

Disables precision transmit rate.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function `CFG_setPrecisionTransmitRate()`. Note that after being disabled there may be a short delay (less than 250 microseconds) before precision transmit rate is actually disabled. Call `CFG_getPrecisionTransmitRateInUse()` to determine if precision transmit rate is no longer in use.

#### Note

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

---

|                 |                                               |
|-----------------|-----------------------------------------------|
| <i>deviceID</i> | Device to disable precision transmit rate on. |
|-----------------|-----------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**8.81.2.2 int STAR\_API\_CC CFG\_enablePrecisionTransmitRate ( STAR\_DEVICE\_ID deviceID )**

Enables precision transmit rate.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#). Note that after being enabled it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_getPrecisionTransmitRateInUse\(\)](#) to determine if precision transmit rate is actually in use.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

---

|                 |                                              |
|-----------------|----------------------------------------------|
| <i>deviceID</i> | Device to enable precision transmit rate on. |
|-----------------|----------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**Examples:**

[routerMk2S\\_configuration\\_example.c](#).

**8.81.2.3 int STAR\_API\_CC CFG\_getPrecisionTransmitRate ( STAR\_DEVICE\_ID deviceID, \_Out\_ double \* pTransmitRate )**

Get the current precision transmit rate on a device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#).

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|     |                      |                                                                                            |
|-----|----------------------|--------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>      | Device to get the precision transmit rate.                                                 |
| out | <i>pTransmitRate</i> | User supplied value that will be updated to contain the precision transmit rate in Mbit/s. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**Examples:**

[routerMk2S\\_configuration\\_example.c](#).

#### 8.81.2.4 int STAR\_API\_CC CFG\_getPrecisionTransmitRateEnabled ( STAR\_DEVICE\_ID deviceId, \_Out\_ int \* pEnabled )

Determine if precision transmit rate is enabled.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number using the function [CFG\\_setPrecisionTransmitRate\(\)](#). Note that although precision transmit rate may be enabled, it can take up to 250 microseconds before precision transmit rate is actually used. Call [CFG\\_getPrecisionTransmitRateInUse\(\)](#) to determine if precision transmit rate is actually in use.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|     |                 |                                                                                              |
|-----|-----------------|----------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get whether precision transmit rate is enabled.                                    |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if precision transmit rate is enabled, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

SpaceWire Router Mk2S

**Examples:**

[routerMk2S\\_configuration\\_example.c](#).

**8.81.2.5** `int STAR_API_CC CFG_getPrecisionTransmitRateInUse ( STAR_DEVICE_ID deviceId, _Out_ int * pInUse )`

Determine whether precision transmit rate is in use.

After being enabled or disabled, it can take up to 250 microseconds before precision transmit is actually used or not used. This function determines whether precision transmit rate is currently in use and can be used to determine if an enable or disable operation has completed.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|            |                 |                                                                                             |
|------------|-----------------|---------------------------------------------------------------------------------------------|
|            | <i>deviceId</i> | Device to get whether precision transmit rate is in use.                                    |
| <i>out</i> | <i>pInUse</i>   | User supplied value that will be updated to 1 if precision transmit rate is in use, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

[STAR-System v4.00 - 19th November 2019](#)

**Supported devices:**

[SpaceWire Router Mk2S](#)

**Examples:**

[routerMk2S\\_configuration\\_example.c](#).

**8.81.2.6** `int STAR_API_CC CFG_setPrecisionTransmitRate ( STAR_DEVICE_ID deviceId, double transmitRate )`

Set the current precision transmit rate on a device in Mbit/s.

Precision transmit rate allows the transmit rate of the device to be expressed accurately in Mbit/s as a floating point number.

**Note**

This function is currently compatible with [SpaceWire Router Mk2S](#) devices.

**Parameters**

|  |                     |                                                |
|--|---------------------|------------------------------------------------|
|  | <i>deviceId</i>     | Device to set the precision transmit rate for. |
|  | <i>transmitRate</i> | The precision transmit rate in Mbit/s to set.  |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

[STAR-System v4.00 - 19th November 2019](#)

**Supported devices:**

[SpaceWire Router Mk2S](#)

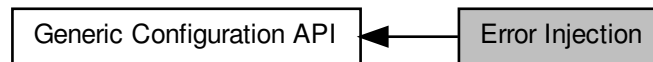
**Examples:**

[routerMk2S\\_configuration\\_example.c](#).

## 8.82 Error Injection

Functions in this group deal with error injection.

Collaboration diagram for Error Injection:



### Functions

- `int STAR_API_CC_CFG_injectError (STAR_DEVICE_ID deviceID, U8 portNum, SPW_ERROR error)`  
*Immediately injects the specified error on a given device's port.*
- `int STAR_API_CC_CFG_injectErrors (STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS *pErrors)`  
*Inject the specified errors on a given port.*

### 8.82.1 Detailed Description

Functions in this group deal with error injection.

### 8.82.2 Function Documentation

#### 8.82.2.1 `int STAR_API_CC_CFG_injectError ( STAR_DEVICE_ID deviceID, U8 portNum, SPW_ERROR error )`

Immediately injects the specified error on a given device's port.

#### Note

Error injection makes use of the same on-board registers as periodic actions on devices which support it, therefore error injection cannot be used on those devices when periodic actions are in progress.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have the error injected on. |
| <i>portNum</i>  | Port the error is to be injected on.                    |
| <i>error</i>    | SpaceWire error to be injected.                         |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v4.00 - 19th November 2019

Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI



Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

**Examples:**

mk2\_configuration\_example.c, and pciMk2\_configuration\_example.c.

**8.82.2.2** `int STAR_API_CC CFG_injectErrors ( STAR_DEVICE_ID deviceID, U8 portNum, _In_ STAR_CFG_BRICK_MK2_ERRORS * pErrors )`

Inject the specified errors on a given port.

**Parameters**

|           |                 |                                                                  |
|-----------|-----------------|------------------------------------------------------------------|
|           | <i>deviceID</i> | Device to inject the errors on.                                  |
|           | <i>portNum</i>  | Port to inject the errors on.                                    |
| <i>in</i> | <i>pErrors</i>  | Pointer to structure specifying which errors are to be injected. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

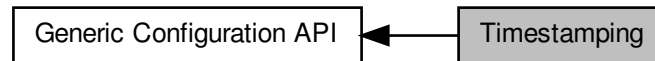
**Supported devices:**

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2

## 8.83 Timestamping

Functions in this group deal with timestampings.

Collaboration diagram for Timestamping:



### Functions

- `int STAR_API_CC_CFG_disableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively disables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_enableRxTimestampEventsOnPort (STAR_DEVICE_ID deviceId, U8 portNum)`  
*Selectively enables receive timestamp events on a given port.*
- `int STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int *pEnabled)`  
*Gets whether receive timestamp events are enabled on a specific port.*
- `int STAR_API_CC_CFG_getPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ *pFrequency)`  
*Gets the frequency of each generated pulse.*
- `int STAR_API_CC_CFG_setPulseGeneratorFrequency (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency)`  
*Sets the frequency of each generated pulse.*
- `int STAR_API_CC_CFG_getTimestampMethod (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD *pMethod)`  
*Gets the timestamping method that is currently in use for the given device.*
- `int STAR_API_CC_CFG_setTimestampMethod (STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method)`  
*Sets the timestamping method to use for the given device.*
- `int STAR_API_CC_CFG_getTimestampValue (STAR_DEVICE_ID deviceId, _Out_ U32 *pValue)`  
*Gets the current timestamp value.*
- `int STAR_API_CC_CFG_setTimestampValue (STAR_DEVICE_ID deviceId, U32 value)`  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*

### 8.83.1 Detailed Description

Functions in this group deal with timestampings.

### 8.83.2 Function Documentation

#### 8.83.2.1 `int STAR_API_CC_CFG_disableRxTimestampEventsOnPort ( STAR_DEVICE_ID deviceId, U8 portNum )`

Selectively disables receive timestamp events on a given port.

Disabling receive timestamp events will result in no timestamp information being provided after the packet.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|                 |                                                           |
|-----------------|-----------------------------------------------------------|
| <i>deviceID</i> | Device to disable receive timestamp events for a port on. |
| <i>portNum</i>  | Port to disable receive timestamp events on.              |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

**8.83.2.2 int STAR\_API\_CC\_CFG\_enableRxTimestampEventsOnPort ( STAR\_DEVICE\_ID deviceID, U8 portNum )**

Selectively enables receive timestamp events on a given port.

Enabling receive timestamp events will result in the timestamp that the last packet was received at to be provided as a STAR\_STREAM\_ITEM\_TYPE\_RX\_TIMESTAMP.

**Note**

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

**Parameters**

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Device to enable receive timestamp events for a port on. |
| <i>portNum</i>  | Port to enable receive timestamp events on.              |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

**Examples:**

[timestamp\\_test.c](#).

### 8.83.2.3 `int STAR_API_CC_CFG_getPulseGeneratorFrequency ( STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_BRICK_MK3_PULSE_FREQ * pFrequency )`

Gets the frequency of each generated pulse.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

#### Parameters

|     |                   |                                                                                    |
|-----|-------------------|------------------------------------------------------------------------------------|
|     | <i>deviceId</i>   | Device to get the pulse generator frequency value for.                             |
| out | <i>pFrequency</i> | Pointer to value that will be updated with the frequency between each pulse cycle. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

### 8.83.2.4 `int STAR_API_CC_CFG_getRxTimestampEventsOnPortEnabled ( STAR_DEVICE_ID deviceId, U8 portNum, _Out_ int * pEnabled )`

Gets whether receive timestamp events are enabled on a specific port.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

#### Parameters

|     |                 |                                                                                                |
|-----|-----------------|------------------------------------------------------------------------------------------------|
|     | <i>deviceId</i> | Device to get whether receive timestamp events are enabled for a port.                         |
|     | <i>portNum</i>  | Port to get receive timestamp events for.                                                      |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if receive timestamp events are enabled, else 0. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

**8.83.2.5** `int STAR_API_CC_CFG_getTimestampMethod ( STAR_DEVICE_ID deviceID, _Out_ STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD * pMethod )`

Gets the timestamping method that is currently in use for the given device.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

#### Parameters

|     |                 |                                                                               |
|-----|-----------------|-------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get the timestamping method for.                                    |
| out | <i>pMethod</i>  | Pointer to value that will be updated with the given link's timestamp method. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

**8.83.2.6** `int STAR_API_CC_CFG_getTimestampValue ( STAR_DEVICE_ID deviceID, _Out_ U32 * pValue )`

Gets the current timestamp value.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

#### Parameters

|     |                 |                                                                              |
|-----|-----------------|------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to get the timestamp value for.                                       |
| out | <i>pValue</i>   | Pointer to value that will be updated with the given link's timestamp value. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

### 8.83.2.7 `int STAR_API_CC CFG_setPulseGeneratorFrequency ( STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_PULSE_FREQ frequency )`

Sets the frequency of each generated pulse.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

#### Parameters

|                  |                                                  |
|------------------|--------------------------------------------------|
| <i>deviceId</i>  | Device to set the pulse generator frequency for. |
| <i>frequency</i> | The frequency value to set.                      |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

[Brick Mk3](#) (version 1.02 or greater), [Brick Mk4](#), [PXI Interface](#) (version 1.04 or greater), [PXI Interface Mk2](#), [SpaceWire PCIe Mk2](#)

#### Examples:

[timestamp\\_test.c](#).

### 8.83.2.8 `int STAR_API_CC CFG_setTimestampMethod ( STAR_DEVICE_ID deviceId, STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD method )`

Sets the timestamping method to use for the given device.

#### Note

This function is currently compatible with [SpaceWire Brick Mk3](#) and [Brick Mk4](#) devices and [SpaceWire PXI Interface](#) and [PXI Interface Mk2](#) Interface and RMAP devices.

The Brick Mk3/Mk4 and PXI Interface will trigger on the rising edge of the external pulse but the Triggering API can be used to change it to trigger on the falling edge using [TRIGGER\\_BRICK\\_MK3\\_enableExtTriggerEdgeDetectMode\(\)](#) and [TRIGGER\\_BRICK\\_MK3\\_enableExtTriggerInvert\(\)](#) or [TRIGGER\\_PXI\\_IF\\_enableExtTriggerEdgeDetectMode\(\)](#) and [TRIGGER\\_PXI\\_IF\\_enableExtTriggerInvert\(\)](#).

#### Parameters

|                 |                                            |
|-----------------|--------------------------------------------|
| <i>deviceId</i> | Device to set the timestamping method for. |
| <i>method</i>   | Timestamp method to enable for device.     |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

**Supported devices:**

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2, SpaceWire PCIe Mk2

**Examples:**

timestamp\_test.c.

**8.83.2.9 int STAR\_API\_CC CFG\_setTimestampValue ( STAR\_DEVICE\_ID *deviceId*, U32 *value* )**

Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.

**Note**

This function is currently compatible with SpaceWire Brick Mk3 and Brick Mk4 devices and SpaceWire PXI Interface and PXI Interface Mk2 Interface and RMAP devices.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceId</i> | Device to set the timestamp value for. |
| <i>value</i>    | Timestamp value to set for device.     |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v4.00 - 19th November 2019

**Supported devices:**

Brick Mk3 (version 1.02 or greater), Brick Mk4, PXI Interface (version 1.04 or greater), PXI Interface Mk2, SpaceWire PCIe Mk2

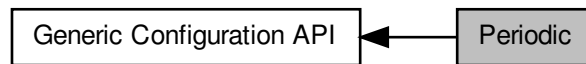
**Examples:**

timestamp\_test.c.

## 8.84 Periodic

Functions in this group deal with periodic functions.

Collaboration diagram for Periodic:



### Functions

- int [STAR\\_API\\_CC\\_CFG\\_getDeviceClockRate](#) ([STAR\\_DEVICE\\_ID](#) deviceID)  
*Gets a device's internal clock rate.*
- int [STAR\\_API\\_CC\\_CFG\\_startPeriodicAction](#) ([STAR\\_DEVICE\\_ID](#) deviceID, unsigned char port, [U32](#) count, [SPW\\_ACTION](#) action)  
*Starts a periodic action on a given device's port.*
- int [STAR\\_API\\_CC\\_CFG\\_stopPeriodicAction](#) ([STAR\\_DEVICE\\_ID](#) deviceID, unsigned char port)  
*Stops a periodic action on a given device's port.*

### 8.84.1 Detailed Description

Functions in this group deal with periodic functions.

### 8.84.2 Function Documentation

#### 8.84.2.1 int [STAR\\_API\\_CC\\_CFG\\_getDeviceClockRate](#) ( [STAR\\_DEVICE\\_ID](#) deviceID )

Gets a device's internal clock rate.

#### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

#### Parameters

|                 |                                    |
|-----------------|------------------------------------|
| <i>deviceID</i> | Identifier of the device to query. |
|-----------------|------------------------------------|

#### Returns

The device's clock rate in Hz, 0 if the device does not support periodic actions, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

[STAR-System v4.00](#) - 19th November 2019

#### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)



#### 8.84.2.2 `int STAR_API_CC CFG_startPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port, U32 count, SPW_ACTION action )`

Starts a periodic action on a given device's port.

To stop a periodic action use `CFG_stopPeriodicAction()`.

##### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

Periodic actions make use of the same on-board registers as error injection, therefore error injection cannot be used when periodic actions are in progress.

##### Parameters

|                 |                                                                                                                                               |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device to have periodic action created for.                                                                                 |
| <i>port</i>     | Port the periodic action is to be performed on.                                                                                               |
| <i>count</i>    | Number of device clock cycles between action repetitions. See <a href="#">CFG_getDeviceClockRate()</a> for obtaining the device's clock rate. |
| <i>action</i>   | Action to be performed. This can be <a href="#">SPW_TRANSMIT_PACKET</a> or any of the <a href="#">SPW_ERR↔OR</a> values.                      |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

[STAR-System v4.00](#) - 19th November 2019

##### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

#### 8.84.2.3 `int STAR_API_CC CFG_stopPeriodicAction ( STAR_DEVICE_ID deviceID, unsigned char port )`

Stops a periodic action on a given device's port.

Periodic actions are started by calling `CFG_startPeriodicAction()`.

##### Note

This function is currently only compatible with [SpaceWire PCI Mk2](#) and [cPCI Mk2](#) devices.

##### Parameters

|                 |                                                          |
|-----------------|----------------------------------------------------------|
| <i>deviceID</i> | Identifier of the device performing the periodic action. |
| <i>port</i>     | Port the periodic action is being performed on.          |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

[STAR-System v4.00](#) - 19th November 2019

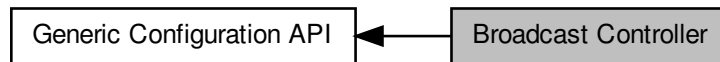
##### Supported devices:

[SpaceWire PCI Mk2](#) and [cPCI Mk2](#)

## 8.85 Broadcast Controller

Functions in this group deal with broadcasts.

Collaboration diagram for Broadcast Controller:



### Functions

- `int STAR_API_CC_CFG_disableBroadcastControllerLegacyMode (STAR_DEVICE_ID deviceID)`  
*Disables broadcast controller legacy mode.*
- `int STAR_API_CC_CFG_enableBroadcastControllerLegacyMode (STAR_DEVICE_ID deviceID)`  
*Enables broadcast controller legacy mode.*
- `int STAR_API_CC_CFG_getBroadcastControllerLegacyModeEnabled (STAR_DEVICE_ID deviceID, _Out_ int *pEnabled)`  
*Gets whether broadcast legacy mode has been enabled for the device.*
- `int STAR_API_CC_CFG_getInterruptCodeDistributionPorts (STAR_DEVICE_ID deviceID, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports interrupt codes are forwarded on.*
- `int STAR_API_CC_CFG_setInterruptCodeDistributionPorts (STAR_DEVICE_ID deviceID, U32 portMask)`  
*This function is used to specify which output ports interrupt codes are forwarded on.*

### 8.85.1 Detailed Description

Functions in this group deal with broadcasts.

These can be used to support distributed interrupt codes.

### 8.85.2 Function Documentation

#### 8.85.2.1 `int STAR_API_CC_CFG_disableBroadcastControllerLegacyMode ( STAR_DEVICE_ID deviceID )`

Disables broadcast controller legacy mode.

When disabled, interrupt codes are supported. A broadcast code with type field 0b00 is interpreted as a time-code. A broadcast code with type field 0b10 is interpreted as a distributed interrupt. Broadcasts with type fields equal to 0b01 or 0b11 are discarded.

#### Parameters

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <i>deviceID</i> | Device to disable broadcast controller legacy mode for. |
|-----------------|---------------------------------------------------------|

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

`router_configuration_example.c`.

### 8.85.2.2 `int STAR_API_CC_CFG_enableBroadcastControllerLegacyMode ( STAR_DEVICE_ID deviceID )`

Enables broadcast controller legacy mode.

When enabled, all broadcast codes are interpreted as time-codes. Codes which do not have flags field set to 0b00 (time-code) will be forwarded according to the time-code flag mode.

**Parameters**

|                 |                                                        |
|-----------------|--------------------------------------------------------|
| <i>deviceID</i> | Device to enable broadcast controller legacy mode for. |
|-----------------|--------------------------------------------------------|

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

### 8.85.2.3 `int STAR_API_CC_CFG_getBroadcastControllerLegacyModeEnabled ( STAR_DEVICE_ID deviceID, _Out_ int * pEnabled )`

Gets whether broadcast legacy mode has been enabled for the device.

When broadcast legacy mode is disabled, interrupt codes are supported. A broadcast code with type field 0b00 is interpreted as a time-code. A broadcast code with type field 0b10 is interpreted as a distributed interrupt. Broadcasts with type fields equal to 0b01 or 0b11 are discarded. When broadcast legacy mode is enabled, all broadcast codes are interpreted as time-codes. Codes which do not have flags field set to 0b00 (time-code) will be forwarded according to the time-code flag mode.

**Parameters**

|     |                 |                                                                                                                      |
|-----|-----------------|----------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Device to check.                                                                                                     |
| out | <i>pEnabled</i> | User supplied value that will be updated to 1 if broadcast controller legacy mode is enabled for the device, else 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[router\\_configuration\\_example.c](#).

**8.85.2.4** `int STAR_API_CC_CFG_getInterruptCodeDistributionPorts ( STAR_DEVICE_ID deviceId, _Out_ U32 * pPortMask )`

This function is used to obtain which output ports interrupt codes are forwarded on.

**Parameters**

|     |                  |                                                                          |
|-----|------------------|--------------------------------------------------------------------------|
|     | <i>deviceId</i>  | Device to get the interrupt code distribution ports for.                 |
| out | <i>pPortMask</i> | Bit mask of ports that interrupt code distribution should be enabled on. |

**Note**

Bit 1 corresponds to port 1.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

**Examples:**

[router\\_configuration\\_example.c](#).

**8.85.2.5** `int STAR_API_CC_CFG_setInterruptCodeDistributionPorts ( STAR_DEVICE_ID deviceId, U32 portMask )`

This function is used to specify which output ports interrupt codes are forwarded on.

**Parameters**

|  |                 |                                                                                                                                                                 |
|--|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <i>deviceId</i> | Device to set the interrupt code distribution ports for.                                                                                                        |
|  | <i>portMask</i> | Bit mask of ports that interrupt code distribution should be enabled on. Valid ports to forward interrupt codes on are 1 through to 13 depending on the device. |

**Note**

Bit 1 corresponds to port 1.

Not all external ports allow interrupt code forwarding. Check your device's user manual for more details.

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v5.01 - 16th September 2022

**Supported devices:**

SpaceWire PCIe Mk2

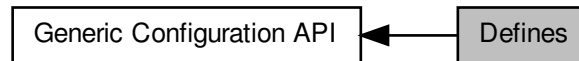
**Examples:**

`router_configuration_example.c`.

## 8.86 Defines

Function return error values are defined here.

Collaboration diagram for Defines:



## Macros

- `#define CFG_STATUS_SUCCESS (0)`  
*Positive retvals are success, negative retvals are errors.*
- `#define CFG_SUCCESS(status) ((status) >= CFG_STATUS_SUCCESS ? 1 : 0)`  
*Macro to allow testing for success or failure.*
- `#define CFG_STATUS_FAILURE (-256)`  
*Return values -1 to -255 are reserved for RMAP status (inverted from +ve to -ve)*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICE_TYPE (-257)`  
*The API call is not compatible with this device type.*
- `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION (-258)`  
*The API call is not compatible with this device's FPGA version.*
- `#define CFG_STATUS_GET_DEVICE_INFO_FAILURE (-259)`  
*Failed to get "Device Information" for this device.*
- `#define CFG_STATUS_TIME_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID (-260)`  
*The Time-code distribution port mask is invalid.*
- `#define CFG_STATUS_TIME_CODE_FLAG_MODE_IS_INVALID (-261)`  
*The Time-code flag mode is invalid.*
- `#define CFG_STATUS_TIME_CODE_PERIOD_IS_INVALID (-262)`  
*The Time-code period is invalid.*
- `#define CFG_STATUS_TIME_CODE_PORT_IS_INVALID (-263)`  
*The Time-code port is invalid.*
- `#define CFG_STATUS_IDENTIFY_SOURCE_PORT_IS_INVALID (-264)`  
*The Identify Source port is invalid.*
- `#define CFG_STATUS_INTERFACE_MODE_PORT_IS_INVALID (-265)`  
*The Interface Mode port is invalid.*
- `#define CFG_STATUS_PORT_IS_INVALID (-266)`  
*The port is invalid.*
- `#define CFG_STATUS_LOGICAL_ADDRESS_IS_INVALID (-267)`  
*The logical address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_IS_INVALID (-268)`  
*The SpaceWire link is invalid.*
- `#define CFG_STATUS_MEASURED_SPEED_PORT_IS_INVALID (-269)`  
*The measured port speed is invalid.*
- `#define CFG_STATUS_SPEED_CHANGE_EVENTS_PORT_IS_INVALID (-270)`  
*The Speed Change Events port is invalid.*

- `#define CFG_STATUS_STATE_CHANGE_EVENTS_PORT_IS_INVALID (-271)`  
*The State Change Events port is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_PORT_IS_INVALID (-272)`  
*The Port Routing port is invalid.*
- `#define CFG_STATUS_INJECT_ERROR_PORT_IS_INVALID (-273)`  
*The Inject Error port is invalid.*
- `#define CFG_STATUS_PRECISION_TRANSMIT_RATE_IS_INVALID (-274)`  
*The Precision Transmit Rate port is invalid.*
- `#define CFG_STATUS_PERIODIC_ACTION_PORT_IS_INVALID (-275)`  
*The Periodic Action port is invalid.*
- `#define CFG_STATUS_TIMESTAMP_EVENTS_PORT_IS_INVALID (-276)`  
*The Timestamp Events port is invalid.*
- `#define CFG_STATUS_CLOCK_LINK_IS_INVALID (-277)`  
*The Clock link is invalid.*
- `#define CFG_STATUS_LINK_RATE_DIVIDER_IS_INVALID (-278)`  
*The Link Rate divider is invalid.*
- `#define CFG_STATUS_TRANSMIT_CLOCK_LINK_IS_INVALID (-279)`  
*The Transmit Clock link is invalid.*
- `#define CFG_STATUS_BIT_RATE_IS_INVALID (-280)`  
*The Bit-rate is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_MODE_IS_INVALID (-281)`  
*The Router Timeout mode is invalid.*
- `#define CFG_STATUS_ROUTER_TIMEOUT_PERIOD_IS_INVALID (-282)`  
*The Router Timeout period is invalid.*
- `#define CFG_STATUS_ROUTER_DISABLE_ON_SILENCE_IS_INVALID (-283)`  
*The Router Disable-on-Silence is invalid.*
- `#define CFG_STATUS_ROUTER_START_ON_REQUEST_IS_INVALID (-284)`  
*The Router Start-on-Request is invalid.*
- `#define CFG_STATUS_ROUTER_ENABLE_SELF_ADDRESSING_IS_INVALID (-285)`  
*The Router Enable-Self-Addressing is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PORT_MASK_IS_INVALID (-286)`  
*The Group Adaptive Routing port mask is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PRIORITY_IS_INVALID (-287)`  
*The Group Adaptive Routing priority is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_DELETE_HEADER_IS_INVALID (-288)`  
*The Group Adaptive Routing delete header is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_ADDRESS_IS_INVALID (-289)`  
*The Group Adaptive Routing address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_TRISTATE_IS_INVALID (-290)`  
*The SpaceWire link tri-state flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_DISABLE_IS_INVALID (-291)`  
*The SpaceWire link disable flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_START_IS_INVALID (-292)`  
*The SpaceWire link start flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_AUTOSTART_IS_INVALID (-293)`  
*The SpaceWire link autostart flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_RUNNING_IS_INVALID (-294)`  
*The SpaceWire link running flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_STATE_IS_INVALID (-295)`  
*The SpaceWire link state is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_ADDRESS_IS_INVALID (-296)`

- The Port Routing address is invalid.*

  - #define `CFG_STATUS_INJECT_ERROR_ERROR_IS_INVALID` (-297)
- The Inject Error error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_PARITY_ERROR_IS_INVALID` (-298)
- The Inject Errors parity error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_ESCAPE_ERROR_IS_INVALID` (-299)
- The Inject Errors escape error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_INSERT_FCT_ERROR_IS_INVALID` (-300)
- The Inject Errors insert FCT error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_SUPPRESS_FCT_ERROR_IS_INVALID` (-301)
- The Inject Errors suppress FCT error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_INCREMENT_CREDIT_ERROR_IS_INVALID` (-302)
- The Inject Errors increment credit error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_DECREMENT_CREDIT_ERROR_IS_INVALID` (-303)
- The Inject Errors decrement credit error is invalid.*

  - #define `CFG_STATUS_INJECT_ERRORS_DISCONNECT_ERROR_IS_INVALID` (-304)
- The Inject Errors disconnect error is invalid.*

  - #define `CFG_STATUS_PERIODIC_ACTION_ACTION_IS_INVALID` (-305)
- The Periodic Action is invalid.*

  - #define `CFG_STATUS_TIMESTAMP_METHOD_IS_INVALID` (-306)
- The Timestamp method is invalid.*

  - #define `CFG_STATUS_PULSE_GENERATOR_FREQUENCY_IS_INVALID` (-307)
- The Pulse Generator frequency is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_MULTIPLIER_IS_INVALID` (-308)
- The Clock Rate multiplier is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_DIVISOR_IS_INVALID` (-309)
- The Clock Rate divisor is invalid.*

  - #define `CFG_STATUS_CLOCK_RATE_PARAMETERS_ARE_INVALID` (-310)
- The Clock Rate parameters are invalid.*

  - #define `CFG_STATUS_CANT_GET_CLOCK_RATE_PARAMETERS_FROM_BIT_RATE` (-311)
- Can't get Clock Rate parameters from bit-rate.*

  - #define `CFG_STATUS_POINTER_PARAMETER_IS_NULL` (-312)
- The pointer parameter is NULL.*

  - #define `CFG_STATUS_TRANSMIT_BIT_RATE_IS_INVALID` (-313)
- The Transmit bit-rate is invalid.*

  - #define `CFG_STATUS_INTERRUPT_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID` (-314)
- The interrupt code distribution port mask is invalid.*

  - #define `CFG_STATUS_GET_FGPA_INFO_FAILURE` (-315)
- Could not get FPGA information from device.*

  - #define `CFG_STATUS_GET_HARDWARE_INFO_FAILURE` (-316)
- Could not get hardware information from device.*

  - #define `CFG_STATUS_ADD_DEVICE_INFO_SHARED_MEMORY_FAILURE` (-317)
- Could not add device information to shared memory.*

### 8.86.1 Detailed Description

Function return error values are defined here.



## 8.86.2 Macro Definition Documentation

### 8.86.2.1 `#define CFG_STATUS_FAILURE (-256)`

Return values -1 to -255 are reserved for RMAP status (inverted from +ve to -ve)

The API call has failed.

### 8.86.2.2 `#define CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION (-258)`

The API call is not compatible with this device's FPGA version.

The FPGA can be updated to allow this additional functionality.

### 8.86.2.3 `#define CFG_STATUS_SUCCESS (0)`

Positive retvals are success, negative retvals are errors.

Values > 0 should be interpreted on a per function basis.

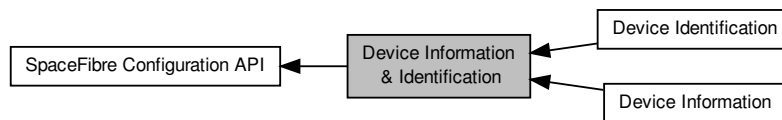
#### Examples:

`data_signalling_rate_example.c`, `device_info_example.c`, `lane_info_example.c`, `link_info_example.c`, `network↵  
_management_example.c`, `port_info_example.c`, `router_configuration_example.c`, `spfi_routing_table_↵  
example.c`, `vendor_specific_example.c`, and `virtual_channel_info_example.c`.

## 8.87 Device Information & Identification

Functions in this group are used to access device information such as the type of node, number of ports, manufacturer identifier and device identifier.

Collaboration diagram for Device Information & Identification:



### Modules

- **Device Information**

*Functions in this group are used to access device information such as the type of node and number of ports.*

- **Device Identification**

*Functions in this group are used to access device identification information such as the manufacturer identifier, device identifier and version number.*

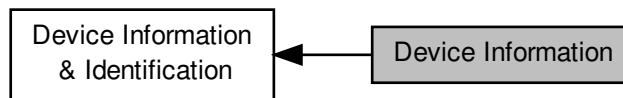
### 8.87.1 Detailed Description

Functions in this group are used to access device information such as the type of node, number of ports, manufacturer identifier and device identifier.

## 8.88 Device Information

Functions in this group are used to access device information such as the type of node and number of ports.

Collaboration diagram for Device Information:



### Functions

- `int STAR_API_CC_CFG_SPFI_getDeviceInformation (STAR_DEVICE_ID deviceId, _Out_ SPFI_DEVICE_INFO *pDeviceInfo)`  
*Reads from the device information register and populates a `SPFI_DEVICE_INFO` value for use with the following sub-accessor functions:*
- `SPFI_NODE_TYPE STAR_API_CC_CFG_SPFI_getDeviceNodeType (SPFI_DEVICE_INFO deviceInfo)`  
*Gets the device node type according to the device information value.*
- `U8 STAR_API_CC_CFG_SPFI_getDevicePortCount (SPFI_DEVICE_INFO deviceInfo)`  
*Gets the number of ports according to the device information value.*
- `int STAR_API_CC_CFG_SPFI_getDevicePortCountByType (STAR_DEVICE_ID deviceId, SPFI_PORT_TYPE portType, _Out_ U8 *pPortCount)`  
*Gets the number of ports of a given type according to the device information value.*

#### 8.88.1 Detailed Description

Functions in this group are used to access device information such as the type of node and number of ports.

#### 8.88.2 Function Documentation

**8.88.2.1** `int STAR_API_CC_CFG_SPFI_getDeviceInformation ( STAR_DEVICE_ID deviceId, _Out_ SPFI_DEVICE_INFO * pDeviceInfo )`

Reads from the device information register and populates a `SPFI_DEVICE_INFO` value for use with the following sub-accessor functions:

- `CFG_SPFI_getDeviceNodeType()`
- `CFG_SPFI_getDevicePortCount()`

#### Parameters

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
|-----------------|--------------------------------------|

---

---

|     |                    |                                                   |
|-----|--------------------|---------------------------------------------------|
| out | <i>pDeviceInfo</i> | SPFI_DEVICE_INFO to populate with the value read. |
|-----|--------------------|---------------------------------------------------|

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

### 8.88.2.2 SPFI\_NODE\_TYPE STAR\_API\_CC\_CFG\_SPFI\_getDeviceNodeType ( SPFI\_DEVICE\_INFO *deviceInfo* )

Gets the device node type according to the device information value.

**Parameters**


---

|                   |                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| <i>deviceInfo</i> | The device information value obtained from a call to <a href="#">CFG_SPFI_getDeviceInformation()</a> . |
|-------------------|--------------------------------------------------------------------------------------------------------|

---

**Returns**

SPFI\_NODE\_TYPE value representing the node type of the device.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

### 8.88.2.3 U8 STAR\_API\_CC\_CFG\_SPFI\_getDevicePortCount ( SPFI\_DEVICE\_INFO *deviceInfo* )

Gets the number of ports according to the device information value.

**Parameters**


---

|                   |                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------|
| <i>deviceInfo</i> | The device information value obtained from a call to <a href="#">CFG_SPFI_getDeviceInformation()</a> . |
|-------------------|--------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 representing the number of ports.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`device_info_example.c.`

**8.88.2.4** `int STAR_API_CC CFG_SPFI_getDevicePortCountByType ( STAR_DEVICE_ID deviceID, SPFI_PORT_TYPE portType, _Out_ U8 * pPortCount )`

Gets the number of ports of a given type according to the device information value.

**Parameters**

|     |                   |                                                                    |
|-----|-------------------|--------------------------------------------------------------------|
|     | <i>deviceID</i>   | Identifier of the SpaceFibre device.                               |
|     | <i>portType</i>   | The type of port to get port count for.                            |
| out | <i>pPortCount</i> | U8 to populate with the number of ports that match the given type. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

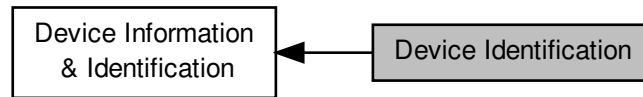
**Examples:**

`device_info_example.c.`

## 8.89 Device Identification

Functions in this group are used to access device identification information such as the manufacturer identifier, device identifier and version number.

Collaboration diagram for Device Identification:



### Functions

- `int STAR_API_CC CFG_SPFI_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceID, _Out_ SPFI_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo)`  
*Reads from the device code register and populates a `SPFI_DEVICE_IDENTIFIER_INFO` value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC CFG_SPFI_getDeviceManufacturer (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)`  
*Gets the device manufacturer code according to the device identification information value.*
- `int STAR_API_CC CFG_SPFI_getDeviceManufacturerAsString (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturer name.*
- `U8 STAR_API_CC CFG_SPFI_getDeviceType (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)`  
*Gets the manufacturer specific device type according to the device identification information value.*
- `int STAR_API_CC CFG_SPFI_getDeviceTypeAsString (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_z_cap_c_(STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC CFG_SPFI_getDeviceVersion (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_U8 *pMajorVersion, _Out_U8 *pMinorVersion, _Out_U8 *pPatchVersion)`  
*Gets the router version number components according to the device identification information value.*

### 8.89.1 Detailed Description

Functions in this group are used to access device identification information such as the manufacturer identifier, device identifier and version number.

### 8.89.2 Function Documentation

- 8.89.2.1 `int STAR_API_CC CFG_SPFI_getDeviceIdentificationInfo ( STAR_DEVICE_ID deviceID, _Out_ SPFI_DEVICE_IDENTIFIER_INFO * pDeviceIdentifierInfo )`

Reads from the device code register and populates a `SPFI_DEVICE_IDENTIFIER_INFO` value for use with the following sub-accessor functions:

- `CFG_SPFI_getDeviceManufacturer()`

- CFG\_SPFI\_getDeviceManufacturerAsString()
- CFG\_SPFI\_getDeviceType()
- CFG\_SPFI\_getDeviceTypeAsString()
- CFG\_SPFI\_getDeviceVersion()

Parameters

|     |                              |                                                              |
|-----|------------------------------|--------------------------------------------------------------|
|     | <i>deviceID</i>              | Identifier of the SpaceFibre device.                         |
| out | <i>pDeviceIdentifierInfo</i> | SPFI_DEVICE_IDENTIFIER_INFO to populate with the value read. |

Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

device\_info\_example.c.

8.89.2.2 U8 STAR\_API\_CC CFG\_SPFI\_getDeviceManufacturer ( SPFI\_DEVICE\_IDENTIFIER\_INFO deviceIdentifierInfo )

Gets the device manufacturer code according to the device identification information value.

Parameters

|  |                             |                                                                                                                      |
|--|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
|  | <i>deviceIdentifierInfo</i> | The device identifier information value obtained from a call to <a href="#">CFG_SPFI_getDeviceIdentifierInfo()</a> . |
|--|-----------------------------|----------------------------------------------------------------------------------------------------------------------|

Returns

U8 containing the device manufacturer code.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

device\_info\_example.c.

8.89.2.3 int STAR\_API\_CC CFG\_SPFI\_getDeviceManufacturerAsString ( SPFI\_DEVICE\_IDENTIFIER\_INFO deviceIdentifierInfo, \_Out\_z\_cap\_c\_(STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN) char \* pManufacturerStr )

If the manufacturer is known, this function obtains a string representation of the manufacturer name.

**Parameters**

|     |                                   |                                                                                                                                                                        |
|-----|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>device↔<br/>IdentifierInfo</i> | The device identifier information value obtained from a call to <a href="#">CFG_SPFI_↔<br/>getDeviceIdentifierInfo()</a> .                                             |
| out | <i>pManufacturer↔<br/>Str</i>     | User allocated zero terminated string of max length <a href="#">STAR_CFG_MANUFA↔<br/>CTURER_STR_MAX_LEN</a> that will be updated with the manufacturer name, if known. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

#### 8.89.2.4 U8 STAR\_API\_CC CFG\_SPFI\_getDeviceType ( SPFI\_DEVICE\_IDENTIFIER\_INFO *deviceIdentifierInfo* )

Gets the manufacturer specific device type according to the device identification information value.

**Parameters**

|  |                                   |                                                                                                                            |
|--|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|
|  | <i>device↔<br/>IdentifierInfo</i> | The device identifier information value obtained from a call to <a href="#">CFG_SPFI_↔<br/>getDeviceIdentifierInfo()</a> . |
|--|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------|

**Returns**

U8 containing the device type.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

#### 8.89.2.5 int STAR\_API\_CC CFG\_SPFI\_getDeviceTypeAsString ( SPFI\_DEVICE\_IDENTIFIER\_INFO *deviceIdentifierInfo*, \_Out\_z\_cap\_c\_(STAR\_CFG\_DEVICE\_STR\_MAX\_LEN) char \* *pDeviceTypeStr* )

If the manufacturer and device type is known, this function obtains a string representation of the device's type.



**Parameters**

|     |                                   |                                                                                                                                                            |
|-----|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>device↔<br/>IdentifierInfo</i> | The device identifier information value obtained from a call to <a href="#">CFG_SPFI_↔<br/>getDeviceIdentificationInfo()</a> .                             |
| out | <i>pDeviceTypeStr</i>             | User allocated zero terminated string of max length <a href="#">STAR_CFG_DEVICE_↔<br/>STR_MAX_LEN</a> that will be updated with the device name, if known. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

**8.89.2.6** `int STAR_API_CC CFG_SPFI_getDeviceVersion ( SPFI_DEVICE_IDENTIFIER_INFO devicelIdentifierInfo, _Out_ U8 * pMajorVersion, _Out_ U8 * pMinorVersion, _Out_ U8 * pPatchVersion )`

Gets the router version number components according to the device identification information value.

**Parameters**

|     |                                   |                                                                                                                                |
|-----|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
|     | <i>device↔<br/>IdentifierInfo</i> | The device identifier information value obtained from a call to <a href="#">CFG_SPFI_↔<br/>getDeviceIdentificationInfo()</a> . |
| out | <i>pMajorVersion</i>              | U8 to populate with the major version number.                                                                                  |
| out | <i>pMinorVersion</i>              | U8 to populate with the minor version number.                                                                                  |
| out | <i>pPatchVersion</i>              | U8 to populate with the patch version number.                                                                                  |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[device\\_info\\_example.c](#).

## 8.90 Routing Table

Functions in this group are used to access and configure routing table entries.

Collaboration diagram for Routing Table:



### Functions

- `int STAR_API_CC_CFG_SPFI_getRoutingTableEntryPorts (STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ U32 *pRoutingTableEntryPorts)`  
*Reads from the routing table entry register for a logical address and populates a U32 value with a bitset indicating the port mask.*
- `int STAR_API_CC_CFG_SPFI_setRoutingTableEntryPorts (STAR_DEVICE_ID deviceID, U8 logicalAddress, U32 routingTableEntryPorts)`  
*Sets the routing table entry for the logical address port mask.*
- `int STAR_API_CC_CFG_SPFI_getRoutingTableEntryFlags (STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ SPFI_ROUTING_TABLE_ENTRY_FLAGS *pRoutingTableEntryFlags)`  
*Reads from the routing table entry flags register for a logical address and populates a SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isRoutingTableEntryEnabled (SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)`  
*Gets the enabled state for the routing table entry.*
- `U8 STAR_API_CC_CFG_SPFI_isRoutingTableEntryDeleteHeaderEnabled (SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)`  
*Gets the delete header enabled state for the routing table entry.*
- `int STAR_API_CC_CFG_SPFI_enableRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress)`  
*Enables the routing table entry for the logical address.*
- `int STAR_API_CC_CFG_SPFI_disableRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress)`  
*Disables the routing table entry for the logical address.*
- `int STAR_API_CC_CFG_SPFI_enableRoutingTableEntryDeleteHeader (STAR_DEVICE_ID deviceID, U8 logicalAddress)`  
*Enables delete header on the routing table entry for the logical address.*
- `int STAR_API_CC_CFG_SPFI_disableRoutingTableEntryDeleteHeader (STAR_DEVICE_ID deviceID, U8 logicalAddress)`  
*Disables delete header on the routing table entry for the logical address.*

### 8.90.1 Detailed Description

Functions in this group are used to access and configure routing table entries.

## 8.90.2 Function Documentation

8.90.2.1 `int STAR_API_CC CFG_SPFI_disableRoutingTableEntry ( STAR_DEVICE_ID deviceId, U8 logicalAddress )`

Disables the routing table entry for the logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.        |
| <i>logicalAddress</i> | Logical address of the routing table entry. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.2** `int STAR_API_CC CFG_SPFI_disableRoutingTableEntryDeleteHeader ( STAR_DEVICE_ID deviceID, U8 logicalAddress )`

Disables delete header on the routing table entry for the logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.        |
| <i>logicalAddress</i> | Logical address of the routing table entry. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.3** `int STAR_API_CC CFG_SPFI_enableRoutingTableEntry ( STAR_DEVICE_ID deviceID, U8 logicalAddress )`

Enables the routing table entry for the logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.        |
| <i>logicalAddress</i> | Logical address of the routing table entry. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.4** `int STAR_API_CC CFG_SPFI_enableRoutingTableEntryDeleteHeader ( STAR_DEVICE_ID deviceID, U8 logicalAddress )`

Enables delete header on the routing table entry for the logical address.

**Parameters**

|                       |                                             |
|-----------------------|---------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.        |
| <i>logicalAddress</i> | Logical address of the routing table entry. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.5** `int STAR_API_CC CFG_SPFI_getRoutingTableEntryFlags ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ SPFI_ROUTING_TABLE_ENTRY_FLAGS * pRoutingTableEntryFlags )`

Reads from the routing table entry flags register for a logical address and populates a [SPFI\\_ROUTING\\_TABLE\\_ENTRY\\_FLAGS](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isRoutingTableEntryEnabled\(\)](#)
- [CFG\\_SPFI\\_isRoutingTableEntryDeleteHeaderEnabled\(\)](#)

**Parameters**

|     |                                      |                                                                 |
|-----|--------------------------------------|-----------------------------------------------------------------|
|     | <i>deviceID</i>                      | Identifier of the SpaceFibre device.                            |
|     | <i>logicalAddress</i>                | Logical address of the routing table entry.                     |
| out | <i>pRoutingTable↔<br/>EntryFlags</i> | SPFI_ROUTING_TABLE_ENTRY_FLAGS to populate with the value read. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.6** `int STAR_API_CC_CFG_SPFI_getRoutingTableEntryPorts ( STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ U32 * pRoutingTableEntryPorts )`

Reads from the routing table entry register for a logical address and populates a **U32** value with a bitset indicating the port mask.

#### Parameters

|     |                                      |                                                                               |
|-----|--------------------------------------|-------------------------------------------------------------------------------|
|     | <i>deviceID</i>                      | Identifier of the SpaceFibre device.                                          |
|     | <i>logicalAddress</i>                | Logical address of the routing table entry.                                   |
| out | <i>pRoutingTable↔<br/>EntryPorts</i> | <b>U32</b> to populate with the value port mask. Bit 0 corresponds to port 0. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[spfi\\_routing\\_table\\_example.c](#).

**8.90.2.7** `U8 STAR_API_CC_CFG_SPFI_isRoutingTableEntryDeleteHeaderEnabled ( SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags )`

Gets the delete header enabled state for the routing table entry.

Parameters

|                                           |                                                                                                            |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <i>routingTable↔</i><br><i>EntryFlags</i> | The routing table entry value obtained from a call to <code>CFG_SPFI_getRoutingTableEntry↔Flags()</code> . |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|

Returns

U8 indicating whether or not the routing table entry has delete header enabled.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

`spfi_routing_table_example.c`.

8.90.2.8 U8 STAR\_API\_CC CFG\_SPFI\_isRoutingTableEntryEnabled ( SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS routingTableEntryFlags )

Gets the enabled state for the routing table entry.

Parameters

|                                           |                                                                                                            |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|
| <i>routingTable↔</i><br><i>EntryFlags</i> | The routing table entry value obtained from a call to <code>CFG_SPFI_getRoutingTableEntry↔Flags()</code> . |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------|

Returns

U8 indicating whether or not the routing table entry is enabled.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

`spfi_routing_table_example.c`.

8.90.2.9 int STAR\_API\_CC CFG\_SPFI\_setRoutingTableEntryPorts ( STAR\_DEVICE\_ID deviceID, U8 logicalAddress, U32 routingTableEntryPorts )

Sets the routing table entry for the logical address port mask.

Parameters

|                                           |                                                                                              |
|-------------------------------------------|----------------------------------------------------------------------------------------------|
| <i>deviceID</i>                           | Identifier of the SpaceFibre device.                                                         |
| <i>logicalAddress</i>                     | Logical address of the routing table entry.                                                  |
| <i>routingTable↔</i><br><i>EntryPorts</i> | Bitmask of output ports the logical address will arbitrate for. Bit 0 corresponds to port 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`spfi_routing_table_example.c`.



## 8.91 Network Management

Functions in this group deal are used to access and configure network management fields such as the device identifier, broadcast timeout, port ready flags, port errors, links errors, current timeslot, time and router broadcast channel.

Collaboration diagram for Network Management:



### Functions

- `int STAR_API_CC_CFG_SPFI_getDeviceIdentifier (STAR_DEVICE_ID deviceId, _Out_ U32 *pDeviceIdentifier)`  
*Reads from the device identifier register and populates a U32.*
- `int STAR_API_CC_CFG_SPFI_setDeviceIdentifier (STAR_DEVICE_ID deviceId, U32 deviceIdIdentifier)`  
*Sets the device identifier value.*
- `int STAR_API_CC_CFG_SPFI_getBroadcastTimeout (STAR_DEVICE_ID deviceId, _Out_ double *pTimeout)`  
*Reads from the broadcast timeout register and populates a double in microseconds.*
- `int STAR_API_CC_CFG_SPFI_setBroadcastTimeout (STAR_DEVICE_ID deviceId, double broadcastTimeout)`  
*Sets the broadcast timeout value specified in microseconds.*
- `int STAR_API_CC_CFG_SPFI_getBroadcastTimeoutMaximum (STAR_DEVICE_ID deviceId, _Out_ double *pMaximumTimeout)`  
*Gets the maximum supported broadcast timeout value specified in microseconds.*
- `int STAR_API_CC_CFG_SPFI_getBroadcastTimeoutResolution (STAR_DEVICE_ID deviceId, _Out_ double *pResolution)`  
*Gets the broadcast timeout resolution in microseconds.*
- `int STAR_API_CC_CFG_SPFI_getPortsReady (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsReady)`  
*Reads from the ports ready register and populates a U32 with a bitset indicating if each port is ready.*
- `int STAR_API_CC_CFG_SPFI_getPortsWithErrors (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsWithErrors)`  
*Reads from the ports error register and populates a U32 with a bitset indicating if each port has an error.*
- `int STAR_API_CC_CFG_SPFI_getLinksWithErrors (STAR_DEVICE_ID deviceId, _Out_ U32 *pLinksWithErrors)`  
*Reads from the links error register and populates a U32 with a bitset indicating if each link has an error.*
- `int STAR_API_CC_CFG_SPFI_getCurrentTimeSlot (STAR_DEVICE_ID deviceId, _Out_ U8 *pCurrentTimeSlot)`  
*Reads from the time-slot register and populates a U8.*
- `int STAR_API_CC_CFG_SPFI_getCurrentTime (STAR_DEVICE_ID deviceId, _Out_ U64 *pCurrentTime, _Out_ U8 *pSynchronised)`  
*Gets the current time and populates a U64.*
- `int STAR_API_CC_CFG_SPFI_getSpaceWireBC (STAR_DEVICE_ID deviceId, _Out_ U8 *pRouterBC)`  
*Gets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.*
- `int STAR_API_CC_CFG_SPFI_setSpaceWireBC (STAR_DEVICE_ID deviceId, U8 routerBC)`  
*Sets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.*

### 8.91.1 Detailed Description

Functions in this group deal are used to access and configure network management fields such as the device identifier, broadcast timeout, port ready flags, port errors, links errors, current timeslot, time and router broadcast channel.

### 8.91.2 Function Documentation

#### 8.91.2.1 `int STAR_API_CC_CFG_SPFI_getBroadcastTimeout ( STAR_DEVICE_ID deviceId, _Out_ double * pTimeout )`

Reads from the broadcast timeout register and populates a double in microseconds.

##### Parameters

|     |                 |                                                                         |
|-----|-----------------|-------------------------------------------------------------------------|
|     | <i>deviceId</i> | Identifier of the SpaceFibre device.                                    |
| out | <i>pTimeout</i> | double to populate with the broadcast timeout interval in microseconds. |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[network\\_management\\_example.c](#).

#### 8.91.2.2 `int STAR_API_CC_CFG_SPFI_getBroadcastTimeoutMaximum ( STAR_DEVICE_ID deviceId, _Out_ double * pMaximumTimeout )`

Gets the maximum supported broadcast timeout value specified in microseconds.

##### Parameters

|     |                        |                                                                                        |
|-----|------------------------|----------------------------------------------------------------------------------------|
|     | <i>deviceId</i>        | Identifier of the SpaceFibre device.                                                   |
| out | <i>pMaximumTimeout</i> | double to populate with the maximum supported broadcast timeout value in microseconds. |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[network\\_management\\_example.c](#).

8.91.2.3 `int STAR_API_CC_CFG_SPFI_getBroadcastTimeoutResolution ( STAR_DEVICE_ID deviceID, _Out_ double * pResolution )`

Gets the broadcast timeout resolution in microseconds.

**Parameters**

|     |                    |                                                                                                                                                                                                                             |
|-----|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>    | Identifier of the SpaceFibre device.                                                                                                                                                                                        |
| out | <i>pResolution</i> | double to populate with the broadcast timeout resolution for use in determining the absolute supported values of the <code>CFG_SPFI_getBroadcastTimeout()</code> and <code>CFG_SPFI_setBroadcastTimeout()</code> functions. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 - 7th October 2024

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.4** `int STAR_API_CC_CFG_SPFI_getCurrentTime ( STAR_DEVICE_ID deviceID, _Out_ U64 * pCurrentTime, _Out_ U8 * pSynchronised )`

Gets the current time and populates a `U64`.

**Parameters**

|     |                      |                                                               |
|-----|----------------------|---------------------------------------------------------------|
|     | <i>deviceID</i>      | Identifier of the SpaceFibre device.                          |
| out | <i>pCurrentTime</i>  | <code>U64</code> to populate with the current time-slot time. |
| out | <i>pSynchronised</i> | <code>U8</code> to indicate if the timer is synchronised.     |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.5** `int STAR_API_CC_CFG_SPFI_getCurrentTimeSlot ( STAR_DEVICE_ID deviceID, _Out_ U8 * pCurrentTimeSlot )`

Reads from the time-slot register and populates a `U8`.

**Parameters**

|     |                         |                                            |
|-----|-------------------------|--------------------------------------------|
|     | <i>deviceID</i>         | Identifier of the SpaceFibre device.       |
| out | <i>pCurrentTimeSlot</i> | U8 to populate with the current time-slot. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.6** `int STAR_API_CC_CFG_SPFI_getDeviceIdentifier ( STAR_DEVICE_ID deviceID, _Out_ U32 * pDeviceIdentifier )`

Reads from the device identifier register and populates a U32.

**Parameters**

|     |                          |                                             |
|-----|--------------------------|---------------------------------------------|
|     | <i>deviceID</i>          | Identifier of the SpaceFibre device.        |
| out | <i>pDeviceIdentifier</i> | U32 to populate with the device identifier. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.7** `int STAR_API_CC_CFG_SPFI_getLinksWithErrors ( STAR_DEVICE_ID deviceID, _Out_ U32 * pLinksWithErrors )`

Reads from the links error register and populates a U32 with a bitset indicating if each link has an error.

**Parameters**

|  |                 |                                      |
|--|-----------------|--------------------------------------|
|  | <i>deviceID</i> | Identifier of the SpaceFibre device. |
|--|-----------------|--------------------------------------|

|     |                         |                                                                                   |
|-----|-------------------------|-----------------------------------------------------------------------------------|
| out | <i>pLinksWithErrors</i> | <a href="#">U32</a> to populate with the links mask. Bit 0 corresponds to port 0. |
|-----|-------------------------|-----------------------------------------------------------------------------------|

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[network\\_management\\_example.c](#).

8.91.2.8 `int STAR_API_CC_CFG_SPFI_getPortsReady ( STAR_DEVICE_ID deviceId, _Out_ U32 * pPortsReady )`

Reads from the ports ready register and populates a [U32](#) with a bitset indicating if each port is ready.

#### Parameters

|     |                    |                                                                                   |
|-----|--------------------|-----------------------------------------------------------------------------------|
|     | <i>deviceId</i>    | Identifier of the SpaceFibre device.                                              |
| out | <i>pPortsReady</i> | <a href="#">U32</a> to populate with the ports mask. Bit 0 corresponds to port 0. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[network\\_management\\_example.c](#).

8.91.2.9 `int STAR_API_CC_CFG_SPFI_getPortsWithErrors ( STAR_DEVICE_ID deviceId, _Out_ U32 * pPortsWithErrors )`

Reads from the ports error register and populates a [U32](#) with a bitset indicating if each port has an error.

#### Parameters

|     |                         |                                                                                   |
|-----|-------------------------|-----------------------------------------------------------------------------------|
|     | <i>deviceId</i>         | Identifier of the SpaceFibre device.                                              |
| out | <i>pPortsWithErrors</i> | <a href="#">U32</a> to populate with the ports mask. Bit 0 corresponds to port 0. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

#### 8.91.2.10 `int STAR_API_CC_CFG_SPFI_getSpaceWireBC ( STAR_DEVICE_ID deviceId, _Out_ U8 * pRouterBC )`

Gets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.

**Parameters**

|     |                  |                                            |
|-----|------------------|--------------------------------------------|
|     | <i>deviceId</i>  | Identifier of the SpaceFibre device.       |
| out | <i>pRouterBC</i> | U8 to populate with the broadcast channel. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

#### 8.91.2.11 `int STAR_API_CC_CFG_SPFI_setBroadcastTimeout ( STAR_DEVICE_ID deviceId, double broadcastTimeout )`

Sets the broadcast timeout value specified in microseconds.

**Parameters**

|  |                         |                                                     |
|--|-------------------------|-----------------------------------------------------|
|  | <i>deviceId</i>         | Identifier of the SpaceFibre device.                |
|  | <i>broadcastTimeout</i> | The broadcast timeout value to set in microseconds. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.12 int STAR\_API\_CC\_CFG\_SPFI\_setDeviceIdentifier ( STAR\_DEVICE\_ID *deviceId*, U32 *deviceIdentifier* )**

Sets the device identifier value.

**Parameters**

|                         |                                      |
|-------------------------|--------------------------------------|
| <i>deviceId</i>         | Identifier of the SpaceFibre device. |
| <i>deviceIdentifier</i> | The device identifier value to set.  |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.

**8.91.2.13 int STAR\_API\_CC\_CFG\_SPFI\_setSpaceWireBC ( STAR\_DEVICE\_ID *deviceId*, U8 *routerBC* )**

Sets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>routerBC</i> | The broadcast channel to set.        |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`network_management_example.c`.



## 8.92 Vendor Specific

Functions in this group are used to access and configure vendor specific functionality including router timeout and broadcast types.

Collaboration diagram for Vendor Specific:



### Functions

- `int STAR_API_CC_CFG_SPFI_getRouterTimeout (STAR_DEVICE_ID deviceId, _Out_ double *pTimeout)`  
*Reads from the router timeout register and populates a double in microseconds.*
- `int STAR_API_CC_CFG_SPFI_isRouterTimeoutEnabled (STAR_DEVICE_ID deviceId, _Out_ U8 *pEnabled)`  
*Gets the router timeout enabled flag.*
- `int STAR_API_CC_CFG_SPFI_setRouterTimeout (STAR_DEVICE_ID deviceId, double routerTimeout)`  
*Sets the router timeout value specified in microseconds.*
- `int STAR_API_CC_CFG_SPFI_enableRouterTimeout (STAR_DEVICE_ID deviceId)`  
*Enables the router timeout function.*
- `int STAR_API_CC_CFG_SPFI_disableRouterTimeout (STAR_DEVICE_ID deviceId)`  
*Disables the router timeout function.*
- `int STAR_API_CC_CFG_SPFI_getRouterTimeoutMaximum (STAR_DEVICE_ID deviceId, _Out_ double *pMaximumTimeout)`  
*Gets the maximum supported router timeout value specified in microseconds.*
- `int STAR_API_CC_CFG_SPFI_getRouterTimeoutResolution (STAR_DEVICE_ID deviceId, _Out_ double *pResolution)`  
*Gets the router timeout resolution in microseconds.*
- `int STAR_API_CC_CFG_SPFI_getBroadcastTypesInformation (STAR_DEVICE_ID deviceId, _Out_ SPFI_BROADCAST_TYPES_INFO *pBroadcastTypesInfo)`  
*Reads from the broadcast types register and populates a SPFI\_BROADCAST\_TYPES\_INFO value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_getTimeSlotBroadcastType (SPFI_BROADCAST_TYPES_INFO broadcastTypesInfo)`  
*Gets the time-slot broadcast type value.*
- `U8 STAR_API_CC_CFG_SPFI_getTimeCodeBroadcastType (SPFI_BROADCAST_TYPES_INFO broadcastTypesInfo)`  
*Gets the time-code broadcast type value.*
- `U8 STAR_API_CC_CFG_SPFI_getSpaceWireInterruptBroadcastType (SPFI_BROADCAST_TYPES_INFO broadcastTypesInfo)`  
*Gets the SpaceWire interrupt broadcast type value.*
- `int STAR_API_CC_CFG_SPFI_setTimeSlotBroadcastType (STAR_DEVICE_ID deviceId, U8 timeSlotValue)`  
*Sets the time-slot broadcast type value.*
- `int STAR_API_CC_CFG_SPFI_setTimeCodeBroadcastType (STAR_DEVICE_ID deviceId, U8 timeCodeValue)`  
*Sets the time-code broadcast type value.*

- `int STAR_API_CC CFG_SPFI_setSpaceWireInterruptBroadcastType (STAR_DEVICE_ID deviceID, U8 interruptValue)`

*Sets the SpaceWire interrupt broadcast type value.*

### 8.92.1 Detailed Description

Functions in this group are used to access and configure vendor specific functionality including router timeout and broadcast types.

### 8.92.2 Function Documentation

#### 8.92.2.1 `int STAR_API_CC CFG_SPFI_disableRouterTimeout ( STAR_DEVICE_ID deviceID )`

Disables the router timeout function.

##### Parameters

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
|-----------------|--------------------------------------|

---

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

`vendor_specific_example.c`.

#### 8.92.2.2 `int STAR_API_CC CFG_SPFI_enableRouterTimeout ( STAR_DEVICE_ID deviceID )`

Enables the router timeout function.

##### Parameters

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
|-----------------|--------------------------------------|

---

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

`vendor_specific_example.c`.

### 8.92.2.3 `int STAR_API_CC_CFG_SPFI_getBroadcastTypesInformation ( STAR_DEVICE_ID deviceId, _Out_ SPFI_BROADCAST_TYPES_INFO * pBroadcastTypesInfo )`

Reads from the broadcast types register and populates a `SPFI_BROADCAST_TYPES_INFO` value for use with the following sub-accessor functions:

- `CFG_SPFI_getTimeSlotBroadcastType()`
- `CFG_SPFI_getTimeCodeBroadcastType()`
- `CFG_SPFI_getSpaceWireInterruptBroadcastType()`

#### Parameters

|     |                            |                                                                         |
|-----|----------------------------|-------------------------------------------------------------------------|
|     | <i>deviceId</i>            | Identifier of the SpaceFibre device.                                    |
| out | <i>pBroadcastTypesInfo</i> | <code>SPFI_BROADCAST_TYPES_INFO</code> to populate with the value read. |

#### Returns

0 for success, or a negative error code upon failure as defined in `Defines`.

#### Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

`vendor_specific_example.c`.

### 8.92.2.4 `int STAR_API_CC_CFG_SPFI_getRouterTimeout ( STAR_DEVICE_ID deviceId, _Out_ double * pTimeout )`

Reads from the router timeout register and populates a double in microseconds.

#### Parameters

|     |                 |                                                                      |
|-----|-----------------|----------------------------------------------------------------------|
|     | <i>deviceId</i> | Identifier of the SpaceFibre device.                                 |
| out | <i>pTimeout</i> | double to populate with the router timeout interval in microseconds. |

#### Returns

0 for success, or a negative error code upon failure as defined in `Defines`.

#### Added in version:

STAR-System v6.00 - 7th October 2024

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

`vendor_specific_example.c`.

### 8.92.2.5 `int STAR_API_CC_CFG_SPFI_getRouterTimeoutMaximum ( STAR_DEVICE_ID deviceId, _Out_ double * pMaximumTimeout )`

Gets the maximum supported router timeout value specified in microseconds.

Parameters

|     |                         |                                                                                     |
|-----|-------------------------|-------------------------------------------------------------------------------------|
|     | <i>deviceID</i>         | Identifier of the SpaceFibre device.                                                |
| out | <i>pMaximum↵Timeout</i> | double to populate with the maximum supported router timeout value in microseconds. |

Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 - 7th October 2024

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

vendor\_specific\_example.c.

8.92.2.6 int STAR\_API\_CC CFG\_SPFI\_getRouterTimeoutResolution ( STAR\_DEVICE\_ID deviceID, \_Out\_ double \* pResolution )

Gets the router timeout resolution in microseconds.

Parameters

|     |                    |                                                                                                                                                                                                                           |
|-----|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>    | Identifier of the SpaceFibre device.                                                                                                                                                                                      |
| out | <i>pResolution</i> | double to populate with the router timeout resolution for use in determining the absolute supported values of the <a href="#">CFG_SPFI_getRouterTimeout()</a> and <a href="#">CFG↵_SPFI_setRouterTimeout()</a> functions. |

Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 - 7th October 2024

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

vendor\_specific\_example.c.

8.92.2.7 U8 STAR\_API\_CC CFG\_SPFI\_getSpaceWireInterruptBroadcastType ( SPFI\_BROADCAST\_TYPES\_INFO broadcastTypeInfo )

Gets the SpaceWire interrupt broadcast type value.

Note

This function is only relevant for devices with SpaceWire ports.

**Parameters**


---

|                                 |                                                                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <i>broadcast↔<br/>TypesInfo</i> | The device information value obtained from a call to <a href="#">CFG_SPFI_getBroadcastTypes↔<br/>Information()</a> . |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the SpaceWire interrupt broadcast type value.

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[vendor\\_specific\\_example.c](#).

#### 8.92.2.8 U8 STAR\_API\_CC CFG\_SPFI\_getTimeCodeBroadcastType ( SPFI\_BROADCAST\_TYPES\_INFO *broadcastTypesInfo* )

Gets the time-code broadcast type value.

**Note**

This function is only relevant for devices with SpaceWire ports.

**Parameters**


---

|                                 |                                                                                                                      |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <i>broadcast↔<br/>TypesInfo</i> | The device information value obtained from a call to <a href="#">CFG_SPFI_getBroadcastTypes↔<br/>Information()</a> . |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the time-code broadcast type value.

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[vendor\\_specific\\_example.c](#).

#### 8.92.2.9 U8 STAR\_API\_CC CFG\_SPFI\_getTimeSlotBroadcastType ( SPFI\_BROADCAST\_TYPES\_INFO *broadcastTypesInfo* )

Gets the time-slot broadcast type value.

**Parameters**

|                           |                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------|
| <i>broadcastTypesInfo</i> | The device information value obtained from a call to <code>CFG_SPFI_getBroadcastTypesInformation()</code> . |
|---------------------------|-------------------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating the time-slot broadcast type value.

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`vendor_specific_example.c`.

**8.92.2.10** `int STAR_API_CC CFG_SPFI_isRouterTimeoutEnabled ( STAR_DEVICE_ID deviceID, _Out_ U8 * pEnabled )`

Gets the router timeout enabled flag.

**Parameters**

|            |                 |                                                                                 |
|------------|-----------------|---------------------------------------------------------------------------------|
|            | <i>deviceID</i> | Identifier of the SpaceFibre device.                                            |
| <i>out</i> | <i>pEnabled</i> | U8 to populate with a flag indicating whether or not router timeout is enabled. |

**Returns**

0 for success, or a negative error code upon failure as defined in `Defines`.

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`vendor_specific_example.c`.

**8.92.2.11** `int STAR_API_CC CFG_SPFI_setRouterTimeout ( STAR_DEVICE_ID deviceID, double routerTimeout )`

Sets the router timeout value specified in microseconds.

**Parameters**

|  |                      |                                                  |
|--|----------------------|--------------------------------------------------|
|  | <i>deviceID</i>      | Identifier of the SpaceFibre device.             |
|  | <i>routerTimeout</i> | The router timeout value to set in microseconds. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 - 7th October 2024

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[vendor\\_specific\\_example.c](#).

**8.92.2.12** `int STAR_API_CC_CFG_SPFI_setSpaceWireInterruptBroadcastType ( STAR_DEVICE_ID deviceId, U8 interruptValue )`

Sets the SpaceWire interrupt broadcast type value.

**Note**

This function is only relevant for devices with SpaceWire ports.

**Parameters**

|                       |                                                      |
|-----------------------|------------------------------------------------------|
| <i>deviceId</i>       | Identifier of the SpaceFibre device.                 |
| <i>interruptValue</i> | The SpaceWire interrupt broadcast type value to set. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[vendor\\_specific\\_example.c](#).

**8.92.2.13** `int STAR_API_CC_CFG_SPFI_setTimeCodeBroadcastType ( STAR_DEVICE_ID deviceId, U8 timeCodeValue )`

Sets the time-code broadcast type value.

**Note**

This function is only relevant for devices with SpaceWire ports.

**Parameters**

|                      |                                            |
|----------------------|--------------------------------------------|
| <i>deviceId</i>      | Identifier of the SpaceFibre device.       |
| <i>timeCodeValue</i> | The time-code broadcast type value to set. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[vendor\\_specific\\_example.c](#).

8.92.2.14 `int STAR_API_CC_CFG_SPFI_setTimeSlotBroadcastType ( STAR_DEVICE_ID deviceId, U8 timeSlotValue )`

Sets the time-slot broadcast type value.

**Parameters**

|                      |                                            |
|----------------------|--------------------------------------------|
| <i>deviceId</i>      | Identifier of the SpaceFibre device.       |
| <i>timeSlotValue</i> | The time-slot broadcast type value to set. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 3 - 5th October 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

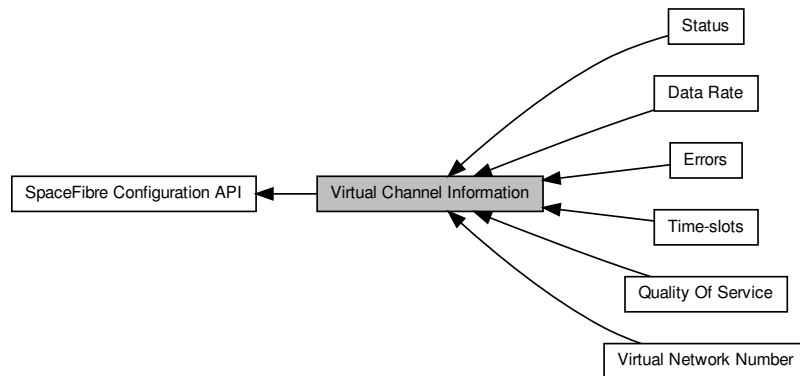
[vendor\\_specific\\_example.c](#).



## 8.93 Virtual Channel Information

Functions in this group are used to access and configure virtual channel information such as the virtual network number, status fields, errors and quality of service parameters.

Collaboration diagram for Virtual Channel Information:



### Modules

- **Virtual Network Number**

*Functions in this group are used to access and configure the virtual network number.*

- **Status**

*Functions in this group are used to access virtual channel status fields.*

- **Errors**

*Functions in this group are used to access and clear virtual channel error fields.*

- **Quality Of Service**

*Functions in this group are used to access and configure virtual channel quality of service fields.*

- **Time-slots**

*Functions in this group are used to access and configure virtual channel time-slots.*

- **Data Rate**

*Functions in this group are used to access virtual channel data rate.*

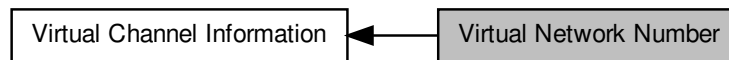
### 8.93.1 Detailed Description

Functions in this group are used to access and configure virtual channel information such as the virtual network number, status fields, errors and quality of service parameters.

## 8.94 Virtual Network Number

Functions in this group are used to access and configure the virtual network number.

Collaboration diagram for Virtual Network Number:



### Functions

- `int STAR_API_CC_CFG_SPFI_getVCVN (STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ U8 *pVirtualNetwork)`  
*Reads from a virtual channel control register and populates a U8.*
- `int STAR_API_CC_CFG_SPFI_setVCVN (STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 virtualNetwork)`  
*Sets the virtual network number for the virtual channel.*

### 8.94.1 Detailed Description

Functions in this group are used to access and configure the virtual network number.

### 8.94.2 Function Documentation

**8.94.2.1** `int STAR_API_CC_CFG_SPFI_getVCVN ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ U8 * pVirtualNetwork )`

Reads from a virtual channel control register and populates a U8.

#### Parameters

|            |                        |                                                 |
|------------|------------------------|-------------------------------------------------|
|            | <i>deviceID</i>        | Identifier of the SpaceFibre device.            |
|            | <i>port</i>            | Port of the virtual channel.                    |
|            | <i>virtualChannel</i>  | The virtual channel number.                     |
| <i>out</i> | <i>pVirtualNetwork</i> | U8 to populate with the virtual network number. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

`virtual_channel_info_example.c`.

8.94.2.2 int **STAR\_API\_CC\_CFG\_SPFI\_setVCVN** ( **STAR\_DEVICE\_ID** *deviceID*, U8 *port*, U8 *virtualChannel*, U8 *virtualNetwork* )

Sets the virtual network number for the virtual channel.

#### Note

Changing the VN of an external port VC will prevent local configuration operations using that external port VC if different from P0 VC 0.

#### Parameters

|                       |                                      |
|-----------------------|--------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device. |
| <i>port</i>           | Port of the virtual channel.         |
| <i>virtualChannel</i> | The virtual channel number.          |
| <i>virtualNetwork</i> | The virtual network number.          |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

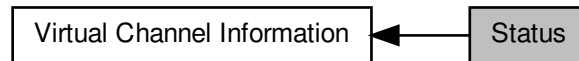
#### Examples:

[virtual\\_channel\\_info\\_example.c](#).

## 8.95 Status

Functions in this group are used to access virtual channel status fields.

Collaboration diagram for Status:



### Functions

- `int STAR_API_CC_CFG_SPFI_getVCStatus (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ SPFI_VC_STATUS *pStatus)`  
*Reads from a virtual channel status register and populates a `SPFI_VC_STATUS` for use with the following sub-accessor functions:*
  - `U8 STAR_API_CC_CFG_SPFI_isVCOverused (SPFI_VC_STATUS status)`  
*Gets the overused state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCUnderused (SPFI_VC_STATUS status)`  
*Gets the underused state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCOutputBufferHalfFull (SPFI_VC_STATUS status)`  
*Gets the output buffer half full state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCOutputBufferFull (SPFI_VC_STATUS status)`  
*Gets the output buffer full state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCInputBufferNotEmpty (SPFI_VC_STATUS status)`  
*Gets the input buffer not empty state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCInputBufferHalfFull (SPFI_VC_STATUS status)`  
*Gets the input buffer half full state of a virtual channel according to the virtual channel status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCInputBufferFull (SPFI_VC_STATUS status)`  
*Gets the input buffer full state of a virtual channel according to the virtual channel status value.*

### 8.95.1 Detailed Description

Functions in this group are used to access virtual channel status fields.

### 8.95.2 Function Documentation

**8.95.2.1** `int STAR_API_CC_CFG_SPFI_getVCStatus ( STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ SPFI_VC_STATUS * pStatus )`

Reads from a virtual channel status register and populates a `SPFI_VC_STATUS` for use with the following sub-accessor functions:

- `CFG_SPFI_isVCOverused()`
- `CFG_SPFI_isVCUnderused()`

- `CFG_SPFI_isVCOutputBufferHalfFull()`
- `CFG_SPFI_isVCOutputBufferFull()`
- `CFG_SPFI_isVCInputBufferNotEmpty()`
- `CFG_SPFI_isVCInputBufferHalfFull()`
- `CFG_SPFI_isVCInputBufferFull()`

**Parameters**

|            |                       |                                                              |
|------------|-----------------------|--------------------------------------------------------------|
|            | <i>deviceId</i>       | Identifier of the SpaceFibre device.                         |
|            | <i>port</i>           | Port of the virtual channel.                                 |
|            | <i>virtualChannel</i> | The virtual channel number.                                  |
| <i>out</i> | <i>pStatus</i>        | <code>SPFI_VC_STATUS</code> to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`virtual_channel_info_example.c`.

### 8.95.2.2 U8 STAR\_API\_CC CFG\_SPFI\_isVCInputBufferFull ( `SPFI_VC_STATUS status` )

Gets the input buffer full state of a virtual channel according to the virtual channel status value.

**Parameters**

|  |               |                                                                                                |
|--|---------------|------------------------------------------------------------------------------------------------|
|  | <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|--|---------------|------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not the input virtual channel buffer is full.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`virtual_channel_info_example.c`.

### 8.95.2.3 U8 STAR\_API\_CC CFG\_SPFI\_isVCInputBufferHalfFull ( `SPFI_VC_STATUS status` )

Gets the input buffer half full state of a virtual channel according to the virtual channel status value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the input virtual channel buffer is half full.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`virtual_channel_info_example.c`.

**8.95.2.4 U8 STAR\_API\_CC CFG\_SPFI\_isVCInputBufferNotEmpty ( SPFI\_VC\_STATUS *status* )**

Gets the input buffer not empty state of a virtual channel according to the virtual channel status value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the input virtual channel buffer is not empty.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`virtual_channel_info_example.c`.

**8.95.2.5 U8 STAR\_API\_CC CFG\_SPFI\_isVCOutputBufferFull ( SPFI\_VC\_STATUS *status* )**

Gets the output buffer full state of a virtual channel according to the virtual channel status value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the output virtual channel buffer is full.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.95.2.6 U8 STAR\_API\_CC\_CFG\_SPFI\_isVCOutputBufferHalfFull ( SPFI\_VC\_STATUS *status* )**

Gets the output buffer half full state of a virtual channel according to the virtual channel status value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the output virtual channel buffer is half full.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.95.2.7 U8 STAR\_API\_CC\_CFG\_SPFI\_isVCOverused ( SPFI\_VC\_STATUS *status* )**

Gets the overused state of a virtual channel according to the virtual channel status value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the virtual channel is using more bandwidth than it has been allocated.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.95.2.8 U8 STAR\_API\_CC\_CFG\_SPFI\_isVCUnderused ( SPFI\_VC\_STATUS *status* )**

Gets the underused state of a virtual channel according to the virtual channel status value.

### Parameters

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>status</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCStatus()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

### Returns

U8 indicating whether or not the virtual channel is using less bandwidth than it has been allocated.

### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

### Supported devices:

STAR-Ultra PCIe Single-Lane Router

### Examples:

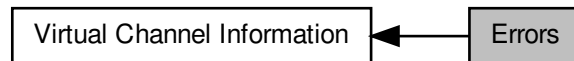
`virtual_channel_info_example.c`.



## 8.96 Errors

Functions in this group are used to access and clear virtual channel error fields.

Collaboration diagram for Errors:



### Functions

- `int STAR_API_CC_CFG_SPFI_getVCErrors (STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _↔ Out_ SPFI_VC_ERRORS *pErrors)`  
*Reads from a virtual channel errors register and populates a `SPFI_VC_ERRORS` for use with the following sub-accessor functions:*
  - `U8 STAR_API_CC_CFG_SPFI_isVCVNErrordetected (SPFI_VC_ERRORS errors)`  
*Gets the VN error flag according to the virtual channel errors value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCAddressErrorDetected (SPFI_VC_ERRORS errors)`  
*Gets the address error flag according to the virtual channel errors value.*
  - `U8 STAR_API_CC_CFG_SPFI_isVCTimeoutDetected (SPFI_VC_ERRORS errors)`  
*Gets the timeout error flag according to the virtual channel errors value.*
- `int STAR_API_CC_CFG_SPFI_clearVCErrors (STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel)`  
*Clears any virtual channel errors on the port.*

#### 8.96.1 Detailed Description

Functions in this group are used to access and clear virtual channel error fields.

#### 8.96.2 Function Documentation

##### 8.96.2.1 `int STAR_API_CC_CFG_SPFI_clearVCErrors ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel )`

Clears any virtual channel errors on the port.

##### Parameters

|                       |                                      |
|-----------------------|--------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device. |
| <i>port</i>           | Port of the virtual channel.         |
| <i>virtualChannel</i> | The virtual channel number.          |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.96.2.2** `int STAR_API_CC CFG_SPFI_getVCErrors ( STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ SPFI_VC_ERRORS * pErrors )`

Reads from a virtual channel errors register and populates a `SPFI_VC_ERRORS` for use with the following sub-accessor functions:

- `CFG_SPFI_isVCVNErrordetected()`
- `CFG_SPFI_isVCAddressErrorDetected()`
- `CFG_SPFI_isVCTimeoutDetected()`

**Parameters**

|            |                       |                                                              |
|------------|-----------------------|--------------------------------------------------------------|
|            | <i>deviceId</i>       | Identifier of the SpaceFibre device.                         |
|            | <i>port</i>           | Port of the virtual channel.                                 |
|            | <i>virtualChannel</i> | The virtual channel number.                                  |
| <i>out</i> | <i>pErrors</i>        | <code>SPFI_VC_ERRORS</code> to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.96.2.3** `U8 STAR_API_CC CFG_SPFI_isVCAddressErrorDetected ( SPFI_VC_ERRORS errors )`

Gets the address error flag according to the virtual channel errors value.

**Parameters**

|  |               |                                                                                                |
|--|---------------|------------------------------------------------------------------------------------------------|
|  | <i>errors</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCErrors()</code> . |
|--|---------------|------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not the output port of a packet is invalid.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.96.2.4 U8 STAR\_API\_CC\_CFG\_SPFI\_isVCTimeoutDetected ( SPFI\_VC\_ERRORS errors )**

Gets the timeout error flag according to the virtual channel errors value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>errors</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCErrors()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not a VC timeout occurred on a packet being received.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

virtual\_channel\_info\_example.c.

**8.96.2.5 U8 STAR\_API\_CC\_CFG\_SPFI\_isVCVNErrorsDetected ( SPFI\_VC\_ERRORS errors )**

Gets the VN error flag according to the virtual channel errors value.

**Parameters**

---

|               |                                                                                                |
|---------------|------------------------------------------------------------------------------------------------|
| <i>errors</i> | The virtual channel status value obtained from a call to <code>CFG_SPFI_getVCErrors()</code> . |
|---------------|------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the output port of a packet arriving at this virtual channel does not have an enabled virtual channel with the same virtual network number.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

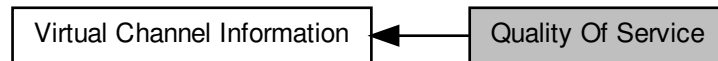
**Examples:**

virtual\_channel\_info\_example.c.

## 8.97 Quality Of Service

Functions in this group are used to access and configure virtual channel quality of service fields.

Collaboration diagram for Quality Of Service:



### Functions

- `int STAR_API_CC_CFG_SPFI_getVCQualityOfService (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ SPFI_QUALITY_OF_SERVICE *pQualityOfService)`  
*Reads from a virtual channel quality of service register and populates a SPFI\_QUALITY\_OF\_SERVICE for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_getVCBandwidthReservation (SPFI_QUALITY_OF_SERVICE qualityOfService)`  
*Gets the VC bandwidth reservation according to the quality of service value.*
- `U8 STAR_API_CC_CFG_SPFI_getVCPriority (SPFI_QUALITY_OF_SERVICE qualityOfService)`  
*Gets the VC priority according to the quality of service value.*
- `U8 STAR_API_CC_CFG_SPFI_isVCContinuousModeEnabled (SPFI_QUALITY_OF_SERVICE qualityOfService)`  
*Gets the continuous mode flag according to the quality of service value.*
- `int STAR_API_CC_CFG_SPFI_setVCBandwidthReservation (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, U8 bandwidth)`  
*Sets the virtual channel bandwidth reservation for the port.*
- `int STAR_API_CC_CFG_SPFI_setVCPriority (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, U8 priority)`  
*Sets the virtual channel priority for the port.*
- `int STAR_API_CC_CFG_SPFI_enableVCContinuousMode (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel)`  
*Enables continuous mode for the virtual channel specified.*
- `int STAR_API_CC_CFG_SPFI_disableVCContinuousMode (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel)`  
*Disables continuous mode for the virtual channel specified.*

### 8.97.1 Detailed Description

Functions in this group are used to access and configure virtual channel quality of service fields.

### 8.97.2 Function Documentation

- 8.97.2.1** `int STAR_API_CC_CFG_SPFI_disableVCContinuousMode ( STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel )`

Disables continuous mode for the virtual channel specified.

When continuous mode is disabled (normal operation), the virtual channel transmit interface will not accept data when the virtual channel buffer is full. When continuous mode is enabled, the virtual channel transmit interface always accepts data i.e. flow control is ignored. This prevents packets from becoming stuck due to a lane failure, a persistent lack of space at the destination or an unexpected low bandwidth utilisation.

#### Note

Loss of data can occur when a virtual channel is configured in continuous mode. Newer data is considered more important than older data. Loss of data is indicated by an EEP.  
Any change to the continuous mode flag of a virtual channel must be followed by a link reset.

#### Parameters

|                       |                                      |
|-----------------------|--------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device. |
| <i>port</i>           | Port of the virtual channel.         |
| <i>virtualChannel</i> | The virtual channel number.          |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[virtual\\_channel\\_info\\_example.c](#).

**8.97.2.2** `int STAR_API_CC_CFG_SPFI_enableVCContinuousMode ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel )`

Enables continuous mode for the virtual channel specified.

When continuous mode is disabled (normal operation), the virtual channel transmit interface will not accept data when the virtual channel buffer is full. When continuous mode is enabled, the virtual channel transmit interface always accepts data i.e. flow control is ignored. This prevents packets from becoming stuck due to a lane failure, a persistent lack of space at the destination or an unexpected low bandwidth utilisation.

#### Note

Loss of data can occur when a virtual channel is configured in continuous mode. Newer data is considered more important than older data. Loss of data is indicated by an EEP.  
Any change to the continuous mode flag of a virtual channel must be followed by a link reset.

#### Parameters

|                       |                                      |
|-----------------------|--------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device. |
| <i>port</i>           | Port of the virtual channel.         |
| <i>virtualChannel</i> | The virtual channel number.          |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

### 8.97.2.3 U8 STAR\_API\_CC\_CFG\_SPFI\_getVCBandwidthReservation ( SPFI\_QUALITY\_OF\_SERVICE *qualityOfService* )

Gets the VC bandwidth reservation according to the quality of service value.

**Parameters**

|                         |                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>qualityOfService</i> | The virtual channel quality of service value obtained from a call to <a href="#">CFG_SPFI_getVC↔QualityOfService()</a> . |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating percentage of the link bandwidth reserved by the corresponding VC.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

### 8.97.2.4 U8 STAR\_API\_CC\_CFG\_SPFI\_getVCPriority ( SPFI\_QUALITY\_OF\_SERVICE *qualityOfService* )

Gets the VC priority according to the quality of service value.

**Parameters**

|                         |                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>qualityOfService</i> | The virtual channel quality of service value obtained from a call to <a href="#">CFG_SPFI_getVC↔QualityOfService()</a> . |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating priority of the virtual channel from 0 (highest priority) to 3 (lowest priority).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

**8.97.2.5** `int STAR_API_CC CFG_SPFI_getVCQualityOfService ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, _Out_ SPFI_QUALITY_OF_SERVICE * pQualityOfService )`

Reads from a virtual channel quality of service register and populates a `SPFI_QUALITY_OF_SERVICE` for use with the following sub-accessor functions:

- `CFG_SPFI_getVCBandwidthReservation()`
- `CFG_SPFI_getVCPriority()`
- `CFG_SPFI_isVCContinuousModeEnabled()`

**Parameters**

|     |                          |                                                                       |
|-----|--------------------------|-----------------------------------------------------------------------|
|     | <i>deviceID</i>          | Identifier of the SpaceFibre device.                                  |
|     | <i>port</i>              | Port of the virtual channel.                                          |
|     | <i>virtualChannel</i>    | The virtual channel number.                                           |
| out | <i>pQualityOfService</i> | <code>SPFI_QUALITY_OF_SERVICE</code> to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

**8.97.2.6** `U8 STAR_API_CC CFG_SPFI_isVCContinuousModeEnabled ( SPFI_QUALITY_OF_SERVICE qualityOfService )`

Gets the continuous mode flag according to the quality of service value.

When continuous mode is disabled (normal operation), the virtual channel transmit interface will not accept data when the virtual channel buffer is full. When continuous mode is enabled, the virtual channel transmit interface always accepts data i.e. flow control is ignored. This prevents packets from becoming stuck due to a lane failure, a persistent lack of space at the destination or an unexpected low bandwidth utilisation.

**Note**

Loss of data can occur when a virtual channel is configured in continuous mode. Newer data is considered more important than older data. Loss of data is indicated by an EEP.

Any change to the continuous mode flag of a virtual channel must be followed by a link reset.

**Parameters**


---

|                         |                                                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>qualityOfService</i> | The virtual channel quality of service value obtained from a call to <a href="#">CFG_SPFI_getVC↔QualityOfService()</a> . |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not continuous mode is enabled for the corresponding VC.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

**8.97.2.7** `int STAR_API_CC CFG_SPFI_setVCBandwidthReservation ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 bandwidth )`

Sets the virtual channel bandwidth reservation for the port.

**Parameters**


---

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.                                            |
| <i>port</i>           | Port of the virtual channel.                                                    |
| <i>virtualChannel</i> | The virtual channel number.                                                     |
| <i>bandwidth</i>      | Percentage of the link bandwidth reserved by the corresponding virtual channel. |

---

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

**8.97.2.8** `int STAR_API_CC CFG_SPFI_setVCPriority ( STAR_DEVICE_ID deviceID, U8 port, U8 virtualChannel, U8 priority )`

Sets the virtual channel priority for the port.



**Parameters**

|                       |                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.                                              |
| <i>port</i>           | Port of the virtual channel.                                                      |
| <i>virtualChannel</i> | The virtual channel number.                                                       |
| <i>priority</i>       | Priority of the virtual channel from 0 (highest priority) to 3 (lowest priority). |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

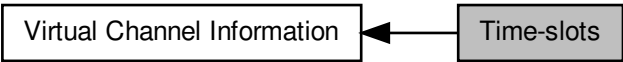
**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

## 8.98 Time-slots

Functions in this group are used to access and configure virtual channel time-slots.

Collaboration diagram for Time-slots:



### Functions

- int `STAR_API_CC_CFG_SPFI_getVCTimeSlots` (`STAR_DEVICE_ID` deviceID, `U8` port, `U8` virtualChannel, `_Out_ U64` \*pVCTimeSlots)  
*Gets a 64-bit mask representing the allowed time-slots for the virtual channel specified.*
- int `STAR_API_CC_CFG_SPFI_setVCTimeSlots` (`STAR_DEVICE_ID` deviceID, `U8` port, `U8` virtualChannel, `U64` timeSlots)  
*Sets a 64-bit mask representing the allowed time-slots for the virtual channel specified.*

### 8.98.1 Detailed Description

Functions in this group are used to access and configure virtual channel time-slots.

### 8.98.2 Function Documentation

**8.98.2.1** int `STAR_API_CC_CFG_SPFI_getVCTimeSlots` ( `STAR_DEVICE_ID` deviceID, `U8` port, `U8` virtualChannel, `_Out_ U64` \* *pVCTimeSlots* )

Gets a 64-bit mask representing the allowed time-slots for the virtual channel specified.

#### Parameters

|     |                       |                                                                                                      |
|-----|-----------------------|------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>       | Identifier of the SpaceFibre device.                                                                 |
|     | <i>port</i>           | Port of the virtual channel.                                                                         |
|     | <i>virtualChannel</i> | The virtual channel number.                                                                          |
| out | <i>pVCTimeSlots</i>   | <code>U64</code> to populate with the time-slot mask. Bits 0 to 63 correspond to time-slots 0 to 63. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

**8.98.2.2 int STAR\_API\_CC\_CFG\_SPFI\_setVCTimeSlots ( STAR\_DEVICE\_ID *deviceID*, U8 *port*, U8 *virtualChannel*, U64 *timeSlots* )**

Sets a 64-bit mask representing the allowed time-slots for the virtual channel specified.

**Parameters**

|                       |                                                                    |
|-----------------------|--------------------------------------------------------------------|
| <i>deviceID</i>       | Identifier of the SpaceFibre device.                               |
| <i>port</i>           | Port of the virtual channel.                                       |
| <i>virtualChannel</i> | The virtual channel number.                                        |
| <i>timeSlots</i>      | The time-slot mask. Bits 0 to 63 correspond to time-slots 0 to 63. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

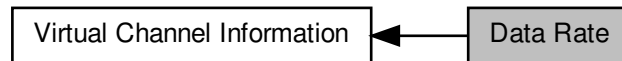
**Examples:**

[virtual\\_channel\\_info\\_example.c](#).

## 8.99 Data Rate

Functions in this group are used to access virtual channel data rate.

Collaboration diagram for Data Rate:



### Functions

- `int STAR_API_CC_CFG_SPFI_getVCDataRate (STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ U64 *pTxDataRate, _Out_ U64 *pRxDataRate)`  
*Gets the virtual channel transmit and receive data rate.*

### 8.99.1 Detailed Description

Functions in this group are used to access virtual channel data rate.

### 8.99.2 Function Documentation

8.99.2.1 `int STAR_API_CC_CFG_SPFI_getVCDataRate ( STAR_DEVICE_ID deviceId, U8 port, U8 virtualChannel, _Out_ U64 * pTxDataRate, _Out_ U64 * pRxDataRate )`

Gets the virtual channel transmit and receive data rate.

#### Parameters

|     |                       |                                                       |
|-----|-----------------------|-------------------------------------------------------|
|     | <i>deviceId</i>       | Identifier of the SpaceFibre device.                  |
|     | <i>port</i>           | Port of the virtual channel.                          |
|     | <i>virtualChannel</i> | The virtual channel number.                           |
| out | <i>pTxDataRate</i>    | U64 to populate with the transmit data rate in bit/s. |
| out | <i>pRxDataRate</i>    | U64 to populate with the receive data rate in bit/s.  |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 - 7th October 2024

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

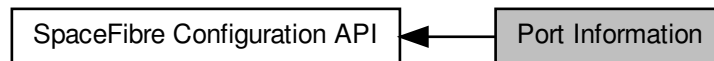
#### Examples:

[virtual\\_channel\\_info\\_example.c](#).

## 8.100 Port Information

Functions in this group are used to access and configure port information such as the port type, number of virtual channels, ready status, broadcast error and virtual channel errors.

Collaboration diagram for Port Information:



### Functions

- `int STAR_API_CC_CFG_SPFI_getPortInformation (STAR_DEVICE_ID deviceId, U8 port, _Out_ SPFI_PORT_INFO *pPortInfo)`  
*Reads from the port information register and populates a `SPFI_PORT_INFO` value for use with the following sub-accessor functions:*
  - `SPFI_PORT_TYPE STAR_API_CC_CFG_SPFI_getPortType (SPFI_PORT_INFO portInfo)`  
*Gets the port type according to the port information value.*
  - `U8 STAR_API_CC_CFG_SPFI_getPortVCCount (SPFI_PORT_INFO portInfo)`  
*Gets the number of virtual channels according to the port information value.*
  - `U8 STAR_API_CC_CFG_SPFI_isPortReady (SPFI_PORT_INFO portInfo)`  
*Gets the SpaceFibre port ready flag according to the port information value.*
  - `U8 STAR_API_CC_CFG_SPFI_isPortBroadcastErrorDetected (SPFI_PORT_INFO portInfo)`  
*Gets the SpaceFibre port broadcast error detected flag according to the port information value.*
  - `U8 STAR_API_CC_CFG_SPFI_getAXIBC (SPFI_PORT_INFO portInfo)`  
*Gets the broadcast channel used by the broadcast interface associated with an AXI port.*
- `int STAR_API_CC_CFG_SPFI_clearPortBroadcastError (STAR_DEVICE_ID deviceId, U8 port)`  
*Clears any broadcast errors on the port.*
- `int STAR_API_CC_CFG_SPFI_setAXIBC (STAR_DEVICE_ID deviceId, U8 port, U8 axiBC)`  
*Sets the broadcast channel used by the broadcast interface associated with an AXI port.*
- `int STAR_API_CC_CFG_SPFI_getPortVCsWithErrors (STAR_DEVICE_ID deviceId, U8 port, _Out_ U32 *pVCsWithErrors)`  
*Reads from a virtual channel status register and populates a `U32` with a bitset indicating if each virtual channel has an error.*

### 8.100.1 Detailed Description

Functions in this group are used to access and configure port information such as the port type, number of virtual channels, ready status, broadcast error and virtual channel errors.

### 8.100.2 Function Documentation

#### 8.100.2.1 `int STAR_API_CC_CFG_SPFI_clearPortBroadcastError ( STAR_DEVICE_ID deviceId, U8 port )`

Clears any broadcast errors on the port.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to clear errors for.            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[port\\_info\\_example.c](#).

**8.100.2.2 U8 STAR\_API\_CC CFG\_SPFI\_getAXIBC ( SPFI\_PORT\_INFO portInfo )**

Gets the broadcast channel used by the broadcast interface associated with an AXI port.

**Parameters**

|                 |                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------|
| <i>portInfo</i> | The port information value obtained from a call to <a href="#">CFG_SPFI_getPortInformation()</a> . |
|-----------------|----------------------------------------------------------------------------------------------------|

**Returns**

U8 representing the AXI broadcast channel of the device.

**Added in version:**

STAR-System v6.00 - 7th October 2024

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[port\\_info\\_example.c](#).

**8.100.2.3 int STAR\_API\_CC CFG\_SPFI\_getPortInformation ( STAR\_DEVICE\_ID deviceID, U8 port, \_Out\_ SPFI\_PORT\_INFO \* pPortInfo )**

Reads from the port information register and populates a [SPFI\\_PORT\\_INFO](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_getPortType\(\)](#)
- [CFG\\_SPFI\\_getPortVCCCount\(\)](#)
- [CFG\\_SPFI\\_isPortReady\(\)](#)
- [CFG\\_SPFI\\_isPortBroadcastErrorDetected\(\)](#)
- [CFG\\_SPFI\\_getAXIBC\(\)](#)

**Note**

The configuration port 0 is not supported by this function.

Parameters

|     |                  |                                                 |
|-----|------------------|-------------------------------------------------|
|     | <i>deviceID</i>  | Identifier of the SpaceFibre device.            |
|     | <i>port</i>      | Port to get information for.                    |
| out | <i>pPortInfo</i> | SPFI_PORT_INFO to populate with the value read. |

Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

[data\\_signalling\\_rate\\_example.c](#), [lane\\_info\\_example.c](#), [link\\_info\\_example.c](#), [port\\_info\\_example.c](#), and [virtual\\_channel\\_info\\_example.c](#).

8.100.2.4 SPFI\_PORT\_TYPE STAR\_API\_CC CFG\_SPFI\_getPortType ( SPFI\_PORT\_INFO portInfo )

Gets the port type according to the port information value.

Parameters

|  |                 |                                                                                                    |
|--|-----------------|----------------------------------------------------------------------------------------------------|
|  | <i>portInfo</i> | The port information value obtained from a call to <a href="#">CFG_SPFI_getPortInformation()</a> . |
|--|-----------------|----------------------------------------------------------------------------------------------------|

Returns

SPFI\_PORT\_TYPE representing the port type of the device.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

[data\\_signalling\\_rate\\_example.c](#), [lane\\_info\\_example.c](#), [link\\_info\\_example.c](#), [port\\_info\\_example.c](#), and [virtual\\_channel\\_info\\_example.c](#).

8.100.2.5 U8 STAR\_API\_CC CFG\_SPFI\_getPortVCCCount ( SPFI\_PORT\_INFO portInfo )

Gets the number of virtual channels according to the port information value.

Parameters

|  |                 |                                                                                                    |
|--|-----------------|----------------------------------------------------------------------------------------------------|
|  | <i>portInfo</i> | The port information value obtained from a call to <a href="#">CFG_SPFI_getPortInformation()</a> . |
|--|-----------------|----------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating the number of virtual channels.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

port\_info\_example.c, and virtual\_channel\_info\_example.c.

**8.100.2.6** `int STAR_API_CC_CFG_SPFI_getPortVCsWithErrors ( STAR_DEVICE_ID deviceId, U8 port, _Out_ U32 * pVCsWithErrors )`

Reads from a virtual channel status register and populates a U32 with a bitset indicating if each virtual channel has an error.

**Parameters**

|            |                       |                                                                          |
|------------|-----------------------|--------------------------------------------------------------------------|
|            | <i>deviceId</i>       | Identifier of the SpaceFibre device.                                     |
|            | <i>port</i>           | Port of the virtual channel.                                             |
| <i>out</i> | <i>pVCsWithErrors</i> | U32 to populate with the VC mask. Bits 0 to 31 correspond to VC 0 to 31. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

port\_info\_example.c.

**8.100.2.7** `U8 STAR_API_CC_CFG_SPFI_isPortBroadcastErrorDetected ( SPFI_PORT_INFO portInfo )`

Gets the SpaceFibre port broadcast error detected flag according to the port information value.

Set when a broadcast received to this port is discarded.

**Parameters**

|  |                 |                                                                                                    |
|--|-----------------|----------------------------------------------------------------------------------------------------|
|  | <i>portInfo</i> | The port information value obtained from a call to <a href="#">CFG_SPFI_getPortInformation()</a> . |
|--|-----------------|----------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not a broadcast error was detected on the port.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023



Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

port\_info\_example.c.

8.100.2.8 U8 STAR\_API\_CC\_CFG\_SPFI\_isPortReady ( SPFI\_PORT\_INFO portInfo )

Gets the SpaceFibre port ready flag according to the port information value.  
This is set when the port is ready to send and receive packets.

Parameters

|          |                                                                                                 |
|----------|-------------------------------------------------------------------------------------------------|
| portInfo | The port information value obtained from a call to <code>CFG_SPFI_getPortInformation()</code> . |
|----------|-------------------------------------------------------------------------------------------------|

Returns

U8 indicating whether or not the port is ready.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

port\_info\_example.c.

8.100.2.9 int STAR\_API\_CC\_CFG\_SPFI\_setAXIBC ( STAR\_DEVICE\_ID deviceID, U8 port, U8 axiBC )

Sets the broadcast channel used by the broadcast interface associated with an AXI port.

Note

This function only applies to AXI ports.  
The broadcast channel must be network unique and should be set by the network manager.  
The STAR-Ultra PCIe Single-Lane Router supports a broadcast channel value of 0 to 63.

Parameters

|          |                                        |
|----------|----------------------------------------|
| deviceID | Identifier of the SpaceFibre device.   |
| port     | Port to set AXI broadcast channel for. |
| axiBC    | The broadcast channel to set.          |

Returns

0 for success, or a negative error code upon failure as defined in `Defines`.

Added in version:

STAR-System v6.00 - 7th October 2024

Supported devices:

STAR-Ultra PCIe Single-Lane Router

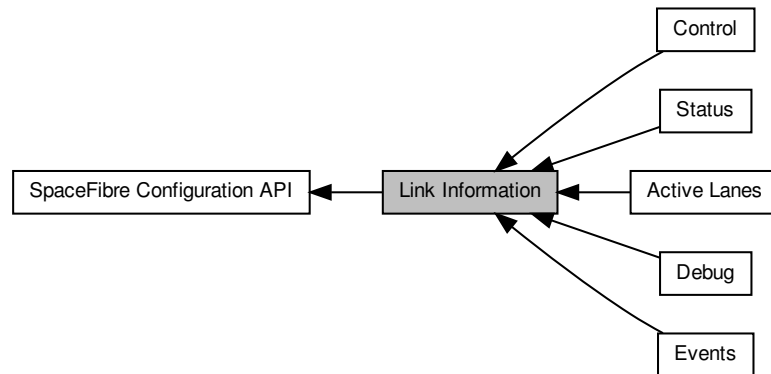
Examples:

`port_info_example.c`.

## 8.101 Link Information

Functions in this group deal are used to access and configure link information such as the control fields, status and errors.

Collaboration diagram for Link Information:



### Modules

- **Control**

*Functions in this group are used to configure the link.*

- **Status**

*Functions in this group are used to access and clear general link status information.*

- **Events**

*Functions in this group are used to access and clear general link event information.*

- **Debug**

*Functions in this group provide additional configuration and information for link debugging.*

- **Active Lanes**

*Functions in this group indicate which lanes are active.*

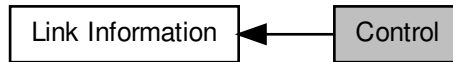
### 8.101.1 Detailed Description

Functions in this group deal are used to access and configure link information such as the control fields, status and errors.

## 8.102 Control

Functions in this group are used to configure the link.

Collaboration diagram for Control:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLinkControlSettings (STAR_DEVICE_ID deviceId, U8 port, _Out_ SPFI_LINK_CONTROL_SETTINGS *pLinkControlSettings)`  
*Reads from the link control register and populates a `SPFI_LINK_CONTROL_SETTINGS` value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isLinkAutoStartEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link autostart enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkStartEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link start enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkResetEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link reset enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkInterfaceResetEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link interface reset enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_getLinkLanesRequestedCount (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the number of lanes requested according to the link control settings value.*
- `int STAR_API_CC_CFG_SPFI_enableLinkAutoStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables autostart for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkAutoStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables autostart for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables start for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables start for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables reset for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables reset for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkInterfaceReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables interface reset for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkInterfaceReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables interface reset for the link.*
- `int STAR_API_CC_CFG_SPFI_setLinkLanesRequestedCount (STAR_DEVICE_ID deviceId, U8 port, U8 lanesRequestedCount)`  
*Sets the lanes requested count for the link.*

### 8.102.1 Detailed Description

Functions in this group are used to configure the link.

### 8.102.2 Function Documentation

#### 8.102.2.1 `int STAR_API_CC_CFG_SPFI_disableLinkAutoStart ( STAR_DEVICE_ID deviceId, U8 port )`

Disables autostart for the link.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable autostart for.       |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[link\\_info\\_example.c](#).

#### 8.102.2.2 `int STAR_API_CC_CFG_SPFI_disableLinkInterfaceReset ( STAR_DEVICE_ID deviceId, U8 port )`

Disables interface reset for the link.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable interface reset for. |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[link\\_info\\_example.c](#).

#### 8.102.2.3 `int STAR_API_CC_CFG_SPFI_disableLinkReset ( STAR_DEVICE_ID deviceId, U8 port )`

Disables reset for the link.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable reset for.           |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.102.2.4 int STAR\_API\_CC\_CFG\_SPFI\_disableLinkStart ( STAR\_DEVICE\_ID *deviceID*, U8 *port* )**

Disables start for the link.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable start for.           |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.102.2.5 int STAR\_API\_CC\_CFG\_SPFI\_enableLinkAutoStart ( STAR\_DEVICE\_ID *deviceID*, U8 *port* )**

Enables autostart for the link.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable autostart for.        |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.102.2.6** `int STAR_API_CC_CFG_SPFI_enableLinkInterfaceReset ( STAR_DEVICE_ID deviceId, U8 port )`

Enables interface reset for the link.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable interface reset for.  |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.102.2.7** `int STAR_API_CC_CFG_SPFI_enableLinkReset ( STAR_DEVICE_ID deviceId, U8 port )`

Enables reset for the link.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable reset for.            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

#### 8.102.2.8 int STAR\_API\_CC\_CFG\_SPFI\_enableLinkStart ( STAR\_DEVICE\_ID *deviceId*, U8 *port* )

Enables start for the link.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable start for.            |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[link\\_info\\_example.c](#).

#### 8.102.2.9 int STAR\_API\_CC\_CFG\_SPFI\_getLinkControlSettings ( STAR\_DEVICE\_ID *deviceId*, U8 *port*, \_Out\_ SPFI\_LINK\_CONTROL\_SETTINGS \* *pLinkControlSettings* )

Reads from the link control register and populates a [SPFI\\_LINK\\_CONTROL\\_SETTINGS](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isLinkAutoStartEnabled\(\)](#)
- [CFG\\_SPFI\\_isLinkStartEnabled\(\)](#)
- [CFG\\_SPFI\\_isLinkResetEnabled\(\)](#)
- [CFG\\_SPFI\\_isLinkInterfaceResetEnabled\(\)](#)
- [CFG\\_SPFI\\_getLinkLanesRequestedCount\(\)](#)

##### Parameters

|     |                             |                                                                             |
|-----|-----------------------------|-----------------------------------------------------------------------------|
|     | <i>deviceId</i>             | Identifier of the SpaceFibre device.                                        |
|     | <i>port</i>                 | The SpaceFibre port.                                                        |
| out | <i>pLinkControlSettings</i> | <a href="#">SPFI_LINK_CONTROL_SETTINGS</a> to populate with the value read. |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[link\\_info\\_example.c](#).



#### 8.102.2.10 U8 STAR\_API\_CC\_CFG\_SPFI\_getLinkLanesRequestedCount ( SPFI\_LINK\_CONTROL\_SETTINGS *linkControlSettings* )

Gets the number of lanes requested according to the link control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>linkControl↔<br/>Settings</i> | The link control settings value obtained from a call to <code>CFG_SPFI_getLinkControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the number of lanes requested.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.102.2.11 U8 STAR\_API\_CC CFG\_SPFI\_isLinkAutoStartEnabled ( SPFI\_LINK\_CONTROL\_SETTINGS *linkControlSettings* )

Gets the link autostart enabled state according to the link control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>linkControl↔<br/>Settings</i> | The link control settings value obtained from a call to <code>CFG_SPFI_getLinkControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is autostart enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.102.2.12 U8 STAR\_API\_CC CFG\_SPFI\_isLinkInterfaceResetEnabled ( SPFI\_LINK\_CONTROL\_SETTINGS *linkControlSettings* )

Gets the link interface reset enabled state according to the link control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>linkControl↔<br/>Settings</i> | The link control settings value obtained from a call to <code>CFG_SPFI_getLinkControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is interface reset enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

link\_info\_example.c.

**8.102.2.13 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkResetEnabled ( SPFI\_LINK\_CONTROL\_SETTINGS linkControlSettings )**

Gets the link reset enabled state according to the link control settings value.

**Parameters**


---

|                            |                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <i>linkControlSettings</i> | The link control settings value obtained from a call to <code>CFG_SPFI_getLinkControlSettings()</code> . |
|----------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is reset enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

link\_info\_example.c.

**8.102.2.14 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkStartEnabled ( SPFI\_LINK\_CONTROL\_SETTINGS linkControlSettings )**

Gets the link start enabled state according to the link control settings value.

**Parameters**


---

|                            |                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <i>linkControlSettings</i> | The link control settings value obtained from a call to <code>CFG_SPFI_getLinkControlSettings()</code> . |
|----------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is start enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.102.2.15** `int STAR_API_CC_CFG_SPFI_setLinkLanesRequestedCount ( STAR_DEVICE_ID deviceId, U8 port, U8 lanesRequestedCount )`

Sets the lanes requested count for the link.

**Parameters**

|                            |                                         |
|----------------------------|-----------------------------------------|
| <i>deviceId</i>            | Identifier of the SpaceFibre device.    |
| <i>port</i>                | Port to set lanes requested count for.  |
| <i>lanesRequestedCount</i> | The lanes requested count value to set. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

## 8.103 Status

Functions in this group are used to access and clear general link status information.

Collaboration diagram for Status:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLinkStatus (STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_STATUS *pLinkStatus)`  
*Reads from the link status register and populates a SPFI\_LINK\_STATUS value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isLinkReady (SPFI_LINK_STATUS linkStatus)`  
*Gets the link ready status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkErrorDetected (SPFI_LINK_STATUS linkStatus)`  
*Gets the link error detected status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkProtocolErrorDetected (SPFI_LINK_STATUS linkStatus)`  
*Gets the link protocol error detected status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkIdle (SPFI_LINK_STATUS linkStatus)`  
*Gets the link idle status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkEventDetected (SPFI_LINK_STATUS linkStatus)`  
*Gets the link event detected status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkLanesEventDetected (SPFI_LINK_STATUS linkStatus)`  
*Gets the lane event detected status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkTransmitting (SPFI_LINK_STATUS linkStatus)`  
*Gets the link transmitting status according to the link status value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkReceiving (SPFI_LINK_STATUS linkStatus)`  
*Gets the link receiving status according to the link status value.*
- `SPFI_LINK_ALIGNMENT_STATE STAR_API_CC_CFG_SPFI_getLinkAlignmentState (SPFI_LINK_STATUS linkStatus)`  
*Gets the link alignment state according to the link status value.*
- `int STAR_API_CC_CFG_SPFI_clearLinkProtocolError (STAR_DEVICE_ID deviceID, U8 port)`  
*Clears any link protocol errors on the port.*

#### 8.103.1 Detailed Description

Functions in this group are used to access and clear general link status information.

#### 8.103.2 Function Documentation

##### 8.103.2.1 `int STAR_API_CC_CFG_SPFI_clearLinkProtocolError ( STAR_DEVICE_ID deviceID, U8 port )`

Clears any link protocol errors on the port.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device.    |
| <i>port</i>     | Port to clear link protocol errors for. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

#### 8.103.2.2 **SPFI\_LINK\_ALIGNMENT\_STATE** STAR\_API\_CC CFG\_SPFI\_getLinkAlignmentState ( **SPFI\_LINK\_STATUS** *linkStatus* )

Gets the link alignment state according to the link status value.

**Note**

Note that this function is only relevant for multi-lane links.

**Parameters**

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <a href="#">CFG_SPFI_getLinkStatus()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

**Returns**

**SPFI\_LINK\_ALIGNMENT\_STATE** value indicating the link alignment state.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

#### 8.103.2.3 **int** STAR\_API\_CC CFG\_SPFI\_getLinkStatus ( **STAR\_DEVICE\_ID** *deviceId*, **U8** *port*, **\_Out\_** **SPFI\_LINK\_STATUS** \* *pLinkStatus* )

Reads from the link status register and populates a **SPFI\_LINK\_STATUS** value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isLinkReady\(\)](#)
- [CFG\\_SPFI\\_isLinkErrorDetected\(\)](#)

- `CFG_SPFI_isLinkProtocolErrorDetected()`
- `CFG_SPFI_isLinkIdle()`
- `CFG_SPFI_isLinkEventDetected()`
- `CFG_SPFI_isLinkLanesEventDetected()`
- `CFG_SPFI_isLinkTransmitting()`
- `CFG_SPFI_isLinkReceiving()`
- `CFG_SPFI_getLinkAlignmentState()`

**Parameters**

|            |                    |                                                                |
|------------|--------------------|----------------------------------------------------------------|
|            | <i>deviceID</i>    | Identifier of the SpaceFibre device.                           |
|            | <i>port</i>        | The SpaceFibre port.                                           |
| <i>out</i> | <i>pLinkStatus</i> | <code>SPFI_LINK_STATUS</code> to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.103.2.4 U8 STAR\_API\_CC CFG\_SPFI\_isLinkErrorDetected ( `SPFI_LINK_STATUS linkStatus` )

Gets the link error detected status according to the link status value.

**Parameters**

|  |                   |                                                                                       |
|--|-------------------|---------------------------------------------------------------------------------------|
|  | <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|--|-------------------|---------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not a link error was detected.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.103.2.5 U8 STAR\_API\_CC CFG\_SPFI\_isLinkEventDetected ( `SPFI_LINK_STATUS linkStatus` )

Gets the link event detected status according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not a link event was detected.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

**8.103.2.6 U8 STAR\_API\_CC CFG\_SPFI\_isLinkIdle ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the link idle status according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is idle.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

**8.103.2.7 U8 STAR\_API\_CC CFG\_SPFI\_isLinkLanesEventDetected ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the lane event detected status according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not a lane event was detected.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023



**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.103.2.8 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkProtocolErrorDetected ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the link protocol error detected status according to the link status value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <a href="#">CFG_SPFI_getLinkStatus()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not a link protocol error was detected.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.103.2.9 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkReady ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the link ready status according to the link status value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <a href="#">CFG_SPFI_getLinkStatus()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is ready.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.103.2.10 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkReceiving ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the link receiving status according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is receiving.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

**8.103.2.11 U8 STAR\_API\_CC CFG\_SPFI\_isLinkTransmitting ( SPFI\_LINK\_STATUS *linkStatus* )**

Gets the link transmitting status according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkStatus</i> | The link status value obtained from a call to <code>CFG_SPFI_getLinkStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the link is transmitting.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

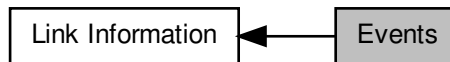
**Examples:**

`link_info_example.c`.

## 8.104 Events

Functions in this group are used to access and clear general link event information.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLinkEvents (STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_EVENTS *pLinkEvents)`  
*Reads from the link events register and populates a `SPFI_LINK_EVENTS` value for use with the following sub-accessor functions:*
  - `U8 STAR_API_CC_CFG_SPFI_isLinkFarEndResetDetected (SPFI_LINK_EVENTS linkEvents)`  
*Gets the link far-end reset state according to the link status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLinkCRC8ErrorDetected (SPFI_LINK_EVENTS linkEvents)`  
*Gets the link CRC-8 error detected state according to the link status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLinkCRC16ErrorDetected (SPFI_LINK_EVENTS linkEvents)`  
*Gets the link CRC-16 error detected state according to the link status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLinkFrameErrorDetected (SPFI_LINK_EVENTS linkEvents)`  
*Gets the link frame error detected state according to the link status value.*
  - `U16 STAR_API_CC_CFG_SPFI_getLinkRetryCount (SPFI_LINK_EVENTS linkEvents)`  
*Gets the link retry count according to the link status value.*
- `int STAR_API_CC_CFG_SPFI_clearLinkEvents (STAR_DEVICE_ID deviceID, U8 port)`  
*Clears any link events on the port.*

#### 8.104.1 Detailed Description

Functions in this group are used to access and clear general link event information.

#### 8.104.2 Function Documentation

##### 8.104.2.1 `int STAR_API_CC_CFG_SPFI_clearLinkEvents ( STAR_DEVICE_ID deviceID, U8 port )`

Clears any link events on the port.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to clear link events for.       |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.104.2.2** `int STAR_API_CC CFG_SPFI_getLinkEvents ( STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_EVENTS * pLinkEvents )`

Reads from the link events register and populates a `SPFI_LINK_EVENTS` value for use with the following sub-accessor functions:

- `CFG_SPFI_isLinkFarEndResetDetected()`
- `CFG_SPFI_isLinkCRC8ErrorDetected()`
- `CFG_SPFI_isLinkCRC16ErrorDetected()`
- `CFG_SPFI_isLinkFrameErrorDetected()`
- `CFG_SPFI_getLinkRetryCount()`

**Parameters**

|            |                    |                                                                |
|------------|--------------------|----------------------------------------------------------------|
|            | <i>deviceID</i>    | Identifier of the SpaceFibre device.                           |
|            | <i>port</i>        | The SpaceFibre port.                                           |
| <i>out</i> | <i>pLinkEvents</i> | <code>SPFI_LINK_EVENTS</code> to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.104.2.3** `U16 STAR_API_CC CFG_SPFI_getLinkRetryCount ( SPFI_LINK_EVENTS linkEvents )`

Gets the link retry count according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkEvents</i> | The link events value obtained from a call to <code>CFG_SPFI_getLinkEvents()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U16 indicating the link retry count value.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

**8.104.2.4 U8 STAR\_API\_CC CFG\_SPFI\_isLinkCRC16ErrorDetected ( SPFI\_LINK\_EVENTS linkEvents )**

Gets the link CRC-16 error detected state according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkEvents</i> | The link events value obtained from a call to <code>CFG_SPFI_getLinkEvents()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the link CRC-16 error state value.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

**8.104.2.5 U8 STAR\_API\_CC CFG\_SPFI\_isLinkCRC8ErrorDetected ( SPFI\_LINK\_EVENTS linkEvents )**

Gets the link CRC-8 error detected state according to the link status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>linkEvents</i> | The link events value obtained from a call to <code>CFG_SPFI_getLinkEvents()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the link CRC-8 error state value.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.104.2.6 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkFarEndResetDetected ( SPFI\_LINK\_EVENTS *linkEvents* )**

Gets the link far-end reset state according to the link status value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>linkEvents</i> | The link events value obtained from a call to <a href="#">CFG_SPFI_getLinkEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the link far-end reset state value.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.104.2.7 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkFrameErrorDetected ( SPFI\_LINK\_EVENTS *linkEvents* )**

Gets the link frame error detected state according to the link status value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>linkEvents</i> | The link events value obtained from a call to <a href="#">CFG_SPFI_getLinkEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the link frame error state value.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

## 8.105 Debug

Functions in this group provide additional configuration and information for link debugging.

Collaboration diagram for Debug:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLinkDebugSettings (STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_DEBUG_SETTINGS *pLinkDebugSettings)`  
*Reads from the link debug register and populates a SPFI\_LINK\_DEBUG\_SETTINGS value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isLinkTxScramblingEnabled (SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)`  
*Gets the link transmit scrambling enabled state according to the link debug value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkNearEndLoopbackEnabled (SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)`  
*Gets the link near-end loopback enabled state according to the link debug value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkRxScramblingEnabled (SPFI_LINK_DEBUG_SETTINGS linkDebugSettings)`  
*Gets the link receive scrambling enabled state according to the link debug value.*
- `int STAR_API_CC_CFG_SPFI_enableLinkTxScrambling (STAR_DEVICE_ID deviceID, U8 port)`  
*Enables transmit scrambling for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkTxScrambling (STAR_DEVICE_ID deviceID, U8 port)`  
*Disables transmit scrambling for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkNearEndLoopback (STAR_DEVICE_ID deviceID, U8 port)`  
*Enables near-end loopback for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkNearEndLoopback (STAR_DEVICE_ID deviceID, U8 port)`  
*Disables near-end loopback for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkRxScrambling (STAR_DEVICE_ID deviceID, U8 port)`  
*Enables receive scrambling for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkRxScrambling (STAR_DEVICE_ID deviceID, U8 port)`  
*Disables receive scrambling for the link.*

### 8.105.1 Detailed Description

Functions in this group provide additional configuration and information for link debugging.

### 8.105.2 Function Documentation

#### 8.105.2.1 `int STAR_API_CC_CFG_SPFI_disableLinkNearEndLoopback ( STAR_DEVICE_ID deviceID, U8 port )`

Disables near-end loopback for the link.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device.   |
| <i>port</i>     | Port to disable near-end loopback for. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.105.2.2 int STAR\_API\_CC\_CFG\_SPFI\_disableLinkRxScrambling ( STAR\_DEVICE\_ID deviceId, U8 port )**

Disables receive scrambling for the link.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device.    |
| <i>port</i>     | Port to disable receive scrambling for. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.105.2.3 int STAR\_API\_CC\_CFG\_SPFI\_disableLinkTxScrambling ( STAR\_DEVICE\_ID deviceId, U8 port )**

Disables transmit scrambling for the link.

**Parameters**

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device.     |
| <i>port</i>     | Port to disable transmit scrambling for. |



**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.105.2.4 int STAR\_API\_CC\_CFG\_SPFI\_enableLinkNearEndLoopback ( STAR\_DEVICE\_ID deviceID, U8 port )**

Enables near-end loopback for the link.

**Parameters**

|                 |                                       |
|-----------------|---------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.  |
| <i>port</i>     | Port to enable near-end loopback for. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

**8.105.2.5 int STAR\_API\_CC\_CFG\_SPFI\_enableLinkRxScrambling ( STAR\_DEVICE\_ID deviceID, U8 port )**

Enables receive scrambling for the link.

**Parameters**

|                 |                                        |
|-----------------|----------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.   |
| <i>port</i>     | Port to enable receive scrambling for. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[link\\_info\\_example.c](#).

### 8.105.2.6 `int STAR_API_CC_CFG_SPFI_enableLinkTxScrambling ( STAR_DEVICE_ID deviceID, U8 port )`

Enables transmit scrambling for the link.

#### Parameters

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.    |
| <i>port</i>     | Port to enable transmit scrambling for. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[link\\_info\\_example.c](#).

### 8.105.2.7 `int STAR_API_CC_CFG_SPFI_getLinkDebugSettings ( STAR_DEVICE_ID deviceID, U8 port, _Out_ SPFI_LINK_DEBUG_SETTINGS * pLinkDebugSettings )`

Reads from the link debug register and populates a [SPFI\\_LINK\\_DEBUG\\_SETTINGS](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isLinkTxScramblingEnabled\(\)](#)
- [CFG\\_SPFI\\_isLinkNearEndLoopbackEnabled\(\)](#)
- [CFG\\_SPFI\\_isLinkRxScramblingEnabled\(\)](#)

#### Parameters

|     |                           |                                                                           |
|-----|---------------------------|---------------------------------------------------------------------------|
|     | <i>deviceID</i>           | Identifier of the SpaceFibre device.                                      |
|     | <i>port</i>               | The SpaceFibre port.                                                      |
| out | <i>pLinkDebugSettings</i> | <a href="#">SPFI_LINK_DEBUG_SETTINGS</a> to populate with the value read. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[link\\_info\\_example.c](#).

#### 8.105.2.8 U8 STAR\_API\_CC\_CFG\_SPFI\_isLinkNearEndLoopbackEnabled ( SPFI\_LINK\_DEBUG\_SETTINGS *linkDebugSettings* )

Gets the link near-end loopback enabled state according to the link debug value.

**Parameters**

---

|                          |                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------|
| <i>linkDebugSettings</i> | The link debug value obtained from a call to <code>CFG_SPFI_getLinkDebugSettings()</code> . |
|--------------------------|---------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not near-end loopback is enabled for the link.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.105.2.9 U8 STAR\_API\_CC CFG\_SPFI\_isLinkRxScramblingEnabled ( SPFI\_LINK\_DEBUG\_SETTINGS *linkDebugSettings* )

Gets the link receive scrambling enabled state according to the link debug value.

**Parameters**

---

|                          |                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------|
| <i>linkDebugSettings</i> | The link debug value obtained from a call to <code>CFG_SPFI_getLinkDebugSettings()</code> . |
|--------------------------|---------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not receive scrambling is enabled for the link.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`link_info_example.c`.

#### 8.105.2.10 U8 STAR\_API\_CC CFG\_SPFI\_isLinkTxScramblingEnabled ( SPFI\_LINK\_DEBUG\_SETTINGS *linkDebugSettings* )

Gets the link transmit scrambling enabled state according to the link debug value.

**Parameters**

---

|                          |                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------|
| <i>linkDebugSettings</i> | The link debug value obtained from a call to <code>CFG_SPFI_getLinkDebugSettings()</code> . |
|--------------------------|---------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not transmit scrambling is enabled for the link.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

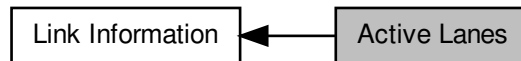
**Examples:**

`link_info_example.c`.

## 8.106 Active Lanes

Functions in this group indicate which lanes are active.

Collaboration diagram for Active Lanes:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLinkActiveLanes (STAR_DEVICE_ID deviceId, U8 port, _Out_ U32 *pActiveLanes)`

*Reads a link active lanes register and populates a U32 with a bitset indicating if each lane is active.*

### 8.106.1 Detailed Description

Functions in this group indicate which lanes are active.

### 8.106.2 Function Documentation

**8.106.2.1** `int STAR_API_CC_CFG_SPFI_getLinkActiveLanes ( STAR_DEVICE_ID deviceId, U8 port, _Out_ U32 *pActiveLanes )`

Reads a link active lanes register and populates a U32 with a bitset indicating if each lane is active.

#### Parameters

|            |                     |                                                                  |
|------------|---------------------|------------------------------------------------------------------|
|            | <i>deviceId</i>     | Identifier of the SpaceFibre device.                             |
|            | <i>port</i>         | The SpaceFibre port.                                             |
| <i>out</i> | <i>pActiveLanes</i> | U32 to populate with the lane mask. Bit 0 corresponds to lane 0. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

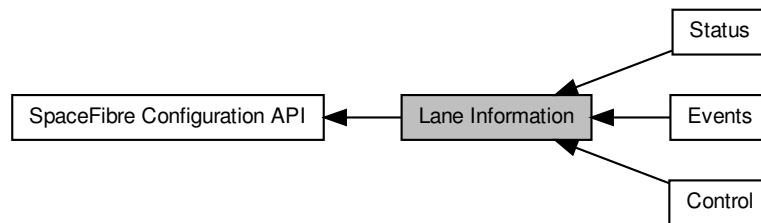
#### Examples:

[link\\_info\\_example.c](#).

## 8.107 Lane Information

Functions in this group are used to access and configure lane information such as the status and errors.

Collaboration diagram for Lane Information:



### Modules

- **Control**

*Functions in this group are used to access and configure lane configuration for a SpaceFibre link.*

- **Status**

*Functions in this group provide status information about a SpaceFibre link.*

- **Events**

*Functions in this group are used to access and clear general lane event information.*

### 8.107.1 Detailed Description

Functions in this group are used to access and configure lane information such as the status and errors.

## 8.108 Control

Functions in this group are used to access and configure lane configuration for a SpaceFibre link.

Collaboration diagram for Control:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLaneControlSettings (STAR_DEVICE_ID deviceID, U8 port, U8 lane, _↔ Out_ SPFI_LANE_CONTROL_SETTINGS *pLaneControlSettings)`  
*Reads from the lane control register and populates a SPFI\_LANE\_CONTROL\_SETTINGS value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isLaneResetEnabled (SPFI_LANE_CONTROL_SETTINGS laneControl↔ Settings)`  
*Gets the lane reset enabled state according to the lane control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_getLaneStandbyReason (SPFI_LANE_CONTROL_SETTINGS lane↔ ControlSettings)`  
*Gets the lane standby reason according to the lane control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLaneTxEnabled (SPFI_LANE_CONTROL_SETTINGS laneControl↔ Settings)`  
*Gets the lane transmit enabled state according to the lane control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLaneRxEnabled (SPFI_LANE_CONTROL_SETTINGS laneControl↔ Settings)`  
*Gets the lane receive enabled state according to the lane control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLaneErrorRecoveryEnabled (SPFI_LANE_CONTROL_SETTINGS lane↔ ControlSettings)`  
*Gets the lane error recovery enabled state according to the lane control settings value.*
- `int STAR_API_CC_CFG_SPFI_enableLaneReset (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Enables reset for the lane.*
- `int STAR_API_CC_CFG_SPFI_disableLaneReset (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Disables reset for the lane.*
- `int STAR_API_CC_CFG_SPFI_setLaneStandbyReason (STAR_DEVICE_ID deviceID, U8 port, U8 lane, U8 standbyReason)`  
*Sets the standby reason for the SpaceFibre port of the device specified.*
- `int STAR_API_CC_CFG_SPFI_enableLaneTx (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Enables transmit for the lane.*
- `int STAR_API_CC_CFG_SPFI_disableLaneTx (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Disables transmit for the lane.*
- `int STAR_API_CC_CFG_SPFI_enableLaneRx (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Enables receive for the lane.*
- `int STAR_API_CC_CFG_SPFI_disableLaneRx (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`  
*Disables receive for the lane.*
- `int STAR_API_CC_CFG_SPFI_enableLaneErrorRecovery (STAR_DEVICE_ID deviceID, U8 port, U8 lane)`



*Enables error recovery for the lane.*

- int `STAR_API_CC_CFG_SPFI_disableLaneErrorRecovery` ( `STAR_DEVICE_ID deviceID`, `U8 port`, `U8 lane` )

*Disables error recovery for the lane.*

### 8.108.1 Detailed Description

Functions in this group are used to access and configure lane configuration for a SpaceFibre link.

### 8.108.2 Function Documentation

#### 8.108.2.1 int `STAR_API_CC_CFG_SPFI_disableLaneErrorRecovery` ( `STAR_DEVICE_ID deviceID`, `U8 port`, `U8 lane` )

Disables error recovery for the lane.

##### Parameters

|                 |                                          |
|-----------------|------------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.     |
| <i>port</i>     | Port to disable lane error recovery for. |
| <i>lane</i>     | Lane to disable error recovery for.      |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

[STAR-System v6.00 Beta 1 - 6th July 2023](#)

##### Supported devices:

[STAR-Ultra PCIe Single-Lane Router](#)

##### Examples:

[lane\\_info\\_example.c](#).

#### 8.108.2.2 int `STAR_API_CC_CFG_SPFI_disableLaneReset` ( `STAR_DEVICE_ID deviceID`, `U8 port`, `U8 lane` )

Disables reset for the lane.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable lane reset for.      |
| <i>lane</i>     | Lane to disable reset for.           |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

[STAR-System v6.00 Beta 1 - 6th July 2023](#)

##### Supported devices:

[STAR-Ultra PCIe Single-Lane Router](#)

##### Examples:

[lane\\_info\\_example.c](#).

#### 8.108.2.3 int STAR\_API\_CC\_CFG\_SPFI\_disableLaneRx ( STAR\_DEVICE\_ID *deviceId*, U8 *port*, U8 *lane* )

Disables receive for the lane.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable lane receive for.    |
| <i>lane</i>     | Lane to disable receive for.         |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[lane\\_info\\_example.c](#).

#### 8.108.2.4 int STAR\_API\_CC\_CFG\_SPFI\_disableLaneTx ( STAR\_DEVICE\_ID *deviceId*, U8 *port*, U8 *lane* )

Disables transmit for the lane.

##### Parameters

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to disable lane transmit for.   |
| <i>lane</i>     | Lane to disable transmit for.        |

##### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

[lane\\_info\\_example.c](#).

#### 8.108.2.5 int STAR\_API\_CC\_CFG\_SPFI\_enableLaneErrorRecovery ( STAR\_DEVICE\_ID *deviceId*, U8 *port*, U8 *lane* )

Enables error recovery for the lane.

**Parameters**

|                 |                                         |
|-----------------|-----------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.    |
| <i>port</i>     | Port to enable lane error recovery for. |
| <i>lane</i>     | Lane to enable error recovery for.      |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

**8.108.2.6** `int STAR_API_CC_CFG_SPFI_enableLaneReset ( STAR_DEVICE_ID deviceID, U8 port, U8 lane )`

Enables reset for the lane.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable lane reset for.       |
| <i>lane</i>     | Lane to enable reset for.            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

**8.108.2.7** `int STAR_API_CC_CFG_SPFI_enableLaneRx ( STAR_DEVICE_ID deviceID, U8 port, U8 lane )`

Enables receive for the lane.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
|-----------------|--------------------------------------|

---

|             |                                  |
|-------------|----------------------------------|
| <i>port</i> | Port to enable lane receive for. |
| <i>lane</i> | Lane to enable receive for.      |

---

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[lane\\_info\\_example.c](#).

**8.108.2.8** `int STAR_API_CC_CFG_SPFI_enableLaneTx ( STAR_DEVICE_ID deviceId, U8 port, U8 lane )`

Enables transmit for the lane.

#### Parameters

---

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceId</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to enable lane transmit for.    |
| <i>lane</i>     | Lane to enable transmit for.         |

---

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

[lane\\_info\\_example.c](#).

**8.108.2.9** `int STAR_API_CC_CFG_SPFI_getLaneControlSettings ( STAR_DEVICE_ID deviceId, U8 port, U8 lane,  
_Out_ SPFI_LANE_CONTROL_SETTINGS * pLaneControlSettings )`

Reads from the lane control register and populates a [SPFI\\_LANE\\_CONTROL\\_SETTINGS](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isLaneResetEnabled\(\)](#)
- [CFG\\_SPFI\\_getLaneStandbyReason\(\)](#)
- [CFG\\_SPFI\\_isLaneTxEnabled\(\)](#)
- [CFG\\_SPFI\\_isLaneRxEnabled\(\)](#)
- [CFG\\_SPFI\\_isLaneErrorRecoveryEnabled\(\)](#)

Parameters

|     |                             |                                                             |
|-----|-----------------------------|-------------------------------------------------------------|
|     | <i>deviceID</i>             | Identifier of the SpaceFibre device.                        |
|     | <i>port</i>                 | The SpaceFibre port.                                        |
|     | <i>lane</i>                 | Lane to get settings for.                                   |
| out | <i>pLaneControlSettings</i> | SPFI_LANE_CONTROL_SETTINGS to populate with the value read. |

Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

[lane\\_info\\_example.c](#).

8.108.2.10 U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneStandbyReason ( SPFI\_LANE\_CONTROL\_SETTINGS laneControlSettings )

Gets the lane standby reason according to the lane control settings value.

Parameters

|  |                            |                                                                                                             |
|--|----------------------------|-------------------------------------------------------------------------------------------------------------|
|  | <i>laneControlSettings</i> | The lane control settings value obtained from a call to <a href="#">CFG_SPFI_getLaneControlSettings()</a> . |
|--|----------------------------|-------------------------------------------------------------------------------------------------------------|

Returns

U8 indicating value of the standby reason data character sent when the lane is disabled.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Supported devices:

STAR-Ultra PCIe Single-Lane Router

Examples:

[lane\\_info\\_example.c](#).

8.108.2.11 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneErrorRecoveryEnabled ( SPFI\_LANE\_CONTROL\_SETTINGS laneControlSettings )

Gets the lane error recovery enabled state according to the lane control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>laneControl↔<br/>Settings</i> | The lane control settings value obtained from a call to <code>CFG_SPFI_getLaneControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is error recovery enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

#### 8.108.2.12 U8 STAR\_API\_CC CFG\_SPFI\_isLaneResetEnabled ( SPFI\_LANE\_CONTROL\_SETTINGS *laneControlSettings* )

Gets the lane reset enabled state according to the lane control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>laneControl↔<br/>Settings</i> | The lane control settings value obtained from a call to <code>CFG_SPFI_getLaneControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is reset enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

#### 8.108.2.13 U8 STAR\_API\_CC CFG\_SPFI\_isLaneRxEnabled ( SPFI\_LANE\_CONTROL\_SETTINGS *laneControlSettings* )

Gets the lane receive enabled state according to the lane control settings value.

**Parameters**


---

|                                  |                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------|
| <i>laneControl↔<br/>Settings</i> | The lane control settings value obtained from a call to <code>CFG_SPFI_getLaneControlSettings()</code> . |
|----------------------------------|----------------------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is receive enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.108.2.14 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneTxEnabled ( SPFI\_LANE\_CONTROL\_SETTINGS laneControlSettings )**

Gets the lane transmit enabled state according to the lane control settings value.

**Parameters**

|                            |                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <i>laneControlSettings</i> | The lane control settings value obtained from a call to <code>CFG_SPFI_getLaneControlSettings()</code> . |
|----------------------------|----------------------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not the lane is transmit enabled.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.108.2.15 int STAR\_API\_CC\_CFG\_SPFI\_setLaneStandbyReason ( STAR\_DEVICE\_ID deviceId, U8 port, U8 lane, U8 standbyReason )**

Sets the standby reason for the SpaceFibre port of the device specified.

**Parameters**

|                      |                                      |
|----------------------|--------------------------------------|
| <i>deviceId</i>      | Identifier of the SpaceFibre device. |
| <i>port</i>          | Port to set standby reason for.      |
| <i>lane</i>          | Lane to set standby reason for.      |
| <i>standbyReason</i> | The standby reason value.            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

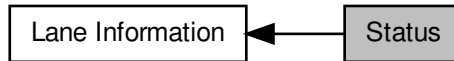
lane\_info\_example.c.



## 8.109 Status

Functions in this group provide status information about a SpaceFibre link.

Collaboration diagram for Status:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLaneStatus (STAR_DEVICE_ID deviceID, U8 port, U8 lane, _Out_ SPFI_LANE_STATUS *pLaneStatus)`  
*Reads from the lane status register and populates a SPFI\_LANE\_STATUS value for use with the following sub-accessor functions:*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneActive (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane active state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneErrorDetected (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane error detected state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneEventDetected (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane event detected state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneNoSignalDetected (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane no signal detected state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_getLaneLOSReasonRx (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane receive loss of signal reason according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_getLaneStandbyReasonRx (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane receive standby reason according to the lane status value.*
  - `SPFI_LANE_STATE STAR_API_CC_CFG_SPFI_getLaneState (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneStarted (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane started state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneTxOnly (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane transmit only state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneRxOnly (SPFI_LANE_STATUS laneStatus)`  
*Gets the lane receive only state according to the lane status value.*
  - `U8 STAR_API_CC_CFG_SPFI_getLaneFarEndINIT3Flags (SPFI_LANE_STATUS laneStatus)`  
*Gets the flags of the last received INIT3 word according to the lane status value.*

#### 8.109.1 Detailed Description

Functions in this group provide status information about a SpaceFibre link.

## 8.109.2 Function Documentation

### 8.109.2.1 U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneFarEndINIT3Flags ( SPFI\_LANE\_STATUS *laneStatus* )

Gets the flags of the last received INIT3 word according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the far end INIT3 flags.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

**8.109.2.2 U8 STAR\_API\_CC CFG\_SPFI\_getLaneLOSReasonRx ( SPFI\_LANE\_STATUS *laneStatus* )**

Gets the lane receive loss of signal reason according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the cause of the last LOST\_SIGNAL control word received.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

**8.109.2.3 U8 STAR\_API\_CC CFG\_SPFI\_getLaneStandbyReasonRx ( SPFI\_LANE\_STATUS *laneStatus* )**

Gets the lane receive standby reason according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating the cause of the last STANDBY control word received.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

#### 8.109.2.4 SPFI\_LANE\_STATE STAR\_API\_CC CFG\_SPFI\_getLaneState ( SPFI\_LANE\_STATUS laneStatus )

Gets the lane state according to the lane status value.

**Parameters**


---

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to CFG_SPFI_getLaneStatus(). |
|-------------------|-------------------------------------------------------------------------|

---

**Returns**

SPFI\_LANE\_STATE value indicating the lane state.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

#### 8.109.2.5 int STAR\_API\_CC CFG\_SPFI\_getLaneStatus ( STAR\_DEVICE\_ID deviceID, U8 port, U8 lane, \_Out\_ SPFI\_LANE\_STATUS \* pLaneStatus )

Reads from the lane status register and populates a SPFI\_LANE\_STATUS value for use with the following sub-accessor functions:

- CFG\_SPFI\_isLaneActive()
- CFG\_SPFI\_isLaneErrorDetected()
- CFG\_SPFI\_isLaneEventDetected()
- CFG\_SPFI\_isLaneNoSignalDetected()
- CFG\_SPFI\_getLaneLOSReasonRx()
- CFG\_SPFI\_getLaneStandbyReasonRx()
- CFG\_SPFI\_getLaneState()
- CFG\_SPFI\_isLaneStarted()
- CFG\_SPFI\_isLaneTxOnly()
- CFG\_SPFI\_isLaneRxOnly()
- CFG\_SPFI\_getLaneFarEndINIT3Flags()

**Parameters**

|     |                    |                                                   |
|-----|--------------------|---------------------------------------------------|
|     | <i>deviceID</i>    | Identifier of the SpaceFibre device.              |
|     | <i>port</i>        | The SpaceFibre port.                              |
|     | <i>lane</i>        | The lane to get the status for.                   |
| out | <i>pLaneStatus</i> | SPFI_LANE_STATUS to populate with the value read. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.109.2.6 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneActive ( SPFI\_LANE\_STATUS laneStatus )**

Gets the lane active state according to the lane status value.

**Parameters**

|  |                   |                                                                                          |
|--|-------------------|------------------------------------------------------------------------------------------|
|  | <i>laneStatus</i> | The lane status value obtained from a call to <a href="#">CFG_SPFI_getLaneStatus()</a> . |
|--|-------------------|------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not the lane is active.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.109.2.7 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneErrorDetected ( SPFI\_LANE\_STATUS laneStatus )**

Gets the lane error detected state according to the lane status value.

**Parameters**

|  |                   |                                                                                          |
|--|-------------------|------------------------------------------------------------------------------------------|
|  | <i>laneStatus</i> | The lane status value obtained from a call to <a href="#">CFG_SPFI_getLaneStatus()</a> . |
|--|-------------------|------------------------------------------------------------------------------------------|

**Returns**

U8 indicating whether or not an error was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.109.2.8 U8 STAR\_API\_CC CFG\_SPFI\_isLaneEventDetected ( SPFI\_LANE\_STATUS laneStatus )**

Gets the lane event detected state according to the lane status value.

**Parameters**

---

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to CFG_SPFI_getLaneStatus(). |
|-------------------|-------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not an event was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.109.2.9 U8 STAR\_API\_CC CFG\_SPFI\_isLaneNoSignalDetected ( SPFI\_LANE\_STATUS laneStatus )**

Gets the lane no signal detected state according to the lane status value.

**Parameters**

---

|                   |                                                                         |
|-------------------|-------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to CFG_SPFI_getLaneStatus(). |
|-------------------|-------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not no signal was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.109.2.10 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneRxOnly ( SPFI\_LANE\_STATUS *laneStatus* )**

Gets the lane receive only state according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is receive only.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

**8.109.2.11 U8 STAR\_API\_CC CFG\_SPFI\_isLaneStarted ( SPFI\_LANE\_STATUS *laneStatus* )**

Gets the lane started state according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is started.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

**8.109.2.12 U8 STAR\_API\_CC CFG\_SPFI\_isLaneTxOnly ( SPFI\_LANE\_STATUS *laneStatus* )**

Gets the lane transmit only state according to the lane status value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneStatus</i> | The lane status value obtained from a call to <code>CFG_SPFI_getLaneStatus()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the lane is transmit only.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023



**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

## 8.110 Events

Functions in this group are used to access and clear general lane event information.

Collaboration diagram for Events:



### Functions

- `int STAR_API_CC_CFG_SPFI_getLaneEvents (STAR_DEVICE_ID deviceId, U8 port, U8 lane, _Out_ SPFI_LANE_EVENTS *pLaneEvents)`  
*Reads from the lane events register and populates a `SPFI_LANE_EVENTS` value for use with the following sub-accessor functions:*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneInitTimeoutDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the lane initialisation timeout detected state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneFarEndStandbyDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the far-end standby state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneFarEndLOSDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the far-end loss of signal state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneFarEndErrorBurstDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the far-end burst state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneFarEndINIT1RxInActiveDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the far-end INIT1 receive in active state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneINIT1RxDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the INIT1 receive detected state according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_isLaneINIT2RxDetected (SPFI_LANE_EVENTS laneEvents)`  
*Gets the INIT2 receive detected state according to the lane events value.*
  - `U16 STAR_API_CC_CFG_SPFI_getLaneRxErrorCount (SPFI_LANE_EVENTS laneEvents)`  
*Gets the lane receive error count according to the lane events value.*
  - `U8 STAR_API_CC_CFG_SPFI_getLaneDisconnectCount (SPFI_LANE_EVENTS laneEvents)`  
*Gets the lane disconnect count according to the lane events value.*
- `int STAR_API_CC_CFG_SPFI_clearLaneEvents (STAR_DEVICE_ID deviceId, U8 port, U8 lane)`  
*Clears any events on the lane.*

#### 8.110.1 Detailed Description

Functions in this group are used to access and clear general lane event information.

#### 8.110.2 Function Documentation

##### 8.110.2.1 `int STAR_API_CC_CFG_SPFI_clearLaneEvents ( STAR_DEVICE_ID deviceId, U8 port, U8 lane )`

Clears any events on the lane.

**Parameters**

|                 |                                      |
|-----------------|--------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device. |
| <i>port</i>     | Port to clear lane events for.       |
| <i>lane</i>     | Lane to clear events for.            |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

#### 8.110.2.2 U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneDisconnectCount ( SPFI\_LANE\_EVENTS *laneEvents* )

Gets the lane disconnect count according to the lane events value.

**Parameters**

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

**Returns**

U8 indicating the disconnect count on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

#### 8.110.2.3 int STAR\_API\_CC\_CFG\_SPFI\_getLaneEvents ( STAR\_DEVICE\_ID *deviceID*, U8 *port*, U8 *lane*, \_Out\_ SPFI\_LANE\_EVENTS \* *pLaneEvents* )

Reads from the lane events register and populates a [SPFI\\_LANE\\_EVENTS](#) value for use with the following sub-accessor functions:

- [CFG\\_SPFI\\_isLaneInitTimeoutDetected\(\)](#)
- [CFG\\_SPFI\\_isLaneFarEndStandbyDetected\(\)](#)
- [CFG\\_SPFI\\_isLaneFarEndLOSDetected\(\)](#)
- [CFG\\_SPFI\\_isLaneFarEndErrorBurstDetected\(\)](#)

- `CFG_SPFI_isLaneFarEndINIT1RxInActiveDetected()`
- `CFG_SPFI_isLaneINIT1RxDetected()`
- `CFG_SPFI_isLaneINIT2RxDetected()`
- `CFG_SPFI_getLaneRxErrorCount()`
- `CFG_SPFI_getLaneDisconnectCount()`

#### Parameters

|     |                    |                                                                |
|-----|--------------------|----------------------------------------------------------------|
|     | <i>deviceId</i>    | Identifier of the SpaceFibre device.                           |
|     | <i>port</i>        | The SpaceFibre port.                                           |
|     | <i>lane</i>        | The lane to get events for.                                    |
| out | <i>pLaneEvents</i> | <code>SPFI_LANE_EVENTS</code> to populate with the value read. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

`lane_info_example.c`.

#### 8.110.2.4 U16 STAR\_API\_CC CFG\_SPFI\_getLaneRxErrorCount ( SPFI\_LANE\_EVENTS laneEvents )

Gets the lane receive error count according to the lane events value.

#### Parameters

|  |                   |                                                                                       |
|--|-------------------|---------------------------------------------------------------------------------------|
|  | <i>laneEvents</i> | The lane events value obtained from a call to <code>CFG_SPFI_getLaneEvents()</code> . |
|--|-------------------|---------------------------------------------------------------------------------------|

#### Returns

U16 indicating the receive error count on the lane.

#### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### Supported devices:

STAR-Ultra PCIe Single-Lane Router

#### Examples:

`lane_info_example.c`.

#### 8.110.2.5 U8 STAR\_API\_CC CFG\_SPFI\_isLaneFarEndErrorBurstDetected ( SPFI\_LANE\_EVENTS laneEvents )

Gets the far-end burst state according to the lane events value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not far-end burst was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

**8.110.2.6 U8 STAR\_API\_CC CFG\_SPFI\_isLaneFarEndINIT1RxInActiveDetected ( SPFI\_LANE\_EVENTS *laneEvents* )**

Gets the far-end INIT1 receive in active state according to the lane events value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not the far-end of the lane indicates a INIT1 control word received while in Active state.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[lane\\_info\\_example.c](#).

**8.110.2.7 U8 STAR\_API\_CC CFG\_SPFI\_isLaneFarEndLOSDetected ( SPFI\_LANE\_EVENTS *laneEvents* )**

Gets the far-end loss of signal state according to the lane events value.

**Parameters**

---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not far-end loss of signal was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.110.2.8 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneFarEndStandbyDetected ( SPFI\_LANE\_EVENTS laneEvents )**

Gets the far-end standby state according to the lane events value.

**Parameters**


---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not far-end standby was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.110.2.9 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneINIT1RxDetected ( SPFI\_LANE\_EVENTS laneEvents )**

Gets the INIT1 receive detected state according to the lane events value.

**Parameters**


---

|                   |                                                                                          |
|-------------------|------------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <a href="#">CFG_SPFI_getLaneEvents()</a> . |
|-------------------|------------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not an INIT1 control word is received by the lane layer.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

lane\_info\_example.c.

**8.110.2.10 U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneINIT2RxDetected ( SPFI\_LANE\_EVENTS laneEvents )**

Gets the INIT2 receive detected state according to the lane events value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <code>CFG_SPFI_getLaneEvents()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not an INIT2 control word is received by the lane layer.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

**8.110.2.11 U8 STAR\_API\_CC CFG\_SPFI\_isLaneInitTimeoutDetected ( SPFI\_LANE\_EVENTS *laneEvents* )**

Gets the lane initialisation timeout detected state according to the lane events value.

**Parameters**

---

|                   |                                                                                       |
|-------------------|---------------------------------------------------------------------------------------|
| <i>laneEvents</i> | The lane events value obtained from a call to <code>CFG_SPFI_getLaneEvents()</code> . |
|-------------------|---------------------------------------------------------------------------------------|

---

**Returns**

U8 indicating whether or not an initialisation timeout was detected on the lane.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

`lane_info_example.c`.

## 8.111 Data Signalling Rate

Functions in this group are used to access and configure SpaceFibre lane data signalling rate.

Collaboration diagram for Data Signalling Rate:



### Functions

- `U64 *STAR_API_CC_CFG_SPFI_getSpFiBitRates (STAR_DEVICE_ID deviceId, _Out_ U32 *pCount)`  
*Returns an array of the SpaceFibre lane data signalling rates supported by the device specified in bit/s.*
- `void STAR_API_CC_CFG_SPFI_destroySpFiBitRates (U64 *pBitRates)`  
*Releases the array of SpaceFibre lane data signalling rates returned by the `CFG_SPFI_getSpFiBitRates()` function.*
- `int STAR_API_CC_CFG_SPFI_getSpFiBitRate (STAR_DEVICE_ID deviceId, U8 port, _Out_ U64 *pBitRate)`  
*Gets the SpaceFibre port lane data signalling rate.*
- `int STAR_API_CC_CFG_SPFI_setSpFiBitRate (STAR_DEVICE_ID deviceId, U8 port, U64 bitRate)`  
*Sets the SpaceFibre port lane data signalling rate.*

### 8.111.1 Detailed Description

Functions in this group are used to access and configure SpaceFibre lane data signalling rate.

### 8.111.2 Function Documentation

#### 8.111.2.1 `void STAR_API_CC_CFG_SPFI_destroySpFiBitRates ( U64 * pBitRates )`

Releases the array of SpaceFibre lane data signalling rates returned by the `CFG_SPFI_getSpFiBitRates()` function.

##### Parameters

|                  |                                  |
|------------------|----------------------------------|
| <i>pBitRates</i> | The data signalling rates array. |
|------------------|----------------------------------|

##### Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

##### Supported devices:

STAR-Ultra PCIe Single-Lane Router

##### Examples:

`data_signalling_rate_example.c`.

#### 8.111.2.2 `int STAR_API_CC_CFG_SPFI_getSpFiBitRate ( STAR_DEVICE_ID deviceId, U8 port, _Out_ U64 * pBitRate )`

Gets the SpaceFibre port lane data signalling rate.



**Parameters**

|     |                 |                                                                                       |
|-----|-----------------|---------------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Identifier of the SpaceFibre device.                                                  |
|     | <i>port</i>     | Port to get lane data signalling rate of.                                             |
| out | <i>pBitRate</i> | U64 to populate with the data signalling rate in bit/s e.g. 2.5 Gbit/s = 25000000000. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[data\\_signalling\\_rate\\_example.c](#).

### 8.111.2.3 U64\* STAR\_API\_CC\_CFG\_SPFI\_getSpFiBitRates ( STAR\_DEVICE\_ID *deviceID*, \_Out\_ U32 \* *pCount* )

Returns an array of the SpaceFibre lane data signalling rates supported by the device specified in bit/s.

**Note**

Release the array of data signalling rates using [CFG\\_SPFI\\_destroySpFiBitRates\(\)](#).

**Parameters**

|     |                 |                                                                                   |
|-----|-----------------|-----------------------------------------------------------------------------------|
|     | <i>deviceID</i> | Identifier of the SpaceFibre device.                                              |
| out | <i>pCount</i>   | U32 to populate with the number of data signalling rates supported by the device. |

**Returns**

An array of the SpaceFibre lane data signalling rates supported by the device specified. Null if error.

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

**Examples:**

[data\\_signalling\\_rate\\_example.c](#).

### 8.111.2.4 int STAR\_API\_CC\_CFG\_SPFI\_setSpFiBitRate ( STAR\_DEVICE\_ID *deviceID*, U8 *port*, U64 *bitRate* )

Sets the SpaceFibre port lane data signalling rate.

**Note**

The data signalling rates supported by the device can be accessed using the function [CFG\\_SPFI\\_getSpFiBitRate\(\)](#).

**Parameters**

|                 |                                                                 |
|-----------------|-----------------------------------------------------------------|
| <i>deviceID</i> | Identifier of the SpaceFibre device.                            |
| <i>port</i>     | Port to set lane data signalling rate of.                       |
| <i>bitRate</i>  | The data signalling rate in bit/s e.g. 2.5 Gbit/s = 2500000000. |

**Returns**

0 for success, or a negative error code upon failure as defined in [Defines](#).

**Added in version:**

STAR-System v6.00 Beta 1 - 6th July 2023

**Supported devices:**

STAR-Ultra PCIe Single-Lane Router

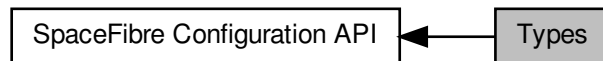
**Examples:**

`data_signalling_rate_example.c`.

## 8.112 Types

Types used with the STAR-Dundee SpaceFibre Configuration API.

Collaboration diagram for Types:



### Typedefs

- typedef [REGISTER SPFI\\_DEVICE\\_INFO](#)  
*The type used to represent SpaceFibre device information.*
- typedef [REGISTER SPFI\\_DEVICE\\_IDENTIFIER\\_INFO](#)  
*The type used to represent SpaceFibre device identification information.*
- typedef [REGISTER SPFI\\_ROUTING\\_TABLE\\_ENTRY\\_FLAGS](#)  
*The type used to represent flags for a SpaceFibre routing table entry.*
- typedef [REGISTER SPFI\\_BROADCAST\\_TYPES\\_INFO](#)  
*The type used to represent broadcast types information.*
- typedef [REGISTER SPFI\\_VC\\_STATUS](#)  
*The type used to represent virtual channel status information.*
- typedef [REGISTER SPFI\\_VC\\_ERRORS](#)  
*The type used to represent virtual channel error information.*
- typedef [REGISTER SPFI\\_QUALITY\\_OF\\_SERVICE](#)  
*The type used to represent virtual channel quality of service information.*
- typedef [REGISTER SPFI\\_PORT\\_INFO](#)  
*The type used to represent SpaceFibre port information.*
- typedef [REGISTER SPFI\\_LINK\\_CONTROL\\_SETTINGS](#)  
*The type used to represent SpaceFibre link control settings.*
- typedef [REGISTER SPFI\\_LINK\\_STATUS](#)  
*The type used to represent SpaceFibre link status.*
- typedef [REGISTER SPFI\\_LINK\\_EVENTS](#)  
*The type used to represent SpaceFibre link events.*
- typedef [REGISTER SPFI\\_LINK\\_DEBUG\\_SETTINGS](#)  
*The type used to represent SpaceFibre link debug settings.*
- typedef [REGISTER SPFI\\_LANE\\_CONTROL\\_SETTINGS](#)  
*The type used to represent SpaceFibre lane control settings.*
- typedef [REGISTER SPFI\\_LANE\\_STATUS](#)  
*The type used to represent SpaceFibre lane status.*
- typedef [REGISTER SPFI\\_LANE\\_EVENTS](#)  
*The type used to represent SpaceFibre lane events.*
- typedef unsigned long long [U64](#)  
*A type that can be used to represent an unsigned 64-bit number.*

## Enumerations

- enum `SPFI_NODE_TYPE` { `SPFI_NODE_TYPE_INVALID` = 0, `SPFI_NODE_TYPE_CONFIG` = 1, `SPFI_NODE_TYPE_SOURCE_AND_DESTINATION` = 2, `SPFI_NODE_TYPE_OTHER` = 3 }

*SpaceFibre node type.*

- enum `SPFI_PORT_TYPE` { `SPFI_PORT_TYPE_UNDEFINED` = -1, `SPFI_PORT_TYPE_SPACEFIBRE` = 0, `SPFI_PORT_TYPE_AXI` = 1, `SPFI_PORT_TYPE_SPACEWIRE` = 2 }

*SpaceFibre port type.*

- enum `SPFI_LINK_ALIGNMENT_STATE` { `SPFI_LINK_ALIGNMENT_STATE_UNDEFINED` = -1, `SPFI_LINK_ALIGNMENT_STATE_NOT_READY` = 0, `SPFI_LINK_ALIGNMENT_STATE_NEAR_END_READY` = 1, `SPFI_LINK_ALIGNMENT_STATE_BOTH_ENDS_READY` = 2 }

*SpaceFibre link alignment state.*

- enum `SPFI_LANE_STATE` { `SPFI_LANE_STATE_UNDEFINED` = -1, `SPFI_LANE_STATE_COLD_RESET` = 0, `SPFI_LANE_STATE_CLEAR_LINE` = 1, `SPFI_LANE_STATE_DISABLED` = 2, `SPFI_LANE_STATE_WAIT` = 3, `SPFI_LANE_STATE_STARTED` = 4, `SPFI_LANE_STATE_INVERT_RX_POLARITY` = 5, `SPFI_LANE_STATE_CONNECTING` = 6, `SPFI_LANE_STATE_CONNECTED` = 7, `SPFI_LANE_STATE_ACTIVE` = 8, `SPFI_LANE_STATE_LOSS_OF_SIGNAL` = 9, `SPFI_LANE_STATE_PREPARE_STANDBY` = 10 }

*SpaceFibre lane state.*

### 8.112.1 Detailed Description

Types used with the STAR-Dundee SpaceFibre Configuration API.

### 8.112.2 Typedef Documentation

#### 8.112.2.1 typedef REGISTER SPFI\_BROADCAST\_TYPES\_INFO

The type used to represent broadcast types information.

Added in version:

STAR-System v6.00 Beta 3 - 5th October 2023

#### 8.112.2.2 typedef REGISTER SPFI\_DEVICE\_IDENTIFIER\_INFO

The type used to represent SpaceFibre device identification information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

#### 8.112.2.3 typedef REGISTER SPFI\_DEVICE\_INFO

The type used to represent SpaceFibre device information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.4 typedef REGISTER SPFI\_LANE\_CONTROL\_SETTINGS**

The type used to represent SpaceFibre lane control settings.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.5 typedef REGISTER SPFI\_LANE\_EVENTS**

The type used to represent SpaceFibre lane events.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.6 typedef REGISTER SPFI\_LANE\_STATUS**

The type used to represent SpaceFibre lane status.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.7 typedef REGISTER SPFI\_LINK\_CONTROL\_SETTINGS**

The type used to represent SpaceFibre link control settings.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.8 typedef REGISTER SPFI\_LINK\_DEBUG\_SETTINGS**

The type used to represent SpaceFibre link debug settings.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.9 typedef REGISTER SPFI\_LINK\_EVENTS**

The type used to represent SpaceFibre link events.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.10 typedef REGISTER SPFI\_LINK\_STATUS**

The type used to represent SpaceFibre link status.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.11 typedef REGISTER SPFI\_PORT\_INFO**

The type used to represent SpaceFibre port information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.12 typedef REGISTER SPFI\_QUALITY\_OF\_SERVICE**

The type used to represent virtual channel quality of service information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.13 typedef REGISTER SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS**

The type used to represent flags for a SpaceFibre routing table entry.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.14 typedef REGISTER SPFI\_VC\_ERRORS**

The type used to represent virtual channel error information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.15 typedef REGISTER SPFI\_VC\_STATUS**

The type used to represent virtual channel status information.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

**8.112.2.16 U64**

A type that can be used to represent an unsigned 64-bit number.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

### 8.112.3 Enumeration Type Documentation

#### 8.112.3.1 enum SPFI\_LANE\_STATE

SpaceFibre lane state.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Enumerator

**SPFI\_LANE\_STATE\_UNDEFINED** Undefined lane state.  
**SPFI\_LANE\_STATE\_COLD\_RESET** Cold Reset.  
**SPFI\_LANE\_STATE\_CLEAR\_LINE** Clear Line.  
**SPFI\_LANE\_STATE\_DISABLED** Disabled.  
**SPFI\_LANE\_STATE\_WAIT** Wait.  
**SPFI\_LANE\_STATE\_STARTED** Started.  
**SPFI\_LANE\_STATE\_INVERT\_RX\_POLARITY** Invert Rx Polarity.  
**SPFI\_LANE\_STATE\_CONNECTING** Connecting.  
**SPFI\_LANE\_STATE\_CONNECTED** Connected.  
**SPFI\_LANE\_STATE\_ACTIVE** Active.  
**SPFI\_LANE\_STATE\_LOSS\_OF\_SIGNAL** Loss of Signal.  
**SPFI\_LANE\_STATE\_PREPARE\_STANDBY** Prepare Standby.

#### 8.112.3.2 enum SPFI\_LINK\_ALIGNMENT\_STATE

SpaceFibre link alignment state.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Enumerator

**SPFI\_LINK\_ALIGNMENT\_STATE\_UNDEFINED** Undefined link alignment state.  
**SPFI\_LINK\_ALIGNMENT\_STATE\_NOT\_READY** Not ready link alignment state.  
**SPFI\_LINK\_ALIGNMENT\_STATE\_NEAR\_END\_READY** Near-end ready alignment link state.  
**SPFI\_LINK\_ALIGNMENT\_STATE\_BOTH\_ENDS\_READY** Both ends ready alignment link state.

#### 8.112.3.3 enum SPFI\_NODE\_TYPE

SpaceFibre node type.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Enumerator

**SPFI\_NODE\_TYPE\_INVALID** Invalid node type.  
**SPFI\_NODE\_TYPE\_CONFIG** Configuration node of a routing switch.  
**SPFI\_NODE\_TYPE\_SOURCE\_AND\_DESTINATION** Source and destination node (with at least VC0 and VC1)  
**SPFI\_NODE\_TYPE\_OTHER** Other configuration node (e.g. Configuration node of another device without VC0)

#### 8.112.3.4 enum SPFI\_PORT\_TYPE

SpaceFibre port type.

Added in version:

STAR-System v6.00 Beta 1 - 6th July 2023

Enumerator

***SPFI\_PORT\_TYPE\_UNDEFINED*** Undefined port type.

***SPFI\_PORT\_TYPE\_SPACEFIBRE*** SpaceFibre port.

***SPFI\_PORT\_TYPE\_AXI*** AXI port.

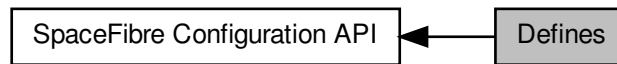
***SPFI\_PORT\_TYPE\_SPACEWIRE*** SpaceWire port.



## 8.113 Defines

Function return error values specific to the SpaceFibre Configuration API are defined here in addition to the [general error codes](#) that may also be returned.

Collaboration diagram for Defines:



### Macros

- `#define CFG_SPFI_STATUS_PORT_INVALID (-1000)`  
*The port value is invalid.*
- `#define CFG_SPFI_STATUS_UNEXPECTED_PORT_TYPE (-1001)`  
*The port type is unexpected.*
- `#define CFG_SPFI_STATUS_VIRTUAL_CHANNEL_INVALID (-1002)`  
*The virtual channel value is invalid.*
- `#define CFG_SPFI_STATUS_LANE_INVALID (-1003)`  
*The lane value is invalid.*
- `#define CFG_SPFI_STATUS_BROADCAST_TIMEOUT_INVALID (-1004)`  
*The broadcast timeout value is invalid.*
- `#define CFG_SPFI_STATUS_VIRTUAL_NETWORK_NUMBER_INVALID (-1005)`  
*The virtual network number value is invalid.*
- `#define CFG_SPFI_STATUS_BANDWIDTH_RESERVATION_INVALID (-1006)`  
*The bandwidth reservation value is invalid.*
- `#define CFG_SPFI_STATUS_PRIORITY_INVALID (-1007)`  
*The priority value is invalid.*
- `#define CFG_SPFI_ROUTER_TIMEOUT_INVALID (-1008)`  
*The router timeout value is invalid.*
- `#define CFG_SPFI_AXI_BC_INVALID (-1009)`  
*The AXI BC value is invalid.*

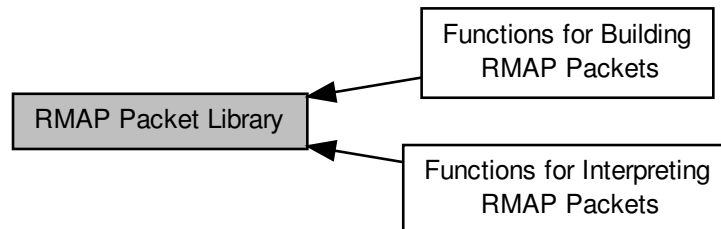
#### 8.113.1 Detailed Description

Function return error values specific to the SpaceFibre Configuration API are defined here in addition to the [general error codes](#) that may also be returned.

## 8.114 RMAP Packet Library

The SpaceWire Remote Memory Access Protocol (RMAP) is a protocol designed to be used over SpaceWire for a number of purposes, including the configuration of devices and networks.

Collaboration diagram for RMAP Packet Library:



### Modules

- [Functions for Building RMAP Packets](#)  
*These functions can be used to build the various types of RMAP packets.*
- [Functions for Interpreting RMAP Packets](#)  
*These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.*

### Data Structures

- [struct RMAP\\_PACKET](#)  
*A structure used to describe an RMAP packet. [More...](#)*

### Macros

- [#define RMAPPACKETLIBRARY\\_CC](#)  
*The calling convention used by functions exported by the RMAP Packet Library.*
- [#define RMAP\\_PROTOCOL\\_IDENTIFIER 1](#)  
*The value used in the protocol identifier field of all RMAP packets.*
- [#define PNP\\_PROTOCOL\\_IDENTIFIER 3](#)  
*The value used in the protocol identifier field of all SpaceWire-PnP packets.*
- [#define RMAP\\_RESERVED\\_BIT 0x80](#)  
*A mask for the reserved bit of the Instruction field.*
- [#define RMAP\\_COMMAND\\_BIT 0x40](#)  
*A packet is a command if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_WRITE\\_OPERATION\\_BIT 0x20](#)  
*A packet is a write operation if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_VERIFY\\_BEFORE\\_WRITE\\_BIT 0x10](#)  
*An operation verifies before writing if the bit specified by this mask is set in the Instruction field.*
- [#define RMAP\\_REPLY\\_BIT 0x08](#)  
*An operation is acknowledged if the bit specified by this mask is set in the Instruction field.*

- `#define RMAP_INCREMENT_ADDRESS_BIT 0x04`  
*A read or write address is incremented if the bit specified by this mask is set in the Instruction field.*
- `#define RMAP_REPLY_ADDRESS_LENGTH_BITS 0x03`  
*A mask for the bits in the Instruction field containing the length of the reply address.*
- `#define RMAP_GET_VERSION_MAJOR(versionInfo) (((versionInfo) & 0xff000000) >> 24)`  
*Return the major version number from the specified version information.*
- `#define RMAP_GET_VERSION_MINOR(versionInfo) (((versionInfo) & 0x00ff0000) >> 16)`  
*Return the minor version number from the specified version information.*
- `#define RMAP_GET_VERSION_EDIT(versionInfo) (((versionInfo) & 0x0000ffc0) >> 6)`  
*Return the edit number from the specified version information.*
- `#define RMAP_GET_VERSION_PATCH(versionInfo) ((versionInfo) & 0x0000003f)`  
*Return the patch level from the specified version information.*

## Enumerations

- `enum RMAP_STATUS {`  
`RMAP_SUCCESS = 0x00, RMAP_GENERAL_ERROR = 0x01, RMAP_UNUSED_PACKET_TYPE_OR_↵`  
`COMMAND_CODE = 0x02, RMAP_INVALID_KEY = 0x03,`  
`RMAP_INVALID_DATA_CRC = 0x04, RMAP_EARLY_EOP = 0x05, RMAP_TOO_MUCH_DATA = 0x06,`  
`RMAP_EEP = 0x07,`  
`RMAP_VERIFY_BUFFER_OVERRUN = 0x09, RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHO↵`  
`RISED = 0x0a, RMAP_RMW_DATA_LENGTH_ERROR = 0x0b, RMAP_INVALID_TARGET_LOGICAL_A↵`  
`DDRESS = 0x0c,`  
`RMAP_INVALID_STATUS = 0xff }`  
*Possible values for the status of RMAP operations.*
- `enum RMAP_PACKET_TYPE {`  
`RMAP_WRITE_COMMAND, RMAP_WRITE_REPLY = (RMAP_WRITE_OPERATION_BIT), RMAP_REA↵`  
`D_COMMAND = (RMAP_COMMAND_BIT), RMAP_READ_REPLY = (0),`  
`RMAP_READ_MODIFY_WRITE_COMMAND, RMAP_READ_MODIFY_WRITE_REPLY = (RMAP_VERIF↵`  
`Y_BEFORE_WRITE_BIT), RMAP_INVALID_PACKET_TYPE = (0xff) }`  
*Possible values for the packet type.*

## Functions

- `U32 RMAPPACKETLIBRARY_CC RMAP_GetVersion (void)`  
*Return the current version information for the RMAP Packet Library.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRC (void *pBuffer, unsigned long len)`  
*Calculate an 8-bit CRC for the given buffer.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRCWithSeed (void *pBuffer, unsigned long len, U8 crc)`  
*Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.*
- `char RMAPPACKETLIBRARY_CC RMAP_IsCRCValid (void *pBuffer, unsigned long len, U8 crc)`  
*Determine if the specified 8-bit CRC is valid for the given buffer.*
- `void RMAPPACKETLIBRARY_CC RMAP_FreeBuffer (void *pBuffer)`  
*Free a previously allocated buffer containing a packet created using one of the RMAP\_Build\* or RMAP\_Fill\* functions.*

### 8.114.1 Detailed Description

The SpaceWire Remote Memory Access Protocol (RMAP) is a protocol designed to be used over SpaceWire for a number of purposes, including the configuration of devices and networks.

More information on SpaceWire and RMAP can be found at the ESA SpaceWire Webpage at <http://spacewire.esa.int>. The STAR-Dundee RMAP Packet Library provides an API which can be used to build RMAP packets, check the validity of RMAP packets, and to read the fields of an RMAP packet.

### 8.114.2 Functionality Provided

The STAR-Dundee RMAP Packet Library provides functions which can be called to create RMAP packets, to check the format of RMAP packets and to determine the values of fields in RMAP packets. It does not send or receive packets using a SpaceWire device; this work must be done by the programmer as this is application and device specific.

Functions are provided to build read, write and read/modify/write commands and replies. Each packet built using the functions which allocate a buffer for the packet must be freed using the [RMAP\\_FreeBuffer](#) function. Functions are also provided to fill previously allocated buffers with specific RMAP packets. Functions are also provided to determine the length of the potential packet so that the amount of memory to allocate is known.

The format of RMAP packets can be checked using the [RMAP\\_CheckPacketValid](#) function, which confirms that the fields have valid values, and that the length of the packet is correct. When building packets or checking a packet's format, the functions can create a structure which can be used in the other functions provided by the library. These functions can be used to determine the values of individual fields within the RMAP packet, such as the packet type and the data.

### 8.114.3 Supported Platforms

The RMAP Packet Library is currently available for Windows (2000, XP, Vista, Windows 7 and Windows 8), Linux (v2.6 and 3 kernels for x86-32 and x86-64), QNX, VxWorks and RTEMS platforms, along with a native ELF version for the LEON processors. As no access to SpaceWire hardware is made, the library can be used with any SpaceWire device.

If you require the library for a different operating system or processor, please contact STAR-Dundee using the contact information in the [Contact Information](#) section.

A Java version of the RMAP Packet Library (described in the [Java Library](#) section) is also available for Windows and Linux.

### 8.114.4 Using The Library

The RMAP Packet Library is provided in the form of two or three files:

- [rmap\\_packet\\_library.h](#)
- [rmap\\_packet\\_library.lib](#) (Windows), [librmap\\_packet\\_library.so](#) (Linux, QNX) or [librmap\\_packet\\_library.a](#) (VxWorks and LEON)
- [rmap\\_packet\\_library.dll](#) (Windows only)

When writing programs which call functions provided by the RMAP Packet Library, the [rmap\\_packet\\_library.h](#) file must be included. This file includes "star\_dundee\_types.h", which must also be provided. To compile programs to use the library "librmap\_packet\_library.so", or "rmap\_packet\_library.lib" on Windows, must be linked in. When running programs which use the library, on Windows "rmap\_packet\_library.dll" must either be in the "Windows\System32" directory of the machine in use, or in the directory from which the program was run. On Linux "librmap\_packet\_library.so" must be present in "/usr/lib/".

### 8.114.5 Java Library

The RMAP Packet Library is also provided with the SpaceWire USB and PCI Drivers for Windows and Linux Driver as a jar file, to allow Java code to be written to use the library. Further information on this can be found in the help files provided in the "spacewire\_java" directory under the driver's install directory (see "index.html"). To compile a Java program using the library, a classpath should be specified which includes "RMAPLibrary.jar". To run an application which uses the library, the jar file should also be in the classpath.

### 8.114.6 Byte Alignment

The functions in the library which can be used to build packets take an alignment argument. Normally this should be set to 1, but if the device being used to send the RMAP packet can only send packets which are a multiple of four bytes, for example, then this alignment parameter can be set to 4. This will result in the packet being built containing extra 0s in the packet's path address, prior to the logical address, to make the packet a multiple of four bytes in length. Other alignment values can also be used. Note that this feature is not part of the RMAP standard but is implemented in a number of SpaceWire devices, including the ESA SpaceWire Router. Use an alignment of 1 to maintain full compliance to the standard.

### 8.114.7 Contact Information

If you have any comments or queries regarding the RMAP Packet Library or documentation please contact:

STAR-Dundee

STAR House

166 Nethergate

Dundee, DD1 4EE

Scotland, UK

E-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Further support information and details of all STAR-Dundee products can be found at <http://www.star-dundee.com>.

### 8.114.8 Data Structure Documentation

#### 8.114.8.1 struct RMAP\_PACKET

A structure used to describe an RMAP packet.

This structure should not be accessed directly, but should be populated with values using one of the RMAP\_Build\* or RMAP\_Fill\* functions. The fields should then be accessed using the RMAP\_Get\* functions.

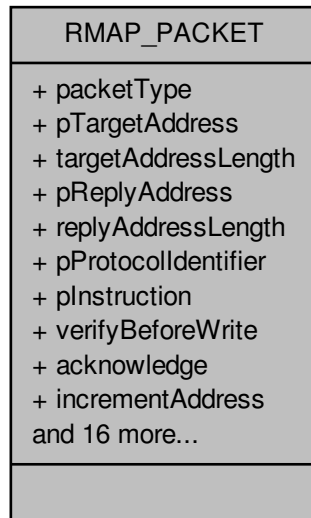
#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[check\\_packet\\_example.c](#).

Collaboration diagram for RMAP\_PACKET:



#### Data Fields

|                         |                                     |                                                                                                                                                                                                                                                                 |
|-------------------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char                    | acknowledge                         | Whether acknowledgement is enabled in the instruction byte in the packet.                                                                                                                                                                                       |
| U32                     | dataLength                          | The length of the data field in the packet, if present, otherwise 0. If the packet is read/modify/write command packet, then this value will be different from the data length field in the packet, which is the total length of both the data and mask fields. |
| unsigned long           | headerLength                        | The length of the header.                                                                                                                                                                                                                                       |
| char                    | increment↔<br>Address               | Whether incrementing of memory addresses is enabled in the instruction byte in the packet.                                                                                                                                                                      |
| U8                      | maskLength                          | The length of the mask field in the packet, if present, otherwise 0. This is half of the data length field in a read/modify write command packet.                                                                                                               |
| RMAP_PACKET↔<br>ET_TYPE | packetType                          | The type of the RMAP packet.                                                                                                                                                                                                                                    |
| U8 *                    | pData                               | A pointer to the data field in the packet, if present, otherwise NULL.                                                                                                                                                                                          |
| U8 *                    | pDataCRC                            | A pointer to the data CRC byte in the packet, if present, otherwise NULL.                                                                                                                                                                                       |
| void *                  | pDataLength                         | A pointer to the three byte data length field in the packet, if present, otherwise NULL.                                                                                                                                                                        |
| U8 *                    | pExtended↔<br>ReadWrite↔<br>Address | A pointer to the extended read or write memory address byte in the packet, if present, otherwise NULL.                                                                                                                                                          |
| U8 *                    | pHeader                             | A pointer to the header in the packet, starting from the last byte in the address at the start of the packet.                                                                                                                                                   |
| U8 *                    | pHeaderCRC                          | A pointer to the header CRC byte in the packet, or NULL if the field could not be identified.                                                                                                                                                                   |

|               |                             |                                                                                                                     |
|---------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------|
| U8 *          | pInstruction                | A pointer to the instruction byte in the packet, or NULL if the field could not be identified.                      |
| U8 *          | pKey                        | A pointer to the key byte in the packet, if present, otherwise NULL.                                                |
| U8 *          | pMask                       | A pointer to the mask field in the packet, if present, otherwise NULL.                                              |
| U8 *          | pProtocol↔<br>Identifier    | A pointer to the protocol identifier byte in the packet, or NULL if the field could not be identified.              |
| U8 *          | pRawPacket                  | A pointer to the raw packet.                                                                                        |
| U32 *         | pReadWrite↔<br>Address      | A pointer to the four byte read or write memory address field in the packet, if present, otherwise NULL.            |
| U8 *          | pReplyAddress               | A pointer to the reply address in the packet, or NULL if the reply address could not be identified.                 |
| U8 *          | pStatus                     | A pointer to the status byte in the packet, if present, otherwise NULL.                                             |
| U8 *          | pTargetAddress              | A pointer to the target address in the packet, or NULL if the target address could not be identified.               |
| U16 *         | pTransaction↔<br>Identifier | A pointer to the two byte transaction identifier field in the packet, or NULL if the field could not be identified. |
| unsigned long | rawPacket↔<br>Length        | The length of the raw packet.                                                                                       |
| unsigned long | replyAddress↔<br>Length     | The length of the reply address in the packet.                                                                      |
| unsigned long | targetAddress↔<br>Length    | The length of the target address in the packet.                                                                     |
| char          | verifyBefore↔<br>Write      | Whether verify before write is enabled in the instruction byte in the packet.                                       |

### 8.114.9 Macro Definition Documentation

#### 8.114.9.1 #define RMAP\_COMMAND\_BIT 0x40

A packet is a command if the bit specified by this mask is set in the Instruction field.

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 8.114.9.2 #define RMAP\_GET\_VERSION\_EDIT( *versionInfo* ) (((versionInfo) & 0x0000ffc0) >> 6)

Return the edit number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the edit number in bits 6 to 15.

##### Parameters

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

##### Returns

the edit number

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

[version\\_example.c](#).

#### 8.114.9.3 `#define RMAP_GET_VERSION_MAJOR( versionInfo ) (((versionInfo) & 0xff000000) >> 24)`

Return the major version number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the major version number in the most significant 8 bits, bits 24 to 31.

##### Parameters

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

##### Returns

the major version number

##### Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

##### Examples:

[version\\_example.c](#).

#### 8.114.9.4 `#define RMAP_GET_VERSION_MINOR( versionInfo ) (((versionInfo) & 0x00ff0000) >> 16)`

Return the minor version number from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the minor version number in bits 16 to 23.

##### Parameters

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---

##### Returns

the minor version number

##### Added in version:

[STAR-System v0.8 \(Beta 1\)](#) - 22nd August 2011

##### Examples:

[version\\_example.c](#).

#### 8.114.9.5 `#define RMAP_GET_VERSION_PATCH( versionInfo ) ((versionInfo) & 0x0000003f)`

Return the patch level from the specified version information.

The version information can be obtained from a call to [RMAP\\_GetVersion](#), which returns a value with the patch level in bits 0 to 5. The patch level should be 0 in a release version of the RMAP Packet Library.

##### Parameters

---

|                    |                                                                               |
|--------------------|-------------------------------------------------------------------------------|
| <i>versionInfo</i> | the version information returned by a call to <a href="#">RMAP_GetVersion</a> |
|--------------------|-------------------------------------------------------------------------------|

---



**Returns**

the patch level

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`version_example.c`.

**8.114.9.6 #define RMAP\_INCREMENT\_ADDRESS\_BIT 0x04**

A read or write address is incremented if the bit specified by this mask is set in the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**8.114.9.7 #define RMAP\_PROTOCOL\_IDENTIFIER 1**

The value used in the protocol identifier field of all RMAP packets.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**8.114.9.8 #define RMAP\_REPLY\_ADDRESS\_LENGTH\_BITS 0x03**

A mask for the bits in the Instruction field containing the length of the reply address.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**8.114.9.9 #define RMAP\_REPLY\_BIT 0x08**

An operation is acknowledged if the bit specified by this mask is set in the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**8.114.9.10 #define RMAP\_RESERVED\_BIT 0x80**

A mask for the reserved bit of the Instruction field.

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### 8.114.9.11 `#define RMAP_VERIFY_BEFORE_WRITE_BIT 0x10`

An operation verifies before writing if the bit specified by this mask is set in the Instruction field.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

#### 8.114.9.12 `#define RMAP_WRITE_OPERATION_BIT 0x20`

A packet is a write operation if the bit specified by this mask is set in the Instruction field.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

#### 8.114.9.13 `#define RMAPPACKETLIBRARY_CC`

The calling convention used by functions exported by the RMAP Packet Library.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

### 8.114.10 Enumeration Type Documentation

#### 8.114.10.1 `enum RMAP_PACKET_TYPE`

Possible values for the packet type.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Enumerator

***RMAP\_WRITE\_COMMAND*** The write command packet type.

***RMAP\_WRITE\_REPLY*** The write reply packet type.

***RMAP\_READ\_COMMAND*** The read command packet type.

***RMAP\_READ\_REPLY*** The read reply packet type.

***RMAP\_READ\_MODIFY\_WRITE\_COMMAND*** The read/modify/write command packet type.

***RMAP\_READ\_MODIFY\_WRITE\_REPLY*** The read/modify/write reply packet type.

***RMAP\_INVALID\_PACKET\_TYPE*** The packet type used when a valid packet type cannot be determined.

#### 8.114.10.2 `enum RMAP_STATUS`

Possible values for the status of RMAP operations.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

Enumerator

***RMAP\_SUCCESS*** The status value indicating success.

***RMAP\_GENERAL\_ERROR*** The status value indicating that the detected error does not fit into the other error cases.

***RMAP\_UNUSED\_PACKET\_TYPE\_OR\_COMMAND\_CODE*** The status value indicating the packet type is reserved or the command is not used by the RMAP protocol.

***RMAP\_INVALID\_KEY*** The status value indicating the key did not match that expected by the target user application.

***RMAP\_INVALID\_DATA\_CRC*** The status value indicating there was an error in the data CRC.

***RMAP\_EARLY\_EOP*** The status value indicating an EOP was detected before the end of the data.

***RMAP\_TOO\_MUCH\_DATA*** The status value indicating there was more data than was expected.

***RMAP\_EEP*** The status value indicating that an EEP was encountered in the packet after the header.

***RMAP\_VERIFY\_BUFFER\_OVERRUN*** The status value indicating that verify before write was enabled in the command but not enough buffer space was available to receive the full command.

***RMAP\_COMMAND\_NOT\_IMPLEMENTED\_OR\_AUTHORISED*** The status value indicating the target user application did not authorise the requested operation.

***RMAP\_RMW\_DATA\_LENGTH\_ERROR*** The status value indicating the amount of data in a read/modify/write command is invalid.

***RMAP\_INVALID\_TARGET\_LOGICAL\_ADDRESS*** The status value indicating the target logical address was not the value expected by the target.

***RMAP\_INVALID\_STATUS*** The status value indicating that an invalid status value was encountered, or the status could not be determined. Note that this is not a standard RMAP error.

## 8.114.11 Function Documentation

### 8.114.11.1 U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRC ( void \* *pBuffer*, unsigned long *len* )

Calculate an 8-bit CRC for the given buffer.

#### Parameters

|                |                                     |
|----------------|-------------------------------------|
| <i>pBuffer</i> | the buffer to calculate the CRC for |
| <i>len</i>     | the length of the buffer            |

#### Returns

the 8-bit CRC for the specified buffer

Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

Examples:

`crc_example.c`.

### 8.114.11.2 U8 RMAPPACKETLIBRARY\_CC RMAP\_CalculateCRCWithSeed ( void \* *pBuffer*, unsigned long *len*, U8 *crc* )

Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.

#### Parameters

---

|                |                                                  |
|----------------|--------------------------------------------------|
| <i>pBuffer</i> | the buffer to calculate the CRC for              |
| <i>len</i>     | the length of the buffer                         |
| <i>crc</i>     | the seed CRC to be used when calculating the CRC |

---

**Returns**

the 8-bit CRC for the specified buffer

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[crc\\_example.c](#).

**8.114.11.3 void RMAPPACKETLIBRARY\_CC RMAP\_FreeBuffer ( void \* pBuffer )**

Free a previously allocated buffer containing a packet created using one of the RMAP\_Build\* or RMAP\_Fill\* functions.

**Parameters**


---

|                |                        |
|----------------|------------------------|
| <i>pBuffer</i> | the buffer to be freed |
|----------------|------------------------|

---

**Returns**

void

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#), [read\\_command\\_packet\\_example.c](#), [read\\_reply\\_packet\\_example.c](#), [rmap\\_target\\_example.c](#), [rmw\\_command\\_packet\\_example.c](#), [rmw\\_reply\\_packet\\_example.c](#), [write\\_command\\_packet\\_example.c](#), and [write\\_reply\\_packet\\_example.c](#).

**8.114.11.4 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetVersion ( void )**

Return the current version information for the RMAP Packet Library.

This can be used with the [RMAP\\_GET\\_VERSION\\_MAJOR](#), [RMAP\\_GET\\_VERSION\\_MINOR](#), [RMAP\\_GET\\_VERSION\\_EDIT](#) and [RMAP\\_GET\\_VERSION\\_PATCH](#) macros to obtain the version number in a useable form.

**Returns**

the version information for the RMAP Packet Library stored in a 32-bit number. The most significant 8 bits will contain the major version number, the next significant 8 bits the minor version number, the next significant 10 bits the edit number, and the 6 least significant bits the patch level. In a release version of the library, the patch level should be 0

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[version\\_example.c](#).

8.114.11.5 char RMAPPACKETLIBRARY\_CC RMAP\_IsCRCValid ( void \* *pBuffer*, unsigned long *len*, U8 *crc* )

Determine if the specified 8-bit CRC is valid for the given buffer.

**Parameters**

|                |                                 |
|----------------|---------------------------------|
| <i>pBuffer</i> | the buffer to check the CRC for |
| <i>len</i>     | the length of the buffer        |
| <i>crc</i>     | the CRC to check against        |

---

**Returns**

1 if the CRC is correct for the buffer, otherwise 0

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

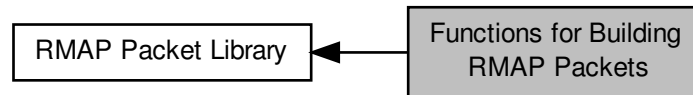
**Examples:**

`crc_example.c`.

## 8.115 Functions for Building RMAP Packets

These functions can be used to build the various types of RMAP packets.

Collaboration diagram for Functions for Building RMAP Packets:



### Functions

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a write command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteReplyPacketLength](#) (unsigned long initiatorAddressLength, char alignment)

*Calculate the number of bytes required for a write reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)

*Calculate the number of bytes required for a read command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadReplyPacketLength](#) (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadModifyWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)

*Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadModifyWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadModifyWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadModifyWriteRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)



*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_CalculateReadModifyWriteReplyPacketLength](#) (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_FillReadModifyWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void \*[RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_BuildReadModifyWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_SetProtocolIdentifier](#) ([RMAP\\_PACKET](#) \*pPacketStruct, U8 protocolIdentifier)

*Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.*

### 8.115.1 Detailed Description

These functions can be used to build the various types of RMAP packets.

These functions are useful when transmitting RMAP packets.

### 8.115.2 Function Documentation

- 8.115.2.1 void\* [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_BuildReadCommandPacket](#) ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *readAddress*, U8 *extendedReadAddress*, U32 *dataLength*, unsigned long \* *pRawPacketLength*, [RMAP\\_PACKET](#) \* *pPacketStruct*, char *alignment* )

Build an RMAP read command packet which can be sent to perform an RMAP read command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                              |                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>        | a pointer to the SpaceWire path or logical address of the device to be read from                                                                       |
| <i>targetAddressLength</i>   | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>         | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddressLength</i>    | the length of the reply address                                                                                                                        |
| <i>incrementAddress</i>      | whether or not the target read address should be incremented when reading                                                                              |
| <i>key</i>                   | the key expected by the destination device                                                                                                             |
| <i>transactionIdentifier</i> | an identifier for the transaction                                                                                                                      |

|                            |                                                                                                                                                                                                                                                                                                   |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>readAddress</i>         | the memory address at the destination to read from                                                                                                                                                                                                                                                |
| <i>extendedReadAddress</i> | the extended memory address at the destination to read from                                                                                                                                                                                                                                       |
| <i>dataLength</i>          | the length of data to read, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                             |
| <i>pRawPacketLength</i>    | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                   |
| <i>pPacketStruct</i>       | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the API. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>           | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                          |

### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### Examples:

[read\\_command\\_packet\\_example.c](#).

```
8.115.2.2 void* RMAPPACKETLIBRARY_CC RMAP_BuildReadModifyWriteCommandPacket (U8 * pTargetAddress,
unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, U8 key,
U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8
dataAndMaskLength, U8 * pData, U8 * pMask, unsigned long * pRawPacketLength, RMAP_PACKET *
pPacketStruct, char alignment)
```

Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

### Parameters

|                               |                                                                                                                                                        |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>         | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddressLength</i>    | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>          | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddressLength</i>     | the length of the reply address                                                                                                                        |
| <i>key</i>                    | the key expected by the destination device                                                                                                             |
| <i>transactionIdentifier</i>  | an identifier for the transaction                                                                                                                      |
| <i>readModifyWriteAddress</i> | the memory address at the destination to read from and write to                                                                                        |

|                                                                 |                                                                                                                                                                                                                                                                                                    |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>extendedRead</i> ↔<br><i>ModifyWrite</i> ↔<br><i>Address</i> | the extended memory address at the destination to read from and write to                                                                                                                                                                                                                           |
| <i>dataAndMask</i> ↔<br><i>Length</i>                           | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                                                                                                                                                                                            |
| <i>pData</i>                                                    | the data to be written                                                                                                                                                                                                                                                                             |
| <i>pMask</i>                                                    | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacket</i> ↔<br><i>Length</i>                            | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                                            | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                                                | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_command\\_packet\\_example.c](#).

**8.115.2.3** `void* RMAPPACKETLIBRARY_CC RMAP_BuildReadModifyWriteRegisterPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment )`

Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

**Parameters**

|                                           |                                                                                                                                                        |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                     | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddress</i> ↔<br><i>Length</i>   | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>                      | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                        |
| <i>key</i>                                | the key expected by the destination device                                                                                                             |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                      |

|                                                   |                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>readModify↔<br/>WriteAddress</i>               | the memory address at the destination to read from and write to                                                                                                                                                                                                                                    |
| <i>extendedRead↔<br/>ModifyWrite↔<br/>Address</i> | the extended memory address at the destination to read from and write to                                                                                                                                                                                                                           |
| <i>registerValue</i>                              | the value to be written to the register                                                                                                                                                                                                                                                            |
| <i>mask</i>                                       | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacket↔<br/>Length</i>                     | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                              | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                                  | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### Examples:

[rmw\\_command\\_packet\\_example.c](#).

**8.115.2.4** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadModifyWriteReplyPacket ( U8 \* *pInitiatorAddress*, unsigned long *initiatorAddressLength*, U8 *targetAddress*, RMAP\_STATUS *status*, U16 *transactionIdentifier*, unsigned long *dataLength*, U8 \* *pData*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadModifyWriteReplyPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

### Parameters

|                                     |                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                |
| <i>targetAddress</i>                | the SpaceWire logical address of the device that sent the command being responded to               |
| <i>status</i>                       | the status of the read operation                                                                   |
| <i>transaction↔<br/>Identifier</i>  | an identifier for the transaction                                                                  |
| <i>dataLength</i>                   | the length in bytes of the data field, this can be at most 4 bytes                                 |
| <i>pData</i>                        | the data read                                                                                      |

|                                      |                                                                                                                                                                                                                                                                                                    |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pRawPacket</i> ↔<br><i>Length</i> | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                 | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                     | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[rmw\\_reply\\_packet\\_example.c](#).

**8.115.2.5** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadRegisterPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *readAddress*, U8 *extendedReadAddress*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.

Note that this function performs a memory allocation, use [RMAP\\_FillReadCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

**Parameters**

|                                           |                                                                                                                                                        |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                     | a pointer to the SpaceWire path or logical address of the device to be read from                                                                       |
| <i>targetAddress</i> ↔<br><i>Length</i>   | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>                      | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                        |
| <i>increment</i> ↔<br><i>Address</i>      | whether or not the target read address should be incremented when reading                                                                              |
| <i>key</i>                                | the key expected by the destination device                                                                                                             |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                      |
| <i>readAddress</i>                        | the memory address at the destination to read from                                                                                                     |
| <i>extendedRead</i> ↔<br><i>Address</i>   | the extended memory address at the destination to read from                                                                                            |
| <i>pRawPacket</i> ↔<br><i>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                        |

|                      |                                                                                                                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>     | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[read\\_command\\_packet\\_example.c](#).

**8.115.2.6** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildReadReplyPacket ( U8 \* *pInitiatorAddress*, unsigned long *initiatorAddressLength*, U8 *targetAddress*, char *incrementAddress*, RMAP\_STATUS *status*, U16 *transactionIdentifier*, U8 \* *pData*, U32 *dataLength*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP read reply packet which can be sent to respond to an RMAP read command.

Note that this function performs a memory allocation, use [RMAP\\_FillReadReplyPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                                |                                                                                                                                                                                                                                                                                                    |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>       | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>           | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>increment↔Address</i>       | whether or not the command indicated that the target read address should be incremented when reading                                                                                                                                                                                               |
| <i>status</i>                  | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transaction↔Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pData</i>                   | the data read                                                                                                                                                                                                                                                                                      |
| <i>dataLength</i>              | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket↔Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>           | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>alignment</i> | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |
|------------------|--------------------------------------------------------------------------------------------------------------------------|

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_reply\\_packet\\_example.c](#).

**8.115.2.7** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildWriteCommandPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *verifyBeforeWrite*, char *acknowledge*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *writeAddress*, U8 *extendedWriteAddress*, U8 \* *pData*, U32 *dataLength*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP write command packet which can be sent to perform an RMAP write command.

Note that this function performs a memory allocation, use [RMAP\\_FillWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

**Parameters**

|                              |                                                                                                                                                        |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>        | a pointer to the SpaceWire path or logical address of the device to be written to                                                                      |
| <i>targetAddressLength</i>   | the length of the target address                                                                                                                       |
| <i>pReplyAddress</i>         | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from |
| <i>replyAddressLength</i>    | the length of the reply address                                                                                                                        |
| <i>verifyBeforeWrite</i>     | whether or not the data should be verified before writing                                                                                              |
| <i>acknowledge</i>           | whether or not the command should be acknowledged                                                                                                      |
| <i>incrementAddress</i>      | whether or not the target write address should be incremented when writing                                                                             |
| <i>key</i>                   | the key expected by the destination device                                                                                                             |
| <i>transactionIdentifier</i> | an identifier for the transaction                                                                                                                      |
| <i>writeAddress</i>          | the memory address at the destination to write to                                                                                                      |
| <i>extendedWriteAddress</i>  | the extended memory address at the destination to write to                                                                                             |
| <i>pData</i>                 | the data to be written                                                                                                                                 |
| <i>dataLength</i>            | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                          |
| <i>pRawPacketLength</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                        |

|                      |                                                                                                                                                                                                                                                                                                    |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>     | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

#### Returns

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmap\\_target\\_example.c](#), and [write\\_command\\_packet\\_example.c](#).

**8.115.2.8** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildWriteRegisterPacket ( U8 \* *pTargetAddress*, unsigned long *targetAddressLength*, U8 \* *pReplyAddress*, unsigned long *replyAddressLength*, char *verifyBeforeWrite*, char *acknowledge*, char *incrementAddress*, U8 *key*, U16 *transactionIdentifier*, U32 *writeAddress*, U8 *extendedWriteAddress*, U32 *registerValue*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.

Note that this function performs a memory allocation, use [RMAP\\_FillWriteCommandPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

#### Parameters

|                               |                                                                                                                                                                   |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>         | a pointer to the SpaceWire path or logical address of the device to be written to                                                                                 |
| <i>targetAddress↔Length</i>   | the length of the target address                                                                                                                                  |
| <i>pReplyAddress</i>          | a pointer to the SpaceWire path or logical address that from will be used the device to send any responses back to the device which the command will be sent from |
| <i>replyAddress↔Length</i>    | the length of the reply address                                                                                                                                   |
| <i>verifyBefore↔Write</i>     | whether or not the data should be verified before writing                                                                                                         |
| <i>acknowledge</i>            | whether or not the command should be acknowledged                                                                                                                 |
| <i>increment↔Address</i>      | whether or not the target write address should be incremented when writing                                                                                        |
| <i>key</i>                    | the key expected by the destination device                                                                                                                        |
| <i>transaction↔Identifier</i> | an identifier for the transaction                                                                                                                                 |
| <i>writeAddress</i>           | the memory address at the destination to write to                                                                                                                 |
| <i>extendedWrite↔Address</i>  | the extended memory address at the destination to write to                                                                                                        |



|                                      |                                                                                                                                                                                                                                                                                                    |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>registerValue</i>                 | the value to be written to the register                                                                                                                                                                                                                                                            |
| <i>pRawPacket</i> ↔<br><i>Length</i> | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                 | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                     | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#), and [write\\_command\\_packet\\_example.c](#).

**8.115.2.9** void\* **RMAPPACKETLIBRARY\_CC** RMAP\_BuildWriteReplyPacket ( U8 \* *pInitiatorAddress*, unsigned long *initiatorAddressLength*, U8 *targetAddress*, char *verifyBeforeWrite*, char *incrementAddress*, RMAP\_STATUS *status*, U16 *transactionIdentifier*, unsigned long \* *pRawPacketLength*, RMAP\_PACKET \* *pPacketStruct*, char *alignment* )

Build an RMAP write reply packet which can be sent to respond to an RMAP write command.

Note that this function performs a memory allocation, use [RMAP\\_FillWriteReplyPacket](#) to build a packet using previously allocated memory. When the buffer allocated by this function is no longer required, it should be freed by calling [RMAP\\_FreeBuffer](#).

**Parameters**

|                                            |                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>                   | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator</i> ↔<br><i>AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>                       | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>verifyBefore</i> ↔<br><i>Write</i>      | whether or not the command indicated that data should be verified before writing                                                                                                                                                                                                                   |
| <i>increment</i> ↔<br><i>Address</i>       | whether or not the command indicated that the target write address should be incremented when writing                                                                                                                                                                                              |
| <i>status</i>                              | the status of the write operation                                                                                                                                                                                                                                                                  |
| <i>transaction</i> ↔<br><i>Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pRawPacket</i> ↔<br><i>Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                       | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |

---

|                  |                                                                                                                          |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>alignment</i> | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |
|------------------|--------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the packet generated, or NULL if there was an error, for example if one of the fields is invalid. The buffer returned should be freed by calling [RMAP\\_FreeBuffer](#)

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_reply\\_packet\\_example.c](#).

**8.115.2.10** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateReadCommandPacketLength ( unsigned long *targetAddressLength*, unsigned long *replyAddressLength*, char *alignment* )

Calculate the number of bytes required for a read command packet, given the properties of the packet.

**Parameters**


---

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>targetAddress↔<br/>Length</i> | the length in bytes of the target address in the packet                                                                  |
| <i>replyAddress↔<br/>Length</i>  | the length in bytes of the reply address in the packet                                                                   |
| <i>alignment</i>                 | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

---

**Returns**

the calculated length of the read command packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_command\\_packet\\_example.c](#).

**8.115.2.11** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateReadModifyWriteCommandPacketLength ( unsigned long *targetAddressLength*, unsigned long *replyAddressLength*, U32 *dataAndMaskLength*, char *alignment* )

Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.

**Parameters**


---

|                                  |                                                         |
|----------------------------------|---------------------------------------------------------|
| <i>targetAddress↔<br/>Length</i> | the length in bytes of the target address in the packet |
| <i>replyAddress↔<br/>Length</i>  | the length in bytes of the reply address in the packet  |

---

|                                |                                                                                                                          |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>dataAndMask↔<br/>Length</i> | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                  |
| <i>alignment</i>               | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the write command packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`rmw_command_packet_example.c`.

**8.115.2.12** `unsigned long RMAPPACKETLIBRARY_CC RMAP_CalculateReadModifyWriteReplyPacketLength ( unsigned long initiatorAddressLength, U32 dataLength, char alignment )`

Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.

**Parameters**

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet                                                               |
| <i>dataLength</i>                   | the length in bytes of the data field, this can be at most 4 bytes                                                       |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the read/modify/write reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`rmw_reply_packet_example.c`.

**8.115.2.13** `unsigned long RMAPPACKETLIBRARY_CC RMAP_CalculateReadReplyPacketLength ( unsigned long initiatorAddressLength, U32 dataLength, char alignment )`

Calculate the number of bytes required for a read reply packet, given the properties of the packet.

**Parameters**

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet                                                               |
| <i>dataLength</i>                   | the length in bytes of the data field, this can be at most 0xfffff bytes                                                 |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the read reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_reply\\_packet\\_example.c](#).

**8.115.2.14** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateWriteCommandPacketLength ( unsigned long *targetAddressLength*, unsigned long *replyAddressLength*, U32 *dataLength*, char *alignment* )

Calculate the number of bytes required for a write command packet, given the properties of the packet.

**Parameters**

|                                  |                                                                                                                          |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>targetAddress↔<br/>Length</i> | the length in bytes of the target address in the packet                                                                  |
| <i>replyAddress↔<br/>Length</i>  | the length in bytes of the reply address in the packet                                                                   |
| <i>dataLength</i>                | the length in bytes of the data field, this can be at most 0xfffff bytes                                                 |
| <i>alignment</i>                 | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the write command packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_command\\_packet\\_example.c](#).

**8.115.2.15** unsigned long **RMAPPACKETLIBRARY\_CC** RMAP\_CalculateWriteReplyPacketLength ( unsigned long *initiatorAddressLength*, char *alignment* )

Calculate the number of bytes required for a write reply packet, given the properties of the packet.

**Parameters**

|                                     |                                                                                                                          |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <i>initiator↔<br/>AddressLength</i> | the length in bytes of the initiator address in the packet                                                               |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information |

**Returns**

the calculated length of the write reply packet in bytes, or 0 if there was an error

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_reply\\_packet\\_example.c](#).

**8.115.2.16** `char RMAPPACKETLIBRARY_CC RMAP_FillReadCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling `RMAP_CalculateReadCommandPacketLength`. This function performs no memory allocation. See `RMAP_BuildReadCommandPacket` for a function which builds a read command packet, performing the memory allocation.

#### Parameters

|                              |                                                                                                                                                                                                                                                                                                   |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>        | a pointer to the SpaceWire path or logical address of the device to be read from                                                                                                                                                                                                                  |
| <i>targetAddressLength</i>   | the length of the target address                                                                                                                                                                                                                                                                  |
| <i>pReplyAddress</i>         | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                            |
| <i>replyAddressLength</i>    | the length of the reply address                                                                                                                                                                                                                                                                   |
| <i>incrementAddress</i>      | whether or not the target read address should be incremented when reading                                                                                                                                                                                                                         |
| <i>key</i>                   | the key expected by the destination device                                                                                                                                                                                                                                                        |
| <i>transactionIdentifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                 |
| <i>readAddress</i>           | the memory address at the destination to read from                                                                                                                                                                                                                                                |
| <i>extendedReadAddress</i>   | the extended memory address at the destination to read from                                                                                                                                                                                                                                       |
| <i>dataLength</i>            | the length of data to read, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                             |
| <i>pRawPacketLength</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                   |
| <i>pPacketStruct</i>         | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the API. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>             | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                          |
| <i>pRawPacket</i>            | a pointer to a previously allocated buffer which will be updated to contain the read command packet                                                                                                                                                                                               |
| <i>rawPacketLength</i>       | the length of the buffer provided for the packet                                                                                                                                                                                                                                                  |

#### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

`read_command_packet_example.c`.

**8.115.2.17** `char RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 * pData, U8 * pMask, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadModifyWriteCommandPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildReadModifyWriteCommandPacket](#) for a function which builds a read/modify/write command packet, performing the memory allocation.

#### Parameters

|                                                   |                                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pTargetAddress</i>                             | a pointer to the SpaceWire path or logical address of the device to be written to                                                                                                                                                                                                                  |
| <i>targetAddress↔<br/>Length</i>                  | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>                              | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress↔<br/>Length</i>                   | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>key</i>                                        | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction↔<br/>Identifier</i>                | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>readModify↔<br/>WriteAddress</i>               | the memory address at the destination to read from and write to                                                                                                                                                                                                                                    |
| <i>extendedRead↔<br/>ModifyWrite↔<br/>Address</i> | the extended memory address at the destination to read from and write to                                                                                                                                                                                                                           |
| <i>dataAndMask↔<br/>Length</i>                    | the combined length in bytes of the data and mask fields, this should be an even number between 0 and 8                                                                                                                                                                                            |
| <i>pData</i>                                      | the data to be written                                                                                                                                                                                                                                                                             |
| <i>pMask</i>                                      | the mask to be applied to the data                                                                                                                                                                                                                                                                 |
| <i>pRawPacket↔<br/>Length</i>                     | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                              | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                                  | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                                 | a pointer to a previously allocated buffer which will be updated to contain the read/modify/write command packet                                                                                                                                                                                   |
| <i>rawPacket↔<br/>Length</i>                      | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

#### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmw\\_command\\_packet\\_example.c](#).

**8.115.2.18** `char RMAPPACKETLIBRARY_CC RMAP_FillReadModifyWriteReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 * pData, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadModifyWriteReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildReadModifyWriteReplyPacket](#) for a function which builds a read/modify/write reply packet, performing the memory allocation.

#### Parameters

|                               |                                                                                                                                                                                                                                                                                                   |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>      | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                |
| <i>initiatorAddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                               |
| <i>targetAddress</i>          | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                              |
| <i>status</i>                 | the status of the read operation                                                                                                                                                                                                                                                                  |
| <i>transactionIdentifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                 |
| <i>dataLength</i>             | the length in bytes of the data field, this can be at most 4 bytes                                                                                                                                                                                                                                |
| <i>pData</i>                  | the data read                                                                                                                                                                                                                                                                                     |
| <i>pRawPacketLength</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                   |
| <i>pPacketStruct</i>          | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the API. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>              | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                          |
| <i>pRawPacket</i>             | a pointer to a previously allocated buffer which will be updated to contain the read reply packet                                                                                                                                                                                                 |
| <i>rawPacketLength</i>        | the length of the buffer provided for the packet                                                                                                                                                                                                                                                  |

#### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[rmw\\_reply\\_packet\\_example.c](#).

**8.115.2.19** `char RMAPPACKETLIBRARY_CC RMAP_FillReadReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateReadReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildReadReplyPacket](#) for a function which builds a read reply packet, performing the memory allocation.

**Parameters**

|                                     |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>                | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>increment↔<br/>Address</i>       | whether or not the command indicated that the target read address should be incremented when reading                                                                                                                                                                                               |
| <i>status</i>                       | the status of the read operation                                                                                                                                                                                                                                                                   |
| <i>transaction↔<br/>Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pData</i>                        | the data read                                                                                                                                                                                                                                                                                      |
| <i>dataLength</i>                   | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket↔<br/>Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                   | a pointer to a previously allocated buffer which will be updated to contain the read reply packet                                                                                                                                                                                                  |
| <i>rawPacket↔<br/>Length</i>        | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[read\\_reply\\_packet\\_example.c](#).

**8.115.2.20** `char RMAPPACKETLIBRARY_CC RMAP_FillWriteCommandPacket ( U8 * pTargetAddress, unsigned long targetAddressLength, U8 * pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 * pData, U32 dataLength, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateWriteCommandPacketLength](#). This function performs no memory allocation. See [RMAP\\_Build↔WriteCommandPacket](#) for a function which builds a write command packet, performing the memory allocation.

**Parameters**

|                       |                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------|
| <i>pTargetAddress</i> | a pointer to the SpaceWire path or logical address of the device to be written to |
|-----------------------|-----------------------------------------------------------------------------------|



|                                           |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>targetAddress</i> ↔<br><i>Length</i>   | the length of the target address                                                                                                                                                                                                                                                                   |
| <i>pReplyAddress</i>                      | a pointer to the SpaceWire path or logical address that will be used to send any responses back to the device from which the command will be sent from                                                                                                                                             |
| <i>replyAddress</i> ↔<br><i>Length</i>    | the length of the reply address                                                                                                                                                                                                                                                                    |
| <i>verifyBefore</i> ↔<br><i>Write</i>     | whether or not the data should be verified before writing                                                                                                                                                                                                                                          |
| <i>acknowledge</i>                        | whether or not the command should be acknowledged                                                                                                                                                                                                                                                  |
| <i>increment</i> ↔<br><i>Address</i>      | whether or not the target write address should be incremented when writing                                                                                                                                                                                                                         |
| <i>key</i>                                | the key expected by the destination device                                                                                                                                                                                                                                                         |
| <i>transaction</i> ↔<br><i>Identifier</i> | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>writeAddress</i>                       | the memory address at the destination to write to                                                                                                                                                                                                                                                  |
| <i>extendedWrite</i> ↔<br><i>Address</i>  | the extended memory address at the destination to write to                                                                                                                                                                                                                                         |
| <i>pData</i>                              | the data to be written                                                                                                                                                                                                                                                                             |
| <i>dataLength</i>                         | the length of data, which is a 24-bit number and so should be at most 0xfffff                                                                                                                                                                                                                      |
| <i>pRawPacket</i> ↔<br><i>Length</i>      | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                      | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                          | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                         | a pointer to a previously allocated buffer which will be updated to contain the write command packet                                                                                                                                                                                               |
| <i>rawPacket</i> ↔<br><i>Length</i>       | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

### Returns

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

### Examples:

[write\\_command\\_packet\\_example.c](#).

**8.115.2.21** `char RMAPPACKETLIBRARY_CC RMAP_FillWriteReplyPacket ( U8 * pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, RMAP_STATUS status, U16 transactionIdentifier, unsigned long * pRawPacketLength, RMAP_PACKET * pPacketStruct, char alignment, U8 * pRawPacket, unsigned long rawPacketLength )`

Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.

The length of the packet to be built, and therefore the amount of memory required, can be determined by calling [RMAP\\_CalculateWriteReplyPacketLength](#). This function performs no memory allocation. See [RMAP\\_BuildWrite↔ReplyPacket](#) for a function which builds a write reply packet, performing the memory allocation.

**Parameters**

|                                     |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pInitiatorAddress</i>            | a pointer to the SpaceWire path or logical address of the device to which the reply should be sent                                                                                                                                                                                                 |
| <i>initiator↔<br/>AddressLength</i> | the length of the initiator address                                                                                                                                                                                                                                                                |
| <i>targetAddress</i>                | the SpaceWire logical address of the device that sent the command being responded to                                                                                                                                                                                                               |
| <i>verifyBefore↔<br/>Write</i>      | whether or not the command indicated that data should be verified before writing                                                                                                                                                                                                                   |
| <i>increment↔<br/>Address</i>       | whether or not the command indicated that the target write address should be incremented when writing                                                                                                                                                                                              |
| <i>status</i>                       | the status of the write operation                                                                                                                                                                                                                                                                  |
| <i>transaction↔<br/>Identifier</i>  | an identifier for the transaction                                                                                                                                                                                                                                                                  |
| <i>pRawPacket↔<br/>Length</i>       | a pointer to a previously allocated variable which will be updated to contain the length of the packet returned                                                                                                                                                                                    |
| <i>pPacketStruct</i>                | an optional structure which will be updated to contain details of the fields in the packet. This structure should not be accessed directly, but by using other functions provided by the A↔PI. Alternatively the parameter can be left as NULL if the properties of the packet are not of interest |
| <i>alignment</i>                    | the word size used by the device sending the packet, normally 1. See <a href="#">Byte Alignment</a> for more information                                                                                                                                                                           |
| <i>pRawPacket</i>                   | a pointer to a previously allocated buffer which will be updated to contain the write reply packet                                                                                                                                                                                                 |
| <i>rawPacket↔<br/>Length</i>        | the length of the buffer provided for the packet                                                                                                                                                                                                                                                   |

**Returns**

1 if the buffer was successfully filled with the RMAP packet, or 0 if there was an error, for example if one of the fields is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[write\\_reply\\_packet\\_example.c](#).

#### 8.115.2.22 void RMAPPACKETLIBRARY\_CC RMAP\_SetProtocolIdentifier ( RMAP\_PACKET \* pPacketStruct, U8 protocolIdentifier )

Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.

This can be used to create packets which use the RMAP packet structure, but use a different protocol identifier, such as SpaceWire-PnP packets. The packet and packet structure can be created by calling one of the RMAP\_Build\* or RMAP\_Fill\* functions.

**Parameters**

|                      |                                                                                                |
|----------------------|------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------|

---

*protocolIdentifier* the value to set the packet's protocol identifier to

---

**Returns**

void

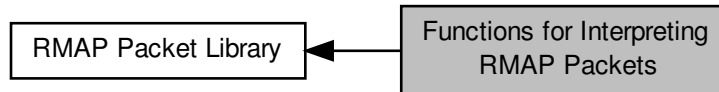
**Added in version:**

STAR-System v2.0 - 3rd July 2013

## 8.116 Functions for Interpreting RMAP Packets

These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.

Collaboration diagram for Functions for Interpreting RMAP Packets:



### Functions

- **RMAP\_STATUS** RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValid (void \*pRawPacket, unsigned long packetLength, RMAP\_PACKET \*pPacketStruct, char checkPacketTooLong)
 

*Check that the packet specified is in the correct format for an RMAP packet.*
- **RMAP\_STATUS** RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValidIgnoreProtocol (void \*pRawPacket, unsigned long packetLength, RMAP\_PACKET \*pPacketStruct, char checkPacketTooLong)
 

*Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.*
- **RMAP\_PACKET\_TYPE** RMAPPACKETLIBRARY\_CC RMAP\_GetPacketType (RMAP\_PACKET \*pPacketStruct)
 

*Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.*
- **U8** \*RMAPPACKETLIBRARY\_CC RMAP\_GetTargetAddress (RMAP\_PACKET \*pPacketStruct, unsigned long \*pTargetAddressLength)
 

*Get the target address of the packet from a packet structure which has already been created.*
- **char** RMAPPACKETLIBRARY\_CC RMAP\_GetVerifyBeforeWrite (RMAP\_PACKET \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.*
- **char** RMAPPACKETLIBRARY\_CC RMAP\_GetPerformAcknowledgement (RMAP\_PACKET \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.*
- **char** RMAPPACKETLIBRARY\_CC RMAP\_GetIncrementAddress (RMAP\_PACKET \*pPacketStruct)
 

*Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.*
- **U8** RMAPPACKETLIBRARY\_CC RMAP\_GetKey (RMAP\_PACKET \*pPacketStruct)
 

*Get the key field in the packet represented by a previously created packet structure.*
- **U8** \*RMAPPACKETLIBRARY\_CC RMAP\_GetReplyAddress (RMAP\_PACKET \*pPacketStruct, unsigned long \*pReplyAddressLength)
 

*Get the reply address of the packet from a packet structure which has already been created.*
- **U16** RMAPPACKETLIBRARY\_CC RMAP\_GetTransactionID (RMAP\_PACKET \*pPacketStruct)
 

*Get the transaction identifier in the packet represented by a previously created packet structure.*
- **U32** RMAPPACKETLIBRARY\_CC RMAP\_GetAddress (RMAP\_PACKET \*pPacketStruct, U8 \*pExtendedAddress)
 

*Get the memory address and extended memory address of the packet from a packet structure which has already been created.*
- **U8** \*RMAPPACKETLIBRARY\_CC RMAP\_GetData (RMAP\_PACKET \*pPacketStruct, U32 \*pDataLength)

*Get the data in the packet from a packet structure which has already been created.*

- [U32 RMAPPACKETLIBRARY\\_CC RMAP\\_GetReadLength \(RMAP\\_PACKET \\*pPacketStruct\)](#)

*Gets the data length property in the read command packet from a packet structure which has already been created.*

- [RMAP\\_STATUS RMAPPACKETLIBRARY\\_CC RMAP\\_GetStatus \(RMAP\\_PACKET \\*pPacketStruct\)](#)

*Get the value of the status field in the packet represented by a previously created packet structure.*

- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetHeaderCRC \(RMAP\\_PACKET \\*pPacketStruct\)](#)

*Get the value of the header CRC field in the packet represented by a previously created packet structure.*

- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetDataCRC \(RMAP\\_PACKET \\*pPacketStruct\)](#)

*Get the value of the data CRC field in the packet represented by a previously created packet structure.*

- [U8 \\*RMAPPACKETLIBRARY\\_CC RMAP\\_GetMask \(RMAP\\_PACKET \\*pPacketStruct, U8 \\*pMaskLength\)](#)

*Get the mask in the packet from a packet structure which has already been created.*

- [U8 RMAPPACKETLIBRARY\\_CC RMAP\\_GetProtocolIdentifier \(RMAP\\_PACKET \\*pPacketStruct\)](#)

*Get the protocol identifier of the packet from a packet structure which has already been created.*

### 8.116.1 Detailed Description

These functions can be used to determine if an RMAP packet is valid, and the fields in that RMAP packet.

These functions are useful when receiving RMAP packets.

### 8.116.2 Function Documentation

#### 8.116.2.1 [RMAP\\_STATUS RMAPPACKETLIBRARY\\_CC RMAP\\_CheckPacketValid \( void \\* pRawPacket, unsigned long packetLength, RMAP\\_PACKET \\* pPacketStruct, char checkPacketTooLong \)](#)

Check that the packet specified is in the correct format for an RMAP packet.

Note that this function expects that the address at the start of the packet (the target address in a command and the reply address in a response) is only one byte in length (e.g. it is the packet when it reaches its destination). This is because without this information it would be otherwise impossible to determine with certainty if packet was a valid RMAP packet. Offsets of packets with longer addresses at the start can be passed in using pointer arithmetic.

#### Parameters

|                                    |                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">pRawPacket</a>         | a buffer containing an RMAP packet                                                                                                                                                                                                                                                                                                                                                                   |
| <a href="#">packetLength</a>       | the length of the RMAP packet                                                                                                                                                                                                                                                                                                                                                                        |
| <a href="#">pPacketStruct</a>      | an optional (previously allocated) structure which can be provided, or left as NULL. If a structure is supplied, this will be updated to contain the packet's properties. Although it is not recommended to access this structure directly, functions such as <a href="#">RMAP_GetReplyAddress</a> and <a href="#">RMAP_GetKey</a> can be called to retrieve the values of fields from the structure |
| <a href="#">checkPacketTooLong</a> | whether to check if the packet specified is longer than it should be, based on the values in the RMAP fields. This should be set to 0 if the packet was received using a device which can only receive multiples of 4 bytes, for example                                                                                                                                                             |

#### Returns

an RMAP error code indicating if the packet contained any errors, for example if any fields have invalid values, or the packet is too short or long

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[check\\_packet\\_example.c](#).

### 8.116.2.2 RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_CheckPacketValidIgnoreProtocol ( void \* *pRawPacket*, unsigned long *packetLength*, RMAP\_PACKET \* *pPacketStruct*, char *checkPacketTooLong* )

Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.

This can be used to check the format of packets for other protocols such as SpaceWire-PnP, which use the RMAP packet format. The protocol identifier can then be checked by calling [RMAP\\_GetProtocolIdentifier\(\)](#). Note that this function expects that the address at the start of the packet (the target address in a command and the reply address in a response) is only one byte in length (e.g. it is the packet when it reaches its destination). This is because without this information it would be otherwise impossible to determine with certainty if the packet was a valid RMAP packet. Offsets of packets with longer addresses at the start can be passed in using pointer arithmetic.

#### Parameters

|                           |                                                                                                                                                                                                                                                                                                                                                                                                      |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pRawPacket</i>         | a buffer containing an RMAP packet                                                                                                                                                                                                                                                                                                                                                                   |
| <i>packetLength</i>       | the length of the RMAP packet                                                                                                                                                                                                                                                                                                                                                                        |
| <i>pPacketStruct</i>      | an optional (previously allocated) structure which can be provided, or left as NULL. If a structure is supplied, this will be updated to contain the packet's properties. Although it is not recommended to access this structure directly, functions such as <a href="#">RMAP_GetReplyAddress</a> and <a href="#">RMAP_GetKey</a> can be called to retrieve the values of fields from the structure |
| <i>checkPacketTooLong</i> | whether to check if the packet specified is longer than it should be, based on the values in the RMAP fields. This should be set to 0 if the packet was received using a device which can only receive multiples of 4 bytes, for example                                                                                                                                                             |

#### Returns

an RMAP error code indicating if the packet contained any errors, for example if any fields have invalid values, or the packet is too short or long

#### Added in version:

STAR-System v2.0 - 3rd July 2013

### 8.116.2.3 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetAddress ( RMAP\_PACKET \* *pPacketStruct*, U8 \* *pExtendedAddress* )

Get the memory address and extended memory address of the packet from a packet structure which has already been created.

The memory address is the address to be read from or written to at the target. The extended address can be used to extend the 32-bit memory address to 40 bits. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_↔\\_CheckPacketValid](#) to check that a packet is in the correct format.

#### Parameters

|                          |                                                                                                                                                                                      |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>     | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pExtended↔Address</i> | a pointer to a previously allocated variable which will be updated to contain the extended address. This can be NULL if the extended address is not of interest                      |

#### Returns

the memory address in the packet, or 0 if the packet is invalid or is of a type which does not contain a memory address

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**8.116.2.4 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetData ( RMAP\_PACKET \* pPacketStruct, U32 \* pDataLength )**

Get the data in the packet from a packet structure which has already been created.

The data bytes are the bytes to be written or the bytes that have been read. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pDataLength</i>   | a pointer to a previously allocated variable which will be updated to contain the length of the data field. This can be NULL if the length is not of interest                        |

**Returns**

a pointer to the data in the packet, or NULL if the packet is invalid or is of a type which does not contain data

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**8.116.2.5 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetDataCRC ( RMAP\_PACKET \* pPacketStruct )**

Get the value of the data CRC field in the packet represented by a previously created packet structure.

The data CRC field is present in packet types with data fields and contains an 8-bit CRC covering each byte in the data and mask (in read/modify/write commands) fields. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

the value of the data CRC field in the packet, if present

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

### 8.116.2.6 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetHeaderCRC ( RMAP\_PACKET \* pPacketStruct )

Get the value of the header CRC field in the packet represented by a previously created packet structure.

The header CRC field contains an 8-bit CRC covering each byte in the header. The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

#### Parameters

---

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

---

#### Returns

the value of the header CRC field in the packet

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[check\\_packet\\_example.c](#).

### 8.116.2.7 char RMAPPACKETLIBRARY\_CC RMAP\_GetIncrementAddress ( RMAP\_PACKET \* pPacketStruct )

Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.

The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

#### Parameters

---

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

---

#### Returns

whether incrementing addresses is enabled in the packet

#### Added in version:

STAR-System v0.8 (Beta 1) - 22nd August 2011

#### Examples:

[check\\_packet\\_example.c](#).

### 8.116.2.8 U8 RMAPPACKETLIBRARY\_CC RMAP\_GetKey ( RMAP\_PACKET \* pPacketStruct )

Get the key field in the packet represented by a previously created packet structure.

The key is matched by the target user application in order for the command to be authorised. The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.



**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the key field in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

### 8.116.2.9 **U8\*** [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_GetMask](#) ( [RMAP\\_PACKET](#) \* *pPacketStruct*, [U8](#) \* *pMaskLength* )

Get the mask in the packet from a packet structure which has already been created.

The mask bytes are present in read/modify/write commands and defines how the data written to memory is formed in an application dependent manner. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pMaskLength</i>   | a pointer to a previously allocated variable which will be updated to contain the length of the mask field. This can be NULL if the length is not of interest                        |

---

**Returns**

a pointer to the mask in the packet, or NULL if the packet is invalid or is not a read/modify/write command

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

### 8.116.2.10 **RMAP\_PACKET\_TYPE** [RMAPPACKETLIBRARY\\_CC](#) [RMAP\\_GetPacketType](#) ( [RMAP\\_PACKET](#) \* *pPacketStruct* )

Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the type of the packet, or [RMAP\\_INVALID\\_PACKET\\_TYPE](#) if the packet is invalid

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

**8.116.2.11** `char RMAPPACKETLIBRARY_CC RMAP_GetPerformAcknowledgement ( RMAP_PACKET * pPacketStruct )`

Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

whether acknowledgement is enabled in the packet

**Added in version:**

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[check\\_packet\\_example.c](#).

**8.116.2.12** `U8 RMAPPACKETLIBRARY_CC RMAP_GetProtocolIdentifier ( RMAP_PACKET * pPacketStruct )`

Get the protocol identifier of the packet from a packet structure which has already been created.

Normally this will be the RMAP protocol ID of 1, but may be the protocol ID of another protocol which uses the RMAP packet format, such as SpaceWire-PnP. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the packet's protocol identifier, or 0 if the packet is invalid

**Added in version:**

STAR-System v2.0 - 3rd July 2013

**8.116.2.13 U32 RMAPPACKETLIBRARY\_CC RMAP\_GetReadLength ( RMAP\_PACKET \* *pPacketStruct* )**

Gets the data length property in the read command packet from a packet structure which has already been created.

The packet structure can be created when calling one of the `RMAP_BuildReadCommandPacket` or `RMAP_FillReadCommandPacket` functions to create a packet to perform a read command operation, or when calling `RMAP_CheckPacketValid` to check that a packet is in the correct format.

**Parameters**

|                      |                                                                                                                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <code>RMAP_CheckPacketValid</code> function or one of the <code>RMAP_BuildReadCommandPacket</code> or <code>RMAP_FillReadCommandPacket</code> functions |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Returns**

the data length of the read command packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**8.116.2.14 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetReplyAddress ( RMAP\_PACKET \* *pPacketStruct*, unsigned long \* *pReplyAddressLength* )**

Get the reply address of the packet from a packet structure which has already been created.

Note that the reply address may contain 0 byte padding if the packet is a command. The packet structure can be created when calling one of the `RMAP_Build*` or `RMAP_Fill*` functions to create a packet to perform a specific operation, or when calling `RMAP_CheckPacketValid` to check that a packet is in the correct format.

**Parameters**

|                            |                                                                                                                                                                             |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>       | a structure representing the packet, created by the <code>RMAP_CheckPacketValid</code> function or one of the <code>RMAP_Build*</code> or <code>RMAP_Fill*</code> functions |
| <i>pReplyAddressLength</i> | a pointer to a previously allocated variable which will be updated to contain the length of the reply address. This can be NULL if the length is not of interest            |

**Returns**

a pointer to the reply address in the packet, or NULL if the packet is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

`check_packet_example.c.`

**8.116.2.15 RMAP\_STATUS RMAPPACKETLIBRARY\_CC RMAP\_GetStatus ( RMAP\_PACKET \* *pPacketStruct* )**

Get the value of the status field in the packet represented by a previously created packet structure.

The status field is present in reply packets to indicate the status of the command. The packet structure can be created when calling one of the `RMAP_Build*` or `RMAP_Fill*` functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the status field in the packet, if present

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

#### 8.116.2.16 U8\* RMAPPACKETLIBRARY\_CC RMAP\_GetTargetAddress ( RMAP\_PACKET \* pPacketStruct, unsigned long \* pTargetAddressLength )

Get the target address of the packet from a packet structure which has already been created.

The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                             |                                                                                                                                                                                      |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>        | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
| <i>pTargetAddressLength</i> | a pointer to a previously allocated variable which will be updated to contain the length of the target address. This can be NULL if the length is not of interest                    |

---

**Returns**

a pointer to the target address in the packet, or NULL if the packet is invalid

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

#### 8.116.2.17 U16 RMAPPACKETLIBRARY\_CC RMAP\_GetTransactionID ( RMAP\_PACKET \* pPacketStruct )

Get the transaction identifier in the packet represented by a previously created packet structure.

The transaction identifier is used to associate a reply with the command that caused the reply. The packet structure can be created when calling one of the [RMAP\\_Build\\*](#) or [RMAP\\_Fill\\*](#) functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**


---

|                      |                                                                                                                                                                                      |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the <a href="#">RMAP_Build*</a> or <a href="#">RMAP_Fill*</a> functions |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

the value of the transaction identifier field in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

**Examples:**

[check\\_packet\\_example.c](#).

**8.116.2.18 char RMAPPACKETLIBRARY\_CC RMAP\_GetVerifyBeforeWrite ( RMAP\_PACKET \* *pPacketStruct* )**

Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.

The packet structure can be created when calling one of the RMAP\_Build\* or RMAP\_Fill\* functions to create a packet to perform a specific operation, or when calling [RMAP\\_CheckPacketValid](#) to check that a packet is in the correct format.

**Parameters**

---

|                      |                                                                                                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | a structure representing the packet, created by the <a href="#">RMAP_CheckPacketValid</a> function or one of the RMAP_Build* or RMAP_Fill* functions |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|

---

**Returns**

whether verify before write is enabled in the packet

**Added in version:**

STAR-System v0.8 (Beta 1) - 22nd August 2011

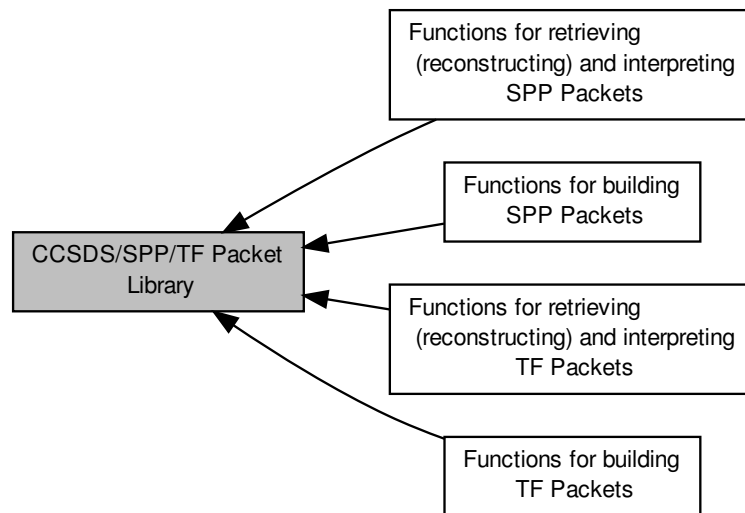
**Examples:**

[check\\_packet\\_example.c](#).

## 8.117 CCSDS/SPP/TF Packet Library

SPP (Space Packet Protocol) is a protocol created by CCSDS (Consultative Committee for Space Data Systems) for space missions to use in order to transfer space application data over any network that involves a ground-to-space, space-to-ground or space-to-space communications link.

Collaboration diagram for CCSDS/SPP/TF Packet Library:



### Modules

- [Functions for building SPP Packets](#)

*These functions can be used to build SPP packets.*

- [Functions for retrieving \(reconstructing\) and interpreting SPP Packets](#)

*These functions can be used to retrieve (reconstruct) SPP packets from an array of (raw) packet bytes that constitute an entire SPP packet.*

- [Functions for building TF Packets](#)

*These functions can be used to build TF packets.*

- [Functions for retrieving \(reconstructing\) and interpreting TF Packets](#)

*These functions can be used to retrieve (reconstruct) TF packets from an array of (raw) packet bytes that constitute an entire TF packet.*

### Data Structures

- struct [CCSDS\\_SPP\\_ENCAPS\\_HEADER](#)

*A structure used to represent the encapsulation header of a CCSDS/SPP/PUS packet. [More...](#)*

- struct [M\\_PDU\\_PACKET](#)

*A structure used to represent a M\_PDU packet Note: The headerSet field is a helper variable to assist in keeping track whether the header pointer has been set for a particular M\_PDU packet. [More...](#)*

- struct [CCSDS\\_TF\\_M\\_PDU\\_HEADER](#)

*A structure used to represent the header of a M\_PDU packet. [More...](#)*

- struct [CCSDS\\_TF\\_PACKET\\_PRIMARY\\_HEADER](#)

- A structure used to represent the primary header of a transfer frame. [More...](#)*

  - struct `CCSDS_TF_INSERT_ZONE`
- A structure used to represent the insert zone field inside a Transfer Frame Note: The insert zone is AOS (Advanced Orbiting Systems) specific field and as such it is optional. [More...](#)*

  - struct `CCSDS_TF_M_PDU_DATA_FIELD`
- A structure used to represent the data field of a M\_PDU packet. [More...](#)*

  - struct `CCSDS_TF_M_PDU_PACKET`
- A structure used to represent a complete transfer frame that is using M\_PDU packets as payload. [More...](#)*

  - struct `CCSDS_SPP_PACKET_PRIMARY_HEADER`
- A structure used to represent the primary header of a CCSDS/SPP/PUS packet. [More...](#)*

  - struct `CCSDS_SPP_PACKET_DATA_FIELD`
- A structure used to represent the data field of a CCSDS/SPP/PUS packet. [More...](#)*

  - struct `CCSDS_SPP_PACKET`
- A structure used to represent the contents of a CCSDS/SPP/PUS packet. [More...](#)*

## Macros

- #define `M_PDU_PACKET_MAX_SIZE` (U32)(2040)
- The maximum size of an M\_PDU packet as defined in the CCSDS/AOS protocol specification.*

## Enumerations

- enum `CCSDS_SPP_STATUS` {  
`CCSDS_SPP_STATUS_SUCCESS = 0, CCSDS_SPP_STATUS_RAW_DATA_POINTER_NULL, CCSDS_SPP_STATUS_STATUS_POINTER_NULL, CCSDS_SPP_STATUS_RAW_PACKET_LENGTH_POINTER_NULL,`  
`CCSDS_SPP_STATUS_PACKET_TYPE_UNKNOWN, CCSDS_SPP_STATUS_APID_INVALID, CCSDS_SPP_STATUS_SEQUENCE_FLAGS_INVALID, CCSDS_SPP_STATUS_DATA_LENGTH_INVALID,`  
`CCSDS_SPP_STATUS_MEMORY_FAILURE, CCSDS_SPP_STATUS_PACKET_STRUCTURE_POINTER_NULL, CCSDS_SPP_STATUS_VALUE_POINTER_NULL, CCSDS_SPP_STATUS_ENCAPSULATION_HEADER_NULL,`  
`CCSDS_SPP_STATUS_PTP_HEADER_INCOMPLETE, CCSDS_SPP_STATUS_SECONDARY_HEADER_POINTER_NULL, CCSDS_SPP_STATUS_SECONDARY_HEADER_LENGTH_INVALID, CCSDS_SPP_STATUS_ENCAPSULATION_HEADER_LENGTH_INVALID,`  
`CCSDS_SPP_STATUS_RECEIVED_RAW_DATA_POINTER_NULL, CCSDS_SPP_STATUS_PTP_PROTOCOL_IDENTIFIER_INCORRECT, CCSDS_SPP_STATUS_PTP_HEADER_INVALID, CCSDS_SPP_STATUS_PTP_HEADER_POINTER_NULL,`  
`CCSDS_SPP_STATUS_TELECOMMAND_SECONDARY_HEADER_INVALID, CCSDS_SPP_STATUS_SECONDARY_HEADER_FLAG_PACKET_TYPE_CONFLICT, CCSDS_SPP_STATUS_TOO_MANY_DEVICE_REGISTER_ADDRESSES_SPECIFIED, CCSDS_SPP_STATUS_BOTH_SECONDARY_HEADER_AND_USER_DATA_FIELD_MISSING,`  
`CCSDS_SPP_STATUS_ZERO_CCSDS_SPP_PACKETS_PROVIDED_FOR_TF_CREATION, CCSDS_TF_STATUS_PACKET_LENGTHS_POINTER_INVALID, CCSDS_TF_STATUS_NUMBER_OF_CCSDS_SPP_PACKETS_ZERO, CCSDS_TF_STATUS_NUMBER_OF_TF_PACKETS_ZERO,`  
`CCSDS_SPP_STATUS_GENERAL_ERROR` }
- Possible values for the status of CCSDS/SPP/TF/PUS related operations.*
- enum `CCSDS_SPP_PACKET_TYPE` {  
`CCSDS_SPP_PACKET_TYPE_TELEMETRY = 0, CCSDS_SPP_PACKET_TYPE_TELECOMMAND, CCSDS_SPP_PACKET_TYPE_CPDU_COMMAND, CCSDS_SPP_PACKET_TYPE_SPACECRAFT_TIME_PACKET,`  
`CCSDS_SPP_PACKET_TYPE_IDLE_PACKET` }

*Possible values for the packet type of CCSDS/SPP packets Note: This enumeration is internal to this library and the values only partially reflect the TYPE field in the primary header.*



### 8.117.1 Detailed Description

SPP (Space Packet Protocol) is a protocol created by CCSDS (Consultative Committee for Space Data Systems) for space missions to use in order to transfer space application data over any network that involves a ground-to-space, space-to-ground or space-to-space communications link.

TF (Transfer Frames) is a protocol used to send formatted frames of data from the mass-memory to a downlink transmitter. The transfer frames are formatted according to the Advanced Orbiting Systems (AOS) Space Data Link Protocol.

### 8.117.2 Functionality Provided

The STAR-Dundee CCSDS/SPP/TF Packet Library provides functions which can be called to create both SPP and/or TF packets, to retrieve the relevant values of all fields of a packet (basically to re-create an entire packet) from an array of raw data bytes of either SPP or TF packets. It does not send or receive packets using a SpaceWire/SpaceFibre device; this work must be done by the programmer as this is application and device specific.

Functions are provided to build any kind of SPP and/or TF packets. Each packet built using the functions which allocate a buffer for the packet must be freed using the `CCSDS_SPP_FreeBuffer()` function. For SPP packets, functions are also provided to fill previously allocated buffers. For SPP packets, in case the user wants to allocate memory that will be filled in using the `CCSDS_SPP_FillPacket()` function, he/she MUST make sure to allocate the right amount of memory for it. Because of the fixed/known structure of SPP packets, the memory size needed to fill in the SPP packet is straightforward to work out.

When building SPP packets, the functions can create a structure which can be used in the other functions provided by the library. These functions can be used to determine the values of individual fields within the SPP packet, such as the packet type and the data.

### 8.117.3 Using The Library

The CCSDS/SPP/TF Packet Library is provided in the form of two or three files:

- `ccsds_spp_packet_library.h`, `ccsds_spp_pus_services.h`
- `ccsds_spp_packet_library.lib` (Windows), `libccsds_spp_packet_library.so` (Linux, QNX) or `libccsds_spp_packet_library.a` (VxWorks and LEON)
- `ccsds_spp_packet_library.dll` (Windows only)

When writing programs which call functions provided by the CCSDS/SPP/TF Packet Library, the `ccsds_spp_packet_library.h` file must be included. This file includes “`star-dundee_types.h`” and “`common.h`” both of which must also be provided. For non-Windows based operating systems, the file “`star-dundee_annotations.h`” must also be included. This file guarantees that the SAL annotations (only understood by Microsoft Visual Studio IDE) are ignored so that the source code can be compiled correctly. To compile programs to use the library “`libccsds_spp_packet_library.so`”, or “`ccsds_spp_packet_library.lib`” on Windows, must be linked in. When running programs which use the library, on Windows “`ccsds_spp_packet_library.dll`” must either be in the “Windows\System32” directory of the machine in use, or in the directory from which the program was run. On Linux “`libccsds_spp_packet_library.so`” must be present in “`/usr/lib/`”.

### 8.117.4 Contact Information

If you have any comments or queries regarding the CCSDS/SPP/TF Packet Library or documentation please contact:

STAR-Dundee  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK

E-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Further support information and details of all STAR-Dundee products can be found at <http://www.star-dundee.com>.

## 8.117.5 Data Structure Documentation

### 8.117.5.1 struct CCSDS\_SPP\_ENCAPS\_HEADER

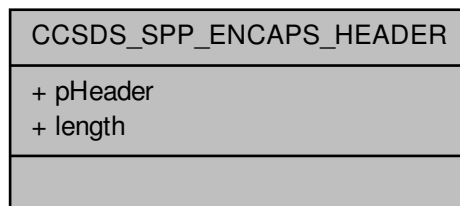
A structure used to represent the encapsulation header of a CCSDS/SPP/PUS packet.

This structure should never be written to directly. Instead, pass the desired contents into one of the `CCSDS_SPP_CreatePacket*` or `CCSDS_SPP_FillPacket*` functions. When retrieving information from this structure, use the `CCSDS_SPP_GetEncapsulationHeaderPointer*` and `CCSDS_SPP_GetDataFieldLength*` functions.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_SPP\_ENCAPS\_HEADER:



#### Data Fields

|      |         |  |
|------|---------|--|
| U16  | length  |  |
| U8 * | pHeader |  |

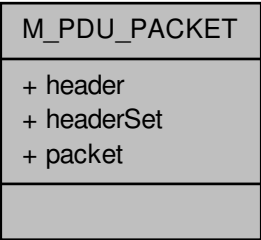
### 8.117.5.2 struct M\_PDU\_PACKET

A structure used to represent a M\_PDU packet Note: The headerSet field is a helper variable to assist in keeping track whether the header pointer has been set for a particular M\_PDU packet.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for M\_PDU\_PACKET:



Data Fields

|     |                               |  |
|-----|-------------------------------|--|
| U16 | header                        |  |
| U8  | headerSet                     |  |
| U8  | packet[M_PDU_PACKET_MAX_SIZE] |  |

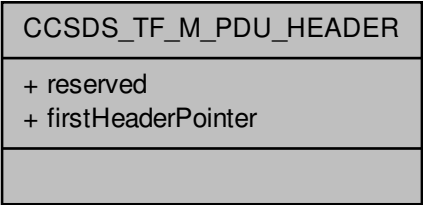
8.117.5.3 struct CCSDS\_TF\_M\_PDU\_HEADER

A structure used to represent the header of a M\_PDU packet.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_TF\_M\_PDU\_HEADER:



Data Fields

|     |                             |  |
|-----|-----------------------------|--|
| U32 | firstHeader↔<br>Pointer: 11 |  |
| U32 | reserved: 5                 |  |

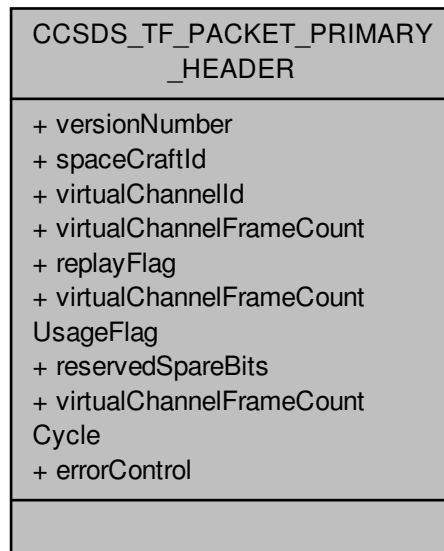
#### 8.117.5.4 struct CCSDS\_TF\_PACKET\_PRIMARY\_HEADER

A structure used to represent the primary header of a transfer frame.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_TF\_PACKET\_PRIMARY\_HEADER:



#### Data Fields

|     |                                      |                                                           |
|-----|--------------------------------------|-----------------------------------------------------------|
| U16 | errorControl                         | Frame header error control Note: This field is optional.  |
| U32 | replayFlag: 1                        | Flag specifying whether the data is "live" or "replayed". |
| U32 | reservedSpare↔<br>Bits: 2            | Bits reserved by the CCSDS standard for future use.       |
| U8  | spaceCraftId                         | ID of the spacecraft.                                     |
| U32 | version↔<br>Number: 2                | Version number of the transfer frame.                     |
| U32 | virtualChannel↔<br>FrameCount:<br>24 | Sequential binary count of the transmitted frame packets. |

|     |                                                   |                                                                   |
|-----|---------------------------------------------------|-------------------------------------------------------------------|
| U32 | virtualChannel↔<br>FrameCount↔<br>Cycle: 4        | Whenever the VC frame count returns to zero, this is incremented. |
| U32 | virtualChannel↔<br>FrameCount↔<br>UsageFlag:<br>1 | Flag specifying whether the frame count is used or not.           |
| U32 | virtualChannel↔<br>Id: 6                          | ID of the virtual channel used.                                   |

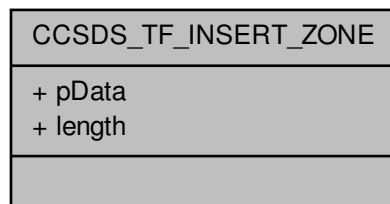
#### 8.117.5.5 struct CCSDS\_TF\_INSERT\_ZONE

A structure used to represent the insert zone field inside a Transfer Frame Note: The insert zone is AOS (Advanced Orbiting Systems) specific field and as such it is optional.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_TF\_INSERT\_ZONE:



#### Data Fields

|      |        |  |
|------|--------|--|
| U8   | length |  |
| U8 * | pData  |  |

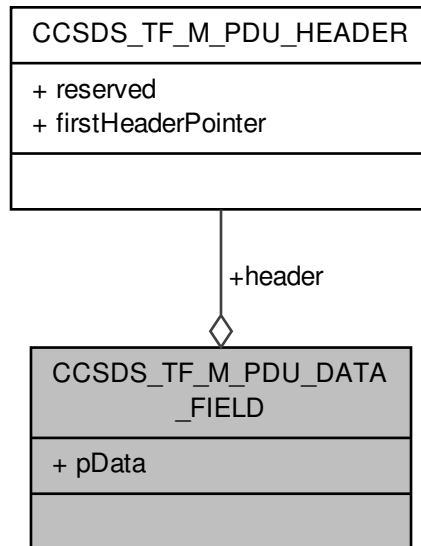
#### 8.117.5.6 struct CCSDS\_TF\_M\_PDU\_DATA\_FIELD

A structure used to represent the data field of a M\_PDU packet.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_TF\_M\_PDU\_DATA\_FIELD:



#### Data Fields

|                       |        |                                        |
|-----------------------|--------|----------------------------------------|
| CCSDS_TF_M_PDU_HEADER | header | Header of the M_PDU data field packet. |
| U8 *                  | pData  | CCSDS_SPP_PACKET data bytes.           |

#### 8.117.5.7 struct CCSDS\_TF\_M\_PDU\_PACKET

A structure used to represent a complete transfer frame that is using M\_PDU packets as payload.

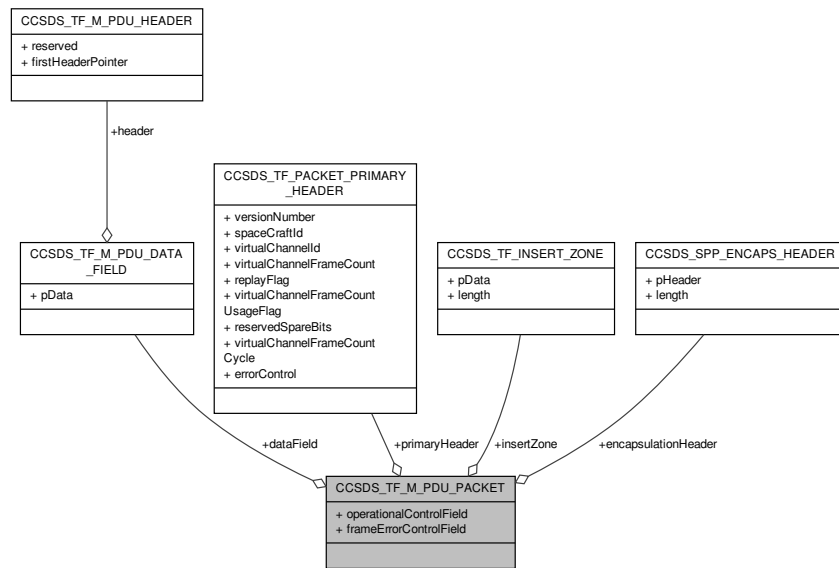
Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Examples:

ccsds\_spp\_tf\_examples.c.

Collaboration diagram for CCSDS\_TF\_M\_PDU\_PACKET:



#### Data Fields

|                                |                         |                                                           |
|--------------------------------|-------------------------|-----------------------------------------------------------|
| CCSDS_TF_M_PDU_DATA_FIELD      | dataField               | Structure holding the data field of the packet.           |
| CCSDS_SPP_ENCAPS_HEADER        | encapsulationHeader     | Structure holding the encapsulation header of the packet. |
| U16                            | frameErrorControlField  | Frame error control field Note: This field is optional.   |
| CCSDS_TF_INSERT_ZONE           | insertZone              | Structure holding the insert zone data.                   |
| U32                            | operationalControlField | Operational control field Note: This field is optional.   |
| CCSDS_TF_PACKET_PRIMARY_HEADER | primaryHeader           | Structure holding the primary header information.         |

#### 8.117.5.8 struct CCSDS\_SPP\_PACKET\_PRIMARY\_HEADER

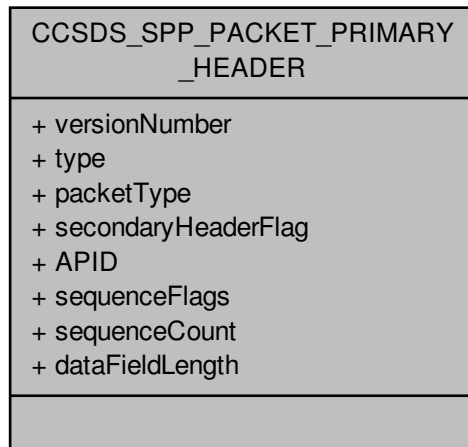
A structure used to represent the primary header of a CCSDS/SPP/PUS packet.

This structure should never be written to directly. Instead, pass some of the desired contents (some fields in this structure are fixed so no need to specify them) into one of the CCSDS\_SPP\_CreatePacket\* or CCSDS\_SPP\_FillPacket\* functions. When retrieving information from this structure, use the CCSDS\_SPP\_Get\* functions (CCSDS\_SPP\_GetPacketVersionNumber for example). functions.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Collaboration diagram for CCSDS\_SPP\_PACKET\_PRIMARY\_HEADER:



#### Data Fields

|                          |                        |                                                                                                                                                        |
|--------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| U32                      | APID: 11               | Application Process Identifier.                                                                                                                        |
| U16                      | dataFieldLength        | The length of the data field Note: This length is one less (- 1) than the sum of the 1) Length of the secondary header AND 2) Length of the user data. |
| CCSDS_SPP_PACKET_TYPE PE | packetType             | The internal type of the packet.                                                                                                                       |
| U32                      | secondaryHeaderFlag: 1 | Flag that shows the presence of the secondary header.                                                                                                  |
| U32                      | sequenceCount: 14      | Sequence count of the packet.                                                                                                                          |
| U32                      | sequenceFlags: 2       | Sequence flags.                                                                                                                                        |
| U32                      | type: 1                | The type of the packet Only one bit is required to represent this As it is either 0 for Telemetry Or 1 for Telecommand.                                |
| U32                      | versionNumber: 3       | The version number of the packet.                                                                                                                      |

#### 8.117.5.9 struct CCSDS\_SPP\_PACKET\_DATA\_FIELD

A structure used to represent the data field of a CCSDS/SPP/PUS packet.

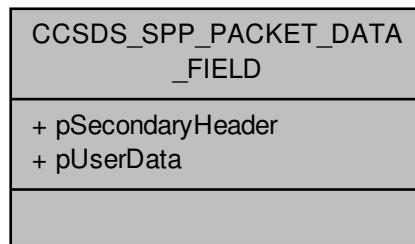
This structure should never be written to directly. Instead, pass the desired contents into one of the CCSDS\_SPP\_CreatePacket\* or CCSDS\_SPP\_FillPacket\* functions. When retrieving information from this structure, use the CCSDS\_SPP\_GetDataFieldPointer\* functions.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021



Collaboration diagram for CCSDS\_SPP\_PACKET\_DATA\_FIELD:



#### Data Fields

|      |                  |                                                        |
|------|------------------|--------------------------------------------------------|
| U8 * | pSecondaryHeader | Pointer pointing to the secondary header of a packet.  |
| U8 * | pUserData        | Pointer pointing to the user data section of a packet. |

#### 8.117.5.10 struct CCSDS\_SPP\_PACKET

A structure used to represent the contents of a CCSDS/SPP/PUS packet.

This structure should never be written to directly. Instead, pass the desired contents into one of the `CCSDS_SPP_P_CreatePacket*` or `CCSDS_SPP_FillPacket*` functions.

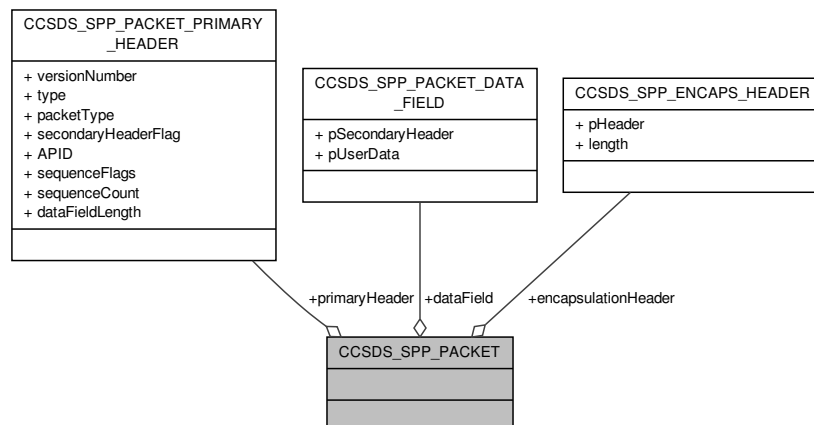
Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

Examples:

[ccsds\\_spp\\_tf\\_examples.c](#).

Collaboration diagram for CCSDS\_SPP\_PACKET:



## Data Fields

|                                              |                     |                                                           |
|----------------------------------------------|---------------------|-----------------------------------------------------------|
| <code>CCSDS_SPP_PACKET_DATA_FIELD</code>     | dataField           | Structure holding the data field of the packet.           |
| <code>CCSDS_SPP_ENCAPSULATION_HEADER</code>  | encapsulationHeader | Structure holding the encapsulation header of the packet. |
| <code>CCSDS_SPP_PACKET_PRIMARY_HEADER</code> | primaryHeader       | Structure holding the primary header of the packet.       |

## 8.117.6 Macro Definition Documentation

8.117.6.1 `#define M_PDU_PACKET_MAX_SIZE (U32)(2040)`

The maximum size of an M\_PDU packet as defined in the CCSDS/AOS protocol specification.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

## 8.117.7 Enumeration Type Documentation

8.117.7.1 `enum CCSDS_SPP_PACKET_TYPE`

Possible values for the packet type of CCSDS/SPP packets Note: This enumeration is internal to this library and the values only partially reflect the TYPE field in the primary header.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

## Enumerator

`CCSDS_SPP_PACKET_TYPE_TELEMETRY` Telemetry packet type.

`CCSDS_SPP_PACKET_TYPE_TELECOMMAND` Telecommand packet type.

`CCSDS_SPP_PACKET_TYPE_CPDU_COMMAND` CPDU command packet type.

`CCSDS_SPP_PACKET_TYPE_SPACECRAFT_TIME_PACKET` Spacecraft time packet type.

`CCSDS_SPP_PACKET_TYPE_IDLE_PACKET` IDLE data packet type.

8.117.7.2 `enum CCSDS_SPP_STATUS`

Possible values for the status of CCSDS/SPP/TF/PUS related operations.

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

## Enumerator

`CCSDS_SPP_STATUS_SUCCESS` The status value indicating success.

`CCSDS_SPP_STATUS_RAW_DATA_POINTER_NULL` The status value indicating that the pointer pointing to the raw data packet is NULL.

- CCSDS\_SPP\_STATUS\_STATUS\_POINTER\_NULL** The status value indicating that the pointer holding the returned status of the CCSDS\_SPP\* operation was NULL.
- CCSDS\_SPP\_STATUS\_RAW\_PACKET\_LENGTH\_POINTER\_NULL** The status value indicating that the pointer holding the length of the raw packet was NULL.
- CCSDS\_SPP\_STATUS\_PACKET\_TYPE\_UNKNOWN** The status value indicating that the packet type is unknown.
- CCSDS\_SPP\_STATUS\_APID\_INVALID** The status value indicating that the APID (Application Process Identifier) is invalid.
- CCSDS\_SPP\_STATUS\_SEQUENCE\_FLAGS\_INVALID** The status value indicating that the value of the sequence flags is invalid.
- CCSDS\_SPP\_STATUS\_DATA\_LENGTH\_INVALID** The status value indicating that the given data length value is invalid.
- CCSDS\_SPP\_STATUS\_MEMORY\_FAILURE** The status value indicating that the requested memory could not be allocated.
- CCSDS\_SPP\_STATUS\_PACKET\_STRUCTURE\_POINTER\_NULL** The status value indicating that the packet structure pointer containing information about the CCSDS/SPP packet was NULL (invalid).
- CCSDS\_SPP\_STATUS\_VALUE\_POINTER\_NULL** The status value indicating that the pointer holding the value of the variable to be retrieved is NULL.
- CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_NULL** The status value indicating that the encapsulation header within the CCSDS/SPP packet was NULL.
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INCOMPLETE** The status value indicating that the PTP protocol header within the CCSDS/SPP packet was incomplete.
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_POINTER\_NULL** The status value indicating that the pointer pointing to the secondary header is NULL.
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_LENGTH\_INVALID** The status value indicating that the length of the secondary header is invalid (zero).
- CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_LENGTH\_INVALID** The status value indicating that the length of the encapsulation header is invalid (zero).
- CCSDS\_SPP\_STATUS\_RECEIVED\_RAW\_DATA\_POINTER\_NULL** The status value indicating that the pointer pointing to the received raw data is NULL.
- CCSDS\_SPP\_STATUS\_PTP\_PROTOCOL\_IDENTIFIER\_INCORRECT** The status value indicating that the PTP protocol identifier value is incorrect.
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INVALID** The status value indicating that the PTP header is invalid.
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_POINTER\_NULL** The status value indicating that the pointer pointing to the PTP header is NULL.
- CCSDS\_SPP\_STATUS\_TELECOMMAND\_SECONDARY\_HEADER\_INVALID** The status value indicating that the secondary header used for telecommand type packets is invalid.
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_FLAG\_PACKET\_TYPE\_CONFLICT** The status value indicating that the secondary header flag and the packet type are in conflict.
- CCSDS\_SPP\_STATUS\_TOO\_MANY\_DEVICE\_REGISTER\_ADDRESSES\_SPECIFIED** The status value indicating that too many device register addresses have been specified.
- CCSDS\_SPP\_STATUS\_BOTH\_SECONDARY\_HEADER\_AND\_USER\_DATA\_FIELD\_MISSING** The status value indicating that both the secondary header and user data field are missing, which is impossible according to the CCSDS/SPP standard.
- CCSDS\_SPP\_STATUS\_ZERO\_CCSDS\_SPP\_PACKETS\_PROVIDED\_FOR\_TF\_CREATION** The status value indicating that the length of the array holding the CCSDS/SPP packets that are being used to create TF packets was zero.
- CCSDS\_TF\_STATUS\_PACKET\_LENGTHS\_POINTER\_INVALID** The status value indicating that the pointer to the array holding the CCSDS/SPP packet length was invalid.

***CCSDS\_TF\_STATUS\_NUMBER\_OF\_CCSDS\_SPP\_PACKETS\_ZERO*** The status value indicating that the number of CCSDS/SPP packets passed into the `CCSDS_TF_DetermineNumberOfTFPackets` function was zero.

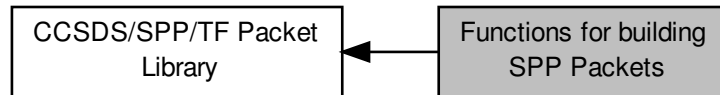
***CCSDS\_TF\_STATUS\_NUMBER\_OF\_TF\_PACKETS\_ZERO*** The status value indicating that the number of TF packets passed into the `Create_M_PDU_Packets` function was zero.

***CCSDS\_SPP\_STATUS\_GENERAL\_ERROR*** The status value indicating that the error does not match any of the other status messages.

## 8.118 Functions for building SPP Packets

These functions can be used to build SPP packets.

Collaboration diagram for Functions for building SPP Packets:



### Functions

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_CreatePacket (_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength, _Out_ CCSDS_SPP_STATUS *const pStatus)`

*Function used to create the raw packet byte buffer.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_FillPacket (_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, _Out_ U8 *const pRawPacket, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength)`

*Function used to fill the raw packet byte buffer that is passed into this function.*

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_CreatePTPHeader (_In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte, _Out_ CCSDS_SPP_STATUS *const pStatus)`

*Function used to create the PTP protocol header.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_FillPTPHeader (_Out_ U8 *const pHeader, _In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte)`

*Function used to fill the PTP protocol header.*

- `void STAR_API_CC CCSDS_SPP_FreeBuffer (_Inout_ void *pBuffer)`

*Function used to free memory that was dynamically allocated by this library.*

### 8.118.1 Detailed Description

These functions can be used to build SPP packets.

These functions are useful when transmitting SPP packets.

### 8.118.2 Function Documentation

8.118.2.1 `_Ret_maybenull_ U8* STAR_API_CC CCSDS_SPP_CreatePacket ( _Out_opt_ CCSDS_SPP_PACKET * pPacketStruct, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength, _Out_ CCSDS_SPP_STATUS *const pStatus )`

Function used to create the raw packet byte buffer.

This buffer represents the complete CCSDS/SPP packet. It will also populate the corresponding fields in the packet structure passed in as a parameter.

#### Parameters

|                                               |                                                                                                       |
|-----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>                          | pointer to a variable holding the structure information of the CCSDS/SPP packet                       |
| <i>p</i> ↔                                    | a pointer to the buffer containing the encapsulation header                                           |
| <i>Encapsulation</i> ↔<br><i>Header</i>       |                                                                                                       |
| <i>encapsulation</i> ↔<br><i>HeaderLength</i> | a variable holding the length of the encapsulation header                                             |
| <i>type</i>                                   | variable holding the type of the CCSDS/SPP packet                                                     |
| <i>secondary</i> ↔<br><i>HeaderFlag</i>       | variable holding the value of the secondary header flag                                               |
| <i>APID</i>                                   | variable holding the APID (Application Process Identifier) value                                      |
| <i>sequenceFlags</i>                          | variable holding the value of the sequence flags                                                      |
| <i>sequenceCount</i>                          | variable holding the value of the sequence count                                                      |
| <i>dataFieldLength</i>                        | variable holding the length of the data field of the CCSDS/SPP packet                                 |
| <i>pSecondary</i> ↔<br><i>Header</i>          | a pointer to a buffer holding the contents of the secondary header                                    |
| <i>secondary</i> ↔<br><i>HeaderLength</i>     | variable holding the length of the secondary header                                                   |
| <i>pUserDataField</i>                         | a pointer to the buffer holding the contents of the user data field                                   |
| <i>pStatus</i>                                | a pointer to a variable holding the status of the previous, present and future operations             |
| <i>pRawPacket</i> ↔<br><i>Length</i>          | a pointer to a value holding the length of the raw packet (this pointer will be set by this function) |

#### Returns

U8\* a pointer to a buffer containing the raw packet bytes of the created CCSDS/SPP packet

#### Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### Examples:

`ccsds_spp_tf_examples.c`.

8.118.2.2 `_Ret_maybenull_ U8* STAR_API_CC CCSDS_SPP_CreatePTPHeader ( _In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte, _Out_ CCSDS_SPP_STATUS *const pStatus )`

Function used to create the PTP protocol header.

#### Parameters

|                             |                                                                 |
|-----------------------------|-----------------------------------------------------------------|
| <i>targetLogicalAddress</i> | a variable containing the logical address of the target device  |
| <i>reservedByte</i>         | variable containing the reserved byte of the PTP header         |
| <i>userApplicationByte</i>  | variable containing the user application byte of the PTP header |
| <i>pStatus</i>              | a pointer to a variable holding the status of this operation    |

**Returns**

a pointer to a buffer containing the PTP encapsulation header

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**8.118.2.3 CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_FillPacket ( \_Out\_opt\_ CCSDS\_SPP\_PACKET \* pPacketStruct, \_Out\_ U8 \*const pRawPacket, \_In\_opt\_ const U8 \*const pEncapsulationHeader, \_In\_ const U16 encapsulationHeaderLength, \_In\_ const CCSDS\_SPP\_PACKET\_TYPE type, \_In\_ const U8 secondaryHeaderFlag, \_In\_ const U16 APID, \_In\_ const U8 sequenceFlags, \_In\_ const U16 sequenceCount, \_In\_ const U16 dataFieldLength, \_In\_opt\_ const U8 \*const pSecondaryHeader, \_In\_ const U16 secondaryHeaderLength, \_In\_opt\_ const U8 \*const pUserDataField, \_Out\_ U32 \*const pRawPacketLength )**

Function used to fill the raw packet byte buffer that is passed into this function.

This buffer represents the complete CCSDS/SPP packet. It will also populate the corresponding fields in the packet structure passed in as a parameter.

**Parameters**

|                                  |                                                                                                       |
|----------------------------------|-------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>             | pointer to a variable holding the structure information of the CCSDS/SPP packet                       |
| <i>pRawPacket</i>                | a pointer to the buffer to be populated by this function                                              |
| <i>pEncapsulationHeader</i>      | a pointer to the buffer containing the encapsulation header                                           |
| <i>encapsulationHeaderLength</i> | a variable holding the length of the encapsulation header                                             |
| <i>type</i>                      | variable holding the type of the CCSDS/SPP packet                                                     |
| <i>secondaryHeaderFlag</i>       | variable holding the value of the secondary header flag                                               |
| <i>APID</i>                      | variable holding the APID (Application Process Identifier) value                                      |
| <i>sequenceFlags</i>             | variable holding the value of the sequence flags                                                      |
| <i>sequenceCount</i>             | variable holding the value of the sequence count                                                      |
| <i>dataFieldLength</i>           | variable holding the length of the data field of the CCSDS/SPP packet                                 |
| <i>pSecondaryHeader</i>          | a pointer to a buffer holding the contents of the secondary header                                    |
| <i>secondaryHeaderLength</i>     | variable holding the length of the secondary header                                                   |
| <i>pUserDataField</i>            | a pointer to the buffer holding the contents of the user data field                                   |
| <i>pRawPacketLength</i>          | a pointer to a value holding the length of the raw packet (this pointer will be set by this function) |

**Returns**

status a variable holding the status of this operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**8.118.2.4** `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_FillPTPHeader ( _Out_ U8 *const pHeader, _In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte )`

Function used to fill the PTP protocol header.



**Parameters**

|                                   |                                                                 |
|-----------------------------------|-----------------------------------------------------------------|
| <i>pHeader</i>                    | a pointer to the buffer to be populated by this function        |
| <i>targetLogical↔<br/>Address</i> | a variable containing the logical address of the target device  |
| <i>reservedByte</i>               | variable containing the reserved byte of the PTP header         |
| <i>user↔<br/>ApplicationByte</i>  | variable containing the user application byte of the PTP header |

**Returns**

a variable holding the status (success or some form of failure) of this operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Examples:**

`ccsds_spp_tf_examples.c`.

**8.118.2.5 void STAR\_API\_CC CCSDS\_SPP\_FreeBuffer ( \_Inout\_ void \* *pBuffer* )**

Function used to free memory that was dynamically allocated by this library.

**Parameters**

|                |                                              |
|----------------|----------------------------------------------|
| <i>pBuffer</i> | pointer to the buffer that needs to be freed |
|----------------|----------------------------------------------|

**Returns**

void

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

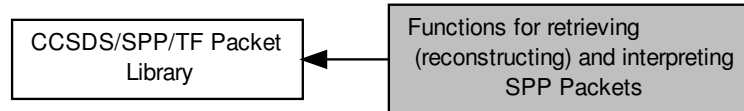
**Examples:**

`ccsds_spp_tf_examples.c`.

## 8.119 Functions for retrieving (reconstructing) and interpreting SPP Packets

These functions can be used to retrieve (reconstruct) SPP packets from an array of (raw) packet bytes that constitute an entire SPP packet.

Collaboration diagram for Functions for retrieving (reconstructing) and interpreting SPP Packets:



### Functions

- **CCSDS\_SPP\_PACKET STAR\_API\_CC CCSDS\_SPP\_RetrievePacket** (*\_In\_* const U8 \*const pReceivedRawPacket, *\_In\_* const U8 retrievePTPHeader, *\_In\_* const U16 encapsulationHeaderLength, *\_Out\_* CCSDS\_SPP\_STATUS \*const pStatus)  
*Function used to receive/retrieve/interpret a CCSDS/SPP packet.*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_CheckAPID** (*\_In\_* const CCSDS\_SPP\_PACKET\_TYPE type, *\_In\_* const U16 APID)  
*Function used to check the validity/correctness of an APID.*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetDataFieldPointer** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U8 \*\*pDataField)  
*Function used to retrieve the pointer pointing to the data field of a CCSDS/SPP packet (represented by the packet structure that was passed in as a parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetDataFieldLength** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U16 \*const pDataFieldLength)  
*Function used to retrieve the length of the data field of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetEncapsulationHeaderPointer** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U8 \*\*pEncapsulationHeader)  
*Function used to retrieve the pointer pointing to the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetEncapsulationHeaderLength** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U16 \*const pEncapsulationHeaderLength)  
*Function used to retrieve the length of the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPacketVersionNumber** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U8 \*const pVersionNumber)  
*Function used to retrieve the version number of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPacketType** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U8 \*const pPacketType)  
*Function used to retrieve the packet type of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_IsSecondaryHeaderPresent** (*\_In\_* const CCSDS\_SPP\_PACKET \*const pPacketStruct, *\_Out\_* U8 \*const plsSecondaryHeaderPresent)  
*Function used to retrieve the secondary header flag of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*

- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetAPID** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pAPID`)  
Function used to retrieve the APID of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetSequenceFlags** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pSequenceFlags`)  
Function used to retrieve the sequence flags of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetSequenceCount** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pSequenceCount`)  
Function used to retrieve the sequence count of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)
- **U8 STAR\_API\_CC CCSDS\_SPP\_GetPTPHeaderLengthBytes** (void)  
Function used to retrieve the size of the PTP protocol header.
- **U8 STAR\_API\_CC CCSDS\_SPP\_GetPrimaryHeaderLengthBytes** (void)  
Function used to retrieve the size of the primary header of a CCSDS/SPP packet.
- **U8 STAR\_API\_CC CCSDS\_SPP\_GetTelecommandSecondaryHeaderLengthBytes** (void)  
Function used to retrieve the size of the secondary header of a Telecommand type CCSDS/SPP packet.
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPTargetLogicalAddress** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pTargetLogicalAddress`)  
Function used to retrieve the PTP target logical address.
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPProtocolIdentifier** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pProtocolIdentifier`)  
Function used to retrieve the PTP protocol identifier byte.
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPReservedByte** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pReservedByte`)  
Function used to retrieve the PTP protocol reserved byte.
- **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPUserApplicationByte** (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pUserApplicationByte`)  
Function used to retrieve the PTP user application byte.

### 8.119.1 Detailed Description

These functions can be used to retrieve (reconstruct) SPP packets from an array of (raw) packet bytes that constitute an entire SPP packet.

These functions are useful when receiving SPP packets.

### 8.119.2 Function Documentation

#### 8.119.2.1 CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_CheckAPID ( \_In\_ const CCSDS\_SPP\_PACKET\_TYPE type, \_In\_ const U16 APID )

Function used to check the validity/correctness of an APID.

##### Parameters

|             |                                                    |
|-------------|----------------------------------------------------|
| <i>type</i> | the type of the CCSDS/SPP packet                   |
| <i>APID</i> | variable holding the ID of the application process |

##### Returns

a status that represents the success (or failure) of the check

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.119.2.2 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC** **CCSDS\_SPP\_GetAPID** ( *\_In\_ const* **CCSDS\_SPP\_PACKET** \**const pPacketStruct*, *\_Out\_ U16* \**const pAPID* )

Function used to retrieve the APID of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

#### Parameters

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pAPID</i>         | pointer to a variable holding the APID of the CCSDS/SPP packet                  |

#### Returns

a status that represents the success (or failure) of the operation

#### Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.119.2.3 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC** **CCSDS\_SPP\_GetDataFieldLength** ( *\_In\_ const* **CCSDS\_SPP\_PACKET** \**const pPacketStruct*, *\_Out\_ U16* \**const pDataFieldLength* )

Function used to retrieve the length of the data field of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

#### Parameters

|                         |                                                                                 |
|-------------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i>    | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pDataFieldLength</i> | pointer to a variable holding length of the data field of the CCSDS/SPP packet  |

#### Returns

a status that represents the success (or failure) of the operation

#### Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### Examples:

`ccsds_spp_tf_examples.c`.

### 8.119.2.4 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC** **CCSDS\_SPP\_GetDataFieldPointer** ( *\_In\_ const* **CCSDS\_SPP\_PACKET** \**const pPacketStruct*, *\_Out\_ U8* \*\* *pDataField* )

Function used to retrieve the pointer pointing to the data field of a CCSDS/SPP packet (represented by the packet structure that was passed in as a parameter to this function)

#### Parameters

|                      |                                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the CCSDS/SPP packet          |
| <i>pDataField</i>    | pointer to a pointer holding the start address of the data field of the CCSDS/SPP packet |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Examples:**

`ccsds_spp_tf_examples.c.`

#### 8.119.2.5 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetEncapsulationHeaderLength** ( *\_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U16 \*const pEncapsulationHeaderLength* )

Function used to retrieve the length of the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                                   |                                                                                 |
|-----------------------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i>              | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pEncapsulationHeaderLength</i> | pointer to a variable holding the length of the encapsulation header            |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.6 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetEncapsulationHeaderPointer** ( *\_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*\* pEncapsulationHeader* )

Function used to retrieve the pointer pointing to the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                             |                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>        | pointer to a variable holding the structure information of the CCSDS/SPP packet     |
| <i>pEncapsulationHeader</i> | a pointer to a pointer pointing to the encapsulation header of the CCSDS/SPP packet |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.7 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPacketType** ( *\_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pPacketType* )

Function used to retrieve the packet type of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pPacketType</i>   | pointer to a variable holding the packet type of the CCSDS/SPP packet           |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.8 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPacketVersionNumber ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVersionNumber )**

Function used to retrieve the version number of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                       |                                                                                   |
|-----------------------|-----------------------------------------------------------------------------------|
| <i>pPacketStruct</i>  | a pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pVersionNumber</i> | a pointer to a variable holding the version number of the CCSDS/SPP packet        |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.9 **U8 STAR\_API\_CC CCSDS\_SPP\_GetPrimaryHeaderLengthBytes ( void )**

Function used to retrieve the size of the primary header of a CCSDS/SPP packet.

**Returns**

a variable holding the size of the primary header of the CCSDS/SPP packet Note: This function should return a constant value (defined within this library)

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.10 **U8 STAR\_API\_CC CCSDS\_SPP\_GetPTPHeaderLengthBytes ( void )**

Function used to retrieve the size of the PTP protocol header.

**Returns**

a variable holding the size of the PTP protocol header Note: This function should return a constant value (defined within this library)

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

8.119.2.11 **CCSDS\_SPP\_STATUS** **STAR\_API\_CC** **CCSDS\_SPP\_GetPTPProtocolIdentifier** ( **\_In\_** **const**  
**CCSDS\_SPP\_PACKET** **\*const** *pPacketStruct*, **\_Out\_** **U8** **\*const** *pProtocolIdentifier* )

Function used to retrieve the PTP protocol identifier byte.

**Parameters**

|                            |                                                                                                 |
|----------------------------|-------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>       | pointer to a variable holding the structure information of the CCSDS/SPP packet                 |
| <i>pProtocolIdentifier</i> | pointer to a variable holding the value of the protocol identifier byte field in the PTP header |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**8.119.2.12 CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPReservedByte ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pReservedByte )**

Function used to retrieve the PTP protocol reserved byte.

**Parameters**

|                      |                                                                                      |
|----------------------|--------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the CCSDS/SPP packet      |
| <i>pReservedByte</i> | pointer to a variable holding the value of the reserved byte field in the PTP header |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**8.119.2.13 CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPTargetLogicalAddress ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pTargetLogicalAddress )**

Function used to retrieve the PTP target logical address.

**Parameters**

|                              |                                                                                               |
|------------------------------|-----------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>         | pointer to a variable holding the structure information of the CCSDS/SPP packet               |
| <i>pTargetLogicalAddress</i> | pointer to a variable holding the value of the target logical address field in the PTP header |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**8.119.2.14 CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetPTPUserApplicationByte ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pUserApplicationByte )**

Function used to retrieve the PTP user application byte.



**Parameters**

|                             |                                                                                              |
|-----------------------------|----------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>        | pointer to a variable holding the structure information of the CCSDS/SPP packet              |
| <i>pUserApplicationByte</i> | pointer to a variable holding the value of the user application byte field in the PTP header |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.15 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetSequenceCount ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U16 \*const pSequenceCount )**

Function used to retrieve the sequence count of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i>  | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pSequenceCount</i> | pointer to a variable holding the sequence count of the CCSDS/SPP packet        |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.16 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_SPP\_GetSequenceFlags ( \_In\_ const CCSDS\_SPP\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pSequenceFlags )**

Function used to retrieve the sequence flags of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                       |                                                                                 |
|-----------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i>  | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>pSequenceFlags</i> | pointer to a variable holding the sequence flags of the CCSDS/SPP packet        |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.119.2.17 **U8 STAR\_API\_CC CCSDS\_SPP\_GetTelecommandSecondaryHeaderLengthBytes ( void )**

Function used to retrieve the size of the secondary header of a Telecommand type CCSDS/SPP packet.

**Returns**

a variable holding the size of the secondary header of a Telecommand type CCSDS/SPP packet Note: This function should return a constant value (defined within this library)

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.119.2.18 **CCSDS\_SPP\_STATUS** STAR\_API\_CC CCSDS\_SPP\_IsSecondaryHeaderPresent ( \_In\_ const CCSDS\_SPP\_PACKET \*const *pPacketStruct*, \_Out\_ U8 \*const *plsSecondaryHeaderPresent* )

Function used to retrieve the secondary header flag of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                                  |                                                                                              |
|----------------------------------|----------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>             | pointer to a variable holding the structure information of the CCSDS/SPP packet              |
| <i>plsSecondaryHeaderPresent</i> | pointer to a variable holding the value of the secondary header flag of the CCSDS/SPP packet |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.119.2.19 **CCSDS\_SPP\_PACKET** STAR\_API\_CC CCSDS\_SPP\_RetrievePacket ( \_In\_ const U8 \*const *pReceivedRawPacket*, \_In\_ const U8 *retrievePTPHeader*, \_In\_ const U16 *encapsulationHeaderLength*, \_Out\_ CCSDS\_SPP\_STATUS \*const *pStatus* )

Function used to receive/retrieve/interpret a CCSDS/SPP packet.

**Parameters**

|                                  |                                                                                                                             |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <i>pReceivedRawPacket</i>        | a pointer to the buffer containing the received raw packet bytes                                                            |
| <i>retrievePTPHeader</i>         | a "boolean" type variable that specifies whether the packet to be received has/hasn't got a PTP header                      |
| <i>encapsulationHeaderLength</i> | variable holding the length of the encapsulation header. Note: If no encapsulation header is used, set this parameter to 0. |
| <i>pStatus</i>                   | a pointer to a variable holding the status of previous present (this) and future operations                                 |

**Returns**

a variable holding the received packet structure (populated with the contents of the packet)

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

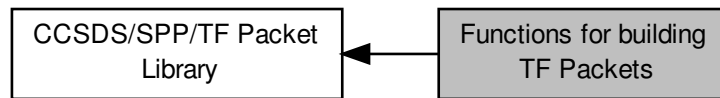
**Examples:**

`ccsds_spp_tf_examples.c`.

## 8.120 Functions for building TF Packets

These functions can be used to build TF packets.

Collaboration diagram for Functions for building TF Packets:



### Functions

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_TF_CreateIdlePacket (const U16 packetSize, U32 *const pRawPacketLength, CCSDS_SPP_STATUS *const pStatus)`

*Function used to create IDLE packets to fill remaining bytes of M PDU packets of a TF (Transfer Frame).*

- `_Ret_maybenull_ U8 **STAR_API_CC CCSDS_TF_CreatePackets (_In_ U8 *const *const pCCSDS_SPP_Packets, _In_ U32 *const pCCSDS_SPP_PacketLengths, _In_ U16 const nrOfCCSDS_SPP_Packets, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const U8 versionNumber, _In_ const U8 spaceCraftId, _In_ const U8 virtualChannelId, _Inout_ U32 *const pVirtualChannelFrameCount, _In_ const U8 replayFlag, _In_ const U8 virtualChannelFrameCountUsageFlag, _Inout_ U8 const virtualChannelFrameCountCycle, _In_ const U8 *const pFrameHeaderErrorControl, _In_opt_ U8 *pInsertZoneData, _In_ U8 insertZoneDataLength, _Out_ U16 *pNumberOfTFPacketsCreated, _Out_ U16 *pTFPacketLengthBytes, _In_opt_ const U8 *const pOperationalControlField, _In_opt_ const U8 *const pFrameErrorControlField, CCSDS_SPP_STATUS *pStatus)`

*Function used to create an array of TF packets (in the form of byte arrays each).*

### 8.120.1 Detailed Description

These functions can be used to build TF packets.

These functions are useful when transmitting TF packets.

### 8.120.2 Function Documentation

#### 8.120.2.1 `_Ret_maybenull_ U8* STAR_API_CC CCSDS_TF_CreateIdlePacket ( const U16 packetSize, U32 *const pRawPacketLength, CCSDS_SPP_STATUS *const pStatus )`

Function used to create IDLE packets to fill remaining bytes of M PDU packets of a TF (Transfer Frame).

#### Parameters

|                               |                                                                                                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>packetSize</code>       | a variable containing the packet size of the idle packet to be created                                   |
| <code>pRawPacketLength</code> | output variable. The length of the entire packet that had just been created is returned in this variable |

---

*pStatus* a pointer to a variable holding the status of this operation

---

#### Returns

a pointer to a buffer containing the bytes of the IDLE packet

#### Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### Examples:

[ccsds\\_spp\\_tf\\_examples.c](#).

**8.120.2.2** `_Ret_maybenull_ U8** STAR_API_CC CCSDS_TF_CreatePackets ( _In_ U8 *const *const pCCSDS_SPP_Packets, _In_ U32 *const pCCSDS_SPP_PacketLengths, _In_ U16 const nrOfCCSDS_SPP_Packets, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const U8 versionNumber, _In_ const U8 spaceCraftId, _In_ const U8 virtualChannelId, _Inout_ U32 *const pVirtualChannelFrameCount, _In_ const U8 replayFlag, _In_ const U8 virtualChannelFrameCountUsageFlag, _Inout_ U8 const virtualChannelFrameCountCycle, _In_ const U8 *const pFrameHeaderErrorControl, _In_opt_ U8 * pInsertZoneData, _In_ U8 insertZoneDataLength, _Out_ U16 * pNumberOfTFPacketsCreated, _Out_ U16 * pTFPacketLengthBytes, _In_opt_ const U8 *const pOperationalControlField, _In_opt_ const U8 *const pFrameErrorControlField, CCSDS_SPP_STATUS * pStatus )`

Function used to create an array of TF packets (in the form of byte arrays each).

The function will also populate several input pointer variables that are passed in as parameters.

#### Parameters

|                                                      |                                                                                                                                                                                                                     |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pCCSDS_SP↔<br/>P_Packets</i>                      | a pointer to a buffer containing the CCSDS/SPP packets (as byte arrays) that will be used to create the TF packets                                                                                                  |
| <i>pCCSDS_SP↔<br/>P_Packet↔<br/>Lengths</i>          | a pointer to a buffer containing the lengths of the CCSDS/SPP packets present in the pointer pCCSDS_SPP_Packets                                                                                                     |
| <i>nrOfCCSDS_↔<br/>SPP_Packets</i>                   | a variable containing the length of the array pointed to by pCCSDS_SPP_Packets                                                                                                                                      |
| <i>p↔<br/>Encapsulation↔<br/>Header</i>              | a pointer to the buffer containing the encapsulation header                                                                                                                                                         |
| <i>encapsulation↔<br/>HeaderLength</i>               | a variable holding the length of the encapsulation header                                                                                                                                                           |
| <i>versionNumber</i>                                 | variable holding the version of the transfer frame                                                                                                                                                                  |
| <i>spaceCraftId</i>                                  | variable holding the spacecraft ID                                                                                                                                                                                  |
| <i>virtualChannelId</i>                              | variable holding the virtual channel ID                                                                                                                                                                             |
| <i>pVirtual↔<br/>ChannelFrame↔<br/>Count</i>         | pointer to a variable holding the most recent frame count of the virtual channel with ID (virtualChannelId) Note: This value is incremented by one in this function (only if the function is executed successfully) |
| <i>replayFlag</i>                                    | variable holding the value whether the TF packet is "Live" or "Replayed"                                                                                                                                            |
| <i>virtualChannel↔<br/>FrameCount↔<br/>UsageFlag</i> | variable holding the value whether the virtual channel frame count values are used/not used                                                                                                                         |

---

|                                                  |                                                                                                                                                           |
|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>virtualChannel↔<br/>FrameCount↔<br/>Cycle</i> | variable holding the value of the virtual channel frame count cycle.                                                                                      |
| <i>pFrame↔<br/>HeaderError↔<br/>Control</i>      | pointer to the frame header error control field (2 bytes in length) Note: This field is optional and can be omitted by passing in NULL.                   |
| <i>pInsertZoneData</i>                           | pointer to the insert zone data field Note: This field is optional and can be omitted by passing in NULL.                                                 |
| <i>insertZone↔<br/>DataLength</i>                | length of the insert zone data Note: If pInsertZoneData is a valid pointer, the length needs to be non-zero. Set to 0 if insert zone data is not present. |
| <i>pNumberOfTF↔<br/>PacketsCreated</i>           | the number of TF packets that had been created by this function is returned in this pointer variable                                                      |
| <i>pTFPacket↔<br/>LengthBytes</i>                | the length of each TF packet (each packet has the same length) is returned in this pointer variable                                                       |
| <i>pOperational↔<br/>ControlField</i>            | pointer to operational control field field (4 bytes in length) Note: This field is optional and can be omitted by passing in NULL.                        |
| <i>pFrameError↔<br/>ControlField</i>             | pointer to the frame error control field (2 bytes in length) Note: This field is optional and can be committed by passing in NULL.                        |
| <i>pStatus</i>                                   | a pointer to a variable holding the status of this operation                                                                                              |

**Returns**

U8\*\* an array of array of bytes that represent the TF packets created by this function

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

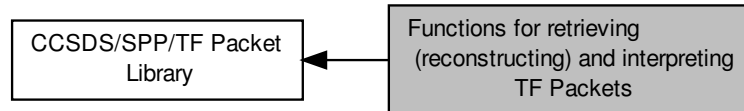
**Examples:**

ccsds\_spp\_tf\_examples.c.

## 8.121 Functions for retrieving (reconstructing) and interpreting TF Packets

These functions can be used to retrieve (reconstruct) TF packets from an array of (raw) packet bytes that constitute an entire TF packet.

Collaboration diagram for Functions for retrieving (reconstructing) and interpreting TF Packets:



### Functions

- `_Out_ CCSDS_TF_M_PDU_PACKET STAR_API_CC CCSDS_TF_RetrievePacket (_In_ const U8 *const pRawPacket, _In_ const U16 encapsulationHeaderLength, _In_ const U8 frameHeaderErrorControlPresent, _In_ const U8 insertZoneDataLength, _In_ const U8 operationalControlFieldPresent, _In_ const U8 frameErrorControlFieldPresent, _Out_ CCSDS_SPP_STATUS *pStatus)`

*Function used to "receive"/reconstruct a TF (Transfer Frame) from an array of raw bytes.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetEncapsulationHeader (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ const U8 *pEncapsulationHeader, _Out_ U16 *const pEncapsulationHeaderLength)`

*Function used to retrieve the encapsulation header of a TF packet.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVersionNumber (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVersionNumber)`

*Function used to retrieve the version number of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetSpacecraftID (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pSpacecraftId)`

*Function used to retrieve the spacecraft ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVirtualChannelID (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVirtualChannelId)`

*Function used to retrieve the virtual channel ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVirtualChannelFrameCount (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U32 *const pVirtualChannelFrameCount)`

*Function used to retrieve the virtual channel frame count of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetReplayFlag (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pReplayFlag)`

*Function used to retrieve the replay flag of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVCFrameCountUsageFlag (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVCFrameCountUsageFlag)`

*Function used to retrieve the virtual channel frame count usage flag of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVCFrameCountCycle (_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *const pVCFrameCountCycle)`

*Function used to retrieve the virtual channel frame count cycle of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_GetFrameHeaderErrorControl` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pFrameHeaderErrorControl`)

*Function used to retrieve the frame header error control field of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_GetInsertZoneData` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U8 *pInsertZoneData, _Out_ U16 *const pInsertZoneDataLength`)

*Function used to retrieve the insert zone data field (and length) of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_Get_M_PDU_DataField` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ const U8 *pM_PDU_DataField`)

*Function used to retrieve the data field of the M PDU packet of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_Get_M_PDU_FirstHeaderPointer` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pM_PDU_FirstHeaderPointer`)

*Function used to retrieve the first header pointer of the M PDU packet of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_GetOperationalControlField` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U32 *const pOperationalControlField`)

*Function used to retrieve the operational control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS_STAR_API_CC_CCSDS_TF_M_PDU_GetFrameErrorControlField` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pFrameErrorControlField`)

*Function used to retrieve the frame error control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `U16 STAR_API_CC_CCSDS_TF_Get_M_PDU_PacketMaxSize` (`void`)

*Function used to retrieve the maximum size of an M PDU packet.*

### 8.121.1 Detailed Description

These functions can be used to retrieve (reconstruct) TF packets from an array of (raw) packet bytes that constitute an entire TF packet.

These functions are useful when receiving TF packets.

### 8.121.2 Function Documentation

#### 8.121.2.1 U16 STAR\_API\_CC\_CCSDS\_TF\_Get\_M\_PDU\_PacketMaxSize ( void )

Function used to retrieve the maximum size of an M PDU packet.

##### Returns

a variable holding the maximum size of an M PDU packet Note: This function should return a constant value (defined within this library)

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.2 CCSDS\_SPP\_STATUS\_STAR\_API\_CC\_CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_DataField ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ const U8 \* pM\_PDU\_DataField )

Function used to retrieve the data field of the M PDU packet of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                               |                                                                                            |
|-------------------------------|--------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>          | pointer to a variable holding the structure information of the TF/M_PDU packet             |
| <i>pM_PDU_↔<br/>DataField</i> | pointer to the data field section of the M PDU packet (which will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.121.2.3 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_FirstHeaderPointer ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U16 \*const pM\_PDU\_FirstHeaderPointer )**

Function used to retrieve the first header pointer of the M PDU packet of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                                        |                                                                                              |
|----------------------------------------|----------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>                   | pointer to a variable holding the structure information of the TF/M_PDU packet               |
| <i>pM_PDU_First↔<br/>HeaderPointer</i> | pointer to the first header pointer of the M PDU packet (which will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

### 8.121.2.4 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetEncapsulationHeader ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ const U8 \* pEncapsulationHeader, \_Out\_ U16 \*const pEncapsulationHeaderLength )**

Function used to retrieve the encapsulation header of a TF packet.

**Parameters**

|                                               |                                                                                                                              |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>                          | pointer to a variable holding the structure information of the TF/M_PDU packet                                               |
| <i>p↔<br/>Encapsulation↔<br/>Header</i>       | pointer to the start of the encapsulation header (the pointer will be set by this function)                                  |
| <i>p↔<br/>Encapsulation↔<br/>HeaderLength</i> | pointer to a variable holding the length of the encapsulation header (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021



#### 8.121.2.5 `CCSDS_SPP_STATUS_STAR_API_CC` `CCSDS_TF_M_PDU_GetFrameErrorControlField` ( `_In_ const` `CCSDS_TF_M_PDU_PACKET` `*const pPacketStruct`, `_Out_ U16` `*const pFrameErrorControlField` )

Function used to retrieve the frame error control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                                      |                                                                                                                                                                     |
|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>                 | pointer to a variable holding the structure information of the TF/M_PDU packet                                                                                      |
| <i>pFrameError↔<br/>ControlField</i> | pointer to the frame error control field the M PDU packet (which will be set by this function)<br>Note: This is an optional field so the returned value may be NULL |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.6 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetFrameHeaderErrorControl ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U16 \*const pFrameHeaderErrorControl )**

Function used to retrieve the frame header error control field of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                                             |                                                                                                                                                                           |
|---------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>                        | pointer to a variable holding the structure information of the TF/M_PDU packet                                                                                            |
| <i>pFrame↔<br/>HeaderError↔<br/>Control</i> | pointer to a variable holding the header error control field (which will be populated by this function) Note: This is an optional field so the returned value may be NULL |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.7 **CCSDS\_SPP\_STATUS STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetInsertZoneData ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \* pInsertZoneData, \_Out\_ U16 \*const pInsertZoneDataLength )**

Function used to retrieve the insert zone data field (and length) of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                                    |                                                                                                                                                                         |
|------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>               | pointer to a variable holding the structure information of the TF/M_PDU packet                                                                                          |
| <i>pInsertZoneData</i>             | pointer to a variable holding the insert zone data bytes (which will be populated by this function) Note: This is an optional field so the returned value may be NULL   |
| <i>pInsertZone↔<br/>DataLength</i> | pointer to a variable holding the length of the insert zone (which will be populated by this function) Note: The length may be 0 if the insert zone data is not present |

**Returns**

a status that represents the success (or failure) of the operation

Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.8 `CCSDS_SPP_STATUS_STAR_API_CC` `CCSDS_TF_M_PDU_GetOperationalControlField` ( `_In_ const` `CCSDS_TF_M_PDU_PACKET` `*const pPacketStruct`, `_Out_ U32` `*const pOperationalControlField` )

Function used to retrieve the operational control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                                 |                                                                                                                                                                        |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>            | pointer to a variable holding the structure information of the TF/M_PDU packet                                                                                         |
| <i>pOperationalControlField</i> | pointer to the operational control field of the M PDU packet (which will be set by this function)<br>Note: This is an optional field so the returned value may be NULL |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.9 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetReplayFlag ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pReplayFlag )**

Function used to retrieve the replay flag of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                      |                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the TF/M_PDU packet                              |
| <i>pReplayFlag</i>   | pointer to a variable holding the replay flag value (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.10 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetSpacecraftID ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pSpacecraftId )**

Function used to retrieve the spacecraft ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                      |                                                                                                     |
|----------------------|-----------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i> | pointer to a variable holding the structure information of the TF/M_PDU packet                      |
| <i>pSpacecraftId</i> | pointer to a variable holding spacecraft ID (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.11 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetVCFrameCountCycle ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVCFrameCountCycle )**

Function used to retrieve the virtual channel frame count cycle of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                           |                                                                                                                             |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>      | pointer to a variable holding the structure information of the TF/M_PDU packet                                              |
| <i>pVCFrameCountCycle</i> | pointer to a variable holding the virtual channel frame count cycle (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.12 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetVCFrameCountUsageFlag ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVCFrameCountUsageFlag )**

Function used to retrieve the virtual channel frame count usage flag of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

**Parameters**

|                               |                                                                                                                                  |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>          | pointer to a variable holding the structure information of the TF/M_PDU packet                                                   |
| <i>pVCFrameCountUsageFlag</i> | pointer to a variable holding the virtual channel frame count usage flag (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.13 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetVersionNumber ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVersionNumber )**

Function used to retrieve the version number of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                       |                                                                                                      |
|-----------------------|------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>  | pointer to a variable holding the structure information of the TF/M_PDU packet                       |
| <i>pVersionNumber</i> | pointer to a variable holding version number (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.14 **CCSDS\_SPP\_STATUS\_STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetVirtualChannelFrameCount ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U32 \*const pVirtualChannelFrameCount )**

Function used to retrieve the virtual channel frame count of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                                  |                                                                                                                       |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>             | pointer to a variable holding the structure information of the TF/M_PDU packet                                        |
| <i>pVirtualChannelFrameCount</i> | pointer to a variable holding the virtual channel frame count (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.15 **CCSDS\_SPP\_STATUS** STAR\_API\_CC CCSDS\_TF\_M\_PDU\_GetVirtualChannelID ( \_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const *pPacketStruct*, \_Out\_ U8 \*const *pVirtualChannelId* )

Function used to retrieve the virtual channel ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)

**Parameters**

|                          |                                                                                                              |
|--------------------------|--------------------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>     | pointer to a variable holding the structure information of the TF/M_PDU packet                               |
| <i>pVirtualChannelId</i> | pointer to a variable holding the virtual channel ID (the value of the pointer will be set by this function) |

**Returns**

a status that represents the success (or failure) of the operation

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

#### 8.121.2.16 \_Out\_ CCSDS\_TF\_M\_PDU\_PACKET STAR\_API\_CC CCSDS\_TF\_RetrievePacket ( \_In\_ const U8 \*const *pRawPacket*, \_In\_ const U16 *encapsulationHeaderLength*, \_In\_ const U8 *frameHeaderErrorControlPresent*, \_In\_ const U8 *insertZoneDataLength*, \_In\_ const U8 *operationalControlFieldPresent*, \_In\_ const U8 *frameErrorControlFieldPresent*, \_Out\_ CCSDS\_SPP\_STATUS \* *pStatus* )

Function used to "receive"/reconstruct a TF (Transfer Frame) from an array of raw bytes.

**Parameters**

|                                       |                                                                                                       |
|---------------------------------------|-------------------------------------------------------------------------------------------------------|
| <i>pRawPacket</i>                     | a pointer to the buffer containing the received raw packet bytes                                      |
| <i>encapsulationHeaderLength</i>      | the length of the encapsulation header                                                                |
| <i>frameHeaderErrorControlPresent</i> | "boolean" type variable that shows whether frame header error control field is present or not         |
| <i>insertZoneDataLength</i>           | the length of the insert zone data field Note: If the insert zone data is not present, set this to 0. |

---

|                                                                 |                                                                                        |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <i>operational</i> ↔<br><i>ControlField</i> ↔<br><i>Present</i> | "boolean" type variable that shows whether operational control field is present or not |
| <i>frameError</i> ↔<br><i>ControlField</i> ↔<br><i>Present</i>  | "boolean" type variable that shows whether frame error control field is present or not |
| <i>pStatus</i>                                                  | a pointer to a variable holding the status of this operation                           |

---

**Returns**

a variable holding the received packet structure (populated with the contents of the packet)

**Added in version:**

STAR-System v5.00 Beta 1 - 24th March 2021

**Examples:**

`ccsds_spp_tf_examples.c`.

## 8.122 STAR-Dundee Types

This section contains the definitions of types used in STAR-Dundee software drivers and APIs.

### Macros

- `#define TRUE 1`  
*A value that can be used to represent the boolean value of true.*
- `#define FALSE 0`  
*A value that can be used to represent the boolean value of false.*
- `#define U16_MAX (0xffff)`  
*The maximum value that can be represented using an unsigned 16-bit number.*

### Typedefs

- `typedef unsigned char U8`  
*A type that can be used to represent an unsigned 8-bit number.*
- `typedef unsigned short U16`  
*A type that can be used to represent an unsigned 16-bit number.*
- `typedef unsigned int U32`  
*A type that can be used to represent an unsigned 32-bit number.*
- `typedef U32 REGISTER`  
*A type that can be used to represent a 4-byte register.*

#### 8.122.1 Detailed Description

This section contains the definitions of types used in STAR-Dundee software drivers and APIs.



## Chapter 9

# File Documentation

### 9.1 ccsds\_spp\_packet\_library.h File Reference

Declarations of enums, structures and functions provided by the CCSDS/SPP library.

```
#include "common.h"
#include "star-dundee_types.h"
#include "star-dundee_annotations.h"
```

#### Data Structures

- struct [CCSDS\\_SPP\\_ENCAPS\\_HEADER](#)  
*A structure used to represent the encapsulation header of a CCSDS/SPP/PUS packet. [More...](#)*
- struct [M\\_PDU\\_PACKET](#)  
*A structure used to represent a M\_PDU packet Note: The headerSet field is a helper variable to assist in keeping track whether the header pointer has been set for a particular M\_PDU packet. [More...](#)*
- struct [CCSDS\\_TF\\_M\\_PDU\\_HEADER](#)  
*A structure used to represent the header of a M\_PDU packet. [More...](#)*
- struct [CCSDS\\_TF\\_PACKET\\_PRIMARY\\_HEADER](#)  
*A structure used to represent the primary header of a transfer frame. [More...](#)*
- struct [CCSDS\\_TF\\_INSERT\\_ZONE](#)  
*A structure used to represent the insert zone field inside a Transfer Frame Note: The insert zone is AOS (Advanced Orbiting Systems) specific field and as such it is optional. [More...](#)*
- struct [CCSDS\\_TF\\_M\\_PDU\\_DATA\\_FIELD](#)  
*A structure used to represent the data field of a M\_PDU packet. [More...](#)*
- struct [CCSDS\\_TF\\_M\\_PDU\\_PACKET](#)  
*A structure used to represent a complete transfer frame that is using M\_PDU packets as payload. [More...](#)*
- struct [CCSDS\\_SPP\\_PACKET\\_PRIMARY\\_HEADER](#)  
*A structure used to represent the primary header of a CCSDS/SPP/PUS packet. [More...](#)*
- struct [CCSDS\\_SPP\\_PACKET\\_DATA\\_FIELD](#)  
*A structure used to represent the data field of a CCSDS/SPP/PUS packet. [More...](#)*
- struct [CCSDS\\_SPP\\_PACKET](#)  
*A structure used to represent the contents of a CCSDS/SPP/PUS packet. [More...](#)*

#### Macros

- [#define M\\_PDU\\_PACKET\\_MAX\\_SIZE \(U32\)\(2040\)](#)  
*The maximum size of an M\_PDU packet as defined in the CCSDS/AOS protocol specification.*

## Enumerations

- enum CCSDS\_SPP\_STATUS {  
CCSDS\_SPP\_STATUS\_SUCCESS = 0, CCSDS\_SPP\_STATUS\_RAW\_DATA\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_STATUS\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_RAW\_PACKET\_LENGTH\_POINTER\_NULL,  
CCSDS\_SPP\_STATUS\_PACKET\_TYPE\_UNKNOWN, CCSDS\_SPP\_STATUS\_APID\_INVALID, CCSDS\_SPP\_STATUS\_SEQUENCE\_FLAGS\_INVALID, CCSDS\_SPP\_STATUS\_DATA\_LENGTH\_INVALID,  
CCSDS\_SPP\_STATUS\_MEMORY\_FAILURE, CCSDS\_SPP\_STATUS\_PACKET\_STRUCTURE\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_VALUE\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_NULL,  
CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INCOMPLETE, CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_LENGTH\_INVALID, CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_LENGTH\_INVALID,  
CCSDS\_SPP\_STATUS\_RECEIVED\_RAW\_DATA\_POINTER\_NULL, CCSDS\_SPP\_STATUS\_PTP\_PROTOCOL\_IDENTIFIER\_INCORRECT, CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INVALID, CCSDS\_SPP\_STATUS\_PTP\_HEADER\_POINTER\_NULL,  
CCSDS\_SPP\_STATUS\_TELECOMMAND\_SECONDARY\_HEADER\_INVALID, CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_FLAG\_PACKET\_TYPE\_CONFLICT, CCSDS\_SPP\_STATUS\_TOO\_MANY\_DEVICE\_REGISTER\_ADDRESSES\_SPECIFIED, CCSDS\_SPP\_STATUS\_BOTH\_SECONDARY\_HEADER\_AND\_USER\_DATA\_FIELD\_MISSING,  
CCSDS\_SPP\_STATUS\_ZERO\_CCSDS\_SPP\_PACKETS\_PROVIDED\_FOR\_TF\_CREATION, CCSDS\_SPP\_STATUS\_PACKET\_LENGTHS\_POINTER\_INVALID, CCSDS\_SPP\_STATUS\_NUMBER\_OF\_CCSDS\_SPP\_PACKETS\_ZERO, CCSDS\_SPP\_STATUS\_NUMBER\_OF\_TF\_PACKETS\_ZERO,  
CCSDS\_SPP\_STATUS\_GENERAL\_ERROR }

Possible values for the status of CCSDS/SPP/TF/PUS related operations.

- enum CCSDS\_SPP\_PACKET\_TYPE {  
CCSDS\_SPP\_PACKET\_TYPE\_TELEMETRY = 0, CCSDS\_SPP\_PACKET\_TYPE\_TELECOMMAND, CCSDS\_SPP\_PACKET\_TYPE\_CPDU\_COMMAND, CCSDS\_SPP\_PACKET\_TYPE\_SPACECRAFT\_TIME\_PACKET,  
CCSDS\_SPP\_PACKET\_TYPE\_IDLE\_PACKET }

Possible values for the packet type of CCSDS/SPP packets Note: This enumeration is internal to this library and the values only partially reflect the TYPE field in the primary header.

## Functions

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_CreatePacket (_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength, _Out_ CCSDS_SPP_STATUS *const pStatus)`

Function used to create the raw packet byte buffer.

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_FillPacket (_Out_opt_ CCSDS_SPP_PACKET *pPacketStruct, _Out_ U8 *const pRawPacket, _In_opt_ const U8 *const pEncapsulationHeader, _In_ const U16 encapsulationHeaderLength, _In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U8 secondaryHeaderFlag, _In_ const U16 APID, _In_ const U8 sequenceFlags, _In_ const U16 sequenceCount, _In_ const U16 dataFieldLength, _In_opt_ const U8 *const pSecondaryHeader, _In_ const U16 secondaryHeaderLength, _In_opt_ const U8 *const pUserDataField, _Out_ U32 *const pRawPacketLength)`

Function used to fill the raw packet byte buffer that is passed into this function.

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_TF_CreateIdlePacket (const U16 packetSize, U32 *const pRawPacketLength, CCSDS_SPP_STATUS *const pStatus)`

Function used to create IDLE packets to fill remaining bytes of M PDU packets of a TF (Transfer Frame).

- `_Ret_maybenull_ U8 **STAR_API_CC CCSDS_TF_CreatePackets (_In_ U8 *const *const pCCSDS_SPP_Packets, _In_ U32 *const pCCSDS_SPP_PacketLengths, _In_ U16 const nrOfCCSDS_SPP_Packets,`

\_In\_opt\_ const U8 \*const pEncapsulationHeader, \_In\_ const U16 encapsulationHeaderLength, \_In\_ const U8 versionNumber, \_In\_ const U8 spaceCraftId, \_In\_ const U8 virtualChannelId, \_Inout\_ U32 \*const pVirtualChannelFrameCount, \_In\_ const U8 replayFlag, \_In\_ const U8 virtualChannelFrameCountUsageFlag, \_Inout\_ U8 const virtualChannelFrameCountCycle, \_In\_ const U8 \*const pFrameHeaderErrorControl, \_In\_opt\_ U8 \*pInsertZoneData, \_In\_ U8 insertZoneDataLength, \_Out\_ U16 \*pNumberOfTFPacketsCreated, \_Out\_ U16 \*pTFPacketLengthBytes, \_In\_opt\_ const U8 \*const pOperationalControlField, \_In\_opt\_ const U8 \*const pFrameErrorControlField, CCSDS\_SPP\_STATUS \*pStatus)

*Function used to create an array of TF packets (in the form of byte arrays each).*

- `_Out_ CCSDS_TF_M_PDU_PACKET STAR_API_CC CCSDS_TF_RetrievePacket` (\_In\_ const U8 \*const pRawPacket, \_In\_ const U16 encapsulationHeaderLength, \_In\_ const U8 frameHeaderErrorControlPresent, \_In\_ const U8 insertZoneDataLength, \_In\_ const U8 operationalControlFieldPresent, \_In\_ const U8 frameErrorControlFieldPresent, \_Out\_ CCSDS\_SPP\_STATUS \*pStatus)

*Function used to "receive"/reconstruct a TF (Transfer Frame) from an array of raw bytes.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetEncapsulationHeader` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ const U8 \*pEncapsulationHeader, \_Out\_ U16 \*const pEncapsulationHeaderLength)

*Function used to retrieve the encapsulation header of a TF packet.*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVersionNumber` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVersionNumber)

*Function used to retrieve the version number of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetSpacecraftID` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pSpacecraftId)

*Function used to retrieve the spacecraft ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVirtualChannelID` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVirtualChannelId)

*Function used to retrieve the virtual channel ID of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVirtualChannelFrameCount` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U32 \*const pVirtualChannelFrameCount)

*Function used to retrieve the virtual channel frame count of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetReplayFlag` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pReplayFlag)

*Function used to retrieve the replay flag of the transfer frame (which is represented by the packet structure passed in as the only parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVCFrameCountUsageFlag` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVCFrameCountUsageFlag)

*Function used to retrieve the virtual channel frame count usage flag of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetVCFrameCountCycle` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*const pVCFrameCountCycle)

*Function used to retrieve the virtual channel frame count cycle of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetFrameHeaderErrorControl` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U16 \*const pFrameHeaderErrorControl)

*Function used to retrieve the frame header error control field of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetInsertZoneData` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ U8 \*pInsertZoneData, \_Out\_ U16 \*const pInsertZoneDataLength)

*Function used to retrieve the insert zone data field (and length) of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)*

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetM_PDU_DataField` (\_In\_ const CCSDS\_TF\_M\_PDU\_PACKET \*const pPacketStruct, \_Out\_ const U8 \*pM\_PDU\_DataField)

Function used to retrieve the data field of the M PDU packet of the transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_Get_M_PDU_FirstHeaderPointer` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pM_PDU_FirstHeaderPointer`)

Function used to retrieve the first header pointer of the M PDU packet of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetOperationalControlField` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U32 *const pOperationalControlField`)

Function used to retrieve the operational control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_TF_M_PDU_GetFrameErrorControlField` (`_In_ const CCSDS_TF_M_PDU_PACKET *const pPacketStruct, _Out_ U16 *const pFrameErrorControlField`)

Function used to retrieve the frame error control field of a transfer frame (which is represented by the packet structure passed in as the only input parameter to this function)

- `U16 STAR_API_CC CCSDS_TF_Get_M_PDU_PacketMaxSize` (`void`)

Function used to retrieve the maximum size of an M PDU packet.

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_CreatePTPHeader` (`_In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte, _Out_ CCSDS_SPP_STATUS *const pStatus`)

Function used to create the PTP protocol header.

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_FillPTPHeader` (`_Out_ U8 *const pHeader, _In_ const U8 targetLogicalAddress, _In_ const U8 reservedByte, _In_ const U8 userApplicationByte`)

Function used to fill the PTP protocol header.

- `CCSDS_SPP_PACKET STAR_API_CC CCSDS_SPP_RetrievePacket` (`_In_ const U8 *const pReceivedRawPacket, _In_ const U8 retrievePTPHeader, _In_ const U16 encapsulationHeaderLength, _Out_ CCSDS_SPP_STATUS *const pStatus`)

Function used to receive/retrieve/interpret a CCSDS/SPP packet.

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_CheckAPID` (`_In_ const CCSDS_SPP_PACKET_TYPE type, _In_ const U16 APID`)

Function used to check the validity/correctness of an APID.

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetDataFieldPointer` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 **pDataField`)

Function used to retrieve the pointer pointing to the data field of a CCSDS/SPP packet (represented by the packet structure that was passed in as a parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetDataFieldLength` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pDataFieldLength`)

Function used to retrieve the length of the data field of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetEncapsulationHeaderPointer` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 **pEncapsulationHeader`)

Function used to retrieve the pointer pointing to the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetEncapsulationHeaderLength` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U16 *const pEncapsulationHeaderLength`)

Function used to retrieve the length of the encapsulation header of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetPacketVersionNumber` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pVersionNumber`)

Function used to retrieve the version number of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

- `CCSDS_SPP_STATUS STAR_API_CC CCSDS_SPP_GetPacketType` (`_In_ const CCSDS_SPP_PACKET *const pPacketStruct, _Out_ U8 *const pPacketType`)

Function used to retrieve the packet type of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)

- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_IsSecondaryHeaderPresent](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pIsSecondaryHeaderPresent)  
*Function used to retrieve the secondary header flag of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetAPID](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U16](#) \*const pAPID)  
*Function used to retrieve the APID of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetSequenceFlags](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pSequenceFlags)  
*Function used to retrieve the sequence flags of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetSequenceCount](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U16](#) \*const pSequenceCount)  
*Function used to retrieve the sequence count of a CCSDS/SPP packet (represented by the packet structure passed in as the only parameter to this function)*
- [U8 STAR\\_API\\_CC CCSDS\\_SPP\\_GetPTPHeaderLengthBytes](#) (void)  
*Function used to retrieve the size of the PTP protocol header.*
- [U8 STAR\\_API\\_CC CCSDS\\_SPP\\_GetPrimaryHeaderLengthBytes](#) (void)  
*Function used to retrieve the size of the primary header of a CCSDS/SPP packet.*
- [U8 STAR\\_API\\_CC CCSDS\\_SPP\\_GetTelecommandSecondaryHeaderLengthBytes](#) (void)  
*Function used to retrieve the size of the secondary header of a Telecommand type CCSDS/SPP packet.*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetPTPTargetLogicalAddress](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pTargetLogicalAddress)  
*Function used to retrieve the PTP target logical address.*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetPTPProtocolIdentifier](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pProtocolIdentifier)  
*Function used to retrieve the PTP protocol identifier byte.*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetPTPReservedByte](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pReservedByte)  
*Function used to retrieve the PTP protocol reserved byte.*
- [CCSDS\\_SPP\\_STATUS STAR\\_API\\_CC CCSDS\\_SPP\\_GetPTPUserApplicationByte](#) ([\\_In\\_](#) const [CCSDS\\_SPP\\_PACKET](#) \*const pPacketStruct, [\\_Out\\_](#) [U8](#) \*const pUserApplicationByte)  
*Function used to retrieve the PTP user application byte.*
- [void STAR\\_API\\_CC CCSDS\\_SPP\\_FreeBuffer](#) ([\\_Inout\\_](#) void \*pBuffer)  
*Function used to free memory that was dynamically allocated by this library.*

### 9.1.1 Detailed Description

Declarations of enums, structures and functions provided by the CCSDS/SPP library.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of enums, structures and declarations of all the functions provided by the CCSDS/SPP packet library

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.2 ccsds\_spp\_pus\_services.h File Reference

Declarations of functions provided by the PUS Services functionality in the CCSDS/SPP library.

```
#include "common.h"
#include "ccsds_spp_packet_library.h"
```

### Functions

- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_PUS_CreateST02DistributeOnOffDeviceCommand`** (`↔`  
`_Out_opt_ CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const`  
`U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ ↔`  
`const U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const`  
`U16 numberOfDeviceAddresses, _In_ const U16 *const pDeviceAddresses, _Out_ CCSDS_SPP_STATUS`  
`*const pStatus, _Out_ U32 *const pRawPacketLength`)  
*Function used to create the ST[02] - Distribute On/Off Device command This is a service defined in the PUS standard.*
- `_Ret_maybenull_ U8 *STAR_API_CC CCSDS_SPP_PUS_CreateST02DistributeRegisterDumpCommand`** (`↔`  
`_Out_opt_ CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const`  
`U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ const`  
`U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const U16`  
`numberOfRegisterAddresses, _In_ const U16 *const pRegisterAddresses, _Out_ CCSDS_SPP_STATUS`  
`*const pStatus, _Out_ U32 *const pRawPacketLength`)  
*Function used to create the ST[02] - Distribute Register Dump command This is a service defined in the PUS standard.*

### 9.2.1 Detailed Description

Declarations of functions provided by the PUS Services functionality in the CCSDS/SPP library.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the declarations of functions for PUS Services provided by the CCSDS/SPP packet library.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.2.2 Function Documentation

- 9.2.2.1 **`_Ret_maybenull_ U8* STAR_API_CC CCSDS_SPP_PUS_CreateST02DistributeOnOffDeviceCommand`** (`↔`  
`_Out_opt_ CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const`  
`U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ const`  
`U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const U16`  
`numberOfDeviceAddresses, _In_ const U16 *const pDeviceAddresses, _Out_ CCSDS_SPP_STATUS *const`  
`pStatus, _Out_ U32 *const pRawPacketLength` )

Function used to create the ST[02] - Distribute On/Off Device command This is a service defined in the PUS standard.



## Parameters

|                                |                                                                                                 |
|--------------------------------|-------------------------------------------------------------------------------------------------|
| <i>pPacketStruct</i>           | pointer to a variable holding the structure information of the CCSDS/SPP packet                 |
| <i>targetLogicalAddress</i>    | a variable containing the logical address of the target device                                  |
| <i>protocolReservedByte</i>    | a variable containing the PTP protocol reserved byte                                            |
| <i>userApplicationByte</i>     | variable containing the user application byte of the PTP protocol header                        |
| <i>destinationAPID</i>         | variable holding the APID of the destination device or process                                  |
| <i>sequenceCount</i>           | variable holding the value of the sequence count                                                |
| <i>acknowledgementFlags</i>    | variable holding the acknowledgement flags                                                      |
| <i>sourceAPID</i>              | variable holding the APID of the source device or process                                       |
| <i>numberOfDeviceAddresses</i> | variable holding the number of device addresses                                                 |
| <i>pDeviceAddresses</i>        | a pointer to the buffer holding the device addresses                                            |
| <i>pStatus</i>                 | a pointer to a variable holding the status of the previous, present and future operations       |
| <i>pRawPacketLength</i>        | a pointer to a variable that is populated by this function holding the length of the raw packet |

## Returns

a pointer to a buffer containing the raw packet bytes of the created CCSDS/SPP packet

## Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

## Examples:

`ccsds_spp_tf_examples.c`.

```
9.2.2.2 _Ret_maybenull_ U8* STAR_API_CC CCSDS_SPP_PUS_CreateST02DistributeRegisterDumpCommand (
 _Out_opt_ CCSDS_SPP_PACKET *const pPacketStruct, _In_ const U8 targetLogicalAddress, _In_ const
 U8 protocolReservedByte, _In_ const U8 userApplicationByte, _In_ const U16 destinationAPID, _In_ const
 U16 sequenceCount, _In_ const U8 acknowledgementFlags, _In_ const U16 sourceAPID, _In_ const U16
 numberOfRegisterAddresses, _In_ const U16 *const pRegisterAddresses, _Out_ CCSDS_SPP_STATUS *const
 pStatus, _Out_ U32 *const pRawPacketLength)
```

Function used to create the ST[02] - Distribute Register Dump command This is a service defined in the PUS standard.

## Parameters

|                             |                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------|
| <i>pPacketStruct</i>        | pointer to a variable holding the structure information of the CCSDS/SPP packet |
| <i>targetLogicalAddress</i> | a variable containing the logical address of the target device                  |
| <i>protocolReservedByte</i> | a variable containing the PTP protocol reserved byte                            |

|                          |                                                                                                 |
|--------------------------|-------------------------------------------------------------------------------------------------|
| <i>user</i> ↔            | variable containing the user application byte of the PTP protocol header                        |
| <i>ApplicationByte</i>   |                                                                                                 |
| <i>destinationAPID</i>   | variable holding the APID of the destination device or process                                  |
| <i>sequenceCount</i>     | variable holding the value of the sequence count                                                |
|                          | variable holding the acknowledgement flags                                                      |
| <i>acknowledgement</i> ↔ |                                                                                                 |
| <i>Flags</i>             |                                                                                                 |
| <i>sourceAPID</i>        | variable holding the APID of the source device or process                                       |
| <i>numberOf</i> ↔        | variable holding the number of register addresses                                               |
| <i>Register</i> ↔        |                                                                                                 |
| <i>Addresses</i>         |                                                                                                 |
| <i>pRegister</i> ↔       | a pointer to the buffer holding the register addresses                                          |
| <i>Addresses</i>         |                                                                                                 |
| <i>pStatus</i>           | a pointer to a variable holding the status of the previous, present and future operations       |
| <i>pRawPacket</i> ↔      | a pointer to a variable that is populated by this function holding the length of the raw packet |
| <i>Length</i>            |                                                                                                 |

### Returns

a pointer to a buffer containing the raw packet bytes of the created CCSDS/SPP packet

### Added in version:

STAR-System v5.00 Beta 1 - 24th March 2021

## 9.3 cfg\_api\_brick\_mk2.h File Reference

Functions used to configure STAR-Dundee Brick Mk2 and compatible devices.

```
#include "star-api.h"
#include "cfg_api_brick_mk2_types.h"
```

### Functions

- int `STAR_API_CC_CFG_BRICK_MK2_enableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Selectively enables interface mode on a given port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_getInterfaceModeOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether interface mode is enabled on a specific port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_disableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Selectively disables interface mode on a given port of a supported USB device.*
- int `STAR_API_CC_CFG_BRICK_MK2_enableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Enables identification of the source port of a received packet on a given port, when in interface mode.*
- int `STAR_API_CC_CFG_BRICK_MK2_getIdentifySourceOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether identify source is enabled for a specific port.*
- int `STAR_API_CC_CFG_BRICK_MK2_disableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceId, U8 port↔Num)  
*Disables source port identification for a given port.*
- int `STAR_API_CC_CFG_BRICK_MK2_setPortRoutingAddress` (STAR\_DEVICE\_ID deviceId, U8 portNum, U8 address)



- Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_getPortRoutingAddress (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U8 \*pAddress)
- Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_setLinkClockFrequency (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ linkFreq)
- Sets the Link Clock Frequency on a given link.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_getLinkClockFrequency (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ \*pLinkFreq)
- Gets the Link Clock Frequency for a given link.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_injectErrors (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_BRICK\_MK2\_ERRORS \*pErrors)
- Inject the specified errors on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_enableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)
- Selectively enables state change events on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_getStateChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)
- Gets whether state change events are enabled on a specific port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_disableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)
- Selectively disables state change events on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_enableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)
- Selectively enables speed change events on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_getSpeedChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)
- Gets whether speed change events are enabled on a specific port.*
- int STAR\_API\_CC CFG\_BRICK\_MK2\_disableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)
- Selectively disables speed change events on a given port.*

### 9.3.1 Detailed Description

Functions used to configure STAR-Dundee Brick Mk2 and compatible devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.4 cfg\_api\_brick\_mk2\_types.h File Reference

Types used with the STAR-Dundee Brick Mk2 Configuration API.

## Data Structures

- struct `STAR_CFG_BRICK_MK2_ERRORS`

Structure used to indicate which errors should be generated when calling `CFG_BRICK_MK2_injectErrors()`. [More...](#)

## Macros

- #define `STAR_CFG_BRICK_MK2_LINK_SPEED_UNITS_KBPS` 200

The units in Kbit/s used to represent the link speed returned when calling `CFG_BRICK_MK2_getMeasuredLinkSpeed()`.

## Enumerations

- enum `STAR_CFG_BRICK_MK2_LINK_FREQ` {  
`STAR_CFG_BRICK_MK2_LINK_FREQ_120 = 0, STAR_CFG_BRICK_MK2_LINK_FREQ_130 = 1, STAR_CFG_BRICK_MK2_LINK_FREQ_140 = 2, STAR_CFG_BRICK_MK2_LINK_FREQ_150 = 3,`  
`STAR_CFG_BRICK_MK2_LINK_FREQ_160 = 4, STAR_CFG_BRICK_MK2_LINK_FREQ_180 = 5, STAR_CFG_BRICK_MK2_LINK_FREQ_200 = 6 }`

Frequencies a link on a Brick Mk2 compatible device may run at.

### 9.4.1 Detailed Description

Types used with the STAR-Dundee Brick Mk2 Configuration API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.4.2 Data Structure Documentation

#### 9.4.2.1 struct `STAR_CFG_BRICK_MK2_ERRORS`

Structure used to indicate which errors should be generated when calling `CFG_BRICK_MK2_injectErrors()`.

If the element is set to a non-zero value, an error of that type will be injected.

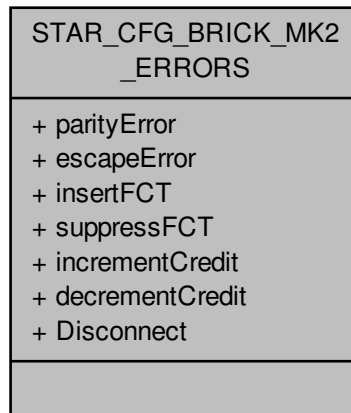
#### Added in version:

STAR-System v2.0 Beta 1 - 8th February 2013

#### Examples:

`brickMk2_configuration_example.c`.

Collaboration diagram for STAR\_CFG\_BRICK\_MK2\_ERRORS:



#### Data Fields

|      |                 |                                                                                                                                                                                                        |
|------|-----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| char | decrementCredit | Whether the transmit credit should be decremented by 1. This can be used to reduce the rate at which data can be transmitted.                                                                          |
| char | Disconnect      | Whether a disconnect error should occur. This causes the link to enter ErrorReset and the Data/Strobe outputs transition LOW.<br><br><b>Added in version:</b><br><br>STAR-System v3.6 - 3rd April 2017 |
| char | escapeError     | Whether an escape error (two escape characters in sequence) should be injected.                                                                                                                        |
| char | incrementCredit | Whether the transmit credit available to send data should be incremented by 8. This can be used to force a data overflow in the opposite end of the link, as too much data will be transmitted.        |
| char | insertFCT       | Whether an extra FCT should be sent. This can be used to force FCT errors on the SpaceWire link.                                                                                                       |
| char | parityError     | Whether a parity error should be injected on the next SpaceWire character to be transmitted by flipping the parity bit.                                                                                |
| char | suppressFCT     | Whether the next FCT character to be transmitted on the link should be (suppressed). This can be used to simulate the loss of an FCT character.                                                        |

### 9.4.3 Macro Definition Documentation

#### 9.4.3.1 #define STAR\_CFG\_BRICK\_MK2\_LINK\_SPEED\_UNITS\_KBPS 200

The units in Kbit/s used to represent the link speed returned when calling CFG\_BRICK\_MK2\_getMeasuredLinkSpeed().

**Added in version:**

STAR-System v2.0 Beta 1 - 8th February 2013

## 9.5 cfg\_api\_brick\_mk3.h File Reference

Functions used to configure STAR-Dundee Brick Mk3 and compatible devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
#include "cfg_api_router.h"
```

### Functions

- int STAR\_API\_CC CFG\_BRICK\_MK3\_setBaseTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the base transmit clock rate on a given link of a Brick Mk3 device:*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getBaseTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, ↵ \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
*Gets the base transmit clock rate parameters for a given link of a Brick Mk3 device.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_setTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, STA↵ R\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the transmit clock rate on a given link.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getTransmitClock (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)  
*Gets the transmit clock rate parameters for a given link.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_setTimestampMethod (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_↵ BRICK\_MK3\_TIMESTAMP\_METHOD timestampMethod)  
*Sets the timestamping method to use for the given device.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getTimestampMethod (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_↵ \_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD \*pTimestampMethod)  
*Gets the timestamping method that is currently in use for the given device.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_enableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively enables receive timestamp events on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getRxTimestampEventsOnPortEnabled (STAR\_DEVICE\_ID device↵ ID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether receive timestamp events are enabled on a specific port.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_disableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively disables receive timestamp events on a given port.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_setTimestampValue (STAR\_DEVICE\_ID deviceId, U32 value)  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getTimestampValue (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*p↵ Value)  
*Gets the current timestamp value.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_setPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, STAR\_↵ \_CFG\_BRICK\_MK3\_PULSE\_FREQ frequency)  
*Sets the frequency of each generated pulse.*
- int STAR\_API\_CC CFG\_BRICK\_MK3\_getPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, \_Out\_ ↵ STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ \*pFrequency)  
*Gets the frequency of each generated pulse.*

### 9.5.1 Detailed Description

Functions used to configure STAR-Dundee Brick Mk3 and compatible devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.6 `cfg_api_brick_mk3_types.h` File Reference

Types used with the STAR-Dundee Brick Mk3 Configuration API.

### Enumerations

- enum `STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD` { `STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE` = 0, `STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER` = 1, `STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR` = 2 }  
*Timestamps can be generated by either an external trigger signal or by an internal pulse generator.*
- enum `STAR_CFG_BRICK_MK3_PULSE_FREQ` { `STAR_CFG_BRICK_MK3_PULSE_FREQ_1` = 0, `STAR_CFG_BRICK_MK3_PULSE_FREQ_10` = 1, `STAR_CFG_BRICK_MK3_PULSE_FREQ_100` = 2, `STAR_CFG_BRICK_MK3_PULSE_FREQ_1000` = 3 }  
*Defines frequencies that can be used by the global pulse generator.*

### 9.6.1 Detailed Description

Types used with the STAR-Dundee Brick Mk3 Configuration API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.7 `cfg_api_gbe_brick_mk1.h` File Reference

Declaration of the functions provided by the STAR-Dundee GbE Brick Mk1 Configuration API.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
#include "cfg_api_router.h"
```

## Functions

- int `STAR_API_CC_CFG_GBE_BRICK_MK1_setTransmitSignallingRate` (STAR\_DEVICE\_ID deviceId, U8 linkNum, U32 signallingRate)  
*Sets the transmit bit rate on a given link.*
- int `STAR_API_CC_CFG_GBE_BRICK_MK1_getTransmitSignallingRate` (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U32 \*pSignallingRate)  
*Gets the transmit bit rate for a given link.*

### 9.7.1 Detailed Description

Declaration of the functions provided by the STAR-Dundee GbE Brick Mk1 Configuration API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.8 cfg\_api\_generic.h File Reference

Types used with the STAR-Dundee Generic Configuration API.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_mk2.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_brick_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
#include "cfg_api_generic_common.h"
```

## Macros

- #define `CFG_STATUS_SUCCESS` (0)  
*Positive retvals are success, negative retvals are errors.*
- #define `CFG_SUCCESS(status)` ((status) >= `CFG_STATUS_SUCCESS` ? 1 : 0)  
*Macro to allow testing for success or failure.*
- #define `CFG_STATUS_FAILURE` (-256)  
*Return values -1 to -255 are reserved for RMAP status (inverted from +ve to -ve)*
- #define `CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICE_TYPE` (-257)

- The API call is not compatible with this device type.*

  - #define `CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICES_FPGA_VERSION` (-258)
- The API call is not compatible with this device's FPGA version.*

  - #define `CFG_STATUS_GET_DEVICE_INFO_FAILURE` (-259)
- Failed to get "Device Information" for this device.*

  - #define `CFG_STATUS_TIME_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID` (-260)
- The Time-code distribution port mask is invalid.*

  - #define `CFG_STATUS_TIME_CODE_FLAG_MODE_IS_INVALID` (-261)
- The Time-code flag mode is invalid.*

  - #define `CFG_STATUS_TIME_CODE_PERIOD_IS_INVALID` (-262)
- The Time-code period is invalid.*

  - #define `CFG_STATUS_TIME_CODE_PORT_IS_INVALID` (-263)
- The Time-code port is invalid.*

  - #define `CFG_STATUS_IDENTIFY_SOURCE_PORT_IS_INVALID` (-264)
- The Identify Source port is invalid.*

  - #define `CFG_STATUS_INTERFACE_MODE_PORT_IS_INVALID` (-265)
- The Interface Mode port is invalid.*

  - #define `CFG_STATUS_PORT_IS_INVALID` (-266)
- The port is invalid.*

  - #define `CFG_STATUS_LOGICAL_ADDRESS_IS_INVALID` (-267)
- The logical address is invalid.*

  - #define `CFG_STATUS_SPACEWIRE_LINK_IS_INVALID` (-268)
- The SpaceWire link is invalid.*

  - #define `CFG_STATUS_MEASURED_SPEED_PORT_IS_INVALID` (-269)
- The measured port speed is invalid.*

  - #define `CFG_STATUS_SPEED_CHANGE_EVENTS_PORT_IS_INVALID` (-270)
- The Speed Change Events port is invalid.*

  - #define `CFG_STATUS_STATE_CHANGE_EVENTS_PORT_IS_INVALID` (-271)
- The State Change Events port is invalid.*

  - #define `CFG_STATUS_PORT_ROUTING_PORT_IS_INVALID` (-272)
- The Port Routing port is invalid.*

  - #define `CFG_STATUS_INJECT_ERROR_PORT_IS_INVALID` (-273)
- The Inject Error port is invalid.*

  - #define `CFG_STATUS_PRECISION_TRANSMIT_RATE_IS_INVALID` (-274)
- The Precision Transmit Rate port is invalid.*

  - #define `CFG_STATUS_PERIODIC_ACTION_PORT_IS_INVALID` (-275)
- The Periodic Action port is invalid.*

  - #define `CFG_STATUS_TIMESTAMP_EVENTS_PORT_IS_INVALID` (-276)
- The Timestamp Events port is invalid.*

  - #define `CFG_STATUS_CLOCK_LINK_IS_INVALID` (-277)
- The Clock link is invalid.*

  - #define `CFG_STATUS_LINK_RATE_DIVIDER_IS_INVALID` (-278)
- The Link Rate divider is invalid.*

  - #define `CFG_STATUS_TRANSMIT_CLOCK_LINK_IS_INVALID` (-279)
- The Transmit Clock link is invalid.*

  - #define `CFG_STATUS_BIT_RATE_IS_INVALID` (-280)
- The Bit-rate is invalid.*

  - #define `CFG_STATUS_ROUTER_TIMEOUT_MODE_IS_INVALID` (-281)
- The Router Timeout mode is invalid.*

  - #define `CFG_STATUS_ROUTER_TIMEOUT_PERIOD_IS_INVALID` (-282)
- The Router Timeout period is invalid.*

- `#define CFG_STATUS_ROUTER_DISABLE_ON_SILENCE_IS_INVALID (-283)`  
*The Router Disable-on-Silence is invalid.*
- `#define CFG_STATUS_ROUTER_START_ON_REQUEST_IS_INVALID (-284)`  
*The Router Start-on-Request is invalid.*
- `#define CFG_STATUS_ROUTER_ENABLE_SELF_ADDRESSING_IS_INVALID (-285)`  
*The Router Enable-Self-Addressing is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PORT_MASK_IS_INVALID (-286)`  
*The Group Adaptive Routing port mask is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_PRIORITY_IS_INVALID (-287)`  
*The Group Adaptive Routing priority is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_DELETE_HEADER_IS_INVALID (-288)`  
*The Group Adaptive Routing delete header is invalid.*
- `#define CFG_STATUS_GROUP_ADAPTIVE_ROUTING_ADDRESS_IS_INVALID (-289)`  
*The Group Adaptive Routing address is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_TRISTATE_IS_INVALID (-290)`  
*The SpaceWire link tri-state flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_DISABLE_IS_INVALID (-291)`  
*The SpaceWire link disable flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_START_IS_INVALID (-292)`  
*The SpaceWire link start flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_AUTOSTART_IS_INVALID (-293)`  
*The SpaceWire link autostart flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_RUNNING_IS_INVALID (-294)`  
*The SpaceWire link running flag is invalid.*
- `#define CFG_STATUS_SPACEWIRE_LINK_STATE_IS_INVALID (-295)`  
*The SpaceWire link state is invalid.*
- `#define CFG_STATUS_PORT_ROUTING_ADDRESS_IS_INVALID (-296)`  
*The Port Routing address is invalid.*
- `#define CFG_STATUS_INJECT_ERROR_ERROR_IS_INVALID (-297)`  
*The Inject Error error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_PARITY_ERROR_IS_INVALID (-298)`  
*The Inject Errors parity error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_ESCAPE_ERROR_IS_INVALID (-299)`  
*The Inject Errors escape error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_INSERT_FCT_ERROR_IS_INVALID (-300)`  
*The Inject Errors insert FCT error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_SUPPRESS_FCT_ERROR_IS_INVALID (-301)`  
*The Inject Errors suppress FCT error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_INCREMENT_CREDIT_ERROR_IS_INVALID (-302)`  
*The Inject Errors increment credit error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_DECREMENT_CREDIT_ERROR_IS_INVALID (-303)`  
*The Inject Errors decrement credit error is invalid.*
- `#define CFG_STATUS_INJECT_ERRORS_DISCONNECT_ERROR_IS_INVALID (-304)`  
*The Inject Errors disconnect error is invalid.*
- `#define CFG_STATUS_PERIODIC_ACTION_ACTION_IS_INVALID (-305)`  
*The Periodic Action is invalid.*
- `#define CFG_STATUS_TIMESTAMP_METHOD_IS_INVALID (-306)`  
*The Timestamp method is invalid.*
- `#define CFG_STATUS_PULSE_GENERATOR_FREQUENCY_IS_INVALID (-307)`  
*The Pulse Generator frequency is invalid.*
- `#define CFG_STATUS_CLOCK_RATE_MULTIPLIER_IS_INVALID (-308)`



- The Clock Rate multiplier is invalid.*
- `#define CFG_STATUS_CLOCK_RATE_DIVISOR_IS_INVALID (-309)`
- The Clock Rate divisor is invalid.*
- `#define CFG_STATUS_CLOCK_RATE_PARAMETERS_ARE_INVALID (-310)`
- The Clock Rate parameters are invalid.*
- `#define CFG_STATUS_CANT_GET_CLOCK_RATE_PARAMETERS_FROM_BIT_RATE (-311)`
- Can't get Clock Rate parameters from bit-rate.*
- `#define CFG_STATUS_POINTER_PARAMETER_IS_NULL (-312)`
- The pointer parameter is NULL.*
- `#define CFG_STATUS_TRANSMIT_BIT_RATE_IS_INVALID (-313)`
- The Transmit bit-rate is invalid.*
- `#define CFG_STATUS_INTERRUPT_CODE_DISTRIBUTION_PORT_MASK_IS_INVALID (-314)`
- The interrupt code distribution port mask is invalid.*
- `#define CFG_STATUS_GET_FPGA_INFO_FAILURE (-315)`
- Could not get FPGA information from device.*
- `#define CFG_STATUS_GET_HARDWARE_INFO_FAILURE (-316)`
- Could not get hardware information from device.*
- `#define CFG_STATUS_ADD_DEVICE_INFO_SHARED_MEMORY_FAILURE (-317)`
- Could not add device information to shared memory.*

## Functions

- `int STAR_API_CC CFG_updateDeviceInfo (STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC CFG_refreshDeviceInfo (STAR_DEVICE_ID deviceId)`
- This function is used to update a device's information held in the shared memory.*
- `int STAR_API_CC CFG_getRouterGlobalSettings (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`
- Gets the router's global settings.*
- `int STAR_API_CC CFG_setRouterGlobalSettings (STAR_DEVICE_ID deviceId, _In_ STAR_CFG_ROUTER_GLOBAL_STATE *pState)`
- Sets the router's global settings.*
- `int STAR_API_CC CFG_getNetworkIdentity (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`
- Gets the value of the network identity register.*
- `int STAR_API_CC CFG_setNetworkIdentity (STAR_DEVICE_ID deviceId, REGISTER value)`
- Sets the value of the network identity register.*
- `int STAR_API_CC CFG_getGeneralPurpose (STAR_DEVICE_ID deviceId, _Out_ REGISTER *pValue)`
- Gets the value of the general purpose register.*
- `int STAR_API_CC CFG_setGeneralPurpose (STAR_DEVICE_ID deviceId, REGISTER value)`
- Sets the value of the general purpose register.*
- `int STAR_API_CC CFG_getFPGAInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_FPGA_INFO *pFPGAInfo)`
- Reads information about the hardware version of a device and updates the STAR\_CFG\_FPGA\_INFO structure pointed to by the passed in pointer to contain the received version information.*
- `void STAR_API_CC CFG_FPGAInfoToString (STAR_DEVICE_ID deviceId, _In_ STAR_CFG_FPGA_INFO *pFPGAInfo, _Out_z_cap_c_ (STAR_CFG_MK2_VERSION_STR_MAX_LEN) char *pVersion, _Out_z_cap_c_ (STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN) char *pBuildDate)`
- Creates a string representation of a STAR\_CFG\_FPGA\_INFO structure.*
- `int STAR_API_CC CFG_identify (STAR_DEVICE_ID deviceId)`
- Flashes the front panel LEDs of a device.*
- `int STAR_API_CC CFG_clearPortErrors (STAR_DEVICE_ID deviceId, U8 portNum)`
- Clears all errors on a given port.*

- `int STAR_API_CC_CFG_getConfigPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_CONFIG_PORT_ERRORS *pErrors)`  
*Gets any errors present on the config port.*
- `int STAR_API_CC_CFG_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceId, U8 linkNum, _In_ STAR_CFG_SPW_LINK_STATUS *pLinkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC_CFG_startLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC_CFG_stopLink (STAR_DEVICE_ID deviceId, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC_CFG_getSpaceWireLinkErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_ERRORS *pErrors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC_CFG_getSpaceWireLinkStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_SPW_LINK_STATUS *pStatus)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC_CFG_getPortStatusControl (STAR_DEVICE_ID deviceId, U8 portNum, _Out_ PORT_STATUS_CONTROL *pPortStatusControl)`  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- `STAR_CFG_PORT_TYPE STAR_API_CC_CFG_getPortType (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the type of port attributed to a given port number.*
- `int STAR_API_CC_CFG_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *pErrors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *pStatus)`  
*Gets the status of an external port.*
- `U8 STAR_API_CC_CFG_getPortConnection (PORT_STATUS_CONTROL portStatusControl)`  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- `int STAR_API_CC_CFG_getNetworkDiscoveryInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_NETWORK_DISCOVERY_INFO *pInfo)`  
*Reads the network discovery information of a device.*
- `int STAR_API_CC_CFG_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pInfo)`  
*Reads the device identifier information of a device, i.e.*
- `void STAR_API_CC_CFG_getDeviceManufacturerAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- `void STAR_API_CC_CFG_getDeviceTypeAsString (_In_ STAR_CFG_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo, _Out_z_cap_c_ (STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC_CFG_getTimeCodeDistributionCapablePorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports are capable of forwarding time-codes on.*
- `int STAR_API_CC_CFG_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceId, U32 portMask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_getTimeCodeFlagMode (STAR_DEVICE_ID deviceId, _Out_ U8 *pMode)`

- Obtains the time-code flag interpretation mode:*

  - int `STAR_API_CC_CFG_setTimeCodeFlagMode` (STAR\_DEVICE\_ID deviceID, U8 mode)

*Sets the time-code flag interpretation mode:*

  - int `STAR_API_CC_CFG_getTimeCodeValue` (STAR\_DEVICE\_ID deviceID, \_Out\_ U8 \*pValue)

*Obtains the current value of the router's internal time-code counter.*

  - int `STAR_API_CC_CFG_getRoutingTableEntry` (STAR\_DEVICE\_ID deviceID, U8 logicalAddress, \_Out\_ S↔  
TAR\_CFG\_GAR\_ENTRY \*pEntry)

*Gets a routing table entry for a given logical address.*

  - int `STAR_API_CC_CFG_setRoutingTableEntry` (STAR\_DEVICE\_ID deviceID, U8 logicalAddress, \_In\_ ST↔  
AR\_CFG\_GAR\_ENTRY \*pEntry)

*Sets a routing table entry for a given logical address.*

  - int `STAR_API_CC_CFG_getPortRoutingAddress` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U8 \*p↔  
Address)

*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*

  - int `STAR_API_CC_CFG_setPortRoutingAddress` (STAR\_DEVICE\_ID deviceID, U8 portNum, U8 address)

*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*

  - int `STAR_API_CC_CFG_disableIdentifySource` (STAR\_DEVICE\_ID deviceID)

*Disables identification of source ports for interface mode globally.*

  - int `STAR_API_CC_CFG_enableIdentifySource` (STAR\_DEVICE\_ID deviceID)

*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*

  - int `STAR_API_CC_CFG_getIdentifySourceEnabled` (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)

*Gets whether identify source is enabled.*

  - int `STAR_API_CC_CFG_disableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Disables source port identification for a given port.*

  - int `STAR_API_CC_CFG_enableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Enables identification of source port during interface mode for a given port.*

  - int `STAR_API_CC_CFG_getIdentifySourceOnPortEnabled` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_↔  
Out\_ int \*pEnabled)

*Gets whether identify source is enabled for a specific port.*

  - int `STAR_API_CC_CFG_disableInterfaceMode` (STAR\_DEVICE\_ID deviceID)

*Disables interface mode on a device.*

  - int `STAR_API_CC_CFG_enableInterfaceMode` (STAR\_DEVICE\_ID deviceID)

*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_setPortRoutingAddress()`).*

  - int `STAR_API_CC_CFG_getInterfaceModeEnabled` (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)

*Gets whether interface mode has been enabled on a device.*

  - int `STAR_API_CC_CFG_disableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables interface mode on a given port.*

  - int `STAR_API_CC_CFG_enableInterfaceModeOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables interface mode on a given port.*

  - int `STAR_API_CC_CFG_getInterfaceModeOnPortEnabled` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_↔  
Out\_ int \*pEnabled)

*Gets whether interface mode is enabled on a specific port.*

  - int `STAR_API_CC_CFG_disableTimeCodeMaster` (STAR\_DEVICE\_ID deviceID)

*Disables the device as a time-code master.*

  - int `STAR_API_CC_CFG_enableTimeCodeMaster` (STAR\_DEVICE\_ID deviceID)

*Enables the device as a time-code master.*

  - int `STAR_API_CC_CFG_getTimeCodeMasterEnabled` (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)

*Gets whether the device has been enabled as a time-code master.*

  - int `STAR_API_CC_CFG_disableExternalTimeCodeSelection` (STAR\_DEVICE\_ID deviceID)

*Disables external time-code selection for the device.*

- int [STAR\\_API\\_CC\\_CFG\\_enableExternalTimeCodeSelection](#) (STAR\_DEVICE\_ID deviceID)  
*Enables external time-code selection for the device.*
- int [STAR\\_API\\_CC\\_CFG\\_getExternalTimeCodeSelectionEnabled](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)  
*Gets whether external time-code selection has been enabled for the device.*
- int [STAR\\_API\\_CC\\_CFG\\_setTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceID, U32 period)  
*Sets the period between time-code master ticks.*
- int [STAR\\_API\\_CC\\_CFG\\_getTimeCodePeriod](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pPeriod)  
*Gets the period between time-code master ticks.*
- int [STAR\\_API\\_CC\\_CFG\\_disableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceID)  
*Disables time-code counter bypass mode for the device.*
- int [STAR\\_API\\_CC\\_CFG\\_enableTimeCodeCounterBypassMode](#) (STAR\_DEVICE\_ID deviceID)  
*Enables time-code counter bypass mode for the device.*
- int [STAR\\_API\\_CC\\_CFG\\_getTimeCodeCounterBypassModeEnabled](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)  
*Gets whether time-code counter bypass mode has been enabled for the device.*
- int [STAR\\_API\\_CC\\_CFG\\_disableSpeedChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables speed change events on a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_enableSpeedChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables speed change events on a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_getSpeedChangeEventsOnPortEnabled](#) (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether speed change events are enabled on a specific port.*
- int [STAR\\_API\\_CC\\_CFG\\_disableStateChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables state change events on a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_enableStateChangeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables state change events on a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_getStateChangeEventsOnPortEnabled](#) (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether state change events are enabled on a specific port.*
- int [STAR\\_API\\_CC\\_CFG\\_disableTimeCodeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively disables time-code notifications on a the channel attached to a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_enableTimeCodeEventsOnPort](#) (STAR\_DEVICE\_ID deviceID, U8 portNum)  
*Selectively enables time-code notifications on a the channel attached to a given port.*
- int [STAR\\_API\\_CC\\_CFG\\_getTimeCodeEventsOnPortEnabled](#) (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether time-code notifications are enabled on a specific port.*
- int [STAR\\_API\\_CC\\_CFG\\_getMeasuredLinkSpeed](#) (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U16 \*pLinkSpeed)
- int [STAR\\_API\\_CC\\_CFG\\_getRxSignallingRate](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ double \*pSignallingRate)  
*Gets the receive bit rate for a given link in Mbit/s.*
- int [STAR\\_API\\_CC\\_CFG\\_getTransmitSignallingRate](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ U32 \*pSignallingRate)
- int [STAR\\_API\\_CC\\_CFG\\_getTxSignallingRate](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ double \*pSignallingRate)  
*Gets the transmit bit rate for a given link in Mbit/s.*
- int [STAR\\_API\\_CC\\_CFG\\_setTransmitSignallingRate](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, U32 signallingRate)
- int [STAR\\_API\\_CC\\_CFG\\_setTxSignallingRate](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, double signallingRate, \_Out\_ double \*pSignallingRateSet)  
*Sets the transmit bit rate on a given link.*

- int STAR\_API\_CC\_CFG\_disablePrecisionTransmitRate (STAR\_DEVICE\_ID deviceId)  
*Disables precision transmit rate.*
- int STAR\_API\_CC\_CFG\_enablePrecisionTransmitRate (STAR\_DEVICE\_ID deviceId)  
*Enables precision transmit rate.*
- int STAR\_API\_CC\_CFG\_getPrecisionTransmitRateEnabled (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pEnabled)  
*Determine if precision transmit rate is enabled.*
- int STAR\_API\_CC\_CFG\_getPrecisionTransmitRateInUse (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pInUse)  
*Determine whether precision transmit rate is in use.*
- int STAR\_API\_CC\_CFG\_getPrecisionTransmitRate (STAR\_DEVICE\_ID deviceId, \_Out\_ double \*pTransmitRate)  
*Get the current precision transmit rate on a device in Mbit/s.*
- int STAR\_API\_CC\_CFG\_setPrecisionTransmitRate (STAR\_DEVICE\_ID deviceId, double transmitRate)  
*Set the current precision transmit rate on a device in Mbit/s.*
- int STAR\_API\_CC\_CFG\_injectError (STAR\_DEVICE\_ID deviceId, U8 portNum, SPW\_ERROR error)  
*Immediately injects the specified error on a given device's port.*
- int STAR\_API\_CC\_CFG\_injectErrors (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_BRICK\_MK2\_ERRORS \*pErrors)  
*Inject the specified errors on a given port.*
- int STAR\_API\_CC\_CFG\_disableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively disables receive timestamp events on a given port.*
- int STAR\_API\_CC\_CFG\_enableRxTimestampEventsOnPort (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively enables receive timestamp events on a given port.*
- int STAR\_API\_CC\_CFG\_getRxTimestampEventsOnPortEnabled (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether receive timestamp events are enabled on a specific port.*
- int STAR\_API\_CC\_CFG\_getPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ \*pFrequency)  
*Gets the frequency of each generated pulse.*
- int STAR\_API\_CC\_CFG\_setPulseGeneratorFrequency (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ frequency)  
*Sets the frequency of each generated pulse.*
- int STAR\_API\_CC\_CFG\_getTimestampMethod (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD \*pMethod)  
*Gets the timestamping method that is currently in use for the given device.*
- int STAR\_API\_CC\_CFG\_setTimestampMethod (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD method)  
*Sets the timestamping method to use for the given device.*
- int STAR\_API\_CC\_CFG\_getTimestampValue (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*pValue)  
*Gets the current timestamp value.*
- int STAR\_API\_CC\_CFG\_setTimestampValue (STAR\_DEVICE\_ID deviceId, U32 value)  
*Sets the current timestamp value for the given device which will be incremented on the next synchronisation pulse.*
- int STAR\_API\_CC\_CFG\_getDeviceClockRate (STAR\_DEVICE\_ID deviceId)  
*Gets a device's internal clock rate.*
- int STAR\_API\_CC\_CFG\_startPeriodicAction (STAR\_DEVICE\_ID deviceId, unsigned char port, U32 count, SPW\_ACTION action)  
*Starts a periodic action on a given device's port.*
- int STAR\_API\_CC\_CFG\_stopPeriodicAction (STAR\_DEVICE\_ID deviceId, unsigned char port)  
*Stops a periodic action on a given device's port.*
- int STAR\_API\_CC\_CFG\_disableBroadcastControllerLegacyMode (STAR\_DEVICE\_ID deviceId)  
*Disables broadcast controller legacy mode.*
- int STAR\_API\_CC\_CFG\_enableBroadcastControllerLegacyMode (STAR\_DEVICE\_ID deviceId)

*Enables broadcast controller legacy mode.*

- int [STAR\\_API\\_CC\\_CFG\\_getBroadcastControllerLegacyModeEnabled](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ int \*pEnabled)

*Gets whether broadcast legacy mode has been enabled for the device.*

- int [STAR\\_API\\_CC\\_CFG\\_getInterruptCodeDistributionPorts](#) (STAR\_DEVICE\_ID deviceID, \_Out\_ U32 \*pPortMask)

*This function is used to obtain which output ports interrupt codes are forwarded on.*

- int [STAR\\_API\\_CC\\_CFG\\_setInterruptCodeDistributionPorts](#) (STAR\_DEVICE\_ID deviceID, U32 portMask)

*This function is used to specify which output ports interrupt codes are forwarded on.*

### 9.8.1 Detailed Description

Types used with the STAR-Dundee Generic Configuration API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2025 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.8.2 Function Documentation

- 9.8.2.1 int [STAR\\_API\\_CC\\_CFG\\_getMeasuredLinkSpeed](#) ( STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U16 \* pLinkSpeed )

**Deprecated** Function superseded by [CFG\\_getRxSignallingRate\(\)](#). Gets the current link speed measured on a specific port.

#### Parameters

|     |                   |                                                                                                                                                           |
|-----|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
|     | <i>deviceID</i>   | Device to get the measured link speed for a port.                                                                                                         |
|     | <i>portNum</i>    | Port to get information for.                                                                                                                              |
| out | <i>pLinkSpeed</i> | User supplied value that will be updated to contain the measured link speed in 100 Kbit/s units, see <a href="#">STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS</a> . |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, SpaceWire Router Mk2S, STAR Fire Mk3, SpaceWire PCIe Mk2, SpaceWire PCIe, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire Physical Layer Tester



9.8.2.2 `int STAR_API_CC CFG_getTransmitSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, _Out_ U32 * pSignallingRate )`

**Deprecated** Function superseded by `CFG_getTxSignallingRate()`. Gets the transmit bit rate for a given link.

#### Note

This function is compatible with all devices.

#### Parameters

|     |                        |                                                                                          |
|-----|------------------------|------------------------------------------------------------------------------------------|
|     | <i>deviceId</i>        | Device to get the transmit bit rate from.                                                |
|     | <i>linkNum</i>         | Link to get the transmit bit rate for.                                                   |
| out | <i>pSignallingRate</i> | Pointer to value that will be updated with the given link's transmit bit rate in Mbit/s. |

#### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

#### Added in version:

STAR-System v4.00 - 19th November 2019

#### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

9.8.2.3 `int STAR_API_CC CFG_setTransmitSignallingRate ( STAR_DEVICE_ID deviceId, U8 linkNum, U32 signallingRate )`

**Deprecated** Function superseded by `CFG_setTxSignallingRate()`. Sets the transmit bit rate on a given link. The transmit bit rate of a link is given in Mbit/s. The bit rate will be between 2 Mbit/s and 400 Mbit/s inclusive, although 400 Mbit/s may not be achievable with every device type.

#### Note

This function is compatible with all devices. However, depending upon the capabilities of the device, an exact bit rate may not be achievable.

For the SpaceWire PXI 12 Port Router device, clocks are shared by pairs of links. For example, setting the transmit bit rate for link 1 will also affect link 2. Similarly, links 3 and 4, 5 and 6, 7 and 8, 9 and 10, and 11 and 12 share clocks. Note that the odd-numbered link must be used to set the transmit clock frequency for each pair of links.

#### Parameters

|  |                       |                                                 |
|--|-----------------------|-------------------------------------------------|
|  | <i>deviceId</i>       | Device to modify a link's transmit bit rate on. |
|  | <i>linkNum</i>        | Link to have its transmit bit rate modified.    |
|  | <i>signallingRate</i> | Transmit bit rate in Mbit/s.                    |

### Returns

0 for success, or a negative error code upon failure as defined in [Defines](#).

### Added in version:

STAR-System v4.00 - 19th November 2019

### Supported devices:

All STAR-System Devices (SpaceWire Router Mk2S, SpaceWire Brick Mk2, SpaceWire Brick Mk3 and Brick Mk4, SpaceWire GbE Brick Mk1, STAR Fire Mk3, SpaceWire Router Mk2S, SpaceWire PXI Interface and PXI Interface Mk2, SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2, SpaceWire PCI Mk2 and cPCI Mk2, SpaceWire PCIe, SpaceWire PCIe Mk2, SpaceWire Physical Layer Tester)

## 9.9 cfg\_api\_generic\_common.h File Reference

Additional types used with the STAR-Dundee Generic Configuration API.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_pci_mk2_types.h"
#include "cfg_api_brick_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
#include "cfg_api_router_types.h"
```

### Data Structures

- struct [STAR\\_CFG\\_FPGA\\_INFO](#)  
*Defines a structure used to store a device's FPGA version info. [More...](#)*
- struct [STAR\\_DEVICE\\_INFO](#)  
*A structure used to store a device's HW info. [More...](#)*

#### 9.9.1 Detailed Description

Additional types used with the STAR-Dundee Generic Configuration API.

### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

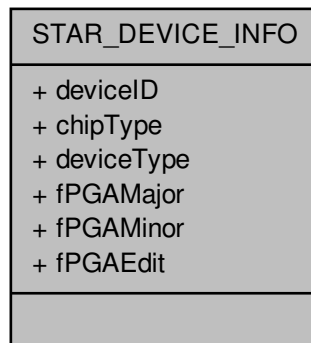
#### 9.9.2 Data Structure Documentation

##### 9.9.2.1 struct STAR\_DEVICE\_INFO

A structure used to store a device's HW info.



Collaboration diagram for STAR\_DEVICE\_INFO:



#### Data Fields

|                |            |                                                |
|----------------|------------|------------------------------------------------|
| U8             | chipType   | Identity code for a device's chip type.        |
| STAR_DEVICE_ID | deviceId   | Device's identifier.                           |
| U8             | deviceType | Identity code for a device's STAR_System type. |
| U16            | fPGAEdit   | FPGA edit number.                              |
| U8             | fPGAMajor  | FPGA major version number.                     |
| U8             | fPGAMinor  | FPGA minor version number.                     |

## 9.10 cfg\_api\_mk2.h File Reference

Functions used to configure STAR-Dundee Mk2 compatible devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
#include "cfg_api_remote.h"
```

#### Macros

- `#define STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN 256`  
*Maximum length for the string representation of a hardware build date.*
- `#define STAR_CFG_MK2_VERSION_STR_MAX_LEN 256`  
*Maximum length for the string representation of a hardware build date.*
- `#define CFG_API_MK2_MODULE_NAME "CFG-API_MK2"`  
*The name of the module, used for logging purposes.*

#### Functions

- `int STAR_API_CC CFG_MK2_getHardwareInfo (STAR_DEVICE_ID deviceId, _Out_ STAR_CFG_MK2_HARDWARE_INFO *pInfo)`  
*Reads information about the hardware version of a device and updates the STAR\_CFG\_MK2\_HARDWARE\_INFO structure pointed to by the passed in pointer to contain the received version information.*

- void `STAR_API_CC_CFG_MK2_hardwareInfoToString` (`STAR_CFG_MK2_HARDWARE_INFO` info, `_Out_ z_cap_c_(STAR_CFG_MK2_VERSION_STR_MAX_LEN)` char \*pVersion, `_Out_ z_cap_c_(STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN)` char \*pBuildDate)  
*Creates a string representation of a `STAR_CFG_MK2_HARDWARE_INFO` structure.*
- int `STAR_API_CC_CFG_MK2_enableTimeCodeMaster` (`STAR_DEVICE_ID` deviceID)  
*Enables the device as a time-code master.*
- int `STAR_API_CC_CFG_MK2_getTimeCodeMasterEnabled` (`STAR_DEVICE_ID` deviceID, int \*pEnabled)  
*Gets whether the device has been enabled as a time-code master.*
- int `STAR_API_CC_CFG_MK2_disableTimeCodeMaster` (`STAR_DEVICE_ID` deviceID)  
*Disables the device as a time-code master.*
- int `STAR_API_CC_CFG_MK2_enableExternalTimeCodeSelection` (`STAR_DEVICE_ID` deviceID)  
*Enables external time-code selection for the device.*
- int `STAR_API_CC_CFG_MK2_getExternalTimeCodeSelectionEnabled` (`STAR_DEVICE_ID` deviceID, int \*pEnabled)  
*Gets whether external time-code selection has been enabled for the device.*
- int `STAR_API_CC_CFG_MK2_disableExternalTimeCodeSelection` (`STAR_DEVICE_ID` deviceID)  
*Disables external time-code selection for the device.*
- int `STAR_API_CC_CFG_MK2_setTimeCodePeriod` (`STAR_DEVICE_ID` deviceID, U32 period)  
*Sets the period between time-code master ticks.*
- int `STAR_API_CC_CFG_MK2_getTimeCodePeriod` (`STAR_DEVICE_ID` deviceID, `_Out_ U32` \*pPeriod)  
*Gets the period between time-code master ticks.*
- int `STAR_API_CC_CFG_MK2_enableTimeCodeCounterBypassMode` (`STAR_DEVICE_ID` deviceID)  
*Enables time-code counter bypass mode for the device.*
- int `STAR_API_CC_CFG_MK2_getTimeCodeCounterBypassModeEnabled` (`STAR_DEVICE_ID` deviceID, int \*pEnabled)  
*Gets whether time-code counter bypass mode has been enabled for the device.*
- int `STAR_API_CC_CFG_MK2_disableTimeCodeCounterBypassMode` (`STAR_DEVICE_ID` deviceID)  
*Disables time-code counter bypass mode for the device.*
- int `STAR_API_CC_CFG_MK2_identify` (`STAR_DEVICE_ID` deviceID)  
*Flashes the front panel LEDs of a device.*
- int `STAR_API_CC_CFG_MK2_enableInterfaceMode` (`STAR_DEVICE_ID` deviceID)  
*In interface mode a packet which is received on an external port, from a SpaceWire link or from the configuration port will be routed to the port specified by the port routing register of the port (set by `CFG_MK2_setPortRoutingAddress()`).*
- int `STAR_API_CC_CFG_MK2_getInterfaceModeEnabled` (`STAR_DEVICE_ID` deviceID, int \*pEnabled)  
*Gets whether interface mode has been enabled on a device.*
- int `STAR_API_CC_CFG_MK2_enableInterfaceModeOnPort` (`STAR_DEVICE_ID` deviceID, U8 portNum)  
*Selectively enables interface mode on a given port.*
- int `STAR_API_CC_CFG_MK2_getInterfaceModeOnPortEnabled` (`STAR_DEVICE_ID` deviceID, U8 portNum, int \*pEnabled)  
*Gets whether interface mode is enabled on a specific port.*
- int `STAR_API_CC_CFG_MK2_disableInterfaceMode` (`STAR_DEVICE_ID` deviceID)  
*Disables interface mode on a device.*
- int `STAR_API_CC_CFG_MK2_disableInterfaceModeOnPort` (`STAR_DEVICE_ID` deviceID, U8 portNum)  
*Selectively disables interface mode on a given port.*
- int `STAR_API_CC_CFG_MK2_enableIdentifySource` (`STAR_DEVICE_ID` deviceID)  
*When interface mode is enabled on a port, this function globally enables the addition of a leading byte to each received packet indicating which port the packet was received on.*
- int `STAR_API_CC_CFG_MK2_getIdentifySourceEnabled` (`STAR_DEVICE_ID` deviceID, int \*pEnabled)  
*Gets whether identify source is enabled.*
- int `STAR_API_CC_CFG_MK2_disableIdentifySource` (`STAR_DEVICE_ID` deviceID)  
*Disables identification of source ports for interface mode globally.*
- int `STAR_API_CC_CFG_MK2_enableIdentifySourceOnPort` (`STAR_DEVICE_ID` deviceID, U8 portNum)

- Enables identification of source port during interface mode for a given port.*

  - int `STAR_API_CC_CFG_MK2_getIdentifySourceOnPortEnabled` (STAR\_DEVICE\_ID deviceID, U8 portNum, int \*pEnabled)

*Gets whether identify source is enabled for a specific port.*
- int `STAR_API_CC_CFG_MK2_disableIdentifySourceOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Disables source port identification for a given port.*
- int `STAR_API_CC_CFG_MK2_setPortRoutingAddress` (STAR\_DEVICE\_ID deviceID, U8 portNum, U8 address)

*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int `STAR_API_CC_CFG_MK2_getPortRoutingAddress` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U8 \*pAddress)

*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*
- int `STAR_API_CC_CFG_MK2_setLinkRateDivider` (STAR\_DEVICE\_ID deviceID, U8 linkNum, U8 divider)

*Sets the Link rate divider for a given link.*
- int `STAR_API_CC_CFG_MK2_getLinkRateDivider` (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ U8 \*pDivider)

*Gets the Link rate divider for a given link.*
- int `STAR_API_CC_CFG_MK2_setBaseTransmitClock` (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)

*Sets the base transmit clock rate on a given link:*
- int `STAR_API_CC_CFG_MK2_getBaseTransmitClock` (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)

*Gets the base transmit clock rate parameters for a given link.*
- int `STAR_API_CC_CFG_MK2_setTransmitClock` (STAR\_DEVICE\_ID deviceID, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)

*Sets the transmit clock rate on a given link.*
- int `STAR_API_CC_CFG_MK2_getTransmitClock` (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*pClockRateParams)

*Gets the transmit clock rate parameters for a given link.*
- int `STAR_API_CC_CFG_MK2_injectError` (STAR\_DEVICE\_ID deviceID, unsigned char port, SPW\_ERROR error)

*Immediately injects the specified error on a given device's port.*
- int `STAR_API_CC_CFG_MK2_startPeriodicAction` (STAR\_DEVICE\_ID deviceID, unsigned char port, U32 count, SPW\_ACTION action)

*Starts a periodic action on a given device's port.*
- int `STAR_API_CC_CFG_MK2_stopPeriodicAction` (STAR\_DEVICE\_ID deviceID, unsigned char port)

*Stops a periodic action on a given device's port.*
- U32 `STAR_API_CC_CFG_MK2_getDeviceClockRate` (STAR\_DEVICE\_ID deviceID)

*Gets a device's internal clock rate.*
- int `STAR_API_CC_CFG_MK2_enableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables state change events on a given port.*
- int `STAR_API_CC_CFG_MK2_getStateChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether state change events are enabled on a specific port.*
- int `STAR_API_CC_CFG_MK2_disableStateChangeEventsOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables state change events on a given port.*
- int `STAR_API_CC_CFG_MK2_enableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables speed change events on a given port.*
- int `STAR_API_CC_CFG_MK2_getSpeedChangeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether speed change events are enabled on a specific port.*

- int `STAR_API_CC_CFG_MK2_disableSpeedChangeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables speed change events on a given port.*

- int `STAR_API_CC_CFG_MK2_enableTimeCodeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively enables time-code notifications on a the channel attached to a given port.*

- int `STAR_API_CC_CFG_MK2_getTimeCodeEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether time-code notifications are enabled on a specific port.*

- int `STAR_API_CC_CFG_MK2_disableTimeCodeEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)

*Selectively disables time-code notifications on a the channel attached to a given port.*

- int `STAR_API_CC_CFG_MK2_getMeasuredLinkSpeed` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ U16 \*pLinkSpeed)

*Gets the current link speed measured on a specific port.*

- int `STAR_API_CC_CFG_MK2_setTimeCodeDistributionPorts` (STAR\_DEVICE\_ID deviceId, U32 portMask)

*This function is used to specify which output ports time-codes are forwarded on.*

- int `STAR_API_CC_CFG_MK2_getTimeCodeDistributionPorts` (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*pPortMask)

*This function is used to obtain which output ports time-codes are forwarded on.*

- int `STAR_API_CC_CFG_MK2_setTimeCodeFlagMode` (STAR\_DEVICE\_ID deviceId, U8 mode)

*Sets the time-code flag interpretation mode:*

- int `STAR_API_CC_CFG_MK2_getTimeCodeFlagMode` (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*pMode)

*Obtains the time-code flag interpretation mode:*

### 9.10.1 Detailed Description

Functions used to configure STAR-Dundee Mk2 compatible devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2025 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.10.2 Macro Definition Documentation

#### 9.10.2.1 #define CFG\_API\_MK2\_MODULE\_NAME "CFG-API\_MK2"

The name of the module, used for logging purposes.

## 9.11 cfg\_api\_mk2\_types.h File Reference

Types used with the STAR-Dundee Mk2 compatible device configuration API.

## Data Structures

- struct `STAR_CFG_MK2_HARDWARE_INFO`  
*Hardware version, and synthesis date of the FPGA code. [More...](#)*
- struct `STAR_CFG_MK2_BASE_TRANSMIT_CLOCK`  
*Base transmit clock rate control properties. [More...](#)*

## Macros

- #define `STAR_CFG_MK2_LINK_SPEED_UNITS_KBPS` 100  
*The units in Kbit/s used to represent the link speed returned when calling `CFG_MK2_getMeasuredLinkSpeed()`.*

## Enumerations

- enum `SPW_ERROR` {  
`SPW_ERROR_DISCONNECT` = 1, `SPW_ERROR_PARITY` = 2, `SPW_ERROR_ESCAPE` = 3, `SPW_ERROR_INSERT_FCT` = 4,  
`SPW_ERROR_SUPPRESS_FCT` = 5, `SPW_ERROR_INCREMENT_CREDIT` = 6, `SPW_ERROR_DECREMENT_CREDIT` = 7 }  
*Errors that can be injected onto the SpaceWire link.*
- enum `SPW_ACTION` { `SPW_TRANSMIT_PACKET` = 0 }  
*Actions which can be preformed at specified periodic intervals.*

### 9.11.1 Detailed Description

Types used with the STAR-Dundee Mk2 compatible device configuration API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.12 cfg\_api\_pci\_mk2.h File Reference

Functions used to configure STAR-Dundee PCI Mk2 devices.

```
#include "star-api.h"
#include "cfg_api_pci_mk2_types.h"
#include "cfg_api_remote.h"
```

## Macros

- #define `CFG_API_PCI_MK2_MODULE_NAME` "CFG-API-PCI-MK2"  
*The name of the module, used for logging purposes.*

## Functions

- int [STAR\\_API\\_CC\\_CFG\\_PCIMK2\\_setLinkClockFrequency](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, ST↔AR\_CFG\_PCIMK2\_LINK\_FREQ linkFreq)  
*Sets the Link Clock Frequency on a given link.*
- int [STAR\\_API\\_CC\\_CFG\\_PCIMK2\\_getLinkClockFrequency](#) (STAR\_DEVICE\_ID deviceID, U8 linkNum, ST↔AR\_CFG\_PCIMK2\_LINK\_FREQ \*pLinkFreq)  
*Gets the Link Clock Frequency for a given link.*

### 9.12.1 Detailed Description

Functions used to configure STAR-Dundee PCI Mk2 devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.12.2 Macro Definition Documentation

#### 9.12.2.1 `#define CFG_API_PCI_MK2_MODULE_NAME "CFG-API-PCI-MK2"`

The name of the module, used for logging purposes.

## 9.13 `cfg_api_pci_mk2_types.h` File Reference

Types used with the STAR-Dundee PCI Mk2 Configuration API.

### Enumerations

- enum [STAR\\_CFG\\_PCIMK2\\_LINK\\_FREQ](#) {  
STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120 = 0, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128 = 5, STAR\_CFG\_P↔CIMK2\_LINK\_FREQ\_140 = 1, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150 = 6,  
STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160 = 2, STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180 = 3, STAR\_CFG\_P↔CIMK2\_LINK\_FREQ\_200 = 4 }  
*Frequencies a link may run at.*

### 9.13.1 Detailed Description

Types used with the STAR-Dundee PCI Mk2 Configuration API.

**Author**

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

**9.14 cfg\_api\_pcie\_mk2.h File Reference**

Functions used to configure STAR-Dundee PCIe Mk2 and compatible devices.

```
#include "star-api.h"
#include "cfg_api_brick_mk2_types.h"
#include "cfg_api_brick_mk3_types.h"
```

**Functions**

- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_setTransmitSignallingRate** (STAR\_DEVICE\_ID deviceId, U8 linkNum, U32 signallingRate)  
*Sets the transmit bit rate on a given link.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_getTransmitSignallingRate** (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ U32 \*pSignallingRate)  
*Gets the transmit bit rate for a given link.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_setDecimalTxSignallingRate** (STAR\_DEVICE\_ID deviceId, U8 linkNum, double signallingRate)  
*Sets the transmit bit rate on a given link.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_getDecimalTxSignallingRate** (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ double \*pSignallingRate)  
*Gets the transmit bit rate for a given link.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_enableBroadcastControllerLegacyMode** (STAR\_DEVICE\_ID deviceId)  
*Enables broadcast controller legacy mode.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_getBroadcastControllerLegacyModeEnabled** (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pEnabled)  
*Gets whether broadcast legacy mode has been enabled for the device.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_disableBroadcastControllerLegacyMode** (STAR\_DEVICE\_ID deviceId)  
*Disables broadcast controller legacy mode.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_setInterruptCodeDistributionPorts** (STAR\_DEVICE\_ID deviceId, U32 portMask)  
*This function is used to specify which output ports interrupt codes are forwarded on.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_getInterruptCodeDistributionPorts** (STAR\_DEVICE\_ID deviceId, \_Out\_ U32 \*pPortMask)  
*This function is used to obtain which output ports interrupt codes are forwarded on.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_setTimestampMethod** (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD timestampMethod)  
*Sets the timestamping method to use for the given device.*
- int **STAR\_API\_CC\_CFG\_PCIE\_MK2\_enableRxTimestampEventsOnPort** (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively enables receive timestamp events on a given port.*

- int `STAR_API_CC_CFG_PCIE_MK2_getRxTimestampEventsOnPortEnabled` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_Out\_ int \*pEnabled)  
*Gets whether receive timestamp events are enabled on a specific port.*
- int `STAR_API_CC_CFG_PCIE_MK2_disableRxTimestampEventsOnPort` (STAR\_DEVICE\_ID deviceId, U8 portNum)  
*Selectively disables receive timestamp events on a given port.*
- int `STAR_API_CC_CFG_PCIE_MK2_setPulseGeneratorFrequency` (STAR\_DEVICE\_ID deviceId, STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ frequency)  
*Sets the frequency of each generated pulse.*
- int `STAR_API_CC_CFG_PCIE_MK2_getPulseGeneratorFrequency` (STAR\_DEVICE\_ID deviceId, \_Out\_ STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ \*pFrequency)  
*Gets the frequency of each generated pulse.*
- int `STAR_API_CC_CFG_PCIE_MK2_injectErrors` (STAR\_DEVICE\_ID deviceId, U8 portNum, \_In\_ STAR\_CFG\_BRICK\_MK2\_ERRORS \*pErrors)  
*Inject the specified errors on a given port.*

### 9.14.1 Detailed Description

Functions used to configure STAR-Dundee PCIe Mk2 and compatible devices.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.15 cfg\_api\_pxi.h File Reference

Functions used to configure STAR-Dundee PXI devices.

```
#include "star-api.h"
#include "cfg_api_mk2_types.h"
```

### Functions

- int `STAR_API_CC_CFG_PXI_setLinkRateDivider` (STAR\_DEVICE\_ID deviceId, U8 divider)  
*Sets the Link rate divider for a PXI device.*
- int `STAR_API_CC_CFG_PXI_getLinkRateDivider` (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*pDivider)  
*Gets the Link rate divider for a PXI device.*
- int `STAR_API_CC_CFG_PXI_setBaseTransmitClock` (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)  
*Sets the base transmit clock rate on a given link:*
- int `STAR_API_CC_CFG_PXI_getBaseTransmitClock` (STAR\_DEVICE\_ID deviceId, U8 linkNum, \_Out\_ STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*clockRateParams)  
*Gets the base transmit clock rate parameters for a given link.*
- int `STAR_API_CC_CFG_PXI_setTransmitClock` (STAR\_DEVICE\_ID deviceId, U8 linkNum, STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK clockRateParams)



*Sets the transmit clock rate on a given link.*

- int STAR\_API\_CC\_CFG\_PXI\_getTransmitClock (STAR\_DEVICE\_ID deviceID, U8 linkNum, \_Out\_ STAR\_API\_CC\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK \*clockRateParams)

*Gets the transmit clock rate parameters for a given link.*

- int STAR\_API\_CC\_CFG\_PXI\_injectError (STAR\_DEVICE\_ID deviceID, unsigned char port, SPW\_ERROR error)

*Immediately injects the specified error on a given device's port.*

- int STAR\_API\_CC\_CFG\_PXI\_enableInterfaceModeOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables interface mode on a given port of a supported PXI device.*

- int STAR\_API\_CC\_CFG\_PXI\_getInterfaceModeOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether interface mode is enabled on a specific port of a supported PXI device.*

- int STAR\_API\_CC\_CFG\_PXI\_disableInterfaceModeOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables interface mode on a given port of a supported PXI device.*

- int STAR\_API\_CC\_CFG\_PXI\_enableIdentifySourceOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Enables identification of the source port of a received packet on a given port, when in interface mode.*

- int STAR\_API\_CC\_CFG\_PXI\_getIdentifySourceOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether identify source is enabled for a specific port.*

- int STAR\_API\_CC\_CFG\_PXI\_disableIdentifySourceOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Disables source port identification for a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_setPortRoutingAddress (STAR\_DEVICE\_ID deviceID, U8 portNum, U8 address)

*Sets the address a packet received on a given port should be routed to when interface mode is enabled.*

- int STAR\_API\_CC\_CFG\_PXI\_getPortRoutingAddress (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ U8 \*pAddress)

*Gets the address a packet received on a given port should be routed to when interface mode is enabled.*

- int STAR\_API\_CC\_CFG\_PXI\_enableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables state change events on a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_getStateChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether state change events are enabled on a specific port.*

- int STAR\_API\_CC\_CFG\_PXI\_disableStateChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables state change events on a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_enableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables speed change events on a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_getSpeedChangeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether speed change events are enabled on a specific port.*

- int STAR\_API\_CC\_CFG\_PXI\_disableSpeedChangeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables speed change events on a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_enableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively enables time-code notifications on a channel attached to a given port.*

- int STAR\_API\_CC\_CFG\_PXI\_getTimeCodeEventsOnPortEnabled (STAR\_DEVICE\_ID deviceID, U8 portNum, \_Out\_ int \*pEnabled)

*Gets whether time-code notifications are enabled on a specific port.*

- int STAR\_API\_CC\_CFG\_PXI\_disableTimeCodeEventsOnPort (STAR\_DEVICE\_ID deviceID, U8 portNum)

*Selectively disables time-code notifications on a channel attached to a given port.*

### 9.15.1 Detailed Description

Functions used to configure STAR-Dundee PXI devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.16 `cfg_api_remote.h` File Reference

Functions used to create, configure and destroy paths to remote devices.

```
#include "star-api.h"
```

#### Functions

- `STAR_DEVICE_ID STAR_API_CC STAR_CFG_createRemoteDeviceIdentifier` (`STAR_DEVICE_ID` deviceId, unsigned char localChannel, char \*name, `STAR_SPACEWIRE_ADDRESS` \*pathTo, `STAR_SPACEWIRE_ADDRESS` \*returnPath)  
*Creates a Remote Device Identifier.*
- `int STAR_API_CC STAR_CFG_destroyRemoteDeviceIdentifier` (`STAR_DEVICE_ID` remoteDeviceId)  
*Destroys a Remote Device Identifier.*
- `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionList` (`_Out_U32` \*count)  
*Gets an array of all remote devices that have been created by all processes.*
- `void STAR_API_CC STAR_CFG_destroyRemoteDeviceDescriptionList` (`_Post_ptr_invalid_ STAR_DEVICE_ID` \*pRemoteDeviceDescriptionList)  
*Frees a remote device description list previously created by a call to `STAR_CFG_getRemoteDeviceDescriptionList()`.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalChannel` (`STAR_DEVICE_ID` remoteDeviceId, unsigned char \*localChannel)  
*Gets the number of the local device channel used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceLocalDevice` (`STAR_DEVICE_ID` remoteDeviceId, `STAR_DEVICE_ID` \*localDeviceId)  
*Gets the local device used to communicate with the remote device on.*
- `int STAR_API_CC STAR_CFG_getRemoteDevicePath` (`STAR_DEVICE_ID` remoteDeviceId, `_Out_opt_ STAR_SPACEWIRE_ADDRESS` \*\*pPath)  
*Gets the path to the specified remote device.*
- `int STAR_API_CC STAR_CFG_getRemoteDeviceRetPath` (`STAR_DEVICE_ID` remoteDeviceId, `_Out_opt_ STAR_SPACEWIRE_ADDRESS` \*\*pRetPath)  
*Gets the return path from to the specified remote device.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_CFG_getRemoteDeviceDescriptionNameString` (`STAR_DEVICE_ID` deviceId)  
*Gets the name of the remote device description identified by a given identifier.*

### 9.16.1 Detailed Description

Functions used to create, configure and destroy paths to remote devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.17 cfg\_api\_router.h File Reference

Functions used to configure STAR-Dundee routing devices.

```
#include "star-api.h"
#include "cfg_api_remote.h"
#include "cfg_api_router_types.h"
```

### Functions

- [int STAR\\_API\\_CC CFG\\_ROUTER\\_getDeviceIdentificationInfo \(STAR\\_DEVICE\\_ID deviceId, \\_Out\\_ STAR\\_CFG\\_DEVICE\\_IDENTIFIER\\_INFO \\*pInfo\)](#)  
*Reads the device identifier information of a device, i.e.*
- [void STAR\\_API\\_CC CFG\\_ROUTER\\_getDeviceManufacturerAsString \(STAR\\_CFG\\_DEVICE\\_IDENTIFIER\\_INFO \\*pDeviceIdentifierInfo, \\_Out\\_z\\_cap\\_c\\_ \(STAR\\_CFG\\_MANUFACTURER\\_STR\\_MAX\\_LEN\) char \\*manufacturerStr\)](#)  
*If the manufacturer is known, this function obtains a string representation of the manufacturers name.*
- [void STAR\\_API\\_CC CFG\\_ROUTER\\_getDeviceTypeAsString \(STAR\\_CFG\\_DEVICE\\_IDENTIFIER\\_INFO \\*pDeviceIdentifierInfo, \\_Out\\_z\\_cap\\_c\\_ \(STAR\\_CFG\\_DEVICE\\_STR\\_MAX\\_LEN\) char \\*deviceTypeStr\)](#)  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- [int STAR\\_API\\_CC CFG\\_ROUTER\\_getNetworkDiscoveryInfo \(STAR\\_DEVICE\\_ID deviceId, \\_Out\\_ STAR\\_CFG\\_NETWORK\\_DISCOVERY\\_INFO \\*pInfo\)](#)  
*Reads the network discovery information of a device.*
- [int STAR\\_API\\_CC CFG\\_ROUTER\\_getPortStatusControl \(STAR\\_DEVICE\\_ID deviceId, U8 portNum, \\_Out\\_ PORT\\_STATUS\\_CONTROL \\*portStatusControl\)](#)  
*Gets the value of a port or link's status/control register, which can then be used by the various functions within the Port Status and Control group.*
- [STAR\\_CFG\\_PORT\\_TYPE STAR\\_API\\_CC CFG\\_ROUTER\\_getPortType \(PORT\\_STATUS\\_CONTROL portStatusControl\)](#)  
*Identifies the type of port attributed to a given port number.*
- [U8 STAR\\_API\\_CC CFG\\_ROUTER\\_getPortConnection \(PORT\\_STATUS\\_CONTROL portStatusControl\)](#)  
*Identifies the output port to which the source port is currently connected whilst routing is in operation.*
- [int STAR\\_API\\_CC CFG\\_ROUTER\\_clearPortErrors \(STAR\\_DEVICE\\_ID deviceId, U8 portNum\)](#)  
*Clears all errors on a given port.*
- [int STAR\\_API\\_CC CFG\\_ROUTER\\_getConfigPortErrors \(PORT\\_STATUS\\_CONTROL portStatusControl, \\_Out\\_ STAR\\_CFG\\_CONFIG\\_PORT\\_ERRORS \\*errors\)](#)  
*Gets any errors present on the config port.*

- `int STAR_API_CC_CFG_ROUTER_getSpaceWireLinkErrors (PORT_STATUS_CONTROL linkStatus↔ Control, _Out_ STAR_CFG_SPW_LINK_ERRORS *errors)`  
*Gets any errors present on a SpaceWire link.*
- `int STAR_API_CC_CFG_ROUTER_getSpaceWireLinkStatus (PORT_STATUS_CONTROL linkStatus↔ Control, _Out_ STAR_CFG_SPW_LINK_STATUS *status)`  
*Gets the status of a SpaceWire Link.*
- `int STAR_API_CC_CFG_ROUTER_setSpaceWireLinkStatus (STAR_DEVICE_ID deviceID, U8 linkNum, S↔ TAR_CFG_SPW_LINK_STATUS linkStatus)`  
*Sets the state of a SpaceWire Link.*
- `int STAR_API_CC_CFG_ROUTER_startLink (STAR_DEVICE_ID deviceID, U8 linkNum)`  
*Starts a SpaceWire link by setting the start bit, and clearing the disable bit.*
- `int STAR_API_CC_CFG_ROUTER_stopLink (STAR_DEVICE_ID deviceID, U8 linkNum)`  
*Stops a SpaceWire link by clearing the start bit, and setting the disable bit.*
- `int STAR_API_CC_CFG_ROUTER_getExternalPortErrors (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_ERRORS *errors)`  
*Gets any errors present on an external port.*
- `int STAR_API_CC_CFG_ROUTER_getExternalPortStatus (PORT_STATUS_CONTROL portStatusControl, _Out_ STAR_CFG_EXTERNAL_PORT_STATUS *status)`  
*Gets the status of an external port.*
- `int STAR_API_CC_CFG_ROUTER_getRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress, _Out_ STAR_CFG_GAR_ENTRY *entry)`  
*Gets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_ROUTER_setRoutingTableEntry (STAR_DEVICE_ID deviceID, U8 logicalAddress, STAR_CFG_GAR_ENTRY entry)`  
*Sets a routing table entry for a given logical address.*
- `int STAR_API_CC_CFG_ROUTER_setNetworkIdentity (STAR_DEVICE_ID deviceID, REGISTER value)`  
*Sets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_getNetworkIdentity (STAR_DEVICE_ID deviceID, _Out_ REGISTE↔ R *value)`  
*Gets the value of the network identity register.*
- `int STAR_API_CC_CFG_ROUTER_setGeneralPurpose (STAR_DEVICE_ID deviceID, REGISTER value)`  
*Sets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_getGeneralPurpose (STAR_DEVICE_ID deviceID, _Out_ REGISTE↔ R *value)`  
*Gets the value of the general purpose register.*
- `int STAR_API_CC_CFG_ROUTER_setTimeCodeDistributionPorts (STAR_DEVICE_ID deviceID, U32 port↔ Mask)`  
*This function is used to specify which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_getTimeCodeDistributionPorts (STAR_DEVICE_ID deviceID, _Out_ U32 *portMask)`  
*This function is used to obtain which output ports time-codes are forwarded on.*
- `int STAR_API_CC_CFG_ROUTER_setTimeCodeFlagMode (STAR_DEVICE_ID deviceID, U8 mode)`  
*Sets the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_ROUTER_getTimeCodeFlagMode (STAR_DEVICE_ID deviceID, _Out_ U8 *mode)`  
*Obtains the time-code flag interpretation mode:*
- `int STAR_API_CC_CFG_ROUTER_getTimeCodeValue (STAR_DEVICE_ID deviceID, _Out_ U8 *value)`  
*Obtains the current value of the router's internal time-code counter.*
- `int STAR_API_CC_CFG_ROUTER_setRouterGlobalSettings (STAR_DEVICE_ID deviceID, STAR_CFG_↔ ROUTER_GLOBAL_STATE state)`  
*Sets the router's global settings.*
- `int STAR_API_CC_CFG_ROUTER_getRouterGlobalSettings (STAR_DEVICE_ID deviceID, _Out_ STAR_↔ CFG_ROUTER_GLOBAL_STATE *state)`  
*Gets the router's global settings.*

### 9.17.1 Detailed Description

Functions used to configure STAR-Dundee routing devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2025 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.18 cfg\_api\_router\_mk2s.h File Reference

Functions used to configure STAR-Dundee Router Mk2S devices.

```
#include "star-api.h"
```

### Functions

- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_enablePrecisionTransmitRate](#) (STAR\_DEVICE\_ID deviceId)  
*Enables precision transmit rate on a SpaceWire Router Mk2S device.*
- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_getPrecisionTransmitRateEnabled](#) (STAR\_DEVICE\_ID deviceId, \_Out\_ int \*pEnabled)  
*Determine if precision transmit rate is enabled on a Router Mk2S device.*
- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_disablePrecisionTransmitRate](#) (STAR\_DEVICE\_ID deviceId)  
*Disables precision transmit rate on a SpaceWire Router Mk2S device.*
- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_getPrecisionTransmitRateInUse](#) (STAR\_DEVICE\_ID deviceId, \_↔\_ Out\_ int \*pInUse)  
*Determine whether precision transmit rate is in use on a SpaceWire Router Mk2S device.*
- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_getPrecisionTransmitRate](#) (STAR\_DEVICE\_ID deviceId, \_↔\_ Out\_ double \*pTransmitRate)  
*Get the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*
- int [STAR\\_API\\_CC\\_CFG\\_ROUTER\\_MK2S\\_setPrecisionTransmitRate](#) (STAR\_DEVICE\_ID deviceId, double transmitRate)  
*Set the current precision transmit rate on a SpaceWire Router Mk2S device in Mbit/s.*

### 9.18.1 Detailed Description

Functions used to configure STAR-Dundee Router Mk2S devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.19 cfg\_api\_router\_types.h File Reference

Types used with the STAR-Dundee Router API.

```
#include "star-api.h"
```

### Data Structures

- struct [STAR\\_CFG\\_DEVICE\\_IDENTIFIER\\_INFO](#)  
*Information obtained from a Device's Identifier Register, identifying the router's version and type. [More...](#)*
- struct [STAR\\_CFG\\_NETWORK\\_DISCOVERY\\_INFO](#)  
*Information obtained from a device's network discovery register. [More...](#)*
- struct [STAR\\_CFG\\_CONFIG\\_PORT\\_ERRORS](#)  
*Errors that may be present on a device's configuration port. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_ERRORS](#)  
*Errors that may be present on a SpaceWire Link. [More...](#)*
- struct [STAR\\_CFG\\_SPW\\_LINK\\_STATUS](#)  
*Status of a SpaceWire link. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_ERRORS](#)  
*Errors that may be present on an external port. [More...](#)*
- struct [STAR\\_CFG\\_EXTERNAL\\_PORT\\_STATUS](#)  
*Status of an external port. [More...](#)*
- struct [STAR\\_CFG\\_GAR\\_ENTRY](#)  
*A Routing table entry. [More...](#)*
- struct [STAR\\_CFG\\_ROUTER\\_GLOBAL\\_STATE](#)  
*Global router settings. [More...](#)*

### Macros

- [#define STAR\\_CFG\\_MANUFACTURER\\_STR\\_MAX\\_LEN 256](#)  
*Maximum length for the string representation of a manufacturer name.*
- [#define STAR\\_CFG\\_DEVICE\\_STR\\_MAX\\_LEN 256](#)  
*Maximum length for the string representation of a device type.*

### Typedefs

- typedef [REGISTER\\_PORT\\_STATUS\\_CONTROL](#)  
*The type used to represent a port status and control register value.*

### Enumerations

- enum [STAR\\_CFG\\_DEVICE\\_TYPE](#) { [STAR\\_CFG\\_DEVICE\\_TYPE\\_ROUTER](#), [STAR\\_CFG\\_DEVICE\\_TYPE\\_UNKNOWN](#), [STAR\\_CFG\\_DEVICE\\_TYPE\\_INVALID](#) }  
*Device types permitted within the network discovery register.*
- enum [STAR\\_CFG\\_PORT\\_TYPE](#) { [STAR\\_CFG\\_PORT\\_TYPE\\_CONFIGURATION](#), [STAR\\_CFG\\_PORT\\_TYPE\\_LINK](#), [STAR\\_CFG\\_PORT\\_TYPE\\_EXTERNAL](#), [STAR\\_CFG\\_PORT\\_TYPE\\_INVALID](#) }

*Port types for Routing devices.*

- enum STAR\_CFG\_SPW\_LINK\_STATE {  
STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RESET, STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT, S↵  
TAR\_CFG\_SPW\_LINK\_STATE\_READY, STAR\_CFG\_SPW\_LINK\_STATE\_STARTED,  
STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING, STAR\_CFG\_SPW\_LINK\_STATE\_RUN, STAR\_CFG\_↵  
SPW\_LINK\_STATE\_INVALID }

*The state of the interface state machine in the SpaceWire link.*

- enum STAR\_CFG\_PORT\_TIMEOUT {  
STAR\_CFG\_PORT\_TIMEOUT\_100US = 0x0, STAR\_CFG\_PORT\_TIMEOUT\_1MS = 0x1, STAR\_CFG\_P↵  
ORT\_TIMEOUT\_10MS = 0x2, STAR\_CFG\_PORT\_TIMEOUT\_100MS = 0x3,  
STAR\_CFG\_PORT\_TIMEOUT\_1S = 0x4 }

*Length of port timeout.*

- enum STAR\_CFG\_TIMEOUT\_MODE { STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING, STAR\_CFG\_TIMEO↵  
UT\_MODE\_WATCHDOG }

*Timeout mode used by the router.*

### 9.19.1 Detailed Description

Types used with the STAR-Dundee Router API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.20 cfg\_api\_spfi.h File Reference

Functions used to configure SpaceFibre devices.

```
#include "star-api.h"
#include "cfg_api_spfi_types.h"
#include "cfg_api_generic.h"
```

#### Macros

- #define CFG\_SPFI\_STATUS\_PORT\_INVALID (-1000)  
*The port value is invalid.*
- #define CFG\_SPFI\_STATUS\_UNEXPECTED\_PORT\_TYPE (-1001)  
*The port type is unexpected.*
- #define CFG\_SPFI\_STATUS\_VIRTUAL\_CHANNEL\_INVALID (-1002)  
*The virtual channel value is invalid.*
- #define CFG\_SPFI\_STATUS\_LANE\_INVALID (-1003)  
*The lane value is invalid.*
- #define CFG\_SPFI\_STATUS\_BROADCAST\_TIMEOUT\_INVALID (-1004)  
*The broadcast timeout value is invalid.*
- #define CFG\_SPFI\_STATUS\_VIRTUAL\_NETWORK\_NUMBER\_INVALID (-1005)



- *The virtual network number value is invalid.*
- `#define CFG_SPFI_STATUS_BANDWIDTH_RESERVATION_INVALID (-1006)`  
*The bandwidth reservation value is invalid.*
- `#define CFG_SPFI_STATUS_PRIORITY_INVALID (-1007)`  
*The priority value is invalid.*
- `#define CFG_SPFI_ROUTER_TIMEOUT_INVALID (-1008)`  
*The router timeout value is invalid.*
- `#define CFG_SPFI_AXI_BC_INVALID (-1009)`  
*The AXI BC value is invalid.*

## Functions

- `int STAR_API_CC CFG_SPFI_getDeviceInformation (STAR_DEVICE_ID deviceId, _Out_ SPFI_DEVICE_INFO *pDeviceInfo)`  
*Reads from the device information register and populates a SPFI\_DEVICE\_INFO value for use with the following sub-accessor functions:*
- `SPFI_NODE_TYPE STAR_API_CC CFG_SPFI_getDeviceNodeType (SPFI_DEVICE_INFO deviceInfo)`  
*Gets the device node type according to the device information value.*
- `U8 STAR_API_CC CFG_SPFI_getDevicePortCount (SPFI_DEVICE_INFO deviceInfo)`  
*Gets the number of ports according to the device information value.*
- `int STAR_API_CC CFG_SPFI_getDevicePortCountByType (STAR_DEVICE_ID deviceId, SPFI_PORT_TYPE portType, _Out_ U8 *pPortCount)`  
*Gets the number of ports of a given type according to the device information value.*
- `int STAR_API_CC CFG_SPFI_getDeviceIdentificationInfo (STAR_DEVICE_ID deviceId, _Out_ SPFI_DEVICE_IDENTIFIER_INFO *pDeviceIdentifierInfo)`  
*Reads from the device code register and populates a SPFI\_DEVICE\_IDENTIFIER\_INFO value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC CFG_SPFI_getDeviceManufacturer (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)`  
*Gets the device manufacturer code according to the device identification information value.*
- `int STAR_API_CC CFG_SPFI_getDeviceManufacturerAsString (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_z_cap_c (STAR_CFG_MANUFACTURER_STR_MAX_LEN) char *pManufacturerStr)`  
*If the manufacturer is known, this function obtains a string representation of the manufacturer name.*
- `U8 STAR_API_CC CFG_SPFI_getDeviceType (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo)`  
*Gets the manufacturer specific device type according to the device identification information value.*
- `int STAR_API_CC CFG_SPFI_getDeviceTypeAsString (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_z_cap_c (STAR_CFG_DEVICE_STR_MAX_LEN) char *pDeviceTypeStr)`  
*If the manufacturer and device type is known, this function obtains a string representation of the device's type.*
- `int STAR_API_CC CFG_SPFI_getDeviceVersion (SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo, _Out_ U8 *pMajorVersion, _Out_ U8 *pMinorVersion, _Out_ U8 *pPatchVersion)`  
*Gets the router version number components according to the device identification information value.*
- `int STAR_API_CC CFG_SPFI_getRoutingTableEntryPorts (STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ U32 *pRoutingTableEntryPorts)`  
*Reads from the routing table entry register for a logical address and populates a U32 value with a bitset indicating the port mask.*
- `int STAR_API_CC CFG_SPFI_setRoutingTableEntryPorts (STAR_DEVICE_ID deviceId, U8 logicalAddress, U32 routingTableEntryPorts)`  
*Sets the routing table entry for the logical address port mask.*
- `int STAR_API_CC CFG_SPFI_getRoutingTableEntryFlags (STAR_DEVICE_ID deviceId, U8 logicalAddress, _Out_ SPFI_ROUTING_TABLE_ENTRY_FLAGS *pRoutingTableEntryFlags)`  
*Reads from the routing table entry flags register for a logical address and populates a SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS value for use with the following sub-accessor functions:*



- `U8 STAR_API_CC CFG_SPFI_isRoutingTableEntryEnabled (SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)`  
*Gets the enabled state for the routing table entry.*
- `U8 STAR_API_CC CFG_SPFI_isRoutingTableEntryDeleteHeaderEnabled (SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags)`  
*Gets the delete header enabled state for the routing table entry.*
- `int STAR_API_CC CFG_SPFI_enableRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress)`  
*Enables the routing table entry for the logical address.*
- `int STAR_API_CC CFG_SPFI_disableRoutingTableEntry (STAR_DEVICE_ID deviceId, U8 logicalAddress)`  
*Disables the routing table entry for the logical address.*
- `int STAR_API_CC CFG_SPFI_enableRoutingTableEntryDeleteHeader (STAR_DEVICE_ID deviceId, U8 logicalAddress)`  
*Enables delete header on the routing table entry for the logical address.*
- `int STAR_API_CC CFG_SPFI_disableRoutingTableEntryDeleteHeader (STAR_DEVICE_ID deviceId, U8 logicalAddress)`  
*Disables delete header on the routing table entry for the logical address.*
- `int STAR_API_CC CFG_SPFI_getDeviceIdentifier (STAR_DEVICE_ID deviceId, _Out_ U32 *pDeviceIdentifier)`  
*Reads from the device identifier register and populates a U32.*
- `int STAR_API_CC CFG_SPFI_setDeviceIdentifier (STAR_DEVICE_ID deviceId, U32 deviceIdentifier)`  
*Sets the device identifier value.*
- `int STAR_API_CC CFG_SPFI_getBroadcastTimeout (STAR_DEVICE_ID deviceId, _Out_ double *pBroadcastTimeout)`  
*Reads from the broadcast timeout register and populates a double in microseconds.*
- `int STAR_API_CC CFG_SPFI_setBroadcastTimeout (STAR_DEVICE_ID deviceId, double broadcastTimeout)`  
*Sets the broadcast timeout value specified in microseconds.*
- `int STAR_API_CC CFG_SPFI_getBroadcastTimeoutMaximum (STAR_DEVICE_ID deviceId, _Out_ double *pMaximumTimeout)`  
*Gets the maximum supported broadcast timeout value specified in microseconds.*
- `int STAR_API_CC CFG_SPFI_getBroadcastTimeoutResolution (STAR_DEVICE_ID deviceId, _Out_ double *pResolution)`  
*Gets the broadcast timeout resolution in microseconds.*
- `int STAR_API_CC CFG_SPFI_getPortsReady (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsReady)`  
*Reads from the ports ready register and populates a U32 with a bitset indicating if each port is ready.*
- `int STAR_API_CC CFG_SPFI_getPortsWithError (STAR_DEVICE_ID deviceId, _Out_ U32 *pPortsWithError)`  
*Reads from the ports error register and populates a U32 with a bitset indicating if each port has an error.*
- `int STAR_API_CC CFG_SPFI_getLinksWithError (STAR_DEVICE_ID deviceId, _Out_ U32 *pLinksWithError)`  
*Reads from the links error register and populates a U32 with a bitset indicating if each link has an error.*
- `int STAR_API_CC CFG_SPFI_getCurrentTimeSlot (STAR_DEVICE_ID deviceId, _Out_ U8 *pCurrentTimeSlot)`  
*Reads from the time-slot register and populates a U8.*
- `int STAR_API_CC CFG_SPFI_getCurrentTime (STAR_DEVICE_ID deviceId, _Out_ U64 *pCurrentTime, _Out_ U8 *pSynchronised)`  
*Gets the current time and populates a U64.*
- `int STAR_API_CC CFG_SPFI_getSpaceWireBC (STAR_DEVICE_ID deviceId, _Out_ U8 *pRouterBC)`  
*Gets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.*
- `int STAR_API_CC CFG_SPFI_setSpaceWireBC (STAR_DEVICE_ID deviceId, U8 routerBC)`  
*Sets the broadcast channel used for SpaceWire time-code messages received over SpaceWire ports.*
- `int STAR_API_CC CFG_SPFI_getRouterTimeout (STAR_DEVICE_ID deviceId, _Out_ double *pTimeout)`

- Reads from the router timeout register and populates a double in microseconds.*

  - int `STAR_API_CC_CFG_SPFI_isRouterTimeoutEnabled` (STAR\_DEVICE\_ID deviceId, \_Out\_ U8 \*pEnabled)

*Gets the router timeout enabled flag.*
- int `STAR_API_CC_CFG_SPFI_setRouterTimeout` (STAR\_DEVICE\_ID deviceId, double routerTimeout)

*Sets the router timeout value specified in microseconds.*
- int `STAR_API_CC_CFG_SPFI_enableRouterTimeout` (STAR\_DEVICE\_ID deviceId)

*Enables the router timeout function.*
- int `STAR_API_CC_CFG_SPFI_disableRouterTimeout` (STAR\_DEVICE\_ID deviceId)

*Disables the router timeout function.*
- int `STAR_API_CC_CFG_SPFI_getRouterTimeoutMaximum` (STAR\_DEVICE\_ID deviceId, \_Out\_ double \*pMaximumTimeout)

*Gets the maximum supported router timeout value specified in microseconds.*
- int `STAR_API_CC_CFG_SPFI_getRouterTimeoutResolution` (STAR\_DEVICE\_ID deviceId, \_Out\_ double \*pResolution)

*Gets the router timeout resolution in microseconds.*
- int `STAR_API_CC_CFG_SPFI_getBroadcastTypesInformation` (STAR\_DEVICE\_ID deviceId, \_Out\_ SPFI\_BROADCAST\_TYPES\_INFO \*pBroadcastTypesInfo)

*Reads from the broadcast types register and populates a SPFI\_BROADCAST\_TYPES\_INFO value for use with the following sub-accessor functions:*
- U8 `STAR_API_CC_CFG_SPFI_getTimeSlotBroadcastType` (SPFI\_BROADCAST\_TYPES\_INFO broadcastTypesInfo)

*Gets the time-slot broadcast type value.*
- U8 `STAR_API_CC_CFG_SPFI_getTimeCodeBroadcastType` (SPFI\_BROADCAST\_TYPES\_INFO broadcastTypesInfo)

*Gets the time-code broadcast type value.*
- U8 `STAR_API_CC_CFG_SPFI_getSpaceWireInterruptBroadcastType` (SPFI\_BROADCAST\_TYPES\_INFO broadcastTypesInfo)

*Gets the SpaceWire interrupt broadcast type value.*
- int `STAR_API_CC_CFG_SPFI_setTimeSlotBroadcastType` (STAR\_DEVICE\_ID deviceId, U8 timeSlotValue)

*Sets the time-slot broadcast type value.*
- int `STAR_API_CC_CFG_SPFI_setTimeCodeBroadcastType` (STAR\_DEVICE\_ID deviceId, U8 timeCodeValue)

*Sets the time-code broadcast type value.*
- int `STAR_API_CC_CFG_SPFI_setSpaceWireInterruptBroadcastType` (STAR\_DEVICE\_ID deviceId, U8 interruptValue)

*Sets the SpaceWire interrupt broadcast type value.*
- int `STAR_API_CC_CFG_SPFI_getVCVN` (STAR\_DEVICE\_ID deviceId, U8 port, U8 virtualChannel, \_Out\_ U8 \*pVirtualNetwork)

*Reads from a virtual channel control register and populates a U8.*
- int `STAR_API_CC_CFG_SPFI_setVCVN` (STAR\_DEVICE\_ID deviceId, U8 port, U8 virtualChannel, U8 virtualNetwork)

*Sets the virtual network number for the virtual channel.*
- int `STAR_API_CC_CFG_SPFI_getVCStatus` (STAR\_DEVICE\_ID deviceId, U8 port, U8 virtualChannel, \_Out\_ SPFI\_VC\_STATUS \*pStatus)

*Reads from a virtual channel status register and populates a SPFI\_VC\_STATUS for use with the following sub-accessor functions:*
- U8 `STAR_API_CC_CFG_SPFI_isVCOverused` (SPFI\_VC\_STATUS status)

*Gets the overused state of a virtual channel according to the virtual channel status value.*
- U8 `STAR_API_CC_CFG_SPFI_isVCUnderused` (SPFI\_VC\_STATUS status)

*Gets the underused state of a virtual channel according to the virtual channel status value.*
- U8 `STAR_API_CC_CFG_SPFI_isVCOutputBufferHalfFull` (SPFI\_VC\_STATUS status)

*Gets the output buffer half full state of a virtual channel according to the virtual channel status value.*
- U8 `STAR_API_CC_CFG_SPFI_isVCOutputBufferFull` (SPFI\_VC\_STATUS status)

- Gets the output buffer full state of a virtual channel according to the virtual channel status value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCInputBufferNotEmpty \(SPFI\\_VC\\_STATUS status\)](#)

*Gets the input buffer not empty state of a virtual channel according to the virtual channel status value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCInputBufferHalfFull \(SPFI\\_VC\\_STATUS status\)](#)

*Gets the input buffer half full state of a virtual channel according to the virtual channel status value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCInputBufferFull \(SPFI\\_VC\\_STATUS status\)](#)

*Gets the input buffer full state of a virtual channel according to the virtual channel status value.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_getVCErrors \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, \\_↔ Out\\_ SPFI\\_VC\\_ERRORS \\*pErrors\)](#)

*Reads from a virtual channel errors register and populates a [SPFI\\_VC\\_ERRORS](#) for use with the following sub-accessor functions:*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCVNErrordetected \(SPFI\\_VC\\_ERRORS errors\)](#)

*Gets the VN error flag according to the virtual channel errors value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCAddressErrordetected \(SPFI\\_VC\\_ERRORS errors\)](#)

*Gets the address error flag according to the virtual channel errors value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCTimeoutdetected \(SPFI\\_VC\\_ERRORS errors\)](#)

*Gets the timeout error flag according to the virtual channel errors value.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_clearVCErrors \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel\)](#)

*Clears any virtual channel errors on the port.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_getVCQualityOfService \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtual↔ Channel, \\_Out\\_ SPFI\\_QUALITY\\_OF\\_SERVICE \\*pQualityOfService\)](#)

*Reads from a virtual channel quality of service register and populates a [SPFI\\_QUALITY\\_OF\\_SERVICE](#) for use with the following sub-accessor functions:*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_getVCBandwidthReservation \(SPFI\\_QUALITY\\_OF\\_SERVICE qualityOf↔ Service\)](#)

*Gets the VC bandwidth reservation according to the quality of service value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_getVCPriority \(SPFI\\_QUALITY\\_OF\\_SERVICE qualityOfService\)](#)

*Gets the VC priority according to the quality of service value.*

  - [U8 STAR\\_API\\_CC CFG\\_SPFI\\_isVCContinuousModeEnabled \(SPFI\\_QUALITY\\_OF\\_SERVICE qualityOf↔ Service\)](#)

*Gets the continuous mode flag according to the quality of service value.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_setVCBandwidthReservation \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, U8 bandwidth\)](#)

*Sets the virtual channel bandwidth reservation for the port.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_setVCPriority \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, U8 priority\)](#)

*Sets the virtual channel priority for the port.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_enableVCContinuousMode \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel\)](#)

*Enables continuous mode for the virtual channel specified.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_disableVCContinuousMode \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel\)](#)

*Disables continuous mode for the virtual channel specified.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_getVCTimeSlots \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, \\_Out\\_ U64 \\*pVCTimeSlots\)](#)

*Gets a 64-bit mask representing the allowed time-slots for the virtual channel specified.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_setVCTimeSlots \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, U64 timeSlots\)](#)

*Sets a 64-bit mask representing the allowed time-slots for the virtual channel specified.*

  - [int STAR\\_API\\_CC CFG\\_SPFI\\_getVCDataRate \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 virtualChannel, \\_Out\\_ U64 \\*pTxDataRate, \\_Out\\_ U64 \\*pRxDataRate\)](#)

*Gets the virtual channel transmit and receive data rate.*

- `int STAR_API_CC_CFG_SPFI_getPortInformation (STAR_DEVICE_ID deviceId, U8 port, _Out_ SPFI_PORT_INFO *pPortInfo)`  
*Reads from the port information register and populates a `SPFI_PORT_INFO` value for use with the following sub-accessor functions:*
- `SPFI_PORT_TYPE STAR_API_CC_CFG_SPFI_getPortType (SPFI_PORT_INFO portInfo)`  
*Gets the port type according to the port information value.*
- `U8 STAR_API_CC_CFG_SPFI_getPortVCCount (SPFI_PORT_INFO portInfo)`  
*Gets the number of virtual channels according to the port information value.*
- `U8 STAR_API_CC_CFG_SPFI_isPortReady (SPFI_PORT_INFO portInfo)`  
*Gets the SpaceFibre port ready flag according to the port information value.*
- `U8 STAR_API_CC_CFG_SPFI_isPortBroadcastErrorDetected (SPFI_PORT_INFO portInfo)`  
*Gets the SpaceFibre port broadcast error detected flag according to the port information value.*
- `U8 STAR_API_CC_CFG_SPFI_getAXIBC (SPFI_PORT_INFO portInfo)`  
*Gets the broadcast channel used by the broadcast interface associated with an AXI port.*
- `int STAR_API_CC_CFG_SPFI_clearPortBroadcastError (STAR_DEVICE_ID deviceId, U8 port)`  
*Clears any broadcast errors on the port.*
- `int STAR_API_CC_CFG_SPFI_setAXIBC (STAR_DEVICE_ID deviceId, U8 port, U8 axiBC)`  
*Sets the broadcast channel used by the broadcast interface associated with an AXI port.*
- `int STAR_API_CC_CFG_SPFI_getPortVCsWithErrors (STAR_DEVICE_ID deviceId, U8 port, _Out_ U32 *pVCsWithErrors)`  
*Reads from a virtual channel status register and populates a `U32` with a bitset indicating if each virtual channel has an error.*
- `int STAR_API_CC_CFG_SPFI_getLinkControlSettings (STAR_DEVICE_ID deviceId, U8 port, _Out_ SPFI_LINK_CONTROL_SETTINGS *pLinkControlSettings)`  
*Reads from the link control register and populates a `SPFI_LINK_CONTROL_SETTINGS` value for use with the following sub-accessor functions:*
- `U8 STAR_API_CC_CFG_SPFI_isLinkAutoStartEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link autostart enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkStartEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link start enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkResetEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link reset enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_isLinkInterfaceResetEnabled (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the link interface reset enabled state according to the link control settings value.*
- `U8 STAR_API_CC_CFG_SPFI_getLinkLanesRequestedCount (SPFI_LINK_CONTROL_SETTINGS linkControlSettings)`  
*Gets the number of lanes requested according to the link control settings value.*
- `int STAR_API_CC_CFG_SPFI_enableLinkAutoStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables autostart for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkAutoStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables autostart for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables start for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkStart (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables start for the link.*
- `int STAR_API_CC_CFG_SPFI_enableLinkReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Enables reset for the link.*
- `int STAR_API_CC_CFG_SPFI_disableLinkReset (STAR_DEVICE_ID deviceId, U8 port)`  
*Disables reset for the link.*

- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLinkInterfaceReset](#) (STAR\_DEVICE\_ID deviceId, U8 port)  
*Enables interface reset for the link.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_disableLinkInterfaceReset](#) (STAR\_DEVICE\_ID deviceId, U8 port)  
*Disables interface reset for the link.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_setLinkLanesRequestedCount](#) (STAR\_DEVICE\_ID deviceId, U8 port, U8 lanesRequestedCount)  
*Sets the lanes requested count for the link.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkStatus](#) (STAR\_DEVICE\_ID deviceId, U8 port, \_Out\_ SPFI\_LINK\_STATUS \*pLinkStatus)  
*Reads from the link status register and populates a SPFI\_LINK\_STATUS value for use with the following sub-accessor functions:*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkReady](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link ready status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkErrorDetected](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link error detected status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkProtocolErrorDetected](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link protocol error detected status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkIdle](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link idle status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkEventDetected](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link event detected status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkLanesEventDetected](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the lane event detected status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkTransmitting](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link transmitting status according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkReceiving](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link receiving status according to the link status value.*
- SPFI\_LINK\_ALIGNMENT\_STATE [STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkAlignmentState](#) (SPFI\_LINK\_STATUS linkStatus)  
*Gets the link alignment state according to the link status value.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_clearLinkProtocolError](#) (STAR\_DEVICE\_ID deviceId, U8 port)  
*Clears any link protocol errors on the port.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkEvents](#) (STAR\_DEVICE\_ID deviceId, U8 port, \_Out\_ SPFI\_LINK\_EVENTS \*pLinkEvents)  
*Reads from the link events register and populates a SPFI\_LINK\_EVENTS value for use with the following sub-accessor functions:*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkFarEndResetDetected](#) (SPFI\_LINK\_EVENTS linkEvents)  
*Gets the link far-end reset state according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkCRC8ErrorDetected](#) (SPFI\_LINK\_EVENTS linkEvents)  
*Gets the link CRC-8 error detected state according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkCRC16ErrorDetected](#) (SPFI\_LINK\_EVENTS linkEvents)  
*Gets the link CRC-16 error detected state according to the link status value.*
- U8 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkFrameErrorDetected](#) (SPFI\_LINK\_EVENTS linkEvents)  
*Gets the link frame error detected state according to the link status value.*
- U16 [STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkRetryCount](#) (SPFI\_LINK\_EVENTS linkEvents)  
*Gets the link retry count according to the link status value.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_clearLinkEvents](#) (STAR\_DEVICE\_ID deviceId, U8 port)  
*Clears any link events on the port.*
- int [STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkDebugSettings](#) (STAR\_DEVICE\_ID deviceId, U8 port, \_Out\_ SPFI\_LINK\_DEBUG\_SETTINGS \*pLinkDebugSettings)  
*Reads from the link debug register and populates a SPFI\_LINK\_DEBUG\_SETTINGS value for use with the following sub-accessor functions:*

- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkTxScramblingEnabled \(SPFI\\_LINK\\_DEBUG\\_SETTINGS linkDebugSettings\)](#)  
*Gets the link transmit scrambling enabled state according to the link debug value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkNearEndLoopbackEnabled \(SPFI\\_LINK\\_DEBUG\\_SETTINGS linkDebugSettings\)](#)  
*Gets the link near-end loopback enabled state according to the link debug value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLinkRxScramblingEnabled \(SPFI\\_LINK\\_DEBUG\\_SETTINGS linkDebugSettings\)](#)  
*Gets the link receive scrambling enabled state according to the link debug value.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLinkTxScrambling \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Enables transmit scrambling for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_disableLinkTxScrambling \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Disables transmit scrambling for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLinkNearEndLoopback \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Enables near-end loopback for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_disableLinkNearEndLoopback \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Disables near-end loopback for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLinkRxScrambling \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Enables receive scrambling for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_disableLinkRxScrambling \(STAR\\_DEVICE\\_ID deviceId, U8 port\)](#)  
*Disables receive scrambling for the link.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLinkActiveLanes \(STAR\\_DEVICE\\_ID deviceId, U8 port, \\_Out\\_ U32 \\*pActiveLanes\)](#)  
*Reads a link active lanes register and populates a U32 with a bitset indicating if each lane is active.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLaneControlSettings \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane, \\_Out\\_ SPFI\\_LANE\\_CONTROL\\_SETTINGS \\*pLaneControlSettings\)](#)  
*Reads from the lane control register and populates a SPFI\_LANE\_CONTROL\_SETTINGS value for use with the following sub-accessor functions:*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLaneResetEnabled \(SPFI\\_LANE\\_CONTROL\\_SETTINGS laneControlSettings\)](#)  
*Gets the lane reset enabled state according to the lane control settings value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLaneStandbyReason \(SPFI\\_LANE\\_CONTROL\\_SETTINGS laneControlSettings\)](#)  
*Gets the lane standby reason according to the lane control settings value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLaneTxEnabled \(SPFI\\_LANE\\_CONTROL\\_SETTINGS laneControlSettings\)](#)  
*Gets the lane transmit enabled state according to the lane control settings value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLaneRxEnabled \(SPFI\\_LANE\\_CONTROL\\_SETTINGS laneControlSettings\)](#)  
*Gets the lane receive enabled state according to the lane control settings value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLaneErrorRecoveryEnabled \(SPFI\\_LANE\\_CONTROL\\_SETTINGS laneControlSettings\)](#)  
*Gets the lane error recovery enabled state according to the lane control settings value.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLaneReset \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane\)](#)  
*Enables reset for the lane.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_disableLaneReset \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane\)](#)  
*Disables reset for the lane.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_setLaneStandbyReason \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane, U8 standbyReason\)](#)  
*Sets the standby reason for the SpaceFibre port of the device specified.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_enableLaneTx \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane\)](#)  
*Enables transmit for the lane.*



- int STAR\_API\_CC\_CFG\_SPFI\_disableLaneTx (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane)  
*Disables transmit for the lane.*
- int STAR\_API\_CC\_CFG\_SPFI\_enableLaneRx (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane)  
*Enables receive for the lane.*
- int STAR\_API\_CC\_CFG\_SPFI\_disableLaneRx (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane)  
*Disables receive for the lane.*
- int STAR\_API\_CC\_CFG\_SPFI\_enableLaneErrorRecovery (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane)  
*Enables error recovery for the lane.*
- int STAR\_API\_CC\_CFG\_SPFI\_disableLaneErrorRecovery (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane)  
*Disables error recovery for the lane.*
- int STAR\_API\_CC\_CFG\_SPFI\_getLaneStatus (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane, \_Out\_ SPFI\_LANE\_STATUS \*pLaneStatus)  
*Reads from the lane status register and populates a SPFI\_LANE\_STATUS value for use with the following sub-accessor functions:*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneActive (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane active state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneErrorDetected (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane error detected state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneEventDetected (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane event detected state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneNoSignalDetected (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane no signal detected state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneLOSReasonRx (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane receive loss of signal reason according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneStandbyReasonRx (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane receive standby reason according to the lane status value.*
- SPFI\_LANE\_STATE STAR\_API\_CC\_CFG\_SPFI\_getLaneState (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneStarted (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane started state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneTxOnly (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane transmit only state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneRxOnly (SPFI\_LANE\_STATUS laneStatus)  
*Gets the lane receive only state according to the lane status value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_getLaneFarEndINIT3Flags (SPFI\_LANE\_STATUS laneStatus)  
*Gets the flags of the last received INIT3 word according to the lane status value.*
- int STAR\_API\_CC\_CFG\_SPFI\_getLaneEvents (STAR\_DEVICE\_ID deviceID, U8 port, U8 lane, \_Out\_ SPFI\_LANE\_EVENTS \*pLaneEvents)  
*Reads from the lane events register and populates a SPFI\_LANE\_EVENTS value for use with the following sub-accessor functions:*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneInitTimeoutDetected (SPFI\_LANE\_EVENTS laneEvents)  
*Gets the lane initialisation timeout detected state according to the lane events value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneFarEndStandbyDetected (SPFI\_LANE\_EVENTS laneEvents)  
*Gets the far-end standby state according to the lane events value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneFarEndLOSDetected (SPFI\_LANE\_EVENTS laneEvents)  
*Gets the far-end loss of signal state according to the lane events value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneFarEndErrorBurstDetected (SPFI\_LANE\_EVENTS laneEvents)  
*Gets the far-end burst state according to the lane events value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneFarEndINIT1RxInActiveDetected (SPFI\_LANE\_EVENTS laneEvents)  
*Gets the far-end INIT1 receive in active state according to the lane events value.*
- U8 STAR\_API\_CC\_CFG\_SPFI\_isLaneInit1RxDetected (SPFI\_LANE\_EVENTS laneEvents)

- Gets the INIT1 receive detected state according to the lane events value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_isLaneINIT2RxDetected \(SPFI\\_LANE\\_EVENTS laneEvents\)](#)
- Gets the INIT2 receive detected state according to the lane events value.*
- [U16 STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLaneRxErrorCount \(SPFI\\_LANE\\_EVENTS laneEvents\)](#)
- Gets the lane receive error count according to the lane events value.*
- [U8 STAR\\_API\\_CC\\_CFG\\_SPFI\\_getLaneDisconnectCount \(SPFI\\_LANE\\_EVENTS laneEvents\)](#)
- Gets the lane disconnect count according to the lane events value.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_clearLaneEvents \(STAR\\_DEVICE\\_ID deviceId, U8 port, U8 lane\)](#)
- Clears any events on the lane.*
- [U64 \\*STAR\\_API\\_CC\\_CFG\\_SPFI\\_getSpFiBitRates \(STAR\\_DEVICE\\_ID deviceId, \\_Out\\_ U32 \\*pCount\)](#)
- Returns an array of the SpaceFibre lane data signalling rates supported by the device specified in bit/s.*
- [void STAR\\_API\\_CC\\_CFG\\_SPFI\\_destroySpFiBitRates \(U64 \\*pBitRates\)](#)
- Releases the array of SpaceFibre lane data signalling rates returned by the [CFG\\_SPFI\\_getSpFiBitRates\(\)](#) function.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_getSpFiBitRate \(STAR\\_DEVICE\\_ID deviceId, U8 port, \\_Out\\_ U64 \\*pBitRate\)](#)
- Gets the SpaceFibre port lane data signalling rate.*
- [int STAR\\_API\\_CC\\_CFG\\_SPFI\\_setSpFiBitRate \(STAR\\_DEVICE\\_ID deviceId, U8 port, U64 bitRate\)](#)
- Sets the SpaceFibre port lane data signalling rate.*

### 9.20.1 Detailed Description

Functions used to configure SpaceFibre devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.21 cfg\_api\_spfi\_types.h File Reference

Types used with the STAR-Dundee SpaceFibre Configuration API.

```
#include "cfg_api_router_types.h"
```

### Typedefs

- typedef unsigned long long [U64](#)
- A type that can be used to represent an unsigned 64-bit number.*
- typedef [REGISTER\\_SPFI\\_DEVICE\\_INFO](#)
- The type used to represent SpaceFibre device information.*
- typedef [REGISTER\\_SPFI\\_DEVICE\\_IDENTIFIER\\_INFO](#)
- The type used to represent SpaceFibre device identification information.*
- typedef [REGISTER\\_SPFI\\_ROUTING\\_TABLE\\_ENTRY\\_FLAGS](#)
- The type used to represent flags for a SpaceFibre routing table entry.*
- typedef [REGISTER\\_SPFI\\_BROADCAST\\_TYPES\\_INFO](#)



- The type used to represent broadcast types information.*

  - typedef REGISTER SPFI\_VC\_STATUS

*The type used to represent virtual channel status information.*
- typedef REGISTER SPFI\_VC\_ERRORS

*The type used to represent virtual channel error information.*
- typedef REGISTER SPFI\_QUALITY\_OF\_SERVICE

*The type used to represent virtual channel quality of service information.*
- typedef REGISTER SPFI\_PORT\_INFO

*The type used to represent SpaceFibre port information.*
- typedef REGISTER SPFI\_LINK\_CONTROL\_SETTINGS

*The type used to represent SpaceFibre link control settings.*
- typedef REGISTER SPFI\_LINK\_STATUS

*The type used to represent SpaceFibre link status.*
- typedef REGISTER SPFI\_LINK\_EVENTS

*The type used to represent SpaceFibre link events.*
- typedef REGISTER SPFI\_LINK\_DEBUG\_SETTINGS

*The type used to represent SpaceFibre link debug settings.*
- typedef REGISTER SPFI\_LANE\_CONTROL\_SETTINGS

*The type used to represent SpaceFibre lane control settings.*
- typedef REGISTER SPFI\_LANE\_STATUS

*The type used to represent SpaceFibre lane status.*
- typedef REGISTER SPFI\_LANE\_EVENTS

*The type used to represent SpaceFibre lane events.*

## Enumerations

- enum SPFI\_NODE\_TYPE { SPFI\_NODE\_TYPE\_INVALID = 0, SPFI\_NODE\_TYPE\_CONFIG = 1, SPFI\_NODE\_TYPE\_SOURCE\_AND\_DESTINATION = 2, SPFI\_NODE\_TYPE\_OTHER = 3 }

*SpaceFibre node type.*
- enum SPFI\_PORT\_TYPE { SPFI\_PORT\_TYPE\_UNDEFINED = -1, SPFI\_PORT\_TYPE\_SPACEFIBRE = 0, SPFI\_PORT\_TYPE\_AXI = 1, SPFI\_PORT\_TYPE\_SPACEWIRE = 2 }

*SpaceFibre port type.*
- enum SPFI\_LINK\_ALIGNMENT\_STATE { SPFI\_LINK\_ALIGNMENT\_STATE\_UNDEFINED = -1, SPFI\_LINK\_ALIGNMENT\_STATE\_NOT\_READY = 0, SPFI\_LINK\_ALIGNMENT\_STATE\_NEAR\_END\_READY = 1, SPFI\_LINK\_ALIGNMENT\_STATE\_BOTH\_ENDS\_READY = 2 }

*SpaceFibre link alignment state.*
- enum SPFI\_LANE\_STATE { SPFI\_LANE\_STATE\_UNDEFINED = -1, SPFI\_LANE\_STATE\_COLD\_RESET = 0, SPFI\_LANE\_STATE\_CLEAR\_LINE = 1, SPFI\_LANE\_STATE\_DISABLED = 2, SPFI\_LANE\_STATE\_WAIT = 3, SPFI\_LANE\_STATE\_STARTED = 4, SPFI\_LANE\_STATE\_INVERT\_RX\_POLARITY = 5, SPFI\_LANE\_STATE\_CONNECTING = 6, SPFI\_LANE\_STATE\_CONNECTED = 7, SPFI\_LANE\_STATE\_ACTIVE = 8, SPFI\_LANE\_STATE\_LOSS\_OF\_SIGNAL = 9, SPFI\_LANE\_STATE\_PREPARE\_STANDBY = 10 }

*SpaceFibre lane state.*

### 9.21.1 Detailed Description

Types used with the STAR-Dundee SpaceFibre Configuration API.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.22 common.h File Reference

STAR-Dundee API macros and types required by STAR API modules.

```
#include <string.h>
#include "star-dundee_annotations.h"
```

**Macros**

- `#define STAR_API_MODULE_NAME "STAR-API"`  
*The name of the module, used for logging purposes.*
- `#define STAR_API_CC`  
*The calling convention used by functions exported by the API.*

### 9.22.1 Detailed Description

STAR-Dundee API macros and types required by STAR API modules.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of macros, structures and types required by modules in the STAR-Dundee software stack.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.22.2 Macro Definition Documentation

#### 9.22.2.1 `#define STAR_API_CC`

The calling convention used by functions exported by the API.

Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

**Examples:**

[channel\\_callback\\_example.c](#), [packet\\_subsystem\\_example.c](#), [rmap\\_target\\_example.c](#), [timestamp\\_test.c](#), and [transfer\\_callback\\_example.c](#).

**9.22.2.2 #define STAR\_API\_MODULE\_NAME "STAR-API"**

The name of the module, used for logging purposes.

**9.23 general.h File Reference**

General functions provided by STAR-API.

```
#include "common.h"
#include "types.h"
#include "stream_item_types.h"
#include "version.h"
```

**Data Structures**

- struct [STAR\\_IP\\_ADDRESS\\_TYPE](#)  
A structure consisting of: [More...](#)

**Enumerations**

- enum [STAR\\_IP\\_VERSION\\_TYPE](#) { [STAR\\_IP\\_VERSION\\_NOT\\_DEFINED](#) = 0, [STAR\\_IPv4](#) = 4, [STAR\\_IPv6](#) = 16 }  
The Internet Protocol version.

**Functions**

- [STAR\\_VERSION\\_INFO \\*STAR\\_API\\_CC STAR\\_getApiVersion \(void\)](#)  
Gets the Version of STAR-API.
- [void STAR\\_API\\_CC STAR\\_destroyVersionInfo \(\\_Post\\_ptr\\_invalid\\_ STAR\\_VERSION\\_INFO \\*pVersionInfo\)](#)  
Frees a version information structure previously created by a call to [STAR\\_getApiVersion\(\)](#), [STAR\\_getDriverVersion\(\)](#) or [STAR\\_getDeviceFirmwareVersion\(\)](#).
- [\\_Ret\\_opt\\_cap\\_ count STAR\\_VERSION\\_INFO \\*STAR\\_API\\_CC STAR\\_getAllVersions \(\\_Out\\_ U32 \\*count\)](#)  
Gets an array of all the version info structures provided by all modules of STAR-System.
- [void STAR\\_API\\_CC STAR\\_destroyVersionInfoList \(\\_Post\\_ptr\\_invalid\\_ STAR\\_VERSION\\_INFO \\*pVersionInfoList\)](#)  
Frees a version information list previously created by a call to [STAR\\_getAllVersions\(\)](#).
- [int STAR\\_API\\_CC STAR\\_setApplicationName \(\\_In\\_z\\_ char \\*name\)](#)  
Sets the name of the application using STAR-API.
- [\\_Check\\_return\\_ \\_Ret\\_opt\\_z\\_ char \\*STAR\\_API\\_CC STAR\\_getApplicationName \(void\)](#)  
Gets the name of the calling process (this process), set by [STAR\\_setApplicationName\(\)](#).
- [STAR\\_APP\\_ID STAR\\_API\\_CC STAR\\_getApplicationID \(void\)](#)  
Gets the application ID of the calling process.
- [\\_Check\\_return\\_ \\_Ret\\_opt\\_z\\_ char \\*STAR\\_API\\_CC STAR\\_getApplicationNameForID \(STAR\\_APP\\_ID appld\)](#)  
Gets the name of the application with the given ID.
- [void STAR\\_API\\_CC STAR\\_destroyString \(\\_Post\\_ptr\\_invalid\\_ \\_In\\_z\\_ char \\*string\)](#)

- Destroy a string returned by STAR-API, such as from `STAR_getApplicationName()` or `STAR_getDeviceTypeAsString()`.*
- `_Ret_opt_cap_ count STAR_DRIVER_ID *STAR_API_CC STAR_getDriverList (int physicalDeviceDrivers, int virtualDeviceDrivers, _Out_ U32 *count)`  
*Gets an array of driver identifiers for all drivers of the requested type(s).*
  - `void STAR_API_CC STAR_destroyDriverList (_Post_ptr_invalid_ STAR_DRIVER_ID *pDriverList)`  
*Frees a driver list previously created by a call to `STAR_getDriverList()`.*
  - `_Check_return_ STAR_DRIVER_LISTENER_ID STAR_API_CC STAR_registerDriverListener (STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`  
*Registers a Call-back function that will be called whenever a Driver is added or removed.*
  - `int STAR_API_CC STAR_unregisterDriverListener (STAR_DRIVER_LISTENER_ID listenerId)`  
*Unregister a Call-back function that was previously registered to be called whenever a driver is added or removed.*
  - `STAR_VERSION_INFO *STAR_API_CC STAR_getDriverVersion (STAR_DRIVER_ID driverId)`  
*Gets the Version of a given driver.*
  - `STAR_DRIVER_TYPE STAR_API_CC STAR_getDriverType (STAR_DRIVER_ID driverId)`  
*Gets the type (USB/PCI/specific virtual type identifier) of a given driver.*
  - `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceList (_Out_ U32 *count)`  
*Gets an array of identifiers for all devices present for all drivers.*
  - `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForType (STAR_DEVICE_TYPE deviceType, _Out_ U32 *pCount)`  
*Gets an array of identifiers for all devices present for the given device type.*
  - `_Ret_opt_cap_ pCount STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForTypes (STAR_DEVICE_TYPE deviceTypes[], U32 numRequestedTypes, _Out_ U32 *pCount)`  
*Gets an array of identifiers for all devices present for the given device types in an array.*
  - `_Ret_opt_cap_ count STAR_DEVICE_ID *STAR_API_CC STAR_getDeviceListForDriver (STAR_DRIVER_ID driverId, _Out_ U32 *count)`  
*Gets an array of device identifiers for all devices present for a specific driver.*
  - `void STAR_API_CC STAR_destroyDeviceList (_Post_ptr_invalid_ STAR_DEVICE_ID *pDeviceList)`  
*Frees a device list previously created by a call to `STAR_getDeviceList()` or `STAR_getDeviceListForDriver()`.*
  - `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListener (_In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`  
*Registers a Call-back function that will be called whenever a device is added or removed.*
  - `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`  
*Registers a Call-back function that will be called whenever a device for a specific driver is added or removed.*
  - `_Check_return_ STAR_DEVICE_LISTENER_ID STAR_API_CC STAR_registerDeviceListenerForDevice (STAR_DEVICE_ID deviceId, _In_ STAR_DeviceEventFunc listenerFunc, _In_opt_ void *pContextInfo)`  
*Registers a Call-back function that will be called whenever a specific device is removed.*
  - `int STAR_API_CC STAR_unregisterDeviceListener (STAR_DEVICE_LISTENER_ID listenerId)`  
*Unregister a Call-back function that was previously registered to be called whenever a device is added or removed.*
  - `int STAR_API_CC STAR_resetDevice (STAR_DEVICE_ID deviceId)`  
*Resets a device.*
  - `STAR_VERSION_INFO *STAR_API_CC STAR_getDeviceFirmwareVersion (STAR_DEVICE_ID deviceId)`  
*Gets the version of a device's firmware.*
  - `STAR_BUS_TYPE STAR_API_CC STAR_getDeviceBusType (STAR_DEVICE_ID deviceId)`  
*Gets the bus type (PCI,USB, Virtual, etc) for the device.*
  - `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceBusTypeAsString (STAR_DEVICE_ID deviceId)`  
*Gets a string representation of a device's bus type.*
  - `int STAR_API_CC STAR_getDeviceIndex (STAR_DEVICE_ID deviceId)`  
*Gets the device index by Driver.*

- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceType (STAR_DEVICE_ID deviceId)`  
*Gets the device's type identifier.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceTypeAsString (STAR_DEVICE_ID deviceId)`  
*Gets a string representation of a device's type.*
- `STAR_DRIVER_ID STAR_API_CC STAR_getDeviceDriver (STAR_DEVICE_ID deviceId)`  
*Gets the identifier of the device's driver.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceName (STAR_DEVICE_ID deviceId)`  
*Gets the name of the device identified by a given identifier.*
- `int STAR_API_CC STAR_setDeviceName (STAR_DEVICE_ID deviceId, _In_z_ char *newName)`  
*Sets the name of the device (friendly name).*
- `int STAR_API_CC STAR_resetDeviceName (STAR_DEVICE_ID deviceId)`  
*Resets a device's name to its default value.*
- `_Check_return_ _Ret_opt_z_ char *STAR_API_CC STAR_getDeviceSerialNumber (STAR_DEVICE_ID deviceId)`  
*Gets the serial number (unique alphanumeric identifier) of the device identified by a given identifier.*
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceTxRxCapabilities (STAR_DEVICE_ID deviceId)`  
*Determine whether the device is capable of transmitting and receiving packets on channels.*
- `STAR_DEVICE_TYPE STAR_API_CC STAR_getDeviceConfigCapabilities (STAR_DEVICE_ID deviceId)`  
*Determine whether the device is capable of being configured.*
- `STAR_CHANNEL_MASK STAR_API_CC STAR_getDeviceChannels (STAR_DEVICE_ID deviceId)`  
*Gets the channels on a device.*
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelBetweenLocalDevices (STAR_DEVICE_ID deviceA, unsigned char channelA, STAR_DEVICE_ID deviceB, unsigned char channelB)`  
*Open a channel between two devices on the specified channel numbers.*
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDevice (STAR_DEVICE_ID device, STAR_CHANNEL_DIRECTION direction, unsigned char channelNumber, int isQueued)`  
*Open a channel to a device on the specified channel number.*
- `_Check_return_ STAR_CHANNEL_ID STAR_API_CC STAR_openChannelToLocalDeviceEx (_In_ STAR_DEVICE_ID device, _In_ STAR_CHANNEL_DIRECTION direction, _In_ unsigned char channelNumber, _In_ int isQueued)`  
*This function is an alternative to `STAR_openChannelToLocalDevice()` that uses an optimised receive strategy.*
- `int STAR_API_CC STAR_closeChannel (STAR_CHANNEL_ID channelId)`  
*Close a previously opened channel.*
- `STAR_CHANNEL_TYPE STAR_API_CC STAR_isChannelOpen (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`  
*Returns whether a given channel on a device is open and, if it is open, whether it is connected to a device or an application.*
- `_Check_return_ STAR_CHANNEL_DIRECTION STAR_API_CC STAR_getChannelDirection (STAR_CHANNEL_ID channelId)`  
*Get the direction information of an open channel.*
- `STAR_DEVICE_ID STAR_API_CC STAR_getLocalDeviceAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`  
*Returns the id of a device attached to a given channel on a given device.*
- `STAR_APP_ID STAR_API_CC STAR_getApplicationAttachedToChannel (STAR_DEVICE_ID deviceId, unsigned int channelNumber)`  
*Returns the ID of an application attached to a given channel on a given device.*
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListener (_In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`  
*Registers a Call-back function that will be called whenever any channel is opened or closed.*
- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDriver (STAR_DRIVER_ID driverId, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

*Registers a Call-back function that will be called whenever a channel is opened or closed on a device which has the specific driver.*

- `_Check_return_ STAR_CHANNEL_LISTENER_ID STAR_API_CC STAR_registerChannelListenerForDevice (STAR_DEVICE_ID deviceld, _In_ STAR_ChannelEventFunc listenerFunc, _In_opt_ void *pContextInfo, int notifyForCurrent)`

*Registers a Call-back function that will be called whenever a channel on the specific device is opened or closed.*

- `int STAR_API_CC STAR_unregisterChannelListener (STAR_CHANNEL_LISTENER_ID listenerId)`

*Unregister a Call-back function that was previously registered to be called whenever a channel is opened or closed.*

- `int STAR_API_CC STAR_isDriverVirtual (STAR_DRIVER_ID driverId)`

*Gets whether or not a given driver is virtual.*

- `int STAR_API_CC STAR_isDeviceVirtual (STAR_DEVICE_ID deviceld)`

*Gets whether or not a given device is virtual.*

- `STAR_DRIVER_ID STAR_API_CC STAR_openEthernetDriver (void)`

*Opens the Ethernet driver.*

- `int STAR_API_CC STAR_closeEthernetDriver (STAR_DRIVER_ID driverId)`

*Closes the Ethernet driver.*

- `STAR_DEVICE_ID STAR_API_CC STAR_openEthernetDevice (unsigned char ipAddr[4])`

*Opens an Ethernet connected device.*

- `int STAR_API_CC STAR_closeEthernetDevice (STAR_DEVICE_ID deviceld)`

*Closes an open Ethernet connected device.*

- `_Check_return_ int STAR_API_CC STAR_getDeviceIPAddress (STAR_DEVICE_ID deviceld, _Out_ bytcap_(IPAddressLength) U8 *pIPAddress, U8 IPAddressLength)`

*Gets the IP address of a specific Ethernet device (i.e.*

- `_Check_return_ STAR_IP_ADDRESS_TYPE **STAR_API_CC STAR_getIPAddresses (STAR_DRIVER_ID driverId, _Out_ int *pNumDevices)`

*Scans for all attached Ethernet devices and stores their IP addresses in an array of IP address structures of STAR\_IP\_ADDRESS\_TYPE.*

- `int STAR_API_CC STAR_destroyIPAddresses (_Post_ptr_invalid_ STAR_IP_ADDRESS_TYPE **ppIPAddrs, _In_ U8 numDevices)`

*Frees the memory allocated by STAR\_getIPAddresses().*

### 9.23.1 Detailed Description

General functions provided by STAR-API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the declarations of the general functions provided by STAR-API, the STAR-Dundee software API, relating to the API itself, along with the functions used for managing channels.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.24 packet\_subsystem.h File Reference

Definitions of the functions provided by the PCI Mk2 packet subsystem API.

```
#include "star-api.h"
#include <limits.h>
```

## Data Structures

- struct [PKT\\_SUBSYS\\_STATISTICS](#)

Statistics that can be obtained from the PCI Mk2 device packet sink subsystem. [More...](#)

## Macros

- `#define` [PKT\\_SUBSYS\\_MODULE\\_NAME](#) "PACKET\_SUBSYSTEM"

The name of the module, used for logging purposes.

## Typedefs

- typedef unsigned long long [U64](#)
- typedef void([STAR\\_API\\_CC](#) \* [PKT\\_SUBSYS\\_StatisticsFunc](#)) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, [PKT\\_SUBSYS\\_STATISTICS](#) statistics)

The function type used by the statistics loop function, which is called when a monitored subsystem's statistics are updated.

- typedef [U32](#) [STATISTICS\\_LOOP\\_ID](#)

Type of identifier used to identify a statistics polling thread.

## Enumerations

- enum [PKT\\_SUBSYS\\_PATTERN](#) {  
[PKT\\_SUBSYS\\_PATTERN\\_ALL\\_ZEROES](#), [PKT\\_SUBSYS\\_PATTERN\\_ALL\\_ONES](#), [PKT\\_SUBSYS\\_PATTERN\\_ALTERNATING\\_BITS](#), [PKT\\_SUBSYS\\_PATTERN\\_INCREMENTING\\_BYTES](#),  
[PKT\\_SUBSYS\\_PATTERN\\_DECREMENTING\\_BYTES](#), [PKT\\_SUBSYS\\_PATTERN\\_WALKING\\_ONES](#),  
[PKT\\_SUBSYS\\_PATTERN\\_WALKING\\_ZEROES](#) }

Available patterns to use with [PKT\\_SUBSYS\\_fillBufferWithPattern\(\)](#)

## Functions

- void [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_fillBufferWithPattern](#) ([PKT\\_SUBSYS\\_PATTERN](#) pattern, [U16](#) bufferSize, [\\_Out\\_bytecap\\_\(bufferSize\)](#) void \*pBuffer)
- Convenience function that fills a user provided buffer with a known pattern.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_writeMemory](#) ([STAR\\_DEVICE\\_ID](#) deviceId, [U16](#) address, [U16](#) writeSize, [\\_In\\_bytecount\\_\(writeSize\)](#) void \*pBuffer)
- Writes a buffer to the PCI Mk2's dual port memory.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_readMemory](#) ([STAR\\_DEVICE\\_ID](#) deviceId, [U16](#) address, [U16](#) readSize, [\\_Inout\\_bytecap\\_\(readSize\)](#) void \*pBuffer)
- Reads from the PCI Mk2's dual port memory to a user allocated buffer.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_setPacketGeneratorFormat](#) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, [U16](#) headerPointer, [U16](#) headerLength, [U16](#) bodyPointer, [U16](#) bodyLength, [U32](#) packetLength, [STAR\\_EOP\\_TYPE](#) eop, [U16](#) gap)
- Sets up the format of the packet to be inserted by the packet generator into the router.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_setGeneratorBlockingParameters](#) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, [U16](#) blockFrequency, [U16](#) blockDuration)
- Sets up the packet generation subsystem blocking parameters.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_startPacketGeneration](#) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, [U16](#) sequenceLength)
- Starts the packet generator inserting packets into the router.
- int [STAR\\_API\\_CC](#) [PKT\\_SUBSYS\\_waitForPacketGenerationSequenceComplete](#) ([STAR\\_DEVICE\\_ID](#) deviceId, unsigned int subsystem, unsigned int timeout)



- Blocks the current thread until packet generation has stopped.*

  - int `STAR_API_CC_PKT_SUBSYS_stopPacketGeneration` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Stops the packet generator inserting packets into the router.*

  - int `STAR_API_CC_PKT_SUBSYS_setPacketSinkParameters` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, U16 memoryPointer, U16 memoryLength)

*Sets up the circular buffer for the packet sink subsystem.*

  - int `STAR_API_CC_PKT_SUBSYS_setSinkBlockingParameters` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, U16 blockFrequency, U16 blockDuration)

*Sets up the packet sink subsystem blocking parameters.*

  - int `STAR_API_CC_PKT_SUBSYS_startSink` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Starts the packet sink reading packets into memory.*

  - int `STAR_API_CC_PKT_SUBSYS_stopSink` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Stops the packet sink reading packets into memory.*

  - int `STAR_API_CC_PKT_SUBSYS_getSinkDataPointers` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, U16 \*startPointer, U16 \*endPointer)

*Gets the address for the start and end of valid data in the circular buffer in device memory.*

  - int `STAR_API_CC_PKT_SUBSYS_startPacketChecking` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, int disableSinkOnError, U16 disableSinkdelay)

*Starts a packet checker running on a packet sink.*

  - int `STAR_API_CC_PKT_SUBSYS_stopPacketChecking` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Stops the packet checker.*

  - int `STAR_API_CC_PKT_SUBSYS_waitForPacketCheckingError` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Blocks the current thread until either a character mismatch is detected and the packet checker is automatically stopped, or the user manually stops the packet checker.*

  - int `STAR_API_CC_PKT_SUBSYS_cancelWaitsForPacketCheckingError` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem)

*Cancels all waits started by `PKT_SUBSYS_waitForPacketCheckingError()` for packet checker errors on a given device and subsystem.*

  - int `STAR_API_CC_PKT_SUBSYS_setPacketCheckingFormat` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, U16 headerPointer, U16 headerLength, U16 bodyPointer, U16 bodyLength, U32 packetLength, STAR\_EOP\_TYPE eop)

*Sets up the packet checker to check packets match the format of packets provided by user entered parameters.*

  - int `STAR_API_CC_PKT_SUBSYS_getPacketCheckingMismatchCount` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, U64 \*errorCount)

*Gets the number of mismatched data characters observed on the SpaceWire interface.*

  - int `STAR_API_CC_PKT_SUBSYS_getStatistics` (STAR\_DEVICE\_ID deviceId, unsigned int subsystem, PK↵T\_SUBSYS\_STATISTICS \*pStatistics)

*Gets statistics for a given subsystem.*

  - `STATISTICS_LOOP_ID STAR_API_CC_PKT_SUBSYS_startGetStatisticsLoop` (STAR\_DEVICE\_ID↵ deviceId, unsigned int subsystem, PKT\_SUBSYS\_StatisticsFunc statisticsHandler)

*Starts a thread that polls statistics on a given subsystem, and calls a given function with the updated statistics when they are retrieved.*

  - void `STAR_API_CC_PKT_SUBSYS_stopGetStatisticsLoop` (STATISTICS\_LOOP\_ID loopId)

*Stops a statistics gathering loop set up by `PKT_SUBSYS_startGetStatisticsLoop()`.*

### 9.24.1 Detailed Description

Definitions of the functions provided by the PCI Mk2 packet subsystem API.



**Author**

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

**9.24.2 Macro Definition Documentation****9.24.2.1 #define PKT\_SUBSYS\_MODULE\_NAME "PACKET\_SUBSYSTEM"**

The name of the module, used for logging purposes.

**9.25 rmap\_packet\_library.h File Reference**

Declarations of the functions provided by the STAR-Dundee RMAP Packet Library.

```
#include "star-dundee_types.h"
```

**Data Structures**

- struct [RMAP\\_PACKET](#)  
*A structure used to describe an RMAP packet. [More...](#)*

**Macros**

- #define [RMAPPACKETLIBRARY\\_CC](#)  
*The calling convention used by functions exported by the RMAP Packet Library.*
- #define [RMAP\\_PROTOCOL\\_IDENTIFIER](#) 1  
*The value used in the protocol identifier field of all RMAP packets.*
- #define [PNP\\_PROTOCOL\\_IDENTIFIER](#) 3  
*The value used in the protocol identifier field of all SpaceWire-PnP packets.*
- #define [RMAP\\_RESERVED\\_BIT](#) 0x80  
*A mask for the reserved bit of the Instruction field.*
- #define [RMAP\\_COMMAND\\_BIT](#) 0x40  
*A packet is a command if the bit specified by this mask is set in the Instruction field.*
- #define [RMAP\\_WRITE\\_OPERATION\\_BIT](#) 0x20  
*A packet is a write operation if the bit specified by this mask is set in the Instruction field.*
- #define [RMAP\\_VERIFY\\_BEFORE\\_WRITE\\_BIT](#) 0x10  
*An operation verifies before writing if the bit specified by this mask is set in the Instruction field.*
- #define [RMAP\\_REPLY\\_BIT](#) 0x08  
*An operation is acknowledged if the bit specified by this mask is set in the Instruction field.*
- #define [RMAP\\_INCREMENT\\_ADDRESS\\_BIT](#) 0x04  
*A read or write address is incremented if the bit specified by this mask is set in the Instruction field.*
- #define [RMAP\\_REPLY\\_ADDRESS\\_LENGTH\\_BITS](#) 0x03  
*A mask for the bits in the Instruction field containing the length of the reply address.*

- `#define RMAP_GET_VERSION_MAJOR(versionInfo) (((versionInfo) & 0xff000000) >> 24)`  
*Return the major version number from the specified version information.*
- `#define RMAP_GET_VERSION_MINOR(versionInfo) (((versionInfo) & 0x00ff0000) >> 16)`  
*Return the minor version number from the specified version information.*
- `#define RMAP_GET_VERSION_EDIT(versionInfo) (((versionInfo) & 0x0000ffc0) >> 6)`  
*Return the edit number from the specified version information.*
- `#define RMAP_GET_VERSION_PATCH(versionInfo) ((versionInfo) & 0x0000003f)`  
*Return the patch level from the specified version information.*

## Enumerations

- `enum RMAP_STATUS {`  
`RMAP_SUCCESS = 0x00, RMAP_GENERAL_ERROR = 0x01, RMAP_UNUSED_PACKET_TYPE_OR_↵`  
`COMMAND_CODE = 0x02, RMAP_INVALID_KEY = 0x03,`  
`RMAP_INVALID_DATA_CRC = 0x04, RMAP_EARLY_EOP = 0x05, RMAP_TOO_MUCH_DATA = 0x06,`  
`RMAP_EEP = 0x07,`  
`RMAP_VERIFY_BUFFER_OVERRUN = 0x09, RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHO_↵`  
`RISED = 0x0a, RMAP_RMW_DATA_LENGTH_ERROR = 0x0b, RMAP_INVALID_TARGET_LOGICAL_A_↵`  
`DDRESS = 0x0c,`  
`RMAP_INVALID_STATUS = 0xff }`  
*Possible values for the status of RMAP operations.*
- `enum RMAP_PACKET_TYPE {`  
`RMAP_WRITE_COMMAND, RMAP_WRITE_REPLY = (RMAP_WRITE_OPERATION_BIT), RMAP_REA_↵`  
`D_COMMAND = (RMAP_COMMAND_BIT), RMAP_READ_REPLY = (0),`  
`RMAP_READ_MODIFY_WRITE_COMMAND, RMAP_READ_MODIFY_WRITE_REPLY = (RMAP_VERIF_↵`  
`Y_BEFORE_WRITE_BIT), RMAP_INVALID_PACKET_TYPE = (0xff) }`  
*Possible values for the packet type.*

## Functions

- `U32 RMAPPACKETLIBRARY_CC RMAP_GetVersion (void)`  
*Return the current version information for the RMAP Packet Library.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRC (void *pBuffer, unsigned long len)`  
*Calculate an 8-bit CRC for the given buffer.*
- `U8 RMAPPACKETLIBRARY_CC RMAP_CalculateCRCWithSeed (void *pBuffer, unsigned long len, U8 crc)`  
*Calculate an 8-bit CRC for the given buffer, starting with a seed CRC.*
- `char RMAPPACKETLIBRARY_CC RMAP_IsCRCValid (void *pBuffer, unsigned long len, U8 crc)`  
*Determine if the specified 8-bit CRC is valid for the given buffer.*
- `RMAP_STATUS RMAPPACKETLIBRARY_CC RMAP_CheckPacketValid (void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)`  
*Check that the packet specified is in the correct format for an RMAP packet.*
- `RMAP_STATUS RMAPPACKETLIBRARY_CC RMAP_CheckPacketValidIgnoreProtocol (void *pRawPacket, unsigned long packetLength, RMAP_PACKET *pPacketStruct, char checkPacketTooLong)`  
*Check that the packet specified is in the correct format for an RMAP packet, but do not check that the protocol identifier used by the packet is the RMAP protocol ID.*
- `RMAP_PACKET_TYPE RMAPPACKETLIBRARY_CC RMAP_GetPacketType (RMAP_PACKET *pPacket_↵`  
`Struct)`  
*Get the type of the packet (whether it is a read, write or read/modify/write command or reply) from a packet structure which has already been created.*
- `U8 *RMAPPACKETLIBRARY_CC RMAP_GetTargetAddress (RMAP_PACKET *pPacketStruct, unsigned long *pTargetAddressLength)`  
*Get the target address of the packet from a packet structure which has already been created.*
- `char RMAPPACKETLIBRARY_CC RMAP_GetVerifyBeforeWrite (RMAP_PACKET *pPacketStruct)`

*Get whether the packet represented by a previously created packet structure has verify before write enabled, to check any data before writing it to memory.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_GetPerformAcknowledgement](#) (RMAP\_PACKET \*pPacketStruct)

*Get whether the packet represented by a previously created packet structure has acknowledgement enabled, to acknowledge the command with a reply packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_GetIncrementAddress](#) (RMAP\_PACKET \*pPacketStruct)

*Get whether the packet represented by a previously created packet structure has incrementing of memory addresses enabled, to read from or write to sequential memory.*

- U8 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetKey](#) (RMAP\_PACKET \*pPacketStruct)

*Get the key field in the packet represented by a previously created packet structure.*

- U8 \*[RMAPPACKETLIBRARY\\_CC RMAP\\_GetReplyAddress](#) (RMAP\_PACKET \*pPacketStruct, unsigned long \*pReplyAddressLength)

*Get the reply address of the packet from a packet structure which has already been created.*

- U16 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetTransactionID](#) (RMAP\_PACKET \*pPacketStruct)

*Get the transaction identifier in the packet represented by a previously created packet structure.*

- U32 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetAddress](#) (RMAP\_PACKET \*pPacketStruct, U8 \*pExtendedAddress)

*Get the memory address and extended memory address of the packet from a packet structure which has already been created.*

- U8 \*[RMAPPACKETLIBRARY\\_CC RMAP\\_GetData](#) (RMAP\_PACKET \*pPacketStruct, U32 \*pDataLength)

*Get the data in the packet from a packet structure which has already been created.*

- U32 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetReadLength](#) (RMAP\_PACKET \*pPacketStruct)

*Gets the data length property in the read command packet from a packet structure which has already been created.*

- RMAP\_STATUS [RMAPPACKETLIBRARY\\_CC RMAP\\_GetStatus](#) (RMAP\_PACKET \*pPacketStruct)

*Get the value of the status field in the packet represented by a previously created packet structure.*

- U8 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetHeaderCRC](#) (RMAP\_PACKET \*pPacketStruct)

*Get the value of the header CRC field in the packet represented by a previously created packet structure.*

- U8 [RMAPPACKETLIBRARY\\_CC RMAP\\_GetDataCRC](#) (RMAP\_PACKET \*pPacketStruct)

*Get the value of the data CRC field in the packet represented by a previously created packet structure.*

- U8 \*[RMAPPACKETLIBRARY\\_CC RMAP\\_GetMask](#) (RMAP\_PACKET \*pPacketStruct, U8 \*pMaskLength)

*Get the mask in the packet from a packet structure which has already been created.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a write command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write command packet which can be sent to perform an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char verifyBeforeWrite, char acknowledge, char incrementAddress, U8 key, U16 transactionIdentifier, U32 writeAddress, U8 extendedWriteAddress, U32 registerValue, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP write command packet which can be sent to perform an RMAP write command on a 4-byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateWriteReplyPacketLength](#) (unsigned long initiatorAddressLength, char alignment)

*Calculate the number of bytes required for a write reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildWriteReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char verifyBeforeWrite, char incrementAddress, [RMAP\\_STAT↵](#)US status, U16 transactionIdentifier, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP write reply packet which can be sent to respond to an RMAP write command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, char alignment)

*Calculate the number of bytes required for a read command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadRegisterPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, char incrementAddress, U8 key, U16 transactionIdentifier, U32 readAddress, U8 extendedReadAddress, unsigned long \*pRawPacket↵Length, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read command packet which can be sent to perform an RMAP read command on a 4-byte register.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadReplyPacketLength](#) (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read reply packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- void [\\*RMAPPACKETLIBRARY\\_CC RMAP\\_BuildReadReplyPacket](#) (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, char incrementAddress, [RMAP\\_STATUS](#) status, U16 transactionIdentifier, U8 \*pData, U32 dataLength, unsigned long \*pRawPacketLength, [RMAP\\_PACKET](#) \*pPacketStruct, char alignment)

*Build an RMAP read reply packet which can be sent to respond to an RMAP read command.*

- unsigned long [RMAPPACKETLIBRARY\\_CC RMAP\\_CalculateReadModifyWriteCommandPacketLength](#) (unsigned long targetAddressLength, unsigned long replyAddressLength, U32 dataAndMaskLength, char alignment)

*Calculate the number of bytes required for a read/modify/write command packet, given the properties of the packet.*

- char [RMAPPACKETLIBRARY\\_CC RMAP\\_FillReadModifyWriteCommandPacket](#) (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16

transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*pMask, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void \*RMAPPACKETLIBRARY\_CC RMAP\_BuildReadModifyWriteCommandPacket (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U8 dataAndMaskLength, U8 \*pData, U8 \*mask, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command.*

- void \*RMAPPACKETLIBRARY\_CC RMAP\_BuildReadModifyWriteRegisterPacket (U8 \*pTargetAddress, unsigned long targetAddressLength, U8 \*pReplyAddress, unsigned long replyAddressLength, U8 key, U16 transactionIdentifier, U32 readModifyWriteAddress, U8 extendedReadModifyWriteAddress, U32 registerValue, U32 mask, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write command packet which can be sent to perform an RMAP read/modify/write command on a 4 byte register.*

- unsigned long RMAPPACKETLIBRARY\_CC RMAP\_CalculateReadModifyWriteReplyPacketLength (unsigned long initiatorAddressLength, U32 dataLength, char alignment)

*Calculate the number of bytes required for a read/modify/write reply packet, given the properties of the packet.*

- char RMAPPACKETLIBRARY\_CC RMAP\_FillReadModifyWriteReplyPacket (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP\_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment, U8 \*pRawPacket, unsigned long rawPacketLength)

*Fill a previously allocated buffer with an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void \*RMAPPACKETLIBRARY\_CC RMAP\_BuildReadModifyWriteReplyPacket (U8 \*pInitiatorAddress, unsigned long initiatorAddressLength, U8 targetAddress, RMAP\_STATUS status, U16 transactionIdentifier, unsigned long dataLength, U8 \*pData, unsigned long \*pRawPacketLength, RMAP\_PACKET \*pPacketStruct, char alignment)

*Build an RMAP read/modify/write reply packet which can be sent to respond to an RMAP read/modify/write command.*

- void RMAPPACKETLIBRARY\_CC RMAP\_FreeBuffer (void \*pBuffer)

*Free a previously allocated buffer containing a packet created using one of the RMAP\_Build\* or RMAP\_Fill\* functions.*

- U8 RMAPPACKETLIBRARY\_CC RMAP\_GetProtocolIdentifier (RMAP\_PACKET \*pPacketStruct)

*Get the protocol identifier of the packet from a packet structure which has already been created.*

- void RMAPPACKETLIBRARY\_CC RMAP\_SetProtocolIdentifier (RMAP\_PACKET \*pPacketStruct, U8 protocolIdentifier)

*Set the protocol identifier of a previously created packet, updating the packet's header CRC so the packet is still valid.*

### 9.25.1 Detailed Description

Declarations of the functions provided by the STAR-Dundee RMAP Packet Library.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the declarations of the functions provided by the STAR-Dundee RMAP Packet Library, along with constants and types used by the library. The RMAP Packet Library provides functions for building and interpreting RMAP packets.

#### IMPORTANT NOTE:

**Note**

If you are experiencing compilation errors indicating that U8 is already defined, for example, please add the following line to your code prior to including this file:

```
#define NO_STAR_TYPES
```

Alternatively you can compile your code with a flag of `-DNO_STAR_TYPES`.

(Copied from `star_dundee_types.h`)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.26 rmap\_target\_pxi\_if.h File Reference

Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXI Interface devices.

```
#include "rmap_target_types.h"
```

**Functions**

- `int STAR_API_CC RMAP_TARGET_PXI_IF_getStatus (STAR_DEVICE_ID deviceId, U32 target, TARGET_STATUS *pStatus)`  
*Gets the RMAP target status for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_setAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 offset)`  
*Sets the address offset for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_getAddressOffset (STAR_DEVICE_ID deviceId, U32 target, U32 *pOffset)`  
*Gets the address offset from a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE mode)`  
*Sets the authorisation control mode for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthControlMode (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_MODE *pMode)`  
*Gets the authorisation control mode from a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 lowest, U8 highest)`  
*Sets the authorised target logical address range for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthLogicalAddressRange (STAR_DEVICE_ID deviceId, U32 target, U8 *pLowest, U8 *pHighest)`  
*Gets the authorised target logical address range from a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 protocolId)`  
*Sets the authorised protocol ID for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthProtocolId (STAR_DEVICE_ID deviceId, U32 target, U8 *pProtocolId)`  
*Gets the authorised protocol ID from a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_setAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK commands)`  
*Sets the authorised RMAP commands for a specific target.*
- `int STAR_API_CC RMAP_TARGET_PXI_IF_getAuthCommands (STAR_DEVICE_ID deviceId, U32 target, TARGET_AUTH_CMD_MASK *pCommands)`  
*Gets the authorised RMAP commands from a specific target.*



- int `STAR_API_CC RMAP_TARGET_PXI_IF_setAuthKeyRange` (STAR\_DEVICE\_ID deviceId, U32 target, U8 lowest, U8 highest)  
*Sets the authorised key range for a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getAuthKeyRange` (STAR\_DEVICE\_ID deviceId, U32 target, U8 \*pLowest, U8 \*pHighest)  
*Gets the authorised key range from a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange` (STAR\_DEVICE\_ID deviceId, U32 target, U32 lowerBoundary, U32 upperBoundary)  
*Sets the authorised memory address range for a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getAuthMemoryAddressRange` (STAR\_DEVICE\_ID deviceId, U32 target, U32 \*pLowerBoundary, U32 \*pUpperBoundary)  
*Gets the authorised memory address range from a specific target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_setInterfaceMode` (STAR\_DEVICE\_ID deviceId, U32 port, IF↔\_MODE mode)  
*Sets the RMAP interface mode for a specific port.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getInterfaceMode` (STAR\_DEVICE\_ID deviceId, U32 port, IF↔\_MODE \*pMode)  
*Gets the RMAP interface mode for a specific port.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_isAuthRequired` (STAR\_DEVICE\_ID deviceId, U32 target)  
*Gets whether or not there is an RMAP command waiting to be authorised.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getWaitingCommand` (STAR\_DEVICE\_ID deviceId, U32 target, RmapCommandParameters \*pCommand)  
*Gets the parameters of the RMAP command waiting to be authorised.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_authoriseWaitingCommand` (STAR\_DEVICE\_ID deviceId, U32 target)  
*Authorises a waiting RMAP command.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_rejectWaitingCommand` (STAR\_DEVICE\_ID deviceId, U32 target, REJECTION\_REASON reason)  
*Rejects a waiting RMAP command.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_setEnabledNotifications` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE\_MASK notifications)  
*Set the notifications that are enabled for a target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_getEnabledNotifications` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE\_MASK \*pNotifications)  
*Get the notifications that are enabled for a target.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListener` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type, NotifListenerFunc listenerFunc)  
*Register a notification listener function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type, NotifListenerWithContextFunc listenerFunc, void \*pContext)  
*Register a notification listener with context function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListener` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type)  
*Unregister a notification listener function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_unregisterNotificationListenerWithContext` (STAR\_DEVICE\_ID deviceId, U32 target, NOTIF\_TYPE type)  
*Unregister a notification listener with context function.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_startReceivingNotifications` (STAR\_DEVICE\_ID deviceId)  
*Start receiving notifications for a device.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_stopReceivingNotifications` (STAR\_DEVICE\_ID deviceId)  
*Stop receiving notifications for a device.*
- int `STAR_API_CC RMAP_TARGET_PXI_IF_readMemory` (STAR\_DEVICE\_ID deviceId, U32 target, U32 address, U32 length, unsigned char \*pBuffer)

*Reads an area of RMAP target memory into a user-supplied buffer.*

- int `STAR_API_CC RMAP_TARGET_PXI_IF_writeMemory` (STAR\_DEVICE\_ID deviceId, U32 target, U32 address, U32 length, const unsigned char \*pBuffer)

*Writes an area of RMAP target memory from a user-supplied buffer.*

### 9.26.1 Detailed Description

Functions used to configure and control the RMAP target capabilities of the STAR-Dundee PXI Interface devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.27 rmap\_target\_types.h File Reference

Types used with the STAR-Dundee RMAP Target API.

```
#include <star-api.h>
```

### Data Structures

- struct `TARGET_ENGINE`  
*Hardware capabilities. [More...](#)*
- struct `RmapCommandParameters`  
*Parameters of an RMAP command. [More...](#)*

### Typedefs

- typedef U32 `TARGET_AUTH_CMD_MASK`  
*Mask describing which commands are authorised.*
- typedef U32 `NOTIF_TYPE_MASK`  
*Mask describing which notifications are enabled.*
- typedef void(`STAR_API_CC * NotifListenerFunc`) (U32 target, void \*pNotif)  
*Notification listener function pointer: target is the index of the target which generated the notification.*
- typedef void(`STAR_API_CC * NotifListenerWithContextFunc`) (STAR\_DEVICE\_ID deviceId, U32 target, void \*pNotif, void \*pContext)  
*Notification listener with context function pointer: deviceId is the ID of the device that issued the notification target is the index of the target which generated the notification.*

### Enumerations

- enum `TARGET_AUTH_MODE` { `TARGET_AUTH_MODE_MANUAL` = 0, `TARGET_AUTH_MODE_AUTO`, `TARGET_AUTH_MODE_MATIC` = 1 }



*Method of authorising incoming RMAP commands.*

- enum `TARGET_STATUS` {  
`TARGET_STATUS_SUCCESS` = 0, `TARGET_STATUS_GENERAL_ERROR` = 1, `TARGET_STATUS_HEADER_EOP_ERROR` = 2, `TARGET_STATUS_HEADER_EEP_ERROR` = 3,  
`TARGET_STATUS_PID_ERROR` = 4, `TARGET_STATUS_REPLY_ERROR` = 5, `TARGET_STATUS_HEADER_CRC_ERROR` = 6, `TARGET_STATUS_HEADER_EEP_AFTER_CRC` = 7,  
`TARGET_STATUS_PACKET_TYPE_ERROR` = 8, `TARGET_STATUS_COMMAND_TYPE_ERROR` = 9, `TARGET_STATUS_RMW_DATALEN_ERROR` = 10, `TARGET_STATUS_CARGO_TOO_LARGE` = 11,  
`TARGET_STATUS_KEY_ERROR` = 12, `TARGET_STATUS_LOGICAL_ADDR_ERROR` = 13, `TARGET_STATUS_AUTHORISED_ERROR` = 14, `TARGET_STATUS_VERIFY_BUFFER_OVERRUN` = 15,  
`TARGET_STATUS_DATA_CRC_ERROR` = 16, `TARGET_STATUS_BUS_ERROR` = 17, `TARGET_STATUS_DATA_EOP_ERROR` = 18, `TARGET_STATUS_DATA_EEP_ERROR` = 19,  
`TARGET_STATUS_DATA_EEP_AFTER_CRC` = 20 }

*Status of the RMAP target.*

- enum `TARGET_AUTH_CMD` {  
`TARGET_AUTH_CMD_READ` = (1 << 0), `TARGET_AUTH_CMD_READ_INCR` = (1 << 1), `TARGET_AUTH_CMD_READ_MODIFY_WRITE` = (1 << 2), `TARGET_AUTH_CMD_WRITE` = (1 << 3),  
`TARGET_AUTH_CMD_WRITE_INCR` = (1 << 4), `TARGET_AUTH_CMD_WRITE_REPLY` = (1 << 5), `TARGET_AUTH_CMD_WRITE_INCR_REPLY` = (1 << 6), `TARGET_AUTH_CMD_WRITE_VERIFY` = (1 << 7),  
`TARGET_AUTH_CMD_WRITE_VERIFY_INCR` = (1 << 8), `TARGET_AUTH_CMD_WRITE_VERIFY_REPLY` = (1 << 9), `TARGET_AUTH_CMD_WRITE_VERIFY_INCR_REPLY` = (1 << 10), `TARGET_AUTH_CMD_ALL` = (0x7FF),  
`TARGET_AUTH_CMD_NONE` = (0) }

*Command types which are authorised by the target.*

- enum `REJECTION_REASON` { `REJECTION_REASON_LOGICAL_ADDRESS`, `REJECTION_REASON_KEY`, `REJECTION_REASON_PROTOCOL_ID`, `REJECTION_REASON_OTHER` }

*Reason for rejecting a waiting RMAP command.*

- enum `IF_MODE` { `IF_MODE_RMAP_DISABLED` = 0, `IF_MODE_RMAP_ENABLED` = 1 }

*Control whether or not the RMAP target is enabled for a port.*

- enum `NOTIF_TYPE` { `NOTIF_TYPE_AUTH_REQUEST` = (1 << 0), `NOTIF_TYPE_CMD_COMPLETE` = (1 << 2), `NOTIF_TYPE_ALL` = (0x5), `NOTIF_TYPE_NONE` = (0) }

*Types of notifications.*

### 9.27.1 Detailed Description

Types used with the STAR-Dundee RMAP Target API.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.28 star-api.h File Reference

Declarations of all functions provided by STAR-API.

```
#include "star-dundee_annotations.h"
#include "general.h"
#include "stream_item.h"
#include "transfer.h"
```

### 9.28.1 Detailed Description

Declarations of all functions provided by STAR-API.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains includes to the declarations of all functions provided by STAR-API, the STAR-Dundee software API.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.29 star-dundee\_annotations.h File Reference

Macros for previously undefined annotations macros.

### 9.29.1 Detailed Description

Macros for previously undefined annotations macros.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of macros of annotations that are defined in some operating systems but not others. The idea is that if we provide blank macros for any undefined annotations then they will compile on operating systems that don't provide the annotation support.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.30 star-dundee\_types.h File Reference

Definitions of STAR-Dundee commonly used types.

```
#include <limits.h>
```

## Macros

- `#define U16_MAX (0xffff)`  
*The maximum value that can be represented using an unsigned 16-bit number.*
- `#define TRUE 1`  
*A value that can be used to represent the boolean value of true.*
- `#define FALSE 0`  
*A value that can be used to represent the boolean value of false.*

## Typedefs

- `typedef unsigned char U8`  
*A type that can be used to represent an unsigned 8-bit number.*
- `typedef unsigned short U16`  
*A type that can be used to represent an unsigned 16-bit number.*
- `typedef unsigned int U32`  
*A type that can be used to represent an unsigned 32-bit number.*
- `typedef U32 REGISTER`  
*A type that can be used to represent a 4-byte register.*

### 9.30.1 Detailed Description

Definitions of STAR-Dundee commonly used types.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the definitions of common types used by STAR-Dundee software drivers and APIs.

#### IMPORTANT NOTE:

##### Note

If you are experiencing compilation errors indicating that U8 is already defined, for example, please add the following line to your code prior to including this file:

```
#define NO_STAR_TYPES
```

Alternatively you can compile your code with a flag of `-DNO_STAR_TYPES`.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.31 stream\_item.h File Reference

Functions create and modify items that can be sent over a SpaceWire network.

```
#include "common.h"
#include "stream_item_types.h"
```

## Functions

- void `STAR_API_CC STAR_destroyStreamItem` (`_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM *item`)  
*Disposes of a given stream item.*
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createPacket` (`_In_opt_ STAR_SPACEWIRE_ADDRESS *address, _In_opt_count_ (dataLen) unsigned char *data, unsigned int dataLen, STAR_EOP_TYPE eopType`)  
*Creates a new SpaceWire packet stream item from user provided data.*
- `_Check_return_ STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_createAddress` (`_In_count_ (pathLen) unsigned char *path, U16 pathLen`)  
*Creates a SpaceWire address.*
- void `STAR_API_CC STAR_destroyAddress` (`_In_ _Post_ptr_invalid_ STAR_SPACEWIRE_ADDRESS *address`)  
*Disposes of a SpaceWire address created with `STAR_createAddress()`.*
- int `STAR_API_CC STAR_setPacketAddress` (`_Inout_ STAR_SPACEWIRE_PACKET *packet, _In_ STAR_SPACEWIRE_ADDRESS *address`)  
*Updates the address of a given packet.*
- `_Ret_opt_bytecap_x_ dataLength unsigned char *STAR_API_CC STAR_getPacketData` (`_In_ STAR_SPACEWIRE_PACKET *packet, _Out_ unsigned int *dataLength`)  
*Gets the packet's data as an array of bytes.*
- void `STAR_API_CC STAR_destroyPacketData` (`_In_ _Post_ptr_invalid_ unsigned char *pData`)  
*Frees packet data previously created by a call to `STAR_getPacketData()`.*
- `STAR_SPACEWIRE_ADDRESS *STAR_API_CC STAR_getPacketAddress` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets a copy of a packet's address structure.*
- unsigned int `STAR_API_CC STAR_getPacketLength` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets the length of a given packet.*
- `STAR_EOP_TYPE STAR_API_CC STAR_getPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet`)  
*Gets the end of packet marker type of a packet.*
- int `STAR_API_CC STAR_setPacketEOP` (`_In_ STAR_SPACEWIRE_PACKET *packet, STAR_EOP_TYPE eopType`)  
*Sets the end of packet marker type of a packet.*
- `_Check_return_ STAR_STREAM_ITEM *STAR_API_CC STAR_createDataChunk` (`_In_opt_count_ (dataLen) unsigned char *data, unsigned int dataLen, int isStart, STAR_EOP_TYPE eopType`)  
*Creates a data chunk stream item, used to refer to part of a packet.*
- unsigned char `*STAR_API_CC STAR_getChunkData` (`_In_ STAR_DATA_CHUNK *chunk, _Out_ unsigned int *len`)  
*Gets a pointer to a data chunk stream item's data.*
- unsigned int `STAR_API_CC STAR_getChunkDataLength` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets the length of a data chunk stream item's data.*
- `STAR_EOP_TYPE STAR_API_CC STAR_getChunkEop` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets the End Of Packet marker type of a data chunk stream item.*
- int `STAR_API_CC STAR_getChunkSop` (`_In_ STAR_DATA_CHUNK *chunk`)  
*Gets whether a data chunk stream item is at the start of a packet.*
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createTimeCode` (U8 value)  
*Creates a Time-code stream item.*
- U8 `STAR_API_CC STAR_getTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode`)  
*Gets the value of a given time-code stream item.*
- void `STAR_API_CC STAR_setTimeCodeValue` (`_In_ STAR_TIMECODE *pTimeCode, U8 value`)  
*Sets the value of a given time-code stream item.*
- U16 `STAR_API_CC STAR_getTimeCodeCount` (`_In_ STAR_TIMECODE *pTimeCode`)  
*Gets the count of a given time-code stream item.*

- `STAR_STREAM_ITEM *STAR_API_CC STAR_createErrorInData (STAR_ERROR_IN_DATA_TYPE errorType)`  
*Creates an error in data stream item.*
- `U8 STAR_API_CC STAR_getLinkStateEventCount (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets the count of a given link state event stream item, indicating the number of events that have occurred.*
- `U8 STAR_API_CC STAR_getLinkStateEventPort (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets the port related to a given link state change event stream item.*
- `int STAR_API_CC STAR_isLinkStateEventDisconnectError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a disconnect error.*
- `int STAR_API_CC STAR_isLinkStateEventEscapeError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes an escape error.*
- `int STAR_API_CC STAR_isLinkStateEventLinkRunning (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a link running state change.*
- `int STAR_API_CC STAR_isLinkStateEventParityError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a parity error.*
- `int STAR_API_CC STAR_isLinkStateEventReceiveCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a receive credit error.*
- `int STAR_API_CC STAR_isLinkStateEventTransmitCreditError (_In_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`  
*Gets whether a link state event stream item includes a transmit credit error.*
- `U32 STAR_API_CC STAR_getLinkSpeedEventSpeed (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`  
*Gets the link speed of a given link speed change event stream item.*
- `U8 STAR_API_CC STAR_getLinkSpeedEventPort (_In_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`  
*Gets the port related to a given link speed change event stream item.*
- `void STAR_API_CC STAR_getTimestampEventStartValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)`  
*Gets the start time value from a timestamp event stream item.*
- `void STAR_API_CC STAR_getTimestampEventEndValue (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, U32 syncPulseFrequency, _Out_ U32 *pSeconds, _Out_ U32 *pRemainder)`  
*Gets the end time value from a timestamp event stream item.*
- `STAR_TIMESTAMP_TYPE STAR_API_CC STAR_getTimestampEventType (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`  
*Gets the timestamp type information from a timestamp event stream item.*
- `STAR_TIMESTAMP_DIRECTION STAR_API_CC STAR_getTimestampEventDirection (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`  
*Gets the timestamp direction information from a timestamp event stream item.*
- `void STAR_API_CC STAR_getTimestampEventRawCounters (_In_ STAR_TIMESTAMP_EVENT *pTimestampEvent, _Out_opt_ U32 *pStartSyncPulseCount, _Out_opt_ U32 *pStartClockCycleCount, _Out_opt_ U32 *pStartTotalCycleCount, _Out_opt_ U32 *pEndSyncPulseCount, _Out_opt_ U32 *pEndClockCycleCount, _Out_opt_ U32 *pEndTotalCycleCount, _Out_opt_ U32 *pClockFrequency)`  
*Gets the timestamp counter values as returned by the hardware, from a timestamp event stream item.*
- `STAR_STREAM_ITEM *STAR_API_CC STAR_createBroadcastMessage (_In_ U8 channel, _In_ U8 type, _In_ U8 status, _In_ U32 dataWord1, _In_ U32 dataWord2)`  
*Creates a broadcast message stream item.*
- `U8 STAR_API_CC STAR_getBroadcastMessageChannel (_In_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`  
*Gets the broadcast message channel from a broadcast message stream item.*

- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageType](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message type from a broadcast message stream item.*
- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageStatus](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message status from a broadcast message stream item.*
- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageLateFlag](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message late flag from a broadcast message stream item.*
- [U8 STAR\\_API\\_CC STAR\\_getBroadcastMessageDelayedFlag](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message delayed flag from a broadcast message stream item.*
- [U32 STAR\\_API\\_CC STAR\\_getBroadcastMessageDataWord1](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message first data word from a broadcast message stream item.*
- [U32 STAR\\_API\\_CC STAR\\_getBroadcastMessageDataWord2](#) ([\\_In\\_ STAR\\_BROADCAST\\_MESSAGE](#) \*p↔ BroadcastMessage)  
*Gets the broadcast message second data word from a broadcast message stream item.*

### 9.31.1 Detailed Description

Functions create and modify items that can be sent over a SpaceWire network.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.32 stream\_item\_types.h File Reference

Structures used to represent stream items such as packets, time-codes and link control tokens.

```
#include "star-dundee_types.h"
```

### Data Structures

- struct [STAR\\_LINK\\_STATE\\_EVENT](#)  
*Events that can occur which affect the state of the SpaceWire link. [More...](#)*
- struct [STAR\\_SPACEWIRE\\_ADDRESS](#)  
*A structure used to describe a SpaceWire address. [More...](#)*
- struct [STAR\\_DATA\\_CHUNK](#)  
*A structure used to describe a chunk of contiguous SpaceWire data, within a single packet. [More...](#)*
- struct [STAR\\_SPACEWIRE\\_PACKET](#)  
*A structure used to describe a SpaceWire packet. [More...](#)*
- struct [STAR\\_TIMECODE](#)

- *A structure used to describe a SpaceWire time-code. [More...](#)*
- struct `STAR_LINK_SPEED_EVENT`
  - A structure used to describe a link speed change event. [More...](#)*
- struct `STAR_ERROR_IN_DATA_INJECT`
  - A structure used to describe an error in data event. [More...](#)*
- struct `STAR_TIMESTAMP_EVENT`
  - A structure used to describe a timestamp on receipt of data on the link. [More...](#)*
- struct `STAR_BROADCAST_MESSAGE`
  - A structure used to describe a SpaceFibre broadcast message. [More...](#)*
- struct `STAR_STREAM_ITEM`
  - A wrapper for Stream Item objects. [More...](#)*

## Enumerations

- enum `STAR_STREAM_ITEM_TYPE` { `STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET`, `STAR_STREAM_ITEM_TYPE_TIMECODE`, `STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT`, `STAR_STREAM_ITEM_TYPE_DATA_CHUNK`, `STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT`, `STAR_STREAM_ITEM_TYPE_ERROR_INJECT`, `STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT`, `STAR_STREAM_ITEM_TYPE_BROADCAST_MESSAGE` }
- The kinds of object that are represented as stream items.*
- enum `STAR_EOP_TYPE` { `STAR_EOP_TYPE_INVALID`, `STAR_EOP_TYPE_EOP`, `STAR_EOP_TYPE_EEP`, `STAR_EOP_TYPE_NONE` }
- The kinds of End of packet marker that may be present.*
- enum `STAR_ERROR_IN_DATA_TYPE` { `STAR_ERROR_IN_DATA_PARITY`, `STAR_ERROR_IN_DATA_DISCONNECT` }
- The types of error that may be injected in to data in a packet.*
- enum `STAR_TIMESTAMP_TYPE` { `STAR_TIMESTAMP_TYPE_DATA`, `STAR_TIMESTAMP_TYPE_LINK_STATE`, `STAR_TIMESTAMP_TYPE_LINK_SPEED`, `STAR_TIMESTAMP_TYPE_TIMECODE`, `STAR_TIMESTAMP_TYPE_ERROR` }
- An enum used to define different types of timestamps that can be received.*
- enum `STAR_TIMESTAMP_DIRECTION` { `STAR_TIMESTAMP_DIRECTION_TRANSMIT`, `STAR_TIMESTAMP_DIRECTION_RECEIVE` }
- An enum used to define the direction of a timestamp.*
- enum `STAR_BROADCAST_MESSAGE_STATUS_FLAG` { `STAR_BROADCAST_MESSAGE_STATUS_FLAG_LATE = 0x1`, `STAR_BROADCAST_MESSAGE_STATUS_FLAG_DELAYED = 0x2` }
- An enum used to define the status flags of a broadcast message.*

### 9.32.1 Detailed Description

Structures used to represent stream items such as packets, time-codes and link control tokens.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

The types in this file are used to represent the traffic and link control tokens that may be sent on a SpaceWire link.

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.33 transfer.h File Reference

Declarations of functions provided by STAR-API relating to transfer operations.

```
#include "types.h"
#include "transfer_types.h"
#include "stream_item_types.h"
#include "common.h"
```

### Functions

- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_receivePacket](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_Out\\_bytecap\\_](#)(\*pPacketLength) void \*pPacketData, [\\_Inout\\_ unsigned int](#) \*pPacketLength, [\\_Out\\_ STAR\\_EOP\\_TYPE](#) \*pEopType, [\\_In\\_ int](#) timeout)  
*Receive a single packet on a previously opened channel in to the buffer specified.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_transmitPacket](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_opt\\_bytecount\\_](#)(packetLength) void \*pPacketData, [\\_In\\_ unsigned int](#) packetLength, [\\_In\\_ STAR\\_EOP\\_TYPE](#) eopType, [\\_In\\_ int](#) timeout)  
*Transmit a single packet on a previously opened channel from the buffer specified.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createTxOperation](#) ([\\_In\\_count\\_](#)(streamItemCount) [STAR\\_STREAM\\_ITEM](#) \*\*ppltems, unsigned int streamItemCount)  
*Creates a transmit operation that can be submitted later.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createRxOperation](#) (int itemCount, [STAR\\_RECEIVE\\_MASK](#) mask)  
*Creates a receive operation that can then be submitted.*
- [\\_Check\\_return\\_ STAR\\_TRANSFER\\_OPERATION \\*STAR\\_API\\_CC STAR\\_createRxOperationEx](#) ([\\_In\\_count\\_](#)(numBuffers) [U8](#) \*\*ppBuffers, [\\_In\\_ unsigned int](#) numBuffers, [\\_In\\_ unsigned int](#) bufferSize, [\\_In\\_ STAR\\_RECEIVE\\_MASK](#) mask)  
*This function is an alternative to [STAR\\_createRxOperation\(\)](#) that creates receive operations to be submitted to a channel opened with the [STAR\\_openChannelToLocalDeviceEx\(\)](#) function.*
- [int STAR\\_API\\_CC STAR\\_submitTransferOperation](#) ([\\_In\\_ STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*transferOp)  
*Submits a single transfer operation.*
- [int STAR\\_API\\_CC STAR\\_submitTransferOperationList](#) ([STAR\\_CHANNEL\\_ID](#) channelId, [\\_In\\_count\\_](#)(count) [STAR\\_TRANSFER\\_OPERATION](#) \*\*transferOps, unsigned int count)  
*Submits a list of transfer operations.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_waitOnTransferOperationCompletion](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation, int timeout)  
*Blocks until a given transfer operation has completed, or the timeout period expires.*
- [STAR\\_TRANSFER\\_STATUS STAR\\_API\\_CC STAR\\_getTransferOperationStatus](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation)  
*Gets the status of a transfer operation.*
- [void STAR\\_API\\_CC STAR\\_cancelTransferOperationWaits](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pTransferOperation)  
*Cancels any waits in progress on a transfer operation.*
- [unsigned int STAR\\_API\\_CC STAR\\_getTransferItemCount](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pReceiveOperation)  
*Gets the current count of stream items received by the operation and available to be obtained using [STAR\\_getTransferItem\(\)](#).*
- [STAR\\_STREAM\\_ITEM \\*STAR\\_API\\_CC STAR\\_getTransferItem](#) ([\\_In\\_ STAR\\_TRANSFER\\_OPERATION](#) \*pReceiveOperation, unsigned int index)  
*Gets the stream item at a given index of a receive operation.*
- [STAR\\_STREAM\\_ITEM \\*STAR\\_API\\_CC STAR\\_getNextTransferItem](#) ([STAR\\_STREAM\\_ITEM](#) \*pStreamItem)



*Gets the next stream item in the list of received stream items.*

- `_Ret_opt_cap_count STAR_STREAM_ITEM **STAR_API_CC STAR_getTransferItemList (_In_ STAR_TRANSFER_OPERATION *pReceiveOperation, _Out_ unsigned int *count)`

*Gets the entire list of transfer items associated with a receive.*

- `void STAR_API_CC STAR_destroyTransferItemList (_In_ _Post_ptr_invalid_ STAR_STREAM_ITEM **transferItemList, unsigned int transferItemListCount, int destroyItems)`

*Destroys a given stream item array, returned by a call to `STAR_getTransferItemList()`, optionally destroying each of the elements in the array.*

- `int STAR_API_CC STAR_cancelTransferOperation (_In_ STAR_TRANSFER_OPERATION *pTransferOperation)`

*Cancels a given transfer operation.*

- `int STAR_API_CC STAR_disposeTransferOperation (_In_ _Post_ptr_invalid_ STAR_TRANSFER_OPERATION *pTransferOperation)`

*Cancels a transfer operation (if it is not already complete) and indicates that the memory associated with the operation should be freed when the operation is no longer in use.*

- `int STAR_API_CC STAR_registerTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc, _In_opt_ void *pContextInfo)`

*Registers a call-back function that will be called when an operation completes.*

- `int STAR_API_CC STAR_unregisterTransferCompletionListener (_In_ STAR_TRANSFER_OPERATION *pTransferOperation, STAR_TransferOperationListenerFunc listenerFunc)`

*Unregisters a call-back function that will be called when an operation completes.*

- `U8 *STAR_API_CC STAR_getSpaceWireAddressPath (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`

*Access function to get a SpaceWire address path.*

- `U16 STAR_API_CC STAR_getSpaceWireAddressPathLength (STAR_SPACEWIRE_ADDRESS *pSpaceWireAddress)`

*Access function to get a SpaceWire address path length.*

- `int STAR_API_CC STAR_rxOperationExGetNumItems (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`

*Gets the number of items received by an ex receive operation.*

- `STAR_STREAM_ITEM_TYPE STAR_API_CC STAR_rxOperationExGetItemType (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item)`

*Gets the stream item type for an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetDataChunk (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_DATA_CHUNK *pDataChunk)`

*Gets a data chunk from an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetBroadcastMessage (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_BROADCAST_MESSAGE *pBroadcastMessage)`

*Gets a broadcast message from an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetTimestampEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMESTAMP_EVENT *pTimestampEvent)`

*Gets a timestamp event from an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetLinkStateEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_STATE_EVENT *pLinkStateEvent)`

*Gets a link state event from an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetLinkSpeedEvent (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_LINK_SPEED_EVENT *pLinkSpeedEvent)`

*Gets a link speed event from an item received by an ex receive operation.*

- `int STAR_API_CC STAR_rxOperationExGetTimeCode (_In_ STAR_TRANSFER_OPERATION *pTransferOp, _In_ unsigned int item, _Out_ STAR_TIMECODE *pTimeCode)`

*Gets a time-code from an item received by an ex receive operation.*

- `STAR_TRANSFER_STATUS STAR_API_CC STAR_rxOperationExGetStatus (_In_ STAR_TRANSFER_OPERATION *pTransferOp)`

*Gets the status of an ex receive operation.*

### 9.33.1 Detailed Description

Declarations of functions provided by STAR-API relating to transfer operations.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.34 transfer\_types.h File Reference

Types used to describe transmit and receive operations.

```
#include "types.h"
```

#### Macros

- `#define STAR_RECEIVE_NULL (STAR_RECEIVE_NULLS)`

#### Typedefs

- typedef void `STAR_TRANSFER_OPERATION`  
*A structure used to identify a transfer operation.*
- typedef void(`STAR_API_CC * STAR_TransferOperationListenerFunc`) (`STAR_TRANSFER_OPERATION`↵  
`N *pOperation`, `STAR_TRANSFER_STATUS` status, void `*pContextInfo`)  
*The function type for transfer operation listener functions which are called when a transfer operation completes, is cancelled, or encounters an error.*

#### Enumerations

- enum `STAR_TRANSFER_STATUS` {  
`STAR_TRANSFER_STATUS_NOT_STARTED`, `STAR_TRANSFER_STATUS_STARTED`, `STAR_TRAN`↵  
`SFER_STATUS_COMPLETE`, `STAR_TRANSFER_STATUS_CANCELLED`,  
`STAR_TRANSFER_STATUS_ERROR` }  
*Possible states a transfer operation can be in.*
- enum `STAR_RECEIVE_MASK` {  
`STAR_RECEIVE_PACKETS` = 1U << 0, `STAR_RECEIVE_CHUNKS` = 1U << 1, `STAR_RECEIVE_TIM`↵  
`ECODES` = 1U << 2, `STAR_RECEIVE_FCTS` = 1U << 3,  
`STAR_RECEIVE_NULLS` = 1U << 4, `STAR_RECEIVE_LINK_STATE_EVENTS` = 1U << 5, `STAR_RE`↵  
`CEIVE_LINK_SPEED_EVENTS` = 1U << 6, `STAR_RECEIVE_TIMESTAMP_EVENTS` = 1U << 7,  
`STAR_RECEIVE_BROADCAST_MESSAGES` = 1U << 8 }  
*Flags used to determine what kind of stream items to receive on a receive operation.*

### 9.34.1 Detailed Description

Types used to describe transmit and receive operations.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.35 triggering\_brick\_mk3.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee [SpaceWire Brick Mk3](#) and [Brick Mk4](#) and [SpaceWire GbE Brick Mk1](#) devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setExtTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)  
*Sets the input events for a port which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getExtTriggerInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getPortInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)  
*Gets the input events from a port which will cause an internal trigger to be set.*
- [int STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getTimeCodeInputEvents](#) (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- `int STAR_API_CC TRIGGER_BRICK_MK3_getTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputActions (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerEdgeDetectMode (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerEdgeDetectModeEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerInvert (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerInvertEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`  
*Gets whether invert is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_enableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Enables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_disableExtTriggerOutput (STAR_DEVICE_ID deviceId, U32 extTrigger)`  
*Disables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_BRICK_MK3_getExtTriggerOutputEnabled (STAR_DEVICE_ID deviceId, U32 extTrigger, U32 *pEnabled)`

- Gets whether output is enabled for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_setExtTriggerExtend](#) (STAR\_DEVICE\_ID deviceId, U32 ext↔Trigger, U32 extend)
- Sets the extend duration for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getExtTriggerExtend](#) (STAR\_DEVICE\_ID deviceId, U32 ext↔Trigger, U32 \*pExtend)
- Gets the extend duration for a specific external trigger.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_enableCounterAutoReload](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables auto reload for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_disableCounterAutoReload](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables auto reload for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterAutoReloadEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether auto reload is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_enableCounterTriggerCount](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables trigger count for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_disableCounterTriggerCount](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables trigger count for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterTriggerCountEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether trigger count is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_enableCounterStartMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_disableCounterStartMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterStartModeEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start mode is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_enableCounterStartStopMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start stop mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_disableCounterStartStopMode](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start stop mode for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterStartStopModeEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start stop mode is enabled for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_enableCounterLoadZero](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables load zero for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_disableCounterLoadZero](#) (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables load zero for a specific counter.*
- int [STAR\\_API\\_CC TRIGGER\\_BRICK\\_MK3\\_getCounterLoadZeroEnabled](#) (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether load zero is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_BRICK_MK3_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)  
*Sets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)  
*Gets the reload value for a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_enablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables spilling for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_disablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables spilling for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getPortSpillEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether spilling is enabled for a specific port.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_BRICK_MK3_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

- Gets the AND/OR mask for a trigger type for a specific internal trigger.
- int `STAR_API_CC TRIGGER_BRICK_MK3_reset (STAR_DEVICE_ID deviceId)`  
Resets the triggering configuration to its default state.
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_BRICK_MK3_getTriggerMatrix ()`  
Retrieves the trigger matrix of the Brick Mk3 device.
- int `STAR_API_CC TRIGGER_BRICK_MK3_getDeviceClockRate ()`  
Returns the clock rate used by the triggering system of the Brick Mk3 device.

### 9.35.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee [SpaceWire Brick Mk3](#) and [Brick Mk4](#) and [SpaceWire GbE Brick Mk1](#) devices.

Those devices have the following triggering resources available: 2 external triggers (indexed from 0-1), 4 internal counters (indexed from 0-3), 2 SpaceWire ports (indexed from 1-2), 1 time-code interface (indexed as 0) and 8 internal triggers (indexed from 0-7).

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.36 triggering\_generic.h File Reference

The Generic Triggering API used to configure triggering functionality on all STAR-Dundee devices.

```
#include "star-api.h"
#include "triggering_types.h"
```

#### Macros

- `#define TRIGGER_CFG_SUCCESS (1)`  
Operation succeeded.
- `#define TRIGGER_CFG_FAILURE (0)`  
Operation failed.
- `#define TRIGGER_CFG_STATUS_GET_DEVICE_INFO_FAILURE (-1)`  
Failed to get "Device Information" for this device.
- `#define TRIGGER_CFG_STATUS_NOT_SUPPORTED_BY_DEVICE (-2)`  
Functionality not supported by this device.

#### Functions

- int `STAR_API_CC TRIGGER_CFG_setExtTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_EVENT_MASK events)`  
Sets the input events for an external trigger which will cause an internal trigger to be set.



- `int STAR_API_CC TRIGGER_CFG_getExtTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_EVENT_MASK *const pEvents)`  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setExtTriggerOutputActions (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, const EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerOutputActions (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 trigger, EXT_TRIGGER_ACTION_MASK *const pActions)`  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerEdgeDetectMode (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerEdgeDetectMode (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables edge detect mode for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerEdgeDetectModeEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether edge detect mode is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerInvert (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerInvert (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables invert for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerInvertEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether invert is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableExtTriggerOutput (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Enables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableExtTriggerOutput (const STAR_DEVICE_ID deviceId, const U32 extTrigger)`  
*Disables output for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerOutputEnabled (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pEnabled)`  
*Gets whether output is enabled for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_setExtTriggerExtend (const STAR_DEVICE_ID deviceId, const U32 extTrigger, const U32 extend)`  
*Sets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getExtTriggerExtend (const STAR_DEVICE_ID deviceId, const U32 extTrigger, U32 *const pExtend)`  
*Gets the extend duration for a specific external trigger.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfExtTriggers (const STAR_DEVICE_ID deviceId, U32 *const pNumExtTriggers)`  
*Gets the number of external trigger resources available on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setCounterInputEvents (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, const COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getCounterInputEvents (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, COUNTER_EVENT_MASK *const pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setCounterOutputActions (const STAR_DEVICE_ID deviceId, const U32 counter, const U32 trigger, const COUNTER_ACTION_MASK actions)`



*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterOutputActions](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, const [U32](#) trigger, [COUNTER\\_ACTION\\_MASK](#) \*const pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableCounterAutoReload](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Enables auto reload for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableCounterAutoReload](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Disables auto reload for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterAutoReloadEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, [U32](#) \*const pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableCounterTriggerCount](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Enables trigger count for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableCounterTriggerCount](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Disables trigger count for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterTriggerCountEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, [U32](#) \*const pEnabled)

*Gets whether trigger count is enabled for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableCounterStartMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Enables start mode for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableCounterStartMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Disables start mode for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterStartModeEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, [U32](#) \*const pEnabled)

*Gets whether start mode is enabled for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableCounterStartStopMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Enables start stop mode for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableCounterStartStopMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Disables start stop mode for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterStartStopModeEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, [U32](#) \*const pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableCounterLoadZero](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Enables load zero for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableCounterLoadZero](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter)

*Disables load zero for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getCounterLoadZeroEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, [U32](#) \*const pEnabled)

*Gets whether load zero is enabled for a specific counter.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_setCounterReloadValue](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) counter, const [U32](#) reloadValue)

*Sets the reload value for a specific counter.*

- `int STAR_API_CC TRIGGER_CFG_getCounterReloadValue (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pReloadValue)`  
*Gets the reload value for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_forceCounterReload (const STAR_DEVICE_ID deviceId, const U32 counter)`  
*Force the reload of a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getCounterValue (const STAR_DEVICE_ID deviceId, const U32 counter, U32 *const pValue)`  
*Gets the counter value for a specific counter.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfCounters (const STAR_DEVICE_ID deviceId, U32 *const pNumCounters)`  
*Gets the number of counter resources available on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_EVENT_MASK events)`
- `int STAR_API_CC TRIGGER_CFG_getPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_EVENT_MASK *const pEvents)`
- `int STAR_API_CC TRIGGER_CFG_setPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, const PORT_ACTION_MASK actions)`
- `int STAR_API_CC TRIGGER_CFG_getPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 port, const U32 trigger, PORT_ACTION_MASK *const pActions)`
- `int STAR_API_CC TRIGGER_CFG_enablePortTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 port)`
- `int STAR_API_CC TRIGGER_CFG_disablePortTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 port)`
- `int STAR_API_CC TRIGGER_CFG_getPortTransmitPacketModeEnabled (const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)`
- `int STAR_API_CC TRIGGER_CFG_enablePortSpill (const STAR_DEVICE_ID deviceId, const U32 port)`
- `int STAR_API_CC TRIGGER_CFG_disablePortSpill (const STAR_DEVICE_ID deviceId, const U32 port)`
- `int STAR_API_CC TRIGGER_CFG_getPortSpillEnabled (const STAR_DEVICE_ID deviceId, const U32 port, U32 *const pEnabled)`
- `int STAR_API_CC TRIGGER_CFG_setTimeCodeInputEvents (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getTimeCodeInputEvents (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_EVENT_MASK *const pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setTimeCodeOutputActions (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, const TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getTimeCodeOutputActions (const STAR_DEVICE_ID deviceId, const U32 timeCode, const U32 trigger, TIME_CODE_ACTION_MASK *const pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfTimeCodeInterfaces (const STAR_DEVICE_ID deviceId, U32 *const pNumInterfaces)`  
*Gets the number of time code resources available on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 causeTrigger, const U32 trigger, const TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerInputEvents (const STAR_DEVICE_ID deviceId, const U32 causeTrigger, const U32 trigger, TRIGGER_EVENT_MASK *const pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerInputMode (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_INPUT_MODE mode)`  
*Sets the input mode for a specific internal trigger.*

- `int STAR_API_CC TRIGGER_CFG_getTriggerInputMode (const STAR_DEVICE_ID deviceId, const U32 trigger, TRIGGER_INPUT_MODE *const pMode)`  
*Gets the input mode for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_enableTrigger (const STAR_DEVICE_ID deviceId, const U32 trigger)`  
*Enables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_disableTrigger (const STAR_DEVICE_ID deviceId, const U32 trigger)`  
*Disables a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerEnabled (const STAR_DEVICE_ID deviceId, const U32 trigger, U32 *const pEnabled)`  
*Gets whether a trigger is enabled.*
- `int STAR_API_CC TRIGGER_CFG_forceTrigger (const STAR_DEVICE_ID deviceId, const U32 trigger)`  
*Force the triggering of a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerInvertMask (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 mask)`  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerInvertMask (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask)`  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerAndMask (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 mask)`  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerAndMask (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, U32 *const pMask)`  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerSourceEventsInverted (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 inverted)`  
*Set the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerSourceEventsInverted (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pInverted)`  
*Get the invert state of the signal generated by the specified source's input events which causes the specified internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setTriggerSourceEventsAnded (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, const U32 anded)`  
*Set whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getTriggerSourceEventsAnded (const STAR_DEVICE_ID deviceId, const U32 trigger, const TRIGGER_TYPE type, const U32 source, U32 *const pAnded)`  
*Get whether AND/OR logic is applied to events from the specified source to generate the signal used to set the specified internal trigger.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfTriggers (const STAR_DEVICE_ID deviceId, U32 *const pNumTriggers)`  
*Gets the number of internal trigger resources available on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_reset (const STAR_DEVICE_ID deviceId)`  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_CFG_getTriggerMatrix (const STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC TRIGGER_CFG_getDeviceClockRate (const STAR_DEVICE_ID deviceId)`
- `int STAR_API_CC TRIGGER_CFG_getDeviceClockFreq (const STAR_DEVICE_ID deviceId, U32 *const pClockFreqHz)`  
*Gets the clock frequency (in hertz) from the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setSpWPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, const SPW_PORT_EVENT_MASK events)`  
*Sets the input events for a SpaceWire port which will cause an internal trigger to be set.*

- `int STAR_API_CC TRIGGER_CFG_getSpWPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, SPW_PORT_EVENT_MASK *const pEvents)`  
*Gets the input events from a SpaceWire port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpWPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, const SPW_PORT_ACTION_MASK actions)`  
*Sets the output actions for a SpaceWire port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spwPort, const U32 trigger, SPW_PORT_ACTION_MASK *const pActions)`  
*Gets the output actions from a SpaceWire port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_enableSpWPortTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 spwPort)`  
*Enables packet transmit mode for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpWPortTransmitPacketMode (const STAR_DEVICE_ID deviceId, const U32 spwPort)`  
*Disables packet transmit mode for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortTransmitPacketModeEnabled (const STAR_DEVICE_ID deviceId, const U32 spwPort, U32 *const pEnabled)`  
*Gets whether packet transmit mode is enabled for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_enableSpWPortSpill (const STAR_DEVICE_ID deviceId, const U32 spwPort)`  
*Enables spilling for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_disableSpWPortSpill (const STAR_DEVICE_ID deviceId, const U32 spwPort)`  
*Disables spilling for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_getSpWPortSpillEnabled (const STAR_DEVICE_ID deviceId, const U32 spwPort, U32 *const pEnabled)`  
*Gets whether spilling is enabled for a specific SpaceWire port.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfSpWPorts (const STAR_DEVICE_ID deviceId, U32 *const pNumSpWPorts)`  
*Gets the number of SpaceWire port resources available on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, const SPFI_PORT_EVENT_MASK events)`  
*Sets the input events for a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiPortInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, SPFI_PORT_EVENT_MASK *const pEvents)`  
*Gets the input events from a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, const SPFI_PORT_ACTION_MASK actions)`  
*Sets the output actions for a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiPortOutputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 trigger, SPFI_PORT_ACTION_MASK *const pActions)`  
*Gets the output actions from a SpaceFibre port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_CFG_getNumberOfSpFiPorts (const STAR_DEVICE_ID deviceId, U32 *const pNumSpFiPorts)`  
*Gets the number of SpaceFibre ports on the specified device.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiVCInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_EVENT_MASK events)`  
*Sets the input events for a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_getSpFiVCInputEvents (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, SPFI_VC_EVENT_MASK *const pEvents)`  
*Gets the input events from a virtual channel on a SpaceFibre port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_CFG_setSpFiVCOuputActions (const STAR_DEVICE_ID deviceId, const U32 spfiPort, const U32 virtualChannel, const U32 trigger, const SPFI_VC_ACTION_MASK actions)`

*Sets the output actions for a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getSpFiVCOOutputActions](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel, const [U32](#) trigger, [SPFI\\_VC\\_ACTION\\_MASK](#) \*const pActions)

*Gets the output actions from a virtual channel on a SpaceFibre port that are caused by an internal trigger being set.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableSpFiVCTransmitPacketMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Enable transmit packet mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableSpFiVCTransmitPacketMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Disable transmit packet mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getSpFiVCTransmitPacketModeEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel, [U32](#) \*const pEnabled)

*Gets whether packet transmit mode is enabled for a specific virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableSpFiVCTransmitDataMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Enable transmit data mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableSpFiVCTransmitDataMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Disable transmit data mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getSpFiVCTransmitDataModeEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel, [U32](#) \*const pEnabled)

*Gets whether transmit data mode is enabled for a specific virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_enableSpFiVCReceiveDataMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Enable receive data mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_disableSpFiVCReceiveDataMode](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel)

*Disable receive data mode on a virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getSpFiVCReceiveDataModeEnabled](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, const [U32](#) spfiPort, const [U32](#) virtualChannel, [U32](#) \*const pEnabled)

*Gets whether receive data mode is enabled for a specific virtual channel of a SpaceFibre port.*

- int [STAR\\_API\\_CC\\_TRIGGER\\_CFG\\_getNumberOfSpFiVCs](#) (const [STAR\\_DEVICE\\_ID](#) deviceId, [U32](#) \*const pNumSpFiVCs)

*Gets the number of virtual channels on each SpaceFibre port on the specified device.*

### 9.36.1 Detailed Description

The Generic Triggering API used to configure triggering functionality on all STAR-Dundee devices.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.37 triggering\_pcie.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- `int STAR_API_CC TRIGGER_PCIE_IF_setCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)`  
*Sets the input events for a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 cause↔Trigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`  
*Gets the input events from a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 cause↔Trigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_PCIE_IF_disableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`  
*Disables auto reload for a specific counter.*
- `int STAR_API_CC TRIGGER_PCIE_IF_getCounterAutoReloadEnabled (STAR_DEVICE_ID deviceId, U32 counter, U32 *pEnabled)`  
*Gets whether auto reload is enabled for a specific counter.*
- `int STAR_API_CC TRIGGER_PCIE_IF_enableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`  
*Enables trigger count for a specific counter.*
- `int STAR_API_CC TRIGGER_PCIE_IF_disableCounterTriggerCount (STAR_DEVICE_ID deviceId, U32 counter)`



- Disables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)
- Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)
- Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)
- Gets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)
- Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)
- Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE mode)
- Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, T↔RIGGER\_INPUT\_MODE \*pMode)
- Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
- Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

- Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PCIE_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)
  - Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)
  - Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)
  - Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)
  - Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)
  - Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_IF_reset` (STAR\_DEVICE\_ID deviceId)
  - Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_PCIE_getTriggerMatrix` ()
  - Retrieves the trigger matrix of the PCIe device.*
- int `STAR_API_CC TRIGGER_PCIE_IF_getDeviceClockRate` ()
  - Returns the clock rate used by the triggering system of the PCIe device.*

### 9.37.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe Interface devices.

The PCIe Interface devices have the following triggering resources available: 6 internal counters (indexed from 0-5), 3 SpaceWire ports (indexed from 1-3) and 6 internal triggers (indexed from 0-5).

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.38 triggering\_pcie\_mk2.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe MK2 device.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- int `STAR_API_CC TRIGGER_PCIE_MK2_reset` (STAR\_DEVICE\_ID deviceId)
  - Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_PCIE_MK2_getTriggerMatrix` ()



- Retrieves the trigger matrix of the PCIe MK2 device.*

  - int `STAR_API_CC TRIGGER_PCIE_MK2_getDeviceClockRate` ()

*Returns the clock rate used by the triggering system of the PCIe MK2 device.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK events)

*Sets the input events for a counter which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterInputEvents` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_EVENT\_MASK \*pEvents)

*Gets the input events from a counter which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)

*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK events)

*Sets the input events for a port which will cause an internal trigger to be set.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵ Trigger, U32 trigger, TRIGGER\_EVENT\_MASK events)

*Sets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getPortInputEvents` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_EVENT\_MASK \*pEvents)

*Gets the input events from a port which will cause an internal trigger to be set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 cause↵Trigger, U32 trigger, TRIGGER\_EVENT\_MASK \*pEvents)

*Gets the input events for an internal trigger which will cause another internal trigger to be set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK actions)

*Sets the output actions for a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK actions)

*Sets the output actions for a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK actions)

*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK events)

*Sets the input events for a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getTimeCodeInputEvents` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_EVENT\_MASK \*pEvents)

*Gets the input events from a time-code engine which will cause an internal trigger to be set.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)

*Sets the reload value for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)

*Gets the reload value for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)

*Sets the input mode for a specific internal trigger.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)

- Gets the input mode for a specific internal trigger.*

  - int `STAR_API_CC TRIGGER_PCIE_MK2_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)

*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)

*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)

*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)

*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)

*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)

*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables edge detect mode for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerEdgeDetectModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether edge detect mode is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_disableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables invert for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_getExtTriggerInvertEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether invert is enabled for a specific external trigger.*
- int `STAR_API_CC TRIGGER_PCIE_MK2_enableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables output for a specific external trigger.*

- int `STAR_API_CC_TRIGGER_PCIE_MK2_disableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 ext↔Trigger)  
*Disables output for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getExtTriggerOutputEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)  
*Gets whether output is enabled for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_setExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 ext↔Trigger, U32 extend)  
*Sets the extend duration for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 ext↔Trigger, U32 \*pExtend)  
*Gets the extend duration for a specific external trigger.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_setExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK events)  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getExtTriggerInputEvents` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_EVENT\_MASK \*pEvents)  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_setExtTriggerOutputActions` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK actions)  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getExtTriggerOutputActions` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 trigger, EXT\_TRIGGER\_ACTION\_MASK \*pActions)  
*Gets the output actions from an external trigger that are caused by an internal trigger being set.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_enablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables spilling for a specific port.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_disablePortSpill` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables spilling for a specific port.*
- int `STAR_API_CC_TRIGGER_PCIE_MK2_getPortSpillEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether spilling is enabled for a specific port.*

### 9.38.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PCIe MK2 device.

The PCIe MK2 device has the following triggering resources available: 4 internal counters (indexed from 0-3), 3 SpaceWire ports (indexed from 1-3) and 8 internal triggers (indexed from 0-7), 1 Time-Code engine (index 0).

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.39 triggering\_pxi\_if.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerInputEvents (STAR_DEVICE_ID deviceld, U32 ext↵Trigger, U32 trigger, EXT_TRIGGER_EVENT_MASK events)`  
*Sets the input events for an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterInputEvents (STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setPortInputEvents (STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK events)`  
*Sets the input events for a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeInputEvents (STAR_DEVICE_ID deviceld, U32 time↵Code, U32 trigger, TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTriggerInputEvents (STAR_DEVICE_ID deviceld, U32 cause↵Trigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInputEvents (STAR_DEVICE_ID deviceld, U32 ext↵Trigger, U32 trigger, EXT_TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events from an external trigger which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getCounterInputEvents (STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getPortInputEvents (STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`  
*Gets the input events from a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTimeCodeInputEvents (STAR_DEVICE_ID deviceld, U32 time↵Code, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getTriggerInputEvents (STAR_DEVICE_ID deviceld, U32 cause↵Trigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setExtTriggerOutputActions (STAR_DEVICE_ID deviceld, U32 ext↵Trigger, U32 trigger, EXT_TRIGGER_ACTION_MASK actions)`  
*Sets the output actions for an external trigger that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setCounterOutputActions (STAR_DEVICE_ID deviceld, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setPortOutputActions (STAR_DEVICE_ID deviceld, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_setTimeCodeOutputActions (STAR_DEVICE_ID deviceld, U32 time↵Code, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputActions (STAR_DEVICE_ID deviceld, U32 ext↵Trigger, U32 trigger, EXT_TRIGGER_ACTION_MASK *pActions)`

*Gets the output actions from an external trigger that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getCounterOutputActions` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 trigger, COUNTER\_ACTION\_MASK \*pActions)

*Gets the output actions from a counter that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getPortOutputActions` (STAR\_DEVICE\_ID deviceId, U32 port, U32 trigger, PORT\_ACTION\_MASK \*pActions)

*Gets the output actions from a port that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_IF_getTimeCodeOutputActions` (STAR\_DEVICE\_ID deviceId, U32 timeCode, U32 trigger, TIME\_CODE\_ACTION\_MASK \*pActions)

*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*

- int `STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables edge detect mode for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerEdgeDetectMode` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables edge detect mode for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerEdgeDetectModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether edge detect mode is enabled for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables invert for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerInvert` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables invert for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerInvertEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether invert is enabled for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_enableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Enables output for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_disableExtTriggerOutput` (STAR\_DEVICE\_ID deviceId, U32 extTrigger)

*Disables output for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerOutputEnabled` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pEnabled)

*Gets whether output is enabled for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_setExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 extend)

*Sets the extend duration for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_getExtTriggerExtend` (STAR\_DEVICE\_ID deviceId, U32 extTrigger, U32 \*pExtend)

*Gets the extend duration for a specific external trigger.*

- int `STAR_API_CC TRIGGER_PXI_IF_enableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_IF_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_IF_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_IF_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*



- int `STAR_API_CC TRIGGER_PXI_IF_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables trigger count for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether trigger count is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables start mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables start stop mode for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether start stop mode is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Enables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Disables load zero for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)  
*Gets whether load zero is enabled for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)  
*Sets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)  
*Gets the reload value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)  
*Force the reload of a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_getCounterValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_IF_enablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_IF_disablePortTransmitPacketMode` (STAR\_DEVICE\_ID deviceId, U32 port)  
*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_IF_getPortTransmitPacketModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 port, U32 \*pEnabled)  
*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_IF_setTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE mode)  
*Sets the input mode for a specific internal trigger.*

- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerInputMode` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_INPUT\_MODE \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_enableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_disableTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerEnabled` (STAR\_DEVICE\_ID deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC_TRIGGER_PXI_IF_forceTrigger` (STAR\_DEVICE\_ID deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerInvertMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_setTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getTriggerAndMask` (STAR\_DEVICE\_ID deviceId, U32 trigger, TRIGGER\_TYPE type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC_TRIGGER_PXI_IF_reset` (STAR\_DEVICE\_ID deviceId)  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC_TRIGGER_PXI_IF_getTriggerMatrix` ()  
*Retrieves the trigger matrix of the PXI Interface device.*
- int `STAR_API_CC_TRIGGER_PXI_IF_getDeviceClockRate` ()  
*Returns the clock rate used by the triggering system of the PXI Interface device.*

### 9.39.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Interface devices.

The PXI Interface devices have the following triggering resources available: 4 external triggers (indexed from 0-3), 4 internal counters (indexed from 0-3), 4 SpaceWire ports (indexed from 1-4), 1 time-code interface (indexed as 0) and 8 internal triggers (indexed from 0-7).

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.



## 9.40 triggering\_pxi\_ro.h File Reference

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices.

```
#include <star-api.h>
#include "triggering_types.h"
```

### Functions

- `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK events)`  
*Sets the input events for a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK events)`  
*Sets the input events for a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK events)`  
*Sets the input events for a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK events)`  
*Sets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterInputEvents (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_EVENT_MASK *pEvents)`  
*Gets the input events from a counter which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortInputEvents (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_EVENT_MASK *pEvents)`  
*Gets the input events from a port which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeInputEvents (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_EVENT_MASK *pEvents)`  
*Gets the input events from a time-code engine which will cause an internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputEvents (STAR_DEVICE_ID deviceId, U32 causeTrigger, U32 trigger, TRIGGER_EVENT_MASK *pEvents)`  
*Gets the input events for an internal trigger which will cause another internal trigger to be set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK actions)`  
*Sets the output actions for a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK actions)`  
*Sets the output actions for a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_setTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK actions)`  
*Sets the output actions for a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getCounterOutputActions (STAR_DEVICE_ID deviceId, U32 counter, U32 trigger, COUNTER_ACTION_MASK *pActions)`  
*Gets the output actions from a counter that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getPortOutputActions (STAR_DEVICE_ID deviceId, U32 port, U32 trigger, PORT_ACTION_MASK *pActions)`  
*Gets the output actions from a port that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_getTimeCodeOutputActions (STAR_DEVICE_ID deviceId, U32 timeCode, U32 trigger, TIME_CODE_ACTION_MASK *pActions)`  
*Gets the output actions from a time-code engine that are caused by an internal trigger being set.*
- `int STAR_API_CC TRIGGER_PXI_ROUTER_enableCounterAutoReload (STAR_DEVICE_ID deviceId, U32 counter)`

*Enables auto reload for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterAutoReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables auto reload for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterAutoReloadEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether auto reload is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables trigger count for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterTriggerCount` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables trigger count for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterTriggerCountEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether trigger count is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start mode for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterStartMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start mode for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterStartModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start mode is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables start stop mode for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterStartStopMode` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables start stop mode for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterStartStopModeEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether start stop mode is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_enableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Enables load zero for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_disableCounterLoadZero` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Disables load zero for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterLoadZeroEnabled` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pEnabled)

*Gets whether load zero is enabled for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_setCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 reloadValue)

*Sets the reload value for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_getCounterReloadValue` (STAR\_DEVICE\_ID deviceId, U32 counter, U32 \*pReloadValue)

*Gets the reload value for a specific counter.*

- int `STAR_API_CC_TRIGGER_PXI_ROUTER_forceCounterReload` (STAR\_DEVICE\_ID deviceId, U32 counter)

*Force the reload of a specific counter.*

- int `STAR_API_CC TRIGGER_PXI_ROUTER_getCounterValue` (`STAR_DEVICE_ID` deviceId, U32 counter, U32 \*pValue)  
*Gets the counter value for a specific counter.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode` (`STAR_DEVICE_ID` deviceId, U32 port)  
*Enables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_disablePortTransmitPacketMode` (`STAR_DEVICE_ID` deviceId, U32 port)  
*Disables packet transmit mode for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getPortTransmitPacketModeEnabled` (`STAR_DEVICE_ID` deviceId, U32 port, U32 \*pEnabled)  
*Gets whether packet transmit mode is enabled for a specific port.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInputMode` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_INPUT_MODE` mode)  
*Sets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInputMode` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_INPUT_MODE` \*pMode)  
*Gets the input mode for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_enableTrigger` (`STAR_DEVICE_ID` deviceId, U32 trigger)  
*Enables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_disableTrigger` (`STAR_DEVICE_ID` deviceId, U32 trigger)  
*Disables a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerEnabled` (`STAR_DEVICE_ID` deviceId, U32 trigger, U32 \*pEnabled)  
*Gets whether a trigger is enabled.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_forceTrigger` (`STAR_DEVICE_ID` deviceId, U32 trigger)  
*Force the triggering of a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerInvertMask` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_TYPE` type, U32 mask)  
*Sets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerInvertMask` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_TYPE` type, U32 \*pMask)  
*Gets the invert mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_setTriggerAndMask` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_TYPE` type, U32 mask)  
*Sets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerAndMask` (`STAR_DEVICE_ID` deviceId, U32 trigger, `TRIGGER_TYPE` type, U32 \*pMask)  
*Gets the AND/OR mask for a trigger type for a specific internal trigger.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_reset` (`STAR_DEVICE_ID` deviceId)  
*Resets the triggering configuration to its default state.*
- `TRIGGER_MATRIX STAR_API_CC TRIGGER_PXI_ROUTER_getTriggerMatrix` ()  
*Retrieves the trigger matrix of the PXI Router device.*
- int `STAR_API_CC TRIGGER_PXI_ROUTER_getDeviceClockRate` ()  
*Returns the clock rate used by the triggering system of the PXI Router device.*

### 9.40.1 Detailed Description

Functions used to configure and control the triggering capabilities of the STAR-Dundee PXI Router devices.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

## 9.41 triggering\_types.h File Reference

Types used with the STAR-Dundee Triggering API.

```
#include <star-api.h>
```

**Data Structures**

- struct [TRIGGER\\_MATRIX](#)

**Typedefs**

- typedef [U32 EXT\\_TRIGGER\\_EVENT\\_MASK](#)  
*External trigger event mask.*
- typedef [U32 EXT\\_TRIGGER\\_ACTION\\_MASK](#)  
*External trigger action mask.*
- typedef [U32 COUNTER\\_EVENT\\_MASK](#)  
*Counter event mask.*
- typedef [U32 COUNTER\\_ACTION\\_MASK](#)  
*Counter action mask.*
- typedef [U32 PORT\\_EVENT\\_MASK](#)
- typedef [U32 SPW\\_PORT\\_EVENT\\_MASK](#)  
*SpaceWire port event mask.*
- typedef [U32 PORT\\_ACTION\\_MASK](#)
- typedef [U32 SPW\\_PORT\\_ACTION\\_MASK](#)  
*SpaceWire port action mask.*
- typedef [U32 SPFI\\_PORT\\_EVENT\\_MASK](#)  
*SpaceFibre port event mask.*
- typedef [U32 SPFI\\_PORT\\_ACTION\\_MASK](#)  
*SpaceFibre port action mask.*
- typedef [U32 SPFI\\_VC\\_EVENT\\_MASK](#)  
*SpaceFibre virtual channel event mask.*
- typedef [U32 SPFI\\_VC\\_ACTION\\_MASK](#)  
*SpaceFibre virtual channel action mask.*
- typedef [U32 TIME\\_CODE\\_EVENT\\_MASK](#)  
*Time-code event mask.*
- typedef [U32 TIME\\_CODE\\_ACTION\\_MASK](#)  
*Time-code action mask.*
- typedef [U32 TRIGGER\\_EVENT\\_MASK](#)  
*Trigger event mask.*

## Enumerations

- enum EXT\_TRIGGER\_EVENT { EXT\_TRIGGER\_EVENT\_IN = 1 << 0, EXT\_TRIGGER\_EVENT\_NONE = 0, EXT\_TRIGGER\_EVENT\_ALL = 0x1 }  
*External trigger events.*
- enum EXT\_TRIGGER\_ACTION { EXT\_TRIGGER\_ACTION\_OUT = 1 << 0, EXT\_TRIGGER\_ACTION\_NONE = 0, EXT\_TRIGGER\_ACTION\_ALL = 0x1 }  
*External trigger actions.*
- enum COUNTER\_EVENT { COUNTER\_EVENT\_COUNT = 1 << 0, COUNTER\_EVENT\_ZERO\_SINGLE = 1 << 1, COUNTER\_EVENT\_ZERO = 1 << 2, COUNTER\_EVENT\_RELOAD = 1 << 3, COUNTER\_EVENT\_NONE = 0, COUNTER\_EVENT\_ALL = 0xf }  
*Counter events.*
- enum COUNTER\_ACTION { COUNTER\_ACTION\_COUNT = 1 << 0, COUNTER\_ACTION\_RELOAD = 1 << 1, COUNTER\_ACTION\_START = 1 << 2, COUNTER\_ACTION\_STOP = 1 << 3, COUNTER\_ACTION\_NONE = 0, COUNTER\_ACTION\_ALL = 0xf }  
*Counter actions.*
- enum PORT\_EVENT { PORT\_EVENT\_RX\_SOP = 1 << 0, PORT\_EVENT\_RX\_EOP = 1 << 1, PORT\_EVENT\_RX\_EEP = 1 << 2, PORT\_EVENT\_TX\_SOP = 1 << 3, PORT\_EVENT\_TX\_EOP = 1 << 4, PORT\_EVENT\_TX\_PKT\_PENDING = 1 << 5, PORT\_EVENT\_RUNNING = 1 << 6, PORT\_EVENT\_PARITY\_ERROR = 1 << 7, PORT\_EVENT\_ESCAPE\_ERROR = 1 << 8, PORT\_EVENT\_CREDIT\_ERROR = 1 << 9, PORT\_EVENT\_DISCONNECT = 1 << 10, PORT\_EVENT\_RX\_TIME\_CODE = 1 << 11, PORT\_EVENT\_TX\_TIME\_CODE = 1 << 12, PORT\_EVENT\_NONE = 0, PORT\_EVENT\_ALL = 0x1fff, PORT\_EVENT\_ALL\_PCIE = 0xffff }  
*Port events.*
- enum SPW\_PORT\_EVENT { SPW\_PORT\_EVENT\_RX\_SOP = PORT\_EVENT\_RX\_SOP, SPW\_PORT\_EVENT\_RX\_EOP = PORT\_EVENT\_RX\_EOP, SPW\_PORT\_EVENT\_RX\_EEP = PORT\_EVENT\_RX\_EEP, SPW\_PORT\_EVENT\_TX\_SOP = PORT\_EVENT\_TX\_SOP, SPW\_PORT\_EVENT\_TX\_EOP = PORT\_EVENT\_TX\_EOP, SPW\_PORT\_EVENT\_TX\_PKT\_PENDING = PORT\_EVENT\_TX\_PKT\_PENDING, SPW\_PORT\_EVENT\_RUNNING = PORT\_EVENT\_RUNNING, SPW\_PORT\_EVENT\_PARITY\_ERROR = PORT\_EVENT\_PARITY\_ERROR, SPW\_PORT\_EVENT\_ESCAPE\_ERROR = PORT\_EVENT\_ESCAPE\_ERROR, SPW\_PORT\_EVENT\_CREDIT\_ERROR = PORT\_EVENT\_CREDIT\_ERROR, SPW\_PORT\_EVENT\_DISCONNECT = PORT\_EVENT\_DISCONNECT, SPW\_PORT\_EVENT\_RX\_TIME\_CODE = PORT\_EVENT\_RX\_TIME\_CODE, SPW\_PORT\_EVENT\_TX\_TIME\_CODE = PORT\_EVENT\_TX\_TIME\_CODE, SPW\_PORT\_EVENT\_NONE = PORT\_EVENT\_NONE, SPW\_PORT\_EVENT\_ALL = PORT\_EVENT\_ALL, SPW\_PORT\_EVENT\_ALL\_PCIE = PORT\_EVENT\_ALL\_PCIE }  
*SpaceWire port events.*
- enum PORT\_ACTION { PORT\_ACTION\_TRANSMIT\_PKT = 1 << 0, PORT\_ACTION\_DISCONNECT = 1 << 1, PORT\_ACTION\_PARITY\_ERROR = 1 << 2, PORT\_ACTION\_ESCAPE\_ERROR = 1 << 3, PORT\_ACTION\_INSERT\_FCT = 1 << 4, PORT\_ACTION\_SUPPRESS\_FCT = 1 << 5, PORT\_ACTION\_INCR\_CREDIT = 1 << 6, PORT\_ACTION\_DECR\_CREDIT = 1 << 7, PORT\_ACTION\_STOP\_RECEPTION = 1 << 8, PORT\_ACTION\_DISABLE\_LVDS = 1 << 9, PORT\_ACTION\_NONE = 0, PORT\_ACTION\_ALL = 0x3ff, PORT\_ACTION\_ALL\_PCIE = 0x1ff }  
*Port actions.*
- enum SPW\_PORT\_ACTION { SPW\_PORT\_ACTION\_TRANSMIT\_PKT = PORT\_ACTION\_TRANSMIT\_PKT, SPW\_PORT\_ACTION\_DISCONNECT = PORT\_ACTION\_DISCONNECT, SPW\_PORT\_ACTION\_PARITY\_ERROR = PORT\_ACTION\_PARITY\_ERROR, SPW\_PORT\_ACTION\_ESCAPE\_ERROR = PORT\_ACTION\_ESCAPE\_ERROR, SPW\_PORT\_ACTION\_INSERT\_FCT = PORT\_ACTION\_INSERT\_FCT, SPW\_PORT\_ACTION\_SUPPRESS\_FCT = PORT\_ACTION\_SUPPRESS\_FCT, SPW\_PORT\_ACTION\_INCR\_CREDIT = PORT\_ACTION\_INCR\_CREDIT, SPW\_PORT\_ACTION\_DECR\_CREDIT = PORT\_ACTION\_DECR\_CREDIT, SPW\_PORT\_ACTION\_STOP\_RECEPTION = PORT\_ACTION\_STOP\_RECEPTION, SPW\_PORT\_ACTION\_DISABLE\_LVDS = PORT\_ACTION\_DISABLE\_LVDS, SPW\_PORT\_ACTION\_NONE = PORT\_ACTION\_NONE, SPW\_PORT\_ACTION\_ALL = PORT\_ACTION\_ALL, SPW\_PORT\_ACTION\_ALL\_PCIE = PORT\_ACTION\_ALL\_PCIE }  
*SpaceWire port actions.*

```
ON_DISABLE_LVDS = PORT_ACTION_DISABLE_LVDS, SPW_PORT_ACTION_NONE = PORT_ACTION_NONE,
SPW_PORT_ACTION_ALL = PORT_ACTION_ALL,
SPW_PORT_ACTION_ALL_PCIE = PORT_ACTION_ALL_PCIE }
```

*SpaceWire port actions.*

- enum SPFI\_PORT\_EVENT {  
 SPFI\_PORT\_EVENT\_CONNECTED = 1 << 0, SPFI\_PORT\_EVENT\_DISCONNECTED = 1 << 1, SPFI\_PORT\_EVENT\_TX\_BM = 1 << 2, SPFI\_PORT\_EVENT\_RX\_BM = 1 << 3,  
 SPFI\_PORT\_EVENT\_NONE = 0, SPFI\_PORT\_EVENT\_ALL = 0xf }

*SpaceFibre port events.*

- enum SPFI\_PORT\_ACTION { SPFI\_PORT\_ACTION\_DISCONNECT = 1 << 0, SPFI\_PORT\_ACTION\_NONE = 0, SPFI\_PORT\_ACTION\_ALL = 0xf }

*SpaceFibre port actions.*

- enum SPFI\_VC\_EVENT {  
 SPFI\_VC\_EVENT\_RX\_SOP = 1 << 0, SPFI\_VC\_EVENT\_RX\_EOP = 1 << 1, SPFI\_VC\_EVENT\_RX\_EEP = 1 << 2, SPFI\_VC\_EVENT\_TX\_SOP = 1 << 3,  
 SPFI\_VC\_EVENT\_TX\_EOP = 1 << 4, SPFI\_VC\_EVENT\_TX\_PKT\_PENDING = 1 << 5, SPFI\_VC\_EVENT\_NONE = 0, SPFI\_VC\_EVENT\_ALL = 0x3f }

*SpaceFibre virtual channel events.*

- enum SPFI\_VC\_ACTION {  
 SPFI\_VC\_ACTION\_TRANSMIT\_PKT = 1 << 0, SPFI\_VC\_ACTION\_TRANSMIT\_DATA = 1 << 1, SPFI\_VC\_ACTION\_RECEIVE\_DATA = 1 << 2, SPFI\_VC\_ACTION\_NONE = 0,  
 SPFI\_VC\_ACTION\_ALL = 0x7 }

*SpaceFibre virtual channel actions.*

- enum TIME\_CODE\_EVENT { TIME\_CODE\_EVENT\_TICK = 1 << 0, TIME\_CODE\_EVENT\_NONE = 0, TIME\_CODE\_EVENT\_ALL = 0x1 }

*Time-code events.*

- enum TIME\_CODE\_ACTION { TIME\_CODE\_ACTION\_TX = 1 << 0, TIME\_CODE\_ACTION\_NONE = 0, TIME\_CODE\_ACTION\_ALL = 0x1 }

*Time-code actions.*

- enum TRIGGER\_EVENT { TRIGGER\_EVENT\_IN = 1 << 0, TRIGGER\_EVENT\_NONE = 0, TRIGGER\_EVENT\_ALL = 0x1 }

*Internal trigger events.*

- enum TRIGGER\_INPUT\_MODE { TRIGGER\_INPUT\_MODE\_OR = 0, TRIGGER\_INPUT\_MODE\_AND = 1 }

*Internal trigger input modes.*

- enum TRIGGER\_TYPE {  
 TRIGGER\_TYPE\_EXT\_TRIGGER = 0, TRIGGER\_TYPE\_COUNTER = 1, TRIGGER\_TYPE\_PORT = 2, TRIGGER\_TYPE\_SPW\_PORT = TRIGGER\_TYPE\_PORT,  
 TRIGGER\_TYPE\_TIME\_CODE = 3, TRIGGER\_TYPE\_TRIGGER = 4, TRIGGER\_TYPE\_SPFI\_PORT = 5,  
 TRIGGER\_TYPE\_SPFI\_VC = 6,  
 TRIGGER\_TYPE\_COUNT = 7 }

*Trigger types.*

- enum TRIGGER\_DEVICE {  
 TRIGGER\_DEVICE\_INVALID = 0, TRIGGER\_DEVICE\_BRICK\_MK3 = 1, TRIGGER\_DEVICE\_PXI\_IF = 2,  
 TRIGGER\_DEVICE\_PXI\_ROUTER = 3,  
 TRIGGER\_DEVICE\_PCIE\_IF = 4, TRIGGER\_DEVICE\_PCIE\_MK2 = 5, TRIGGER\_DEVICE\_STAR\_ULTRA\_PCIE\_IF = 6, TRIGGER\_DEVICE\_COUNT = 7 }

*Trigger devices.*

### 9.41.1 Detailed Description

Types used with the STAR-Dundee Triggering API.

**Author**

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

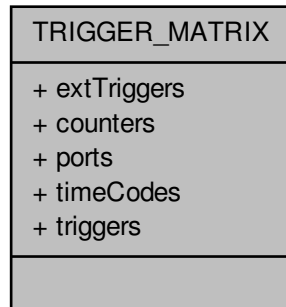
**9.41.2 Data Structure Documentation****9.41.2.1 struct TRIGGER\_MATRIX**

**Deprecated** Structure no longer needed due to new resource count retrieval functions e.g., TRIGGER\_CFG\_getNumberOfExtTriggers. Hardware capabilities.

**Added in version:**

STAR-System v3.0 - 26th August 2016

Collaboration diagram for TRIGGER\_MATRIX:

**Data Fields**

|     |             |  |
|-----|-------------|--|
| U32 | counters    |  |
| U32 | extTriggers |  |
| U32 | ports       |  |
| U32 | timeCodes   |  |
| U32 | triggers    |  |

**9.41.3 Typedef Documentation****9.41.3.1 typedef U32 PORT\_ACTION\_MASK**

**Deprecated** Replaced by SPW\_PORT\_ACTION\_MASK.

Port action mask.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

#### 9.41.3.2 typedef U32 PORT\_EVENT\_MASK

**Deprecated** Replaced by SPW\_PORT\_EVENT\_MASK.

Port event mask.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

### 9.41.4 Enumeration Type Documentation

#### 9.41.4.1 enum PORT\_ACTION

**Deprecated** Replaced by SPW\_PORT\_ACTION\_MASK.

Port actions.

Added in version:

[STAR-System v3.0 - 26th August 2016](#)

Declaration changed in version:

[STAR-System v4.00 Beta 4 - 16th October 2019](#)

Enumerator

**PORT\_ACTION\_TRANSMIT\_PKT** Transmit a pending packet.

**PORT\_ACTION\_DISCONNECT** Disconnect the port.

**PORT\_ACTION\_PARITY\_ERROR** Inject a parity error.

**PORT\_ACTION\_ESCAPE\_ERROR** Inject an escape error.

**PORT\_ACTION\_INSERT\_FCT** Insert an FCT.

**PORT\_ACTION\_SUPPRESS\_FCT** Suppress the next FCT to be transmitted.

**PORT\_ACTION\_INCR\_CREDIT** Increment credit.

**PORT\_ACTION\_DECR\_CREDIT** Decrement credit.

**PORT\_ACTION\_STOP\_RECEPTION** Stop reception.

Added in version:

[STAR-System v3.7 - 13th October 2017](#)

Supported devices:

[SpaceWire Brick Mk3 and Brick Mk4 \(version 1.03 or greater\).](#)

**PORT\_ACTION\_DISABLE\_LVDS** Disable LVDS drivers.

Added in version:

[STAR-System v4.00 Beta 4 - 16th October 2019](#)

Supported devices:

[SpaceWire Brick Mk3 and Brick Mk4 \(version 1.05 or greater\), SpaceWire PXI Interface and PXI Interface Mk2 \(v1.05 or later\), SpaceWire PXI 12 Port Router and PXI 12 Port Router Mk2 \(v1.05 or later\).](#)

**PORT\_ACTION\_NONE** No actions.

**PORT\_ACTION\_ALL** All actions.



#### 9.41.4.2 enum PORT\_EVENT

**Deprecated** Replaced by SPW\_PORT\_EVENT.

Port events.

Added in version:

STAR-System v3.0 - 26th August 2016

Enumerator

**PORT\_EVENT\_RX\_SOP** Event occurs when the port receives an SOP.  
**PORT\_EVENT\_RX\_EOP** Event occurs when the port receives an EOP.  
**PORT\_EVENT\_RX\_EEP** Event occurs when the port receives an EEP.  
**PORT\_EVENT\_TX\_SOP** Event occurs when the port transmits an SOP.  
**PORT\_EVENT\_TX\_EOP** Event occurs when the port transmits an EOP.  
**PORT\_EVENT\_TX\_PKT\_PENDING** Event is active when the port has a packet queued for transmission.  
**PORT\_EVENT\_RUNNING** Event is active when the link attached to the port is running.  
**PORT\_EVENT\_PARITY\_ERROR** Event occurs when the port detects a parity error.  
**PORT\_EVENT\_ESCAPE\_ERROR** Event occurs when the port detects an escape error.  
**PORT\_EVENT\_CREDIT\_ERROR** Event occurs when the port detects a credit error.  
**PORT\_EVENT\_DISCONNECT** Event occurs when the port disconnects.  
**PORT\_EVENT\_RX\_TIME\_CODE** Event occurs when the port receives a time-code.  
**PORT\_EVENT\_TX\_TIME\_CODE** Event occurs when the port transmits a time-code.  
**PORT\_EVENT\_NONE** No events.  
**PORT\_EVENT\_ALL** All events.

#### 9.41.4.3 enum TRIGGER\_DEVICE

Trigger devices.

Added in version:

STAR-System v3.0 - 26th August 2016

Declaration changed in version:

STAR-System v6.01 - 13th November 2025

#### 9.41.4.4 enum TRIGGER\_TYPE

Trigger types.

Added in version:

STAR-System v3.0 - 26th August 2016

Declaration changed in version:

STAR-System v6.01 - 13th November 2025

## 9.42 types.h File Reference

Types, structures and function types used by STAR-API.

```
#include "common.h"
#include "star-dundee_types.h"
```

### Macros

- `#define CFG_INFO_ID_TYPE U32`  
*Type used for driver, device, channel, application and listener identifiers.*
- `#define STAR_DEVICE_TYPE U32`  
*Type of device that a device is.*
- `#define STAR_DEVICE_SERIAL_SIZE 16`  
*Size in bytes (including null terminator) of a device serial number.*
- `#define STAR_CHANNEL_MASK U32`  
*Type used to represent channels that exist on a device.*
- `#define STAR_DEVICE_UNKNOWN 0U`  
*The "unknown" device type.*
- `#define STAR_DEVICE_PCI 1U`  
*The STAR-Dundee SpaceWire PCI-2 and cPCI device type.*
- `#define STAR_DEVICE_ROUTER_USB 2U`  
*The STAR-Dundee SpaceWire Router-USB device type.*
- `#define STAR_DEVICE_USB_BRICK 3U`  
*The STAR-Dundee SpaceWire-USB Brick device type.*
- `#define STAR_DEVICE_LINK_ANALYSER 4U`  
*The STAR-Dundee SpaceWire Link Analyser device type.*
- `#define STAR_DEVICE_CONFORMANCE_TESTER 5U`  
*The STAR-Dundee SpaceWire Conformance Tester device type.*
- `#define STAR_DEVICE_IP_TUNNEL 6U`  
*The STAR-Dundee SpaceWire IP-Tunnel device type.*
- `#define STAR_DEVICE_ROUTER_MK2S 7U`  
*The STAR-Dundee SpaceWire Router Mk2S device type.*
- `#define STAR_DEVICE_PCI_MK2 8U`  
*The STAR-Dundee SpaceWire PCI Mk2 device type.*
- `#define STAR_DEVICE_BRICK_MK2 9U`  
*The STAR-Dundee SpaceWire Brick Mk2 device type.*
- `#define STAR_DEVICE_LINK_ANALYSER_MK2 10U`  
*The STAR-Dundee SpaceWire Link Analyser Mk2 device type.*
- `#define STAR_DEVICE_PCIE 11U`  
*The STAR-Dundee SpaceWire PCI Express device type.*
- `#define STAR_DEVICE_RTC 12U`  
*The STAR-Dundee SpaceWire RTC device type.*
- `#define STAR_DEVICE_EGSE 13U`  
*The STAR-Dundee SpaceWire EGSE device type.*
- `#define STAR_DEVICE_CPCI_MK2 14U`  
*The STAR-Dundee SpaceWire cPCI Mk2 device type.*
- `#define STAR_DEVICE_SPLT 15U`  
*The STAR-Dundee SpaceWire Physical Layer Tester device type.*
- `#define STAR_DEVICE_STAR_FIRE 16U`  
*The STAR-Dundee STAR Fire device type.*

- `#define STAR_DEVICE_WBS_II 17U`  
The STAR-Dundee Wide Band Spectrometer II device type.
- `#define STAR_DEVICE_RECORDER 18U`  
The STAR-Dundee SpaceWire Recorder device type.
- `#define STAR_DEVICE_BRICK_MK3 19U`  
The STAR-Dundee SpaceWire Brick Mk3 device type.
- `#define STAR_DEVICE_TRAFFIC_GENERATOR 20U`  
The Shimafuji Traffic Generator device type.
- `#define STAR_DEVICE_PXI_INTERFACE 21U`  
The STAR-Dundee SpaceWire PXI Interface device type.
- `#define STAR_DEVICE_PXI_RMAP 22U`  
The STAR-Dundee SpaceWire PXI RMAP device type.
- `#define STAR_DEVICE_PXI_ROUTER_8 23U`  
The STAR-Dundee SpaceWire PXI 8 port Router device type.
- `#define STAR_DEVICE_PXI_ROUTER_12 24U`  
The STAR-Dundee SpaceWire PXI 12 port Router device type.
- `#define STAR_DEVICE_SPFPCIE 25U`  
The STAR-Dundee SpaceFibre PCI Express device type.
- `#define STAR_DEVICE_STAR_FIRE_MK3 26U`  
The STAR-Dundee STAR Fire Mk3 device type.
- `#define STAR_DEVICE_PCI_MK3 27U`  
The STAR-Dundee SpaceWire PCI Mk3 device type.
- `#define STAR_DEVICE_LINK_ANALYSER_MK3 28U`  
The STAR-Dundee SpaceWire Link Analyser Mk3 device type.
- `#define STAR_DEVICE_CONFORMANCE_TESTER_MK2 29U`  
The STAR-Dundee SpaceWire Conformance Tester Mk2 device type.
- `#define STAR_DEVICE_EGSE_MK2 30U`  
The STAR-Dundee SpaceWire EGSE Mk2 device type.
- `#define STAR_DEVICE_GBE_BRICK 31U`  
The STAR-Dundee SpaceWire GbE Brick device type.
- `#define STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE 32U`  
The STAR-Dundee STAR-Ultra PCIe Interface device type.
- `#define STAR_DEVICE_STAR_ULTRA_PCIE (STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE)`
- `#define STAR_DEVICE_PXI_INTERFACE_MK2 33U`  
The STAR-Dundee SpaceWire PXI Interface Mk2 device type.
- `#define STAR_DEVICE_PXI_RMAP_MK2 34U`  
The STAR-Dundee SpaceWire PXI RMAP Mk2 device type.
- `#define STAR_DEVICE_PXI_ROUTER_MK2 35U`  
The STAR-Dundee SpaceWire PXI Router Mk2 device type.
- `#define STAR_DEVICE_BRICK_MK4 36U`  
The STAR-Dundee SpaceWire Brick Mk4 device type.
- `#define STAR_DEVICE_RECORDER_MK2 37U`  
The STAR-Dundee SpaceWire Recorder Mk2 device type.
- `#define STAR_DEVICE_PCIE_MK2 38U`  
The STAR-Dundee SpaceWire PCIe Mk2 device type.
- `#define STAR_DEVICE_STAR_ULTRA_PCIE_SL_ROUTER 39U`  
The STAR-Dundee STAR-Ultra PCIe Single-Lane Router device type.
- `#define STAR_DEVICE_CONFIG_SUPPORTED 0x00ffffbU`  
A device which is capable of being configured.
- `#define STAR_DEVICE_CONFIG_NOT_SUPPORTED 0x00ffffcU`  
A device which is not capable of being configured.

- `#define STAR_DEVICE_TXRX_SUPPORTED 0x00ffffdU`  
*A device which is capable of transmitting and receiving packets.*
- `#define STAR_DEVICE_TXRX_NOT_SUPPORTED 0x00ffffeU`  
*A device which is not capable of transmitting and receiving packets.*
- `#define STAR_DEVICE_ALL 0x00fffffU`  
*All devices.*

## Typedefs

- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_ID`  
*Unique value used to identify an individual driver.*
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_ID`  
*Unique value used to identify an individual device.*
- `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_ID`  
*Unique value used to identify an open channel.*
- `typedef CFG_INFO_ID_TYPE STAR_APP_ID`  
*Unique value used to identify an application using STAR-System.*
- `typedef CFG_INFO_ID_TYPE STAR_DEVICE_LISTENER_ID`  
*Unique value used to identify an individual device listener.*
- `typedef CFG_INFO_ID_TYPE STAR_DRIVER_LISTENER_ID`  
*Unique value used to identify an individual driver listener.*
- `typedef CFG_INFO_ID_TYPE STAR_CHANNEL_LISTENER_ID`  
*Unique value used to identify an individual channel listener.*
- `typedef void(STAR_API_CC * STAR_DriverEventFunc) (STAR_DRIVER_LISTENER_ID driverListenerIdentifier, STAR_DRIVER_ID driverIdentifier, int driverAdded, void *pContextInfo)`  
*The function type for driver listener functions which are called when a new driver is added, or an existing one removed.*
- `typedef void(STAR_API_CC * STAR_DeviceEventFunc) (STAR_DEVICE_LISTENER_ID deviceListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, int deviceAdded, void *pContextInfo)`  
*The function type for device listener functions which are called when a device is added or removed.*
- `typedef void(STAR_API_CC * STAR_ChannelEventFunc) (STAR_CHANNEL_LISTENER_ID channelListenerIdentifier, STAR_DRIVER_ID driverIdentifier, STAR_DEVICE_ID deviceIdIdentifier, STAR_CHANNEL_ID channelIdIdentifier, int channelOpened, U8 channelNumber, void *pContextInfo)`  
*The function type for channel listener functions which are called when a device channel is opened or closed.*

## Enumerations

- `enum STAR_BUS_TYPE {`  
`STAR_BUS_UNKNOWN = 0, STAR_BUS_PCI = 1, STAR_BUS_USB = 2, STAR_BUS_PCIE = 3,`  
`STAR_BUS_TCP = 4, STAR_BUS_CPCI = 5, STAR_BUS_VIRTUAL = 255 }`  
*The different types of bus that can be used to connect a SpaceWire device to a PC.*
- `enum STAR_DRIVER_TYPE {`  
`STAR_DRIVER_INVALID = 0, STAR_DRIVER_TYPE_USB = 1, STAR_DRIVER_TYPE_PCI = 2, STAR_DRIVER_TYPE_TCPIP = 4,`  
`STAR_DRIVER_TYPE_VIRTUAL = 5, STAR_DRIVER_TYPE_ULTRA_PCIE = 6, STAR_DRIVER_TYPE_ULTRA_PCIE = 7 }`  
*Available driver types.*
- `enum STAR_CHANNEL_TYPE { STAR_CHANNEL_TYPE_NOT_OPEN, STAR_CHANNEL_TYPE_DEVICE, STAR_CHANNEL_TYPE_APPLICATION }`  
*Channel types.*
- `enum STAR_CHANNEL_DIRECTION { STAR_CHANNEL_DIRECTION_IN = 1, STAR_CHANNEL_DIRECTION_OUT = 2, STAR_CHANNEL_DIRECTION_INOUT }`  
*Channel directions.*

### 9.42.1 Detailed Description

Types, structures and function types used by STAR-API.

Types used to describe drivers, devices and channels.

#### Author

STAR-Dundee Ltd.  
STAR House  
166 Nethergate  
Dundee, DD1 4EE  
Scotland, UK  
e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.42.2 Macro Definition Documentation

#### 9.42.2.1 `#define CFG_INFO_ID_TYPE U32`

Type used for driver, device, channel, application and listener identifiers.

#### Added in version:

[STAR-System v0.8 \(Beta 1\) - 22nd August 2011](#)

#### 9.42.2.2 `#define STAR_DEVICE_STAR_ULTRA_PCIE (STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE)`

**Deprecated** Replaced by [STAR\\_DEVICE\\_STAR\\_ULTRA\\_PCIE\\_INTERFACE](#). The STAR-Dundee STAR-Ultra PCIe Interface device type.

#### Added in version:

[STAR-System v4.00 - 19th November 2019](#)

## 9.43 version.h File Reference

Version information for a module.

```
#include <stdio.h>
#include "star-dundee_types.h"
```

### Data Structures

- struct [STAR\\_VERSION\\_INFO](#)  
*A structure used to store version information for a module. [More...](#)*

### Macros

- `#define VERSION_MAJOR (6)`  
*The module's major version number.*

- `#define VERSION_MINOR (1)`  
*The module's minor version number.*
- `#define VERSION_EDIT (12653)`  
*The module's edit number.*
- `#define VERSION_PATCH (0)`  
*The module's patch number.*
- `#define STAR_STR_MAX_LEN 256`  
*Length of strings stored in a version information structure, inclusive of null terminator.*
- `#define PRINT_FULL_VERSION_INFO(o)`  
*Macro to print the full version information, including name and author.*
- `#define PRINT_VERSION(o)`  
*Macro to print the version information.*
- `#define PRINT_VERSION_EDIT(o) (void)fprintf(o, "(%d)", VERSION_EDIT)`  
*Macro to print the version's edit number if present.*
- `#define PRINT_VERSION_PATCH(o)`  
*Macro to print the version's patch number if present.*
- `#define PRINT_VERSION_NUM(o)`  
*Macro to print the version number.*
- `#define POPULATE_VERSION(v)`  
*Macro to populate a version information structure.*

### 9.43.1 Detailed Description

Version information for a module.

#### Author

STAR-Dundee Ltd.  
 STAR House  
 166 Nethergate  
 Dundee, DD1 4EE  
 Scotland, UK  
 e-mail: [support@star-dundee.com](mailto:support@star-dundee.com)

This file contains the current version information used by each of the modules in STAR-System, the STAR-Dundee software stack. Before including this file, `VERSION_NAME` and `VERSION_AUTHOR` macros should be defined to provide the name and authors of the module, e.g.

```
#define VERSION_NAME ("SpaceWire PCI Driver for Windows")
#define VERSION_AUTHOR ("Stuart Mills, STAR-Dundee Ltd.")
```

Copyright © 2024 STAR-Dundee Ltd.

Unpublished - rights reserved under the Copyright Laws of the United States and International treaties.

### 9.43.2 Data Structure Documentation

#### 9.43.2.1 struct STAR\_VERSION\_INFO

A structure used to store version information for a module.

#### Examples:

[all\\_version\\_example.c](#), [api\\_version\\_example.c](#), [firmware\\_version\\_example.c](#), [star-test.c](#), and [utilities.c](#).

Collaboration diagram for STAR\_VERSION\_INFO:



Data Fields

|      |                          |                                                                      |
|------|--------------------------|----------------------------------------------------------------------|
| char | author[STAR_STR_MAX_LEN] | The author of the module.                                            |
| U16  | edit                     | The edit number of this module.                                      |
| U16  | major                    | The major version number of this module.                             |
| U16  | minor                    | The minor version number of this module.                             |
| char | name[STAR_STR_MAX_LEN]   | The name of the module.                                              |
| U16  | patch                    | The patch number of this module. Edit will be 0 if patch is non zero |

9.43.3 Macro Definition Documentation

9.43.3.1 #define POPULATE\_VERSION( v )

Value:

```
{
 (v).major = VERSION_MAJOR;
 (v).minor = VERSION_MINOR;
 (v).edit = VERSION_EDIT;
 (v).patch = VERSION_PATCH;

 strcpy_s((v).name, STAR_STR_MAX_LEN, VERSION_NAME);
 strcpy_s((v).author, STAR_STR_MAX_LEN, VERSION_AUTHOR);
}
```

Macro to populate a version information structure.

Parameters

|   |                                                 |
|---|-------------------------------------------------|
| v | the version information structure to be updated |
|---|-------------------------------------------------|

9.43.3.2 #define PRINT\_FULL\_VERSION\_INFO( o )

Value:

```
{
 fprintf(o, "Module Name: %s\n", VERSION_NAME); \
 fprintf(o, "Module Author: %s\n", VERSION_AUTHOR); \
 fprintf(o, "Version: "); \
 PRINT_VERSION_NUM(o); \
 fprintf(o, "\n"); \
}
```

Macro to print the full version information, including name and author.

#### Parameters

---

*o*    the output stream

---

#### 9.43.3.3 #define PRINT\_VERSION( *o* )

##### Value:

```
{
 (void)fprintf(o, "%s", VERSION_NAME); \
 PRINT_VERSION_NUM(o); \
}
```

Macro to print the version information.

#### Parameters

---

*o*    the output stream

---

#### 9.43.3.4 #define PRINT\_VERSION\_EDIT( *o* ) (void)fprintf(o, "(%d)", VERSION\_EDIT)

Macro to print the version's edit number if present.

Used by [PRINT\\_VERSION\\_NUM\(FILE \\*\)](#) but contained in a separate macro to avoid "conditional expression is constant" warnings.

#### Parameters

---

*o*    the output stream

---

#### 9.43.3.5 #define PRINT\_VERSION\_NUM( *o* )

##### Value:

```
{
 (void)fprintf(o, "v%d.%02d", VERSION_MAJOR, VERSION_MINOR); \
 PRINT_VERSION_EDIT(o); \
 PRINT_VERSION_PATCH(o); \
}
```

Macro to print the version number.

#### Parameters

---

*o*    the output stream

---

#### 9.43.3.6 #define PRINT\_VERSION\_PATCH( *o* )

Macro to print the version's patch number if present.

Used by [PRINT\\_VERSION\\_NUM\(FILE \\*\)](#) but contained in a separate macro to avoid "conditional expression is constant" warnings.



---

**Parameters**

---

*o* the output stream

---

**9.43.3.7 #define VERSION\_EDIT (12653)**

The module's edit number.

**9.43.3.8 #define VERSION\_MAJOR (6)**

The module's major version number.

**9.43.3.9 #define VERSION\_MINOR (1)**

The module's minor version number.

**9.43.3.10 #define VERSION\_PATCH (0)**

The module's patch number.



## Chapter 10

# Example Documentation

### 10.1 advanced\_address\_example.c

This example assumes that channel 1 (defined in general header for api\_test\_app project) is connected to link 2 on a SpaceWire USB Brick or other routing device. Any packets sent out of channel 1 will end up coming back to where they started.

#### PCI Mk2 Brick

Link 1  Link 2

Link 2 

Link 3 

```
#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedAddressExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open channel to send out of and receive into */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice
 (firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* If transmit channel was opened */
 if(channel)
 {
 /* Create address path */
 unsigned char addressPath[] = {2};

 /* Get address path length */
 U16 addressPathLength = sizeof(addressPath) / sizeof(unsigned char);

 /* Create SpaceWire address */
 STAR_SPACEWIRE_ADDRESS * address =
 STAR_createAddress(addressPath,
 addressPathLength);

 /* If address was created */
 if(address)
 {
 /* Define packet data to be sent out */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Get packet length */
 unsigned int packetDataLength = sizeof(packetData) /
 sizeof(unsigned char);
```

```

/* Create packet */
STAR_STREAM_ITEM * packet = STAR_createPacket(address,
 (unsigned char *)packetData, packetDataLength,
 STAR_EOP_TYPE_EOP);

/* If packet was created */
if(packet)
{
 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If send transfer operation was created */
 if (sendTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel,
 sendTransferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion
(
 sendTransferOperation, -1);

 /* Close transmit channel */
 STAR_closeChannel(channel);

 /* Open receive channel */
 channel = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 0);

 /* If receive channel was opened */
 if(channel)
 {
 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION
 *receiveTransferOperation =
 STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);

 /* If receive transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Start receiving packet */
 STAR_submitTransferOperation(channel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status =
STAR_waitOnTransferOperationCompletion
 (receiveTransferOperation, -1);

 /* If valid packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
)
 {
 /* Get received packet */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data =
 STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);
 if(data != NULL)
 {
 /* Print received packet */
 printPacketContents(data,
 dataLength);

 /* Destroy packet data now that it
 has been output */
 STAR_destroyPacketData(data);
 }
 }
 }
 }
 }
}

```

```

 /* Dispose receive transfer operation */
 STAR_disposeTransferOperation(
 receiveTransferOperation);
 }
}

/* Dispose send transfer operation */
STAR_disposeTransferOperation(sendTransferOperation);
}

/* Destroy packet */
STAR_destroyStreamItem(packet);
}

/* Dispose address */
STAR_destroyAddress(address);
}

/* Close channel */
STAR_closeChannel(channel);
}
}
}

```

## 10.2 advanced\_continuous\_receive\_example.c

Loops over the receive functions until 10 packets have been received. After a packet is received, the contents are displayed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedContinuousReceiveExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 1);

 /* Initialise received packet count to 0 */
 unsigned int receivedPacketCount = 0;

 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);

 /* While less than 10 packets have been received */
 while(receivedPacketCount < 10)
 {
 /* Define transfer status for receive transfer status updates */
 STAR_TRANSFER_STATUS status;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion(
 receiveTransferOperation, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);
 }
 }
 }
}

```

```

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
item,
 &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }

 /* Increment received packet count */
 receivedPacketCount++;
 }
}

/* Dispose receive transfer operation */
STAR_disposeTransferOperation(receiveTransferOperation);

/* Close receive channel */
STAR_closeChannel(receiveChannel);
}
}

```

### 10.3 advanced\_multiple\_packet\_example.c

Creates 3 packets, puts them in an array and then transmits the array in a single transfer operation. The packets are then received and printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedMultiplePacketExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Create first packet data */
 unsigned char packetData1[] = {1, 2, 3, 4, 5};

 /* Get length of first packet */
 unsigned int packetData1Length = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Create first packet */
 STAR_STREAM_ITEM * packet1 = STAR_createPacket(NULL, packetData1,
 packetData1Length, eopType);

 /* Create second packet data */
 unsigned char packetData2[] = {6, 7, 8, 9, 10};

 /* Get length of second packet */
 unsigned int packetData2Length = sizeof(packetData2) /
 sizeof(unsigned char);

 /* Create second packet */
 STAR_STREAM_ITEM * packet2 = STAR_createPacket(NULL, packetData2,
 packetData2Length, eopType);

 /* Create third packet data */
 unsigned char packetData3[] = {11, 12, 13, 14, 15};
 }
}

```

```

/* Get length of third packet */
unsigned int packetData3Length = sizeof(packetData3) /
 sizeof(unsigned char);

/* Create third packet */
STAR_STREAM_ITEM * packet3 = STAR_createPacket(NULL, packetData3,
 packetData3Length, eopType);

/* If valid packets */
if(packet1 && packet2 && packet3)
{
 /* Open channel to send packets out of */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Define send transfer operation for sending packets */
 STAR_TRANSFER_OPERATION * sendTransferOperation;

 /* Wrap individual packets in an array */
 STAR_STREAM_ITEM * streamItems[3];
 streamItems[0] = packet1;
 streamItems[1] = packet2;
 streamItems[2] = packet3;

 /* Create send operation to send packets */
 sendTransferOperation =
 STAR_createTxOperation(streamItems, 3);

 /* If transfer operation was created */
 if(sendTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS transmitStatus;

 /* Start transmitting the packets */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait for packets to be transmitted */
 transmitStatus = STAR_waitOnTransferOperationCompletion
(
 sendTransferOperation, -1);

 /* If packets were transmitted */
 if(transmitStatus == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Counter for loop */
 unsigned int index;

 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(3,
STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS receiveStatus;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 receiveStatus =
 STAR_waitOnTransferOperationCompletion
(
 receiveTransferOperation, -1);

 /* If valid stream item was received */
 if (receiveStatus ==
 STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get transfer item count */

```





```

 &selectedChannelNumber, TRANSMIT_TYPE_RECEIVE);

/* If channel was opened */
if(selectedChannel)
{
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * transferOperation =
 STAR_createRxOperation
 (1, STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(transferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Start receiving the packet */
 STAR_submitTransferOperation(selectedChannel, transferOperation);

 /* Print waiting on incoming packet message */
 printf("Waiting on incoming packet...\n");

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 transferOperation, 0);

 /* If valid stream item was received */
 if(streamItem)
 {
 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
item,
 &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
 }
}

```

## 10.5 advanced\_send\_and\_receive\_example.c

A simple packet is created, sent using a transfer operation out of one channel which is then received on another. The received packet is printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void advancedSendAndReceiveExample()

```

```

{
 /* Define transfer status for send and receive status information */
 STAR_TRANSFER_STATUS transmitStatus;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData) /
 sizeof(unsigned char);

 /* Create packet */
 STAR_STREAM_ITEM * packet = STAR_createPacket(NULL,
 (unsigned char *)packetData, packetLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)
 {
 /* Create transfer operation to send packet */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If transfer operation was created */
 if(sendTransferOperation)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait for packet to be transmitted */
 transmitStatus = STAR_waitOnTransferOperationCompletion

(
 sendTransferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet);

 /* If packet was sent */
 if(transmitStatus == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1,
STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS receiveStatus;

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 receiveStatus =
 STAR_waitOnTransferOperationCompletion

(
 receiveTransferOperation, -1);

 /* If valid stream item was received */

```

```

 if(receiveStatus ==
 STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char *data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the
 receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has
 been output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose receive transfer operation */
 STAR_disposeTransferOperation(
 receiveTransferOperation);
 }

 /* Close receive channel */
 STAR_closeChannel(receiveChannel);
}

}

}

/* Close send channel */
STAR_closeChannel(sendChannel);
}

}

```

## 10.6 advanced\_send\_example.c

The user is prompted for the device to use, the channel to send out of and a series of bytes to be sent in the packet. The packet is sent using the advanced transfer functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void advancedSendExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = NULL;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Allocate array of bytes to be populated */

```

```

unsigned char bytes[256];
U16 readBytesLength = 0;

/* Define packet that will hold the packet buffer data */
STAR_STREAM_ITEM * packet;

/* Ask user to enter a series of bytes to send */
printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

/* If valid bytes read input bytes from console */
if(readBytes(bytes, 256, &readBytesLength))
{
 /* Create packet */
 packet = STAR_createPacket(NULL, bytes,
 (unsigned int)readBytesLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)
 {
 /* Create transfer operation */
 STAR_TRANSFER_OPERATION * transferOperation =
 STAR_createTxOperation(&packet, 1);

 /* If transfer operation was created */
 if(transferOperation)
 {
 /* Define transfer status for result of transfer
 operation */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* Check transfer status */
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Unexpected transfer status: %d\n", status);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet);
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
}
else
{
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 10.7 advanced\_two\_way\_example.c

Two packets are constructed and then sent in either direction using two way channels and the advanced transfer functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

```

```

void receiveAdvancedPacket(STAR_CHANNEL_ID channel)
{
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Define transfer status for receive status information */
 STAR_TRANSFER_STATUS status;

 /* Start receiving packet */
 STAR_submitTransferOperation(channel,
 receiveTransferOperation);

 /* Wait for packet to be received */
 status = STAR_waitOnTransferOperationCompletion
 (receiveTransferOperation, -1);

 /* If valid stream item was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 if (data != NULL)
 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(receiveTransferOperation);
 }
 }
}

void advancedTwoWayExample()
{
 /* Define transfer status for send and receive status information */
 STAR_TRANSFER_STATUS status;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent out of first channel */
 unsigned char packetData1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Define packet data to be sent out of second channel */
 unsigned char packetData2[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};

 /* If there is a device to use */
 if(firstDevice)
 {
 /* Open channel to send and receive on */
 STAR_CHANNEL_ID channel1 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* Open channel to send and receive on */
 STAR_CHANNEL_ID channel2 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL2, 1);

 /* If both channels were opened */
 if(channel1 && channel2)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Create packet to send out of first channel */
 STAR_STREAM_ITEM * packet1 = STAR_createPacket(NULL,

```

```

 (unsigned char *)packetData1, packetLength, STAR_EOP_TYPE_EOP);

/* If packet was created */
if(packet1)
{
 /* Create transfer operation to send packet out of first
 channel */
 STAR_TRANSFER_OPERATION * sendTransferOperation1 =
 STAR_createTxOperation(&packet1, 1);

 /* If transfer operation was created */
 if(sendTransferOperation1)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel1,
 sendTransferOperation1);

 /* Wait for packet to be trasmitted */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation1, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation1);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet1);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Define packet to be sent out of second channel */
 STAR_STREAM_ITEM * packet2;

 /* Receive packet on channel 2 */
 receiveAdvancedPacket(channel2);

 /* Get packet length */
 packetLength = sizeof(packetData2) /
 sizeof(unsigned char);

 /* Create packet to send out of second channel */
 packet2 = STAR_createPacket(NULL,
 (unsigned char *)packetData2, packetLength,
 STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet2)
 {
 /* Create transfer operation to send packet out of
 second channel */
 STAR_TRANSFER_OPERATION * sendTransferOperation2 =
 STAR_createTxOperation(&packet2, 1);

 /* If transfer operation was created */
 if(sendTransferOperation2)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(channel2,
 sendTransferOperation2);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation2, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(
 sendTransferOperation2);

 /* Destroy packet after it was sent */
 STAR_destroyStreamItem(packet2);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on first channel */
 receiveAdvancedPacket(channel1);
 }
 }
 }
 }
 }
}

/* Close first channel */
STAR_closeChannel(channel1);

```

```

 /* Close second channel */
 STAR_closeChannel(channel2);
 }
}

```

## 10.8 all\_version\_example.c

Uses the `STAR_getAllVersions()` function to retrieve a list of versions of the modules that are in use and prints the version information.

```

#include "examples.h"

#include "utilities.h"

#include "star-api.h"

void allVersionExample()
{
 /* Define number of versions */
 U32 numberOfVersions = 0;

 /* Define counter for loop */
 unsigned int index;

 /* Get all versions */
 STAR_VERSION_INFO *versionInfos = STAR_getAllVersions(&
 numberOfVersions);

 /* For number of versions */
 for(index = 0; (versionInfos != NULL) && (index < numberOfVersions);
 index++)
 {
 /* Print version information string */
 printVersionInfo(&versionInfos[index]);

 /* If not the last version */
 if (index < (numberOfVersions - 1))
 {
 /* Print blank line to separate */
 printf("\n");
 }
 }

 /* Destroy version info string */
 STAR_destroyVersionInfoList(versionInfos);
}

```

## 10.9 api\_test\_app.c

This file is the entry point to various example functions that demonstrate the capabilities of STAR-API including transmitting and receiving packets.

```

#include "examples.h"

#if !defined(UNREFERENCED_PARAMETER)
 #define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

#include <stdlib.h>

#include "star-api.h"

int __cdecl main(int argc, char *argv[])
{
 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System API Test Application");

 /* Configure the transmit and receive channels in general.h
 Uncomment the example(s) below to run */

```

```

//advancedAddressExample();
//advancedContinuousReceiveExample();
//advancedMultiplePacketExample();
//advancedReceiveExample();
//advancedSendAndReceiveExample();
//advancedSendExample();
//advancedTwoWayExample();
//allVersionExample();
//apiVersionExample();
//applicationNameExample();
//broadcastMessageExample();
//channelCallbackExample();
//driverListExample();
//exReceiveExample();
//firmwareVersionExample();
//simpleReceiveExample();
//simpleSendAndReceiveExample();
//simpleSendExample();
//simpleTwoWayExample();
//timecodeExample();
//transferCallbackExample();
//transferOperationListSendExample();
//transferOperationListSendReceiveExample();
//transmitErrorsExample();

return 0;
}

```

## 10.10 api\_version\_example.c

Uses the `STAR_getApiVersion()` function to retrieve the current API version that is in use and prints it.

```

#include "examples.h"

#include "utilities.h"

#include "star-api.h"

void apiVersionExample()
{
 /* Get API version info */
 STAR_VERSION_INFO * versionInfo = STAR_getApiVersion();

 /* Print version info */
 printVersionInfo(versionInfo);

 /* Destroy version info */
 STAR_destroyVersionInfo(versionInfo);
}

```

## 10.11 application\_name\_example.c

Prints the initial application name, sets it to a new value and then prints the new value.

```

#include "examples.h"

#include "star-api.h"

void applicationNameExample()
{
 /* Get initial application name */
 char * applicationName = STAR_getApplicationName();

 if (applicationName != NULL)
 {
 /* Print initial application name */
 printf("Initial application name: %s\n", applicationName);

 /* Destroy application name */
 STAR_destroyString(applicationName);
 }

 /* Set application name */
 STAR_setApplicationName("Test STAR-System Application");
}

```



```

/* Get application name */
applicationName = STAR_getApplicationName();

if (applicationName != NULL)
{
 /* Print application name */
 printf("Application name: %s\n", applicationName);

 /* Destroy application name */
 STAR_destroyString(applicationName);
}
}

```

## 10.12 brickMk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include <time.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

#ifdef _WIN32

#include <windows.h>

#define SLEEP(time) Sleep(time)

#else

#if defined(__vxworks)

void SLEEP(unsigned int delay)
{
 double rate = (double)sysClkRateGet();
 unsigned int ticks = (int)((rate / 1000.0) * delay + 0.5);
 taskDelay(ticks);
}

#else

#include <unistd.h>
#define SLEEP(time) usleep(time * 1000)

#endif

#endif

int __cdecl main()
{
 STAR_DEVICE_ID deviceID, remoteDeviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 int enabled, success, transferItemCount;
 double linkSpeed;
 U32 originalTimeCodePeriod;
 double signallingRate, newSignallingRate, signallingRateSet;

 STAR_CHANNEL_ID channelID, remoteChannelID;
 STAR_TRANSFER_OPERATION *linkEventRxOp;
 STAR_STREAM_ITEM *linkEventZero, *linkEventOne;
 STAR_CFG_BRICK_MK2_ERRORS linkErrors;
 U32 newLinkSpeedZero, newLinkSpeedOne;
 STAR_CFG_SPW_LINK_STATUS linkState, originalLinkState;
 PORT_STATUS_CONTROL portStatusControl;

 unsigned char aPath[] = { 0, 254 };
 unsigned char aRetPath[] = { 254 };

 STAR_SPACEWIRE_ADDRESS* pPath;
 STAR_SPACEWIRE_ADDRESS* pRetPath;

 STAR_DEVICE_TYPE aDeviceTypes[2];
 aDeviceTypes[0] = STAR_DEVICE_BRICK_MK2;
 aDeviceTypes[1] = STAR_DEVICE_ROUTER_MK2S;

 /* Select device to configure */

```

```

deviceID = STAR_UI_chooseDevice(aDeviceTypes, 2);
if (!deviceID)
{
 return 0;
}

STAR_getDeviceType(deviceID);

/* Get hardware info*/
CFG_getFPGAInfo(deviceID, &fpgaInfo);

CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the device's LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Get current time-code period */
CFG_getTimeCodePeriod(deviceID, &originalTimeCodePeriod);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Enable interface mode on port 1 only */
CFG_enableInterfaceModeOnPort(deviceID, 1);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

CFG_getInterfaceModeOnPortEnabled(deviceID, 2, &enabled);

printf("\nInterface mode %s on port 2", enabled ? "enabled" : "disabled");

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 2 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 2);

/* Check if identify source is enabled for port 1 */
CFG_getIdentifySourceOnPortEnabled(deviceID, 1, &enabled);
printf("\nIdentify source %s on port 1", enabled ? "enabled" : "disabled");

CFG_disableIdentifySourceOnPort(deviceID, 1);

/* Enable Link Speed Events */
CFG_enableSpeedChangeEventsOnPort(deviceID, 2);

pPath = STAR_createAddress(aPath, sizeof(aPath));
pRetPath = STAR_createAddress(aRetPath, sizeof(aRetPath));

/* To receive link speed events, we must open a connection to device
 channel 0. This will prevent configuration operations from being
 performed until we close this channel again. */

/* Sleep for several hundreds of ms so that the config channel
 closes after the last configuration function had been called
 In this case that is CFG_enableSpeedChangeEventsOnPort */
SLEEP(300);

channelID = STAR_openChannelToLocalDevice(deviceID,
 STAR_CHANNEL_DIRECTION_IN,
 0,
 0);

printf("\nConfiguration channel ID: %d", channelID);

remoteDeviceID = STAR_CFG_createRemoteDeviceIdentifier(
 deviceID, 1, NULL, pPath, pRetPath);

linkEventRxOp = STAR_createRxOperation(2,
 STAR_RECEIVE_LINK_SPEED_EVENTS);

/* Make sure that the link is running */

```

```

/* Get port status control */
success = CFG_getPortStatusControl(remoteDeviceID, 1, &portStatusControl);

/* Get SpW Link Status */
success = CFG_getSpaceWireLinkStatus(portStatusControl, &linkState);

/* Save original link state */
originalLinkState = linkState;

/* Modify the values of the link state structure, to start the link */
linkState.start = 1;
linkState.running = 1;
success = CFG_setSpaceWireLinkStatus(remoteDeviceID, 1, &linkState);

/* Get receive signalling rate */
CFG_getRxSignallingRate(remoteDeviceID, 2, &linkSpeed);

printf("\nRx Link Speed: %g Mbit/s", linkSpeed);

success = STAR_submitTransferOperation(channelID, linkEventRxOp);

/* Get link transmit signalling rate */
CFG_getTxSignallingRate(remoteDeviceID, 1, &signallingRate);
printf("\nCurrent tx signalling rate on port 1: %g Mbit/s", signallingRate);

/* Set link speed for link 1 */
if (signallingRate != 130)
{
 newSignallingRate = 130;
}
else
{
 newSignallingRate = 200;
}

printf("\nSetting the signalling rate to: %g Mbit/s.", newSignallingRate);
success = CFG_setTxSignallingRate(remoteDeviceID, 1,
 newSignallingRate, &signallingRateSet);

/* Wait for up to 2 seconds for link speed event to occur */
printf("\nWaiting for Link speed event(s)...\n");
STAR_waitOnTransferOperationCompletion(linkEventRxOp, 2000);

/* Display new link speed */
switch(STAR_getTransferOperationStatus(linkEventRxOp)) {
case STAR_TRANSFER_STATUS_COMPLETE:
 transferItemCount = STAR_getTransferItemCount(linkEventRxOp);

 printf("Received %d Link Event items", transferItemCount);

 if (transferItemCount > 1)
 {
 linkEventZero = STAR_getTransferItem(linkEventRxOp, 0);
 linkEventOne = STAR_getTransferItem(linkEventRxOp, 1);

 newLinkSpeedZero = STAR_getLinkSpeedEventSpeed(
 (STAR_LINK_SPEED_EVENT*)(linkEventZero->
item));

 newLinkSpeedOne = STAR_getLinkSpeedEventSpeed(
 (STAR_LINK_SPEED_EVENT*)(linkEventOne->item));

 printf("\nNew Link Speed: %u Kbit/s", newLinkSpeedOne / 1000);
 }
 else
 {
 printf("2 Link Event items should have been received.");
 printf(" Received only: %d", transferItemCount);
 }

 break;
case STAR_TRANSFER_STATUS_STARTED:
case STAR_TRANSFER_STATUS_NOT_STARTED:
 printf("\nLink speed event did not occur");
 STAR_cancelTransferOperation(linkEventRxOp);
 break;
default:
 printf("\nError occurred while waiting for link speed event");
}

/* Destroy receive transfer operation */
STAR_disposeTransferOperation(linkEventRxOp);

/* Close configuration channel */
STAR_closeChannel(channelID);

/* Set the time code period back to the original */

```

```

CFG_setTimeCodePeriod(deviceID, originalTimeCodePeriod);

/* Disable the device to be a time-code master*/
CFG_disableTimeCodeMaster(deviceID);

/* Enable interface mode on port 2 */
CFG_enableInterfaceModeOnPort(deviceID, 2);

/* Disable link */
originalLinkState.disable = 1;
success = CFG_setSpaceWireLinkStatus(deviceID, 1, &originalLinkState);
success = CFG_setSpaceWireLinkStatus(deviceID, 2, &originalLinkState);

CFG_clearPortErrors(deviceID, 1);
CFG_clearPortErrors(deviceID, 2);

/* Set back original link state */
originalLinkState.disable = 0;
originalLinkState.autoStart = 1;
originalLinkState.linkState = STAR_CFG_SPW_LINK_STATE_READY;
success = CFG_setSpaceWireLinkStatus(deviceID, 1, &originalLinkState);
success = CFG_setSpaceWireLinkStatus(deviceID, 2, &originalLinkState);

CFG_enableIdentifySourceOnPort(deviceID, 1);

CFG_disableIdentifySource(deviceID);

success = CFG_disableSpeedChangeEventsOnPort(deviceID, 2);

/* Destroy addresses */
STAR_destroyAddress(pPath);
STAR_destroyAddress(pRetPath);

/* Destroy remote device identifier */
STAR_CFG_destroyRemoteDeviceIdentifier(remoteDeviceID);

printf("\nPress Enter to exit.");
getchar();

return 0;
}

```

## 10.13 broadcast\_message\_example.c

Sends and receives a broadcast message.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

static void broadcastMessageExampleCleanup(
 STAR_STREAM_ITEM *pTransmitBroadcast,
 STAR_TRANSFER_OPERATION *pTransmitOp,
 STAR_TRANSFER_OPERATION *pReceiveOp, STAR_CHANNEL_ID
 transmitChannelId,
 STAR_CHANNEL_ID receiveChannelId);

void broadcastMessageExample()
{
 STAR_STREAM_ITEM *pTransmitStreamItem = NULL;
 STAR_STREAM_ITEM *pReceiveStreamItem = NULL;
 STAR_BROADCAST_MESSAGE *pReceiveBroadcast = NULL;
 STAR_TRANSFER_OPERATION *pTransmitOp = NULL;
 STAR_TRANSFER_OPERATION *pReceiveOp = NULL;
 STAR_CHANNEL_ID transmitChannelId = 0;
 STAR_CHANNEL_ID receiveChannelId = 0;
 STAR_TRANSFER_STATUS transmitStatus =
 STAR_TRANSFER_STATUS_NOT_STARTED;
 STAR_TRANSFER_STATUS receiveStatus =
 STAR_TRANSFER_STATUS_NOT_STARTED;

 /* Get the first STAR-Ultra PCIe Interface device */
 STAR_DEVICE_ID deviceId = getFirstDeviceForType(
 STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE);
 if (!deviceId)
 {
 printf("Error: no STAR-Ultra PCIe Interface devices detected\n");
 }
}

```

```

 return;
 }

 /* Create a broadcast message */
 pTransmitStreamItem = STAR_createBroadcastMessage(
 0x00, 0x20, 0x00, 123, 456);
 if (!pTransmitStreamItem)
 {
 printf("Error: failed to create broadcast message\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Create a transmit operation */
 pTransmitOp = STAR_createTxOperation(&pTransmitStreamItem, 1);
 if (!pTransmitOp)
 {
 printf("Error: failed to create transmit operation\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Open the transmit channel */
 transmitChannelId = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_OUT, 0, 1);
 if (!transmitChannelId)
 {
 printf("Error: failed to open channel 0\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Open the receive channel */
 receiveChannelId = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_IN, 8, 1);
 if (!receiveChannelId)
 {
 printf("Error: failed to open channel 8\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Create a receive operation */
 pReceiveOp = STAR_createRxOperation(1,
 STAR_RECEIVE_BROADCAST_MESSAGES);
 if (!pReceiveOp)
 {
 printf("Error: failed to create receive operation\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Submit the receive operation */
 if (!STAR_submitTransferOperation(receiveChannelId, pReceiveOp))
 {
 printf("Error: failed to submit receive operation\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Submit the transmit operation */
 if (!STAR_submitTransferOperation(transmitChannelId, pTransmitOp))
 {
 printf("Error: failed to submit transmit operation\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }

 /* Wait on the transmit operation completing */
 transmitStatus = STAR_waitOnTransferOperationCompletion(
 pTransmitOp, -1);
 if (transmitStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Error: transmit operation didn't complete (status=%d)\n",
 transmitStatus);
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
 }
}

```

```

/* Wait on the receive operation completing */
receiveStatus = STAR_waitOnTransferOperationCompletion(pReceiveOp
, -1);
if (receiveStatus != STAR_TRANSFER_STATUS_COMPLETE)
{
 printf("Error: receive operation didn't complete (status=%d)\n",
 receiveStatus);
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
}

/* Get the stream item */
pReceiveStreamItem = STAR_getTransferItem(pReceiveOp, 0);
if (!pReceiveStreamItem)
{
 printf("Error: failed to get transfer item\n");
 broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
 return;
}

/* Get the broadcast message */
pReceiveBroadcast = (STAR_BROADCAST_MESSAGE *)pReceiveStreamItem->
 item;

/* Print the broadcast message */
printf("Broadcast received: channel=%u, type=%u, status=%u, data=[%u, %u]\n",
 STAR_getBroadcastMessageChannel(pReceiveBroadcast),
 STAR_getBroadcastMessageType(pReceiveBroadcast),
 STAR_getBroadcastMessageStatus(pReceiveBroadcast),
 STAR_getBroadcastMessageDataWord1(pReceiveBroadcast),
 STAR_getBroadcastMessageDataWord2(pReceiveBroadcast));

/* Clean up and close channels */
broadcastMessageExampleCleanUp(pTransmitStreamItem, pTransmitOp,
 pReceiveOp, transmitChannelId, receiveChannelId);
}

void broadcastMessageExampleCleanUp(
 STAR_STREAM_ITEM *pTransmitBroadcast,
 STAR_TRANSFER_OPERATION *pTransmitOp,
 STAR_TRANSFER_OPERATION *pReceiveOp, STAR_CHANNEL_ID
 transmitChannelId,
 STAR_CHANNEL_ID receiveChannelId)
{
 if (pTransmitBroadcast)
 {
 STAR_destroyStreamItem(pTransmitBroadcast);
 }

 if (pTransmitOp)
 {
 STAR_disposeTransferOperation(pTransmitOp);
 }

 if (pReceiveOp)
 {
 STAR_disposeTransferOperation(pReceiveOp);
 }

 if (transmitChannelId)
 {
 STAR_closeChannel(transmitChannelId);
 }

 if (receiveChannelId)
 {
 STAR_closeChannel(receiveChannelId);
 }
}

```

## 10.14 ccsds\_spp\_tf\_examples.c

This file contains several example functions that demonstrate how to use some of the basic functions the CCSD↵S/SPP/TF library.

```

#include "ccsds_spp_packet_library.h"
#include "ccsds_spp_pus_services.h"

```

```

#include <stdio.h>
#include <stdlib.h>

static void CCSDS_SPP_PUS_ST02_Example()
{
 CCSDS_SPP_PACKET* pPacketStruct = { 0 };
 U8* pRawPacket = NULL;
 CCSDS_SPP_STATUS status = CCSDS_SPP_STATUS_GENERAL_ERROR;

 /* Define fields of the primary header */
 const U8 targetLogicalAddress = 0xAB;
 const U8 protocolReservedByte = 0x02;
 const U8 userApplicationByte = 0x56;
 const U16 destAPID = 15, sourceAPID = 30;
 const U16 sequenceCount = 0;
 const U8 acknowledgmentFlags = 2;

 /* Raw packet length of the PUS packet */
 U32 rawPacketLength = 0;

 /* Assign some dummy device addresses */
 const U16 nrOfDeviceAddresses = 2;
 U16 deviceAddresses[2];
 deviceAddresses[0] = 0xAB98;
 deviceAddresses[1] = 0x5D45;

 /* Create PUS command */
 pRawPacket = CCSDS_SPP_PUS_CreateST02DistributeOnOffDeviceCommand
 (
 pPacketStruct, targetLogicalAddress, protocolReservedByte,
 userApplicationByte, destAPID, sequenceCount, acknowledgmentFlags,
 sourceAPID, nrOfDeviceAddresses, (const U16 * const)&deviceAddresses,
 &status, &rawPacketLength);

 /* Check the feedback */
 if (status != CCSDS_SPP_STATUS_SUCCESS || pRawPacket == NULL
 || rawPacketLength == 0) return;

 /* Do something with the raw packet: send it, etc. */
}

static void CCSDS_TF_Packet_Creation_Example()
{
 /* Helper variable used in for loops */
 U16 i = 0;

 /* First we create the CCSDS/SPP packet */
 /* Set up a secondary header */
 const U8 secondaryHeader[10] = { 0x01, 0x02, 0x03,
 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x0A };

 /* Set up the user data field */
 const U8 userDataField[10] = { 0x11, 0x12, 0x13,
 0x14, 0x15, 0x16, 0x17, 0x18, 0x19, 0x1A };

 /* Raw packet length of the CCSDS/SPP packet */
 U32 rawPacketLength = 0;
 CCSDS_SPP_STATUS status = CCSDS_SPP_STATUS_GENERAL_ERROR;

 /* VC Frame count */
 U32 VCFrameCount = 25;

 /* VC Frame count cycle */
 U8 VCFrameCountCycle = 3;

 /* Number of TF packets that will be created by the function in
 the library */
 U16 numberOfTFPacketsCreated = 0;

 /* Size of each of the created TF packets in bytes */
 U16 TFPacketLengthBytes = 0;

 /* Pointer variables */
 U8* pCCSDS_SPP_Packet = NULL;
 U32* pCCSDS_SPP_PacketLengths = NULL;
 U8** CCSDS_SPP_Packets = NULL;
 U8** TFPackets = NULL;

 CCSDS_TF_M_PDU_PACKET retrievedPacket = { 0 }, retrievedPacket1 = { 0 };

 /* Define fields present in the primary header fields */
 U8 secondaryHeaderPresent = 1;
 U8 APID = 0x34;
 U8 sequenceFlags = 0x03;
 U16 sequenceCount = 0x10;
 U16 dataFieldLength = sizeof(secondaryHeader) / sizeof(U8) + sizeof(userDataField) / sizeof(
 U8);

```

```

U8 versionNumber = 0x02;
U8 spaceCraftID = 0x55;
U8 virtualChannelID = 0x15;
U8 replayFlag = 0;
U8 virtualChannelFrameCountUsageFlag = 0;

/* Create the CCSDS/SPP Packet (with the parameters as follows):
 * Temporary packet structs will be used
 * No encapsulation header (encapsulation header length 0)
 * Packet type is TELECOMMAND
 * Secondary header present
 * APID as defined above
 * Sequence flags as defined above
 * Sequence count as defined above
 * Data field length as defined above
 * Secondary header and length
 * User data field (data)
 */
pCCSDS_SPP_Packet = CCSDS_SPP_CreatePacket(NULL, NULL, 0,
 CCSDS_SPP_PACKET_TYPE_TELECOMMAND, secondaryHeaderPresent,
 APID, sequenceFlags, sequenceCount, dataFieldLength,
 &secondaryHeader[0], 10, &userDataField[0], &rawPacketLength, &status);

/* Allocate memory for the array holding the packet lengths of the
 CCSDS/SPP packets */
pCCSDS_SPP_PacketLengths = (U32*)malloc(sizeof(U32) * 1);

/* Set the value of the packet length for the CCSDS/SPP packet created */
pCCSDS_SPP_PacketLengths[0] = rawPacketLength;

CCSDS_SPP_Packets = (U8**)malloc(sizeof(U8*) * 1);
CCSDS_SPP_Packets[0] = pCCSDS_SPP_Packet;

/* Create TF packets (with parameters as follows):
 * CCSDS/SPP Packets that had been previously created
 * Array containing the CCSDS/SPP packet length
 * Number of CCSDS/SPP packets (1 in this case)
 * No encapsulation header (NULL and 0 length)
 * Version number as defined above
 * Spacecraft ID as defined above
 * Virtual channel ID as defined above
 * Virtual Channel Frame Count as defined above
 * Replay flag as defined above
 * Virtual Channel Frame Count Usage Flag as defined above
 * Virtual Channel Frame Count Cycle
 * No Frame Header Error Control (NULL)
 * No Insert Zone Data (NULL and 0 length)
 * No operational control field (NULL)
 * No frame error control field (NULL)
 */
TFPackets = CCSDS_TF_CreatePackets(CCSDS_SPP_Packets,
 pCCSDS_SPP_PacketLengths, 1, NULL, 0, versionNumber, spaceCraftID,
 virtualChannelID, &VCFFrameCount, replayFlag,
 virtualChannelFrameCountUsageFlag, VCFFrameCountCycle, NULL, NULL, 0,
 &numberOfTFPacketsCreated, &TFPacketLengthBytes, NULL, NULL,
 &status);

/* Demonstrate how to retrieve TF packets */
/* When retrieving TF packets, one needs to specify the following:
 1) Encapsulation header length (non-zero values means it is present)
 2) Whether the frame header error control field is present
 3) Whether the insert zone data field is present
 4) Whether the operational control field is present
 5) Whether the frame error control field is present */
retrievedPacket = CCSDS_TF_RetrievePacket(TFPackets[0], 0, 0, 0, 0, 0,
 &status);

/* It is then up to the user to retrieve the CCSDS/SPP packets from the
 M PDU type transfer frames */

/* Free up the memory allocated in this sample program */
CCSDS_SPP_FreeBuffer(CCSDS_SPP_Packets[0]);
CCSDS_SPP_FreeBuffer(CCSDS_SPP_Packets);

/* Free up memory that was allocated by the library */
for (i = 0; i < numberOfTFPacketsCreated; i++)
{
 CCSDS_SPP_FreeBuffer(TFPackets[i]);
}

CCSDS_SPP_FreeBuffer(TFPackets);
}

static void CCSDS_TF_Idle_Packet_Creation_Example()
{
 /* Status variable used for feedback and error checking */

```



```

CCSDS_SPP_STATUS status = CCSDS_SPP_STATUS_GENERAL_ERROR;

/* Size of the idle packet */
U16 idlePacketSize = 10;

/* Length of the raw packet */
U32 rawPacketLength = 0;

/* Create idle packet */
U8* idlePacket = CCSDS_TF_CreateIdlePacket(idlePacketSize,
&rawPacketLength, &status);

/* Check for success */
if (status != CCSDS_SPP_STATUS_SUCCESS || rawPacketLength == 0) return;
}

static void CCSDS_SPP_Basic_Example()
{
/* Define CCSDS/SPP packet structures */
CCSDS_SPP_PACKET* pPacketStruct = NULL, pRetrievedPacketStruct;

/* Define data pointers, header pointers, etc. */
U8* pRawPacket = NULL, *pDataField = NULL;
U8* pEncapsulationHeader = NULL, *pSecondaryHeader = NULL,
*pUserDataField = NULL;

/* Status variable used for feedback and error checking */
CCSDS_SPP_STATUS status = CCSDS_SPP_STATUS_GENERAL_ERROR;

/* Helper variable used in for loops */
U32 i = 0;

/* Length of the data field of the retrieved CCSDS/SPP packet */
U16 retrievedDataFieldLength = 0;

/* Length of the raw packet of the CCSDS/SPP packet */
U32 rawPacketLength = 0;

/* Defined the primary header fields */
const U8 targetLogicalAddress = 0xAB;
const U8 reservedByte = 0x02;
const U8 userApplicationByte = 0x56;
const CCSDS_SPP_PACKET_TYPE type =
CCSDS_SPP_PACKET_TYPE_TELECOMMAND;
const U8 secondaryHeaderFlag = 1;
const U16 APID = 15;
const U8 sequenceFlags = 3;
const U16 sequenceCount = 0;
const U16 encapsulationHeaderLength = 4;
const U16 dataLength = 16;
const U16 secondaryHeaderLength = 10;
const U16 userDataLength = dataLength - secondaryHeaderLength;

/* Allocate memory for the encapsulation header, secondary header
and user data field */
pEncapsulationHeader = (U8*)malloc(encapsulationHeaderLength);
pSecondaryHeader = (U8*)malloc(secondaryHeaderLength);
pUserDataField = (U8*)malloc(userDataLength);

/* Check that the dynamic memory allocation was successful */
if (pEncapsulationHeader == NULL || pSecondaryHeader == NULL
|| pUserDataField == NULL) return;

/* Create PTP encapsulation header */
status = CCSDS_SPP_FillPTPHeader(pEncapsulationHeader,
targetLogicalAddress, reservedByte, userApplicationByte);

/* Check that the previous operation executed successfully */
if (status != CCSDS_SPP_STATUS_SUCCESS) return;

/* Create dummy secondary header */
for (i = 0; i < secondaryHeaderLength; i++)
{
*(pSecondaryHeader + i) = (U8)i;
}

/* Create dummy user data */
for (i = 0; i < userDataLength; i++)
{
*(pUserDataField + i) = (U8)(i * 2);
}

/* Create raw data packet and fill it with the contents of the primary header */
pRawPacket = CCSDS_SPP_CreatePacket(pPacketStruct, pEncapsulationHeader,
encapsulationHeaderLength, type, secondaryHeaderFlag, APID,
sequenceFlags, sequenceCount, dataLength, pSecondaryHeader,
secondaryHeaderLength, pUserDataField, &rawPacketLength, &status);

```

```

/* Check that the previous operation executed successfully */
if (pRawPacket == NULL || status != CCSDS_SPP_STATUS_SUCCESS) return;

/* Retrieve received packet */
pRetrievedPacketStruct = CCSDS_SPP_RetrievePacket(pRawPacket, 1,
 encapsulationHeaderLength, &status);

/* Get pointer that points to the data field */
status = CCSDS_SPP_GetDataFieldPointer(&pRetrievedPacketStruct,
 &pDataField);

/* Check that the previous operation executed successfully */
if (status != CCSDS_SPP_STATUS_SUCCESS || pDataField == NULL) return;

/* Get user data field length */
status = CCSDS_SPP_GetDataFieldLength(
 &pRetrievedPacketStruct, &retrievedDataFieldLength);

/* Check that the previous operation executed successfully */
if (status != CCSDS_SPP_STATUS_SUCCESS) return;

/* Free memory allocated for the encapsulation header */
if (pEncapsulationHeader)
{
 free(pEncapsulationHeader);
 pEncapsulationHeader = NULL;
}

/* Free memory allocated for the secondary header */
if (pSecondaryHeader)
{
 free(pSecondaryHeader);
 pSecondaryHeader = NULL;
}

/* Free memory allocated for the user data field */
if (pUserDataField)
{
 free(pUserDataField);
 pUserDataField = NULL;
}

/* Free memory of the raw packet */
CCSDS_SPP_FreeBuffer(pRawPacket);
}

int main()
{
 //CCSDS_SPP_PUS_ST02_Example();
 //CCSDS_SPP_Basic_Example();

 //CCSDS_TF_Idle_Packet_Creation_Example();
 //CCSDS_TF_Packet_Creation_Example();
}

```

## 10.15 channel\_callback\_example.c

Opens and closes various channels, printing messages when the channel callback notifications are received.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void STAR_API_CC ChannelCallback(STAR_CHANNEL_LISTENER_ID
 channelListenerIdentifier, STAR_DRIVER_ID driverIdentifier,
 STAR_DEVICE_ID deviceIdentifier, STAR_CHANNEL_ID channelIdentifier,
 int channelOpened, U8 channelNumber, void *pContextInfo)
{
 /* Unreferenced parameters */
 UNREFERENCED_PARAMETER(pContextInfo);
 UNREFERENCED_PARAMETER(deviceIdentifier);
 UNREFERENCED_PARAMETER(driverIdentifier);
 UNREFERENCED_PARAMETER(channelListenerIdentifier);
 UNREFERENCED_PARAMETER(channelIdentifier);
}

```

```

/* If channel was opened */
if(channelOpened)
{
 /* Print channel opened message */
 printf("Opened channel %u.\n", channelNumber);
}
else
{
 /* Print channel closed message */
 printf("Closed channel %u.\n", channelNumber);
}
}

void channelCallbackExample()
{
 /* Define first out channel */
 STAR_CHANNEL_ID outChannel1;

 /* Define second out channel */
 STAR_CHANNEL_ID outChannel2;

 /* Define first in channel */
 STAR_CHANNEL_ID inChannel1;

 /* Define second in channel */
 STAR_CHANNEL_ID inChannel2;

 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Register channel listener */
 STAR_CHANNEL_LISTENER_ID channelListener =
 STAR_registerChannelListener(
 ChannelCallback, NULL, 0);

 /* If channel listener was registered */
 if(channelListener)
 {
 /* Open first channel with out direction */
 outChannel1 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 0);

 /* Open second channel with out direction */
 outChannel2 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL2, 0);

 /* Open first channel with in direction */
 inChannel1 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 0);

 /* Open second channel with in direction */
 inChannel2 = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 0);

 /* Close first out channel */
 STAR_closeChannel(outChannel1);

 /* Close second out channel */
 STAR_closeChannel(outChannel2);

 /* Close first in channel */
 STAR_closeChannel(inChannel1);

 /* Close second in channel */
 STAR_closeChannel(inChannel2);

 /* Unregister channel listener */
 STAR_unregisterChannelListener(channelListener);
 }
}

```

## 10.16 check\_packet\_example.c

This is an example of how to check that the format of a received RMAP packet is correct, then access the fields in the packet.

```

#include <stdio.h>

#include "rmap_packet_library.h"

```

```

/* Display the type of a packet. */
void display_packet_type(RMAP_PACKET_TYPE type)
{
 switch (type)
 {
 case RMAP_WRITE_COMMAND:
 puts("The packet is an RMAP write command.");
 break;

 case RMAP_WRITE_REPLY:
 puts("The packet is an RMAP write reply.");
 break;

 case RMAP_READ_COMMAND:
 puts("The packet is an RMAP read command.");
 break;

 case RMAP_READ_REPLY:
 puts("The packet is an RMAP read reply.");
 break;

 case RMAP_READ_MODIFY_WRITE_COMMAND:
 puts("The packet is an RMAP read/modify/write command.");
 break;

 case RMAP_READ_MODIFY_WRITE_REPLY:
 puts("The packet is an RMAP read/modify/write reply.");
 break;

 case RMAP_INVALID_PACKET_TYPE:
 puts("The packet is of an invalid RMAP packet type.");
 break;

 default:
 puts("The packet is of an unexpected type.");
 }
}

/* Display the status of a packet. */
void display_packet_status(RMAP_STATUS status)
{
 switch (status)
 {
 case RMAP_SUCCESS:
 puts("The status indicates the command completed successfully.");
 break;

 case RMAP_GENERAL_ERROR:
 puts("The status indicates a general error that does not fit into the other error cases.");
 break;

 case RMAP_UNUSED_PACKET_TYPE_OR_COMMAND_CODE:
 puts("The status indicates the packet type or command is an unexpected value.");
 break;

 case RMAP_INVALID_KEY:
 puts("The status indicates the key did not match that expected by the target user application.");
 break;

 case RMAP_INVALID_DATA_CRC:
 puts("The status indicates an invalid data CRC.");
 break;

 case RMAP_EARLY_EOP:
 puts("The status indicates an EOP was detected before the end of the data.");
 break;

 case RMAP_TOO_MUCH_DATA:
 puts("The status indicates there was more data than was expected.");
 break;

 case RMAP_EEP:
 puts("The status indicates an EEP was encountered in the packet after the header.");
 break;

 case RMAP_VERIFY_BUFFER_OVERRUN:
 puts("The status indicates verify before write was enabled in the command but not enough buffer
space was available to receive the full command.");
 break;

 case RMAP_COMMAND_NOT_IMPLEMENTED_OR_AUTHORISED:
 puts("The status indicates the target user application did not authorise the requested
operation.");
 break;

 case RMAP_RMW_DATA_LENGTH_ERROR:

```

```

 puts("The status indicates the amount of data in a read/modify/write command is invalid.");
 break;

 case RMAP_INVALID_TARGET_LOGICAL_ADDRESS:
 puts("The status indicates the target logical address was not the value expected by the target.");
 break;

 default:
 puts("The status is an unexpected value.");
 }
}

void check_packet_example()
{
 void *pPacket;
 unsigned long packetLen, addressLen, i;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 RMAP_PACKET packetStruct;
 RMAP_STATUS status;
 RMAP_PACKET_TYPE type;
 U8 *pAddress, *pData, *pMask;
 char enabled;
 U8 key, extendedMemoryAddress, crc, maskLen;
 U16 transactionID;
 U32 memoryAddress, dataLen;

 /* Build an RMAP write command packet to write to a 4-byte register */
 pPacket = RMAP_BuildWriteRegisterPacket(pTarget, 2, pReply, 1, 1, 1, 0,
 0x20, 0, 0x106, 0, 0x12345678, &packetLen, &packetStruct, 1);

 /* Check the format of the packet, making sure it's not longer than */
 /* expected. Note that the first byte is skipped as RMAP_CheckPacketValid */
 /* expects the address at the start to be 1 byte long. */
 status = RMAP_CheckPacketValid((U8 *)pPacket + 1, packetLen - 1,
 &packetStruct, 1);
 printf("The packet is %sa valid RMAP packet.\n",
 (status == RMAP_SUCCESS) ? "" : "not ");

 /* Get the type of the packet */
 type = RMAP_GetPacketType(&packetStruct);

 /* Display the type of the packet */
 display_packet_type(type);

 /* Get the target address of the packet */
 pAddress = RMAP_GetTargetAddress(&packetStruct, &addressLen);
 if ((!pAddress) || (addressLen == 0))
 {
 puts("No valid target address is present in the packet.");
 }
 else
 {
 /* Display the target address of the packet */
 printf("The packet has a target address of:");
 for (i = 0; i < addressLen; i++)
 {
 printf(" %x", pAddress[i]);
 }
 puts("");
 }

 /* Determine if verify before write is enabled */
 enabled = RMAP_GetVerifyBeforeWrite(&packetStruct);

 /* Display whether verify before write is enabled */
 printf("Verify before write is %senabled for the packet.\n",
 enabled ? "" : "not ");

 /* Determine if acknowledgement is enabled */
 enabled = RMAP_GetPerformAcknowledgement(&packetStruct);

 /* Display whether acknowledgement is enabled */
 printf("Acknowledgement is %senabled for the packet.\n",
 enabled ? "" : "not ");

 /* Determine if incrementing addresses is enabled */
 enabled = RMAP_GetIncrementAddress(&packetStruct);

 /* Display whether incrementing addresses is enabled */
 printf("Incrementing addresses is %senabled for the packet.\n",
 enabled ? "" : "not ");

 /* Get the key of the packet */
 key = RMAP_GetKey(&packetStruct);

```

```

/* Display the key of the packet */
printf("The value of the key is 0x%x.\n", key);

/* Get the transaction identifier of the packet */
transactionID = RMAP_GetTransactionID(&packetStruct);

/* Display the transaction identifier of the packet */
printf("The value of the transaction identifier is %d.\n", transactionID);

/* Get the reply address of the packet */
pAddress = RMAP_GetReplyAddress(&packetStruct, &addressLen);
if ((!pAddress) || (addressLen == 0))
{
 puts("No valid reply address is present in the packet.");
}
else
{
 /* Display the reply address of the packet */
 printf("The packet has a reply address of:");
 for (i = 0; i < addressLen; i++)
 {
 printf(" %x", pAddress[i]);
 }
 puts("");
}

/* Get the memory address and extended memory address of the packet */
memoryAddress = RMAP_GetAddress(&packetStruct, &extendedMemoryAddress);

/* Display the transaction identifier of the packet */
printf("The value of the memory address is 0x%x ", memoryAddress);
printf("and the extended address is 0x%x.\n", extendedMemoryAddress);

/* Get the data in the packet */
pData = RMAP_GetData(&packetStruct, &dataLen);
if ((!pData) || (dataLen == 0))
{
 puts("No valid data is present in the packet.");
}
else
{
 /* Display the data in the packet */
 printf("The packet has the following data:");
 for (i = 0; i < dataLen; i++)
 {
 printf(" %x", pData[i]);
 }
 puts("");
}

/* Get the status of the packet */
status = RMAP_GetStatus(&packetStruct);

/* Display the status of the packet */
display_packet_status(status);

/* Get the header CRC of the packet */
crc = RMAP_GetHeaderCRC(&packetStruct);

/* Display the header CRC of the packet */
printf("The value of the header CRC is 0x%x.\n", crc);

/* Get the data CRC of the packet */
crc = RMAP_GetDataCRC(&packetStruct);

/* Display the data CRC of the packet */
printf("The value of the data CRC is 0x%x.\n", crc);

/* Get the mask in the packet */
pMask = RMAP_GetMask(&packetStruct, &maskLen);
if ((!pMask) || (maskLen == 0))
{
 puts("No valid mask is present in the packet.");
}
else
{
 /* Display the data in the packet */
 printf("The packet has the following mask:");
 for (i = 0; i < maskLen; i++)
 {
 printf(" %x", pMask[i]);
 }
 puts("");
}

/* Free the RMAP write command packet created above */
RMAP_FreeBuffer(pPacket);

```

```

}
```

## 10.17 crc\_example.c

This is an example of how to use the CRC functions.

```

#include <stdio.h>

#include "rmap_packet_library.h"

#define BUFFER_LEN 32
#define HEADER_LEN 10

void crc_example()
{
 U8 pBuffer[BUFFER_LEN], crc;
 int i;

 /* Fill the data buffer */
 for (i = 0; i < BUFFER_LEN; i++)
 {
 pBuffer[i] = i;
 }

 /* Calculate a CRC for the header part of the buffer */
 crc = RMAP_CalculateCRC(pBuffer, HEADER_LEN);

 printf("The CRC of the first %d bytes of the buffer is 0x%x\n", HEADER_LEN,
 crc);

 /* Calculate a CRC for the entire buffer, */
 /* using the CRC already calculated as a seed */
 crc = RMAP_CalculateCRCWithSeed(pBuffer + HEADER_LEN,
 BUFFER_LEN - HEADER_LEN, crc);

 printf("The CRC of the full buffer is 0x%x\n", crc);

 /* Check the CRC is correct for the buffer */
 printf("The CRC of the buffer is %svalid\n",
 RMAP_IsCRCValid(pBuffer, BUFFER_LEN, crc) ? "" : "not ");
}

```

## 10.18 data\_signalling\_rate\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Data Signalling Rate functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void getSpFiBitRatesExample(STAR_DEVICE_ID deviceID)
{
 U64 * pSpFiBitRates;
 U32 bitRateCount = 0U;

 printf("Get SpaceFibre Bit Rates Example\n");
 printf("-----\n");

 /* Get SpaceFibre lane data signalling rates supported by the device */
 pSpFiBitRates = CFG_SPFI_getSpFiBitRates(deviceID, &bitRateCount);
 if (!pSpFiBitRates)
 {
 /* Print error and exit example */
 printf("Failed to get supported SpaceFibre data signalling rates.\n");
 return;
 }
 else
 {
 U32 spfiBitRatesIndex;

 /* Print success */
 printf("Supported SpaceFibre data signalling rates (Gbit/s):\n");
 }
}

```

```

 /* Loop for each supported SpaceFibre bit rate */
 for (spfiBitRatesIndex = 0U;
 spfiBitRatesIndex < bitRateCount; spfiBitRatesIndex++)
 {
 /* Convert from bit/s to Gbit/s */
 float bitRateGbps = (float) (pSpFiBitRates[spfiBitRatesIndex]
 / 1000000000.0);

 /* Print Gbit/s bit rate */
 printf("%g", bitRateGbps);

 /* If not last bit rate */
 if (spfiBitRatesIndex != (bitRateCount - 1U))
 {
 /* Print delimiter */
 printf(", ");
 }
 }
 printf("\n");

 /* Release SpaceFibre lane data signalling rates */
 CFG_SPFI_destroySpFiBitRates(pSpFiBitRates);
}

printf("\n");
}

void getSpFiBitRateExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get SpaceFibre Bit Rate Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U64 spFiBitRate = 0ULL;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get the SpaceFibre bit rate */
 if (CFG_SPFI_getSpFiBitRate(deviceID, port, &spFiBitRate) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get SpaceFibre bit rate", port);
 return;
 }
 else
 {
 /* Print success */
 printf("SpaceFibre bit rate is %llu bit/s", spFiBitRate);
 printPortMessage("", port);
 }
 }
 }
}

```



```

 }
}

void setSpFiBitRateExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Set SpaceFibre Bit Rate Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U64 spFiBitRate;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Use bit rate of 2.5 Gbit/s */
 spFiBitRate = 2500000000ULL;

 /* Set the SpaceFibre bit rate */
 if (CFG_SPFI_setSpFiBitRate(deviceID, port, spFiBitRate) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to set SpaceFibre bit rate", port);
 return;
 }
 else
 {
 /* Print success */
 printf("SpaceFibre bit rate set to %llu bit/s", spFiBitRate);
 printPortMessage("", port);
 }
 }
 }

 printf("\n");
}

void dataSignallingRateExample(STAR_DEVICE_ID deviceID)
{
 printf("Data Signalling Rate Example\n");
 printf("===== \n");

 /* Get SpaceFibre bit rates example */
 getSpFiBitRatesExample(deviceID);

 /* Set SpaceFibre bit rate example */
 setSpFiBitRateExample(deviceID);

 /* Get SpaceFibre bit rate example */
 getSpFiBitRateExample(deviceID);
}

```

## 10.19 device\_identifier\_example.c

This example demonstrates the use of the device identifier functions.

```
#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID *devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
```

```

STAR_CFG_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo;
STAR_CFG_NETWORK_DISCOVERY_INFO networkInfo;
char manufacturerStr[STAR_CFG_MANUFACTURER_STR_MAX_LEN];
char deviceStr[STAR_CFG_DEVICE_STR_MAX_LEN];
int i;

/* Select device to configure */
deviceID = chooseStarDevice();

/* Get the router Identification Information */
CFG_getDeviceIdentificationInfo(deviceID, &deviceIdentifierInfo);

/* Display the Identification Information*/
printf("\nManufacturer ID:\t%u", deviceIdentifierInfo.manufacturerID);
printf("\nChip ID:\t%u", deviceIdentifierInfo.chipType);
printf("\nVersion:\t%u", deviceIdentifierInfo.versionNum);

/* Display parsed information*/
CFG_getDeviceManufacturerAsString(&deviceIdentifierInfo,
 manufacturerStr);
printf("\nManufacturer Name:\t%s", manufacturerStr);

CFG_getDeviceTypeAsString(&deviceIdentifierInfo, deviceStr);
printf("\nDevice Type:\t%s", deviceStr);

/* Get Device network information */
CFG_getNetworkDiscoveryInfo(deviceID, &networkInfo);

/* Display device network information */
switch(networkInfo.deviceType)
{
case STAR_CFG_DEVICE_TYPE_ROUTER:
 printf("\nDevice Type:\tRouter");
 break;
case STAR_CFG_DEVICE_TYPE_UNKNOWN:
 printf("\nDevice Type:\tUnknown");
 break;
case STAR_CFG_DEVICE_TYPE_INVALID:
 printf("\nDevice Type:\tInvalid");
 break;
}

printf("\nNumber of Ports (Total/Running):\t%u/%u", networkInfo.portCount, networkInfo.
 runningPortsCount);
printf("\nRunning port numbers: ");
/* Test each bit in mask to see if it is set */
for(i = 1; i < 32; i++)
{
 if((1 << i) & networkInfo.runningPortsMask)
 printf(" %d", i);
}

printf("\nReturn Port:\t%u\n", networkInfo.returnPort);

return 0;
}

```

## 10.20 device\_info\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Device Information & Identification functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void deviceInformationExample(STAR_DEVICE_ID deviceID)
{
 SPFI_DEVICE_INFO deviceInfo = 0U;

 printf("Device Information Example\n");
 printf("-----\n");

 /* Get the device information */
 if (CFG_SPFI_getDeviceInformation(deviceID, &deviceInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device information.\n");
 }
}

```

```

 return;
 }
 else
 {
 SPFI_NODE_TYPE nodeType;
 U8 portCount;
 U8 spfiPortCount = 0U;
 U8 axiPortCount = 0U;

 /* Get node type and port count */
 nodeType = CFG_SPFI_getDeviceNodeType(deviceInfo);
 portCount = CFG_SPFI_getDevicePortCount(deviceInfo);

 /* Print device information */
 printf("Node Type : %s\n", nodeTypeToString(nodeType));
 printf("Port Count : %u\n", portCount);

 /* Get SpaceFibre port count */
 if (CFG_SPFI_getDevicePortCountByType(deviceID,
 SPFI_PORT_TYPE_SPACEFIBRE, &spfiPortCount) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get SpaceFibre port count.\n");
 return;
 }
 else
 {
 printf("SpaceFibre Port Count : %u\n", spfiPortCount);
 }

 /* Get AXI port count */
 if (CFG_SPFI_getDevicePortCountByType(deviceID,
 SPFI_PORT_TYPE_AXI, &axiPortCount) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get AXI port count.\n");
 return;
 }
 else
 {
 printf("AXI Port Count : %u\n", axiPortCount);
 }

 printf("\n");
 }
}

void deviceIdentificationExample(STAR_DEVICE_ID deviceID)
{
 SPFI_DEVICE_IDENTIFIER_INFO deviceIdentifierInfo = 0U;

 printf("Device Identification Example\n");
 printf("-----\n");

 /* Get the device identification info */
 if (CFG_SPFI_getDeviceIdentificationInfo(deviceID, &
 deviceIdentifierInfo)
 != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device identification information.\n");
 return;
 }
 else
 {
 char deviceManufacturerString[STAR_CFG_MANUFACTURER_STR_MAX_LEN];
 U8 deviceType;
 char deviceTypeString[STAR_CFG_DEVICE_STR_MAX_LEN];
 U8 majorVersion = 0U;
 U8 minorVersion = 0U;
 U8 patchVersion = 0U;

 /* Get device manufacturer ID and string values */
 U8 deviceManufacturer =
 CFG_SPFI_getDeviceManufacturer(deviceIdentifierInfo);
 if (CFG_SPFI_getDeviceManufacturerAsString(
 deviceIdentifierInfo,
 deviceManufacturerString) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device manufacturer string.\n");
 return;
 }
 else
 {

```

```

 printf("Manufacturer : %s (%u)\n", deviceManufacturerString,
 deviceManufacturer);
 }

 /* Get device type ID and string values */
 deviceType = CFG_SPFI_getDeviceType(deviceIdentifierInfo);
 if (CFG_SPFI_getDeviceTypeAsString(deviceIdentifierInfo,
 deviceTypeString) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device type string.\n");
 return;
 }
 else
 {
 printf("Device Type : %s (%u)\n", deviceTypeString, deviceType);
 }

 /* Get device version numbers */
 if (CFG_SPFI_getDeviceVersion(deviceIdentifierInfo, &majorVersion,
 &minorVersion, &patchVersion) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device version.\n");
 return;
 }
 else
 {
 printf("Device Version : v%u.%02u", majorVersion,
 minorVersion);
 if (patchVersion != 0U)
 {
 printf("p%u", patchVersion);
 }
 printf("\n");
 }
}

printf("\n");
}

void deviceIdentificationInfoExample(STAR_DEVICE_ID deviceID)
{
 printf("Device Information & Identification Example\n");
 printf("===== \n");

 /* Device information example */
 deviceInformationExample(deviceID);

 /* Device identification example */
 deviceIdentificationExample(deviceID);
}

```

## 10.21 driver\_list\_example.c

Gets a list of available drivers using `STAR_getDriverList()` then iterates over them and prints the type of driver.

```

#include "examples.h"

#include "utilities.h"

#include "star-api.h"

void printDriverType(STAR_DRIVER_TYPE driverType)
{
 /* Switch on driver type */
 switch(driverType)
 {
 /* Case Invalid */
 case STAR_DRIVER_INVALID:
 /* Print invalid driver */
 printf("Invalid driver detected.\n");

 break;
 /* Case USB */
 case STAR_DRIVER_TYPE_USB:
 /* Print USB driver type */
 printf("USB driver detected.\n");
 }
}

```

```

 break;
 /* Case PCI */
 case STAR_DRIVER_TYPE_PCI:
 /* Print PCI driver type */
 printf("PCI driver detected.\n");

 break;
 /* Case TCP/IP */
 case STAR_DRIVER_TYPE_TCP/IP:
 /* Print TCP/IP driver type */
 printf("TCP/IP driver detected.\n");

 break;
 /* Case Virtual */
 case STAR_DRIVER_TYPE_VIRTUAL:
 /* Print virtual driver type */
 printf("Virtual driver detected.\n");

 break;
 }
}

void driverListExample()
{
 /* The number of drivers that were found */
 U32 driverCount = 0;

 /* The current driver index */
 unsigned int index;

 /* Get list of drivers */
 STAR_DRIVER_ID * drivers = STAR_getDriverList(1, 1, &driverCount);

 /* For all drivers */
 for(index = 0; (drivers != NULL) && (index < driverCount); index++)
 {
 /* Get driver type */
 STAR_DRIVER_TYPE driverType = STAR_getDriverType(drivers[index]);

 /* Print driver type */
 printDriverType(driverType);
 }

 /* Destroy device list */
 STAR_destroyDeviceList(drivers);
}

```

## 10.22 ex\_receive\_example.c

Opens a channel in ex mode and receives a single data chunk using an ex receive operation.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"
#include <stdlib.h>

#define EX_BUFFER_SIZE (16384)

void exReceiveExample()
{
 STAR_DEVICE_ID deviceId;
 STAR_CHANNEL_ID channelId;
 STAR_TRANSFER_OPERATION *pExRxOp;
 STAR_TRANSFER_STATUS status;
 U8 *pBuffer;

 /* Get selected device */
 deviceId = promptForDevice(TRANSMIT_TYPE_RECEIVE);
 if (!deviceId)
 {
 printf("Error: no device selected\n");
 return;
 }

 /* Open the channel in ex mode */
 channelId = STAR_openChannelToLocalDeviceEx(deviceId,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 1);
}

```

```

if (!channelId)
{
 printf("Error: failed to open channel\n");
 return;
}

/* Allocate the buffer */
pBuffer = calloc(1, EX_BUFFER_SIZE);
if (!pBuffer)
{
 printf("Error: failed to allocate buffer\n");
 STAR_closeChannel(channelId);
 return;
}

/* Create ex receive operation */
pExRxOp = STAR_createRxOperationEx(&pBuffer, 1, EX_BUFFER_SIZE,
 STAR_RECEIVE_CHUNKS);
if (!pExRxOp)
{
 printf("Error: failed to create ex operation\n");
 STAR_closeChannel(channelId);
 free(pBuffer);
 return;
}

/* Submit the ex receive operation */
STAR_submitTransferOperation(channelId, pExRxOp);

/* Wait for an item to be received */
printf("\nWaiting on item to be received...\n\n");
status = STAR_waitOnTransferOperationCompletion(pExRxOp, -1);

/* Check if the ex receive operation completed */
if (status == STAR_TRANSFER_STATUS_COMPLETE)
{
 STAR_STREAM_ITEM_TYPE itemType;

 /* Get the number of items received */
 int numItems = STAR_rxOperationExGetNumItems(pExRxOp);
 if (numItems != -1)
 {
 printf("Number of items received: %d\n\n", numItems);
 }

 /* Get the first item type */
 itemType = STAR_rxOperationExGetItemType(pExRxOp, 0);

 /* Check and print the item details */
 if (itemType == STAR_STREAM_ITEM_TYPE_DATA_CHUNK)
 {
 /* Get the chunk */
 STAR_DATA_CHUNK chunk;
 if (STAR_rxOperationExGetDataChunk(pExRxOp, 0, &chunk) != -1)
 {
 /* Print the chunk details */
 printf("Chunk:\n");
 printf("Start of packet = %s\n", chunk.isStart ? "Yes" : "No");
 printf("Data length = %u\n", chunk.dataLength);
 printf("End of packet = ");
 if (chunk.eop == STAR_EOP_TYPE_EOP)
 {
 printf("EOP\n");
 }
 else if (chunk.eop == STAR_EOP_TYPE_EEP)
 {
 printf("EEP\n");
 }
 else
 {
 printf("No\n");
 }
 }
 else
 {
 printf("Error: failed to get chunk from ex operation\n");
 }
 }
 else if (itemType == STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT)
 {
 /* Get the link state event */
 STAR_LINK_STATE_EVENT linkStateEvent;
 if (STAR_rxOperationExGetLinkStateEvent(pExRxOp, 0,
 &linkStateEvent) != -1)
 {
 /* Print the link state event details */
 printf("Link state event:\n");

```

```

 printf("Port = %u\n", linkStateEvent.port);
 printf("Count = %u\n", linkStateEvent.count);
 printf("Link running = %u\n",
 linkStateEvent.linkRunning);
 printf("Disconnect error = %u\n",
 linkStateEvent.disconnectError);
 printf("Escape error = %u\n",
 linkStateEvent.escapeError);
 printf("Parity error = %u\n",
 linkStateEvent.parityError);
 printf("Receive credit error = %u\n",
 linkStateEvent.receiveCreditError);
 printf("Transmit credit error = %u\n",
 linkStateEvent.transmitCreditError);
 }
 else
 {
 printf("Error: failed to get link state event from ex operation\n");
 }
}
else if (itemType == STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT)
{
 /* Get the link speed event */
 STAR_LINK_SPEED_EVENT linkSpeedEvent;
 if (STAR_rxOperationExGetLinkSpeedEvent(pExRxOp, 0,
 &linkSpeedEvent) != -1)
 {
 /* Print the link speed event details */
 printf("Link speed event:\n");
 printf("Port = %u\n", linkSpeedEvent.port);
 printf("Speed = %u\n", linkSpeedEvent.linkSpeed);
 }
 else
 {
 printf("Error: failed to get link speed event from ex operation\n");
 }
}
else if (itemType == STAR_STREAM_ITEM_TYPE_TIMECODE)
{
 /* Get the time-code */
 STAR_TIMECODE timeCode;
 if (STAR_rxOperationExGetTimeCode(pExRxOp, 0, &timeCode) != -1)
 {
 /* Print the time-code details */
 printf("Time-code:\n");
 printf("Number since last tx = %u\n", timeCode.numSinceLastTx);
 printf("Value = 0x%02x\n", timeCode.value);
 }
 else
 {
 printf("Error: failed to get time-code from ex operation\n");
 }
}
}

/* Close channel after transmitting packet */
STAR_closeChannel(channelId);

/* Dispose transfer operation */
STAR_disposeTransferOperation(pExRxOp);

/* Free the buffer */
free(pBuffer);
}

```

## 10.23 firmware\_version\_example.c

Gets a list of available devices using `STAR_getDeviceList()` and then iterates over them. The firmware version for each device is retrieved and printed.

```

#include "examples.h"

#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void firmwareVersionExample()
{
 /* Initialise device count to 0 */

```



```

U32 deviceCount = 0;

/* Counter for loop */
unsigned int index;

/* Get device list */
STAR_DEVICE_ID *devices = STAR_getDeviceList(&deviceCount);

/* For all devices */
for(index = 0; (devices != NULL) && (index < deviceCount); index++)
{
 /* Get firmware version */
 STAR_VERSION_INFO * firmwareVersion =
 STAR_getDeviceFirmwareVersion(
 devices[index]);

 /* Print firmware version label */
 printf("Firmware version:");

 /* Print firmware version */
 printVersionInfo(firmwareVersion);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 10.24 lane\_info\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Lane Information functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void laneControlExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;
 U8 lane;

 printf("Lane Control Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Use lane 0 */
 lane = 0U;

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;
 SPFI_LANE_CONTROL_SETTINGS laneControlSettings = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 laneStandbyReason = 0U;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }
 }
 }
}

```

```

}

/* Set lane standby reason */
if (CFG_SPFI_setLaneStandbyReason(deviceID, port, lane,
 laneStandbyReason) != CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printLaneMessage("Failed to set lane standby reason", port,
 lane);
 return;
}
else
{
 printf("Lane standby reason set to %u", laneStandbyReason);
 printLaneMessage("", port, lane);
}

/* Disable lane error recovery */
if (CFG_SPFI_disableLaneErrorRecovery(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to disable lane error recovery.\n");
 return;
}
else
{
 printLaneMessage("Lane error recovery disabled", port, lane);
}

/* Enable lane error recovery */
if (CFG_SPFI_enableLaneErrorRecovery(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to enable lane error recovery.\n");
 return;
}
else
{
 printLaneMessage("Lane error recovery enabled", port, lane);
}

/* Enable lane reset */
if (CFG_SPFI_enableLaneReset(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to enable lane reset.\n");
 return;
}
else
{
 printLaneMessage("Lane reset enabled", port, lane);
}

/* Disable lane reset */
if (CFG_SPFI_disableLaneReset(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to disable lane reset.\n");
 return;
}
else
{
 printLaneMessage("Lane reset disabled", port, lane);
}

/* Disable lane receive */
if (CFG_SPFI_disableLaneRx(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to disable lane receive.\n");
 return;
}
else
{
 printLaneMessage("Lane receive disabled", port, lane);
}

/* Enable lane receive */
if (CFG_SPFI_enableLaneRx(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */

```

```

 printf("Failed to enable lane receive.\n");
 return;
 }
 else
 {
 printLaneMessage("Lane receive enabled", port, lane);
 }

 /* Disable lane transmit */
 if (CFG_SPFI_disableLaneTx(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable lane transmit.\n");
 return;
 }
 else
 {
 printLaneMessage("Lane transmit disabled", port, lane);
 }

 /* Enable lane transmit */
 if (CFG_SPFI_enableLaneTx(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable lane transmit.\n");
 return;
 }
 else
 {
 printLaneMessage("Lane transmit enabled", port, lane);
 }

 printf("\n");

 /* Get lane control settings */
 if (CFG_SPFI_getLaneControlSettings(deviceID, port, lane,
 &laneControlSettings) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printLaneMessage("Failed to get lane control settings", port,
 lane);
 return;
 }
 else
 {
 /* Get lane error recovery enabled flag */
 U8 laneErrorRecovery = CFG_SPFI_isLaneErrorRecoveryEnabled
(
 laneControlSettings);

 /* Get lane reset enabled flag */
 U8 laneReset = CFG_SPFI_isLaneResetEnabled(laneControlSettings
);

 /* Get lane receive enabled flag */
 U8 laneRx = CFG_SPFI_isLaneRxEnabled(laneControlSettings);

 /* Get lane transmit enabled flag */
 U8 laneTx = CFG_SPFI_isLaneTxEnabled(laneControlSettings);

 /* Get lane standby reason */
 laneStandbyReason = CFG_SPFI_getLaneStandbyReason(
 laneControlSettings);

 /* Print the SpaceFibre link control settings */
 printf("SpaceFibre port %u link control settings:\n",
 port);
 printf(" Lane Standby Reason : %u\n", laneStandbyReason);
 printf(" Lane Error Recovery : %s\n",
 laneErrorRecovery ? "enabled" : "disabled");
 printf(" Lane Reset : %s\n",
 laneReset ? "enabled" : "disabled");
 printf(" Lane Rx : %s\n",
 laneRx ? "enabled" : "disabled");
 printf(" Lane Tx : %s\n",
 laneTx ? "enabled" : "disabled");
 }

 printf("\n");
}

}

void laneStatusExample(STAR_DEVICE_ID deviceID)
{

```

```

U8 portCount;
U8 port;
U8 lane;

printf("Lane Status Example\n");
printf("-----\n");

/* Get the number of ports */
portCount = getPortCount(deviceID);
if (!portCount)
{
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
}

/* Use lane 0 */
lane = 0U;

/* Loop through the ports */
for (port = 1U; port < portCount; port++)
{
 SPFI_PORT_INFO portInfo = 0U;
 SPFI_LANE_STATUS laneStatus = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get lane status */
 if (CFG_SPFI_getLaneStatus(deviceID, port, lane, &laneStatus) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printLaneMessage("Failed to get lane status", port, lane);
 return;
 }
 else
 {
 /* Get lane far-end INIT3 flags */
 U8 laneFarEndINIT3Flags = CFG_SPFI_getLaneFarEndINIT3Flags
(
 laneStatus);

 /* Get lane lost receive signal reason */
 U8 laneLOSReasonRx = CFG_SPFI_getLaneLOSReasonRx(laneStatus);

 /* Get lane receive standby reason */
 U8 laneStandbyReasonRx = CFG_SPFI_getLaneStandbyReasonRx(
 laneStatus);

 /* Get lane state */
 SPFI_LANE_STATE laneState = CFG_SPFI_getLaneState(
 laneStatus);

 /* Get lane active flag */
 U8 laneActive = CFG_SPFI_isLaneActive(laneStatus);

 /* Get lane error detected flag */
 U8 laneError = CFG_SPFI_isLaneErrorDetected(laneStatus);

 /* Get lane event detected flag */
 U8 laneEvent = CFG_SPFI_isLaneEventDetected(laneStatus);

 /* Get lane no signal detected flag */
 U8 laneNoSignal = CFG_SPFI_isLaneNoSignalDetected(
 laneStatus);

 /* Get lane receive only flag */
 U8 laneRxOnly = CFG_SPFI_isLaneRxOnly(laneStatus);

```

```

 /* Get lane started flag */
 U8 laneStarted = CFG_SPFI_isLaneStarted(laneStatus);

 /* Get lane transmit only flag */
 U8 laneTxOnly = CFG_SPFI_isLaneTxOnly(laneStatus);

 /* Print the SpaceFibre lane status */
 printf("SpaceFibre port %u lane %u status:\n", port, lane);
 printf(" Lane Far-end INIT3 Flags : %u\n",
 laneFarEndINIT3Flags);
 printf(" Lane Lost Receive Signal Reason : %u\n",
 laneLOSReasonRx);
 printf(" Lane Receive Standby Reason : %u\n",
 laneStandbyReasonRx);
 printf(" Lane State : %s\n",
 laneStateToString(laneState));
 printf(" Lane Active : %s\n",
 laneActive ? "yes" : "no");
 printf(" Lane Error : %s\n",
 laneError ? "yes" : "no");
 printf(" Lane Event : %s\n",
 laneEvent ? "yes" : "no");
 printf(" Lane No Signal : %s\n",
 laneNoSignal ? "yes" : "no");
 printf(" Lane Receive Only : %s\n",
 laneRxOnly ? "yes" : "no");
 printf(" Lane Started : %s\n",
 laneStarted ? "yes" : "no");
 printf(" Lane Transmit Only : %s\n",
 laneTxOnly ? "yes" : "no");
}

}

printf("\n");
}

}

void laneEventsExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;
 U8 lane;

 printf("Lane Events Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Use lane 0 */
 lane = 0U;

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 SPFI_LANE_EVENTS laneEvents = 0U;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get lane events */
 if (CFG_SPFI_getLaneEvents(deviceID, port, lane, &laneEvents) !=

```

```

 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printLaneMessage("Failed to get lane events", port, lane);
 return;
 }
 else
 {
 /* Get lane disconnect count */
 U8 laneDisconnectCount = CFG_SPFI_getLaneDisconnectCount(
 laneEvents);

 /* Get lane receive error count */
 U16 laneRxErrorCount = CFG_SPFI_getLaneRxErrorCount(
 laneEvents);

 /* Get lane far-end error burst detected flag */
 U8 laneFarEndErrorBurst =
 CFG_SPFI_isLaneFarEndErrorBurstDetected(
 laneEvents);

 /* Get lane far-end INIT1 receive in active detected flag */
 U8 laneFarEndINIT1RxInActive =
 CFG_SPFI_isLaneFarEndINIT1RxInActiveDetected(
 laneEvents);

 /* Get lane far-end lost signal detected flag */
 U8 laneFarEndLOS = CFG_SPFI_isLaneFarEndLOSDetected(
 laneEvents);

 /* Get lane far-end standby detected flag */
 U8 laneFarEndStandby = CFG_SPFI_isLaneFarEndStandbyDetected(
 laneEvents);

 /* Get lane INIT1 receive detected flag */
 U8 laneINIT1Rx = CFG_SPFI_isLaneINIT1RxDetected(laneEvents)
;

 /* Get lane INIT2 receive detected flag */
 U8 laneINIT2Rx = CFG_SPFI_isLaneINIT2RxDetected(laneEvents)
;

 /* Get lane init timeout detected flag */
 U8 laneInitTimeout = CFG_SPFI_isLaneInitTimeoutDetected(
 laneEvents);

 /* Print the SpaceFibre lane status */
 printf("SpaceFibre port %u lane %u status:\n", port, lane);
 printf(" Lane Disconnect Count : %u\n",
 laneDisconnectCount);
 printf(" Lane Receive Error Count : %u\n",
 laneRxErrorCount);
 printf(" Lane Far-end Error Burst : %s\n",
 laneFarEndErrorBurst ? "yes" : "no");
 printf(" Lane Far-end INIT1 Receive In Active : %s\n",
 laneFarEndINIT1RxInActive ? "yes" : "no");
 printf(" Lane Far-end Lost Signal : %s\n",
 laneFarEndLOS ? "yes" : "no");
 printf(" Lane Far-end Standby : %s\n",
 laneFarEndStandby ? "yes" : "no");
 printf(" Lane INIT1 Receive : %s\n",
 laneINIT1Rx ? "yes" : "no");
 printf(" Lane INIT2 Receive : %s\n",
 laneINIT2Rx ? "yes" : "no");
 printf(" Lane Init Timeout : %s\n",
 laneInitTimeout ? "yes" : "no");

 printf("\n");

 /* Clear lane events */
 if (CFG_SPFI_clearLaneEvents(deviceID, port, lane) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printLaneMessage("Failed to clear lane errors", port,
 lane);
 return;
 }
 else
 {
 printLaneMessage("Lane errors cleared", port, lane);
 }
 }
}

```

```

 printf("\n");
 }
}

void laneInfoExample(STAR_DEVICE_ID deviceID)
{
 printf("Lane Information Example\n");
 printf("=====\n");

 /* Lane control example */
 laneControlExample(deviceID);

 /* Lane status example */
 laneStatusExample(deviceID);

 /* Lane events example */
 laneEventsExample(deviceID);
}

```

## 10.25 link\_events.c

This file contains example code demonstrating how to receive link speed and state change events.

```

#include <stdio.h>

#include "star-api.h"
#include "cfg_api_mk2.h"
#include "cfg_api_brick_mk2.h"
#include "cfg_api_router.h"
#include "cfg_api_brick_mk3.h"
#include "cfg_api_pxi.h"
#include "cfg_api_pci_mk2.h"
#include "cfg_api_generic.h"
#include "ui.h"

//-----

#define TEST_ERROR_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 HandleError(sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_FUNC_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 printf("%s", sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_SUB_FUNC_STATUS(Status,sText,sError) \
if (Status == 0) \
{ \
 printf("%s", sError); \
 return (0); \
} \
else \
{ \
 printf("%s", sText); \
}

#define TEST_GENERIC_ERROR_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 HandleError(sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
}

```

```

}

#define TEST_GENERIC_FUNC_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 printf("%s", sError); \
 return (1); \
} \
else \
{ \
 printf("%s", sText); \
} \

#define TEST_GENERIC_SUB_FUNC_STATUS(Status,sText,sError) \
if (!CFG_SUCCESS(Status)) \
{ \
 printf("%s", sError); \
 return (0); \
} \
else \
{ \
 printf("%s", sText); \
} \

//-----

#ifdef _WIN32
#include <windows.h>
#elif (defined(__linux__) || defined(__CYGWIN__) || defined(__QNX__) || defined(__rtems__) || \
 defined(__vxworks))
#include <pthread.h>
#include <unistd.h>
#endif

//-----

void DelayMS(U32 _DelayMS)
{
#ifdef _WIN32
 Sleep(_DelayMS);
#elif (defined(__linux__) || defined(__CYGWIN__) || defined(__QNX__) || defined(__rtems__))
 usleep(_DelayMS * 1000);
#elif defined(__vxworks)
 taskDelay(_DelayMS);
#endif
}

//-----

void HandleError(const char* _psText)
{
 printf("%s", _psText);
 printf("\nPress Enter to exit...\n");
 getchar();
}

//-----

int SetLinkSpeed(STAR_DEVICE_ID _DeviceID, U8 _Port, U8 _Divisor)
{
 double SignallingRateSet;

 int Status = CFG_setTxSignallingRate(_DeviceID, _Port, 200.0 / _Divisor,
 &SignallingRateSet);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");
 return (Status);
}

//-----

int PrintLinkSpeed(STAR_DEVICE_ID _DeviceID, U8 _TxPort, U8 _RxPort, const char* _psText)
{
 double SignallingRate;
 int Status = CFG_getRxSignallingRate(_DeviceID, _RxPort, &SignallingRate);
 if (Status >= 0)
 {
 printf(_psText, _TxPort, _RxPort, _RxPort, SignallingRate);
 }
 return (Status);
}

//-----

STAR_DEVICE_ID GetRemoteDeviceID(STAR_DEVICE_ID _DeviceID,
 U8 _RemoteChannel)
{
 unsigned char aPath[] = { 0,254 };

```



```

unsigned char aRetPath[] = { 254 };

STAR_SPACEWIRE_ADDRESS* pPath = STAR_createAddress(aPath,
sizeof(aPath));
STAR_SPACEWIRE_ADDRESS* pRetPath = STAR_createAddress(aRetPath,
sizeof(aRetPath));

STAR_DEVICE_ID RemoteDeviceID =
 STAR_CFG_createRemoteDeviceIdentifier(
 _DeviceID, _RemoteChannel, NULL, pPath, pRetPath);

STAR_destroyAddress(pRetPath);
STAR_destroyAddress(pPath);

return (RemoteDeviceID);
}

//-----

int EnableLinkEvents(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 int Status = CFG_enableSpeedChangeEventsOnPort(_DeviceID,_Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status,"","");
 Status = CFG_enableStateChangeEventsOnPort(_DeviceID,_Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status,"","");
 return (Status);
}

//-----

int DisableLinkEvents(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 int Status = CFG_disableSpeedChangeEventsOnPort(_DeviceID,_Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status,"","");
 Status = CFG_disableStateChangeEventsOnPort(_DeviceID,_Port);
 TEST_GENERIC_SUB_FUNC_STATUS(Status,"","");
 return (Status);
}

//-----

int WaitEvents(STAR_CHANNEL_ID _ChannelID,STAR_TRANSFER_OPERATION*
 _pLinkEventRxOperation)
{
 int Status = 0;
 U32 Index;

 STAR_TRANSFER_STATUS TransferOperationStatus;
 do
 {
 TransferOperationStatus = STAR_waitOnTransferOperationCompletion
 (_pLinkEventRxOperation,2000);
 switch (TransferOperationStatus)
 {
 case STAR_TRANSFER_STATUS_COMPLETE:
 {
 U32 ItemCount = STAR_getTransferItemCount(
 _pLinkEventRxOperation);
 for (Index=0;Index<ItemCount;Index++)
 {
 STAR_STREAM_ITEM* pLinkEvent =
 STAR_getTransferItem(_pLinkEventRxOperation, Index);
 STAR_STREAM_ITEM_TYPE EventType = (
 STAR_STREAM_ITEM_TYPE)pLinkEvent->itemType;
 if (EventType == STAR_STREAM_ITEM_TYPE_LINK_STATE_EVENT)
 {
 STAR_LINK_STATE_EVENT* pLinkStateEvent = (
 STAR_LINK_STATE_EVENT*)pLinkEvent->item;
 U8 Port = STAR_getLinkStateEventPort(pLinkStateEvent);

 int bIsLinkRunning = STAR_isLinkStateEventLinkRunning
 (pLinkStateEvent);
 if (bIsLinkRunning == FALSE)
 {
 printf("\nPort %i state event: Link not running.", Port);
 }
 else
 {
 printf("\nPort %i state event: Link running.", Port);
 }
 }
 else if (EventType == STAR_STREAM_ITEM_TYPE_LINK_SPEED_EVENT)
 {
 STAR_LINK_SPEED_EVENT* pLinkSpeedEvent = (
 STAR_LINK_SPEED_EVENT*)pLinkEvent->item;

```

```

 U8 Port = STAR_getLinkSpeedEventPort(pLinkSpeedEvent);
 U32 NewLinkSpeed = STAR_getLinkSpeedEventSpeed(
pLinkSpeedEvent);
 printf("\nPort %i speed event: Link speed %u.%u Mbit/s", Port, NewLinkSpeed / 1000
000, (NewLinkSpeed % 1000000) / 100000);
 }
 break;
}
case STAR_TRANSFER_STATUS_STARTED:
{
 printf("\nNo more events\n");
 Status = STAR_cancelTransferOperation(_pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "");
 break;
}
case STAR_TRANSFER_STATUS_NOT_STARTED:
{
 printf("\nXfer not started and link speed event did not occur");
 Status = STAR_cancelTransferOperation(_pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "");
 break;
}
case STAR_TRANSFER_STATUS_CANCELLED:
{
 //printf("\nCancelled");
 break;
}
case STAR_TRANSFER_STATUS_ERROR:
default:
{
 printf("\nError occurred while waiting for link speed event");
 break;
}
}

if (TransferOperationStatus != STAR_TRANSFER_STATUS_STARTED)
{
 Status = STAR_submitTransferOperation(_ChannelID,
_pLinkEventRxOperation);
 TEST_SUB_FUNC_STATUS(Status, "", "\nSTAR_submitTransferOperation Status = 0.");
}
while (TransferOperationStatus != STAR_TRANSFER_STATUS_STARTED);

return (Status);
}

//-----

void PrintDeviceType(STAR_DEVICE_ID _DeviceID)
{
 char* psDeviceType = STAR_getDeviceTypeAsString(_DeviceID);
 if (psDeviceType == NULL)
 {
 puts("Testing an unknown device type");
 }
 else
 {
 printf("\nTesting the %s\n", psDeviceType);
 STAR_destroyString(psDeviceType);
 }
}

//-----

int DisableTriState(STAR_DEVICE_ID _DeviceID, U8 _Port)
{
 PORT_STATUS_CONTROL PortStatusControl;
 STAR_CFG_SPW_LINK_STATUS LinkStatus;

 int Status = CFG_getPortStatusControl(_DeviceID, _Port, &PortStatusControl);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 Status = CFG_getSpaceWireLinkStatus(PortStatusControl, &LinkStatus);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 LinkStatus.triState = FALSE;
 Status = CFG_setSpaceWireLinkStatus(_DeviceID, _Port, &LinkStatus);
 TEST_GENERIC_SUB_FUNC_STATUS(Status, "", "");

 return (Status);
}

//-----

int bAreLinkEventsOnChannel0Only(U32 _DeviceType)

```

```

{
 if ((_DeviceType == STAR_DEVICE_BRICK_MK2) ||
 (_DeviceType == STAR_DEVICE_BRICK_MK3) ||
 (_DeviceType == STAR_DEVICE_ROUTER_MK2S) ||
 (_DeviceType == STAR_DEVICE_SPLT) ||
 (_DeviceType == STAR_DEVICE_STAR_FIRE_MK3))
 {
 return (TRUE);
 }
 return (FALSE);
}

//-----

#define NUM_DEVICE_TYPES (16)

int __cdecl main()
{
 STAR_DEVICE_ID DeviceID;
 STAR_DEVICE_ID RemoteDeviceID;
 STAR_CHANNEL_ID ChannelID;
 STAR_CFG_FPGA_INFO FPGAInfo;
 STAR_TRANSFER_OPERATION* pLinkEventRxOperation;
 STAR_CHANNEL_MASK ChannelMask;
 STAR_CHANNEL_MASK UsableChannelsMask;

 U32 DeviceType;
 int Status;
 U8 TxPort;
 U8 RxPort;
 U8 Channel;
 U8 RemoteChannel;
 U32 Index;
 char sBuffer[96];

 char sVersion[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char sBuildDate[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];

 STAR_DEVICE_TYPE aDeviceTypes[NUM_DEVICE_TYPES];
 aDeviceTypes[0] = STAR_DEVICE_BRICK_MK2;
 aDeviceTypes[1] = STAR_DEVICE_ROUTER_MK2S;
 aDeviceTypes[2] = STAR_DEVICE_BRICK_MK3;
 aDeviceTypes[3] = STAR_DEVICE_BRICK_MK4;
 aDeviceTypes[4] = STAR_DEVICE_PCIE;
 aDeviceTypes[5] = STAR_DEVICE_PCIE_MK2;
 aDeviceTypes[6] = STAR_DEVICE_PXI_INTERFACE;
 aDeviceTypes[7] = STAR_DEVICE_PXI_RMAP;
 aDeviceTypes[8] = STAR_DEVICE_PXI_ROUTER_12;
 aDeviceTypes[9] = STAR_DEVICE_SPLT;
 aDeviceTypes[10] = STAR_DEVICE_PCI_MK2;
 aDeviceTypes[11] = STAR_DEVICE_CPCI_MK2;
 aDeviceTypes[12] = STAR_DEVICE_STAR_FIRE_MK3;
 aDeviceTypes[13] = STAR_DEVICE_PXI_INTERFACE_MK2;
 aDeviceTypes[14] = STAR_DEVICE_PXI_RMAP_MK2;
 aDeviceTypes[15] = STAR_DEVICE_PXI_ROUTER_MK2;

 STAR_setApplicationName("STAR-System Link Events Example Application");

 // Select device to configure
 DeviceID = STAR_UI_chooseDevice(aDeviceTypes, NUM_DEVICE_TYPES);
 TEST_ERROR_STATUS(DeviceID, "", "\nChosen device has invalid ID");

 DeviceType = STAR_getDeviceType(DeviceID);
 TEST_ERROR_STATUS(DeviceType, "", "\nCan't get device type");

 PrintDeviceType(DeviceID);

 // Get hardware info
 Status = CFG_getFPGAInfo(DeviceID, &FPGAInfo);
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't get device FPGA Info");
 CFG_FPGAInfoToString(DeviceID, &FPGAInfo, sVersion, sBuildDate);

 // Display the hardware info
 printf("\nVersion: %s", sVersion);
 printf("\nBuildDate: %s\n", sBuildDate);

 if (DeviceType == STAR_DEVICE_PCIE)
 {
 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.minor >= 11))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
 }
 else if ((DeviceType == STAR_DEVICE_PCI_MK2) ||
 (DeviceType == STAR_DEVICE_CPCI_MK2))
 {

```

```

 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.
minor >= 10))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
 }
else if ((DeviceType == STAR_DEVICE_PXI_INTERFACE) ||
(DeviceType == STAR_DEVICE_PXI_RMAP) ||
(DeviceType == STAR_DEVICE_PXI_ROUTER_12))
{
 if (!((FPGAInfo.major > 1) || ((FPGAInfo.major == 1) && (FPGAInfo.
minor >= 1))))
 {
 TEST_ERROR_STATUS(0, "", "\nPlease upgrade this device's firmware\n");
 }
}

//----
Status = CFG_disableInterfaceMode(DeviceID);
TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't disable interface mode");
DelayMS(3000); // Waiting for Config Service to close
 channel 0

ChannelMask = STAR_getDeviceChannels(DeviceID);
UsableChannelsMask = (ChannelMask >> 1); // Step over channel 0

RemoteChannel = 0;

/* To receive link speed events on some devices, we must open a connection
to device channel 0. This will prevent local configuration operations
from being performed until we close this channel again, so we create a
remote device for configuration operations, using a different channel, if
necessary. */
if (bAreLinkEventsOnChannel0Only(DeviceType) == TRUE)
{
 Channel = 0;
 RemoteChannel = 1;

 RemoteDeviceID = GetRemoteDeviceID(DeviceID, RemoteChannel);
 TEST_ERROR_STATUS(RemoteDeviceID, "", "\nCan't get remote device ID");
}
else
{
 RemoteDeviceID = DeviceID;
}

Index = 1;
while (UsableChannelsMask != 0)
{
 UsableChannelsMask = (UsableChannelsMask >> 1);

 if ((Index % 2) == 1)
 {
 TxPort = Index;
 RxPort = (Index + 1);
 }
 else
 {
 TxPort = Index;
 RxPort = (Index - 1);
 }
 Index++;

 if (bAreLinkEventsOnChannel0Only(DeviceType) == FALSE)
 {
 Channel = RxPort;
 }

 printf("\n\nTx port %d, Rx port %d, Channel %d\n", TxPort, RxPort, Channel);

 if ((DeviceType == STAR_DEVICE_PXI_INTERFACE) ||
 (DeviceType == STAR_DEVICE_PXI_RMAP) ||
 (DeviceType == STAR_DEVICE_PXI_ROUTER_12) ||
 (DeviceType == STAR_DEVICE_PXI_INTERFACE_MK2) ||
 (DeviceType == STAR_DEVICE_PXI_RMAP_MK2) ||
 (DeviceType == STAR_DEVICE_PXI_ROUTER_MK2))
 {
 DisableTriState(RemoteDeviceID, TxPort);
 DisableTriState(RemoteDeviceID, RxPort);
 }

 Status = EnableLinkEvents(RemoteDeviceID, RxPort); // Enable Link Speed and Link State Events
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't enable link events");

 ChannelID = STAR_openChannelToLocalDevice(DeviceID,
STAR_CHANNEL_DIRECTION_IN, Channel, TRUE);
 TEST_ERROR_STATUS(ChannelID, "", "\nCan't open channel to local device");
}

```

```

 pLinkEventRxOperation = STAR_createRxOperation(1, (
STAR_RECEIVE_MASK) (STAR_RECEIVE_LINK_STATE_EVENTS |
STAR_RECEIVE_LINK_SPEED_EVENTS));
 TEST_ERROR_STATUS(pLinkEventRxOperation, "", "\nSTAR_createRxOperation returned NULL pointer");

 Status = STAR_submitTransferOperation(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nCan't submit transfer operation");

 Status = CFG_startLink(RemoteDeviceID, TxPort); // Remote in to start link
 sprintf(sBuffer, "\nStart link from port %i to port %i", TxPort, RxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't start link");

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = SetLinkSpeed(RemoteDeviceID, TxPort, 2);
 sprintf(sBuffer, "\nSet port %i tx speed : 100 Mbit/s", TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't set link speed");

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = SetLinkSpeed(RemoteDeviceID, TxPort, 1);
 sprintf(sBuffer, "\nSet port %i tx speed : 200 Mbit/s", TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, sBuffer, "\nCan't set link speed");

 Status = WaitEvents(ChannelID, pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nWaitEvents failed");

 Status = STAR_disposeTransferOperation(pLinkEventRxOperation);
 TEST_ERROR_STATUS(Status, "", "\nCan't dispose transfer operation");

 Status = STAR_closeChannel(ChannelID);
 TEST_ERROR_STATUS(Status, "", "\nCan't close channel");

 Status = DisableLinkEvents(RemoteDeviceID, RxPort); // Disable Link Speed and Link State Events
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't disable link events");

 Status = PrintLinkSpeed(RemoteDeviceID, TxPort, RxPort, "\nLink from port %i to port %i running.
 Link speed measured at port %i : %g Mbit/s");
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't measure link speed");

 Status = CFG_stopLink(RemoteDeviceID, TxPort);
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't stop link");
 DelayMS(1000); // Wait for link to stop

 Status = PrintLinkSpeed(RemoteDeviceID, TxPort, RxPort, "\nLink from port %i to port %i stopped.
 Link speed measured at port %i : %g Mbit/s");
 TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't measure link speed");
}

if (RemoteChannel != 0)
{
 Status = STAR_CFG_destroyRemoteDeviceIdentifier(
RemoteDeviceID);
 TEST_ERROR_STATUS(Status, "", "\nCan't destroy remote device ID");
}

Status = CFG_enableInterfaceMode(DeviceID);
TEST_GENERIC_ERROR_STATUS(Status, "", "\nCan't enable interface mode");

printf("\n\nPress Enter to exit...\n");
getchar();
return 0;
}

//-----

```

## 10.26 link\_info\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Link Information functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void linkControlExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

```

```

printf("Link Control Example\n");
printf("-----\n");

/* Get the number of ports */
portCount = getPortCount(deviceID);
if (!portCount)
{
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
}

/* Loop through the ports */
for (port = 1U; port < portCount; port++)
{
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 SPFI_LINK_CONTROL_SETTINGS linkControlSettings = 0U;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get link control settings */
 if (CFG_SPFI_getLinkControlSettings(deviceID, port,
 &linkControlSettings) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get link control settings", port);
 return;
 }
 else
 {
 U8 linkLanesRequestedCount = 1U;
 U8 autoStart;
 U8 linkInterfaceReset;
 U8 linkReset;
 U8 linkStart;
 U8 linkLaneRequested;

 /* Set link lanes requested count */
 if (CFG_SPFI_setLinkLanesRequestedCount(deviceID, port,
 linkLanesRequestedCount) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set link lanes requested count to %u",
 linkLanesRequestedCount);
 printPortMessage("", port);
 return;
 }
 else
 {
 printf("Link lanes requested count set to %u",
 linkLanesRequestedCount);
 printPortMessage("", port);
 }
 }

 /* Disable link autostart */
 if (CFG_SPFI_disableLinkAutoStart(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to disable link autostart", port);
 return;
 }
 else
 {
 printPortMessage("Link autostart disabled", port);
 }

 /* Enable link autostart */
 }
}

```

```
if (CFG_SPFI_enableLinkAutoStart(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to enable link autostart", port);
 return;
}
else
{
 printPortMessage("Link autostart enabled", port);
}

/* Enable link interface reset */
if (CFG_SPFI_enableLinkInterfaceReset(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to enable link interface reset",
 port);
 return;
}
else
{
 printPortMessage("Link interface reset enabled", port);
}

/* Disable link interface reset */
if (CFG_SPFI_disableLinkInterfaceReset(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to disable link interface reset",
 port);
 return;
}
else
{
 printPortMessage("Link interface reset disabled", port);
}

/* Enable link reset */
if (CFG_SPFI_enableLinkReset(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to enable link reset", port);
 return;
}
else
{
 printPortMessage("Link reset enabled", port);
}

/* Disable link reset */
if (CFG_SPFI_disableLinkReset(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to disable link reset", port);
 return;
}
else
{
 printPortMessage("Link reset disabled", port);
}

/* Disable link start */
if (CFG_SPFI_disableLinkStart(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to disable link start", port);
 return;
}
else
{
 printPortMessage("Link start disabled", port);
}

/* Enable link start */
if (CFG_SPFI_enableLinkStart(deviceID, port) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printPortMessage("Failed to enable link start", port);
 return;
}
}
```

```

 else
 {
 printPortMessage("Link start enabled", port);
 }

 printf("\n");

 /* Get link lanes requested count */
 linkLanesRequestedCount = CFG_SPFI_getLinkLanesRequestedCount
(
 linkControlSettings);

 /* Get autostart enabled flag */
 autoStart = CFG_SPFI_isLinkAutoStartEnabled(
 linkControlSettings);

 /* Get link interface reset enabled flag */
 linkInterfaceReset = CFG_SPFI_isLinkInterfaceResetEnabled
(
 linkControlSettings);

 /* Get link reset enabled flag */
 linkReset = CFG_SPFI_isLinkResetEnabled(linkControlSettings);

 /* Get link start enabled flag */
 linkStart = CFG_SPFI_isLinkStartEnabled(linkControlSettings);

 /* Print the SpaceFibre link control settings */
 printf("SpaceFibre port %u link control settings:\n", port);
 printf(" Link Autostart : %s\n",
 autoStart ? "enabled" : "disabled");
 printf(" Link Interface Reset : %s\n",
 linkInterfaceReset ? "enabled" : "disabled");
 printf(" Link Reset : %s\n",
 linkReset ? "enabled" : "disabled");
 printf(" Link Start : %s\n",
 linkStart ? "enabled" : "disabled");
 printf(" Link Lanes Requested Count : %u\n",
 linkLanesRequestedCount);
 }
}

printf("\n");
}

}

void linkStatusExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Link Status Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;
 SPFI_LINK_STATUS linkStatus = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }
 }
 }
}

```



```

 }

 /* Get link status */
 if (CFG_SPFI_getLinkStatus(deviceID, port, &linkStatus) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get link status", port);
 return;
 }
 else
 {
 /* Get link alignment state */
 SPFI_LINK_ALIGNMENT_STATE linkAlignmentState =
 CFG_SPFI_getLinkAlignmentState(linkStatus);

 /* Get link error detected flag */
 U8 linkError = CFG_SPFI_isLinkErrorDetected(linkStatus);

 /* Get link event detected flag */
 U8 linkEvent = CFG_SPFI_isLinkEventDetected(linkStatus);

 /* Get link idle flag */
 U8 linkIdle = CFG_SPFI_isLinkIdle(linkStatus);

 /* Get link lanes event detected flag */
 U8 linkLanesEvent = CFG_SPFI_isLinkLanesEventDetected(
 linkStatus);

 /* Get link protocol error detected flag */
 U8 linkProtocolError = CFG_SPFI_isLinkProtocolErrorDetected
(
 linkStatus);

 /* Get link ready flag */
 U8 linkReady = CFG_SPFI_isLinkReady(linkStatus);

 /* Get link receiving flag */
 U8 linkReceiving = CFG_SPFI_isLinkReceiving(linkStatus);

 /* Get link transmitting flag */
 U8 linkTransmitting = CFG_SPFI_isLinkTransmitting(linkStatus);

 /* Print the SpaceFibre link status */
 printf("SpaceFibre port %u link status:\n", port);
 printf(" Link Alignment State : %s\n",
 linkAlignmentStateToString(linkAlignmentState));
 printf(" Link Error : %s\n",
 linkError ? "yes" : "no");
 printf(" Link Event : %s\n",
 linkEvent ? "yes" : "no");
 printf(" Link Idle : %s\n",
 linkIdle ? "yes" : "no");
 printf(" Link Lanes Event : %s\n",
 linkLanesEvent ? "yes" : "no");
 printf(" Link Protocol Error : %s\n",
 linkProtocolError ? "yes" : "no");
 printf(" Link Ready : %s\n",
 linkReady ? "yes" : "no");
 printf(" Link Receiving : %s\n",
 linkReceiving ? "yes" : "no");
 printf(" Link Transmitting : %s\n",
 linkTransmitting ? "yes" : "no");

 printf("\n");

 /* Clear link protocol error */
 if (CFG_SPFI_clearLinkProtocolError(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to clear link protocol error",
 port);
 return;
 }
 else
 {
 printPortMessage("Link protocol error cleared", port);
 }
 }
}

printf("\n");
}

void linkEventsExample(STAR_DEVICE_ID deviceID)

```

```

{
 U8 portCount;
 U8 port;

 printf("Link Events Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;
 SPFI_LINK_EVENTS linkEvents = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get link events */
 if (CFG_SPFI_getLinkEvents(deviceID, port, &linkEvents) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get link events", port);
 return;
 }
 else
 {
 /* Get link retry count */
 U16 linkRetryCount = CFG_SPFI_getLinkRetryCount(linkEvents);

 /* Get link CRC-16 error detected flag */
 U8 linkCRC16Error = CFG_SPFI_isLinkCRC16ErrorDetected(
 linkEvents);

 /* Get link CRC-8 error detected flag */
 U8 linkCRC8Error = CFG_SPFI_isLinkCRC8ErrorDetected(
 linkEvents);

 /* Get link far-end reset detected flag */
 U8 linkFarEndReset = CFG_SPFI_isLinkFarEndResetDetected(
 linkEvents);

 /* Get link frame error detected flag */
 U8 linkFrameError = CFG_SPFI_isLinkFrameErrorDetected(
 linkEvents);

 /* Print the SpaceFibre link events information */
 printf("SpaceFibre port %u link events:\n", port);
 printf(" Link Retry Count : %u\n", linkRetryCount);
 printf(" Link CRC-16 Error : %s\n",
 linkCRC16Error ? "yes" : "no");
 printf(" Link CRC-8 Error : %s\n",
 linkCRC8Error ? "yes" : "no");
 printf(" Link Far-end Reset : %s\n",
 linkFarEndReset ? "yes" : "no");
 printf(" Link Frame Error : %s\n",
 linkFrameError ? "yes" : "no");

 printf("\n");

 /* Clear link events */

```

```

 if (CFG_SPFI_clearLinkEvents(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to clear link events",
 port);
 return;
 }
 else
 {
 printPortMessage("Link events cleared", port);
 }
 }
}

printf("\n");
}

void linkDebugExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Link Debug Settings Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;
 SPFI_LINK_DEBUG_SETTINGS linkDebugSettings = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get link debug settings */
 if (CFG_SPFI_getLinkDebugSettings(deviceID, port,
 &linkDebugSettings) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get link debug settings", port);
 return;
 }
 else
 {
 U8 linkNearEndLoopback;
 U8 linkRxScrambling;
 U8 linkTxScrambling;

 /* Enable link near-end loopback */
 if (CFG_SPFI_enableLinkNearEndLoopback(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable link near end loopback.\n");
 return;
 }
 else
 {
 printPortMessage("Link near-end loopback enabled", port);
 }
 }
 }
 }
}

```

```

 }

 /* Disable link near-end loopback */
 if (CFG_SPFI_disableLinkNearEndLoopback(deviceID, port)
 !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable link near-end loopback.\n");
 return;
 }
 else
 {
 printPortMessage("Link near-end loopback disabled", port);
 }

 /* Enable link receive scrambling */
 if (CFG_SPFI_enableLinkRxScrambling(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable link receive scrambling.\n");
 return;
 }
 else
 {
 printPortMessage("Link receive scrambling enabled", port);
 }

 /* Disable link receive scrambling */
 if (CFG_SPFI_disableLinkRxScrambling(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable link receive scrambling.\n");
 return;
 }
 else
 {
 printPortMessage("Link receive scrambling disabled", port);
 }

 /* Disable link transmit scrambling */
 if (CFG_SPFI_disableLinkTxScrambling(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable transmit scrambling.\n");
 return;
 }
 else
 {
 printPortMessage("Link transmit scrambling disabled", port);
 }

 /* Disable link transmit scrambling */
 if (CFG_SPFI_enableLinkTxScrambling(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable transmit scrambling.\n");
 return;
 }
 else
 {
 printPortMessage("Link transmit scrambling enabled", port);
 }

 printf("\n");

 /* Get link near-end loopback enabled flag */
 linkNearEndLoopback = CFG_SPFI_isLinkNearEndLoopbackEnabled(
 (
 linkDebugSettings);

 /* Get link receive scrambling enabled flag */
 linkRxScrambling = CFG_SPFI_isLinkRxScramblingEnabled(
 linkDebugSettings);

 /* Get link transmit scrambling enabled flag */
 linkTxScrambling = CFG_SPFI_isLinkTxScramblingEnabled(
 linkDebugSettings);

 /* Print the SpaceFibre link events information */
 printf("SpaceFibre port %u link debug settings:\n", port);
 printf(" Link Near-end Loopback : %s\n",
 linkNearEndLoopback ? "enabled" : "disabled");

```

```

 printf(" Link Rx Scrambling : %s\n",
 linkRxScrambling ? "enabled" : "disabled");
 printf(" Link Tx Scrambling : %s\n",
 linkTxScrambling ? "enabled" : "disabled");
 }
}

printf("\n");
}

}

void linkActiveLanesExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Link Active Lanes Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U32 linkActiveLanes = 0U;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get link active lanes */
 if (CFG_SPFI_getLinkActiveLanes(deviceID, port, &linkActiveLanes)
 != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get link active lanes", port);
 return;
 }
 else
 {
 printf("Port %u link active lanes :\n", port);

 /* If at least one active lane on the link */
 if (linkActiveLanes)
 {
 U32 lane;

 /* Test each bit in mask to see if it is set */
 for (lane = 0U; lane < 32U; lane++)
 {
 if ((1U << lane) & linkActiveLanes)
 {
 printf(" Lane %u", lane);
 }
 }
 }
 else
 {
 printf(" none");
 }
 printf("\n");
 }
 }
 }
}

```

```

 }

 printf("\n");
}

void linkInfoExample(STAR_DEVICE_ID deviceID)
{
 printf("Link Information Example\n");
 printf("=====\n");

 /* Link control example */
 linkControlExample(deviceID);

 /* Link status example */
 linkStatusExample(deviceID);

 /* Link events example */
 linkEventsExample(deviceID);

 /* Link debug example */
 linkDebugExample(deviceID);

 /* Link active lanes example */
 linkActiveLanesExample(deviceID);
}

```

## 10.27 mk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

#if defined(__rtems__)
#include <bsp.h>
#include <pthread.h>
#include "cfg_service_api.h"

int __cdecl main();

void *POSIX_Init(void *pArgs)
{
 UNREFERENCED_PARAMETER(pArgs);

 /* Start the Config Service */
 STAR_runConfigService();

 main();

 exit(0);
 return NULL;
}

#define CONFIGURE_APPLICATION_NEEDS_CONSOLE_DRIVER
#define CONFIGURE_APPLICATION_NEEDS_CLOCK_DRIVER

#define CONFIGURE_OBJECTS_UNLIMITED
#define CONFIGURE_UNIFIED_WORK_AREAS

#define CONFIGURE_MAXIMUM_POSIX_THREADS rtems_resource_unlimited(4)
#define CONFIGURE_MAXIMUM_POSIX_MUTEXES rtems_resource_unlimited(4)
#define CONFIGURE_MAXIMUM_POSIX_SEMAPHORES rtems_resource_unlimited(4)

#define CONFIGURE_POSIX_INIT_THREAD_TABLE

#define CONFIGURE_INIT
#include <rtems/confdefs.h>

#include <drvmgr/drvmgr_confdefs.h>
#endif

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID *devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;

```

```

char *deviceName, s[256];

/* Get the list of devices present which can be configured */
devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

/* If there was an error getting the list of devices */
if (!devices)
{
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
}

/* Display the devices detected */
if (devCount == 1)
{
 puts("One SpaceWire device detected:");
}
else
{
 printf("%u SpaceWire devices detected:", devCount);
}

/* For each device */
for (i = 0; i < devCount; i++)
{
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1)
{
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which device to open: ");
fgets(s, 256, stdin);
status = sscanf(s, "%d", &chosen);
if ((!status) || (chosen > devCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 double signallingRateSet;

 /* Select device to configure */
 deviceID = chooseStarDevice();
 if (!deviceID)
 {
 return 0;
 }

 /* Get hardware info*/
 CFG_getFPGAInfo(deviceID, &fpgaInfo);

 CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);

```

```

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s\n", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the devices LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

/* Enable adding the source port number as a leading byte
 to received packets on port 1 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);

/* Set port 2 to have a base transmit rate of 150 Mbit/s */
CFG_setTxSignallingRate(deviceID, 2, 150.0, &signallingRateSet);

/* Inject a disconnect error on link 1 */
CFG_injectError(deviceID, 1, SPW_ERROR_DISCONNECT);

return 0;
}

```

## 10.28 network\_management\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Network Management functions.

```

#include "cfg_api_spfi.h"

void getBroadcastTimeoutExample(STAR_DEVICE_ID deviceID)
{
 double broadcastTimeout = 0.0;
 double broadcastTimeoutMaximum = 0.0;
 double broadcastTimeoutResolution = 0.0;;

 /* Get broadcast timeout value */
 if (CFG_SPFI_getBroadcastTimeout(deviceID, &broadcastTimeout) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get broadcast timeout value.\n");
 return;
 }
 else
 {
 printf("Broadcast Timeout : %.2f microseconds\n",
 broadcastTimeout);
 }

 /* Get maximum broadcast timeout value */
 if (CFG_SPFI_getBroadcastTimeoutMaximum(deviceID, &
 broadcastTimeoutMaximum)
 != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get maximum broadcast timeout value.\n");
 return;
 }
 else
 {
 printf("Maximum Broadcast Timeout : %.2f microseconds\n",
 broadcastTimeoutMaximum);
 }

 /* Get broadcast timeout resolution value */
 if (CFG_SPFI_getBroadcastTimeoutResolution(deviceID,
 &broadcastTimeoutResolution) != CFG_STATUS_SUCCESS)

```



```

 {
 /* Print error and exit example */
 printf("Failed to get broadcast timeout resolution value.\n");
 return;
 }
 else
 {
 printf("Broadcast Timeout Resolution : %.2f microseconds\n",
 broadcastTimeoutResolution);
 }
}

void setBroadcastTimeoutExample(STAR_DEVICE_ID deviceID)
{
 double broadcastTimeout = 256.0;

 /* Set broadcast timeout value */
 if (CFG_SPFI_setBroadcastTimeout(deviceID, broadcastTimeout) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set broadcast timeout value.\n");
 return;
 }
 else
 {
 printf("Broadcast timeout set to %.2f microseconds.\n",
 broadcastTimeout);
 }
}

void getDeviceIdentifierExample(STAR_DEVICE_ID deviceID)
{
 U32 deviceIdentifier = 0U;

 /* Get device identifier value */
 if (CFG_SPFI_getDeviceIdentifier(deviceID, &deviceIdentifier) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get device identifier.\n");
 return;
 }
 else
 {
 printf("Device Identifier : %u\n", deviceIdentifier);
 }
}

void setDeviceIdentifierExample(STAR_DEVICE_ID deviceID)
{
 U32 deviceIdentifier = 1U;

 /* Set device identifier value */
 if (CFG_SPFI_setDeviceIdentifier(deviceID, deviceIdentifier) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set device identifier value.\n");
 return;
 }
 else
 {
 printf("Device identifier set to %u.\n", deviceIdentifier);
 }
}

void getSpaceWireBCExample(STAR_DEVICE_ID deviceID)
{
 U8 spaceWireBC = 0U;

 /* Get SpaceWire broadcast channel */
 if (CFG_SPFI_getSpaceWireBC(deviceID, &spaceWireBC) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to read SpaceWire broadcast channel.\n");
 return;
 }
 else
 {
 printf("Broadcast Channel : %u\n", spaceWireBC);
 }
}

void setSpaceWireBCExample(STAR_DEVICE_ID deviceID)
{

```

```

 U8 spaceWireBC = 1U;

 /* Set SpaceWire broadcast channel */
 if (CFG_SPFI_setSpaceWireBC(deviceID, spaceWireBC) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set SpaceWire broadcast channel.\n");
 return;
 }
 else
 {
 printf("SpaceWire broadcast channel set to %u.\n", spaceWireBC);
 }
}

void getTimeSlotExample(STAR_DEVICE_ID deviceID)
{
 U8 currentTimeSlot = 0U;

 /* Get current time-slot value */
 if (CFG_SPFI_getCurrentTimeSlot(deviceID, ¤tTimeSlot) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get current time-slot value.\n");
 return;
 }
 else
 {
 printf("Current Time-slot : %u\n", currentTimeSlot);
 }
}

void getCurrentTimeExample(STAR_DEVICE_ID deviceID)
{
 U64 currentTime = 0ULL;
 U8 synchronised = 0U;

 /* Get current time value */
 if (CFG_SPFI_getCurrentTime(deviceID, ¤tTime,
 &synchronised) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get current time value.\n");
 return;
 }
 else
 {
 printf("Current Time : %llu\n", currentTime);
 printf("Synchronised : %s\n",
 synchronised ? "yes" : "no");
 }
}

void getPortsReadyExample(STAR_DEVICE_ID deviceID)
{
 U32 portsReady = 0U;

 /* Get bitmask containing ports that are ready */
 if (CFG_SPFI_getPortsReady(deviceID, &portsReady) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get ports that are ready.\n");
 return;
 }
 else
 {
 printf("Ports ready :");

 /* If at least one port is ready */
 if (portsReady)
 {
 U8 port;

 /* Test each bit in mask to see if it is set */
 for (port = 1U; port < 32U; port++)
 {
 if ((1U << port) & portsReady)
 {
 printf(" %u", port);
 }
 }
 }
 else
 {

```

```

 printf(" none");
 }
 printf("\n");
}

void getLinksWithErrorsExample(STAR_DEVICE_ID deviceID)
{
 U32 linksWithErrors = 0U;

 /* Get bitmask containing links with errors */
 if (CFG_SPFI_getLinksWithErrors(deviceID, &linksWithErrors) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get links with errors.\n");
 return;
 }
 else
 {
 printf("Links with errors :");

 /* If at least one link has an error */
 if (linksWithErrors)
 {
 U8 port;

 /* Test each bit in mask to see if it is set */
 for (port = 1U; port < 32U; port++)
 {
 if ((1U << port) & linksWithErrors)
 {
 printf(" %u", port);
 }
 }
 }
 else
 {
 printf(" none");
 }
 printf("\n");
 }
}

void getPortsWithErrorsExample(STAR_DEVICE_ID deviceID)
{
 U32 portsWithErrors = 0U;

 /* Get bitmask containing ports with errors */
 if (CFG_SPFI_getPortsWithErrors(deviceID, &portsWithErrors) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get ports with errors.\n");
 return;
 }
 else
 {
 printf("Ports with errors :");

 /* If at least one port has an error */
 if (portsWithErrors)
 {
 U8 port;

 /* Test each bit in mask to see if it is set */
 for (port = 1U; port < 32U; port++)
 {
 if ((1U << port) & portsWithErrors)
 {
 printf(" %u", port);
 }
 }
 }
 else
 {
 printf(" none");
 }
 printf("\n");
 }
}

void networkManagementExample(STAR_DEVICE_ID deviceID)
{
 printf("Network Management Example\n");
 printf("=====");
}

```

```

/* Set broadcast timeout example */
setBroadcastTimeoutExample(deviceID);

/* Set device identifier example */
setDeviceIdentifierExample(deviceID);

/* Set SpaceWire broadcast channel example */
setSpaceWireBCExample(deviceID);

printf("\n");

/* Get broadcast timeout example */
getBroadcastTimeoutExample(deviceID);

/* Get device identifier example */
getDeviceIdentifierExample(deviceID);

/* Get SpaceWire broadcast channel example */
getSpaceWireBCExample(deviceID);

/* Get time-slot example */
getTimeSlotExample(deviceID);

/* Get current time example */
getCurrentTimeExample(deviceID);

/* Get ports ready example */
getPortsReadyExample(deviceID);

/* Get links with errors example */
getLinksWithErrorExample(deviceID);

/* Get ports with errors example */
getPortsWithErrorExample(deviceID);

printf("\n");
}

```

## 10.29 packet\_subsystem\_example.c

Example usage of this API.

```

#include "packet_subsystem.h"
#include "cfg_api_generic.h"
#include "ui.h"
#include <stdlib.h>
#include <stdio.h>

#define VERSION_INFO "PCI Mk2 Packet Subsystem Test Application"

/* Currently the PCI Mk2 only contains a single packet subsystem on port 8*/
#define PACKET_SUBSYSTEM_NUMBER 1
#define SUBSYSTEM_PORT_START 7

#define MIN(x, y) ((x) < (y)) ? (x) : (y)

#if !defined(UNREFERENCED_PARAMETER)
#define UNREFERENCED_PARAMETER(a) ((void)(a))
#endif

void printVersionInformation()
{
 puts(VERSION_INFO);
 puts("Copyright (C) 2012 STAR-Dundee Ltd.");
 puts("http://www.star-dundee.com/ \n");
}

int runTest(STAR_DEVICE_ID generatorDevice,
 STAR_DEVICE_ID checkerDevice)
{
 /* Buffer that will be filled with bytes to be used for generated packet headers */
 U8 *pHeaderBuffer;

 /* Number of header bytes to use */
 U16 headerBytes = 2;
 /* Determine how many words header bytes occupy */
 U16 headerWords = (headerBytes + 3) / 4;

```

```

/* Sizes for expected headerdata used for checker*/
U16 expectedHeaderWords = 0;

/* Buffer that will be filled with bytes to be used for generated packet bodies */
U8 *pBodyBuffer;

/* Number of 4 byte words to use when generating packet bodies */
U16 bodyWords = 12;

/* Size of packets to be generated. Generated packets will be made up of the specified
 header bytes, followed by the body bytes specified repeating as necessary to make
 the packet up to the length specified.*/
U16 packetLen = 100 + headerBytes;

/* Size of expected packets received by the packet checker. Header deletion by the router
 will strip the path address. */
U16 expectedPacketLen = packetLen - headerBytes;

/* Here we decide on the the physical location in the device dual port memory to use
 for the packet generator and checker information, and for the sink's receive buffer.
 These locations and the order we specify them are arbitrary. */

/* We decide to put the header data at address 0 */
U16 memHeaderGenStart = 0;

/* We put the body data after the end of the header data */
U16 memBodyGenStart = memHeaderGenStart + headerWords;

/* Put the checker memory after the end of the generator data */
U16 memHeaderCheckerStart = memBodyGenStart + bodyWords;
U16 memBodyCheckerStart = memHeaderCheckerStart + expectedHeaderWords;

/* Put the sink buffer after the checker memory */
U16 memSinkStart = memBodyCheckerStart + bodyWords;

U16 sinkLenWords;

/* Number of bytes to wait before turning off sink after detecting a character mismatch */
U16 sinkDelayOnMismatch = 16;

/* Count of byte mismatches caught by the packet checker */
U64 mismatchCount = 0;

/* Circular buffer pointers used when reading data back from the packet sink */
U16 startPointer = 0 , endPointer = 0;

/* Buffer that will be filled with data read back from the packet sink*/
U8* pReadBackBuffer;

/* Values used when displaying results*/
unsigned int offset= 0, address = 0;

/* Create local buffers which we will copy to the devices on board memory */
pHeaderBuffer = (U8*)malloc(headerWords * 4);
if(!pHeaderBuffer)
{
 return 0;
}
pBodyBuffer = (U8*)malloc(bodyWords * 4);
if(!pBodyBuffer)
{
 free(pHeaderBuffer);
 return 0;
}

printVersionInformation();

/* In this example (with a cable looped back from port 1 to port 3) we want the packet
 to have a path address of [1, 8]. 1 to send packet out of port one, 8 to receive the
 packet back at the packet subsystem.*/
*pHeaderBuffer = 1;
*(pHeaderBuffer + 1) = SUBSYSTEM_PORT_START + PACKET_SUBSYSTEM_NUMBER;

/* Fill the packet body with a repeating pattern*/
PKT_SUBSYS_fillBufferWithPattern(PKT_SUBSYS_PATTERN_ALTERNATING_BITS,
 bodyWords * 4, pBodyBuffer);

/* Write packet header and body to device memory */
PKT_SUBSYS_writeMemory(generatorDevice, memHeaderGenStart, headerWords * 4,
 pHeaderBuffer);
PKT_SUBSYS_writeMemory(generatorDevice, memBodyGenStart, bodyWords * 4,
 pBodyBuffer);
free(pHeaderBuffer);
pHeaderBuffer = NULL;

/* Write expected packet body to checker device memory. As we don't expect the packet body to change
 we can just use the same buffer as before.

```

```

 Note that header deletion will strip the path address, so we will not expect to see a header*/
PKT_SUBSYS_writeMemory(checkerDevice, memBodyCheckerStart, bodyWords * 4,
 pBodyBuffer);
free(pBodyBuffer);
pBodyBuffer = NULL;

/* Instruct subsystem A packet generator to use supplied buffers */
PKT_SUBSYS_setPacketGeneratorFormat(generatorDevice,
 PACKET_SUBSYSTEM_NUMBER,
 memHeaderGenStart,
 headerBytes,
 memBodyGenStart,
 bodyWords,
 packetLen,
 STAR_EOP_TYPE_EOP,
 0);

/* Set up a packet sink on subsystem B to receive generated packets:
 We set up circular buffer with room for 20 packets,
 ie sink size in words = ((20 packets * (packetLen in bytes + 2 bytes for end of packet marker) +
 3) / 4
 Note that 3 is added to ensure there are enough words if (packet length + EOP) is not a multiple of
 4
*/
sinkLenWords = ((20 * (expectedPacketLen+2)) + 3) / 4;
PKT_SUBSYS_setPacketSinkParameters(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER, memSinkStart, sinkLenWords);

/* Set up the packet checker on subsystem B to check the received generated packets
 Here we just re-use the buffers already written to device to generate packets from.
 As we do not expect a packet header, we set its length to 0.*/
PKT_SUBSYS_setPacketCheckingFormat(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER,
 memHeaderCheckerStart,
 0,
 memBodyCheckerStart,
 bodyWords,
 expectedPacketLen,
 STAR_EOP_TYPE_EOP);

/* Initiate packet generation and sinking run */
PKT_SUBSYS_startSink(checkerDevice, PACKET_SUBSYSTEM_NUMBER);
/* Start the packet checker, disable the the packet sink after a short delay on receiving a mismatched
 character */
PKT_SUBSYS_startPacketChecking(checkerDevice, PACKET_SUBSYSTEM_NUMBER, 1,
 sinkDelayOnMismatch);
/* Kick off packet generation of 10 packets */
PKT_SUBSYS_startPacketGeneration(generatorDevice,
 PACKET_SUBSYSTEM_NUMBER, 10);
/* Wait for packet generation to complete */
PKT_SUBSYS_waitForPacketGenerationSequenceComplete(
 generatorDevice, PACKET_SUBSYSTEM_NUMBER, 0);
/* Get a count of the number of received bytes that did not match the expected pattern */
PKT_SUBSYS_getPacketCheckingMismatchCount(checkerDevice,
 PACKET_SUBSYSTEM_NUMBER, &mismatchCount);
/* Stop packet checking*/
PKT_SUBSYS_stopPacketChecking(checkerDevice, PACKET_SUBSYSTEM_NUMBER);
PKT_SUBSYS_stopSink(checkerDevice, PACKET_SUBSYSTEM_NUMBER);

printf("Mismatch count: %llu\n", mismatchCount);

/* Display sink contents if we received a mismatched character */
if(mismatchCount)
{
 /* Read back the packet sink data from the device */
 PKT_SUBSYS_getSinkDataPointers(checkerDevice, PACKET_SUBSYSTEM_NUMBER
 , &startPointer, &endPointer);
 if(endPointer < startPointer)
 {
 /* wraparound */
 pReadBackBuffer = (U8*)calloc(sinkLenWords, sizeof(U32));
 if(!pReadBackBuffer)
 {
 return 0;
 }

 PKT_SUBSYS_readMemory(checkerDevice, startPointer, ((memSinkStart+
 sinkLenWords) - startPointer) * sizeof(U32), pReadBackBuffer);
 PKT_SUBSYS_readMemory(checkerDevice, memSinkStart, (endPointer -
 memSinkStart) * sizeof(U32), pReadBackBuffer + ((memSinkStart+sinkLenWords) - endPointer) * sizeof(
 U32));
 }
 else
 {

```

```

 /* no wraparound */
 sinkLenWords = endPoint - startPointer;
 pReadBackBuffer = (U8*)calloc(sinkLenWords, sizeof(U32));
 if(!pReadBackBuffer)
 {
 return 0;
 }
 PKT_SUBSYS_readMemory(checkerDevice, startPointer, (endPointer -
startPointer) * sizeof(U32), pReadBackBuffer);
 }

 /* Display all data received after the packet sink was stopped automatically on
character mismatch. Note that if the packet sink was stopped manually before this
delay was reached, some displayed data will be from before the first mismatch occurred. */

 if((sinkDelayOnMismatch+3)/4 >= sinkLenWords)
 {
 /* sink disable delay is larger than sink; display whole sink */
 offset = address = 0;
 }
 else
 {
 /* display only last (sink delay words + mismatch word) of sink */
 offset = (sinkLenWords * 4 - sinkDelayOnMismatch) - 4;
 address = (sinkLenWords - (sinkDelayOnMismatch+3)/4) - 1;
 }

 printf("Packet Sink contents (Last %d words):\n", MIN(sinkLenWords, ((sinkDelayOnMismatch+3)/4) + 1
));

 for(; offset <= sinkLenWords * sizeof(U32) - 4; offset += 4)
 {
 printf(" %-5d | %02x %02x %02x %02x\n", address++,
 ((U8 *)pReadBackBuffer)[offset], ((U8 *)pReadBackBuffer)[offset + 1],
 ((U8 *)pReadBackBuffer)[offset + 2], ((U8 *)pReadBackBuffer)[offset + 3]);
 }

 free(pReadBackBuffer);
 pReadBackBuffer = NULL;
}

return 0;
}

void STAR_API_CC displayStats(
 STAR_DEVICE_ID deviceIdentifier,
 unsigned int subsystem,
 PKT_SUBSYS_STATISTICS statistics)
{
 UNREFERENCED_PARAMETER(deviceIdentifier);
 UNREFERENCED_PARAMETER(subsystem);

 printf("Data Character Rate: %u/s \n", statistics.dataCharacterRate);
 printf("Data Characters Received: %llu \n", statistics.
dataCharactersReceived);
 printf("EEP Character Rate: %u/s \n", statistics.eepCharacterRate);
 printf("EEP Characters Received: %llu \n", statistics.eepCharactersReceived);
 printf("EOP Character Rate: %u/s \n", statistics.eopCharacterRate);
 printf("EOP Characters Received: %llu \n", statistics.eopCharactersReceived);
 puts("Press enter to stop");
}

void showStatistics(STAR_DEVICE_ID deviceId)
{
 char string [256];
 STATISTICS_LOOP_ID id;

 /* Enable packet sink */
 PKT_SUBSYS_startSink(deviceId, PACKET_SUBSYSTEM_NUMBER);

 puts("Ensure the device is not running in interface mode.");
 puts("Press enter to start:");
 fgets(string, 256, stdin);

 /* Begin polling statistics */
 id = PKT_SUBSYS_startGetStatisticsLoop(deviceId,
 PACKET_SUBSYSTEM_NUMBER, displayStats);
 if(!id)
 {
 puts("Could not start statistics loop");
 return;
 }
 puts("Press enter to stop:");
 fgets(string, 256, stdin);
}

```

```

 /* End loop, disable packet sink again */
 PKT_SUBSYS_stopGetStatisticsLoop(id);
 PKT_SUBSYS_stopSink(deviceId, PACKET_SUBSYSTEM_NUMBER);
}

void usage(char *argv[], int error)
{
 puts("");

 if(error)
 fprintf(stderr, "usage: %s [-v][-s][-help]\n", argv[0]);
 else
 fprintf(stdout, "usage: %s [-v][-s][-help]\n", argv[0]);
}

void showHelp(void)
{
 puts("");
 puts("Description:");
 puts(" A program to demonstrate usage of the PCI Mk2");
 puts(" packet subsystem API. Sets up a packet generator");
 puts(" and checker, starts them, then displays results.");
 puts("");
 puts(" Note that interface mode will be disabled on the");
 puts(" the device(s) chosen.");
 puts("");
 puts("Optional Switches:");
 puts("");
 puts(" -v Displays version information and exits.");
 puts("");
 puts(" -help Displays this help message and exits.");
 puts("");
 puts(" -s Polls and displays device statistics until");
 puts(" user hits enter.");
 puts("");
 puts(" When called without arguments, the program sets up a packet");
 puts(" generator a chosen device to send 10 102 byte packets,");
 puts(" addressed to the packet checking subsystem on a second chosen");
 puts(" device. The packets are built from 2 address bytes");
 puts(" (1, then subsystem port) followed by 100 bytes of a repeating");
 puts(" alternating bits pattern.");
}

/*Set up a packet generator on subsystem one to send packets to a sink on subsystem 2*/
int __cdecl main(int argc, char* argv[])
{
 int i;
 int version = 0; /* Value for the "-v" optional argument. */
 int help = 0; /* Value for the "-help" optional argument. */
 int statistics = 0; /* Value for the "-s" optional argument. */
 STAR_DEVICE_ID deviceA, deviceB;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_PCI_MK2;

 /* If no args provided, run with default settings */
 if (argc <= 1)
 {
 puts("Select device to run packet generator on:");
 deviceA = STAR_UI_chooseDevice(aDeviceTypes,1);

 puts("Select device to run packet checker on:");
 deviceB = STAR_UI_chooseDevice(aDeviceTypes,1);

 if (!deviceA || !deviceB)
 {
 return 1;
 }

 CFG_disableInterfaceMode(deviceB);

 runTest(deviceA, deviceB);
 return 0;
 }

 /* Currently we only expect 1 optional argument */
 if(argc > 2)
 {
 usage(argv, 1);
 return 1;
 }

 /* Process command line args*/
 for (i = 1; i < argc; i++)
 {
 if (strcmp(argv[i], "-v") == 0)

```



```

 {
 version = 1;
 }
 else if(strcmp(argv[i], "-help") == 0)
 {
 help = 1;
 }
 else if(strcmp(argv[i], "-s") == 0)
 {
 statistics = 1;
 }
 }

 if(help)
 {
 usage(argv, 0);
 showHelp();
 }
 else if(version)
 {
 printVersionInformation();
 }
 else if (statistics)
 {
 puts("Select device:");
 //deviceA = chooseDevice();
 deviceA = STAR_UI_chooseDevice(aDeviceTypes, 1);

 CFG_disableInterfaceMode(deviceA);

 if(!deviceA)
 {
 return 1;
 }

 showStatistics(deviceA);
 }
 else
 {
 usage(argv, 1);
 return 1;
 }

 return 0;
}

```

## 10.30 pciMk2\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 double signallingRate;
 double signallingRateSet;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_PCI_MK2;

 /* Select device to configure */
 deviceID = STAR_UI_chooseDevice(aDeviceTypes, 1);
 if (!deviceID)
 {
 return 0;
 }

 STAR_getDeviceType(deviceID);

 /* Get hardware info*/
 CFG_getFPGAInfo(deviceID, &fpgaInfo);

 CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);
}

```

```

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the device's LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 3 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 3);

/* Set port 2 to have a link speed of 100 Mbit/s */
CFG_setTxSignallingRate(deviceID, 2, 100.0, &signallingRateSet);

/* Read Link speed of port 1 */
CFG_getTxSignallingRate(deviceID, 1, &signallingRate);
printf("\nLink 1 transmit rate: %g Mbit/s", signallingRate);

/* Inject a disconnect error on link 1 */
CFG_injectError(deviceID, 1, SPW_ERROR_DISCONNECT);

return 0;
}

```

## 10.31 port\_control\_status\_example.c

This example demonstrates the use of the Port Status and Control functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */

```

```

for (i = 0; i < devCount; i++)
{
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1)
{
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which device to open: ");
fgets(s, 256, stdin);
status = sscanf(s, "%d", &chosen);
if ((!status) || (chosen > devCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 PORT_STATUS_CONTROL registerValue;
 U8 portNum = 0; /* Configuration port */

 STAR_CFG_CONFIG_PORT_ERRORS configPortErrors;
 STAR_CFG_SPW_LINK_STATUS spwPortStatus;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get port information */
 CFG_getPortStatusControl(deviceID, portNum, ®isterValue);

 /* Display port type */
 switch(CFG_getPortType(registerValue))
 {
 case STAR_CFG_PORT_TYPE_CONFIGURATION:
 printf("\nPort type of port %u: Configuration Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_LINK:
 printf("\nPort type of port %u: SpaceWire Link Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_EXTERNAL:
 printf("\nPort type of port %u: External Port", portNum);
 break;
 case STAR_CFG_PORT_TYPE_INVALID:
 printf("\nPort type of port %u: Invalid port value", portNum);
 break;
 }

 /* Display current internal connection */
 printf("\nPort %d is currently connected to port %d ",
 portNum, CFG_getPortConnection(registerValue));

 /* Get configuration port errors (if portNum == 0) */
 CFG_getConfigPortErrors(registerValue, &configPortErrors);
 if (configPortErrors.errorCount)
 {
 printf("\n%d error(s) present on configuration port ", configPortErrors.
 errorCount);
 }
}

```

```

 /* Clear port errors */
 CFG_clearPortErrors(deviceID, portNum);

 /* Get SpaceWire port Status */
 portNum = 1; /* Assume for the purpose of this example that port One
 for this device is a SpaceWire port */

 CFG_getPortStatusControl(deviceID, portNum, ®isterValue);

 /* Obtain the links status values */
 CFG_getSpaceWireLinkStatus(registerValue, &spwPortStatus);

 /* Display some details about the links status
 Note that the links state machine state can be obtained from the linkState member*/
 printf("\nLink %u %s running.", portNum, spwPortStatus.running ? "is":"is not");
 printf("\nLink %u %s set to autostart.", portNum, spwPortStatus.autoStart ? "is":"is not");
 printf("\nLink %u %s set to start.", portNum, spwPortStatus.start ? "is":"is not");
 printf("\nLink %u %s disabled.", portNum, spwPortStatus.disable ? "is":"is not");
 printf("\nLink %u %s in tri-state mode.\n", portNum, spwPortStatus.triState ? "is":"is not");

 /* Disable the link (Set the disable bit, clear the start bit)*/
 spwPortStatus.disable = 1;
 spwPortStatus.start = 0;
 CFG_setSpaceWireLinkStatus(deviceID, portNum, &spwPortStatus);

 /* Start and stop the link using the shortcut functions.*/
 CFG_startLink(deviceID, portNum);
 CFG_stopLink(deviceID, portNum);

 return 0;
}

```

## 10.32 port\_info\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Port Information functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void getAXIBCExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get AXI BC Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 axiBC;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not an AXI port */
 if (portType != SPFI_PORT_TYPE_AXI)
 {
 /* Skip port */
 }
 }
 }
}

```

```

 continue;
 }

 /* Get AXI BC value */
 axiBC = CFG_SPFI_getAXIBC(portInfo);
 printf("Port %2u AXI Broadcast Channel : %u\n", port, axiBC);
}

printf("\n");
}

void setAXIBCExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Set AXI BC Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 axiBC;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not an AXI port */
 if (portType != SPFI_PORT_TYPE_AXI)
 {
 /* Skip port */
 continue;
 }

 /* Use 63 for AXI BC */
 axiBC = 63U;

 /* Set AXI BC value */
 if (CFG_SPFI_setAXIBC(deviceID, port, axiBC) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to set AXI broadcast channel value",
 port);
 return;
 }
 else
 {
 printf("AXI broadcast channel set to %u", axiBC);
 printPortMessage("", port);
 }
 }
 }

 printf("\n");
}

void getPortInfoExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Port Info Example\n");
 printf("-----\n");

```

```

/* Get the number of ports */
portCount = getPortCount(deviceID);
if (!portCount)
{
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
}

/* Loop through the ports */
for (port = 1U; port < portCount; port++)
{
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U32 portVCsWithErrors = 0U;

 /* Get port properties */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);
 U8 portVcCount = CFG_SPFI_getPortVcCount(portInfo);
 U8 portReady = CFG_SPFI_isPortReady(portInfo);
 U8 portBroadcastErrorDetected =
 CFG_SPFI_isPortBroadcastErrorDetected(portInfo);

 /* Print port properties */
 printf("Port Number : %u\n", port);
 printf("Port Type : %s\n",
 portTypeToString(portType));
 printf("VC Count : %u\n", portVcCount);
 printf("Ready : %s\n",
 portReady ? "yes" : "no");
 printf("Broadcast Error Detected : %s\n",
 portBroadcastErrorDetected ? "yes" : "no");

 /* Get bitmask containing port virtual channels with errors */
 if (CFG_SPFI_getPortVCsWithErrors(deviceID, port,
 &portVCsWithErrors) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get port virtual channels with errors.\n");
 return;
 }
 else
 {
 printf("Port VCs With Errors :");

 /* If at least one VC has an error */
 if (portVCsWithErrors)
 {
 U32 vc;

 /* Test each bit in mask to see if it is set */
 for (vc = 1U; vc < 32U; vc++)
 {
 if ((1U << port) & portVCsWithErrors)
 {
 printf(" %u", vc);
 }
 }
 }
 else
 {
 printf(" none");
 }
 printf("\n");
 }

 printf("\n");

 /* Clear port broadcast error */
 if (CFG_SPFI_clearPortBroadcastError(deviceID, port) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to clear port broadcast error", port);
 return;
 }
 else
 }
}

```

```

 {
 printPortMessage("Port broadcast error cleared", port);
 }
 }

 printf("\n");
}

void portInfoExample(STAR_DEVICE_ID deviceID)
{
 printf("Port Information Example\n");
 printf("=====\n");

 /* Set AXI broadcast channel example */
 setAXIBCExample(deviceID);

 /* Get AXI broadcast channel example */
 getAXIBCExample(deviceID);

 /* Get port info example */
 getPortInfoExample(deviceID);
}

```

## 10.33 read\_command\_packet\_example.c

This is an example of how to build RMAP read commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void read_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 char status;

 /* Calculate the length of a read command packet, with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateReadCommandPacketLength(2,
 1, 1);
 printf("The read command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillReadCommandPacket(pTarget, 2, pReply, 1, 0, 0x20, 0,
 0x106, 0, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the read command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP read command packet to read 4-bytes */
 pBuildPacket = RMAP_BuildReadCommandPacket(pTarget, 2, pReply, 1, 0, 0x20,
 0, 0x106, 0, 4, &buildPacketLen, NULL, 1);
 if (!pBuildPacket)
 {
 puts("Couldn't build the read command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP read command packet to read from a 4-byte register */

```

```

pBuildPacketRegister = RMAP_BuildReadRegisterPacket(pTarget, 2, pReply, 1,
0, 0x20, 0, 0x106, 0, &buildPacketRegisterLen, NULL, 1);
if (!pBuildPacketRegister)
{
 puts("Couldn't build the read register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen) ||
 (buildPacketLen != buildPacketRegisterLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketRegisterLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacketRegister);
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 10.34 read\_reply\_packet\_example.c

This is an example of how to build RMAP read replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void read_reply_packet_example()
{
 void *pFillPacket, *pBuildPacket;
 unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
 U8 pReply[] = {254};
 U8 target = 254;
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 char status;

 /* Calculate the length of a read reply packet, with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateReadReplyPacketLength(1, 4,
1);
 printf("The read reply packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the reply packet");
 }
}

```



```

 return;
}

/* Fill the memory with the packet */
status = RMAP_FillReadReplyPacket(pReply, 1, target, 0,
 RMAP_SUCCESS, 0,
 pData, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
if (!status)
{
 puts("Couldn't fill the read reply packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read reply packet */
pBuildPacket = RMAP_BuildReadReplyPacket(pReply, 1, target, 0,
 RMAP_SUCCESS, 0, pData, 4, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the read reply packet");
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 10.35 rmap\_target\_example.c

This file contains example code demonstrating how to use the RMAP Target API.

```

#include "rmap_target_pxi_if.h"

#include <stdlib.h>

#include "rmap_packet_library.h"
#include "cfg_api_generic.h"

#ifdef _WIN32
 #include "windows.h"

 #define SLEEP(milliseconds) Sleep(milliseconds);
#else
 #include <unistd.h>

 #define SLEEP(milliseconds) usleep(milliseconds * 1000);
#endif

#if !defined(UNREFERENCED_PARAMETER)

```

```

#define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

/* The four RMAP targets are accessed through external ports 6-9 */
#define TARGET_PORT_START (6)
#define TARGET_PORT_END (9)

typedef enum
{
 MENU_CHOICE_MANUAL_AUTH = 1,
 MENU_CHOICE_NOTIFICATIONS,
 MENU_CHOICE_EXIT,
 MENU_CHOICE_INVALID
} MENU_CHOICE;

void STAR_API_CC notifListener(STAR_DEVICE_ID deviceId,
 U32 target,
 void *pNotif, void *pContext)
{
 int i;

 /* Cast the notification parameter to an array of bytes */
 U8 *notif = (U8*)pNotif;

 /* Cast the context variable to a U32 */
 U32 context = *((U32*)pContext);

 /* Print the device ID and the target index */
 printf("Device %u (Target = %u, Context = %u): ", deviceId, target,
 context);

 /* Print each of the notification bytes */
 /* See the STAR-System documentation for the notification structure */
 for (i = 0; i < 22; ++i)
 {
 printf("0x%02x ", notif[i]);
 }
 printf("\n");
}

void sendRandomCommand(STAR_DEVICE_ID deviceId, U32 target)
{
 U8 buffer[1024];
 U8 cmdPath[2], repPath[1];
 U32 length, address;
 U8 iniLa, tarLa, key;
 STAR_CHANNEL_ID channelId;
 STAR_STREAM_ITEM *streamItem;
 STAR_TRANSFER_OPERATION *transferOp;
 STAR_TRANSFER_STATUS status;
 void *command;
 unsigned long rawPacketLength;

 /* Set the RMAP command parameters to random values */
 length = rand() % 1024;
 address = rand();
 iniLa = 32 + (rand() % 222);
 tarLa = 32 + (rand() % 222);
 key = rand();

 /* Set the buffer to 0xAB, repeating */
 memset(buffer, 0xAB, 1024);

 /* Set the command and reply paths */
 cmdPath[0] = 6 + target, cmdPath[1] = tarLa;
 repPath[0] = iniLa;

 /* Open a channel */
 channelId = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_OUT, 4, 1);
 if (!channelId)
 {
 printf("Failed to open channel 4\n");
 return;
 }

 /* Build the command */
 command = RMAP_BuildWriteCommandPacket(cmdPath, 2, repPath, 1, 0, 0, 1,
 key, 0, address, 0, buffer, length, &rawPacketLength, NULL, 1);

 /* Create the stream item */
 streamItem = STAR_createPacket(0, command, rawPacketLength,
 STAR_EOP_TYPE_EOP);
 if (!streamItem)
 {
 RMAP_FreeBuffer(command);
 printf("Failed to create stream item\n");
 }
}

```

```

 return;
 }

 /* Create the transfer operation */
 transferOp = STAR_createTxOperation(&streamItem, 1);
 if (!transferOp)
 {
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to create transfer operation\n");
 return;
 }

 /* Submit the transfer operation */
 if (!STAR_submitTransferOperation(channelId, transferOp))
 {
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to submit transfer operation\n");
 return;
 }

 /* Wait on the transfer operation completing */
 status = STAR_waitOnTransferOperationCompletion(transferOp, -1);
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 printf("Failed to transmit the command\n");
 return;
 }

 STAR_destroyStreamItem(streamItem);
 RMAP_FreeBuffer(command);
 STAR_closeChannel(channelId);
}

void reset(STAR_DEVICE_ID deviceId)
{
 int i;

 /* Reset the RMAP parameters for each target */
 for (i = 0; i < 4; ++i)
 {
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, TARGET_PORT_START
 + i,
 IF_MODE_RMAP_ENABLED);
 RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, i,
 TARGET_AUTH_MODE_AUTOMATIC);
 RMAP_TARGET_PXI_IF_setAuthCommands(deviceId, i,
 TARGET_AUTH_CMD_ALL);
 RMAP_TARGET_PXI_IF_setAuthKeyRange(deviceId, i, 0, 255);
 RMAP_TARGET_PXI_IF_setAuthLogicalAddressRange(deviceId
 , i, 32, 255);
 RMAP_TARGET_PXI_IF_setAuthProtocolId(deviceId, i, 1);
 RMAP_TARGET_PXI_IF_setAuthMemoryAddressRange(deviceId,
 i, 0,
 0xffffffff);
 RMAP_TARGET_PXI_IF_setAddressOffset(deviceId, i, 0);
 RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, i,
 NOTIF_TYPE_NONE);
 RMAP_TARGET_PXI_IF_unregisterNotificationListener(
 deviceId, i,
 NOTIF_TYPE_AUTH_REQUEST);
 RMAP_TARGET_PXI_IF_unregisterNotificationListener(
 deviceId, i,
 NOTIF_TYPE_CMD_COMPLETE);
 RMAP_TARGET_PXI_IF_stopReceivingNotifications(deviceId
);
 }
}

MENU_CHOICE getMenuChoice(void)
{
 char buffer[32];
 unsigned int choice;

 /* Show menu to user */
 printf("Please select:\n");
 printf("%d. Manual RMAP command authorisation example.\n",
 MENU_CHOICE_MANUAL_AUTH);
 printf("%d. Command complete notifications example.\n",
 MENU_CHOICE_NOTIFICATIONS);
 printf("%d. Exit\n", MENU_CHOICE_EXIT);
 printf("> ");

 /* Validate chosen number */

```

```

 if (!fgets(buffer, 32, stdin)) return MENU_CHOICE_INVALID;

 /* Get menu choice value */
 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) return MENU_CHOICE_INVALID;

 /* Return the chosen value */
 return choice;
}

void manualAuthExample(STAR_DEVICE_ID deviceId)
{
 char buffer[32];
 unsigned int choice;
 RmapCommandParameters command;

 /* Disable router timeouts so RMAP commands aren't truncated whilst
 awaiting authorisation */
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 CFG_getRouterGlobalSettings(deviceId, &globalState);
 globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_BLOCKING;
 CFG_setRouterGlobalSettings(deviceId, &globalState);

 /* Reset the RMAP targets */
 reset(deviceId);

 /* Disable the RMAP target for target 3 (port 9) so the channel can be used
 to transmit commands */
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, 9,
 IF_MODE_RMAP_DISABLED);

 /* Enable manual authorisation mode for target 0 */
 RMAP_TARGET_PXI_IF_setAuthControlMode(deviceId, 0,
 TARGET_AUTH_MODE_MANUAL);

 printf("\nGenerating randomised RMAP command for target 0...\n");

 /* Send a random command to target 0 */
 sendRandomCommand(deviceId, 0);

 /* Get the waiting command */
 RMAP_TARGET_PXI_IF_getWaitingCommand(deviceId, 0, &command);

 /* Print the command parameters */
 printf("\nParameters: tar LA = 0x%x, command = 0x%x, key = 0x%x, "
 "address = 0x%x, length = 0x%x, ini LA = 0x%x\n",
 command.targetLogicalAddress, command.command, command.key,
 command.address, command.dataLength, command.initiatorLogicalAddress);

 /* Print the authorisation menu */
 printf("1. Authorise.\n");
 printf("2. Reject.\n");
 printf("> ");

 if (!fgets(buffer, 32, stdin)) choice = 0;

 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) choice = 0;

 /* Authorise or reject the command */
 if (choice == 1)
 {
 printf("Authorising command...\n");
 RMAP_TARGET_PXI_IF_authoriseWaitingCommand(deviceId, 0);
 }
 else if (choice == 2)
 {
 printf("Rejecting command...\n");
 RMAP_TARGET_PXI_IF_rejectWaitingCommand(deviceId, 0,
 REJECTION_REASON_OTHER);
 }
}

void notificationsExample(STAR_DEVICE_ID deviceId)
{
 U32 target;
 U32 context0, context1, context2;

 /* Reset the RMAP targets */
 reset(deviceId);

 /* Disable the RMAP target for target 3 (port 9) so the channel can be used
 to transmit commands */
 RMAP_TARGET_PXI_IF_setInterfaceMode(deviceId, 9,
 IF_MODE_RMAP_DISABLED);

 /* Enable notifications for targets 0-2 */

```

```

RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 0,
 NOTIF_TYPE_CMD_COMPLETE);
RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 1,
 NOTIF_TYPE_CMD_COMPLETE);
RMAP_TARGET_PXI_IF_setEnabledNotifications(deviceId, 2,
 NOTIF_TYPE_CMD_COMPLETE);

/* Set the context variables */
context0 = 123;
context1 = 456;
context2 = 789;

/* Set the command complete notification listener for targets 0-2 */
RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
(deviceId, 0,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context0);
RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
(deviceId, 1,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context1);
RMAP_TARGET_PXI_IF_registerNotificationListenerWithContext
(deviceId, 2,
 NOTIF_TYPE_CMD_COMPLETE, notifListener, &context2);

/* Start receiving notifications for the device */
RMAP_TARGET_PXI_IF_startReceivingNotifications(deviceId);

printf("\nGenerating randomised RMAP commands for target 0-2...\n");

/* Send random commands to targets 0-2 every 200 milliseconds */
while (1)
{
 target = rand() % 3;
 sendRandomCommand(deviceId, target);
 SLEEP(200);
}

}

STAR_DEVICE_ID getDeviceId(void)
{
 U32 count;
 STAR_DEVICE_ID *devices = NULL;
 STAR_DEVICE_ID deviceId = 0;

 /* Get the first PXI RMAP Target device */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_PXI_RMAP, &count);

 /* Try to get the first PXI RMAP Target Mk2 device */
 if (0 == count)
 {
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_PXI_RMAP_MK2, &count);
 }

 /* In case devices is still NULL, return 0 as device ID */
 if (NULL == devices)
 {
 return 0;
 }

 deviceId = count ? devices[0] : 0;

 STAR_destroyDeviceList(devices);
 return deviceId;
}

int main(int argc, char **argv)
{
 STAR_DEVICE_ID deviceId;
 MENU_CHOICE choice;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 /* Print the program header */
 printf("----- RMAP Target API Examples ----- \n");

 deviceId = getDeviceId();
 if (!deviceId)
 {
 printf("No PXI IF with RMAP Target devices were found.\n");
 return 0;
 }

 /* Reset RMAP targets */
 reset(deviceId);

```

```

/* This example requires interface mode to be disabled */
CFG_disableInterfaceMode(deviceId);

/* Loop menu */
choice = MENU_CHOICE_INVALID;
do
{
 choice = getMenuChoice();
 switch (choice)
 {
 /* Manual authorisation example */
 case MENU_CHOICE_MANUAL_AUTH: manualAuthExample(deviceId); break;
 /* Notifications example */
 case MENU_CHOICE_NOTIFICATIONS: notificationsExample(deviceId); break;

 /* Handle unused cases */
 case MENU_CHOICE_INVALID: break;
 case MENU_CHOICE_EXIT: break;
 }
} while (choice != MENU_CHOICE_EXIT);

return 0;
}

```

## 10.36 rmw\_command\_packet\_example.c

This is an example of how to build RMAP read/modify/write commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmw_packet_library.h"

void rmw_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 U8 pMask[] = {0x0f, 0x0f, 0x0f, 0x0f};
 char status;

 /* Calculate the length of a read/modify/write command packet, */
 /* with 4 bytes of data and mask */
 fillPacketLenCalculated = RMAP_CalculateReadModifyWriteCommandPacketLength
 (
 2, 1, 8, 1);
 printf("The read/modify/write command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillReadModifyWriteCommandPacket(pTarget, 2, pReply,
 1, 0x20,
 0, 0x106, 0, 8, pData, pMask, &fillPacketLen, NULL, 1,
 (U8 *)pFillPacket, fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the read/modify/write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP read/modify/write command packet to write 4-bytes */
 pBuildPacket = RMAP_BuildReadModifyWriteCommandPacket(pTarget, 2,
 pReply, 1,
 0x20, 0, 0x106, 0, 8, pData, pMask, &buildPacketLen, NULL, 1);
 if (!pBuildPacket)
 {
 puts("Couldn't build the read/modify/write command packet");
 }
}

```

```

 free(pFillPacket);
 return;
 }

 /* Build an RMAP read/modify/write command packet */
 /* to write to a 4-byte register */
 pBuildPacketRegister = RMAP_BuildReadModifyWriteRegisterPacket(
 pTarget, 2,
 pReply, 1, 0x20, 0, 0x106, 0, 0x12345678, 0x0f0f0f0f,
 &buildPacketRegisterLen, NULL, 1);
 if (!pBuildPacketRegister)
 {
 puts("Couldn't build the read/modify/write register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
 }

 /* Check the lengths are all the same */
 if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen) ||
 (buildPacketLen != buildPacketRegisterLen))
 {
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketRegisterLen);
 }
 else
 {
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
 }

 /* Free the memory allocated for each of the packets. Note that the */
 /* memory allocated by the library should be freed by calling */
 /* RMAP_FreeBuffer. */
 RMAP_FreeBuffer(pBuildPacketRegister);
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
}

```

## 10.37 rmw\_reply\_packet\_example.c

This is an example of how to build RMAP read/modify/write replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmw_packet_library.h"

void rmw_reply_packet_example()
{
 void *pFillPacket, *pBuildPacket;
 unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
 U8 pReply[] = {254};
 U8 target = 254;
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 char status;

 /* Calculate the length of a read/modify/write reply packet, */
 /* with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateReadModifyWriteReplyPacketLength

```

```

 (1,
 4, 1);
printf("The read/modify/write reply packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

/* Allocate memory for the packet */
pFillPacket = malloc(fillPacketLenCalculated);
if (!pFillPacket)
{
 puts("Couldn't allocate the memory for the reply packet");
 return;
}

/* Fill the memory with the packet */
status = RMAP_FillReadModifyWriteReplyPacket(pReply, 1, target,
 RMAP_SUCCESS, 0, 4, pData, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,
 fillPacketLenCalculated);
if (!status)
{
 puts("Couldn't fill the read/modify/write reply packet");
 free(pFillPacket);
 return;
}

/* Build an RMAP read/modify/write reply packet */
pBuildPacket = RMAP_BuildReadModifyWriteReplyPacket(pReply, 1,
 target,
 RMAP_SUCCESS, 0, 4, pData, &buildPacketLen, NULL, 1);
if (!pBuildPacket)
{
 puts("Couldn't build the read/modify/write reply packet");
 free(pFillPacket);
 return;
}

/* Check the lengths are all the same */
if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen))
{
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
}
else
{
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
}

/* Free the memory allocated for each of the packets. Note that the */
/* memory allocated by the library should be freed by calling */
/* RMAP_FreeBuffer. */
RMAP_FreeBuffer(pBuildPacket);
free(pFillPacket);
}

```

## 10.38 router\_configuration\_example.c

This example demonstrates how to use the user router state configuration functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

#ifdef TRUE
#define TRUE 1
#endif
#ifdef FALSE

```



```

#define FALSE 0
#endif

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

void setInterruptCodeDistributionPorts(STAR_DEVICE_ID deviceID)
{
 int status = 0;
 int bcLegacyModeEnabled = 0;
 U32 portMask = 0;
 unsigned int portMaskBit = 0;

 /* Get broadcast controller legacy mode enabled flag */
 status = CFG_getBroadcastControllerLegacyModeEnabled(
 deviceID,
 &bcLegacyModeEnabled);

```

```

/* If successfully accessed broadcast controller legacy mode enabled flag */
if (status == CFG_STATUS_SUCCESS)
{
 /* If broadcast controller legacy mode is enabled */
 if (bcLegacyModeEnabled)
 {
 /* Disable broadcast controller legacy mode */
 /* i.e. enable interrupt code support */
 CFG_disableBroadcastControllerLegacyMode(deviceID);
 }

 /* Get interrupt code distribution ports */
 CFG_getInterruptCodeDistributionPorts(deviceID, &portMask);

 printf("\n\nInterrupt codes are being forwarded on ports: ");

 /* Test each bit in mask to see if it is set */
 for (portMaskBit = 1; portMaskBit < 32; portMaskBit++)
 {
 if ((1 << portMaskBit) & portMask)
 {
 printf(" %u", portMaskBit);
 }
 }

 /* Forward interrupt codes on ports 1, 2 and 3 only */
 CFG_setInterruptCodeDistributionPorts(deviceID, 0xE);
}
/* Else if function is not compatible with device type */
else if (status == CFG_STATUS_NOT_COMPATIBLE_WITH_THIS_DEVICE_TYPE)
{
 printf("\n\nCFG_getBroadcastControllerLegacyModeEnabled "
 "function unsupported by device");
}
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 U32 portMask;
 U8 tcFlagMode;
 U8 tcValue;
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 unsigned int i;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get Time-code output ports */
 CFG_getTimeCodeDistributionPorts(deviceID, &portMask);

 printf("\n\nTime-codes are being forwarded on ports: ");

 /* Test each bit in mask to see if it is set */
 for(i = 1; i < 32; i++)
 {
 if((1 << i) & portMask)
 printf(" %u", i);
 }

 /* Forward time codes on ports 1, 2 and 3 only */
 CFG_setTimeCodeDistributionPorts(deviceID, 0xE);

 /* Get time-code flags mode */
 CFG_getTimeCodeFlagMode(deviceID, &tcFlagMode);

 printf("\n\nTime-code flag interpretation mode: %s", tcFlagMode ?
 "Flags ignored" :
 "Time-code discarded on flag not \"00\"");

 /* Get current time-code value */
 CFG_getTimeCodeValue(deviceID, &tcValue);
 printf("\n\nCurrent time-code value: %u", tcValue);

 /* If supported, set interrupt code distribution ports */
 setInterruptCodeDistributionPorts(deviceID);

 /* Get Router global settings */
 CFG_getRouterGlobalSettings(deviceID, &globalState);

 printf("\n\nDisable on silence: %s", globalState.disableOnSilence ? "Enabled" : "
 Disabled");
 printf("\n\nSelf addressing: %s", globalState.enableSelfAddressing ? "Enabled" : "
 Disabled");
 printf("\n\nStart on request: %s", globalState.startOnRequest ? "Enabled" : "Disabled");
}

```

```

switch(globalState.timeoutMode)
{
case STAR_CFG_TIMEOUT_MODE_WATCHDOG:
 printf("\nWatchdog mode enabled.");
 break;
case STAR_CFG_TIMEOUT_MODE_BLOCKING:
 printf("\nWatchdog mode disabled.");
 break;
}

printf("\nTimeout period: ");
switch(globalState.timeoutPeriod)
{
case STAR_CFG_PORT_TIMEOUT_100US:
 printf("60-100us");
 break;
case STAR_CFG_PORT_TIMEOUT_1MS:
 printf("1.3ms");
 break;
case STAR_CFG_PORT_TIMEOUT_10MS:
 printf("10ms");
 break;
case STAR_CFG_PORT_TIMEOUT_100MS:
 printf("82ms");
 break;
case STAR_CFG_PORT_TIMEOUT_1S:
 printf("1.3s");
 break;
}
printf("\n");

/* Set Global settings */
globalState.disableOnSilence = FALSE;
globalState.enableSelfAddressing = TRUE;
globalState.startOnRequest = TRUE;
globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_WATCHDOG;
globalState.timeoutPeriod = STAR_CFG_PORT_TIMEOUT_10MS;

CFG_setRouterGlobalSettings(deviceID, &globalState);

return 0;
}

```

## 10.39 routerMk2S\_configuration\_example.c

Example usage of this API.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"
#include "ui.h"

int main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_FPGA_INFO fpgaInfo;
 char versionStr[STAR_CFG_MK2_VERSION_STR_MAX_LEN];
 char buildDateStr[STAR_CFG_MK2_BUILD_DATE_STR_MAX_LEN];
 int enabled;
 double precisionTxRate;

 STAR_DEVICE_TYPE aDeviceTypes[1];
 aDeviceTypes[0] = STAR_DEVICE_ROUTER_MK2S;

 /* Select device to configure */
 deviceID = STAR_UI_chooseDevice(aDeviceTypes, 1);
 if (!deviceID)
 {
 return 0;
 }

 STAR_getDeviceType(deviceID);

 /* Get hardware info*/
 CFG_getFPGAInfo(deviceID, &fpgaInfo);

 CFG_FPGAInfoToString(deviceID, &fpgaInfo, versionStr, buildDateStr);
}

```

```

/* Display the hardware info*/
printf("\nVersion: %s", versionStr);
printf("\nBuildDate: %s", buildDateStr);

/* Set general purpose register to 0xABCD */
CFG_setGeneralPurpose(deviceID, 0xABCD);

/* Flash the device's LEDs*/
CFG_identify(deviceID);

/* Set the device to be a time-code master*/
CFG_enableTimeCodeMaster(deviceID);

/* Set 2 second delay between time-codes */
CFG_setTimeCodePeriod(deviceID, 2000000);

/* Enable interface mode */
CFG_enableInterfaceMode(deviceID);

/* Enable interface mode on port 1 only */
CFG_enableInterfaceModeOnPort(deviceID, 1);

/* Disable interface mode on port 2 only */
CFG_disableInterfaceModeOnPort(deviceID, 2);

CFG_getInterfaceModeOnPortEnabled(deviceID, 2, &enabled);

printf("\nInterface mode %s on port 2", enabled ? "enabled" : "disabled");

/* Enable adding the source port number as a leading byte
 to received packets on ports 1 and 2 */

CFG_enableIdentifySource(deviceID);
CFG_enableIdentifySourceOnPort(deviceID, 1);
CFG_enableIdentifySourceOnPort(deviceID, 2);

/* Check if identify source is enabled for port 1 */
CFG_getIdentifySourceOnPortEnabled(deviceID, 1, &enabled);
printf("\nIdentify source %s on port 1", enabled ? "enabled" : "disabled");

CFG_disableIdentifySourceOnPort(deviceID, 1);

/* Test for precision transmit rate */
CFG_getPrecisionTransmitRateEnabled(deviceID, &enabled);
printf("\nPrecision transmit rate is %s", enabled ? "enabled" : "disabled");

/* Enable precision transmit rate */
CFG_enablePrecisionTransmitRate(deviceID);

/* Wait until precision transmit rate is in use. This can sometimes take
 250us */
do
{
 CFG_getPrecisionTransmitRateInUse(deviceID, &enabled);
}
while(!enabled);

/* Get current precision transmit rate */
CFG_getPrecisionTransmitRate(deviceID, &precisionTxRate);

printf("\nPrecision transmit rate is %f Mbit/s", precisionTxRate);

/* Set precision transmit rate */
CFG_setPrecisionTransmitRate(deviceID, 123.4);
CFG_getPrecisionTransmitRate(deviceID, &precisionTxRate);
printf("\nPrecision transmit rate is %f Mbit/s", precisionTxRate);

return 0;
}

```

## 10.40 routing\_table\_example.c

This example demonstrates the use of the routing table functions.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

#define BIT1 0x02
#define BIT3 0x08

```

```

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID * devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 STAR_CFG_GAR_ENTRY entry;
 U8 logicalAddress = 106;
 unsigned int i;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Get a routing table entry */
 CFG_getRoutingTableEntry(deviceID, logicalAddress, &entry);
}

```

```

/* Display routing table entry details */

printf("\nRouting table entry for address %u.\n", logicalAddress);

printf("\n\tAddress is %s.", entry.invalidAddress ? "invalid":"valid");
printf("\n\tHeader deletion %s.", entry.deleteHeader ? "enabled":"disabled");
printf("\n\tPriority: %s.", entry.priority ? "high":"low");

printf("\n\tOutput ports: ");

/* Test each bit in mask to see if it is set */
for(i = 1; i < 32; i++)
{
 if((1 << i) & entry.portMask)
 printf(" %u", i);
}
printf("\n");

/* Set a routing table entry:
 Logical address 44, Header deletion enabled, High priority
 Output ports: 1,3 */
entry.deleteHeader = 1;
entry.invalidAddress = 0;
entry.priority = 1;
entry.portMask = BIT1 | BIT3;

CFG_setRoutingTableEntry(deviceID, 44, &entry);

return 0;
}

```

## 10.41 simple\_receive\_example.c

Prompts the user for the device to use and the channel to receive into. The simple receive function is then used to wait for an incoming packet. The received packet contents are then printed.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void simpleReceiveExample()
{
 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_RECEIVE);

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_RECEIVE);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define eop type */
 STAR_EOP_TYPE eopType;

 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Print waiting on incoming packet message */
 printf("Waiting on incoming packet...\n");

 /* Receive packet on selected channel */
 status = STAR_receivePacket(selectedChannel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)

```

```

 {
 /* Prints the packet contents from the receive buffer */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }
 else
 {
 /* Switch on transfer status */
 switch(status)
 {
 /* Case cancelled */
 case STAR_TRANSFER_STATUS_CANCELLED:
 /* Print error when packet transfer was cancelled */
 printf("The packet was not received because the transfer "
 "was cancelled.\n");
 break;
 default:
 /* Print unexpected error when receiving packet */
 printf("An unexpected error occurred when receiving the "
 "packet.\n");
 break;
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
}
}

```

## 10.42 simple\_send\_and\_receive\_example.c

Creates a simple packet and sends it out of a channel and receives it into another channel. Uses the simple send and receive functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void simpleSendAndReceiveExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData) /
 sizeof(unsigned char);

 /* Transmit packet and wait indefinitely */
 STAR_TRANSFER_STATUS status =
 STAR_transmitPacket(sendChannel,
 packetData, packetLength, STAR_EOP_TYPE_EOP, -1);

 /* Close send channel */
 STAR_closeChannel(sendChannel);

 /* If packet was sent */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Open receive channel */
 STAR_CHANNEL_ID receiveChannel =
 STAR_openChannelToLocalDevice
 (firstDevice, STAR_CHANNEL_DIRECTION_IN, CHANNEL2, 1);
 }
 }
 }
}

```

```

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive
 buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Receive packet and wait indefinitely */
 status = STAR_receivePacket(receiveChannel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print packet contents */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }

 /* Close receive channel */
 STAR_closeChannel(receiveChannel);
 }
 }
}

```

## 10.43 simple\_send\_example.c

Creates a simple packet and sends it out of a channel. Uses the simple send and receive functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void simpleSendExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = NULL;

 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Allocate array of bytes to be populated */
 unsigned char bytes[256];
 U16 readBytesLength = 0;

 /* Ask user to enter a series of bytes to send */
 printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

 /* If valid bytes read input bytes from console */
 if(readBytes(bytes, 256, &readBytesLength))
 {
 /* Send packet out of channel */
 status = STAR_transmitPacket(selectedChannel, bytes,
 (unsigned int)readBytesLength, STAR_EOP_TYPE_EOP, 2);
 }
 }
}

```



```

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success */
 printf("\nPacket was sent successfully over channel %d.\n",
 selectedChannelNumber);
 }
 else
 {
 /* Print error */
 printf("\nPacket could not be sent.\n");
 }
 }

 /* Close channel after transmitting packet */
 STAR_closeChannel(selectedChannel);
}
else
{
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 10.44 simple\_two\_way\_example.c

Two packets are constructed and then sent in either direction using two way channels and the simple transmit functions.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void receiveSimplePacket(STAR_CHANNEL_ID channel)
{
 /* Define transfer status for receive status information */
 STAR_TRANSFER_STATUS status;

 /* Define receive buffer of maximum packet length */
 unsigned char receiveBuffer[MAX_PACKET_LENGTH];

 /* Initialise receive buffer length to size of receive
 buffer array */
 unsigned int receiveBufferLength = sizeof(receiveBuffer);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Receive packet and wait indefinitely */
 status = STAR_receivePacket(channel, receiveBuffer,
 &receiveBufferLength, &eopType, -1);

 /* If packet was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print packet contents */
 printPacketContents(receiveBuffer, receiveBufferLength);
 }
}

void simpleTwoWayExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* Define packet data to be sent out of first channel */
 unsigned char packetData1[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Define packet data to be sent out of second channel */
 unsigned char packetData2[] = {9, 8, 7, 6, 5, 4, 3, 2, 1};

 /* If there is a device to use */

```

```

if(firstDevice)
{
 /* Open first channel to send and receive on */
 STAR_CHANNEL_ID channel1 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* Open second channel to send and receive on */
 STAR_CHANNEL_ID channel2 = STAR_openChannelToLocalDevice
 (
 firstDevice, STAR_CHANNEL_DIRECTION_INOUT, CHANNEL2, 1);

 /* If both channels were opened */
 if(channel1 && channel2)
 {
 /* Get packet length */
 unsigned int packetLength = sizeof(packetData1) /
 sizeof(unsigned char);

 /* Define EOP type */
 STAR_EOP_TYPE eopType = STAR_EOP_TYPE_EOP;

 /* Transmit packet out of first channel */
 STAR_TRANSFER_STATUS status =
 STAR_transmitPacket(channel1,
 packetData1, packetLength, eopType, -1);

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on second channel */
 receiveSimplePacket(channel2);

 /* Get packet length */
 packetLength = sizeof(packetData2) / sizeof(unsigned char);

 /* Transmit packet out of second channel */
 status = STAR_transmitPacket(channel2, packetData2,
 packetLength, eopType, -1);

 /* If packet was transmitted */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Receive packet on first channel */
 receiveSimplePacket(channel1);
 }
 }

 /* Close first channel */
 STAR_closeChannel(channel1);

 /* Close second channel */
 STAR_closeChannel(channel2);
 }
}
}

```

## 10.45 spfi\_routing\_table\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Routing Table functions.

```

#include "cfg_api_spfi.h"

#define BIT1 0x02U
#define BIT3 0x08U

void getRoutingTableEntryExample(STAR_DEVICE_ID deviceID)
{
 U8 logicalAddress;
 SPFI_ROUTING_TABLE_ENTRY_FLAGS routingTableEntryFlags = 0U;
 U32 routingTableEntryPorts = 0U;

 printf("Get Routing Table Entry Example\n");
 printf("-----\n");

 /* Use logical address 106 */
 logicalAddress = 106U;

 /* Get routing table entry flags */
 if (CFG_SPFI_getRoutingTableEntryFlags(deviceID, logicalAddress,
 &routingTableEntryFlags) != CFG_STATUS_SUCCESS)

```

```

{
 /* Print error and exit example */
 printf("Failed to get routing table entry flags.\n");
 return;
}
else
{
 /* Get header deletion and enabled flags */
 U8 deleteHeaderEnabled =
 CFG_SPFI_isRoutingTableEntryDeleteHeaderEnabled(
 routingTableEntryFlags);
 U8 enabled = CFG_SPFI_isRoutingTableEntryEnabled(
 routingTableEntryFlags);

 printf("Header deletion %s for logical address %d.\n",
 deleteHeaderEnabled ? "enabled" : "disabled", logicalAddress);
 printf("Routing table entry %s for logical address %d.\n",
 enabled ? "enabled" : "disabled", logicalAddress);
}

/* Get routing table entry ports */
if (CFG_SPFI_getRoutingTableEntryPorts(deviceID, logicalAddress,
 &routingTableEntryPorts) != CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to get routing table entry ports.\n");
 return;
}
else
{
 printf("Output ports for logical address %d:", logicalAddress);

 /* If at least one routing table entry output port is enabled */
 if (routingTableEntryPorts)
 {
 U32 outputPort;

 /* Test each bit in mask to see if it is set */
 for (outputPort = 1U; outputPort < 32U; outputPort++)
 {
 if ((1U << outputPort) & routingTableEntryPorts)
 {
 printf(" %u", outputPort);
 }
 }
 }
 else
 {
 printf(" none");
 }
 printf("\n");
}

printf("\n");
}

void setRoutingTableEntryExample(STAR_DEVICE_ID deviceID)
{
 U8 logicalAddress;
 U8 enablePorts;

 printf("Set Routing Table Entry Example\n");
 printf("-----\n");

 /* Use logical address 106 */
 logicalAddress = 106U;

 /* Enable ports 1 and 3 */
 enablePorts = BIT1 | BIT3;

 /* Disable routing table entry header deletion */
 if (CFG_SPFI_disableRoutingTableEntryDeleteHeader(deviceID,
 logicalAddress) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable header deletion.\n");
 return;
 }
 else
 {
 printf("Header deletion disabled for logical address %u.\n",
 logicalAddress);
 }

 /* Disable routing table entry */
 if (CFG_SPFI_disableRoutingTableEntry(deviceID,

```

```

 logicalAddress) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable routing table entry.\n");
 return;
 }
 else
 {
 printf("Routing table entry disabled for logical address %u.\n",
 logicalAddress);
 }

 /* Enable routing table entry header deletion */
 if (CFG_SPFI_enableRoutingTableEntryDeleteHeader(deviceID,
 logicalAddress) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable header deletion.\n");
 return;
 }
 else
 {
 printf("Header deletion enabled for logical address %u.\n",
 logicalAddress);
 }

 /* Enable routing table entry */
 if (CFG_SPFI_enableRoutingTableEntry(deviceID,
 logicalAddress) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable routing table entry.\n");
 return;
 }
 else
 {
 printf("Routing table entry enabled for logical address %u.\n",
 logicalAddress);
 }

 /* Set routing table entry ports */
 if (CFG_SPFI_setRoutingTableEntryPorts(deviceID, logicalAddress,
 enablePorts) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set routing table entry output ports.\n");
 return;
 }
 else
 {
 U32 outputPort;

 printf("Routing table entry output ports set to");
 for (outputPort = 1U; outputPort < 32U; outputPort++)
 {
 if ((1U << outputPort) & enablePorts)
 {
 printf(" %u", outputPort);
 }
 }
 printf(" for logical address %u.\n", logicalAddress);
 }

 printf("\n");
}

void routingTableExample(STAR_DEVICE_ID deviceID)
{
 printf("Routing Table Example\n");
 printf("=====\n");

 /* Set routing table entry example */
 setRoutingTableEntryExample(deviceID);

 /* Get routing table entry example */
 getRoutingTableEntryExample(deviceID);
}

```

## 10.46 star-test.c

This file contains the bulk of the functions used by the STAR-System Test program. This is provided as an example for developers wishing to write programs using STAR-API.

```
#include "star-test.h"
#include "star-api.h"
#include "cfg_api_generic.h"

#define VERSION_INFO "star-system_test v2.0"

#include <errno.h>

/* Menu option codes */
#define MENU_DISPLAY_INFORMATION 1
#define MENU_LOOPBACK_SINGLE 2
#define MENU_LOOPBACK_MULTI 3
#define MENU_LOOPBACK_DOUBLE_SINGLE 4
#define MENU_LOOPBACK_DOUBLE_MULTI 5

#define MENU_TRANSMIT_SINGLE 6
#define MENU_TRANSMIT_MULTI 7
#define MENU_RECEIVE_SINGLE 8
#define MENU_RECEIVE_MULTI 9
#define MENU_TRANSMIT_FILE_WHOLE 10
#define MENU_RECEIVE_FILE_WHOLE 11
#define MENU_TRANSMIT_FILE_SPLIT 12
#define MENU_RECEIVE_FILE_SPLIT 13
#define MENU_RESET_DEVICE 14
#define MENU_IDENTIFY_DEVICE 15

#define MENU_EXIT 0

#define STAR_INFINITE -1

void DisplayMenu(void)
{
 puts("\nSelect Option:");
 printf(" (%d) Display Device and Version Information\n",
 MENU_DISPLAY_INFORMATION);
 printf(" (%d) Loopback Test, Single Packet (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_SINGLE);
 printf(" (%d) Loopback Test, Multiple Packets (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_MULTI);
 printf(" (%d) Double Loopback Test, Single Packet (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_DOUBLE_SINGLE);
 printf(" (%d) Double Loopback Test, Multiple Packets (Data Compare, Link Speed)\n",
 MENU_LOOPBACK_DOUBLE_MULTI);
 printf(" (%d) Transmit Single Packet (Link Speed)\n", MENU_TRANSMIT_SINGLE);
 printf(" (%d) Transmit Multiple Packets (Link Speed)\n",
 MENU_TRANSMIT_MULTI);
 printf(" (%d) Receive Packet (Link Speed)\n", MENU_RECEIVE_SINGLE);
 printf(" (%d) Receive Multiple Packets (Link Speed)\n", MENU_RECEIVE_MULTI);
 printf(" (%d) Transmit From File, Single Packet\n", MENU_TRANSMIT_FILE_WHOLE);
 printf(" (%d) Receive To File, Single Packet\n", MENU_RECEIVE_FILE_WHOLE);
 printf(" (%d) Transmit From File, Multiple Packets\n", MENU_TRANSMIT_FILE_SPLIT);
 printf(" (%d) Receive To File, Multiple Packets (Link Speed)\n", MENU_RECEIVE_FILE_SPLIT);
 printf(" (%d) Reset Device\n", MENU_RESET_DEVICE);
 printf(" (%d) Identify Device\n", MENU_IDENTIFY_DEVICE);
 printf(" (%d) Exit\n", MENU_EXIT);
 printf("Please Select Menu Option: ");
 fflush(stdout);
}

void DisplayInformation(void)
{
 STAR_VERSION_INFO *versions;
 STAR_CFG_FPGA_INFO fpgaVersion;
 int getFpgaResult = 0;

 U32 versionCount = 0U;

 STAR_DEVICE_ID* devices;
 U32 deviceCount = 0U;

 char *deviceStr;
 STAR_CHANNEL_MASK channelMask;
 unsigned int deviceNum, i;

 STAR_CFG_DEVICE_IDENTIFIER_INFO deviceInfo;
 char manufacturerStr[STAR_CFG_MANUFACTURER_STR_MAX_LEN];
 char deviceTypeStr[STAR_CFG_DEVICE_STR_MAX_LEN];
 STAR_DEVICE_TYPE configCapabilities;
```

```

puts("\n\nMODULE VERSION INFORMATION");
puts("=====");

/* Get the list of module versions */
versions = STAR_getAllVersions(&versionCount);

/* If there are modules present */
if (versions != NULL)
{
 /* For each module version item in the list */
 for (i = 0U; i < versionCount; i++)
 {
 /* Display the module's version information */
 printf(" Module name: %s\n", versions[i].name);
 printf(" Module author: %s\n", versions[i].author);
 printf(" Module version: v%u.%02u", versions[i].major,
 versions[i].minor);
 if (versions[i].edit != 0U)
 {
 printf(" (%u)", versions[i].edit);
 }
 if (versions[i].patch != 0U)
 {
 printf(" p%u", versions[i].patch);
 }
 puts("\n");
 }

 /* Dispose of the version list */
 STAR_destroyVersionInfoList(versions);
}
else
{
 puts("No modules present.\n");
}

puts("\n\nDEVICE INFORMATION");
puts("=====");

/* Get the list of devices */
devices = STAR_getDeviceList(&deviceCount);

/* If there are devices present */
if (devices != NULL)
{
 /* For each device in the list */
 for (deviceNum = 0U; deviceNum < deviceCount; deviceNum++)
 {
 /* Get and display the device's name */
 deviceStr = STAR_getDeviceName(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Name: Unable to get device name!");
 }
 else
 {
 printf(" Name: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 /* Get and display the device's type */
 deviceStr = STAR_getDeviceTypeAsString(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Device type: Unable to get device type!");
 }
 else
 {
 printf(" Device type: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 deviceStr = STAR_getDeviceBusTypeAsString(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Bus type: Unable to get bus type!");
 }
 else
 {
 printf(" Bus type: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 /* Get and display the device's firmware version */
 versions = STAR_getDeviceFirmwareVersion(devices[deviceNum]);
 if (versions == NULL)

```

```

 {
 puts("Firmware version: Unable to get the firmware version!");
 }
 else
 {
 printf("Firmware version: v%u.%u", versions->major,
 versions->minor);
 if (versions->edit != 0U)
 {
 printf("(%u)", versions->edit);
 }
 if (versions->patch != 0U)
 {
 printf("p%u", versions->patch);
 }
 puts("");
 STAR_destroyVersionInfo(versions);
 }

 /* If the device can be configured */
 configCapabilities = STAR_getDeviceConfigCapabilities(
 devices[deviceNum]);
 if (configCapabilities == STAR_DEVICE_CONFIG_SUPPORTED)
 {
 /* Get and display the device's FPGA version */
 getFpgaResult = CFG_getFPGAInfo(devices[deviceNum],
 &fpgaVersion);
 if (!CFG_SUCCESS(getFpgaResult))
 {
 puts(" FPGA version: Unable to get the FPGA version!");
 }
 else
 {
 printf(" FPGA version: v%u.%02u", fpgaVersion.major,
 fpgaVersion.minor);
 if (fpgaVersion.edit != 0U)
 {
 printf("(%u)", fpgaVersion.edit);
 }
 if (fpgaVersion.patch != 0U)
 {
 printf("p%u", fpgaVersion.patch);
 }
 puts("");
 }
 }

 /* Get and display the device's serial number */
 deviceStr = STAR_getDeviceSerialNumber(devices[deviceNum]);
 if (deviceStr == NULL)
 {
 puts(" Serial number: Unable to get serial number!");
 }
 else
 {
 printf(" Serial number: %s\n", deviceStr);
 STAR_destroyString(deviceStr);
 }

 /* Get and display the device's index */
 printf(" Index: %d\n",
 STAR_getDeviceIndex(devices[deviceNum]));

 /* Get and display the device's channels */
 printf(" Channels:");
 channelMask = STAR_getDeviceChannels(devices[deviceNum]);
 if (channelMask == 0U)
 {
 puts(" None");
 }
 else
 {
 for (i = 0U; i < 32U; i++)
 {
 if (((channelMask >> i) & 1U) != 0U)
 {
 printf(" %u", i);
 }
 }
 printf("\n");
 }

 if (configCapabilities == STAR_DEVICE_CONFIG_SUPPORTED)
 {
 memset(&deviceInfo, 0, sizeof(deviceInfo));
 if (!CFG_SUCCESS(CFG_getDeviceIdentificationInfo(
 devices[deviceNum],

```

```

 &deviceInfo)))
 {
 strcpy(manufacturerStr, "Unable to get the manufacturer!");
 strcpy(deviceTypeStr, "Unable to get the chip type!");
 }
 else
 {
 CFG_getDeviceManufacturerAsString(&deviceInfo,
 manufacturerStr);
 CFG_getDeviceTypeAsString(&deviceInfo,
 deviceTypeStr);
 }
 printf(" Manufacturer ID: %u\t(%s)\n",
 deviceInfo.manufacturerID, manufacturerStr);
 printf(" Chip Type: %u\t(%s)\n", deviceInfo.chipType,
 deviceTypeStr);
}
puts("\n");
}

/* Dispose of the device list */
STAR_destroyDeviceList(devices);
}
else
{
 puts("No devices present.\n");
}

puts("\n");
}

STAR_DEVICE_ID chooseDevice(const char * const descriptionStr,
 const STAR_DEVICE_TYPE deviceType)
{
 STAR_DEVICE_ID* devices;
 U32 deviceCount = 0U, i;
 unsigned int chosen;
 int status;

 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

 /* Get the list of devices of the specified type */
 devices = STAR_getDeviceListForType(deviceType, &deviceCount);

 /* If there are no devices present */
 if (devices == NULL)
 {
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return STAR_DEVICE_UNKNOWN;
 }

 /* Display the devices detected */
 if (deviceCount == 1U)
 {
 printf("One %s device detected:", descriptionStr);
 }
 else
 {
 printf("%u %s devices detected:\n", deviceCount, descriptionStr);
 }

 /* For each device */
 for (i = 0U; i < deviceCount; i++)
 {
 /* Display its name */
 if (devices[i] != STAR_DEVICE_UNKNOWN)
 {
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName != NULL)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
 }

 /* If there's only 1 device on the list */

```



```

if (deviceCount == 1U)
{
 /* Return that device */
 deviceId = devices[0U];
 STAR_destroyDeviceList(devices);
 return deviceId;
}

/* Ask the user which device to use */
printf("Please select which %s device to use: ", descriptionStr);
fflush(stdout);
if (fgetc(s, 256, stdin) == NULL)
{
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return STAR_DEVICE_UNKNOWN;
}
status = sscanf(s, "%u", &chosen);
if ((status == 0) || (chosen > (deviceCount - 1U)))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return STAR_DEVICE_UNKNOWN;
}

/* Return the chosen device */
deviceId = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceId;
}

unsigned char chooseChannel(const char * const descriptionStr,
 const STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_MASK channelMask;
 unsigned int channelNumber;
 unsigned char channelCount;
 int status;
 unsigned char i;
 char s[256];

 /* Get the channels present on the device */
 channelMask = STAR_getDeviceChannels(deviceId);
 if ((channelMask == 0U) || (channelMask == 1U))
 {
 printf("ERROR: The %s device doesn't appear to have any valid channels.\n",
 descriptionStr);
 return 0U;
 }

 /* Determine the number of channels on the device, */
 /* ignoring the configuration channel */
 channelCount = 0U;
 channelNumber = 0U;
 for (i = 1U; i < 32U; i++)
 {
 if (((channelMask >> i) & 1U) != 0U)
 {
 channelCount++;
 channelNumber = i;
 }
 }

 /* If there's only one channel on the device, use this channel */
 if (channelCount == 1U)
 {
 printf("Using channel %u as the %s channel, as this is the only channel on the device.\n",
 channelNumber, descriptionStr);
 return (unsigned char)channelNumber;
 }

 /* Display the available channels */
 printf("Enter %s channel (", descriptionStr);
 for (i = 1U; i < 32U; i++)
 {
 /* If the channel exists */
 if (((channelMask >> i) & 1U) != 0U)
 {
 printf("%u", i);
 channelCount--;
 if (channelCount == 1U)
 {
 printf(" or ");
 }
 else if (channelCount > 1U)
 {
 printf(", ");
 }
 }
 }
}

```

```

 }
 else
 {
 break;
 }
 }
}
printf("): ");
fflush(stdout);

/* Read in the channel to use */
if (fgets(s, 256, stdin) == NULL)
{
 puts("\nERROR: No channel specified");
 return 0U;
}
status = sscanf(s, "%u", &channelNumber);
if ((status == 0) || ((1U << channelNumber) & channelMask) == 0U)
{
 puts("\nERROR: The channel specified is not present");
 return 0U;
}

return (unsigned char)channelNumber;
}

int chooseDeviceAndChannel(const char * const descriptionStr,
 STAR_CHANNEL_ID * const pChannelId,
 const STAR_CHANNEL_DIRECTION direction)
{
 STAR_DEVICE_ID deviceId;
 unsigned char channelNumber;

 deviceId = chooseDevice(descriptionStr, STAR_DEVICE_TXRX_SUPPORTED);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return 0;
 }

 channelNumber = chooseChannel(descriptionStr, deviceId);
 if (channelNumber == 0U)
 {
 return 0;
 }

 /* Open the channel */
 *pChannelId = STAR_openChannelToLocalDevice(deviceId, direction,
 channelNumber, TRUE);
 if ((*pChannelId) == 0U)
 {
 printf("\nERROR: Unable to open %s channel\n", descriptionStr);
 return 0;
 }

 puts("");

 return 1;
}

STAR_SPACEWIRE_ADDRESS *getTransmitPathAddress()
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0U;
 char *pos = (char *)s;
 unsigned long value;

 /* Ask the user to enter the path to add to the front of packets */
 puts("Enter an optional path to add to the front of the packets to be sent");
 puts("(Values should be in hex, separated by a space, i.e. 01 0f 02):");

 /* Read in the path */
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("No address entered");
 return NULL;
 }

 /* Read until end of buffer or line feed */
 while ((*pos != '\0') && (pathLen < 256U) && (*pos != '\n'))
 {
 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in address");

```

```

 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
}

/* Create a SpaceWire address from the path */
pAddress = STAR_createAddress(newPath, pathLen);

/* Return the completed address */
return pAddress;
}

int getPacketSize(unsigned long * const pPacketSize)
{
 char s[256];
 int status;

 printf("Enter buffer size for each packet in bytes: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No buffer size specified");
 return 0;
 }
 status = sscanf(s, "%lu", pPacketSize);
 if ((status == 0) || (*pPacketSize == 0U))
 {
 puts("\nERROR: Invalid buffer size specified");
 return 0;
 }

 return 1;
}

int chooseCheckData(int *pCompare)
{
 char s[256];

 printf("Would you like to check received data for errors? (y/n): ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("ERROR: No value selected");
 return 0;
 }
 if ((s[0U] == 'y') || (s[0U] == 'Y') || (s[0U] == 't') || (s[0U] == 'T') ||
 (s[0U] == '1'))
 {
 *pCompare = 1;
 }
 else if ((s[0U] == 'n') || (s[0U] == 'N') || (s[0U] == 'f') ||
 (s[0U] == 'F') || (s[0U] == '0'))
 {
 *pCompare = 0;
 }
 else
 {
 puts("ERROR: Invalid value selected");
 return 0;
 }

 return 1;
}

int getLoopCount(unsigned long * const pLoopCount)
{
 char s[256];
 int status;

 printf("Enter the number of times to run the test: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No loop count specified");
 return 0;
 }
 status = sscanf(s, "%lu", pLoopCount);
 if ((status == 0) || (*pLoopCount == 0U))
 {
 puts("\nERROR: Invalid loop count specified");
 return 0;
 }

 return 1;
}

```

```

int getPacketCount(unsigned long * const pPacketCount)
{
 char s[256];
 int status;

 printf("Enter the number of packets to transmit/receive in the test: ");
 fflush(stdout);
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No packet count specified");
 return 0;
 }
 status = sscanf(s, "%lu", pPacketCount);
 if ((status == 0) || (*pPacketCount == 0U))
 {
 puts("\nERROR: Invalid packet count specified");
 return 0;
 }

 return 1;
}

unsigned long comparePackets(STAR_TRANSFER_OPERATION * const pTransferOp,
 const unsigned long packetCount, const unsigned long packetSize,
 char * const pBuffer)
{
 unsigned long errorCount = 0U, i;
 unsigned int rxPacketCount;
 STAR_STREAM_ITEM* pRxStreamItem = NULL;
 char *pRxBuffer;
 U32 rxPacketLength;
 U32 bRestart;

 /* Get the number of traffic items received */
 rxPacketCount = STAR_getTransferItemCount(pTransferOp);
 if (rxPacketCount != packetCount)
 {
 printf("\nERROR expected to receive %lu packets but received %u.\n",
 packetCount, rxPacketCount);
 errorCount++;
 }
 else
 {
 bRestart = TRUE;
 /* For each traffic item received */
 for (i = 0U; i < rxPacketCount; i++)
 {
 if (bRestart == TRUE)
 {
 pRxStreamItem = STAR_getTransferItem(pTransferOp, i);
 bRestart = FALSE;
 }
 else
 {
 pRxStreamItem = STAR_getNextTransferItem(pRxStreamItem);
 }

 /* Get the packet */
 if ((pRxStreamItem == NULL) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET) ||
 (pRxStreamItem->item == NULL))
 {
 printf("\nERROR received an unexpected traffic type, or empty traffic item in item %lu\n",
 i);
 errorCount++;
 bRestart = TRUE;
 }
 else
 {
 pRxBuffer = (char *)STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)pRxStreamItem->
item,
 &rxPacketLength);
 if ((pRxBuffer == NULL) || (rxPacketLength != packetSize))
 {
 printf("\nERROR received a packet of length %u, expected length %lu in item %lu\n",
 rxPacketLength, packetSize, i);
 errorCount++;
 }
 else
 {
 /* Compare the buffers and increment the error count if */
 /* the buffers do not match */
 errorCount += BufferCompareChar(pBuffer + (packetSize * i),
 pRxBuffer, packetSize);
 }
 }
 }
 }
}

```

```

 if (pRxBuffer != NULL)
 {
 STAR_destroyPacketData((unsigned char *)pRxBuffer);
 }
 }
}

/* Return the error count */
return errorCount;
}

unsigned long getPacketLengths(STAR_TRANSFER_OPERATION * const pTransferOp,
 const unsigned long packetCount)
{
 unsigned long rxPacketLength = 0U, i;
 unsigned int rxPacketCount;
 STAR_STREAM_ITEM *pRxStreamItem;

 /* Get the number of traffic items received */
 rxPacketCount = STAR_getTransferItemCount(pTransferOp);
 if (rxPacketCount != packetCount)
 {
 printf("\nERROR expected to receive %lu packets but received %u.\n",
 packetCount, rxPacketCount);
 }

 /* For each traffic item received */
 for (i = 0U; i < rxPacketCount; i++)
 {
 /* Get the packet */
 pRxStreamItem = STAR_getTransferItem(pTransferOp, i);
 if ((pRxStreamItem == NULL) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET) ||
 (pRxStreamItem->item == NULL))
 {
 printf("\nERROR received an unexpected traffic type, or empty traffic item in item %lu\n",
 i);
 }
 else
 {
 rxPacketLength += STAR_getPacketLength(
 (STAR_SPACEWIRE_PACKET *)pRxStreamItem->
 item);
 }
 }

 /* Return the total packet length */
 return rxPacketLength;
}

void DisplayResults(const clock_t start, const clock_t finish,
 const int compared, const unsigned long byteSize,
 const unsigned long packetCount, const unsigned long loopCount,
 const unsigned long errorCount, const char * const descriptionStr)
{
 double bitsSent, duration;

 puts("Test complete.");

 /* Calculate the time taken and the number of bits sent */
 duration = (double)(finish - start) / TIME_DIVIDER;
 bitsSent = (double)byteSize * (double)8 * (double)packetCount *
 (double)loopCount;

 /* Display the data rates */
 printf("\n**** %s, Results **** \n", descriptionStr);
 printf("\tTime Taken = %-9.5g Seconds\n", duration);
 if (duration > 0)
 {
 printf("\tAverage Speed = %-9.2E bit/s (%-3.2f Mbit/s)\n\n",
 bitsSent / duration, (bitsSent / duration) / 1000000.0);
 }
 else
 {
 printf("\tAverage Speed = N/A (time taken too small)\n\n");
 }

 /* Display the number of errors encountered */
 if (compared != 0)
 {
 printf("Total Errors = 0x%-7lx \n", errorCount);
 }

 /* Display whether the test was successful */
 if (errorCount == 0U)
 {

```

```

 puts("Test successful.\n");
 }
 else
 {
 puts("Test failed.\n");
 }
}

void LoopBack_SinglePacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U, txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL, *pRxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 unsigned long loopCount = 0U, errorCount = 0U, byteSize = 0U, testNum;
 clock_t start, finish;
 int compare;
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Compare? */
 if (chooseCheckData(&compare) == 0)
 {
 goto LoopBack_SinglePacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBack_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer, byteSize, DATA_TYPE_RANDOM);

 /* Create receive operation to receive 1 packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBack_SinglePacket_Finish;
 }

 /* Create the packet to be transmitted */
 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, byteSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto LoopBack_SinglePacket_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
 if (pTxTransferOp == NULL)

```

```

 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBack_SinglePacket_Finish;
 }

 puts("Running Single-Packet Loopback Test...");

 /* Start time */
 start = GET_TIME();

 for (testNum = 0U; testNum < loopCount; testNum++)
 {
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_SinglePacket_Finish;
 }

 /* If the received packet is to be compared */
 if (compare != 0)
 {
 /* Compare the received packet to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp, 1U, byteSize,
 pTxBuffer);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, compare, byteSize, 1U, loopCount, errorCount,
 "Single-Packet Loopback Test");

LoopBack_SinglePacket_Finish:

 /* Dispose of the transfer operations */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }
 if (pRxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pRxTransferOp);
 }

 /* Destroy the packet transmitted */
 if (pTxStreamItem != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

```

```

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address path */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channels */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void LoopBack_MultiPacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U, txChannelId = 0U;
 unsigned long loopCount, errorCount = 0U, byteSize, testNum;
 unsigned long i, packetCount = 0U;
 int compare;
 char *pTxBuffer = NULL;
 STAR_STREAM_ITEM **pTxStreamItems = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL, *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Compare? */
 if (chooseCheckData(&compare) == 0)
 {
 goto LoopBack_MultiPacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize * packetCount);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBack_MultiPacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer, byteSize * packetCount, DATA_TYPE_RANDOM);
}

```



```

/* Create receive operation to receive the packets */
pRxTransferOp = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
if (pRxTransferOp == NULL)
{
 puts("\nERROR: Unable to create receive operation");
 goto LoopBack_MultiPacket_Finish;
}

/* Allocate memory for the transmit stream item pointers */
ppTxStreamItems = (STAR_STREAM_ITEM **) calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
if (ppTxStreamItems == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto LoopBack_MultiPacket_Finish;
}

/* For each packet */
for (i = 0U; i < packetCount; i++)
{
 /* Create the stream item */
 ppTxStreamItems[i] = STAR_createPacket(pAddressPath,
 (U8 *)pTxBuffer + (byteSize * i), byteSize, STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i] == NULL)
 {
 printf("\nERROR: Unable to create packet %lu to be transmitted\n", i);
 goto LoopBack_MultiPacket_Finish;
 }
}

/* Create the transmit transfer operation for the packets */
pTxTransferOp = STAR_createTxOperation(ppTxStreamItems, packetCount);
if (pTxTransferOp == NULL)
{
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBack_MultiPacket_Finish;
}

puts("Running Multi-Packet Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId,
 pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBack_MultiPacket_Finish;
 }

 /* If the received packets are to be compared */

```

```

 if (compare != 0)
 {
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp, packetCount, byteSize,
 pTxBuffer);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, compare, byteSize, packetCount, loopCount,
 errorCount, "Multiple-Packet Loopback Test");
}

LoopBack_MultiPacket_Finish:

/* Dispose of the transfer operations */
if (pTxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pTxTransferOp);
}
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Destroy the packets transmitted */
if (ppTxStreamItems != NULL)
{
 for (i = 0U; i < packetCount; i++)
 {
 if (ppTxStreamItems[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i]);
 }
 }
 free(ppTxStreamItems);
}

/* Free the transmit buffer */
if (pTxBuffer != NULL)
{
 free(pTxBuffer);
}

/* Free the address path */
if (pAddressPath != NULL)
{
 STAR_destroyAddress(pAddressPath);
}

/* Close the opened channels */
if (rxChannelId != 0U)
{
 STAR_closeChannel(rxChannelId);
}
if (txChannelId != 0U)
{
 STAR_closeChannel(txChannelId);
}
}

void LoopBackDouble_SinglePacket(void)
{
 STAR_CHANNEL_ID channelId[2] = { 0U, 0U };
 unsigned long loopCount, errorCount = 0U, byteSize, testNum, i;
 int compare;
 char *pTxBuffer[2] = { NULL, NULL };
 STAR_STREAM_ITEM *pTxStreamItem[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pRxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTransferOpList[2][2] = { {NULL} };
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath[2] = { NULL, NULL };

 /* Select the first device and channel to be used */
 if (chooseDeviceAndChannel("first", &channelId[0U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {
 return;
 }
}

```

```

/* Select the first path to be added to the front of packets transmitted */
pAddressPath[0U] = getTransmitPathAddress();

/* Select the second device and channel to be used */
if (chooseDeviceAndChannel("second", &channelId[1U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Select the second path to be added to the front of packets transmitted */
pAddressPath[1U] = getTransmitPathAddress();

/* Get packet size */
if (getPacketSize(&byteSize) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Get loop count */
if (getLoopCount(&loopCount) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* Compare? */
if (chooseCheckData(&compare) == 0)
{
 goto LoopBackDouble_SinglePacket_Finish;
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Allocate memory for the transmit buffer */
 pTxBuffer[i] = (char *)calloc(1U, byteSize);
 if (pTxBuffer[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer[i], byteSize, DATA_TYPE_RANDOM);

 /* Create receive operation to receive 1 packet */
 pRxTransferOp[i] = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Create the packet to be transmitted */
 pTxStreamItem[i] = STAR_createPacket(pAddressPath[i],
 (U8 *)pTxBuffer[i], byteSize, STAR_EOP_TYPE_EOP);
 if (pTxStreamItem[i] == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp[i] = STAR_createTxOperation(&pTxStreamItem[i], 1U);
 if (pTxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Add transfer operation to list for channel */
 pTransferOpList[i][0U] = pRxTransferOp[i];
 pTransferOpList[i][1U] = pTxTransferOp[i];
}

puts("Starting Single-Packet Double Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operations */

 /* First Channel Rx */
 if (STAR_submitTransferOperation(channelId[0U],

```

```

 pTransferOpList[0U][0U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
 /* Second Channel Rx */
 if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][0U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* Submit the transmit operations */

 /* First Channel Tx */
 if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
 /* Second Channel Tx */
 if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp[i],
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp[i],
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_SinglePacket_Finish;
 }
 }

 if (compare != 0)
 {
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp[1U], 1U, byteSize,
 pTxBuffer[0U]);
 errorCount += comparePackets(pRxTransferOp[0U], 1U, byteSize,
 pTxBuffer[1U]);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, 1U, loopCount, errorCount,
 "Single-Packet Double Loopback Test");

LoopBackDouble_SinglePacket_Finish:

 /* For each channel */

```

```

for (i = 0U; i < 2U; i++)
{
 /* Dispose of the transfer operations */
 STAR_disposeTransferOperation(pTransferOpList[i][0U]);
 STAR_disposeTransferOperation(pTransferOpList[i][1U]);

 /* Destroy the packet transmitted */
 if (pTxStreamItem[i] != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem[i]);
 }

 /* Free the transmit buffer */
 if (pTxBuffer[i] != NULL)
 {
 free(pTxBuffer[i]);
 }

 /* Free the address path */
 if (pAddressPath[i] != NULL)
 {
 STAR_destroyAddress(pAddressPath[i]);
 }

 /* Close the channel */
 if (channelId[i] != 0U)
 {
 STAR_closeChannel(channelId[i]);
 }
}

void LoopBackDouble_MultiPacket(void)
{
 STAR_CHANNEL_ID channelId[2] = { 0U, 0U };
 unsigned long loopCount, errorCount = 0U, byteSize, testNum, i;
 unsigned long packetNum, packetCount = 0U;
 int compare;
 char *pTxBuffer[2] = { NULL, NULL };
 STAR_STREAM_ITEM **ppTxStreamItems[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pRxTransferOp[2] = { NULL, NULL };
 STAR_TRANSFER_OPERATION *pTransferOpList[2][2] = { {NULL} };
 STAR_TRANSFER_STATUS rxStatus, txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath[2] = { NULL, NULL };

 /* Select the first device and channel to be used */
 if (chooseDeviceAndChannel("first", &channelId[0U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {
 return;
 }

 /* Select the first path to be added to the front of packets transmitted */
 pAddressPath[0U] = getTransmitPathAddress();

 /* Select the second device and channel to be used */
 if (chooseDeviceAndChannel("second", &channelId[1U],
 STAR_CHANNEL_DIRECTION_INOUT) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Select the second path to be added to the front of packets transmitted */
 pAddressPath[1U] = getTransmitPathAddress();

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Compare? */
 if (chooseCheckData(&compare) == 0)

```

```

{
 goto LoopBackDouble_MultiPacket_Finish;
}

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Allocate memory for the transmit buffer */
 pTxBuffer[i] = (char *)calloc(1U, byteSize * packetCount);
 if (pTxBuffer[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Fill transmit buffer with random data */
 FillBufferRandomChar(pTxBuffer[i], byteSize * packetCount,
 DATA_TYPE_RANDOM);

 /* Create receive operation to receive the packets */
 pRxTransferOp[i] = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Allocate memory for the transmit stream item pointers */
 ppTxStreamItems[i] = (STAR_STREAM_ITEM **)calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
 if (ppTxStreamItems[i] == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* For each packet */
 for (packetNum = 0U; packetNum < packetCount; packetNum++)
 {
 /* Create the packet to be transmitted */
 ppTxStreamItems[i][packetNum] = STAR_createPacket(pAddressPath[i],
 (U8 *)pTxBuffer[i] + (byteSize * packetNum), byteSize,
 STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i][packetNum] == NULL)
 {
 printf("\nERROR: Unable to create the packet %lu to be transmitted\n",
 packetNum);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 /* Create the transmit transfer operation for the packets */
 pTxTransferOp[i] = STAR_createTxOperation(ppTxStreamItems[i],
 packetCount);
 if (pTxTransferOp[i] == NULL)
 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Add transfer operation to list for channel */
 pTransferOpList[i][0U] = pRxTransferOp[i];
 pTransferOpList[i][1U] = pTxTransferOp[i];
}

puts("Running Multi-Packet Double Loopback Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operations */

 /* First Channel Rx */
 if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][0U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Second Channel Rx */
 if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][0U]) == 0)
 {

```

```

 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* Submit the transmit operations */

 /* First Channel Tx */
 if (STAR_submitTransferOperation(channelId[0U],
 pTransferOpList[0U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 /* Second Channel Tx */
 if (STAR_submitTransferOperation(channelId[1U],
 pTransferOpList[1U][1U]) == 0)
 {
 printf("\nERROR occurred submitting operation list. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp[i],
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 /* For each channel */
 for (i = 0U; i < 2U; i++)
 {
 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp[i],
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed - status %d.\n",
 testNum + 1U, rxStatus);
 goto LoopBackDouble_MultiPacket_Finish;
 }
 }

 if (compare != 0)
 {
 /* Compare the received packets to the buffer transmitted */
 errorCount += comparePackets(pRxTransferOp[1U], packetCount,
 byteSize, pTxBuffer[0U]);
 errorCount += comparePackets(pRxTransferOp[0U], packetCount,
 byteSize, pTxBuffer[1U]);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, compare, byteSize, packetCount, loopCount,
 errorCount, "Multi-Packet Double Loopback Test");

LoopBackDouble_MultiPacket_Finish:

/* For each channel */
for (i = 0U; i < 2U; i++)
{
 /* Dispose of transfer operations, */
 STAR_disposeTransferOperation(pTransferOpList[i][0U]);
 STAR_disposeTransferOperation(pTransferOpList[i][1U]);

 /* Destroy the packets transmitted */
 if (ppTxStreamItems[i] != NULL)
 {
 for (packetNum = 0U; packetNum < packetCount; packetNum++)

```

```

 {
 if (ppTxStreamItems[i][packetNum] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i][packetNum]);
 }
 }
 free(ppTxStreamItems[i]);
 }

 /* Free the transmit buffer */
 if (pTxBuffer[i] != NULL)
 {
 free(pTxBuffer[i]);
 }

 /* Free the address path */
 if (pAddressPath[i] != NULL)
 {
 STAR_destroyAddress(pAddressPath[i]);
 }

 /* Close the channel */
 if (channelId[i] != 0U)
 {
 STAR_closeChannel(channelId[i]);
 }
}

void Transmit_SinglePacket(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 unsigned long loopCount, byteSize, testNum;
 clock_t start, finish;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get packet size */
 if (getPacketSize(&byteSize) == 0)
 {
 goto Transmit_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Transmit_SinglePacket_Finish;
 }

 /* Allocate memory for the transmit buffer */
 pTxBuffer = (char *)calloc(1U, byteSize);
 if (pTxBuffer == NULL)
 {
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto Transmit_SinglePacket_Finish;
 }

 /* Fill transmit buffer with random data, receive buffer with zeros */
 FillBufferRandomChar(pTxBuffer, byteSize, DATA_TYPE_RANDOM);

 /* Create the packet to be transmitted */
 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, byteSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("\nERROR: Unable to create the packet to be transmitted");
 goto Transmit_SinglePacket_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
 if (pTxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 }
}

```



```

 goto Transmit_SinglePacket_Finish;
 }

 puts("Starting Single-Packet Transmit Test...");

 /* Start time */
 start = GET_TIME();

 for (testNum = 0U; testNum < loopCount; testNum++)
 {
 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_SinglePacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_SinglePacket_Finish;
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, byteSize, 1U, loopCount, 0U,
 "Single-Packet Transmit Test");

Transmit_SinglePacket_Finish:

 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packet transmitted */
 if (pTxStreamItem != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0u)
 {
 STAR_closeChannel(txChannelId);
 }
}

void Transmit_MultiPacket(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 unsigned long loopCount, byteSize, testNum, packetCount = 0U, i;
 char *pTxBuffer = NULL;
 STAR_STREAM_ITEM **ppTxStreamItems = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_TRANSFER_STATUS txStatus;
 clock_t start, finish;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)

```

```

{
 return;
}

/* Select the path to be added to the front of packets transmitted */
pAddressPath = getTransmitPathAddress();

/* Get packet size */
if (getPacketSize(&byteSize) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Get packet count */
if (getPacketCount(&packetCount) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Get loop count */
if (getLoopCount(&loopCount) == 0)
{
 goto Transmit_MultiPacket_Finish;
}

/* Allocate memory for the transmit buffer */
pTxBuffer = (char *)calloc(1U, byteSize * packetCount);
if (pTxBuffer == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit buffer");
 goto Transmit_MultiPacket_Finish;
}

/* Fill transmit buffer with random data */
FillBufferRandomChar(pTxBuffer, byteSize * packetCount, DATA_TYPE_RANDOM);

/* Allocate memory for the transmit stream item pointers */
ppTxStreamItems = (STAR_STREAM_ITEM **)calloc(packetCount,
 sizeof(STAR_STREAM_ITEM *));
if (ppTxStreamItems == NULL)
{
 puts("\nERROR: Unable to allocate memory for transmit stream items");
 goto Transmit_MultiPacket_Finish;
}

/* For each packet */
for (i = 0U; i < packetCount; i++)
{
 /* Create the stream item */
 ppTxStreamItems[i] = STAR_createPacket(pAddressPath,
 (U8 *)pTxBuffer + (byteSize * i), byteSize, STAR_EOP_TYPE_EOP);
 if (ppTxStreamItems[i] == NULL)
 {
 printf("\nERROR: Unable to create packet %lu to be transmitted\n",
 i);
 goto Transmit_MultiPacket_Finish;
 }
}

/* Create the transmit transfer operation for the packets */
pTxTransferOp = STAR_createTxOperation(ppTxStreamItems, packetCount);
if (pTxTransferOp == NULL)
{
 puts("\nERROR: Unable to create the transfer operation to be transmitted");
 goto Transmit_MultiPacket_Finish;
}

puts("Starting Multi-Packet Transmit Test...");

/* Start time */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the transmit operation */
 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_MultiPacket_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(
 pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)

```

```

 {
 printf("\nERROR occurred during transmit. Test %lu failed.\n",
 testNum + 1U);
 goto Transmit_MultiPacket_Finish;
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, byteSize, packetCount, loopCount, 0U,
 "Multiple-Packet Transmit Test");

Transmit_MultiPacket_Finish:

 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packets transmitted */
 if (ppTxStreamItems != NULL)
 {
 for (i = 0U; i < packetCount; i++)
 {
 if (ppTxStreamItems[i] != NULL)
 {
 STAR_destroyStreamItem(ppTxStreamItems[i]);
 }
 }
 free(ppTxStreamItems);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void Receive_SinglePacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 unsigned long loopCount, rxByteSize = 0U, testNum;
 clock_t start, finish;
 STAR_TRANSFER_STATUS rxStatus;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_SinglePacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Receive_SinglePacket_Finish;
 }

 /* Create receive operation to receive 1 packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto Receive_SinglePacket_Finish;
 }
}

```

```

puts("Running Single-Packet Receive Test...");

/* Start time (this should be updated when the first packet is received) */
start = GET_TIME();

for (testNum = 0U; testNum < loopCount; testNum++)
{
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_SinglePacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_SinglePacket_Finish;
 }

 /* Start time after first packet received */
 if (testNum == 0U)
 {
 start = GET_TIME();
 }
 else
 {
 rxByteSize += getPacketLengths(pRxTransferOp, 1U);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
}
printf("\n");

/* End time */
finish = GET_TIME();

/* Display the results */
DisplayResults(start, finish, 0, rxByteSize, 1U, 1U, 0U,
 "Single-Packet Receive Test");

Receive_SinglePacket_Finish:

/* Dispose of the transfer operation */
if (pRxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pRxTransferOp);
}

/* Close the channel */
if (rxChannelId != 0U)
{
 STAR_closeChannel(rxChannelId);
}
}

void Receive_MultiPacket(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 unsigned long loopCount, rxByteSize = 0U, testNum, packetCount;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 clock_t start, finish;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }

 /* Get packet count */
 if (getPacketCount(&packetCount) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }

 /* Get loop count */
 if (getLoopCount(&loopCount) == 0)
 {
 goto Receive_MultiPacket_Finish;
 }
}

```

```

 }

 /* Create receive operation to receive the packets */
 pRxTransferOp = STAR_createRxOperation(packetCount,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("\nERROR: Unable to create receive operation");
 goto Receive_MultiPacket_Finish;
 }

 puts("Running Multiple-Packet Receive Test...");

 /* Start time */
 /* (this should be updated when the first packets are received) */
 start = GET_TIME();

 for (testNum = 0U; testNum < loopCount; testNum++)
 {
 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_MultiPacket_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("\nERROR occurred during receive. Test %lu failed.\n",
 testNum + 1U);
 goto Receive_MultiPacket_Finish;
 }

 /* Start time after first packet received */
 if (testNum == 0U)
 {
 start = GET_TIME();
 }
 else
 {
 rxByteSize += getPacketLengths(pRxTransferOp, packetCount);
 }
 printf("\rTest - %lu", testNum + 1);
 fflush(stdout);
 }
 printf("\n");

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, rxByteSize, 1U, 1U, 0U,
 "Multi-Packet Receive Test");

Receive_MultiPacket_Finish:

 /* Dispose of the transfer operation */
 if (pRxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pRxTransferOp);
 }

 /* Close the channel */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
}

int ConfirmYes(void)
{
 char response[256] = { '\0' };
 size_t responseLen = 0U;

 for (;;)
 {
 if (fgets(response, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return 0;
 }
 }
}

```

```

 /* Strip newline */
 responseLen = strlen(response);
 if (response[responseLen - 1U] == '\n')
 {
 response[responseLen - 1U] = '\0';
 }

 if ((strcmp(response, "y") == 0) || (strcmp(response, "Y") == 0))
 {
 return 1;
 }
 else if ((strcmp(response, "n") == 0) || (strcmp(response, "N") == 0))
 {
 return 0;
 }
 else
 {
 puts("Please enter Y/N");
 }
 }
}

void TransmitFile_Whole(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 char *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM *pTxStreamItem = NULL;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;
 char sFile[256];
 int fileSize = 0;

 size_t sFileLen;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get the File name */
 printf("Enter name of file to transmit: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 puts("File Transfer aborted");
 goto Transmit_File_Whole_Finish;
 }

 fileSize = SizeOfFile(sFile);
 if (fileSize <= 0)
 {
 puts("File size invalid, aborting transfer.");
 goto Transmit_File_Whole_Finish;
 }

 /* Allocate memory to read file into */
 pTxBuffer = (char *)malloc(fileSize);
 if (pTxBuffer == NULL)
 {
 puts("ERROR: Could not allocate buffer for file");
 goto Transmit_File_Whole_Finish;
 }

 /* Read the file */

```

```

 if (file_read_chunk(sFile, pTxBuffer, 0U, fileSize) == 0)
 {
 puts("ERROR: Could not read file");
 goto Transmit_File_Whole_Finish;
 }

 pTxStreamItem = STAR_createPacket(pAddressPath, (U8 *)pTxBuffer, fileSize,
 STAR_EOP_TYPE_EOP);
 if (pTxStreamItem == NULL)
 {
 puts("ERROR: Unable to create the packet to be transmitted");
 goto Transmit_File_Whole_Finish;
 }

 /* Create the transmit transfer operation for the packet */
 pTxTransferOp = STAR_createTxOperation(&pTxStreamItem, 1U);
 if (pTxTransferOp == NULL)
 {
 puts("ERROR: Unable to create the transfer operation to be transmitted");
 goto Transmit_File_Whole_Finish;
 }

 puts("Starting File Transmit...");

 if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
 {
 puts("ERROR occurred during transmit");
 goto Transmit_File_Whole_Finish;
 }

 /* Wait on the transmit operation completing */
 txStatus = STAR_waitOnTransferOperationCompletion(pTxTransferOp,
 STAR_INFINITE);
 if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 puts("ERROR occurred during transmit");
 goto Transmit_File_Whole_Finish;
 }

 puts("Complete.");

Transmit_File_Whole_Finish:
 /* Dispose of the transfer operation */
 if (pTxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pTxTransferOp);
 }

 /* Destroy the packet transmitted */
 if (pTxStreamItem != NULL)
 {
 STAR_destroyStreamItem(pTxStreamItem);
 }

 /* Free the transmit buffer */
 if (pTxBuffer != NULL)
 {
 free(pTxBuffer);
 }

 /* Free the address */
 if (pAddressPath != NULL)
 {
 STAR_destroyAddress(pAddressPath);
 }

 /* Close the channel */
 if (txChannelId != 0U)
 {
 STAR_closeChannel(txChannelId);
 }
}

void ReceiveFile_Whole(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 char sFile[256];
 int writeStatus;
 STAR_SPACEWIRE_PACKET *pReceivedPacket = NULL;
 size_t sFileLen;
 U8 *receivedData = NULL;
 unsigned int receivedDataLen = 0U;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,

```

```

 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_File_Whole_Finish;
 }

 /* Get the File name */
 printf("Enter name of file to write data to: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 printf("File Transfer aborted\n");
 goto Receive_File_Whole_Finish;
 }

 puts("Running File Receive...");

 /* Create receive operation to receive 1 packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("ERROR: Unable to create receive operation");
 goto Receive_File_Whole_Finish;
 }

 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Whole_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Error of %d occurred during receive: \n", rxStatus);
 goto Receive_File_Whole_Finish;
 }

 /* Get received data*/
 pReceivedPacket = (STAR_SPACEWIRE_PACKET*)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;

 receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
 if (receivedData == NULL)
 {
 puts("ERROR: No data received");
 goto Receive_File_Whole_Finish;
 }

 writeStatus = file_append_chunk(receivedData, receivedDataLen, sFile);

 STAR_destroyPacketData(receivedData);

 if (writeStatus == 0)
 {
 puts("ERROR: Could not write file");
 goto Receive_File_Whole_Finish;
 }

 puts("Complete.");

Receive_File_Whole_Finish:

 /* Dispose of the transfer operation */
 if (pRxTransferOp != NULL)
 {

```



```

 STAR_disposeTransferOperation(pRxTransferOp);
 }

 /* Close the channel */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
}

void TransmitFile_Split(void)
{
 STAR_CHANNEL_ID txChannelId = 0U;
 unsigned long chunkSize, dataPacketCount = 0U, i;
 U8 *pTxBuffer = NULL;
 STAR_TRANSFER_OPERATION *pTxTransferOp = NULL;
 STAR_STREAM_ITEM** packets = NULL;
 STAR_TRANSFER_STATUS txStatus;
 STAR_SPACEWIRE_ADDRESS *pAddressPath;
 char sFile[256], sChunkSize[256];
 int status;
 int fileSize = 0;
 U32 fileSizeToSend = 0U;
 unsigned int offset = 0U;
 size_t sFileLen;

 /* Select the transmit device and channel to be used */
 if (chooseDeviceAndChannel("transmit", &txChannelId,
 STAR_CHANNEL_DIRECTION_OUT) == 0)
 {
 return;
 }

 /* Select the path to be added to the front of packets transmitted */
 pAddressPath = getTransmitPathAddress();

 /* Get the File name */
 printf("Enter name of file to transmit: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid Input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 printf("File Transfer aborted\n");
 goto Transmit_File_Packets_Finish;
 }

 /* Allocate memory to read file into */
 fileSize = SizeOfFile(sFile);
 if (fileSize <= 0)
 {
 printf("File size invalid, aborting transfer.\n");
 goto Transmit_File_Packets_Finish;
 }

 /* Read the file into a buffer */
 pTxBuffer = (U8 *)malloc(fileSize);
 if (pTxBuffer == NULL)
 {
 puts("ERROR: Could not allocate buffer for file");
 goto Transmit_File_Packets_Finish;
 }

 if (file_read_chunk(sFile, (char*)pTxBuffer, 0U, fileSize) == 0)
 {
 puts("ERROR: Could not read file");
 goto Transmit_File_Packets_Finish;
 }

 /* Get packet size */
 printf("Enter maximum size of packets to transmit, in bytes: ");
 fflush(stdout);

```

```

if (fgets(sChunkSize, 256, stdin) == NULL)
{
 puts("ERROR: Invalid input");
 goto Transmit_File_Packets_Finish;
}
status = sscanf(sChunkSize, "%lu", &chunkSize);
if (status == 0)
{
 puts("ERROR: Invalid input");
 goto Transmit_File_Packets_Finish;
}

/* Calculate number of packets to send */
dataPacketCount = ((fileSize / chunkSize) +
 ((fileSize % chunkSize) > 0U) ? 1U : 0U));

/* Prepare packets for transmission */
packets = (STAR_STREAM_ITEM **)calloc(dataPacketCount + 1U,
 sizeof(STAR_STREAM_ITEM *));
if (packets == NULL)
{
 puts("ERROR: Could not allocate buffer for packets");
 goto Transmit_File_Packets_Finish;
}

/* Initial four byte packet size */
CopyNumberToMemory(&fileSizeToSend, fileSize, sizeof(U32));
packets[0U] = STAR_createPacket(pAddressPath, (U8*)&fileSizeToSend,
 sizeof(U32), STAR_EOP_TYPE_EOP);

/* Data packets*/
for (i = 1U; i < dataPacketCount; i++)
{
 packets[i] = STAR_createPacket(pAddressPath, pTxBuffer + offset,
 chunkSize, STAR_EOP_TYPE_EOP);
 offset+= chunkSize;
}

/* Final Data packet*/
if ((fileSize % chunkSize) > 0)
{
 packets[dataPacketCount] = STAR_createPacket(pAddressPath,
 pTxBuffer + offset, fileSize % chunkSize, STAR_EOP_TYPE_EOP);
}
else
{
 packets[dataPacketCount] = STAR_createPacket(pAddressPath,
 pTxBuffer + offset, chunkSize, STAR_EOP_TYPE_EOP);
}

pTxTransferOp = STAR_createTxOperation(packets, dataPacketCount + 1U);

puts("Starting File Transmit...");

if (STAR_submitTransferOperation(txChannelId, pTxTransferOp) == 0)
{
 puts("ERROR: Could not submit transmit operation");
 goto Packet_Cleanup;
}

/* Wait on the transmit operation completing */
txStatus = STAR_waitOnTransferOperationCompletion(pTxTransferOp,
 STAR_INFINITE);
if (txStatus != STAR_TRANSFER_STATUS_COMPLETE)
{
 printf("ERROR: Error %d occurred during transmit\n", txStatus);
 goto Packet_Cleanup;
}

puts("Complete.");

Packet_Cleanup:
/* Cleanup packets */
for (i = 0U; i <= dataPacketCount; i++)
{
 STAR_destroyStreamItem(packets[i]);
}

Transmit_File_Packets_Finish:

free(packets);

/* Dispose of the transfer operation */
if (pTxTransferOp != NULL)
{
 STAR_disposeTransferOperation(pTxTransferOp);
}

```

```

/* Free the transmit buffer */
if (pTxBuffer != NULL)
{
 free(pTxBuffer);
}

/* Free the address path */
if (pAddressPath != NULL)
{
 STAR_destroyAddress(pAddressPath);
}

/* Close the channel */
if (txChannelId != 0U)
{
 STAR_closeChannel(txChannelId);
}
}

void ReceiveFile_Split(void)
{
 STAR_CHANNEL_ID rxChannelId = 0U;
 STAR_TRANSFER_OPERATION *pRxTransferOp = NULL;
 STAR_TRANSFER_STATUS rxStatus;
 char sFile[256];
 int writeStatus;
 STAR_SPACEWIRE_PACKET *pReceivedPacket = NULL;
 size_t sFileLen = 0U;
 clock_t start, finish;
 U8 *receivedData = NULL;
 unsigned int receivedDataLen = 0U, expectedFileSize = 0U;
 unsigned int actualFileSize = 0U;

 /* Select the receive device and channel to be used */
 if (chooseDeviceAndChannel("receive", &rxChannelId,
 STAR_CHANNEL_DIRECTION_IN) == 0)
 {
 goto Receive_File_Split_Finish;
 }

 /* Get the File name */
 printf("Enter name of file to write data to: ");
 fflush(stdout);
 if (fgets(sFile, 256, stdin) == NULL)
 {
 puts("ERROR: Invalid input");
 return;
 }

 /* Strip newline */
 sFileLen = strlen(sFile);
 if (sFile[sFileLen - 1U] == '\n')
 {
 sFile[sFileLen - 1U] = '\0';
 }

 printf("Filename = \"%s\"\n", sFile);
 printf("Continue with these values ? (Y/N): ");
 fflush(stdout);

 if (ConfirmYes() == 0)
 {
 puts("File Transfer aborted");
 goto Receive_File_Split_Finish;
 }

 puts("Running File Receive...");

 /* Create receive operation to receive Header packet */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);
 if (pRxTransferOp == NULL)
 {
 puts("ERROR: Unable to create receive operation");
 goto Receive_File_Split_Finish;
 }

 /* Submit the receive operation */
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Split_Finish;
 }

 /* Wait on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(pRxTransferOp,

```

```

 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Error %d occurred during receive\n", rxStatus);
 goto Receive_File_Split_Finish;
 }

 pReceivedPacket = (STAR_SPACEWIRE_PACKET *)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;
 receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
 if (receivedData == NULL)
 {
 puts("ERROR: No data received");
 goto Receive_File_Split_Finish;
 }

 CopyNumberFromMemory(&expectedFileSize, receivedData,
 sizeof(expectedFileSize));

 STAR_destroyPacketData(receivedData);

 /* Start the timer */
 start = GET_TIME();

 /* Read a packet at a time until we get all expected data */
 while(expectedFileSize > actualFileSize)
 {
 if (STAR_submitTransferOperation(rxChannelId, pRxTransferOp) == 0)
 {
 puts("ERROR: Could not submit receive operation");
 goto Receive_File_Split_Finish;
 }

 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp,
 STAR_INFINITE);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Error %d occurred during receive\n", rxStatus);
 goto Receive_File_Split_Finish;
 }

 pReceivedPacket = (STAR_SPACEWIRE_PACKET *)
 STAR_getTransferItem(
 pRxTransferOp, 0U)->item;

 receivedData = STAR_getPacketData(pReceivedPacket, &receivedDataLen);
 if (receivedData == NULL)
 {
 puts("ERROR: No data received");
 goto Receive_File_Split_Finish;
 }

 writeStatus = file_append_chunk(receivedData, receivedDataLen, sFile);

 STAR_destroyPacketData(receivedData);

 if (writeStatus == 0)
 {
 puts("ERROR: Could not write to file");
 goto Receive_File_Split_Finish;
 }

 actualFileSize+=receivedDataLen;
 }

 if (actualFileSize != expectedFileSize)
 {
 puts("ERROR: Expected and actual file size do not match");
 }

 /* End time */
 finish = GET_TIME();

 /* Display the results */
 DisplayResults(start, finish, 0, actualFileSize, 1U, 1U, 0U,
 "Receive File from multiple packets");

Receive_File_Split_Finish:

 /* Dispose of the transfer operation */
 if (pRxTransferOp != NULL)
 {
 STAR_disposeTransferOperation(pRxTransferOp);
 }

```

```

 /* Close the channel */
 if (rxChannelId != 0U)
 {
 STAR_closeChannel(rxChannelId);
 }
}

void ResetDevice(void)
{
 STAR_DEVICE_ID deviceId;

 deviceId = chooseDevice("", STAR_DEVICE_ALL);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return;
 }

 if (STAR_resetDevice(deviceId) == 0)
 {
 puts("ERROR: Unable to reset device");
 }
}

void IdentifyDevice(void)
{
 STAR_DEVICE_ID deviceId;

 deviceId = chooseDevice("", STAR_DEVICE_CONFIG_SUPPORTED);
 if (deviceId == STAR_DEVICE_UNKNOWN)
 {
 return;
 }

 if (!CFG_SUCCESS(CFG_identify(deviceId)))
 {
 puts("ERROR: Unable to identify device");
 }
}

void runInteractive(void)
{
 char s[256];
 int menuSelect;
 int bExit = 0;

 puts("STAR-System Test Program");
 puts("Copyright STAR-Dundee Ltd. (c) 2024");
 puts("www.star-dundee.com");

 /* Loop until exit option is chosen */
 while (bExit == 0)
 {
 DisplayMenu();

 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No value specified");
 }
 else if (sscanf(s, "%d", &menuSelect) == 0)
 {
 puts("\nERROR: Invalid value specified");
 }
 else
 {
 switch(menuSelect)
 {
 case MENU_DISPLAY_INFORMATION:
 DisplayInformation();
 break;

 case MENU_LOOPBACK_SINGLE:
 LoopBack_SinglePacket();
 break;

 case MENU_LOOPBACK_MULTI:
 LoopBack_MultiPacket();
 break;

 case MENU_LOOPBACK_DOUBLE_SINGLE:
 LoopBackDouble_SinglePacket();
 break;

 case MENU_LOOPBACK_DOUBLE_MULTI:
 LoopBackDouble_MultiPacket();
 break;

 case MENU_TRANSMIT_SINGLE:

```

```

 Transmit_SinglePacket();
 break;

 case MENU_TRANSMIT_MULTI:
 Transmit_MultiPacket();
 break;

 case MENU_RECEIVE_SINGLE:
 Receive_SinglePacket();
 break;

 case MENU_RECEIVE_MULTI:
 Receive_MultiPacket();
 break;

 case MENU_TRANSMIT_FILE_WHOLE:
 TransmitFile_Whole();
 break;

 case MENU_RECEIVE_FILE_WHOLE:
 ReceiveFile_Whole();
 break;

 case MENU_TRANSMIT_FILE_SPLIT:
 TransmitFile_Split();
 break;

 case MENU_RECEIVE_FILE_SPLIT:
 ReceiveFile_Split();
 break;

 case MENU_RESET_DEVICE:
 ResetDevice();
 break;

 case MENU_IDENTIFY_DEVICE:
 IdentifyDevice();
 break;

 case MENU_EXIT:
 puts("\nExiting SpaceWire test program");
 bExit = 1;
 break;

 default:
 puts("\nERROR: Incorrect menu option");
 break;
 }
}

puts("Exiting...\n");
}

void usage(char *argv[], const int error)
{
 puts("");

 if (error != 0)
 {
 fprintf(stderr, "usage: %s [-v][-help]\n", argv[0]);
 }
 else
 {
 fprintf(stdout, "usage: %s [-v][-help]\n", argv[0]);
 }
}

void showHelp(void)
{
 puts("");
 puts("Description:");
 puts(" A program to display information about STAR-System,");
 puts(" the STAR-Dundee API stack, and perform basic transmit");
 puts(" and receive tests using attached hardware.");
 puts("");
 puts("Optional Arguments:");
 puts("");
 puts(" -v Displays version information and exits.");
 puts("");
 puts(" -help Displays this help message and exits.");
 puts("");
 puts(" When called without arguments, the program is run in");
 puts(" interactive mode.");
}

int __cdecl main(int argc, char *argv[])

```

```

{
 int i;
 int version = 0; /* Value for the "-v" optional argument. */
 int help = 0; /* Value for the "-help" optional argument. */

 STAR_setApplicationName("STAR-System Test Application");

 /* If no args provided, interactive mode */
 if (argc <= 1)
 {
 runInteractive();
 return 0;
 }

 /* Currently we only expect 1 optional argument */
 if (argc > 2)
 {
 usage(argv, 1);
 return 1;
 }

 /* Process command line args*/
 for (i = 1; i < argc; i++)
 {
 if (strcmp(argv[i], "-v") == 0)
 {
 version = 1;
 }
 else if (strcmp(argv[i], "-help") == 0)
 {
 help = 1;
 }
 }

 if (help != 0)
 {
 usage(argv, 0);
 showHelp();
 }
 else if (version != 0)
 {
 puts(VERSION_INFO);
 puts("Copyright STAR-Dundee Ltd. (c) 2024");
 puts("www.star-dundee.com");

 DisplayInformation();
 }
 else
 {
 usage(argv, 1);
 return 1;
 }

 return 0;
}

```

## 10.47 star\_performance\_tester.c

This file contains the functions for the SpaceWire Performance Tester, a program to test the performance of SpaceWire devices using the STAR-System software stack.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <errno.h>

#include "star-api.h"

#if !defined(__vxworks)
 #define TRUE 1
 #define FALSE 0
#else
 /* Required for strcasecmp() */
 #include <strings.h>
#endif

#define STAR_INFINITE -1

```

```

#ifdef _WIN32

#include <windows.h>

#define strcasecmp(s1, s2) _stricmp((s1), (s2))

#define GET_TIME() clock()

#define TIME_DIVIDER CLOCKS_PER_SEC

#define SLEEP(time) Sleep(time)

#define DISABLE_SYSTEM_SLEEP() \
 SetThreadExecutionState(ES_CONTINUOUS | ES_SYSTEM_REQUIRED)

#define ENABLE_SYSTEM_SLEEP() \
 SetThreadExecutionState(ES_CONTINUOUS)

typedef struct
{
 FILETIME idleTime;

 FILETIME kernelTime;

 FILETIME userTime;

 FILETIME processKernelTime;

 FILETIME processUserTime;
} CPU_USAGE_PROPERTIES;

static void STORE_CPU_USAGE(CPU_USAGE_PROPERTIES * const pCpuProperties)
{
 FILETIME creationTime, exitTime;

 GetSystemTimes(&pCpuProperties->idleTime, &pCpuProperties->kernelTime,
 &pCpuProperties->userTime);
 GetProcessTimes(GetCurrentProcess(), &creationTime, &exitTime,
 &pCpuProperties->processKernelTime,
 &pCpuProperties->processUserTime);
}

static void PRINT_CPU_USAGE_HEADERS(FILE * const stream)
{
 fprintf(stream, "\tProcess CPU usage (%) \tTotal CPU usage (%) \n");
}

static void POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(ULARGE_INTEGER *pInt,
 const FILETIME * const pStartTime, const FILETIME * const pEndTime)
{
 ULARGE_INTEGER startTime;
 startTime.HighPart = pStartTime->dwHighDateTime;
 startTime.LowPart = pStartTime->dwLowDateTime;

 pInt->HighPart = pEndTime->dwHighDateTime;
 pInt->LowPart = pEndTime->dwLowDateTime;

 pInt->QuadPart -= startTime.QuadPart;
}

static void PRINT_CPU_USAGE(FILE * const stream,
 const CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 const CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 ULARGE_INTEGER idleTime, kernelTime, userTime, totalTime;
 double totalUsage, processUsage;

 /* Subtract the original total CPU figures from the final figures */
 POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(&idleTime,
 &pStartCpuProperties->idleTime, &pEndCpuProperties->idleTime);
 POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(&kernelTime,
 &pStartCpuProperties->kernelTime, &pEndCpuProperties->kernelTime);
 POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(&userTime,
 &pStartCpuProperties->userTime, &pEndCpuProperties->userTime);

 /* Calculate the total time */
 totalTime.QuadPart = userTime.QuadPart + kernelTime.QuadPart;

 /* Calculate the total usage */
 if (totalTime.QuadPart != 0U)
 {
 totalUsage =
 (double)((totalTime.QuadPart - idleTime.QuadPart) * 100U) /
 (double)totalTime.QuadPart;
 }
}

```



```

 else
 {
 totalUsage = 0.0;
 }

 /* Subtract the original process CPU figures from the final figures */
 POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(&kernelTime,
 &pStartCpuProperties->processKernelTime,
 &pEndCpuProperties->processKernelTime);
 POPULATE_ULARGE_INT_FROM_FILETIME_DIFF(&userTime,
 &pStartCpuProperties->processUserTime,
 &pEndCpuProperties->processUserTime);

 /* Calculate the process's usage */
 if (totalTime.QuadPart != 0U)
 {
 processUsage =
 (double)((userTime.QuadPart + kernelTime.QuadPart) * 100U) /
 (double)totalTime.QuadPart;
 }
 else
 {
 processUsage = 0.0;
 }

 if (stream != NULL)
 {
 fprintf(stream, "\t%-3.2f\t%-3.2f\n", processUsage, totalUsage);
 }
 else
 {
 printf("\t\t%-3.2f%% Process CPU usage\t%-3.2f%% Total CPU usage\n",
 processUsage, totalUsage);
 }
}

#else

#if defined(__vxworks)
#include <time.h>
#include <utime.h>
#include <sys/times.h>
#include <taskLib.h>
#include <sysLib.h>
#else
#include <sys/time.h>
#include <sys/times.h>
#endif

#include <unistd.h>

clock_t GET_TIME()
{
 struct timeval tv;
 struct timezone tz;

 gettimeofday(&tv, &tz);
 return tv.tv_sec * 1000000 + tv.tv_usec;
}

#define TIME_DIVIDER 1000000

#if defined(__vxworks)
void SLEEP(unsigned int delay)
{
 double rate = (double)sysClkRateGet();
 unsigned int ticks = (int)((rate / 1000.0) * delay + 0.5);
 taskDelay(ticks);
}
#else
#define SLEEP(time) usleep(time * 1000)
#endif

#define MAX_PATH 1024

#define DISABLE_SYSTEM_SLEEP()

#define ENABLE_SYSTEM_SLEEP()

typedef struct
{
#if !defined(__vxworks)
 struct tms processTimes;
#endif

```

```

 clock_t currentTime;

 unsigned long userTime;

 unsigned long systemTime;

 unsigned long niceTime;

 unsigned long idleTime;

 unsigned long iowaitTime;

 unsigned long irqTime;

 unsigned long softirqTime;

} CPU_USAGE_PROPERTIES;

void STORE_CPU_USAGE(CPU_USAGE_PROPERTIES *pCpuProperties)
{
 FILE *procFile;
#ifdef __vxworks
 pCpuProperties->currentTime = times(&pCpuProperties->processTimes);
#endif
 pCpuProperties->userTime = 0;
 pCpuProperties->systemTime = 0;
 pCpuProperties->niceTime = 0;
 pCpuProperties->idleTime = 0;
 pCpuProperties->iowaitTime = 0;
 pCpuProperties->irqTime = 0;
 pCpuProperties->softirqTime = 0;
 procFile = fopen("/proc/stat", "r");
 if (procFile)
 {
 char line[256];
 if (fgets(line, 256, procFile))
 {
 sscanf(line, "%*s %lu %lu %lu %lu %lu %lu %lu",
 &pCpuProperties->userTime, &pCpuProperties->systemTime,
 &pCpuProperties->niceTime, &pCpuProperties->idleTime,
 &pCpuProperties->iowaitTime, &pCpuProperties->irqTime,
 &pCpuProperties->softirqTime);
 }

 fclose(procFile);
 }
}

void PRINT_CPU_USAGE_HEADERS(FILE *stream)
{
 fprintf(stream, "\tProcess CPU usage (%) \tTotal CPU usage (%) \n");
}

#ifdef __QNX__
#include <sys/syspage.h>
int GetCpuCount()
{
 return _syspage_ptr->num_cpu;
}
#elif defined(__vxworks)
/* First version of vxworks driver is for single core targets only */
int GetCpuCount()
{
 return 1;
}
#else
int GetCpuCount()
{
 return (int)sysconf(_SC_NPROCESSORS_ONLN);
}
#endif

void PRINT_CPU_USAGE(FILE *stream,
 CPU_USAGE_PROPERTIES const *pStartCpuProperties,
 CPU_USAGE_PROPERTIES const *pEndCpuProperties)
{
 unsigned long userTime, systemTime, niceTime, idleTime, iowaitTime,
 irqTime, softirqTime;
 unsigned long totalTime;
 clock_t processKernelTime, processUserTime, processTotalTime;
 double totalUsage, processUsage;
 int cpuCount;

```

```

/* Determine the number of CPUs */
cpuCount = GetCpuCount();

/* Subtract the original total CPU figures from the final figures */
userTime = pEndCpuProperties->userTime -
 pStartCpuProperties->userTime;
systemTime = pEndCpuProperties->systemTime -
 pStartCpuProperties->systemTime;
niceTime = pEndCpuProperties->niceTime -
 pStartCpuProperties->niceTime;
idleTime = pEndCpuProperties->idleTime -
 pStartCpuProperties->idleTime;
iowaitTime = pEndCpuProperties->iowaitTime -
 pStartCpuProperties->iowaitTime;
irqTime = pEndCpuProperties->irqTime -
 pStartCpuProperties->irqTime;
softirqTime = pEndCpuProperties->softirqTime -
 pStartCpuProperties->softirqTime;

/* Calculate the total time */
totalTime = userTime + systemTime + niceTime + idleTime + iowaitTime +
 irqTime + softirqTime;

/* Calculate the total usage */
if (totalTime)
{
 totalUsage =
 (double)((totalTime - idleTime) * 100) / (double)totalTime;
}
else
{
 totalUsage = 0;
}

#if !defined(__vxworks)
/* Subtract the original process CPU figures from the final figures */
processKernelTime = pEndCpuProperties->processTimes.tms_stime -
 pStartCpuProperties->processTimes.tms_stime;
processUserTime = pEndCpuProperties->processTimes.tms_utime -
 pStartCpuProperties->processTimes.tms_utime;
#else
processKernelTime = 0;
processUserTime = 0;
#endif

/* Calculate the total process time */
processTotalTime = pEndCpuProperties->currentTime -
 pStartCpuProperties->currentTime;

/* Calculate the process's usage */
if (processTotalTime && cpuCount)
{
 processUsage =
 (double)((processUserTime + processKernelTime) * 100) /
 (double)(processTotalTime * cpuCount);
}
else
{
 processUsage = 0;
}

/* Note: the kernel time may not be needed, if this is included in */
/* the user time (at least according to some manuals!) */
/* processUsage = (double)(processUserTime * 100) / */
/* (double)(processTotalTime * cpuCount); */

if (stream)
{
 fprintf(stream, "\t%-3.2f\t%-3.2f\n", processUsage, totalUsage);
}
else
{
 printf("\t\t%-3.2f%% Process CPU usage\t%-3.2f%% Total CPU usage\n",
 processUsage, totalUsage);
}
}

#endif

#define PERFORM_RECEIVE 1U

#define PERFORM_TRANSMIT 2U

#define PERFORMING_RECEIVES(_mode_) (((_mode_) & PERFORM_RECEIVE) != 0U)

```

```

#define PERFORMING_TRANSMITS(_mode_) (((_mode_) & PERFORM_TRANSMIT) != 0U)

#define MIN(a, b) (((a) > (b)) ? (b) : (a))

#define MAX(a, b) (((a) < (b)) ? (b) : (a))

#define BUFFER_NUM 4

/* The following values ensure that each test runs for at least 10 seconds */
/* (at 200 Mbit/s) */

#define BUFFER_SIZE 200000

#define LOOP_NUM 1000

#define TIMEOUT_TIME 1000

typedef enum
{
 RATE_TEST,

 RANDOM_TEST,

 LATENCY_TEST,

 TIME_TEST
} TEST_TYPE;

typedef struct
{
 U8 mode;

 TEST_TYPE testType;

 unsigned int testCount;

 unsigned int packetStart;

 unsigned int packetEnd;

 unsigned int packetStep;

 double testTime;

 STAR_CHANNEL_ID *pTxChannelIds;

 STAR_SPACEWIRE_ADDRESS **pAddressPaths;

 STAR_CHANNEL_ID *pRxChannelIds;

 char pFilePath[MAX_PATH + 1];

 char pFileDescription[1000];

 int useChannel0;
} TEST_PROPERTIES;

static STAR_DEVICE_ID chooseDevice(const char * const deviceTypeStr,
 const int argCount, const char * const arguments[],
 int * const pCurrentArgument)
{
 STAR_DEVICE_ID *devices;
 U32 devCount = 0U, i, chosen;
 int status;
 STAR_DEVICE_ID deviceID;
 char *deviceName, s[256];

 /* Get the list of devices which can transmit and receive packets */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_TXRX_SUPPORTED, &devCount);

 /* If there was an error getting the list of devices */
 if (devices == NULL)
 {
 /* Return an error */
 puts("Error occurred detecting devices!");
 return 0U;
 }

 /* Display the devices detected */
 if (devCount == 1U)
 {
 puts("One SpaceWire device detected:");
 }
}

```

```

else
{
 printf("%u SpaceWire devices detected:\n", devCount);
}

/* For each device */
for (i = 0U; i < devCount; i++)
{
 if (devices[i] != 0U)
 {
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName != NULL)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
}

/* If there's only 1 device on the list */
if (devCount == 1U)
{
 /* Return that device */
 deviceID = devices[0U];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

/* Ask the user which device to use */
printf("\nPlease select which %s device to use: ", deviceTypeStr);
fflush(stdout);
if ((*pCurrentArgument) < argCount)
{
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return 0U;
 }
}
status = sscanf(s, "%u", &chosen);
if ((status == 0) || (chosen >= devCount))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0U;
}

/* Return the chosen device */
deviceID = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceID;
}

static STAR_CHANNEL_ID openChannel(const char * const channelTypeStr,
 const int testNumber, const STAR_CHANNEL_DIRECTION direction,
 const int argCount, const char * const arguments[],
 int * const pCurrentArgument, const int lowestChannel)
{
 STAR_CHANNEL_MASK channelMask;
 int maximumChannelNumber = 0, i, channelNumber;
 STAR_DEVICE_ID deviceId;
 STAR_CHANNEL_ID channelId;

 /* Choose the device to be used */
 deviceId = chooseDevice(channelTypeStr, argCount, arguments,
 pCurrentArgument);

 /* Check for an invalid device id */
 if (deviceId == 0U)

```

```

{
 printf("\nERROR: Failed to open %s device for test %d.\n",
 channelTypeStr, testNumber);
 return 0U;
}

/* Check for a device with zero channels or an invalid list of channels */
channelMask = STAR_getDeviceChannels(deviceId);
if (channelMask == 0U)
{
 printf("\nERROR: The chosen %s device doesn't appear to have any valid channels.\n",
 channelTypeStr);
 return 0U;
}

/* For each possible channel */
for (i = 31; i >= lowestChannel; i--)
{
 /* If the channel exists */
 if (((channelMask >> (U32)i) & 1U) != 0U)
 {
 /* This is the maximum channel number */
 maximumChannelNumber = i;
 break;
 }
}

if (maximumChannelNumber < lowestChannel)
{
 printf("\nERROR: The chosen %s device doesn't appear to have any valid channels.\n",
 channelTypeStr);
 return 0U;
}
else if (maximumChannelNumber == lowestChannel)
{
 printf("\nUsing %s channel %d for test %d\n", channelTypeStr,
 lowestChannel, testNumber);
 channelNumber = lowestChannel;
}
else
{
 char s[256];
 int status;

 /* Get channel and check for validity */
 printf("\nEnter %s channel (%d..%d) for test %d: ", channelTypeStr,
 lowestChannel, maximumChannelNumber, testNumber);
 fflush(stdout);
 if ((*pCurrentArgument) < argCount)
 {
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No channel number specified");
 return 0U;
 }
 }
 status = sscanf(s, "%d", &channelNumber);
 if ((status == 0) || (channelNumber < lowestChannel) ||
 (channelNumber > maximumChannelNumber))
 {
 puts("\nERROR: Incorrect channel number specified");
 return 0U;
 }
 if (((1U << (U32)channelNumber) & channelMask) == 0U)
 {
 puts("\nERROR: The channel specified is not present");
 return 0U;
 }
}

/* Open the channel */
channelId = STAR_openChannelToLocalDevice(deviceId, direction,
 (unsigned char)channelNumber, TRUE);
if (channelId == 0U)
{
 printf("\nFailed to open %s channel %d for test %d\n", channelTypeStr,
 channelNumber, testNumber);
 return 0U;
}

```

```

 return channelId;
}

static void cleanupTest(const TEST_PROPERTIES * const pTestProperties)
{
 unsigned int i;

 /* Close any channels which were opened */
 if (pTestProperties->pRxChannelIds != NULL)
 {
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 STAR_closeChannel(pTestProperties->pRxChannelIds[i]);
 }
 free(pTestProperties->pRxChannelIds);
 }
 if (pTestProperties->pTxChannelIds != NULL)
 {
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 STAR_closeChannel(pTestProperties->pTxChannelIds[i]);
 }
 free(pTestProperties->pTxChannelIds);
 }

 /* Clean up the address paths */
 if (pTestProperties->pAddressPaths != NULL)
 {
 for (i = 0; i < pTestProperties->testCount; i++)
 {
 if (pTestProperties->pAddressPaths[i] != NULL)
 {
 STAR_destroyAddress(pTestProperties->pAddressPaths[i]);
 }
 }
 free(pTestProperties->pAddressPaths);
 }
}

static STAR_SPACEWIRE_ADDRESS *readSpaceWireAddress(const int argCount,
 const char * const arguments[], int * const pCurrentArgument)
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0U;
 char *pos = (char *)s;

 /* Read in the path */
 if ((*pCurrentArgument) < argCount)
 {
 puts(arguments[*pCurrentArgument]);
 strncpy(s, arguments[*pCurrentArgument], 255U);
 s[255U] = '\0';
 (*pCurrentArgument)++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("No address entered");
 return NULL;
 }
 }

 /* Read until end of buffer or line feed */
 while ((*pos != '\0') && (pathLen < 256U) && (*pos != '\n'))
 {
 unsigned long value;

 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in address");
 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
 }

 /* Create a SpaceWire address from the path */
 pAddress = STAR_createAddress(newPath, pathLen);
}

```

```

 /* Return the completed address */
 return pAddress;
}

static int readTestParameters(TEST_PROPERTIES * const pTestProperties,
 const int argCount, const char * const arguments[])
{
 char s[256], txRxStr[100];
 size_t len;
 int status, currentArgument = 0;
 unsigned int testNumber;

 puts("STAR-System Performance Tester");
 puts("Copyright STAR-Dundee Ltd. (c) 2024");
 puts("www.star-dundee.com\n");

 /* Ask the user which test they wish to perform */
 puts("Which test type do you wish to perform?");
 puts("\t(m) Maximum data rate test");
 puts("\t(r) Random packet size data rate test");
 puts("\t(l) Latency test");
 puts("\t(t) Timed test");

 /* Read in the test type */
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("ERROR: No test type selected");
 return 0;
 }
 }
 if ((s[0U] == 'm') || (s[0U] == 'M'))
 {
 pTestProperties->testType = RATE_TEST;
 }
 else if ((s[0U] == 'r') || (s[0U] == 'R'))
 {
 pTestProperties->testType = RANDOM_TEST;
 }
 else if ((s[0U] == 'l') || (s[0U] == 'L'))
 {
 pTestProperties->testType = LATENCY_TEST;
 }
 else if ((s[0U] == 't') || (s[0U] == 'T'))
 {
 pTestProperties->testType = TIME_TEST;
 }
 else
 {
 puts("ERROR: Invalid test type selected");
 return 0;
 }
 }

 /* Ask the user to how many tests they wish to perform */
 printf("\nHow many tests do you wish to perform: ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No test count specified");
 return 0;
 }
 }
 }
 status = sscanf(s, "%u", &pTestProperties->testCount);
 if ((status == 0) || (pTestProperties->testCount < 1) ||
 (pTestProperties->testCount > 100))
 {
 puts("\nERROR: No valid test count specified, enter a value between 1 and 100");
 return 0;
 }
}

```



```

/* Ask the user if the test should transmit, receive or */
/* both transmit and receive packets */
printf("Should these tests transmit packets only (\t\), receive packets only (\r\), or both (\b\
)? ");
fflush(stdout);
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No transmit/receive option specified");
 return 0;
 }
}
pTestProperties->mode = 0U;
if ((s[0U] == 't') || (s[0U] == 'T') || (s[0U] == 's') || (s[0U] == 'S') ||
 (s[0U] == 'b') || (s[0U] == 'B'))
{
 strcpy(txRxStr, "transmit");
 pTestProperties->mode |= PERFORM_TRANSMIT;
}
if ((s[0U] == 'r') || (s[0U] == 'R') || (s[0U] == 'b') || (s[0U] == 'B'))
{
 if (pTestProperties->mode != 0U)
 {
 strcat(txRxStr, "/receive");
 }
 else
 {
 strcpy(txRxStr, "receive");
 }
 pTestProperties->mode |= PERFORM_RECEIVE;
}
if (pTestProperties->mode == 0U)
{
 puts("\nERROR: No valid transmit/receive option specified");
 return 0;
}

/* Ask the user to enter a starting packet size */
printf("\nEnter the minimum size of packet to %s: ", txRxStr);
fflush(stdout);
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No minimum packet size specified");
 return 0;
 }
}
status = sscanf(s, "%u", &pTestProperties->packetStart);
if ((status == 0) || (pTestProperties->packetStart < 1) ||
 (pTestProperties->packetStart > (1024 * 1024)))
{
 printf("\nERROR: No valid minimum packet size specified, enter a packet size between 1 and %d\n",
 1024 * 1024);
 return 0;
}

/* Ask the user to enter an ending packet size */
printf("\nEnter the maximum size of packet to %s: ", txRxStr);
fflush(stdout);
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
}
else
{
 if (fgets(s, 256, stdin) == NULL)
 {

```

```

 puts("\nERROR: No maximum packet size specified");
 return 0;
 }
}
status = sscanf(s, "%u", &pTestProperties->packetEnd);
if ((status == 0) || (pTestProperties->packetEnd < 1) ||
 (pTestProperties->packetEnd > (1024 * 1024)))
{
 printf("\nERROR: No valid maximum packet size specified, enter a packet size between 1 and %d\n",
 1024 * 1024);
 return 0;
}
if (pTestProperties->packetStart > pTestProperties->packetEnd)
{
 puts("\nERROR: Enter a maximum packet size greater than or equal to the minimum packet size");
 return 0;
}

/* Ask the user to enter the number of bytes to increment the packet size */
/* by in each loop */
if (pTestProperties->packetStart == pTestProperties->packetEnd)
{
 pTestProperties->packetStep = 1;
}
else
{
 printf(
 "\nEnter the number of bytes to increment the packet size by in each loop (0 for log-like
 pattern): ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No packet size increment specified");
 return 0;
 }
 }
 status = sscanf(s, "%u", &pTestProperties->packetStep);
 if (status == 0)
 {
 puts("\nERROR: No valid packet size increment specified");
 return 0;
 }
}

/* Ask the user to enter the number of seconds to transmit each packet size for in timed transmits */
if (pTestProperties->testType == TIME_TEST && PERFORMING_TRANSMITS(pTestProperties->mode))
{
 printf("\nEnter number of seconds each packet size should be transmitted for: ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(s, arguments[currentArgument], 255U);
 s[255U] = '\0';
 currentArgument++;
 }
 else
 {
 if (fgets(s, 256, stdin) == NULL)
 {
 puts("\nERROR: No time specified");
 return 0;
 }
 }
 status = sscanf(s, "%lf", &pTestProperties->testTime);
 if (status == 0)
 {
 puts("\nERROR: No valid time specified");
 return 0;
 }
}

/* Allocate memory for the channels */
if (PERFORMING_TRANSMITS(pTestProperties->mode))
{
 pTestProperties->pTxChannelIds = (STAR_CHANNEL_ID *)calloc(
 pTestProperties->testCount, sizeof(STAR_CHANNEL_ID));
}

```

```

 if (pTestProperties->pTxChannelIds == NULL)
 {
 puts(
 "\nERROR: Unable to allocate memory for the transmit channels");
 return 0;
 }
 pTestProperties->pAddressPaths = (STAR_SPACEWIRE_ADDRESS *)calloc(
 pTestProperties->testCount, sizeof(STAR_SPACEWIRE_ADDRESS *));
 if (pTestProperties->pAddressPaths == NULL)
 {
 puts(
 "\nERROR: Unable to allocate memory for the address paths");
 return 0;
 }
}
if (PERFORMING_RECEIVES(pTestProperties->mode))
{
 pTestProperties->pRxChannelIds = (STAR_CHANNEL_ID *)calloc(
 pTestProperties->testCount, sizeof(STAR_CHANNEL_ID));
 if (pTestProperties->pRxChannelIds == NULL)
 {
 puts("\nERROR: Unable to allocate memory for the receive channels");
 return 0;
 }
}

/* For each test to perform */
for (testNumber = 0; testNumber < pTestProperties->testCount; testNumber++)
{
 /* If transmitting packets */
 if (PERFORMING_TRANSMITS(pTestProperties->mode))
 {
 /* Choose the transmit channel */
 pTestProperties->pTxChannelIds[testNumber] = openChannel("transmit",
 testNumber + 1, STAR_CHANNEL_DIRECTION_OUT, argCount, arguments,
 ¤tArgument, (pTestProperties->useChannel0 != 0) ? 0 : 1);
 if (pTestProperties->pTxChannelIds[testNumber] == 0U)
 {
 return 0;
 }

 /* Ask the user to enter the path to add to the front of packets */
 printf("\nEnter the path to add to the front of packets sent for test %d:\n",
 testNumber);
 puts("(Values should be in hex, separated by a space, i.e.: 01 0f 02)");

 /* Read in the path */
 pTestProperties->pAddressPaths[testNumber] = readSpaceWireAddress(
 argCount, arguments, ¤tArgument);
 }

 /* If receiving packets */
 if (PERFORMING_RECEIVES(pTestProperties->mode))
 {
 /* Choose the receive channel */
 pTestProperties->pRxChannelIds[testNumber] = openChannel("receive",
 testNumber + 1, STAR_CHANNEL_DIRECTION_IN, argCount, arguments,
 ¤tArgument, (pTestProperties->useChannel0 != 0) ? 0 : 1);
 if (pTestProperties->pRxChannelIds[testNumber] == 0U)
 {
 return 0;
 }
 }
}

/* Ask the user to enter the path to the file to use, */
/* or to press enter to not store to a file */
printf("\nEnter the path and the file to be used to store the results (hit enter if the results should
 not be stored): ");
fflush(stdout);

/* Read in the file path */
if (currentArgument < argCount)
{
 puts(arguments[currentArgument]);
 strncpy(pTestProperties->pFilePath, arguments[currentArgument],
 MAX_PATH);
 pTestProperties->pFilePath[MAX_PATH] = '\0';
 currentArgument++;
}
else
{
 if (fgets(pTestProperties->pFilePath, MAX_PATH, stdin) == NULL)
 {
 strcpy(pTestProperties->pFilePath, "");
 }
}
}

```

```

len = strlen(pTestProperties->pFilePath);
if (len != 0U)
{
 while ((len > 0U) &&
 ((pTestProperties->pFilePath[len - 1] == '\r') ||
 (pTestProperties->pFilePath[len - 1] == '\n')))
 {
 pTestProperties->pFilePath[len - 1] = '\0';
 len--;
 }
}

/* if writing the results to file */
if (len > 0U)
{
 printf("\nEnter a description of the test: ");
 fflush(stdout);
 if (currentArgument < argCount)
 {
 puts(arguments[currentArgument]);
 strncpy(pTestProperties->pFileDescription,
 arguments[currentArgument], 999U);
 pTestProperties->pFileDescription[999U] = '\0';
 }
 else
 {
 if (fgets(pTestProperties->pFileDescription, 999U, stdin) == NULL)
 {
 strcpy(pTestProperties->pFileDescription, "");
 }
 }
}

puts("\n");
return 1;
}

static int waitAndCheckContentOfReceivedItems(
 STAR_TRANSFER_OPERATION ** const ppRxTransOp, const unsigned int bufferNum,
 const unsigned int testCount, const int packetSize, int timeout,
 int noTimeoutError)
{
 STAR_TRANSFER_STATUS status;
 STAR_STREAM_ITEM **receivedItems;
 unsigned int receivedItemCount, packetItem;
 STAR_SPACEWIRE_PACKET *pReceivedPacket;
 unsigned int rxPacketSize;
 STAR_EOP_TYPE rxEopType;
 unsigned int i;

 for (i = 0; i < testCount; i++)
 {
 /* Wait on the receive operation completing */
 status = STAR_waitOnTransferOperationCompletion(
 ppRxTransOp[(bufferNum * testCount) + i], timeout);
 /* If timeout */
 if (status == STAR_TRANSFER_STATUS_STARTED)
 {
 if (!noTimeoutError)
 {
 printf("ERROR: Could not complete receive, error of %d\n",
 status);
 }
 return -1;
 }
 /* If non-timeout error */
 else if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("ERROR: Could not complete receive, error of %d\n", status);
 return 0;
 }

 /* Get the packets received */
 receivedItems = STAR_getTransferItemList(
 ppRxTransOp[(bufferNum * testCount) + i], &receivedItemCount);
 if (receivedItems == NULL)
 {
 puts("ERROR: Receive completed but no data was received");
 return 0;
 }

 /* Check the content of each packet */
 for (packetItem = 0U; packetItem < receivedItemCount; packetItem++)
 {
 pReceivedPacket = (STAR_SPACEWIRE_PACKET *)receivedItems[

```

```

 packetItem]->item;

 rxPacketSize = STAR_getPacketLength(pReceivedPacket);
 if (rxPacketSize != (unsigned int)packetSize)
 {
 printf("Rx operation completed, received incorrect packet size: %u\n",
 rxPacketSize);
 }

 rxEopType = STAR_getPacketEOP(pReceivedPacket);
 if (rxEopType != STAR_EOP_TYPE_EOP)
 {
 printf("Rx operation completed, received unexpected EOP type: %d\n",
 rxEopType);
 }
 }

 /* Destroy the packet list */
 STAR_destroyTransferItemList(receivedItems, receivedItemCount, 0);
}

return 1;
}

static STAR_TRANSFER_OPERATION **allocateAndCreateRxOperations(
 const unsigned int testNumber, const unsigned int operationsPerTest,
 const unsigned int packetNumber)
{
 STAR_TRANSFER_OPERATION **ppRxTransOp;
 unsigned int i;

 /* Allocate memory for the receive transfer operations */
 ppRxTransOp = (STAR_TRANSFER_OPERATION **)calloc((size_t)testNumber *
 (size_t)operationsPerTest, sizeof(STAR_TRANSFER_OPERATION *));
 if (ppRxTransOp == NULL)
 {
 puts("Couldn't allocate memory for receive operations, exiting.");
 return NULL;
 }

 /* Create transfer operations to receive all packets in each buffer */
 for (i = 0; i < (testNumber * operationsPerTest); i++)
 {
 ppRxTransOp[i] = STAR_createRxOperation(packetNumber,
 STAR_RECEIVE_PACKETS);
 if (ppRxTransOp[i] == NULL)
 {
 puts("Couldn't create receive operation, exiting.");
 return NULL;
 }
 }

 return ppRxTransOp;
}

static void freeRxOperations(STAR_TRANSFER_OPERATION ** const ppRxTransOp,
 const unsigned int testNumber, const unsigned int operationsPerTest)
{
 unsigned int i;

 /* Dispose of all receive transfer operations */
 if (ppRxTransOp != NULL)
 {
 for (i = 0; i < (testNumber * operationsPerTest); i++)
 {
 if (ppRxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(ppRxTransOp[i]);
 }
 }

 free(ppRxTransOp);
 }
}

typedef struct
{
 unsigned char *pTransmitBuffer;

 STAR_STREAM_ITEM **ppTxPackets;

 STAR_TRANSFER_OPERATION **ppTxTransOp;

 unsigned int testNumber;

 unsigned int operationNumber;

```

```

 unsigned int packetNumber;

} TEST_TRANSMIT_PROPERTIES;

static int allocateAndCreateTxOperations(const int packetSize,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const int * const pPacketLengths,
 TEST_TRANSMIT_PROPERTIES * const pTestProperties)
{
 unsigned int i, j;

 /* Clear test properties */
 pTestProperties->pTransmitBuffer = NULL;
 pTestProperties->ppTxPackets = NULL;
 pTestProperties->ppTxTransOp = NULL;

 /* Allocate transmit buffer */
 /* This buffer will be reused for all packets sent */
 pTestProperties->pTransmitBuffer = (unsigned char *)calloc(1U, packetSize);
 if (pTestProperties->pTransmitBuffer == NULL)
 {
 puts("Couldn't allocate memory for transmit buffer, exiting.");
 return 0;
 }

 /* Set the first byte of the transmit buffer to 0xfe. This is the */
 /* byte that will be at the front of the packet at the destination */
 /* (if path addressing is used) */
 if (packetSize > 0)
 {
 pTestProperties->pTransmitBuffer[0U] = 0xfeU;
 }

 /* Allocate memory for packet pointers */
 pTestProperties->ppTxPackets = (STAR_STREAM_ITEM **)calloc(
 (size_t)pTestProperties->packetNumber *
 (size_t)pTestProperties->testNumber *
 (size_t)pTestProperties->operationNumber,
 sizeof(STAR_STREAM_ITEM *));
 if (pTestProperties->ppTxPackets == NULL)
 {
 puts("Couldn't allocate memory for transmit packets, exiting.");
 return 0;
 }

 /* Allocate memory for the transmit operations */
 pTestProperties->ppTxTransOp = (STAR_TRANSFER_OPERATION **)calloc(
 (size_t)pTestProperties->testNumber *
 (size_t)pTestProperties->operationNumber,
 sizeof(STAR_TRANSFER_OPERATION *));
 if (pTestProperties->ppTxTransOp == NULL)
 {
 puts("Couldn't allocate memory for transmit operations, exiting.");
 return 0;
 }

 /* Construct packets */
 for (i = 0; i < (pTestProperties->packetNumber *
 pTestProperties->operationNumber); i++)
 {
 for (j = 0; j < pTestProperties->testNumber; j++)
 {
 /* Create stream item using the transmit buffer */
 pTestProperties->ppTxPackets[(i * pTestProperties->testNumber) +
 j] = STAR_createPacket(pAddressPaths[j],
 pTestProperties->pTransmitBuffer,
 ((pPacketLengths != NULL) ? pPacketLengths[j] : packetSize),
 STAR_EOP_TYPE_EOP);
 if (pTestProperties->ppTxPackets[(i * pTestProperties->testNumber) +
 j] == NULL)
 {
 puts("Couldn't create packet for transmit operation, exiting.");
 return 0;
 }
 }
 }

 /* Create transfer operations to transmit all packets in each buffer */
 for (i = 0; i < (pTestProperties->testNumber *
 pTestProperties->operationNumber); i++)
 {
 pTestProperties->ppTxTransOp[i] = STAR_createTxOperation(
 pTestProperties->ppTxPackets +
 ((size_t)pTestProperties->packetNumber * (size_t)i),
 pTestProperties->packetNumber);
 if (pTestProperties->ppTxTransOp[i] == NULL)

```

```

 {
 puts("Couldn't create transmit operation, exiting.");
 return 0;
 }
 }

 return 1;
}

static void freeTxOperations(TEST_TRANSMIT_PROPERTIES * const pTestProperties)
{
 unsigned int i;

 /* Free the transfer operation pointers */
 if (pTestProperties->ppTxTransOp != NULL)
 {
 /* Dispose of all transmit transfer operations */
 for (i = 0; i < (pTestProperties->testNumber *
 pTestProperties->operationNumber); i++)
 {
 if (pTestProperties->ppTxTransOp[i] != NULL)
 {
 STAR_disposeTransferOperation(pTestProperties->ppTxTransOp[i])
 }
 }

 free(pTestProperties->ppTxTransOp);
 }

 if (pTestProperties->ppTxPackets != NULL)
 {
 /* Dispose of stream item buffer */
 for (i = 0; i < (pTestProperties->packetNumber *
 pTestProperties->testNumber * pTestProperties->operationNumber);
 i++)
 {
 if (pTestProperties->ppTxPackets[i] != NULL)
 {
 STAR_destroyStreamItem(pTestProperties->ppTxPackets[i]);
 }
 }

 /* Free the packet pointer array*/
 free(pTestProperties->ppTxPackets);
 }

 /* Free the transmit buffer */
 if (pTestProperties->pTransmitBuffer != NULL)
 {
 free(pTestProperties->pTransmitBuffer);
 }
}

static double getTotalAddressLengths(
 STAR_SPACEWIRE_ADDRESS **const pAddressPaths, const unsigned int testCount)
{
 double bytesSent = 0;
 unsigned int i;

 if (pAddressPaths != NULL)
 {
 for (i = 0; i < testCount; i++)
 {
 if (pAddressPaths[i] != NULL)
 {
 bytesSent += STAR_getSpaceWireAddressPathLength(
 pAddressPaths[i]);
 }
 }
 }

 return bytesSent;
}

static int submitTransferOperationForEachTest(
 const STAR_CHANNEL_ID *const pChannels,
 STAR_TRANSFER_OPERATION **const ppTransOp, const unsigned int bufferNum,
 const unsigned int testCount, const char *const operationType)
{
 unsigned int i;
 STAR_TRANSFER_STATUS status;

 for (i = 0; i < testCount; i++)
 {
 if (STAR_submitTransferOperation(pChannels[i],
 ppTransOp[(bufferNum * testCount) + i]) == 0)
 }

```

```

 {
 printf("ERROR: Could not submit %s operation\n",
 operationType);
 return 0;
 }

 status = STAR_getTransferOperationStatus(
 ppTransOp[(bufferNum * testCount) + i]);
 if ((status != STAR_TRANSFER_STATUS_STARTED) &&
 (status != STAR_TRANSFER_STATUS_COMPLETE))
 {
 printf("ERROR: Could not perform %s operation, error of %d\n",
 operationType, status);
 return 0;
 }
 }

 return 1;
}

static int waitOnTransferOperationCompletionForEachTest(
 STAR_TRANSFER_OPERATION ** const ppTransOp, const unsigned int bufferNum,
 const unsigned int testCount, const char * const operationType,
 int timeout, int noError)
{
 unsigned int i;
 STAR_TRANSFER_STATUS status;

 for (i = 0; i < testCount; i++)
 {
 status = STAR_waitOnTransferOperationCompletion(
 ppTransOp[(bufferNum * testCount) + i],
 timeout);
 /* If not complete and errors are to be shown */
 /* In the case of the timed test, it receives until it can't, */
 /* so it's better not to show an error message then. */
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 if (!noError)
 {
 printf("ERROR: Could not complete %s operation, error of %d\n",
 operationType, status);
 }
 return 0;
 }
 }

 return 1;
}

static void writeTestHeaders(FILE *const pOutputFile)
{
 /* If a file is to be written to */
 if (pOutputFile != NULL)
 {
 /* Write the statistics column headers to the file */
 fprintf(pOutputFile, "Packet Size\tData Rate (Mbit/s)\t");
 fprintf(pOutputFile,
 "Packet Rate (packets/s)\tAverage Packet Time (microseconds)");
 PRINT_CPU_USAGE_HEADERS(pOutputFile);
 }
}

static void writeTestResults(FILE *const pOutputFile,
 const unsigned int packetSize, const double dataRate,
 const double packetRate, const double packetTime,
 const CPU_USAGE_PROPERTIES *const pStartCpuProperties,
 const CPU_USAGE_PROPERTIES *const pEndCpuProperties)
{
 /* Display the results of the test */
 printf(
 "\t%-3.2f Mbit/s %-3.2f packets/s %-3.2f microseconds/packet\n",
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(NULL, pStartCpuProperties, pEndCpuProperties);

 /* If writing the results to file */
 if (pOutputFile != NULL)
 {
 /* Write the results to the file */
 fprintf(pOutputFile, "%8d\t%-3.2f\t%-3.2f\t%-3.2f", packetSize,
 dataRate, packetRate, packetTime);
 PRINT_CPU_USAGE(pOutputFile, pStartCpuProperties, pEndCpuProperties);
 fflush(pOutputFile);
 }
}

```



```

static void performRateTestForPacketSize(const unsigned int packetSize,
 const U8 mode, const unsigned int testCount,
 const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 unsigned int i, bufferNum, buffersUsed, loopNum, packetNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bitsSent;
 TEST_TRANSMIT_PROPERTIES testProperties;

 /* Clear the data rate values */
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;

 /* Calculate the number of packets to transmit in each loop and */
 /* the number of loops to perform */
 packetNum = MAX(BUFFER_NUM, BUFFER_SIZE / packetSize);
 testProperties.testNumber = testCount;
 loopNum = LOOP_NUM;

#ifdef DEBUG
 printf("\nPackets per loop:\t%d", packetNum);
 printf("\nNumber of loops:\t%d", loopNum);
 printf("\nBytes per packet:\t%d", packetSize);
 printf("\nExpected packets:\t%u", loopNum * packetNum);
 printf("\nExpected bytes:\t%f\n",
 (double)packetSize * (double)packetNum * (double)loopNum);
 printf("\n");
#endif

 /* if performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Allocate and create each of the receive operations */
 ppRxTransOp = allocateAndCreateRxOperations(testCount, BUFFER_NUM,
 packetNum);
 if (ppRxTransOp == NULL)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 testProperties.packetNumber = packetNum;
 testProperties.testNumber = testCount;
 testProperties.operationNumber = BUFFER_NUM;

 /* Create the transmit operations and their packets */
 if (allocateAndCreateTxOperations(packetSize, pAddressPaths, NULL,
 &testProperties) == 0)
 {
 goto Rate_End_Cleanup;
 }

 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }

 bufferNum = 0;

 /* for each loop to perform */
 for (i = 0; i < loopNum; i++)
 {
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* If a receive has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on the receive completing and check its content */
 if (waitAndCheckContentOfReceivedItems(ppRxTransOp, bufferNum,
 testCount, packetSize, STAR_INFINITE, 0) == 0)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* If this is the first receive and not performing transmits */
 if ((i == BUFFER_NUM) && (!PERFORMING_TRANSMITS(mode)))

```

```

 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }

 /* Start receiving the next group of packets */
 if (submitTransferOperationForEachTest(pRxChannels, ppRxTransOp,
 bufferNum, testCount, "receive") == 0)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* If a transmit has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on the last transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, bufferNum, testCount,
 "transmit", STAR_INFINITE, 0) == 0)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* Start transmitting the next packets */
 if (submitTransferOperationForEachTest(pTxChannels,
 testProperties.ppTxTransOp, bufferNum, testCount, "transmit") ==
 0)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* Move to the next buffer */
 bufferNum++;
 if (bufferNum == BUFFER_NUM)
 {
 bufferNum = 0;
 }
}

/* Account for LOOP_NUM less than BUFFER_NUM */
buffersUsed = BUFFER_NUM;
#ifdef (LOOP_NUM < BUFFER_NUM)
 buffersUsed = LOOP_NUM;
#endif
/* For each buffer used */
for (bufferNum = 0; bufferNum < buffersUsed; bufferNum++)
{
 /* If performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Wait on the next transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, bufferNum, testCount, "transmit",
 STAR_INFINITE, 0) == 0)
 {
 goto Rate_End_Cleanup;
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Wait on packets being received */
 if (waitAndCheckContentOfReceivedItems(ppRxTransOp, bufferNum,
 testCount, packetSize, STAR_INFINITE, 0) == 0)
 {
 goto Rate_End_Cleanup;
 }
 }
}

/* Stop the clock */
finish = GET_TIME();
STORE_CPU_USAGE(pEndCpuProperties);

duration = ((double)finish - (double)start) / TIME_DIVIDER;

bitsSent = (double)testCount * (double)packetSize;
bitsSent += getTotalAddressLengths(pAddressPaths, testCount);

```

```

bitsSent = bitsSent * (double)packetNum * (double)loopNum * (double)8;

/* Calculate the data and packet rate */
if (duration > 0.0)
{
 *pDataRate = (bitsSent / duration) / 1000000.0;
 *pPacketRate = ((double)packetNum * (double)testCount *
 (double)loopNum) / duration;
}
else
{
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
}
*pPacketTime = (duration * 1000000.0) /
 ((double)packetNum * (double)testCount * (double)loopNum);

/* Clean-up */
Rate_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Free all receive operations */
 freeRxOperations(ppRxTransOp, BUFFER_NUM, testCount);
}

/* If performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Free the transmit operations and their packets */
 freeTxOperations(&testProperties);
}
}

static void performRateTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 unsigned int packetSize;
 double dataRate = 0, packetRate = 0, packetTime = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* Write the statistics column headers to the file */
 writeTestHeaders(pOutputFile);

 /* for each packet size to test */
 packetSize = pTestProperties->packetStart;
 while (packetSize <= pTestProperties->packetEnd)
 {
 /* Display the packet size on screen */
 printf(
 "Performing maximum data rate test using packets of %d bytes...\n",
 packetSize);

 /* Perform the test for the current packet size */
 performRateTestForPacketSize(packetSize, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &startCpuProperties,
 &endCpuProperties);

 /* Display the results of the test and write them to file */
 writeTestResults(pOutputFile, packetSize, dataRate, packetRate,
 packetTime, &startCpuProperties, &endCpuProperties);

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (packetSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }

 /* Move to the next packet size */
 if (packetSize == pTestProperties->packetEnd)
 {
 packetSize++;
 }
 else if (pTestProperties->packetStep == 0)
 {
 packetSize = MIN(2 * packetSize, pTestProperties->packetEnd);
 }
 else
 {
 packetSize = MIN(packetSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
 }
}

```

```

 }
}

static void performRandomTestForPacketSize(const unsigned int minPacketSize,
 const unsigned int maxPacketSize, const U8 mode,
 const unsigned int testCount, const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 double * const pPacketSize,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 unsigned int i, j, bufferNum, buffersUsed, packetNum, loopNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bytesSent = 0.0;
 int *pPacketLengths = NULL;
 TEST_TRANSMIT_PROPERTIES testProperties;

 /* Clear the data rate values */
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;

 /* Seed random number function */
 srand((unsigned int)time(NULL));

 /* Calculate the number of packets to transmit in each loop and */
 /* the number of loops to perform */
 packetNum = MAX(BUFFER_NUM, BUFFER_SIZE / maxPacketSize);
 loopNum = LOOP_NUM;

 /* Allocate memory to hold length info for packets */
 pPacketLengths = (int *)calloc((size_t)packetNum * BUFFER_NUM, sizeof(int));
 if (pPacketLengths == NULL)
 {
 puts("Couldn't allocate memory for the packet lengths, exiting.");
 goto Random_End_Cleanup;
 }

 for (i = 0; i < (packetNum * BUFFER_NUM); i++)
 {
 /* Set the initial random lengths for the packets */
 pPacketLengths[i] = ((int)(((double)rand() / ((double)RAND_MAX + 1.0)) *
 ((double)maxPacketSize - (double)minPacketSize))) + minPacketSize;
 }

 /* if performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Allocate and create each of the receive operations */
 ppRxTransOp = allocateAndCreateRxOperations(testCount, BUFFER_NUM,
 packetNum);
 if (ppRxTransOp == NULL)
 {
 goto Random_End_Cleanup;
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 testProperties.packetNumber = packetNum;
 testProperties.testNumber = testCount;
 testProperties.operationNumber = BUFFER_NUM;

 /* Create the transmit operations and their packets */
 if (allocateAndCreateTxOperations(maxPacketSize, pAddressPaths,
 pPacketLengths, &testProperties) == 0)
 {
 goto Random_End_Cleanup;
 }

 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }

 bufferNum = 0;

 /* for each loop to perform */
 for (i = 0; i < loopNum; i++)
 {
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))

```

```

{
 /* If a receive has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on packets being received */
 if (waitOnTransferOperationCompletionForEachTest(ppRxTransOp,
 bufferNum, testCount, "receive", STAR_INFINITE, 0) == 0)
 {
 goto Random_End_Cleanup;
 }

 /* If this is the first receive and not performing transmits */
 if ((i == BUFFER_NUM) && (!PERFORMING_TRANSMITS(mode)))
 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }
 }

 /* Start receiving the next group of packets */
 if (submitTransferOperationForEachTest(pRxChannels, ppRxTransOp,
 bufferNum, testCount, "receive") == 0)
 {
 goto Random_End_Cleanup;
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* If a transmit op has already been submitted for this buffer */
 if (i >= BUFFER_NUM)
 {
 /* Wait on the last transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, bufferNum, testCount,
 "transmit", STAR_INFINITE, 0) == 0)
 {
 goto Random_End_Cleanup;
 }
 }
}

/* Update the number of bytes transmitted/received */
for (j = 0; j < packetNum; j++)
{
 bytesSent += ((double)pPacketLengths[(packetNum * bufferNum) + j] *
 (double)testCount);
}
bytesSent += packetNum *
 getTotalAddressLengths(pAddressPaths, testCount);

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Note that this current version of the Performance Tester does */
 /* not change the size of the data in each loop. This is due to */
 /* an as yet unfixed bug in STAR_setPacketData */

 /* Start transmitting the next packets */
 if (submitTransferOperationForEachTest(pTxChannels,
 testProperties.ppTxTransOp, bufferNum, testCount, "transmit") ==
 0)
 {
 goto Random_End_Cleanup;
 }
}

/* Move to the next buffer */
bufferNum++;
if (bufferNum == BUFFER_NUM)
{
 bufferNum = 0;
}
}

/* Account for LOOP_NUM less than BUFFER_NUM */
buffersUsed = BUFFER_NUM;
#if (LOOP_NUM < BUFFER_NUM)
 buffersUsed = LOOP_NUM;
#endif

/* For each buffer used */
for (bufferNum = 0; bufferNum < buffersUsed; bufferNum++)
{
 /* If performing transmits */

```

```

 if (PERFORMING_TRANSMITS(mode))
 {
 /* Wait on the next transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, bufferNum, testCount,
 "transmit", STAR_INFINITE, 0) == 0)
 {
 goto Random_End_Cleanup;
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Wait on packets being received */
 if (waitOnTransferOperationCompletionForEachTest(ppRxTransOp,
 bufferNum, testCount, "receive", STAR_INFINITE, 0) == 0)
 {
 goto Random_End_Cleanup;
 }
 }
 }

 /* Stop the clock */
 finish = GET_TIME();
 STORE_CPU_USAGE(pEndCpuProperties);

 duration = ((double)finish - (double)start) / TIME_DIVIDER;

 /* Calculate the average packet size */
 *pPacketSize = (double)bytesSent / ((double)packetNum * (double)testCount *
 (double)loopNum);

 /* Calculate the data and packet rate */
 if (duration > 0.0)
 {
 *pDataRate = ((bytesSent * 8.0) / duration) / 1000000.0;
 *pPacketRate = ((double)packetNum * (double)testCount *
 (double)loopNum) / duration;
 }
 else
 {
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 }

 *pPacketTime = (duration * 1000000.0) /
 ((double)packetNum * (double)testCount * (double)loopNum);

/* Clean-up */
Random_End_Cleanup:

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Free all receive operations */
 freeRxOperations(ppRxTransOp, BUFFER_NUM, testCount);
 }

 /* If performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Free the transmit operations and their packets */
 freeTxOperations(&testProperties);
 }

 /* Free the array of packet lengths */
 if (pPacketLengths != NULL)
 {
 free(pPacketLengths);
 }
}

static void performRandomTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 unsigned int minPacketSize;
 double dataRate = 0, packetRate = 0, packetTime = 0, packetSize = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* If a file is to be written to */
 if (pOutputFile != NULL)
 {
 /* Write the statistics column headers to the file */
 fprintf(pOutputFile,
 "Minimum Packet Size\tMaximum Packet Size\tData Rate (Mbit/s)\t");
 }

```

```

 fprintf(pOutputFile,
 "Packet Rate (packets/s)\tAverage Packet Time (microseconds)\t");
 fprintf(pOutputFile,
 "Average Packet Size (bytes)");
 PRINT_CPU_USAGE_HEADERS(pOutputFile);
}

/* for each packet size to test */
minPacketSize = pTestProperties->packetStart;
while (minPacketSize <= pTestProperties->packetEnd)
{
 /* Display the packet size on screen */
 printf("Performing random packet size data rate test using packets of size %d to %d bytes...\n",
 minPacketSize, pTestProperties->packetEnd);

 /* Perform the test for the current packet size */
 performRandomTestForPacketSize(minPacketSize,
 pTestProperties->packetEnd, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &packetSize,
 &startCpuProperties, &endCpuProperties);

 /* Display the results of the test */
 printf("\t%-3.2f Mbit/s %-3.2f packets/s %-3.2f microseconds/packet %-3.2f average packet size\n",
 dataRate, packetRate, packetTime, packetSize);
 PRINT_CPU_USAGE(NULL, &startCpuProperties, &endCpuProperties);

 /* If writing the results to file */
 if (pOutputFile != NULL)
 {
 /* Write the results to the file */
 fprintf(pOutputFile, "%8d\t%8d\t%-3.2f\t%-3.2f\t%-3.2f\t%-3.2f",
 minPacketSize, pTestProperties->packetEnd, dataRate, packetRate, packetTime,
 packetSize);
 PRINT_CPU_USAGE(pOutputFile, &startCpuProperties,
 &endCpuProperties);
 fflush(pOutputFile);
 }

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (minPacketSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }

 /* Move to the next packet size */
 if (minPacketSize == pTestProperties->packetEnd)
 {
 minPacketSize++;
 }
 else if ((minPacketSize == pTestProperties->packetStart) &&
 (pTestProperties->packetStep != 0) && ((minPacketSize +
 pTestProperties->packetStep) > pTestProperties->packetEnd))
 {
 minPacketSize = pTestProperties->packetEnd + 1;
 }
 else if (pTestProperties->packetStep == 0)
 {
 minPacketSize = MIN(2 * minPacketSize, pTestProperties->packetEnd);
 }
 else
 {
 minPacketSize = MIN(minPacketSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
}

static void performLatencyTestForPacketSize(const unsigned int packetSize,
 const U8 mode, const unsigned int testCount,
 const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties)
{
 unsigned int i, loopNum;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL;
 clock_t start = 0, finish;
 double duration, bitsSent;
 TEST_TRANSMIT_PROPERTIES testProperties;

```

```

/* Clear the data rate values */
*pDataRate = 0.0;
*pPacketRate = 0.0;
*pPacketTime = 0.0;

/* Calculate the number of loops to perform */
loopNum = LOOP_NUM;

/* if performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Allocate and create each of the receive operations */
 ppRxTransOp = allocateAndCreateRxOperations(testCount, 1, 1);
 if (ppRxTransOp == NULL)
 {
 goto Latency_End_Cleanup;
 }
}

/* if performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 testProperties.packetNumber = 1;
 testProperties.operationNumber = 1;
 testProperties.testNumber = testCount;

 /* Create the transmit operations and their packets */
 if (allocateAndCreateTxOperations(packetSize, pAddressPaths, NULL,
 &testProperties) == 0)
 {
 goto Latency_End_Cleanup;
 }

 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
}

/* for each loop to perform */
for (i = 0; i < loopNum; i++)
{
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Start receiving the next packet */
 if (submitTransferOperationForEachTest(pRxChannels, ppRxTransOp,
 0, testCount, "receive") == 0)
 {
 goto Latency_End_Cleanup;
 }
 }

 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 /* Start transmitting the packet */
 if (submitTransferOperationForEachTest(pTxChannels,
 testProperties.ppTxTransOp, 0, testCount, "transmit") ==
 0)
 {
 goto Latency_End_Cleanup;
 }
 }

 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Wait on packets being received */
 if (waitOnTransferOperationCompletionForEachTest(ppRxTransOp, 0,
 testCount, "receive", STAR_INFINITE, 0) == 0)
 {
 goto Latency_End_Cleanup;
 }

 /* If this is the first receive and not performing transmits */
 if ((i == 0) && (!PERFORMING_TRANSMITS(mode)))
 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }
 }

 /* If performing transmits but not performing receives */
 if (PERFORMING_TRANSMITS(mode) && (!PERFORMING_RECEIVES(mode)))
 {

```



```

 /* Wait on the transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, 0, testCount, "transmit", STAR_INFINITE, 0) == 0)
 {
 goto Latency_End_Cleanup;
 }
 }
}

/* Stop the clock */
finish = GET_TIME();
STORE_CPU_USAGE(pEndCpuProperties);

duration = ((double)finish - (double)start) / TIME_DIVIDER;
bitsSent = (double)packetSize * (double)testCount;
bitsSent += getTotalAddressLengths(pAddressPaths, testCount);
bitsSent = bitsSent * (double)loopNum * (double)8;

/* Calculate the data and packet rate */
if (duration > 0.0)
{
 *pDataRate = (bitsSent / duration) / 1000000.0;
 *pPacketRate = ((double)loopNum * (double)testCount) / duration;
}
else
{
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
}
*pPacketTime = (duration * 1000000.0) /
 ((double)loopNum * (double)testCount);

/* Clean-up */
Latency_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Free all receive operations */
 freeRxOperations(ppRxTransOp, 1, 1);
}

/* If performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Free the transmit operations and their packets */
 freeTxOperations(&testProperties);
}
}

static void performLatencyTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 unsigned int packetSize;
 double dataRate = 0, packetRate = 0, packetTime = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* Write the statistics column headers to the file */
 writeTestHeaders(pOutputFile);

 /* for each packet size to test */
 packetSize = pTestProperties->packetStart;
 while (packetSize <= pTestProperties->packetEnd)
 {
 /* Display the packet size on screen */
 printf("Performing latency test using packets of %d bytes...\n",
 packetSize);

 /* Perform the test for the current packet size */
 performLatencyTestForPacketSize(packetSize, pTestProperties->mode,
 pTestProperties->testCount, pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime, &startCpuProperties,
 &endCpuProperties);

 /* Display the results of the test and write them to file */
 writeTestResults(pOutputFile, packetSize, dataRate, packetRate,
 packetTime, &startCpuProperties, &endCpuProperties);

 /* if not performing receives and this isn't the last loop */
 if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (packetSize < pTestProperties->packetEnd))
 {
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
 }
 }
}

```

```

 /* Move to the next packet size */
 if (packetSize == pTestProperties->packetEnd)
 {
 packetSize++;
 }
 else if (pTestProperties->packetStep == 0)
 {
 packetSize = MIN(2 * packetSize, pTestProperties->packetEnd);
 }
 else
 {
 packetSize = MIN(packetSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
 }
 }
}

static void performTimedTestForPacketSize(const unsigned int packetSize,
 const U8 mode, const unsigned int testCount,
 const double testTime,
 const STAR_CHANNEL_ID * const pTxChannels,
 STAR_SPACEWIRE_ADDRESS ** const pAddressPaths,
 const STAR_CHANNEL_ID * const pRxChannels, double * const pDataRate,
 double * const pPacketRate, double * const pPacketTime,
 CPU_USAGE_PROPERTIES * const pStartCpuProperties,
 CPU_USAGE_PROPERTIES * const pEndCpuProperties,
 double * const pDuration)
{
 unsigned int loopNum, packetNum, bufferNum, extraLoops;
 STAR_TRANSFER_OPERATION **ppRxTransOp = NULL;
 clock_t start = 0;
 clock_t finish = 0;
 double bitsSent;
 TEST_TRANSMIT_PROPERTIES testProperties;
 unsigned int isTimeout = 1;
 unsigned int stillTransmitting = 1;

 /* Clear the data rate values */
 *pDataRate = 0.0;
 *pPacketRate = 0.0;
 *pPacketTime = 0.0;

 /* Calculate the number of packets to transmit in each loop */
 packetNum = MAX(BUFFER_NUM, BUFFER_SIZE / packetSize);

 testProperties.testNumber = testCount;

 /* if performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* If only performing receives i.e. timeout is needed */
 if (!PERFORMING_TRANSMITS(mode))
 {
 /* No timeout to begin with */
 isTimeout = 0;
 }
 /* Allocate and create each of the receive operations */
 ppRxTransOp = allocateAndCreateRxOperations(testCount, BUFFER_NUM,
 packetNum);
 if (ppRxTransOp == NULL)
 {
 goto Time_End_Cleanup;
 }
 }
 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode))
 {
 testProperties.packetNumber = packetNum;
 testProperties.testNumber = testCount;
 testProperties.operationNumber = BUFFER_NUM;

 /* Create the transmit operations and their packets */
 if (allocateAndCreateTxOperations(packetSize, pAddressPaths, NULL,
 &testProperties) == 0)
 {
 goto Time_End_Cleanup;
 }

 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }

 loopNum = 0;

```

```

bufferNum = 0;
extraLoops = 0;

/* Run for time specified */
do
{
 /* If performing receives and receive already submitted for this buffer */
 if (PERFORMING_RECEIVES(mode) && loopNum >= BUFFER_NUM)
 {
 /* If on first receive */
 if (loopNum == BUFFER_NUM)
 {
 /* Wait on packets being received */
 if (waitAndCheckContentOfReceivedItems(ppRxTransOp,
 bufferNum, testCount, packetSize, STAR_INFINITE, 0) == 0)
 {
 goto Time_End_Cleanup;
 }
 /* If this is the first receive and not performing transmits */
 if (!PERFORMING_TRANSMITS(mode))
 {
 /* Start the clock */
 start = GET_TIME();
 STORE_CPU_USAGE(pStartCpuProperties);
 }
 }
 /* If not on first loop */
 else
 {
 isTimeout = (waitAndCheckContentOfReceivedItems(ppRxTransOp,
 bufferNum, testCount, packetSize, TIMEOUT_TIME, 1) == -1);
 /* If timed out */
 if (isTimeout)
 {
 /* Cancel receive op */
 STAR_cancelTransferOperation(*ppRxTransOp);
 /* Don't count this loop when calculating bits sent */
 extraLoops++;
 }
 }
 }
 /* If performing transmits and transmit submitted for this buffer */
 if (PERFORMING_TRANSMITS(mode) && stillTransmitting &&
 loopNum >= BUFFER_NUM)
 {
 /* Wait on the transmit completing */
 if (waitOnTransferOperationCompletionForEachTest(
 testProperties.ppTxTransOp, bufferNum, testCount, "transmit",
 STAR_INFINITE, 0) == 0)
 {
 goto Time_End_Cleanup;
 }
 }
 /* If performing receives */
 if (PERFORMING_RECEIVES(mode))
 {
 /* Start receiving the next group of packets */
 if (submitTransferOperationForEachTest(pRxChannels, ppRxTransOp,
 bufferNum, testCount, "receive") == 0)
 {
 goto Time_End_Cleanup;
 }
 }
 /* if performing transmits */
 if (PERFORMING_TRANSMITS(mode) && stillTransmitting)
 {
 /* Start transmitting the packets */
 if (submitTransferOperationForEachTest(pTxChannels,
 testProperties.ppTxTransOp, bufferNum, testCount, "transmit")
 == 0)
 {
 goto Time_End_Cleanup;
 }
 }
 /* Update the finish time */
 if (!isTimeout || stillTransmitting)
 {
 finish = GET_TIME();
 }
 /* Move onto the next buffer */
 bufferNum++;
 if (bufferNum == BUFFER_NUM)
 {
 bufferNum = 0;
 }
}

```

```

 /* If the user specified time hasn't yet elapsed, keep transmitting */
 stillTransmitting = (((double)GET_TIME() - (double)start)
 < (testTime * TIME_DIVIDER));
 }
 loopNum++;
 /* Repeat while still transmitting, not timed out or not filled buffer */
} while (stillTransmitting || !isTimeout || bufferNum);

STORE_CPU_USAGE(pEndCpuProperties);

bitsSent = (double)packetSize * (double)testCount;
bitsSent += getTotalAddressLengths(pAddressPaths, testCount);
bitsSent = bitsSent * (double)packetNum * (double)(loopNum - extraLoops)
 * (double)8;

/* Calculate the data and packet rate */
*pDuration = ((double)finish - (double)start) / TIME_DIVIDER;
if (*pDuration > 0.0)
{
 *pDataRate = (bitsSent / *pDuration) / 1000000.0;
 *pPacketRate = ((double)packetNum * (double)(loopNum - extraLoops) *
 (double)testCount) / *pDuration;
}
else
{
 *pDataRate = 0;
 *pPacketRate = 0;
}
*pPacketTime = (*pDuration * 1000000.0) /
 ((double)packetNum * (double)(loopNum - extraLoops) *
 (double)testCount);

/* Clean-up */
Time_End_Cleanup:

/* If performing receives */
if (PERFORMING_RECEIVES(mode))
{
 /* Free all receive operations */
 freeRxOperations(ppRxTransOp, BUFFER_NUM, testCount);
}

/* If performing transmits */
if (PERFORMING_TRANSMITS(mode))
{
 /* Free the transmit operations and their packets */
 freeTxOperations(&testProperties);
}
}

static void performTimedTest(const TEST_PROPERTIES * const pTestProperties,
 FILE * const pOutputFile)
{
 unsigned int packetSize;
 double testTime;
 double dataRate = 0, packetRate = 0, packetTime = 0, duration = 0;
 CPU_USAGE_PROPERTIES startCpuProperties, endCpuProperties;

 /* Write the statistics column headers to the file */
 writeTestHeaders(pOutputFile);

 /* for each packet size to test */
 packetSize = pTestProperties->packetStart;
 testTime = pTestProperties->testTime;
 while (packetSize <= pTestProperties->packetEnd)
 {
 /* If the seconds taken needs to be shown */
 if (PERFORMING_TRANSMITS(pTestProperties->mode))
 {
 /* Display the packet size and test time on screen */
 printf("Performing %f second timed test using packets of %d bytes...\n",
 testTime, packetSize);
 }
 /* Otherwise if the test simply receive packets until a timeout */
 else
 {
 /* Display the packet size on screen */
 printf("Performing timed test using packets of %d bytes...\n",
 packetSize);
 }
 /* Perform the test for the current packet size */
 performTimedTestForPacketSize(packetSize, pTestProperties->mode,
 pTestProperties->testCount, testTime,
 pTestProperties->pTxChannelIds,
 pTestProperties->pAddressPaths, pTestProperties->pRxChannelIds,
 &dataRate, &packetRate, &packetTime,

```

```

 &startCpuProperties, &endCpuProperties,
 &duration);

/* Display the results of the test and write them to file */
printf("\tActual time taken = %lfs", duration);
if (PERFORMING_RECEIVES(pTestProperties->mode))
{
 printf(" + %ds wait to ensure all packets received.",
 ((TIMEOUT_TIME * BUFFER_NUM) / 1000));
}
printf("\n");
writeTestResults(pOutputFile, packetSize, dataRate, packetRate,
 packetTime, &startCpuProperties, &endCpuProperties);

/* if not performing receives and this isn't the last loop */
if ((!PERFORMING_RECEIVES(pTestProperties->mode)) &&
 (packetSize < pTestProperties->packetEnd))
{
 /* Sleep for 5 seconds to allow all packets to be sent */
 SLEEP(5000U);
}

/* Move to the next packet size */
if (packetSize == pTestProperties->packetEnd)
{
 packetSize++;
}
else if (pTestProperties->packetStep == 0)
{
 packetSize = MIN(2 * packetSize, pTestProperties->packetEnd);
}
else
{
 packetSize = MIN(packetSize + pTestProperties->packetStep,
 pTestProperties->packetEnd);
}
}
}

static int readArguments(TEST_PROPERTIES * const pTestProperties,
 const int argCount, const char * const arguments[])
{
 int processedArguments = 1;

 /* While not reached the end of the arguments */
 while (processedArguments < argCount)
 {
 /* If the argument begins with a hyphen */
 if ((strlen(arguments[processedArguments]) > 1U) &&
 (arguments[processedArguments][0U] == '-'))
 {
 /* If the argument is to use channel 0, enable its use */
 if (strcasecmp(arguments[processedArguments], "-usechannel0") == 0)
 {
 pTestProperties->useChannel0 = 1;
 }
 /* Otherwise this argument isn't supported, so display a warning */
 else
 {
 printf("Unsupported argument \"%s\" ignored\n",
 arguments[processedArguments]);
 }
 processedArguments++;
 }
 else
 {
 break;
 }
 }

 return processedArguments;
}

int main(int argc, char* argv[])
{
 TEST_PROPERTIES testProperties = {{ 0 }};
 FILE *pOutputFile = NULL;
 int processedArguments;

 STAR_setApplicationName("STAR-System Performance Tester Application");

 /* Read in any arguments */
 processedArguments = readArguments(&testProperties, argc,
 (const char * const *)argv);

 /* Read the test parameters */

```

```

if (readTestParameters(&testProperties, argc - processedArguments,
 (const char * const *) (argv + processedArguments)) == 0)
{
 cleanupTest(&testProperties);
 return 0;
}

/* if writing the results to file */
if (strlen(testProperties.pFilePath) > 0U)
{
 /* Open the file */
 pOutputFile = fopen(testProperties.pFilePath, "wt");
 if (pOutputFile == NULL)
 {
 puts("Couldn't open the file to store the statistics, exiting");
 cleanupTest(&testProperties);
 return 0;
 }

 /* Write the description to the file */
 fprintf(pOutputFile, "%s\n\n", testProperties.pFileDescription);
}

/* Test is beginning, so prevent system sleeping */
DISABLE_SYSTEM_SLEEP();

switch (testProperties.testType)
{
 case RATE_TEST:
 performRateTest(&testProperties, pOutputFile);
 break;

 case RANDOM_TEST:
 performRandomTest(&testProperties, pOutputFile);
 break;

 case LATENCY_TEST:
 performLatencyTest(&testProperties, pOutputFile);
 break;

 case TIME_TEST:
 performTimedTest(&testProperties, pOutputFile);
 break;

 default:
 puts("Unexpected test type encountered, exiting");
 break;
}

ENABLE_SYSTEM_SLEEP();

/* if writing the results to file */
if (pOutputFile != NULL)
{
 /* Close the file */
 fclose(pOutputFile);
}

/* Clean-up the test by freeing any memory and closing any open channels */
cleanupTest(&testProperties);

return 0;
}

```

## 10.48 time-code\_example.c

Demonstrates sending a time-code using the advanced transmit operations.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void timecodeExample()
{

```

```

/* Get first device */
STAR_DEVICE_ID firstDevice = getFirstDevice();

/* If there is a first device */
if(firstDevice)
{
 /* Open send channel */
 STAR_CHANNEL_ID sendChannel =
 STAR_openChannelToLocalDevice(
 firstDevice, STAR_CHANNEL_DIRECTION_OUT, CHANNEL0, 1);

 /* If send channel was opened */
 if(sendChannel)
 {
 /* Define transfer status for transfer operation status updates */
 STAR_TRANSFER_STATUS status;

 /* Define transfer operation for sending packets */
 STAR_TRANSFER_OPERATION * sendTransferOperation;

 /* Initialise time-code value to 1 */
 U8 timecodeValue = 1;

 /* Create time-code */
 STAR_STREAM_ITEM * timecode = STAR_createTimeCode(
 timecodeValue);

 /* If time-code was created */
 if(timecode)
 {
 /* Create transmit operation to send time-code */
 sendTransferOperation = STAR_createTxOperation(&timecode, 1);

 /* Start transmitting the time-code */
 STAR_submitTransferOperation(sendChannel,
 sendTransferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation, -1);

 /* If time-code was sent successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success message */
 printf("Time-code sent successfully.\n");
 }

 /* Dispose of transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);

 /* Destroy time-code */
 STAR_destroyStreamItem(timecode);
 }

 /* Close send channel */
 STAR_closeChannel(sendChannel);
 }
}
}

```

## 10.49 time-code\_test.c

The STAR-System Time-code Test application is a command line program that can be used to configure the time-code properties of a device, and to transmit and receive time-codes. The source code also provides useful examples for performing these tasks.

```

#include <stdio.h>
#include <stdlib.h>

#include "star-api.h"
#include "cfg_api_generic.h"

#if defined(_WIN32)

 #include <process.h>
 #include <windows.h>

 #define STAR_THREAD HANDLE
 #define STAR_THREAD_RETURN_TYPE unsigned int WINAPI

```

```

#define STAR_THREAD_RETURN_VAL 0

#define STAR_CreateThread(thread, func, arg) \
 ((thread = (HANDLE)_beginthreadex(NULL, 0, func, arg, 0, NULL)) != 0)

#define STAR_CloseThread(thread) CloseHandle(thread)

#define STAR_WaitForThreadCompletion(thread) \
 (WaitForSingleObject(thread, INFINITE) == WAIT_OBJECT_0)

#else

#include <pthread.h>
#include <unistd.h>

#define STAR_THREAD pthread_t
#define STAR_THREAD_RETURN_TYPE void *
#define STAR_THREAD_RETURN_VAL NULL

#define STAR_CreateThread(thread, func, arg) \
 (!pthread_create(&(thread), NULL, func, arg))

#define STAR_CloseThread(thread) \
 ((thread) ? (!pthread_detach(thread)) : 1)

#define STAR_WaitForThreadCompletion(thread) \
 (!!pthread_join(thread, NULL) && (!(thread) = 0)))

#endif

#if !defined(UNREFERENCED_PARAMETER)
#define UNREFERENCED_PARAMETER(a) ((void)(a))
#endif

U8 g_currentTimecode = 0;

STAR_THREAD g_receiveThread = 0;

int g_receiveThreadRunning = 0;

STAR_CHANNEL_ID g_receiveChannel = 0;

static void transmitTimecode(STAR_DEVICE_ID deviceId)
{
 /* Open channel 0 to transmit */
 /* Note that STAR-System devices currently only support transmitting and */
 /* receiving time-codes on channel 0 */
 STAR_CHANNEL_ID txChannel = STAR_openChannelToLocalDevice(
 deviceId,
 STAR_CHANNEL_DIRECTION_OUT, 0, 0);

 /* If transmit channel was opened */
 if (txChannel)
 {
 STAR_TRANSFER_STATUS status;
 STAR_TRANSFER_OPERATION * transferOperation;

 /* Create a time-code */
 STAR_STREAM_ITEM *timecode = STAR_createTimeCode(
 g_currentTimecode);

 /* If time-code was created */
 if (timecode)
 {
 /* Create transmit operation to transmit time-code */
 transferOperation = STAR_createTxOperation(&timecode, 1);

 /* Start transmitting the time-code */
 STAR_submitTransferOperation(txChannel, transferOperation);

 /* Wait indefinitely for transfer to complete */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation,
 -1);

 /* If time-code was transmitted successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Print success message */
 printf("Time-code %u transmitted successfully.\n",
 g_currentTimecode);
 }
 }
 }
}

```



```

 else
 {
 /* Print error message */
 printf("Error transmitting time-code %u, status = %d.\n",
 g_currentTimecode, status);
 }

 /* Dispose of transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy time-code */
 STAR_destroyStreamItem(timecode);

 /* Move to the next time-code value */
 g_currentTimecode++;
 if (g_currentTimecode == 64)
 {
 g_currentTimecode = 0;
 }
 }

 /* Close transmit channel */
 STAR_closeChannel(txChannel);
}

static void setTimecodePeriod(STAR_DEVICE_ID deviceId)
{
 char s[256];

 /* Display the current time-code period */
 U32 period;
 if (CFG_SUCCESS(CFG_getTimeCodePeriod(deviceId, &period)))
 {
 printf("Current time-code period: %u microseconds\n", period);
 }
 else
 {
 puts("Unable to read the current time-code period");
 }

 /* Read in the time-code period to be set */
 printf("Please enter the time-code period in microseconds: ");
 fflush(stdout);
 if (!fgets(s, 256, stdin))
 {
 puts("No period specified.");
 }
 else
 {
 {
 int status = sscanf(s, "%u", &period);
 if (!status)
 {
 puts("Invalid time-code period specified.");
 }
 else
 {
 {
 /* Set the time-code period */
 if (CFG_SUCCESS(CFG_setTimeCodePeriod(deviceId, period)))
 {
 printf("Time-code period set to %u\n", period);
 }
 else
 {
 puts("Error setting time-code period");
 }
 }
 }
 }
 }
}

static void enableTimecodeMaster(STAR_DEVICE_ID deviceId)
{
 /* Enable the device as a time-code master */
 if (CFG_SUCCESS(CFG_enableTimeCodeMaster(deviceId)))
 {
 puts("Device enabled as a time-code master");
 }
 else
 {
 puts("Error enabling device as a time-code master");
 }
}

static void disableTimecodeMaster(STAR_DEVICE_ID deviceId)

```

```

{
 /* Disable the device as a time-code master */
 if (CFG_SUCCESS(CFG_disableTimeCodeMaster(deviceId)))
 {
 puts("Device disabled as a time-code master");
 }
 else
 {
 puts("Error disabling device as a time-code master");
 }
}

STAR_THREAD_RETURN_TYPE timecodeReceiveThread(void *pArg)
{
 STAR_TRANSFER_OPERATION *pRxTransferOp;

 /* Get the device ID from the argument passed to the thread */
 STAR_DEVICE_ID deviceId = *(STAR_DEVICE_ID *)pArg;
 free(pArg);

 /* Open channel 0 to receive */
 /* Note that STAR-System devices currently only support transmitting and */
 /* receiving time-codes on channel 0 */
 g_receiveChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_IN, 0, 1);
 if (!g_receiveChannel)
 {
 puts("Unable to open channel to receive time-codes");
 goto timecodeReceiveThread_openChannelFail;
 }

 /* Create a receive operation to receive 1 time-code */
 pRxTransferOp = STAR_createRxOperation(1,
 STAR_RECEIVE_TIMECODES);
 if (!pRxTransferOp)
 {
 puts("Unable to create receive operation to receive a time-code");
 goto timecodeReceiveThread_createRxOperationFail;
 }

 /* While the thread has not been stopped */
 while (g_receiveThreadRunning)
 {
 STAR_TRANSFER_STATUS rxStatus;
 STAR_STREAM_ITEM *pRxStreamItem;

 /* Submit the receive operation */
 if (!STAR_submitTransferOperation(g_receiveChannel, pRxTransferOp))
 {
 if (g_receiveThreadRunning)
 {
 puts("Unable to submit receive operation to receive a time-code");
 }
 break;
 }

 /* Wait indefinitely on the receive operation completing */
 rxStatus = STAR_waitOnTransferOperationCompletion(
 pRxTransferOp, -1);
 if (rxStatus != STAR_TRANSFER_STATUS_COMPLETE)
 {
 if (g_receiveThreadRunning)
 {
 printf("Error waiting for receive operation to receive a time-code, status = %d\n",
 rxStatus);
 }
 break;
 }

 /* Get the received time-code */
 pRxStreamItem = STAR_getTransferItem(pRxTransferOp, 0);
 if ((!pRxStreamItem) || (pRxStreamItem->itemType !=
 STAR_STREAM_ITEM_TYPE_TIMECODE) || (!pRxStreamItem->
 item))
 {
 if (g_receiveThreadRunning)
 {
 puts("Error getting receive stream item");
 }
 break;
 }

 /* Display the received time-code */
 printf("\t\tReceived time-code with value: %2u\n",
 STAR_getTimeCodeValue((STAR_TIMECODE *)pRxStreamItem->
 item));
 }
}

```

```

 }

 /* Dispose the receive operation */
 STAR_disposeTransferOperation(pRxTransferOp);

timecodeReceiveThread_createRxOperationFail:
 /* Close the receive channel */
 STAR_closeChannel(g_receiveChannel);

timecodeReceiveThread_openChannelFail:
 g_receiveThreadRunning = 0;

 return STAR_THREAD_RETURN_VAL;
}

static void startReceivingTimecodes(STAR_DEVICE_ID deviceId)
{
 /* Check the thread isn't already running */
 if (g_receiveThread)
 {
 puts("The receive thread is already running");
 }
 else
 {
 /* Create the argument to be passed to the thread */
 STAR_DEVICE_ID *pDeviceId;
 pDeviceId = (STAR_DEVICE_ID *)calloc(1, sizeof(
STAR_DEVICE_ID));
 if (!pDeviceId)
 {
 puts("Unable to create argument for thread to receive time-codes");
 }
 else
 {
 *pDeviceId = deviceId;

 /* Start a thread to receive time-codes */
 g_receiveThreadRunning = 1;
 if (!STAR_CreateThread(g_receiveThread, timecodeReceiveThread,
pDeviceId))
 {
 puts("Unable to create thread to receive time-codes");
 g_receiveThreadRunning = 0;
 g_receiveThread = 0;
 }
 }
 }
}

static void stopReceivingTimecodes()
{
 if (!g_receiveThread)
 {
 puts("The receive thread is not currently running");
 }
 else
 {
 /* Close the receive channel to stop the thread receiving time-codes */
 g_receiveThreadRunning = 0;
 STAR_closeChannel(g_receiveChannel);

 /* Wait on the thread completing */
 if (!STAR_WaitForThreadCompletion(g_receiveThread))
 {
 puts("Error waiting for the receive thread to complete");
 }

 /* Close the thread */
 (void)STAR_CloseThread(g_receiveThread);
 g_receiveThread = 0;
 }
}

static void enableExternalTimecodeSelection(STAR_DEVICE_ID deviceId)
{
 /* Enable external time-codes selection for the device */
 if (CFG_SUCCESS(CFG_enableExternalTimeCodeSelection(
deviceId)))
 {
 puts("External time-code selection enabled for the device");
 }
 else
 {
 puts("Error enabling external time-code selection for the device");
 }
}

```

```

 }
}

static void disableExternalTimecodeSelection(STAR_DEVICE_ID deviceId)
{
 /* Disable external time-codes selection for the device */
 if (CFG_SUCCESS(CFG_disableExternalTimeCodeSelection(
 deviceId)))
 {
 puts("External time-code selection disabled for the device");
 }
 else
 {
 puts("Error disabling external time-code selection for the device");
 }
}

static STAR_DEVICE_ID displayDeviceList(void)
{
 U32 deviceCount, deviceNum;
 char *deviceName, s[256];
 unsigned int chosenDevice;
 int status;

 /* Get the list of configurable devices */
 STAR_DEVICE_ID *devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED, &deviceCount);
 STAR_DEVICE_ID deviceId = 0;

 /* If there are devices present */
 if (devices)
 {
 if (!deviceCount)
 {
 puts("No devices present.\n");
 }
 else if (deviceCount == 1)
 {
 deviceName = STAR_getDeviceName(devices[0]);
 if (deviceName)
 {
 printf("One device present: %s\n", deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 puts("One device present: Unable to determine device name");
 }
 deviceId = devices[0];
 }
 else
 {
 puts("Select device:");

 /* For each device in the list */
 for (deviceNum = 0; deviceNum < deviceCount; deviceNum++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[deviceNum]);
 if (deviceName)
 {
 printf("%u: %s\n", deviceNum, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("%u: Unable to determine device name\n", deviceNum);
 }
 }

 /* Get the device to use */
 if (!fgets(s, 256, stdin))
 {
 puts("No device number selected.");
 }
 else
 {
 status = sscanf(s, "%u", &chosenDevice);
 if ((!status) || (chosenDevice > deviceCount - 1))
 {
 puts("Incorrect device number selected.");
 }
 else
 {
 deviceId = devices[chosenDevice];
 }
 }
 }
 }
}

```

```

 }
 }
}

/* Destroy the device list */
STAR_destroyDeviceList(devices);
}
else
{
 puts("No devices present.\n");
}

return deviceId;
}

int __cdecl main(int argc, char *argv[])
{
 char exitSelected = 0;
 char s[256];
 STAR_DEVICE_ID deviceId;
 U32 timeCodePorts = 0;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System Time-code Test Application");

 /* Get the device to be used */
 deviceId = displayDeviceList();
 if (!deviceId)
 {
 return 0;
 }

 /* Determine which ports of the device are time-code capable */
 if (!CFG_SUCCESS(CFG_getTimeCodeDistributionCapablePorts
 (deviceId, &timeCodePorts)))
 {
 puts("Unable to determine time-code distribution capable ports");
 }

 /* Enable time-code forwarding on all ports of the device */
 if (!CFG_SUCCESS(CFG_setTimeCodeDistributionPorts(deviceId,
 timeCodePorts)))
 {
 puts("Unable to enable time-code distribution on all ports");
 }

 do
 {
 /* Display Menu */
 puts("\n");
 puts("Select option:");
 puts(" 0: Exit");
 puts(" 1: Transmit a time-code");
 puts(" 2: Set period of time-code master (cannot be done while receiving)");
 puts(" 3: Enable time-code master (cannot be done while receiving)");
 puts(" 4: Disable time-code master (cannot be done while receiving)");
 puts(" 5: Start receiving time-codes");
 puts(" 6: Stop receiving time-codes");
 puts(" 7: Enable external time-code selection (cannot be done while receiving)");
 puts(" 8: Disable external time-code selection (cannot be done while receiving)");
 puts("");

 /* Read in the operation type */
 if (!fgets(s, 256, stdin))
 {
 puts("Error reading input, exiting");
 exitSelected = 1;
 }
 else
 {
 puts("");
 switch (s[0])
 {
 case '0':
 exitSelected = 1;
 break;

 case '1':
 transmitTimecode(deviceId);
 break;

 case '2':
 setTimecodePeriod(deviceId);
 break;
 }
 }
 }
 while (exitSelected == 0);
}

```

```

 case '3':
 enableTimecodeMaster(deviceId);
 break;

 case '4':
 disableTimecodeMaster(deviceId);
 break;

 case '5':
 startReceivingTimecodes(deviceId);
 break;

 case '6':
 stopReceivingTimecodes();
 break;

 case '7':
 enableExternalTimecodeSelection(deviceId);
 break;

 case '8':
 disableExternalTimecodeSelection(deviceId);
 break;

 default:
 printf("Invalid menu option selected: %c\n", s[0]);
 }
}
} while (!exitSelected);

puts("Exiting");

return 0;
}

```

## 10.50 timestamp\_test.c

This file contains example code demonstrating how to use the timestamping API for the Brick Mk3.

```

#include "star-api.h"
#include "cfg_api_generic.h"
#include "triggering_brick_mk3.h"
#include "triggering_pcie_mk2.h"
#include "triggering_pxi_if.h"

#include <stdio.h>

#ifdef _WIN32
 #include "windows.h"

 #define SLEEP(milliseconds) Sleep(milliseconds);
#else
 #include <unistd.h>

 #define SLEEP(milliseconds) usleep(milliseconds * 1000);
#endif

#if !defined(UNREFERENCED_PARAMETER)
 #define UNREFERENCED_PARAMETER(a) ((void) (a))
#endif

/* Comment this out to use external trigger in example. */
#define GLOBAL_PULSE_GENERATOR

/* Uncomment this to trigger on falling edge of external trigger */
// #define EXTERNAL_TRIGGER_FALLING_EDGE

#ifdef GLOBAL_PULSE_GENERATOR

/* Set sync pulse frequency to 1Hz (global pulse generator supports 1Hz,
 10Hz, 100Hz or 1000Hz). */
#define SYNC_PULSE_FREQUENCY 1 /* 1Hz */

#else

/* Set sync pulse frequency to match external trigger pulse generation
 frequency. */
#define SYNC_PULSE_FREQUENCY 1 /* 1Hz */

```

```

#endif

/* Define transmit and receive links */
#define TRANSMIT_LINK 1
#define RECEIVE_LINK 2

STAR_DEVICE_ID getDeviceId()
{
 /* The selected device to use for the example */
 STAR_DEVICE_ID selectedDevice = 0;

 /* List of device types that support timestamping */
 U32 deviceTypes[] = { STAR_DEVICE_BRICK_MK3,
 STAR_DEVICE_BRICK_MK4,
 STAR_DEVICE_PXI_INTERFACE,
 STAR_DEVICE_PXI_INTERFACE_MK2,
 STAR_DEVICE_PXI_RMAP,
 STAR_DEVICE_PXI_RMAP_MK2,
 STAR_DEVICE_PCIE_MK2 };

 /* Get all connected devices that support timestamping */
 U32 deviceCount = 0;
 STAR_DEVICE_ID *pDeviceList
 = STAR_getDeviceListForTypes(
 deviceTypes,
 sizeof(deviceTypes) / sizeof(deviceTypes[0]),
 &deviceCount);

 /* If at least one device */
 if(deviceCount > 0)
 {
 U32 deviceIndex = 0;

 /* For all devices */
 for(deviceIndex = 0; deviceIndex < deviceCount; deviceIndex++)
 {
 /* Get the current device */
 STAR_DEVICE_ID currentDevice = pDeviceList[deviceIndex];

 /* Get current device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(
 currentDevice);

 /* If Brick Mk3 */
 if (deviceType == STAR_DEVICE_BRICK_MK3)
 {
 /* Get hardware info for current device */
 STAR_CFG_FPGA_INFO hardwareInfo;
 CFG_getFPGAInfo(currentDevice, &hardwareInfo);

 /* If Brick Mk3 is v1.02 or later */
 if (hardwareInfo.major > 1 ||
 (hardwareInfo.major == 1 && hardwareInfo.minor >= 2))
 {
 /* Use current device for test */
 selectedDevice = currentDevice;
 break;
 }
 }
 /* Else if SpW PXI Interface (with or without RMAP target) */
 else if (deviceType == STAR_DEVICE_PXI_INTERFACE
 || deviceType == STAR_DEVICE_PXI_RMAP)
 {
 /* Get hardware info for current device */
 STAR_CFG_FPGA_INFO hardwareInfo;
 CFG_getFPGAInfo(currentDevice, &hardwareInfo);

 /* If PXI Interface is v1.04 or later */
 if (hardwareInfo.major > 1 ||
 (hardwareInfo.major == 1 && hardwareInfo.minor >= 4))
 {
 /* Use current device for test */
 selectedDevice = currentDevice;
 break;
 }
 }
 else
 {
 /* Use current device for test */
 selectedDevice = currentDevice;
 break;
 }
 }
 }

 /* Destroy the device list */
 STAR_destroyDeviceList(pDeviceList);
}

```

```

 /* Return the selected device */
 return selectedDevice;
}

void printPacket(unsigned char *pPacketData, unsigned int packetLength)
{
 /* Print packet contents */
 unsigned int x;
 for(x = 0; x < packetLength; x++)
 {
 printf("%02X ", pPacketData[x]);
 }
}

void transmitPackets(STAR_CHANNEL_ID channel, unsigned int transmitPacketCount)
{
 U8 buffer[32];
 unsigned int x;
 STAR_STREAM_ITEM *pPacket;
 STAR_TRANSFER_OPERATION *pTransmitOperation;

 /* For all packets to be transmitted */
 for(x = 1; x <= transmitPacketCount; x++)
 {
 /* Populate packet buffer */
 memset(buffer, 0xAB, 32);
 memset(buffer, x, 1);

 /* Create packet from buffer */
 pPacket = STAR_createPacket(NULL, buffer, 32,
 STAR_EOP_TYPE_EOP);

 /* Create transmit operation from packet */
 pTransmitOperation = STAR_createTxOperation(&pPacket, 1);

 /* Transmit the packet */
 STAR_submitTransferOperation(channel, pTransmitOperation);

 /* Wait for transmit operation to complete */
 STAR_waitOnTransferOperationCompletion(pTransmitOperation, -1
);

 /* Cleanup resources required for transmit operation */
 STAR_disposeTransferOperation(pTransmitOperation);
 STAR_destroyStreamItem(pPacket);

 /* Print packet transmit status */
 printf("Packet %d transmitted : ", (int)x);
 printPacket(buffer, 32);
 printf("\n");

 /* Sleep for 100 milliseconds */
 SLEEP(100);
 }
}

double convertToDouble(U32 seconds, U32 nanoseconds)
{
 /* Convert value to double */
 double doubleValue = (double)seconds + ((double)nanoseconds / 1E9);

 /* Return value as double */
 return doubleValue;
}

void receivePacketsAndTimestamps(STAR_CHANNEL_ID channel,
 unsigned int receiveCount, U32 syncPulseFrequency)
{
 STAR_TRANSFER_OPERATION *pReceiveOperation;
 unsigned int x;
 unsigned int actualReceiveCount;

 /* Create receive operation for packets and timestamp events */
 pReceiveOperation = STAR_createRxOperation(receiveCount,
 STAR_RECEIVE_PACKETS | STAR_RECEIVE_TIMESTAMP_EVENTS
);

 /* Submit the receive operation */
 STAR_submitTransferOperation(channel, pReceiveOperation);

 /* Wait for receive operation to complete */
 STAR_waitOnTransferOperationCompletion(pReceiveOperation, -1);

 /* Get the number of stream items that were received */
 actualReceiveCount = STAR_getTransferItemCount(pReceiveOperation);
}

```



```

/* For all stream items that were received */
for(x = 0; x < actualReceiveCount; x++)
{
 /* Get current stream item */
 STAR_STREAM_ITEM *pReceivedItem =
 STAR_getTransferItem(pReceiveOperation, x);

 /* If current stream item is a packet */
 if(pReceivedItem->itemType ==
 STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET)
 {
 unsigned char *pPacketData;
 unsigned int packetLength;

 /* Get the packet data */
 STAR_SPACEWIRE_PACKET *pPacket =
 (STAR_SPACEWIRE_PACKET *)pReceivedItem->
item;
 pPacketData = STAR_getPacketData(pPacket, &packetLength);

 /* Print the packet data */
 printf("Packet %d received : ", pPacketData[0]);
 printPacket(pPacketData, packetLength);
 printf("\n");

 /* Destroy the packet data now that it has been used */
 STAR_destroyPacketData(pPacketData);
 }
 /* Else if current stream item is a timestamp event */
 else if(pReceivedItem->itemType ==
 STAR_STREAM_ITEM_TYPE_TIMESTAMP_EVENT)
 {
 U32 startSeconds;
 U32 startRemainder;
 U32 endSeconds;
 U32 endRemainder;

 /* Get the timestamp data */
 STAR_TIMESTAMP_EVENT *pTimestampEvent =
 (STAR_TIMESTAMP_EVENT *)pReceivedItem->item;

 /* Get start and end timestamp values in seconds and a remainder of
nanoseconds */
 STAR_getTimestampEventStartValue(pTimestampEvent,
 syncPulseFrequency, &startSeconds, &startRemainder);
 STAR_getTimestampEventEndValue(pTimestampEvent,
 syncPulseFrequency, &endSeconds, &endRemainder);

 /* Print the start and end of packet timestamps */
 printf("Start of packet timestamp : %f seconds.\n",
 convertToDouble(startSeconds, startRemainder));
 printf("End of packet timestamp : %f seconds.\n\n",
 convertToDouble(endSeconds, endRemainder));
 }
}

/* Cleanup receive operation */
STAR_disposeTransferOperation(pReceiveOperation);
}

int STAR_API_CC TRIGGERSetCounterReloadValue(STAR_DEVICE_ID deviceId,
 U32 counter,
 U32 reloadValue)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Set counter reload value for device type */
 switch (deviceType)
 {
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId,
 counter,
 reloadValue);

 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_setCounterReloadValue(deviceId,
 counter,
 reloadValue);

 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId,
 counter,
 reloadValue);

 default:

```

```

 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGEREnableCounterAutoReload(STAR_DEVICE_ID deviceId,
 U32 counter)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable counter auto reload for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_enableCounterAutoReload(deviceId,
 counter);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_enableCounterAutoReload(deviceId,
 counter);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_enableCounterAutoReload(deviceId,
 counter);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERSetCounterInputEvents(STAR_DEVICE_ID deviceId,
 U32 counter,
 U32 trigger,
 COUNTER_EVENT_MASK events)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable counter auto reload for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId,
 counter,
 trigger, events);

 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_setCounterInputEvents(deviceId, counter,
 trigger, events);

 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId,
 counter,
 trigger, events);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGEREnableExtTriggerOutput(STAR_DEVICE_ID deviceId,
 U32 extTrigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable external trigger output for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_enableExtTriggerOutput(deviceId,
 extTrigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_enableExtTriggerOutput(deviceId,
 extTrigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_enableExtTriggerOutput(deviceId,
 extTrigger);
 }
}

```

```

 default:
 /* Return error, unsupported device type */
 return 0;
 }
 }
}

int STAR_API_CC TRIGGERDisableExtTriggerOutput(STAR_DEVICE_ID deviceId,
 U32 extTrigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Disable external trigger output for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_disableExtTriggerOutput(deviceId,
 extTrigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_disableExtTriggerOutput(deviceId,
 extTrigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_disableExtTriggerOutput(deviceId,
 extTrigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERSetExtTriggerExtend(STAR_DEVICE_ID deviceId,
 U32 extTrigger,
 U32 extend)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Set external trigger extend duration for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_setExtTriggerExtend(deviceId,
 extTrigger,
 extend);

 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_setExtTriggerExtend(deviceId,
 extTrigger,
 extend);

 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_setExtTriggerExtend(deviceId,
 extTrigger,
 extend);

 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERSetExtTriggerOutputActions(
 STAR_DEVICE_ID deviceId,
 U32 extTrigger,
 U32 trigger,
 EXT_TRIGGER_ACTION_MASK actions)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Set external trigger output actions for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_setExtTriggerOutputActions(
 deviceId,
 extTrigger,
 trigger,
 actions);

 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 }
}

```

```

 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_setExtTriggerOutputActions(deviceId,
 extTrigger,
 trigger,
 actions);

 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_setExtTriggerOutputActions(
 deviceId,
 extTrigger,
 trigger,
 actions);

 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGEREnableTrigger(STAR_DEVICE_ID deviceId,
 U32 trigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable internal trigger for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_enableTrigger(deviceId, trigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_enableTrigger(deviceId, trigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_enableTrigger(deviceId, trigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERDisableTrigger(STAR_DEVICE_ID deviceId,
 U32 trigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Disable internal trigger for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_disableTrigger(deviceId, trigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_disableTrigger(deviceId, trigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_disableTrigger(deviceId, trigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int TRIGGEREnableExtTriggerEdgeDetectMode(STAR_DEVICE_ID deviceId,
 U32 extTrigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable external trigger edge detect mode for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId,
 extTrigger);

 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(

```

```

 deviceId,
 extTrigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode
 (deviceId,
 extTrigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGEREnableExtTriggerInvert(STAR_DEVICE_ID deviceId,
 U32 extTrigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Enable external trigger invert for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_enableExtTriggerInvert(deviceId,
 extTrigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_enableExtTriggerInvert(deviceId,
 extTrigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_enableExtTriggerInvert(deviceId,
 extTrigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERDisableExtTriggerInvert(STAR_DEVICE_ID deviceId,
 U32 extTrigger)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Disable external trigger invert for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_disableExtTriggerInvert(deviceId,
 extTrigger);
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_disableExtTriggerInvert(deviceId,
 extTrigger);
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_disableExtTriggerInvert(deviceId,
 extTrigger);
 default:
 /* Return error, unsupported device type */
 return 0;
 }
}

int STAR_API_CC TRIGGERGetDeviceClockRate(STAR_DEVICE_ID deviceId)
{
 /* Get device type */
 STAR_DEVICE_TYPE deviceType = STAR_getDeviceType(deviceId);

 /* Set counter reload value for device type */
 switch (deviceType)
 {
 case STAR_DEVICE_BRICK_MK3:
 case STAR_DEVICE_BRICK_MK4:
 return TRIGGER_BRICK_MK3_getDeviceClockRate();
 case STAR_DEVICE_PXI_INTERFACE:
 case STAR_DEVICE_PXI_RMAP:
 case STAR_DEVICE_PXI_INTERFACE_MK2:
 case STAR_DEVICE_PXI_RMAP_MK2:
 return TRIGGER_PXI_IF_getDeviceClockRate();
 case STAR_DEVICE_PCIE_MK2:
 return TRIGGER_PCIE_MK2_getDeviceClockRate();
 default:

```

```

 /* Return error, unsupported device type */
 return 0;
 }
}

void enableExternalSyncPulse(STAR_DEVICE_ID deviceId)
{
 /* The following code uses the STAR-System triggering API to generate a
 pulse every second. An SMB lead should be connected between triggers
 A and B. */

 int clockCyclesPerSecond = 0;
 clockCyclesPerSecond = TRIGGERGetDeviceClockRate(deviceId);

 /* Set counter 0 reload value to 1 second */
 TRIGGERSetCounterReloadValue(deviceId, 0, clockCyclesPerSecond);

 /* Enable counter auto reload */
 TRIGGEREnableCounterAutoReload(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on counter 0 causes internal trigger 0 to
 be set */
 TRIGGERSetCounterInputEvents(deviceId, 0, 0, COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
 engine 0 */
 TRIGGEREnableExtTriggerOutput(deviceId, 1);
 TRIGGERSetExtTriggerExtend(deviceId, 1, 50);
 TRIGGERSetExtTriggerOutputActions(deviceId, 1, 0, EXT_TRIGGER_ACTION_OUT);

 /* Enable internal trigger 0 */
 TRIGGEREnableTrigger(deviceId, 0);
}

void disableExternalSyncPulse(STAR_DEVICE_ID deviceId)
{
 /* Disable the constant pulse on the external trigger */
 TRIGGERDisableExtTriggerOutput(deviceId, 1);
 TRIGGERDisableTrigger(deviceId, 0);
}

void timestampTest(STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_ID transmitChannel;
 STAR_CHANNEL_ID receiveChannel;

 /* Transmit 10 packets */
 unsigned int transmitPacketCount = 10;

 printf("Setting up timestamp system...\n");

 /* Ensure that timestamp counters are disabled before configuring */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE);

 /* Set initial timestamp value of 10 seconds */
 CFG_setTimestampValue(deviceId, 10);

 /* Enable timestamp events on port 2 */
 CFG_enableRxTimestampEventsOnPort(deviceId, RECEIVE_LINK);

#ifdef GLOBAL_PULSE_GENERATOR
 #if SYNC_PULSE_FREQUENCY == 1

 /* Set pulse generator frequency to 1Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_1);

 #elif SYNC_PULSE_FREQUENCY == 10

 /* Set pulse generator frequency to 10Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_10);

 #elif SYNC_PULSE_FREQUENCY == 100

 /* Set pulse generator frequency to 100Hz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_100);

 #elif SYNC_PULSE_FREQUENCY == 1000

 /* Set pulse generator frequency to 1KHz */
 CFG_setPulseGeneratorFrequency(deviceId,
 STAR_CFG_BRICK_MK3_PULSE_FREQ_1000);
 #endif
#endif
}

```

```

#else

 /* Print error when unexpected global pulse frequency is detected */
 printf("ERROR: Unexpected global pulse frequency of %dHz (valid values are: 1Hz, 10Hz, 100Hz, 1000Hz).\n");
 return;

#endif

 /* Enable global pulse generator */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_PULSE_GENERATOR
);

#else

 /* Enable external trigger pulse detection */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_EXTERNAL_TRIGGER
);

 /* Enable the external sync pulse before transmitting packets */
 enableExternalSyncPulse(deviceId);

#endif

 /* Create transmit and receive channels */
 transmitChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_OUT, TRANSMIT_LINK, 1);
 receiveChannel = STAR_openChannelToLocalDevice(deviceId,
 STAR_CHANNEL_DIRECTION_IN, RECEIVE_LINK, 1);

 /* Sleep to allow timer to run before transmitting packets */
 SLEEP(2000);

 /* Transmit a group of packets */
 transmitPackets(transmitChannel, transmitPacketCount);

 /* Receive the packets and timestamps */
 receivePacketsAndTimestamps(receiveChannel, transmitPacketCount * 2,
 SYNC_PULSE_FREQUENCY);

#ifndef GLOBAL_PULSE_GENERATOR

 /* Disable the external sync pulse after receiving packets and timestamps */
 disableExternalSyncPulse(deviceId);

#endif

 /* Disable timestamps after receiving packets and timestamps */
 CFG_setTimestampMethod(deviceId,
 STAR_CFG_BRICK_MK3_TIMESTAMP_METHOD_NONE);

 /* Close the channels */
 STAR_closeChannel(transmitChannel);
 STAR_closeChannel(receiveChannel);
}

int main(int argc, char **argv)
{
 /* Select device to use in examples */
 STAR_DEVICE_ID deviceId = getDeviceId();

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 STAR_setApplicationName("STAR-System Timestamp Test Application");

 /* Print the program header */
 printf("----- Timestamping API Example -----\\n\\n");

 /* If compatible device was found */
 if(deviceId)
 {
 STAR_CFG_ROUTER_GLOBAL_STATE globalState;
 PORT_STATUS_CONTROL portStatusControl;
 STAR_CFG_SPW_LINK_STATUS linkStatus;

 /* Get device name and serial */
 char * pDeviceName = STAR_getDeviceName(deviceId);
 char * pDeviceSerial = STAR_getDeviceSerialNumber(deviceId);

 /* Print selected serial */
 printf("%s with serial %s selected.\\n\\n", pDeviceName, pDeviceSerial);

 /* Disable router timeouts so that timestamps are not blocked */
 CFG_getRouterGlobalSettings(deviceId, &globalState);
 }
}

```

```

globalState.timeoutMode = STAR_CFG_TIMEOUT_MODE_BLOCKING;
CFG_setRouterGlobalSettings(deviceId, &globalState);

/* Timestamping requires interface mode to be enabled */
CFG_enableInterfaceMode(deviceId);

/* Start the link since watchdog timer is disabled */
CFG_getPortStatusControl(deviceId, 1, &portStatusControl);
CFG_getSpaceWireLinkStatus(portStatusControl, &linkStatus);
linkStatus.start = 1;
CFG_setSpaceWireLinkStatus(deviceId, 1, &linkStatus);

/* Enable external trigger edge detection */
TRIGGEREnableExtTriggerEdgeDetectMode(deviceId, 1);

#ifdef EXTERNAL_TRIGGER_FALLING_EDGE
/* Trigger on falling edge */
TRIGGEREnableExtTriggerInvert(deviceId, 1);
#else
/* Trigger on rising edge */
TRIGGERDisableExtTriggerInvert(deviceId, 1);
#endif

/* Perform timestamp test */
timestampTest(deviceId);
}
else
{
 printf("No compatible devices were found.\n");
}

return 0;
}

```

## 10.51 transfer\_callback\_example.c

Creates a simple packet and sends it using the advanced transmit operations. The packet is received using the transfer completion listener function.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

/* Define receive channel */
STAR_CHANNEL_ID receiveChannel;

/* Initialise transfer complete to false */
unsigned int transferComplete = 0;

void STAR_API_CC TransferCallback(STAR_TRANSFER_OPERATION *pOperation,
STAR_TRANSFER_STATUS status, void *pContextInfo)
{
 /* Unreferenced parameters */
 UNREFERENCED_PARAMETER(pContextInfo);

 /* Unregister transfer completion listener */
 STAR_unregisterTransferCompletionListener(pOperation,
 TransferCallback);

 /* If valid stream item was received */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received stream item */
 STAR_STREAM_ITEM * streamItem = STAR_getTransferItem(pOperation,
 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData((
 STAR_SPACEWIRE_PACKET *)
 streamItem->item, &dataLength);

 if (data != NULL)
 {

```



```

 /* Prints the packet contents from the receive buffer */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been output */
 STAR_destroyPacketData(data);
 }
}

/* Dispose transfer operation */
STAR_disposeTransferOperation(pOperation);

/* Close receive channel */
STAR_closeChannel(receiveChannel);

/* Set transfer complete to true */
transferComplete = 1;
}

void transferCallbackExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open receive channel */
 receiveChannel = STAR_openChannelToLocalDevice(firstDevice,
 STAR_CHANNEL_DIRECTION_IN, CHANNEL1, 1);

 /* If receive channel was opened */
 if(receiveChannel)
 {
 /* Create transfer operation to receive 1 packet */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);

 /* If transfer operation was created */
 if(receiveTransferOperation)
 {
 /* Register transfer completion listener */
 STAR_registerTransferCompletionListener(
 receiveTransferOperation, TransferCallback, NULL);

 /* Start receiving packet */
 STAR_submitTransferOperation(receiveChannel,
 receiveTransferOperation);

 /* Transfer completion listener is unregistered in callback
 function */
 }

 /* Receive transfer operation is disposed in callback function */

 /* Receive channel is closed in callback function */
 }
 }

 /* While transfer isn't complete, keep application alive */
 while(!transferComplete)
 {
 /* Sleep to relieve the processor */
 sleep(0);
 }
}

```

## 10.52 transfer\_operation\_list\_send\_example.c

This example dynamically creates multiple packets and sends them using the `STAR_submitTransferOperationList()` function.

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

```

```

#include <stdlib.h>

void transferOperationListSendExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open channel to send out of */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice
(firstDevice,
 STAR_CHANNEL_DIRECTION_OUT, CHANNEL1, 1);

 /* If channel was opened */
 if(channel)
 {
 /* Initialise number of packets to 10 */
 unsigned int numberOfPackets = 10;

 /* Create array to hold transfer operations */
 STAR_TRANSFER_OPERATION **transferOperations =
(STAR_TRANSFER_OPERATION **)malloc(numberOfPackets *
sizeof(STAR_TRANSFER_OPERATION *));

 /* Create array to hold packets */
 STAR_STREAM_ITEM **packets = (STAR_STREAM_ITEM **)malloc(
numberOfPackets * sizeof(STAR_STREAM_ITEM *));

 if ((transferOperations != NULL) && (packets != NULL))
 {
 /* Define counter for loop */
 unsigned char index;

 /* For number of packets */
 for (index = 0; index < numberOfPackets; index++)
 {
 /* Create packet data with current index */
 unsigned char packetData[1];
 packetData[0] = index;

 {
 /* Create packet from packet data */
 STAR_STREAM_ITEM * packet =
STAR_createPacket(NULL,
 packetData, 1, STAR_EOP_TYPE_EOP);

 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
STAR_createTxOperation(&packet, 1);

 /* Store send transfer operation in array */
 transferOperations[index] = sendTransferOperation;

 /* Store packet so that it can be disposed later */
 packets[index] = packet;
 }
 }

 /* Start transmitting packets */
 STAR_submitTransferOperationList(channel,
transferOperations,
 numberOfPackets);

 /* For number of packets */
 for (index = 0; index < numberOfPackets; index++)
 {
 /* Get current transfer operation */
 STAR_TRANSFER_OPERATION * transferOperation =
transferOperations[index];

 /* Get current packet */
 STAR_STREAM_ITEM * packet = packets[index];

 /* Wait for transfer operation to complete */
 STAR_waitOnTransferOperationCompletion(
transferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* Destroy packet */
 STAR_destroyStreamItem(packet);
 }
 }
 }
 }
}

```

```

 if (packets != NULL)
 {
 /* Free array of packets */
 free(packets);
 }

 if (transferOperations != NULL)
 {
 /* Free array of transfer operations */
 free(transferOperations);
 }

 /* Close channel */
 STAR_closeChannel(channel);
 }
}

```

## 10.53 transfer\_operation\_list\_send\_receive\_example.c

This example assumes that channel 1 (defined in general header for api\_test\_app project) is connected to link 2 on a SpaceWire USB Brick or other routing device. Any packets sent out of channel 1 will end up coming back to where they started.

### PCI Mk2 Brick

Link 1  Link 2

Link 2 

Link 3 

```

#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

void transferOperationListSendReceiveExample()
{
 /* Get first device */
 STAR_DEVICE_ID firstDevice = getFirstDevice();

 /* If there is a first device */
 if(firstDevice)
 {
 /* Open channel to send out of */
 STAR_CHANNEL_ID channel = STAR_openChannelToLocalDevice(
 firstDevice,
 STAR_CHANNEL_DIRECTION_INOUT, CHANNEL1, 1);

 /* If channel was opened */
 if(channel)
 {
 /* Create address path */
 unsigned char addressPath[] = {2};

 /* Get address path length */
 U16 addressPathLength = sizeof(addressPath) / sizeof(unsigned char);

 /* Create SpaceWire address */
 STAR_SPACEWIRE_ADDRESS * address =
 STAR_createAddress(addressPath,
 addressPathLength);

 /* Define packet data to be sent out */
 unsigned char packetData[] = {1, 2, 3, 4, 5, 6, 7, 8, 9};

 /* Get packet length */
 unsigned int packetDataLength = sizeof(packetData) /
 sizeof(unsigned char);

 /* Create packet */
 STAR_STREAM_ITEM * packet = STAR_createPacket(address,
 packetData,
 packetDataLength, STAR_EOP_TYPE_EOP);

 /* If packet was created */
 if(packet)

```

```

{
 /* Create send transfer operation */
 STAR_TRANSFER_OPERATION * sendTransferOperation =
 STAR_createTxOperation(&packet, 1);

 /* Create receive transfer operation */
 STAR_TRANSFER_OPERATION * receiveTransferOperation =
 STAR_createRxOperation(1,
 STAR_RECEIVE_PACKETS);

 /* If send and receive transfer operations were created */
 if ((sendTransferOperation != NULL) &&
 (receiveTransferOperation != NULL))
 {
 /* Define transfer status */
 STAR_TRANSFER_STATUS status;

 /* Put send and receive transfer operations in an array */
 STAR_TRANSFER_OPERATION * transferOperations[2];
 transferOperations[0] = sendTransferOperation;
 transferOperations[1] = receiveTransferOperation;

 /* Submit transfer operation list */
 STAR_submitTransferOperationList(channel,
 transferOperations, 2);

 /* Wait for send transfer operation to complete */
 status = STAR_waitOnTransferOperationCompletion(
 sendTransferOperation, -1);

 /* If packet was sent successfully */
 if(status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Wait for receive transfer operation to complete */
 status = STAR_waitOnTransferOperationCompletion
(
 receiveTransferOperation, -1);

 /* If packet was received successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Get received packet */
 STAR_STREAM_ITEM * streamItem =
 STAR_getTransferItem(
 receiveTransferOperation, 0);

 /* Initialise data length to 0 */
 unsigned int dataLength = 0;

 /* Get packet data and its length */
 unsigned char * data = STAR_getPacketData(
 (STAR_SPACEWIRE_PACKET *)streamItem->
 item, &dataLength);

 /* Check that packet data is valid */
 if (data)
 {
 /* Print received packet */
 printPacketContents(data, dataLength);

 /* Destroy packet data now that it has been
 output */
 STAR_destroyPacketData(data);
 }
 }

 /* Dispose send transfer operation */
 STAR_disposeTransferOperation(sendTransferOperation);
 }

 /* Dispose receive transfer operation */
 STAR_disposeTransferOperation(receiveTransferOperation);
 }

 /* Destroy packet */
 STAR_destroyStreamItem(packet);
}

/* Close channel */
STAR_closeChannel(channel);
}
}

```

## 10.54 transmit\_errors\_example.c

The user is prompted for the device to use, the channel to send out of and a series of bytes to be sent in the packets. The packet is sent twice, the first time containing a disconnect error in the middle and the second time containing a parity error in the middle.

```
#include "examples.h"

#include "general.h"
#include "utilities.h"

#include "star-dundee_types.h"
#include "star-api.h"

#include <stdlib.h>

void transmitErrorsExample()
{
 /* Pointer to a list of available SpaceWire devices */
 STAR_DEVICE_ID *devices = NULL;

 /* Define selected channel */
 int selectedChannelNumber;

 /* Get selected device */
 STAR_DEVICE_ID selectedDevice = promptForDevice(TRANSMIT_TYPE_SEND);

 /* Get selected channel */
 STAR_CHANNEL_ID selectedChannel = promptForChannel(selectedDevice,
 &selectedChannelNumber, TRANSMIT_TYPE_SEND);

 /* If channel was opened */
 if(selectedChannel)
 {
 /* Allocate array of bytes to be populated */
 unsigned char bytes[256];
 U16 readBytesLength = 0;

 /* Length of start of packet to be transmitted */
 U16 packetStartLength;

 /* Define the stream items that will hold the packet and errors */
 STAR_STREAM_ITEM *packetStart, *packetEnd;
 STAR_STREAM_ITEM *disconnectErrorItem, *parityErrorItem;
 STAR_STREAM_ITEM *packet[3];

 /* Ask user to enter a series of bytes to send */
 printf("\nPlease enter a series of bytes to be transmitted over "
 "SpaceWire:\n");

 /* If valid bytes read input bytes from console */
 if(readBytes(bytes, 256, &readBytesLength))
 {
 /* Get length of packet string */
 packetStartLength = readBytesLength / 2;

 /* Create the stream items */
 packetStart = STAR_createDataChunk(bytes,
 packetStartLength, 1, STAR_EOP_TYPE_NONE);
 packetEnd = STAR_createDataChunk(
 bytes + packetStartLength,
 readBytesLength - packetStartLength, 0, STAR_EOP_TYPE_EOP);
 disconnectErrorItem = STAR_createErrorInData(
 STAR_ERROR_IN_DATA_DISCONNECT);
 parityErrorItem = STAR_createErrorInData(
 STAR_ERROR_IN_DATA_PARITY);

 /* Make packet with from stream items with disconnect error */
 packet[0] = packetStart;
 packet[1] = disconnectErrorItem;
 packet[2] = packetEnd;

 /* If the stream items were created */
 if((packetStart != NULL) && (packetEnd != NULL) &&
 (disconnectErrorItem != NULL) && (parityErrorItem != NULL))
 {
 /* Create transfer operation */
 STAR_TRANSFER_OPERATION *transferOperation =
 STAR_createTxOperation(packet, 3);

 /* If transfer operation was created */
 if(transferOperation != NULL)
 {

```

```

 /* Define transfer status for result of transfer */
 /* operation */
 STAR_TRANSFER_STATUS status;

 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion(
 transferOperation, -1);

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);

 /* If the transfer operation completed successfully */
 if (status == STAR_TRANSFER_STATUS_COMPLETE)
 {
 /* Change the packet to contain a parity error */
 packet[1] = parityErrorItem;

 /* Create transfer operation */
 transferOperation = STAR_createTxOperation(packet, 3);

 /* If transfer operation was created */
 if (transferOperation != NULL)
 {
 /* Start transmitting the packet */
 STAR_submitTransferOperation(selectedChannel,
 transferOperation);

 /* Wait for packet to be transmitted */
 status = STAR_waitOnTransferOperationCompletion
 (
 transferOperation, -1);

 /* Check transfer status */
 if (status != STAR_TRANSFER_STATUS_COMPLETE)
 {
 printf("Unexpected transfer status: %d\n",
 status);
 }

 /* Dispose transfer operation */
 STAR_disposeTransferOperation(transferOperation);
 }
 }
 else
 {
 /* Unexpected transfer status */
 printf("Unexpected transfer status: %d\n", status);
 }
 }

 /* Destroy the stream items */
 if(packetStart != NULL)
 {
 STAR_destroyStreamItem(packetStart);
 }
 if(packetEnd != NULL)
 {
 STAR_destroyStreamItem(packetEnd);
 }
 if(disconnectErrorItem != NULL)
 {
 STAR_destroyStreamItem(disconnectErrorItem);
 }
 if(parityErrorItem != NULL)
 {
 STAR_destroyStreamItem(parityErrorItem);
 }
}

/* Close channel after transmitting packet */
STAR_closeChannel(selectedChannel);
}
else
{
 /* Print error */
 printf("Channel %d could not be opened.\n", selectedChannelNumber);
}

/* Destroy device list */
STAR_destroyDeviceList(devices);
}

```

## 10.55 triggering\_example.c

This file contains example code demonstrating how to use the Triggering API.

```
#include "utility.h"

#include "triggering_brick_mk3.h"
#include "triggering_pxi_if.h"
#include "triggering_pxi_ro.h"
#include "triggering_pcie.h"
#include "triggering_pcie_mk2.h"
#include "cfg_api_generic.h"

typedef enum
{
 MENU_CHOICE_TX_TC_ON_EXT_TRIGGER = 1,
 MENU_CHOICE_TX_EXT_TRIGGER_ON_TC,
 MENU_CHOICE_TX_EXT_TRIGGER_ON_COUNTER,
 MENU_CHOICE_TX_TC_ON_COUNTER,
 MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER,
 MENU_CHOICE_PKT_TX_ON_COUNTER,
 MENU_CHOICE_PKT_TX_ON_TC,
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER,
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC,
 MENU_CHOICE_TIMED_PKT_TX_ON_TC,
 MENU_CHOICE_STOP_RECEIVING,
 MENU_CHOICE_PRINT_TRIGGER_CONF,
 MENU_CHOICE_RESET,
 MENU_CHOICE_EXIT,
 MENU_CHOICE_INVALID
} MENU_CHOICE;

void txTcOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Transmit time-code on external trigger activity) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Disable external trigger output */
 configResult = TRIGGER_BRICK_MK3_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Internal trigger 0 causes TIME_CODE_ACTION_RX action on external
 trigger 0 */
 configResult = TRIGGER_BRICK_MK3_setTimeCodeOutputActions(
 deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Disable external trigger output */
 configResult = TRIGGER_PXI_IF_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
```

```

 trigger 0 to be set */
 configResult = TRIGGER_PXI_IF_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Internal trigger 0 causes TIME_CODE_ACTION_RX action on external
 trigger 0 */
 configResult = TRIGGER_PXI_IF_setTimeCodeOutputActions(
 deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 }
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Disable output on external trigger 0 */
 configResult = TRIGGER_PCIE_MK2_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Internal trigger 0 causes TIME_CODE_ACTION_RX action on external
 trigger 0 */
 configResult = TRIGGER_PCIE_MK2_setTimeCodeOutputActions(
 deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("Time-codes will now be transmitted when external trigger 0 is "
 "pulsed.\n\n");
}

void txExtTriggerOnTc(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Generate external trigger on received time-code) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Generate external trigger */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerOutputActions(
 deviceId, 0, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Enable external trigger output */
 configResult = TRIGGER_BRICK_MK3_enableExtTriggerOutput(
 deviceId, 0);

 /* External trigger extend value */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerExtend(deviceId,
 0, 2000);

 /* Internal trigger 0 is triggered on time-code TIME_CODE_EVENT_TICK */
 configResult = TRIGGER_BRICK_MK3_setTimeCodeInputEvents(
 deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Generate external trigger */
 configResult = TRIGGER_PXI_IF_setExtTriggerOutputActions(
 deviceId, 0, 0,

```



```

 EXT_TRIGGER_ACTION_OUT);

 /* Enable external trigger output */
 configResult = TRIGGER_PXI_IF_enableExtTriggerOutput(deviceId,
0);

 /* External trigger extend value */
 configResult = TRIGGER_PXI_IF_setExtTriggerExtend(deviceId, 0, 20
00);

 /* Internal trigger 0 is triggered on time-code TIME_CODE_EVENT_TICK */
 configResult = TRIGGER_PXI_IF_setTimeCodeInputEvents(deviceId,
0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Generate external trigger */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerOutputActions
(deviceId, 0, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Enable external trigger output */
 configResult = TRIGGER_PCIE_MK2_enableExtTriggerOutput (
deviceId, 0);

 /* External trigger extend value */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerExtend(deviceId, 0
, 2000);

 /* Internal trigger 0 is triggered on time-code TIME_CODE_EVENT_TICK */
 configResult = TRIGGER_PCIE_MK2_setTimeCodeInputEvents (
deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("External trigger on trigger 0 is going to be generated on each received time-code.\n\n");
}

void txTcOnCounter(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;
 int clockRate = 0;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Transmit time-code on timer) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_BRICK_MK3_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
be set */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
engine 0 */
 configResult = TRIGGER_BRICK_MK3_setTimeCodeOutputActions
(deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */

```

```

 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_IF_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
be set */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
engine 0 */
 configResult = TRIGGER_PXI_IF_setTimeCodeOutputActions(
deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_ROUTER_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PXI_ROUTER_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
be set */
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
engine 0 */
 configResult = TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(
deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_PCIE_MK2_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(
deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
be set */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes TIME_CODE_ACTION_TX action on time-code
engine 0 */
 configResult = TRIGGER_PCIE_MK2_setTimeCodeOutputActions(
deviceId, 0, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */

```

```

 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
 }

 printf("Time-codes will now be transmitted every second.\n\n");
}

void txExtTriggerOnCounter(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;
 int clockRate = 0;

 /* Define the external trigger extend period */
 const U32 extTriggerExtendValue = 2000;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Transmit time-code on timer) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_BRICK_MK3_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes EXT_TRIGGER_ACTION_OUT action on external trigger 0 */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerOutputActions(
 deviceId, 0, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Set external trigger extend value */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerExtend(deviceId,
 0,
 extTriggerExtendValue);

 /* Enable external trigger output */
 configResult = TRIGGER_BRICK_MK3_enableExtTriggerOutput(
 deviceId, 0);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_IF_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
 ,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
 deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
 , 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes EXT_TRIGGER_ACTION_OUT action on external trigger 0 */
 configResult = TRIGGER_PXI_IF_setExtTriggerOutputActions(
 deviceId, 0, 0,
 EXT_TRIGGER_ACTION_OUT);
 }
}

```

```

 /* Set external trigger extend value */
 configResult = TRIGGER_PXI_IF_setExtTriggerExtend(deviceId, 0,
 extTriggerExtendValue);

 /* Enable external trigger output */
 configResult = TRIGGER_PXI_IF_enableExtTriggerOutput(deviceId,
0);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PCIE_MK2_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(
deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes EXT_TRIGGER_ACTION_OUT action on external trigger 0 */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerOutputActions
(deviceId, 0, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Set external trigger extend value */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerExtend(deviceId, 0
,
 extTriggerExtendValue);

 /* Enable external trigger output */
 configResult = TRIGGER_PCIE_MK2_enableExtTriggerOutput(
deviceId, 0);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("Pulse on External trigger will now be generated every second.\n\n");
}

void pktTxOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Packet transmit on external trigger) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_BRICK_MK3_disableExtTriggerOutput(
deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode
(deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerInputEvents
(deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode
(deviceId, 1);
 }
}

```

```

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_PXI_IF_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_IF_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_PCIE_MK2_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("8 packets have been queued and will now be transmitted on port 1 "
 "when external trigger 0 is pulsed.\n\n");
}

void pktTxOnCounter(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;
 int clockRate = 0;

```

```

reset(deviceId);
printf("\n----- Trigger Example (Packet transmit on timer) -----\\n");

if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_BRICK_MK3_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_IF_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
 deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_ROUTER_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
 deviceId, 0,
 clockRate);

 /* Enable timer auto reload */

```

```

 configResult = TRIGGER_PXI_ROUTER_enableCounterAutoReload
(deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode
(deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_ROUTER_setPortOutputActions(
deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PCIE_IF_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PCIE_IF_setCounterReloadValue(deviceId,
0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PCIE_IF_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PCIE_IF_setCounterInputEvents(deviceId,
0, 0,
 COUNTER_EVENT_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_IF_setPortOutputActions(deviceId, 1
, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_IF_enableTrigger(deviceId, 0);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PCIE_MK2_getDeviceClockRate();

 /* Set timer 0 reload value to 1 second */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(
deviceId, 0,
 clockRate);

 /* Enable timer auto reload */
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 0);

 /* COUNTER_EVENT_RELOAD event on timer 0 causes internal trigger 0 to
 be set */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(
deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode
(deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 0,
 PORT_ACTION_TRANSMIT_PKT);

```

```

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("8 packets have been queued and will now be transmitted every "
 "second.\n\n");
}

void pktTxOnTc(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Packet transmit on time-code reception) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_BRICK_MK3_setTimeCodeInputEvents(
 deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode
 (deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_IF_setTimeCodeInputEvents(deviceId,
 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode
 (deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(
 deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode
 (deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */

```



```

 configResult = TRIGGER_PXI_ROUTER_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
}
if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PCIE_MK2_setTimeCodeInputEvents(
 deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8);

 /* Enable internal trigger 0 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
}

printf("8 packets have been queued and will now be transmitted when "
 "valid time-codes are received.\n\n");
}

void multiplePktTxOnExtTrigger(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF ||
 gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Multiple packet transmit on external "
 "trigger) ----- \n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 configResult = TRIGGER_BRICK_MK3_enableCounterTriggerCount(
 deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 0, 3);

 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_BRICK_MK3_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_BRICK_MK3_enableExtTriggerEdgeDetectMode(
 deviceId, 0);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_BRICK_MK3_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* External trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 0, 0,

```

```

 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 configResult = TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1
, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 configResult = TRIGGER_PXI_IF_enableCounterTriggerCount(
 deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
, 3);

 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_PXI_IF_disableExtTriggerOutput(
 deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PXI_IF_enableExtTriggerEdgeDetectMode
(deviceId, 0);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode
(deviceId, 1);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_IF_setExtTriggerInputEvents(
 deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* External trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
 deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 configResult = TRIGGER_PXI_IF_setPortInputEvents(deviceId, 1, 1,
PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
 deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

```

```

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 configResult = TRIGGER_PCIE_MK2_enableCounterTriggerCount(
deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId
, 0, 3);

 /* Disable external trigger output (basically enable it as input) */
 configResult = TRIGGER_PCIE_MK2_disableExtTriggerOutput(
deviceId, 0);

 /* Enable external trigger 0 edge detect mode */
 configResult = TRIGGER_PCIE_MK2_enableExtTriggerEdgeDetectMode
(deviceId, 0);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode
(deviceId, 1);

 /* EXT_TRIGGER_EVENT_IN event on external trigger 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PCIE_MK2_setExtTriggerInputEvents(
deviceId, 0, 0,
 EXT_TRIGGER_EVENT_IN);

 /* External trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 configResult = TRIGGER_PCIE_MK2_setPortInputEvents(deviceId, 1, 1
, PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 1);

```

```

 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 2);
 }

 printf("32 packets have been queued and will now be transmitted in groups "
 "of four on port 1 when external trigger 0 is pulsed.\n\n");
}

void multiplePktTxOnTc(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Multiple packet transmit on time-code) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 configResult = TRIGGER_BRICK_MK3_enableCounterTriggerCount(
 deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 0, 3);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode(
 deviceId, 1);

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_BRICK_MK3_setTimeCodeInputEvents(
 deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 configResult = TRIGGER_BRICK_MK3_setPortInputEvents(deviceId, 1
 , 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on

```

```

 triggers */
 configResult = TRIGGER_PXI_IF_enableCounterTriggerCount(
deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
, 3);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode
(deviceId, 1);

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_IF_setTimeCodeInputEvents(deviceId,
0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
0,
 PORT_ACTION_TRANSMIT_PKT);

 /* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
 configResult = TRIGGER_PXI_IF_setPortInputEvents(deviceId, 1, 1,
 PORT_EVENT_TX_EOP);

 /* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

 /* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
 set */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
, 2,
 COUNTER_EVENT_COUNT);

 /* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
 }
 else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
 {
 /* Enable timer trigger count on timer 0 so the timer only counts on
 triggers */
 configResult = TRIGGER_PXI_ROUTER_enableCounterTriggerCount
(deviceId, 0);

 /* Set timer 0 reload value to 3 */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
deviceId, 0, 3);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode
(deviceId, 1);

 /* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
 trigger 0 to be set */
 configResult = TRIGGER_PXI_ROUTER_setTimeCodeInputEvents(
deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

 /* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
 configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions
(deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

 /* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
 port 1 */
 configResult = TRIGGER_PXI_ROUTER_setPortOutputActions(

```

```

deviceId, 1, 0,
 PORT_ACTION_TRANSMIT_PKT);

/* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
configResult = TRIGGER_PXI_ROUTER_setPortInputEvents(deviceId,
1, 1,
 PORT_EVENT_TX_EOP);

/* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions
(deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

/* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
set */
configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
deviceId, 0, 2,
 COUNTER_EVENT_COUNT);

/* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
port 1 */
configResult = TRIGGER_PXI_ROUTER_setPortOutputActions(
deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

/* Queue up some packets */
queuePackets(deviceId, 32);

/* Enable internal triggers 0, 1 and 2 */
configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 1);
configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 2);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
/* Enable timer trigger count on timer 0 so the timer only counts on
triggers */
configResult = TRIGGER_PCIE_MK2_enableCounterTriggerCount(
deviceId, 0);

/* Set timer 0 reload value to 3 */
configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId
, 0, 3);

/* Enable port transmit packet mode on port 1 */
configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode
(deviceId, 1);

/* TIME_CODE_EVENT_TICK event on time-code engine 0 causes internal
trigger 0 to be set */
configResult = TRIGGER_PCIE_MK2_setTimeCodeInputEvents(
deviceId, 0, 0,
 TIME_CODE_EVENT_TICK);

/* Internal trigger 0 causes COUNTER_ACTION_RELOAD action on timer 0 */
configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 0, 0,
 COUNTER_ACTION_RELOAD);

/* Internal trigger 0 causes PORT_ACTION_TRANSMIT_PKT action on
port 1 */
configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 0,
 PORT_ACTION_TRANSMIT_PKT);

/* PORT_EVENT_TX_EOP event causes internal trigger 1 to be set */
configResult = TRIGGER_PCIE_MK2_setPortInputEvents(deviceId, 1, 1
,
 PORT_EVENT_TX_EOP);

/* Internal trigger 1 causes COUNTER_ACTION_COUNT action on timer 0 */
configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 0, 1,
 COUNTER_ACTION_COUNT);

/* COUNTER_EVENT_COUNT event on timer 0 causes internal trigger 2 to be
set */
configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 0, 2,
 COUNTER_EVENT_COUNT);

/* Internal trigger 2 causes PORT_ACTION_TRANSMIT_PKT action on
port 1 */
configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 2,
 PORT_ACTION_TRANSMIT_PKT);

```

```

 /* Queue up some packets */
 queuePackets(deviceId, 32);

 /* Enable internal triggers 0, 1 and 2 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 2);
}

printf("32 packets have been queued and will now be transmitted in groups "
 "of four on port 1 when time-codes are received.\n\n");
}

void timedPktTxOnTc(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;
 int clockRate = 0;

 if (gTriggerDevice == TRIGGER_DEVICE_PCIE_IF)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Timed Packet Transmit on Time-Code) "
 "-----\n");

 if (gTriggerDevice == TRIGGER_DEVICE_BRICK_MK3)
 {
 /* Retrieve clock rate */
 clockRate = TRIGGER_BRICK_MK3_getDeviceClockRate();

 /* Setup timer 0 to transmit time-codes every second and reload/start
 timer 1 */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 0, clockRate);
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 configResult = TRIGGER_BRICK_MK3_setTimeCodeOutputActions(
 deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 1,
 clockRate / 5);
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 1);
 configResult = TRIGGER_BRICK_MK3_enableCounterStartStopMode(
 deviceId, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(
 deviceId, 2, 4);
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 2);
 configResult = TRIGGER_BRICK_MK3_enableCounterTriggerCount(
 deviceId, 2);

 /* Setup timer 2 to transmit packets */
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
 deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(
 deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 }
}

```

```

 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(
deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
deviceId, 1, 3,
 COUNTER_ACTION_STOP);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_BRICK_MK3_enablePortTransmitPacketMode
(deviceId, 1);

 /* Reload timers */
 configResult = TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 0
);
 configResult = TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 1
);
 configResult = TRIGGER_BRICK_MK3_forceCounterReload(deviceId, 2
);

 /* Enable internal triggers 0-3 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 2);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_IF)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_IF_getDeviceClockRate();

 /* Setup timer 0 to transmit time-codes every second and reload/start
timer 1 */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 0
, clockRate);
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
deviceId, 0);
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 0
, 0,
 COUNTER_EVENT_RELOAD);
 configResult = TRIGGER_PXI_IF_setTimeCodeOutputActions(
deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 1
,
 clockRate / 5);
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
deviceId, 1);
 configResult = TRIGGER_PXI_IF_enableCounterStartStopMode(
deviceId, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 1
, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 configResult = TRIGGER_PXI_IF_setCounterReloadValue(deviceId, 2
, 4);
 configResult = TRIGGER_PXI_IF_enableCounterAutoReload(
deviceId, 2);
 configResult = TRIGGER_PXI_IF_enableCounterTriggerCount(
deviceId, 2);

 /* Setup timer 2 to transmit packets */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 2
, 2,
 COUNTER_EVENT_COUNT);
 configResult = TRIGGER_PXI_IF_setPortOutputActions(deviceId, 1,
2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 configResult = TRIGGER_PXI_IF_setCounterInputEvents(deviceId, 2
, 3,

```



```

 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PXI_IF_setCounterOutputActions(
 deviceId, 1, 3,
 COUNTER_ACTION_STOP);

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_IF_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Reload timers */
 configResult = TRIGGER_PXI_IF_forceCounterReload(deviceId, 0);
 configResult = TRIGGER_PXI_IF_forceCounterReload(deviceId, 1);
 configResult = TRIGGER_PXI_IF_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 2);
 configResult = TRIGGER_PXI_IF_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PXI_ROUTER)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PXI_ROUTER_getDeviceClockRate();

 /* Setup timer 0 to transmit time-codes every second and reload/start
 timer 1 */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
 deviceId, 0,
 clockRate);
 configResult = TRIGGER_PXI_ROUTER_enableCounterAutoReload(
 deviceId, 0);
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
 deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 configResult = TRIGGER_PXI_ROUTER_setTimeCodeOutputActions(
 deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions(
 deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions(
 deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
 deviceId, 1,
 clockRate / 5);
 configResult = TRIGGER_PXI_ROUTER_enableCounterAutoReload(
 deviceId, 1);
 configResult = TRIGGER_PXI_ROUTER_enableCounterStartStopMode(
 deviceId, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
 deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions(
 deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 configResult = TRIGGER_PXI_ROUTER_setCounterReloadValue(
 deviceId, 2, 4);
 configResult = TRIGGER_PXI_ROUTER_enableCounterAutoReload(
 deviceId, 2);
 configResult = TRIGGER_PXI_ROUTER_enableCounterTriggerCount(
 deviceId, 2);

 /* Setup timer 2 to transmit packets */
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
 deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 configResult = TRIGGER_PXI_ROUTER_setPortOutputActions(
 deviceId, 1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 configResult = TRIGGER_PXI_ROUTER_setCounterInputEvents(
 deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PXI_ROUTER_setCounterOutputActions(
 deviceId, 1, 3,
 COUNTER_ACTION_STOP);
}

```

```

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PXI_ROUTER_enablePortTransmitPacketMode
(deviceId, 1);

 /* Reload timers */
 configResult = TRIGGER_PXI_ROUTER_forceCounterReload(deviceId,
0);
 configResult = TRIGGER_PXI_ROUTER_forceCounterReload(deviceId,
1);
 configResult = TRIGGER_PXI_ROUTER_forceCounterReload(deviceId,
2);

 /* Enable internal triggers 0-3 */
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 2);
 configResult = TRIGGER_PXI_ROUTER_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}
else if (gTriggerDevice == TRIGGER_DEVICE_PCIE_MK2)
{
 /* Retrieve clock rate */
 clockRate = TRIGGER_PCIE_MK2_getDeviceClockRate();

 /* Setup timer 0 to transmit time-codes every second and reload/start
timer 1 */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId
, 0,
 clockRate);
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 0);
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 0, 0,
 COUNTER_EVENT_RELOAD);
 configResult = TRIGGER_PCIE_MK2_setTimeCodeOutputActions(
deviceId, 0, 0,
 TIME_CODE_ACTION_TX);
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup timer 1 to expire every 200 ms and enable start/stop mode */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId
, 1,
 clockRate / 5);
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 1);
 configResult = TRIGGER_PCIE_MK2_enableCounterStartStopMode
(deviceId, 1);

 /* Setup timer 1 to decrement timer 2 (packet counter timer) */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 /* Setup timer 2 to count from 4 to 0 and enable trigger count mode */
 configResult = TRIGGER_PCIE_MK2_setCounterReloadValue(deviceId
, 2, 4);
 configResult = TRIGGER_PCIE_MK2_enableCounterAutoReload(
deviceId, 2);
 configResult = TRIGGER_PCIE_MK2_enableCounterTriggerCount(
deviceId, 2);

 /* Setup timer 2 to transmit packets */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 2, 2,
 COUNTER_EVENT_COUNT);
 configResult = TRIGGER_PCIE_MK2_setPortOutputActions(deviceId,
1, 2,
 PORT_ACTION_TRANSMIT_PKT);

 /* Setup timer 2 to stop timer 1 when it hits 0 */
 configResult = TRIGGER_PCIE_MK2_setCounterInputEvents(deviceId
, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 configResult = TRIGGER_PCIE_MK2_setCounterOutputActions(
deviceId, 1, 3,
 COUNTER_ACTION_STOP);
}

```

```

 /* Enable port transmit packet mode on port 1 */
 configResult = TRIGGER_PCIE_MK2_enablePortTransmitPacketMode(
 deviceId, 1);

 /* Reload timers */
 configResult = TRIGGER_PCIE_MK2_forceCounterReload(deviceId, 0);
 configResult = TRIGGER_PCIE_MK2_forceCounterReload(deviceId, 1);
 configResult = TRIGGER_PCIE_MK2_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 0);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 1);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 2);
 configResult = TRIGGER_PCIE_MK2_enableTrigger(deviceId, 3);

 queuePackets(deviceId, 32);
}

printf("32 packets have been queued and will now be transmitted in groups "
 "of 4 at 200 ms intervals when time-codes are transmitted.\n\n");
}

void stopReceiving(STAR_DEVICE_ID deviceId)
{
 /* Keep track of the result of each config call, this is used for debugging purposes */
 int configResult = -1;
 int clockRate = 0;

 if (gTriggerDevice != TRIGGER_DEVICE_BRICK_MK3)
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 reset(deviceId);
 printf("\n----- Trigger Example (Stop Receiving on Port 2 For 5 Seconds "
 "Every 10 Seconds) ----- \n");

 /* Retrieve clock rate */
 clockRate = TRIGGER_BRICK_MK3_getDeviceClockRate();

 /* Setup counter 0 to fire every 10 seconds and reload counter 1 */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId,
 0,
 10 * clockRate);
 configResult = TRIGGER_BRICK_MK3_enableCounterAutoReload(
 deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId,
 0, 0,
 COUNTER_EVENT_RELOAD);
 configResult = TRIGGER_BRICK_MK3_setCounterOutputActions(
 deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);

 /* Setup counter 1 to fire a single-shot 5 second timer which, while */
 /* not zero, causes the STOP_RECEPTION action on port 2 */
 configResult = TRIGGER_BRICK_MK3_setCounterReloadValue(deviceId,
 1,
 5 * clockRate);
 configResult = TRIGGER_BRICK_MK3_disableCounterAutoReload(
 deviceId, 1);
 configResult = TRIGGER_BRICK_MK3_setCounterInputEvents(deviceId,
 1, 1,
 COUNTER_EVENT_ZERO);
 configResult = TRIGGER_BRICK_MK3_setPortOutputActions(deviceId, 2
 , 1,
 PORT_ACTION_STOP_RECEPTION);

 /* Setup internal trigger 1 to receive inverted input from counter 1 */
 configResult = TRIGGER_BRICK_MK3_setTriggerInvertMask(deviceId, 1
 ,
 TRIGGER_TYPE_COUNTER, 0x2);

 /* Enable internal triggers 0-1 */
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 0);
 configResult = TRIGGER_BRICK_MK3_enableTrigger(deviceId, 1);

 printf("Reception will now be stopped for 5 seconds every 10 seconds.\n\n");
}

void resetTriggerConf(STAR_DEVICE_ID deviceId)
{
 reset(deviceId);
 printf("\n----- Trigger Example (Reset triggers) ----- \n");
 printf("Triggers have been reset.\n\n");
}

```

```

MENU_CHOICE getMenuChoice(void)
{
 char buffer[32];
 unsigned int choice;

 /* Show menu to user */
 printf("Please select example to run:\n");
 printf("%d. Transmit time-code on external trigger.\n",
 MENU_CHOICE_TX_TC_ON_EXT_TRIGGER);
 printf("%d. Generate external trigger on received time-code.\n",
 MENU_CHOICE_TX_EXT_TRIGGER_ON_TC);
 printf("%d. Generate external trigger on counter.\n",
 MENU_CHOICE_TX_EXT_TRIGGER_ON_COUNTER);
 printf("%d. Transmit time-code on timer.\n",
 MENU_CHOICE_TX_TC_ON_COUNTER);
 printf("%d. Transmit packet on external trigger.\n",
 MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER);
 printf("%d. Transmit packet on timer.\n",
 MENU_CHOICE_PKT_TX_ON_COUNTER);
 printf("%d. Transmit packet on time-code.\n",
 MENU_CHOICE_PKT_TX_ON_TC);
 printf("%d. Transmit multiple packets on external trigger.\n",
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER);
 printf("%d. Transmit multiple packets on time-code.\n",
 MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC);
 printf("%d. Transmit packets at intervals with time-code.\n",
 MENU_CHOICE_TIMED_PKT_TX_ON_TC);
 printf("%d. Stop receiving for 5 seconds every 10 seconds.\n",
 MENU_CHOICE_STOP_RECEIVING);
 printf("%d. Print trigger configuration.\n",
 MENU_CHOICE_PRINT_TRIGGER_CONF);
 printf("%d. Reset triggers.\n", MENU_CHOICE_RESET);
 printf("%d. Exit\n", MENU_CHOICE_EXIT);
 printf("> ");

 /* Validate chosen number */
 if (!fgets(buffer, 32, stdin)) return MENU_CHOICE_INVALID;

 /* Get menu choice value */
 if (!sscanf(buffer, "%u", &choice) ||
 choice >= MENU_CHOICE_INVALID) return MENU_CHOICE_INVALID;

 /* Return the chosen value */
 return choice;
}

STAR_DEVICE_ID getTriggerDeviceID()
{
 char *pDeviceName, *pDeviceSerial, buffer[256];
 unsigned int chosen;
 int status;

 /* List devices that support triggering API */
 U32 deviceTypes[11] = { STAR_DEVICE_BRICK_MK3,
 STAR_DEVICE_BRICK_MK4,
 STAR_DEVICE_GBE_BRICK,
 STAR_DEVICE_PXI_INTERFACE,
 STAR_DEVICE_PXI_INTERFACE_MK2,
 STAR_DEVICE_PXI_RMAP,
 STAR_DEVICE_PXI_RMAP_MK2,
 STAR_DEVICE_PXI_ROUTER_12,
 STAR_DEVICE_PXI_ROUTER_MK2,
 STAR_DEVICE_PCIE,
 STAR_DEVICE_PCIE_MK2};

 /* Get available devices that support triggering */
 U32 deviceCount;
 STAR_DEVICE_ID* pDevices = STAR_getDeviceListForTypes(
 deviceTypes, sizeof(deviceTypes)/sizeof(deviceTypes[0]),
 &deviceCount);
 STAR_DEVICE_ID selectedDeviceID = STAR_DEVICE_UNKNOWN;

 /* If more than one device is available */
 if (deviceCount > 1)
 {
 U32 x;

 printf("The following devices are available that support triggering:"
 "\n\n");

 /* For each device */
 for (x = 0; x < deviceCount; x++)
 {
 /* Display its name */
 if (pDevices[x] != STAR_DEVICE_UNKNOWN)
 {
 pDeviceName = STAR_getDeviceName(pDevices[x]);
 }
 }
 }
}

```

```

 pDeviceSerial = STAR_getDeviceSerialNumber(pDevices[x]);
 if (pDeviceName != NULL && pDeviceSerial != NULL)
 {
 printf("\t%u - %s with serial number %s\n", x, pDeviceName,
 pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", x);
 }

 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unable to access device\n", x);
 }
}

printf("\n");

/* Ask the user which device to use */
do
{
 printf("Please select which device to use: ");
 fflush(stdout);

 /* Read selected device number */
 if (fgets(buffer, 256, stdin) == NULL)
 {
 puts("No device number selected.");
 continue;
 }
 status = sscanf(buffer, "%u", &chosen);
 if ((status == 0) || (chosen > (deviceCount - 1U)))
 {
 puts("Invalid device number selected.");
 continue;
 }

 /* Get the selected device id */
 selectedDeviceID = pDevices[chosen];
}
while (selectedDeviceID == STAR_DEVICE_UNKNOWN);
}
/* Else if just one device is available */
else if (deviceCount == 1)
{
 /* Select the first device id */
 selectedDeviceID = pDevices[0];
}

/* If valid device was selected */
if (selectedDeviceID != STAR_DEVICE_UNKNOWN)
{
 /* Get device name and serial */
 pDeviceName = STAR_getDeviceName(selectedDeviceID);
 pDeviceSerial = STAR_getDeviceSerialNumber(selectedDeviceID);

 /* Print selected device */
 printf("%s selected with serial number %s\n\n", pDeviceName,
 pDeviceSerial);

 /* Destroy device and serial */
 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
}

STAR_destroyDeviceList(pDevices);

/* Return the selected device id */
return selectedDeviceID;
}

TRIGGER_DEVICE getTriggerDeviceType(U32 deviceType)
{
 if (deviceType == STAR_DEVICE_BRICK_MK3 ||
 deviceType == STAR_DEVICE_BRICK_MK4 ||
 deviceType == STAR_DEVICE_GBE_BRICK)
 {
 return TRIGGER_DEVICE_BRICK_MK3;
 }
 else if (deviceType == STAR_DEVICE_PXI_INTERFACE ||
 deviceType == STAR_DEVICE_PXI_RMAP ||

```

```

 deviceType == STAR_DEVICE_PXI_INTERFACE_MK2 ||
 deviceType == STAR_DEVICE_PXI_RMAP_MK2)
 {
 return TRIGGER_DEVICE_PXI_IF;
 }
 else if (deviceType == STAR_DEVICE_PXI_ROUTER_12 ||
 deviceType == STAR_DEVICE_PXI_ROUTER_MK2)
 {
 return TRIGGER_DEVICE_PXI_ROUTER;
 }
 else if (deviceType == STAR_DEVICE_PCIE)
 {
 return TRIGGER_DEVICE_PCIE_IF;
 }
 else if (deviceType == STAR_DEVICE_PCIE_MK2)
 {
 return TRIGGER_DEVICE_PCIE_MK2;
 }

 return TRIGGER_DEVICE_INVALID;
}

int main(int argc, char **argv)
{
 STAR_DEVICE_ID deviceId;
 MENU_CHOICE choice;
 U32 deviceType = STAR_DEVICE_UNKNOWN;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 /* Print the program header */
 printf("----- Trigger API Examples -----\\n");

 /* Select the device to use */
 deviceId = getTriggerDeviceID();

 /* Check that a device was selected */
 if (deviceId == 0)
 {
 printf("No devices were found.\\n");
 return 0;
 }

 /* Get type of device */
 deviceType = STAR_getDeviceType(deviceId);
 gTriggerDevice = getTriggerDeviceType(deviceType);

 /* Reset triggers */
 reset(deviceId);

 /* Loop menu */
 choice = MENU_CHOICE_INVALID;
 do
 {
 choice = getMenuChoice();
 switch (choice)
 {
 /* Time-code on external trigger (not supported on PCIe or
 PXI Router) */
 case MENU_CHOICE_TX_TC_ON_EXT_TRIGGER:
 txTcOnExtTrigger(deviceId);
 break;
 /* Generate external trigger on received time-code */
 case MENU_CHOICE_TX_EXT_TRIGGER_ON_TC:
 txExtTriggerOnTc(deviceId);
 break;
 /* Time-code on counter (not supported on PCIe) */
 case MENU_CHOICE_TX_TC_ON_COUNTER:
 txTcOnCounter(deviceId);
 break;
 /* Time-code on counter (not supported on PCIe) */
 case MENU_CHOICE_TX_EXT_TRIGGER_ON_COUNTER:
 txExtTriggerOnCounter(deviceId);
 break;
 /* Transmit packet on external trigger (not supported on PCIe or
 PXI Router) */
 case MENU_CHOICE_PKT_TX_ON_EXT_TRIGGER:
 pktTxOnExtTrigger(deviceId);
 break;
 /* Transmit packet on counter */
 case MENU_CHOICE_PKT_TX_ON_COUNTER:
 pktTxOnCounter(deviceId);
 break;
 /* Transmit packet on time-code (not supported on PCIe) */
 case MENU_CHOICE_PKT_TX_ON_TC:
 pktTxOnTc(deviceId);

```

```

 break;
 /* Transmit multiple packets on external trigger
 (not supported on PCIe or PXI Router) */
 case MENU_CHOICE_MULTIPLE_PKT_TX_ON_EXT_TRIGGER:
 multiplePktTxOnExtTrigger(deviceId);
 break;
 /* Transmit multiple packets on time-code (not supported on PCIe) */
 case MENU_CHOICE_MULTIPLE_PKT_TX_ON_TC:
 multiplePktTxOnTc(deviceId);
 break;
 /* Timed packet transmit on time-code (not supported on PCIe) */
 case MENU_CHOICE_TIMED_PKT_TX_ON_TC:
 timedPktTxOnTc(deviceId);
 break;
 /* Stop receiving for 5 seconds every 10 seconds (only
 supported on Brick Mk3) */
 case MENU_CHOICE_STOP_RECEIVING:
 stopReceiving(deviceId);
 break;
 /* Print trigger configuration */
 case MENU_CHOICE_PRINT_TRIGGER_CONF:
 printTriggerConf(deviceId);
 break;
 /* Reset trigger configuration */
 case MENU_CHOICE_RESET:
 resetTriggerConf(deviceId);
 break;

 /* Handle unused cases */
 case MENU_CHOICE_INVALID: break;
 case MENU_CHOICE_EXIT: break;
}
} while (choice != MENU_CHOICE_EXIT);

/* Reset triggers */
reset(deviceId);

return 0;
}

```

## 10.56 triggering\_generic\_example.c

This file contains example code demonstrating how to use the Generic Triggering API.

```

#include "utility.h"
#include "triggering_generic.h"
#include "cfg_api_generic.h"

typedef enum
{
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_TIME_CODE_TX = 1,
 MENU_CHOICE_TIME_CODE_RX_CAUSES_EXT_TRIGGER_PULSE,
 MENU_CHOICE_PERIODIC_EXT_TRIGGER_PULSE,
 MENU_CHOICE_PERIODIC_TIME_CODE_TX,
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_PKT_TX,
 MENU_CHOICE_PERIODIC_SPW_PORT_PKT_TX,
 MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_PKT_TX,
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_MULTIPLE_PKT_TX,
 MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_MULTIPLE_PKT_TX,
 MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_TIMED_PKT_TX,
 MENU_CHOICE_SPW_PORT_STOP_RECEIVING_EVERY_5S,
 MENU_CHOICE_PERIODIC_SPFI_VC_PKT_TX,
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPFI_PORT_DISCONNECT,
 MENU_CHOICE_PRINT_TRIGGER_CONF,
 MENU_CHOICE_RESET,
 MENU_CHOICE_EXIT,
 MENU_CHOICE_INVALID
} MENU_CHOICE;

static U32 resourceIsValid(const STAR_DEVICE_ID deviceId,
 const TRIGGER_TYPE resourceType, const U32 resourceId)
{
 U32 resourceCount = 0;
 U32 idOffset = 0;

 switch (resourceType)
 {
 case TRIGGER_TYPE_EXT_TRIGGER:
 TRIGGER_CFG_getNumberOfExtTriggers(deviceId, &resourceCount);
 break;
 }
}

```

```

 case TRIGGER_TYPE_COUNTER:
 TRIGGER_CFG_getNumberOfCounters(deviceId, &resourceCount);
 break;
 case TRIGGER_TYPE_SPW_PORT:
 TRIGGER_CFG_getNumberOfSpWPorts(deviceId, &resourceCount);
 /* SpW port index is offset by 1 */
 idOffset = 1;
 break;
 case TRIGGER_TYPE_TIME_CODE:
 TRIGGER_CFG_getNumberOfTimeCodeInterfaces(deviceId, &
 resourceCount);
 break;
 case TRIGGER_TYPE_TRIGGER:
 TRIGGER_CFG_getNumberOfTriggers(deviceId, &resourceCount);
 break;
 case TRIGGER_TYPE_SPFI_PORT:
 TRIGGER_CFG_getNumberOfSpFiPorts(deviceId, &resourceCount);
 /* SpFi port index is offset by 1 */
 idOffset = 1;
 break;
 case TRIGGER_TYPE_SPFI_VC:
 TRIGGER_CFG_getNumberOfSpFiVCs(deviceId, &resourceCount);
 break;
 default:
 return 0;
}

return (resourceId - idOffset) < resourceCount;
}

void extTriggerPulseCausesTimeCodeTx(const STAR_DEVICE_ID deviceId)
{
 const U32 extTriggerId = 0;
 const U32 tcInterfaceId = 0;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (External trigger pulse causes time-code "
 "transmit) ----- \n");

 /* Reset the device's triggering configuration */
 int resetSuccess = TRIGGER_CFG_reset(deviceId);

 if (resetSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Resetting the triggering configuration failed.\n");
 return;
 }

 /* Disable external trigger output (enables it as input) */
 int disableOutputSuccess = TRIGGER_CFG_disableExtTriggerOutput(
 deviceId,
 extTriggerId);

 if (disableOutputSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Disabling external trigger output failed.\n");
 return;
 }

 /* Enable external trigger edge detect mode */
 int enableEdgeDetectSuccess = TRIGGER_CFG_enableExtTriggerEdgeDetectMode(
 (
 deviceId, extTriggerId);

 if (enableEdgeDetectSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Enabling external trigger edge detect mode failed.\n");
 return;
 }

 int setEventSuccess = TRIGGER_CFG_setExtTriggerInputEvents(deviceId
 ,
 extTriggerId, 0, EXT_TRIGGER_EVENT_IN);

 if (setEventSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Setting the external trigger input events failed.\n");
 return;
 }

 int setActionSuccess = TRIGGER_CFG_setTimeCodeOutputActions(

```



```

 deviceId,
 tcInterfaceId, 0, TIME_CODE_ACTION_TX);

if (setActionSuccess != TRIGGER_CFG_SUCCESS)
{
 printf("Setting the time-code interface output actions failed.\n");
 return;
}

/* Enable internal trigger 0 */
int enableTriggerSuccess = TRIGGER_CFG_enableTrigger(deviceId, 0);

if (enableTriggerSuccess != TRIGGER_CFG_SUCCESS)
{
 printf("Enabling internal trigger 0 failed.\n");
 return;
}

printf("Time-codes will now be transmitted when external trigger %u is "
 "pulsed.\n\n", extTriggerId);
}

void timeCodeRxCausesExtTriggerPulse(const STAR_DEVICE_ID deviceId)
{
 const U32 extTriggerId = 0;
 const U32 tcInterfaceId = 0;
 const U32 extTriggerExtend = 2000;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Received time-code causes external "
 "trigger pulse) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 TRIGGER_CFG_setTimeCodeInputEvents(deviceId, tcInterfaceId, 0,
 TIME_CODE_EVENT_TICK);

 TRIGGER_CFG_setExtTriggerOutputActions(deviceId, extTriggerId, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Set the external trigger's extend value */
 TRIGGER_CFG_setExtTriggerExtend(deviceId, extTriggerId, extTriggerExtend
);

 /* Enable output on the external trigger */
 TRIGGER_CFG_enableExtTriggerOutput(deviceId, extTriggerId);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

 printf("A pulse on external trigger %u will now be generated on each "
 "received time-code.\n\n", extTriggerId);
}

void periodicTimeCodeTx(const STAR_DEVICE_ID deviceId)
{
 const U32 counterId = 0;
 const U32 tcInterfaceId = 0;
 U32 cyclesPerSecond = 0U;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Periodically transmit time-code) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Get the number of clock cycles per second for the device */
 TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

 /* Set the counter's reload value to 1 second */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId, cyclesPerSecond
);

 /* Enable auto-reload on the counter */

```

```

 TRIGGER_CFG_enableCounterAutoReload(deviceId, counterId);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 0,
 COUNTER_EVENT_RELOAD);

 TRIGGER_CFG_setTimeCodeOutputActions(deviceId, tcInterfaceId, 0,
 TIME_CODE_ACTION_TX);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

 printf("Time-codes will now be transmitted every second.\n\n");
}

void periodicExtTriggerPulse(const STAR_DEVICE_ID deviceId)
{
 const U32 extTriggerId = 0;
 const U32 counterId = 0;
 const U32 extendValue = 2000;
 U32 cyclesPerSecond = 0U;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Periodically output an external trigger "
 "pulse) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Get the number of clock cycles per second for the device */
 TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

 /* Set counter's reload value to 1 second */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId, cyclesPerSecond
);

 /* Enable auto-reload on the counter */
 TRIGGER_CFG_enableCounterAutoReload(deviceId, counterId);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 0,
 COUNTER_EVENT_RELOAD);

 TRIGGER_CFG_setExtTriggerOutputActions(deviceId, extTriggerId, 0,
 EXT_TRIGGER_ACTION_OUT);

 /* Set the external trigger's extend value */
 TRIGGER_CFG_setExtTriggerExtend(deviceId, extTriggerId, extendValue);

 /* Enable output on the external trigger */
 TRIGGER_CFG_enableExtTriggerOutput(deviceId, extTriggerId);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

 printf("External trigger %u will now output a pulse every second.\n\n",
 extTriggerId);
}

void extTriggerPulseCausesSpwPortPktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spwPortId = 1;
 const U32 extTriggerId = 0;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (External trigger pulse causes a packet "
 "transmit on a SpaceWire port) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Disable external trigger output (enables it as input) */
 TRIGGER_CFG_disableExtTriggerOutput(deviceId, extTriggerId);

 /* Enable external trigger edge detect mode */
 TRIGGER_CFG_enableExtTriggerEdgeDetectMode(deviceId,
 extTriggerId);
}

```

```

 TRIGGER_CFG_setExtTriggerInputEvents(deviceId, extTriggerId, 0,
 EXT_TRIGGER_EVENT_IN);

 if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
 {
 TRIGGER_CFG_enableSpWPortTransmitPacketMode(deviceId,
 spwPortId);
 }

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 0,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8, spwPortId);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

 printf("8 packets have been queued and will now be transmitted on "
 "SpaceWire port %u when external trigger %u is pulsed.\n\n", spwPortId,
 extTriggerId);
}

void periodicSpwPortPktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spwPortId = 1;
 const U32 counterId = 0;
 U32 cyclesPerSecond = 0;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Periodically transmit a packet from a "
 "SpaceWire port) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Get the number of clock cycles per second for the device */
 TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

 /* Set the counter's reload value to 1 second */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId, cyclesPerSecond
);

 /* Enable auto-reload on the counter */
 TRIGGER_CFG_enableCounterAutoReload(deviceId, counterId);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 0,
 COUNTER_EVENT_RELOAD);

 if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
 {
 TRIGGER_CFG_enableSpWPortTransmitPacketMode(deviceId,
 spwPortId);
 }

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 0,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, 8, spwPortId);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

 printf("8 packets have been queued and will now be transmitted on "
 "SpaceWire port %u every second.\n\n", spwPortId);
}

void timeCodeRxCausesSpwPortPktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spwPortId = 1;
 const U32 tcInterfaceId = 0;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }
}

```

```

printf("\n----- Trigger Example (Time-code reception causes a packet "
 "transmit from a SpaceWire port) -----\\n");

/* Reset the device's triggering configuration */
TRIGGER_CFG_reset(deviceId);

TRIGGER_CFG_setTimeCodeInputEvents(deviceId, tcInterfaceId, 0,
 TIME_CODE_EVENT_TICK);

if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
{
 TRIGGER_CFG_enableSpWPortTransmitPacketMode(deviceId,
 spwPortId);
}

TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 0,
 SPW_PORT_ACTION_TRANSMIT_PKT);

/* Queue up some packets */
queuePackets(deviceId, 8, spwPortId);

/* Enable internal trigger 0 */
TRIGGER_CFG_enableTrigger(deviceId, 0);

printf("8 packets have been queued and will now be transmitted on "
 "SpaceWire port %u when valid time-codes are received.\\n\\n", spwPortId);
}

void extTriggerPulseCausesSpwPortMultiplePktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spwPortId = 1;
 const U32 extTriggerId = 0;
 const U32 counterId = 0;
 const U32 numberPktsPerGroup = 3;
 const U32 numberOfGroups = 8;
 const U32 totalNumberOfPackets = numberPktsPerGroup * numberOfGroups;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId))
 {
 printf("\\n Test not supported by this device.\\n");
 return;
 }

 printf("\\n----- Trigger Example (External trigger pulse causes multiple "
 "packets to transmit from a SpaceWire port) -----\\n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 TRIGGER_CFG_enableCounterTriggerCount(deviceId, counterId);

 /* Set the counter's reload value to the number of packets required */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId,
 numberPktsPerGroup);

 /* Disable external trigger output (enables it as input) */
 TRIGGER_CFG_disableExtTriggerOutput(deviceId, extTriggerId);

 /* Enable edge detect mode on the external trigger */
 TRIGGER_CFG_enableExtTriggerEdgeDetectMode(deviceId,
 extTriggerId);

 if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
 {
 TRIGGER_CFG_enableSpWPortTransmitPacketMode(deviceId,
 spwPortId);
 }

 TRIGGER_CFG_setExtTriggerInputEvents(deviceId, extTriggerId, 0,
 EXT_TRIGGER_EVENT_IN);

 TRIGGER_CFG_setCounterOutputActions(deviceId, counterId, 0,
 COUNTER_ACTION_RELOAD);

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 0,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 TRIGGER_CFG_setSpWPortInputEvents(deviceId, spwPortId, 1,
 SPW_PORT_EVENT_TX_EOP);

 TRIGGER_CFG_setCounterOutputActions(deviceId, counterId, 1,
 COUNTER_ACTION_COUNT);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 2,
 COUNTER_EVENT_COUNT);
}

```

```

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 2,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, totalNumberOfPackets, spwPortId);

 /* Enable the internal triggers */
 TRIGGER_CFG_enableTrigger(deviceId, 0);
 TRIGGER_CFG_enableTrigger(deviceId, 1);
 TRIGGER_CFG_enableTrigger(deviceId, 2);

 printf("%u packets have been queued and will now be transmitted in groups "
 "of %u on SpaceWire port %u when external trigger %u is pulsed.\n\n",
 totalNumberOfPackets, numberPktsPerGroup, spwPortId, extTriggerId);
}

void timeCodeRxCausesSpwPortMultiplePktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spwPortId = 1;
 const U32 counterId = 0;
 const U32 tcInterfaceId = 0;
 const U32 numberPktsPerGroup = 3;
 const U32 numberOfGroups = 8;
 const U32 totalNumberOfPackets = numberPktsPerGroup * numberOfGroups;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Time-code reception causes multiple "
 "packets to transmit from a SpaceWire port) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 TRIGGER_CFG_enableCounterTriggerCount(deviceId, counterId);

 /* Set the counter's reload value to numberPktsPerGroup */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId,
 numberPktsPerGroup);

 if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
 {
 TRIGGER_CFG_enableSpWPortTransmitPacketMode(deviceId,
 spwPortId);
 }

 TRIGGER_CFG_setTimeCodeInputEvents(deviceId, tcInterfaceId, 0,
 TIME_CODE_EVENT_TICK);

 TRIGGER_CFG_setCounterOutputActions(deviceId, counterId, 0,
 COUNTER_ACTION_RELOAD);

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 0,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 TRIGGER_CFG_setSpWPortInputEvents(deviceId, spwPortId, 1,
 SPW_PORT_EVENT_TX_EOP);

 TRIGGER_CFG_setCounterOutputActions(deviceId, counterId, 1,
 COUNTER_ACTION_COUNT);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 2,
 COUNTER_EVENT_COUNT);

 TRIGGER_CFG_setSpWPortOutputActions(deviceId, spwPortId, 2,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 /* Queue up some packets */
 queuePackets(deviceId, totalNumberOfPackets, spwPortId);

 /* Enable the internal triggers */
 TRIGGER_CFG_enableTrigger(deviceId, 0);
 TRIGGER_CFG_enableTrigger(deviceId, 1);
 TRIGGER_CFG_enableTrigger(deviceId, 2);

 printf("%u packets have been queued and will now be transmitted in groups "
 "of %u on SpaceWire port %u when time-codes are received.\n\n",
 totalNumberOfPackets, numberPktsPerGroup, spwPortId);
}

void timeCodeRxCausesSpwPortTimedPktTx(const STAR_DEVICE_ID deviceId)

```

```

{
 const U32 tcInterfaceId = 0;
 const U32 spwPortId = 1;
 const U32 numberPktsPerGroup = 4;
 const U32 numberOfGroups = 8;
 const U32 totalNumberOfPackets = numberPktsPerGroup * numberOfGroups;
 U32 cyclesPerSecond = 0U;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_TIME_CODE, tcInterfaceId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Time-code reception causes timed packet "
 "transmission from a SpaceWire port) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Get the number of clock cycles per second for the device */
 TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

 TRIGGER_CFG_setCounterReloadValue(deviceId, 0, cyclesPerSecond);
 TRIGGER_CFG_enableCounterAutoReload(deviceId, 0);
 TRIGGER_CFG_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
 TRIGGER_CFG_setTimeCodeOutputActions(deviceId, tcInterfaceId, 0,
 TIME_CODE_ACTION_TX);
 TRIGGER_CFG_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_START);

 /* Setup counter 1 to expire every 200 ms and enable start/stop mode */
 TRIGGER_CFG_setCounterReloadValue(deviceId, 1, cyclesPerSecond / 5U);
 TRIGGER_CFG_enableCounterAutoReload(deviceId, 1);
 TRIGGER_CFG_enableCounterStartStopMode(deviceId, 1);

 /* Setup counter 1 to decrement counter 2 (packet counter) */
 TRIGGER_CFG_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_CFG_setCounterOutputActions(deviceId, 2, 1,
 COUNTER_ACTION_COUNT);

 TRIGGER_CFG_setCounterReloadValue(deviceId, 2, numberPktsPerGroup);
 TRIGGER_CFG_enableCounterAutoReload(deviceId, 2);
 TRIGGER_CFG_enableCounterTriggerCount(deviceId, 2);

 /* Setup counter 2 to transmit packets */
 TRIGGER_CFG_setCounterInputEvents(deviceId, 2, 2,
 COUNTER_EVENT_COUNT);
 TRIGGER_CFG_setSpwPortOutputActions(deviceId, spwPortId, 2,
 SPW_PORT_ACTION_TRANSMIT_PKT);

 /* Setup counter 2 to stop counter 1 when it hits 0 */
 TRIGGER_CFG_setCounterInputEvents(deviceId, 2, 3,
 COUNTER_EVENT_ZERO_SINGLE);
 TRIGGER_CFG_setCounterOutputActions(deviceId, 1, 3,
 COUNTER_ACTION_STOP);

 if (TRIGGER_DEVICE_PCIE_IF != gTriggerDevice)
 {
 TRIGGER_CFG_enableSpwPortTransmitPacketMode(deviceId,
 spwPortId);
 }

 /* Reload counters */
 TRIGGER_CFG_forceCounterReload(deviceId, 0);
 TRIGGER_CFG_forceCounterReload(deviceId, 1);
 TRIGGER_CFG_forceCounterReload(deviceId, 2);

 /* Enable internal triggers 0-3 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);
 TRIGGER_CFG_enableTrigger(deviceId, 1);
 TRIGGER_CFG_enableTrigger(deviceId, 2);
 TRIGGER_CFG_enableTrigger(deviceId, 3);

 queuePackets(deviceId, totalNumberOfPackets, spwPortId);

 printf("%u packets have been queued and will now be transmitted on "
 "SpaceWire port %u in groups of %u at 200 ms intervals when "
 "time-codes are transmitted.\n\n", totalNumberOfPackets, spwPortId,
 numberPktsPerGroup);
}

void spwPortStopReceivingEvery5s(const STAR_DEVICE_ID deviceId)
{

```

```

const U32 spwPortId = 2;
U32 cyclesPerSecond = 0U;

if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPW_PORT, spwPortId))
{
 printf("\n Test not supported by this device.\n");
 return;
}

printf("\n----- Trigger Example (Stop receiving on SpaceWire port For 5 "
 "seconds every 10 seconds) ----- \n");

/* Reset the device's triggering configuration */
TRIGGER_CFG_reset(deviceId);

/* Get the number of clock cycles per second for the device */
TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

/* Setup counter 0 to fire every 10 seconds and reload counter 1 */
TRIGGER_CFG_setCounterReloadValue(deviceId, 0, 10U * cyclesPerSecond);
TRIGGER_CFG_enableCounterAutoReload(deviceId, 0);
TRIGGER_CFG_setCounterInputEvents(deviceId, 0, 0,
 COUNTER_EVENT_RELOAD);
TRIGGER_CFG_setCounterOutputActions(deviceId, 1, 0,
 COUNTER_ACTION_RELOAD);

TRIGGER_CFG_setCounterReloadValue(deviceId, 1, 5U * cyclesPerSecond);
TRIGGER_CFG_disableCounterAutoReload(deviceId, 1);
TRIGGER_CFG_setCounterInputEvents(deviceId, 1, 1,
 COUNTER_EVENT_ZERO);
TRIGGER_CFG_setSpwPortOutputActions(deviceId, spwPortId, 1,
 SPW_PORT_ACTION_STOP_RECEPTION);

/* Setup internal trigger 1 to receive inverted input from counter 1 */
TRIGGER_CFG_setTriggerSourceEventsInverted(deviceId, 1,
 TRIGGER_TYPE_COUNTER, 1, 1);

/* Enable internal triggers 0-1 */
TRIGGER_CFG_enableTrigger(deviceId, 0);
TRIGGER_CFG_enableTrigger(deviceId, 1);
printf("Reception on SpaceWire port %u will now be stopped for 5 seconds "
 "every 10 seconds.\n\n", spwPortId);
}

void periodicSpFiVcPktTx(const STAR_DEVICE_ID deviceId)
{
 const U32 spfiPortId = 1;
 const U32 spfiVcId = 1;
 const U32 counterId = 0;
 U32 cyclesPerSecond = 0U;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPFI_PORT, spfiPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_SPFI_VC, spfiVcId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_COUNTER, counterId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (Periodically transmit a packet from a "
 "SpaceFibre virtual channel) ----- \n");

 /* Reset the device's triggering configuration */
 TRIGGER_CFG_reset(deviceId);

 /* Get the number of clock cycles per second for the device */
 TRIGGER_CFG_getDeviceClockFreq(deviceId, &cyclesPerSecond);

 TRIGGER_CFG_setCounterInputEvents(deviceId, counterId, 0,
 COUNTER_EVENT_RELOAD);

 TRIGGER_CFG_setSpFiVcOutputActions(deviceId, spfiPortId, spfiVcId, 0,
 SPFI_VC_ACTION_TRANSMIT_PKT);

 /* Set the counter's reload value to 1 second */
 TRIGGER_CFG_setCounterReloadValue(deviceId, counterId, cyclesPerSecond
);

 /* Enable auto-reload on the counter */
 TRIGGER_CFG_enableCounterAutoReload(deviceId, counterId);

 /* Enable transmit packet mode on the SpaceFibre virtual channel */
 TRIGGER_CFG_enableSpFiVcTransmitPacketMode(deviceId,
 spfiPortId, spfiVcId);

 /* Enable internal trigger 0 */
 TRIGGER_CFG_enableTrigger(deviceId, 0);

```

```

 printf("Triggering configuration sent to device. Once packets have been "
 "queued and the link has been started, packets will periodically be "
 "transmitted over SpFi P%u VC%u.\n\n", spfiPortId, spfiVcId);
 }

void extTriggerPulseCausesSpfiPortDisconnect(const STAR_DEVICE_ID deviceId)
{
 const U32 spfiPortId = 1;
 const U32 extTriggerId = 0;

 if (!resourceIsValid(deviceId, TRIGGER_TYPE_SPFI_PORT, spfiPortId) ||
 !resourceIsValid(deviceId, TRIGGER_TYPE_EXT_TRIGGER, extTriggerId))
 {
 printf("\n Test not supported by this device.\n");
 return;
 }

 printf("\n----- Trigger Example (External trigger pulse causes a "
 "SpaceFibre port disconnect) ----- \n");

 /* Reset the device's triggering configuration */
 int resetSuccess = TRIGGER_CFG_reset(deviceId);

 if (resetSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Resetting the triggering configuration failed.\n");
 return;
 }

 int setEventSuccess = TRIGGER_CFG_setExtTriggerInputEvents(deviceId
 , extTriggerId, 0, EXT_TRIGGER_EVENT_IN);

 if (setEventSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Setting the external trigger input event failed.\n");
 return;
 }

 int setActionSuccess = TRIGGER_CFG_setSpfiPortOutputActions(
 deviceId,
 spfiPortId, 0, SPFI_PORT_ACTION_DISCONNECT);

 if (setActionSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Setting the SpFi port output action failed.\n");
 return;
 }

 /* Disable external trigger output (enables it as input) */
 int disableOutputSuccess = TRIGGER_CFG_disableExtTriggerOutput(
 deviceId,
 extTriggerId);

 if (disableOutputSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Disabling external trigger output failed.\n");
 return;
 }

 /* Enable external trigger edge detect mode */
 int enableEdgeDetectSuccess = TRIGGER_CFG_enableExtTriggerEdgeDetectMode(
 (deviceId, extTriggerId);

 if (enableEdgeDetectSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Enabling external trigger edge detect mode failed.\n");
 return;
 }

 /* Enable internal trigger 0 */
 int enableTriggerSuccess = TRIGGER_CFG_enableTrigger(deviceId, 0);

 if (enableTriggerSuccess != TRIGGER_CFG_SUCCESS)
 {
 printf("Enabling the internal trigger failed.\n");
 return;
 }

 printf("Triggering configuration sent to device. Once the link has "
 "been started, an input pulse on external trigger %u will cause "
 "SpaceFibre port %u to disconnect.\n\n", extTriggerId, spfiPortId);
}

void resetConfiguration(const STAR_DEVICE_ID deviceId)

```



```

{
 TRIGGER_CFG_reset(deviceId);
 printf("\n----- Trigger Example (Reset triggers) -----\\n");
 printf("Triggers have been reset.\\n\\n");
}

MENU_CHOICE getMenuChoice(void)
{
 char buffer[32] = {0};
 unsigned int choice = 0;

 /* Show menu to user */
 printf("Please select example to run:\\n");
 printf("%d. External trigger pulse causes time-code transmit.\\n",
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_TIME_CODE_TX);
 printf("%d. Received time-code causes external trigger pulse.\\n",
 MENU_CHOICE_TIME_CODE_RX_CAUSES_EXT_TRIGGER_PULSE);
 printf("%d. Periodically output an external trigger pulse.\\n",
 MENU_CHOICE_PERIODIC_EXT_TRIGGER_PULSE);
 printf("%d. Periodically transmit a time-code.\\n",
 MENU_CHOICE_PERIODIC_TIME_CODE_TX);
 printf("%d. External trigger pulse causes a SpaceWire port to transmit a "
 "packet.\\n", MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_PKT_TX);
 printf("%d. Periodically transmit a packet from a SpaceWire port.\\n",
 MENU_CHOICE_PERIODIC_SPW_PORT_PKT_TX);
 printf("%d. Received time-code causes a SpaceWire port to transmit a "
 "packet.\\n", MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_PKT_TX);
 printf("%d. External trigger pulse causes a SpaceWire port to transmit "
 "multiple packets.\\n",
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_MULTIPLE_PKT_TX);
 printf("%d. Received time-code causes a SpaceWire port to transmit "
 "multiple packets.\\n",
 MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_MULTIPLE_PKT_TX);
 printf("%d. Received time-code causes a SpaceWire port to transmit packets "
 "at intervals.\\n",
 MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_TIMED_PKT_TX);
 printf("%d. Stop receiving on a SpaceWire port for 5 seconds every 10 "
 "seconds.\\n", MENU_CHOICE_SPW_PORT_STOP_RECEIVING_EVERY_5S);
 printf("%d. Periodically transmit a packet from a SpaceFibre virtual "
 "channel.\\n", MENU_CHOICE_PERIODIC_SPFI_VC_PKT_TX);
 printf("%d. External trigger pulse causes a SpaceFibre port disconnect.\\n",
 MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPFI_PORT_DISCONNECT);
 printf("%d. Print trigger configuration.\\n",
 MENU_CHOICE_PRINT_TRIGGER_CONF);
 printf("%d. Reset triggers.\\n", MENU_CHOICE_RESET);
 printf("%d. Exit\\n", MENU_CHOICE_EXIT);
 printf("> ");

 /* Validate chosen number */
 if (!fgets(buffer, 32, stdin))
 {
 return MENU_CHOICE_INVALID;
 }

 /* Get menu choice value */
 if (!sscanf(buffer, "%u", &choice) ||
 (choice >= MENU_CHOICE_INVALID))
 {
 return MENU_CHOICE_INVALID;
 }

 /* Return the chosen value */
 return choice;
}

STAR_DEVICE_ID getTriggerDeviceID()
{
 char* pDeviceName = 0;
 char* pDeviceSerial = 0;
 char buffer[256] = {0};
 unsigned int chosen;
 int status;

 /* List devices that support triggering API */
 U32 deviceTypes[12] = { STAR_DEVICE_BRICK_MK3,
 STAR_DEVICE_BRICK_MK4,
 STAR_DEVICE_GBE_BRICK,
 STAR_DEVICE_PXI_INTERFACE,
 STAR_DEVICE_PXI_INTERFACE_MK2,
 STAR_DEVICE_PXI_RMAP,
 STAR_DEVICE_PXI_RMAP_MK2,
 STAR_DEVICE_PXI_ROUTER_12,
 STAR_DEVICE_PXI_ROUTER_MK2,
 STAR_DEVICE_PCIE,
 STAR_DEVICE_PCIE_MK2,
 STAR_DEVICE_STAR_ULTRA_PCIE_INTERFACE };

```

```

/* Get available devices that support triggering */
U32 deviceCount = 0;
STAR_DEVICE_ID* pDevices = STAR_getDeviceListForTypes(
 deviceTypes,
 sizeof(deviceTypes) / sizeof(deviceTypes[0]), &deviceCount);
STAR_DEVICE_ID selectedDeviceID = STAR_DEVICE_UNKNOWN;

/* If more than one device is available */
if (deviceCount > 1U)
{
 U32 deviceIndex = 0;

 printf("The following devices are available that support triggering:"
 "\n\n");

 /* For each device */
 for (deviceIndex = 0U; deviceIndex < deviceCount; deviceIndex++)
 {
 /* Display its name */
 if (STAR_DEVICE_UNKNOWN != pDevices[deviceIndex])
 {
 pDeviceName = STAR_getDeviceName(pDevices[deviceIndex]);
 pDeviceSerial = STAR_getDeviceSerialNumber(
 pDevices[deviceIndex]);
 if ((NULL != pDeviceName) && (NULL != pDeviceSerial))
 {
 printf("\t%u - %s with serial number %s\n", deviceIndex,
 pDeviceName, pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", deviceIndex);
 }

 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
 }
 else
 {
 printf("\t%u - Unable to access device\n", deviceIndex);
 }
 }

 printf("\n");

 /* Ask the user which device to use */
 do
 {
 printf("Please select which device to use: ");
 fflush(stdout);

 /* Read selected device number */
 if (fgets(buffer, 256, stdin) == NULL)
 {
 puts("No device number selected.");

 continue;
 }
 status = sscanf(buffer, "%u", &chosen);
 if ((0 == status) || (chosen > (deviceCount - 1U)))
 {
 puts("Invalid device number selected.");

 continue;
 }

 /* Get the selected device id */
 selectedDeviceID = pDevices[chosen];
 } while (STAR_DEVICE_UNKNOWN == selectedDeviceID);
}
/* Else if just one device is available */
else if (1U == deviceCount)
{
 /* Select the first device id */
 selectedDeviceID = pDevices[0];
}
else
{
 /* Nothing to do */
}

/* If valid device was selected */
if (selectedDeviceID != STAR_DEVICE_UNKNOWN)
{
 /* Get device name and serial */
 pDeviceName = STAR_getDeviceName(selectedDeviceID);
 pDeviceSerial = STAR_getDeviceSerialNumber(selectedDeviceID);
}

```

```

 if (pDeviceName && pDeviceSerial)
 {
 /* Print selected device */
 printf("%s selected with serial number %s\n\n", pDeviceName,
 pDeviceSerial);
 }

 /* Destroy device and serial */
 STAR_destroyString(pDeviceName);
 STAR_destroyString(pDeviceSerial);
 }

 STAR_destroyDeviceList(pDevices);

 /* Return the selected device id */
 return selectedDeviceID;
}

TRIGGER_DEVICE getTriggerDeviceType(const U32 deviceType)
{
 if (STAR_DEVICE_BRICK_MK3 == deviceType ||
 STAR_DEVICE_BRICK_MK4 == deviceType ||
 STAR_DEVICE_GBE_BRICK == deviceType)
 {
 return TRIGGER_DEVICE_BRICK_MK3;
 }
 else if (STAR_DEVICE_PXI_INTERFACE == deviceType ||
 STAR_DEVICE_PXI_RMAP == deviceType ||
 STAR_DEVICE_PXI_INTERFACE_MK2 == deviceType ||
 STAR_DEVICE_PXI_RMAP_MK2 == deviceType)
 {
 return TRIGGER_DEVICE_PXI_IF;
 }
 else if (STAR_DEVICE_PXI_ROUTER_12 == deviceType ||
 STAR_DEVICE_PXI_ROUTER_MK2 == deviceType)
 {
 return TRIGGER_DEVICE_PXI_ROUTER;
 }
 else if (STAR_DEVICE_PCIE == deviceType)
 {
 return TRIGGER_DEVICE_PCIE_IF;
 }
 else if (STAR_DEVICE_PCIE_MK2 == deviceType)
 {
 return TRIGGER_DEVICE_PCIE_MK2;
 }
 else
 {
 /* Do nothing */
 }

 return TRIGGER_DEVICE_INVALID;
}

int main(int argc, char** argv)
{
 STAR_DEVICE_ID deviceId = 0;
 MENU_CHOICE choice = MENU_CHOICE_INVALID;
 U32 deviceType = STAR_DEVICE_UNKNOWN;

 UNREFERENCED_PARAMETER(argc);
 UNREFERENCED_PARAMETER(argv);

 /* Print the program header */
 printf("----- Trigger API Examples -----\\n");

 /* Select the device to use */
 deviceId = getTriggerDeviceID();

 /* Check that a device was selected */
 if (0U == deviceId)
 {
 printf("No devices were found.\\n");
 return 0;
 }

 /* Get type of device */
 deviceType = STAR_getDeviceType(deviceId);
 gTriggerDevice = getTriggerDeviceType(deviceType);

 /* Reset triggers */
 TRIGGER_CFG_reset(deviceId);

 /* Loop menu */
 do
 {

```

```

 choice = getMenuChoice();
 switch (choice)
 {
 case MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_TIME_CODE_TX:
 extTriggerPulseCausesTimeCodeTx(deviceId);
 break;
 case MENU_CHOICE_TIME_CODE_RX_CAUSES_EXT_TRIGGER_PULSE:
 timeCodeRxCausesExtTriggerPulse(deviceId);
 break;
 case MENU_CHOICE_PERIODIC_TIME_CODE_TX:
 periodicTimeCodeTx(deviceId);
 break;
 case MENU_CHOICE_PERIODIC_EXT_TRIGGER_PULSE:
 periodicExtTriggerPulse(deviceId);
 break;
 case MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_PKT_TX:
 extTriggerPulseCausesSpwPortPktTx(deviceId);
 break;
 case MENU_CHOICE_PERIODIC_SPW_PORT_PKT_TX:
 periodicSpwPortPktTx(deviceId);
 break;
 case MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_PKT_TX:
 timeCodeRxCausesSpwPortPktTx(deviceId);
 break;
 case MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPW_PORT_MULTIPLE_PKT_TX:
 extTriggerPulseCausesSpwPortMultiplePktTx(deviceId);
 break;
 case MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_MULTIPLE_PKT_TX:
 timeCodeRxCausesSpwPortMultiplePktTx(deviceId);
 break;
 case MENU_CHOICE_TIME_CODE_RX_CAUSES_SPW_PORT_TIMED_PKT_TX:
 timeCodeRxCausesSpwPortTimedPktTx(deviceId);
 break;
 case MENU_CHOICE_SPW_PORT_STOP_RECEIVING_EVERY_5S:
 spwPortStopReceivingEvery5s(deviceId);
 break;
 case MENU_CHOICE_PERIODIC_SPFI_VC_PKT_TX:
 periodicSpFiVcPktTx(deviceId);
 break;
 case MENU_CHOICE_EXT_TRIGGER_PULSE_CAUSES_SPFI_PORT_DISCONNECT:
 extTriggerPulseCausesSpfiPortDisconnect(deviceId);
 break;
 case MENU_CHOICE_PRINT_TRIGGER_CONF:
 printConfiguration(deviceId);
 break;
 case MENU_CHOICE_RESET:
 resetConfiguration(deviceId);
 break;
 /* Handle unused cases */
 case MENU_CHOICE_INVALID: break;
 case MENU_CHOICE_EXIT: break;
 default: break;
 }
} while (choice != MENU_CHOICE_EXIT);

/* Reset triggers */
TRIGGER_CFG_reset(deviceId);

return 0;
}

```

## 10.57 ui.c

This file contains UI functions that are common across the example applications.

```

#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include "star-api.h"

STAR_DEVICE_ID STAR_UI_chooseDevice(STAR_DEVICE_TYPE aDeviceTypes[],
 U32 NumRequestedTypes)
{
 STAR_DEVICE_ID* devices;
 U32 deviceCount = 0, i;
 unsigned int chosen;
 int status;

 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

```

```

devices = STAR_getDeviceListForTypes(aDeviceTypes, NumRequestedTypes, &
 deviceCount);

/* If there are no devices present */
if (!devices)
{
 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
}

/* Display the devices detected */
if (deviceCount == 1)
{
 printf("One device detected:");
}
else
{
 printf("%u devices detected:\n", deviceCount);
}

/* For each device */
for (i = 0; i < deviceCount; i++)
{
 /* Display its name */
 if (devices[i])
 {
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }
 else
 {
 printf("\t%u - Unable to access device\n", i);
 }
}

/* If there's only 1 device on the list */
if (deviceCount == 1)
{
 /* Return that device */
 deviceId = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceId;
}

/* Ask the user which device to use */
printf("Please select which device to use: ");
if (!fgets(s, 256, stdin))
{
 puts("No device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}
status = sscanf(s, "%u", &chosen);
if ((!status) || (chosen > deviceCount - 1))
{
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
}

/* Return the chosen device */
deviceId = devices[chosen];
STAR_destroyDeviceList(devices);
return deviceId;
}

unsigned char STAR_UI_chooseChannel(STAR_DEVICE_ID deviceId)
{
 STAR_CHANNEL_MASK channelMask;
 unsigned int channelNumber;
 unsigned char channelCount;
 int status;
 unsigned char i;
 char s[256];

 /* Get the channels present on the device */
 channelMask = STAR_getDeviceChannels(deviceId);

```

```

 if (channelMask == 0 || channelMask == 1)
 {
 printf("ERROR: The device doesn't appear to have any valid channels.");
 return 0;
 }

 /* Determine the number of channels on the device, */
 /* ignoring the configuration channel */
 channelCount = 0;
 channelNumber = 0;
 for (i = 1; i < 32; i++)
 {
 if ((channelMask >> i) & 1)
 {
 channelCount++;
 channelNumber = i;
 }
 }

 /* If there's only one channel on the device, use this channel */
 if (channelCount == 1)
 {
 printf("Using channel %u as the channel, as this is the only channel on the device.",
 channelNumber);
 return (unsigned char)channelNumber;
 }

 /* Display the available channels */
 printf("Enter channel (");
 for (i = 1; i < 32; i++)
 {
 /* If the channel exists */
 if ((channelMask >> i) & 1)
 {
 printf("%d", i);
 channelCount--;
 if (channelCount == 1)
 {
 printf(" or ");
 }
 else if (channelCount > 1)
 {
 printf(", ");
 }
 else
 {
 break;
 }
 }
 }
 printf("): ");

 /* Read in the channel to use */
 if (!fgets(s, 256, stdin))
 {
 puts("\nERROR: No channel specified");
 return 0;
 }
 status = sscanf(s, "%u", &channelNumber);
 if ((!status) || (!(1 << channelNumber) & channelMask))
 {
 puts("\nERROR: The channel specified is not present");
 return 0;
 }

 return (unsigned char)channelNumber;
}

int STAR_UI_chooseDeviceAndChannel(STAR_DEVICE_TYPE aDeviceTypes[],
 U32 NumRequestedTypes,
 STAR_DEVICE_ID *pDeviceId,
 STAR_CHANNEL_ID *pChannelId,
 STAR_CHANNEL_DIRECTION direction)
{
 unsigned char channelNumber;

 *pDeviceId = STAR_UI_chooseDevice(aDeviceTypes, NumRequestedTypes);
 if (!*pDeviceId)
 {
 return 0;
 }

 channelNumber = STAR_UI_chooseChannel(*pDeviceId);
 if (!channelNumber)
 {
 return 0;
 }
}

```

```

/* Open the channel */
*pChannelId = STAR_openChannelToLocalDevice(*pDeviceId, direction,
 channelNumber, TRUE);
if (!(*pChannelId))
{
 printf("\nERROR: Unable to open channel\n");
 return 0;
}

puts("");
return 1;
}

STAR_SPACEWIRE_ADDRESS *STAR_UI_getAddress()
{
 char s[256];
 STAR_SPACEWIRE_ADDRESS *pAddress;
 unsigned char newPath[256];
 U16 pathLen = 0;
 char *pos = (char *)s;

 /* Ask the user to enter the path to add to the front of packets */
 puts("Enter SpaceWire Address:");
 puts("(Values should be in hex, separated by a space, i.e.: 01 0f 02)");

 /* Read in the path */
 if (!fgets(s, 256, stdin))
 {
 puts("No address entered");
 return NULL;
 }

 /* Read until end of buffer or line feed */
 while ((*pos) && (pathLen < 256) && (*pos != '\n'))
 {
 unsigned long value;

 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno)
 {
 puts("Invalid value entered in address");
 return NULL;
 }

 newPath[pathLen] = (unsigned char)value;
 pathLen++;
 }

 /* Create a SpaceWire address from the path */
 pAddress = STAR_createAddress(newPath, pathLen);

 /* Return the completed address */
 return pAddress;
}

```

## 10.58 user\_registers\_example.c

This example demonstrates how to use the user configurable device registers.

```

#include <stdio.h>
#include "star-api.h"
#include "cfg_api_generic.h"

STAR_DEVICE_ID chooseStarDevice()
{
 STAR_DEVICE_ID *devices;
 unsigned int devCount = 0, chosen, i, status;
 STAR_DEVICE_ID deviceId;
 char *deviceName, s[256];

 /* Get the list of devices present which can be configured */
 devices = STAR_getDeviceListForType(
 STAR_DEVICE_CONFIG_SUPPORTED,
 &devCount);

 /* If there was an error getting the list of devices */
 if (!devices)
 {

```

```

 /* Return an error */
 puts("No SpaceWire devices detected!");
 return 0;
 }

 /* Display the devices detected */
 if (devCount == 1)
 {
 puts("One SpaceWire device detected:");
 }
 else
 {
 printf("%u SpaceWire devices detected:", devCount);
 }

 /* For each device */
 for (i = 0; i < devCount; i++)
 {
 /* Display its name */
 deviceName = STAR_getDeviceName(devices[i]);
 if (deviceName)
 {
 printf("\t%u - %s\n", i, deviceName);
 STAR_destroyString(deviceName);
 }
 else
 {
 printf("\t%u - Unknown SpaceWire Device\n", i);
 }
 }

 /* If there's only 1 device on the list */
 if (devCount == 1)
 {
 /* Return that device */
 deviceID = devices[0];
 STAR_destroyDeviceList(devices);
 return deviceID;
 }

 /* Ask the user which device to use */
 printf("\nPlease select which device to open: ");
 fgets(s, 256, stdin);
 status = sscanf(s, "%d", &chosen);
 if ((!status) || (chosen > devCount - 1))
 {
 puts("Incorrect device number selected.");
 STAR_destroyDeviceList(devices);
 return 0;
 }

 /* Return the chosen device */
 deviceID = devices[chosen];
 STAR_destroyDeviceList(devices);
 return deviceID;
}

int __cdecl main()
{
 STAR_DEVICE_ID deviceID;
 REGISTER value;

 /* Select device to configure */
 deviceID = chooseStarDevice();

 /* Set general purpose register to 0xABCD */
 CFG_setGeneralPurpose(deviceID, 0xABCD);

 /* Set device identity register to 0xCAFE */
 CFG_setNetworkIdentity(deviceID, 0xCAFE);

 /* Display user register values*/
 CFG_getGeneralPurpose(deviceID, &value);
 printf("General Purpose register value: %x", value);
 CFG_getNetworkIdentity(deviceID, &value);
 printf("\nNetwork Identity register value: %x\n", value);

 return 0;
}

```

## 10.59 utilities.c

This file provides common utilities used throughout the `api_test_app` example project.



```

#include "utilities.h"

#include "star-api.h"
#include "star-dundee_types.h"

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

int readBytes(unsigned char * bytes, unsigned int maximumBytesLength,
 U16 * actualBytesLength)
{
 U16 bytesRead = 0U;
 unsigned long value;
 char currentChar;
 char * pos;

 /* Allocate string for input */
 char * input = (char *)calloc(maximumBytesLength, sizeof(char));

 /* Check that input was allocated successfully */
 if (input == NULL)
 {
 return 0;
 }

 /* Flush the input stream */
 while ((currentChar = getchar()) != '\n' && currentChar != EOF);

 /* Read in the path */
 if (fgets(input, maximumBytesLength, stdin) == NULL)
 {
 puts("No bytes entered.");
 free(input);
 return 0;
 }

 /* Get start of input */
 pos = (char *)input;

 /* Detect no bytes */
 if (*pos == '\n')
 {
 puts("No bytes entered.");
 free(input);
 return 0;
 }

 /* Read until end of buffer or line feed */
 while ((*pos != '\0') && (bytesRead < maximumBytesLength) && (*pos != '\n'))
 {
 errno = 0;
 value = strtoul(pos, &pos, 16);
 if (errno != 0)
 {
 puts("Invalid value entered in bytes.");
 free(input);
 return 0;
 }

 bytes[bytesRead] = (unsigned char)value;
 bytesRead++;
 }

 /* Free the input string */
 free(input);

 /* Update actual number of bytes that were read */
 *actualBytesLength = bytesRead;

 return 1;
}

STAR_DEVICE_ID promptForDevice(TRANSMIT_TYPE transmitType)
{
 /* The number of devices that were found */
 U32 deviceCount = 0;

 /* Get the list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = STAR_getDeviceList(&deviceCount);

 /* If at least one device is present */
 if(devices)
 {
 /* Initialise selected device index to -1 */

```

```

int selectedDeviceIndex = -1;

/* Define selected device */
STAR_DEVICE_ID selectedDevice;

/* Counter to iterate over device array */
int index;

/* Initialise transmit type string to null by default */
char * transmitTypeString = NULL;

/* Print header statement */
printf("The following SpaceWire devices were detected:\n");

/* For all devices found on system */
for(index = 0; index < (int)deviceCount; index++)
{
 /* Get current device */
 STAR_DEVICE_ID device = devices[index];

 /* Get current device name */
 char * deviceName = STAR_getDeviceName(device);

 /* Get current device serial number */
 char * deviceSerial = STAR_getDeviceSerialNumber(device);

 if ((deviceSerial != NULL) && (deviceName != NULL))
 {
 /* Print current device */
 printf("%d: %s (Serial Number: %s)\n", index + 1, deviceName,
 deviceSerial);
 }

 if (deviceName != NULL)
 {
 /* Destroy device name string */
 STAR_destroyString(deviceName);
 }

 if (deviceSerial)
 {
 /* Destroy device serial string */
 STAR_destroyString(deviceSerial);
 }
}

/* Switch on transmit type */
switch(transmitType)
{
 /* Case send */
 case TRANSMIT_TYPE_SEND:
 /* Set transmit type string to "send out of" */
 transmitTypeString = "send out of";

 break;
 /* Case receive */
 case TRANSMIT_TYPE_RECEIVE:
 /* Set transmit type string to "receive into" */
 transmitTypeString = "receive into";

 break;
}

/* If there is a transmit type string */
if(transmitTypeString)
{
 /* Ask which device to use with transmit type string */
 printf("\nPlease enter the number of the device that you would like"
 " to %s:\n", transmitTypeString);
}
else
{
 /* Ask which device to use */
 printf("\nPlease enter the number of the device that you would like"
 " use:\n");
}

/* Get user input */
(void)scanf("%d", &selectedDeviceIndex);

/* While invalid user input */
while(selectedDeviceIndex < 1 || selectedDeviceIndex > (int)deviceCount)
{
 /* Flush input buffer */
 fflush(stdin);

 /* Ask again */

```

```

 printf("Invalid device number. Please enter a number between 1 and"
 " %u.\n", deviceCount);

 /* Get user input */
 (void)scanf("%d", &selectedDeviceIndex);
 }

 /* Decrement selected device index */
 selectedDeviceIndex--;

 /* Get selected device */
 selectedDevice = devices[selectedDeviceIndex];

 /* Destroy device list */
 STAR_destroyDeviceList(devices);

 /* Return selected device */
 return selectedDevice;
}

/* Return 0 if no selected device */
return 0;
}

STAR_DEVICE_ID getFirstDevice()
{
 /* The number of devices that were found */
 unsigned int deviceCount = 0;

 /* Get the list of available SpaceWire devices */
 STAR_DEVICE_ID * devices = STAR_getDeviceList(&deviceCount);

 /* If at least one device is present */
 if(devices)
 {
 /* Get first device */
 STAR_DEVICE_ID firstDevice = devices[0];

 /* Destroy device list */
 STAR_destroyDriverList(devices);

 /* Return first device */
 return firstDevice;
 }

 /* Return 0 if no devices exist */
 return 0;
}

STAR_DEVICE_ID getFirstDeviceForType(STAR_DEVICE_TYPE deviceType)
{
 /* The number of devices that were found */
 unsigned int deviceCount = 0;

 /* Get the list of available devices of the specific type */
 STAR_DEVICE_ID * devices = STAR_getDeviceListForType(deviceType,
 &deviceCount);

 /* If at least one device is present */
 if(devices)
 {
 /* Get first device */
 STAR_DEVICE_ID firstDevice = devices[0];

 /* Destroy device list */
 STAR_destroyDriverList(devices);

 /* Return first device */
 return firstDevice;
 }

 /* Return 0 if no devices exist */
 return 0;
}

int isChannelValid(STAR_DEVICE_ID device, unsigned int channel)
{
 /* Get available channels */
 U32 channelMask = STAR_getDeviceChannels(device);

 /* Define counter for loop */
 unsigned int index;

 /* For all possible channels (apart from port 0) */
 for (index = 1; index < 32; index++)
 {
 /* If the channel exists */

```

```

 if ((channelMask >> index) & 1)
 {
 /* If this is the channel that we are looking for */
 if(index == channel)
 {
 /* Return true */
 return 1;
 }
 }

 /* Return false if channel is invalid */
 return 0;
 }

STAR_CHANNEL_ID promptForChannel(STAR_DEVICE_ID device, int *
 selectedChannelNumber, TRANSMIT_TYPE transmitType)
{
 /* Define selected channel id */
 STAR_CHANNEL_ID selectedChannel;

 /* Initialise transmit type string to null by default */
 char * transmitTypeString = NULL;

 /* Initialise channel direction to inout by default */
 STAR_CHANNEL_DIRECTION channelDirection =
 STAR_CHANNEL_DIRECTION_INOUT;

 /* Initialise selected channel number to -1 */
 * selectedChannelNumber = -1;

 /* Switch on transmit type */
 switch(transmitType)
 {
 /* Case send */
 case TRANSMIT_TYPE_SEND:
 /* Set transmit type string to "send out of" */
 transmitTypeString = "send out of";

 /* Set channel direction to out */
 channelDirection = STAR_CHANNEL_DIRECTION_OUT;

 break;
 /* Case receive */
 case TRANSMIT_TYPE_RECEIVE:
 /* Set transmit type string to "receive into" */
 transmitTypeString = "receive into";

 /* Set channel direction to in */
 channelDirection = STAR_CHANNEL_DIRECTION_IN;

 break;
 }

 /* If there is a transmit type string */
 if(transmitTypeString)
 {
 /* Ask user for channel number to use with transmit type string */
 printf("\nPlease enter a channel number between 1 and 31 to %s:\n",
 transmitTypeString);
 }
 else
 {
 /* Ask user for channel number to use */
 printf("\nPlease enter a channel number between 1 and 31 to use:\n");
 }

 /* Get user input */
 (void) scanf("%d", selectedChannelNumber);

 /* While invalid user input */
 while(!isChannelValid(device, * selectedChannelNumber))
 {
 /* Flush input buffer */
 fflush(stdin);

 /* Ask again */
 printf("Invalid channel number. Please enter a valid channel number "
 "between 1 and 31:\n");

 /* Get user input */
 (void) scanf("%d", selectedChannelNumber);
 }

 /* Open channel to device */
 selectedChannel = STAR_openChannelToLocalDevice(device, channelDirection,
 (unsigned char)* selectedChannelNumber, 1);
}

```

```

 /* Return selected channel */
 return selectedChannel;
}

void printPacketContents(unsigned char * packetContents,
 unsigned int receiveBufferLength)
{
 /* Counter for loop */
 unsigned int index;

 /* Print packet contents label */
 printf("Received packet of length %u bytes, contents:", receiveBufferLength);

 /* For all characters in receive buffer */
 for(index = 0; index < receiveBufferLength; index++)
 {
 /* Print current character with leading zero */
 printf(" %02x", packetContents[index]);
 }

 /* Print full stop */
 printf(".\n");
}

void printVersionInfo(STAR_VERSION_INFO * versionInfo)
{
 /* Get module name */
 char * moduleName = versionInfo->name;

 /* Get module author */
 char * moduleAuthor = versionInfo->author;

 /* Get major version number */
 U16 major = versionInfo->major;

 /* Get minor version number */
 U16 minor = versionInfo->minor;

 /* Get edit version number */
 U16 edit = versionInfo->edit;

 /* Get patch version number */
 U16 patch = versionInfo->patch;

 /* Print version information string */
 printf("%s %u.%u.%u.%u\n", moduleName, major, minor, edit, patch);

 /* If module has a name */
 if(strlen(moduleName) > 0)
 {
 /* Print module name */
 printf(" Module name: %s\n", moduleName);
 }

 /* If module has an author */
 if(strlen(moduleAuthor) > 0)
 {
 /* Print module author */
 printf(" Module author: %s\n", moduleAuthor);
 }

 /* Print module version */
 printf("Module version: v%u.%u", major, minor);

 /* If there is an edit version number */
 if (edit)
 {
 /* Print edit version number */
 printf("(%u)", edit);
 }

 /* If there is a patch version number */
 if (patch)
 {
 /* Print patch version number */
 printf("p%u", patch);
 }

 /* Take new line */
 printf("\n");
}

```

## 10.60 utility.c

This file contains utility functions used by the STAR-System Test program.

```
#include <stdio.h>
#include <stdlib.h>
#ifdef __vxworks
 #include <memLib.h>
#elif defined(__APPLE__)
#else
 #include <malloc.h>
#endif
#include <string.h>
#include <errno.h>

#include "utility.h"

/*****
/*
/* Generates a 16-bit random number
/*
/*
*****/
unsigned int random16(void)
{
 unsigned int value = rand();

 if ((unsigned int)rand() > (RAND_MAX / 2))
 {
 value += RAND_MAX;
 }

 return value;
}

/*****
/*
/* Generates a 32-bit random number
/*
/*
*****/
unsigned int random32(void)
{
 unsigned int value;

 value = random16();
 value *= 0x10000U;
 value += random16();

 return value;
}

/*****
/*
/* Plain format-less File read/write
/* Reads a file into a buffer
/*
/*
*****/
int file_read(const char fname[], char * const pBuffer,
 unsigned int * const byteSize)
{
 FILE *infile;
 unsigned int count;

 *byteSize = 0U;
 infile = fopen(fname, "rb");
 if (infile == NULL)
 {
 printf("file_read:Trouble opening file\n");
 return 0;
 }

 /* Cycle until end of file reached: */
 while (feof(infile) == 0)
 {
 /* Attempt to read in a chunk of bytes */
 count = fread(pBuffer + *byteSize, sizeof(char), 100U, infile);
 if (ferror(infile) != 0)
 {
 printf("Read error in file_read: After reading %u\n", *byteSize);
 fclose(infile);
 return 0;
 }
 }
}
```

```

 *byteSize += count;
 }

 fclose(infile);
 return 1;
}

/*****
/*
/* Plain format-less File read/write
/* Reads PART of a file into a buffer
/*
/*
/*****/
int file_read_chunk(const char fname[], char * const pBuffer, const long offset,
 const unsigned int size)
{
 FILE *infile;
 unsigned int count;

 infile = fopen(fname, "rb");
 if (infile == NULL)
 {
 printf("file_read:Trouble opening file\n");
 return 0;
 }

 /* Move file steam pointer to offset */
 fseek(infile,offset,SEEK_SET);

 /* Attempt to read in a chunk of bytes */
 count = (unsigned int)fread(pBuffer, sizeof(char), size, infile);

 if ((ferror(infile) != 0) || (count < size))
 {
 printf("ERROR trying to read file %s at byte point %ld.\n", fname, offset);
 fclose(infile);
 return 0;
 }

 fclose(infile);
 return 1;
}

/*****
/*
/* Receives a fixed size message.
/* Returns FALSE if data could not be read.
/*
/*
/*****/
int file_append_chunk(const unsigned char * const pData, const long dataSize,
 const char fname[])
{
 FILE *outfile;
 int returnValue = 1;

 /* Open the file to write to */
 outfile = fopen(fname, "ab");
 if (outfile == NULL)
 {
 printf("\nfile_append_chunk: Trouble opening file: %s\n", fname);
 return 0;
 }

 fwrite(pData, 1U, dataSize, outfile);

 if (ferror(outfile) != 0)
 {
 printf("\nWrite error in file_append_chunk.\n");
 returnValue = 0;
 }

 fclose(outfile);

 return returnValue;
}

/*****
/*
/* FillBuffer
/* Fills a buffer with different data types. (zeros, ones, random, count)
/*
/*
/*****/

```

```

/*****
void FillBufferRandomChar(char * const pBuffer, const unsigned int size,
 const int dataType)
{
 unsigned int n;
 switch (dataType)
 {
 case DATA_TYPE_0: /* Fill with zeros */
 memset(pBuffer, 0, size);
 break;

 case DATA_TYPE_1: /* Fill with ones */
 memset(pBuffer, 1, size);
 break;

 case DATA_TYPE_RANDOM: /* Fill with random values */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = (char) random32();
 }
 break;

 case DATA_TYPE_COUNT: /* Fill with simple count */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = (char)n;
 }
 break;

 case DATA_TYPE_NOT_COUNT: /* Fill with not of simple count */
 for (n = 0U; n < size; n++)
 {
 *(pBuffer + n) = 0xFF - (char)n;
 }
 break;
 }
}

/*****
/*
/* Buffer compare function: Used instead of memcmp() for debugging
/* Returns the number of errors found
/*
/*
/*****
unsigned long BufferCompareChar(const char * const pBuffer1,
 const char * const pBuffer2, const unsigned long size)
{
 unsigned int errorCount = 0U;
 unsigned long i;

 for (i = 0U; i < size; i++)
 {
 if (*(pBuffer1 + i) != *(pBuffer2 + i))
 {
 errorCount++;
 printf("Error in byte %8lu, should be 0x%02x actually 0x%02x\n", i,
 (unsigned char)*(pBuffer1 + i), (unsigned char)*(pBuffer2 + i));
 }
 }

 return errorCount;
}

void CopyNumberToMemory(void * const pBuffer, const U32 number,
 const unsigned long len)
{
 unsigned long i;

 for (i = 0U; i < len; i++)
 {
 ((U8 *)pBuffer)[i] = (U8)((number >> ((len - i) - 1U) * 8U) & 0xffU);
 }
}

void CopyNumberFromMemory(U32 * const pNumber, void * const pBuffer,
 const unsigned long len)
{
 unsigned long i;

 *pNumber = 0U;

```



```

 for (i = 0U; i < len; i++)
 {
 *pNumber = (*pNumber << 8U) + ((U8 *)pBuffer)[i];
 }
}

int SizeOfFile(const char filePath[])
{
 FILE *infile;
 int size;

 infile = fopen(filePath, "rb");
 if (infile == NULL)
 {
 printf("Error: Trouble obtaining file size\n");
 return 0;
 }

 fseek(infile, 0, SEEK_END); /* seek to end of file */
 size = ftell(infile); /* get current file pointer */
 fseek(infile, 0, SEEK_SET); /* seek back to beginning of file */

 fclose(infile);

 return size;
}

```

## 10.61 vendor\_specific\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Vendor Specific functions.

```

#include "cfg_api_spfi.h"

void disableRouterTimeoutExample(STAR_DEVICE_ID deviceID)
{
 /* Disable router timeout */
 if (CFG_SPFI_disableRouterTimeout(deviceID) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to disable router timeout.\n");
 return;
 }
 else
 {
 printf("Router timeout disabled.\n");
 }
}

void enableRouterTimeoutExample(STAR_DEVICE_ID deviceID)
{
 /* Enable router timeout */
 if (CFG_SPFI_enableRouterTimeout(deviceID) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to enable router timeout.\n");
 return;
 }
 else
 {
 printf("Router timeout enabled.\n");
 }
}

void getBroadcastTypesInformationExample(STAR_DEVICE_ID deviceID)
{
 SPFI_BROADCAST_TYPES_INFO broadcastTypesInfo = 0U;

 /* Get broadcast types information */
 if (CFG_SPFI_getBroadcastTypesInformation(deviceID, &
 broadcastTypesInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get broadcast types information.\n");
 }
}

```

```

 return;
 }
 else
 {
 /* Get broadcast type properties */
 U8 spwInterruptBroadcastType =
 CFG_SPFI_getSpaceWireInterruptBroadcastType(deviceID
);
 U8 timeCodeBroadcastType = CFG_SPFI_getTimeCodeBroadcastType(
 deviceID);
 U8 timeSlotBroadcastType = CFG_SPFI_getTimeSlotBroadcastType(
 deviceID);

 /* Print broadcast type properties */
 printf("SpaceWire Interrupt Broadcast Type : %u\n",
 spwInterruptBroadcastType);
 printf("Time-code Broadcast Type : %u\n",
 timeCodeBroadcastType);
 printf("Time-slot Broadcast Type : %u\n",
 timeSlotBroadcastType);
 }
}

void setSpaceWireInterruptBroadcastTypeExample(STAR_DEVICE_ID deviceID)
{
 U8 spwInterruptBroadcastType = 1U;

 /* Set SpaceWire interrupt broadcast type */
 if (CFG_SPFI_setSpaceWireInterruptBroadcastType(deviceID,
 spwInterruptBroadcastType) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set SpaceWire interrupt broadcast type.\n");
 return;
 }
 else
 {
 printf("SpaceWire interrupt broadcast type set to %u.\n",
 spwInterruptBroadcastType);
 }
}

void setTimeCodeBroadcastTypeExample(STAR_DEVICE_ID deviceID)
{
 U8 timeCodeBroadcastType = 2U;

 /* Set time-code broadcast type */
 if (CFG_SPFI_setTimeCodeBroadcastType(deviceID,
 timeCodeBroadcastType) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set time-code broadcast type.\n");
 return;
 }
 else
 {
 printf("Time-code broadcast type set to %u.\n",
 timeCodeBroadcastType);
 }
}

void setTimeSlotBroadcastTypeExample(STAR_DEVICE_ID deviceID)
{
 U8 timeSlotBroadcastType = 3U;

 /* Set time-slot broadcast type */
 if (CFG_SPFI_setTimeSlotBroadcastType(deviceID,
 timeSlotBroadcastType) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set time-slot broadcast type.\n");
 return;
 }
 else
 {
 printf("Time-slot broadcast type set to %u.\n",
 timeSlotBroadcastType);
 }
}

void getRouterTimeoutExample(STAR_DEVICE_ID deviceID)
{
 double routerTimeout = 0.0;
 double routerTimeoutMaximum = 0.0;
 double routerTimeoutResolution = 0.0;

 /* Get router timeout value */

```

```

if (CFG_SPFI_getRouterTimeout(deviceID, &routerTimeout) !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to get router timeout value.\n");
 return;
}
else
{
 printf("Router Timeout : %.2f microseconds\n",
 routerTimeout);
}

/* Get maximum router timeout value */
if (CFG_SPFI_getRouterTimeoutMaximum(deviceID, &routerTimeoutMaximum)
 !=
 CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to get maximum router timeout value.\n");
 return;
}
else
{
 printf("Maximum Router Timeout : %.2f microseconds\n",
 routerTimeout);
}

/* Get router timeout resolution value */
if (CFG_SPFI_getRouterTimeoutResolution(deviceID, &
 routerTimeoutResolution)
 != CFG_STATUS_SUCCESS)
{
 /* Print error and exit example */
 printf("Failed to get router timeout resolution value.\n");
 return;
}
else
{
 printf("Router Timeout Resolution : %.2f microseconds\n",
 routerTimeoutResolution);
}
}

void setRouterTimeoutExample(STAR_DEVICE_ID deviceID)
{
 double maximumRouterTimeout = 0.0;
 double routerTimeout = 999936.0;

 /* Get router timeout maximum */
 if (CFG_SPFI_getRouterTimeoutMaximum(deviceID, &maximumRouterTimeout)
 !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get maximum router timeout value.\n");
 return;
 }

 /* Validate router timeout */
 if (routerTimeout > maximumRouterTimeout)
 {
 /* Print error and exit example */
 printf("Router timeout value is too large to be set.\n");
 return;
 }

 /* Set router timeout value */
 if (CFG_SPFI_setRouterTimeout(deviceID, routerTimeout) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to set router timeout value.\n");
 return;
 }
 else
 {
 printf("Router timeout set to %.2f microseconds.\n", routerTimeout);
 }
}

void isRouterTimeoutEnabledExample(STAR_DEVICE_ID deviceID)
{
 U8 routerTimeoutEnabled = 0U;

 /* Get router timeout enabled value */
 if (CFG_SPFI_isRouterTimeoutEnabled(deviceID, &routerTimeoutEnabled) !=

```

```

 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printf("Failed to get router timeout enabled value.\n");
 return;
 }
 else
 {
 printf("Router Timeout Enabled : %s\n",
 routerTimeoutEnabled ? "yes" : "no");
 }
}

void vendorSpecificExample(STAR_DEVICE_ID deviceID)
{
 printf("Vendor Specific Example\n");
 printf("=====\n");

 /* Disable router timeout example */
 disableRouterTimeoutExample(deviceID);

 /* Enable router timeout example */
 enableRouterTimeoutExample(deviceID);

 /* Set SpaceWire interrupt broadcast type example */
 setSpaceWireInterruptBroadcastTypeExample(deviceID);

 /* Set time-code broadcast type example */
 setTimeCodeBroadcastTypeExample(deviceID);

 /* Set time-slot broadcast type example */
 setTimeSlotBroadcastTypeExample(deviceID);

 /* Set router timeout example */
 setRouterTimeoutExample(deviceID);

 printf("\n");

 /* Get broadcast types information example */
 getBroadcastTypesInformationExample(deviceID);

 /* Get router timeout example */
 getRouterTimeoutExample(deviceID);

 /* Is router timeout enabled example */
 isRouterTimeoutEnabledExample(deviceID);

 printf("\n");
}

```

## 10.62 version\_example.c

This is an example of how to obtain the version information of the RMAP Packet Library.

```

#include <stdio.h>

#include "rmap_packet_library.h"

void version_example()
{
 U32 version;
 U16 edit;
 U8 patch;

 version = RMAP_GetVersion();

 printf("RMAP Packet Library v%d.%02d", RMAP_GET_VERSION_MAJOR(version),
 RMAP_GET_VERSION_MINOR(version));
 edit = RMAP_GET_VERSION_EDIT(version);
 if (edit)
 {
 printf(" edit %d", edit);
 }
 patch = RMAP_GET_VERSION_PATCH(version);
 if (patch)
 {
 printf(" patch level %d", patch);
 }
 puts("");
}

```

```

}
```

## 10.63 virtual\_channel\_info\_example.c

This example demonstrates the use of the SpaceFibre Configuration API Virtual Channel Information functions.

```

#include "utility.h"
#include "cfg_api_spfi.h"

void getVCVirtualNetworkNumberExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Virtual Network Number Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre or AXI port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE &&
 portType != SPFI_PORT_TYPE_AXI)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 U8 vn = 0U;

 /* Get VC VN value */
 if (CFG_SPFI_getVCVN(deviceID, port, vc, &vn) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC VN value", port, vc);
 return;
 }
 else
 {
 printf("Port %u VC %u VN : %u\n", port, vc, vn);
 }
 }
 }
 }

 printf("\n");
}
```

```

 }
}

void setVCVirtualNetworkNumberExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Set VC Virtual Network Number Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre or AXI port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE &&
 portType != SPFI_PORT_TYPE_AXI)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 U8 vn;

 vn = vc;

 /* Set VC VN value */
 if (CFG_SPFI_setVCVN(deviceID, port, vc, vn) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to set VC VN value", port, vc);
 return;
 }
 else
 {
 printf("VC VN set to %u", vn);
 printVCMessage("", port, vc);
 }
 }

 printf("\n");
 }
 }
}

void getVCStatusExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Status Example\n");
 printf("-----\n");

```

```

/* Get the number of ports */
portCount = getPortCount(deviceID);
if (!portCount)
{
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
}

/* Loop through the ports */
for (port = 1U; port < portCount; port++)
{
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 SPFI_VC_STATUS vcStatus = 0U;

 /* Get VC status */
 if (CFG_SPFI_getVCStatus(deviceID, port, vc, &vcStatus) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC status", port, vc);
 return;
 }
 else
 {
 /* Get VC status values */
 U8 vcInputBufferFull = CFG_SPFI_isVCInputBufferFull(
 vcStatus);
 U8 vcInputBufferHalfFull =
 CFG_SPFI_isVCInputBufferHalfFull(
 vcStatus);
 U8 vcInputBufferNotEmpty =
 CFG_SPFI_isVCInputBufferNotEmpty(
 vcStatus);
 U8 vcOutputBufferFull = CFG_SPFI_isVCOutputBufferFull(
 vcStatus);
 U8 vcOutputBufferHalfFull =
 CFG_SPFI_isVCOutputBufferHalfFull(vcStatus);
 U8 vcOverused = CFG_SPFI_isVCOverused(vcStatus);
 U8 vcUnderused = CFG_SPFI_isVCUnderused(vcStatus);

 /* Print the VC status values */
 printf("Port %u VC %u Input Buffer Full : %s\n",
 port, vc, vcInputBufferFull ? "yes" : "no");
 printf("Port %u VC %u Input Buffer Half Full : %s\n",
 port, vc, vcInputBufferHalfFull ? "yes" : "no");
 printf("Port %u VC %u Input Buffer Not Empty : %s\n",
 port, vc, vcInputBufferNotEmpty ? "yes" : "no");
 printf("Port %u VC %u Output Buffer Full : %s\n",
 port, vc, vcOutputBufferFull ? "yes" : "no");
 printf("Port %u VC %u Output Buffer Half Full : %s\n",
 port, vc, vcOutputBufferHalfFull ? "yes" : "no");
 printf("Port %u VC %u Overused : %s\n",
 port, vc, vcOverused ? "yes" : "no");
 printf("Port %u VC %u Underused : %s\n",
 port, vc, vcUnderused ? "yes" : "no");
 }
 }
 }
}

```

```

 if (vc < (vcCount - 1U))
 {
 printf("\n");
 }
 }
}

printf("\n");
}

void getVCErrorsExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Errors Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Try to get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 SPFI_VC_ERRORS vcErrors = 0U;

 /* Get VC errors */
 if (CFG_SPFI_getVCErrors(deviceID, port, vc, &vcErrors) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC errors", port, vc);
 return;
 }
 else
 {
 /* Get VC error values */
 U8 vcAddressErrorDetected =
 CFG_SPFI_isVCAddressErrorDetected(vcErrors);
 U8 vcTimeoutDetected = CFG_SPFI_isVCTimeoutDetected(
 vcErrors);
 U8 vcVNEErrorDetected = CFG_SPFI_isVCVNEErrorDetected(
 vcErrors);

 /* Print the VC error values */
 printf("Port %u VC %u Address Error Detected : %s\n", port,
 vc, vcAddressErrorDetected ? "yes" : "no");
 printf("Port %u VC %u Timeout Detected : %s\n", port,

```



```

 vc, vcTimeoutDetected ? "yes" : "no");
 printf("Port %u VC %u VN Error Detected : %s\n", port,
 vc, vcVNErrordetected ? "yes" : "no");

 printf("\n");

 /* Clear VC errors */
 if (CFG_SPFI_clearVCErrors(deviceID, port, vc) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to clear VC errors", port, vc);
 return;
 }
 else
 {
 printVCMessage("VC errors cleared", port, vc);
 }

 if (vc < (vcCount - 1U))
 {
 printf("\n");
 }
}

}

printf("\n");
}

}

void getVCBandwidthReservationExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Bandwidth Reservation Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 SPFI_QUALITY_OF_SERVICE qos = 0U;

 /* Get VC quality of service */
 if (CFG_SPFI_getVCQualityOfService(deviceID, port, vc, &qos)
 !=

```

```

 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC quality of service", port,
 vc);
 return;
 }
 else
 {
 U8 bandwidth;

 /* Get VC bandwidth reservation */
 bandwidth = CFG_SPFI_getVCBandwidthReservation(qos);
 printf("Port %u VC %u Bandwidth : %u%%\n", port, vc,
 bandwidth);
 }
}

printf("\n");
}

void setVCBandwidthReservationExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;
 U8 bandwidth;

 printf("Set VC Bandwidth Reservation Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Use bandwidth of 5% */
 bandwidth = 5U;

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 /* Set VC bandwidth reservation */
 if (CFG_SPFI_setVCBandwidthReservation(deviceID, port, vc
 ,
 bandwidth) != CFG_STATUS_SUCCESS)
 {
 printVCMessage("Failed to set VC bandwidth value",
 port, vc);
 }
 else

```

```

 {
 printf("VC bandwidth set to %u%%", bandwidth);
 printVCMessage("", port, vc);
 }
 }

 printf("\n");
}

void getVCPriorityExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Priority Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 SPFI_QUALITY_OF_SERVICE qos = 0U;

 /* Get VC quality of service */
 if (CFG_SPFI_getVCQualityOfService(deviceID, port, vc, &qos)
 !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC quality of service", port,
 vc);
 return;
 }
 else
 {
 /* Get VC priority */
 U8 priority = CFG_SPFI_getVCPriority(qos);
 printf("Port %u VC %u Priority : %u\n", port, vc,
 priority);
 }
 }
 }

 printf("\n");
 }
}

```

```

void setVCPriorityExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;
 U8 priority;

 printf("Set VC Priority Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Use priority of 3 */
 priority = 3U;

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 /* Set VC priority */
 if (CFG_SPFI_setVCPriority(deviceID, port, vc, priority)
 != CFG_STATUS_SUCCESS)
 {
 printVCMessage("Failed to set VC priority value", port,
 vc);
 }
 else
 {
 printf("VC priority set to %u", priority);
 printVCMessage("", port, vc);
 }
 }

 printf("\n");
 }
 }
}

void disableVCContinuousModeExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Disable VC Continuous Mode Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {

```

```

 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
}

/* Loop through the ports */
for (port = 1U; port < portCount; port++)
{
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 /* Disable VC continuous mode */
 if (CFG_SPFI_disableVCContinuousMode(deviceID, port, vc) !=
 CFG_STATUS_SUCCESS)
 {
 printVCMessage("Failed to disable VC continuous mode",
 port, vc);
 }
 else
 {
 printf("Port %u VC %u continuous mode disabled.\n",
 port, vc);
 }
 }

 printf("\n");
 }
}

void enableVCContinuousModeExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Enable VC Continuous Mode Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 }
}

```

```

 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 /* Enable VC continuous mode */
 if (CFG_SPFI_enableVCContinuousMode(deviceID, port, vc) !=
 CFG_STATUS_SUCCESS)
 {
 printVCMessage("Failed to enable VC continuous mode",
 port, vc);
 }
 else
 {
 printf("Port %u VC %u continuous mode enabled.\n",
 port, vc);
 }
 }

 printf("\n");
 }
}

void isVCContinuousModeEnabledExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Is VC Continuous Mode Enabled Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */

```

```

 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 SPFI_QUALITY_OF_SERVICE qos = 0U;

 /* Get VC quality of service */
 if (CFG_SPFI_getVCQualityOfService(deviceID, port, vc, &qos)
 != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC quality of service", port,
 vc);
 return;
 }
 else
 {
 /* Get continuous mode enabled flag */
 U8 continuousModeEnabled =
 CFG_SPFI_isVCContinuousModeEnabled(qos);

 printf("Port %u, VC %u Continuous Mode Enabled : %s\n",
 port, vc, continuousModeEnabled ? "yes" : "no");
 }
 }

 printf("\n");
 }
}

void getVCTimeSlotsExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Get VC Time-slots Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 U64 timeSlots = 0ULL;

 /* Get VC time-slots */

```

```

 if (CFG_SPFI_getVCTimeSlots(deviceID, port, vc, &timeSlots) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC time-slots", port, vc);
 return;
 }
 else
 {
 printf("Port %u, VC %u Time-slots : 0x%llx\n", port, vc,
 timeSlots);
 }
 }
}

printf("\n");
}

void setVCTimeSlotsExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("Set VC Time-slots Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 /* Time-slots value to set */
 U64 timeSlots = 0x1111111122222222U;

 /* Set VC time-slots value */
 if (CFG_SPFI_setVCTimeSlots(deviceID, port, vc, timeSlots) !=
 CFG_STATUS_SUCCESS)
 {
 printVCMessage("Failed to set VC time-slots value", port,
 vc);
 }
 else
 {
 printf("VC time-slots set to 0x%llx", timeSlots);
 printVCMessage("", port, vc);
 }
 }
 }
 }
}

```



```

 printf("\n");
 }
}

void getVCDataRateExample(STAR_DEVICE_ID deviceID)
{
 U8 portCount;
 U8 port;

 printf("VC Data Rate Example\n");
 printf("-----\n");

 /* Get the number of ports */
 portCount = getPortCount(deviceID);
 if (!portCount)
 {
 /* Print error and exit example */
 printf("Failed to get port count.\n");
 return;
 }

 /* Loop through the ports */
 for (port = 1U; port < portCount; port++)
 {
 SPFI_PORT_INFO portInfo = 0U;

 /* Get port information */
 if (CFG_SPFI_getPortInformation(deviceID, port, &portInfo) !=
 CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printPortMessage("Failed to get port information", port);
 return;
 }
 else
 {
 U8 vcCount;
 U8 vc;

 /* Get port type */
 SPFI_PORT_TYPE portType = CFG_SPFI_getPortType(portInfo);

 /* If not a SpaceFibre port */
 if (portType != SPFI_PORT_TYPE_SPACEFIBRE)
 {
 /* Skip port */
 continue;
 }

 /* Get count of virtual channels associated with SpaceFibre port */
 vcCount = CFG_SPFI_getPortVCCount(portInfo);

 /* Loop through the virtual channels */
 for (vc = 0U; vc < vcCount; vc++)
 {
 U64 txDataRate = 0ULL;
 U64 rxDataRate = 0ULL;

 /* Get VC data rate */
 if (CFG_SPFI_getVCDataRate(deviceID, port, vc, &txDataRate,
 &rxDataRate) != CFG_STATUS_SUCCESS)
 {
 /* Print error and exit example */
 printVCMessage("Failed to get VC data rate", port,
 vc);
 return;
 }
 else
 {
 printf("Port %u VC %u Data Rate (Tx, Rx) : %llu bit/s, "
 "%llu bit/s\n", port, vc, txDataRate, rxDataRate);
 }
 }

 printf("\n");
 }
 }
}

void virtualChannelInfoExample(STAR_DEVICE_ID deviceID)
{
 printf("Virtual Channel Information Example\n");
 printf("=====\n");

 /* Set virtual channel virtual network number example */
 setVCVirtualNetworkNumberExample(deviceID);
}

```

```

/* Get virtual channel virtual network number example */
getVCVirtualNetworkNumberExample(deviceID);

/* Get virtual channel status example */
getVCStatusExample(deviceID);

/* Get virtual channel errors example */
getVCErrorsExample(deviceID);

/* Set virtual channel bandwidth reservation example */
setVCBandwidthReservationExample(deviceID);

/* Get virtual channel bandwidth reservation example */
getVCBandwidthReservationExample(deviceID);

/* Set virtual channel priority example */
setVCPriorityExample(deviceID);

/* Get virtual channel priority example */
getVCPriorityExample(deviceID);

/* Disable virtual channel continuous mode example */
disableVCContinuousModeExample(deviceID);

/* Enable virtual channel continuous mode example */
enableVCContinuousModeExample(deviceID);

/* Is virtual channel continuous mode enabled example */
isVCContinuousModeEnabledExample(deviceID);

/* Set virtual channel time-slots example */
setVCTimeSlotsExample(deviceID);

/* Get virtual channel time-slots example */
getVCTimeSlotsExample(deviceID);

/* Get virtual channel data rate example */
getVCDataRateExample(deviceID);
}

```

## 10.64 write\_command\_packet\_example.c

This is an example of how to build RMAP write commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

void write_command_packet_example()
{
 void *pFillPacket, *pBuildPacket, *pBuildPacketRegister;
 unsigned long fillPacketLenCalculated, fillPacketLen;
 unsigned long buildPacketLen, buildPacketRegisterLen;
 U8 pTarget[] = {0, 254};
 U8 pReply[] = {254};
 U8 pData[] = {0x12, 0x34, 0x56, 0x78};
 char status;

 /* Calculate the length of a write command packet, with 4 bytes of data */
 fillPacketLenCalculated = RMAP_CalculateWriteCommandPacketLength(
 2, 1, 4,
 1);
 printf("The write command packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the command packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillWriteCommandPacket(pTarget, 2, pReply, 1, 1, 1, 0, 0x20,
 0, 0x106, 0, pData, 4, &fillPacketLen, NULL, 1, (U8 *)pFillPacket,

```

```

 fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP write command packet to write 4-bytes */
 pBuildPacket = RMAP_BuildWriteCommandPacket(pTarget, 2, pReply, 1, 1, 1, 0,
 0x20, 0, 0x106, 0, pData, 4, &buildPacketLen, NULL, 1);
 if (!pBuildPacket)
 {
 puts("Couldn't build the write command packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP write command packet to write to a 4-byte register */
 pBuildPacketRegister = RMAP_BuildWriteRegisterPacket(pTarget, 2, pReply, 1,
 1, 1, 0, 0x20, 0, 0x106, 0, 0x12345678, &buildPacketRegisterLen, NULL,
 1);
 if (!pBuildPacketRegister)
 {
 puts("Couldn't build the write register packet");
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
 return;
 }

 /* Check the lengths are all the same */
 if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen) ||
 (buildPacketLen != buildPacketRegisterLen))
 {
 puts("The lengths are different:");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 printf("\tLength of built register packet: %ld\n", buildPacketRegisterLen);
 }
 else
 {
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else if (memcmp(pFillPacket, pBuildPacketRegister, fillPacketLen) != 0)
 {
 puts("The built register packet is different to the built packet and the filled packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
 }

 /* Free the memory allocated for each of the packets. Note that the */
 /* memory allocated by the library should be freed by calling */
 /* RMAP_FreeBuffer. */
 RMAP_FreeBuffer(pBuildPacketRegister);
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
}

```

## 10.65 write\_reply\_packet\_example.c

This is an example of how to build RMAP write replies.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "rmap_packet_library.h"

```

```

void write_reply_packet_example()
{
 void *pFillPacket, *pBuildPacket;
 unsigned long fillPacketLenCalculated, fillPacketLen, buildPacketLen;
 U8 pReply[] = {254};
 U8 target = 254;
 char status;

 /* Calculate the length of a write reply packet */
 fillPacketLenCalculated = RMAP_CalculateWriteReplyPacketLength(1, 1
);
 printf("The write reply packet will have a length of %ld bytes\n",
 fillPacketLenCalculated);

 /* Allocate memory for the packet */
 pFillPacket = malloc(fillPacketLenCalculated);
 if (!pFillPacket)
 {
 puts("Couldn't allocate the memory for the reply packet");
 return;
 }

 /* Fill the memory with the packet */
 status = RMAP_FillWriteReplyPacket(pReply, 1, target, 1, 0,
 RMAP_SUCCESS, 0,
 &fillPacketLen, NULL, 1, (U8 *)pFillPacket, fillPacketLenCalculated);
 if (!status)
 {
 puts("Couldn't fill the write reply packet");
 free(pFillPacket);
 return;
 }

 /* Build an RMAP write reply packet */
 pBuildPacket = RMAP_BuildWriteReplyPacket(pReply, 1, target, 1, 0,
 RMAP_SUCCESS, 0, &buildPacketLen, NULL, 1);
 if (!pBuildPacket)
 {
 puts("Couldn't build the write reply packet");
 free(pFillPacket);
 return;
 }

 /* Check the lengths are all the same */
 if ((fillPacketLenCalculated != fillPacketLen) ||
 (fillPacketLen != buildPacketLen))
 {
 puts("The lengths are different.");
 printf("\tLength calculated for packet: %ld\n",
 fillPacketLenCalculated);
 printf("\tLength of filled packet: %ld\n", fillPacketLen);
 printf("\tLength of built packet: %ld\n", buildPacketLen);
 }
 else
 {
 puts("The packet lengths are all the same.");

 /* Check the packets are identical, despite being built differently */
 if (memcmp(pFillPacket, pBuildPacket, fillPacketLen) != 0)
 {
 puts("The filled packet is different to the built packet.");
 }
 else
 {
 puts("The packets are identical.");
 }
 }

 /* Free the memory allocated for each of the packets. Note that the */
 /* memory allocated by the library should be freed by calling */
 /* RMAP_FreeBuffer. */
 RMAP_FreeBuffer(pBuildPacket);
 free(pFillPacket);
}

```

# Index

## Actions, 720

- COUNTER\_ACTION, 723
- COUNTER\_ACTION\_ALL, 723
- COUNTER\_ACTION\_COUNT, 723
- COUNTER\_ACTION\_NONE, 723
- COUNTER\_ACTION\_RELOAD, 723
- COUNTER\_ACTION\_START, 723
- COUNTER\_ACTION\_STOP, 723
- EXT\_TRIGGER\_ACTION, 723
- EXT\_TRIGGER\_ACTION\_ALL, 724
- EXT\_TRIGGER\_ACTION\_NONE, 724
- EXT\_TRIGGER\_ACTION\_OUT, 724
- SPFI\_PORT\_ACTION, 724
- SPFI\_PORT\_ACTION\_ALL, 724
- SPFI\_PORT\_ACTION\_DISCONNECT, 724
- SPFI\_PORT\_ACTION\_MASK, 723
- SPFI\_PORT\_ACTION\_NONE, 724
- SPFI\_VC\_ACTION, 724
- SPFI\_VC\_ACTION\_ALL, 724
- SPFI\_VC\_ACTION\_MASK, 723
- SPFI\_VC\_ACTION\_NONE, 724
- SPFI\_VC\_ACTION\_RECEIVE\_DATA, 724
- SPFI\_VC\_ACTION\_TRANSMIT\_DATA, 724
- SPFI\_VC\_ACTION\_TRANSMIT\_PKT, 724
- SPW\_PORT\_ACTION, 724
- SPW\_PORT\_ACTION\_ALL, 725
- SPW\_PORT\_ACTION\_DECR\_CREDIT, 725
- SPW\_PORT\_ACTION\_DISABLE\_LVDS, 725
- SPW\_PORT\_ACTION\_DISCONNECT, 724
- SPW\_PORT\_ACTION\_ESCAPE\_ERROR, 725
- SPW\_PORT\_ACTION\_INCR\_CREDIT, 725
- SPW\_PORT\_ACTION\_INSERT\_FCT, 725
- SPW\_PORT\_ACTION\_MASK, 723
- SPW\_PORT\_ACTION\_NONE, 725
- SPW\_PORT\_ACTION\_PARITY\_ERROR, 725
- SPW\_PORT\_ACTION\_STOP\_RECEPTION, 725
- SPW\_PORT\_ACTION\_SUPPRESS\_FCT, 725
- SPW\_PORT\_ACTION\_TRANSMIT\_PKT, 724
- TIME\_CODE\_ACTION, 725
- TIME\_CODE\_ACTION\_ALL, 725
- TIME\_CODE\_ACTION\_NONE, 725
- TIME\_CODE\_ACTION\_TX, 725
- TRIGGER\_BRICK\_MK3\_getCounterOutput↔  
Actions, 725
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutput↔  
Actions, 725
- TRIGGER\_BRICK\_MK3\_getPortOutputActions,  
726
- TRIGGER\_BRICK\_MK3\_getTimeCodeOutput↔

## Actions, 726

- TRIGGER\_BRICK\_MK3\_setCounterOutput↔  
Actions, 727
- TRIGGER\_BRICK\_MK3\_setExtTriggerOutput↔  
Actions, 727
- TRIGGER\_BRICK\_MK3\_setPortOutputActions,  
728
- TRIGGER\_BRICK\_MK3\_setTimeCodeOutput↔  
Actions, 728
- TRIGGER\_PCIE\_IF\_getCounterOutputActions,  
729
- TRIGGER\_PCIE\_IF\_getPortOutputActions, 729
- TRIGGER\_PCIE\_IF\_setCounterOutputActions,  
730
- TRIGGER\_PCIE\_IF\_setPortOutputActions, 730
- TRIGGER\_PCIE\_MK2\_getCounterOutputActions,  
731
- TRIGGER\_PCIE\_MK2\_getExtTriggerOutput↔  
Actions, 731
- TRIGGER\_PCIE\_MK2\_getPortOutputActions, 731
- TRIGGER\_PCIE\_MK2\_getTimeCodeOutput↔  
Actions, 732
- TRIGGER\_PCIE\_MK2\_setCounterOutputActions,  
732
- TRIGGER\_PCIE\_MK2\_setExtTriggerOutput↔  
Actions, 733
- TRIGGER\_PCIE\_MK2\_setPortOutputActions, 733
- TRIGGER\_PCIE\_MK2\_setTimeCodeOutput↔  
Actions, 734
- TRIGGER\_PXI\_IF\_getCounterOutputActions, 734
- TRIGGER\_PXI\_IF\_getExtTriggerOutputActions,  
735
- TRIGGER\_PXI\_IF\_getPortOutputActions, 735
- TRIGGER\_PXI\_IF\_getTimeCodeOutputActions,  
736
- TRIGGER\_PXI\_IF\_setCounterOutputActions, 736
- TRIGGER\_PXI\_IF\_setExtTriggerOutputActions,  
736
- TRIGGER\_PXI\_IF\_setPortOutputActions, 737
- TRIGGER\_PXI\_IF\_setTimeCodeOutputActions,  
737
- TRIGGER\_PXI\_ROUTER\_getCounterOutput↔  
Actions, 738
- TRIGGER\_PXI\_ROUTER\_getPortOutputActions,  
738
- TRIGGER\_PXI\_ROUTER\_getTimeCodeOutput↔  
Actions, 739
- TRIGGER\_PXI\_ROUTER\_setCounterOutput↔  
Actions, 739

- TRIGGER\_PXI\_ROUTER\_setPortOutputActions, 740
- TRIGGER\_PXI\_ROUTER\_setTimeCodeOutputActions, 740
- Active Lanes, 1008
  - CFG\_SPFI\_getLinkActiveLanes, 1008
- Brick Mk2 Configuration API, 631
- Brick Mk3 Configuration API, 632
- Broadcast Controller, 390, 916
  - CFG\_PCIE\_MK2\_disableBroadcastControllerLegacyMode, 390
  - CFG\_PCIE\_MK2\_enableBroadcastControllerLegacyMode, 391
  - CFG\_PCIE\_MK2\_getBroadcastControllerLegacyModeEnabled, 391
  - CFG\_PCIE\_MK2\_getInterruptCodeDistributionPorts, 391
  - CFG\_PCIE\_MK2\_setInterruptCodeDistributionPorts, 393
  - CFG\_disableBroadcastControllerLegacyMode, 916
  - CFG\_enableBroadcastControllerLegacyMode, 917
  - CFG\_getBroadcastControllerLegacyModeEnabled, 917
  - CFG\_getInterruptCodeDistributionPorts, 918
  - CFG\_setInterruptCodeDistributionPorts, 918
- CCSDS/SPP/TF Packet Library, 1089
  - CCSDS\_SPP\_PACKET\_TYPE, 1100
  - CCSDS\_SPP\_PACKET\_TYPE\_CPDU\_COMMAND, 1100
  - CCSDS\_SPP\_PACKET\_TYPE\_IDLE\_PACKET, 1100
  - CCSDS\_SPP\_PACKET\_TYPE\_SPACECRAFT\_TIME\_PACKET, 1100
  - CCSDS\_SPP\_PACKET\_TYPE\_TELECOMMAND, 1100
  - CCSDS\_SPP\_PACKET\_TYPE\_TELEMETRY, 1100
  - CCSDS\_SPP\_STATUS, 1100
  - CCSDS\_SPP\_STATUS\_APID\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_BOTH\_SECONDARY\_HEADER\_AND\_USER\_DATA\_FIELD\_MISSED, 1101
  - CCSDS\_SPP\_STATUS\_DATA\_LENGTH\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_LENGTH\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_GENERAL\_ERROR, 1102
  - CCSDS\_SPP\_STATUS\_MEMORY\_FAILURE, 1101
  - CCSDS\_SPP\_STATUS\_PACKET\_STRUCTURE\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_PACKET\_TYPE\_UNKNOWN, 1101
  - CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INCOMPLETE, 1101
  - CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_PTP\_HEADER\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_PTP\_PROTOCOL\_IDENTIFIER\_INCORRECT, 1101
  - CCSDS\_SPP\_STATUS\_RAW\_DATA\_POINTER\_NULL, 1100
  - CCSDS\_SPP\_STATUS\_RAW\_PACKET\_LENGTH\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_RECEIVED\_RAW\_DATA\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_FLAG\_PACKET\_TYPE\_CONFLICT, 1101
  - CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_LENGTH\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_SEQUENCE\_FLAGS\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_STATUS\_POINTER\_NULL, 1100
  - CCSDS\_SPP\_STATUS\_SUCCESS, 1100
  - CCSDS\_SPP\_STATUS\_TELECOMMAND\_SECONDARY\_HEADER\_INVALID, 1101
  - CCSDS\_SPP\_STATUS\_TOO\_MANY\_DEVICE\_REGISTER\_ADDRESSES\_SPECIFIED, 1101
  - CCSDS\_SPP\_STATUS\_VALUE\_POINTER\_NULL, 1101
  - CCSDS\_SPP\_STATUS\_ZERO\_CCSDS\_SPP\_PACKETS\_PROVIDED\_FOR\_TF\_CREATION, 1101
  - CCSDS\_TF\_STATUS\_NUMBER\_OF\_CCSDS\_SPP\_PACKETS\_ZERO, 1101
  - CCSDS\_TF\_STATUS\_NUMBER\_OF\_TF\_PACKETS\_ZERO, 1102
  - CCSDS\_TF\_STATUS\_PACKET\_LENGTHS\_POINTER\_INVALID, 1101
  - M\_PDU\_PACKET\_MAX\_SIZE, 1100
- CCSDS\_SPP\_CheckAPIID
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, 1109
- CCSDS\_SPP\_CreatePTPHeader
  - Functions for building SPP Packets, 1104
- CCSDS\_SPP\_CreatePacket
  - Functions for building SPP Packets, 1103
- CCSDS\_SPP\_ENCAPS\_HEADER, 1092
- CCSDS\_SPP\_FillPTPHeader
  - Functions for building SPP Packets, 1105
- CCSDS\_SPP\_FillPacket
  - Functions for building SPP Packets, 1105
- CCSDS\_SPP\_FreeBuffer
  - Functions for building SPP Packets, 1107
- CCSDS\_SPP\_GetAPIID
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, 1109

- preting SPP Packets, [1109](#)
- CCSDS\_SPP\_GetDataFieldLength
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1110](#)
- CCSDS\_SPP\_GetDataFieldPointer
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1110](#)
- CCSDS\_SPP\_GetEncapsulationHeaderLength
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1111](#)
- CCSDS\_SPP\_GetEncapsulationHeaderPointer
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1111](#)
- CCSDS\_SPP\_GetPTPHeaderLengthBytes
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1112](#)
- CCSDS\_SPP\_GetPTPProtocolIdentifier
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1112](#)
- CCSDS\_SPP\_GetPTPReservedByte
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1114](#)
- CCSDS\_SPP\_GetPTPTargetLogicalAddress
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1114](#)
- CCSDS\_SPP\_GetPTPUserApplicationByte
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1114](#)
- CCSDS\_SPP\_GetPacketType
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1111](#)
- CCSDS\_SPP\_GetPacketVersionNumber
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1112](#)
- CCSDS\_SPP\_GetPrimaryHeaderLengthBytes
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1112](#)
- CCSDS\_SPP\_GetSequenceCount
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1115](#)
- CCSDS\_SPP\_GetSequenceFlags
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1115](#)
- CCSDS\_SPP\_GetTelecommandSecondaryHeaderLengthBytes
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1115](#)
- CCSDS\_SPP\_IsSecondaryHeaderPresent
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1116](#)
- CCSDS\_SPP\_PACKET, [1099](#)
- CCSDS\_SPP\_PACKET\_DATA\_FIELD, [1098](#)
- CCSDS\_SPP\_PACKET\_PRIMARY\_HEADER, [1097](#)
- CCSDS\_SPP\_PACKET\_TYPE
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PACKET\_TYPE\_CPDU\_COMMAND
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PACKET\_TYPE\_IDLE\_PACKET
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PACKET\_TYPE\_SPACECRAFT\_TIME\_PACKET
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PACKET\_TYPE\_TELECOMMAND
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PACKET\_TYPE\_TELEMETRY
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_PUS\_CreateST02DistributeOnOffDeviceCommand
  - ccsds\_spp\_pus\_services.h, [1136](#)
- CCSDS\_SPP\_PUS\_CreateST02DistributeRegisterDumpCommand
  - ccsds\_spp\_pus\_services.h, [1137](#)
- CCSDS\_SPP\_RetrievePacket
  - Functions for retrieving (reconstructing) and interpreting SPP Packets, [1116](#)
- CCSDS\_SPP\_STATUS
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_STATUS\_APID\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_BOTH\_SECONDARY\_HEADER\_AND\_USER\_DATA\_FIELD\_MISSING
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_DATA\_LENGTH\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_LENGTH\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_ENCAPSULATION\_HEADER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_GENERAL\_ERROR
  - CCSDS/SPP/TF Packet Library, [1102](#)
- CCSDS\_SPP\_STATUS\_MEMORY\_FAILURE
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PACKET\_STRUCTURE\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PACKET\_TYPE\_UNKNOWN
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INCOMPLETE
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PTP\_HEADER\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_PTP\_PROTOCOL\_IDENTIFIER\_INCORRECT
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_RAW\_DATA\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_STATUS\_RAW\_PACKET\_LENGTH\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)



- CCSDS\_SPP\_STATUS\_RECEIVED\_RAW\_DATA\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_FLAG\_PACKET\_TYPE\_CONFLICT
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_LENGTH\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_SECONDARY\_HEADER\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_SEQUENCE\_FLAGS\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_STATUS\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_STATUS\_SUCCESS
  - CCSDS/SPP/TF Packet Library, [1100](#)
- CCSDS\_SPP\_STATUS\_TELECOMMAND\_SECONDARY\_HEADER\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_TOO\_MANY\_DEVICE\_REGISTERS\_ADDRESSES\_SPECIFIED
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_VALUE\_POINTER\_NULL
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_SPP\_STATUS\_ZERO\_CCSDS\_SPP\_PACKETS\_PROVIDED\_FOR\_TF\_CREATION
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_TF\_CreateIdlePacket
  - Functions for building TF Packets, [1117](#)
- CCSDS\_TF\_CreatePackets
  - Functions for building TF Packets, [1118](#)
- CCSDS\_TF\_Get\_M\_PDU\_PacketMaxSize
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1121](#)
- CCSDS\_TF\_INSERT\_ZONE, [1095](#)
- CCSDS\_TF\_M\_PDU\_DATA\_FIELD, [1095](#)
- CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_DataField
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1121](#)
- CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_FirstHeaderPointer
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1122](#)
- CCSDS\_TF\_M\_PDU\_GetEncapsulationHeader
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1122](#)
- CCSDS\_TF\_M\_PDU\_GetFrameErrorControlField
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1122](#)
- CCSDS\_TF\_M\_PDU\_GetFrameHeaderErrorControl
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1124](#)
- CCSDS\_TF\_M\_PDU\_GetInsertZoneData
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1124](#)
- CCSDS\_TF\_M\_PDU\_GetOperationalControlField
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1124](#)
- CCSDS\_TF\_M\_PDU\_GetReplayFlag
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1126](#)
- CCSDS\_TF\_M\_PDU\_GetSpacecraftID
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1126](#)
- CCSDS\_TF\_M\_PDU\_GetVCFrameCountCycle
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1126](#)
- CCSDS\_TF\_M\_PDU\_GetVCFrameCountUsageFlag
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1127](#)
- CCSDS\_TF\_M\_PDU\_GetVersionNumber
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1127](#)
- CCSDS\_TF\_M\_PDU\_GetVirtualChannelFrameCount
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1127](#)
- CCSDS\_TF\_M\_PDU\_GetVirtualChannelID
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1128](#)
- CCSDS\_TF\_M\_PDU\_HEADER, [1093](#)
- CCSDS\_TF\_M\_PDU\_PACKET, [1096](#)
- CCSDS\_TF\_PACKET\_PRIMARY\_HEADER, [1094](#)
- CCSDS\_TF\_RetrievePacket
  - Functions for retrieving (reconstructing) and interpreting TF Packets, [1128](#)
- CCSDS\_TF\_STATUS\_NUMBER\_OF\_CCSDS\_SPP\_PACKETS\_ZERO
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CCSDS\_TF\_STATUS\_NUMBER\_OF\_TF\_PACKETS\_ZERO
  - CCSDS/SPP/TF Packet Library, [1102](#)
- CCSDS\_TF\_STATUS\_PACKET\_LENGTHS\_POINTER\_INVALID
  - CCSDS/SPP/TF Packet Library, [1101](#)
- CFG\_API\_MK2\_MODULE\_NAME
  - cfg\_api\_mk2.h, [1158](#)
- CFG\_API\_PCI\_MK2\_MODULE\_NAME
  - cfg\_api\_pci\_mk2.h, [1160](#)
- CFG\_BRICK\_MK2\_disableIdentifySourceOnPort
  - Interface Mode, [402](#)
- CFG\_BRICK\_MK2\_disableInterfaceModeOnPort
  - Interface Mode, [403](#)
- CFG\_BRICK\_MK2\_disableSpeedChangeEventsOnPort
  - Events, [408](#)
- CFG\_BRICK\_MK2\_disableStateChangeEventsOnPort
  - Events, [410](#)
- CFG\_BRICK\_MK2\_enableIdentifySourceOnPort
  - Interface Mode, [403](#)
- CFG\_BRICK\_MK2\_enableInterfaceModeOnPort
  - Interface Mode, [404](#)
- CFG\_BRICK\_MK2\_enableSpeedChangeEventsOnPort
  - Events, [410](#)
- CFG\_BRICK\_MK2\_enableStateChangeEventsOnPort
  - Events, [411](#)



- CFG\_BRICK\_MK2\_getIdentifySourceOnPortEnabled
  - Interface Mode, [404](#)
- CFG\_BRICK\_MK2\_getInterfaceModeOnPortEnabled
  - Interface Mode, [405](#)
- CFG\_BRICK\_MK2\_getLinkClockFrequency
  - Link Speed, [400](#)
- CFG\_BRICK\_MK2\_getPortRoutingAddress
  - Interface Mode, [405](#)
- CFG\_BRICK\_MK2\_getSpeedChangeEventsOnPort↔
  - Enabled
  - Events, [411](#)
- CFG\_BRICK\_MK2\_getStateChangeEventsOnPort↔
  - Enabled
  - Events, [412](#)
- CFG\_BRICK\_MK2\_injectErrors
  - Error Injection, [407](#)
- CFG\_BRICK\_MK2\_setLinkClockFrequency
  - Link Speed, [400](#)
- CFG\_BRICK\_MK2\_setPortRoutingAddress
  - Interface Mode, [406](#)
- CFG\_BRICK\_MK3\_disableRxTimestampEventsOnPort
  - Timestamping, [418](#)
- CFG\_BRICK\_MK3\_enableRxTimestampEventsOnPort
  - Timestamping, [419](#)
- CFG\_BRICK\_MK3\_getBaseTransmitClock
  - Link Speed, [413](#)
- CFG\_BRICK\_MK3\_getPulseGeneratorFrequency
  - Timestamping, [419](#)
- CFG\_BRICK\_MK3\_getRxTimestampEventsOnPort↔
  - Enabled
  - Timestamping, [420](#)
- CFG\_BRICK\_MK3\_getTimestampMethod
  - Timestamping, [420](#)
- CFG\_BRICK\_MK3\_getTimestampValue
  - Timestamping, [421](#)
- CFG\_BRICK\_MK3\_getTransmitClock
  - Link Speed, [414](#)
- CFG\_BRICK\_MK3\_setBaseTransmitClock
  - Link Speed, [414](#)
- CFG\_BRICK\_MK3\_setPulseGeneratorFrequency
  - Timestamping, [421](#)
- CFG\_BRICK\_MK3\_setTimestampMethod
  - Timestamping, [422](#)
- CFG\_BRICK\_MK3\_setTimestampValue
  - Timestamping, [422](#)
- CFG\_BRICK\_MK3\_setTransmitClock
  - Link Speed, [415](#)
- CFG\_FPGAInfoToString
  - Hardware, [857](#)
- CFG\_GBE\_BRICK\_MK1\_getTransmitSignallingRate
  - Link Speed, [424](#)
- CFG\_GBE\_BRICK\_MK1\_setTransmitSignallingRate
  - Link Speed, [425](#)
- CFG\_INFO\_ID\_TYPE
  - types.h, [1239](#)
- CFG\_MK2\_disableExternalTimeCodeSelection
  - Time-code Master, [575](#)
- CFG\_MK2\_disableIdentifySource
  - Interface Mode, [585](#)
- CFG\_MK2\_disableIdentifySourceOnPort
  - Interface Mode, [585](#)
- CFG\_MK2\_disableInterfaceMode
  - Interface Mode, [585](#)
- CFG\_MK2\_disableInterfaceModeOnPort
  - Interface Mode, [586](#)
- CFG\_MK2\_disableSpeedChangeEventsOnPort
  - Events, [561](#)
- CFG\_MK2\_disableStateChangeEventsOnPort
  - Events, [561](#)
- CFG\_MK2\_disableTimeCodeCounterBypassMode
  - Time-code Master, [575](#)
- CFG\_MK2\_disableTimeCodeEventsOnPort
  - Events, [562](#)
- CFG\_MK2\_disableTimeCodeMaster
  - Time-code Master, [575](#)
- CFG\_MK2\_enableExternalTimeCodeSelection
  - Time-code Master, [576](#)
- CFG\_MK2\_enableIdentifySource
  - Interface Mode, [586](#)
- CFG\_MK2\_enableIdentifySourceOnPort
  - Interface Mode, [587](#)
- CFG\_MK2\_enableInterfaceMode
  - Interface Mode, [587](#)
- CFG\_MK2\_enableInterfaceModeOnPort
  - Interface Mode, [587](#)
- CFG\_MK2\_enableSpeedChangeEventsOnPort
  - Events, [562](#)
- CFG\_MK2\_enableStateChangeEventsOnPort
  - Events, [563](#)
- CFG\_MK2\_enableTimeCodeCounterBypassMode
  - Time-code Master, [576](#)
- CFG\_MK2\_enableTimeCodeEventsOnPort
  - Events, [563](#)
- CFG\_MK2\_enableTimeCodeMaster
  - Time-code Master, [577](#)
- CFG\_MK2\_getBaseTransmitClock
  - Link Speed, [567](#)
- CFG\_MK2\_getDeviceClockRate
  - Periodic Actions, [594](#)
- CFG\_MK2\_getExternalTimeCodeSelectionEnabled
  - Time-code Master, [577](#)
- CFG\_MK2\_getHardwareInfo
  - Device Information, [582](#)
- CFG\_MK2\_getIdentifySourceEnabled
  - Interface Mode, [588](#)
- CFG\_MK2\_getIdentifySourceOnPortEnabled
  - Interface Mode, [588](#)
- CFG\_MK2\_getInterfaceModeEnabled
  - Interface Mode, [589](#)
- CFG\_MK2\_getInterfaceModeOnPortEnabled
  - Interface Mode, [589](#)
- CFG\_MK2\_getLinkRateDivider
  - Link Speed, [568](#)
- CFG\_MK2\_getMeasuredLinkSpeed
  - Measured Link Speed, [572](#)
- CFG\_MK2\_getPortRoutingAddress

- Interface Mode, 590
- CFG\_MK2\_getSpeedChangeEventsOnPortEnabled  
Events, 564
- CFG\_MK2\_getStateChangeEventsOnPortEnabled  
Events, 564
- CFG\_MK2\_getTimeCodeCounterBypassModeEnabled  
Time-code Master, 578
- CFG\_MK2\_getTimeCodeDistributionPorts  
User Configurable Registers, 660
- CFG\_MK2\_getTimeCodeEventsOnPortEnabled  
Events, 565
- CFG\_MK2\_getTimeCodeFlagMode  
Router Control and Status Configuration, 668
- CFG\_MK2\_getTimeCodeMasterEnabled  
Time-code Master, 578
- CFG\_MK2\_getTimeCodePeriod  
Time-code Master, 579
- CFG\_MK2\_getTransmitClock  
Link Speed, 568
- CFG\_MK2\_hardwareInfoToString  
Device Information, 583
- CFG\_MK2\_identify  
Device Information, 583
- CFG\_MK2\_injectError  
Error Injection, 593
- CFG\_MK2\_setBaseTransmitClock  
Link Speed, 569
- CFG\_MK2\_setLinkRateDivider  
Link Speed, 570
- CFG\_MK2\_setPortRoutingAddress  
Interface Mode, 590
- CFG\_MK2\_setTimeCodeDistributionPorts  
User Configurable Registers, 661
- CFG\_MK2\_setTimeCodeFlagMode  
Router Control and Status Configuration, 668
- CFG\_MK2\_setTimeCodePeriod  
Time-code Master, 579
- CFG\_MK2\_setTransmitClock  
Link Speed, 570
- CFG\_MK2\_startPeriodicAction  
Periodic Actions, 595
- CFG\_MK2\_stopPeriodicAction  
Periodic Actions, 595
- CFG\_PCIE\_MK2\_disableBroadcastControllerLegacy↔  
Mode  
Broadcast Controller, 390
- CFG\_PCIE\_MK2\_disableRxTimestampEventsOnPort  
Timestamping, 394
- CFG\_PCIE\_MK2\_enableBroadcastControllerLegacy↔  
Mode  
Broadcast Controller, 391
- CFG\_PCIE\_MK2\_enableRxTimestampEventsOnPort  
Timestamping, 395
- CFG\_PCIE\_MK2\_getBroadcastControllerLegacy↔  
ModeEnabled  
Broadcast Controller, 391
- CFG\_PCIE\_MK2\_getDecimalTxSignallingRate  
Link Speed, 387
- CFG\_PCIE\_MK2\_getInterruptCodeDistributionPorts  
Broadcast Controller, 391
- CFG\_PCIE\_MK2\_getPulseGeneratorFrequency  
Timestamping, 395
- CFG\_PCIE\_MK2\_getRxTimestampEventsOnPort↔  
Enabled  
Timestamping, 395
- CFG\_PCIE\_MK2\_getTransmitSignallingRate  
Link Speed, 388
- CFG\_PCIE\_MK2\_injectErrors  
Error Injection, 398
- CFG\_PCIE\_MK2\_setDecimalTxSignallingRate  
Link Speed, 388
- CFG\_PCIE\_MK2\_setInterruptCodeDistributionPorts  
Broadcast Controller, 393
- CFG\_PCIE\_MK2\_setPulseGeneratorFrequency  
Timestamping, 396
- CFG\_PCIE\_MK2\_setTimestampMethod  
Timestamping, 396
- CFG\_PCIE\_MK2\_setTransmitSignallingRate  
Link Speed, 388
- CFG\_PCIMK2\_getLinkClockFrequency  
Link Speed, 385
- CFG\_PCIMK2\_setLinkClockFrequency  
Link Speed, 385
- CFG\_PXI\_disableIdentifySourceOnPort  
Interface Mode, 432
- CFG\_PXI\_disableInterfaceModeOnPort  
Interface Mode, 433
- CFG\_PXI\_disableSpeedChangeEventsOnPort  
Events, 437
- CFG\_PXI\_disableStateChangeEventsOnPort  
Events, 438
- CFG\_PXI\_disableTimeCodeEventsOnPort  
Events, 438
- CFG\_PXI\_enableIdentifySourceOnPort  
Interface Mode, 433
- CFG\_PXI\_enableInterfaceModeOnPort  
Interface Mode, 433
- CFG\_PXI\_enableSpeedChangeEventsOnPort  
Events, 439
- CFG\_PXI\_enableStateChangeEventsOnPort  
Events, 439
- CFG\_PXI\_enableTimeCodeEventsOnPort  
Events, 439
- CFG\_PXI\_getBaseTransmitClock  
Link Speed, 426
- CFG\_PXI\_getIdentifySourceOnPortEnabled  
Interface Mode, 434
- CFG\_PXI\_getInterfaceModeOnPortEnabled  
Interface Mode, 434
- CFG\_PXI\_getLinkRateDivider  
Link Speed, 427
- CFG\_PXI\_getPortRoutingAddress  
Interface Mode, 435
- CFG\_PXI\_getSpeedChangeEventsOnPortEnabled  
Events, 440
- CFG\_PXI\_getStateChangeEventsOnPortEnabled

- Events, [440](#)
- CFG\_PXI\_getTimeCodeEventsOnPortEnabled
  - Events, [441](#)
- CFG\_PXI\_getTransmitClock
  - Link Speed, [427](#)
- CFG\_PXI\_injectError
  - Error Injection, [431](#)
- CFG\_PXI\_setBaseTransmitClock
  - Link Speed, [428](#)
- CFG\_PXI\_setLinkRateDivider
  - Link Speed, [428](#)
- CFG\_PXI\_setPortRoutingAddress
  - Interface Mode, [435](#)
- CFG\_PXI\_setTransmitClock
  - Link Speed, [429](#)
- CFG\_ROUTER\_MK2S\_disablePrecisionTransmitRate
  - Link Speed, [556](#)
- CFG\_ROUTER\_MK2S\_enablePrecisionTransmitRate
  - Link Speed, [557](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate
  - Link Speed, [557](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate↔
  - Enabled
  - Link Speed, [558](#)
- CFG\_ROUTER\_MK2S\_getPrecisionTransmitRateInUse
  - Link Speed, [558](#)
- CFG\_ROUTER\_MK2S\_setPrecisionTransmitRate
  - Link Speed, [558](#)
- CFG\_ROUTER\_clearPortErrors
  - Port Status and Control, [650](#)
- CFG\_ROUTER\_getConfigPortErrors
  - Port Status and Control, [650](#)
- CFG\_ROUTER\_getDeviceIdentificationInfo
  - Device Information, [640](#)
- CFG\_ROUTER\_getDeviceManufacturerAsString
  - Device Information, [640](#)
- CFG\_ROUTER\_getDeviceTypeAsString
  - Device Information, [641](#)
- CFG\_ROUTER\_getExternalPortErrors
  - Port Status and Control, [651](#)
- CFG\_ROUTER\_getExternalPortStatus
  - Port Status and Control, [651](#)
- CFG\_ROUTER\_getGeneralPurpose
  - User Configurable Registers, [661](#)
- CFG\_ROUTER\_getNetworkDiscoveryInfo
  - Device Information, [641](#)
- CFG\_ROUTER\_getNetworkIdentity
  - User Configurable Registers, [662](#)
- CFG\_ROUTER\_getPortConnection
  - Port Status and Control, [652](#)
- CFG\_ROUTER\_getPortStatusControl
  - Port Status and Control, [652](#)
- CFG\_ROUTER\_getPortType
  - Port Status and Control, [653](#)
- CFG\_ROUTER\_getRouterGlobalSettings
  - Router Control and Status Configuration, [668](#)
- CFG\_ROUTER\_getRoutingTableEntry
  - Routing Table, [658](#)
- CFG\_ROUTER\_getSpaceWireLinkErrors
  - Port Status and Control, [653](#)
- CFG\_ROUTER\_getSpaceWireLinkStatus
  - Port Status and Control, [654](#)
- CFG\_ROUTER\_getTimeCodeDistributionPorts
  - User Configurable Registers, [662](#)
- CFG\_ROUTER\_getTimeCodeFlagMode
  - Router Control and Status Configuration, [670](#)
- CFG\_ROUTER\_getTimeCodeValue
  - Router Control and Status Configuration, [670](#)
- CFG\_ROUTER\_setGeneralPurpose
  - User Configurable Registers, [663](#)
- CFG\_ROUTER\_setNetworkIdentity
  - User Configurable Registers, [663](#)
- CFG\_ROUTER\_setRouterGlobalSettings
  - Router Control and Status Configuration, [671](#)
- CFG\_ROUTER\_setRoutingTableEntry
  - Routing Table, [659](#)
- CFG\_ROUTER\_setSpaceWireLinkStatus
  - Port Status and Control, [654](#)
- CFG\_ROUTER\_setTimeCodeDistributionPorts
  - User Configurable Registers, [664](#)
- CFG\_ROUTER\_setTimeCodeFlagMode
  - Router Control and Status Configuration, [671](#)
- CFG\_ROUTER\_startLink
  - Port Status and Control, [655](#)
- CFG\_ROUTER\_stopLink
  - Port Status and Control, [655](#)
- CFG\_SPFI\_clearLaneEvents
  - Events, [1028](#)
- CFG\_SPFI\_clearLinkEvents
  - Events, [997](#)
- CFG\_SPFI\_clearLinkProtocolError
  - Status, [991](#)
- CFG\_SPFI\_clearPortBroadcastError
  - Port Information, [975](#)
- CFG\_SPFI\_clearVCErrors
  - Errors, [963](#)
- CFG\_SPFI\_destroySpFiBitRates
  - Data Signalling Rate, [1034](#)
- CFG\_SPFI\_disableLaneErrorRecovery
  - Control, [1011](#)
- CFG\_SPFI\_disableLaneReset
  - Control, [1011](#)
- CFG\_SPFI\_disableLaneRx
  - Control, [1011](#)
- CFG\_SPFI\_disableLaneTx
  - Control, [1012](#)
- CFG\_SPFI\_disableLinkAutoStart
  - Control, [983](#)
- CFG\_SPFI\_disableLinkInterfaceReset
  - Control, [983](#)
- CFG\_SPFI\_disableLinkNearEndLoopback
  - Debug, [1001](#)
- CFG\_SPFI\_disableLinkReset
  - Control, [983](#)
- CFG\_SPFI\_disableLinkRxCrambling
  - Debug, [1002](#)

- CFG\_SPFI\_disableLinkStart
  - Control, [984](#)
- CFG\_SPFI\_disableLinkTxScrambling
  - Debug, [1002](#)
- CFG\_SPFI\_disableRouterTimeout
  - Vendor Specific, [948](#)
- CFG\_SPFI\_disableRoutingTableEntry
  - Routing Table, [933](#)
- CFG\_SPFI\_disableRoutingTableEntryDeleteHeader
  - Routing Table, [934](#)
- CFG\_SPFI\_disableVCCContinuousMode
  - Quality Of Service, [966](#)
- CFG\_SPFI\_enableLaneErrorRecovery
  - Control, [1012](#)
- CFG\_SPFI\_enableLaneReset
  - Control, [1013](#)
- CFG\_SPFI\_enableLaneRx
  - Control, [1013](#)
- CFG\_SPFI\_enableLaneTx
  - Control, [1014](#)
- CFG\_SPFI\_enableLinkAutoStart
  - Control, [984](#)
- CFG\_SPFI\_enableLinkInterfaceReset
  - Control, [985](#)
- CFG\_SPFI\_enableLinkNearEndLoopback
  - Debug, [1003](#)
- CFG\_SPFI\_enableLinkReset
  - Control, [985](#)
- CFG\_SPFI\_enableLinkRxScrambling
  - Debug, [1003](#)
- CFG\_SPFI\_enableLinkStart
  - Control, [985](#)
- CFG\_SPFI\_enableLinkTxScrambling
  - Debug, [1003](#)
- CFG\_SPFI\_enableRouterTimeout
  - Vendor Specific, [948](#)
- CFG\_SPFI\_enableRoutingTableEntry
  - Routing Table, [934](#)
- CFG\_SPFI\_enableRoutingTableEntryDeleteHeader
  - Routing Table, [935](#)
- CFG\_SPFI\_enableVCCContinuousMode
  - Quality Of Service, [967](#)
- CFG\_SPFI\_getAXIBC
  - Port Information, [976](#)
- CFG\_SPFI\_getBroadcastTimeout
  - Network Management, [940](#)
- CFG\_SPFI\_getBroadcastTimeoutMaximum
  - Network Management, [940](#)
- CFG\_SPFI\_getBroadcastTimeoutResolution
  - Network Management, [940](#)
- CFG\_SPFI\_getBroadcastTypesInformation
  - Vendor Specific, [948](#)
- CFG\_SPFI\_getCurrentTime
  - Network Management, [942](#)
- CFG\_SPFI\_getCurrentTimeSlot
  - Network Management, [942](#)
- CFG\_SPFI\_getDeviceIdentificationInfo
  - Device Identification, [928](#)
- CFG\_SPFI\_getDeviceIdentifier
  - Network Management, [943](#)
- CFG\_SPFI\_getDeviceInformation
  - Device Information, [925](#)
- CFG\_SPFI\_getDeviceManufacturer
  - Device Identification, [929](#)
- CFG\_SPFI\_getDeviceManufacturerAsString
  - Device Identification, [929](#)
- CFG\_SPFI\_getDeviceNodeType
  - Device Information, [926](#)
- CFG\_SPFI\_getDevicePortCount
  - Device Information, [926](#)
- CFG\_SPFI\_getDevicePortCountByType
  - Device Information, [927](#)
- CFG\_SPFI\_getDeviceType
  - Device Identification, [930](#)
- CFG\_SPFI\_getDeviceTypeAsString
  - Device Identification, [930](#)
- CFG\_SPFI\_getDeviceVersion
  - Device Identification, [931](#)
- CFG\_SPFI\_getLaneControlSettings
  - Control, [1014](#)
- CFG\_SPFI\_getLaneDisconnectCount
  - Events, [1029](#)
- CFG\_SPFI\_getLaneEvents
  - Events, [1029](#)
- CFG\_SPFI\_getLaneFarEndINIT3Flags
  - Status, [1020](#)
- CFG\_SPFI\_getLaneLOSReasonRx
  - Status, [1021](#)
- CFG\_SPFI\_getLaneRxErrorCount
  - Events, [1030](#)
- CFG\_SPFI\_getLaneStandbyReason
  - Control, [1015](#)
- CFG\_SPFI\_getLaneStandbyReasonRx
  - Status, [1021](#)
- CFG\_SPFI\_getLaneState
  - Status, [1022](#)
- CFG\_SPFI\_getLaneStatus
  - Status, [1022](#)
- CFG\_SPFI\_getLinkActiveLanes
  - Active Lanes, [1008](#)
- CFG\_SPFI\_getLinkAlignmentState
  - Status, [992](#)
- CFG\_SPFI\_getLinkControlSettings
  - Control, [986](#)
- CFG\_SPFI\_getLinkDebugSettings
  - Debug, [1004](#)
- CFG\_SPFI\_getLinkEvents
  - Events, [998](#)
- CFG\_SPFI\_getLinkLanesRequestedCount
  - Control, [986](#)
- CFG\_SPFI\_getLinkRetryCount
  - Events, [998](#)
- CFG\_SPFI\_getLinkStatus
  - Status, [992](#)
- CFG\_SPFI\_getLinksWithErrors
  - Network Management, [943](#)

- CFG\_SPFI\_getPortInformation
  - Port Information, [976](#)
- CFG\_SPFI\_getPortType
  - Port Information, [977](#)
- CFG\_SPFI\_getPortVCCount
  - Port Information, [977](#)
- CFG\_SPFI\_getPortVCsWithErrors
  - Port Information, [978](#)
- CFG\_SPFI\_getPortsReady
  - Network Management, [944](#)
- CFG\_SPFI\_getPortsWithErrors
  - Network Management, [944](#)
- CFG\_SPFI\_getRouterTimeout
  - Vendor Specific, [949](#)
- CFG\_SPFI\_getRouterTimeoutMaximum
  - Vendor Specific, [949](#)
- CFG\_SPFI\_getRouterTimeoutResolution
  - Vendor Specific, [950](#)
- CFG\_SPFI\_getRoutingTableEntryFlags
  - Routing Table, [935](#)
- CFG\_SPFI\_getRoutingTableEntryPorts
  - Routing Table, [936](#)
- CFG\_SPFI\_getSpFiBitRate
  - Data Signalling Rate, [1034](#)
- CFG\_SPFI\_getSpFiBitRates
  - Data Signalling Rate, [1035](#)
- CFG\_SPFI\_getSpaceWireBC
  - Network Management, [945](#)
- CFG\_SPFI\_getSpaceWireInterruptBroadcastType
  - Vendor Specific, [950](#)
- CFG\_SPFI\_getTimeCodeBroadcastType
  - Vendor Specific, [951](#)
- CFG\_SPFI\_getTimeSlotBroadcastType
  - Vendor Specific, [951](#)
- CFG\_SPFI\_getVCBandwidthReservation
  - Quality Of Service, [968](#)
- CFG\_SPFI\_getVCDataRate
  - Data Rate, [974](#)
- CFG\_SPFI\_getVCErrors
  - Errors, [964](#)
- CFG\_SPFI\_getVCPriority
  - Quality Of Service, [968](#)
- CFG\_SPFI\_getVCQualityOfService
  - Quality Of Service, [969](#)
- CFG\_SPFI\_getVCStatus
  - Status, [958](#)
- CFG\_SPFI\_getVCTimeSlots
  - Time-slots, [972](#)
- CFG\_SPFI\_getVVCVN
  - Virtual Network Number, [956](#)
- CFG\_SPFI\_isLaneActive
  - Status, [1023](#)
- CFG\_SPFI\_isLaneErrorDetected
  - Status, [1023](#)
- CFG\_SPFI\_isLaneErrorRecoveryEnabled
  - Control, [1015](#)
- CFG\_SPFI\_isLaneEventDetected
  - Status, [1024](#)
- CFG\_SPFI\_isLaneFarEndErrorBurstDetected
  - Events, [1030](#)
- CFG\_SPFI\_isLaneFarEndINIT1RxInActiveDetected
  - Events, [1031](#)
- CFG\_SPFI\_isLaneFarEndLOSDetected
  - Events, [1031](#)
- CFG\_SPFI\_isLaneFarEndStandbyDetected
  - Events, [1032](#)
- CFG\_SPFI\_isLaneINIT1RxDetected
  - Events, [1032](#)
- CFG\_SPFI\_isLaneINIT2RxDetected
  - Events, [1032](#)
- CFG\_SPFI\_isLaneInitTimeoutDetected
  - Events, [1033](#)
- CFG\_SPFI\_isLaneNoSignalDetected
  - Status, [1024](#)
- CFG\_SPFI\_isLaneResetEnabled
  - Control, [1016](#)
- CFG\_SPFI\_isLaneRxEnabled
  - Control, [1016](#)
- CFG\_SPFI\_isLaneRxOnly
  - Status, [1024](#)
- CFG\_SPFI\_isLaneStarted
  - Status, [1026](#)
- CFG\_SPFI\_isLaneTxEnabled
  - Control, [1017](#)
- CFG\_SPFI\_isLaneTxOnly
  - Status, [1026](#)
- CFG\_SPFI\_isLinkAutoStartEnabled
  - Control, [988](#)
- CFG\_SPFI\_isLinkCRC16ErrorDetected
  - Events, [999](#)
- CFG\_SPFI\_isLinkCRC8ErrorDetected
  - Events, [999](#)
- CFG\_SPFI\_isLinkErrorDetected
  - Status, [993](#)
- CFG\_SPFI\_isLinkEventDetected
  - Status, [993](#)
- CFG\_SPFI\_isLinkFarEndResetDetected
  - Events, [1000](#)
- CFG\_SPFI\_isLinkFrameErrorDetected
  - Events, [1000](#)
- CFG\_SPFI\_isLinkIdle
  - Status, [994](#)
- CFG\_SPFI\_isLinkInterfaceResetEnabled
  - Control, [988](#)
- CFG\_SPFI\_isLinkLanesEventDetected
  - Status, [994](#)
- CFG\_SPFI\_isLinkNearEndLoopbackEnabled
  - Debug, [1004](#)
- CFG\_SPFI\_isLinkProtocolErrorDetected
  - Status, [995](#)
- CFG\_SPFI\_isLinkReady
  - Status, [995](#)
- CFG\_SPFI\_isLinkReceiving
  - Status, [995](#)
- CFG\_SPFI\_isLinkResetEnabled
  - Control, [989](#)



- CFG\_SPFI\_isLinkRxScramblingEnabled
  - Debug, [1006](#)
- CFG\_SPFI\_isLinkStartEnabled
  - Control, [989](#)
- CFG\_SPFI\_isLinkTransmitting
  - Status, [996](#)
- CFG\_SPFI\_isLinkTxScramblingEnabled
  - Debug, [1006](#)
- CFG\_SPFI\_isPortBroadcastErrorDetected
  - Port Information, [978](#)
- CFG\_SPFI\_isPortReady
  - Port Information, [979](#)
- CFG\_SPFI\_isRouterTimeoutEnabled
  - Vendor Specific, [952](#)
- CFG\_SPFI\_isRoutingTableEntryDeleteHeaderEnabled
  - Routing Table, [936](#)
- CFG\_SPFI\_isRoutingTableEntryEnabled
  - Routing Table, [937](#)
- CFG\_SPFI\_isVCAddressErrorDetected
  - Errors, [964](#)
- CFG\_SPFI\_isVCContinuousModeEnabled
  - Quality Of Service, [969](#)
- CFG\_SPFI\_isVCInputBufferFull
  - Status, [959](#)
- CFG\_SPFI\_isVCInputBufferHalfFull
  - Status, [959](#)
- CFG\_SPFI\_isVCInputBufferNotEmpty
  - Status, [960](#)
- CFG\_SPFI\_isVCOOutputBufferFull
  - Status, [960](#)
- CFG\_SPFI\_isVCOOutputBufferHalfFull
  - Status, [961](#)
- CFG\_SPFI\_isVCOOverused
  - Status, [961](#)
- CFG\_SPFI\_isVCTimeoutDetected
  - Errors, [965](#)
- CFG\_SPFI\_isVCUnderused
  - Status, [961](#)
- CFG\_SPFI\_isVCVNErrordetected
  - Errors, [965](#)
- CFG\_SPFI\_setAXIBC
  - Port Information, [979](#)
- CFG\_SPFI\_setBroadcastTimeout
  - Network Management, [945](#)
- CFG\_SPFI\_setDeviceIdentifier
  - Network Management, [946](#)
- CFG\_SPFI\_setLaneStandbyReason
  - Control, [1017](#)
- CFG\_SPFI\_setLinkLanesRequestedCount
  - Control, [990](#)
- CFG\_SPFI\_setRouterTimeout
  - Vendor Specific, [952](#)
- CFG\_SPFI\_setRoutingTableEntryPorts
  - Routing Table, [937](#)
- CFG\_SPFI\_setSpFiBitRate
  - Data Signalling Rate, [1035](#)
- CFG\_SPFI\_setSpaceWireBC
  - Network Management, [946](#)
- CFG\_SPFI\_setSpaceWireInterruptBroadcastType
  - Vendor Specific, [953](#)
- CFG\_SPFI\_setTimeCodeBroadcastType
  - Vendor Specific, [953](#)
- CFG\_SPFI\_setTimeSlotBroadcastType
  - Vendor Specific, [954](#)
- CFG\_SPFI\_setVCBandwidthReservation
  - Quality Of Service, [970](#)
- CFG\_SPFI\_setVCPriority
  - Quality Of Service, [970](#)
- CFG\_SPFI\_setVCTimeSlots
  - Time-slots, [973](#)
- CFG\_SPFI\_setVCVN
  - Virtual Network Number, [956](#)
- CFG\_STATUS\_FAILURE
  - Defines, [923](#)
- CFG\_STATUS\_NOT\_COMPATIBLE\_WITH\_THIS\_DEVICE\_FPGA\_VERSION
  - Defines, [923](#)
- CFG\_STATUS\_SUCCESS
  - Defines, [923](#)
- CFG\_clearPortErrors
  - Port Status and Control, [861](#)
- CFG\_disableBroadcastControllerLegacyMode
  - Broadcast Controller, [916](#)
- CFG\_disableExternalTimeCodeSelection
  - Time-codes, [883](#)
- CFG\_disableIdentifySource
  - Interface Mode, [874](#)
- CFG\_disableIdentifySourceOnPort
  - Interface Mode, [874](#)
- CFG\_disableInterfaceMode
  - Interface Mode, [874](#)
- CFG\_disableInterfaceModeOnPort
  - Interface Mode, [875](#)
- CFG\_disablePrecisionTransmitRate
  - Precision Links, [902](#)
- CFG\_disableRxTimestampEventsOnPort
  - Timestamping, [908](#)
- CFG\_disableSpeedChangeEventsOnPort
  - Events, [892](#)
- CFG\_disableStateChangeEventsOnPort
  - Events, [893](#)
- CFG\_disableTimeCodeCounterBypassMode
  - Time-codes, [883](#)
- CFG\_disableTimeCodeEventsOnPort
  - Events, [893](#)
- CFG\_disableTimeCodeMaster
  - Time-codes, [884](#)
- CFG\_enableBroadcastControllerLegacyMode
  - Broadcast Controller, [917](#)
- CFG\_enableExternalTimeCodeSelection
  - Time-codes, [884](#)
- CFG\_enableIdentifySource
  - Interface Mode, [875](#)
- CFG\_enableIdentifySourceOnPort
  - Interface Mode, [876](#)
- CFG\_enableInterfaceMode

- Interface Mode, [876](#)
- CFG\_enableInterfaceModeOnPort
  - Interface Mode, [878](#)
- CFG\_enablePrecisionTransmitRate
  - Precision Links, [903](#)
- CFG\_enableRxTimestampEventsOnPort
  - Timestamping, [909](#)
- CFG\_enableSpeedChangeEventsOnPort
  - Events, [894](#)
- CFG\_enableStateChangeEventsOnPort
  - Events, [894](#)
- CFG\_enableTimeCodeCounterBypassMode
  - Time-codes, [885](#)
- CFG\_enableTimeCodeEventsOnPort
  - Events, [895](#)
- CFG\_enableTimeCodeMaster
  - Time-codes, [885](#)
- CFG\_getBroadcastControllerLegacyModeEnabled
  - Broadcast Controller, [917](#)
- CFG\_getConfigPortErrors
  - Port Status and Control, [861](#)
- CFG\_getDeviceClockRate
  - Periodic, [914](#)
- CFG\_getDeviceIdentificationInfo
  - Device Identifier, [868](#)
- CFG\_getDeviceManufacturerAsString
  - Device Identifier, [869](#)
- CFG\_getDeviceTypeAsString
  - Device Identifier, [869](#)
- CFG\_getExternalPortErrors
  - Port Status and Control, [862](#)
- CFG\_getExternalPortStatus
  - Port Status and Control, [862](#)
- CFG\_getExternalTimeCodeSelectionEnabled
  - Time-codes, [886](#)
- CFG\_getFPGAInfo
  - Hardware, [858](#)
- CFG\_getGeneralPurpose
  - General, [851](#)
- CFG\_getIdentifySourceEnabled
  - Interface Mode, [878](#)
- CFG\_getIdentifySourceOnPortEnabled
  - Interface Mode, [879](#)
- CFG\_getInterfaceModeEnabled
  - Interface Mode, [879](#)
- CFG\_getInterfaceModeOnPortEnabled
  - Interface Mode, [880](#)
- CFG\_getInterruptCodeDistributionPorts
  - Broadcast Controller, [918](#)
- CFG\_getMeasuredLinkSpeed
  - cfg\_api\_generic.h, [1152](#)
- CFG\_getNetworkDiscoveryInfo
  - Device Identifier, [870](#)
- CFG\_getNetworkIdentity
  - General, [852](#)
- CFG\_getPortConnection
  - Port Status and Control, [863](#)
- CFG\_getPortRoutingAddress
  - Interface Mode, [880](#)
- CFG\_getPortStatusControl
  - Port Status and Control, [863](#)
- CFG\_getPortType
  - Port Status and Control, [864](#)
- CFG\_getPrecisionTransmitRate
  - Precision Links, [903](#)
- CFG\_getPrecisionTransmitRateEnabled
  - Precision Links, [904](#)
- CFG\_getPrecisionTransmitRateInUse
  - Precision Links, [904](#)
- CFG\_getPulseGeneratorFrequency
  - Timestamping, [909](#)
- CFG\_getRouterGlobalSettings
  - General, [852](#)
- CFG\_getRoutingTableEntry
  - Group Adaptive Routing, [871](#)
- CFG\_getRxSignallingRate
  - Receive Link Speed, [901](#)
- CFG\_getRxTimestampEventsOnPortEnabled
  - Timestamping, [910](#)
- CFG\_getSpaceWireLinkErrors
  - Port Status and Control, [864](#)
- CFG\_getSpaceWireLinkStatus
  - Port Status and Control, [865](#)
- CFG\_getSpeedChangeEventsOnPortEnabled
  - Events, [895](#)
- CFG\_getStateChangeEventsOnPortEnabled
  - Events, [897](#)
- CFG\_getTimeCodeCounterBypassModeEnabled
  - Time-codes, [886](#)
- CFG\_getTimeCodeDistributionCapablePorts
  - Time-codes, [887](#)
- CFG\_getTimeCodeDistributionPorts
  - Time-codes, [887](#)
- CFG\_getTimeCodeEventsOnPortEnabled
  - Events, [897](#)
- CFG\_getTimeCodeFlagMode
  - Time-codes, [888](#)
- CFG\_getTimeCodeMasterEnabled
  - Time-codes, [888](#)
- CFG\_getTimeCodePeriod
  - Time-codes, [889](#)
- CFG\_getTimeCodeValue
  - Time-codes, [889](#)
- CFG\_getTimestampMethod
  - Timestamping, [910](#)
- CFG\_getTimestampValue
  - Timestamping, [911](#)
- CFG\_getTransmitSignallingRate
  - cfg\_api\_generic.h, [1152](#)
- CFG\_getTxSignallingRate
  - Transmit Link Speed, [899](#)
- CFG\_identify
  - Hardware, [858](#)
- CFG\_injectError
  - Error Injection, [906](#)
- CFG\_injectErrors

- Error Injection, 907
- CFG\_refreshDeviceInfo
  - General, 853
- CFG\_setGeneralPurpose
  - General, 853
- CFG\_setInterruptCodeDistributionPorts
  - Broadcast Controller, 918
- CFG\_setNetworkIdentity
  - General, 854
- CFG\_setPortRoutingAddress
  - Interface Mode, 881
- CFG\_setPrecisionTransmitRate
  - Precision Links, 905
- CFG\_setPulseGeneratorFrequency
  - Timestamping, 911
- CFG\_setRouterGlobalSettings
  - General, 854
- CFG\_setRoutingTableEntry
  - Group Adaptive Routing, 872
- CFG\_setSpaceWireLinkStatus
  - Port Status and Control, 865
- CFG\_setTimeCodeDistributionPorts
  - Time-codes, 890
- CFG\_setTimeCodeFlagMode
  - Time-codes, 890
- CFG\_setTimeCodePeriod
  - Time-codes, 891
- CFG\_setTimestampMethod
  - Timestamping, 912
- CFG\_setTimestampValue
  - Timestamping, 913
- CFG\_setTransmitSignallingRate
  - cfg\_api\_generic.h, 1153
- CFG\_setTxSignallingRate
  - Transmit Link Speed, 900
- CFG\_startLink
  - Port Status and Control, 866
- CFG\_startPeriodicAction
  - Periodic, 914
- CFG\_stopLink
  - Port Status and Control, 866
- CFG\_stopPeriodicAction
  - Periodic, 915
- CFG\_updateDeviceInfo
  - General, 855
- COUNTER\_ACTION
  - Actions, 723
- COUNTER\_ACTION\_ALL
  - Actions, 723
- COUNTER\_ACTION\_COUNT
  - Actions, 723
- COUNTER\_ACTION\_MASK
  - Events, 692
- COUNTER\_ACTION\_NONE
  - Actions, 723
- COUNTER\_ACTION\_RELOAD
  - Actions, 723
- COUNTER\_ACTION\_START
  - Actions, 723
- COUNTER\_ACTION\_STOP
  - Actions, 723
- COUNTER\_EVENT
  - Events, 693
- COUNTER\_EVENT\_ALL
  - Events, 694
- COUNTER\_EVENT\_COUNT
  - Events, 693
- COUNTER\_EVENT\_MASK
  - Events, 692
- COUNTER\_EVENT\_NONE
  - Events, 694
- COUNTER\_EVENT\_RELOAD
  - Events, 694
- COUNTER\_EVENT\_ZERO
  - Events, 693
- COUNTER\_EVENT\_ZERO\_SINGLE
  - Events, 693
- ccsds\_spp\_packet\_library.h, 1131
- ccsds\_spp\_pus\_services.h, 1136
  - CCSDS\_SPP\_PUS\_CreateST02DistributeOnOff↔
    - DeviceCommand, 1136
  - CCSDS\_SPP\_PUS\_CreateST02Distribute↔
    - RegisterDumpCommand, 1137
- cfg\_api\_brick\_mk2.h, 1138
- cfg\_api\_brick\_mk2\_types.h, 1139
  - STAR\_CFG\_BRICK\_MK2\_LINK\_SPEED\_UNIT↔
    - S\_KBPS, 1141
- cfg\_api\_brick\_mk3.h, 1142
- cfg\_api\_brick\_mk3\_types.h, 1143
- cfg\_api\_gbe\_brick\_mk1.h, 1143
- cfg\_api\_generic.h, 1144
  - CFG\_getMeasuredLinkSpeed, 1152
  - CFG\_getTransmitSignallingRate, 1152
  - CFG\_setTransmitSignallingRate, 1153
- cfg\_api\_generic\_common.h, 1154
- cfg\_api\_mk2.h, 1155
  - CFG\_API\_MK2\_MODULE\_NAME, 1158
- cfg\_api\_mk2\_types.h, 1158
- cfg\_api\_pci\_mk2.h, 1159
  - CFG\_API\_PCI\_MK2\_MODULE\_NAME, 1160
- cfg\_api\_pci\_mk2\_types.h, 1160
- cfg\_api\_pcie\_mk2.h, 1161
- cfg\_api\_pxi.h, 1162
- cfg\_api\_remote.h, 1164
- cfg\_api\_router.h, 1165
- cfg\_api\_router\_mk2s.h, 1167
- cfg\_api\_router\_types.h, 1168
- cfg\_api\_spfi.h, 1169
- cfg\_api\_spfi\_types.h, 1178
- Channel Management, 313
  - STAR\_CHANNEL\_DIRECTION, 315
  - STAR\_CHANNEL\_DIRECTION\_IN, 315
  - STAR\_CHANNEL\_DIRECTION\_INOUT, 315
  - STAR\_CHANNEL\_DIRECTION\_OUT, 315
  - STAR\_CHANNEL\_ID, 314
  - STAR\_CHANNEL\_LISTENER\_ID, 314



- STAR\_CHANNEL\_TYPE, 315
- STAR\_CHANNEL\_TYPE\_APPLICATION, 315
- STAR\_CHANNEL\_TYPE\_DEVICE, 315
- STAR\_CHANNEL\_TYPE\_NOT\_OPEN, 315
- STAR\_ChannelEventFunc, 314
- STAR\_closeChannel, 315
- STAR\_getChannelDirection, 316
- STAR\_openChannelBetweenLocalDevices, 316
- STAR\_openChannelToLocalDevice, 316
- STAR\_openChannelToLocalDeviceEx, 317
- STAR\_registerChannelListener, 318
- STAR\_registerChannelListenerForDevice, 319
- STAR\_registerChannelListenerForDriver, 319
- STAR\_unregisterChannelListener, 320
- common.h, 1180
  - STAR\_API\_CC, 1180
  - STAR\_API\_MODULE\_NAME, 1181
- Configuration, 448, 741
  - IF\_MODE, 451
  - IF\_MODE\_RMAP\_DISABLED, 451
  - IF\_MODE\_RMAP\_ENABLED, 451
  - RMAP\_TARGET\_PXI\_IF\_getAddressOffset, 452
  - RMAP\_TARGET\_PXI\_IF\_getAuthCommands, 452
  - RMAP\_TARGET\_PXI\_IF\_getAuthControlMode, 452
  - RMAP\_TARGET\_PXI\_IF\_getAuthKeyRange, 453
  - RMAP\_TARGET\_PXI\_IF\_getAuthLogical↔
    - AddressRange, 453
  - RMAP\_TARGET\_PXI\_IF\_getAuthMemory↔
    - AddressRange, 454
  - RMAP\_TARGET\_PXI\_IF\_getAuthProtocolId, 454
  - RMAP\_TARGET\_PXI\_IF\_getInterfaceMode, 454
  - RMAP\_TARGET\_PXI\_IF\_getStatus, 456
  - RMAP\_TARGET\_PXI\_IF\_readMemory, 456
  - RMAP\_TARGET\_PXI\_IF\_setAddressOffset, 457
  - RMAP\_TARGET\_PXI\_IF\_setAuthCommands, 457
  - RMAP\_TARGET\_PXI\_IF\_setAuthControlMode, 458
  - RMAP\_TARGET\_PXI\_IF\_setAuthKeyRange, 458
  - RMAP\_TARGET\_PXI\_IF\_setAuthLogicalAddress↔
    - Range, 459
  - RMAP\_TARGET\_PXI\_IF\_setAuthMemory↔
    - AddressRange, 459
  - RMAP\_TARGET\_PXI\_IF\_setAuthProtocolId, 459
  - RMAP\_TARGET\_PXI\_IF\_setInterfaceMode, 460
  - RMAP\_TARGET\_PXI\_IF\_writeMemory, 460
  - TARGET\_AUTH\_CMD, 451
  - TARGET\_AUTH\_CMD\_MASK, 451
  - TARGET\_AUTH\_MODE, 451
  - TARGET\_AUTH\_MODE\_AUTOMATIC, 451
  - TARGET\_AUTH\_MODE\_MANUAL, 451
  - TARGET\_STATUS, 451
  - TRIGGER\_BRICK\_MK3\_disableCounterAuto↔
    - Reload, 753
  - TRIGGER\_BRICK\_MK3\_disableCounterLoad↔
    - Zero, 753
  - TRIGGER\_BRICK\_MK3\_disableCounterStart↔
    - Mode, 753
  - TRIGGER\_BRICK\_MK3\_disableCounterStart↔
    - StopMode, 754
  - TRIGGER\_BRICK\_MK3\_disableCounterTrigger↔
    - Count, 754
  - TRIGGER\_BRICK\_MK3\_disableExtTriggerEdge↔
    - DetectMode, 754
  - TRIGGER\_BRICK\_MK3\_disableExtTriggerInvert, 755
  - TRIGGER\_BRICK\_MK3\_disableExtTriggerOutput, 755
  - TRIGGER\_BRICK\_MK3\_disablePortSpill, 756
  - TRIGGER\_BRICK\_MK3\_disablePortTransmit↔
    - PacketMode, 756
  - TRIGGER\_BRICK\_MK3\_disableTrigger, 757
  - TRIGGER\_BRICK\_MK3\_enableCounterAuto↔
    - Reload, 757
  - TRIGGER\_BRICK\_MK3\_enableCounterLoadZero, 758
  - TRIGGER\_BRICK\_MK3\_enableCounterStart↔
    - Mode, 758
  - TRIGGER\_BRICK\_MK3\_enableCounterStart↔
    - StopMode, 759
  - TRIGGER\_BRICK\_MK3\_enableCounterTrigger↔
    - Count, 759
  - TRIGGER\_BRICK\_MK3\_enableExtTriggerEdge↔
    - DetectMode, 760
  - TRIGGER\_BRICK\_MK3\_enableExtTriggerInvert, 760
  - TRIGGER\_BRICK\_MK3\_enableExtTriggerOutput, 760
  - TRIGGER\_BRICK\_MK3\_enablePortSpill, 761
  - TRIGGER\_BRICK\_MK3\_enablePortTransmit↔
    - PacketMode, 761
  - TRIGGER\_BRICK\_MK3\_enableTrigger, 762
  - TRIGGER\_BRICK\_MK3\_forceCounterReload, 762
  - TRIGGER\_BRICK\_MK3\_forceTrigger, 763
  - TRIGGER\_BRICK\_MK3\_getCounterAutoReload↔
    - Enabled, 763
  - TRIGGER\_BRICK\_MK3\_getCounterLoadZero↔
    - Enabled, 763
  - TRIGGER\_BRICK\_MK3\_getCounterReloadValue, 765
  - TRIGGER\_BRICK\_MK3\_getCounterStartMode↔
    - Enabled, 765
  - TRIGGER\_BRICK\_MK3\_getCounterStartStop↔
    - ModeEnabled, 766
  - TRIGGER\_BRICK\_MK3\_getCounterTrigger↔
    - CountEnabled, 766
  - TRIGGER\_BRICK\_MK3\_getCounterValue, 766
  - TRIGGER\_BRICK\_MK3\_getDeviceClockRate, 767
  - TRIGGER\_BRICK\_MK3\_getExtTriggerEdge↔
    - DetectModeEnabled, 767
  - TRIGGER\_BRICK\_MK3\_getExtTriggerExtend, 767
  - TRIGGER\_BRICK\_MK3\_getExtTriggerInvert↔
    - Enabled, 768
  - TRIGGER\_BRICK\_MK3\_getExtTriggerOutput↔

- Enabled, [768](#)
- TRIGGER\_BRICK\_MK3\_getPortSpillEnabled, [769](#)
- TRIGGER\_BRICK\_MK3\_getPortTransmitPacket↔  
ModeEnabled, [769](#)
- TRIGGER\_BRICK\_MK3\_getTriggerAndMask, [769](#)
- TRIGGER\_BRICK\_MK3\_getTriggerEnabled, [771](#)
- TRIGGER\_BRICK\_MK3\_getTriggerInputMode,  
[771](#)
- TRIGGER\_BRICK\_MK3\_getTriggerInvertMask,  
[772](#)
- TRIGGER\_BRICK\_MK3\_getTriggerMatrix, [772](#)
- TRIGGER\_BRICK\_MK3\_reset, [772](#)
- TRIGGER\_BRICK\_MK3\_setCounterReloadValue,  
[773](#)
- TRIGGER\_BRICK\_MK3\_setExtTriggerExtend, [773](#)
- TRIGGER\_BRICK\_MK3\_setTriggerAndMask, [774](#)
- TRIGGER\_BRICK\_MK3\_setTriggerInputMode,  
[774](#)
- TRIGGER\_BRICK\_MK3\_setTriggerInvertMask,  
[775](#)
- TRIGGER\_INPUT\_MODE, [752](#)
- TRIGGER\_INPUT\_MODE\_AND, [752](#)
- TRIGGER\_INPUT\_MODE\_OR, [752](#)
- TRIGGER\_PCIE\_IF\_disableCounterAutoReload,  
[775](#)
- TRIGGER\_PCIE\_IF\_disableCounterLoadZero,  
[776](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartMode,  
[776](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartStop↔  
Mode, [776](#)
- TRIGGER\_PCIE\_IF\_disableCounterTriggerCount,  
[777](#)
- TRIGGER\_PCIE\_IF\_disableTrigger, [777](#)
- TRIGGER\_PCIE\_IF\_enableCounterAutoReload,  
[778](#)
- TRIGGER\_PCIE\_IF\_enableCounterLoadZero, [778](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartMode,  
[778](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartStop↔  
Mode, [779](#)
- TRIGGER\_PCIE\_IF\_enableCounterTriggerCount,  
[779](#)
- TRIGGER\_PCIE\_IF\_enableTrigger, [780](#)
- TRIGGER\_PCIE\_IF\_forceCounterReload, [780](#)
- TRIGGER\_PCIE\_IF\_forceTrigger, [780](#)
- TRIGGER\_PCIE\_IF\_getCounterAutoReload↔  
Enabled, [781](#)
- TRIGGER\_PCIE\_IF\_getCounterLoadZero↔  
Enabled, [781](#)
- TRIGGER\_PCIE\_IF\_getCounterReloadValue, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterStartMode↔  
Enabled, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterStartStopMode↔  
Enabled, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterTriggerCount↔  
Enabled, [784](#)
- TRIGGER\_PCIE\_IF\_getCounterValue, [784](#)
- TRIGGER\_PCIE\_IF\_getDeviceClockRate, [784](#)
- TRIGGER\_PCIE\_IF\_getTriggerAndMask, [785](#)
- TRIGGER\_PCIE\_IF\_getTriggerEnabled, [785](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputMode, [786](#)
- TRIGGER\_PCIE\_IF\_getTriggerInvertMask, [786](#)
- TRIGGER\_PCIE\_IF\_reset, [787](#)
- TRIGGER\_PCIE\_IF\_setCounterReloadValue, [787](#)
- TRIGGER\_PCIE\_IF\_setTriggerAndMask, [787](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputMode, [788](#)
- TRIGGER\_PCIE\_IF\_setTriggerInvertMask, [788](#)
- TRIGGER\_PCIE\_MK2\_disableCounterAuto↔  
Reload, [789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterLoadZero,  
[789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterStartMode,  
[789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterStart↔  
StopMode, [791](#)
- TRIGGER\_PCIE\_MK2\_disableCounterTrigger↔  
Count, [791](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerEdge↔  
DetectMode, [791](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerInvert,  
[792](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerOutput,  
[792](#)
- TRIGGER\_PCIE\_MK2\_disablePortSpill, [793](#)
- TRIGGER\_PCIE\_MK2\_disablePortTransmit↔  
PacketMode, [793](#)
- TRIGGER\_PCIE\_MK2\_disableTrigger, [794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterAuto↔  
Reload, [794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterLoadZero,  
[794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterStartMode,  
[795](#)
- TRIGGER\_PCIE\_MK2\_enableCounterStartStop↔  
Mode, [795](#)
- TRIGGER\_PCIE\_MK2\_enableCounterTrigger↔  
Count, [796](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerEdge↔  
DetectMode, [796](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerInvert,  
[797](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerOutput,  
[797](#)
- TRIGGER\_PCIE\_MK2\_enablePortSpill, [798](#)
- TRIGGER\_PCIE\_MK2\_enablePortTransmit↔  
PacketMode, [798](#)
- TRIGGER\_PCIE\_MK2\_enableTrigger, [799](#)
- TRIGGER\_PCIE\_MK2\_forceCounterReload, [799](#)
- TRIGGER\_PCIE\_MK2\_forceTrigger, [800](#)
- TRIGGER\_PCIE\_MK2\_getCounterAutoReload↔  
Enabled, [800](#)
- TRIGGER\_PCIE\_MK2\_getCounterLoadZero↔  
Enabled, [800](#)
- TRIGGER\_PCIE\_MK2\_getCounterReloadValue,  
[801](#)

- TRIGGER\_PCIE\_MK2\_getCounterStartMode↔  
Enabled, 801
- TRIGGER\_PCIE\_MK2\_getCounterStartStop↔  
ModeEnabled, 802
- TRIGGER\_PCIE\_MK2\_getCounterTriggerCount↔  
Enabled, 802
- TRIGGER\_PCIE\_MK2\_getCounterValue, 802
- TRIGGER\_PCIE\_MK2\_getDeviceClockRate, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerEdge↔  
DetectModeEnabled, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerExtend, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerInvert↔  
Enabled, 804
- TRIGGER\_PCIE\_MK2\_getExtTriggerOutput↔  
Enabled, 804
- TRIGGER\_PCIE\_MK2\_getPortSpillEnabled, 805
- TRIGGER\_PCIE\_MK2\_getPortTransmitPacket↔  
ModeEnabled, 805
- TRIGGER\_PCIE\_MK2\_getTriggerAndMask, 805
- TRIGGER\_PCIE\_MK2\_getTriggerEnabled, 807
- TRIGGER\_PCIE\_MK2\_getTriggerInputMode, 807
- TRIGGER\_PCIE\_MK2\_getTriggerInvertMask, 808
- TRIGGER\_PCIE\_MK2\_getTriggerMatrix, 808
- TRIGGER\_PCIE\_MK2\_reset, 808
- TRIGGER\_PCIE\_MK2\_setCounterReloadValue,  
809
- TRIGGER\_PCIE\_MK2\_setExtTriggerExtend, 809
- TRIGGER\_PCIE\_MK2\_setTriggerAndMask, 810
- TRIGGER\_PCIE\_MK2\_setTriggerInputMode, 810
- TRIGGER\_PCIE\_MK2\_setTriggerInvertMask, 811
- TRIGGER\_PCIE\_getTriggerMatrix, 775
- TRIGGER\_PXI\_IF\_disableCounterAutoReload,  
811
- TRIGGER\_PXI\_IF\_disableCounterLoadZero, 812
- TRIGGER\_PXI\_IF\_disableCounterStartMode, 812
- TRIGGER\_PXI\_IF\_disableCounterStartStopMode,  
812
- TRIGGER\_PXI\_IF\_disableCounterTriggerCount,  
813
- TRIGGER\_PXI\_IF\_disableExtTriggerEdge↔  
DetectMode, 813
- TRIGGER\_PXI\_IF\_disableExtTriggerInvert, 813
- TRIGGER\_PXI\_IF\_disableExtTriggerOutput, 814
- TRIGGER\_PXI\_IF\_disablePortTransmitPacket↔  
Mode, 814
- TRIGGER\_PXI\_IF\_disableTrigger, 815
- TRIGGER\_PXI\_IF\_enableCounterAutoReload,  
815
- TRIGGER\_PXI\_IF\_enableCounterLoadZero, 816
- TRIGGER\_PXI\_IF\_enableCounterStartMode, 816
- TRIGGER\_PXI\_IF\_enableCounterStartStopMode,  
816
- TRIGGER\_PXI\_IF\_enableCounterTriggerCount,  
817
- TRIGGER\_PXI\_IF\_enableExtTriggerEdgeDetect↔  
Mode, 817
- TRIGGER\_PXI\_IF\_enableExtTriggerInvert, 818
- TRIGGER\_PXI\_IF\_enableExtTriggerOutput, 818
- TRIGGER\_PXI\_IF\_enablePortTransmitPacket↔  
Mode, 819
- TRIGGER\_PXI\_IF\_enableTrigger, 819
- TRIGGER\_PXI\_IF\_forceCounterReload, 820
- TRIGGER\_PXI\_IF\_forceTrigger, 820
- TRIGGER\_PXI\_IF\_getCounterAutoReload↔  
Enabled, 821
- TRIGGER\_PXI\_IF\_getCounterLoadZeroEnabled,  
821
- TRIGGER\_PXI\_IF\_getCounterReloadValue, 821
- TRIGGER\_PXI\_IF\_getCounterStartModeEnabled,  
823
- TRIGGER\_PXI\_IF\_getCounterStartStopMode↔  
Enabled, 823
- TRIGGER\_PXI\_IF\_getCounterTriggerCount↔  
Enabled, 823
- TRIGGER\_PXI\_IF\_getCounterValue, 824
- TRIGGER\_PXI\_IF\_getDeviceClockRate, 824
- TRIGGER\_PXI\_IF\_getExtTriggerEdgeDetect↔  
ModeEnabled, 825
- TRIGGER\_PXI\_IF\_getExtTriggerExtend, 825
- TRIGGER\_PXI\_IF\_getExtTriggerInvertEnabled,  
825
- TRIGGER\_PXI\_IF\_getExtTriggerOutputEnabled,  
826
- TRIGGER\_PXI\_IF\_getPortTransmitPacketMode↔  
Enabled, 826
- TRIGGER\_PXI\_IF\_getTriggerAndMask, 826
- TRIGGER\_PXI\_IF\_getTriggerEnabled, 827
- TRIGGER\_PXI\_IF\_getTriggerInputMode, 827
- TRIGGER\_PXI\_IF\_getTriggerInvertMask, 828
- TRIGGER\_PXI\_IF\_getTriggerMatrix, 828
- TRIGGER\_PXI\_IF\_reset, 828
- TRIGGER\_PXI\_IF\_setCounterReloadValue, 830
- TRIGGER\_PXI\_IF\_setExtTriggerExtend, 830
- TRIGGER\_PXI\_IF\_setTriggerAndMask, 831
- TRIGGER\_PXI\_IF\_setTriggerInputMode, 831
- TRIGGER\_PXI\_IF\_setTriggerInvertMask, 832
- TRIGGER\_PXI\_ROUTER\_disableCounterAuto↔  
Reload, 832
- TRIGGER\_PXI\_ROUTER\_disableCounterLoad↔  
Zero, 833
- TRIGGER\_PXI\_ROUTER\_disableCounterStart↔  
Mode, 833
- TRIGGER\_PXI\_ROUTER\_disableCounterStart↔  
StopMode, 833
- TRIGGER\_PXI\_ROUTER\_disableCounter↔  
TriggerCount, 835
- TRIGGER\_PXI\_ROUTER\_disablePortTransmit↔  
PacketMode, 835
- TRIGGER\_PXI\_ROUTER\_disableTrigger, 835
- TRIGGER\_PXI\_ROUTER\_enableCounterAuto↔  
Reload, 836
- TRIGGER\_PXI\_ROUTER\_enableCounterLoad↔  
Zero, 836
- TRIGGER\_PXI\_ROUTER\_enableCounterStart↔  
Mode, 837
- TRIGGER\_PXI\_ROUTER\_enableCounterStart↔

- StopMode, [837](#)
- TRIGGER\_PXI\_ROUTER\_enableCounter↔
  - TriggerCount, [838](#)
- TRIGGER\_PXI\_ROUTER\_enablePortTransmit↔
  - PacketMode, [838](#)
- TRIGGER\_PXI\_ROUTER\_enableTrigger, [839](#)
- TRIGGER\_PXI\_ROUTER\_forceCounterReload, [839](#)
- TRIGGER\_PXI\_ROUTER\_forceTrigger, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterAuto↔
  - ReloadEnabled, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterLoadZero↔
  - Enabled, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterReload↔
  - Value, [842](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStart↔
  - ModeEnabled, [842](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartStop↔
  - ModeEnabled, [843](#)
- TRIGGER\_PXI\_ROUTER\_getCounterTrigger↔
  - CountEnabled, [843](#)
- TRIGGER\_PXI\_ROUTER\_getCounterValue, [843](#)
- TRIGGER\_PXI\_ROUTER\_getDeviceClockRate, [844](#)
- TRIGGER\_PXI\_ROUTER\_getPortTransmit↔
  - PacketModeEnabled, [844](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerAndMask, [844](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerEnabled, [846](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerInputMode, [846](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerInvertMask, [847](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerMatrix, [847](#)
- TRIGGER\_PXI\_ROUTER\_reset, [847](#)
- TRIGGER\_PXI\_ROUTER\_setCounterReload↔
  - Value, [848](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerAndMask, [848](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerInputMode, [849](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerInvertMask, [849](#)
- Control, [982](#), [1010](#)
  - CFG\_SPFI\_disableLaneErrorRecovery, [1011](#)
  - CFG\_SPFI\_disableLaneReset, [1011](#)
  - CFG\_SPFI\_disableLaneRx, [1011](#)
  - CFG\_SPFI\_disableLaneTx, [1012](#)
  - CFG\_SPFI\_disableLinkAutoStart, [983](#)
  - CFG\_SPFI\_disableLinkInterfaceReset, [983](#)
  - CFG\_SPFI\_disableLinkReset, [983](#)
  - CFG\_SPFI\_disableLinkStart, [984](#)
  - CFG\_SPFI\_enableLaneErrorRecovery, [1012](#)
  - CFG\_SPFI\_enableLaneReset, [1013](#)
  - CFG\_SPFI\_enableLaneRx, [1013](#)
  - CFG\_SPFI\_enableLaneTx, [1014](#)
  - CFG\_SPFI\_enableLinkAutoStart, [984](#)
  - CFG\_SPFI\_enableLinkInterfaceReset, [985](#)
  - CFG\_SPFI\_enableLinkReset, [985](#)
  - CFG\_SPFI\_enableLinkStart, [985](#)
  - CFG\_SPFI\_getLaneControlSettings, [1014](#)
  - CFG\_SPFI\_getLaneStandbyReason, [1015](#)
  - CFG\_SPFI\_getLinkControlSettings, [986](#)
  - CFG\_SPFI\_getLinkLanesRequestedCount, [986](#)
  - CFG\_SPFI\_isLaneErrorRecoveryEnabled, [1015](#)
  - CFG\_SPFI\_isLaneResetEnabled, [1016](#)
  - CFG\_SPFI\_isLaneRxEnabled, [1016](#)
  - CFG\_SPFI\_isLaneTxEnabled, [1017](#)
  - CFG\_SPFI\_isLinkAutoStartEnabled, [988](#)
  - CFG\_SPFI\_isLinkInterfaceResetEnabled, [988](#)
  - CFG\_SPFI\_isLinkResetEnabled, [989](#)
  - CFG\_SPFI\_isLinkStartEnabled, [989](#)
  - CFG\_SPFI\_setLaneStandbyReason, [1017](#)
  - CFG\_SPFI\_setLinkLanesRequestedCount, [990](#)
- Counters, [502](#)
  - TRIGGER\_CFG\_disableCounterAutoReload, [503](#)
  - TRIGGER\_CFG\_disableCounterLoadZero, [504](#)
  - TRIGGER\_CFG\_disableCounterStartMode, [504](#)
  - TRIGGER\_CFG\_disableCounterStartStopMode, [505](#)
  - TRIGGER\_CFG\_disableCounterTriggerCount, [505](#)
  - TRIGGER\_CFG\_enableCounterAutoReload, [506](#)
  - TRIGGER\_CFG\_enableCounterLoadZero, [506](#)
  - TRIGGER\_CFG\_enableCounterStartMode, [507](#)
  - TRIGGER\_CFG\_enableCounterStartStopMode, [507](#)
  - TRIGGER\_CFG\_enableCounterTriggerCount, [508](#)
  - TRIGGER\_CFG\_forceCounterReload, [508](#)
  - TRIGGER\_CFG\_getCounterAutoReloadEnabled, [509](#)
  - TRIGGER\_CFG\_getCounterInputEvents, [509](#)
  - TRIGGER\_CFG\_getCounterLoadZeroEnabled, [510](#)
  - TRIGGER\_CFG\_getCounterOutputActions, [510](#)
  - TRIGGER\_CFG\_getCounterReloadValue, [511](#)
  - TRIGGER\_CFG\_getCounterStartModeEnabled, [511](#)
  - TRIGGER\_CFG\_getCounterStartStopMode↔
    - Enabled, [512](#)
  - TRIGGER\_CFG\_getCounterTriggerCountEnabled, [512](#)
  - TRIGGER\_CFG\_getCounterValue, [512](#)
  - TRIGGER\_CFG\_getNumberOfCounters, [513](#)
  - TRIGGER\_CFG\_setCounterInputEvents, [513](#)
  - TRIGGER\_CFG\_setCounterOutputActions, [514](#)
  - TRIGGER\_CFG\_setCounterReloadValue, [514](#)
- Data Rate, [974](#)
  - CFG\_SPFI\_getVCDDataRate, [974](#)
- Data Signalling Rate, [1034](#)
  - CFG\_SPFI\_destroySpFiBitRates, [1034](#)
  - CFG\_SPFI\_getSpFiBitRate, [1034](#)
  - CFG\_SPFI\_getSpFiBitRates, [1035](#)
  - CFG\_SPFI\_setSpFiBitRate, [1035](#)
- Debug, [1001](#)
  - CFG\_SPFI\_disableLinkNearEndLoopback, [1001](#)
  - CFG\_SPFI\_disableLinkRxScrambling, [1002](#)

- CFG\_SPFI\_disableLinkTxScrambling, 1002
- CFG\_SPFI\_enableLinkNearEndLoopback, 1003
- CFG\_SPFI\_enableLinkRxScrambling, 1003
- CFG\_SPFI\_enableLinkTxScrambling, 1003
- CFG\_SPFI\_getLinkDebugSettings, 1004
- CFG\_SPFI\_isLinkNearEndLoopbackEnabled, 1004
- CFG\_SPFI\_isLinkRxScramblingEnabled, 1006
- CFG\_SPFI\_isLinkTxScramblingEnabled, 1006
- Defines, 555, 920, 1043
  - CFG\_STATUS\_FAILURE, 923
  - CFG\_STATUS\_NOT\_COMPATIBLE\_WITH\_TH←IS\_DEVICES\_FPGA\_VERSION, 923
  - CFG\_STATUS\_SUCCESS, 923
- Device and Driver Management, 282
  - STAR\_BUS\_CPCI, 296
  - STAR\_BUS\_PCI, 296
  - STAR\_BUS\_PCIE, 296
  - STAR\_BUS\_TCP, 296
  - STAR\_BUS\_TYPE, 296
  - STAR\_BUS\_UNKNOWN, 296
  - STAR\_BUS\_USB, 296
  - STAR\_BUS\_VIRTUAL, 296
  - STAR\_CHANNEL\_MASK, 286
  - STAR\_DEVICE\_ALL, 286
  - STAR\_DEVICE\_BRICK\_MK2, 287
  - STAR\_DEVICE\_BRICK\_MK3, 287
  - STAR\_DEVICE\_BRICK\_MK4, 287
  - STAR\_DEVICE\_CONFIG\_NOT\_SUPPORTED, 287
  - STAR\_DEVICE\_CONFIG\_SUPPORTED, 287
  - STAR\_DEVICE\_CONFORMANCE\_TESTER, 288
  - STAR\_DEVICE\_CONFORMANCE\_TESTER\_M←K2, 288
  - STAR\_DEVICE\_CPCI\_MK2, 288
  - STAR\_DEVICE\_EGSE, 288
  - STAR\_DEVICE\_EGSE\_MK2, 288
  - STAR\_DEVICE\_ID, 295
  - STAR\_DEVICE\_IP\_TUNNEL, 288
  - STAR\_DEVICE\_LINK\_ANALYSER, 289
  - STAR\_DEVICE\_LINK\_ANALYSER\_MK2, 289
  - STAR\_DEVICE\_LINK\_ANALYSER\_MK3, 289
  - STAR\_DEVICE\_LISTENER\_ID, 295
  - STAR\_DEVICE\_PCI, 289
  - STAR\_DEVICE\_PCI\_MK2, 289
  - STAR\_DEVICE\_PCIE, 289
  - STAR\_DEVICE\_PCIE\_MK2, 290
  - STAR\_DEVICE\_PXI\_INTERFACE, 290
  - STAR\_DEVICE\_PXI\_INTERFACE\_MK2, 290
  - STAR\_DEVICE\_PXI\_RMAP, 290
  - STAR\_DEVICE\_PXI\_RMAP\_MK2, 290
  - STAR\_DEVICE\_PXI\_ROUTER\_12, 291
  - STAR\_DEVICE\_PXI\_ROUTER\_8, 291
  - STAR\_DEVICE\_PXI\_ROUTER\_MK2, 291
  - STAR\_DEVICE\_RECORDER, 291
  - STAR\_DEVICE\_RECORDER\_MK2, 291
  - STAR\_DEVICE\_ROUTER\_MK2S, 292
  - STAR\_DEVICE\_ROUTER\_USB, 292
  - STAR\_DEVICE\_RTC, 292
  - STAR\_DEVICE\_SERIAL\_SIZE, 292
  - STAR\_DEVICE\_SPLT, 292
  - STAR\_DEVICE\_STAR\_FIRE, 292
  - STAR\_DEVICE\_STAR\_FIRE\_MK3, 293
  - STAR\_DEVICE\_STAR\_ULTRA\_PCIE\_INTERF←ACE, 293
  - STAR\_DEVICE\_STAR\_ULTRA\_PCIE\_SL\_ROU←TER, 293
  - STAR\_DEVICE\_TRAFFIC\_GENERATOR, 293
  - STAR\_DEVICE\_TXRX\_NOT\_SUPPORTED, 293
  - STAR\_DEVICE\_TXRX\_SUPPORTED, 293
  - STAR\_DEVICE\_TYPE, 294
  - STAR\_DEVICE\_UNKNOWN, 294
  - STAR\_DEVICE\_USB\_BRICK, 294
  - STAR\_DEVICE\_WBS\_II, 294
  - STAR\_DRIVER\_ID, 295
  - STAR\_DRIVER\_INVALID, 296
  - STAR\_DRIVER\_LISTENER\_ID, 295
  - STAR\_DRIVER\_TYPE, 296
  - STAR\_DRIVER\_TYPE\_PCI, 296
  - STAR\_DRIVER\_TYPE\_PEP\_PCIE, 296
  - STAR\_DRIVER\_TYPE\_TCPIP, 296
  - STAR\_DRIVER\_TYPE\_ULTRA\_PCIE, 296
  - STAR\_DRIVER\_TYPE\_USB, 296
  - STAR\_DRIVER\_TYPE\_VIRTUAL, 296
  - STAR\_DeviceEventFunc, 295
  - STAR\_DriverEventFunc, 295
  - STAR\_destroyDeviceList, 297
  - STAR\_destroyDriverList, 298
  - STAR\_getDeviceBusType, 298
  - STAR\_getDeviceBusTypeAsString, 298
  - STAR\_getDeviceChannels, 299
  - STAR\_getDeviceConfigCapabilities, 299
  - STAR\_getDeviceDriver, 299
  - STAR\_getDeviceFirmwareVersion, 300
  - STAR\_getDeviceIndex, 300
  - STAR\_getDeviceList, 300
  - STAR\_getDeviceListForDriver, 302
  - STAR\_getDeviceListForType, 302
  - STAR\_getDeviceListForTypes, 303
  - STAR\_getDeviceName, 303
  - STAR\_getDeviceSerialNumber, 304
  - STAR\_getDeviceTxRxCapabilities, 304
  - STAR\_getDeviceType, 306
  - STAR\_getDeviceTypeAsString, 306
  - STAR\_getDriverList, 306
  - STAR\_getDriverType, 308
  - STAR\_getDriverVersion, 308
  - STAR\_getLocalDeviceAttachedToChannel, 309
  - STAR\_isChannelOpen, 309
  - STAR\_registerDeviceListener, 309
  - STAR\_registerDeviceListenerForDevice, 310
  - STAR\_registerDeviceListenerForDriver, 310
  - STAR\_registerDriverListener, 310
  - STAR\_resetDevice, 311
  - STAR\_resetDeviceName, 311
  - STAR\_setDeviceName, 311



- STAR\_unregisterDeviceListener, 312
- STAR\_unregisterDriverListener, 312
- Device Identification, 928
  - CFG\_SPFI\_getDeviceIdentificationInfo, 928
  - CFG\_SPFI\_getDeviceManufacturer, 929
  - CFG\_SPFI\_getDeviceManufacturerAsString, 929
  - CFG\_SPFI\_getDeviceType, 930
  - CFG\_SPFI\_getDeviceTypeAsString, 930
  - CFG\_SPFI\_getDeviceVersion, 931
- Device Identifier, 868
  - CFG\_getDeviceIdentificationInfo, 868
  - CFG\_getDeviceManufacturerAsString, 869
  - CFG\_getDeviceTypeAsString, 869
  - CFG\_getNetworkDiscoveryInfo, 870
- Device Information, 581, 637, 925
  - CFG\_MK2\_getHardwareInfo, 582
  - CFG\_MK2\_hardwareInfoToString, 583
  - CFG\_MK2\_identify, 583
  - CFG\_ROUTER\_getDeviceIdentificationInfo, 640
  - CFG\_ROUTER\_getDeviceManufacturerAsString, 640
  - CFG\_ROUTER\_getDeviceTypeAsString, 641
  - CFG\_ROUTER\_getNetworkDiscoveryInfo, 641
  - CFG\_SPFI\_getDeviceInformation, 925
  - CFG\_SPFI\_getDeviceNodeType, 926
  - CFG\_SPFI\_getDevicePortCount, 926
  - CFG\_SPFI\_getDevicePortCountByType, 927
  - STAR\_CFG\_DEVICE\_STR\_MAX\_LEN, 639
  - STAR\_CFG\_DEVICE\_TYPE, 640
  - STAR\_CFG\_DEVICE\_TYPE\_INVALID, 640
  - STAR\_CFG\_DEVICE\_TYPE\_ROUTER, 640
  - STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN, 640
  - STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN, 639
- Device Information & Identification, 924
- EXT\_TRIGGER\_ACTION
  - Actions, 723
- EXT\_TRIGGER\_ACTION\_ALL
  - Actions, 724
- EXT\_TRIGGER\_ACTION\_MASK
  - Events, 692
- EXT\_TRIGGER\_ACTION\_NONE
  - Actions, 724
- EXT\_TRIGGER\_ACTION\_OUT
  - Actions, 724
- EXT\_TRIGGER\_EVENT
  - Events, 694
- EXT\_TRIGGER\_EVENT\_ALL
  - Events, 694
- EXT\_TRIGGER\_EVENT\_IN
  - Events, 694
- EXT\_TRIGGER\_EVENT\_MASK
  - Events, 692
- EXT\_TRIGGER\_EVENT\_NONE
  - Events, 694
- Error Injection, 398, 407, 431, 592, 906
  - CFG\_BRICK\_MK2\_injectErrors, 407
  - CFG\_MK2\_injectError, 593
  - CFG\_PCIE\_MK2\_injectErrors, 398
  - CFG\_PXI\_injectError, 431
  - CFG\_injectError, 906
  - CFG\_injectErrors, 907
  - SPW\_ERROR, 592
  - SPW\_ERROR\_DECREMENT\_CREDIT, 593
  - SPW\_ERROR\_DISCONNECT, 592
  - SPW\_ERROR\_ESCAPE, 592
  - SPW\_ERROR\_INCREMENT\_CREDIT, 593
  - SPW\_ERROR\_INSERT\_FCT, 592
  - SPW\_ERROR\_PARITY, 592
  - SPW\_ERROR\_SUPPRESS\_FCT, 592
- Errors, 963
  - CFG\_SPFI\_clearVCErrors, 963
  - CFG\_SPFI\_getVCErrors, 964
  - CFG\_SPFI\_isVCAAddressErrorDetected, 964
  - CFG\_SPFI\_isVCTimeoutDetected, 965
  - CFG\_SPFI\_isVCVNErrorsDetected, 965
- Ethernet Devices and Drivers, 381
  - STAR\_closeEthernetDevice, 381
  - STAR\_closeEthernetDriver, 382
  - STAR\_destroyIPAddresses, 382
  - STAR\_getDeviceIPAddress, 382
  - STAR\_getIPAddresses, 383
  - STAR\_openEthernetDevice, 383
  - STAR\_openEthernetDriver, 383
- Events, 408, 437, 560, 688, 892, 997, 1028
  - CFG\_BRICK\_MK2\_disableSpeedChangeEventsOnPort, 408
  - CFG\_BRICK\_MK2\_disableStateChangeEventsOnPort, 410
  - CFG\_BRICK\_MK2\_enableSpeedChangeEventsOnPort, 410
  - CFG\_BRICK\_MK2\_enableStateChangeEventsOnPort, 411
  - CFG\_BRICK\_MK2\_getSpeedChangeEventsOnPortEnabled, 411
  - CFG\_BRICK\_MK2\_getStateChangeEventsOnPortEnabled, 412
  - CFG\_MK2\_disableSpeedChangeEventsOnPort, 561
  - CFG\_MK2\_disableStateChangeEventsOnPort, 561
  - CFG\_MK2\_disableTimeCodeEventsOnPort, 562
  - CFG\_MK2\_enableSpeedChangeEventsOnPort, 562
  - CFG\_MK2\_enableStateChangeEventsOnPort, 563
  - CFG\_MK2\_enableTimeCodeEventsOnPort, 563
  - CFG\_MK2\_getSpeedChangeEventsOnPortEnabled, 564
  - CFG\_MK2\_getStateChangeEventsOnPortEnabled, 564
  - CFG\_MK2\_getTimeCodeEventsOnPortEnabled, 565
  - CFG\_PXI\_disableSpeedChangeEventsOnPort, 437
  - CFG\_PXI\_disableStateChangeEventsOnPort, 438
  - CFG\_PXI\_disableTimeCodeEventsOnPort, 438

- CFG\_PXI\_enableSpeedChangeEventsOnPort, 439
- CFG\_PXI\_enableStateChangeEventsOnPort, 439
- CFG\_PXI\_enableTimeCodeEventsOnPort, 439
- CFG\_PXI\_getSpeedChangeEventsOnPort↔ Enabled, 440
- CFG\_PXI\_getStateChangeEventsOnPortEnabled, 440
- CFG\_PXI\_getTimeCodeEventsOnPortEnabled, 441
- CFG\_SPFI\_clearLaneEvents, 1028
- CFG\_SPFI\_clearLinkEvents, 997
- CFG\_SPFI\_getLaneDisconnectCount, 1029
- CFG\_SPFI\_getLaneEvents, 1029
- CFG\_SPFI\_getLaneRxErrorCount, 1030
- CFG\_SPFI\_getLinkEvents, 998
- CFG\_SPFI\_getLinkRetryCount, 998
- CFG\_SPFI\_isLaneFarEndErrorBurstDetected, 1030
- CFG\_SPFI\_isLaneFarEndINIT1RxInActive↔ Detected, 1031
- CFG\_SPFI\_isLaneFarEndLOSDetected, 1031
- CFG\_SPFI\_isLaneFarEndStandbyDetected, 1032
- CFG\_SPFI\_isLaneINIT1RxDetected, 1032
- CFG\_SPFI\_isLaneINIT2RxDetected, 1032
- CFG\_SPFI\_isLaneInitTimeoutDetected, 1033
- CFG\_SPFI\_isLinkCRC16ErrorDetected, 999
- CFG\_SPFI\_isLinkCRC8ErrorDetected, 999
- CFG\_SPFI\_isLinkFarEndResetDetected, 1000
- CFG\_SPFI\_isLinkFrameErrorDetected, 1000
- CFG\_disableSpeedChangeEventsOnPort, 892
- CFG\_disableStateChangeEventsOnPort, 893
- CFG\_disableTimeCodeEventsOnPort, 893
- CFG\_enableSpeedChangeEventsOnPort, 894
- CFG\_enableStateChangeEventsOnPort, 894
- CFG\_enableTimeCodeEventsOnPort, 895
- CFG\_getSpeedChangeEventsOnPortEnabled, 895
- CFG\_getStateChangeEventsOnPortEnabled, 897
- CFG\_getTimeCodeEventsOnPortEnabled, 897
- COUNTER\_ACTION\_MASK, 692
- COUNTER\_EVENT, 693
- COUNTER\_EVENT\_ALL, 694
- COUNTER\_EVENT\_COUNT, 693
- COUNTER\_EVENT\_MASK, 692
- COUNTER\_EVENT\_NONE, 694
- COUNTER\_EVENT\_RELOAD, 694
- COUNTER\_EVENT\_ZERO, 693
- COUNTER\_EVENT\_ZERO\_SINGLE, 693
- EXT\_TRIGGER\_ACTION\_MASK, 692
- EXT\_TRIGGER\_EVENT, 694
- EXT\_TRIGGER\_EVENT\_ALL, 694
- EXT\_TRIGGER\_EVENT\_IN, 694
- EXT\_TRIGGER\_EVENT\_MASK, 692
- EXT\_TRIGGER\_EVENT\_NONE, 694
- SPFI\_PORT\_EVENT, 694
- SPFI\_PORT\_EVENT\_ALL, 694
- SPFI\_PORT\_EVENT\_CONNECTED, 694
- SPFI\_PORT\_EVENT\_DISCONNECTED, 694
- SPFI\_PORT\_EVENT\_MASK, 692
- SPFI\_PORT\_EVENT\_NONE, 694
- SPFI\_PORT\_EVENT\_RX\_BM, 694
- SPFI\_PORT\_EVENT\_TX\_BM, 694
- SPFI\_VC\_EVENT, 694
- SPFI\_VC\_EVENT\_ALL, 694
- SPFI\_VC\_EVENT\_MASK, 692
- SPFI\_VC\_EVENT\_NONE, 694
- SPFI\_VC\_EVENT\_RX\_EEP, 694
- SPFI\_VC\_EVENT\_RX\_EOP, 694
- SPFI\_VC\_EVENT\_RX\_SOP, 694
- SPFI\_VC\_EVENT\_TX\_EOP, 694
- SPFI\_VC\_EVENT\_TX\_PKT\_PENDING, 694
- SPFI\_VC\_EVENT\_TX\_SOP, 694
- SPW\_PORT\_EVENT, 694
- SPW\_PORT\_EVENT\_ALL, 695
- SPW\_PORT\_EVENT\_CREDIT\_ERROR, 695
- SPW\_PORT\_EVENT\_DISCONNECT, 695
- SPW\_PORT\_EVENT\_ESCAPE\_ERROR, 695
- SPW\_PORT\_EVENT\_MASK, 693
- SPW\_PORT\_EVENT\_NONE, 695
- SPW\_PORT\_EVENT\_PARITY\_ERROR, 695
- SPW\_PORT\_EVENT\_RUNNING, 695
- SPW\_PORT\_EVENT\_RX\_EEP, 695
- SPW\_PORT\_EVENT\_RX\_EOP, 695
- SPW\_PORT\_EVENT\_RX\_SOP, 695
- SPW\_PORT\_EVENT\_RX\_TIME\_CODE, 695
- SPW\_PORT\_EVENT\_TX\_EOP, 695
- SPW\_PORT\_EVENT\_TX\_PKT\_PENDING, 695
- SPW\_PORT\_EVENT\_TX\_SOP, 695
- SPW\_PORT\_EVENT\_TX\_TIME\_CODE, 695
- TIME\_CODE\_ACTION\_MASK, 693
- TIME\_CODE\_EVENT, 695
- TIME\_CODE\_EVENT\_ALL, 695
- TIME\_CODE\_EVENT\_MASK, 693
- TIME\_CODE\_EVENT\_NONE, 695
- TIME\_CODE\_EVENT\_TICK, 695
- TRIGGER\_BRICK\_MK3\_getCounterInputEvents, 696
- TRIGGER\_BRICK\_MK3\_getExtTriggerInput↔ Events, 697
- TRIGGER\_BRICK\_MK3\_getPortInputEvents, 697
- TRIGGER\_BRICK\_MK3\_getTimeCodeInput↔ Events, 698
- TRIGGER\_BRICK\_MK3\_getTriggerInputEvents, 698
- TRIGGER\_BRICK\_MK3\_setCounterInputEvents, 698
- TRIGGER\_BRICK\_MK3\_setExtTriggerInput↔ Events, 700
- TRIGGER\_BRICK\_MK3\_setPortInputEvents, 700
- TRIGGER\_BRICK\_MK3\_setTimeCodeInput↔ Events, 701
- TRIGGER\_BRICK\_MK3\_setTriggerInputEvents, 701
- TRIGGER\_EVENT, 695
- TRIGGER\_EVENT\_ALL, 695
- TRIGGER\_EVENT\_IN, 695

- TRIGGER\_EVENT\_MASK, 693
- TRIGGER\_EVENT\_NONE, 695
- TRIGGER\_PCIE\_IF\_getCounterInputEvents, 702
- TRIGGER\_PCIE\_IF\_getPortInputEvents, 702
- TRIGGER\_PCIE\_IF\_getTriggerInputEvents, 703
- TRIGGER\_PCIE\_IF\_setCounterInputEvents, 703
- TRIGGER\_PCIE\_IF\_setPortInputEvents, 703
- TRIGGER\_PCIE\_IF\_setTriggerInputEvents, 704
- TRIGGER\_PCIE\_MK2\_getCounterInputEvents, 704
- TRIGGER\_PCIE\_MK2\_getExtTriggerInputEvents, 705
- TRIGGER\_PCIE\_MK2\_getPortInputEvents, 705
- TRIGGER\_PCIE\_MK2\_getTimeCodeInputEvents, 705
- TRIGGER\_PCIE\_MK2\_getTriggerInputEvents, 707
- TRIGGER\_PCIE\_MK2\_setCounterInputEvents, 707
- TRIGGER\_PCIE\_MK2\_setExtTriggerInputEvents, 708
- TRIGGER\_PCIE\_MK2\_setPortInputEvents, 708
- TRIGGER\_PCIE\_MK2\_setTimeCodeInputEvents, 709
- TRIGGER\_PCIE\_MK2\_setTriggerInputEvents, 709
- TRIGGER\_PXI\_IF\_getCounterInputEvents, 710
- TRIGGER\_PXI\_IF\_getExtTriggerInputEvents, 710
- TRIGGER\_PXI\_IF\_getPortInputEvents, 710
- TRIGGER\_PXI\_IF\_getTimeCodeInputEvents, 712
- TRIGGER\_PXI\_IF\_getTriggerInputEvents, 712
- TRIGGER\_PXI\_IF\_setCounterInputEvents, 713
- TRIGGER\_PXI\_IF\_setExtTriggerInputEvents, 713
- TRIGGER\_PXI\_IF\_setPortInputEvents, 714
- TRIGGER\_PXI\_IF\_setTimeCodeInputEvents, 714
- TRIGGER\_PXI\_IF\_setTriggerInputEvents, 714
- TRIGGER\_PXI\_ROUTER\_getCounterInput↔Events, 715
- TRIGGER\_PXI\_ROUTER\_getPortInputEvents, 715
- TRIGGER\_PXI\_ROUTER\_getTimeCodeInput↔Events, 716
- TRIGGER\_PXI\_ROUTER\_getTriggerInputEvents, 716
- TRIGGER\_PXI\_ROUTER\_setCounterInputEvents, 716
- TRIGGER\_PXI\_ROUTER\_setPortInputEvents, 718
- TRIGGER\_PXI\_ROUTER\_setTimeCodeInput↔Events, 718
- TRIGGER\_PXI\_ROUTER\_setTriggerInputEvents, 719
- External Triggers, 493
  - TRIGGER\_CFG\_disableExtTriggerEdgeDetect↔Mode, 494
  - TRIGGER\_CFG\_disableExtTriggerInvert, 494
  - TRIGGER\_CFG\_disableExtTriggerOutput, 495
  - TRIGGER\_CFG\_enableExtTriggerEdgeDetect↔Mode, 495
  - TRIGGER\_CFG\_enableExtTriggerInvert, 496
  - TRIGGER\_CFG\_enableExtTriggerOutput, 496
  - TRIGGER\_CFG\_getExtTriggerEdgeDetectMode↔Enabled, 497
  - TRIGGER\_CFG\_getExtTriggerExtend, 497
  - TRIGGER\_CFG\_getExtTriggerInputEvents, 498
  - TRIGGER\_CFG\_getExtTriggerInvertEnabled, 498
  - TRIGGER\_CFG\_getExtTriggerOutputActions, 499
  - TRIGGER\_CFG\_getExtTriggerOutputEnabled, 499
  - TRIGGER\_CFG\_getNumberOfExtTriggers, 499
  - TRIGGER\_CFG\_setExtTriggerExtend, 500
  - TRIGGER\_CFG\_setExtTriggerInputEvents, 500
  - TRIGGER\_CFG\_setExtTriggerOutputActions, 501
- Functions for Building RMAP Packets, 1057
  - RMAP\_BuildReadCommandPacket, 1059
  - RMAP\_BuildReadModifyWriteCommandPacket, 1060
  - RMAP\_BuildReadModifyWriteRegisterPacket, 1061
  - RMAP\_BuildReadModifyWriteReplyPacket, 1062
  - RMAP\_BuildReadRegisterPacket, 1063
  - RMAP\_BuildReadReplyPacket, 1064
  - RMAP\_BuildWriteCommandPacket, 1065
  - RMAP\_BuildWriteRegisterPacket, 1066
  - RMAP\_BuildWriteReplyPacket, 1067
  - RMAP\_CalculateReadCommandPacketLength, 1068
  - RMAP\_CalculateReadModifyWriteCommand↔PacketLength, 1068
  - RMAP\_CalculateReadModifyWriteReplyPacket↔Length, 1069
  - RMAP\_CalculateReadReplyPacketLength, 1069
  - RMAP\_CalculateWriteCommandPacketLength, 1070
  - RMAP\_CalculateWriteReplyPacketLength, 1070
  - RMAP\_FillReadCommandPacket, 1070
  - RMAP\_FillReadModifyWriteCommandPacket, 1071
  - RMAP\_FillReadModifyWriteReplyPacket, 1072
  - RMAP\_FillReadReplyPacket, 1073
  - RMAP\_FillWriteCommandPacket, 1074
  - RMAP\_FillWriteReplyPacket, 1075
  - RMAP\_SetProtocolIdentifier, 1076
- Functions for building SPP Packets, 1103
  - CCSDS\_SPP\_CreatePTPHeader, 1104
  - CCSDS\_SPP\_CreatePacket, 1103
  - CCSDS\_SPP\_FillPTPHeader, 1105
  - CCSDS\_SPP\_FillPacket, 1105
  - CCSDS\_SPP\_FreeBuffer, 1107
- Functions for building TF Packets, 1117
  - CCSDS\_TF\_CreateIdlePacket, 1117
  - CCSDS\_TF\_CreatePackets, 1118
- Functions for Interpreting RMAP Packets, 1078
  - RMAP\_CheckPacketValid, 1079
  - RMAP\_CheckPacketValidIgnoreProtocol, 1079
  - RMAP\_GetAddress, 1080
  - RMAP\_GetData, 1081
  - RMAP\_GetDataCRC, 1081
  - RMAP\_GetHeaderCRC, 1081



- RMAP\_GetIncrementAddress, 1082
- RMAP\_GetKey, 1082
- RMAP\_GetMask, 1083
- RMAP\_GetPacketType, 1083
- RMAP\_GetPerformAcknowledgement, 1084
- RMAP\_GetProtocolIdentifier, 1084
- RMAP\_GetReadLength, 1085
- RMAP\_GetReplyAddress, 1085
- RMAP\_GetStatus, 1085
- RMAP\_GetTargetAddress, 1087
- RMAP\_GetTransactionID, 1087
- RMAP\_GetVerifyBeforeWrite, 1088
- Functions for retrieving (reconstructing) and interpreting
  - SPP Packets, 1108
  - CCSDS\_SPP\_CheckAPID, 1109
  - CCSDS\_SPP\_GetAPID, 1109
  - CCSDS\_SPP\_GetDataFieldLength, 1110
  - CCSDS\_SPP\_GetDataFieldPointer, 1110
  - CCSDS\_SPP\_GetEncapsulationHeaderLength, 1111
  - CCSDS\_SPP\_GetEncapsulationHeaderPointer, 1111
  - CCSDS\_SPP\_GetPTPHeaderLengthBytes, 1112
  - CCSDS\_SPP\_GetPTPProtocolIdentifier, 1112
  - CCSDS\_SPP\_GetPTPReservedByte, 1114
  - CCSDS\_SPP\_GetPTPTargetLogicalAddress, 1114
  - CCSDS\_SPP\_GetPTPUserApplicationByte, 1114
  - CCSDS\_SPP\_GetPacketType, 1111
  - CCSDS\_SPP\_GetPacketVersionNumber, 1112
  - CCSDS\_SPP\_GetPrimaryHeaderLengthBytes, 1112
  - CCSDS\_SPP\_GetSequenceCount, 1115
  - CCSDS\_SPP\_GetSequenceFlags, 1115
  - CCSDS\_SPP\_GetTelecommandSecondaryHeaderLengthBytes, 1115
  - CCSDS\_SPP\_IsSecondaryHeaderPresent, 1116
  - CCSDS\_SPP\_RetrievePacket, 1116
- Functions for retrieving (reconstructing) and interpreting
  - TF Packets, 1120
  - CCSDS\_TF\_Get\_M\_PDU\_PacketMaxSize, 1121
  - CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_DataField, 1121
  - CCSDS\_TF\_M\_PDU\_Get\_M\_PDU\_FirstHeaderPointer, 1122
  - CCSDS\_TF\_M\_PDU\_GetEncapsulationHeader, 1122
  - CCSDS\_TF\_M\_PDU\_GetFrameErrorControlField, 1122
  - CCSDS\_TF\_M\_PDU\_GetFrameHeaderErrorControl, 1124
  - CCSDS\_TF\_M\_PDU\_GetInsertZoneData, 1124
  - CCSDS\_TF\_M\_PDU\_GetOperationalControlField, 1124
  - CCSDS\_TF\_M\_PDU\_GetReplayFlag, 1126
  - CCSDS\_TF\_M\_PDU\_GetSpacecraftID, 1126
  - CCSDS\_TF\_M\_PDU\_GetVCFrameCountCycle, 1126
  - CCSDS\_TF\_M\_PDU\_GetVCFrameCountUsageFlag, 1127
  - CCSDS\_TF\_M\_PDU\_GetVersionNumber, 1127
  - CCSDS\_TF\_M\_PDU\_GetVirtualChannelFrameCount, 1127
  - CCSDS\_TF\_M\_PDU\_GetVirtualChannelID, 1128
  - CCSDS\_TF\_RetrievePacket, 1128
- GbE Brick Mk1 Configuration API, 633
- General, 490, 851
  - CFG\_getGeneralPurpose, 851
  - CFG\_getNetworkIdentity, 852
  - CFG\_getRouterGlobalSettings, 852
  - CFG\_refreshDeviceInfo, 853
  - CFG\_setGeneralPurpose, 853
  - CFG\_setNetworkIdentity, 854
  - CFG\_setRouterGlobalSettings, 854
  - CFG\_updateDeviceInfo, 855
  - TRIGGER\_CFG\_getDeviceClockFreq, 490
  - TRIGGER\_CFG\_getDeviceClockRate, 491
  - TRIGGER\_CFG\_getTriggerMatrix, 491
  - TRIGGER\_CFG\_reset, 491
- General API Functions, 277
  - STAR\_APP\_ID, 278
  - STAR\_destroyString, 278
  - STAR\_destroyVersionInfo, 278
  - STAR\_destroyVersionInfoList, 278
  - STAR\_getAllVersions, 279
  - STAR\_getApiVersion, 279
  - STAR\_getApplicationAttachedToChannel, 279
  - STAR\_getApplicationID, 280
  - STAR\_getApplicationName, 280
  - STAR\_getApplicationNameForID, 280
  - STAR\_setApplicationName, 281
- general.h, 1181
- Generic Configuration API, 614
- Generic Triggering API, 470
- Group Adaptive Routing, 871
  - CFG\_getRoutingTableEntry, 871
  - CFG\_setRoutingTableEntry, 872
- Hardware, 856
  - CFG\_FPGAInfoToString, 857
  - CFG\_getFPGAInfo, 858
  - CFG\_identify, 858
- IF\_MODE
  - Configuration, 451
- IF\_MODE\_RMAP\_DISABLED
  - Configuration, 451
- IF\_MODE\_RMAP\_ENABLED
  - Configuration, 451
- Interface Mode, 402, 432, 584, 873
  - CFG\_BRICK\_MK2\_disableIdentifySourceOnPort, 402
  - CFG\_BRICK\_MK2\_disableInterfaceModeOnPort, 403
  - CFG\_BRICK\_MK2\_enableIdentifySourceOnPort, 403

- CFG\_BRICK\_MK2\_enableInterfaceModeOnPort, 404
- CFG\_BRICK\_MK2\_getIdentifySourceOnPort↔ Enabled, 404
- CFG\_BRICK\_MK2\_getInterfaceModeOnPort↔ Enabled, 405
- CFG\_BRICK\_MK2\_getPortRoutingAddress, 405
- CFG\_BRICK\_MK2\_setPortRoutingAddress, 406
- CFG\_MK2\_disableIdentifySource, 585
- CFG\_MK2\_disableIdentifySourceOnPort, 585
- CFG\_MK2\_disableInterfaceMode, 585
- CFG\_MK2\_disableInterfaceModeOnPort, 586
- CFG\_MK2\_enableIdentifySource, 586
- CFG\_MK2\_enableIdentifySourceOnPort, 587
- CFG\_MK2\_enableInterfaceMode, 587
- CFG\_MK2\_enableInterfaceModeOnPort, 587
- CFG\_MK2\_getIdentifySourceEnabled, 588
- CFG\_MK2\_getIdentifySourceOnPortEnabled, 588
- CFG\_MK2\_getInterfaceModeEnabled, 589
- CFG\_MK2\_getInterfaceModeOnPortEnabled, 589
- CFG\_MK2\_getPortRoutingAddress, 590
- CFG\_MK2\_setPortRoutingAddress, 590
- CFG\_PXI\_disableIdentifySourceOnPort, 432
- CFG\_PXI\_disableInterfaceModeOnPort, 433
- CFG\_PXI\_enableIdentifySourceOnPort, 433
- CFG\_PXI\_enableInterfaceModeOnPort, 433
- CFG\_PXI\_getIdentifySourceOnPortEnabled, 434
- CFG\_PXI\_getInterfaceModeOnPortEnabled, 434
- CFG\_PXI\_getPortRoutingAddress, 435
- CFG\_PXI\_setPortRoutingAddress, 435
- CFG\_disableIdentifySource, 874
- CFG\_disableIdentifySourceOnPort, 874
- CFG\_disableInterfaceMode, 874
- CFG\_disableInterfaceModeOnPort, 875
- CFG\_enableIdentifySource, 875
- CFG\_enableIdentifySourceOnPort, 876
- CFG\_enableInterfaceMode, 876
- CFG\_enableInterfaceModeOnPort, 878
- CFG\_getIdentifySourceEnabled, 878
- CFG\_getIdentifySourceOnPortEnabled, 879
- CFG\_getInterfaceModeEnabled, 879
- CFG\_getInterfaceModeOnPortEnabled, 880
- CFG\_getPortRoutingAddress, 880
- CFG\_setPortRoutingAddress, 881
- Internal Triggers, 531
  - TRIGGER\_CFG\_disableTrigger, 532
  - TRIGGER\_CFG\_enableTrigger, 532
  - TRIGGER\_CFG\_forceTrigger, 533
  - TRIGGER\_CFG\_getNumberOfTriggers, 533
  - TRIGGER\_CFG\_getTriggerAndMask, 534
  - TRIGGER\_CFG\_getTriggerEnabled, 534
  - TRIGGER\_CFG\_getTriggerInputEvents, 535
  - TRIGGER\_CFG\_getTriggerInputMode, 535
  - TRIGGER\_CFG\_getTriggerInvertMask, 536
  - TRIGGER\_CFG\_getTriggerSourceEventsAnded, 536
  - TRIGGER\_CFG\_getTriggerSourceEventsInverted, 537
  - TRIGGER\_CFG\_setTriggerAndMask, 537
  - TRIGGER\_CFG\_setTriggerInputEvents, 539
  - TRIGGER\_CFG\_setTriggerInputMode, 539
  - TRIGGER\_CFG\_setTriggerInvertMask, 540
  - TRIGGER\_CFG\_setTriggerSourceEventsAnded, 540
  - TRIGGER\_CFG\_setTriggerSourceEventsInverted, 541
- Lane Information, 1009
- Link Information, 981
- Link Speed, 384, 387, 399, 413, 424, 426, 556, 566
  - CFG\_BRICK\_MK2\_getLinkClockFrequency, 400
  - CFG\_BRICK\_MK2\_setLinkClockFrequency, 400
  - CFG\_BRICK\_MK3\_getBaseTransmitClock, 413
  - CFG\_BRICK\_MK3\_getTransmitClock, 414
  - CFG\_BRICK\_MK3\_setBaseTransmitClock, 414
  - CFG\_BRICK\_MK3\_setTransmitClock, 415
  - CFG\_GBE\_BRICK\_MK1\_getTransmitSignalling↔ Rate, 424
  - CFG\_GBE\_BRICK\_MK1\_setTransmitSignalling↔ Rate, 425
  - CFG\_MK2\_getBaseTransmitClock, 567
  - CFG\_MK2\_getLinkRateDivider, 568
  - CFG\_MK2\_getTransmitClock, 568
  - CFG\_MK2\_setBaseTransmitClock, 569
  - CFG\_MK2\_setLinkRateDivider, 570
  - CFG\_MK2\_setTransmitClock, 570
  - CFG\_PCIE\_MK2\_getDecimalTxSignallingRate, 387
  - CFG\_PCIE\_MK2\_getTransmitSignallingRate, 388
  - CFG\_PCIE\_MK2\_setDecimalTxSignallingRate, 388
  - CFG\_PCIE\_MK2\_setTransmitSignallingRate, 388
  - CFG\_PCIMK2\_getLinkClockFrequency, 385
  - CFG\_PCIMK2\_setLinkClockFrequency, 385
  - CFG\_PXI\_getBaseTransmitClock, 426
  - CFG\_PXI\_getLinkRateDivider, 427
  - CFG\_PXI\_getTransmitClock, 427
  - CFG\_PXI\_setBaseTransmitClock, 428
  - CFG\_PXI\_setLinkRateDivider, 428
  - CFG\_PXI\_setTransmitClock, 429
  - CFG\_ROUTER\_MK2S\_disablePrecisionTransmit↔ Rate, 556
  - CFG\_ROUTER\_MK2S\_enablePrecisionTransmit↔ Rate, 557
  - CFG\_ROUTER\_MK2S\_getPrecisionTransmitRate, 557
  - CFG\_ROUTER\_MK2S\_getPrecisionTransmit↔ RateEnabled, 558
  - CFG\_ROUTER\_MK2S\_getPrecisionTransmit↔ RateInUse, 558
  - CFG\_ROUTER\_MK2S\_setPrecisionTransmitRate, 558
  - STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ, 399
  - STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_120, 399
  - STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_130, 399
  - STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_140, 399
  - STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_150, 400

- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_160, 400
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_180, 400
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_200, 400
- STAR\_CFG\_PCIMK2\_LINK\_FREQ, 384
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120, 384
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128, 384
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_140, 385
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150, 385
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160, 385
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180, 385
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_200, 385
- M\_PDU\_PACKET, 1092
- M\_PDU\_PACKET\_MAX\_SIZE
  - CCSDS/SPP/TF Packet Library, 1100
- Manual Authorisation, 444
  - REJECTION\_REASON, 445
  - RMAP\_TARGET\_PXI\_IF\_authoriseWaiting↔
    - Command, 446
  - RMAP\_TARGET\_PXI\_IF\_getWaitingCommand, 446
  - RMAP\_TARGET\_PXI\_IF\_isAuthRequired, 446
  - RMAP\_TARGET\_PXI\_IF\_rejectWaitingCommand, 447
- Measured Link Speed, 572
  - CFG\_MK2\_getMeasuredLinkSpeed, 572
  - STAR\_CFG\_MK2\_LINK\_SPEED\_UNITS\_KBPS, 572
- Mk2 Compatible Interface and Router Device Configuration API, 628
- NOTIF\_TYPE
  - Notifications, 464
- NOTIF\_TYPE\_ALL
  - Notifications, 464
- NOTIF\_TYPE\_AUTH\_REQUEST
  - Notifications, 464
- NOTIF\_TYPE\_CMD\_COMPLETE
  - Notifications, 464
- NOTIF\_TYPE\_MASK
  - Notifications, 463
- NOTIF\_TYPE\_NONE
  - Notifications, 464
- Network Management, 939
  - CFG\_SPFI\_getBroadcastTimeout, 940
  - CFG\_SPFI\_getBroadcastTimeoutMaximum, 940
  - CFG\_SPFI\_getBroadcastTimeoutResolution, 940
  - CFG\_SPFI\_getCurrentTime, 942
  - CFG\_SPFI\_getCurrentTimeSlot, 942
  - CFG\_SPFI\_getDeviceIdentifier, 943
  - CFG\_SPFI\_getLinksWithErrors, 943
  - CFG\_SPFI\_getPortsReady, 944
  - CFG\_SPFI\_getPortsWithErrors, 944
  - CFG\_SPFI\_getSpaceWireBC, 945
  - CFG\_SPFI\_setBroadcastTimeout, 945
  - CFG\_SPFI\_setDeviceIdentifier, 946
  - CFG\_SPFI\_setSpaceWireBC, 946
- NotifListenerFunc
  - Notifications, 463
- NotifListenerWithContextFunc
  - Notifications, 463
- Notifications, 462
  - NOTIF\_TYPE, 464
  - NOTIF\_TYPE\_ALL, 464
  - NOTIF\_TYPE\_AUTH\_REQUEST, 464
  - NOTIF\_TYPE\_CMD\_COMPLETE, 464
  - NOTIF\_TYPE\_MASK, 463
  - NOTIF\_TYPE\_NONE, 464
  - NotifListenerFunc, 463
  - NotifListenerWithContextFunc, 463
  - RMAP\_TARGET\_PXI\_IF\_getEnabledNotifications, 464
  - RMAP\_TARGET\_PXI\_IF\_registerNotification↔
    - Listener, 466
  - RMAP\_TARGET\_PXI\_IF\_registerNotification↔
    - ListenerWithContext, 466
  - RMAP\_TARGET\_PXI\_IF\_setEnabledNotifications, 467
  - RMAP\_TARGET\_PXI\_IF\_startReceivingNotifications, 467
  - RMAP\_TARGET\_PXI\_IF\_stopReceivingNotifications, 468
  - RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔
    - Listener, 468
  - RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔
    - ListenerWithContext, 468
- Other Device Configuration APIs, 618
- PCI Mk2 Configuration API, 630
- PCI Mk2 Packet Subsystem, 597
  - PKT\_SUBSYS\_PATTERN, 600
  - PKT\_SUBSYS\_StatisticsFunc, 600
  - PKT\_SUBSYS\_cancelWaitsForPacketChecking↔
    - Error, 600
  - PKT\_SUBSYS\_fillBufferWithPattern, 602
  - PKT\_SUBSYS\_getPacketCheckingMismatch↔
    - Count, 602
  - PKT\_SUBSYS\_getSinkDataPointers, 603
  - PKT\_SUBSYS\_getStatistics, 603
  - PKT\_SUBSYS\_readMemory, 603
  - PKT\_SUBSYS\_setGeneratorBlockingParameters, 604
  - PKT\_SUBSYS\_setPacketCheckingFormat, 604
  - PKT\_SUBSYS\_setPacketGeneratorFormat, 606
  - PKT\_SUBSYS\_setPacketSinkParameters, 606
  - PKT\_SUBSYS\_setSinkBlockingParameters, 608
  - PKT\_SUBSYS\_startGetStatisticsLoop, 608
  - PKT\_SUBSYS\_startPacketChecking, 609
  - PKT\_SUBSYS\_startPacketGeneration, 609
  - PKT\_SUBSYS\_startSink, 610
  - PKT\_SUBSYS\_stopGetStatisticsLoop, 610
  - PKT\_SUBSYS\_stopPacketChecking, 611
  - PKT\_SUBSYS\_stopPacketGeneration, 611
  - PKT\_SUBSYS\_stopSink, 611
  - PKT\_SUBSYS\_waitForPacketCheckingError, 612
  - PKT\_SUBSYS\_waitForPacketGeneration↔
    - SequenceComplete, 612

- PKT\_SUBSYS\_writeMemory, 613
- STATISTICS\_LOOP\_ID, 600
- PCIe Mk2 Configuration API, 636
- PKT\_SUBSYS\_MODULE\_NAME
  - packet\_subsystem.h, 1187
- PKT\_SUBSYS\_PATTERN
  - PCI Mk2 Packet Subsystem, 600
- PKT\_SUBSYS\_STATISTICS, 599
- PKT\_SUBSYS\_StatisticsFunc
  - PCI Mk2 Packet Subsystem, 600
- PKT\_SUBSYS\_cancelWaitsForPacketCheckingError
  - PCI Mk2 Packet Subsystem, 600
- PKT\_SUBSYS\_fillBufferWithPattern
  - PCI Mk2 Packet Subsystem, 602
- PKT\_SUBSYS\_getPacketCheckingMismatchCount
  - PCI Mk2 Packet Subsystem, 602
- PKT\_SUBSYS\_getSinkDataPointers
  - PCI Mk2 Packet Subsystem, 603
- PKT\_SUBSYS\_getStatistics
  - PCI Mk2 Packet Subsystem, 603
- PKT\_SUBSYS\_readMemory
  - PCI Mk2 Packet Subsystem, 603
- PKT\_SUBSYS\_setGeneratorBlockingParameters
  - PCI Mk2 Packet Subsystem, 604
- PKT\_SUBSYS\_setPacketCheckingFormat
  - PCI Mk2 Packet Subsystem, 604
- PKT\_SUBSYS\_setPacketGeneratorFormat
  - PCI Mk2 Packet Subsystem, 606
- PKT\_SUBSYS\_setPacketSinkParameters
  - PCI Mk2 Packet Subsystem, 606
- PKT\_SUBSYS\_setSinkBlockingParameters
  - PCI Mk2 Packet Subsystem, 608
- PKT\_SUBSYS\_startGetStatisticsLoop
  - PCI Mk2 Packet Subsystem, 608
- PKT\_SUBSYS\_startPacketChecking
  - PCI Mk2 Packet Subsystem, 609
- PKT\_SUBSYS\_startPacketGeneration
  - PCI Mk2 Packet Subsystem, 609
- PKT\_SUBSYS\_startSink
  - PCI Mk2 Packet Subsystem, 610
- PKT\_SUBSYS\_stopGetStatisticsLoop
  - PCI Mk2 Packet Subsystem, 610
- PKT\_SUBSYS\_stopPacketChecking
  - PCI Mk2 Packet Subsystem, 611
- PKT\_SUBSYS\_stopPacketGeneration
  - PCI Mk2 Packet Subsystem, 611
- PKT\_SUBSYS\_stopSink
  - PCI Mk2 Packet Subsystem, 611
- PKT\_SUBSYS\_waitForPacketCheckingError
  - PCI Mk2 Packet Subsystem, 612
- PKT\_SUBSYS\_waitForPacketGenerationSequence↔
  - Complete
  - PCI Mk2 Packet Subsystem, 612
- PKT\_SUBSYS\_writeMemory
  - PCI Mk2 Packet Subsystem, 613
- POPULATE\_VERSION
  - version.h, 1241
- PORT\_ACTION
  - triggering\_types.h, 1234
- PORT\_ACTION\_ALL
  - triggering\_types.h, 1234
- PORT\_ACTION\_DECR\_CREDIT
  - triggering\_types.h, 1234
- PORT\_ACTION\_DISABLE\_LVDS
  - triggering\_types.h, 1234
- PORT\_ACTION\_DISCONNECT
  - triggering\_types.h, 1234
- PORT\_ACTION\_ESCAPE\_ERROR
  - triggering\_types.h, 1234
- PORT\_ACTION\_INCR\_CREDIT
  - triggering\_types.h, 1234
- PORT\_ACTION\_INSERT\_FCT
  - triggering\_types.h, 1234
- PORT\_ACTION\_MASK
  - triggering\_types.h, 1233
- PORT\_ACTION\_NONE
  - triggering\_types.h, 1234
- PORT\_ACTION\_PARITY\_ERROR
  - triggering\_types.h, 1234
- PORT\_ACTION\_STOP\_RECEPTION
  - triggering\_types.h, 1234
- PORT\_ACTION\_SUPPRESS\_FCT
  - triggering\_types.h, 1234
- PORT\_ACTION\_TRANSMIT\_PKT
  - triggering\_types.h, 1234
- PORT\_EVENT
  - triggering\_types.h, 1234
- PORT\_EVENT\_ALL
  - triggering\_types.h, 1235
- PORT\_EVENT\_CREDIT\_ERROR
  - triggering\_types.h, 1235
- PORT\_EVENT\_DISCONNECT
  - triggering\_types.h, 1235
- PORT\_EVENT\_ESCAPE\_ERROR
  - triggering\_types.h, 1235
- PORT\_EVENT\_MASK
  - triggering\_types.h, 1234
- PORT\_EVENT\_NONE
  - triggering\_types.h, 1235
- PORT\_EVENT\_PARITY\_ERROR
  - triggering\_types.h, 1235
- PORT\_EVENT\_RUNNING
  - triggering\_types.h, 1235
- PORT\_EVENT\_RX\_EEP
  - triggering\_types.h, 1235
- PORT\_EVENT\_RX\_EOP
  - triggering\_types.h, 1235
- PORT\_EVENT\_RX\_SOP
  - triggering\_types.h, 1235
- PORT\_EVENT\_RX\_TIME\_CODE
  - triggering\_types.h, 1235
- PORT\_EVENT\_TX\_EOP
  - triggering\_types.h, 1235
- PORT\_EVENT\_TX\_PKT\_PENDING
  - triggering\_types.h, 1235
- PORT\_EVENT\_TX\_SOP



- triggering\_types.h, 1235
- PORT\_EVENT\_TX\_TIME\_CODE
  - triggering\_types.h, 1235
- PORT\_STATUS\_CONTROL
  - Port Status and Control, 649
- PRINT\_FULL\_VERSION\_INFO
  - version.h, 1241
- PRINT\_VERSION
  - version.h, 1242
- PRINT\_VERSION\_EDIT
  - version.h, 1242
- PRINT\_VERSION\_NUM
  - version.h, 1242
- PRINT\_VERSION\_PATCH
  - version.h, 1242
- PXI Configuration API, 635
- packet\_subsystem.h, 1184
  - PKT\_SUBSYS\_MODULE\_NAME, 1187
- Periodic, 914
  - CFG\_getDeviceClockRate, 914
  - CFG\_startPeriodicAction, 914
  - CFG\_stopPeriodicAction, 915
- Periodic Actions, 594
  - CFG\_MK2\_getDeviceClockRate, 594
  - CFG\_MK2\_startPeriodicAction, 595
  - CFG\_MK2\_stopPeriodicAction, 595
  - SPW\_ACTION, 594
  - SPW\_TRANSMIT\_PACKET, 594
- Port Information, 975
  - CFG\_SPFI\_clearPortBroadcastError, 975
  - CFG\_SPFI\_getAXIBC, 976
  - CFG\_SPFI\_getPortInformation, 976
  - CFG\_SPFI\_getPortType, 977
  - CFG\_SPFI\_getPortVCCount, 977
  - CFG\_SPFI\_getPortVCsWithErrors, 978
  - CFG\_SPFI\_isPortBroadcastErrorDetected, 978
  - CFG\_SPFI\_isPortReady, 979
  - CFG\_SPFI\_setAXIBC, 979
- Port Status and Control, 643, 860
  - CFG\_ROUTER\_clearPortErrors, 650
  - CFG\_ROUTER\_getConfigPortErrors, 650
  - CFG\_ROUTER\_getExternalPortErrors, 651
  - CFG\_ROUTER\_getExternalPortStatus, 651
  - CFG\_ROUTER\_getPortConnection, 652
  - CFG\_ROUTER\_getPortStatusControl, 652
  - CFG\_ROUTER\_getPortType, 653
  - CFG\_ROUTER\_getSpaceWireLinkErrors, 653
  - CFG\_ROUTER\_getSpaceWireLinkStatus, 654
  - CFG\_ROUTER\_setSpaceWireLinkStatus, 654
  - CFG\_ROUTER\_startLink, 655
  - CFG\_ROUTER\_stopLink, 655
  - CFG\_clearPortErrors, 861
  - CFG\_getConfigPortErrors, 861
  - CFG\_getExternalPortErrors, 862
  - CFG\_getExternalPortStatus, 862
  - CFG\_getPortConnection, 863
  - CFG\_getPortStatusControl, 863
  - CFG\_getPortType, 864
  - CFG\_getSpaceWireLinkErrors, 864
  - CFG\_getSpaceWireLinkStatus, 865
  - CFG\_setSpaceWireLinkStatus, 865
  - CFG\_startLink, 866
  - CFG\_stopLink, 866
  - PORT\_STATUS\_CONTROL, 649
  - STAR\_CFG\_PORT\_TYPE, 649
  - STAR\_CFG\_PORT\_TYPE\_CONFIGURATION, 649
  - STAR\_CFG\_PORT\_TYPE\_EXTERNAL, 650
  - STAR\_CFG\_PORT\_TYPE\_INVALID, 650
  - STAR\_CFG\_PORT\_TYPE\_LINK, 650
  - STAR\_CFG\_SPW\_LINK\_STATE, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RES←ET, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_INVALID, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_READY, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_RUN, 650
  - STAR\_CFG\_SPW\_LINK\_STATE\_STARTED, 650
- Precision Links, 902
  - CFG\_disablePrecisionTransmitRate, 902
  - CFG\_enablePrecisionTransmitRate, 903
  - CFG\_getPrecisionTransmitRate, 903
  - CFG\_getPrecisionTransmitRateEnabled, 904
  - CFG\_getPrecisionTransmitRateInUse, 904
  - CFG\_setPrecisionTransmitRate, 905
- Quality Of Service, 966
  - CFG\_SPFI\_disableVCContinuousMode, 966
  - CFG\_SPFI\_enableVCContinuousMode, 967
  - CFG\_SPFI\_getVCBandwidthReservation, 968
  - CFG\_SPFI\_getVCPriority, 968
  - CFG\_SPFI\_getVCQualityOfService, 969
  - CFG\_SPFI\_isVCContinuousModeEnabled, 969
  - CFG\_SPFI\_setVCBandwidthReservation, 970
  - CFG\_SPFI\_setVCPriority, 970
- REJECTION\_REASON
  - Manual Authorisation, 445
- RMAP Packet Library, 1044
  - RMAP\_COMMAND\_BIT, 1049
  - RMAP\_COMMAND\_NOT\_IMPLEMENTED\_OR←\_AUTHORISED, 1053
  - RMAP\_CalculateCRC, 1053
  - RMAP\_CalculateCRCWithSeed, 1053
  - RMAP\_EARLY\_EOP, 1053
  - RMAP\_EEP, 1053
  - RMAP\_FreeBuffer, 1054
  - RMAP\_GENERAL\_ERROR, 1052
  - RMAP\_GET\_VERSION\_EDIT, 1049
  - RMAP\_GET\_VERSION\_MAJOR, 1049
  - RMAP\_GET\_VERSION\_MINOR, 1050
  - RMAP\_GET\_VERSION\_PATCH, 1050
  - RMAP\_GetVersion, 1054
  - RMAP\_INCREMENT\_ADDRESS\_BIT, 1051

- [RMAP\\_INVALID\\_DATA\\_CRC, 1053](#)
  - [RMAP\\_INVALID\\_KEY, 1053](#)
  - [RMAP\\_INVALID\\_PACKET\\_TYPE, 1052](#)
  - [RMAP\\_INVALID\\_STATUS, 1053](#)
  - [RMAP\\_INVALID\\_TARGET\\_LOGICAL\\_ADDRESS, 1053](#)
  - [RMAP\\_IsCRCValid, 1054](#)
  - [RMAP\\_PACKET\\_TYPE, 1052](#)
  - [RMAP\\_PROTOCOL\\_IDENTIFIER, 1051](#)
  - [RMAP\\_READ\\_COMMAND, 1052](#)
  - [RMAP\\_READ\\_MODIFY\\_WRITE\\_COMMAND, 1052](#)
  - [RMAP\\_READ\\_MODIFY\\_WRITE\\_REPLY, 1052](#)
  - [RMAP\\_READ\\_REPLY, 1052](#)
  - [RMAP\\_REPLY\\_ADDRESS\\_LENGTH\\_BITS, 1051](#)
  - [RMAP\\_REPLY\\_BIT, 1051](#)
  - [RMAP\\_RESERVED\\_BIT, 1051](#)
  - [RMAP\\_RMW\\_DATA\\_LENGTH\\_ERROR, 1053](#)
  - [RMAP\\_STATUS, 1052](#)
  - [RMAP\\_SUCCESS, 1052](#)
  - [RMAP\\_TOO\\_MUCH\\_DATA, 1053](#)
  - [RMAP\\_UNUSED\\_PACKET\\_TYPE\\_OR\\_COMMAND\\_CODE, 1053](#)
  - [RMAP\\_VERIFY\\_BEFORE\\_WRITE\\_BIT, 1051](#)
  - [RMAP\\_VERIFY\\_BUFFER\\_OVERRUN, 1053](#)
  - [RMAP\\_WRITE\\_COMMAND, 1052](#)
  - [RMAP\\_WRITE\\_OPERATION\\_BIT, 1052](#)
  - [RMAP\\_WRITE\\_REPLY, 1052](#)
  - [RMAPPACKETLIBRARY\\_CC, 1052](#)
- [RMAP Target API, 442](#)
- [RMAP\\_BuildReadCommandPacket](#)
  - [Functions for Building RMAP Packets, 1059](#)
- [RMAP\\_BuildReadModifyWriteCommandPacket](#)
  - [Functions for Building RMAP Packets, 1060](#)
- [RMAP\\_BuildReadModifyWriteRegisterPacket](#)
  - [Functions for Building RMAP Packets, 1061](#)
- [RMAP\\_BuildReadModifyWriteReplyPacket](#)
  - [Functions for Building RMAP Packets, 1062](#)
- [RMAP\\_BuildReadRegisterPacket](#)
  - [Functions for Building RMAP Packets, 1063](#)
- [RMAP\\_BuildReadReplyPacket](#)
  - [Functions for Building RMAP Packets, 1064](#)
- [RMAP\\_BuildWriteCommandPacket](#)
  - [Functions for Building RMAP Packets, 1065](#)
- [RMAP\\_BuildWriteRegisterPacket](#)
  - [Functions for Building RMAP Packets, 1066](#)
- [RMAP\\_BuildWriteReplyPacket](#)
  - [Functions for Building RMAP Packets, 1067](#)
- [RMAP\\_COMMAND\\_BIT](#)
  - [RMAP Packet Library, 1049](#)
- [RMAP\\_COMMAND\\_NOT\\_IMPLEMENTED\\_OR\\_AUTHORIZED](#)
  - [RMAP Packet Library, 1053](#)
- [RMAP\\_CalculateCRC](#)
  - [RMAP Packet Library, 1053](#)
- [RMAP\\_CalculateCRCWithSeed](#)
  - [RMAP Packet Library, 1053](#)
- [RMAP\\_CalculateReadCommandPacketLength](#)
  - [Functions for Building RMAP Packets, 1068](#)
- [RMAP\\_CalculateReadModifyWriteCommandPacketLength](#)
  - [Functions for Building RMAP Packets, 1068](#)
- [RMAP\\_CalculateReadModifyWriteReplyPacketLength](#)
  - [Functions for Building RMAP Packets, 1069](#)
- [RMAP\\_CalculateReadReplyPacketLength](#)
  - [Functions for Building RMAP Packets, 1069](#)
- [RMAP\\_CalculateWriteCommandPacketLength](#)
  - [Functions for Building RMAP Packets, 1070](#)
- [RMAP\\_CalculateWriteReplyPacketLength](#)
  - [Functions for Building RMAP Packets, 1070](#)
- [RMAP\\_CheckPacketValid](#)
  - [Functions for Interpreting RMAP Packets, 1079](#)
- [RMAP\\_CheckPacketValidIgnoreProtocol](#)
  - [Functions for Interpreting RMAP Packets, 1079](#)
- [RMAP\\_EARLY\\_EOP](#)
  - [RMAP Packet Library, 1053](#)
- [RMAP\\_EEP](#)
  - [RMAP Packet Library, 1053](#)
- [RMAP\\_FillReadCommandPacket](#)
  - [Functions for Building RMAP Packets, 1070](#)
- [RMAP\\_FillReadModifyWriteCommandPacket](#)
  - [Functions for Building RMAP Packets, 1071](#)
- [RMAP\\_FillReadModifyWriteReplyPacket](#)
  - [Functions for Building RMAP Packets, 1072](#)
- [RMAP\\_FillReadReplyPacket](#)
  - [Functions for Building RMAP Packets, 1073](#)
- [RMAP\\_FillWriteCommandPacket](#)
  - [Functions for Building RMAP Packets, 1074](#)
- [RMAP\\_FillWriteReplyPacket](#)
  - [Functions for Building RMAP Packets, 1075](#)
- [RMAP\\_FreeBuffer](#)
  - [RMAP Packet Library, 1054](#)
- [RMAP\\_GENERAL\\_ERROR](#)
  - [RMAP Packet Library, 1052](#)
- [RMAP\\_GET\\_VERSION\\_EDIT](#)
  - [RMAP Packet Library, 1049](#)
- [RMAP\\_GET\\_VERSION\\_MAJOR](#)
  - [RMAP Packet Library, 1049](#)
- [RMAP\\_GET\\_VERSION\\_MINOR](#)
  - [RMAP Packet Library, 1050](#)
- [RMAP\\_GET\\_VERSION\\_PATCH](#)
  - [RMAP Packet Library, 1050](#)
- [RMAP\\_GetAddress](#)
  - [Functions for Interpreting RMAP Packets, 1080](#)
- [RMAP\\_GetData](#)
  - [Functions for Interpreting RMAP Packets, 1081](#)
- [RMAP\\_GetDataCRC](#)
  - [Functions for Interpreting RMAP Packets, 1081](#)
- [RMAP\\_GetHeaderCRC](#)
  - [Functions for Interpreting RMAP Packets, 1081](#)
- [RMAP\\_GetIncrementAddress](#)
  - [Functions for Interpreting RMAP Packets, 1082](#)
- [RMAP\\_GetKey](#)
  - [Functions for Interpreting RMAP Packets, 1082](#)
- [RMAP\\_GetMask](#)
  - [Functions for Interpreting RMAP Packets, 1083](#)

RMAP\_GetPacketType  
     Functions for Interpreting RMAP Packets, [1083](#)  
 RMAP\_GetPerformAcknowledgement  
     Functions for Interpreting RMAP Packets, [1084](#)  
 RMAP\_GetProtocolIdentifier  
     Functions for Interpreting RMAP Packets, [1084](#)  
 RMAP\_GetReadLength  
     Functions for Interpreting RMAP Packets, [1085](#)  
 RMAP\_GetReplyAddress  
     Functions for Interpreting RMAP Packets, [1085](#)  
 RMAP\_GetStatus  
     Functions for Interpreting RMAP Packets, [1085](#)  
 RMAP\_GetTargetAddress  
     Functions for Interpreting RMAP Packets, [1087](#)  
 RMAP\_GetTransactionID  
     Functions for Interpreting RMAP Packets, [1087](#)  
 RMAP\_GetVerifyBeforeWrite  
     Functions for Interpreting RMAP Packets, [1088](#)  
 RMAP\_GetVersion  
     RMAP Packet Library, [1054](#)  
 RMAP\_INCREMENT\_ADDRESS\_BIT  
     RMAP Packet Library, [1051](#)  
 RMAP\_INVALID\_DATA\_CRC  
     RMAP Packet Library, [1053](#)  
 RMAP\_INVALID\_KEY  
     RMAP Packet Library, [1053](#)  
 RMAP\_INVALID\_PACKET\_TYPE  
     RMAP Packet Library, [1052](#)  
 RMAP\_INVALID\_STATUS  
     RMAP Packet Library, [1053](#)  
 RMAP\_INVALID\_TARGET\_LOGICAL\_ADDRESS  
     RMAP Packet Library, [1053](#)  
 RMAP\_IsCRCValid  
     RMAP Packet Library, [1054](#)  
 RMAP\_PACKET, [1047](#)  
 RMAP\_PACKET\_TYPE  
     RMAP Packet Library, [1052](#)  
 RMAP\_PROTOCOL\_IDENTIFIER  
     RMAP Packet Library, [1051](#)  
 RMAP\_READ\_COMMAND  
     RMAP Packet Library, [1052](#)  
 RMAP\_READ\_MODIFY\_WRITE\_COMMAND  
     RMAP Packet Library, [1052](#)  
 RMAP\_READ\_MODIFY\_WRITE\_REPLY  
     RMAP Packet Library, [1052](#)  
 RMAP\_READ\_REPLY  
     RMAP Packet Library, [1052](#)  
 RMAP\_REPLY\_ADDRESS\_LENGTH\_BITS  
     RMAP Packet Library, [1051](#)  
 RMAP\_REPLY\_BIT  
     RMAP Packet Library, [1051](#)  
 RMAP\_RESERVED\_BIT  
     RMAP Packet Library, [1051](#)  
 RMAP\_RMW\_DATA\_LENGTH\_ERROR  
     RMAP Packet Library, [1053](#)  
 RMAP\_STATUS  
     RMAP Packet Library, [1052](#)  
 RMAP\_SUCCESS  
     RMAP Packet Library, [1052](#)  
 RMAP\_SetPacketLibrary, [1052](#)  
 RMAP\_SetProtocolIdentifier  
     Functions for Building RMAP Packets, [1076](#)  
 RMAP\_TARGET\_PXI\_IF\_authoriseWaitingCommand  
     Manual Authorisation, [446](#)  
 RMAP\_TARGET\_PXI\_IF\_getAddressOffset  
     Configuration, [452](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthCommands  
     Configuration, [452](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthControlMode  
     Configuration, [452](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthKeyRange  
     Configuration, [453](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthLogicalAddress↔  
     Range  
     Configuration, [453](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthMemoryAddress↔  
     Range  
     Configuration, [454](#)  
 RMAP\_TARGET\_PXI\_IF\_getAuthProtocolId  
     Configuration, [454](#)  
 RMAP\_TARGET\_PXI\_IF\_getEnabledNotifications  
     Notifications, [464](#)  
 RMAP\_TARGET\_PXI\_IF\_getInterfaceMode  
     Configuration, [454](#)  
 RMAP\_TARGET\_PXI\_IF\_getStatus  
     Configuration, [456](#)  
 RMAP\_TARGET\_PXI\_IF\_getWaitingCommand  
     Manual Authorisation, [446](#)  
 RMAP\_TARGET\_PXI\_IF\_isAuthRequired  
     Manual Authorisation, [446](#)  
 RMAP\_TARGET\_PXI\_IF\_readMemory  
     Configuration, [456](#)  
 RMAP\_TARGET\_PXI\_IF\_registerNotificationListener  
     Notifications, [466](#)  
 RMAP\_TARGET\_PXI\_IF\_registerNotificationListener↔  
     WithContext  
     Notifications, [466](#)  
 RMAP\_TARGET\_PXI\_IF\_rejectWaitingCommand  
     Manual Authorisation, [447](#)  
 RMAP\_TARGET\_PXI\_IF\_setAddressOffset  
     Configuration, [457](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthCommands  
     Configuration, [457](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthControlMode  
     Configuration, [458](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthKeyRange  
     Configuration, [458](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthLogicalAddress↔  
     Range  
     Configuration, [459](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthMemoryAddress↔  
     Range  
     Configuration, [459](#)  
 RMAP\_TARGET\_PXI\_IF\_setAuthProtocolId  
     Configuration, [459](#)  
 RMAP\_TARGET\_PXI\_IF\_setEnabledNotifications  
     Notifications, [467](#)

- RMAP\_TARGET\_PXI\_IF\_setInterfaceMode Configuration, [460](#)
- RMAP\_TARGET\_PXI\_IF\_startReceivingNotifications Notifications, [467](#)
- RMAP\_TARGET\_PXI\_IF\_stopReceivingNotifications Notifications, [468](#)
- RMAP\_TARGET\_PXI\_IF\_unregisterNotificationListener Notifications, [468](#)
- RMAP\_TARGET\_PXI\_IF\_unregisterNotification↔ ListenerWithContext Notifications, [468](#)
- RMAP\_TARGET\_PXI\_IF\_writeMemory Configuration, [460](#)
- RMAP\_TOO\_MUCH\_DATA RMAP Packet Library, [1053](#)
- RMAP\_UNUSED\_PACKET\_TYPE\_OR\_COMMAND↔ \_CODE RMAP Packet Library, [1053](#)
- RMAP\_VERIFY\_BEFORE\_WRITE\_BIT RMAP Packet Library, [1051](#)
- RMAP\_VERIFY\_BUFFER\_OVERRUN RMAP Packet Library, [1053](#)
- RMAP\_WRITE\_COMMAND RMAP Packet Library, [1052](#)
- RMAP\_WRITE\_OPERATION\_BIT RMAP Packet Library, [1052](#)
- RMAP\_WRITE\_REPLY RMAP Packet Library, [1052](#)
- RMAPPACKETLIBRARY\_CC RMAP Packet Library, [1052](#)
- Receive Link Speed, [901](#)
  - CFG\_getRxSignallingRate, [901](#)
- Remote Devices, [621](#)
  - STAR\_CFG\_createRemoteDeviceIdentifier, [622](#)
  - STAR\_CFG\_destroyRemoteDeviceDescription↔ List, [622](#)
  - STAR\_CFG\_destroyRemoteDeviceIdentifier, [622](#)
  - STAR\_CFG\_getRemoteDeviceDescriptionList, [623](#)
  - STAR\_CFG\_getRemoteDeviceDescriptionName↔ String, [623](#)
  - STAR\_CFG\_getRemoteDeviceLocalChannel, [623](#)
  - STAR\_CFG\_getRemoteDeviceLocalDevice, [625](#)
  - STAR\_CFG\_getRemoteDevicePath, [625](#)
  - STAR\_CFG\_getRemoteDeviceRetPath, [625](#)
- rmap\_packet\_library.h, [1187](#)
- rmap\_target\_pxi\_if.h, [1192](#)
- rmap\_target\_types.h, [1194](#)
- RmapCommandParameters, [444](#)
- Router Configuration API, [627](#)
- Router Control and Status Configuration, [665](#)
  - CFG\_MK2\_getTimeCodeFlagMode, [668](#)
  - CFG\_MK2\_setTimeCodeFlagMode, [668](#)
  - CFG\_ROUTER\_getRouterGlobalSettings, [668](#)
  - CFG\_ROUTER\_getTimeCodeFlagMode, [670](#)
  - CFG\_ROUTER\_getTimeCodeValue, [670](#)
  - CFG\_ROUTER\_setRouterGlobalSettings, [671](#)
  - CFG\_ROUTER\_setTimeCodeFlagMode, [671](#)
- STAR\_CFG\_PORT\_TIMEOUT, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_100MS, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_100US, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_10MS, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_1MS, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_1S, [667](#)
- STAR\_CFG\_TIMEOUT\_MODE, [667](#)
- STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING, [667](#)
- STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG, [667](#)
- Router Mk2S Configuration API, [634](#)
- Routing Table, [657](#), [932](#)
  - CFG\_ROUTER\_getRoutingTableEntry, [658](#)
  - CFG\_ROUTER\_setRoutingTableEntry, [659](#)
  - CFG\_SPFI\_disableRoutingTableEntry, [933](#)
  - CFG\_SPFI\_disableRoutingTableEntryDelete↔ Header, [934](#)
  - CFG\_SPFI\_enableRoutingTableEntry, [934](#)
  - CFG\_SPFI\_enableRoutingTableEntryDelete↔ Header, [935](#)
  - CFG\_SPFI\_getRoutingTableEntryFlags, [935](#)
  - CFG\_SPFI\_getRoutingTableEntryPorts, [936](#)
  - CFG\_SPFI\_isRoutingTableEntryDeleteHeader↔ Enabled, [936](#)
  - CFG\_SPFI\_isRoutingTableEntryEnabled, [937](#)
  - CFG\_SPFI\_setRoutingTableEntryPorts, [937](#)
- SPFI\_BROADCAST\_TYPES\_INFO Types, [1038](#)
- SPFI\_DEVICE\_IDENTIFIER\_INFO Types, [1038](#)
- SPFI\_DEVICE\_INFO Types, [1038](#)
- SPFI\_LANE\_CONTROL\_SETTINGS Types, [1038](#)
- SPFI\_LANE\_EVENTS Types, [1039](#)
- SPFI\_LANE\_STATE Types, [1041](#)
- SPFI\_LANE\_STATE\_ACTIVE Types, [1041](#)
- SPFI\_LANE\_STATE\_CLEAR\_LINE Types, [1041](#)
- SPFI\_LANE\_STATE\_COLD\_RESET Types, [1041](#)
- SPFI\_LANE\_STATE\_CONNECTED Types, [1041](#)
- SPFI\_LANE\_STATE\_CONNECTING Types, [1041](#)
- SPFI\_LANE\_STATE\_DISABLED Types, [1041](#)
- SPFI\_LANE\_STATE\_INVERT\_RX\_POLARITY Types, [1041](#)
- SPFI\_LANE\_STATE\_LOSS\_OF\_SIGNAL Types, [1041](#)
- SPFI\_LANE\_STATE\_PREPARE\_STANDBY Types, [1041](#)
- SPFI\_LANE\_STATE\_STARTED Types, [1041](#)



- SPFI\_LANE\_STATE\_UNDEFINED
  - Types, [1041](#)
- SPFI\_LANE\_STATE\_WAIT
  - Types, [1041](#)
- SPFI\_LANE\_STATUS
  - Types, [1039](#)
- SPFI\_LINK\_ALIGNMENT\_STATE
  - Types, [1041](#)
- SPFI\_LINK\_ALIGNMENT\_STATE\_BOTH\_ENDS\_READY
  - Types, [1041](#)
- SPFI\_LINK\_ALIGNMENT\_STATE\_NEAR\_END\_READY
  - Types, [1041](#)
- SPFI\_LINK\_ALIGNMENT\_STATE\_NOT\_READY
  - Types, [1041](#)
- SPFI\_LINK\_ALIGNMENT\_STATE\_UNDEFINED
  - Types, [1041](#)
- SPFI\_LINK\_CONTROL\_SETTINGS
  - Types, [1039](#)
- SPFI\_LINK\_DEBUG\_SETTINGS
  - Types, [1039](#)
- SPFI\_LINK\_EVENTS
  - Types, [1039](#)
- SPFI\_LINK\_STATUS
  - Types, [1039](#)
- SPFI\_NODE\_TYPE
  - Types, [1041](#)
- SPFI\_NODE\_TYPE\_CONFIG
  - Types, [1041](#)
- SPFI\_NODE\_TYPE\_INVALID
  - Types, [1041](#)
- SPFI\_NODE\_TYPE\_OTHER
  - Types, [1041](#)
- SPFI\_NODE\_TYPE\_SOURCE\_AND\_DESTINATION
  - Types, [1041](#)
- SPFI\_PORT\_ACTION
  - Actions, [724](#)
- SPFI\_PORT\_ACTION\_ALL
  - Actions, [724](#)
- SPFI\_PORT\_ACTION\_DISCONNECT
  - Actions, [724](#)
- SPFI\_PORT\_ACTION\_MASK
  - Actions, [723](#)
- SPFI\_PORT\_ACTION\_NONE
  - Actions, [724](#)
- SPFI\_PORT\_EVENT
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_ALL
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_CONNECTED
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_DISCONNECTED
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_MASK
  - Events, [692](#)
- SPFI\_PORT\_EVENT\_NONE
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_RX\_BM
  - Events, [694](#)
- SPFI\_PORT\_EVENT\_TX\_BM
  - Events, [694](#)
- SPFI\_PORT\_INFO
  - Types, [1039](#)
- SPFI\_PORT\_TYPE
  - Types, [1041](#)
- SPFI\_PORT\_TYPE\_AXI
  - Types, [1042](#)
- SPFI\_PORT\_TYPE\_SPACEFIBRE
  - Types, [1042](#)
- SPFI\_PORT\_TYPE\_SPACEWIRE
  - Types, [1042](#)
- SPFI\_PORT\_TYPE\_UNDEFINED
  - Types, [1042](#)
- SPFI\_QUALITY\_OF\_SERVICE
  - Types, [1040](#)
- SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS
  - Types, [1040](#)
- SPFI\_VC\_ACTION
  - Actions, [724](#)
- SPFI\_VC\_ACTION\_ALL
  - Actions, [724](#)
- SPFI\_VC\_ACTION\_MASK
  - Actions, [723](#)
- SPFI\_VC\_ACTION\_NONE
  - Actions, [724](#)
- SPFI\_VC\_ACTION\_RECEIVE\_DATA
  - Actions, [724](#)
- SPFI\_VC\_ACTION\_TRANSMIT\_DATA
  - Actions, [724](#)
- SPFI\_VC\_ACTION\_TRANSMIT\_PKT
  - Actions, [724](#)
- SPFI\_VC\_ERRORS
  - Types, [1040](#)
- SPFI\_VC\_EVENT
  - Events, [694](#)
- SPFI\_VC\_EVENT\_ALL
  - Events, [694](#)
- SPFI\_VC\_EVENT\_MASK
  - Events, [692](#)
- SPFI\_VC\_EVENT\_NONE
  - Events, [694](#)
- SPFI\_VC\_EVENT\_RX\_EEP
  - Events, [694](#)
- SPFI\_VC\_EVENT\_RX\_EOP
  - Events, [694](#)
- SPFI\_VC\_EVENT\_RX\_SOP
  - Events, [694](#)
- SPFI\_VC\_EVENT\_TX\_EOP
  - Events, [694](#)
- SPFI\_VC\_EVENT\_TX\_PKT\_PENDING
  - Events, [694](#)
- SPFI\_VC\_EVENT\_TX\_SOP
  - Events, [694](#)
- SPFI\_VC\_STATUS
  - Types, [1040](#)

- SPW\_ACTION
  - Periodic Actions, [594](#)
- SPW\_ERROR
  - Error Injection, [592](#)
- SPW\_ERROR\_DECREMENT\_CREDIT
  - Error Injection, [593](#)
- SPW\_ERROR\_DISCONNECT
  - Error Injection, [592](#)
- SPW\_ERROR\_ESCAPE
  - Error Injection, [592](#)
- SPW\_ERROR\_INCREMENT\_CREDIT
  - Error Injection, [593](#)
- SPW\_ERROR\_INSERT\_FCT
  - Error Injection, [592](#)
- SPW\_ERROR\_PARITY
  - Error Injection, [592](#)
- SPW\_ERROR\_SUPPRESS\_FCT
  - Error Injection, [592](#)
- SPW\_PORT\_ACTION
  - Actions, [724](#)
- SPW\_PORT\_ACTION\_ALL
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_DECR\_CREDIT
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_DISABLE\_LVDS
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_DISCONNECT
  - Actions, [724](#)
- SPW\_PORT\_ACTION\_ESCAPE\_ERROR
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_INCR\_CREDIT
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_INSERT\_FCT
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_MASK
  - Actions, [723](#)
- SPW\_PORT\_ACTION\_NONE
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_PARITY\_ERROR
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_STOP\_RECEPTION
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_SUPPRESS\_FCT
  - Actions, [725](#)
- SPW\_PORT\_ACTION\_TRANSMIT\_PKT
  - Actions, [724](#)
- SPW\_PORT\_EVENT
  - Events, [694](#)
- SPW\_PORT\_EVENT\_ALL
  - Events, [695](#)
- SPW\_PORT\_EVENT\_CREDIT\_ERROR
  - Events, [695](#)
- SPW\_PORT\_EVENT\_DISCONNECT
  - Events, [695](#)
- SPW\_PORT\_EVENT\_ESCAPE\_ERROR
  - Events, [695](#)
- SPW\_PORT\_EVENT\_MASK
  - Events, [693](#)
- SPW\_PORT\_EVENT\_NONE
  - Events, [695](#)
- SPW\_PORT\_EVENT\_PARITY\_ERROR
  - Events, [695](#)
- SPW\_PORT\_EVENT\_RUNNING
  - Events, [695](#)
- SPW\_PORT\_EVENT\_RX\_EEP
  - Events, [695](#)
- SPW\_PORT\_EVENT\_RX\_EOP
  - Events, [695](#)
- SPW\_PORT\_EVENT\_RX\_SOP
  - Events, [695](#)
- SPW\_PORT\_EVENT\_RX\_TIME\_CODE
  - Events, [695](#)
- SPW\_PORT\_EVENT\_TX\_EOP
  - Events, [695](#)
- SPW\_PORT\_EVENT\_TX\_PKT\_PENDING
  - Events, [695](#)
- SPW\_PORT\_EVENT\_TX\_SOP
  - Events, [695](#)
- SPW\_PORT\_EVENT\_TX\_TIME\_CODE
  - Events, [695](#)
- SPW\_TRANSMIT\_PACKET
  - Periodic Actions, [594](#)
- STAR-API, [275](#)
- STAR-Dundee Types, [1130](#)
- STAR\_API\_CC
  - common.h, [1180](#)
- STAR\_API\_MODULE\_NAME
  - common.h, [1181](#)
- STAR\_APP\_ID
  - General API Functions, [278](#)
- STAR\_BROADCAST\_MESSAGE, [332](#)
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG
  - Stream Items, [334](#)
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG\_↵
  - DELAYED
  - Stream Items, [334](#)
- STAR\_BROADCAST\_MESSAGE\_STATUS\_FLAG\_L↵
  - ATE
  - Stream Items, [334](#)
- STAR\_BUS\_CPCI
  - Device and Driver Management, [296](#)
- STAR\_BUS\_PCI
  - Device and Driver Management, [296](#)
- STAR\_BUS\_PCIE
  - Device and Driver Management, [296](#)
- STAR\_BUS\_TCP
  - Device and Driver Management, [296](#)
- STAR\_BUS\_TYPE
  - Device and Driver Management, [296](#)
- STAR\_BUS\_UNKNOWN
  - Device and Driver Management, [296](#)
- STAR\_BUS\_USB
  - Device and Driver Management, [296](#)
- STAR\_BUS\_VIRTUAL
  - Device and Driver Management, [296](#)
- STAR\_CFG\_BRICK\_MK2\_ERRORS, [1140](#)

- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ
  - Link Speed, [399](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_120
  - Link Speed, [399](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_130
  - Link Speed, [399](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_140
  - Link Speed, [399](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_150
  - Link Speed, [400](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_160
  - Link Speed, [400](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_180
  - Link Speed, [400](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_FREQ\_200
  - Link Speed, [400](#)
- STAR\_CFG\_BRICK\_MK2\_LINK\_SPEED\_UNITS\_KB↔PS
  - cfg\_api\_brick\_mk2\_types.h, [1141](#)
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_10
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_100
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1000
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔EXTERNAL\_TRIGGER
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔NONE
  - Timestamping, [418](#)
- STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METHOD↔PULSE\_GENERATOR
  - Timestamping, [418](#)
- STAR\_CFG\_CONFIG\_PORT\_ERRORS, [644](#)
- STAR\_CFG\_DEVICE\_IDENTIFIER\_INFO, [638](#)
- STAR\_CFG\_DEVICE\_STR\_MAX\_LEN
  - Device Information, [639](#)
- STAR\_CFG\_DEVICE\_TYPE
  - Device Information, [640](#)
- STAR\_CFG\_DEVICE\_TYPE\_INVALID
  - Device Information, [640](#)
- STAR\_CFG\_DEVICE\_TYPE\_ROUTER
  - Device Information, [640](#)
- STAR\_CFG\_DEVICE\_TYPE\_UNKNOWN
  - Device Information, [640](#)
- STAR\_CFG\_EXTERNAL\_PORT\_ERRORS, [648](#)
- STAR\_CFG\_EXTERNAL\_PORT\_STATUS, [648](#)
- STAR\_CFG\_FPGA\_INFO, [856](#)
- STAR\_CFG\_GAR\_ENTRY, [657](#)
- STAR\_CFG\_MANUFACTURER\_STR\_MAX\_LEN
  - Device Information, [639](#)
- STAR\_CFG\_MK2\_BASE\_TRANSMIT\_CLOCK, [566](#)
- STAR\_CFG\_MK2\_HARDWARE\_INFO, [581](#)
- STAR\_CFG\_MK2\_LINK\_SPEED\_UNITS\_KBPS
  - Measured Link Speed, [572](#)
- STAR\_CFG\_NETWORK\_DISCOVERY\_INFO, [638](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ
  - Link Speed, [384](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_120
  - Link Speed, [384](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_128
  - Link Speed, [384](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_140
  - Link Speed, [385](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_150
  - Link Speed, [385](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_160
  - Link Speed, [385](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_180
  - Link Speed, [385](#)
- STAR\_CFG\_PCIMK2\_LINK\_FREQ\_200
  - Link Speed, [385](#)
- STAR\_CFG\_PORT\_TIMEOUT
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_100MS
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_100US
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_10MS
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_1MS
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TIMEOUT\_1S
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_PORT\_TYPE
  - Port Status and Control, [649](#)
- STAR\_CFG\_PORT\_TYPE\_CONFIGURATION
  - Port Status and Control, [649](#)
- STAR\_CFG\_PORT\_TYPE\_EXTERNAL
  - Port Status and Control, [650](#)
- STAR\_CFG\_PORT\_TYPE\_INVALID
  - Port Status and Control, [650](#)
- STAR\_CFG\_PORT\_TYPE\_LINK
  - Port Status and Control, [650](#)
- STAR\_CFG\_ROUTER\_GLOBAL\_STATE, [666](#)
- STAR\_CFG\_SPW\_LINK\_ERRORS, [646](#)
- STAR\_CFG\_SPW\_LINK\_STATE
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_CONNECTING
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_RESET
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_ERROR\_WAIT
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_INVALID
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_READY
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_RUN

- Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATE\_STARTED
  - Port Status and Control, [650](#)
- STAR\_CFG\_SPW\_LINK\_STATUS, [647](#)
- STAR\_CFG\_TIMEOUT\_MODE
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_TIMEOUT\_MODE\_BLOCKING
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_TIMEOUT\_MODE\_WATCHDOG
  - Router Control and Status Configuration, [667](#)
- STAR\_CFG\_createRemoteDeviceIdentifier
  - Remote Devices, [622](#)
- STAR\_CFG\_destroyRemoteDeviceDescriptionList
  - Remote Devices, [622](#)
- STAR\_CFG\_destroyRemoteDeviceIdentifier
  - Remote Devices, [622](#)
- STAR\_CFG\_getRemoteDeviceDescriptionList
  - Remote Devices, [623](#)
- STAR\_CFG\_getRemoteDeviceDescriptionNameString
  - Remote Devices, [623](#)
- STAR\_CFG\_getRemoteDeviceLocalChannel
  - Remote Devices, [623](#)
- STAR\_CFG\_getRemoteDeviceLocalDevice
  - Remote Devices, [625](#)
- STAR\_CFG\_getRemoteDevicePath
  - Remote Devices, [625](#)
- STAR\_CFG\_getRemoteDeviceRetPath
  - Remote Devices, [625](#)
- STAR\_CHANNEL\_DIRECTION
  - Channel Management, [315](#)
- STAR\_CHANNEL\_DIRECTION\_IN
  - Channel Management, [315](#)
- STAR\_CHANNEL\_DIRECTION\_INOUT
  - Channel Management, [315](#)
- STAR\_CHANNEL\_DIRECTION\_OUT
  - Channel Management, [315](#)
- STAR\_CHANNEL\_ID
  - Channel Management, [314](#)
- STAR\_CHANNEL\_LISTENER\_ID
  - Channel Management, [314](#)
- STAR\_CHANNEL\_MASK
  - Device and Driver Management, [286](#)
- STAR\_CHANNEL\_TYPE
  - Channel Management, [315](#)
- STAR\_CHANNEL\_TYPE\_APPLICATION
  - Channel Management, [315](#)
- STAR\_CHANNEL\_TYPE\_DEVICE
  - Channel Management, [315](#)
- STAR\_CHANNEL\_TYPE\_NOT\_OPEN
  - Channel Management, [315](#)
- STAR\_ChannelEventFunc
  - Channel Management, [314](#)
- STAR\_DATA\_CHUNK, [327](#)
- STAR\_DEVICE\_ALL
  - Device and Driver Management, [286](#)
- STAR\_DEVICE\_BRICK\_MK2
  - Device and Driver Management, [287](#)
- STAR\_DEVICE\_BRICK\_MK3
  - Device and Driver Management, [287](#)
- STAR\_DEVICE\_BRICK\_MK4
  - Device and Driver Management, [287](#)
- STAR\_DEVICE\_CONFIG\_NOT\_SUPPORTED
  - Device and Driver Management, [287](#)
- STAR\_DEVICE\_CONFIG\_SUPPORTED
  - Device and Driver Management, [287](#)
- STAR\_DEVICE\_CONFORMANCE\_TESTER
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_CONFORMANCE\_TESTER\_MK2
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_CPCI\_MK2
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_EGSE
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_EGSE\_MK2
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_ID
  - Device and Driver Management, [295](#)
- STAR\_DEVICE\_INFO, [1154](#)
- STAR\_DEVICE\_IP\_TUNNEL
  - Device and Driver Management, [288](#)
- STAR\_DEVICE\_LINK\_ANALYSER
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_LINK\_ANALYSER\_MK2
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_LINK\_ANALYSER\_MK3
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_LISTENER\_ID
  - Device and Driver Management, [295](#)
- STAR\_DEVICE\_PCI
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_PCI\_MK2
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_PCIE
  - Device and Driver Management, [289](#)
- STAR\_DEVICE\_PCIE\_MK2
  - Device and Driver Management, [290](#)
- STAR\_DEVICE\_PXI\_INTERFACE
  - Device and Driver Management, [290](#)
- STAR\_DEVICE\_PXI\_INTERFACE\_MK2
  - Device and Driver Management, [290](#)
- STAR\_DEVICE\_PXI\_RMAP
  - Device and Driver Management, [290](#)
- STAR\_DEVICE\_PXI\_RMAP\_MK2
  - Device and Driver Management, [290](#)
- STAR\_DEVICE\_PXI\_ROUTER\_12
  - Device and Driver Management, [291](#)
- STAR\_DEVICE\_PXI\_ROUTER\_8
  - Device and Driver Management, [291](#)
- STAR\_DEVICE\_PXI\_ROUTER\_MK2
  - Device and Driver Management, [291](#)
- STAR\_DEVICE\_RECORDER
  - Device and Driver Management, [291](#)
- STAR\_DEVICE\_RECORDER\_MK2
  - Device and Driver Management, [291](#)
- STAR\_DEVICE\_ROUTER\_MK2S
  - Device and Driver Management, [292](#)

- STAR\_DEVICE\_ROUTER\_USB
  - Device and Driver Management, [292](#)
- STAR\_DEVICE\_RTC
  - Device and Driver Management, [292](#)
- STAR\_DEVICE\_SERIAL\_SIZE
  - Device and Driver Management, [292](#)
- STAR\_DEVICE\_SPLT
  - Device and Driver Management, [292](#)
- STAR\_DEVICE\_STAR\_FIRE
  - Device and Driver Management, [292](#)
- STAR\_DEVICE\_STAR\_FIRE\_MK3
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_STAR\_ULTRA\_PCIE
  - types.h, [1239](#)
- STAR\_DEVICE\_STAR\_ULTRA\_PCIE\_INTERFACE
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_STAR\_ULTRA\_PCIE\_SL\_ROUTER
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_TRAFFIC\_GENERATOR
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_TXRX\_NOT\_SUPPORTED
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_TXRX\_SUPPORTED
  - Device and Driver Management, [293](#)
- STAR\_DEVICE\_TYPE
  - Device and Driver Management, [294](#)
- STAR\_DEVICE\_UNKNOWN
  - Device and Driver Management, [294](#)
- STAR\_DEVICE\_USB\_BRICK
  - Device and Driver Management, [294](#)
- STAR\_DEVICE\_WBS\_II
  - Device and Driver Management, [294](#)
- STAR\_DRIVER\_ID
  - Device and Driver Management, [295](#)
- STAR\_DRIVER\_INVALID
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_LISTENER\_ID
  - Device and Driver Management, [295](#)
- STAR\_DRIVER\_TYPE
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_PCI
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_PEP\_PCIE
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_TCPIP
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_ULTRA\_PCIE
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_USB
  - Device and Driver Management, [296](#)
- STAR\_DRIVER\_TYPE\_VIRTUAL
  - Device and Driver Management, [296](#)
- STAR\_DeviceEventFunc
  - Device and Driver Management, [295](#)
- STAR\_DriverEventFunc
  - Device and Driver Management, [295](#)
- STAR\_EOP\_TYPE
  - Stream Items, [334](#)
- STAR\_EOP\_TYPE\_EEP
  - Stream Items, [335](#)
- STAR\_EOP\_TYPE\_EOP
  - Stream Items, [334](#)
- STAR\_EOP\_TYPE\_INVALID
  - Stream Items, [334](#)
- STAR\_EOP\_TYPE\_NONE
  - Stream Items, [335](#)
- STAR\_ERROR\_IN\_DATA\_DISCONNECT
  - Stream Items, [335](#)
- STAR\_ERROR\_IN\_DATA\_INJECT, [330](#)
- STAR\_ERROR\_IN\_DATA\_PARITY
  - Stream Items, [335](#)
- STAR\_ERROR\_IN\_DATA\_TYPE
  - Stream Items, [335](#)
- STAR\_IP\_ADDRESS\_TYPE, [324](#)
- STAR\_IP\_VERSION\_NOT\_DEFINED
  - Stream Items, [335](#)
- STAR\_IP\_VERSION\_TYPE
  - Stream Items, [335](#)
- STAR\_IPv4
  - Stream Items, [335](#)
- STAR\_IPv6
  - Stream Items, [335](#)
- STAR\_LINK\_SPEED\_EVENT, [329](#)
- STAR\_LINK\_STATE\_EVENT, [325](#)
- STAR\_RECEIVE\_BROADCAST\_MESSAGES
  - Transfer Operations, [361](#)
- STAR\_RECEIVE\_CHUNKS
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_FCTS
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_LINK\_SPEED\_EVENTS
  - Transfer Operations, [361](#)
- STAR\_RECEIVE\_LINK\_STATE\_EVENTS
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_MASK
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_NULL
  - Transfer Operations, [359](#)
- STAR\_RECEIVE\_NULLS
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_PACKETS
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_TIMECODES
  - Transfer Operations, [360](#)
- STAR\_RECEIVE\_TIMESTAMP\_EVENTS
  - Transfer Operations, [361](#)
- STAR\_SPACEWIRE\_ADDRESS, [326](#)
- STAR\_SPACEWIRE\_PACKET, [328](#)
- STAR\_STREAM\_ITEM, [333](#)
- STAR\_STREAM\_ITEM\_TYPE
  - Stream Items, [335](#)
- STAR\_STREAM\_ITEM\_TYPE\_BROADCAST\_MESSAGE
  - Stream Items, [336](#)
- STAR\_STREAM\_ITEM\_TYPE\_DATA\_CHUNK
  - Stream Items, [335](#)

- STAR\_STREAM\_ITEM\_TYPE\_ERROR\_INJECT
  - Stream Items, [336](#)
- STAR\_STREAM\_ITEM\_TYPE\_LINK\_SPEED\_EVENT
  - Stream Items, [335](#)
- STAR\_STREAM\_ITEM\_TYPE\_LINK\_STATE\_EVENT
  - Stream Items, [335](#)
- STAR\_STREAM\_ITEM\_TYPE\_SPACEWIRE\_PACKET
  - Stream Items, [335](#)
- STAR\_STREAM\_ITEM\_TYPE\_TIMECODE
  - Stream Items, [335](#)
- STAR\_STREAM\_ITEM\_TYPE\_TIMESTAMP\_EVENT
  - Stream Items, [336](#)
- STAR\_TIMECODE, [329](#)
- STAR\_TIMESTAMP\_DIRECTION
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_DIRECTION\_RECEIVE
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_DIRECTION\_TRANSMIT
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_EVENT, [331](#)
- STAR\_TIMESTAMP\_TYPE
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_TYPE\_DATA
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_TYPE\_ERROR
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_TYPE\_LINK\_SPEED
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_TYPE\_LINK\_STATE
  - Stream Items, [336](#)
- STAR\_TIMESTAMP\_TYPE\_TIMECODE
  - Stream Items, [336](#)
- STAR\_TRANSFER\_OPERATION
  - Transfer Operations, [360](#)
- STAR\_TRANSFER\_STATUS
  - Transfer Operations, [361](#)
- STAR\_TRANSFER\_STATUS\_CANCELLED
  - Transfer Operations, [361](#)
- STAR\_TRANSFER\_STATUS\_COMPLETE
  - Transfer Operations, [361](#)
- STAR\_TRANSFER\_STATUS\_ERROR
  - Transfer Operations, [361](#)
- STAR\_TRANSFER\_STATUS\_NOT\_STARTED
  - Transfer Operations, [361](#)
- STAR\_TRANSFER\_STATUS\_STARTED
  - Transfer Operations, [361](#)
- STAR\_TransferOperationListenerFunc
  - Transfer Operations, [360](#)
- STAR\_VERSION\_INFO, [1240](#)
- STAR\_cancelTransferOperation
  - Transfer Operations, [361](#)
- STAR\_cancelTransferOperationWaits
  - Transfer Operations, [362](#)
- STAR\_closeChannel
  - Channel Management, [315](#)
- STAR\_closeEthernetDevice
  - Ethernet Devices and Drivers, [381](#)
- STAR\_closeEthernetDriver
  - Ethernet Devices and Drivers, [382](#)
- STAR\_createAddress
  - Stream Items, [336](#)
- STAR\_createBroadcastMessage
  - Stream Items, [337](#)
- STAR\_createDataChunk
  - Stream Items, [337](#)
- STAR\_createErrorInData
  - Stream Items, [338](#)
- STAR\_createPacket
  - Stream Items, [338](#)
- STAR\_createRxOperation
  - Transfer Operations, [362](#)
- STAR\_createRxOperationEx
  - Transfer Operations, [363](#)
- STAR\_createTimeCode
  - Stream Items, [339](#)
- STAR\_createTxOperation
  - Transfer Operations, [364](#)
- STAR\_destroyAddress
  - Stream Items, [339](#)
- STAR\_destroyDeviceList
  - Device and Driver Management, [297](#)
- STAR\_destroyDriverList
  - Device and Driver Management, [298](#)
- STAR\_destroyIPAddresses
  - Ethernet Devices and Drivers, [382](#)
- STAR\_destroyPacketData
  - Stream Items, [340](#)
- STAR\_destroyStreamItem
  - Stream Items, [340](#)
- STAR\_destroyString
  - General API Functions, [278](#)
- STAR\_destroyTransferItemList
  - Transfer Operations, [364](#)
- STAR\_destroyVersionInfo
  - General API Functions, [278](#)
- STAR\_destroyVersionInfoList
  - General API Functions, [278](#)
- STAR\_disposeTransferOperation
  - Transfer Operations, [365](#)
- STAR\_getAllVersions
  - General API Functions, [279](#)
- STAR\_getApiVersion
  - General API Functions, [279](#)
- STAR\_getApplicationAttachedToChannel
  - General API Functions, [279](#)
- STAR\_getApplicationID
  - General API Functions, [280](#)
- STAR\_getApplicationName
  - General API Functions, [280](#)
- STAR\_getApplicationNameForID
  - General API Functions, [280](#)
- STAR\_getBroadcastMessageChannel
  - Stream Items, [340](#)
- STAR\_getBroadcastMessageDataWord1
  - Stream Items, [342](#)



- STAR\_getBroadcastMessageDataWord2
  - Stream Items, 342
- STAR\_getBroadcastMessageDelayedFlag
  - Stream Items, 342
- STAR\_getBroadcastMessageLateFlag
  - Stream Items, 344
- STAR\_getBroadcastMessageStatus
  - Stream Items, 344
- STAR\_getBroadcastMessageType
  - Stream Items, 344
- STAR\_getChannelDirection
  - Channel Management, 316
- STAR\_getChunkData
  - Stream Items, 345
- STAR\_getChunkDataLength
  - Stream Items, 345
- STAR\_getChunkEop
  - Stream Items, 345
- STAR\_getChunkSop
  - Stream Items, 346
- STAR\_getDeviceBusType
  - Device and Driver Management, 298
- STAR\_getDeviceBusTypeAsString
  - Device and Driver Management, 298
- STAR\_getDeviceChannels
  - Device and Driver Management, 299
- STAR\_getDeviceConfigCapabilities
  - Device and Driver Management, 299
- STAR\_getDeviceDriver
  - Device and Driver Management, 299
- STAR\_getDeviceFirmwareVersion
  - Device and Driver Management, 300
- STAR\_getDeviceIPAddress
  - Ethernet Devices and Drivers, 382
- STAR\_getDeviceIndex
  - Device and Driver Management, 300
- STAR\_getDeviceList
  - Device and Driver Management, 300
- STAR\_getDeviceListForDriver
  - Device and Driver Management, 302
- STAR\_getDeviceListForType
  - Device and Driver Management, 302
- STAR\_getDeviceListForTypes
  - Device and Driver Management, 303
- STAR\_getDeviceName
  - Device and Driver Management, 303
- STAR\_getDeviceSerialNumber
  - Device and Driver Management, 304
- STAR\_getDeviceTxRxCapabilities
  - Device and Driver Management, 304
- STAR\_getDeviceType
  - Device and Driver Management, 306
- STAR\_getDeviceTypeAsString
  - Device and Driver Management, 306
- STAR\_getDriverList
  - Device and Driver Management, 306
- STAR\_getDriverType
  - Device and Driver Management, 308
- STAR\_getDriverVersion
  - Device and Driver Management, 308
- STAR\_getIPAddresses
  - Ethernet Devices and Drivers, 383
- STAR\_getLinkSpeedEventPort
  - Stream Items, 346
- STAR\_getLinkSpeedEventSpeed
  - Stream Items, 346
- STAR\_getLinkStateEventCount
  - Stream Items, 347
- STAR\_getLinkStateEventPort
  - Stream Items, 347
- STAR\_getLocalDeviceAttachedToChannel
  - Device and Driver Management, 309
- STAR\_getNextTransferItem
  - Transfer Operations, 365
- STAR\_getPacketAddress
  - Stream Items, 347
- STAR\_getPacketData
  - Stream Items, 348
- STAR\_getPacketEOP
  - Stream Items, 348
- STAR\_getPacketLength
  - Stream Items, 349
- STAR\_getSpaceWireAddressPath
  - Transfer Operations, 366
- STAR\_getSpaceWireAddressPathLength
  - Transfer Operations, 366
- STAR\_getTimeCodeCount
  - Stream Items, 349
- STAR\_getTimeCodeValue
  - Stream Items, 349
- STAR\_getTimestampEventDirection
  - Stream Items, 350
- STAR\_getTimestampEventEndValue
  - Stream Items, 350
- STAR\_getTimestampEventRawCounters
  - Stream Items, 350
- STAR\_getTimestampEventStartValue
  - Stream Items, 352
- STAR\_getTimestampEventType
  - Stream Items, 352
- STAR\_getTransferItem
  - Transfer Operations, 366
- STAR\_getTransferItemCount
  - Transfer Operations, 367
- STAR\_getTransferItemList
  - Transfer Operations, 367
- STAR\_getTransferOperationStatus
  - Transfer Operations, 368
- STAR\_isChannelOpen
  - Device and Driver Management, 309
- STAR\_isDeviceVirtual
  - Virtual Devices and Drivers, 379
- STAR\_isDriverVirtual
  - Virtual Devices and Drivers, 379
- STAR\_isLinkStateEventDisconnectError
  - Stream Items, 353

- STAR\_isLinkStateEventEscapeError
  - Stream Items, [353](#)
- STAR\_isLinkStateEventLinkRunning
  - Stream Items, [353](#)
- STAR\_isLinkStateEventParityError
  - Stream Items, [354](#)
- STAR\_isLinkStateEventReceiveCreditError
  - Stream Items, [354](#)
- STAR\_isLinkStateEventTransmitCreditError
  - Stream Items, [354](#)
- STAR\_openChannelBetweenLocalDevices
  - Channel Management, [316](#)
- STAR\_openChannelToLocalDevice
  - Channel Management, [316](#)
- STAR\_openChannelToLocalDeviceEx
  - Channel Management, [317](#)
- STAR\_openEthernetDevice
  - Ethernet Devices and Drivers, [383](#)
- STAR\_openEthernetDriver
  - Ethernet Devices and Drivers, [383](#)
- STAR\_receivePacket
  - Transfer Operations, [368](#)
- STAR\_registerChannelListener
  - Channel Management, [318](#)
- STAR\_registerChannelListenerForDevice
  - Channel Management, [319](#)
- STAR\_registerChannelListenerForDriver
  - Channel Management, [319](#)
- STAR\_registerDeviceListener
  - Device and Driver Management, [309](#)
- STAR\_registerDeviceListenerForDevice
  - Device and Driver Management, [310](#)
- STAR\_registerDeviceListenerForDriver
  - Device and Driver Management, [310](#)
- STAR\_registerDriverListener
  - Device and Driver Management, [310](#)
- STAR\_registerTransferCompletionListener
  - Transfer Operations, [369](#)
- STAR\_resetDevice
  - Device and Driver Management, [311](#)
- STAR\_resetDeviceName
  - Device and Driver Management, [311](#)
- STAR\_rxOperationExGetBroadcastMessage
  - Transfer Operations, [369](#)
- STAR\_rxOperationExGetDataChunk
  - Transfer Operations, [370](#)
- STAR\_rxOperationExGetItemType
  - Transfer Operations, [370](#)
- STAR\_rxOperationExGetLinkSpeedEvent
  - Transfer Operations, [370](#)
- STAR\_rxOperationExGetLinkStateEvent
  - Transfer Operations, [372](#)
- STAR\_rxOperationExGetNumItems
  - Transfer Operations, [372](#)
- STAR\_rxOperationExGetStatus
  - Transfer Operations, [372](#)
- STAR\_rxOperationExGetTimeCode
  - Transfer Operations, [374](#)
- STAR\_rxOperationExGetTimestampEvent
  - Transfer Operations, [374](#)
- STAR\_setApplicationName
  - General API Functions, [281](#)
- STAR\_setDeviceName
  - Device and Driver Management, [311](#)
- STAR\_setPacketAddress
  - Stream Items, [354](#)
- STAR\_setPacketEOP
  - Stream Items, [356](#)
- STAR\_setTimeCodeValue
  - Stream Items, [356](#)
- STAR\_submitTransferOperation
  - Transfer Operations, [374](#)
- STAR\_submitTransferOperationList
  - Transfer Operations, [376](#)
- STAR\_transmitPacket
  - Transfer Operations, [376](#)
- STAR\_unregisterChannelListener
  - Channel Management, [320](#)
- STAR\_unregisterDeviceListener
  - Device and Driver Management, [312](#)
- STAR\_unregisterDriverListener
  - Device and Driver Management, [312](#)
- STAR\_unregisterTransferCompletionListener
  - Transfer Operations, [377](#)
- STAR\_waitOnTransferOperationCompletion
  - Transfer Operations, [377](#)
- STATISTICS\_LOOP\_ID
  - PCI Mk2 Packet Subsystem, [600](#)
- SpaceFibre Configuration API, [616](#)
- SpaceFibre Ports, [543](#)
  - TRIGGER\_CFG\_getNumberOfSpFiPorts, [543](#)
  - TRIGGER\_CFG\_getSpFiPortInputEvents, [544](#)
  - TRIGGER\_CFG\_getSpFiPortOutputActions, [544](#)
  - TRIGGER\_CFG\_setSpFiPortInputEvents, [544](#)
  - TRIGGER\_CFG\_setSpFiPortOutputActions, [546](#)
- SpaceFibre Virtual Channels, [547](#)
  - TRIGGER\_CFG\_disableSpFiVCReceiveData↔
    - Mode, [548](#)
  - TRIGGER\_CFG\_disableSpFiVCTransmitData↔
    - Mode, [548](#)
  - TRIGGER\_CFG\_disableSpFiVCTransmitPacket↔
    - Mode, [549](#)
  - TRIGGER\_CFG\_enableSpFiVCReceiveData↔
    - Mode, [549](#)
  - TRIGGER\_CFG\_enableSpFiVCTransmitData↔
    - Mode, [549](#)
  - TRIGGER\_CFG\_enableSpFiVCTransmitPacket↔
    - Mode, [550](#)
  - TRIGGER\_CFG\_getNumberOfSpFIVCs, [550](#)
  - TRIGGER\_CFG\_getSpFIVCInputEvents, [551](#)
  - TRIGGER\_CFG\_getSpFIVCOutputActions, [551](#)
  - TRIGGER\_CFG\_getSpFIVCReceiveDataMode↔
    - Enabled, [552](#)
  - TRIGGER\_CFG\_getSpFIVCTransmitDataMode↔
    - Enabled, [552](#)



- TRIGGER\_CFG\_getSpFiVCTransmitPacket↔  
ModeEnabled, 552
- TRIGGER\_CFG\_setSpFiVCInputEvents, 553
- TRIGGER\_CFG\_setSpFiVCOOutputActions, 553
- SpaceWire Ports, 516
  - TRIGGER\_CFG\_disablePortSpill, 517
  - TRIGGER\_CFG\_disablePortTransmitPacketMode,  
517
  - TRIGGER\_CFG\_disableSpWPortSpill, 518
  - TRIGGER\_CFG\_disableSpWPortTransmit↔  
PacketMode, 518
  - TRIGGER\_CFG\_enablePortSpill, 519
  - TRIGGER\_CFG\_enablePortTransmitPacketMode,  
519
  - TRIGGER\_CFG\_enableSpWPortSpill, 519
  - TRIGGER\_CFG\_enableSpWPortTransmitPacket↔  
Mode, 521
  - TRIGGER\_CFG\_getNumberOfSpWPorts, 521
  - TRIGGER\_CFG\_getPortInputEvents, 522
  - TRIGGER\_CFG\_getPortOutputActions, 522
  - TRIGGER\_CFG\_getPortSpillEnabled, 523
  - TRIGGER\_CFG\_getPortTransmitPacketMode↔  
Enabled, 523
  - TRIGGER\_CFG\_getSpWPortInputEvents, 524
  - TRIGGER\_CFG\_getSpWPortOutputActions, 524
  - TRIGGER\_CFG\_getSpWPortSpillEnabled, 525
  - TRIGGER\_CFG\_getSpWPortTransmitPacket↔  
ModeEnabled, 525
  - TRIGGER\_CFG\_setPortInputEvents, 525
  - TRIGGER\_CFG\_setPortOutputActions, 526
  - TRIGGER\_CFG\_setSpWPortInputEvents, 526
  - TRIGGER\_CFG\_setSpWPortOutputActions, 527
- star-api.h, 1195
- star-dundee\_annotations.h, 1196
- star-dundee\_types.h, 1196
- Status, 958, 991, 1019
  - CFG\_SPFI\_clearLinkProtocolError, 991
  - CFG\_SPFI\_getLaneFarEndINIT3Flags, 1020
  - CFG\_SPFI\_getLaneLOSReasonRx, 1021
  - CFG\_SPFI\_getLaneStandbyReasonRx, 1021
  - CFG\_SPFI\_getLaneState, 1022
  - CFG\_SPFI\_getLaneStatus, 1022
  - CFG\_SPFI\_getLinkAlignmentState, 992
  - CFG\_SPFI\_getLinkStatus, 992
  - CFG\_SPFI\_getVCStatus, 958
  - CFG\_SPFI\_isLaneActive, 1023
  - CFG\_SPFI\_isLaneErrorDetected, 1023
  - CFG\_SPFI\_isLaneEventDetected, 1024
  - CFG\_SPFI\_isLaneNoSignalDetected, 1024
  - CFG\_SPFI\_isLaneRxOnly, 1024
  - CFG\_SPFI\_isLaneStarted, 1026
  - CFG\_SPFI\_isLaneTxOnly, 1026
  - CFG\_SPFI\_isLinkErrorDetected, 993
  - CFG\_SPFI\_isLinkEventDetected, 993
  - CFG\_SPFI\_isLinkIdle, 994
  - CFG\_SPFI\_isLinkLanesEventDetected, 994
  - CFG\_SPFI\_isLinkProtocolErrorDetected, 995
  - CFG\_SPFI\_isLinkReady, 995
  - CFG\_SPFI\_isLinkReceiving, 995
  - CFG\_SPFI\_isLinkTransmitting, 996
  - CFG\_SPFI\_isVCInputBufferFull, 959
  - CFG\_SPFI\_isVCInputBufferHalfFull, 959
  - CFG\_SPFI\_isVCInputBufferNotEmpty, 960
  - CFG\_SPFI\_isVCOOutputBufferFull, 960
  - CFG\_SPFI\_isVCOOutputBufferHalfFull, 961
  - CFG\_SPFI\_isVCOOverused, 961
  - CFG\_SPFI\_isVCUnderused, 961
- Stream Items, 321
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔  
AG, 334
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔  
AG\_DELAYED, 334
  - STAR\_BROADCAST\_MESSAGE\_STATUS\_FL↔  
AG\_LATE, 334
  - STAR\_EOP\_TYPE, 334
  - STAR\_EOP\_TYPE\_EEP, 335
  - STAR\_EOP\_TYPE\_EOP, 334
  - STAR\_EOP\_TYPE\_INVALID, 334
  - STAR\_EOP\_TYPE\_NONE, 335
  - STAR\_ERROR\_IN\_DATA\_DISCONNECT, 335
  - STAR\_ERROR\_IN\_DATA\_PARITY, 335
  - STAR\_ERROR\_IN\_DATA\_TYPE, 335
  - STAR\_IP\_VERSION\_NOT\_DEFINED, 335
  - STAR\_IP\_VERSION\_TYPE, 335
  - STAR\_IPv4, 335
  - STAR\_IPv6, 335
  - STAR\_STREAM\_ITEM\_TYPE, 335
  - STAR\_STREAM\_ITEM\_TYPE\_BROADCAST↔  
MESSAGE, 336
  - STAR\_STREAM\_ITEM\_TYPE\_DATA\_CHUNK,  
335
  - STAR\_STREAM\_ITEM\_TYPE\_ERROR\_INJECT,  
336
  - STAR\_STREAM\_ITEM\_TYPE\_LINK\_SPEED\_E↔  
VENT, 335
  - STAR\_STREAM\_ITEM\_TYPE\_LINK\_STATE\_E↔  
VENT, 335
  - STAR\_STREAM\_ITEM\_TYPE\_SPACEWIRE\_P↔  
ACKET, 335
  - STAR\_STREAM\_ITEM\_TYPE\_TIMECODE, 335
  - STAR\_STREAM\_ITEM\_TYPE\_TIMESTAMP\_E↔  
VENT, 336
  - STAR\_TIMESTAMP\_DIRECTION, 336
  - STAR\_TIMESTAMP\_DIRECTION\_RECEIVE, 336
  - STAR\_TIMESTAMP\_DIRECTION\_TRANSMIT,  
336
  - STAR\_TIMESTAMP\_TYPE, 336
  - STAR\_TIMESTAMP\_TYPE\_DATA, 336
  - STAR\_TIMESTAMP\_TYPE\_ERROR, 336
  - STAR\_TIMESTAMP\_TYPE\_LINK\_SPEED, 336
  - STAR\_TIMESTAMP\_TYPE\_LINK\_STATE, 336
  - STAR\_TIMESTAMP\_TYPE\_TIMECODE, 336
  - STAR\_createAddress, 336
  - STAR\_createBroadcastMessage, 337
  - STAR\_createDataChunk, 337
  - STAR\_createErrorInData, 338

- STAR\_createPacket, 338
- STAR\_createTimeCode, 339
- STAR\_destroyAddress, 339
- STAR\_destroyPacketData, 340
- STAR\_destroyStreamItem, 340
- STAR\_getBroadcastMessageChannel, 340
- STAR\_getBroadcastMessageDataWord1, 342
- STAR\_getBroadcastMessageDataWord2, 342
- STAR\_getBroadcastMessageDelayedFlag, 342
- STAR\_getBroadcastMessageLateFlag, 344
- STAR\_getBroadcastMessageStatus, 344
- STAR\_getBroadcastMessageType, 344
- STAR\_getChunkData, 345
- STAR\_getChunkDataLength, 345
- STAR\_getChunkEop, 345
- STAR\_getChunkSop, 346
- STAR\_getLinkSpeedEventPort, 346
- STAR\_getLinkSpeedEventSpeed, 346
- STAR\_getLinkStateEventCount, 347
- STAR\_getLinkStateEventPort, 347
- STAR\_getPacketAddress, 347
- STAR\_getPacketData, 348
- STAR\_getPacketEOP, 348
- STAR\_getPacketLength, 349
- STAR\_getTimeCodeCount, 349
- STAR\_getTimeCodeValue, 349
- STAR\_getTimestampEventDirection, 350
- STAR\_getTimestampEventEndValue, 350
- STAR\_getTimestampEventRawCounters, 350
- STAR\_getTimestampEventStartValue, 352
- STAR\_getTimestampEventType, 352
- STAR\_isLinkStateEventDisconnectError, 353
- STAR\_isLinkStateEventEscapeError, 353
- STAR\_isLinkStateEventLinkRunning, 353
- STAR\_isLinkStateEventParityError, 354
- STAR\_isLinkStateEventReceiveCreditError, 354
- STAR\_isLinkStateEventTransmitCreditError, 354
- STAR\_setPacketAddress, 354
- STAR\_setPacketEOP, 356
- STAR\_setTimeCodeValue, 356
- stream\_item.h, 1197
- stream\_item\_types.h, 1200
- TARGET\_AUTH\_CMD
  - Configuration, 451
- TARGET\_AUTH\_CMD\_MASK
  - Configuration, 451
- TARGET\_AUTH\_MODE
  - Configuration, 451
- TARGET\_AUTH\_MODE\_AUTOMATIC
  - Configuration, 451
- TARGET\_AUTH\_MODE\_MANUAL
  - Configuration, 451
- TARGET\_ENGINE, 450
- TARGET\_STATUS
  - Configuration, 451
- TIME\_CODE\_ACTION
  - Actions, 725
- TIME\_CODE\_ACTION\_ALL
  - Actions, 725
- TIME\_CODE\_ACTION\_MASK
  - Events, 693
- TIME\_CODE\_ACTION\_NONE
  - Actions, 725
- TIME\_CODE\_ACTION\_TX
  - Actions, 725
- TIME\_CODE\_EVENT
  - Events, 695
- TIME\_CODE\_EVENT\_ALL
  - Events, 695
- TIME\_CODE\_EVENT\_MASK
  - Events, 693
- TIME\_CODE\_EVENT\_NONE
  - Events, 695
- TIME\_CODE\_EVENT\_TICK
  - Events, 695
- TRIGGER\_BRICK\_MK3\_disableCounterAutoReload
  - Configuration, 753
- TRIGGER\_BRICK\_MK3\_disableCounterLoadZero
  - Configuration, 753
- TRIGGER\_BRICK\_MK3\_disableCounterStartMode
  - Configuration, 753
- TRIGGER\_BRICK\_MK3\_disableCounterStartStop↔
  - Mode
  - Configuration, 754
- TRIGGER\_BRICK\_MK3\_disableCounterTriggerCount
  - Configuration, 754
- TRIGGER\_BRICK\_MK3\_disableExtTriggerEdge↔
  - DetectMode
  - Configuration, 754
- TRIGGER\_BRICK\_MK3\_disableExtTriggerInvert
  - Configuration, 755
- TRIGGER\_BRICK\_MK3\_disableExtTriggerOutput
  - Configuration, 755
- TRIGGER\_BRICK\_MK3\_disablePortSpill
  - Configuration, 756
- TRIGGER\_BRICK\_MK3\_disablePortTransmitPacket↔
  - Mode
  - Configuration, 756
- TRIGGER\_BRICK\_MK3\_disableTrigger
  - Configuration, 757
- TRIGGER\_BRICK\_MK3\_enableCounterAutoReload
  - Configuration, 757
- TRIGGER\_BRICK\_MK3\_enableCounterLoadZero
  - Configuration, 758
- TRIGGER\_BRICK\_MK3\_enableCounterStartMode
  - Configuration, 758
- TRIGGER\_BRICK\_MK3\_enableCounterStartStopMode
  - Configuration, 759
- TRIGGER\_BRICK\_MK3\_enableCounterTriggerCount
  - Configuration, 759
- TRIGGER\_BRICK\_MK3\_enableExtTriggerEdge↔
  - DetectMode
  - Configuration, 760
- TRIGGER\_BRICK\_MK3\_enableExtTriggerInvert
  - Configuration, 760
- TRIGGER\_BRICK\_MK3\_enableExtTriggerOutput

- Configuration, 760
- TRIGGER\_BRICK\_MK3\_enablePortSpill
  - Configuration, 761
- TRIGGER\_BRICK\_MK3\_enablePortTransmitPacket↔
  - Mode
  - Configuration, 761
- TRIGGER\_BRICK\_MK3\_enableTrigger
  - Configuration, 762
- TRIGGER\_BRICK\_MK3\_forceCounterReload
  - Configuration, 762
- TRIGGER\_BRICK\_MK3\_forceTrigger
  - Configuration, 763
- TRIGGER\_BRICK\_MK3\_getCounterAutoReload↔
  - Enabled
  - Configuration, 763
- TRIGGER\_BRICK\_MK3\_getCounterInputEvents
  - Events, 696
- TRIGGER\_BRICK\_MK3\_getCounterLoadZeroEnabled
  - Configuration, 763
- TRIGGER\_BRICK\_MK3\_getCounterOutputActions
  - Actions, 725
- TRIGGER\_BRICK\_MK3\_getCounterReloadValue
  - Configuration, 765
- TRIGGER\_BRICK\_MK3\_getCounterStartModeEnabled
  - Configuration, 765
- TRIGGER\_BRICK\_MK3\_getCounterStartStopMode↔
  - Enabled
  - Configuration, 766
- TRIGGER\_BRICK\_MK3\_getCounterTriggerCount↔
  - Enabled
  - Configuration, 766
- TRIGGER\_BRICK\_MK3\_getCounterValue
  - Configuration, 766
- TRIGGER\_BRICK\_MK3\_getDeviceClockRate
  - Configuration, 767
- TRIGGER\_BRICK\_MK3\_getExtTriggerEdgeDetect↔
  - ModeEnabled
  - Configuration, 767
- TRIGGER\_BRICK\_MK3\_getExtTriggerExtend
  - Configuration, 767
- TRIGGER\_BRICK\_MK3\_getExtTriggerInputEvents
  - Events, 697
- TRIGGER\_BRICK\_MK3\_getExtTriggerInvertEnabled
  - Configuration, 768
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutputActions
  - Actions, 725
- TRIGGER\_BRICK\_MK3\_getExtTriggerOutputEnabled
  - Configuration, 768
- TRIGGER\_BRICK\_MK3\_getPortInputEvents
  - Events, 697
- TRIGGER\_BRICK\_MK3\_getPortOutputActions
  - Actions, 726
- TRIGGER\_BRICK\_MK3\_getPortSpillEnabled
  - Configuration, 769
- TRIGGER\_BRICK\_MK3\_getPortTransmitPacket↔
  - ModeEnabled
  - Configuration, 769
- TRIGGER\_BRICK\_MK3\_getTimeCodeInputEvents
  - Events, 698
- TRIGGER\_BRICK\_MK3\_getTimeCodeOutputActions
  - Actions, 726
- TRIGGER\_BRICK\_MK3\_getTriggerAndMask
  - Configuration, 769
- TRIGGER\_BRICK\_MK3\_getTriggerEnabled
  - Configuration, 771
- TRIGGER\_BRICK\_MK3\_getTriggerInputEvents
  - Events, 698
- TRIGGER\_BRICK\_MK3\_getTriggerInputMode
  - Configuration, 771
- TRIGGER\_BRICK\_MK3\_getTriggerInvertMask
  - Configuration, 772
- TRIGGER\_BRICK\_MK3\_getTriggerMatrix
  - Configuration, 772
- TRIGGER\_BRICK\_MK3\_reset
  - Configuration, 772
- TRIGGER\_BRICK\_MK3\_setCounterInputEvents
  - Events, 698
- TRIGGER\_BRICK\_MK3\_setCounterOutputActions
  - Actions, 727
- TRIGGER\_BRICK\_MK3\_setCounterReloadValue
  - Configuration, 773
- TRIGGER\_BRICK\_MK3\_setExtTriggerExtend
  - Configuration, 773
- TRIGGER\_BRICK\_MK3\_setExtTriggerInputEvents
  - Events, 700
- TRIGGER\_BRICK\_MK3\_setExtTriggerOutputActions
  - Actions, 727
- TRIGGER\_BRICK\_MK3\_setPortInputEvents
  - Events, 700
- TRIGGER\_BRICK\_MK3\_setPortOutputActions
  - Actions, 728
- TRIGGER\_BRICK\_MK3 setTimeCodeInputEvents
  - Events, 701
- TRIGGER\_BRICK\_MK3 setTimeCodeOutputActions
  - Actions, 728
- TRIGGER\_BRICK\_MK3\_setTriggerAndMask
  - Configuration, 774
- TRIGGER\_BRICK\_MK3\_setTriggerInputEvents
  - Events, 701
- TRIGGER\_BRICK\_MK3\_setTriggerInputMode
  - Configuration, 774
- TRIGGER\_BRICK\_MK3\_setTriggerInvertMask
  - Configuration, 775
- TRIGGER\_CFG\_disableCounterAutoReload
  - Counters, 503
- TRIGGER\_CFG\_disableCounterLoadZero
  - Counters, 504
- TRIGGER\_CFG\_disableCounterStartMode
  - Counters, 504
- TRIGGER\_CFG\_disableCounterStartStopMode
  - Counters, 505
- TRIGGER\_CFG\_disableCounterTriggerCount
  - Counters, 505
- TRIGGER\_CFG\_disableExtTriggerEdgeDetectMode
  - External Triggers, 494
- TRIGGER\_CFG\_disableExtTriggerInvert

- External Triggers, [494](#)
- TRIGGER\_CFG\_disableExtTriggerOutput
  - External Triggers, [495](#)
- TRIGGER\_CFG\_disablePortSpill
  - SpaceWire Ports, [517](#)
- TRIGGER\_CFG\_disablePortTransmitPacketMode
  - SpaceWire Ports, [517](#)
- TRIGGER\_CFG\_disableSpFiVCReceiveDataMode
  - SpaceFibre Virtual Channels, [548](#)
- TRIGGER\_CFG\_disableSpFiVCTransmitDataMode
  - SpaceFibre Virtual Channels, [548](#)
- TRIGGER\_CFG\_disableSpFiVCTransmitPacketMode
  - SpaceFibre Virtual Channels, [549](#)
- TRIGGER\_CFG\_disableSpWPortSpill
  - SpaceWire Ports, [518](#)
- TRIGGER\_CFG\_disableSpWPortTransmitPacketMode
  - SpaceWire Ports, [518](#)
- TRIGGER\_CFG\_disableTrigger
  - Internal Triggers, [532](#)
- TRIGGER\_CFG\_enableCounterAutoReload
  - Counters, [506](#)
- TRIGGER\_CFG\_enableCounterLoadZero
  - Counters, [506](#)
- TRIGGER\_CFG\_enableCounterStartMode
  - Counters, [507](#)
- TRIGGER\_CFG\_enableCounterStartStopMode
  - Counters, [507](#)
- TRIGGER\_CFG\_enableCounterTriggerCount
  - Counters, [508](#)
- TRIGGER\_CFG\_enableExtTriggerEdgeDetectMode
  - External Triggers, [495](#)
- TRIGGER\_CFG\_enableExtTriggerInvert
  - External Triggers, [496](#)
- TRIGGER\_CFG\_enableExtTriggerOutput
  - External Triggers, [496](#)
- TRIGGER\_CFG\_enablePortSpill
  - SpaceWire Ports, [519](#)
- TRIGGER\_CFG\_enablePortTransmitPacketMode
  - SpaceWire Ports, [519](#)
- TRIGGER\_CFG\_enableSpFiVCReceiveDataMode
  - SpaceFibre Virtual Channels, [549](#)
- TRIGGER\_CFG\_enableSpFiVCTransmitDataMode
  - SpaceFibre Virtual Channels, [549](#)
- TRIGGER\_CFG\_enableSpFiVCTransmitPacketMode
  - SpaceFibre Virtual Channels, [550](#)
- TRIGGER\_CFG\_enableSpWPortSpill
  - SpaceWire Ports, [519](#)
- TRIGGER\_CFG\_enableSpWPortTransmitPacketMode
  - SpaceWire Ports, [521](#)
- TRIGGER\_CFG\_enableTrigger
  - Internal Triggers, [532](#)
- TRIGGER\_CFG\_forceCounterReload
  - Counters, [508](#)
- TRIGGER\_CFG\_forceTrigger
  - Internal Triggers, [533](#)
- TRIGGER\_CFG\_getCounterAutoReloadEnabled
  - Counters, [509](#)
- TRIGGER\_CFG\_getCounterInputEvents
  - Counters, [509](#)
- TRIGGER\_CFG\_getCounterLoadZeroEnabled
  - Counters, [510](#)
- TRIGGER\_CFG\_getCounterOutputActions
  - Counters, [510](#)
- TRIGGER\_CFG\_getCounterReloadValue
  - Counters, [511](#)
- TRIGGER\_CFG\_getCounterStartModeEnabled
  - Counters, [511](#)
- TRIGGER\_CFG\_getCounterStartStopModeEnabled
  - Counters, [512](#)
- TRIGGER\_CFG\_getCounterTriggerCountEnabled
  - Counters, [512](#)
- TRIGGER\_CFG\_getCounterValue
  - Counters, [512](#)
- TRIGGER\_CFG\_getDeviceClockFreq
  - General, [490](#)
- TRIGGER\_CFG\_getDeviceClockRate
  - General, [491](#)
- TRIGGER\_CFG\_getExtTriggerEdgeDetectMode↔
  - Enabled
  - External Triggers, [497](#)
- TRIGGER\_CFG\_getExtTriggerExtend
  - External Triggers, [497](#)
- TRIGGER\_CFG\_getExtTriggerInputEvents
  - External Triggers, [498](#)
- TRIGGER\_CFG\_getExtTriggerInvertEnabled
  - External Triggers, [498](#)
- TRIGGER\_CFG\_getExtTriggerOutputActions
  - External Triggers, [499](#)
- TRIGGER\_CFG\_getExtTriggerOutputEnabled
  - External Triggers, [499](#)
- TRIGGER\_CFG\_getNumberOfCounters
  - Counters, [513](#)
- TRIGGER\_CFG\_getNumberOfExtTriggers
  - External Triggers, [499](#)
- TRIGGER\_CFG\_getNumberOfSpFiPorts
  - SpaceFibre Ports, [543](#)
- TRIGGER\_CFG\_getNumberOfSpFiVCs
  - SpaceFibre Virtual Channels, [550](#)
- TRIGGER\_CFG\_getNumberOfSpWPorts
  - SpaceWire Ports, [521](#)
- TRIGGER\_CFG\_getNumberOfTimeCodeInterfaces
  - Time-codes, [528](#)
- TRIGGER\_CFG\_getNumberOfTriggers
  - Internal Triggers, [533](#)
- TRIGGER\_CFG\_getPortInputEvents
  - SpaceWire Ports, [522](#)
- TRIGGER\_CFG\_getPortOutputActions
  - SpaceWire Ports, [522](#)
- TRIGGER\_CFG\_getPortSpillEnabled
  - SpaceWire Ports, [523](#)
- TRIGGER\_CFG\_getPortTransmitPacketModeEnabled
  - SpaceWire Ports, [523](#)
- TRIGGER\_CFG\_getSpFiPortInputEvents
  - SpaceFibre Ports, [544](#)
- TRIGGER\_CFG\_getSpFiPortOutputActions
  - SpaceFibre Ports, [544](#)

- TRIGGER\_CFG\_getSpFiVCInputEvents  
SpaceFibre Virtual Channels, [551](#)
- TRIGGER\_CFG\_getSpFiVCOOutputActions  
SpaceFibre Virtual Channels, [551](#)
- TRIGGER\_CFG\_getSpFiVCReceiveDataModeEnabled  
SpaceFibre Virtual Channels, [552](#)
- TRIGGER\_CFG\_getSpFiVCTransmitDataModeEnabled  
SpaceFibre Virtual Channels, [552](#)
- TRIGGER\_CFG\_getSpFiVCTransmitPacketMode↔  
Enabled  
SpaceFibre Virtual Channels, [552](#)
- TRIGGER\_CFG\_getSpWPortInputEvents  
SpaceWire Ports, [524](#)
- TRIGGER\_CFG\_getSpWPortOutputActions  
SpaceWire Ports, [524](#)
- TRIGGER\_CFG\_getSpWPortSpillEnabled  
SpaceWire Ports, [525](#)
- TRIGGER\_CFG\_getSpWPortTransmitPacketMode↔  
Enabled  
SpaceWire Ports, [525](#)
- TRIGGER\_CFG\_getTimeCodeInputEvents  
Time-codes, [529](#)
- TRIGGER\_CFG\_getTimeCodeOutputActions  
Time-codes, [529](#)
- TRIGGER\_CFG\_getTriggerAndMask  
Internal Triggers, [534](#)
- TRIGGER\_CFG\_getTriggerEnabled  
Internal Triggers, [534](#)
- TRIGGER\_CFG\_getTriggerInputEvents  
Internal Triggers, [535](#)
- TRIGGER\_CFG\_getTriggerInputMode  
Internal Triggers, [535](#)
- TRIGGER\_CFG\_getTriggerInvertMask  
Internal Triggers, [536](#)
- TRIGGER\_CFG\_getTriggerMatrix  
General, [491](#)
- TRIGGER\_CFG\_getTriggerSourceEventsAnded  
Internal Triggers, [536](#)
- TRIGGER\_CFG\_getTriggerSourceEventsInverted  
Internal Triggers, [537](#)
- TRIGGER\_CFG\_reset  
General, [491](#)
- TRIGGER\_CFG\_setCounterInputEvents  
Counters, [513](#)
- TRIGGER\_CFG\_setCounterOutputActions  
Counters, [514](#)
- TRIGGER\_CFG\_setCounterReloadValue  
Counters, [514](#)
- TRIGGER\_CFG\_setExtTriggerExtend  
External Triggers, [500](#)
- TRIGGER\_CFG\_setExtTriggerInputEvents  
External Triggers, [500](#)
- TRIGGER\_CFG\_setExtTriggerOutputActions  
External Triggers, [501](#)
- TRIGGER\_CFG\_setPortInputEvents  
SpaceWire Ports, [525](#)
- TRIGGER\_CFG\_setPortOutputActions  
SpaceWire Ports, [526](#)
- TRIGGER\_CFG\_setSpFiPortInputEvents  
SpaceFibre Ports, [544](#)
- TRIGGER\_CFG\_setSpFiPortOutputActions  
SpaceFibre Ports, [546](#)
- TRIGGER\_CFG\_setSpFiVCInputEvents  
SpaceFibre Virtual Channels, [553](#)
- TRIGGER\_CFG\_setSpFiVCOOutputActions  
SpaceFibre Virtual Channels, [553](#)
- TRIGGER\_CFG\_setSpWPortInputEvents  
SpaceWire Ports, [526](#)
- TRIGGER\_CFG\_setSpWPortOutputActions  
SpaceWire Ports, [527](#)
- TRIGGER\_CFG\_setTimeCodeInputEvents  
Time-codes, [530](#)
- TRIGGER\_CFG\_setTimeCodeOutputActions  
Time-codes, [530](#)
- TRIGGER\_CFG\_setTriggerAndMask  
Internal Triggers, [537](#)
- TRIGGER\_CFG\_setTriggerInputEvents  
Internal Triggers, [539](#)
- TRIGGER\_CFG\_setTriggerInputMode  
Internal Triggers, [539](#)
- TRIGGER\_CFG\_setTriggerInvertMask  
Internal Triggers, [540](#)
- TRIGGER\_CFG\_setTriggerSourceEventsAnded  
Internal Triggers, [540](#)
- TRIGGER\_CFG\_setTriggerSourceEventsInverted  
Internal Triggers, [541](#)
- TRIGGER\_DEVICE  
triggering\_types.h, [1235](#)
- TRIGGER\_EVENT  
Events, [695](#)
- TRIGGER\_EVENT\_ALL  
Events, [695](#)
- TRIGGER\_EVENT\_IN  
Events, [695](#)
- TRIGGER\_EVENT\_MASK  
Events, [693](#)
- TRIGGER\_EVENT\_NONE  
Events, [695](#)
- TRIGGER\_INPUT\_MODE  
Configuration, [752](#)
- TRIGGER\_INPUT\_MODE\_AND  
Configuration, [752](#)
- TRIGGER\_INPUT\_MODE\_OR  
Configuration, [752](#)
- TRIGGER\_MATRIX, [1233](#)
- TRIGGER\_PCIE\_IF\_disableCounterAutoReload  
Configuration, [775](#)
- TRIGGER\_PCIE\_IF\_disableCounterLoadZero  
Configuration, [776](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartMode  
Configuration, [776](#)
- TRIGGER\_PCIE\_IF\_disableCounterStartStopMode  
Configuration, [776](#)
- TRIGGER\_PCIE\_IF\_disableCounterTriggerCount  
Configuration, [777](#)
- TRIGGER\_PCIE\_IF\_disableTrigger



- Configuration, [777](#)
- TRIGGER\_PCIE\_IF\_enableCounterAutoReload
  - Configuration, [778](#)
- TRIGGER\_PCIE\_IF\_enableCounterLoadZero
  - Configuration, [778](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartMode
  - Configuration, [778](#)
- TRIGGER\_PCIE\_IF\_enableCounterStartStopMode
  - Configuration, [779](#)
- TRIGGER\_PCIE\_IF\_enableCounterTriggerCount
  - Configuration, [779](#)
- TRIGGER\_PCIE\_IF\_enableTrigger
  - Configuration, [780](#)
- TRIGGER\_PCIE\_IF\_forceCounterReload
  - Configuration, [780](#)
- TRIGGER\_PCIE\_IF\_forceTrigger
  - Configuration, [780](#)
- TRIGGER\_PCIE\_IF\_getCounterAutoReloadEnabled
  - Configuration, [781](#)
- TRIGGER\_PCIE\_IF\_getCounterInputEvents
  - Events, [702](#)
- TRIGGER\_PCIE\_IF\_getCounterLoadZeroEnabled
  - Configuration, [781](#)
- TRIGGER\_PCIE\_IF\_getCounterOutputActions
  - Actions, [729](#)
- TRIGGER\_PCIE\_IF\_getCounterReloadValue
  - Configuration, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterStartModeEnabled
  - Configuration, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterStartStopMode↔
  - Enabled
  - Configuration, [782](#)
- TRIGGER\_PCIE\_IF\_getCounterTriggerCountEnabled
  - Configuration, [784](#)
- TRIGGER\_PCIE\_IF\_getCounterValue
  - Configuration, [784](#)
- TRIGGER\_PCIE\_IF\_getDeviceClockRate
  - Configuration, [784](#)
- TRIGGER\_PCIE\_IF\_getPortInputEvents
  - Events, [702](#)
- TRIGGER\_PCIE\_IF\_getPortOutputActions
  - Actions, [729](#)
- TRIGGER\_PCIE\_IF\_getTriggerAndMask
  - Configuration, [785](#)
- TRIGGER\_PCIE\_IF\_getTriggerEnabled
  - Configuration, [785](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputEvents
  - Events, [703](#)
- TRIGGER\_PCIE\_IF\_getTriggerInputMode
  - Configuration, [786](#)
- TRIGGER\_PCIE\_IF\_getTriggerInvertMask
  - Configuration, [786](#)
- TRIGGER\_PCIE\_IF\_reset
  - Configuration, [787](#)
- TRIGGER\_PCIE\_IF\_setCounterInputEvents
  - Events, [703](#)
- TRIGGER\_PCIE\_IF\_setCounterOutputActions
  - Actions, [730](#)
- TRIGGER\_PCIE\_IF\_setCounterReloadValue
  - Configuration, [787](#)
- TRIGGER\_PCIE\_IF\_setPortInputEvents
  - Events, [703](#)
- TRIGGER\_PCIE\_IF\_setPortOutputActions
  - Actions, [730](#)
- TRIGGER\_PCIE\_IF\_setTriggerAndMask
  - Configuration, [787](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputEvents
  - Events, [704](#)
- TRIGGER\_PCIE\_IF\_setTriggerInputMode
  - Configuration, [788](#)
- TRIGGER\_PCIE\_IF\_setTriggerInvertMask
  - Configuration, [788](#)
- TRIGGER\_PCIE\_MK2\_disableCounterAutoReload
  - Configuration, [789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterLoadZero
  - Configuration, [789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterStartMode
  - Configuration, [789](#)
- TRIGGER\_PCIE\_MK2\_disableCounterStartStopMode
  - Configuration, [791](#)
- TRIGGER\_PCIE\_MK2\_disableCounterTriggerCount
  - Configuration, [791](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerEdgeDetect↔
  - Mode
  - Configuration, [791](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerInvert
  - Configuration, [792](#)
- TRIGGER\_PCIE\_MK2\_disableExtTriggerOutput
  - Configuration, [792](#)
- TRIGGER\_PCIE\_MK2\_disablePortSpill
  - Configuration, [793](#)
- TRIGGER\_PCIE\_MK2\_disablePortTransmitPacket↔
  - Mode
  - Configuration, [793](#)
- TRIGGER\_PCIE\_MK2\_disableTrigger
  - Configuration, [794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterAutoReload
  - Configuration, [794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterLoadZero
  - Configuration, [794](#)
- TRIGGER\_PCIE\_MK2\_enableCounterStartMode
  - Configuration, [795](#)
- TRIGGER\_PCIE\_MK2\_enableCounterStartStopMode
  - Configuration, [795](#)
- TRIGGER\_PCIE\_MK2\_enableCounterTriggerCount
  - Configuration, [796](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerEdgeDetect↔
  - Mode
  - Configuration, [796](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerInvert
  - Configuration, [797](#)
- TRIGGER\_PCIE\_MK2\_enableExtTriggerOutput
  - Configuration, [797](#)
- TRIGGER\_PCIE\_MK2\_enablePortSpill
  - Configuration, [798](#)
- TRIGGER\_PCIE\_MK2\_enablePortTransmitPacketMode

- Configuration, 798
- TRIGGER\_PCIE\_MK2\_enableTrigger
  - Configuration, 799
- TRIGGER\_PCIE\_MK2\_forceCounterReload
  - Configuration, 799
- TRIGGER\_PCIE\_MK2\_forceTrigger
  - Configuration, 800
- TRIGGER\_PCIE\_MK2\_getCounterAutoReloadEnabled
  - Configuration, 800
- TRIGGER\_PCIE\_MK2\_getCounterInputEvents
  - Events, 704
- TRIGGER\_PCIE\_MK2\_getCounterLoadZeroEnabled
  - Configuration, 800
- TRIGGER\_PCIE\_MK2\_getCounterOutputActions
  - Actions, 731
- TRIGGER\_PCIE\_MK2\_getCounterReloadValue
  - Configuration, 801
- TRIGGER\_PCIE\_MK2\_getCounterStartModeEnabled
  - Configuration, 801
- TRIGGER\_PCIE\_MK2\_getCounterStartStopMode↔
  - Enabled
  - Configuration, 802
- TRIGGER\_PCIE\_MK2\_getCounterTriggerCount↔
  - Enabled
  - Configuration, 802
- TRIGGER\_PCIE\_MK2\_getCounterValue
  - Configuration, 802
- TRIGGER\_PCIE\_MK2\_getDeviceClockRate
  - Configuration, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerEdgeDetect↔
  - ModeEnabled
  - Configuration, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerExtend
  - Configuration, 803
- TRIGGER\_PCIE\_MK2\_getExtTriggerInputEvents
  - Events, 705
- TRIGGER\_PCIE\_MK2\_getExtTriggerInvertEnabled
  - Configuration, 804
- TRIGGER\_PCIE\_MK2\_getExtTriggerOutputActions
  - Actions, 731
- TRIGGER\_PCIE\_MK2\_getExtTriggerOutputEnabled
  - Configuration, 804
- TRIGGER\_PCIE\_MK2\_getPortInputEvents
  - Events, 705
- TRIGGER\_PCIE\_MK2\_getPortOutputActions
  - Actions, 731
- TRIGGER\_PCIE\_MK2\_getPortSpillEnabled
  - Configuration, 805
- TRIGGER\_PCIE\_MK2\_getPortTransmitPacketMode↔
  - Enabled
  - Configuration, 805
- TRIGGER\_PCIE\_MK2\_getTimeCodeInputEvents
  - Events, 705
- TRIGGER\_PCIE\_MK2\_getTimeCodeOutputActions
  - Actions, 732
- TRIGGER\_PCIE\_MK2\_getTriggerAndMask
  - Configuration, 805
- TRIGGER\_PCIE\_MK2\_getTriggerEnabled
  - Configuration, 807
- TRIGGER\_PCIE\_MK2\_getTriggerInputEvents
  - Events, 707
- TRIGGER\_PCIE\_MK2\_getTriggerInputMode
  - Configuration, 807
- TRIGGER\_PCIE\_MK2\_getTriggerInvertMask
  - Configuration, 808
- TRIGGER\_PCIE\_MK2\_getTriggerMatrix
  - Configuration, 808
- TRIGGER\_PCIE\_MK2\_reset
  - Configuration, 808
- TRIGGER\_PCIE\_MK2\_setCounterInputEvents
  - Events, 707
- TRIGGER\_PCIE\_MK2\_setCounterOutputActions
  - Actions, 732
- TRIGGER\_PCIE\_MK2\_setCounterReloadValue
  - Configuration, 809
- TRIGGER\_PCIE\_MK2\_setExtTriggerExtend
  - Configuration, 809
- TRIGGER\_PCIE\_MK2\_setExtTriggerInputEvents
  - Events, 708
- TRIGGER\_PCIE\_MK2\_setExtTriggerOutputActions
  - Actions, 733
- TRIGGER\_PCIE\_MK2\_setPortInputEvents
  - Events, 708
- TRIGGER\_PCIE\_MK2\_setPortOutputActions
  - Actions, 733
- TRIGGER\_PCIE\_MK2\_setTimeCodeInputEvents
  - Events, 709
- TRIGGER\_PCIE\_MK2\_setTimeCodeOutputActions
  - Actions, 734
- TRIGGER\_PCIE\_MK2\_setTriggerAndMask
  - Configuration, 810
- TRIGGER\_PCIE\_MK2\_setTriggerInputEvents
  - Events, 709
- TRIGGER\_PCIE\_MK2\_setTriggerInputMode
  - Configuration, 810
- TRIGGER\_PCIE\_MK2\_setTriggerInvertMask
  - Configuration, 811
- TRIGGER\_PCIE\_getTriggerMatrix
  - Configuration, 775
- TRIGGER\_PXI\_IF\_disableCounterAutoReload
  - Configuration, 811
- TRIGGER\_PXI\_IF\_disableCounterLoadZero
  - Configuration, 812
- TRIGGER\_PXI\_IF\_disableCounterStartMode
  - Configuration, 812
- TRIGGER\_PXI\_IF\_disableCounterStartStopMode
  - Configuration, 812
- TRIGGER\_PXI\_IF\_disableCounterTriggerCount
  - Configuration, 813
- TRIGGER\_PXI\_IF\_disableExtTriggerEdgeDetectMode
  - Configuration, 813
- TRIGGER\_PXI\_IF\_disableExtTriggerInvert
  - Configuration, 813
- TRIGGER\_PXI\_IF\_disableExtTriggerOutput
  - Configuration, 814
- TRIGGER\_PXI\_IF\_disablePortTransmitPacketMode

- Configuration, [814](#)
- TRIGGER\_PXI\_IF\_disableTrigger
  - Configuration, [815](#)
- TRIGGER\_PXI\_IF\_enableCounterAutoReload
  - Configuration, [815](#)
- TRIGGER\_PXI\_IF\_enableCounterLoadZero
  - Configuration, [816](#)
- TRIGGER\_PXI\_IF\_enableCounterStartMode
  - Configuration, [816](#)
- TRIGGER\_PXI\_IF\_enableCounterStartStopMode
  - Configuration, [816](#)
- TRIGGER\_PXI\_IF\_enableCounterTriggerCount
  - Configuration, [817](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerEdgeDetectMode
  - Configuration, [817](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerInvert
  - Configuration, [818](#)
- TRIGGER\_PXI\_IF\_enableExtTriggerOutput
  - Configuration, [818](#)
- TRIGGER\_PXI\_IF\_enablePortTransmitPacketMode
  - Configuration, [819](#)
- TRIGGER\_PXI\_IF\_enableTrigger
  - Configuration, [819](#)
- TRIGGER\_PXI\_IF\_forceCounterReload
  - Configuration, [820](#)
- TRIGGER\_PXI\_IF\_forceTrigger
  - Configuration, [820](#)
- TRIGGER\_PXI\_IF\_getCounterAutoReloadEnabled
  - Configuration, [821](#)
- TRIGGER\_PXI\_IF\_getCounterInputEvents
  - Events, [710](#)
- TRIGGER\_PXI\_IF\_getCounterLoadZeroEnabled
  - Configuration, [821](#)
- TRIGGER\_PXI\_IF\_getCounterOutputActions
  - Actions, [734](#)
- TRIGGER\_PXI\_IF\_getCounterReloadValue
  - Configuration, [821](#)
- TRIGGER\_PXI\_IF\_getCounterStartModeEnabled
  - Configuration, [823](#)
- TRIGGER\_PXI\_IF\_getCounterStartStopModeEnabled
  - Configuration, [823](#)
- TRIGGER\_PXI\_IF\_getCounterTriggerCountEnabled
  - Configuration, [823](#)
- TRIGGER\_PXI\_IF\_getCounterValue
  - Configuration, [824](#)
- TRIGGER\_PXI\_IF\_getDeviceClockRate
  - Configuration, [824](#)
- TRIGGER\_PXI\_IF\_getExtTriggerEdgeDetectMode↔
  - Enabled
  - Configuration, [825](#)
- TRIGGER\_PXI\_IF\_getExtTriggerExtend
  - Configuration, [825](#)
- TRIGGER\_PXI\_IF\_getExtTriggerInputEvents
  - Events, [710](#)
- TRIGGER\_PXI\_IF\_getExtTriggerInvertEnabled
  - Configuration, [825](#)
- TRIGGER\_PXI\_IF\_getExtTriggerOutputActions
  - Actions, [735](#)
- TRIGGER\_PXI\_IF\_getExtTriggerOutputEnabled
  - Configuration, [826](#)
- TRIGGER\_PXI\_IF\_getPortInputEvents
  - Events, [710](#)
- TRIGGER\_PXI\_IF\_getPortOutputActions
  - Actions, [735](#)
- TRIGGER\_PXI\_IF\_getPortTransmitPacketMode↔
  - Enabled
  - Configuration, [826](#)
- TRIGGER\_PXI\_IF\_getTimeCodeInputEvents
  - Events, [712](#)
- TRIGGER\_PXI\_IF\_getTimeCodeOutputActions
  - Actions, [736](#)
- TRIGGER\_PXI\_IF\_getTriggerAndMask
  - Configuration, [826](#)
- TRIGGER\_PXI\_IF\_getTriggerEnabled
  - Configuration, [827](#)
- TRIGGER\_PXI\_IF\_getTriggerInputEvents
  - Events, [712](#)
- TRIGGER\_PXI\_IF\_getTriggerInputMode
  - Configuration, [827](#)
- TRIGGER\_PXI\_IF\_getTriggerInvertMask
  - Configuration, [828](#)
- TRIGGER\_PXI\_IF\_getTriggerMatrix
  - Configuration, [828](#)
- TRIGGER\_PXI\_IF\_reset
  - Configuration, [828](#)
- TRIGGER\_PXI\_IF\_setCounterInputEvents
  - Events, [713](#)
- TRIGGER\_PXI\_IF\_setCounterOutputActions
  - Actions, [736](#)
- TRIGGER\_PXI\_IF\_setCounterReloadValue
  - Configuration, [830](#)
- TRIGGER\_PXI\_IF\_setExtTriggerExtend
  - Configuration, [830](#)
- TRIGGER\_PXI\_IF\_setExtTriggerInputEvents
  - Events, [713](#)
- TRIGGER\_PXI\_IF\_setExtTriggerOutputActions
  - Actions, [736](#)
- TRIGGER\_PXI\_IF\_setPortInputEvents
  - Events, [714](#)
- TRIGGER\_PXI\_IF\_setPortOutputActions
  - Actions, [737](#)
- TRIGGER\_PXI\_IF setTimeCodeInputEvents
  - Events, [714](#)
- TRIGGER\_PXI\_IF setTimeCodeOutputActions
  - Actions, [737](#)
- TRIGGER\_PXI\_IF\_setTriggerAndMask
  - Configuration, [831](#)
- TRIGGER\_PXI\_IF\_setTriggerInputEvents
  - Events, [714](#)
- TRIGGER\_PXI\_IF\_setTriggerInputMode
  - Configuration, [831](#)
- TRIGGER\_PXI\_IF\_setTriggerInvertMask
  - Configuration, [832](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterAutoReload
  - Configuration, [832](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterLoadZero



- Configuration, [833](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStartMode
  - Configuration, [833](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterStartStop↔
  - Mode
  - Configuration, [833](#)
- TRIGGER\_PXI\_ROUTER\_disableCounterTriggerCount
  - Configuration, [835](#)
- TRIGGER\_PXI\_ROUTER\_disablePortTransmit↔
  - PacketMode
  - Configuration, [835](#)
- TRIGGER\_PXI\_ROUTER\_disableTrigger
  - Configuration, [835](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterAutoReload
  - Configuration, [836](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterLoadZero
  - Configuration, [836](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStartMode
  - Configuration, [837](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterStartStop↔
  - Mode
  - Configuration, [837](#)
- TRIGGER\_PXI\_ROUTER\_enableCounterTriggerCount
  - Configuration, [838](#)
- TRIGGER\_PXI\_ROUTER\_enablePortTransmitPacket↔
  - Mode
  - Configuration, [838](#)
- TRIGGER\_PXI\_ROUTER\_enableTrigger
  - Configuration, [839](#)
- TRIGGER\_PXI\_ROUTER\_forceCounterReload
  - Configuration, [839](#)
- TRIGGER\_PXI\_ROUTER\_forceTrigger
  - Configuration, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterAutoReload↔
  - Enabled
  - Configuration, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterInputEvents
  - Events, [715](#)
- TRIGGER\_PXI\_ROUTER\_getCounterLoadZero↔
  - Enabled
  - Configuration, [840](#)
- TRIGGER\_PXI\_ROUTER\_getCounterOutputActions
  - Actions, [738](#)
- TRIGGER\_PXI\_ROUTER\_getCounterReloadValue
  - Configuration, [842](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartMode↔
  - Enabled
  - Configuration, [842](#)
- TRIGGER\_PXI\_ROUTER\_getCounterStartStopMode↔
  - Enabled
  - Configuration, [843](#)
- TRIGGER\_PXI\_ROUTER\_getCounterTriggerCount↔
  - Enabled
  - Configuration, [843](#)
- TRIGGER\_PXI\_ROUTER\_getCounterValue
  - Configuration, [843](#)
- TRIGGER\_PXI\_ROUTER\_getDeviceClockRate
  - Configuration, [844](#)
- TRIGGER\_PXI\_ROUTER\_getPortInputEvents
  - Events, [715](#)
- TRIGGER\_PXI\_ROUTER\_getPortOutputActions
  - Actions, [738](#)
- TRIGGER\_PXI\_ROUTER\_getPortTransmitPacket↔
  - ModeEnabled
  - Configuration, [844](#)
- TRIGGER\_PXI\_ROUTER\_getTimeCodeInputEvents
  - Events, [716](#)
- TRIGGER\_PXI\_ROUTER\_getTimeCodeOutputActions
  - Actions, [739](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerAndMask
  - Configuration, [844](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerEnabled
  - Configuration, [846](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerInputEvents
  - Events, [716](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerInputMode
  - Configuration, [846](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerInvertMask
  - Configuration, [847](#)
- TRIGGER\_PXI\_ROUTER\_getTriggerMatrix
  - Configuration, [847](#)
- TRIGGER\_PXI\_ROUTER\_reset
  - Configuration, [847](#)
- TRIGGER\_PXI\_ROUTER\_setCounterInputEvents
  - Events, [716](#)
- TRIGGER\_PXI\_ROUTER\_setCounterOutputActions
  - Actions, [739](#)
- TRIGGER\_PXI\_ROUTER\_setCounterReloadValue
  - Configuration, [848](#)
- TRIGGER\_PXI\_ROUTER\_setPortInputEvents
  - Events, [718](#)
- TRIGGER\_PXI\_ROUTER\_setPortOutputActions
  - Actions, [740](#)
- TRIGGER\_PXI\_ROUTER\_setTimeCodeInputEvents
  - Events, [718](#)
- TRIGGER\_PXI\_ROUTER\_setTimeCodeOutputActions
  - Actions, [740](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerAndMask
  - Configuration, [848](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerInputEvents
  - Events, [719](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerInputMode
  - Configuration, [849](#)
- TRIGGER\_PXI\_ROUTER\_setTriggerInvertMask
  - Configuration, [849](#)
- TRIGGER\_TYPE
  - triggering\_types.h, [1235](#)
- Time-code Master, [574](#)
  - CFG\_MK2\_disableExternalTimeCodeSelection, [575](#)
  - CFG\_MK2\_disableTimeCodeCounterBypass↔
    - Mode, [575](#)
  - CFG\_MK2\_disableTimeCodeMaster, [575](#)
  - CFG\_MK2\_enableExternalTimeCodeSelection, [576](#)

- CFG\_MK2\_enableTimeCodeCounterBypassMode, 576
- CFG\_MK2\_enableTimeCodeMaster, 577
- CFG\_MK2\_getExternalTimeCodeSelection↔ Enabled, 577
- CFG\_MK2\_getTimeCodeCounterBypassMode↔ Enabled, 578
- CFG\_MK2\_getTimeCodeMasterEnabled, 578
- CFG\_MK2\_getTimeCodePeriod, 579
- CFG\_MK2\_setTimeCodePeriod, 579
- Time-codes, 528, 882
  - CFG\_disableExternalTimeCodeSelection, 883
  - CFG\_disableTimeCodeCounterBypassMode, 883
  - CFG\_disableTimeCodeMaster, 884
  - CFG\_enableExternalTimeCodeSelection, 884
  - CFG\_enableTimeCodeCounterBypassMode, 885
  - CFG\_enableTimeCodeMaster, 885
  - CFG\_getExternalTimeCodeSelectionEnabled, 886
  - CFG\_getTimeCodeCounterBypassModeEnabled, 886
  - CFG\_getTimeCodeDistributionCapablePorts, 887
  - CFG\_getTimeCodeDistributionPorts, 887
  - CFG\_getTimeCodeFlagMode, 888
  - CFG\_getTimeCodeMasterEnabled, 888
  - CFG\_getTimeCodePeriod, 889
  - CFG\_getTimeCodeValue, 889
  - CFG\_setTimeCodeDistributionPorts, 890
  - CFG\_setTimeCodeFlagMode, 890
  - CFG\_setTimeCodePeriod, 891
  - TRIGGER\_CFG\_getNumberOfTimeCode↔ Interfaces, 528
  - TRIGGER\_CFG\_getTimeCodeInputEvents, 529
  - TRIGGER\_CFG\_getTimeCodeOutputActions, 529
  - TRIGGER\_CFG\_setTimeCodeInputEvents, 530
  - TRIGGER\_CFG\_setTimeCodeOutputActions, 530
- Time-slots, 972
  - CFG\_SPFI\_getVCTimeSlots, 972
  - CFG\_SPFI\_setVCTimeSlots, 973
- Timestamping, 394, 417, 908
  - CFG\_BRICK\_MK3\_disableRxTimestampEvents↔ OnPort, 418
  - CFG\_BRICK\_MK3\_enableRxTimestampEvents↔ OnPort, 419
  - CFG\_BRICK\_MK3\_getPulseGeneratorFrequency, 419
  - CFG\_BRICK\_MK3\_getRxTimestampEventsOn↔ PortEnabled, 420
  - CFG\_BRICK\_MK3\_getTimestampMethod, 420
  - CFG\_BRICK\_MK3\_getTimestampValue, 421
  - CFG\_BRICK\_MK3\_setPulseGeneratorFrequency, 421
  - CFG\_BRICK\_MK3\_setTimestampMethod, 422
  - CFG\_BRICK\_MK3\_setTimestampValue, 422
  - CFG\_PCIE\_MK2\_disableRxTimestampEvents↔ OnPort, 394
  - CFG\_PCIE\_MK2\_enableRxTimestampEvents↔ OnPort, 395
  - CFG\_PCIE\_MK2\_getPulseGeneratorFrequency, 395
  - CFG\_PCIE\_MK2\_getRxTimestampEventsOn↔ PortEnabled, 395
  - CFG\_PCIE\_MK2\_setPulseGeneratorFrequency, 396
  - CFG\_PCIE\_MK2\_setTimestampMethod, 396
  - CFG\_disableRxTimestampEventsOnPort, 908
  - CFG\_enableRxTimestampEventsOnPort, 909
  - CFG\_getPulseGeneratorFrequency, 909
  - CFG\_getRxTimestampEventsOnPortEnabled, 910
  - CFG\_getTimestampMethod, 910
  - CFG\_getTimestampValue, 911
  - CFG\_setPulseGeneratorFrequency, 911
  - CFG\_setTimestampMethod, 912
  - CFG\_setTimestampValue, 913
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ, 418
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1, 418
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_10, 418
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_100, 418
  - STAR\_CFG\_BRICK\_MK3\_PULSE\_FREQ\_1000, 418
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD, 418
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_EXTERNAL\_TRIGGER, 418
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_NONE, 418
  - STAR\_CFG\_BRICK\_MK3\_TIMESTAMP\_METH↔ OD\_PULSE\_GENERATOR, 418
- Transfer Operations, 357
  - STAR\_RECEIVE\_BROADCAST\_MESSAGE↔ S, 361
  - STAR\_RECEIVE\_CHUNKS, 360
  - STAR\_RECEIVE\_FCTS, 360
  - STAR\_RECEIVE\_LINK\_SPEED\_EVENTS, 361
  - STAR\_RECEIVE\_LINK\_STATE\_EVENTS, 360
  - STAR\_RECEIVE\_MASK, 360
  - STAR\_RECEIVE\_NULL, 359
  - STAR\_RECEIVE\_NULLS, 360
  - STAR\_RECEIVE\_PACKETS, 360
  - STAR\_RECEIVE\_TIMECODES, 360
  - STAR\_RECEIVE\_TIMESTAMP\_EVENTS, 361
  - STAR\_TRANSFER\_OPERATION, 360
  - STAR\_TRANSFER\_STATUS, 361
  - STAR\_TRANSFER\_STATUS\_CANCELLED, 361
  - STAR\_TRANSFER\_STATUS\_COMPLETE, 361
  - STAR\_TRANSFER\_STATUS\_ERROR, 361
  - STAR\_TRANSFER\_STATUS\_NOT\_STARTED, 361
  - STAR\_TRANSFER\_STATUS\_STARTED, 361
  - STAR\_TransferOperationListenerFunc, 360
  - STAR\_cancelTransferOperation, 361
  - STAR\_cancelTransferOperationWaits, 362
  - STAR\_createRxOperation, 362
  - STAR\_createRxOperationEx, 363
  - STAR\_createTxOperation, 364

- STAR\_destroyTransferItemList, 364
- STAR\_disposeTransferOperation, 365
- STAR\_getNextTransferItem, 365
- STAR\_getSpaceWireAddressPath, 366
- STAR\_getSpaceWireAddressPathLength, 366
- STAR\_getTransferItem, 366
- STAR\_getTransferItemCount, 367
- STAR\_getTransferItemList, 367
- STAR\_getTransferOperationStatus, 368
- STAR\_receivePacket, 368
- STAR\_registerTransferCompletionListener, 369
- STAR\_rxOperationExGetBroadcastMessage, 369
- STAR\_rxOperationExGetDataChunk, 370
- STAR\_rxOperationExGetItemType, 370
- STAR\_rxOperationExGetLinkSpeedEvent, 370
- STAR\_rxOperationExGetLinkStateEvent, 372
- STAR\_rxOperationExGetNumItems, 372
- STAR\_rxOperationExGetStatus, 372
- STAR\_rxOperationExGetTimeCode, 374
- STAR\_rxOperationExGetTimestampEvent, 374
- STAR\_submitTransferOperation, 374
- STAR\_submitTransferOperationList, 376
- STAR\_transmitPacket, 376
- STAR\_unregisterTransferCompletionListener, 377
- STAR\_waitOnTransferOperationCompletion, 377
- transfer.h, 1202
- transfer\_types.h, 1204
- Transmit Link Speed, 899
  - CFG\_getTxSignallingRate, 899
  - CFG\_setTxSignallingRate, 900
- Triggering API, 673
- triggering\_brick\_mk3.h, 1205
- triggering\_generic.h, 1209
- triggering\_pcie.h, 1216
- triggering\_pcie\_mk2.h, 1218
- triggering\_pxi\_if.h, 1223
- triggering\_pxi\_ro.h, 1227
- triggering\_types.h, 1230
  - PORT\_ACTION, 1234
  - PORT\_ACTION\_ALL, 1234
  - PORT\_ACTION\_DECR\_CREDIT, 1234
  - PORT\_ACTION\_DISABLE\_LVDS, 1234
  - PORT\_ACTION\_DISCONNECT, 1234
  - PORT\_ACTION\_ESCAPE\_ERROR, 1234
  - PORT\_ACTION\_INCR\_CREDIT, 1234
  - PORT\_ACTION\_INSERT\_FCT, 1234
  - PORT\_ACTION\_MASK, 1233
  - PORT\_ACTION\_NONE, 1234
  - PORT\_ACTION\_PARITY\_ERROR, 1234
  - PORT\_ACTION\_STOP\_RECEPTION, 1234
  - PORT\_ACTION\_SUPPRESS\_FCT, 1234
  - PORT\_ACTION\_TRANSMIT\_PKT, 1234
  - PORT\_EVENT, 1234
  - PORT\_EVENT\_ALL, 1235
  - PORT\_EVENT\_CREDIT\_ERROR, 1235
  - PORT\_EVENT\_DISCONNECT, 1235
  - PORT\_EVENT\_ESCAPE\_ERROR, 1235
  - PORT\_EVENT\_MASK, 1234
  - PORT\_EVENT\_NONE, 1235
  - PORT\_EVENT\_PARITY\_ERROR, 1235
  - PORT\_EVENT\_RUNNING, 1235
  - PORT\_EVENT\_RX\_EEP, 1235
  - PORT\_EVENT\_RX\_EOP, 1235
  - PORT\_EVENT\_RX\_SOP, 1235
  - PORT\_EVENT\_RX\_TIME\_CODE, 1235
  - PORT\_EVENT\_TX\_EOP, 1235
  - PORT\_EVENT\_TX\_PKT\_PENDING, 1235
  - PORT\_EVENT\_TX\_SOP, 1235
  - PORT\_EVENT\_TX\_TIME\_CODE, 1235
  - TRIGGER\_DEVICE, 1235
  - TRIGGER\_TYPE, 1235
- Types, 1037
  - SPFI\_BROADCAST\_TYPES\_INFO, 1038
  - SPFI\_DEVICE\_IDENTIFIER\_INFO, 1038
  - SPFI\_DEVICE\_INFO, 1038
  - SPFI\_LANE\_CONTROL\_SETTINGS, 1038
  - SPFI\_LANE\_EVENTS, 1039
  - SPFI\_LANE\_STATE, 1041
  - SPFI\_LANE\_STATE\_ACTIVE, 1041
  - SPFI\_LANE\_STATE\_CLEAR\_LINE, 1041
  - SPFI\_LANE\_STATE\_COLD\_RESET, 1041
  - SPFI\_LANE\_STATE\_CONNECTED, 1041
  - SPFI\_LANE\_STATE\_CONNECTING, 1041
  - SPFI\_LANE\_STATE\_DISABLED, 1041
  - SPFI\_LANE\_STATE\_INVERT\_RX\_POLARITY, 1041
  - SPFI\_LANE\_STATE\_LOSS\_OF\_SIGNAL, 1041
  - SPFI\_LANE\_STATE\_PREPARE\_STANDBY, 1041
  - SPFI\_LANE\_STATE\_STARTED, 1041
  - SPFI\_LANE\_STATE\_UNDEFINED, 1041
  - SPFI\_LANE\_STATE\_WAIT, 1041
  - SPFI\_LANE\_STATUS, 1039
  - SPFI\_LINK\_ALIGNMENT\_STATE, 1041
  - SPFI\_LINK\_ALIGNMENT\_STATE\_BOTH\_END↔S\_READY, 1041
  - SPFI\_LINK\_ALIGNMENT\_STATE\_NEAR\_END↔\_READY, 1041
  - SPFI\_LINK\_ALIGNMENT\_STATE\_NOT\_READY, 1041
  - SPFI\_LINK\_ALIGNMENT\_STATE\_UNDEFINED, 1041
  - SPFI\_LINK\_CONTROL\_SETTINGS, 1039
  - SPFI\_LINK\_DEBUG\_SETTINGS, 1039
  - SPFI\_LINK\_EVENTS, 1039
  - SPFI\_LINK\_STATUS, 1039
  - SPFI\_NODE\_TYPE, 1041
  - SPFI\_NODE\_TYPE\_CONFIG, 1041
  - SPFI\_NODE\_TYPE\_INVALID, 1041
  - SPFI\_NODE\_TYPE\_OTHER, 1041
  - SPFI\_NODE\_TYPE\_SOURCE\_AND\_DESTINATION, 1041
  - SPFI\_PORT\_INFO, 1039
  - SPFI\_PORT\_TYPE, 1041
  - SPFI\_PORT\_TYPE\_AXI, 1042
  - SPFI\_PORT\_TYPE\_SPACEFIBRE, 1042
  - SPFI\_PORT\_TYPE\_SPACEWIRE, 1042

- SPFI\_PORT\_TYPE\_UNDEFINED, 1042
- SPFI\_QUALITY\_OF\_SERVICE, 1040
- SPFI\_ROUTING\_TABLE\_ENTRY\_FLAGS, 1040
- SPFI\_VC\_ERRORS, 1040
- SPFI\_VC\_STATUS, 1040
- U64, 1040
- types.h, 1236
  - CFG\_INFO\_ID\_TYPE, 1239
  - STAR\_DEVICE\_STAR\_ULTRA\_PCIE, 1239
- U64
  - Types, 1040
- User Configurable Registers, 660
  - CFG\_MK2\_getTimeCodeDistributionPorts, 660
  - CFG\_MK2\_setTimeCodeDistributionPorts, 661
  - CFG\_ROUTER\_getGeneralPurpose, 661
  - CFG\_ROUTER\_getNetworkIdentity, 662
  - CFG\_ROUTER\_getTimeCodeDistributionPorts, 662
  - CFG\_ROUTER\_setGeneralPurpose, 663
  - CFG\_ROUTER\_setNetworkIdentity, 663
  - CFG\_ROUTER\_setTimeCodeDistributionPorts, 664
- VERSION\_EDIT
  - version.h, 1243
- VERSION\_MAJOR
  - version.h, 1243
- VERSION\_MINOR
  - version.h, 1243
- VERSION\_PATCH
  - version.h, 1243
- Vendor Specific, 947
  - CFG\_SPFI\_disableRouterTimeout, 948
  - CFG\_SPFI\_enableRouterTimeout, 948
  - CFG\_SPFI\_getBroadcastTypesInformation, 948
  - CFG\_SPFI\_getRouterTimeout, 949
  - CFG\_SPFI\_getRouterTimeoutMaximum, 949
  - CFG\_SPFI\_getRouterTimeoutResolution, 950
  - CFG\_SPFI\_getSpaceWireInterruptBroadcastType, 950
  - CFG\_SPFI\_getTimeCodeBroadcastType, 951
  - CFG\_SPFI\_getTimeSlotBroadcastType, 951
  - CFG\_SPFI\_isRouterTimeoutEnabled, 952
  - CFG\_SPFI\_setRouterTimeout, 952
  - CFG\_SPFI\_setSpaceWireInterruptBroadcastType, 953
  - CFG\_SPFI\_setTimeCodeBroadcastType, 953
  - CFG\_SPFI\_setTimeSlotBroadcastType, 954
- version.h, 1239
  - POPULATE\_VERSION, 1241
  - PRINT\_FULL\_VERSION\_INFO, 1241
  - PRINT\_VERSION, 1242
  - PRINT\_VERSION\_EDIT, 1242
  - PRINT\_VERSION\_NUM, 1242
  - PRINT\_VERSION\_PATCH, 1242
  - VERSION\_EDIT, 1243
  - VERSION\_MAJOR, 1243
  - VERSION\_MINOR, 1243
  - VERSION\_PATCH, 1243
  - Virtual Channel Information, 955
  - Virtual Devices and Drivers, 379
    - STAR\_isDeviceVirtual, 379
    - STAR\_isDriverVirtual, 379
  - Virtual Network Number, 956
    - CFG\_SPFI\_getVCVN, 956
    - CFG\_SPFI\_setVCVN, 956